台灣自編大學教科書

影像處理

Image Processing

影像處理是當代人工智慧產業的發展基礎



大學教科書

李世淵（機器人叫獸）

台灣自編大學教科書

影　像　處　理

Digital Image Processing

大學部　智慧機器人學程 | 高中　AI 機器人課程

編著　李世淵（機器人叫獸）

助教　李天宇、李宇晴

課程資訊與補充教材　@prof

序言　從像素到世界

這本書是為國立雲林科技大學智慧科技學院智慧機器人學程的《影像處理》課程而寫的,同時也希望能為高中的 AI 機器人課程提供一份可用的教材。它從一張影像最基本的像素講起,一路走到讓機器自己學習、自己辨識,最後把這一切裝進機器人裡,讓它能在真實世界中看見與行動。

寫這本書的起心動念很單純。這些年在課堂上,我常看到兩種讓人惋惜的情況。一種是學生很快學會了呼叫函式庫,能做出漂亮的成果,卻在遇到問題時完全不知道該從哪裡查起——因為他不知道那行程式底下發生了什麼事。另一種是學生把公式背得滾瓜爛熟,卻答不出「這個方法什麼時候該用、什麼時候不該用」。這本書想做的,是把「理解原理」與「動手做出來」重新縫合在一起。

因此本書有幾個刻意的安排,想在這裡先說明。

其一,深度學習被放在第 15 章,而不是開頭。這在今天或許顯得保守,但我相信:唯有先理解卷積、頻率、多尺度、特徵這些基本觀念,你才能真正看懂卷積神經網路在做什麼。事實上,當你讀到第 15 章,會發現網路自己學出來的第一層濾波器,與前面各章人工設計的邊緣偵測器驚人地相似——深度學習沒有推翻傳統影像處理的智慧,而是把它自動化了。

其二,每一章開頭都有一則「叫獸開講」。它們不是花絮。傅立葉的論文被退了稿,他繼續改;哈伯太空望遠鏡近乎失明,工程師用數學把影像救回來;張正友用一張列印的棋盤格,取代了昂貴的精密標定物;一位劍橋的老師為了讓學生有一台「弄壞也不心疼」的電腦,做出了二十五美元的樹莓派;YOLO 的創造者在成就的頂點,因為對技術用途的疑慮而選擇放下。這些故事想告訴你:技術從來不是冷冰冰的公式,而是活生生的人,帶著各自的處境、熱情與判斷,一步步做出來的。而每一則故事的史料,都經過查證。

其三,本書的實作橫跨三個層級的平台。入門的 ESP32-CAM 只要一頓便當的價錢,用來理解端上運算的能力與限制;樹莓派 5 配上 AI Camera,把推論直接做在感測器晶片裡,主機拿到的是「畫面裡有什麼」而不是「畫面」;而 NVIDIA Jetson Orin Nano Super 則能讓深度學習模型真正「上車」。這樣的安排是希望你建立一種判斷力:選對工具,遠比用最強的工具重要。這也是全書反覆出現的主題——適合勝於最強。

其四,課程的教學設計遵循 MIT 媒體實驗室 Mitchel Resnick 提出的 4P 創意學習理念:專題（Projects）、熱情（Passion）、同儕（Peers）、遊玩（Play）。每章都有動手做的 Lab、每題習題都附上提示與參考解答、而全書以一個由你自己選題的期末專題收尾。我希望你做出來的東西是你的,不是老師的、不是課本的。

書中也有一條不那麼顯眼、卻同樣重要的暗線:技術背後的責任。從第 1 章那張退役的測試影像、第 7 章「還原或腦補」的區分、第 14 章模型學到的無意義捷徑,到第 16 章關於軍事應用的抉擇——技術能力愈強大,對後果的想像力就愈是專業素養不可分割的一部分。這門課教你把東西做出來,但沒有任何一行程式碼能替你決定「該不該

做」。

最後要說明的是,本書的每一章都經過「加厚」的改寫:除了正文之外,每章都附有與全書其他章節的觀念連結總表、多個應用案例、六個程式實作 Lab、二十五題附提示與參考解答的習題,以及數段置於節末、把關鍵算式從頭推一遍的推導。這是因為我相信,影像處理真正困難的地方不在單一演算法,而在於看見那些反覆出現的共通結構——換一個域看問題、多解析度、投票對抗雜訊、正則化對抗病態、精確率與召回率的取捨。技術會更迭,這些思考方式不會。

感謝助教李天宇與李宇晴在教材整理與實驗驗證上的協助,也感謝歷屆修課同學的提問——很多章節的補充,都是因為你們問了好問題。書中若有錯漏,責任在我,也歡迎透過 @prof 與我聯繫指正。

祝你讀得愉快,做得盡興。我們在你的作品裡見。

李世淵（機器人叫獸）

國立雲林科技大學　智慧機器人學程/智慧製造學程

目錄

(頁碼為正文頁碼)

第一篇　基礎

第 1 章　緒論 1

1.1 什麼是影像處理 2
1.2 影像處理發展史：四個階段 5
1.3 應用領域巡禮 7
1.4 標準測試影像與技術社群文化 9
1.5 本書架構與學習地圖 10
1.6 應用案例 13
1.7 本章與全書的觀念連結總表 15

第 2 章　開發環境與程式基礎 22

2.1 開發環境的建立 23
2.2 影像就是陣列：NumPy 基礎 25
2.3 OpenCV 基本操作 31
2.4 三種 AI 辨識架構與實作平台 35
2.5 工程習慣的養成 37
2.6 應用案例 39
2.7 本章與全書的觀念連結總表 43

第 3 章　數位影像基礎 52

3.1 從連續到離散：取樣與量化 53
3.2 取樣定理與混疊 56
3.3 量化與位元深度 59
3.4 影像的產生：感測器與色彩 62
3.5 影像品質的客觀評估 65
3.6 像素的鄰域、連通性與距離 68
3.7 應用案例 71
3.8 本章與全書的觀念連結總表 73

第二篇　核心

第 4 章　幾何轉換 81

4.1 基本幾何轉換與齊次座標 82
4.2 轉換家族：從剛體到透視 86
4.3 影像內插：轉換後的像素從哪裡來 90
4.4 相機模型、鏡頭畸變與張氏標定 94
4.5 幾何轉換在全書中的角色 99
4.6 應用案例 100
4.7 本章與全書的觀念連結總表 101

第 5 章　空間域影像強化 109

5.1 空間域處理的框架與灰階轉換 110
5.2 直方圖處理 115
5.3 空間濾波：卷積與相關 119
5.4 平滑濾波：對付雜訊 122
5.5 銳化濾波：強調細節 125

5.6 多影格合成：以軟體突破硬體限制 129
5.7 應用案例 132
5.8 本章與全書的觀念連結總表 133
第 6 章　頻率域影像處理 141
6.1 從弦波到影像：傅立葉的基本觀念 142
6.2 二維離散傅立葉轉換(DFT)與 FFT 145
6.3 頻率域濾波：低通、高通、帶通 150
6.4 帶阻與陷波濾波：狙擊週期性雜訊 155
6.5 卷積定理：兩個域的橋梁 157
6.6 頻率域觀點與全書的觀念連結 159
6.7 應用案例 162
6.8 本章與全書的觀念連結總表 165
第 7 章　影像還原 173
7.1 影像還原的定位、模型與病態性 174
7.2 雜訊模型、辨識與參數估計 178
7.3 空間域去雜訊：從排序統計到非局部平均 183
7.4 頻率域觀點：還原與濾波的血緣關係 184
7.5 反卷積：逆濾波、維納濾波與約束最小平方 188
7.6 應用案例 193
7.7 影像還原對後續處理的影響 195
7.8 AI 影像修復：從還原到生成 197
7.9 本章與全書的觀念連結總表 198
第 8 章　色彩影像處理 207
8.1 色彩的基礎：從人眼到 RGB 208
8.2 HSL 與 HSV：符合人類直覺的色彩模型 210
8.3 YCbCr 與 CIE Lab 215
8.4 虛擬色彩：讓看不見的資訊現形 219
8.5 白平衡與色彩校正 221
8.6 顏色分割與物件追蹤 225
8.7 應用案例 227
8.8 本章與全書的觀念連結總表 229
第 9 章　影像分割 237
9.1 什麼是影像分割 238
9.2 門檻法：最簡單的分割 241
9.3 邊緣偵測 246
9.4 霍夫轉換：參數空間的投票 251
9.5 基於區域的分割 255
9.6 從傳統分割到深度學習語意分割 259
9.7 應用案例 260
9.8 本章與全書的觀念連結總表 261
第 10 章　二值影像處理 269
10.1 二值影像與型態學的基礎 270
10.2 兩個基本運算：侵蝕與膨脹 272
10.3 兩個組合運算：斷開與閉合 276
10.4 進階型態學運算 281
10.5 連通元件標記與距離轉換 284
10.6 Haar 級聯分類器與人臉偵測 287
10.7 應用案例 292
10.8 本章與全書的觀念連結總表 293
第 11 章　小波與正交轉換 301
11.1 正交轉換的共同思想 302

11.2 傅立葉的弱點與小波的動機 303
11.3 Haar 小波與多解析度分析 306
11.4 離散餘弦轉換與 JPEG 壓縮 310
11.5 小波壓縮與 JPEG 2000 315
11.6 小波的其他應用 317
11.7 小波在深度學習時代的定位 321
11.8 應用案例 321
11.9 本章與全書的觀念連結總表 323
第 12 章　視訊與動態影像處理 331
12.1 視訊的本質：多了一個時間維度 332
12.2 視訊的讀取、處理與寫出 334
12.3 移動偵測：影格差分與背景相減 336
12.4 光流：估計像素的運動 342
12.5 物件追蹤 348
12.6 進階主題：穩定、動作辨識與時序深度學習 352
12.7 應用案例 353
12.8 本章與全書的觀念連結總表 354

第三篇　學習

第 13 章　特徵擷取 363
13.1 什麼是好的特徵 364
13.2 角點偵測：Harris 與 Shi-Tomasi 366
13.3 尺度不變特徵：SIFT 與 ORB 371
13.4 特徵匹配與外點去除 375
13.5 區域描述子與形狀特徵：HOG、LBP 與幾何量測 379
13.6 應用：影像拼接、模板比對與物件定位 389
13.7 案例深探：車牌偵測 391
13.8 從手工特徵到學習特徵 393
13.9 本章與全書的觀念連結總表 393
第 14 章　機器學習基礎 401
14.1 什麼是機器學習 402
14.2 影像分類的完整流程 404
14.3 影像資料集：機器學習的燃料 405
14.4 兩個經典分類器：kNN 與 SVM 407
14.5 資料切分與模型評估 414
14.6 過度擬合、擬合不足與偏差-變異權衡 420
14.7 應用案例 424
14.8 本章與全書的觀念連結總表 426
第 15 章　深度學習 434
15.1 從手工特徵到學習特徵：典範轉移 436
15.2 神經網路的基本構件 437
15.3 卷積神經網路(CNN) 444
15.4 經典 CNN 架構譜系 449
15.5 三大框架與 MNIST 實作 450
15.6 遷移學習與資料增強 452
15.7 深度學習的侷限與展望 456
15.8 本章與全書的觀念連結總表 457
第 16 章　物件偵測 YOLO 實戰 466
16.1 四種視覺任務 467
16.2 物件偵測的演進與 YOLO 的核心思想 469

16.3 物件偵測的關鍵概念 472
16.4 YOLO 實戰：推論、訓練與驗證 478
16.5 延伸功能：分割、姿態估計與追蹤 483
16.6 應用案例 488
16.7 技術倫理：工程師的責任 491
16.8 本章與全書的觀念連結總表 493

第四篇　整合

第 17 章　邊緣 AI 與機器人視覺部署 502
17.1 為什麼需要邊緣運算 503
17.2 兩大實作平台：樹莓派 5 與 Jetson Orin Nano Super 505
17.3 模型最佳化：讓大模型跑在小裝置上 508
17.4 效能量測與系統設計 513
17.5 整合進機器人系統：ROS 2 519
17.6 應用案例 521
17.7 部署的實務課題 523
17.8 本章與全書的觀念連結總表 527
第 18 章　期末專題：4P 實踐 537
18.1 4P 創意學習 538
18.2 創意學習螺旋 539
18.3 專題的規劃與執行 541
18.4 專題方向與技術地圖 548
18.5 成果展示與評估 550
18.6 專題案例參考 553
18.7 全書技術整合地圖 558

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 1 章 緒論

本章學習目標

- 說明影像處理的定義與範疇，區分影像處理、電腦視覺與電腦圖學三者的關係。
- 闡述低階、中階、高階視覺的三層架構，並理解它與本書章節安排的對應。
- 概述影像處理的發展史，從報紙傳真、太空探索到深度學習的四個階段。
- 舉例說明影像處理在醫療、製造、農業、交通、機器人等領域的應用。
- 認識標準測試影像的角色，並反思技術社群的文化與包容性議題。
- 掌握本書的學習地圖、三層實作平台與 4P 學習理念。

叫獸開講|萊娜圖退役：一張測試影像與一個領域的自省

如果你翻開 1980 至 2000 年代的任何一本影像處理教科書或論文集，幾乎必然會遇到同一張照片：一位側身回眸、戴著羽飾寬邊帽的年輕女子。她的名字叫萊娜(Lenna)，而她的臉，曾被戲稱為「比蒙娜麗莎還要被研究得更透徹的一張臉」。半個世紀以來，無數的濾波器、壓縮演算法、邊緣偵測器，都是先在她身上試過才發表的。

這張影像的來歷，是一個關於「偶然」的故事。1973 年六、七月間，南加州大學訊號與影像處理研究所的助理教授 Alexander Sawchuk 與同事，正急著替一位同僚的研討會論文找一張測試影像。實驗室裡現成的測試圖都是 1960 年代電視標準時代留下的老素材，單調乏味;他們想要一張印刷精美、動態範圍好、而且有人臉的照片。就在此時，有人拿著一本當期的《花花公子》雜誌走進實驗室。他們撕下了中頁摺頁的上方三分之一(僅保留頭肩部分)，把它捲上一台 Muirhead 滾

筒式傳真掃描器——那台機器被他們自行改裝過，接上了紅綠藍三組類比數位轉換器與一台 HP 2100 迷你電腦——掃出了一張 512×512 的數位影像。

接下來的事出乎所有人意料。這張影像因為兼具平滑的皮膚、銳利的羽毛紋理、豐富的色彩層次與明暗對比，成了絕佳的測試素材;南加大甚至會把拷貝發送給來訪的研究者。它就這樣一傳十、十傳百，成了整個領域不成文的標準測試影像，甚至參與了 JPEG 壓縮標準的發展歷程。照片中的瑞典模特兒 Lena Forsén 對此渾然不知，直到多年後才知道自己在一個素未謀面的技術圈裡如此有名——1997 年，她還受邀出席了影像科學與技術學會的年會。

但這個故事有它的另一面。從 2010 年代中期起，愈來愈多研究者提出質疑：一個以《花花公子》中頁作為半世紀標準素材的領域，對想要進入這個領域的女性傳遞了什麼樣的訊息?2019 年的紀錄短片《失去萊娜》(Losing Lena)直指這張影像象徵了科技產業長期的偏狹文化。多家期刊陸續勸退或禁用它。而 Lena Forsén 本人也表達了希望「退休」的意願。

2024 年 3 月，全球最大的計算機專業學會之一——IEEE 計算機協會——正式發函通知會員：自 4 月 1 日起，不再接受包含萊娜圖的投稿論文。函中援引了學會的多元共融聲明與倫理守則，並明確表示這是「尊重影像當事人 Lena Forsén 的意願」。一張見證了整個數位影像時代的照片，就這樣正式退役。

叫獸把這個故事放在全書的第一頁，有三個用意。第一，它讓你看見這個領域的真實歷史——不是無菌的教科書敘事，而是由一群人在特定時空下的偶然選擇所堆疊起來的。第二，它預告了本書的一條暗線：技術從來不只是技術，它承載著使用者的價值觀，也形塑著誰會覺得自己屬於這裡。從第 7 章的「還原或腦補」、第 14 章的資料偏誤、到第 16 章 YOLO 之父的選擇，我們會不斷回到這個主題。第三，它示範了一個健康的專業社群該有的樣子：能夠回頭檢視自己的習慣，即使那個習慣已經存在了五十年。

思考題：萊娜圖之所以流行，一部分原因是它「技術上很好用」(紋理豐富、對比鮮明)。當「技術上好用」與「對某些人不友善」發生衝突時，你認為該怎麼權衡?有沒有可能找到兩全的方案?(提示：今天已有 Kodak 影像庫、DIV2K、BSDS 等大量高品質的替代測試集)

1.1 什麼是影像處理

1.1.1 定義與範疇

數位影像處理(Digital Image Processing)是以電腦對數位影像進行分析與操作的技術。最基本的定義是：輸入一張影像，輸出另一張影像或從中萃取出的資訊。前者例如去除雜訊、增強對比、校正變形;後者例如判斷影像中有幾個物件、它們在哪裡、是什麼。

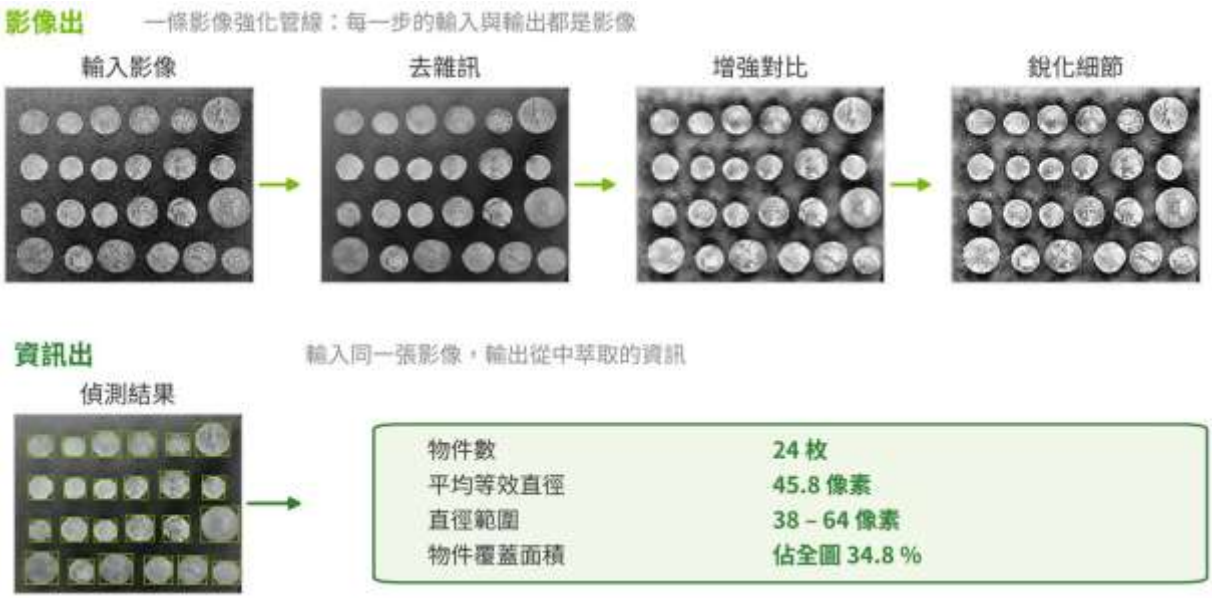


圖 1-1　影像處理的兩種輸出：影像出與資訊出

為什麼需要它?因為人眼雖然強大，卻有三個限制。其一，人眼只能看見電磁波譜中極窄的一段(可見光)，而 X 光、紅外線、超音波、雷達所攜帶的資訊，必須經過處理才能轉換成人眼可讀的形式。其二，人會疲勞、判斷會不一致——一位品檢員連續工作八小時後的判斷，與剛上班時未必相同，而機器不會。其三，人的處理速度有限——一條產線每秒通過數十個工件、一台自駕車每秒接收數十影格影像，這是人力無

法應付的規模與速度。

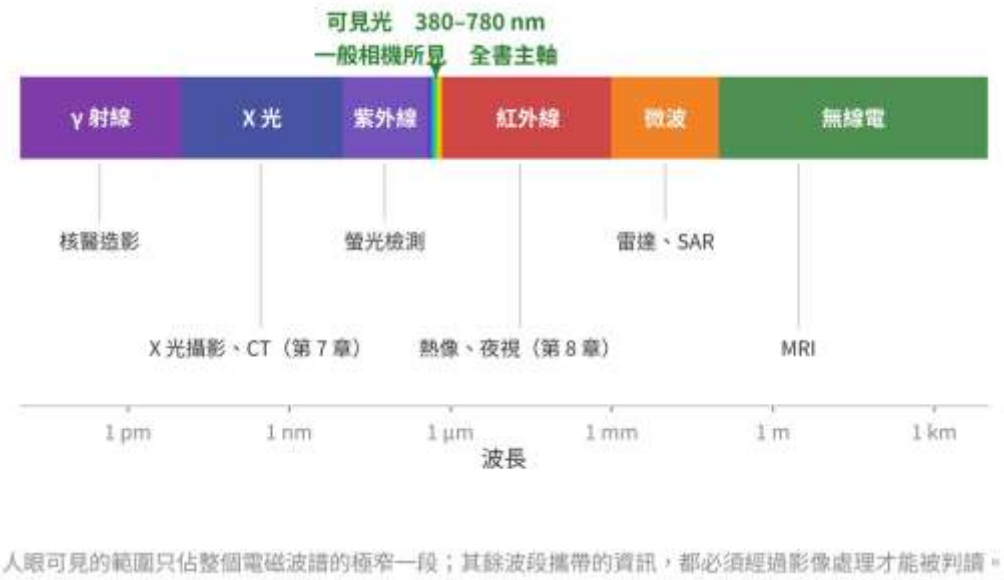


圖 1-2　電磁波譜與各波段的成像技術

1.1.2 三個相鄰領域的關係

影像處理常與另外兩個領域混淆，釐清它們有助於建立整體樣貌：

1. 影像處理(Image Processing)：影像進、影像出。重點在改善影像品質或轉換其表示方式，例如去雜訊、銳化、壓縮、幾何校正。本書第 4 至 12 章大多屬於此範疇。
2. 電腦視覺(Computer Vision)：影像進、資訊或理解出。重點在讓機器「看懂」影像的內容，例如辨識物件、估計距離、理解場景。本書第 13 至 17 章走向此範疇。
3. 電腦圖學(Computer Graphics)：資訊進、影像出。重點在由模型與參數合成影像，例如 3D 渲染、動畫、遊戲畫面。它與影像處理恰好方向相反。

三者的界線在實務上早已模糊。第 15 章的生成式模型「由文字生成影像」，某種意義上正是圖學與視覺的融合;而第 16 章的物件偵測，既需要影像處理的前處理，也需要電腦視覺的理解。本書採取的立場是：以影像處理為地基，自然地延伸到電腦視覺——這也是為什麼書名是《影像處理》，但後半部大量談論辨識與學習。



圖 1-3　影像處理、電腦視覺與電腦圖學的方向關係

1.1.3 低階、中階與高階視覺

一個更有用的分類方式，是依「抽象程度」把整個處理流程分成三層。這個三層架構是理解本書章節安排的鑰匙，請務必牢記：

層級	輸入與輸出	典型工作	本書對應章節
低階視覺	影像進、影像出	去雜訊、增強、還原、幾何轉換、壓縮	第 3-8、11 章
中階視覺	影像進、屬性出	分割、型態學、特徵擷取、追蹤	第 9、10、12、13 章
高階視覺	屬性進、理解出	辨識、偵測、場景理解、決策	第 14-17 章

表 1-1 低階、中階、高階視覺與本書章節的對應

這三層不是彼此獨立的階段，而是層層堆疊的支撐關係：低階處理的品質決定了中階分割的成敗(第 7.7 節會量化這一點)，而中階萃取的特徵又決定了高階辨識的上限。這也解釋了為什麼本書要花費大量篇幅在看似「基礎」的前半部——沒有穩固的地基，再先進的深度學習模型也蓋不高。

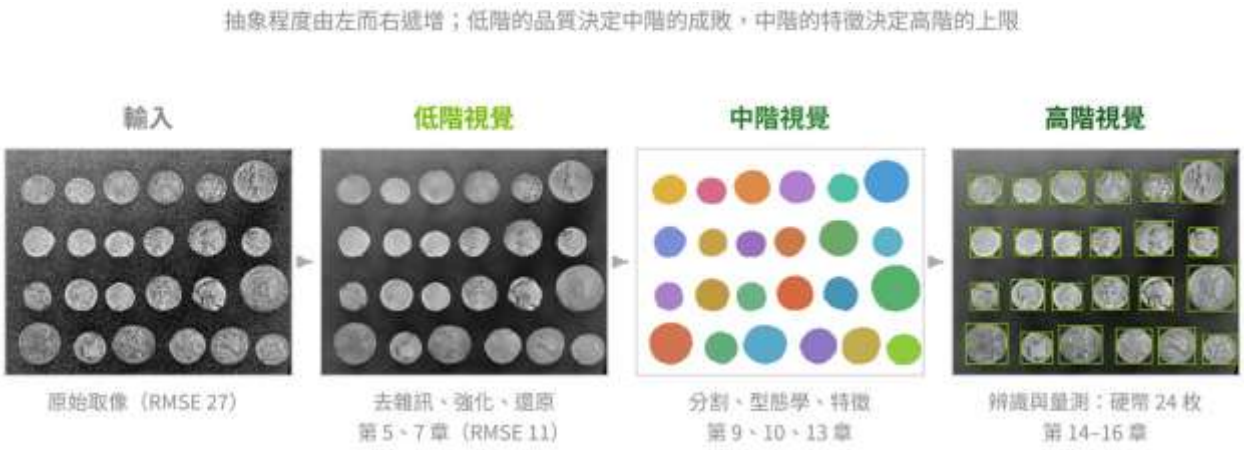


圖 1-4　同一張影像走過低階、中階與高階視覺

1.2 影像處理發展史：四個階段

理解一項技術的歷史，能幫助你看清它「為什麼長成現在這樣」。影像處理的發展大致可分為四個階段，而每個階段的驅動力都不相同。

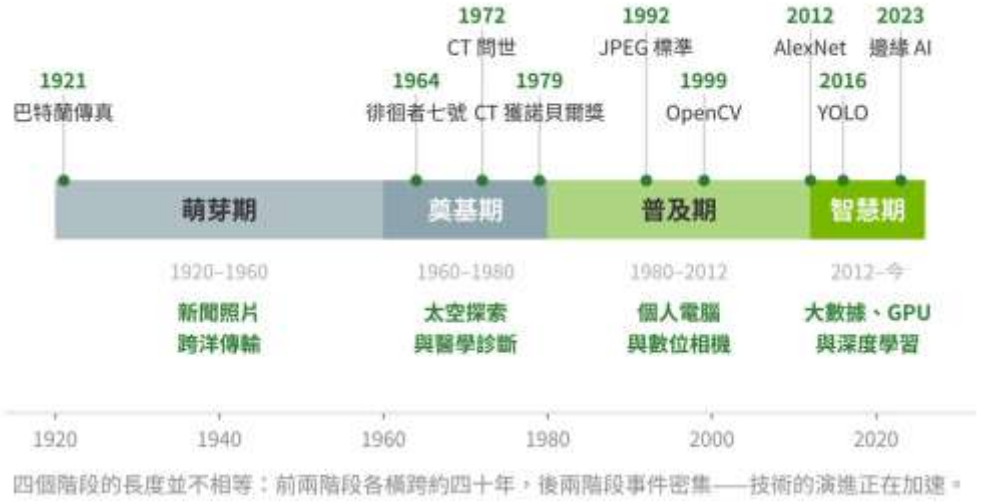


圖 1-5　影像處理發展的四個階段與代表事件

1.2.1 萌芽期：報紙傳真(1920 年代)

數位影像的最早應用之一，出現在 1920 年代橫越大西洋的海底電纜。當時的巴特蘭(Bartlane)傳輸系統把照片編碼成訊號，經由電纜傳送，再在另一端重建成影像——原本需要一週船運的新聞照片，縮短到數小時。早期系統只能表現五個灰階，後來提升到十五階。這個階段還談不上「處理」，但它建立了一個關鍵觀念：影像可以被離散化成數字來傳輸與儲存。這正是第 3 章取樣與量化的先聲。



圖 1-6　灰階數對影像品質的影響：從 2 階到 256 階

1.2.2 奠基期：太空探索與醫學影像(1960-1970 年代)

真正推動影像處理成為一門學科的，是兩股力量。第一股來自太空：1964 年，美國噴射推進實驗室(JPL)處理由徘徊者七號探測器傳回的月球照片，以電腦校正相機造成的幾何失真與雜訊——這被公認為現代數位影像處理的起點。往後的水手號、阿波羅任務持續推動技術演進，而第 7 章叫獸開講中哈伯望遠鏡的反卷積救援，正是這條血脈的延續。

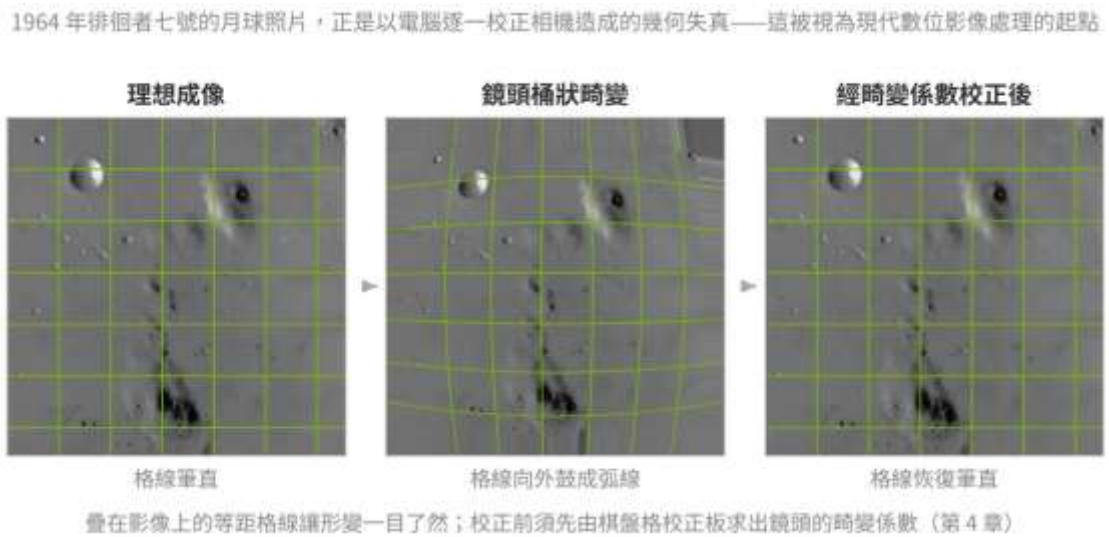


圖 1-7　鏡頭幾何失真與校正

第二股力量來自醫學。1970 年代電腦斷層掃描(CT)問世，它從不同角度拍攝的 X 光投影中「重建」出人體的橫切面影像——這項成就讓 Hounsfield 與 Cormack 獲得 1979 年諾貝爾生理醫學獎，也讓影像處理第一次直接關乎人命。此後的磁振造影

(MRI，第 7 章的維納濾波案例)、超音波、正子斷層造影，共同構成了現代醫學不可或缺的「看見身體內部」的能力。

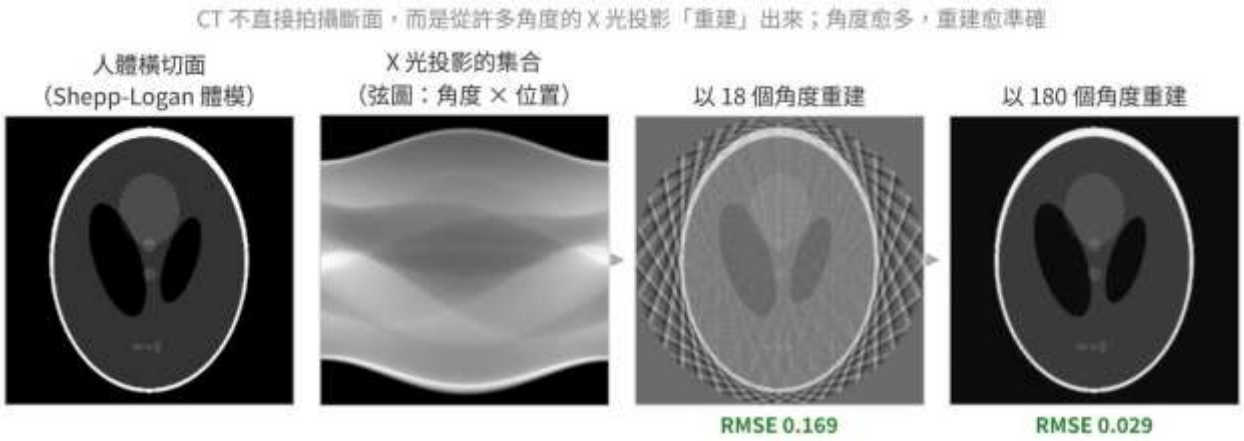


圖 1-8　CT 的斷面重建：從多角度投影到斷層影像

值得一提的是，這個階段的驅動力都不是「為了做影像處理而做影像處理」，而是有一個非做不可的真實需求——看清月球、看見身體內部。這個模式在往後的每個階段都不斷重演，也是叫獸希望你在期末專題(第 18 章)中記住的：好的技術，總是從真實的問題出發。

1.2.3 普及期：個人電腦與數位相機(1980-2000 年代)

隨著電腦運算能力提升與成本下降，影像處理從國家級實驗室走進了企業與家庭。這個階段的關鍵發展包括：JPEG 壓縮標準的制定(1992 年，第 11 章)讓數位影像得以在有限的儲存與頻寬中流通;數位相機取代底片，使影像的產生本身就是數位的;工業自動光學檢測(AOI)開始取代人工目檢;而 1999 年 Intel 發起的 OpenCV 專案(第 2 章叫獸開講)則把影像處理的工具開放給了全世界。這也是萊娜圖最風光的年代——叫獸開講中那些一代代被測試的演算法，大多誕生於此時。

1.2.4 智慧期：深度學習與邊緣運算(2012 年至今)

2012 年是分水嶺。AlexNet 在 ImageNet 競賽上的壓倒性勝利(第 15 章叫獸開講)，讓「讓機器自己學特徵」取代了「人工設計特徵」成為主流。此後十餘年，辨識準確率快速逼近甚至超越人類，應用從實驗室湧向產業：自駕車、人臉支付、醫療輔

助診斷、智慧製造。

近年的發展則呈現兩個方向：一是往「更大」走——基礎模型、多模態大模型、生成式 AI 模糊了辨識與創造的界線;二是往「更小」走——邊緣運算讓 AI 走出資料中心，進入手機、感測器與機器人(第 17 章)。而對本學程的學生而言，後者尤其重要：機器人要在真實世界中行動，運算就必須發生在現場。

階段	年代	驅動力	代表性成果
萌芽期	1920 年代	新聞照片的跨洋傳輸	巴特蘭電纜傳真系統
奠基期	1960-70 年代	太空探索、醫學診斷	月球照片校正、CT 斷層重建
普及期	1980-2000 年代	個人電腦、數位相機、工業自動化	JPEG 標準、AOI 檢測、OpenCV
智慧期	2012 年至今	大數據、GPU 算力、深度學習	AlexNet、YOLO、生成式 AI、邊緣 AI

表 1-2 影像處理發展的四個階段

1.3 應用領域巡禮

影像處理已滲透到現代生活的每個角落。以下巡禮不求詳盡，而是希望讓你看見「這門課能做什麼」，並在其中找到你想投入的方向(第 18 章的專題選題會用到)。

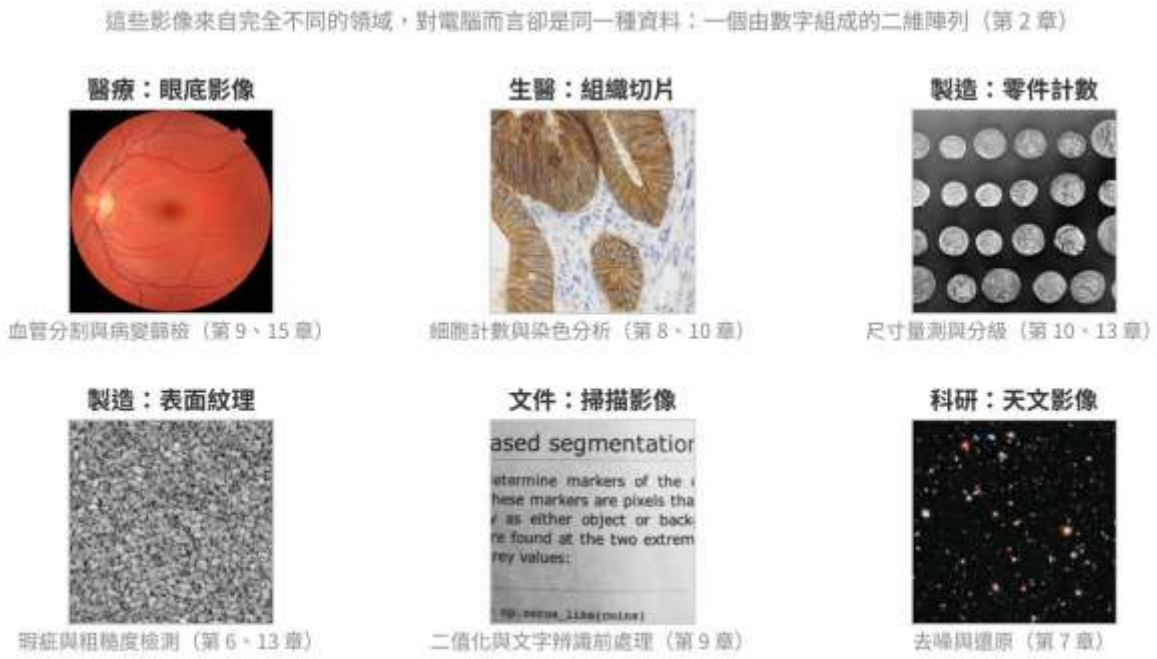


圖 1-10　不同領域的影像，對電腦而言都是同一種資料

- 醫療與生醫：CT/MRI 影像的重建與去噪(第 7 章)、病灶分割與量測(第 9 章)、細胞計數(第 10 章)、輔助診斷模型(第 15 章)。這是影像處理最早也最深刻改變人類生活的領域。
- 智慧製造：自動光學檢測(AOI)找出工件瑕疵(第 6、9、16 章)、零件計數與尺寸量測(第 10 章)、機械手臂的視覺定位(第 4、13 章)、產線人員的安全監控(第 16 章)。這是本學程畢業生最主要的就業領域之一。
- 農業與環境：果實計數估產、病蟲害辨識、雜草偵測、無人機田間巡檢(第 16、17 章)、衛星遙測的植生分析。對雲林這個農業大縣而言，這是最具在地價值的應用方向。
- 交通與自駕：車道線偵測(第 9 章)、車輛與行人偵測追蹤(第 12、16 章)、車牌辨識(第 13 章)、車流分析與測速。
- 機器人：視覺導航與避障、視覺里程計與 SLAM(第 12、13、17 章)、物件抓取的姿態估計、人機協作的安全防護。這是本書的最終目的地。

- 消費性應用：手機的夜景模式與計算攝影(第 5 章)、人臉解鎖(第 10、15 章)、影像美化與濾鏡、擴增實境。這些是你每天都在使用、卻可能沒意識到的影像處理。
- 科學研究：天文影像的還原(第 7 章)、顯微影像分析、材料表面檢測、生態調查的自動物種辨識。

觀察這份清單，你會發現一個共通點：影像處理很少是獨立的產品，它總是「某個更大系統中的一環」。醫師需要的不是分割演算法，而是更準確的診斷;工廠需要的不是 YOLO 模型，而是更低的不良率。這個認知很重要——它提醒我們，技術的價值取決於它解決了什麼問題，而不是它有多先進(這個判準會在第 9、11、16、17 章反覆出現)。

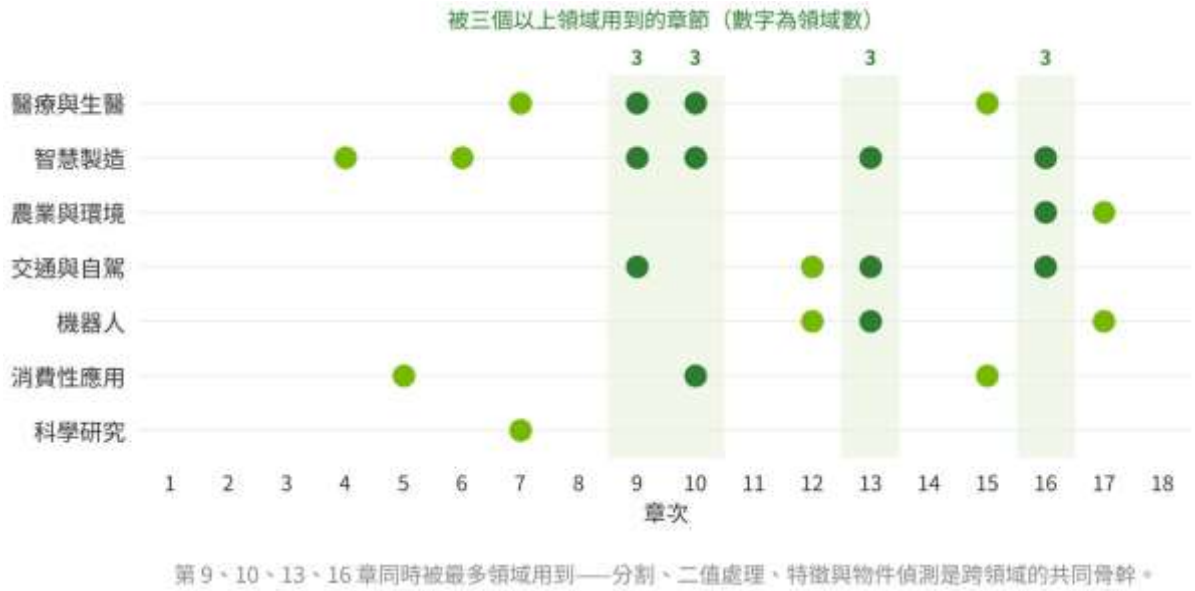


圖 1-9　七大應用領域與本書章節的對照

1.4 標準測試影像與技術社群文化

承叫獸開講，本節深入談一個容易被忽略、卻對整個領域影響深遠的主題：標準測試影像。

1.4.1 為什麼需要標準測試影像

當研究者甲宣稱他的去雜訊演算法比研究者乙的好，如何驗證?若兩人各自用自己

的照片測試，結果無從比較——也許甲的照片本來就比較容易處理。因此，領域內需要共同的基準影像與資料集，讓不同方法能在相同的立足點上比較。這與第 14 章的 MNIST、ImageNet、COCO 扮演的角色完全一致，只是規模不同：標準測試影像是「單張」的基準，而資料集是「大規模」的基準。

好的測試影像需要具備多樣的視覺特性：平滑區域(檢驗是否過度平滑)、銳利邊緣(檢驗是否模糊)、細緻紋理(檢驗細節保留)、豐富色彩與明暗層次(檢驗動態範圍)。萊娜圖恰好同時具備這些特性——這正是它當年被廣泛採用的技術理由。

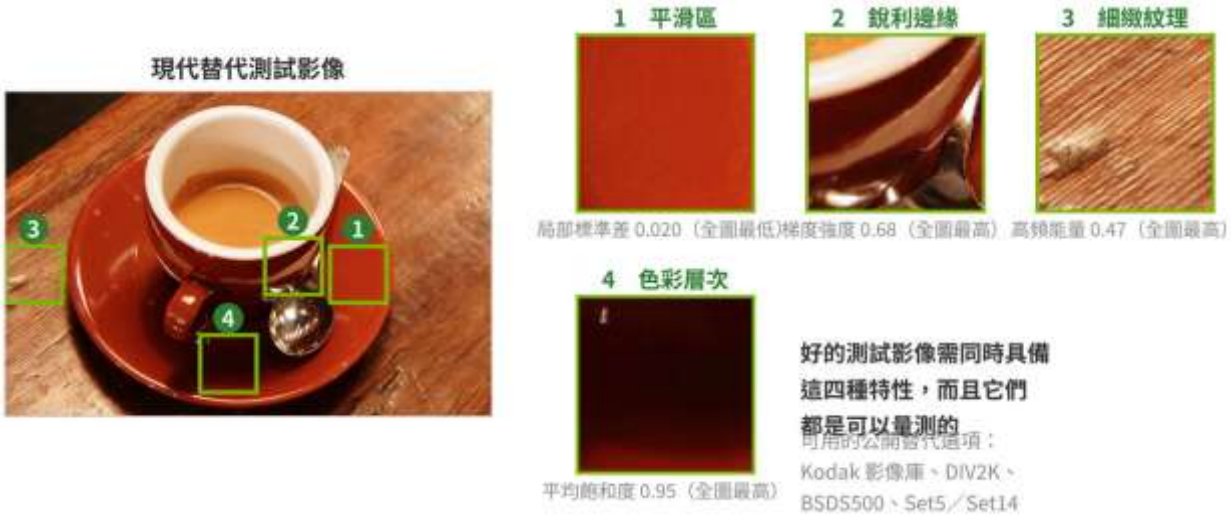


圖 1-11　好的測試影像應具備的四種視覺特性

1.4.2 現代的替代選項

今天已有大量高品質且無爭議的替代選擇，建議同學在作業與專題中優先採用：柯達(Kodak)影像庫的二十四張自然影像、DIV2K 高解析度資料集、BSDS 邊緣與分割基準集、Set5 與 Set14 超解析度基準，以及各領域的專用資料集。OpenCV 與 scikit-image 等函式庫也內建了可自由使用的範例影像。技術上完全不遜色，而且沒有任何倫理包袱——這正是叫獸開講思考題所說的「兩全方案」。

1.4.3 從測試影像看社群文化

萊娜圖事件真正的意義，不在於一張照片的存廢，而在於它揭示了一個道理：一個專業社群的日常習慣，會無聲地傳遞「誰屬於這裡」的訊息。當一個領域的入門教

材、研討會投影片、示範程式碼中反覆出現某一類影像時，它就在定義這個領域的文化樣貌。

這個道理在人工智慧時代更加重要，因為現在我們不只是「選擇測試影像」，而是「選擇訓練資料」——而模型會忠實地學習資料中的一切，包括其中的偏誤。第 14 章的肺炎 X 光案例、第 15 章談到的人臉超解析度偏誤，都是這個問題的延伸。身為未來的工程師，你的每一個資料選擇，都在形塑技術的樣貌與它服務的對象。

1.5 本書架構與學習地圖

1.5.1 四篇十八章

本書共十八章，分為四篇，對應 1.1.3 節的三層視覺架構，並以整合實踐收尾：

篇	章節	內容與定位
第一篇 基礎	第 1-3 章	緒論、開發環境與程式基礎、數位影像基礎(取樣量化、色彩、品質評估)
第二篇 核心	第 4-12 章	幾何轉換、空間域強化、頻率域、影像還原、色彩處理、分割、二值型態學、小波、視訊
第三篇 學習	第 13-16 章	特徵擷取、機器學習基礎、深度學習、物件偵測 YOLO 實戰
第四篇 整合	第 17-18 章	邊緣 AI 與機器人視覺部署、期末專題 4P 實踐

表 1-3 本書四篇架構

這個安排有其刻意：深度學習被放在第 15 章而非開頭，是因為叫獸相信——唯有先理解卷積、頻率、多尺度、特徵這些基本觀念，你才能真正看懂 CNN 在做什麼(第 15.1 節會回頭印證這一點)。反之，若一開始就直接學 YOLO，你會很快做出成果，卻只是在呼叫別人寫好的函式，遇到問題時無從診斷。



箭頭代表主要的先後依賴：第 5 章的濾波是第 7、9 章的基礎，第 9、10 章的輸出是第 13 章的輸入，而第 13 章的特徵正是第 14、15 章要讓機器自己學會的東西。

圖 1-12　本書十八章的學習地圖與章間依賴

1.5.2 貫穿全書的核心思想

比起個別的演算法，更重要的是那些會在不同章節以不同面貌反覆出現的思想。請在閱讀時留意它們，遇到時在心中打個記號：

- 換一個域看問題：空間域解不開的問題，到頻率域可能一目了然(第 6 章);傅立葉看不見位置，小波補上了(第 11 章)。
- 多解析度與由粗到細：小波金字塔(第 11 章)、SIFT 尺度空間(第 13 章)、CNN 的層級特徵(第 15 章)，骨子裡是同一種思維。
- 投票對抗雜訊：霍夫轉換讓邊緣點投票給直線(第 9 章)、RANSAC 讓匹配投票給幾何模型(第 13 章)——以多數證據壓制少數錯誤。
- 正則化對抗病態：維納濾波在分母加項以避免雜訊爆炸(第 7 章)、資料增強與 Dropout 避免模型死背(第 14、15 章)，都是「以約束換穩定」。
- 精確率與召回率的取捨：Canny 的雙門檻(第 9 章)、偵測器的信心門檻(第 10、16 章)、評估指標的選擇(第 14 章)——沒有最好的設定，只有最適合任務的設定。

- 適合勝於最強：ESP32 上跑得動的 Haar 級聯勝過跑不動的 YOLO(第 10 章)、夠好又普及的 JPEG 勝過技術更優的 JPEG 2000(第 11 章)。
- 技術背後的責任：從本章的萊娜圖、第 7 章的還原與腦補、第 14 章的資料偏誤，到第 16 章 YOLO 之父的選擇——技術能力愈強，選擇就愈重要。

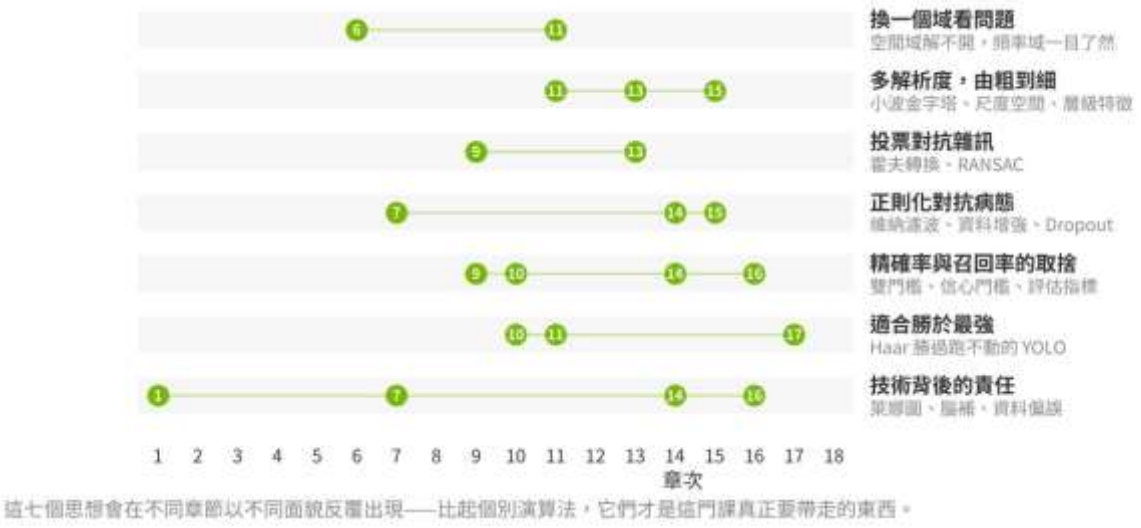


圖 1-13　七個貫穿全書的核心思想及其出現章節

1.5.3 三層實作平台

本書的實作橫跨三個層級的平台，對應第 2 章與第 17 章詳述的三種 AI 辨識架構：

4. ESP32-CAM(入門平台)：極低成本、極低功耗的視覺感測節點。用於理解「端上運算」的能力與限制，以及大規模佈署的可能性。第 2 章會帶你把它接起來，第 10 章會在它上面部署一個真正會動的人臉偵測門禁。
5. 樹莓派 5 加 AI Camera(部署平台)：一台完整的 Linux 單板電腦，配上把推論做在感測器晶片裡的相機模組。主機拿到的是「畫面裡有什麼」而不是「畫面」，因此省下頻寬與主機時間，也讓影像根本不必離開感測器。第 17 章會完整介紹它的能力與限制。
6. NVIDIA Jetson Orin Nano Super(進階平台)：內建 GPU 的邊緣 AI 電腦，能即時執行深度學習模型、ROS 2 與開放詞彙偵測，是機器人的視覺大腦(第 16、17 章)。

比較面向	ESP32-CAM 入門平台	樹莓派 5 + AI Camera 感測器上推論	Jetson Orin Nano Super 邊緣 AI 電腦
運算核心	Xtensa LX6 雙核 MCU	ARM CPU + 感測器內建推論加速器	ARM CPU + CUDA GPU
記憶體量級	數百 KB	數 GB	數 GB 以上
功耗量級	百毫瓦	數瓦	可設定的十瓦量級
成本量級	數美元	數十美元	數百美元
能跑的模型	傳統演算法 Haar 級聯	感測器上的 INT8 小模型 (YOLO nano 這一級)	YOLO、CNN、ROS 2 開放詞彙模型
最適合的事	偵測、觸發、串流 大規模佈點	類別固定、影像不出裝置 主機要留著做別的事	即時辨識、追蹤、SLAM 機器人的視覺大腦
本書章節	第 2、10 章	第 17 章	第 16、17 章

多數章節的 Lab 可先在個人電腦上驗證邏輯，再移植到目標平台——這正是嵌入式開發的標準工作流程。

圖 1-14　三層實作平台對照

多數章節的 Lab 可以先在個人電腦上完成(使用 Python 與 OpenCV，第 2 章會建立環境)，再移植到目標平台。這個「先在電腦上驗證邏輯、再移植到目標硬體」的流程，本身就是嵌入式開發的標準工作方式。

1.5.4 4P 學習理念

本課程的教學設計遵循 MIT 媒體實驗室 Mitchel Resnick 提出的 4P 創意學習理念——專題(Projects)、熱情(Passion)、同儕(Peers)、遊玩(Play)。具體體現在：每章都有動手做的 Lab(Projects)、鼓勵你選擇自己在乎的專題題目(Passion)、分組協作與互相回饋(Peers)、以及允許失敗、鼓勵嘗試的評量方式(Play)。



每章的 Lab 是「專題」，叫獸開講點燃「熱情」，課堂討論是「同儕」，而 Lab 的開放題是「遊玩」。

圖 1-15　4P 創意學習螺旋

這也是為什麼每章開頭都有「叫獸開講」——它們不是花絮，而是想讓你看見：每一項技術背後都有活生生的人，帶著各自的困境、熱情與選擇。第 18 章會完整展開這套理念，並帶你把它應用到期末專題上。

1.6 應用案例

1.6.1 案例一：從月球照片到哈伯望遠鏡

1964 年，徘徊者七號傳回的月球照片因相機特性與傳輸雜訊而失真，JPL 的工程師以電腦逐一校正——這是現代數位影像處理公認的起點。二十六年後的 1990 年，哈伯太空望遠鏡因主鏡研磨誤差而近乎失明，天文學家再次以數學方法(反卷積)從模糊資料中救回科學成果(第 7 章叫獸開講)。

這兩件相隔四分之一世紀的事，體現了同一個精神：當硬體有其物理極限時，數學與運算可以把資訊救回來。這正是第 7 章影像還原的核心命題，也是叫獸希望你在遇到「感測器不夠好」的困境時，能想起的另一條路。

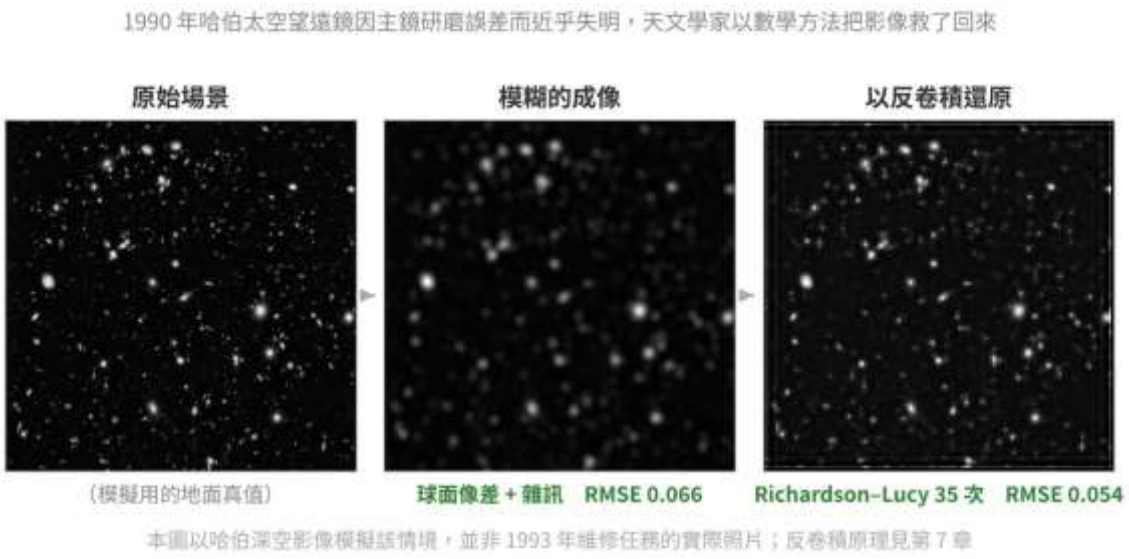


圖 1-16　當硬體有極限時：模糊成像與反卷積還原

1.6.2 案例二：AOI 如何改變製造業

在自動光學檢測普及之前，工廠的品質檢驗仰賴人工目視。問題不只是速度慢，更在於一致性——不同檢驗員的標準不同、同一個人在不同時段的判斷也不同，且長

時間注視細小物件極易疲勞。AOI 系統以固定的打光、相機與演算法，提供了穩定且可追溯的檢測。

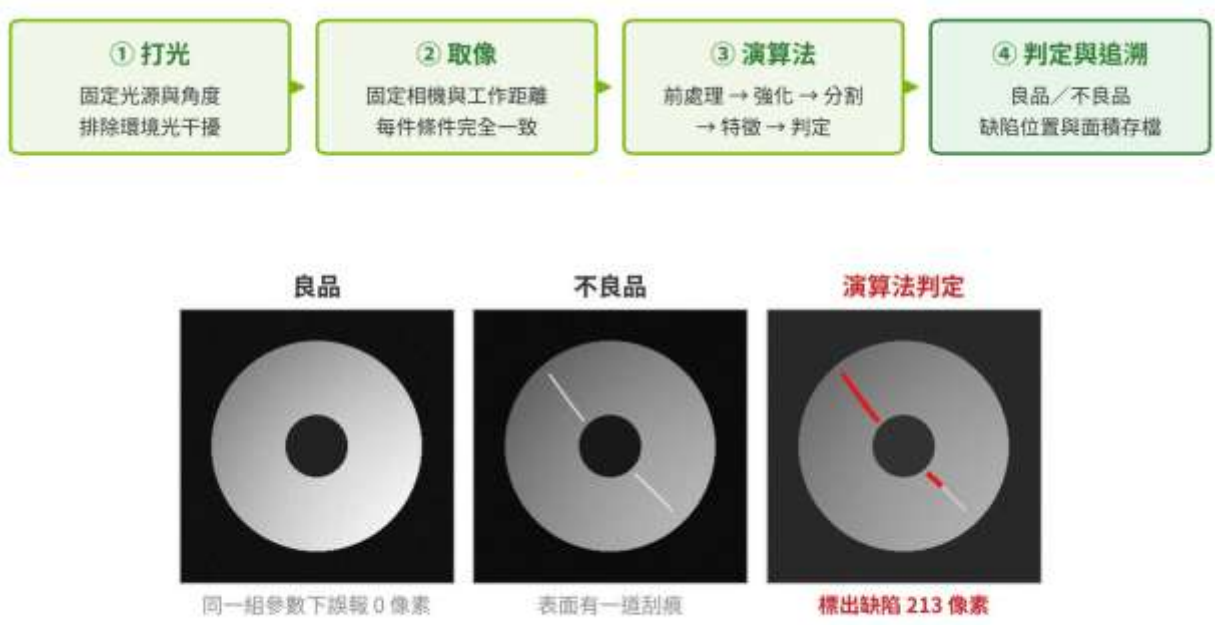


圖 1-17　AOI 自動光學檢測的流程與判定

這個案例的教學價值在於：它示範了影像處理的典型價值主張——不是「比人更聰明」，而是「比人更穩定、更快、不會累」。這個定位很重要，它也提醒我們設計系統時該把人放在哪裡：讓機器做重複而穩定的判斷，讓人做需要脈絡與經驗的決策(第 9 章醫學影像、第 16 章人機協作都會回到這個主題)。

1.6.3 案例三：手機裡的計算攝影

你口袋裡的手機，是當代最普及的影像處理裝置。按下快門的那一刻，它其實做了遠不只「拍一張照」的事：連拍多張影像對齊後合併以降低雜訊、依場景自動調整白平衡與色調、以深度學習分離主體與背景製造淺景深、甚至用超解析度重建細節。第 5 章叫獸開講會完整拆解夜景模式的原理。

這個案例說明了一件事：當硬體受限(手機鏡頭與感光元件都很小)時，軟體可以補上差距。它也是本書多章技術的綜合展示——多影格平均(第 5 章)、影像對齊(第 6、

13 章)、色彩處理(第 8 章)、分割(第 9 章)、深度學習(第 15 章)全都在你按下快門的那一秒內發生。

1.6.4 案例四：雲林在地的農業視覺

雲林是台灣重要的農業縣，而農業正面臨人口老化與缺工的挑戰。影像處理在此有明確的著力點：以無人機或定點相機自動巡檢作物、辨識病蟲害的早期徵兆、估計產量以協助銷售規劃、辨識雜草以精準施藥減少用量。

這類應用的技術挑戰很典型：戶外光線變化劇烈(第 5、8 章的前處理成為必要)、標註資料稀少(第 15 章的遷移學習與資料增強)、田間沒有穩定網路(第 17 章的邊緣運算)。它們不是最尖端的研究題目，卻是最能產生在地價值的方向——也是叫獸最鼓勵同學在期末專題(第 18 章)嘗試的領域。

1.7 本章與全書的觀念連結總表

作為全書的開篇，本章的每一節都是後續章節的入口。這張表既是本章的整理，也是你日後回頭查找的索引。

本章觀念	後續展開的章節	如何深化
影像的數位化	第 3 章	取樣定理、量化、位元深度、品質評估指標
低階視覺	第 4-8、11 章	幾何轉換、強化、頻率域、還原、色彩、壓縮
中階視覺	第 9、10、12、13 章	分割、型態學、追蹤、特徵擷取
高階視覺	第 14-16 章	機器學習、深度學習、物件偵測
太空與醫學的推力	第 7、9 章	哈伯反卷積、MRI 去噪、病灶分割
標準測試與基準	第 3、14 章	PSNR/SSIM 指標、MNIST/ImageNet/COCO 資料集

資料與文化偏誤	第 14、15、16 章	捷徑學習、模型偏誤、技術倫理與工程師責任
三種辨識架構	第 2、10、17 章	端上、近端邊緣、雲端的取捨與混合架構
4P 學習理念	第 18 章與各章 Lab	專題、熱情、同儕、遊玩;創意學習螺旋

表 1-4 第 1 章與全書其他章節的觀念連結

程式實作 Lab 1

Lab 1-1 影像處理應用觀察

目標：建立對這門課的整體感受。(a)在你的日常生活中，找出五個「背後一定用到影像處理」的產品或服務(例如手機相機、超商的商品辨識、停車場的車牌辨識、社群軟體的濾鏡、醫院的檢查儀器);(b)對每一個，推測它可能用到本書哪幾章的技術，並說明理由;(c)挑其中一個做深入調查：它解決什麼問題?若沒有它，人們原本怎麼做?它的限制是什麼?(d)小組分享並互相補充。

Lab 1-2 我的第一張影像

目標：在正式學習前先建立直覺(第 2 章會建立完整環境，本 Lab 可用線上環境或手機 App 完成)。(a)用手機拍攝三張照片：一張明亮清晰、一張昏暗、一張模糊;(b)使用任何影像編輯工具(手機內建、小畫家、GIMP 或線上工具)，嘗試調整亮度、對比、銳利度，觀察哪些問題「救得回來」、哪些「救不回來」;(c)特別觀察：昏暗的照片調亮後，是否出現原本看不見的雜訊?模糊的照片銳化後，細節是真的回來了嗎?(d)把你的觀察寫下來——第 5 章(強化)與第 7 章(還原)會回頭解釋這些現象背後的原理，屆時請翻回你的筆記對照。

Lab 1-3 標準測試影像調查

目標：實踐叫獸開講的反思。(a)上網查詢 Kodak 影像庫、DIV2K、BSDS500 等

公開測試資料集，各下載幾張影像;(b)觀察它們具備哪些視覺特性(平滑區、銳利邊緣、細緻紋理、色彩層次)，說明為什麼這些特性對測試演算法很重要;(c)選出三張你認為最適合作為本課程測試影像的圖，說明理由，並在往後各章的 Lab 中固定使用它們——這樣你就能跨章比較不同演算法在同一張影像上的效果;(d)討論題：承叫獸開講，當「技術上好用」與「對某些人不友善」衝突時該如何權衡?請提出你的看法與理由。

Lab 1-4 學習地圖與目標設定

目標：為這門課設定屬於你的方向。(a)瀏覽本書目錄與表 1-3 的四篇架構，標出你最感興趣的三章與最擔心的三章;(b)參考 1.3 節的應用領域，選出一個你最想投入的方向，寫下你想解決的一個具體問題;(c)為自己設定一個學期目標(例如「做出能辨識我家果園病害的系統」、「弄懂深度學習到底在學什麼」);(d)把這份紀錄收好，第 18 章期末專題與反思時會請你回頭檢視——那時你會很驚訝自己走了多遠。

Lab 1-5 重現本章的插圖

目標：把本章的圖變成你自己的第一支程式(建議在第 2 章建立好環境後再進行)。(a)取用 scikit-image 內建的硬幣影像，重現圖 1-6 的量化效果——分別量化到 2、5、15、256 階並排比較，說明從幾階開始你看不出差異;(b)對同一張影像加入高斯雜訊，計算它與原圖的均方根誤差，再用任一種去雜訊方法處理後重算一次，確認誤差下降(本章圖 1-4 的數值為 27 降到 11);(c)以邊緣偵測加填洞的方式數出影像中有幾枚硬幣，與圖 1-1 標註的 24 枚對照，若數字不同請找出原因;(d)把你的結果與本章的圖並列，說明哪些步驟你已經做得出來、哪些還需要後面哪一章的知識。

Lab 1-6 設計你自己的 AOI 題目

目標：把 1.6.2 節與圖 1-17 的流程套用到真實物件上，而且全程不需要寫程式。(a)從生活中挑一種可以分出「良品 / 不良品」的小物件(餅乾、螺絲、名片、瓷杯、電路板皆可)，各準備至少五個樣本;(b)設計取像條件：固定相機高度與角度、固定背景，並嘗試至少三種打光方式(正面光、側向光、背光)拍攝同一個缺陷;(c)比較三種打光下

缺陷的可見程度，說明哪一種最適合、為什麼——這是整條管線最關鍵的一步;(d)手繪一張你的檢測流程圖，標出每一步會用到本書哪一章;(e)預測哪些缺陷用傳統影像處理就能找到、哪些非得靠深度學習不可，並說明你的判斷依據。

本章重點整理

- 數位影像處理以電腦分析與操作影像，輸出可以是影像(強化、還原)或資訊(辨識、量測);它彌補了人眼在光譜範圍、一致性與速度上的三個限制。
- 影像處理(影像進影像出)、電腦視覺(影像進資訊出)、電腦圖學(資訊進影像出)三者方向不同但界線日益模糊;本書以影像處理為地基延伸到電腦視覺。
- 三層視覺架構：低階(去雜訊、強化、還原)、中階(分割、特徵)、高階(辨識、理解)，層層堆疊——低階品質決定中階成敗，中階特徵決定高階上限。
- 發展史四階段：萌芽期(1920 年代電纜傳真)、奠基期(1960-70 年代太空與 CT)、普及期(1980-2000 年代 PC、JPEG、AOI、OpenCV)、智慧期(2012 年 AlexNet 至今的深度學習與邊緣運算);每個階段都由真實需求驅動。
- 標準測試影像與資料集讓不同方法能公平比較;萊娜圖因其技術特性流行半世紀，亦因文化包容性爭議於 2024 年被 IEEE 計算機協會停用，現有 Kodak、DIV2K、BSDS 等替代選項。
- 本書四篇十八章對應三層視覺並以專題整合;貫穿全書的核心思想包括換域看問題、多解析度、投票對抗雜訊、正則化對抗病態、精確率召回率取捨、適合勝於最強，以及技術背後的責任。

習題

1. 請以自己的話定義數位影像處理，並說明為什麼需要它(人眼有哪三個限制)。

> *提示與參考解答：定義：以電腦對數位影像進行分析與操作，輸入影像、輸出另一張影像或從中萃取的資訊。人眼的三個限制：一、只能看見可見光，X 光/紅外線/超音波等須經處理才能判讀;二、會疲勞且判斷不一致(檢驗員八小時後的標準與剛上班*

不同);三、速度有限，無法應付產線每秒數十件或自駕每秒數十影格的規模。

2. 請區分影像處理、電腦視覺與電腦圖學，各以「輸入與輸出」說明，並各舉一例。

   提示與參考解答：影像處理：影像進、影像出，重點在改善品質或轉換表示(如去雜訊、壓縮)。電腦視覺：影像進、資訊或理解出，重點在看懂內容(如辨識物件、估計距離)。電腦圖學：資訊進、影像出，由模型與參數合成影像(如 3D 渲染、遊戲畫面)，方向與影像處理相反。三者界線在生成式 AI 與現代系統中日益模糊。

3. 說明低階、中階、高階視覺的差異與對應的典型工作。為什麼說三者是「層層堆疊」而非彼此獨立?

   提示與參考解答：低階：影像進影像出，如去雜訊、強化、還原、幾何轉換。中階：影像進屬性出，如分割、型態學、特徵擷取、追蹤。高階：屬性進理解出，如辨識、偵測、場景理解。層層堆疊是因為低階處理的品質決定中階分割的成敗(雜訊產生假邊緣、模糊使邊界偏移)，而中階萃取的特徵又決定高階辨識的上限——地基不穩，上層再先進也蓋不高。

4. 影像處理發展史的四個階段分別由什麼力量驅動?這對「如何找到好的研究或專題題目」有什麼啟示?

   提示與參考解答：萌芽期(1920s)：新聞照片跨洋傳輸的需求。奠基期(1960-70s)：太空探索(看清月球)與醫學診斷(看見身體內部)。普及期(1980-2000s)：個人電腦、數位相機、工業自動化。智慧期(2012-)：大數據、GPU 算力與深度學習。啟示：每個階段都不是「為做技術而做技術」，而是有一個非解決不可的真實需求——好的題目應從真實問題出發(第 18 章專題選題的核心原則)。

5. 1964 年的月球照片校正為什麼被視為現代數位影像處理的起點?它與第 7 章的哈伯望遠鏡案例有什麼共通的精神?

   提示與參考解答：因為 JPL 首次以電腦系統性地校正探測器傳回影像的幾何失真與雜訊，是數位影像處理作為學科的開端。與哈伯案例的共通精神：當硬體有其物理極限(相機特性失真、主鏡研磨誤差)時，數學與運算可以把資訊救回來——這正是第 7 章影像還原的核心命題，提醒工程師在「感測器不夠好」時還有另一條路。

6. 為什麼一個領域需要「標準測試影像」?一張好的測試影像應具備哪些視覺特性?

   提示與參考解答：因為若各研究者用自己的影像測試，結果無從比較(也許某人的影像本來就容易處理);需要共同基準讓不同方法在相同立足點上競爭。這與第 14 章 MNIST、ImageNet、COCO 的角色一致，只是規模不同。好的測試影像應具備：平滑區域(檢驗是否過度平滑)、銳利邊緣(是否模糊)、細緻紋理(細節保留)、豐富色彩與明暗層次(動態範圍)。

7. 承叫獸開講：萊娜圖為什麼會流行半世紀?IEEE 計算機協會又為什麼在 2024 年停用它?

   提示與參考解答：流行原因：1973 年南加大 Sawchuk 團隊掃描後，發現它兼具平滑皮膚、銳利羽毛紋理、豐富色彩與明暗對比，是絕佳測試素材;拷貝在社群中廣為流傳，甚至參與 JPEG 標準的發展歷程。停用原因：自 2010 年代中期起，愈來愈多人質疑以《花花公子》中頁作為領域標準素材對女性傳遞的訊息;2024 年 3 月 IEEE 計算機協會發函，自 4 月 1 日起不再接受含該影像的投稿，援引學會多元共融聲明與倫理守則，並尊重當事人 Lena Forsén 的意願。

8. 請提出至少三個可取代萊娜圖的現代測試影像或資料集，並說明為什麼「技術上不遜色」很重要。

   提示與參考解答：替代選項：Kodak 影像庫(二十四張自然影像)、DIV2K(高解析度)、BSDS500(邊緣與分割基準)、Set5/Set14(超解析度基準)，以及 OpenCV 與 scikit-image 內建範例影像。「技術上不遜色」很重要，是因為它證明了叫獸開講思考題所問的「兩全方案」確實存在——不必在技術品質與文化包容之間二選一，這使得改變沒有正當的技術阻力。

9. 本書為什麼把深度學習安排在第 15 章而非開頭?請說明這個安排的理由與可能的代價。

   提示與參考解答：理由：唯有先理解卷積、頻率、多解析度、特徵等基本觀念，才能真正看懂 CNN 在做什麼(第 15.1 節會印證卷積層即可學習的濾波核、首層自發學出邊緣偵測器);若一開始就學 YOLO，雖能快速做出成果，卻只是呼叫他人函式，遇到問題無從診斷。可能的代價：前期較缺乏「炫目」的成果、學習動機需靠 Lab 與叫獸開講維持;此為刻意的取捨，重視長期理解勝於短期成就感。

10. 請說出至少三個貫穿本書的核心思想，並預測它們可能會在哪些章節出現。

    提示與參考解答：可任選三個：一、換一個域看問題(第 6 章頻率域、第 11 章小波)。二、多解析度由粗到細(第 11 章小波金字塔、第 13 章 SIFT、第 15 章 CNN 層級特徵)。三、投票對抗雜訊(第 9 章霍夫轉換、第 13 章 RANSAC)。四、正則化對抗病態(第 7 章維納濾波、第 14 章對抗過度擬合)。五、精確率與召回率取捨(第 9、10、14、16 章)。六、適合勝於最強(第 10、11、16、17 章)。七、技術背後的責任(第 1、7、14、16 章)。

11. 本書為什麼要橫跨三個層級的平台?它們各適合什麼樣的任務?

    提示與參考解答：因為它們對應不同的 AI 辨識架構與資源條件，培養「選對工具」的判斷力。ESP32-CAM(第 2、10 章)：極低成本與功耗的視覺感測節點，適合簡單偵測、觸發、影像串流與大規模佈署，但無法執行深度模型。樹莓派 5 加 AI Camera(第 17 章)：把推論做在感測器晶片裡，主機拿到的是結果而不是影像，適合類別固定、要保護隱私、而且主機還有別的正事要做的場合。Jetson Orin Nano Super(第 16、17 章)：內建 GPU 的邊緣 AI 電腦，能即時執行深度學習、ROS 2 與開放詞彙偵測，適合機器人主控、即時偵測、SLAM。第 17 章會完整比較，實務上也常分層搭配——端上的節點負責第一層過濾，Jetson 作為集中處理與決策的大腦。

12. (反思題)承叫獸開講與 1.4.3 節：一個專業社群的日常習慣如何傳遞「誰屬於這裡」的訊息?在人工智慧時代，這個問題為什麼變得更加重要?

    提示與參考解答：本題無標準答案，重點在論證品質。可能論點：當一個領域的教材、投影片、示範程式中反覆出現某類影像或範例時，它就在無聲地定義這個領域的文化樣貌與預設的參與者形象，影響誰覺得自己受歡迎。AI 時代更重要的理由：現在我們不只選擇測試影像，而是選擇訓練資料，而模型會忠實學習資料中的一切偏誤並大規模複製(第 14 章肺炎 X 光的捷徑學習、第 15 章人臉超解析度的族裔偏誤)。工程師的每一個資料選擇，都在形塑技術的樣貌與它服務的對象。

13. 承圖 1-2：請列舉三種「人眼看不見、必須經過影像處理才能判讀」的成像技術，說明各自所在的波段與本書的對應章節。

    提示與參考解答：X 光攝影與 CT(X 光波段,第 7 章的重建與去噪)、熱像與夜視(紅外線波段,第 8 章以虛擬色彩把溫度映射成人眼可讀的顏色)、雷達與 SAR(微波波段,可

穿透雲層與黑夜)。另有核醫造影(γ 射線)、螢光檢測(紫外線)、MRI(無線電波段)。共通點是：感測器先把人眼接收不到的物理量轉成數值陣列，再由影像處理把數值映射成可見的形式——這正是 1.1.1 節「人眼只能看見極窄一段」這個限制的解法。

14. 承圖 1-1：請各舉三個「影像進、影像出」與「影像進、資訊出」的實例，並說明兩者的輸出在資料型態上有何根本差異。

    提示與參考解答：影像出——去雜訊、增強對比、幾何校正、壓縮、超解析度;輸出仍是一個與輸入同維度的陣列。資訊出——物件計數、尺寸量測、分類標籤、人臉位置、車牌字串;輸出是純量、向量或結構化紀錄，維度遠低於輸入。根本差異在資訊壓縮的程度：前者維持資訊量級，後者把數十萬個像素壓成幾個數字。而「該壓掉什麼、該留下什麼」正是整門課在回答的問題。

15. 承圖 1-4：若低階的去雜訊沒做好，對中階的分割與高階的辨識分別會造成什麼後果?請各舉一個具體現象。

    提示與參考解答：中階——雜訊會在梯度上產生大量假邊緣，使邊緣偵測的輸出破碎、門檻化產生雜點、連通元件標記把一個物件切成數塊或把雜點當成物件，計數因而出錯。高階——特徵描述子建立在梯度之上，雜訊會讓同一個物件在不同影像中算出不同的描述子，匹配失敗，分類器也因拿到不穩定的輸入而準確率下降。這就是「低階的品質決定中階的成敗，中階的特徵決定高階的上限」的具體意義，第 7.7 節會量化這個影響。

16. 承圖 1-6：2 階、5 階、15 階、256 階各需要多少位元?請寫出計算式，並說明巴特蘭系統為什麼只能做到 5 個灰階。

    提示與參考解答：位元數為 $\log_2$(灰階數)。2 階為 1 位元;5 階約 2.32 位元;15 階約 3.91 位元;256 階為 8 位元。實作上不足整數位元時須無條件進位，故 5 階實際需 3 位元、15 階需 4 位元。巴特蘭系統受限於 1920 年代海底電纜的傳輸頻寬與電報編碼方式——每個像素能攜帶的符號數有限，灰階愈多則傳輸時間愈長，只能在「畫質」與「傳得完」之間取捨。這是本書第一次出現「資源限制決定技術選擇」，第 11 章的壓縮與第 17 章的邊緣部署還會再遇到。

17. 承圖 1-7：桶狀畸變與枕狀畸變有何不同?為什麼校正之前要先拍攝棋盤格校正板，而不能直接從一張自然影像推算?

    提示與參考解答：兩者都是徑向畸變。畸變係數為負時影像向外鼓、直線變成外凸

的弧線，稱為桶狀畸變，常見於廣角鏡頭;係數為正時向內縮，稱為枕狀畸變，常見於望遠端。要校正就必須先知道該顆鏡頭的畸變係數，而係數無法從單張自然影像推得——因為我們不知道畫面中的線條原本是不是直的。棋盤格的角點在真實世界中等距且共面，是一組「已知幾何」;比對拍到的角點位置與理論上應在的位置，即可反推係數。第 4 章的張氏標定法就是這套做法。

18. 承圖 1-8：為什麼 CT 的投影角度愈多，重建結果愈準確?本章以 18 個角度與 180 個角度重建，均方根誤差分別為 0.169 與 0.029，請解釋這個差距的來源。

    提示與參考解答：每一個角度的投影只提供「沿該方向的積分值」，是原始斷面的一種壓縮。角度不足時，未被觀測到的方向沒有任何約束，反投影只能沿投影方向把能量均勻塗抹回去，形成放射狀的條紋假影——這正是 18 角度那張圖上明顯的星芒。角度增加後，不同方向的約束互相抵銷了這些假影，誤差因而由 0.169 降到 0.029。這是「資訊量決定可還原的程度」的典型例子，與第 3 章的取樣定理、第 7 章的病態反問題同源。

19. 承圖 1-9：為什麼第 9、10、13、16 章同時被最多領域用到?這對你安排學習的優先順序有什麼啟示?

    提示與參考解答：這四章分別是分割、二值影像處理、特徵擷取與物件偵測，它們回答的都是跨領域的共同問題——物件在哪裡、有幾個、長什麼樣、是什麼。不論領域是醫療的病灶、製造的瑕疵、農業的果實還是交通的車輛，一旦把領域語彙拿掉，剩下的技術需求幾乎相同。啟示是：時間有限時應優先把這幾章讀透，它們的投資報酬率最高;領域專屬的知識可以留到專題階段再補。

20. 承圖 1-17 與 1.6.2 節：為什麼說 AOI 的價值不是「比人更聰明」?又為什麼「打光」會是整條管線的第一步，而不是演算法?

    提示與參考解答：AOI 的強項在穩定度而非智慧——同一套光源、相機與參數下，第一件與第一萬件的判準完全相同，不會因疲勞或換班而漂移，而且每一次判定都可追溯。人類檢驗員在處理需要脈絡與經驗的少數例外時仍然更強，所以合理的分工是「機器做重複而穩定的判斷，人做需要脈絡的決策」。打光排第一，是因為它決定了缺陷與背景的對比:對比若不足，後面再強的演算法也只是在放大雜訊;反之，一個角度對的側向光可以讓刮痕直接變成高對比亮線，使演算法只需一次型態學運

算就能找出來。這呼應全書反覆出現的原則——把問題在最前端解決最便宜。

21. 承 1.6.3 節：手機如何以軟體補足硬體的不足?請列出至少四項技術與其對應章節，並說明它們的共同邏輯。

    提示與參考解答：手機的鏡頭與感光元件都很小，單張曝光的進光量不足、雜訊高。補救方式包括:連拍多張對齊後合併以降低雜訊(第 5 章多影格合成、第 6、13 章的影像對齊)、依場景自動調整白平衡與色調(第 8 章)、以深度學習分離主體與背景製造淺景深(第 9 章分割、第 15 章)、超解析度重建細節(第 7、15 章)、HDR 合成擴大動態範圍(第 5 章)。共同邏輯是「以運算換取硬體規格」，與 1.6.1 節哈伯望遠鏡的精神完全一致。

22. 承 1.6.4 節：農業視覺有哪三類典型的技術挑戰?各自對應本書哪幾章的解法?

    提示與參考解答：其一，戶外光線變化劇烈——晴天、陰天、逆光與不同時段的色溫都不同，需要第 5 章的對比強化與第 8 章的白平衡做前處理，或改用對光照不敏感的特徵。其二，標註資料稀少——作物種類多、病徵樣本難蒐集，需要第 15 章的遷移學習與資料增強，以少量樣本達到可用精度。其三，田間沒有穩定網路與電力——無法把影像傳回雲端推論，需要第 17 章的邊緣運算與模型最佳化，讓模型在裝置上跑得動。這三項挑戰在其他戶外應用中也幾乎原封不動地重演。

23. 承圖 1-12 的學習地圖：為什麼第 13 章同時依賴第 9、10、12 章?請說明這三條依賴各自提供了什麼。

    提示與參考解答：第 9 章提供梯度與邊緣——角點偵測與 SIFT 都建立在影像梯度之上，DoG 近似 LoG 的關係也源自該章。第 10 章提供型態學與連通元件標記——分割後的遮罩需先清理才能計算形狀特徵，連通元件則提供「物件清單」作為特徵擷取的輸入。第 12 章提供追蹤的需求與光流——Shi-Tomasi 角點正是為了光流追蹤而設計，特徵匹配也用於視訊穩定。這說明第 13 章是本書前半部多條線索的匯流處，也是為什麼它被排在第三篇的開頭。

24. 承圖 1-15：創意學習螺旋包含哪五個環節?本課程的哪些設計分別對應其中一環?你認為哪一環最容易被忽略，為什麼?

    提示與參考解答：五個環節為想像、創造、遊玩、分享、反思。對應上：叫獸開講與 1.3 節的應用巡禮點燃「想像」，每章的 Lab 是「創造」，Lab 的開放題與參數調整是「遊玩」，課堂展示與同儕回饋是「分享」，章末重點整理與反思題是「反

> *思」。最容易被忽略的通常是「遊玩」與「反思」——學生往往程式一跑出結果就停手，既不試極限也不回頭問為什麼有效，於是學到的是「做出來了」而不是「我知道為什麼」。*

25. (綜合題)請構思一個期末專題的雛形：說明你要解決的具體問題、它在低中高三層各需要什麼處理、你會選 ESP32-CAM 還是 Jetson 並說明理由，以及你打算如何評估成果。

> *提示與參考解答：本題無標準答案，評分重點在這四件事是否互相自洽。問題要具體到可以驗收(「偵測停車場空位數量」優於「做一個停車場系統」)。三層分工要寫得出來:低階做哪些前處理、中階如何取出候選物件、高階如何判定。平台選擇要有依據——若只需偵測有無變化、可接受低影格率、且要佈設多點，選 ESP32-CAM;若需即時執行深度模型或接 ROS 2，選 Jetson。評估方式要可量化，例如在自備測試集上的精確率與召回率、每秒處理影格數，以及失敗案例的歸類。第 18 章會把這套流程完整展開。*

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 1(緒論與發展史)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 1.
- IEEE Computer Society, 停止接受含 Lena 影像投稿之公告，2024 年 3 月。(本章叫獸開講之史料來源)
- Losing Lena(紀錄短片),2019.(萊娜圖爭議之文化討論)
- USC-SIPI Image Database:https://sipi.usc.edu/database/(標準測試影像庫)
- Kodak Lossless True Color Image Suite、DIV2K、BSDS500 等公開測試資料集。
- Resnick, M., Lifelong Kindergarten, MIT Press, 2017.(4P 創意學習理念，第 18 章詳述)

- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 2 章 開發環境與程式基礎

本章學習目標

- 建立 Python 與 OpenCV 的開發環境，理解虛擬環境與套件管理的必要性。
- 掌握 NumPy 陣列運算，理解「影像就是陣列」這個貫穿全書的核心觀念。
- 熟練 OpenCV 的影像讀取、顯示、儲存與基本操作，並避開 BGR 等常見陷阱。
- 運用 ROI、切片與遮罩對影像的局部區域進行操作。
- 比較端上、近端邊緣、雲端三種 AI 辨識架構，並建立本書的實作平台環境。
- 養成除錯、查閱文件與版本管理的工程習慣。

叫獸開講|OpenCV 的誕生：一個「不要再重造輪子」的決定

1998 年，英特爾研究院的一位研究員 Gary Bradski 有了一個念頭。當時他觀察到一個奇怪的現象：電腦視覺這個領域，每一個實驗室、每一間公司，都在重複寫著同樣的東西——讀取影像、做高斯模糊、找邊緣、算直方圖。這些基礎功能人人都需要，卻沒有人共用;研究生入學的第一年，往往就耗在重寫前人早已寫過無數次的程式碼上。整個領域的能量，大量浪費在重造輪子。

Bradski 的構想是：打造一套共用的、經過最佳化的電腦視覺函式庫，讓研究者能站在同一個基礎上，把時間花在真正的創新上。他在英特爾內部組建了團隊，由 Vadim Pisarevsky 擔任技術主管，並得到英特爾俄羅斯團隊與效能函式庫團隊的大力支援。

但真正關鍵的轉折點在 1999 年。英特爾原本的計畫是做一套閉源的商業產品，名字叫 CVL(Computer Vision Library)。是 Bradski 力主開源——他認為只有開放，才能真正達成「不要再重造輪子」的目標，也才能讓整個領域一起前進。對一家以販售技術為業的晶片巨頭來說，把辛苦開發的程式碼免費送出去，是個相當大膽的決定，而英特爾接受了。專案也因此改名為 OpenCV，靈感來自當時已相當流行的 OpenGL。

2000 年 6 月，OpenCV 的第一個公開版本在電腦視覺頂級會議 CVPR 上發表，獲得與會者高度讚譽。它的目標在早期文件中寫得很清楚：提供不只是開放、而且是經過最佳化的基礎視覺程式碼，讓大家不必再重造輪子;透過共用的基礎設施傳播視覺知識;並以不強制開源的授權條款，讓商業應用也能自由採用。這最後一點尤其重要——正是這個寬鬆的授權，讓 OpenCV 得以進入無數的商業產品(對照第 13 章 SIFT 專利的故事，你會更能體會授權條款的份量)。

接下來的故事你已經身在其中。2005 年，史丹佛大學的無人車 Stanley 使用 OpenCV，贏得了美國國防高等研究計畫署的無人車大挑戰，在莫哈維沙漠中自主行駛超過兩百公里;2006 年 1.0 正式版問世;此後歷經 Willow Garage、Itseez 的接手維護，加入了 C++ 與 Python 介面、GPU 加速、深度學習模組。今天，OpenCV 收錄超過兩千五百個演算法，累積數千萬次下載，是全世界電腦視覺工作者共同的工具箱——包括你即將在本章安裝的這一套。

叫獸想從這個故事裡點出兩件事。第一，你在這門課裡寫的每一行 OpenCV 程式，背後都站著二十多年來成千上萬人的貢獻。這正是第 18 章 4P 中「同儕」(Peers)在全球尺度上的展現——開源社群本身，就是人類最大規模的協作學習實驗。第二，Bradski 當年那個「不要再重造輪子」的判斷，今天依然適用於你：寫程式時，先問問「這個功能是不是已經有人寫得比我好了?」把時間留給真正需要你創造的那一部分。

思考題：英特爾把辛苦開發的程式碼免費開源，短期看是「損失」，長期卻

讓公司在電腦視覺生態系中佔據了核心位置。你能想到其他「開放反而獲益」的例子嗎?這對你思考「知識該不該分享」有什麼啟發?

2.1 開發環境的建立

2.1.1 為什麼選 Python 與 OpenCV

本書選用 Python 搭配 OpenCV 作為主要工具，理由有四：其一，Python 語法簡潔，讓你能專注在影像處理的觀念而非語言細節;其二，OpenCV 是業界標準，學會它等於學會了一項可直接用於職場的技能;其三，Python 擁有最完整的科學運算與深度學習生態系(NumPy、Matplotlib、PyTorch、Ultralytics)，第 14 至 16 章會大量使用;其四，同樣的程式碼稍加調整就能在 Jetson 等邊緣裝置上執行(第 17 章)。

需要提醒的是：OpenCV 的核心是以 C++ 撰寫並高度最佳化，Python 只是它的介面。這代表你享受了 Python 的易用性，卻仍能獲得接近 C++ 的執行速度——前提是你使用 OpenCV 或 NumPy 的向量化運算，而非用 Python 的迴圈逐像素處理(2.3.3 節會用實測數據說明這個差異有多大)。

2.1.2 虛擬環境：為什麼不要直接安裝

初學者常直接用系統的 Python 安裝套件，這會在專案變多時造成災難：A 專案需要某套件的舊版本、B 專案需要新版本，兩者無法共存。虛擬環境(virtual environment)為每個專案建立獨立的套件空間，彼此互不干擾。這是專業開發的基本習慣，請從第一天就養成。

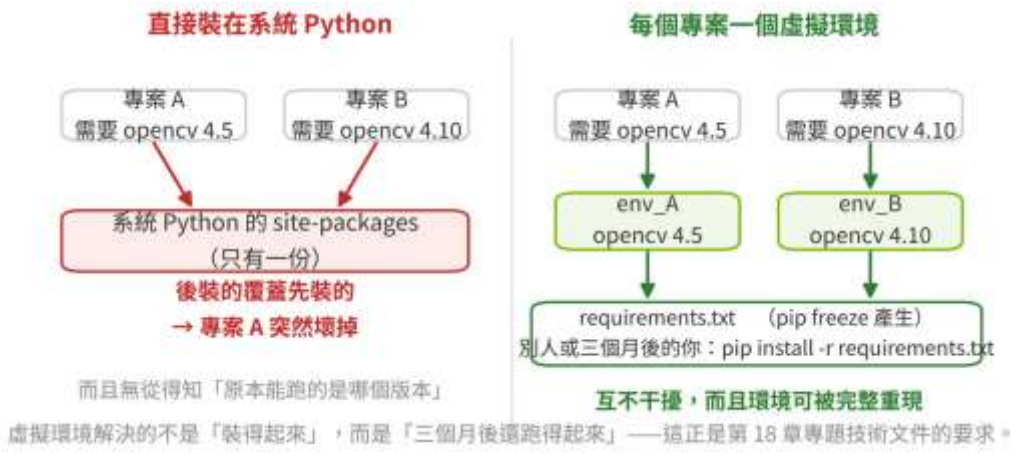


圖 2-1　虛擬環境解決的是什麼問題

程式 2-1 建立虛擬環境與安裝套件

```
# --- 建立虛擬環境(以 Python 內建的 venv 為例) ---
python -m venv cv_env

# 啟動(Windows)
cv_env\Scripts\activate
# 啟動(macOS / Linux)
source cv_env/bin/activate

# --- 安裝本課程所需套件 ---
pip install opencv-python        # OpenCV 主套件
pip install opencv-contrib-python  # 含額外模組(第 12 章追蹤器需要)
pip install numpy matplotlib     # 陣列運算與繪圖

# --- 記錄環境,方便他人重現(團隊協作與第 18 章專題必備) ---
pip freeze > requirements.txt
# 他人只需:pip install -r requirements.txt

# --- 離開虛擬環境 ---
deactivate
```

請注意 requirements.txt 這個檔案。它記錄了專案所需的所有套件與版本，讓別人(或三個月後的你)能完整重現你的環境。第 18 章期末專題的技術文件要求「另一個人照著做能重現你的成果」，這個檔案就是關鍵的一環。

2.1.3 驗證安裝與開發工具

程式 2-2 驗證環境安裝

```
import cv2
import numpy as np
import sys

print("Python 版本:", sys.version)
```

```
print("OpenCV 版本:", cv2.__version__)
print("NumPy 版本 :", np.__version__)

# 測試:建立一張純色影像並顯示
img = np.zeros((300, 400, 3), dtype=np.uint8)   # 全黑影像
img[:] = (0, 185, 118)                          # BGR:綠色
cv2.putText(img, "Hello OpenCV!", (60, 160),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
cv2.imshow("test", img)
cv2.waitKey(0)              # 按任意鍵關閉
cv2.destroyAllWindows()
```

開發工具方面，建議三選一：Jupyter Notebook 適合探索與教學(能逐格執行、即時看結果，本課程的 Lab 很適合);VS Code 適合開發完整專案(有除錯器、Git 整合);Google Colab 則適合沒有本機環境或需要免費 GPU 的場合(第 15、16 章訓練模型時特別有用)。三者可以並用，依任務選擇。



圖 2-2　Jupyter、VS Code、Colab 的適用場合

2.2 影像就是陣列：NumPy 基礎

這一節是全書最重要的觀念之一。如果你只記得本章的一句話，請記得這一句：對電腦而言，一張影像就是一個數字陣列。



圖 2-3　把影像放大到看見數字：眼睛看明暗，電腦看數值

2.2.1 影像的資料結構

OpenCV 讀進來的影像，型別是 NumPy 的 ndarray。灰階影像是二維陣列，形狀為(高度， 寬度);彩色影像是三維陣列，形狀為(高度， 寬度， 通道數)。每個元素的資料型別通常是 uint8(無號八位元整數)，數值範圍 0 到 255——這正是第 3 章要深入探討的量化與位元深度。

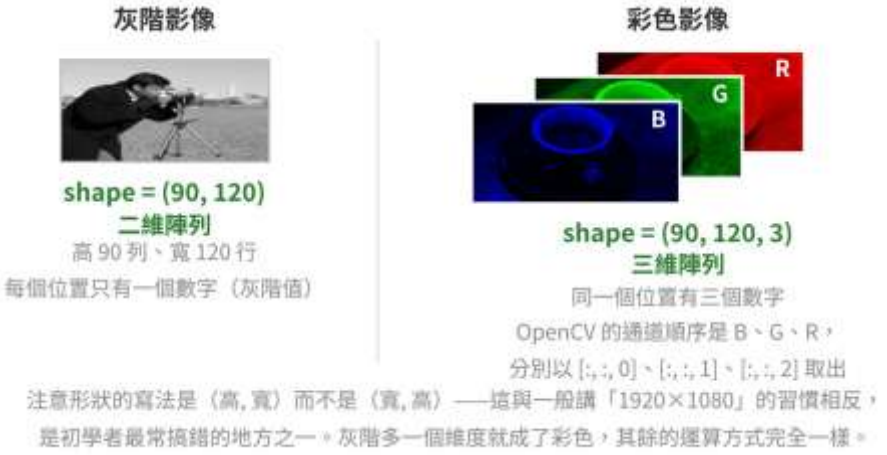


圖 2-4　灰階是二維陣列，彩色是三維陣列

請特別留意兩個容易搞混的順序。第一，陣列的索引順序是「先列後行」，也就是 img[y, x]，與數學上習慣的 (x, y) 相反;第二，OpenCV 的彩色通道順序是 BGR 而非

RGB(2.3.2 節會說明這個歷史包袱)。這兩點是初學者最常見的錯誤來源。



圖 2-5　初學者最常踩的兩個坑：索引順序與通道順序

程式 2-3 影像的陣列結構

```
import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
color = cv2.imread("photo.jpg")            # 預設讀成彩色(BGR)

print("灰階形狀:", gray.shape)             # (高, 寬)
print("彩色形狀:", color.shape)            # (高, 寬, 3)
print("資料型別:", color.dtype)            # uint8
print("總像素數:", color.size)             # 高 × 寬 × 3

# 存取單一像素(注意:先 y 後 x!)
y, x = 100, 250
print("該點灰階值:", gray[y, x])
b, g, r = color[y, x]                      # BGR 順序,不是 RGB
print(f"該點色彩:B={b}, G={g}, R={r}")

# 修改像素
color[y, x] = (0, 0, 255)                  # 設為紅色(BGR)
```

2.2.2 建立與初始化影像

程式 2-4 以 NumPy 建立測試影像

```
import numpy as np

black = np.zeros((300, 400, 3), dtype=np.uint8)        # 全黑
white = np.ones((300, 400, 3), dtype=np.uint8) * 255   # 全白
```

```
gray  = np.full((300, 400), 128, dtype=np.uint8)        # 中灰(單通道)
noise = np.random.randint(0, 256, (300, 400),
                          dtype=np.uint8)               # 隨機雜訊

# 漸層(用於測試第 5 章的灰階轉換)
ramp = np.tile(np.arange(256, dtype=np.uint8), (100, 1))

# 棋盤格(用於測試第 4 章相機校正與第 6 章頻譜)
board = np.indices((320, 320)).sum(axis=0) // 40 % 2
board = (board * 255).astype(np.uint8)
```

自己合成測試影像是一個非常實用的技巧：當演算法出錯時，用一張你完全知道內容的簡單影像(純色塊、漸層、棋盤格)去測試，遠比用複雜的自然照片容易找出問題。第 6 章 Lab 的「頻譜偵探」就大量使用這個技巧。



圖 2-6　六種可以用十行程式合成的測試影像

2.2.3 向量化運算：為什麼不要用迴圈

NumPy 的威力在於向量化——對整個陣列一次運算，而非逐元素迴圈。這不只是寫法簡潔的問題，而是效能的天壤之別：NumPy 的運算底層以 C 實作並最佳化，而 Python 的迴圈每次疊代都有直譯器的額外開銷。



圖 2-7　向量化與 Python 迴圈的實測效能差距

程式 2-5 向量化 vs. 迴圈的效能比較

```
import cv2, numpy as np, time

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
h, w = img.shape

# --- 壞方法:Python 迴圈逐像素(慢) ---
t0 = time.time()
out1 = img.copy()
for y in range(h):
    for x in range(w):
        out1[y, x] = 255 - img[y, x]        # 負片
print(f"迴圈:{time.time()-t0:.3f} 秒")

# --- 好方法:向量化(快數百倍) ---
t0 = time.time()
out2 = 255 - img                            # 一行搞定
print(f"向量化:{time.time()-t0:.5f} 秒")

print("結果相同:", np.array_equal(out1, out2))

# 常見向量化操作
brighter = np.clip(img.astype(np.int16) + 50, 0, 255).astype(np.uint8)
mask = img > 128                            # 布林陣列(第 9 章門檻分割的基礎)
print("亮部像素比例:", mask.mean())
```

請務必實際跑一次程式 2-5，親眼看看那個倍數差距。這個習慣在第 17 章邊緣部署時格外重要——在算力有限的 Jetson 或樹莓派上，一個寫成迴圈的前處理步驟，就足以讓整個即時系統掉影格。

另一個常見陷阱是溢位：uint8 的範圍是 0 到 255，若直接對它做加法，超過 255

會「繞回」變成小數字(例如 250 + 50 得到 44)，而非停在 255。正確做法是先轉成較大的型別、運算後再用 np.clip 限制範圍——程式 2-5 的 brighter 就是示範。第 5 章的亮度調整會反覆用到這個技巧。



圖 2-8　uint8 溢位：250 加 50 為什麼會變成 44

2.2.4 深入：廣播機制——NumPy 最常被低估的一招

到目前為止我們寫的 img * 1.2、255 - img，都用到了一個沒有明講的機制：廣播（Broadcasting）。它是 NumPy 在形狀不同的陣列之間自動對齊維度的規則，也是「影像就是陣列」這個觀念真正發揮威力的地方。

規則只有一條，而且只要三十秒就能記住：把兩個陣列的形狀從最後一個維度往前對齊比較，每一組維度必須相等，或其中之一為 1，否則無法廣播。為 1 的那一維會被「複製」到另一邊的長度——但這個複製是虛擬的，NumPy 並不會真的配置一塊大記憶體，而是用步幅為零的方式反覆讀取同一份資料，所以既省空間又快。

這條規則在影像處理上有三種典型用法，請務必對照形狀理解。其一是純量：img * 1.2，純量視為形狀 ()，可廣播到任何形狀，效果是整體調整亮度。其二是逐通道：對一張 (480, 640, 3) 的彩色影像乘上一個形狀為 (3,) 的陣列，例如 img *

np.array([0.9, 1.0, 1.2])，三個係數分別作用在 B、G、R 三個通道上——這正是白平衡與色溫調整的做法（第 8 章）。其三是逐列或逐行：一個形狀 (480, 1) 的陣列可以廣播到 (480, 640)，效果是「每一列乘上不同的係數」，這是製作漸層遮罩與暈影（vignette）效果的標準手法。

最常見的錯誤是維度對不齊。例如想對 (480, 640, 3) 的影像逐列加權，若直接乘上形狀 (480,) 的陣列會失敗，因為對齊是從最後一維開始：480 與 3 既不相等、也都不是 1。正解是先把它變成 (480, 1, 1)，用 w[:, None, None] 或 w.reshape(-1, 1, 1) 即可。一個實用的除錯習慣是：遇到廣播錯誤時，先把兩邊的 shape 印出來，右對齊寫在紙上比對，問題通常一眼就看得出來。

最後提醒一個與第 14 至 16 章直接相關的應用：深度學習的輸入正規化 (img - mean) / std，其中 mean 與 std 都是形狀 (3,) 的陣列，靠的就是逐通道廣播。許多初學者在這一步把形狀搞錯卻不自知——因為程式不會報錯，只是模型永遠訓練不起來。

2.3 OpenCV 基本操作

2.3.1 讀取、顯示與儲存

程式 2-6 影像的讀取、顯示與儲存

```
import cv2

# --- 讀取 ---
img  = cv2.imread("photo.jpg")                    # 彩色(BGR)
gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
raw  = cv2.imread("photo.png", cv2.IMREAD_UNCHANGED)  # 含 alpha 通道

# 務必檢查是否讀取成功(路徑錯誤時 imread 回傳 None 而不報錯!)
if img is None:
    raise FileNotFoundError("找不到影像,請檢查路徑")

# --- 顯示 ---
cv2.imshow("window title", img)
key = cv2.waitKey(0)          # 0 = 無限等待;數字 = 等待毫秒數
cv2.destroyAllWindows()

# --- 儲存(副檔名決定格式與壓縮方式,第 11 章詳述) ---
cv2.imwrite("output.png", img)                    # 無損
```

```
cv2.imwrite("output.jpg", img,
            [cv2.IMWRITE_JPEG_QUALITY, 95])       # 有損,可設品質
```

三個實務提醒：第一，imread 找不到檔案時不會拋出例外，而是安靜地回傳 None，若不檢查會在後續操作時出現難以理解的錯誤訊息——請養成檢查的習慣。第二，中文路徑在某些系統上會導致讀取失敗，建議檔名與資料夾都用英文。第三，waitKey 的回傳值是按鍵碼，可用於互動控制(第 8、12 章的 Lab 會用到)。

2.3.2 BGR 之謎：一個歷史包袱

OpenCV 使用 BGR 而非 RGB 的通道順序，是初學者最常踩的坑——用 Matplotlib 顯示 OpenCV 讀進來的影像時，紅色和藍色會對調。這個設計來自 OpenCV 早期(2000 年代初)的歷史背景：當時許多相機硬體與 Windows 的點陣圖格式採用 BGR 排列，為了效能而直接沿用。等到 RGB 成為主流，OpenCV 已有大量既有程式碼依賴 BGR，改動的代價太高，只好一路保留至今。



圖 2-9　BGR 與 RGB：同一個陣列，兩種讀法

這個小小的歷史包袱其實是一堂很好的工程課：早期的設計決定會長期影響一個系統，即使當初的理由早已消失。你在第 18 章的專題中做架構決定時，不妨想想這件事。

程式 2-7 BGR 與 RGB 的轉換

```
import cv2, matplotlib.pyplot as plt

img = cv2.imread("photo.jpg")        # BGR

# 用 Matplotlib 顯示時務必轉換,否則紅藍顛倒
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off"); plt.show()

# 通道的分離與合併
b, g, r = cv2.split(img)
merged  = cv2.merge([b, g, r])

# 用 NumPy 切片更快(向量化思維)
blue_channel = img[:, :, 0]
rgb = img[:, :, ::-1]                # 反轉通道順序即 BGR→RGB
```

2.3.3 ROI、切片與遮罩

ROI(Region of Interest，感興趣區域)是只對影像的某個區域進行操作的技巧，在 NumPy 中用切片即可完成。它在整本書中反覆出現：第 9 章車道線偵測只處理路面梯形區、第 10 章人臉偵測後只在臉部區域找眼睛、第 13 章裁切車牌區域做字元分割。



圖 2-10　ROI 就是切片：座標範圍與語法的對照

程式 2-8 ROI、切片與遮罩

```
import cv2, numpy as np

img = cv2.imread("photo.jpg")
```

```
# --- ROI:用切片取出區域(注意:先 y 後 x) ---
roi = img[100:300, 200:500]          # y 從 100 到 300,x 從 200 到 500
cv2.imwrite("roi.jpg", roi)

# 對 ROI 做處理後貼回(注意:切片是「檢視」不是複製!)
img[100:300, 200:500] = cv2.GaussianBlur(roi, (21, 21), 0)

# 若不想影響原圖,務必用 copy()
roi_safe = img[100:300, 200:500].copy()

# --- 遮罩:對不規則區域操作 ---
mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.circle(mask, (320, 240), 120, 255, -1)      # 白色實心圓
masked = cv2.bitwise_and(img, img, mask=mask)   # 只保留圓內

# 遮罩也可用布林索引(NumPy 風格)
img[mask == 255] = (0, 255, 0)                  # 圓內塗綠
```

程式中「切片是檢視不是複製」這一點請特別留意：NumPy 的切片回傳的是原陣列的一個視圖，修改它會直接改到原影像。這既是效能優勢(不必複製資料)，也是常見的 bug 來源。需要獨立副本時，務必呼叫 copy()。



圖 2-11　切片是「檢視」不是「複製」的實測對照

2.3.4 深入：記憶體佈局——為什麼切片不用複製

前一段說「切片是檢視不是複製」，但沒有解釋它為什麼做得到。答案藏在 NumPy 描述陣列的方式裡，而這個機制會在本書反覆出現，值得花兩頁弄懂。

一個 NumPy 陣列由三樣東西描述：一塊連續的記憶體、一個形狀（shape），

以及一組步幅（strides）——步幅記錄「在這一維上前進一格，記憶體位址要跳幾個位元組」。以一張 (480, 640, 3) 的 uint8 彩色影像為例，步幅是 (1920, 3, 1)：往下一列要跳 640 × 3 = 1920 個位元組，往右一行跳 3 個，換一個通道跳 1 個。這也印證了彩色影像在記憶體中是以 BGRBGRBGR… 交錯排列的，而不是三個通道各自成塊。

有了步幅，切片就只是「換一組起點、形狀與步幅」而已，完全不必搬動資料——這就是 img[100:300, 200:420] 幾乎零成本的原因。同理，img[:, :, ::-1] 做 BGR 轉 RGB 只是把最後一維的步幅改成 −1，比 cv2.cvtColor 更快;img.T 轉置也只是把前兩維的步幅互換。

代價是這些結果不再是「C 連續」的——也就是說，元素在記憶體中的排列順序不再與索引順序一致。純粹在 NumPy 之內運算時這沒有問題，但 OpenCV 的底層是 C++ 實作，多數函式要求輸入連續排列，拿到非連續的陣列時可能拋出錯誤，或由綁定層偷偷複製一份（於是你省下的時間又還回去了）。用 img.flags["C_CONTIGUOUS"] 可以檢查，用 np.ascontiguousarray() 可以明確複製一份連續的。

實務上的判準很簡單：只在 NumPy 之內運算時，盡量用切片與步幅操作，省時省記憶體;要把結果交給 OpenCV、寫入檔案、或送進深度學習框架時，先確保它是連續的。這個觀念在第 12 章處理視訊（每秒三十張影格都要搬一次就來不及了）與第 17 章的邊緣部署（記憶體頻寬是瓶頸）時格外重要。

2.4 三種 AI 辨識架構與實作平台

本節建立本書實作的硬體框架。這三種架構會在第 10、16、17 章反覆出現，是理解「演算法該跑在哪裡」的關鍵。

2.4.1 三種架構

1. 端上運算(On-device)：在感測裝置本身完成運算，例如 ESP32-CAM 直接執行輕量的人臉偵測。優點是延遲最低、不需網路、資料不出裝置(隱私最佳)、功耗低;限制是算力極為有限。
2. 近端邊緣運算(Edge)：在鄰近的較強裝置上運算，例如產線旁的 Jetson 或工業電腦處理多台相機的影像。它在算力與即時性之間取得平衡，資料不出廠區，是機器人與工業視覺的主力架構。
3. 雲端運算(Cloud)：把資料上傳到遠端資料中心處理。算力最強、適合訓練大模型與複雜分析，但延遲高、需要網路、且涉及資料外流的隱私考量。

關鍵觀念是三者可以分層協作。第 10 章的門禁案例就是典型：ESP32-CAM 在端上判斷「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把影像送到雲端做「是誰」的辨識(低頻、高算力)。第 17 章會用完整的比較表深入這個主題。



圖 2-13　端上、近端邊緣、雲端三種架構的取捨

2.4.2 本書的實作平台

本書的入門實作圍繞兩個平台展開，對應上述架構(第 17 章還會再加入樹莓派 5 配 AI Camera 這一層)：

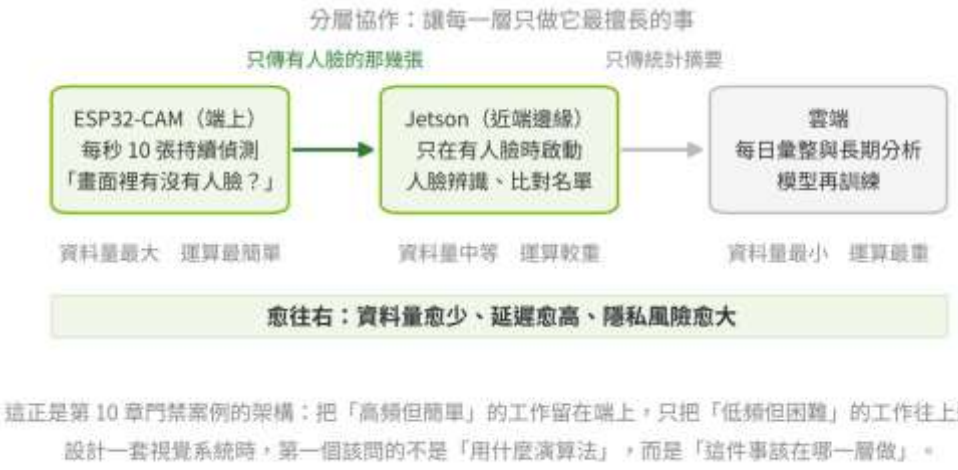


圖 2-14　分層協作：門禁案例的資料流

比較面向	ESP32-CAM(入門)	NVIDIA Jetson(進階)
定位	會看的感測節點	機器人的視覺大腦
對應架構	端上運算	近端邊緣運算
開發方式	Arduino IDE 或 ESP-IDF(C/C++)	Linux + Python + CUDA(同桌機)
可跑的任務	移動偵測、顏色追蹤、Haar 人臉偵測、影像串流	YOLO 即時偵測、分割、姿態、SLAM、ROS 2
本書出現章節	第 8、10、12 章 Lab	第 15-17 章與期末專題

表 2-1 入門平台與進階平台比較

學習策略上，建議的流程是：先在個人電腦上用 Python 與 OpenCV 把演算法邏輯驗證清楚，再移植到目標平台。這個「先驗證邏輯、再移植硬體」的工作方式，是嵌入式與機器人開發的標準做法——因為在電腦上除錯遠比在裝置上容易。

2.4.3 ESP32-CAM 快速上手

ESP32-CAM 最常見的用法是作為網路攝影機：燒錄官方的 CameraWebServer 範例後，它會提供一個串流網址，而 OpenCV 可以直接讀取這個網址，把它當成一台普通的相機。這讓你能在電腦上用 Python 開發，卻使用 ESP32-CAM 作為影像來源——本書多章的 Lab 都採用這個模式。

程式 2-9 以 OpenCV 讀取 ESP32-CAM 串流

```
import cv2

# ESP32-CAM 燒錄 CameraWebServer 範例後取得的串流網址
STREAM_URL = "http://192.168.1.50:81/stream"

cap = cv2.VideoCapture(STREAM_URL)      # 與讀取本機相機的用法完全相同
if not cap.isOpened():
    raise ConnectionError("無法連線,請確認 ESP32-CAM 與電腦在同一網路")

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # === 此處可套用本書任何一章的處理 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow("ESP32-CAM", frame)
    if cv2.waitKey(1) == 27:            # ESC 離開
        break

cap.release()
cv2.destroyAllWindows()
```

2.5 工程習慣的養成

這一節談的不是影像處理，而是讓你在往後十六章(以及日後職涯)少走彎路的習慣。它們看似瑣碎，卻是區分「會寫程式」與「會做專案」的關鍵。

2.5.1 除錯的基本功

影像處理的除錯有其特殊性：錯誤往往不會拋出例外，而是「結果看起來怪怪的」。三個實用技巧：其一，隨時印出陣列的 shape 與 dtype——大多數錯誤都源於形狀不符或型別溢位;其二，把中間結果存檔或顯示出來，不要一路算到底才看結果(所謂「視覺化除錯」);其三，用自己合成的簡單影像(2.2.2 節)測試，你知道正確答案應該長什麼樣。



圖 2-15　影像處理的除錯：每個階段都印出 shape 與值域

程式 2-10 除錯輔助函式

```
import cv2, numpy as np

def debug_info(name, arr):
    """印出陣列的關鍵資訊,除錯必備"""
    print(f"{name:12s} shape={arr.shape} dtype={arr.dtype} "
          f"min={arr.min()} max={arr.max()} mean={arr.mean():.1f}")

img = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
debug_info("原圖", img)

blur = cv2.GaussianBlur(img, (5, 5), 0)
debug_info("模糊後", blur)

edges = cv2.Canny(blur, 80, 160)
debug_info("邊緣", edges)

# 視覺化除錯:把各階段並排顯示
stacked = np.hstack([img, blur, edges])
cv2.imwrite("debug_stages.jpg", stacked)
```

2.5.2 查閱文件與求助

沒有人記得住 OpenCV 兩千多個函式的所有參數。專業工程師的能力不在於「記得多少」，而在於「多快能找到並理解」。建議的順序：先看官方文件(docs.opencv.org，有完整的參數說明與範例)、再看官方教學(tutorial 章節)、最後才是搜尋引擎與問答網站。使用 AI 助手時也請記得驗證——它可能給你舊版本的 API

或看似合理但錯誤的參數。

一個實用技巧：在 Python 中直接用 help(cv2.GaussianBlur) 或在 Jupyter 中用問號查詢函式簽名，比切換到瀏覽器更快。

2.5.3 版本管理與程式碼組織

從第一個 Lab 就開始用 Git。它的價值不只是備份，更在於「讓你敢於嘗試」——知道隨時能回復到能用的版本，你才會放手實驗(這正是第 18 章 Play 精神的技術基礎)。程式碼組織上，建議把常用的處理函式抽成獨立模組(例如把第 5 章的前處理管線寫成 preprocess.py)，往後各章直接引用——這既是良好習慣，也呼應叫獸開講「不要重造輪子」的精神。



圖 2-16　有版本控制，才敢放手做實驗

2.5.4 深入：從「會跑」到「可重現」

一支程式在你的電腦上跑得出結果，與這個結果能被別人（或三個月後的你）重現，是兩件完全不同的事。前者是作業的及格線，後者是第 18 章專題與日後職涯的及格線。兩者之間的差距，通常來自四個容易被忽略的細節。

第一是環境。虛擬環境加 requirements.txt 解決了套件版本（圖 2-1），但還有兩件事值得記錄：Python 本身的版本，以及作業系統——某些 OpenCV 函式在不同平台上的預設行為確實有細微差異。最省事的做法是在 README 的開頭寫下程式 2-2

的三行輸出。

第二是隨機性。只要你的流程裡出現 np.random、資料集的隨機切分、或第 15 章的權重初始化，每次執行的結果就會不同。解法是固定亂數種子（np.random.default_rng(42)），並且把種子值寫進紀錄而不是埋在程式中段。注意：固定種子是為了「可重現」，不是為了「挑一個好看的結果」——若你換了十個種子才得到想要的數字，該報告的是十次的分布，而不是最好的那一次。這是研究誠信的底線。

第三是路徑與資料。把 "C:/Users/你的名字/Desktop/photo.jpg" 寫死在程式裡，別人拿到就跑不動。做法是把所有路徑集中成檔案開頭的常數、使用相對路徑、並固定 raw / processed / output 的資料夾結構，其中 raw 永遠唯讀——覆寫原始資料是這門課唯一無法復原的錯誤。

第四是紀錄。每次執行時把參數、輸入檔名、輸出檔名與關鍵數值寫進一個簡單的文字檔或 CSV。這件事做起來只要五行程式，卻能在三週後你盯著一張效果很好的結果圖、卻完全想不起來當初參數調到多少時，救你一命。

把這四件事加上 Git，就構成了一份最小可行的「可重現」清單。它們看起來都與影像處理無關，但第 18 章評分時真正拉開差距的，往往正是這些地方。

2.6 應用案例

2.6.1 案例一：批次影像處理工具

情境：你有一資料夾的照片需要統一處理——調整尺寸、轉灰階、加浮水印、重新命名。這是影像處理最基本也最常見的實務需求。做法：用 Python 的 glob 或 pathlib 列出所有檔案，逐一讀取、處理、儲存到輸出資料夾。

這個看似簡單的案例，實際上是所有後續工作的基礎：第 14 至 16 章訓練模型前，你都需要先把原始影像批次處理成統一的尺寸與格式。及早寫好一個可重複使用的批

次處理腳本，會替你省下大量時間。



圖 2-17　批次處理：把尺寸各異的照片統一成同一規格

程式 2-11 批次影像處理

```
import cv2, glob, os

IN_DIR, OUT_DIR = "raw/", "processed/"
os.makedirs(OUT_DIR, exist_ok=True)

for i, path in enumerate(sorted(glob.glob(IN_DIR + "*.jpg"))):
    img = cv2.imread(path)
    if img is None:                          # 務必檢查
        print(f"跳過無法讀取的檔案:{path}")
        continue

    img = cv2.resize(img, (640, 480))     # 統一尺寸(第 4 章)
    cv2.putText(img, "YunTech", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 185, 118), 2)

    out_path = os.path.join(OUT_DIR, f"img_{i:04d}.jpg")
    cv2.imwrite(out_path, img)

print("批次處理完成")
```

2.6.2 案例二：即時攝影機處理管線

情境：打開筆電相機，即時顯示原始畫面與處理後的結果並排對照。這個「讀取一影格、處理、顯示」的迴圈，是第 12 章視訊處理、第 16 章 YOLO 即時偵測、第 17 章邊緣部署的共同骨架——本書後半的所有即時應用，結構都與它相同。

建議在這個案例中加入按鍵切換功能(例如按 1 顯示原圖、按 2 顯示灰階、按 3 顯示邊緣)，它會成為你整學期最常用的實驗工具：每學一個新演算法，就把它加進這個管線，即時看效果。

程式 2-12 即時處理管線（按鍵切換模式）

```
import cv2

cap = cv2.VideoCapture(0)          # 0 = 預設相機;換成網址即可讀 ESP32-CAM
if not cap.isOpened():
    raise ConnectionError("無法開啟相機")

mode = 1                           # 目前的處理模式
NAMES = {1: "original", 2: "gray", 3: "edges", 4: "blur"}

while True:
    ok, frame = cap.read()
    if not ok:
        break

    # === 依模式處理:每學一個新演算法就在這裡加一個分支 ===
    if mode == 1:
        out = frame
    elif mode == 2:
        out = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    elif mode == 3:
        g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        out = cv2.Canny(cv2.GaussianBlur(g, (5, 5), 0), 70, 150)
    else:
        out = cv2.GaussianBlur(frame, (21, 21), 0)

    # 灰階結果轉回三通道,才能疊彩色文字並與原圖並排
    if out.ndim == 2:
        out = cv2.cvtColor(out, cv2.COLOR_GRAY2BGR)

    cv2.putText(out, f"[{mode}] {NAMES[mode]}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 185, 118), 2)
    cv2.imshow("pipeline", cv2.hconcat([frame, out]))   # 原圖與結果並排

    key = cv2.waitKey(1) & 0xFF
    if key == 27:                  # ESC 離開
        break
    elif key in (ord("1"), ord("2"), ord("3"), ord("4")):
        mode = key - ord("0")
    elif key == ord("s"):          # 按 s 存下目前這一格
        cv2.imwrite(f"capture_{NAMES[mode]}.png", out)
        print("已存檔")

cap.release()
cv2.destroyAllWindows()
```

這支程式的骨架——讀一影格、依模式處理、顯示、等按鍵——會在第 12 章的視訊處理與第 16 章的 YOLO 即時偵測中原封不動地再出現。建議你每學完一章就回來加一個分支（按 5 顯示直方圖等化、按 6 顯示頻譜、按 7 顯示分割結果），到學期末它就是一台屬於你的即時影像處理實驗機。

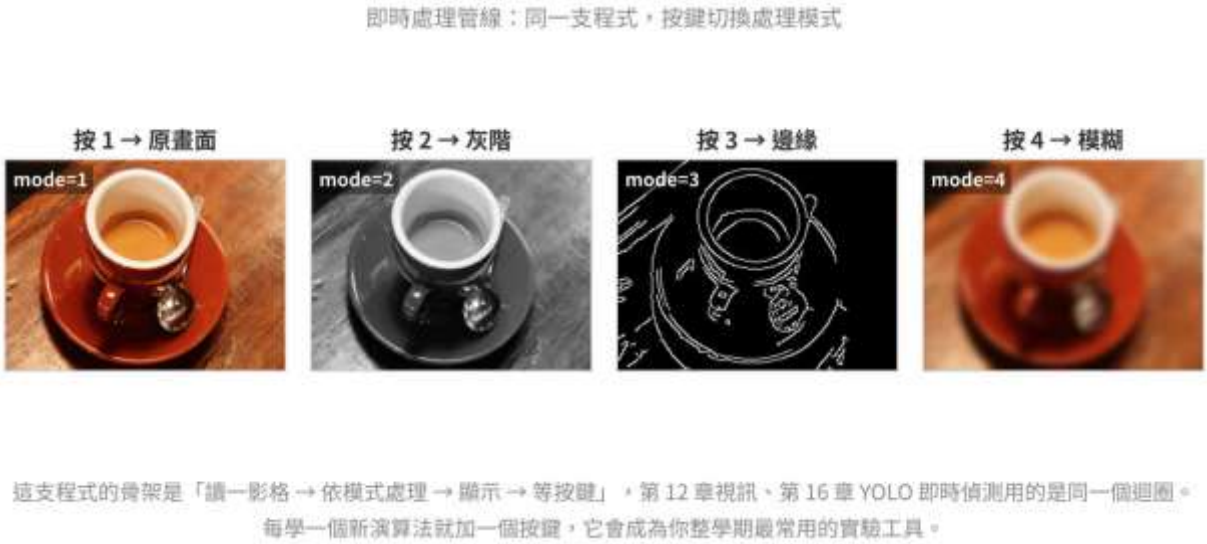


圖 2-18　即時處理管線：按鍵切換四種處理模式

2.6.3 案例三：ESP32-CAM 遠端監看

情境：把 ESP32-CAM 放在實驗室或家中，從電腦即時監看，並在偵測到變化時自動存檔。這個案例整合了本章的 2.4.3 節(串流讀取)與後續第 12 章的移動偵測。它的價值在於讓你及早體驗「感測器在遠端、運算在本機」的分散式架構——這正是 2.4.1 節三種架構中「端上取像、近端運算」的實例，也是第 17 章分散式監測網案例的雛形。

2.6.4 案例四：建立你的測試影像庫

情境：承第 1 章 Lab 1-3 的討論，建立一組屬於你自己的標準測試影像，供整學期使用。做法：挑選或拍攝六到十張涵蓋不同特性的影像——平滑漸層、銳利邊緣、細緻紋理、高對比、低光雜訊、含文字、含人臉(需取得同意)、規律圖案。統一尺寸與命名，建立資料夾與說明文件。



圖 2-19　一組為了考驗不同技術而挑選的測試影像

為什麼這值得花時間?因為當你在第 5 章調對比、第 6 章看頻譜、第 7 章去雜訊、第 9 章找邊緣時，都使用同一組影像，你就能跨章比較不同演算法在同一素材上的表現，建立起連貫的直覺。這也呼應第 1 章叫獸開講的反思——選擇測試素材本身，就是一個值得思考的決定。

2.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
影像即陣列	全書所有章節	所有演算法都是對陣列的運算
uint8 與溢位	第 3、5 章	位元深度、量化;亮度調整需 clip
向量化運算	第 5、6、17 章	濾波效能;邊緣裝置的即時性
BGR 通道順序	第 8 章	色彩空間轉換與 HSV 範圍陷阱
ROI 與遮罩	第 9、10、13 章	車道 ROI、人臉內找眼睛、車牌裁切
合成測試影像	第 4、6、7 章	棋盤格校正、頻譜驗證、雜訊模擬

VideoCapture 迴圈	第 12、16、17 章	視訊處理、即時偵測、邊緣部署的骨架
三種辨識架構	第 10、16、17 章	端上預篩加雲端精算的混合設計
實作平台	第 8、10、12、15-17 章	ESP32-CAM 感測、Jetson 推論與 ROS 2
工程習慣	第 18 章	版本管理、工程筆記、可重現的文件

表 2-2 第 2 章與全書其他章節的觀念連結

程式實作 Lab 2

Lab 2-1 環境建立與驗證

目標：建立可用一整個學期的開發環境。(a)依程式 2-1 建立虛擬環境並安裝套件;(b)執行程式 2-2 驗證安裝，記下三個套件的版本號;(c)產出 requirements.txt 並確認內容;(d)三種開發工具(Jupyter、VS Code、Colab)各試用一次，寫下你覺得各自適合什麼場合;(e)建立本課程的專案資料夾結構(建議：images/ 放素材、labs/ 放各章程式、outputs/ 放結果、utils/ 放共用函式)，並用 git init 開始版本管理。

Lab 2-2 陣列即影像

目標：徹底內化「影像就是陣列」。(a)讀取一張照片，印出 shape、dtype、size，並解釋每個數字的意義;(b)不使用任何影像檔，純以 NumPy 合成：純色塊、水平漸層、垂直漸層、棋盤格、隨機雜訊、以及一個中心亮四周暗的圓形漸層;(c)只用陣列運算(不用 OpenCV 函式)實作：負片、亮度加減、左右翻轉、上下翻轉、裁切中央區域;(d)執行程式 2-5，實測迴圈與向量化的效能差距並記錄倍數;(e)刻意製造一次 uint8 溢位(如對亮部加 100 而不 clip)，觀察結果並解釋為什麼會出現詭異的暗塊。

Lab 2-3 ROI 與遮罩實作

目標：掌握局部操作。(a)用切片取出照片中的一塊區域並存檔;(b)對該區域做模

糊後貼回原圖(製造「馬賽克隱私保護」的效果);(c)驗證「切片是檢視不是複製」：先不加 copy() 修改 ROI，觀察原圖是否被改變，再加上 copy() 重試;(d)建立圓形與多邊形遮罩，只保留遮罩內的影像;(e)進階：用滑鼠事件(cv2.setMouseCallback)讓使用者框選 ROI——這個工具在第 4 章文件掃描與第 12 章物件追蹤都會用到。

Lab 2-4 即時處理管線與 ESP32-CAM

目標：建立整學期最常用的實驗工具。(a)寫一支程式，讀取筆電相機並即時顯示;(b)加入按鍵切換模式(1 原圖、2 灰階、3 邊緣、4 模糊)，並在畫面上顯示目前模式與 FPS;(c)加入按鍵存檔功能(按 s 儲存目前影格);(d)若有 ESP32-CAM，燒錄 CameraWebServer 範例，依程式 2-9 改用串流網址作為輸入，比較兩者的畫質與延遲;(e)把這支程式存成可重複使用的工具(如 live_demo.py)，往後每學一個新演算法就把它加進去——這將是你這門課最有價值的個人資產。

Lab 2-5 效能實測與瓶頸定位

目標：把圖 2-7 的實驗做成你自己的數據，並學會找出瓶頸在哪裡。(a)寫一支程式，對同一張影像分別以 Python 迴圈與 NumPy 向量化做負片，用 time.perf_counter 量測並印出倍數;(b)把影像尺寸由 128×128 逐步放大到 1024×1024，畫出「尺寸對耗時」與「尺寸對倍數」兩條曲線，並解釋為什麼倍數在某個尺寸之後反而下降;(c)量測三種寫法的差異：out = 255 - img（產生新陣列）、np.subtract(255, img, out=out)（就地寫入）、以及 cv2.bitwise_not(img)，說明哪一種最快、為什麼;(d)用 cProfile 或簡單的計時裝飾器，對一條「讀檔→轉灰階→模糊→Canny→存檔」的管線逐步計時，找出最耗時的那一步——結果可能與你的直覺不同;(e)寫下結論：在你的機器上，什麼樣的操作值得優化、什麼不值得?(f)把這支測速程式存進你的工具箱，第 17 章在 Jetson 上部署時會再用到它。

Lab 2-6 打造你的影像處理工具箱

目標：把本章學到的東西整理成一個可重用的模組，供接下來十六章使用。這是

本書最有長期價值的一個 Lab。(a)建立 imtools.py，至少包含四個函式：imread_safe(path)（讀不到就明確拋錯，而不是回傳 None）、debug_info(name, arr)（印出 shape、dtype、值域與平均）、debug_show(name, img)（自動處理灰階 / 彩色與各種值域，見習題 23）、make_test_images()（產生圖 2-6 的六張合成影像）;(b)加上一個 side_by_side(*imgs, titles=None) 函式，把多張影像並排成一張圖，尺寸不同時自動補邊;(c)為每個函式寫一段 docstring 與一個最小測試，確認它在灰階、彩色、float、二值四種輸入下都不會壞;(d)建立 Git 倉庫，寫好 .gitignore 與 README，並以有意義的訊息提交（例如「加入 debug_show，支援 float 輸入」）;(e)寫一支 demo.py，只用 import imtools 就能跑出一條完整的「讀檔→處理→並排顯示」流程;(f)把這個倉庫留著。接下來每一章你都會往裡面加東西——到了第 18 章做專題時，它就是你的個人函式庫。

本章重點整理

- OpenCV 源於 Gary Bradski 1998 年「不要再重造輪子」的構想，1999 年英特爾決定開源(原計畫為閉源的 CVL),2000 年 6 月於 CVPR 首次公開發表;其寬鬆授權讓商業應用得以自由採用，是它普及的關鍵。
- 虛擬環境為每個專案建立獨立套件空間，requirements.txt 讓環境可被他人重現;開發工具 Jupyter(探索)、VS Code(專案)、Colab(免費 GPU)可依任務並用。
- 影像就是 NumPy 陣列：灰階為二維、彩色為三維(高， 寬， 通道);索引順序為 img[y, x](先列後行)，資料型別多為 uint8(0 至 255)，超出範圍會溢位需以 clip 處理。
- 向量化運算比 Python 迴圈快數百倍，因 NumPy 與 OpenCV 底層以最佳化的 C/C++ 實作;這個習慣在第 17 章邊緣部署的即時系統中尤其關鍵。

- OpenCV 使用 BGR 而非 RGB(早期硬體與 Windows 點陣圖格式的歷史包袱)，與 Matplotlib 互通時須轉換;imread 失敗時安靜回傳 None，務必檢查。
- ROI 以切片實作，但切片是「檢視」非複製，修改會影響原圖，需獨立副本時應用 copy();不規則區域則以遮罩配合 bitwise 運算或布林索引處理。



圖 2-12　遮罩的四個步驟：建立、套用、反轉、布林索引

- 三種 AI 辨識架構：端上(ESP32-CAM，低延遲、保護隱私、算力有限)、近端邊緣(Jetson，算力與即時性平衡)、雲端(算力最強但延遲高);三者可分層協作。學習策略為先在電腦驗證邏輯、再移植目標平台。

習題

1. 承叫獸開講：Gary Bradski 為什麼要創立 OpenCV?英特爾在 1999 年做了什麼關鍵決定，這個決定為何重要?

   提示與參考解答：因為他觀察到電腦視覺領域每個實驗室都在重複實作同樣的基礎功能(讀影像、模糊、找邊緣)，整個領域的能量浪費在「重造輪子」上。1999 年的關鍵決定是：原本計畫做閉源商業產品 CVL，經 Bradski 力主改為開源，並更名 OpenCV(靈感來自 OpenGL)。重要性在於只有開放才能真正達成共用基礎、讓領域一起前進;而不強制開源的寬鬆授權，更讓它得以進入無數商業產品。

2. 為什麼要使用虛擬環境?requirements.txt 的用途是什麼?它與第 18 章的哪個要求相關?

提示與參考解答：虛擬環境為每個專案建立獨立的套件空間，避免不同專案需要同一套件的不同版本而衝突。requirements.txt 記錄專案所需的所有套件與版本，讓他人(或日後的自己)能以一行指令完整重現環境。它與第 18 章期末專題技術文件的要求「另一個人照著做能重現你的成果」直接相關。

3. 請說明灰階影像與彩色影像的陣列形狀差異。存取像素時為什麼是 img[y, x] 而非 img[x, y]?

   提示與參考解答：灰階為二維陣列(高度， 寬度);彩色為三維陣列(高度， 寬度， 通道數)。索引為 img[y, x] 是因為 NumPy 陣列的索引順序是「先列(row)後行(column)」，而列對應影像的垂直方向(y)、行對應水平方向(x)，與數學習慣的 (x, y) 相反——這是初學者最常見的錯誤來源之一。

4. 為什麼要避免用 Python 迴圈逐像素處理影像?請估算一張 1920×1080 彩色影像用迴圈與向量化處理的差距量級，並說明原因。

   提示與參考解答：因為向量化比迴圈快數百倍。一張 1920×1080×3 約六百二十萬個元素，Python 迴圈每次疊代都有直譯器的額外開銷;而 NumPy 與 OpenCV 的運算底層以最佳化的 C/C++ 實作、且能利用 SIMD 指令與快取。實測差距常達數百倍。這個習慣在第 17 章邊緣裝置的即時系統上格外關鍵——一個寫成迴圈的前處理就足以讓系統掉影格。

5. 什麼是 uint8 溢位?對一個值為 250 的像素加上 50 會得到什麼?正確做法是什麼?

   提示與參考解答：uint8 範圍是 0 至 255，超出會「繞回」(模 256)。250 + 50 = 300，溢位後得到 44——一個原本很亮的像素反而變暗，產生詭異的暗塊。正確做法：先轉成較大的型別(如 int16 或 float)運算，再用 np.clip(結果， 0, 255) 限制範圍，最後轉回 uint8。第 5 章的亮度調整會反覆用到。

6. OpenCV 為什麼使用 BGR 而非 RGB?這個歷史包袱會在什麼情況下造成問題?這給了我們什麼工程啟示?

   提示與參考解答：因為 OpenCV 早期(2000 年代初)許多相機硬體與 Windows 點陣圖格式採用 BGR 排列，為效能而沿用;待 RGB 成為主流時，已有大量既有程式碼依賴 BGR，改動代價太高。問題常出現在與 Matplotlib、PIL 等使用 RGB 的函式庫互通時，紅藍會顛倒。工程啟示：早期的設計決定會長期影響一個系統，即使當初的

理由早已消失——做架構決定時需考慮長遠影響。

7. imread 讀取失敗時會發生什麼?為什麼這個行為特別容易造成困擾?應如何防範?

   提示與參考解答：它不會拋出例外，而是安靜地回傳 None。困擾在於程式不會在出錯的地方停下，而是在後續操作(如 img.shape)才報出難以理解的錯誤訊息，不易追查根源。防範：讀取後立即檢查 if img is None 並拋出明確的錯誤。另外中文路徑在某些系統會失敗，建議檔名與資料夾都用英文。

8. 什麼是 ROI?為什麼說「切片是檢視不是複製」?這個特性何時是優點、何時是陷阱?

   提示與參考解答：ROI 是感興趣區域，以 NumPy 切片取出影像的局部(如 img[100:300, 200:500])。切片回傳的是原陣列的視圖而非副本，修改它會直接改到原影像。優點：不必複製資料，效能好，且能方便地「處理後貼回」。陷阱：當你只想取出區域來分析、卻不小心改到原圖時，會產生難以察覺的 bug。需要獨立副本時務必呼叫 copy()。

9. 請說明端上、近端邊緣、雲端三種 AI 辨識架構的差異，並舉一個「分層協作」的實例。

   提示與參考解答：端上：在感測裝置本身運算(如 ESP32-CAM)，延遲最低、不需網路、隱私最佳，但算力極有限。近端邊緣：在鄰近的較強裝置運算(如 Jetson)，算力與即時性平衡、資料不出廠區。雲端：遠端資料中心，算力最強但延遲高、需網路、有隱私考量。分層協作實例(第 10 章門禁):ESP32-CAM 端上判斷「有沒有人臉」(高頻低算力)，偵測到才上傳雲端做「是誰」的辨識(低頻高算力)。

10. 本課程為什麼建議「先在電腦上驗證邏輯、再移植到目標平台」?

    提示與參考解答：因為在個人電腦上開發與除錯遠比在嵌入式裝置上容易：有完整的除錯器、可即時顯示中間結果、修改後立即執行不需重新燒錄、且不受裝置算力與記憶體限制。先確認演算法邏輯正確，移植時就只需處理硬體相關的問題(效能、記憶體、介面)，能大幅縮短開發時間。這是嵌入式與機器人開發的標準工作方式。

11. 影像處理的除錯有什麼特殊性?請說明三個實用的除錯技巧。

    提示與參考解答：特殊性：錯誤往往不拋出例外，而是「結果看起來怪怪的」，不

易定位。三個技巧：一、隨時印出陣列的 shape 與 dtype(大多數錯誤源於形狀不符或型別溢位)，可寫成 debug_info 輔助函式;二、視覺化除錯——把中間結果存檔或並排顯示，不要一路算到底才看;三、用自己合成的簡單影像(純色塊、漸層、棋盤格)測試，因為你知道正確答案該長什麼樣。

12. (綜合題)承叫獸開講「不要再重造輪子」：請說明這個原則在你的學習與專題中該如何實踐，以及它有沒有需要注意的界線。

    提示與參考解答：實踐：寫程式前先查是否已有成熟的函式庫或實作(OpenCV 兩千多個演算法、PyPI 生態系)，把時間留給真正需要創造的部分;把自己常用的處理抽成共用模組供各章重複使用;善用開源社群的成果並回饋。界線：一、學習階段有時值得「重造一次輪子」以理解原理(如本書多章要求手動實作演算法再與函式庫對照);二、需確認授權條款是否允許你的用途(對照第 13 章 SIFT 專利的故事);三、直接使用而不理解，遇到問題時無從診斷——理解與效率需要平衡。

13. 計算題：一張 1920×1080 的彩色影像以 uint8 儲存，未壓縮時佔多少記憶體?若轉成 float32 做運算，又會佔多少?若要同時保留原圖、灰階版與三個中間結果（皆為 float32），總共需要多少?

    提示與參考解答：uint8 彩色：1920 × 1080 × 3 × 1 位元組 ≈ 6.22 MB。轉 float32 後每個元素 4 位元組，變成約 24.9 MB，整整四倍。灰階 float32 為 1920 × 1080 × 4 ≈ 8.3 MB;三個彩色 float32 中間結果為 3 × 24.9 ≈ 74.6 MB;加上原圖 6.2 MB，合計約 113 MB。這個數字值得記住：在第 17 章只有 4 GB 記憶體的 Jetson 上同時跑多路相機時，「不必要的中間結果」是最常見的記憶體殺手。務實的做法是盡量就地運算（out 參數）、用完的中間結果及早釋放、以及只在真正需要小數精度的步驟才轉 float。

14. 承圖 2-7：為什麼 NumPy 的向量化會比 Python 迴圈快上千倍?請從「直譯器開銷」與「記憶體存取」兩個角度說明。另外，為什麼在最大的那張影像上，倍數反而比中等尺寸時低?

    提示與參考解答：兩個原因。其一是直譯器開銷：Python 的每次迴圈疊代都要做型別檢查、物件裝箱拆箱與邊界檢查，這些成本遠大於「255 減一個數字」本身;NumPy 把整個迴圈下放到預先編譯好的 C 程式碼，一次付出一份開銷。其二是記憶體存取：NumPy 陣列在記憶體中是連續的，CPU 可以一次載入一整條快取線

> *並以 SIMD 指令同時處理多個元素，而 Python 的迴圈逐一存取、無法向量化。至於大影像倍數下降，是因為向量化版本此時已經快到「瓶頸不在運算而在把資料搬進搬出記憶體」——也就是受限於記憶體頻寬。這個現象在第 17 章的邊緣裝置上更明顯，因為嵌入式平台的記憶體頻寬通常遠低於桌機。*

15. 承圖 2-8：請說明為什麼 img + 50 會讓亮部變黑，而 cv2.add(img, 50) 不會。若要把一張影像的亮度降低 50，有沒有對稱的陷阱?該怎麼寫才安全?

> *提示與參考解答：NumPy 對 uint8 採「繞回」（模 256）運算，250 + 50 = 300 會變成 44，於是最亮的區域變成接近黑色。cv2.add 採「飽和」運算，超過 255 就停在 255，所以不會出現黑點。減法有完全對稱的陷阱：30 − 50 = −20，繞回後變成 236，最暗的區域反而變成接近白色——這在第 7 章做影像相減找差異時特別容易中招。安全的寫法有三種：用 cv2.add / cv2.subtract;或先轉成有號的較寬型別再 np.clip 回來，如 np.clip(img.astype(np.int16) - 50, 0, 255).astype(np.uint8);或整條管線都以 float32 運算，最後一步才轉回 uint8。本書後續採第三種做法的場合最多，因為它同時避免了中間步驟的精度損失。*

16. 承圖 2-11：請解釋「切片是檢視不是複製」。下列三行程式分別會不會動到原圖?(a) roi = img[0:100, 0:100]; roi += 10　(b) roi = img[0:100, 0:100].copy(); roi += 10　(c) roi = img[0:100, 0:100]; roi = roi + 10

> *提示與參考解答：(a) 會。roi 是原陣列的視圖，+= 是就地運算，直接寫回原本的記憶體。(b) 不會。copy() 配置了新的記憶體，兩者從此無關。(c) 不會，但理由與 (b) 不同：roi + 10 產生一個全新的陣列，再把名字 roi 綁到新陣列上，原本的視圖只是被改了名字，原圖沒被碰到。這題的關鍵在於分辨「就地運算」（+=、[:] =、cv2 的 dst 參數）與「產生新物件的運算」（+、np.add 不帶 out）。判準：如果你希望原圖保持不變，就加 .copy();如果你是刻意要改原圖的某一塊（例如打馬賽克），視圖反而讓你省去「貼回去」這一步。*

17. 承 2.4 節：請為下列三個情境各選一種架構（端上 / 近端邊緣 / 雲端），並說明理由：(a) 高鐵月台的跌倒偵測;(b) 病理切片的腫瘤分級;(c) 自家門口的訪客通知。每一題請至少指出一個「選錯架構會出什麼事」。

> *提示與參考解答：本題無唯一答案，評分重點在理由的完整性。一組合理的選擇：(a) 近端邊緣。跌倒必須在數百毫秒內示警，雲端的往返延遲與斷線風險不可接受;且*

月台影像涉及大量乘客隱私，不宜離開站體。選雲端的後果是：網路一塞，警報就慢了，而這正是最需要它的時候。(b) 雲端。切片影像可達數 GB、模型極大，且屬非即時分析，算力才是瓶頸;選端上的後果是根本跑不動，但要注意病歷屬特種個資，上傳前的去識別化與法遵是前提。(c) 端上為主。隱私最敏感（自家影像）、事件稀疏、算力需求低，ESP32-CAM 在端上判斷「有沒有人」即可，只把有人的那幾張送出（圖 2-14 的分層協作）;選雲端的後果是把自家門口的連續影像交給第三方，而且斷網時完全失效。

18. 承 2.5.1 節：以下三個除錯情境，請指出最可能的原因與檢查方法：(a) 用 cv2.imshow 顯示一張剛算好的影像，整個視窗全白;(b) 程式在 img.shape 這一行報 AttributeError;(c) 兩張照片相減後，結果幾乎全黑但偶有極亮的雜點。

    提示與參考解答：(a) 最可能是 dtype 與值域不匹配：若陣列是 float 且值域在 0~1，cv2.imshow 會把 1.0 以上全當成白色;或值域是 0~255 但型別已成 float64。檢查：印出 dtype、min、max，必要時用 cv2.normalize 或 .astype(np.uint8) 修正。(b) img 是 None——imread 找不到檔案時不拋例外而是安靜回傳 None，於是錯誤延後到使用它的地方才爆。檢查：在 imread 後立刻加 if img is None: raise。(c) 幾乎全黑代表兩張確實很像（正常），極亮雜點則是 uint8 減法的負值繞回（第 15 題的對稱陷阱）。檢查：改用 cv2.absdiff，或先轉 int16 再取絕對值。這三題涵蓋了影像處理除錯的三大類：值域、None、型別溢位。

19. 比較 cv2.imshow 與 matplotlib 的 plt.imshow，列出至少三項差異，並說明各自適合在什麼場合使用。

    提示與參考解答：至少三項：其一，通道順序：cv2.imshow 預期 BGR，plt.imshow 預期 RGB，互通時必須 cvtColor（圖 2-9）。其二，灰階影像：plt.imshow 預設套用彩色對應表（viridis），要顯示真正的灰階必須指定 cmap="gray"，而且還要留意 vmin/vmax——未指定時 matplotlib 會自動拉伸對比，讓你誤以為影像比實際更有層次。其三，視窗行為：cv2.imshow 需要搭配 waitKey 才會更新畫面，適合即時影像的連續播放;plt.imshow 產生靜態圖，適合並排比較與輸出到報告。場合：即時攝影機、按鍵互動用 cv2;報告、論文、Jupyter 中的多圖比較用 matplotlib。另外在 Colab 上 cv2.imshow 無法使用，須改用 google.colab.patches 的 cv2_imshow，或直接改用 matplotlib。

20. 什麼是 NumPy 的廣播（Broadcasting）?請說明 img * 1.2、img - img.mean(axis=(0,1))、以及 img * np.array([0.5, 1.0, 1.5]) 這三個運算各自發生了什麼事，並指出第三個運算在影像處理上的實際用途。

    *提示與參考解答：廣播是 NumPy 在形狀不同的陣列間自動對齊維度的規則：從最後一個維度往前比對，兩者相等或其中之一為 1 才能廣播。img * 1.2：純量被廣播到每一個元素，整體變亮兩成。img - img.mean(axis=(0,1))：mean 的結果形狀是 (3,)，對 (H, W, 3) 廣播後等於「每個通道各自減去自己的平均」，這是第 14 至 16 章做資料正規化的標準動作。img * np.array([0.5, 1.0, 1.5])：三個係數分別乘到 B、G、R 三個通道，用途是白平衡與色溫調整（第 8 章）——把藍色壓低、紅色拉高，影像就會偏暖。注意三者都會讓結果變成 float，最後務必 np.clip 並轉回 uint8。*

21. 承 2.3.4 節：什麼是記憶體的連續性（contiguous）?為什麼 img[:, :, ::-1] 做 BGR→RGB 很快，但某些 OpenCV 函式拿到它卻會報錯或變慢?該怎麼處理?

    提示與參考解答：NumPy 以「起點 + 步幅（stride）」描述陣列如何對應到一塊連續的記憶體。img[:, :, ::-1] 只是把最後一維的步幅改成負的，完全沒有搬動任何資料，所以幾乎是零成本——這正是它比 cv2.cvtColor 快的原因。但這樣得到的陣列不是 C 連續的，而 OpenCV 的底層是 C++ 實作、多數函式要求輸入在記憶體中連續排列，因此可能拋出錯誤，或由綁定層偷偷複製一份（於是你省下的時間又還回去了）。處理方式是用 np.ascontiguousarray(img[:, :, ::-1]) 明確複製一份，或乾脆用 cv2.cvtColor。判準是：只在 NumPy 之內運算時用切片反轉最划算;要把結果交給 OpenCV 或寫入檔案時，先確保它是連續的。可用 img.flags["C_CONTIGUOUS"] 檢查。

22. 承 2.5.3 節與圖 2-16：請列出三件「絕對不該提交到 Git」的東西，並說明理由。另外說明 .gitignore 的用途，以及它與 requirements.txt 在「讓別人重現你的專案」這件事上如何分工。

    提示與參考解答：三件不該提交的東西：其一，大型資料集與影像素材——Git 為文字設計，二進位大檔會讓倉庫迅速膨脹且無法有效比對差異，應改用資料集下載腳本或 Git LFS。其二，訓練好的模型權重檔，理由同上（第 15、16 章特別容易犯）。其三，金鑰、密碼與 API token——一旦提交就永遠留在歷史紀錄裡，即使後來刪除也救不回來，應放在環境變數或不被追蹤的設定檔中。此外，虛擬環境資

料夾（cv_env/）、__pycache__、以及程式的輸出結果也都不該提交。.gitignore 用來宣告「這些路徑不要追蹤」，讓 git status 保持乾淨。分工是：.gitignore 負責「不要帶什麼」，requirements.txt 負責「要補什麼」——兩者合起來，別人 clone 之後才能一行指令重建出可執行的環境。

23. 設計題：請設計一個 debug_show(name, img) 函式，要能自動處理下列情況並正確顯示：(a) 灰階與彩色;(b) uint8 與值域 0~1 的 float;(c) 值域超出範圍的中間結果;(d) 二值遮罩。請說明你的判斷邏輯。

    提示與參考解答：核心是「先判斷，再正規化，最後顯示」。一個合理的邏輯：先印出 shape、dtype、min、max（延續程式 2-10）;接著依 dtype 分流——若是浮點數且 max ≤ 1.0，乘以 255 後轉 uint8;若是浮點數但值域超出 0~255，用 cv2.normalize 線性拉伸到 0~255 並在標題註明「已正規化」（這點很重要，否則你會誤以為原始值域本來就正常）;若是 uint8 則直接顯示。維度方面：二維視為灰階（cmap="gray", vmin=0, vmax=255）;三維且第三維為 3 則視為 BGR，顯示前轉 RGB;若只有 0 與 255 兩種值則標註為二值遮罩。這個函式值得你真的寫出來放進工具箱（Lab 2-6），因為接下來十六章的每一個 Lab 都會用到它。

24. 承 2.6.4 節與圖 2-19：為什麼建立一組「自己的標準測試影像」值得花時間?請說明挑選時應涵蓋哪些特性，並舉出兩個「只用網路名圖不做自己的測試影像」會遇到的問題。

    提示與參考解答：價值在於可比較性：整學期用同一組素材，你才能回答「第 5 章的 CLAHE 與第 7 章的去雜訊，在同一張暗部照片上誰比較有效」這類跨章問題;而且每一張都是為了考驗某件事而存在，出問題時容易定位。應涵蓋的特性至少有：平滑漸層（測灰階轉換）、銳利邊緣與細密紋理（測銳化與頻譜）、高雜訊與暗部（測去雜訊）、低對比（測直方圖處理）、規則紋理（測陷波與 AOI）、多個可數的物件（測分割與連通元件）。兩個問題：其一，網路名圖多半已經過壓縮與銳化處理，你量到的是「別人處理後的結果」而非真實感測資料;其二，它們與你未來實際要處理的場景（你的相機、你的打光、你的產品）差距極大，調好的參數移植過去往往完全失效——這正是第 18 章專題最常踩的坑。

25. (綜合設計題)請為本課程設計一份「專案起手式」檢查清單，涵蓋環境、程式組織、資料管理與除錯四個面向，每個面向至少三項。接著說明：如果只能保留其中三項，你會留哪三項?為什麼?

> *提示與參考解答：本題無標準答案，評分重點在取捨的理由。一份合理的清單——環境：建立虛擬環境、產生 requirements.txt、記錄 Python 與 OpenCV 版本。程式組織：把常用函式抽成模組（Lab 2-6）、所有路徑寫成常數集中在檔案開頭、主流程寫在 if __name__ == "__main__" 之內。資料管理：原始資料唯讀且永不覆寫、輸出寫到獨立資料夾、資料夾結構固定為 raw / processed / output。除錯：每個步驟後印 shape 與 dtype、把中間結果存檔或並排顯示、隨機過程固定亂數種子以便重現。若只能留三項，一個站得住腳的選擇是：(1) 原始資料唯讀——因為覆寫原始資料是唯一無法復原的錯誤;(2) 虛擬環境加 requirements.txt——因為環境壞掉會讓其他所有努力都無法重現;(3) 每步印 shape 與 dtype——因為它以最低的成本攔截了最大比例的錯誤。其餘項目雖然有用，但都還有補救餘地。*

參考資料與延伸閱讀

- OpenCV 官方文件與教學：https://docs.opencv.org(函式參考與 tutorial)
- OpenCV Anniversary — 專案發展歷程：https://opencv.org/anniversary/(本章叫獸開講之史料來源)
- Bradski, G., & Kaehler, A., Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly, 2008.(創始者親撰的經典入門書)
- NumPy 官方文件：https://numpy.org/doc/(陣列運算與廣播機制)
- Espressif ESP32-CAM 官方範例與文件：https://www.espressif.com
- NVIDIA Jetson 開發者資源：https://developer.nvidia.com/embedded-computing
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 3 章 數位影像基礎

本章學習目標

- 說明影像數位化的兩個步驟——取樣與量化，並理解像素與解析度的意義。
- 陳述 Nyquist 取樣定理，解釋混疊現象的成因與防範方式。
- 說明位元深度與動態範圍的關係，理解量化不足造成的假輪廓現象。
- 區分空間解析度、灰階解析度與時間解析度，並判斷各自的取捨。
- 描述影像感測器的成像原理與色彩濾鏡陣列，理解 RAW 與 JPEG 的差異。
- 運用 MSE、PSNR、SSIM 客觀評估影像品質，並理解其侷限。
- 說明像素鄰域、連通性與距離度量，為後續分割與型態學打底。

叫獸開講|第一張數位影像：一位父親、一台巨型電腦與 176 個像素

1957 年初的某一天，一位二十七歲的年輕科學家把一張兒子的照片帶到辦公室。這件事聽起來平凡無奇——多少新手爸媽都做過同樣的事。但他的辦公室不太一樣：那是美國國家標準局(NBS，今日的 NIST)，而他是少數獲准操作 SEAC 的人之一。SEAC 是美國第一台可程式化的儲存程式電腦，大到能塞滿一整個房間，平時的工作是社會安全帳務、空軍後勤運算，以及氫彈相關的計算。

這位科學家名叫 Russell Kirsch。他心裡有一個當時幾乎沒人問過的問題：「如果電腦能夠看見圖片，會發生什麼事?」為了回答它，他和同事打造了一台滾筒式掃描器——照片貼在旋轉的滾筒上，一端是馬達、另一端是閃頻圓盤，搭配一支能偵測極微弱光線的光電倍增管，隨著滾筒緩緩轉動，透過箱壁上一個方形的觀察孔逐點掃描影像的明暗變化，再轉換成電腦看得懂的二進位數字。

他選來測試的，是三個月大的兒子 Walden 的頭肩照。原本那是一張他與兒子的合影，但因為當時的設備最多只能處理 176×176(共 30,976 個)像素，他把照

片裁成了 5 公分見方的小方塊——順帶讓兒子的臉，比他自己更有名。掃出來的影像模糊、顆粒粗大、灰階層次極少，像個幽靈，卻是人類史上第一張數位影像。Kirsch 後來自嘲說，他「坦承曾竊取原本該用於更有用產品(比如熱核武器計算)的機器時間」。

這 30,976 個像素，大約只有 2010 年代數位相機一張照片的千分之一資訊量，卻成了往後一切的源頭：衛星影像、電腦斷層掃描、商品條碼、桌上排版、數位攝影，全都從這口井裡湧出。2003 年，《生活》雜誌的編輯把它列入「改變世界的一百張照片」。

故事還有一個耐人尋味的尾聲。Kirsch 晚年回顧時，對自己當年的一個選擇感到不滿意：他把像素做成了方形。他說那是為了實作方便而做的權宜，卻讓全世界沿用了半世紀——方形的網格在表現曲線與斜線時天生笨拙(這也是你在放大影像時看到鋸齒的根本原因，第 4 章的內插正是為了緩解它)。晚年他甚至還在研究以三角形等其他形狀來組成影像。這是一位工程師難得的誠實：承認自己的一個「暫時的決定」變成了整個領域的預設，而它未必是最好的答案。

叫獸把這個故事放在第 3 章開頭，是因為本章要談的正是那個「把連續的世界切成一格一格數字」的過程——取樣與量化。Kirsch 當年受限於設備只能取 176×176、只能表現極少的灰階;今天你的手機能拍下數千萬像素、每個通道 12 位元以上。但本質完全相同：我們永遠是在用有限的格子，去逼近一個無限細緻的真實世界。理解這個「逼近」會遺失什麼、又該怎麼把遺失降到最低，就是本章的全部功課。

思考題：Kirsch 為了實作方便選了方形像素，結果影響了半世紀。回想第 2 章 OpenCV 的 BGR 順序，也是類似的故事。你認為工程師在做「暫時的權宜決定」時，該如何判斷它會不會變成難以更改的長期標準?

3.1 從連續到離散：取樣與量化

3.1.1 影像數位化的兩個步驟

真實世界的影像是連續的：在空間上，兩個點之間永遠可以再找到一個點;在亮度上，兩個灰階之間永遠可以再分出更細的差異。但電腦只能處理有限的數字，因此必須把連續的影像轉換成離散的數值陣列。這個轉換分成兩個獨立的步驟：

1. 取樣(Sampling)：在空間上離散化。把影像切成一格一格的小方塊，每一格取一個代表值——這就是像素(pixel,picture element 的縮寫)。取樣的密度決定了空間解析度，也就是我們常說的「幾百萬畫素」。



圖 3-1　數位化的兩個步驟：取樣切空間，量化切亮度

2. 量化(Quantization)：在亮度上離散化。把每個像素的連續亮度值，對應到有限個階層中的一個。階層數由位元深度決定：8 位元有 256 階(0 至 255)，這正是第 2 章 uint8 的由來。

用一個比喻幫助理解：取樣像是決定「用多細的網格去描摹一幅畫」，量化則是決定「調色盤上有幾種顏色可用」。兩者都會造成資訊損失，而且是不可逆的——一旦取樣或量化完成，失去的細節就永遠回不來了(這也預告了第 7 章影像還原的物理上限)。承叫獸開講，Kirsch 當年在兩個維度上都受到極大限制：176×176 的取樣、極少的量化階層。



圖 3-2　取樣率由高到低：先消失的是細節

3.1.2 像素、解析度與尺寸的混淆

這裡有一組經常被混用、實則不同的概念，值得釐清：

- 影像尺寸(Image Size)：影像的像素數，例如 1920×1080。這是陣列的形狀(第 2 章的 shape)。
- 空間解析度(Spatial Resolution)：單位長度內的像素數，例如掃描器的 300 dpi(每英寸點數)、感測器的每毫米像素數。它才真正描述「細節的精細程度」。
- 顯示尺寸(Display Size)：影像在螢幕或紙上實際呈現的大小，取決於顯示裝置的密度。

為什麼要區分?因為「像素多」不等於「細節多」。把一張模糊的照片放大四倍存檔，像素數變成十六倍，細節卻一點也沒增加——第 4 章的內插只是在既有資訊之間「猜」出新像素。真正決定細節的，是拍攝當下的光學品質與取樣密度。這也是為什麼手機廠商的「億級畫素」宣傳需要打個問號：感測器再多像素，若鏡頭的解析力跟不上，多出來的像素只是在記錄模糊。



圖 3-3　像素多不等於細節多

3.1.3 三種解析度

完整地說，數位影像其實有三個維度的解析度，各有取捨：



圖 3-4　三種解析度不足時的三種現象

解析度類型	離散化的維度	不足時的現象	相關章節
空間解析度	空間(取樣)	鋸齒、細節喪失、混疊	第 4 章內插、第 6 章混疊
灰階解析度	亮度(量化)	假輪廓、色階斷層	第 5 章灰階轉換、第 11

			章壓縮
時間解析度	時間(影格率)	運動模糊、時間混疊	第 7 章運動模糊、第 12 章視訊

表 3-1 三種解析度與其不足時的現象

三者之間常有取捨關係。以攝影為例：要提高時間解析度(縮短曝光以凍結動作)，進光量就變少，雜訊相對變大;要提高空間解析度(更多像素)，每個像素接收的光就變少，同樣影響訊噪比。這種「此消彼長」的張力，是感測器設計的核心課題，也會在第 5 章夜景模式、第 7 章運動模糊、第 17 章產線取像中反覆出現。

3.2 取樣定理與混疊

3.2.1 Nyquist 取樣定理

取樣要多密才夠?這個問題有一個精確的答案，由 Nyquist 與 Shannon 給出：取樣頻率必須至少是訊號中最高頻率成分的兩倍，才能完整重建原訊號。用影像的語言說：若影像中最細的紋理是每毫米五個明暗週期，取樣密度就必須至少每毫米十個像素。

為什麼是兩倍?直覺上可以這樣理解：要記錄一個完整的波形週期，至少需要兩個取樣點——一個記錄波峰、一個記錄波谷。若每個週期只取到一個點，你完全無從得知這個波的存在，甚至會誤以為它是別的東西。這個「別的東西」，就是下一節的混疊。

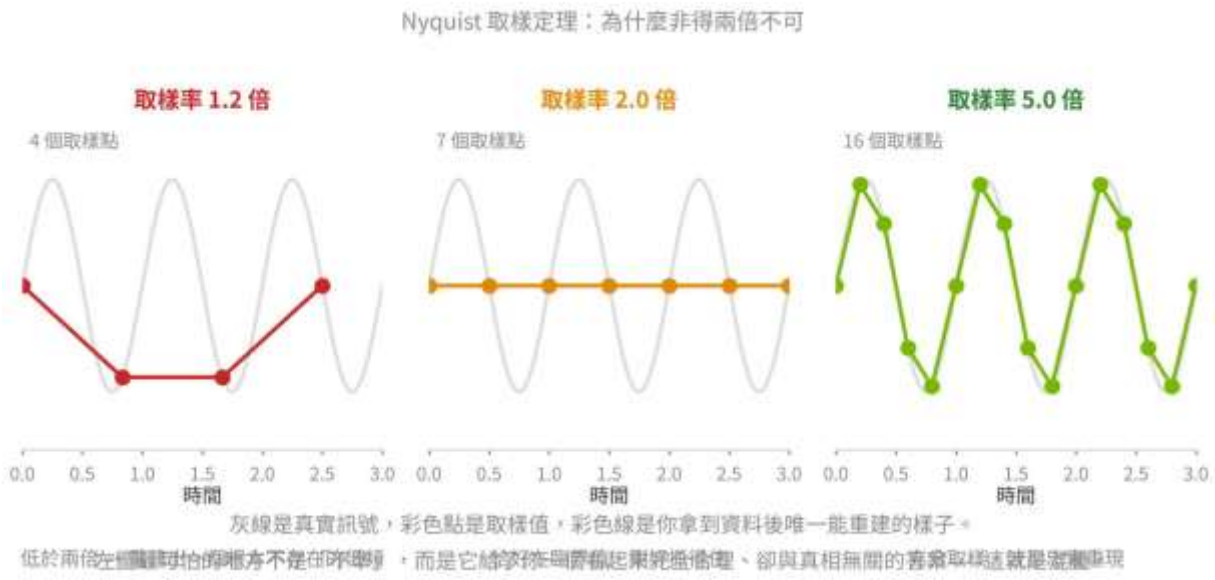


圖 3-5　取樣率 1.2 倍、2 倍、5 倍的重建結果

這條定理的完整意義，要到第 6 章的頻率域才說得清楚：取樣的動作在頻域上會使原訊號的頻譜以取樣頻率為間隔無限複製;取樣夠密時複製品彼此分離、可用低通濾波取回原訊號，不夠密則相鄰複製品重疊、高頻成分「摺疊」進低頻區。第 6.6.1 節會完整展開這個解釋。這裡先記住結論：取樣不足，高頻會偽裝成低頻。

3.2.2 混疊的三張面孔

混疊(Aliasing)是取樣不足的直接後果，它在不同場合有不同的樣貌，但成因完全相同：

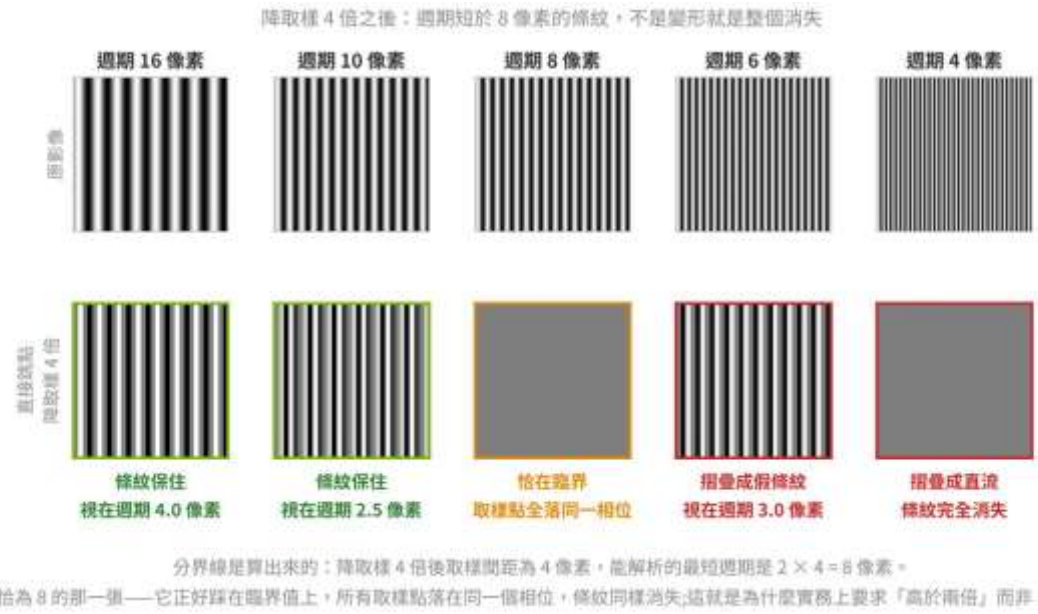


圖 3-6　混疊的分界線：降取樣四倍後的五種條紋

3. 空間混疊——摩爾紋：拍攝細密的條紋(格子襯衫、螢幕、紗窗)時出現的詭異波紋。這是螢幕像素網格與相機感測器網格相互干涉，高頻紋理摺疊成了低頻的粗大波紋。第 6.7.1 節會示範如何以陷波濾波移除它。
4. 鋸齒(Jaggies)：斜線與曲線在低解析度下呈現階梯狀。這是叫獸開講中方形像素的直接後果，也是抗鋸齒(anti-aliasing)技術要對付的問題——其原理正是先做低通模糊，再取樣。
5. 時間混疊——車輪倒轉：影片中高速旋轉的車輪或電風扇看似靜止或倒轉。這是影格率(時間取樣)不足，高頻的旋轉運動被摺疊成了低頻甚至反向的運動。第 12 章叫獸開講的思考題正是這個現象。

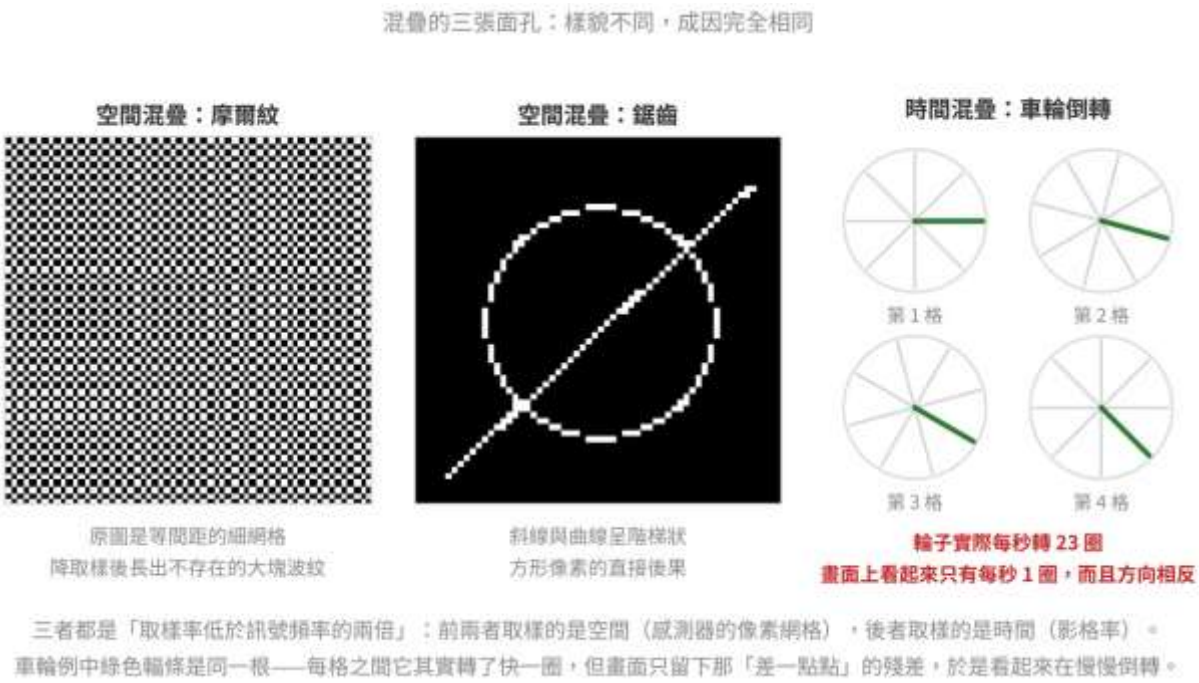


圖 3-7　混疊的三張面孔：摩爾紋、鋸齒、車輪倒轉

3.2.3 如何防範

防範混疊的方法只有一個原則：在取樣之前，先把超過取樣率一半的高頻成分濾掉。這稱為抗混疊濾波(anti-aliasing filter)，硬體上以光學低通濾鏡實現，軟體上則以模糊處理實現。

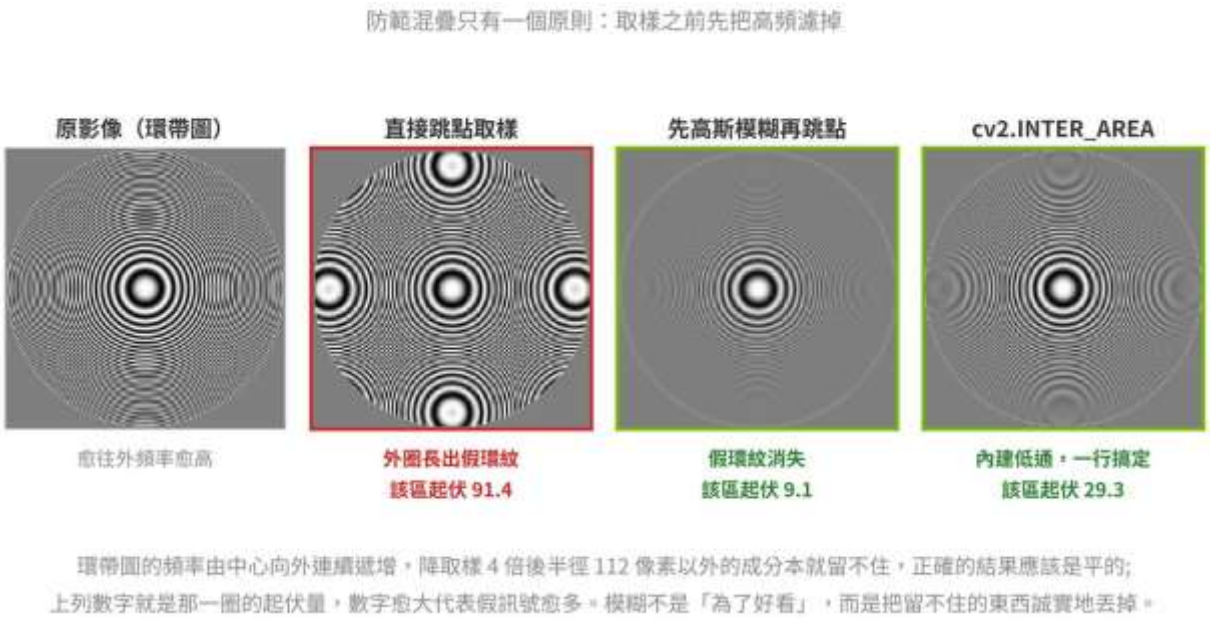


圖 3-8　三種縮圖做法在環帶圖上的實測比較

這解釋了一個常令初學者困惑的建議：縮小影像前應先做模糊。因為縮小就是降低取樣率，若不先把高頻濾掉，它們就會摺疊成假訊號。OpenCV 的 INTER_AREA 內插之所以適合縮小，正是因為它內建了區域平均(即低通)的動作——第 4.3 節的口訣「縮小用 AREA」，根源就在這裡。

程式 3-1 混疊的產生與防範

```
import cv2, numpy as np

# --- 製造混疊:合成高頻條紋後直接降取樣 ---
N = 512
x = np.arange(N)
stripes = ((np.sin(2 * np.pi * x / 4) + 1) * 127).astype(np.uint8)
img = np.tile(stripes, (N, 1))              # 每 4 像素一個週期(高頻)

# 錯誤做法:直接跳點取樣 → 混疊
bad = img[::4, ::4]                          # 條紋消失或變成假圖案

# 正確做法:先低通模糊,再降取樣
blurred = cv2.GaussianBlur(img, (0, 0), sigmaX=2)
good = blurred[::4, ::4]

# OpenCV 的 INTER_AREA 內建低通,是縮小的建議選項(第 4 章)
area = cv2.resize(img, (N//4, N//4), interpolation=cv2.INTER_AREA)

cv2.imwrite("alias_bad.png", bad)
cv2.imwrite("alias_good.png", good)
cv2.imwrite("alias_area.png", area)
# 並排比較:bad 會出現原圖沒有的假圖案
```

3.2.4 推導：取樣在頻率域上究竟做了什麼

3.2.1 節說「這條定理的完整意義，要到第 6 章的頻率域才說得清楚」。這裡先把骨架推一次——它只有四個步驟，而且能讓「摺疊」這個比喻變成字面上的事實。

第一步：取樣這個動作，在數學上等於把連續訊號乘上一列等間距的脈衝。若取樣間距為 Δ，這列脈衝就位於 0、Δ、2Δ、3Δ……的位置，相乘之後只有這些位置的值被留下來，其餘全部歸零。

第二步：依卷積定理（6.5 節），空間域的相乘等於頻率域的卷積。而脈衝列有一個漂亮的性質——它的傅立葉轉換仍然是一列脈衝，只是間距變成了 1/Δ。於是「取樣」在頻率域上就是「原訊號的頻譜與一列間距 1/Δ 的脈衝做卷積」。

第三步：與脈衝列卷積的效果，是把原頻譜原封不動地複製到每一個脈衝的位置。換句話說，取樣讓原本只有一份的頻譜，變成以 1/Δ 為間隔無限延伸的一排複製品。

第四步，也是結論：這些複製品會不會互相重疊，取決於原訊號的最高頻率 f_max 與間隔 1/Δ 的關係。若 f_max < 1/(2Δ)，相鄰的複製品之間留有空隙，彼此分離，此時只要用一個低通濾波器把中央那一份取出來，就能完整重建原訊號——這正是取樣定理。反之若 f_max > 1/(2Δ)，複製品的裙襬互相重疊，高頻的尾巴侵入了鄰居的低頻區，而且重疊之後再也分不開：你無法判斷某個低頻成分究竟是原本就有的，還是從高頻摺疊過來的。

「摺疊」兩個字是字面意義，而且可以直接算。一個頻率為 f 的成分，取樣後會出現在 $|f - k/\Delta|$ 的位置（k 為整數，取使結果最小的那個）。以圖 3-6 週期 6 像素的條紋為例：原頻率為 1/6 週期 / 像素，降取樣 4 倍後在新影像中的頻率是 $4/6 \approx 0.667$，超過了 Nyquist 頻率 0.5，於是摺疊到 $|0.667 - 1| = 0.333$，對應的視在週期是 $1/0.333 = 3$ 像素——這與圖中量到的數字完全一致。

這個推導也一次解釋了三件事。其一，為什麼抗混疊濾波一定要在取樣之前：重

疊一旦發生就不可分離，事後再怎麼濾波都救不回來。其二，為什麼圖 3-6 中週期恰為 8 的條紋也失敗：f_max 恰好等於 1/(2Δ) 時，相鄰複製品剛好相切，理論上仍可分離，但需要一個理想的低通濾波器（第 6.3.2 節會說明它不存在），而且結果對取樣相位極度敏感。其三，為什麼第 6 章要花一整章談頻率域——本章的現象在那裡都只是這張頻譜複製圖的不同切面。

3.3 量化與位元深度

3.3.1 位元深度與動態範圍

位元深度(bit depth)決定每個像素能表示多少個灰階層次：1 位元只有黑與白兩階(二值影像，第 10 章的主角);8 位元有 256 階，是最常見的規格;10、12、14 位元則常見於專業相機的 RAW 檔與醫學影像。彩色影像的位元深度通常以「每通道」計算，8 位元的 RGB 影像共有約一千六百萬種顏色組合。

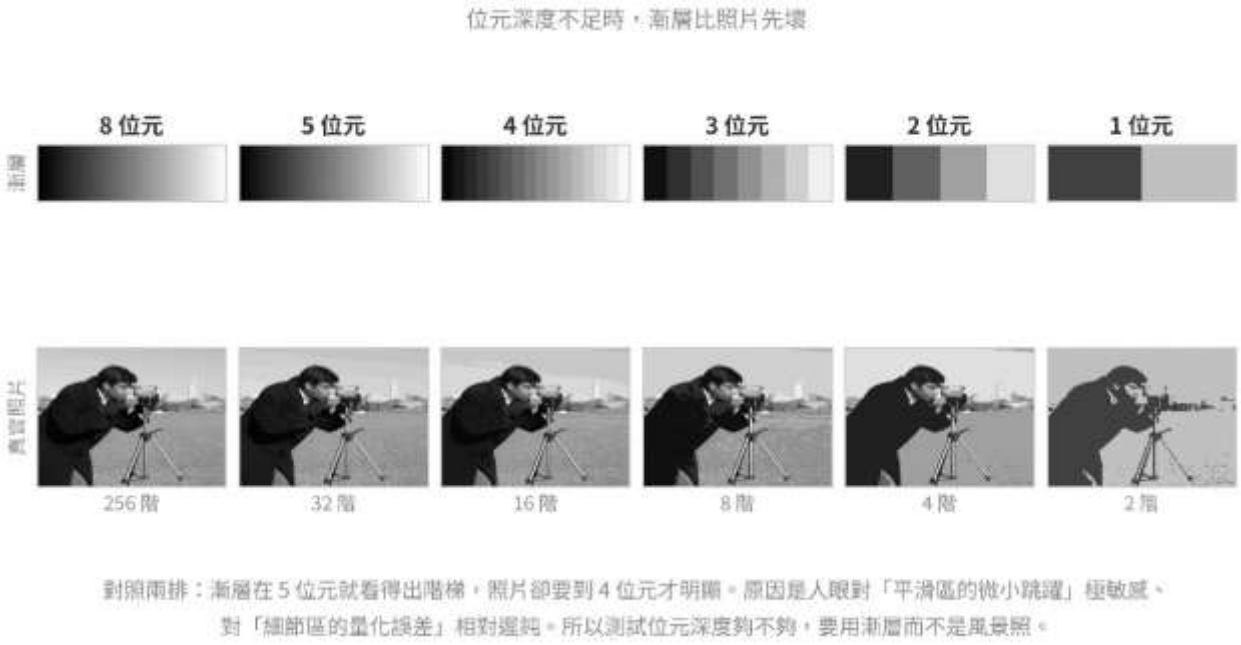


圖 3-9　位元深度 8 到 1：漸層比照片先壞

動態範圍(Dynamic Range)則描述影像能同時記錄的最亮與最暗之間的比值。人眼在單一場景中約可分辨十多個 f-stop 的亮度差異，而一般 8 位元 JPEG 只能記錄較窄的範圍——這就是為什麼逆光拍攝時，若曝光在人臉，背景的天空就會過曝成一片

白。高動態範圍(HDR)技術以不同曝光的多張影像合成，正是為了突破這個限制，與第 5 章夜景模式的多影格合併是同源的思路。

3.3.2 量化不足：假輪廓

當位元深度不足時，最明顯的現象是假輪廓(false contouring，或稱色階斷層、posterization)：在本該平滑漸變的區域(晴朗的天空、柔和的陰影、漸層背景)出現一圈圈生硬的階梯狀邊界。原因很單純：真實的漸變是連續的，但量化後只能用有限的幾階去表示，相鄰階層之間就出現了肉眼可見的跳躍。

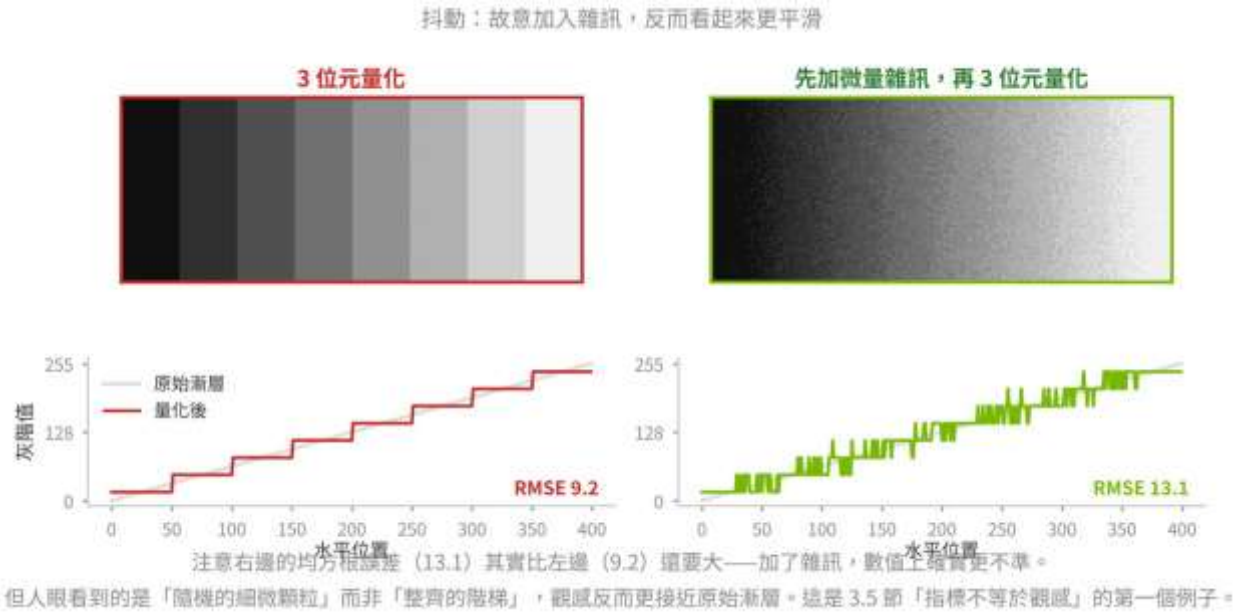


圖 3-11　抖動：加入微量雜訊反而看起來更平滑

有趣的是，人眼對「平滑區域的微小跳躍」特別敏感，對「細節豐富區域的量化誤差」反而遲鈍。這個特性有兩個實務意義：其一，漸層區域是判斷位元深度是否足夠的最佳測試素材;其二，刻意加入極微量的雜訊(稱為抖動，dithering)反而能打散階梯邊界、讓漸變看起來更平滑——這是「用雜訊改善觀感」的少見例子。



圖 3-10　假輪廓為什麼只長在平滑的區域

程式 3-2 量化與假輪廓

```
import cv2, numpy as np

# 建立平滑漸層(最容易看出量化不足的素材)
ramp = np.tile(np.linspace(0, 255, 512, dtype=np.uint8), (200, 1))

# 模擬不同位元深度的量化
for bits in [8, 5, 4, 3, 2, 1]:
    levels = 2 ** bits
    step = 256 // levels
    q = (ramp // step) * step              # 量化
    cv2.imwrite(f"quant_{bits}bit.png", q)
    print(f"{bits} 位元 = {levels} 階")
    # 觀察:位元數愈少,漸層上的階梯(假輪廓)愈明顯

# 抖動:加入微量雜訊打散階梯邊界
noise = np.random.normal(0, 8, ramp.shape)
dithered = np.clip(ramp + noise, 0, 255).astype(np.uint8)
q_dither = (dithered // 32) * 32           # 同樣量化成 3 位元
cv2.imwrite("quant_3bit_dither.png", q_dither)
# 與 quant_3bit.png 比較:加了雜訊反而看起來更平滑
```

3.3.3 量化的一體多面

「用有限的階層逼近連續值」這個思想，會在本書中以三種面貌出現，值得在此先建立連結：本章談的是感測器把光強度量化成灰階;第 11 章 JPEG 壓縮的量化步驟，

是把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻;第 17 章的模型量化，則是把神經網路的權重從 32 位元浮點降到 8 位元整數以加速推論。三者的數學核心完全相同——都是在可接受的品質損失下，換取儲存、傳輸或運算的大幅節省。當你在第 11、17 章再次遇到「量化」時，請記得回到本章。

3.3.4 推導：每多一個位元，訊噪比為什麼提升約 6 dB

位元深度與影像品質的關係，可以用一條簡潔的經驗法則概括：每增加一個位元，訊噪比提升約 6 分貝。這條法則在選感測器、決定存檔格式、評估量化影響時都用得上，而它的推導只需要兩步。

第一步，量化誤差有多大。設量化步長為 Δ（8 位元時 $\Delta = 1$，即一個灰階）。四捨五入的量化，其誤差必落在 $-\Delta/2$ 與 $+\Delta/2$ 之間;當訊號本身變化夠豐富時，可以合理地假設誤差在這個區間內均勻分布。均勻分布於 $[-\Delta/2, \Delta/2]$ 的隨機變數，其變異數為 $\Delta^2 / 12$——這是把 e^2 在該區間上積分再除以區間長度的結果，是機率論的標準練習。於是量化雜訊的功率就是 $\Delta^2 / 12$。

第二步，位元數改變時會發生什麼。位元數由 b 增加到 $b + 1$，階層數加倍，同樣的滿刻度範圍被切成兩倍多的階，因此步長 Δ 減半。誤差功率正比於 Δ^2，所以變成原本的四分之一。換算成分貝：$10 \times \log_{10}(4) = 6.02$ 分貝。這就是 6 dB 法則的全部內容——它其實只是「$20 \log_{10} 2$」換個說法。

若把訊號也一併算進去，對一個佔滿整個量化範圍的正弦訊號，可以推出常見的工程公式：$\text{SNR} \approx 6.02b + 1.76$ 分貝。代入幾個數字就很有感覺：8 位元約 50 分貝，12 位元約 74 分貝，14 位元約 86 分貝。由 8 位元升到 12 位元，等於多了 $4 \times 6.02 \approx 24$ 分貝的餘裕——這正是圖 3-14 中 RAW 經得起提亮三倍、JPEG 經不起的量化版本。

兩個必要的提醒。第一，這是理論上限，實務上達不到：感測器的讀取雜訊、暗電流、散粒雜訊（3.4.1 節）都會疊加上去，當它們大於量化雜訊時，再多的位元也不

會改善訊噪比——這就是為什麼手機感測器不會一路堆到 16 位元。一個實用的判準是：量化步長應小於感測器本身的雜訊水準，再小就是在精確地記錄雜訊。第二，均勻分布的假設在訊號幾乎不變時會失效——這正是圖 3-10 的平滑區：該區域的起伏小於一個量化步長，誤差不再隨機，而是整片同向偏移，於是表現為假輪廓而非雜訊。圖 3-11 的抖動之所以有效，正是因為它人為地讓誤差重新變回隨機。

3.4 影像的產生：感測器與色彩

3.4.1 感測器如何看見

數位相機的感測器(CCD 或 CMOS)由數百萬個微小的光電二極體組成，每一個對應一個像素。曝光期間，光子撞擊感光元件產生電子，累積的電荷量正比於接收到的光量;曝光結束後，電荷被讀出、放大、經類比數位轉換器量化成數字。



圖 3-12　感測器的成像鏈與其中兩個不可逆的環節

這個過程解釋了幾個重要現象。其一，光子到達是隨機的，因此即使拍攝完全均勻的畫面，各像素的數值也會有隨機起伏——這就是第 7 章的 Poisson 雜訊(散粒雜訊)，它在低光時特別明顯，也是第 5 章夜景模式要對付的主角。其二，讀出與放大電路本身也會引入雜訊(讀取雜訊)。其三，像素若接收過多光子會「滿溢」，數值卡在

最大值——這就是過曝，一旦發生，該區域的資訊就徹底喪失，任何後製都救不回來。

3.4.2 色彩濾鏡陣列與去馬賽克

這裡有一個常被忽略的事實：感光元件本身只能感測光的強度，不能分辨顏色。那彩色照片是怎麼來的?答案是在感測器表面覆蓋一層色彩濾鏡陣列(CFA)，讓每個像素只接收特定顏色的光。最常見的是拜爾陣列(Bayer pattern)，它以「一紅、兩綠、一藍」為單位重複排列——綠色佔一半，是因為人眼對綠光最敏感(這個設計思想與第 8 章 YCbCr 對色度降取樣、第 11 章 JPEG 的 4:2:0 完全一致：把資源分配給人眼最在意的地方)。



圖 3-13　拜爾陣列與去馬賽克：彩色是濾出來又猜回來的

於是每個像素其實只記錄了三個顏色中的一個，另外兩個必須由鄰近像素「猜」出來——這個過程稱為去馬賽克(demosaicing)，本質上是一種內插(第 4 章)。這也解釋了為什麼相機的彩色細節解析力，實際上低於它的像素數所暗示的水準，以及為什麼在高對比的細節邊緣容易出現偽色。

3.4.3 RAW 與 JPEG：一個關鍵的取捨

相機可以輸出兩種檔案。RAW 保存的是感測器讀出的原始資料——高位元深度(通常 12 至 14 位元)、線性(數值正比於光量)、未經壓縮或僅無損壓縮、未套用白平衡

與銳化。JPEG 則是相機內部已經完成去馬賽克、白平衡、色調曲線、Gamma 編碼、銳化，並以有損壓縮(第 11 章)存成 8 位元的成品。

對影像處理而言，這個差異至關重要，有三個理由：第一，位元深度——12 位元的 RAW 有 4096 階，遠多於 JPEG 的 256 階，後製調整曝光時不易產生假輪廓。第二，線性——RAW 的數值正比於實際光量，而 JPEG 經過 Gamma 編碼後並非線性(第 5.1.2 節詳述)，做精確的光度量測時必須留意。第三，未經破壞——JPEG 的壓縮與銳化都是不可逆的，且反覆存檔會累積失真(第 11.4.2 節)。



圖 3-14　RAW 與 JPEG 各自提亮三倍後的灰階存量

因此本書的建議是：做嚴肅的影像處理與科學量測時，盡可能從 RAW 出發;第 7 章的還原、第 14 至 16 章的模型訓練資料蒐集，都會因為輸入品質而受益。當然，RAW 檔案大、處理慢，一般應用用 JPEG 就夠——這又是一個依需求判斷的工程取捨。

3.5 影像品質的客觀評估

當你用某個演算法處理影像後，如何判斷它「好不好」?主觀的目視評估重要但不可量化、不可重現;因此我們需要客觀指標。這一節建立的工具，將在第 5、6、7、11 章的每一個 Lab 中反覆使用。

3.5.1 MSE 與 PSNR

均方誤差(MSE)是最基本的指標：把處理後影像與參考影像逐像素相減、取平方、再平均。它直接反映「差多少」，值愈小愈好，但它的數值大小與影像的亮度範圍有關，不同影像之間難以比較。

峰值訊噪比(PSNR)以對數形式改良了這一點，單位是分貝(dB)，值愈大愈好。實務上的經驗參考值：超過 40 dB 表示品質極佳、幾乎看不出差異;30 至 40 dB 為良好;20 至 30 dB 可見明顯劣化;低於 20 dB 則品質很差。它是影像壓縮與還原領域最廣泛使用的指標(第 7、11 章)。



圖 3-15　五種劣化、幾乎相同的 PSNR、天差地別的觀感

3.5.2 SSIM 與 PSNR 的盲點

但 PSNR 有一個根本盲點：它逐像素比較，完全不考慮人類視覺的特性。舉一個經典的例子——把整張影像的亮度平均提高一點點，PSNR 會顯著下降，但人眼幾乎看不出差異;反之，在平坦區域加入少數幾個突兀的雜點，PSNR 下降有限，人眼卻一眼就看到刺眼的瑕疵。換言之，PSNR 高不代表「看起來好」。



圖 3-16　全域指標衡量平均，而你在乎的是最壞的那一塊

結構相似性指標(SSIM)為此而生。它不逐像素比較，而是分別衡量兩張影像在亮度、對比度、結構三個面向的相似程度，再綜合成一個 0 到 1 之間的分數(1 代表完全相同)。因為它著眼於「結構」，更貼近人類的感知。實務建議是兩者並用：PSNR 反映數值誤差，SSIM 反映感知品質，兩者一致時結論可靠，不一致時就值得深究原因。



圖 3-17　SSIM 差異圖：不只「差多少」，還有「差在哪裡」

還有一個更根本的提醒，將在第 7.6.1 節的 MRI 案例中詳述：全域指標衡量的是

平均表現，而許多應用關心的是最壞情況。一個把微小病灶抹掉的去噪演算法，PSNR 可能不降反升(因為背景更乾淨)，但在醫學上是災難。指標是工具，不是目的——這個判斷力，是本書希望你建立的核心素養之一。

程式 3-3 MSE、PSNR 與 SSIM

```
import cv2, numpy as np

clean = cv2.imread("original.png", cv2.IMREAD_GRAYSCALE)
noisy = cv2.imread("processed.png", cv2.IMREAD_GRAYSCALE)

# --- MSE ---
mse = np.mean((clean.astype(np.float64) - noisy.astype(np.float64)) ** 2)
print(f"MSE  = {mse:.2f}")

# --- PSNR(OpenCV 內建) ---
print(f"PSNR = {cv2.PSNR(clean, noisy):.2f} dB")

# --- SSIM(需安裝 scikit-image:pip install scikit-image) ---
from skimage.metrics import structural_similarity as ssim
score, diff_map = ssim(clean, noisy, full=True)
print(f"SSIM = {score:.4f}")

# SSIM 差異圖:可視化「哪裡不像」,除錯很有用
diff_vis = (diff_map * 255).astype(np.uint8)
cv2.imwrite("ssim_map.png", diff_vis)

# --- 示範 PSNR 的盲點 ---
shifted = np.clip(clean.astype(np.int16) + 5, 0, 255).astype(np.uint8)
print(f"整體加亮 5:PSNR={cv2.PSNR(clean, shifted):.2f} dB,"
      f" SSIM={ssim(clean, shifted):.4f}")
# 人眼幾乎看不出差異,但 PSNR 顯著下降 → 指標未必等於觀感
```

3.5.3 推導：PSNR 的那些經驗值從哪裡來

3.5.1 節給了幾個經驗參考值——40 分貝以上極佳、30 到 40 分貝良好。這些數字不是慣例，而是可以算出來的，而且算一次之後你對 PSNR 的理解會完全不同。

由定義出發。均方誤差為 MSE，峰值訊噪比定義為 $\text{PSNR} = 10 \times \log_{10}(\text{MAX}^2 / \text{MSE})$，其中 MAX 是像素的最大可能值，8 位元影像為 255。把常數拆出來：$\text{PSNR} = 20 \times \log_{10}(255) - 10 \times \log_{10}(\text{MSE})$，而 $20 \times \log_{10}(255) = 48.13$。於是對所有 8 位元灰階影像，都有 $\text{PSNR} = 48.13 - 10 \times \log_{10}(\text{MSE})$。

把經驗值代進去反推，意義就具體了。PSNR = 40 分貝對應 MSE = 6.5，換算成

均方根誤差是每個像素平均差 2.5 個灰階——在 256 階中差 2.5 階，人眼確實看不出來。PSNR = 30 分貝對應 MSE = 65，平均差 8.1 個灰階，開始在平滑區域看得出端倪。PSNR = 20 分貝對應 MSE = 650，平均差 25.5 個灰階，那已經是一眼就看得出的劣化。更簡潔的記法是：PSNR 每下降 10 分貝，MSE 增為十倍，平均誤差增為 $\sqrt{10} \approx 3.16$ 倍。

這個推導也揭露了一件常被誤解的事：PSNR 與 MSE 之間是一對一的單調對應，兩者攜帶的資訊完全相同。PSNR 並沒有比 MSE「更懂」影像，它的價值只有兩點——其一，除以 MAX^2 之後數值與影像的動態範圍無關，因此不同影像之間可以比較（這正是 3.5.1 節說 MSE 做不到的事）；其二，取對數之後的尺度比較接近人對「差多少」的感受，也讓數值落在一個好記的範圍。除此之外，MSE 的所有盲點，PSNR 一個也沒少——圖 3-15 與 3-16 示範的正是這一點。

最後補一個實務細節：比較不同論文或不同工具的 PSNR 數字時要小心，因為計算方式可能不同。彩色影像有人先轉灰階再算、有人三個通道各自算再平均、也有人只算 Y 通道（第 8 章）；有人把邊界幾個像素排除、有人不排除。這些差別足以造成一兩分貝的落差，而一兩分貝在論文的比較表上往往就是勝負。自己做實驗時，請把計算方式明確寫下來——這是 2.5.4 節「可重現」的具體實踐。

3.6 像素的鄰域、連通性與距離

本節建立的三個概念看似瑣碎，卻是第 9 章分割、第 10 章型態學與連通元件標記的直接基礎。

3.6.1 鄰域與連通性

一個像素的「鄰居」有兩種定義方式：4-鄰域只算上下左右四個像素；8-鄰域則加上四個對角，共八個。由此衍生出兩種連通性，而選擇哪一種會直接影響結果——這在第 10 章計算「畫面中有幾個物件」時尤其關鍵。



圖 3-18　4-鄰域、8-鄰域，與對角相接的經典困境

舉一個經典的困境：兩個只在對角相接的白色區塊，在 8-連通下算一個物件，在 4-連通下算兩個;而一條斜向的細線，在 4-連通下會被判定為斷裂的一群孤立點。更麻煩的是所謂的連通性悖論——若前景與背景採用同一種連通性，會出現「一條封閉的曲線卻沒有把背景分成內外兩部分」這類矛盾。實務上的慣例是前景與背景採用互補的連通性(例如前景 8-連通、背景 4-連通)。

3.6.2 距離度量

兩個像素有多遠?三種常見的定義各有用途：歐氏距離(直線距離)最符合直覺、也最常用於量測;城市街區距離(D4，只能上下左右移動的步數)與棋盤距離(D8，可斜走的步數)則計算快速，且與 4-連通、8-連通對應。

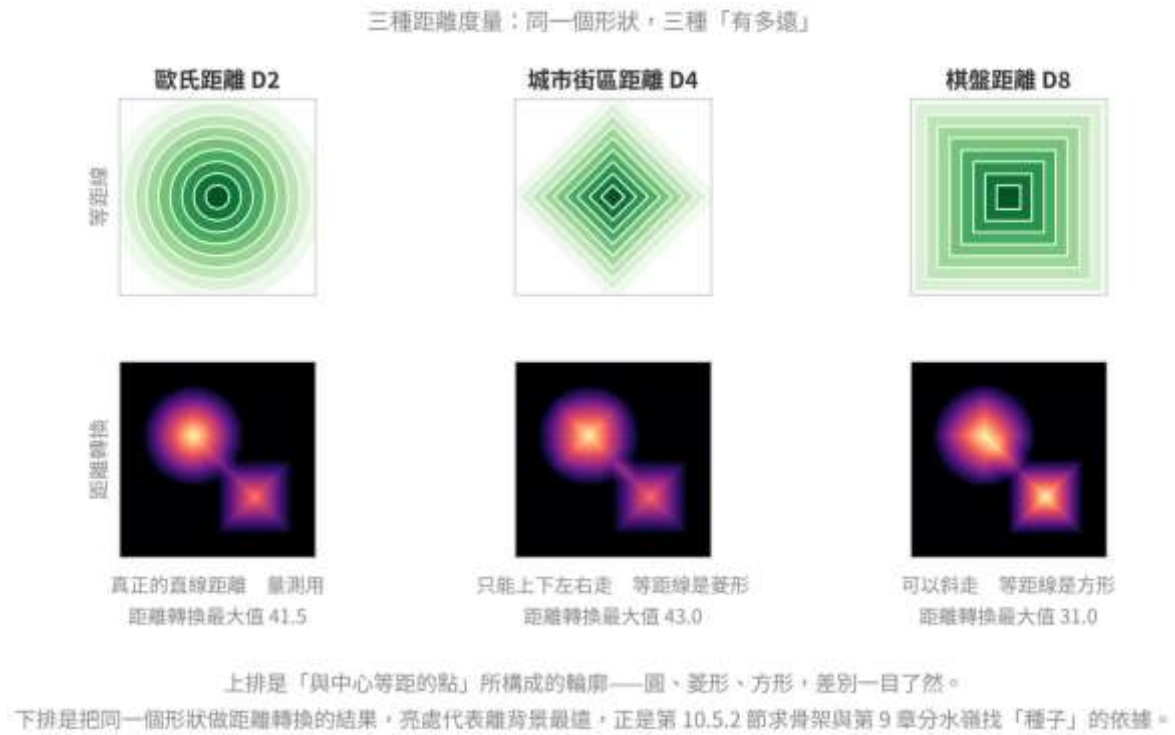


圖 3-19　三種距離度量的等距線與距離轉換

這些度量在第 10.5.2 節的距離轉換中扮演核心角色：為每個前景像素計算它到最近背景像素的距離，得到的「距離圖」可用於求物件骨架，更是第 9 章分水嶺分割「分開相黏物件」的關鍵——每個相黏圓形物件的中心會在距離圖上形成一個局部極大值，正好作為分水嶺的標記。屆時請記得回到本節。

程式 3-4 連通性與距離度量

```
import cv2, numpy as np

# 建立一個測試二值影像:兩個對角相接的方塊
img = np.zeros((100, 100), np.uint8)
img[20:50, 20:50] = 255
img[50:80, 50:80] = 255            # 與前者僅在一個角相接

# 連通性的影響:同一張圖,不同答案
n4, _ = cv2.connectedComponents(img, connectivity=4)
n8, _ = cv2.connectedComponents(img, connectivity=8)
print(f"4-連通:{n4-1} 個物件")     # 2(視為分開)
print(f"8-連通:{n8-1} 個物件")     # 1(視為相連)

# 三種距離度量的距離轉換(第 10 章)
for name, dist_type in [("歐氏", cv2.DIST_L2),
                        ("城市街區", cv2.DIST_L1),
                        ("棋盤", cv2.DIST_C)]:
    d = cv2.distanceTransform(img, dist_type, 5)
```

```
    print(f"{name}距離:最大值 = {d.max():.1f}")
    vis = cv2.normalize(d, None, 0, 255, cv2.NORM_MINMAX)
    cv2.imwrite(f"dist_{dist_type}.png", vis.astype(np.uint8))
```

3.7 應用案例

3.7.1 案例一：掃描解析度該設多少

情境：要把一批紙本文件或老照片數位化典藏，掃描解析度該設 150、300 還是 600 dpi?這是取樣定理最貼近生活的應用。判斷依據是「你要保留的最細特徵有多細」：一般文字文件的筆畫寬度約 0.2 至 0.3 毫米，依取樣定理需要每毫米至少八到十個取樣點，對應約 200 至 300 dpi;若要保留印刷網點或做後續的文字辨識(OCR)，則建議 300 至 600 dpi;而老照片的細節(如髮絲、織品紋理)則可能需要更高。

反過來說，盲目提高解析度也有代價：檔案大小以平方成長(600 dpi 是 300 dpi 的四倍大)、掃描時間拉長、且若原稿本身的細節有限，多出來的像素只是在記錄底片顆粒或紙張纖維。這個「依需求決定規格」的思考，正是第 17 章延遲預算與第 18 章專題規劃的同一種能力。

3.7.2 案例二：醫學影像為什麼需要高位元深度

情境：CT 與 MRI 影像通常以 12 至 16 位元儲存，遠高於一般照片的 8 位元。為什麼?因為人體組織之間的密度差異可能極小——一個早期的腫瘤與周圍正常組織的訊號差，可能只有幾個灰階單位。若量化成 256 階，這個差異可能被歸進同一階而完全消失。

但這帶來一個顯示的難題：一般螢幕只能顯示 256 階灰度，人眼也分辨不了那麼多階。解法是「視窗化」(windowing)：醫師依觀察目標，把 12 位元資料中的某一段範圍拉伸到完整的顯示範圍——看肺部用一組視窗、看骨骼用另一組、看軟組織再一組。同一份資料，不同的視窗設定會看到截然不同的內容。這其實就是第 5 章灰階轉換的醫學版本，也說明了為什麼高位元深度的原始資料必須完整保存：它承載了顯示時尚未取用的資訊。

3.7.3 案例三：工業檢測的取像規格計算

情境：要檢測一個工件表面上寬度 0.1 毫米的刮痕，相機該選多少像素?這是產線設計的第一道題，而它是可以精確計算的。

計算方式：依取樣定理，要可靠地偵測 0.1 毫米的特徵，至少需要每 0.05 毫米一個像素(兩倍取樣);實務上為了穩健，常要求三到五個像素涵蓋最小特徵，即每像素約 0.02 至 0.03 毫米。若工件的視野寬度是 60 毫米，則所需的水平像素數約為 60 除以 0.025，即 2400 像素。加上安全餘裕，選一台 3000 萬像素等級的相機或縮小視野分次拍攝。



圖 3-20　由最小特徵推算相機規格的四個步驟

這個案例展示了工程與「調參數」的差別：規格不是試出來的，而是從需求推算出來的。同樣的推算思維會出現在第 7.6.3 節(由產線速度與曝光時間推算運動模糊長度)與第 17.4.2 節(由機器人速度推算延遲預算)。這種「從需求反推規格」的能力，是叫獸最希望你從這門課帶走的東西之一。

3.7.4 案例四：影像壓縮的品質決策

情境：網站要上傳數千張商品照片，JPEG 品質參數該設多少?這是 3.5 節評估指標的實戰。做法：取一組代表性的照片，分別以品質 95、85、75、60、40 存檔，記

錄每個設定的檔案大小、PSNR 與 SSIM，繪製「檔案大小對品質」的曲線(即第 11 章的率失真曲線)，找出「再降就明顯劣化」的轉折點。

經驗上，品質 85 附近往往是甜蜜點——檔案大小比 95 小得多，而肉眼幾乎看不出差異。但這個結論會因影像內容而異：含大面積平滑漸層的照片對壓縮較敏感(容易出現第 11 章的方塊效應)，而紋理豐富的照片則較耐壓。這也再次呼應 3.5.2 節：指標要看，眼睛也要看。

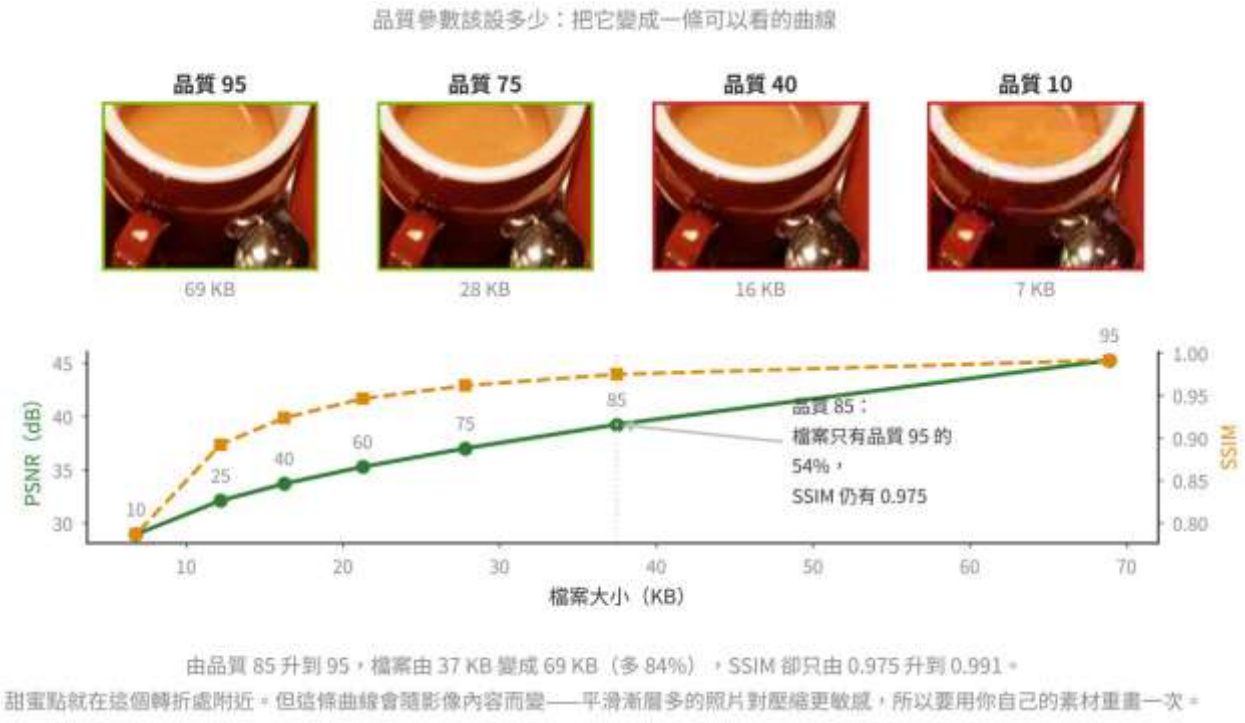


圖 3-21　JPEG 品質參數掃描：檔案大小與品質的取捨曲線

3.8 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
取樣與內插	第 4 章	縮放的內插法選擇;縮小用 INTER_AREA 防混疊
取樣定理與混疊	第 6、12 章	頻域解釋(頻譜複製與摺疊);摩爾紋移除;時間混疊
量化	第 11、17 章	JPEG 的 DCT 係數量化;神經網路的模

		型量化
位元深度與動態範圍	第 5、7 章	Gamma 與對數轉換;HDR;RAW 為還原的理想輸入
感測器雜訊	第 5、7 章	Poisson 與讀取雜訊;夜景多影格平均;去雜訊方法
拜爾陣列與去馬賽克	第 4、8、11 章	內插;人眼對綠光敏感;色度降取樣的同源思想
MSE、PSNR、SSIM	第 5-7、11、14 章	所有處理成效的客觀驗收;全域指標的盲點
鄰域與連通性	第 9、10 章	連通元件標記的計數依據;型態學的結構元素
距離度量	第 9、10 章	距離轉換;分水嶺標記;骨架化

表 3-2 第 3 章與全書其他章節的觀念連結

程式實作 Lab 3

Lab 3-1 取樣、混疊與抗混疊

目標：親眼見證取樣定理。(a)以 np.sin 合成一組不同疏密的條紋影像(每 2、4、8、16 像素一個週期);(b)對每張直接跳點降取樣四倍，觀察哪些出現了「原圖沒有的假圖案」，並用取樣定理解釋為什麼;(c)改為先高斯模糊再降取樣，比較結果;(d)用 OpenCV 的四種內插旗標(NEAREST、LINEAR、CUBIC、AREA)各縮小一次，比較混疊程度，驗證第 4 章「縮小用 AREA」的建議;(e)實拍：用手機拍攝格子襯衫、紗窗或電腦螢幕，重現摩爾紋，並保存這張影像——第 6 章 Lab 會用它來做陷波濾波移除。

Lab 3-2 量化與假輪廓

目標：理解位元深度的意義。(a)以程式 3-2 對一張平滑漸層與一張真實照片分別做 8、5、4、3、2、1 位元的量化，並排比較;(b)記錄「從幾位元開始肉眼可見假輪

廓」，並比較漸層與紋理豐富的照片，何者較早出現、為什麼;(c)實作抖動：量化前加入微量高斯雜訊，比較有無抖動的視覺差異;(d)量化計算 PSNR 與 SSIM 隨位元數的變化，繪成曲線;(e)討論：為什麼醫學影像需要 12 位元以上?(對照 3.7.2 節)

Lab 3-3 品質評估工具箱

目標：建立整學期都會用到的評估工具。(a)寫一個函式 evaluate(original, processed)，一次回傳 MSE、PSNR、SSIM 三個指標，並輸出 SSIM 差異圖;(b)驗證 PSNR 的盲點：對同一張影像做兩種處理——整體亮度加 5、以及在平坦區加入十個突兀的雜點，比較兩者的 PSNR 與 SSIM，並用肉眼判斷哪個比較難看;(c)討論結果與你的視覺判斷是否一致，不一致的原因是什麼;(d)把這個函式存成共用模組(如 utils/metrics.py)，往後第 5、6、7、11 章的 Lab 直接引用——呼應第 2 章「不要重造輪子」。

Lab 3-4 連通性與距離轉換

目標：為第 9、10 章打底。(a)以程式 3-4 建立「對角相接的兩個方塊」，分別用 4-連通與 8-連通做連通元件標記，確認計數結果不同;(b)畫一條斜向的一像素細線，觀察 4-連通下它是否被判為斷裂;(c)對一張含多個圓形物件(可用 cv2.circle 合成，刻意讓兩個相接)的二值影像做距離轉換，把結果以第 8 章的虛擬色彩顯示;(d)找出距離轉換的局部極大值，觀察它們是否落在每個圓的中心——這正是第 9 章分水嶺分割的標記來源;(e)比較歐氏、城市街區、棋盤三種距離度量的結果差異。

Lab 3-5 把混疊做出來，再把它修好

目標：讓取樣定理從一條公式變成你親手量過的數字。(a)用 np.sin 合成週期為 16、10、8、6、4 像素的條紋影像各一張，直接跳點降取樣 4 倍，並排顯示，標出哪幾張失真;(b)對每一張計算「降取樣後的視在週期」，與圖 3-6 的公式對照，確認你算出的數字與看到的現象一致;(c)重做一次，但這次先以不同 sigma 的高斯模糊再降取樣，找出「剛好能消除假圖案」的最小 sigma，並解釋它為什麼是這個值;(d)合成一張環帶

圖（提示：亮度為 $\cos(\pi r^2/k)$ 的形式），分別以直接跳點、先模糊、cv2.INTER_AREA、cv2.INTER_NEAREST 縮小四倍，量測臨界半徑以外區域的標準差，排出優劣;(e)拿一張你自己拍的、含細密紋理的照片（格子衣服、紗窗、遠處磁磚）重做 (d)，確認結論在真實影像上也成立;(f)寫下結論：你會把哪一種縮圖方法設成專案的預設?為什麼?

Lab 3-6 品質指標的極限

目標：親手做出「指標說好、眼睛說壞」的反例，並建立整學期都會用到的評估工具。(a)寫一個函式 evaluate(ref, test)，一次回傳 MSE、PSNR、SSIM 與 SSIM 差異圖;(b)對同一張參考影像製造五種劣化（雜訊、模糊、重度 JPEG、整體加亮、抹掉一小塊），調整各自的參數，讓五者的 PSNR 落在 1 dB 之內——這一步本身就很有教育意義，你會發現要湊到一樣有多容易;(c)把五張圖與三個指標做成一張對照表，請三位同學在不看數字的情況下排序「哪張看起來最糟」，再與指標的排序比較;(d)針對「抹掉一小塊」，試著設計一個能抓出它的指標（提示：考慮局部 SSIM 的最小值、或誤差最大的前 1% 區域），驗證它確實能把這個案例排到最後;(e)以你自己的照片重做一次，確認你設計的指標沒有過度擬合到課本的範例影像;(f)把 evaluate 放進第 2 章 Lab 2-6 的工具箱。第 5、6、7、11 章的每一次「這樣有沒有比較好」都會用到它。

本章重點整理

- 影像數位化分為取樣(空間離散化，決定像素與空間解析度)與量化(亮度離散化，決定位元深度);兩者皆造成不可逆的資訊損失。
- 影像尺寸、空間解析度、顯示尺寸三者不同;像素多不等於細節多——放大既有影像只是內插猜值，並未增加資訊。
- Nyquist 取樣定理要求取樣頻率至少為最高頻率的兩倍;不足則產生混疊，表現為摩爾紋(空間)、鋸齒、車輪倒轉(時間)。防範之道是取樣前先低通濾波，故縮小影像應先模糊或用 INTER_AREA。

- 位元深度決定灰階層次(8 位元 256 階)，不足時在平滑區產生假輪廓;動態範圍描述可同時記錄的明暗比值;加入微量雜訊(抖動)反而能打散階梯。量化思想在第 11 章 JPEG 與第 17 章模型量化中重現。
- 感測器以光電效應成像，帶來 Poisson 與讀取雜訊、以及過曝的不可逆資訊喪失;拜爾陣列使每像素僅記錄一個顏色，需經去馬賽克內插;RAW 保有高位元深度與線性數值，是嚴肅影像處理的理想輸入。
- 客觀品質評估：MSE(誤差平方均值)、PSNR(對數形式，40 dB 以上極佳)、SSIM(衡量亮度、對比、結構，更貼近感知);PSNR 有逐像素比較的盲點，全域指標亦掩蓋最壞情況，須與目視並用。
- 鄰域分 4-鄰域與 8-鄰域，連通性的選擇直接影響物件計數;距離度量有歐氏、城市街區、棋盤三種，為第 10 章距離轉換與第 9 章分水嶺的基礎。

習題

1. 影像數位化的兩個步驟是什麼?各自離散化了哪一個維度?為什麼說這個過程不可逆?

   提示與參考解答：取樣：在空間上離散化，把影像切成一格格像素，決定空間解析度。量化：在亮度上離散化，把連續亮度對應到有限階層，由位元深度決定。不可逆是因為兩者都丟棄了原始的連續資訊——落在同一格內的空間細節、落在同一階層內的亮度差異，都永遠無法還原。這也是第 7 章影像還原的物理上限。

2. 請區分影像尺寸、空間解析度與顯示尺寸。為什麼說「像素多不等於細節多」?

   提示與參考解答：影像尺寸是像素數(如 1920×1080，即陣列的 shape);空間解析度是單位長度內的像素數(如 300 dpi)，才真正描述細節精細程度;顯示尺寸是實際呈現的大小，取決於顯示裝置密度。像素多不等於細節多：把模糊照片放大四倍，像素變十六倍但細節未增(內插只是猜值);感測器像素再多，若鏡頭解析力不足，多出的像素只是記錄模糊。

3. 請陳述 Nyquist 取樣定理，並用「記錄一個波形週期」的直覺解釋為什麼是兩倍。

提示與參考解答：取樣頻率必須至少是訊號中最高頻率成分的兩倍，才能完整重建原訊號。直覺解釋：要記錄一個完整的波形週期，至少需要兩個取樣點——一個抓波峰、一個抓波谷;若每週期只取到一個點，就無從得知這個波的存在，甚至會誤判成別的低頻訊號(即混疊)。

4. 混疊在空間與時間上分別表現為什麼現象?請各舉一例並說明成因相同之處。

提示與參考解答：空間混疊：摩爾紋(拍格子襯衫或螢幕時的詭異波紋)、鋸齒(斜線呈階梯狀)。時間混疊：車輪或電風扇在影片中看似靜止或倒轉。成因相同：取樣率(空間的像素密度或時間的影格率)不足以滿足取樣定理，高頻成分被「摺疊」進低頻區，偽裝成原本不存在的低頻訊號。

5. 為什麼縮小影像前應先做模糊?這與 OpenCV 的 INTER_AREA 有何關聯?

提示與參考解答：縮小即降低取樣率，依取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊(假圖案)。先低通模糊把這些高頻濾掉，就無物可摺疊。INTER_AREA 內插的原理是區域平均，本身就內建了低通動作，因此是縮小的建議選項(第 4.3 節「縮小用 AREA」的根源)。

6. 什麼是假輪廓?它為什麼最容易出現在平滑漸層區域?為什麼加入微量雜訊反而能改善?

提示與參考解答：假輪廓是位元深度不足時，在本該平滑漸變的區域出現生硬的階梯狀邊界。最容易出現在漸層區，是因為真實漸變是連續的，量化後只能用有限階層表示，相鄰階層間出現肉眼可見的跳躍;而人眼對平滑區的微小跳躍特別敏感。加入微量雜訊(抖動)能打散規則的階梯邊界，使量化誤差在視覺上呈隨機分布而非整齊的界線，反而看起來更平滑。

7. 本書中「量化」這個思想出現在哪三個地方?它們的共通核心是什麼?

提示與參考解答：一、本章：感測器把連續的光強度量化成有限灰階。二、第 11 章：JPEG 把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻。三、第 17 章：模型量化把神經網路權重從 32 位元浮點降到 8 位元整數。共通核心：在可接受的品質損失下，用有限的階層逼近連續值，換取儲存、傳輸或運算的大幅節省。

8. 感光元件本身能分辨顏色嗎?彩色照片是如何產生的?拜爾陣列為什麼綠色佔一半?

提示與參考解答：不能，感光元件只能感測光的強度。彩色是靠感測器表面的色彩濾鏡陣列(CFA)，讓每像素只接收特定顏色的光，再以去馬賽克內插補出另外兩色。拜爾陣列以「一紅兩綠一藍」重複排列，綠色佔一半是因為人眼對綠光最敏感——這與第 8 章 YCbCr 對色度降取樣、第 11 章 JPEG 的 4:2:0 是同一個設計思想：把資源分配給人眼最在意的地方。

9. RAW 與 JPEG 有哪三個關鍵差異?為什麼做嚴肅的影像處理時建議從 RAW 出發?

提示與參考解答：一、位元深度：RAW 通常 12 至 14 位元(4096 階以上),JPEG 僅 8 位元(256 階)，後製調整不易產生假輪廓。二、線性：RAW 數值正比於實際光量，JPEG 經 Gamma 編碼後非線性(第 5.1.2 節)，精確光度量測須留意。三、未經破壞：JPEG 已套用白平衡、銳化並有損壓縮，皆不可逆且反覆存檔會累積失真。因此第 7 章還原與模型訓練資料蒐集都會因 RAW 的輸入品質而受益。

10. 請說明 PSNR 的盲點，並解釋 SSIM 如何改善。為什麼說「指標是工具，不是目的」?

提示與參考解答：PSNR 逐像素比較、不考慮人類視覺特性：整張影像亮度微幅提高會使 PSNR 顯著下降但人眼幾乎無感;平坦區的少數突兀雜點對 PSNR 影響有限卻極為刺眼。SSIM 改為衡量亮度、對比度、結構三面向的相似度，更貼近感知。「指標是工具不是目的」：全域指標衡量平均表現，而許多應用關心最壞情況——一個抹掉微小病灶的去噪演算法，PSNR 可能不降反升(第 7.6.1 節 MRI 案例)，但在醫學上是災難。

11. 4-連通與 8-連通有何差異?請舉一個「同一張影像、不同連通性得到不同物件數」的例子，並說明為什麼實務上前景與背景要用互補的連通性。

提示與參考解答：4-鄰域只算上下左右，8-鄰域再加四個對角。例子：兩個僅在對角相接的白色方塊，8-連通視為一個物件、4-連通視為兩個;一條斜向細線在 4-連通下會被判為斷裂的孤立點。前景與背景用互補連通性(如前景 8、背景 4)是為了避免連通性悖論——若兩者同用一種，會出現「封閉曲線卻沒把背景分成內外」這類矛盾。

12. (綜合題)承 3.7.3 節：要檢測工件上寬度 0.2 毫米的裂紋，視野寬度為 100 毫米，請推算所需的相機水平像素數，並說明你的假設與安全餘裕。

提示與參考解答：依取樣定理，偵測 0.2 毫米特徵至少需每 0.1 毫米一像素(兩倍取樣);但實務上為穩健，常要求三到五像素涵蓋最小特徵，取四像素則每像素約 0.05 毫米。所需水平像素數 = 100 ÷ 0.05 = 2000 像素。加上安全餘裕(考慮鏡頭邊緣解析力下降、對焦誤差、工件擺放偏移)，建議選 2500 至 3000 像素以上的相機，或縮小視野分次拍攝。關鍵在於說明假設(每特徵幾個像素)與餘裕的理由，而非單一數字。

13. 計算題：承圖 3-6。一張影像中有週期分別為 16、10、8、6、4 像素的條紋，若直接跳點降取樣 4 倍，哪些會失真?請算出分界線，並說明週期恰為 8 的那一組為什麼也靠不住。

    提示與參考解答：降取樣 4 倍後，新的取樣間距是 4 像素，依取樣定理能可靠解析的最短週期為 2 × 4 = 8 像素，因此週期 6 與 4 必然失真。週期 16 與 10 的新週期分別為 4 與 2.5 像素，高於臨界值，條紋保住。週期恰為 8 的情形最微妙：它的新週期正好是 2 像素，剛好踩在 Nyquist 頻率上。此時結果完全取決於取樣的相位——本例中每個取樣點都落在同一個相位（正弦的零點），於是整片變成均勻的灰色，條紋完全消失。這正是定理要求「高於兩倍」而非「至少兩倍」的原因：等於兩倍時，理論上資訊仍在，實務上卻可能完全取不到。

14. 計算題：承圖 3-8。環帶圖的瞬時徑向頻率為 r / 900 週期 / 像素。若降取樣 4 倍，從哪個半徑開始會出現混疊?若改為降取樣 2 倍呢?請說明這個半徑與「假環紋出現在外圈」的關係。

    提示與參考解答：降取樣 D 倍後的 Nyquist 頻率為 1 / (2D)。令 r / 900 = 1 / (2D) 得臨界半徑 r = 900 / (2D)。D = 4 時 r = 112.5 像素;D = 2 時 r = 225 像素。因為環帶圖的頻率由中心向外遞增，所以超過臨界半徑的區域才會混疊——這就是假環紋只出現在外圈的原因，中央完全不受影響。降取樣倍率愈小，臨界半徑愈往外推，受影響的範圍愈小。這張圖之所以是檢驗縮圖演算法的標準素材，正是因為「哪裡開始出錯」可以直接換算成「該演算法的有效截止頻率是多少」。

15. 計算題：承圖 3-7 的車輪。一個 8 輻的輪子以每秒 23 圈順時針旋轉，用 24 fps 拍攝。請計算每一影格之間輪子轉過的角度、以及「看起來」轉了多少，並說明為什麼看起來會倒轉。若輪子改為每秒 24 圈呢?

    提示與參考解答：每格之間輪子轉過 23 × 360 / 24 = 345 度。但 8 輻的輪子每轉過

360 / 8 = 45 度就與原本的樣子完全一樣，因此真正「看得出來」的只有 345 mod 45 = 30 度。又因為 30 度大於 45 / 2 = 22.5 度，大腦會選擇較短的那條路徑來解釋，也就是「往回轉 15 度」——於是看起來以每秒 15 × 24 / 360 = 1 圈的速度反向旋轉。若改為每秒 24 圈：每格轉過 360 度，360 mod 45 = 0，輪子看起來完全靜止。這與圖 3-6 中「週期恰為 8」的條紋消失是同一個數學現象——取樣頻率與訊號頻率成整數倍關係時，你什麼都取不到。

16. 計算題：承圖 3-9 與 3-10。一張 8 位元影像中有一塊天空，其灰階標準差為 4.6。若量化到 4 位元（量化步長 16）與 6 位元（步長 4），分別會發生什麼？請以「標準差與量化步長的比值」說明判準，並據此推算這塊天空至少需要幾位元才不會出現假輪廓。

    提示與參考解答：判準是把區域的灰階起伏與量化步長相比。4 位元時步長 16，遠大於起伏 4.6，整塊天空只會落在一兩個階上，階與階的交界形成生硬的假輪廓。6 位元時步長 4，小於起伏 4.6，漸變能被多個階層描述，假輪廓不明顯。一個實用的經驗法則是：量化步長應小於該區域的灰階標準差，最好小於其一半。要讓步長 ≤ 4.6 / 2 ≈ 2.3，需 256 / 2.3 ≈ 111 階，即至少 7 位元（128 階）。這解釋了為什麼天空、膚色漸層、柔和陰影是測試位元深度的最佳素材，也解釋了為什麼 HDR 影像需要 10 位元以上。

17. 承圖 3-11：加了雜訊之後的均方根誤差反而變大，為什麼我們還說「抖動讓它看起來更好」？這與 3.5 節的哪一個論點是同一回事？請再舉一個「指標變差但觀感變好」的例子。

    提示與參考解答：因為均方根誤差衡量的是逐像素的數值偏差，而人眼判斷平滑度靠的是「有沒有看得見的結構」。抖動把規則的階梯邊界打散成隨機顆粒：數值上偏差更大，但隨機顆粒在視覺上不構成結構，大腦會把它平均掉，規則的階梯卻會被視覺系統強化（馬赫帶效應）。這與 3.5 節的論點完全同源：指標衡量的面向不一定是你在乎的面向。另一個例子是適度的銳化（第 5 章）：它會讓 PSNR 下降（因為加入了原本沒有的過衝），卻普遍被認為看起來更清晰。底線是：指標用來比較同一類方法的優劣，不能用來跨類別排名，更不能取代目視確認。

18. 承圖 3-13：拜爾陣列中綠色佔一半，為什麼？去馬賽克在什麼樣的影像內容上最容易出錯？這種錯誤與 3.2 節的混疊有什麼關係？

提示與參考解答：綠色佔一半有兩個理由：其一，人眼的視錐細胞對綠光（約 555 奈米）最敏感，綠通道對亮度感知的貢獻最大（第 8 章的亮度公式中綠的權重約 0.59）;其二，多一倍的綠色取樣點等於提高了亮度資訊的空間取樣率，而人眼對亮度細節遠比對色彩細節敏感。去馬賽克在「細密的高頻紋理」上最容易出錯，例如細格子襯衫、屋頂瓦片、遠處的欄杆，會產生原本不存在的彩色條紋（偽色）與拉鍊狀邊緣。關係很直接：每個顏色的實際取樣率只有影像解析度的二分之一到四分之一，當紋理頻率超過該色通道的 Nyquist 頻率時就會混疊——只是這次摺疊出來的不是假的亮度條紋，而是假的顏色。這也是為什麼高階相機會在感測器前加光學低通濾鏡。

19. 承圖 3-14：為什麼同樣提亮三倍，12 位元 RAW 的結果仍有兩百多個灰階，8 位元 JPEG 卻只剩七十幾個?請用數字說明，並解釋為什麼「先存成 JPEG 再後製」與「存 RAW 後製」在拍攝的那一刻看起來沒有差別。

    提示與參考解答：關鍵在量化發生的時機。12 位元有 4096 階，拍得過暗時訊號只佔其中低位的一小段，但那一段裡仍有數百個可用的階;提亮等於把這一段拉伸到整個 0 至 255，階層數足夠，拉伸後仍然平滑。8 位元只有 256 階，過暗的畫面可能只佔其中二三十階，提亮三倍等於把這二三十階拉開成橫跨整個範圍，相鄰階層之間出現空隙——這就是圖 3-14 右側直方圖呈梳齒狀的原因。至於拍攝當下看不出差別，是因為相機顯示的本來就是處理過的 8 位元預覽圖，兩者在那個階段確實一模一樣;差別要到你動手調整時才顯現。這也是 3.1.1 節「不可逆」最具體的一個例子。

20. 計算題：承 3.7.1 節。要掃描一份報紙，其最細的特徵是 6 點大小的鉛字，筆畫寬度約 0.1 毫米。(a) 依取樣定理，最低需要多少 dpi?(b) 若要能辨識而非只是偵測，實務上建議多少?(c) 若改設為 600 dpi，檔案會大幅增加，這些多出來的像素一定是浪費嗎?

    提示與參考解答：(a) 1 英寸 = 25.4 毫米。偵測 0.1 毫米的筆畫至少需每 0.05 毫米一個像素，即每英寸 25.4 / 0.05 ≈ 508 dpi——也就是說連「理論下限」都已經超過 300 dpi。(b) 實務上要三到五個像素涵蓋最小特徵，取四像素則每 0.025 毫米一像素，約 1016 dpi;若目標只是文字辨識（OCR），600 dpi 通常已足夠，因為 OCR 靠的是筆畫的整體形狀而非單一筆畫的精確寬度。(c) 不一定是浪費。多出來的像素在三種情況下有價值：要做 OCR 時更高的解析度能提升辨識率;原稿有網點（半色

調）時需要足夠解析度才能用第 6 章的陷波濾波移除;以及典藏用途——原稿會劣化，但掃描檔不會，一次掃到位比日後重掃便宜得多。真正的浪費是「原稿本身的細節有限卻硬提高解析度」，例如掃描一張已經模糊的老照片。

21. 承 3.7.2 節：醫學影像為什麼需要 12 至 16 位元?既然螢幕只能顯示 256 階、人眼也分辨不了那麼多，多出來的位元有什麼用?請說明「視窗化」的作用，並與第 5 章的對比強化比較。

提示與參考解答：多出來的位元用在「儲存」而非「顯示」。人體組織之間的密度差異可能極小——早期腫瘤與正常組織的 CT 值可能只差幾個 Hounsfield 單位，若一開始就量化成 256 階，這個差異會被量化步長吃掉，資訊在存檔的那一刻就消失了（3.1.1 節的不可逆）。保留 12 至 16 位元，等於把「要看哪一段」的決定權留到判讀時。視窗化就是行使這個決定權：醫師設定視窗中心與視窗寬度，把 12 位元資料中關心的那一段（例如肺窗、骨窗、腦窗）線性映射到螢幕的 256 階，其餘部分壓成全黑或全白。與第 5 章對比強化的關係是：兩者在數學上都是灰階轉換，但起點不同——視窗化是從高位元深度的原始資料中「選一段來看」，資訊一直都在;而對一張已經是 8 位元的影像做對比拉伸，是在有限的階層上硬拉，必然伴隨假輪廓。這個對比值得記住：有沒有保住位元深度，決定了你是在「選擇」還是在「硬拉」。

22. 承圖 3-15 與 3-16：五種劣化的 PSNR 幾乎相同，但「整體加亮」與「抹掉一小塊」連 SSIM 的排名也不符合人的判斷。請說明為什麼，並提出至少兩種補救的評估做法。

提示與參考解答：原因在於兩個指標都是「全域平均」。整體加亮改動了 100% 的像素但每個像素只差一點點;抹塊只改動不到 2% 的像素但那一塊被完全破壞。平均下來兩者的誤差總量相近，所以 PSNR 相近;SSIM 雖然考慮了結構，但它取的是所有局部視窗的平均，98% 完好的區域把那 2% 的災難稀釋掉了，於是反而給了抹塊更高的分數。補救做法至少有三種：其一，報告局部指標的最壞值而非平均值，例如 SSIM 差異圖的最小值、或誤差最大的前 1% 區域的平均誤差;其二，使用任務導向的指標——若目的是偵測瑕疵，就直接量偵出率與誤報率（第 16 章的 mAP 正是這種思路）;其三，永遠保留目視檢查，並把 SSIM 差異圖（圖 3-17）當成除錯工具，它會直接指出「哪裡不像」。一句話總結：指標是用來篩選候選方案的，不是用來下最終結論的。

23. 承圖 3-18：為什麼實務上前景與背景要採用互補的連通性?若兩者都用 8-連通，會出現什麼矛盾?請以「一條斜線把區域分成兩半」為例說明。

    提示與參考解答：考慮一條由左上到右下、只在對角相接的斜線，把一個白色區域分成左下與右上兩塊背景。若前景（線）與背景都用 8-連通：線本身是連通的（對角相接算相鄰），所以它「是一條完整的線」;但左下與右上的背景也因為對角相接而算連通，於是這條線「並沒有把區域分開」——一條完整的線卻沒有分割作用，這在拓撲上是矛盾的。反之若兩者都用 4-連通：線本身是斷開的一群孤立點，但背景被它分成了兩塊——一條斷掉的線卻有分割作用，同樣矛盾。採用互補的連通性（前景 8、背景 4，或前景 4、背景 8）可以消除矛盾，讓「連通」與「分離」這兩個概念自洽。這就是為什麼 OpenCV 的 connectedComponents 要明確指定 connectivity，而第 10 章的形態學運算也必須留意結構元素的形狀。

24. 承圖 3-19：同一個形狀用三種距離度量做距離轉換，最大值分別為 41.5、43.0、31.0。請解釋為什麼歐氏距離的值介於另外兩者之間，並說明在什麼情況下該選哪一種。

    提示與參考解答：三者的大小關係是固定的：D8 ≤ D2 ≤ D4。直覺上，棋盤距離（D8）允許斜走，一步可以同時前進一格橫向與一格縱向，所以步數最少、數值最小;城市街區距離（D4）只能上下左右走，斜向移動要拆成兩步，數值最大;歐氏距離是真正的直線距離，自然介於兩者之間。以一個 (3, 4) 的位移為例：D4 = 7、D2 = 5、D8 = 4，恰好印證這個順序。選用時機：要做實際的長度量測（例如量瑕疵的尺寸、第 10 章求內切圓半徑）必須用歐氏距離，因為只有它與真實世界的距離成正比;若目的只是快速估算或做形態學的膨脹腐蝕，D4 與 D8 的計算成本低得多（只需整數加法與比較），且 D8 恰好對應方形結構元素、D4 對應十字形結構元素，在實作上更自然。

25. (綜合設計題)某產線要以相機檢測印刷電路板上的焊點，最小焊點直徑 0.3 毫米，板子尺寸 200×150 毫米，產線每 2 秒通過一片，焊點的良與不良差別在於亮度分布而非形狀。請提出完整的取像規格（解析度、位元深度、曝光時間），每一項都要說明推算依據，並指出你最不確定的假設是什麼、要如何驗證。

    提示與參考解答：本題無標準答案，評分重點在推算鏈的完整性與自我檢驗。一組

合理的推算——解析度：焊點 0.3 毫米，取 5 像素涵蓋（因為要判斷亮度分布而非只偵測存在，需要較多取樣點），即每像素 0.06 毫米;水平需 200 / 0.06 ≈ 3333 像素、垂直 150 / 0.06 = 2500 像素，故選 4096×3000 左右的相機，或以兩台相機分區拍攝。位元深度：既然判準是亮度分布的細微差異，8 位元可能不足，建議 10 至 12 位元，並依習題 16 的判準實測——取一批良品與不良品，量測兩者亮度差的標準差，確認量化步長小於該差異的一半。曝光時間：每 2 秒一片，若板子是連續移動的，需由移動速度與可容忍的運動模糊（建議小於半個像素，即 0.03 毫米）反推曝光上限（第 7.6.3 節的做法）;若可停止取像則限制大幅放寬，但要確認機構的穩定時間。最不確定的假設通常有三個：「5 個像素夠不夠」（應以實拍的良品與不良品做 ROC 曲線驗證）、「打光是否穩定且均勻」（應量測同一片板子連拍數十張的亮度變異，若變異大於良與不良的差異，再高的解析度也沒用）、以及「板子是否會翹曲造成失焦」（應量測景深並實測邊角）。這題的核心訊息是：規格要從需求推算，但推算建立在假設上，而工程的專業在於知道自己假設了什麼、以及如何驗證它。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2(數位影像基礎)。
- NIST, First Digital Image — Russell Kirsch, 1957:https://www.nist.gov/mathematics-statistics/first-digital-image(本章叫獸開講之史料來源)
- Wang, Z., Bovik, A. C., et al., Image Quality Assessment: From Error Visibility to Structural Similarity, IEEE TIP, 2004.(SSIM 原始論文)
- Shannon, C. E., Communication in the Presence of Noise, Proc. IRE, 1949.(取樣定理)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2(影像形成與感測器)。
- scikit-image 官方文件(SSIM 等品質指標):https://scikit-image.org

- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 4 章 幾何轉換

本章學習目標

- 以矩陣表示平移、旋轉、縮放等基本幾何轉換，並說明齊次座標存在的理由。
- 區分剛體、相似、仿射與透視轉換的自由度、不變性質與適用場景。
- 解釋反向映射為何需要內插，比較四種內插法的品質、速度與適用時機。
- 描述針孔相機模型的內部參數與外部參數，辨識桶狀與枕狀鏡頭畸變。
- 以張氏標定法完成相機校正與去畸變，並以重投影誤差驗收成果。
- 運用透視轉換完成文件拉正與機器人導航用的鳥瞰圖。
- 理解幾何轉換在資料增強、視訊穩定、影像拼接與機器人座標系中的角色。

叫獸開講|張正友與一張棋盤格：讓相機校正走進每個人口袋

要讓相機從「拍照的機器」升級成「測量的儀器」，第一步是校正(Calibration)：精確求出它的焦距、光心位置與鏡頭畸變參數。在 1990 年代，這是一件昂貴的苦差事。經典方法需要精密加工的三維標定物——例如兩三個互相垂直、貼滿精確標記點的金屬平面，加工精度直接決定校正精度，一般實驗室根本玩不起。另一派「自我校正」方法不需標定物，數學卻極其艱深且不穩定。相機校正，就這樣卡在「太貴」與「太難」之間。

1998 年 12 月 2 日，微軟研究院的張正友(Zhengyou Zhang)發表了技術報告 MSR-TR-98-71《A Flexible New Technique for Camera Calibration》，給出了第三條路。他的方法簡單到令人難以置信：只需要把一張棋盤格圖案印在紙上、

貼到一塊平板上，讓相機從至少兩個不同方位觀察它就行了——而且相機動或棋盤動都可以，更關鍵的是，你完全不需要知道它是怎麼動的。

技術上，這個方法先利用「平面與其影像之間的單應性」導出對內部參數的約束，以封閉解求出初始估計，再以最大概似準則做非線性最佳化，同時把徑向畸變一併建模。論文於 1999 年在頂級會議 ICCV 發表，並於 2000 年正式刊登在頂級期刊 IEEE TPAMI 上。張正友在論文中寫下的一句話，精準地概括了它的意義：相較於需要兩三個正交平面等昂貴設備的傳統方法，這個技術簡單而靈活，讓三維電腦視覺從實驗室環境往真實世界前進了一步。

「一張紙取代精密機械」的威力是巨大的。任何人花五分鐘列印一張棋盤格，就能完成過去要專業設備才能做的事。這個方法後來成為 OpenCV 內建的標準校正流程(cv2.calibrateCamera 的核心正是張氏標定法)，也是全球手機、無人機、機器人出廠校正管線的基礎，論文引用數以萬計，是電腦視覺史上影響力最大的工作之一。張正友後來持續在工業界深耕電腦視覺與機器人研究——而你今天在 Lab 4-2 印出的那張棋盤格，正是這段歷史的延續。

叫獸想從這裡點出一個貫穿全書的觀察。回想第 2 章 OpenCV 的開源、第 13 章 ORB 對 SIFT 專利的回應、第 17 章二十五美元的樹莓派——這些改變領域的突破，往往不是「做出更強大的東西」，而是「大幅降低了做這件事的門檻」。把一項只有少數人玩得起的技術，變成每個人都能用的工具，其影響力常常遠超過在精度上多推進幾個百分點。這是工程創新中最容易被低估、卻最有力量的一種。

思考題：張氏標定法要求棋盤格「至少兩個不同方位」，但實務上大家都拍 10 到 20 張。想一想：多拍的那些照片，是在對抗什麼?(提示：雜訊、角點定位誤差，以及涵蓋畫面各個角落的畸變。張正友原始論文的實驗也顯示，估計結果的不確定度會隨影像張數增加而下降)

4.1 基本幾何轉換與齊次座標

4.1.1 幾何轉換改變什麼

先釐清一件事：幾何轉換改變的是像素的「位置」，而非像素的「數值」。第 5 章的灰階轉換是「位置不變、數值改變」，本章的幾何轉換則恰好相反——把影像中的內容搬到別的地方去，而每個點的亮度或顏色原則上跟著它一起搬。這個對照有助於你建立整體樣貌：影像處理的操作，不是在動數值，就是在動位置，或兩者兼有。

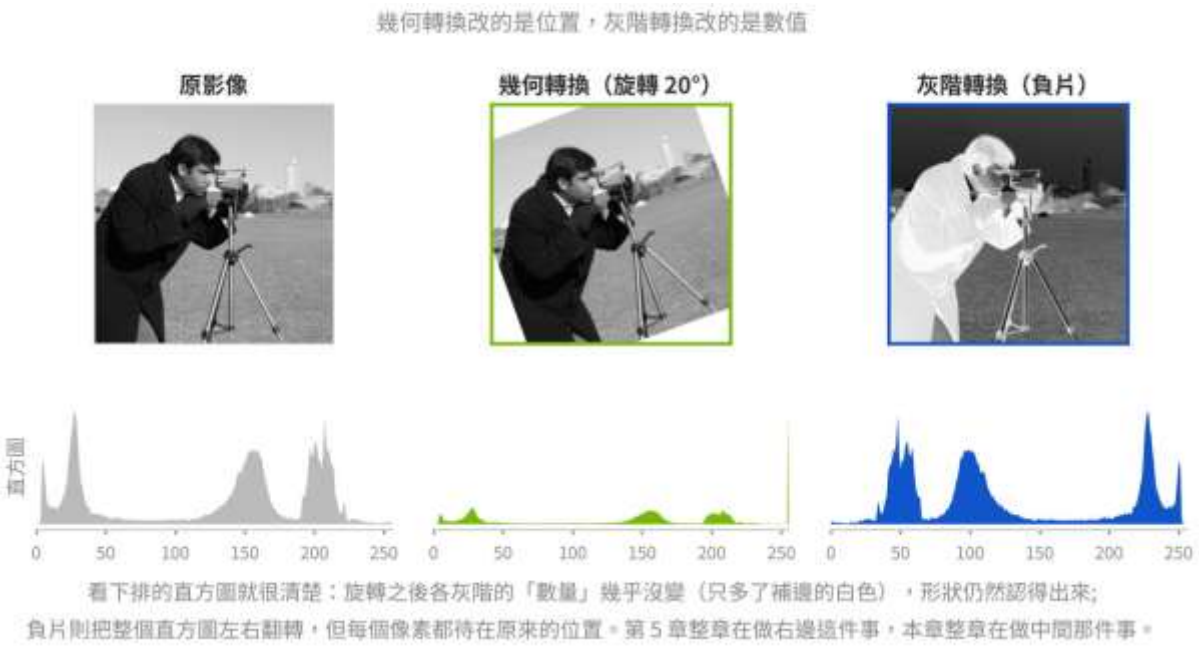


圖 4-1　幾何轉換改位置，灰階轉換改數值

4.1.2 三種基本轉換

三種最基本的轉換是：平移把每個點 (x, y) 移到 (x + tx, y + ty);縮放把座標乘上倍率，得到 (sx·x, sy·y);旋轉將點繞原點轉 θ 角，新座標為 x' = x·cosθ - y·sinθ,y' = x·sinθ + y·cosθ。

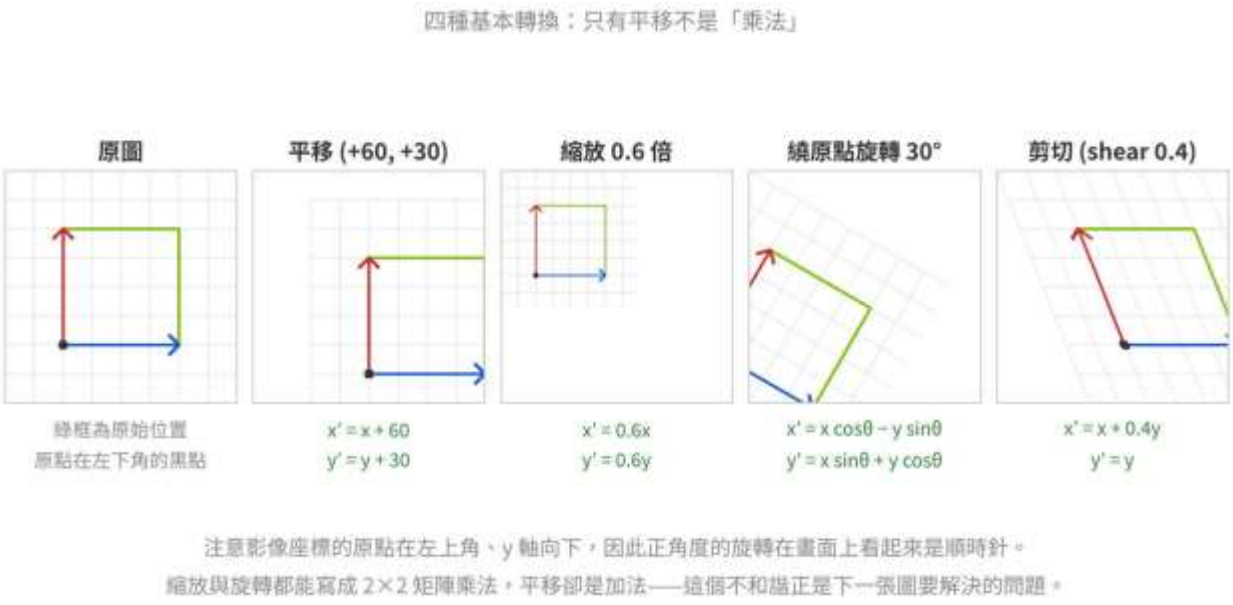


圖 4-2　四種基本轉換：只有平移不是乘法

縮放與旋轉都能寫成 2×2 矩陣乘法，唯獨平移是「加法」，格格不入。這個小小的不和諧看似無關痛癢，實際上很麻煩：當你要串接多個轉換時，乘法與加法混在一起，無法合成為單一運算。這正是齊次座標登場的理由。

4.1.3 齊次座標：讓所有轉換說同一種語言

齊次座標的做法很簡單：替每個點多加一維，把 (x, y) 寫成 (x, y, 1)。如此一來，平移也能表示成 3×3 矩陣乘法，三種轉換全部統一成「矩陣乘以向量」的形式。



圖 4-3　齊次座標：多加一維，把平移也變成乘法

統一的好處立刻顯現。第一，多個轉換的串接，等於把矩陣依序相乘成一個矩陣——「先旋轉 30 度、再放大 1.5 倍、再平移」三個動作，可以合成一個 3×3 矩陣一次做完。第二，這不只是寫法漂亮：一次完成比分三次做更快，而且更重要的是避免了多次取樣造成的品質劣化(每做一次轉換就要內插一次，而每次內插都會損失一點細節，第 4.3 節詳述)。

請特別留意矩陣相乘的順序：轉換是從右往左作用的。若寫成 M = T · S · R，代表先旋轉、再縮放、最後平移。順序不可交換——先旋轉再平移，與先平移再旋轉，結果完全不同(想像繞原點旋轉的圓心位置)。這是初學者最常見的錯誤之一。

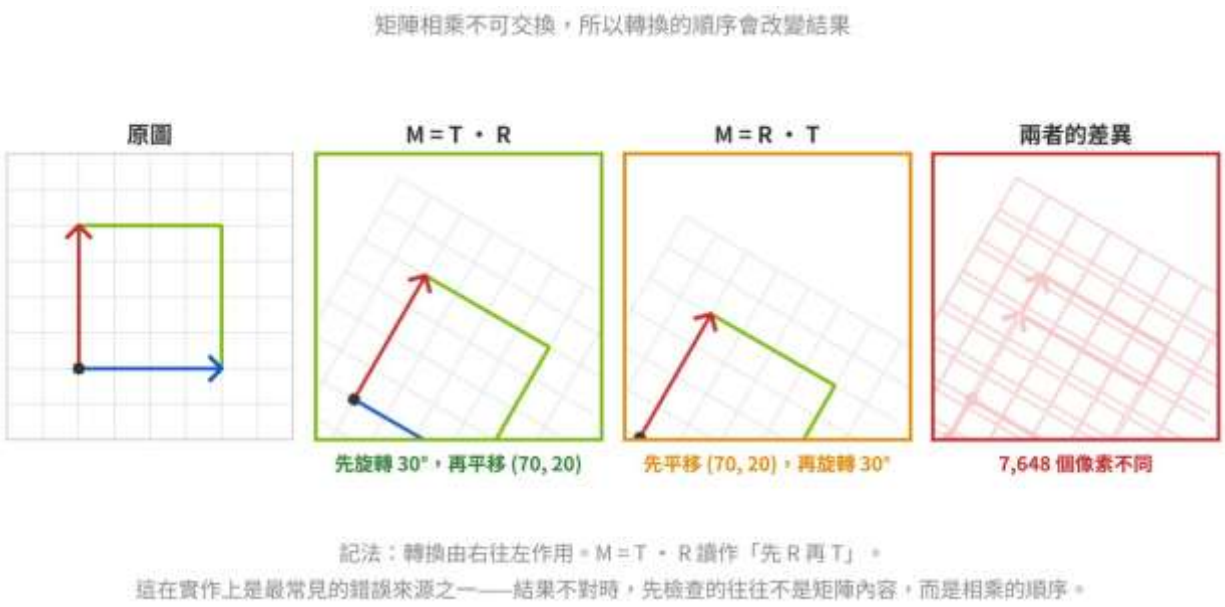


圖 4-4　轉換順序不可交換

「升一維換取統一性」這個技巧，其影響遠超出本章。第 4.4 節的相機模型、第 17 章 ROS 2 的座標轉換系統(tf2)，背後都是齊次座標。一台機器人身上有相機座標系、機械臂座標系、底盤座標系、世界座標系，而 tf2 就是以齊次座標矩陣管理這些座標系之間的關係——當你在第 17 章看到 tf2 時，請記得它的根就在這裡。



圖 4-5　分三次做與合成一次做的品質差距

程式 4-1 平移、旋轉、縮放與串接

```
import cv2
import numpy as np

img = cv2.imread("robot.jpg")
h, w = img.shape[:2]

# --- 平移:x 方向 +100,y 方向 +50 ---
M_t = np.float32([[1, 0, 100],
                  [0, 1, 50]])          # 2x3 仿射矩陣
shifted = cv2.warpAffine(img, M_t, (w, h))

# --- 旋轉:繞影像中心轉 30 度,順便縮放 0.8 倍 ---
M_r = cv2.getRotationMatrix2D(center=(w/2, h/2), angle=30, scale=0.8)
rotated = cv2.warpAffine(img, M_r, (w, h))

# --- 縮放:resize 是最常用的幾何轉換 ---
half = cv2.resize(img, None, fx=0.5, fy=0.5,
                  interpolation=cv2.INTER_AREA)   # 縮小用 AREA(第 3 章)

# --- 串接:合成單一矩陣,只取樣一次 ---
R = np.vstack([M_r, [0, 0, 1]])          # 補成 3x3 齊次矩陣
T = np.float32([[1, 0, 100], [0, 1, 50], [0, 0, 1]])
M = T @ R                                 # 先旋轉、再平移(從右往左)
combined = cv2.warpAffine(img, M[:2], (w, h))
# 比較:分兩次做 vs. 合成一次做,後者更快且細節損失更少
```

4.1.4 推導：旋轉矩陣從哪裡來，以及繞任意點旋轉

旋轉矩陣的四個元素常被當成公式背起來，但它其實可以在三十秒內推出來，而

且推完之後「繞任意點旋轉」也就順便解決了。

先推旋轉矩陣。任何線性轉換，都完全由「它把兩個基底向量送到哪裡」決定——因為任意向量都是基底的線性組合，而線性轉換保持線性組合。所以只要問兩個問題：把 x 軸的單位向量 (1, 0) 繞原點轉 θ 角，它會到哪裡?答案直接由三角函數定義：(cos θ, sin θ)。把 y 軸的單位向量 (0, 1) 轉 θ 角呢?它會到 (−sin θ, cos θ)。而矩陣的第一行就是「x 軸被送到哪」、第二行就是「y 軸被送到哪」，於是旋轉矩陣必然是 [[cos θ, −sin θ], [sin θ, cos θ]]。這個「看基底去了哪裡」的技巧對所有線性轉換都適用，縮放、剪切都能這樣一眼寫出矩陣。

提醒一個影像特有的陷阱：上面的推導假設 y 軸向上，但影像座標的 y 軸向下（第 2.2.1 節）。因此同一個矩陣作用在影像上時，正的 θ 在畫面上看起來是順時針旋轉。這不是矩陣錯了，是座標系的方向不同——圖 4-2 標示的就是影像座標下的實際效果。

接著是繞任意點旋轉。旋轉矩陣是繞原點轉的，而影像的原點在左上角，所以直接套用會讓影像繞著左上角甩出去。要繞中心 (cx, cy) 轉，做法是三步：先把中心平移到原點、在原點做旋轉、再平移回去。寫成矩陣就是 M = T(cx, cy) · R(θ) · T(−cx, −cy)，記得由右往左讀。

這個「搬過去、做事、搬回來」的三明治結構，在數學上稱為共軛（conjugation），是幾何轉換中最常用的一招。繞任意點縮放、沿任意一條直線鏡射、繞任意軸旋轉（三維），用的都是同一個模式：先用一個轉換把問題搬到「簡單的位置」，在那裡做事，再用反轉換搬回來。OpenCV 的 cv2.getRotationMatrix2D(center, angle, scale) 內部做的正是這件事，它回傳的是合成後的 2×3 矩陣——這也解釋了為什麼它需要 center 這個參數。

最後補一個在實作上救命的檢查法。當結果不如預期時，不要盯著影像猜，直接拿幾個已知的點代進矩陣算一遍：影像的四個角分別被送到哪裡?中心點有沒有留在原

處?三十秒的手算往往比三十分鐘的目視除錯更有效，而且它會明確告訴你問題出在矩陣內容還是相乘順序。

4.2 轉換家族：從剛體到透視

幾何轉換構成一個層層包含的家族，自由度愈高、能表達的變形愈多、但保持的性質愈少。理解這個家族譜，你才知道面對一個問題該選哪一種。

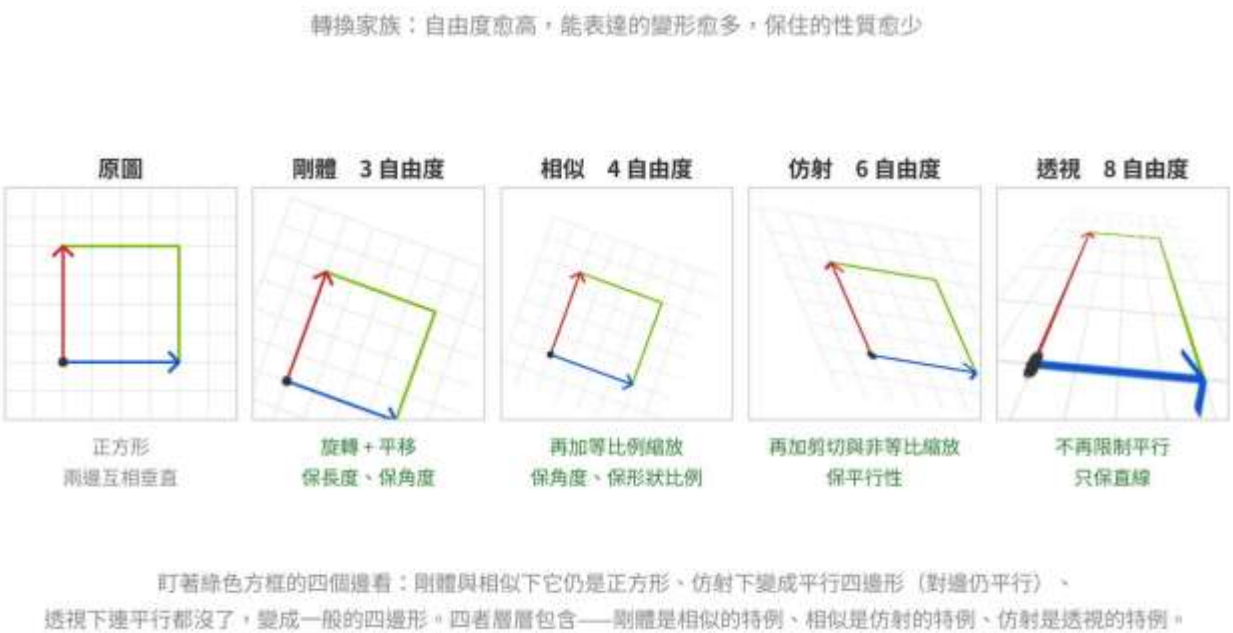


圖 4-6　轉換家族：自由度愈高，保住的性質愈少

1. 剛體轉換(Rigid / Euclidean)：只有旋轉與平移，3 個自由度。保持長度與角度——物體不變形，只是換了位置與朝向。機器人的移動、物件的擺放姿態屬於此類。
2. 相似轉換(Similarity)：剛體再加上等比例縮放，4 個自由度。保持角度與形狀比例，但大小可變。照片的等比例縮放、無人機拉高後看到的地面屬於此類。
3. 仿射轉換(Affine)：再加上剪切與非等比例縮放，2×3 矩陣、6 個自由度、需 3 組對應點。保持平行性——轉換前平行的線，轉換後仍平行(長度與角度可變)。

4. 透視轉換(Perspective / Homography):3×3 矩陣、8 個自由度、需 4 組對應點。不保平行但保直線——平行線可以相交(這正是近大遠小的視覺效果)，而直線轉換後仍是直線。

4.2.1 仿射轉換：平行線的守護者

仿射轉換是平移、旋轉、縮放、剪切的任意組合。因為 6 個自由度對應 3 個點的對應關係，cv2.getAffineTransform() 只需要「轉換前後各三個點」就能求出矩陣。



圖 4-7　自由度決定需要幾組對應點

典型應用有二。其一是影像對齊：把兩張有輕微位移旋轉的影像疊合(第 12 章的視訊穩定即屬此類)。其二是資料增強——第 14、15、16 章訓練神經網路時的隨機旋轉、平移、翻轉、剪切，本質上全是仿射轉換。這個連結值得記住：當你在第 15 章看到 RandomRotation 這類增強參數時，它呼叫的就是本章的技術。

4.2.2 透視轉換：拍歪的世界拉回正面

用相機斜拍一張 A4 紙，紙的影像不再是矩形而是梯形——原本平行的紙邊，在影像中相交了。要描述這種「近大遠小」的投影，就需要透視轉換，又稱單應性(Homography)。它需要 4 組對應點，由 cv2.getPerspectiveTransform() 求解。



圖 4-8　透視轉換：把拍歪的文件拉回正面

透視轉換是本章最實用的工具，它的應用橫跨全書：文件掃描 App 的「拉正」功能、第 13 章車牌影像的正面化、機器人導航的鳥瞰圖、第 13 章影像拼接求兩張照片間的關係、以及第 13.4.3 節 RANSAC 求出的正是這個矩陣。

一個重要的觀念補充：單應性成立有其前提——要嘛拍攝的是一個平面(如紙張、地面、牆面)，要嘛相機只做純旋轉而不平移。若場景有明顯的深度變化而相機又平移了，單一的單應性就無法描述兩張影像的關係(這也是為什麼手持平移拍攝的照片較難拼接，而原地旋轉拍攝的全景圖很容易)。理解這個限制，能幫你判斷什麼時候該用透視轉換、什麼時候需要更複雜的三維方法。

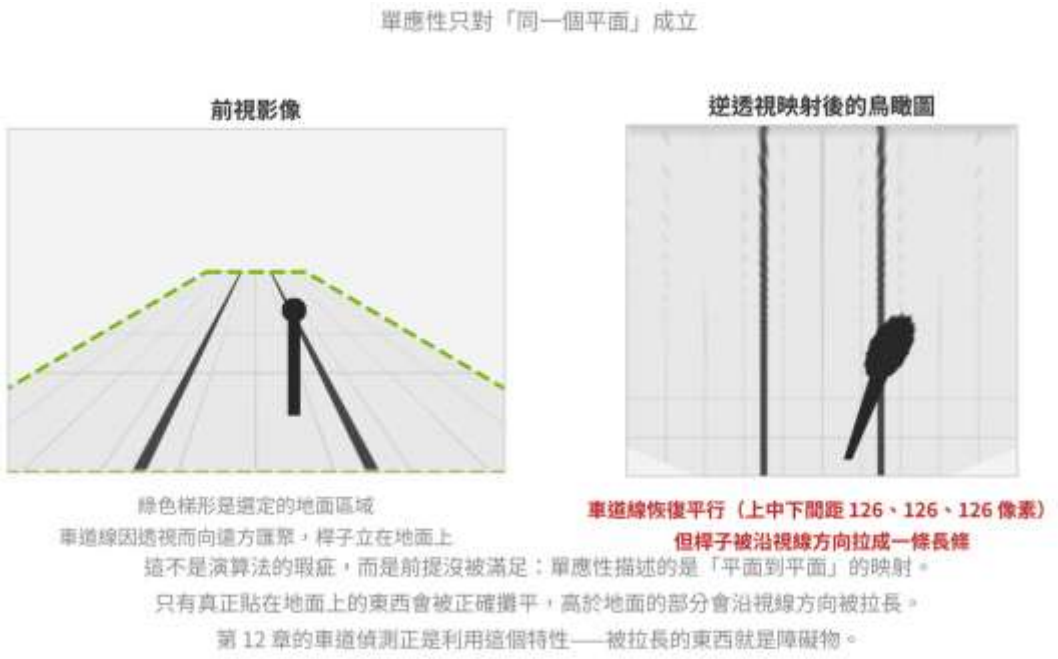


圖 4-9　單應性只對同一個平面成立

轉換	自由度	對應點數	保持的性質	典型應用
剛體	3	2	長度、角度	物件姿態、機器人位姿
相似	4	2	角度、形狀比例	等比縮放、尺度變化
仿射	6	3	平行性、面積比	影像對齊、資料增強
透視	8	4	直線性、交比	文件拉正、鳥瞰圖、拼接

表 4-1 幾何轉換家族比較

程式 4-2 透視轉換：文件拉正

```
import cv2
import numpy as np

img = cv2.imread("tilted_doc.jpg")

# 斜拍文件的四個角(左上、右上、右下、左下)
src = np.float32([[168, 92], [583, 128], [612, 707], [95, 664]])

# 自動計算合理的長寬比,避免文件被不自然拉伸
def dist(a, b): return np.linalg.norm(a - b)
W = int((dist(src[0], src[1]) + dist(src[3], src[2])) / 2)
H = int((dist(src[0], src[3]) + dist(src[1], src[2])) / 2)

dst = np.float32([[0, 0], [W, 0], [W, H], [0, H]])
```

```
M = cv2.getPerspectiveTransform(src, dst)   # 3x3 透視矩陣
flat = cv2.warpPerspective(img, M, (W, H))
cv2.imwrite("document_flat.jpg", flat)      # 文件掃描 App 的核心

# 反向:也可把矩陣反轉,把正面圖貼回原始視角(擴增實境的原理)
M_inv = np.linalg.inv(M)
```

4.2.3 推導：單應性為什麼是八個自由度，四個點又是怎麼解出來的

4.2 節說透視轉換是「3×3 矩陣、8 個自由度、需 4 組對應點」。這三個數字彼此相關，而且都能推出來。

先看自由度。3×3 矩陣有九個元素，看起來是九個自由度，但在齊次座標下，把整個矩陣乘上任一個非零常數 c，轉換後的點 (cx′, cy′, cw′) 與 (x′, y′, w′) 在除以最後一維之後會得到完全相同的結果。換句話說，整體尺度是看不見的，九個元素中只有八個是真正獨立的。實作上通常把 h33 固定為 1 來消去這個多餘的自由度，或要求矩陣的範數為 1。

再看為什麼是四個點。每一組對應點 (x, y) → (x′, y′) 提供兩個方程式。把齊次座標的轉換展開：x′ = (h11·x + h12·y + h13) / (h31·x + h32·y + h33)，y′ = (h21·x + h22·y + h23) / (h31·x + h32·y + h33)。注意分母——這個除法正是透視與仿射的根本差異，也是「近大遠小」的數學來源：分母隨位置而變，所以遠處的東西被壓得比較小。把分母乘過去，兩式就變成對 h 的線性方程。八個未知數需要八個方程式，而每組點給兩個，故需 4 組——而且這四個點不能有任何三點共線，否則方程式之間會線性相依，解不出來。實作上這代表：選文件的四個角時要選真正的四個角，不要選同一條邊上的四個點。

解法的標準流程稱為直接線性轉換（DLT）：把八個方程式整理成 A·h = 0 的形式，其中 h 是把九個元素排成的向量、A 是 8×9 的矩陣，再以奇異值分解（SVD）求 A 的最小奇異值對應的奇異向量。這個解在最小平方的意義下最接近零空間，正好對應「尺度不計」的性質。OpenCV 的 cv2.getPerspectiveTransform 做的就是四點的精確解，cv2.findHomography 則接受更多點並以最小平方或 RANSAC 求解。

最後回到前提。這整套推導從頭到尾假設了「存在一個 3×3 矩陣能描述兩張影像之間的對應」，而這件事只在兩種情況下成立（4.2.2 節與圖 4-9）：場景是一個平面，或相機只做純旋轉。若前提不成立，上面的方程組仍然解得出一個答案——這正是危險的地方。演算法不會報錯，它會給你一個「在最小平方意義下最好」的矩陣，而那個矩陣誰也對不齊。遇到這種情形時，要換的不是參數而是模型：第 12 章的光流或完整的三維重建才描述得了「不同深度有不同位移」。

4.3 影像內插：轉換後的像素從哪裡來

4.3.1 為什麼要反向映射

直覺上，執行幾何轉換似乎應該「把每個輸入像素搬到它該去的位置」——這稱為正向映射。但這樣做會產生兩個嚴重問題：某些輸出位置沒有任何輸入像素落上去，形成破洞;而某些位置則有多個輸入像素重疊，不知該用哪一個。

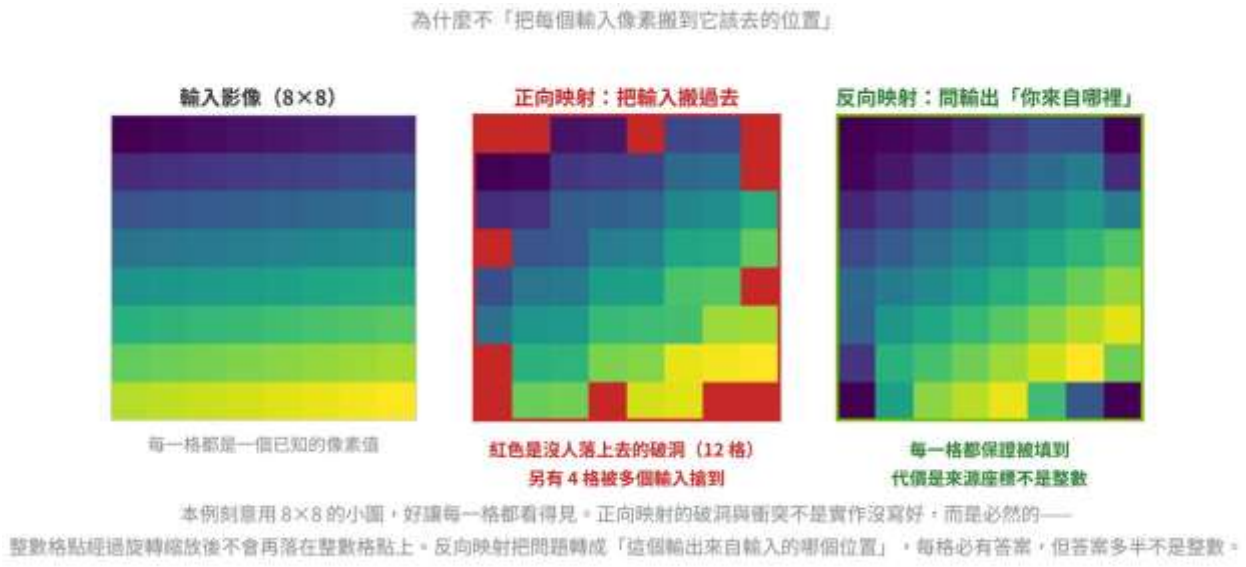


圖 4-10　正向映射的破洞與衝突，以及反向映射的解法

因此 OpenCV 實際採用反向映射(Backward Mapping)：對輸出影像的每一個像素位置，用轉換矩陣的反矩陣反算「它來自輸入影像的哪裡」。這樣每個輸出像素都保證有值、且只有一個來源，沒有破洞也沒有衝突。

但新問題來了：反算出的來源座標幾乎都不是整數——像 (217.3, 154.8) 這樣的

位置根本沒有現成的像素值。從鄰近的整數像素「估」出這個值，就是內插(Interpolation)。這也解釋了為什麼幾何轉換必然伴隨品質損失：每一次轉換都要內插一次，而內插本質上是猜測。

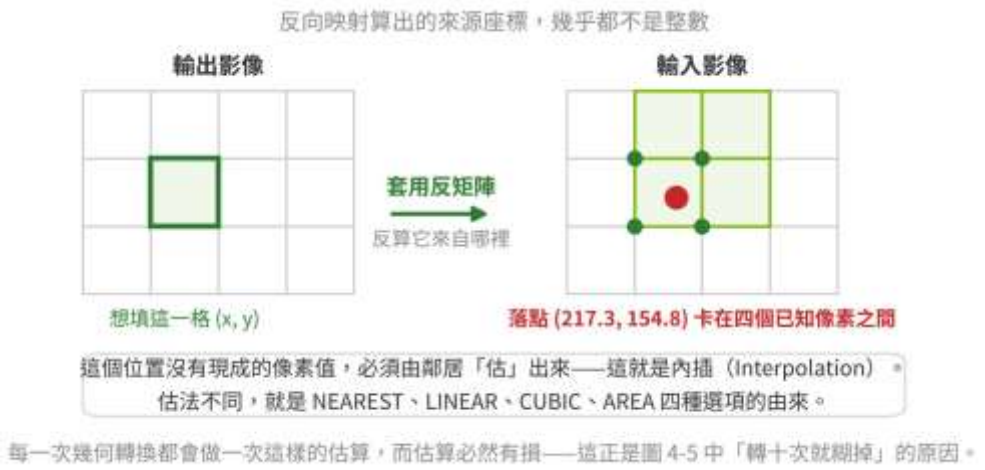


圖 4-11　反算出的來源座標幾乎都不是整數

4.3.2 四種內插法

5. 最近鄰內插(INTER_NEAREST)：直接取最近的那個像素值。最快，但放大後鋸齒與方塊感明顯。它唯一但關鍵的優點是不會產生「新的數值」——因此處理標籤圖、分割遮罩(第 9、16 章)時必須用它。若對一張只有 0 與 1 的遮罩用雙線性內插，會產生 0.5 這種無意義的中間值。

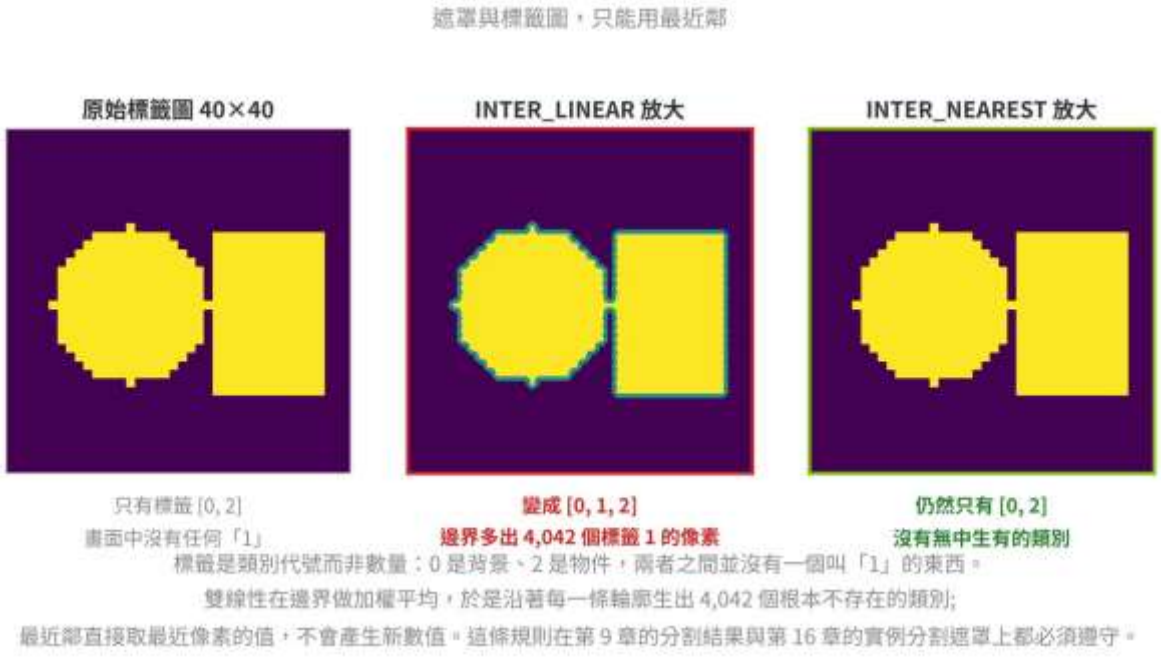


圖 4-13　遮罩與標籤圖只能用最近鄰

6. 雙線性內插(INTER_LINEAR,OpenCV 預設)：取周圍 2×2 共 4 個像素，依距離加權平均。速度與品質的甜蜜點，絕大多數場合的首選。
7. 雙三次內插(INTER_CUBIC)：取周圍 4×4 共 16 個像素，以三次多項式加權。邊緣更平滑銳利，代價是約數倍的運算量，適合離線的高品質放大。

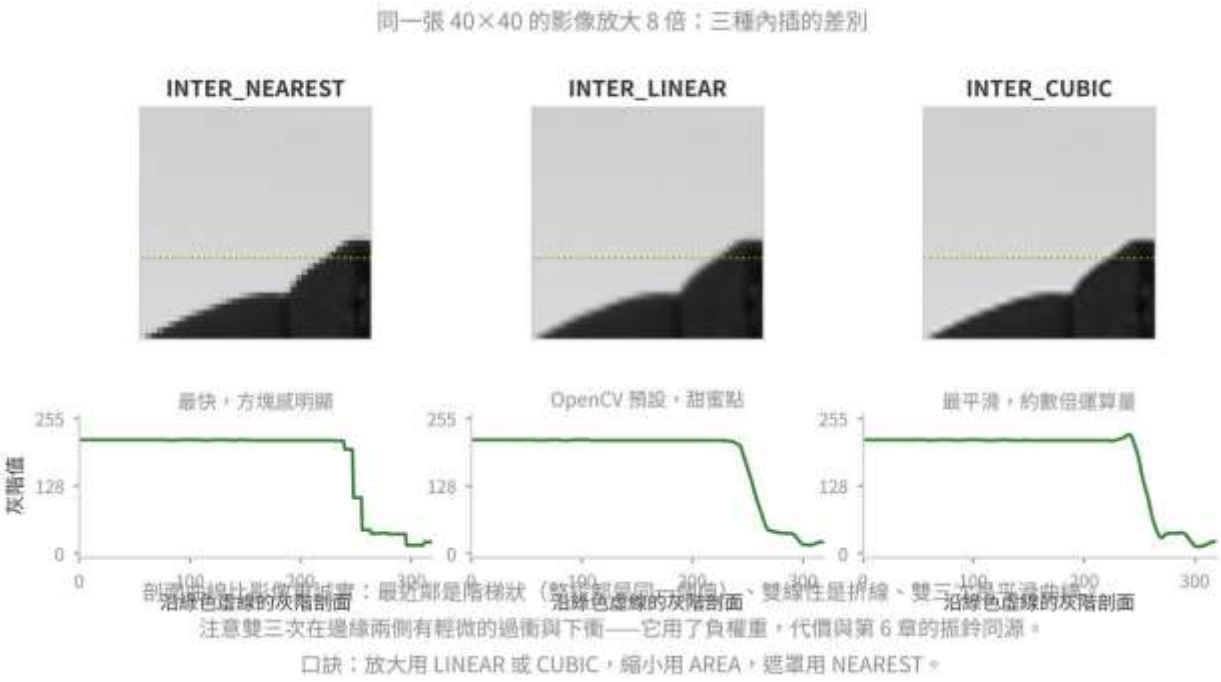


圖 4-12　三種內插法放大八倍：影像與灰階剖面

8. 區域內插(INTER_AREA)：以區域平均進行降取樣。它是縮小影像的建議選項——原因在第 3 章已經說明：縮小就是降取樣，不先做低通就會產生混疊，而 INTER_AREA 的區域平均正是內建的低通濾波。

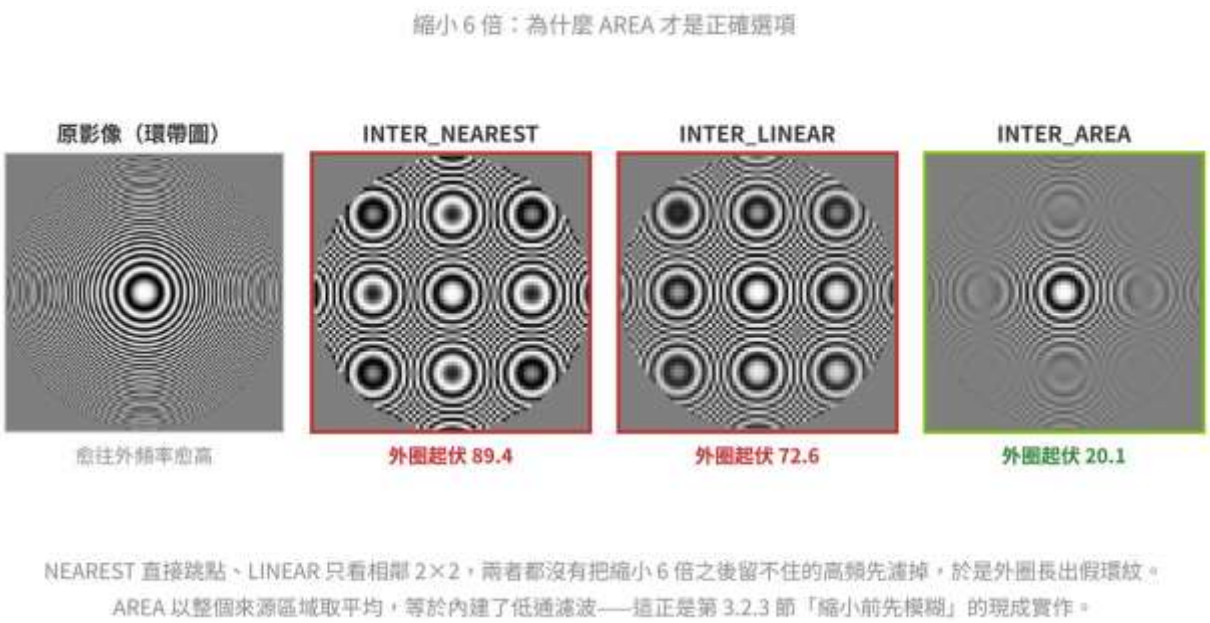


圖 4-14　縮小影像為什麼要用 INTER_AREA

口訣：放大用 LINEAR 或 CUBIC，縮小用 AREA，遮罩用 NEAREST。這個口訣背後串起了第 3 章(取樣定理與混疊)、第 5 章(平滑濾波)、第 9 章(分割遮罩)三章的觀念。

程式 4-3 四種內插法與多次轉換的代價

```
import cv2, numpy as np

img = cv2.imread("small_photo.jpg")

# --- 放大:比較三種內插的品質 ---
for name, flag in [("nearest", cv2.INTER_NEAREST),
                   ("linear",  cv2.INTER_LINEAR),
                   ("cubic",   cv2.INTER_CUBIC)]:
    big = cv2.resize(img, None, fx=4, fy=4, interpolation=flag)
    cv2.imwrite(f"up_{name}.png", big)
# 放大 4 倍後檢視:nearest 滿是方塊,cubic 邊緣最平滑

# --- 遮罩必須用 NEAREST,否則會產生無意義的中間值 ---
mask = np.zeros((100, 100), np.uint8)
mask[30:70, 30:70] = 1                          # 標籤圖:只有 0 與 1

bad  = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_LINEAR)
good = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_NEAREST)
print("雙線性後的唯一值:", np.unique(bad))    # 出現 0 與 1 以外的值!
print("最近鄰後的唯一值:", np.unique(good))   # 仍只有 0 與 1

# --- 多次轉換 vs. 合成一次:品質差異 ---
a = img.copy()
for _ in range(10):                             # 連續旋轉 10 次,每次 36 度
    M = cv2.getRotationMatrix2D((img.shape[1]/2, img.shape[0]/2), 36, 1)
    a = cv2.warpAffine(a, M, img.shape[1::-1])
```

```
print("轉一圈後與原圖的 PSNR:", cv2.PSNR(img, a))
# 理論上轉回原位,實際上因十次內插而明顯劣化 → 應合成矩陣一次做
```

4.3.3 推導：雙線性內插的公式，與它為什麼會讓影像變糊

雙線性內插是本書使用最頻繁的內插法，它的公式只有一行，但「它為什麼會讓影像變糊」這個問題要到第 6 章才真正說得清楚。這裡把兩件事一次講完。

先看公式。設反算出的來源座標為 (x + a, y + b)，其中 x、y 是整數、a 與 b 是 0 到 1 之間的小數部分。周圍四個已知像素是 I(x, y)、I(x+1, y)、I(x, y+1)、I(x+1, y+1)。做法是先在水平方向做兩次一維線性內插：上列得到 (1−a)·I(x, y) + a·I(x+1, y)，下列得到 (1−a)·I(x, y+1) + a·I(x+1, y+1);再把這兩個結果在垂直方向依 b 內插一次。展開後四個像素的權重分別是 (1−a)(1−b)、a(1−b)、(1−a)b、ab，四者相加恰為 1——這是檢查算式有沒有寫錯的最快方法。

順序可以顛倒：先垂直再水平，結果完全相同。這個性質稱為可分離性，與第 5.3.3 節的可分離濾波同源，也是雙線性內插比雙三次快得多的原因——二維的問題被拆成兩次一維運算。

接著是「為什麼會糊」。關鍵在於換個角度看這件事：雙線性內插其實是把輸入影像與一個三角形的權重函數做卷積，再在新的位置取樣。而卷積就是濾波（第 5 章）——這個三角形權重是一個低通濾波器，它必然衰減高頻成分。第 6 章會給出定量的答案：三角形函數的頻率響應是 sinc 的平方，在 Nyquist 頻率處的增益已經掉到相當低，這就是為什麼每做一次內插，細節就少一分。圖 4-5 實測的「分三次做比合成一次做少了兩成銳利度」，量的正是這個效應。

這個觀點也一併解釋了另外兩件事。其一，雙三次內插為什麼比較銳利卻會過衝：它的權重函數帶有負的部分，等於一個輕微的銳化濾波器，代價就是在強邊緣兩側產生過衝與下衝（圖 4-12 的剖面曲線）——與第 6.3.2 節的振鈴效應是同一個現象的不同程度。其二，最近鄰為什麼不糊卻有鋸齒：它的權重函數是一個方形脈衝，在空間

域完全不平均（所以不糊），但方形在頻率域的旁瓣極多（第 6 章），表現出來就是階梯狀的鋸齒。

結論回到實務：既然每次內插都要付出高頻的代價，最好的策略就是少做幾次。把所有轉換合成單一矩陣、只取樣一次，不只是為了省時間，更是為了省細節。

4.4 相機模型、鏡頭畸變與張氏標定

4.4.1 針孔相機模型

針孔相機模型把成像抽象成一句話：三維空間中的點沿直線投影穿過光心，落在成像平面上。以齊次座標表示，整個投影過程可寫成「內參矩陣 K 乘以外參 [R|t] 乘以世界座標點」——注意這裡再次用到了 4.1.3 節的齊次座標，它讓三維到二維的投影也能寫成單一的矩陣乘法。



圖 4-15　針孔相機模型：三維世界如何變成二維影像

參數分成兩組，區分它們是理解校正的關鍵：

- 內部參數(Intrinsics)：屬於相機本身，包括焦距 fx、fy(以像素為單位)與主點 cx、cy(光軸與成像面的交點，理想上在影像中心)。只要相機與鏡頭組合不變，這組參數就固定，與相機放在哪裡無關。



圖 4-16　內部參數的幾何意義：四個數字各自改變什麼

- 外部參數(Extrinsics)：描述相機在世界中的位置與朝向，包括旋轉矩陣 R 與平移向量 t。相機一移動，這組參數就改變。

這個區分有實務意義：換一顆鏡頭或調整變焦，必須重新校正內參;而把相機搬到另一個腳架上，內參不變、只有外參改變。在機器人系統中，內參通常在出廠時校正一次並存檔，外參則由第 17 章的 tf2 系統即時追蹤。

4.4.2 鏡頭畸變

真實鏡頭不是理想的針孔，光線經過鏡片組會產生畸變(Distortion)。徑向畸變沿半徑方向作用，以係數 k1、k2、k3 建模：係數為負時影像向外鼓、直線變成外凸的弧線，稱為桶狀畸變(廣角鏡、GoPro、ESP32-CAM 的魚眼感都是它);係數為正時向內縮，稱為枕狀畸變，常見於望遠端。切向畸變(p1、p2)則來自鏡片與感光元件未能完全平行。

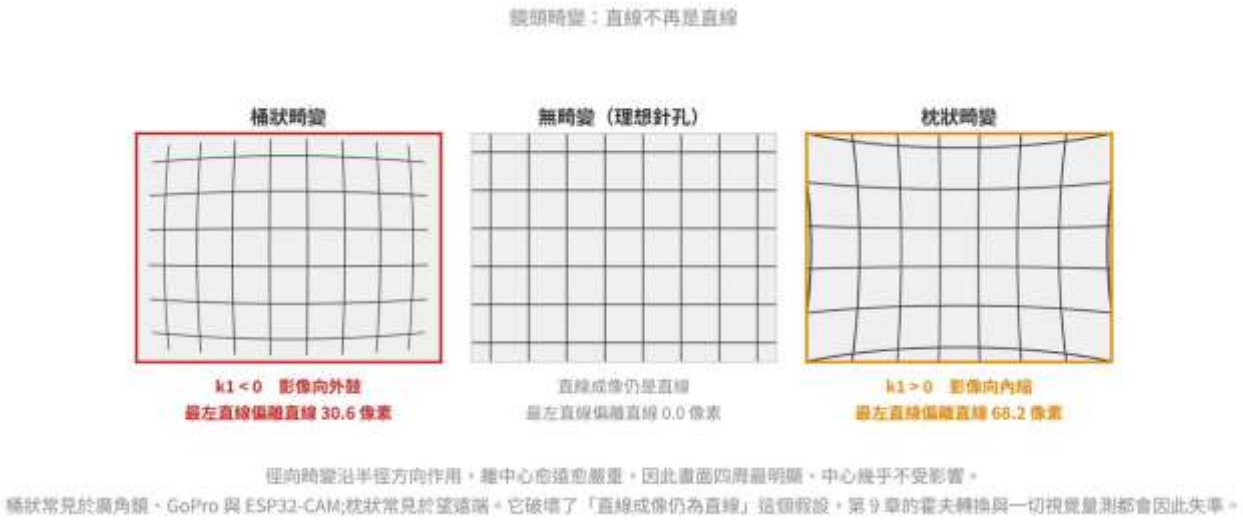


圖 4-17　桶狀與枕狀畸變：直線不再是直線

畸變對機器人與量測應用是致命的，因為它破壞了「直線成像仍是直線」這個假設：第 9 章的霍夫轉換車道偵測會把彎曲的車道線判為多段短線、第 13 章的特徵匹配位置會系統性偏移、第 17 章以像素換算實際距離的計算會失準——而且愈靠近畫面邊緣誤差愈大。因此本書的原則是：先校正，再談視覺量測。

4.4.3 張氏標定法實作

依叫獸開講的原理，OpenCV 的標定流程分四步：拍攝棋盤格多張、偵測角點、求解參數、去畸變。

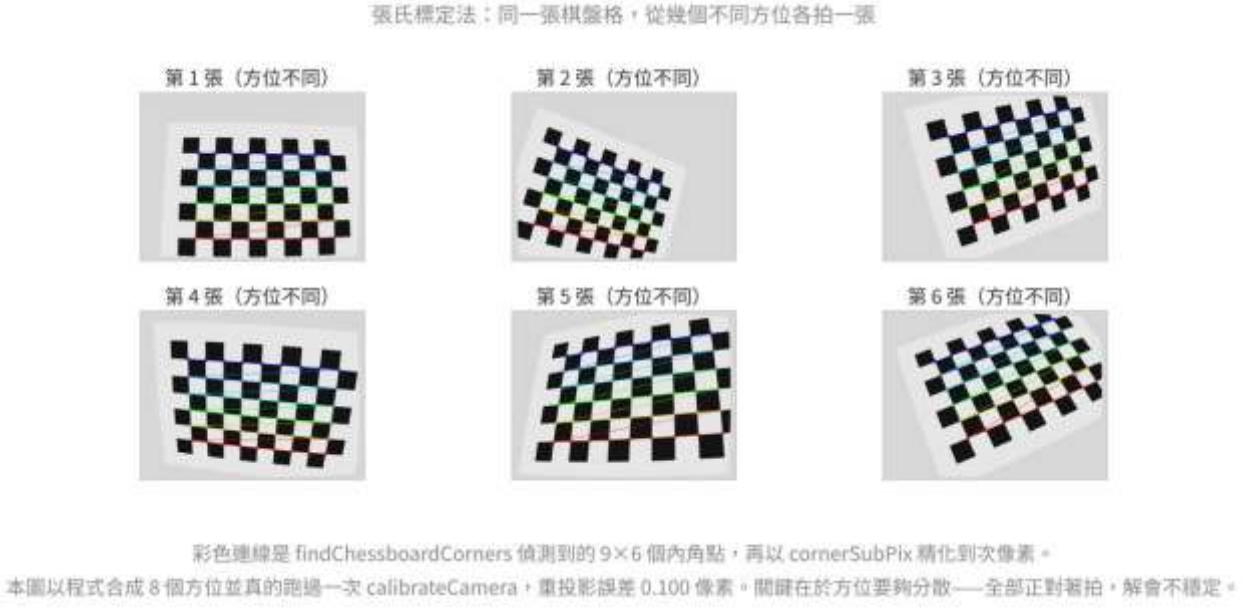


圖 4-18　張氏標定法：從多個方位拍同一張棋盤格

程式 4-4 張氏標定法完整流程

```
import cv2
import numpy as np
import glob

CB = (9, 6)   # 棋盤格「內角點」數:10x7 格的板子內角點為 9x6

# 棋盤格角點的世界座標(z=0 平面,單位:格)
objp = np.zeros((CB[0]*CB[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CB[0], 0:CB[1]].T.reshape(-1, 2)

objpoints, imgpoints = [], []
for fname in glob.glob("calib/*.jpg"):      # 10~20 張不同方位
    gray = cv2.imread(fname, cv2.IMREAD_GRAYSCALE)
    ok, corners = cv2.findChessboardCorners(gray, CB, None)
    if ok:
        # 次像素精化:把角點定位精度提升到小數點以下
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
            (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001))
        objpoints.append(objp)
        imgpoints.append(corners)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None)

print("重投影誤差(愈小愈好,<0.5 佳):", ret)
print("內參矩陣 K:\n", K)          # fx, fy, cx, cy
print("畸變係數:", dist.ravel())    # k1 k2 p1 p2 k3

# 存檔:整學期所有實驗共用(第 12、17 章都會用到)
np.savez("camera_params.npz", K=K, dist=dist)

# 去畸變:彎掉的直線拉回來
img = cv2.imread("calib/test.jpg")
undist = cv2.undistort(img, K, dist)
cv2.imwrite("undistorted.jpg", undist)
```

驗收標準是重投影誤差：把求得的參數反過來把世界點投影回影像，與實際偵測到的角點位置比較，取平均距離。一般以低於 0.5 像素為佳。若誤差偏大，常見原因有三：棋盤格不平整(貼在軟紙板上會微微彎曲)、照片模糊或動態模糊、方位變化不足(所有照片都從相似角度拍)。這正好回應了叫獸開講的思考題——多拍不同方位的照片，就是在對抗這些誤差來源。

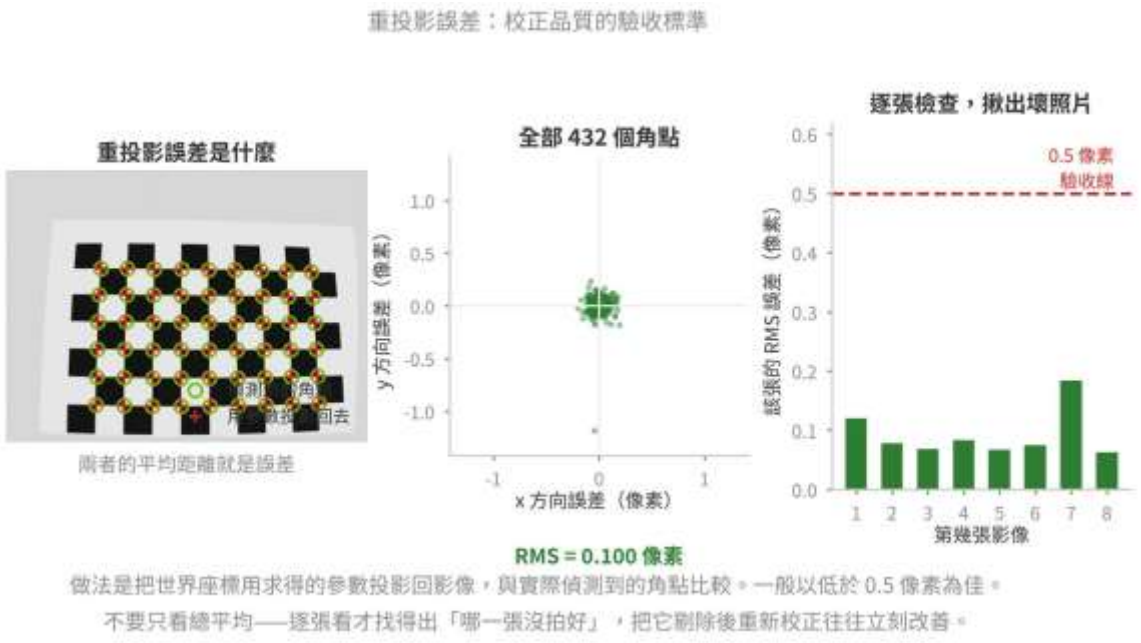


圖 4-20　重投影誤差：校正品質的驗收標準

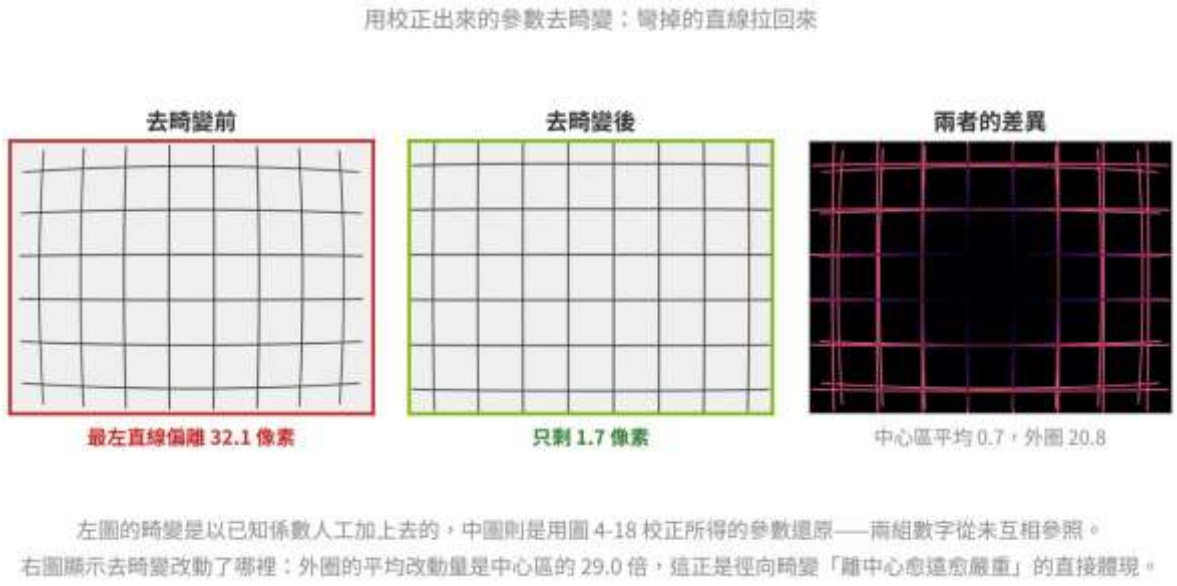


圖 4-19　用校正出來的參數去畸變

4.4.4 從像素到公尺

校正的最終價值，是讓相機成為量測工具。一旦知道內參，加上一個已知的參考(例如物體在已知距離的平面上、或畫面中有已知尺寸的參考物)，就能把「像素」換算成「公釐」。這個換算是後續多章的基礎：第 7.6.3 節由產線速度與像素尺寸推算運動模糊長度、第 12.7.2 節由車輛的像素位移換算實際車速、第 17 章機器人以視覺估計障礙物距離——它們全都建立在本節的校正之上。

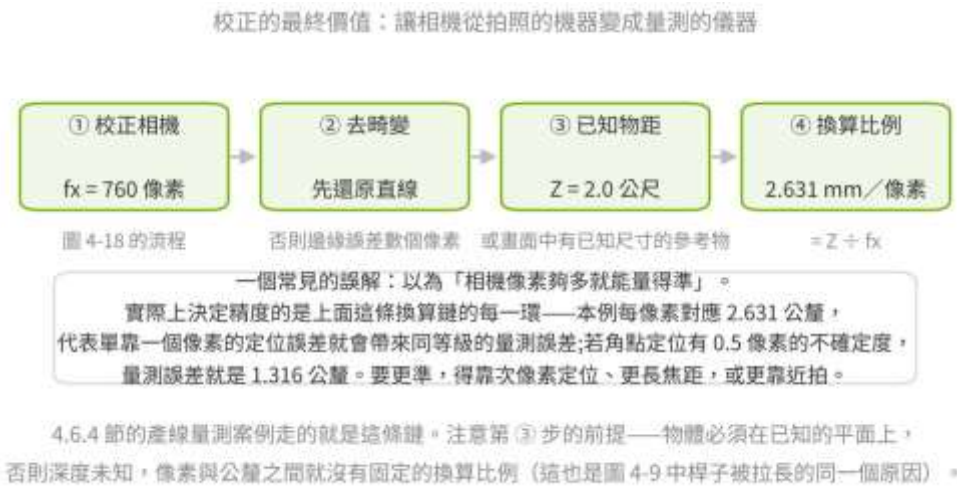


圖 4-21　從像素到公釐：讓相機變成量測儀器

4.5 幾何轉換在全書中的角色

幾何轉換是本書中出現頻率最高的工具之一，值得在此把散落各章的應用收攏起來：

- 資料增強(第 14、15、16 章)：訓練模型時的隨機旋轉、平移、翻轉、縮放，是對抗過度擬合最有效的手段之一，而它們全是本章的仿射轉換。第 16 章 YOLO 訓練參數中的 degrees、fliplr 正是如此。
- 視訊穩定(第 12 章)：估計相鄰影格之間的抖動(以第 6 章相位相關或第 13 章特徵匹配)，再用本章的幾何轉換反向補償，把每一影格「拉回」穩定位置。
- 影像拼接(第 13 章)：求出兩張照片間的透視轉換，把一張拉到與另一張對齊後融合，即成全景圖。
- 物件定位(第 13 章)：特徵匹配加 RANSAC 求出的單應性矩陣，能把參考影像的四個角投影到場景中，框出物件的位置與姿態。
- 車牌辨識(第 13 章)：斜拍的車牌先以透視轉換拉正，字元分割與辨識才能可靠進行。
- 鳥瞰圖與導航(本章 Lab、第 12 章)：把地面透視轉換成俯視圖，是循跡機器人與車道偵測的標準前處理。

- 機器人座標系(第 17 章):ROS 2 的 tf2 以齊次座標矩陣管理相機、機械臂、底盤、世界之間的座標轉換關係。

這份清單說明了一件事：本章不是一個「學完就過去」的獨立章節，而是往後每一章都會回頭取用的工具箱。建議你把程式 4-4 產出的相機參數檔妥善保存，它會陪你走完整個學期。

4.6 應用案例

4.6.1 案例一：文件掃描 App

情境：用手機斜拍一張紙本文件，自動偵測紙張邊界並拉正成正面的掃描檔。這是透視轉換最貼近生活的應用，完整流程串起本書多章：轉灰階與去雜訊(第 5 章)→ Canny 邊緣偵測(第 9 章)→ 找出最大的四邊形輪廓(第 9、10 章)→ 取四個角以透視轉換拉正(本章)→ 自適應門檻二值化成清晰的黑白掃描檔(第 9 章)。

工程細節值得留意：目標矩形的長寬比不能隨便設，否則文件會被不自然拉伸——程式 4-2 用「量測對邊長度取平均」的方式估計合理比例。若已知是 A4 紙，也可直接指定 1:1.414 的比例。這種「利用先驗知識提升結果」的思路，在工程上總是划算的。

4.6.2 案例二：機器人導航的鳥瞰圖

情境：循跡機器人或自駕車的前視相機，看到的地面是有透視變形的——遠處的車道線看起來會匯聚。若直接在原始影像上判斷「線在左邊還是右邊、彎曲程度多少」，會因透視而失真。

解法是逆透視映射(IPM)：選取地面上的一塊梯形區域，把它透視轉換成矩形，得到俯視的鳥瞰圖。在鳥瞰圖上，平行的車道線真的是平行的、彎道的曲率也能正確量測，後續的第 9 章霍夫轉換或曲線擬合都變得單純許多。這是車道保持與循跡機器人的標準前處理，也是本章 Lab 4-2 的目標。

前提條件必須說清楚：IPM 假設所有內容都在同一個平面(地面)上。因此地面上的車道線會被正確攤平，但立體的物件(其他車輛、行人)在鳥瞰圖中會被拉成怪異的長條——因為它們違反了平面假設。理解這個限制，你就知道鳥瞰圖適合用來分析路面標線，而不適合用來判斷障礙物的形狀。

4.6.3 案例三：資料增強的幾何操作

情境：訓練一個工件瑕疵偵測模型，但標註樣本只有兩百張。資料增強能以幾何轉換人工擴充資料的多樣性：隨機旋轉正負十五度、隨機平移百分之十、水平翻轉、隨機縮放。每一張原始影像因此能衍生出許多變化，逼迫模型學習「不管工件怎麼擺放都要認得」的穩健特徵。

但增強必須合理，這是最容易被忽略的一點：若你的工件永遠以固定方向進料，那麼旋轉一百八十度的增強反而是在教模型學習不會發生的情況，可能稀釋了有效的學習。同理，數字辨識任務不該做垂直翻轉(6 會變成 9 的樣子)、醫學影像不該做左右翻轉(器官有左右之分)。增強要模擬「真實世界可能出現的變化」，而非隨意變形。這個判斷力，是第 15 章訓練模型時的必修課。

4.6.4 案例四：相機校正在產線量測的價值

情境：以視覺量測工件尺寸，要求精度達到零點一毫米。此時鏡頭畸變絕不能忽略——未校正的廣角鏡頭在畫面邊緣可能有數個像素的位置偏差，換算成實際尺寸就是零點幾毫米的誤差，直接讓量測失效。

標準做法：出廠時以張氏標定法校正相機並存檔內參與畸變係數(程式 4-4)→ 每次取像後先去畸變 → 以已知尺寸的標準件或標定板建立「像素對實際尺寸」的換算比例 → 定期(如每季或更換鏡頭後)重新校正驗證。

這個案例呼應了叫獸開講的核心：在張氏標定法問世之前，要做到這件事需要昂貴的三維標定物，只有大企業玩得起;而現在，一張列印的棋盤格加上開源的 OpenCV，

任何一間中小企業、任何一個學生專題都能達成工業級的量測校正。降低門檻的力量，莫過於此。

4.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
齊次座標	第 17 章	ROS 2 的 tf2 座標框架管理
仿射轉換	第 12、14-16 章	視訊穩定的運動補償;訓練時的資料增強
透視轉換/單應性	第 13 章	RANSAC 求單應性做物件定位與影像拼接
內插與 INTER_AREA	第 3、9、16 章	取樣定理與防混疊;遮罩須用 NEAREST
相機校正與內參	第 7、12、17 章	推算運動模糊長度;像素換算車速;機器人測距
鏡頭畸變	第 9、13、17 章	直線偵測失準;特徵位置偏移;量測誤差
鳥瞰圖(IPM)	第 9、12 章	車道偵測與循跡機器人的前處理

表 4-2 第 4 章與全書其他章節的觀念連結

程式實作 Lab 4

Lab 4-1 轉換家族與內插實驗

目標：建立對轉換與內插的直覺。(a)以程式 4-1 對一張照片分別做平移、旋轉、縮放，並驗證「先旋轉再平移」與「先平移再旋轉」結果不同;(b)實作矩陣串接：比較「分三次做」與「合成一次做」的執行時間與 PSNR;(c)執行程式 4-3 的「連續旋轉十次轉回原位」實驗，記錄 PSNR 的劣化程度，說明為什麼理論上該回到原圖卻沒有;(d)對一張分割遮罩分別用 LINEAR 與 NEAREST 放大，用 np.unique 驗證前者產生了無

意義的中間值;(e)整理出一張「什麼情況用哪種內插」的決策表。

Lab 4-2 相機校正與機器人鳥瞰圖

目標：替課程相機建立專屬校正檔，並產生導航鳥瞰圖。(a)列印棋盤格(OpenCV 官網可下載)，固定於硬板上——務必平整，不可貼在會彎曲的軟紙板;(b)拍攝 15 張涵蓋畫面四角、遠近與傾斜各異的照片;(c)執行程式 4-4，記錄重投影誤差與內參，將 K 與 dist 存成 npz 檔供整學期所有實驗使用;(d)去畸變後，在地面貼四張定位貼紙圍成已知尺寸的矩形，以透視轉換產生俯視鳥瞰圖;(e)量化實驗：刻意只用 3 張照片重新校正，比較重投影誤差與 15 張時的差異，驗證叫獸開講思考題的結論;(f)ESP32-CAM 的廣角鏡畸變特別明顯，校正前後直線的彎曲程度差多少?請量化它。

Lab 4-3 文件掃描器

目標：做出手機掃描 App 的核心功能。(a)以程式 4-2 為基礎，用 cv2.setMouseCallback 讓使用者在影像上依序點四個角，取代寫死的座標;(b)依四點自動計算目標矩形的長寬比，避免文件被不自然拉伸;(c)串接第 3 章的成果——輸出前將文件轉灰階並提高對比，存成清晰的掃描檔;(d)挑戰題：第 9 章學會輪廓偵測後，回來把「手動點四角」升級成「自動找四角」，做成全自動掃描器;(e)延伸：實作反向轉換，把一張圖片貼到原始影像中那張紙的位置上——這正是擴增實境的基本原理。

Lab 4-4 資料增強管線(第 15 章前導)

目標：提前建立資料增強的觀念與工具。(a)寫一個 augment(img) 函式，隨機套用旋轉(正負 15 度)、平移(10%)、縮放(0.9 到 1.1 倍)、水平翻轉;(b)對一張影像產生 20 個增強版本並拼貼顯示，檢查是否有出現不合理的變形(如內容被轉出畫面外);(c)思辨題：針對三種不同任務——工件瑕疵偵測、手寫數字辨識、胸部 X 光判讀——各自判斷哪些增強該用、哪些不該用，並說明理由;(d)這個函式將在第 15、16 章訓練模型時直接使用，請存成共用模組。

Lab 4-5 內插法的代價實測

目標：把圖 4-5、4-12、4-14 的現象變成你自己量過的數據。(a)取一張清晰的照片，連續旋轉 36 度十次，每次都存檔;計算每一步與原圖的 PSNR 並畫成曲線，觀察劣化如何累積;(b)改成把十個矩陣相乘後只做一次 warpAffine，比較最終的 PSNR 與執行時間;(c)以 Laplacian 的標準差當作銳利度指標，量測兩種做法的差距——注意只能在兩者都有內容的中央區域比較，想一想為什麼不能直接比整張;(d)把同一張圖分別以 NEAREST、LINEAR、CUBIC、LANCZOS4 放大八倍，沿一條穿過強邊緣的水平線畫出灰階剖面，找出哪幾種會產生過衝，並解釋它與第 6 章振鈴的關係;(e)把一張圖縮小 1.5、2、4、8 倍，每個倍率都用 LINEAR 與 AREA 各做一次，量測混疊程度（提示：可用第 3 章的環帶圖當素材），找出「從幾倍開始非用 AREA 不可」;(f)把結論寫成三行口訣，貼在你的工具箱模組開頭。

Lab 4-6 校正你自己的相機

目標：為課程用的相機建立一份會陪你走完整學期的校正檔。(a)列印棋盤格並貼在硬質平板上——務必平整，貼在軟紙板上是重投影誤差偏大的頭號原因;(b)拍攝 15 到 20 張，刻意涵蓋多種傾角與畫面的各個區域（特別是四個角落，那裡的畸變最大、資訊最多）;(c)跑程式 4-4，記錄重投影誤差與內參，並把每張照片各自的誤差也印出來——找出誤差最大的那一兩張，看看它們有什麼共同點;(d)刻意做一組對照實驗：只用三張方位相似的照片校正一次，比較它的重投影誤差與用全部照片的結果。若前者的數字反而比較好看，請解釋為什麼（見習題 23）;(e)用校正參數對一張含直線的照片去畸變，量測某條直線在去畸變前後偏離直線的最大距離，以數字驗證效果;(f)拿一把尺或一個已知尺寸的物件，在固定距離下量測它的像素長度，建立「每像素幾公釐」的換算，再量另一個已知物件驗證誤差有多大;(g)把 camera_params.npz 存進你的工具箱倉庫。第 12 章的視訊穩定與第 17 章的部署都會直接讀它。

本章重點整理

- 幾何轉換改變像素位置而非數值;齊次座標以「升一維」把平移、旋轉、縮放統一為矩陣乘法，多重轉換可預先相乘合成單一矩陣，一次取樣完成(更快且品質更好);矩陣相乘順序不可交換，從右往左作用。
- 轉換家族層層包含：剛體(3 自由度，保長度角度)、相似(4)、仿射(6 自由度、3 組對應點、保平行)、透視(8 自由度、4 組對應點、保直線)。單應性成立的前提是拍攝平面或相機純旋轉。
- 反向映射避免破洞與衝突，但來源座標非整數故需內插;NEAREST 最快且不產生新值(遮罩專用)、LINEAR 為預設甜蜜點、CUBIC 高品質放大、AREA 為縮小首選(內建低通防混疊，呼應第 3 章)。
- 針孔模型的內部參數(fx、fy、cx、cy)屬於相機本身，外部參數(R、t)描述相機位姿;換鏡頭需重新校正內參，搬動相機只改外參。
- 徑向畸變分桶狀(係數負，廣角鏡)與枕狀(係數正，望遠端)，另有切向畸變;畸變破壞「直線成像仍為直線」的假設，使邊緣偵測、特徵匹配、視覺量測全部失準，故須先校正再量測。
- 張氏標定法(1998 MSR-TR-98-71,2000 TPAMI)只需印一張棋盤格、從至少兩個未知方位拍攝即可校正，為 OpenCV calibrateCamera 之核心;以重投影誤差驗收，一般小於 0.5 像素為佳。
- 幾何轉換是全書最常取用的工具：資料增強(第 14-16 章)、視訊穩定(第 12 章)、影像拼接與物件定位(第 13 章)、車牌拉正(第 13 章)、鳥瞰圖(第 9、12 章)、ROS 2 座標系(第 17 章)。

習題

1. 為什麼平移無法用 2×2 矩陣表示?齊次座標如何解決這個問題?統一之後帶來哪兩個好處?

   提示與參考解答：因為平移是「加法」而非線性的矩陣乘法，縮放與旋轉可寫成 2×2 矩陣乘法，平移不行。齊次座標替每個點多加一維(x, y, 1)，使平移也能寫成

3×3 矩陣乘法。兩個好處：一、多個轉換可預先相乘合成單一矩陣，一次做完;二、只需取樣內插一次，避免多次內插造成的品質劣化。

2. 寫出「先繞原點旋轉 90 度、再平移 (10, 20)」的合成 3×3 矩陣，並驗算點 (1, 0) 的落點。若順序顛倒，結果會相同嗎?

   提示與參考解答：旋轉 90 度矩陣 R = [[0,-1,0],[1,0,0],[0,0,1]];平移 T = [[1,0,10],[0,1,20],[0,0,1]]。合成 M = T·R(從右往左：先旋轉後平移)。點 (1,0,1) 經 R 得 (0,1,1)，再經 T 得 (10,21)。順序顛倒(M = R·T)結果不同：(1,0) 先平移得 (11,20)，再旋轉得 (-20,11)。矩陣乘法不可交換，順序必須明確。

3. 請比較剛體、相似、仿射、透視四種轉換的自由度、所需對應點數與保持的性質。

   提示與參考解答：剛體：3 自由度、2 組對應點、保長度與角度。相似：4 自由度、2 組點、保角度與形狀比例。仿射：6 自由度、3 組點、保平行性與面積比。透視：8 自由度、4 組點、保直線性與交比(不保平行，故能表現近大遠小)。可對照表 4-1。

4. 斜拍的棋盤，格線在影像中不再平行。應以仿射還是透視轉換拉正?為什麼?單應性成立的前提是什麼?

   提示與參考解答：應用透視轉換。因為仿射保持平行性，無法描述「平行線相交」的透視效果;透視轉換不保平行但保直線，正好對應近大遠小的投影。單應性成立的前提：拍攝的是一個平面(如紙張、地面、牆面)，或相機只做純旋轉而不平移。若場景有深度變化且相機平移，單一單應性無法描述兩影像的關係。

5. 為什麼幾何轉換採用反向映射而非正向映射?反向映射帶來什麼新問題?

   提示與參考解答：正向映射(把輸入像素搬到目標位置)會產生兩個問題：某些輸出位置沒有任何輸入落上去而形成破洞，某些位置則多個輸入重疊而衝突。反向映射對每個輸出像素反算來源，保證每個輸出都有值且唯一。新問題是：反算出的來源座標幾乎都不是整數，必須從鄰近整數像素估值——這就是內插，也是幾何轉換必然伴隨品質損失的原因。

6. 放大分割遮罩(數值只有 0 與 1)時，為什麼必須用最近鄰內插?用雙線性會發生什麼事?

提示與參考解答：因為最近鄰直接取最近像素的值，不會產生新數值，遮罩仍只有 0 與 1。雙線性是加權平均，會在邊界產生 0.3、0.5 這類中間值，而標籤圖的「0.5 類」毫無意義，後續處理會出錯。這也適用於第 9、16 章的分割結果與第 14 章的標籤資料。

7. 縮小影像為什麼建議用 INTER_AREA?這與第 3 章的哪個定理有關?

提示與參考解答：縮小就是降低取樣率，依第 3 章的 Nyquist 取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊(假圖案)。INTER_AREA 以區域平均進行降取樣，本身內建了低通濾波的動作，先把高頻濾掉再取樣，因此能有效抑制混疊，是縮小的建議選項。

8. 說明內部參數與外部參數的差別。換一顆鏡頭後，哪一組需要重新校正?把相機搬到另一個腳架上呢?

提示與參考解答：內部參數(fx、fy、cx、cy)屬於相機與鏡頭本身，描述成像的幾何特性，與相機放在哪裡無關。外部參數(R、t)描述相機在世界中的位置與朝向。換鏡頭或調整變焦會改變成像特性，必須重新校正內參;搬到另一個腳架只改變位置與朝向，內參不變、只有外參改變(在機器人中由第 17 章的 tf2 即時追蹤)。

9. GoPro 影像中筆直的地平線變成弧形，這是哪種畸變?對應的係數正負為何?為什麼畸變對視覺量測是致命的?

提示與參考解答：這是桶狀畸變(barrel distortion)，徑向畸變係數為負，影像向外鼓、直線變成外凸弧線，常見於廣角鏡、GoPro、ESP32-CAM。致命原因：它破壞了「直線成像仍為直線」的假設，使第 9 章霍夫直線偵測失準、第 13 章特徵位置系統性偏移、第 17 章像素換算實際距離產生誤差，且愈靠畫面邊緣誤差愈大。故須先校正再量測。

10. 張氏標定法相較「精密三維標定物」方法的核心優勢是什麼?實務上為何要拍 10 張以上?

提示與參考解答：核心優勢：只需一張列印的平面棋盤格，從至少兩個不同方位觀察即可，且相機或棋盤皆可自由移動、運動不需已知;傳統方法需要精密加工的兩三個正交平面，昂貴且門檻高。張正友論文稱它「讓三維電腦視覺從實驗室往真實世界前進一步」。拍 10 張以上是為了對抗影像雜訊與角點定位誤差、並涵蓋畫面各角落以準確估計畸變;原論文實驗亦顯示估計的不確定度隨影像張數增加而下降。

11. 重投影誤差 1.8 像素算好嗎?請列出三個可能原因與對應的改善做法。

    提示與參考解答：不好，一般以低於 0.5 像素為佳，1.8 表示校正品質不佳。三個可能原因與對策：一、棋盤格不平整(貼在會彎曲的軟紙板上)——改貼在硬質平板;二、照片模糊或有動態模糊，角點偵測不準——用腳架、加強打光、縮短曝光;三、方位變化不足，所有照片角度相似——刻意涵蓋畫面四角、不同傾斜與遠近。另可檢查是否誤設棋盤格內角點數。

12. (綜合題)承 4.6.3 節：針對「工件瑕疵偵測」、「手寫數字辨識」、「胸部 X 光判讀」三種任務，請各自判斷旋轉 180 度與左右翻轉這兩種增強是否適用，並說明理由。

    提示與參考解答：工件瑕疵偵測：若工件以固定方向進料，旋轉 180 度是在教模型學習不會發生的情況，可能稀釋有效學習，不建議;若工件會任意擺放則適用。左右翻轉通常可行(瑕疵無左右之分)。手寫數字辨識：旋轉 180 度不適用(6 與 9 會混淆);左右翻轉不適用(鏡像數字不存在於真實資料)。胸部 X 光：旋轉 180 度不適用(解剖方向固定);左右翻轉不適用，因人體器官有左右之分(心臟偏左)，翻轉會產生醫學上錯誤的影像。原則：增強應模擬真實世界可能出現的變化。

13. 計算題：承圖 4-4。寫出「繞影像中心 (cx, cy) 旋轉 θ 角」的 3×3 齊次矩陣。提示：這不能只用一個旋轉矩陣。請說明為什麼要用三個矩陣相乘，以及它們的順序。

    提示與參考解答：單純的旋轉矩陣是繞原點（影像左上角）轉，要繞中心轉必須先把中心搬到原點、轉完再搬回去，因此答案是 $M = T(cx, cy) \cdot R(\theta) \cdot T(-cx, -cy)$。順序由右往左讀：先平移 $(-cx, -cy)$ 把中心移到原點，再旋轉 θ，最後平移 (cx, cy) 搬回原位。三者缺一不可，順序也不能調換——若寫成 $R \cdot T \cdot T^{-1}$ 就等於只做了旋轉。OpenCV 的 cv2.getRotationMatrix2D(center, angle, scale) 內部做的正是這件事，它直接回傳合成後的 2×3 矩陣。這個「搬到原點、做事、搬回去」的模式在圖形學中稱為共軛轉換，繞任意點縮放、沿任意軸鏡射都用同一招。

14. 計算題：承圖 4-5。對一張影像連續做十次「旋轉 36 度」，理論上會回到原位。請說明為什麼實際結果會明顯劣化，以及正確的做法。若改成連續做十次「旋轉 36 度再轉回 −36 度」呢?

提示與參考解答：因為每一次 warpAffine 都要做一次內插，而內插本質上是加權平均，等同於一次輕微的低通濾波（第 5 章）。十次內插就是十次低通，高頻細節被逐次抹平，影像愈來愈糊，而且邊界每轉一次就被裁掉一點。正確做法是先把十個矩陣相乘得到單一矩陣（本例乘起來恰好是單位矩陣），最後只呼叫一次 warpAffine——如果合成結果真的是單位矩陣，甚至連呼叫都不必。至於「轉 36 度再轉回 –36 度」重複十次：劣化會更嚴重，因為這樣做了二十次內插。一個常見的誤解是「反向轉換可以抵銷正向轉換」——幾何上可以，資訊上不行：第一次內插丟掉的高頻，第二次內插不會還給你。這與第 3 章「取樣與量化不可逆」是同一個道理。

15. 承圖 4-6 與圖 4-7：為什麼仿射需要 3 組對應點、透視需要 4 組?請由自由度推算。若你手上有 10 組對應點，多出來的那些有什麼用?該如何使用?

提示與參考解答：每一組對應點提供兩個方程式（x 與 y 各一）。仿射有 6 個自由度，6 ÷ 2 = 3 組;透視的 3×3 矩陣有 9 個元素，但整體乘上一個常數不改變結果（齊次座標的尺度不計），所以實際自由度是 8，8 ÷ 2 = 4 組。多出來的對應點有兩個用途。其一是降低雜訊影響：點的座標本身有定位誤差，用最小平方法在所有點上求一個「整體最吻合」的解，比只用最少點數穩定得多。其二是偵測並排除錯誤：若其中有幾組是錯誤匹配（outlier），最小平方會被它們嚴重拉偏，此時要改用 RANSAC（第 13 章）——隨機取最少點數求解、統計有多少點同意這個解、重複多次取最多人同意的那個。OpenCV 的 cv2.findHomography(src, dst, cv2.RANSAC) 正是這個流程。

16. 承圖 4-9：單應性成立的兩個前提是什麼?請各舉一個「前提成立」與「前提不成立」的實例，並說明前提不成立時會看到什麼現象。

提示與參考解答：兩個前提，滿足其一即可。其一，被拍攝的內容全部位於同一個平面上（紙張、地面、牆面、棋盤格）。其二，相機只做純旋轉而不平移（原地轉動拍全景，此時深度差異不產生視差）。前提成立的實例：手機掃描 App 拉正桌上的文件、循跡機器人把地面轉成鳥瞰圖、原地旋轉拍攝的全景接圖。前提不成立的實例：在街上邊走邊拍大樓做全景——相機平移了，而場景有明顯深度（大樓、路樹、遠山各在不同距離）。現象是：你可以讓其中一個深度層對齊得很好，但其他深度層會出現明顯的錯位與鬼影，而且怎麼調參數都無法同時對齊。這不是演算法不夠好，是單一個 8 自由度的矩陣在數學上就描述不了「不同深度有不同位移」這件事——那需要第 12 章的光流或完整的三維重建。

17. 承圖 4-10：為什麼幾何轉換要用反向映射而非正向映射?正向映射的兩個問題各自在什麼情況下最嚴重?反向映射完全沒有缺點嗎?

    提示與參考解答：正向映射有兩個問題。破洞：放大時最嚴重——輸出像素比輸入多，必然有些輸出位置沒有任何輸入落上去。衝突：縮小時最嚴重——多個輸入擠到同一個輸出位置，後寫的覆蓋先寫的，等於隨機取樣（而非平均），會產生第 3 章的混疊。反向映射對每個輸出像素反算來源，保證每個輸出都有值、也不會重複寫入，兩個問題一次解決。但它不是沒有代價：其一，需要轉換矩陣可逆（所幸本章的轉換都可逆）;其二，反算出的來源座標不是整數，必須內插，而內插本身會損失高頻（圖 4-5 與圖 4-12）;其三，縮小時反向映射只取樣一個點，同樣會混疊——這正是縮小要特別用 INTER_AREA 的原因（圖 4-14）。換句話說，反向映射解決的是幾何上的完整性，取樣理論上的問題仍然要另外處理。

18. 計算題：一個像素的來源座標被反算為 (217.3, 154.8)。已知四個鄰近像素的值為 I(217,154)=100、I(218,154)=140、I(217,155)=160、I(218,155)=200。請以雙線性內插算出該像素的值，並說明計算的順序。

    提示與參考解答：令小數部分 a = 0.3（x 方向）、b = 0.8（y 方向）。先在上面那一列做一維線性內插：100 × (1 − 0.3) + 140 × 0.3 = 70 + 42 = 112。再在下面那一列做同樣的事：160 × 0.7 + 200 × 0.3 = 112 + 60 = 172。最後在兩個結果之間依 b 內插：112 × (1 − 0.8) + 172 × 0.8 = 22.4 + 137.6 = 160.0。所以該像素的值為 160。順序可以顛倒（先直後橫）而結果相同，這稱為可分離性，與第 5 章可分離濾波是同一個性質，也是雙線性內插快的原因。補充一個檢查習慣：四個權重 (1−a)(1−b)、a(1−b)、(1−a)b、ab 相加必為 1，算完可以驗算一下。

19. 承圖 4-13：放大一張只有 0 與 2 兩種值的標籤圖，用雙線性內插後會發生什麼事?為什麼這在語意分割中是嚴重錯誤?那麼「把雙線性的結果四捨五入回整數」可行嗎?

    提示與參考解答：雙線性是加權平均，在 0 與 2 的交界處會產生 0.4、1.0、1.7 這類中間值，取整後出現「1」——而標籤 1 可能代表完全不同的類別（例如 0 是背景、2 是車輛，1 卻是行人）。於是模型憑空在每個物件的邊緣繞上一圈「行人」，這在語意分割的評分（第 16 章的 IoU）中是直接的錯誤。至於四捨五入：不可行，因為標籤值是「名目尺度」而非「數值尺度」——類別 0、1、2 之間沒有大小或距

離關係，在它們之間做平均再取整，本身就沒有意義。正確做法只有一個：標籤圖一律用 INTER_NEAREST。同樣的原則適用於第 10 章的二值遮罩、第 9 章的分割結果，以及任何「數值代表身分而非數量」的影像。

20. 承圖 4-14：縮小影像時 INTER_AREA 與 INTER_LINEAR 的差別是什麼?為什麼 INTER_LINEAR 在縮小時會失效?請用第 3 章的取樣定理解釋，並說明縮小 1.5 倍與縮小 8 倍哪一種情況差別更大。

    提示與參考解答：INTER_LINEAR 不論縮放倍率，永遠只看來源座標周圍的 2×2 個像素。縮小 8 倍時，輸出的每一個像素其實代表輸入的 8×8 = 64 個像素，但雙線性只看了其中 4 個，其餘 60 個像素的資訊被直接忽略——這等同於第 3 章的「直接跳點取樣」，必然混疊。INTER_AREA 則會依縮放倍率自動把對應區域內的所有像素平均，等於內建了一個隨倍率調整的低通濾波器，先濾掉留不住的高頻再取樣，符合取樣定理。倍率愈大差別愈明顯：縮小 1.5 倍時，每個輸出像素對應 2.25 個輸入像素，雙線性看的 4 個已涵蓋大部分，差別不大;縮小 8 倍時只看了 64 分之 4，差別極大。實務上的判準是：縮小超過 2 倍就該用 AREA，或先用高斯模糊再縮。

21. 承圖 4-16：內部參數 fx、fy、cx、cy 各自的幾何意義是什麼?若把相機的解析度由 1920×1080 改成 960×540（二分之一取樣），這四個數字該怎麼換算?

    提示與參考解答：fx 與 fy 是以像素為單位的焦距，決定「同樣的物體在影像中有多大」;兩者相等代表感光元件的像素是正方形，不相等則影像在某個方向被拉長。cx、cy 是主點，也就是光軸與成像面的交點，理想上落在影像中心，實際上因組裝公差而略有偏移。解析度減半時，四個數字全部乘以 0.5：fx、fy 減半是因為同一個視角現在對應一半的像素數;cx、cy 減半是因為影像中心的像素座標也減半了。畸變係數則不變，因為它們定義在正規化座標上，與解析度無關。這是實務上極常見的陷阱：用高解析度校正、卻在低解析度上推論，忘了換算內參，結果整條管線的量測全錯而且錯得很一致，不容易發現。

22. 承圖 4-17 與 4-19：為什麼徑向畸變在畫面中心幾乎不影響、愈往邊緣愈嚴重?請由畸變模型的數學形式說明。這對「只用畫面中央區域」的設計策略有什麼啟示?

    提示與參考解答：徑向畸變的模型是把正規化座標乘上一個只與半徑有關的倍率：$(1 + k1 \cdot r^2 + k2 \cdot r^4 + k3 \cdot r^6)$，其中 r 是該點到主點的距離。在中心 r = 0，倍率恰為

1，位置完全不變;離中心愈遠 r 愈大，而且因為是 r 的平方、四次方、六次方，偏移量增長得非常快——這就是為什麼畫面四角最誇張。圖 4-19 實測的數字正是這個形狀：去畸變在中心區的平均改動量接近零，外圈卻大上數十倍。啟示是：若應用允許，把關鍵的量測區域安排在畫面中央，可以大幅降低對校正精度的依賴。這在產線設計上很實用——寧可換一顆視角較窄的鏡頭並把工件放在中央，也不要用廣角鏡硬塞進整個視野再靠軟體救。但要注意這是降低風險而非免除校正：要達到零點一毫米等級的量測，中央區域仍然需要去畸變。

23. 承圖 4-18 與 4-20：重投影誤差是怎麼算出來的?為什麼它能作為校正品質的驗收標準?若某次校正的重投影誤差是 0.08 像素，這一定是好事嗎?

提示與參考解答：計算方式是把求得的內參、畸變係數與每張照片的外參，拿去把棋盤格角點的「世界座標」重新投影回影像，再與當初偵測到的角點位置相比，取所有點的均方根距離。它之所以能當驗收標準，是因為它直接衡量「這組參數能不能解釋觀測到的資料」——參數愈接近真實，投影回去就愈吻合。但 0.08 像素不一定是好事，這題的重點在這裡。重投影誤差只衡量「在這批照片上吻合得多好」，並不保證參數本身正確。兩種常見的假好成績：其一，照片太少或方位太相似，模型有足夠自由度去遷就這幾張，等於過度擬合;其二，誤差被平均稀釋——若多數角點很準、少數幾張照片很糟，平均起來仍然好看。正確的驗收應該包含三件事：看每張照片各自的誤差而非只看總平均、確認拍攝方位涵蓋了畫面各區域與多種傾角、以及最重要的——拿一個已知尺寸的標準件實際量一次。這與第 3.5 節「指標是工具，不是目的」完全同源。

24. 承圖 4-21 與 4.6.4 節：要以視覺量測工件尺寸達到 0.1 毫米的精度，為什麼一定要先做相機校正?請估算：若未校正的廣角鏡在畫面邊緣有 5 像素的位置偏差，而每像素對應 0.05 毫米，這會造成多大的量測誤差?這個誤差有什麼特性讓它特別危險?

提示與參考解答：5 像素 × 0.05 毫米／像素 = 0.25 毫米，已經是要求精度 0.1 毫米的兩倍半，完全不可接受。若量測的是跨越畫面的長度，兩端各有偏差，誤差還會加倍。這個誤差有兩個危險的特性。其一，它是系統性的而非隨機的——多量幾次取平均完全沒有用，因為每次測量都朝同一個方向偏同樣的量。其二，它隨位置而變：同一個工件放在畫面中央量到的尺寸，與放在角落量到的不一樣。於是你會看到「同一批產品的量測值隨著它在輸送帶上的位置而漂移」這種極難除錯的現象——

> *—工程師通常會先去懷疑光源、機構、甚至產品本身，而不會想到是鏡頭。正確流程是：出廠時校正一次並存檔、每次取像後先去畸變、再以已知尺寸的標準件建立像素與公釐的換算比例，並定期用標準件複驗。*

25. (綜合設計題)你要為一台循跡機器人建立鳥瞰圖（逆透視映射）。相機裝在車體前方，俯角約 30 度，距地面 20 公分。請說明：(a) 你需要哪些資訊才能求出那個透視矩陣?有哪兩種取得方式?(b) 鳥瞰圖中「每像素代表幾公分」該怎麼決定?(c) 這套方法在哪些情況下會失效?(d) 車子行進中震動造成俯角改變 2 度，會有什麼後果?該怎麼辦?

> *提示與參考解答：本題無標準答案，評分重點在前提意識與失效分析。(a) 需要「地面上四個點在影像中的座標」與「這四個點在地面上的實際相對位置」。兩種取得方式：一是手動標定——在地面上貼四個標記、量好它們的實際間距，拍一張照後在影像上點出四個角，用 getPerspectiveTransform 求解，簡單可靠，適合相機固定的場合;二是由幾何推算——已知相機高度、俯角與內參，可直接計算地平面與影像平面之間的單應性，不必貼標記，但對安裝角度的精度要求高。實務上建議先用方法一做出可用版本，再用方法二做自動化。(b) 由你要偵測的最小特徵決定（第 3.7.3 節的思路）：若車道線寬 2 公分、希望有 4 個像素涵蓋，則每像素 0.5 公分;再由機器人的前視距離需求（例如要看到前方 1 公尺）決定鳥瞰圖的高度為 200 像素。(c) 至少四種失效情況：地面不平（上下坡、減速丘）使平面假設破裂;立體物件（其他機器人、人的腳）會被拉成長條，因為它們不在地面上;遠處區域在原影像中只佔少數像素，攤平後被嚴重放大而模糊，可用距離有上限;以及光照造成的陰影在鳥瞰圖中與車道線難以分辨。(d) 俯角改變 2 度會讓整張鳥瞰圖產生系統性的梯形偏移，遠處的誤差遠大於近處，表現為「車道線在鳥瞰圖中突然變寬或變斜」，導致控制器誤判而產生震盪。三種對策：機構上加裝減震並降低重心;軟體上對連續數影格的轉換矩陣做時間平滑;或加裝慣性感測器即時估計俯角並動態修正矩陣（這已是第 17 章感測器融合的範疇）。*

參考資料與延伸閱讀

- Zhang, Z., A Flexible New Technique for Camera Calibration, IEEE TPAMI, 22(11), 1330-1334, 2000.(技術報告版 MSR-TR-98-71, 1998 年 12 月;本章叫獸開講之史料來源)

- Zhang, Z., Flexible Camera Calibration by Viewing a Plane from Unknown Orientations, ICCV, 1999.
- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2.4(幾何轉換與內插)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2、11(投影幾何與立體視覺)。
- Hartley, R., & Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge, 2004.(進階必讀)
- OpenCV 官方教學：Camera Calibration(含可下載之棋盤格圖檔)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 5 章 空間域影像強化

本章學習目標

- 區分點運算與鄰域運算，說明空間域強化的基本框架與它在全書中的定位。
- 運用負片、對數與 Gamma 校正等灰階轉換調整影像明暗與對比，並理解 LUT 最佳化。
- 讀懂直方圖，並以直方圖等化與 CLAHE 改善對比不足的影像。
- 正確定義卷積與相關，說明濾波器核、邊界處理與可分離濾波的運算量差異。
- 比較均值、高斯、中值、雙邊四種平滑濾波的特性與適用雜訊型態。
- 以 Laplacian、Unsharp Masking 與 High-Boost 進行影像銳化，並理解其代價。

- 說明多影格合成的原理，並理解計算攝影如何以軟體突破硬體限制。

叫獸開講|手機如何在黑夜裡看見：夜景模式的祕密

2018 年 11 月，Google 為 Pixel 手機推出 Night Sight(夜視模式)。它的能力在當時令人難以置信：在照度低至 0.3 勒克斯的環境——那是連人類視覺都已退化成模糊單色的暗度——不用腳架、不用閃光燈，手持就能拍出乾淨、銳利、色彩正確的照片。負責這項技術的是 Marc Levoy，史丹佛大學計算攝影的先驅、後來成為 Google 的傑出工程師。

低光攝影為什麼困難?Levoy 團隊在技術文件中解釋得很清楚：手機的鏡頭與感光元件都很小，收集到的光子本來就少;而主要的雜訊來源是進入鏡頭的光子數量本身的隨機波動(散粒雜訊，第 3、7 章)，加上電荷轉數字時的讀取雜訊，兩者共同拉低訊噪比。物理學給的解方很單純：訊噪比隨曝光時間的平方根上升，曝光愈久畫面愈乾淨。但問題是——手持拍照，你會抖;被拍的人，也在動。長曝光換來的是一張明亮的糊照。

Google 的答案不是換更大的鏡頭，而是換一種思路：連拍一疊短曝光影像，在軟體中對齊後合併。一張張「不夠亮但夠銳利」的照片疊起來，隨機的雜訊互相抵消，亮度與訊噪比同時提升。這個「以多換好」的想法，正是本章 5.6 節要展開的多影格合成。

Night Sight 把這套做法推到極致，其中幾個設計極具巧思。其一是運動測光(motion metering)：在你按下快門之前，系統就已經在量測手抖程度與場景中的運動量，據此預測接下來的運動並自動決定每張的曝光時間——場景夠靜時，單張曝光甚至可超過 300 毫秒。其二是影格數的動態調整：若偵測到手機放在腳架上就拍 6 張，手持則最多 15 張，並限制總擷取時間以免使用者失去耐心。其三是穩健合併：採用專為高雜訊影像設計的分塊對齊策略，能丟棄運動過大的區塊，避免鬼影。其四是學習式白平衡——因為暗處的光源色溫難以判斷，他們訓練了

專門針對夜景的演算法(第 8 章會回到這個主題)。

最後一個設計尤其耐人尋味：色調映射刻意受到繪畫的啟發。團隊發現，若只是單純把夜景調亮，結果會像大白天拍的，失去夜的氛圍。因此他們仿效畫家描繪夜景的手法——提高對比、把陰影壓到近乎全黑、讓畫面周圍沉入黑暗。這是一個很好的提醒：影像強化的目標，未必是「還原真實」，有時是「傳達感受」。這正是本章「強化」與第 7 章「還原」的根本差異。

拆開來看，Night Sight 用到的技術幾乎全在本章：多張平均去雜訊(5.6 節)、對比拉伸與色調曲線(5.1 節)、局部對比增強(5.2 節)，而對齊則用到第 6 章的相位相關與第 13 章的特徵匹配。學完這一章，你就有能力自己寫一個「窮人版夜景模式」——這正是 Lab 5-4 的任務。

思考題：「連拍多張再平均」為什麼能降低雜訊?如果雜訊是隨機且零均值的，平均 N 張後雜訊的標準差會變成原來的幾分之一?(提示：與叫獸開講中「訊噪比隨曝光時間平方根上升」是同一個數學原理)

5.1 空間域處理的框架與灰階轉換

5.1.1 本章的定位

空間域處理是指直接對像素數值進行運算，相對於第 6 章在頻率域的處理。它可寫成 g(x, y) = T[f(x, y)]，其中 f 是輸入影像、g 是輸出影像、T 是作用在 (x, y) 鄰域上的運算子。當鄰域只有一個像素時，T 退化為灰階轉換函數，稱為點運算(Point Operation)：輸出只取決於該像素自己的值，與鄰居無關。當鄰域擴大到 3×3、5×5 時，就成為 5.3 節的空間濾波(鄰域運算)。

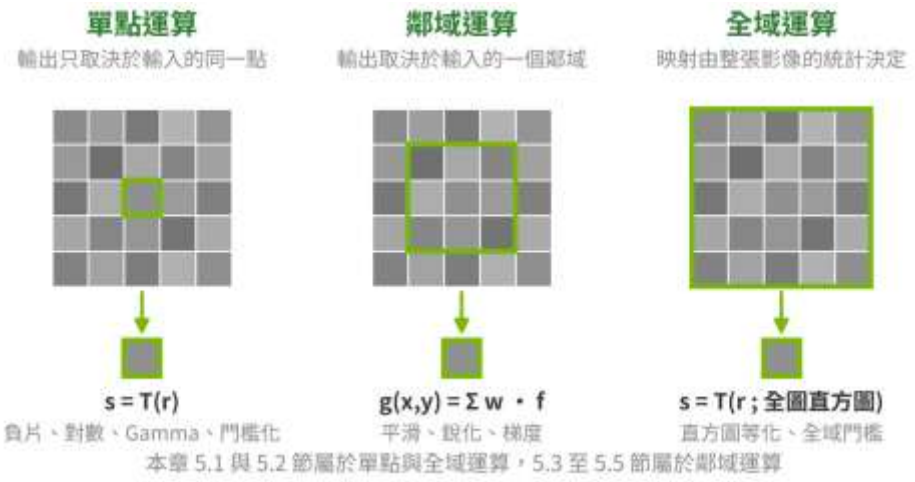


圖 5-1　空間域處理的三種作用範圍

這裡也要釐清本章與第 7 章的關係，因為兩者常被混淆。本章談的是影像強化(Enhancement)：目標是讓影像「看起來更好」或「更適合後續處理」，判準相當主觀——就像叫獸開講中 Night Sight 刻意把陰影壓黑以營造夜的氛圍，那並非還原真實，卻是更好的照片。第 7 章的影像還原(Restoration)則是客觀的逆問題：先建立劣化的數學模型，再反推最接近原始的估計。一句話總結：強化是化妝，還原是手術。

5.1.2 負片與對數轉換

負片轉換 s = L - 1 - r(8 位元時為 255 - r)把黑白顛倒。它在醫學影像中特別有用：X 光片上細小的白色病灶，反轉成黑色後在淺色背景中反而更醒目——這利用了人眼對暗背景上的亮物與亮背景上的暗物，敏感度並不相同的特性。

對數轉換 s = c·log(1 + r) 則大幅拉伸暗部、壓縮亮部，適合處理動態範圍極大的影像。最經典的例子是第 6 章的傅立葉頻譜：直流分量的數值往往比其他成分大好幾個數量級，不取對數幾乎只看得到中央一個亮點。當你在第 6 章看到「顯示頻譜前要取對數」時，用的就是這裡的工具。

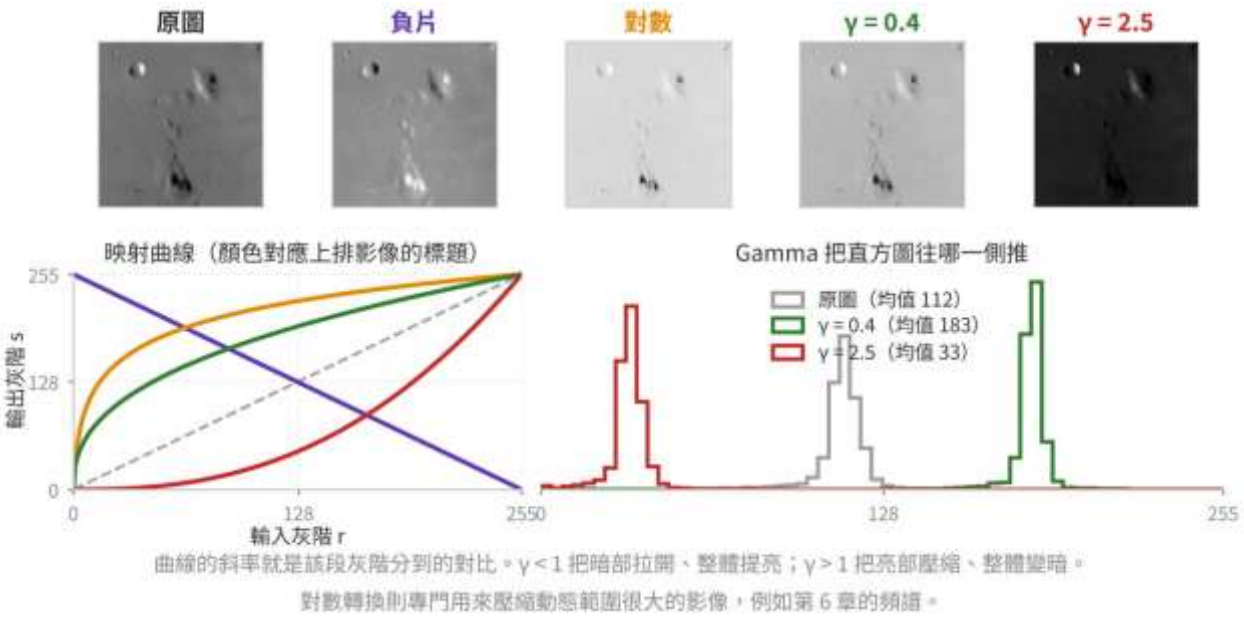


圖 5-2　四種灰階轉換的映射曲線與效果

5.1.3 Gamma 校正與冪次法則

冪次法則轉換 s = c·r^γ 是實務上最常用的明暗調整工具。γ 小於 1 時，暗部被拉亮(整體變亮、暗部細節浮現);γ 大於 1 時，暗部被壓暗(整體變暗、亮部細節保留)。

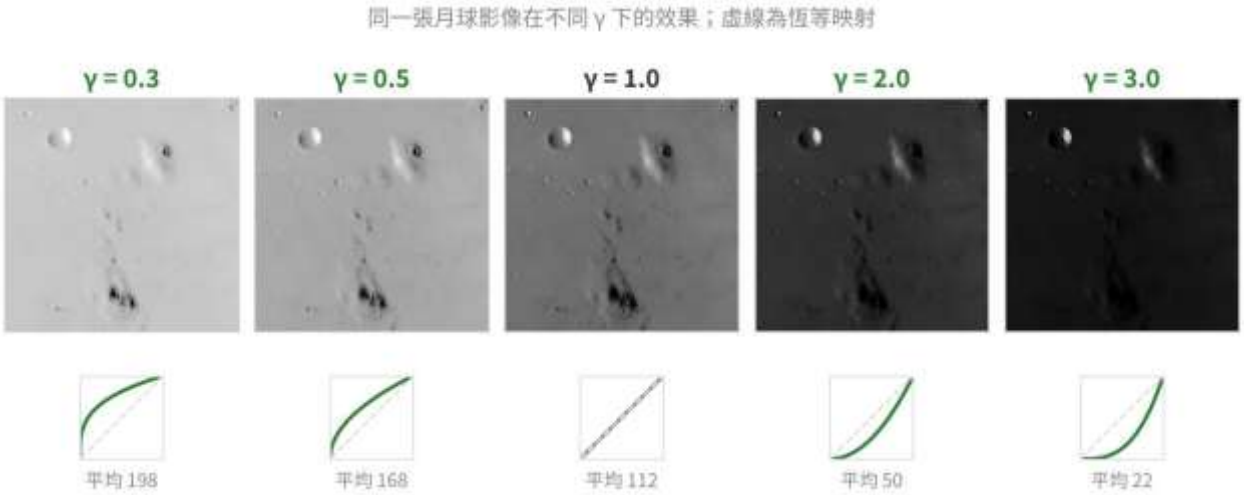


圖 5-3　Gamma 值對影像的影響

它的重要性不只在美觀，還有一段硬體的歷史淵源：早期映像管螢幕的亮度響應天生近似 γ = 2.2 的冪次曲線，因此影像在儲存時預先做了反向的 Gamma 編碼來抵消。這就是為什麼 8 位元 JPEG 的像素值並非線性正比於光強度，而第 3 章提到的 RAW 檔保留的才是線性感測值。

這個差別在做精確的光度量測時會咬人：若你要判斷「這個區域的實際亮度是那個區域的幾倍」，直接用 JPEG 的像素值相除會得到錯誤答案，必須先做反 Gamma 轉換回到線性空間。這也是第 7 章影像還原、第 8 章白平衡計算時要留意的細節——許多演算法在推導時都假設輸入是線性的。

程式 5-1 灰階轉換：負片、Gamma、對數

```
import cv2
import numpy as np

img = cv2.imread("dark_room.jpg")

# --- 負片 ---
negative = 255 - img

# --- Gamma 校正:用查表(LUT)一次算好 256 個對應值,最快 ---
def gamma_correct(img, gamma):
    inv = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv) * 255
                      for i in range(256)]).astype("uint8")
    return cv2.LUT(img, table)

brighter = gamma_correct(img, 2.2)   # gamma>1 → 提亮暗部
darker   = gamma_correct(img, 0.45)  # gamma<1 → 壓暗

# --- 對數轉換(常用於顯示頻譜,第 6 章) ---
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)
log_img = 255 * np.log1p(gray) / np.log1p(255)
log_img = log_img.astype(np.uint8)

# --- 反 Gamma:回到線性空間再做光度計算(第 7、8 章需要) ---
linear = np.power(img.astype(np.float32) / 255.0, 2.2)
```

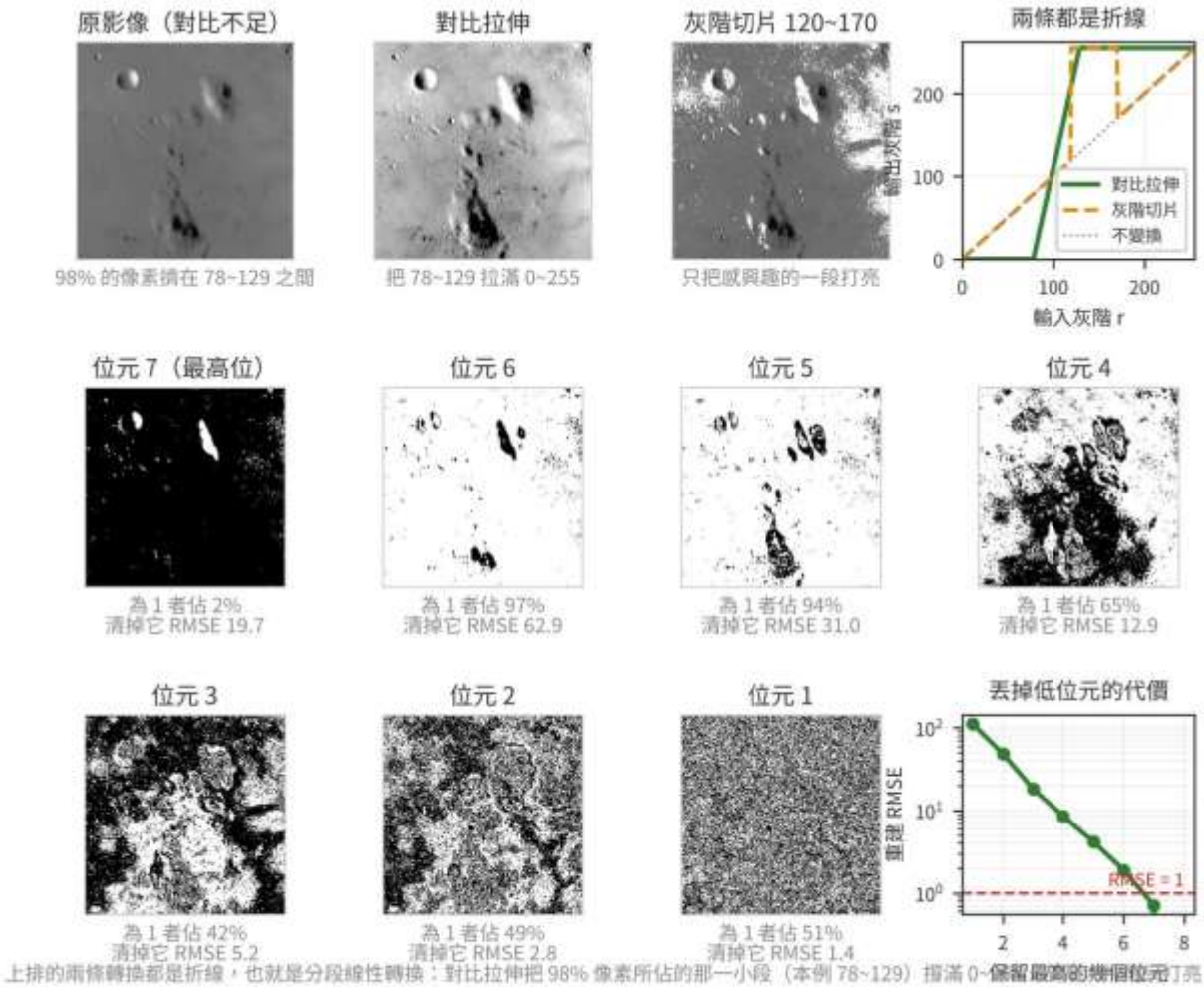
請注意程式中的 cv2.LUT()：灰階轉換只有 256 種可能的輸入，先建好查表再一次對映，遠比對每個像素重算冪次快——這正是第 2 章「向量化思維」的延伸，也是影像處理最常見的最佳化手法之一。在第 17 章的樹莓派或 Jetson 等邊緣裝置上，這類最佳化尤其值得。

5.1.4 分段線性轉換與位元平面切片

前面三種轉換——負片、對數、Gamma——都是一條平滑的曲線。但實務上還有一整類轉換長得完全不同：它們是折線，由使用者自己指定幾個轉折點。這一類叫

做分段線性轉換(Piecewise Linear Transformation)。它的優點是完全可控：你想把哪一段灰階拉開、哪一段壓扁，直接畫出來就好，不必去湊一個數學函數;缺點則是參數變多，而且轉折處會有不連續的視覺跳躍。

最常用的兩種分段線性轉換如下。第一是對比拉伸(Contrast Stretching):找出影像實際用到的灰階範圍，把它線性撐滿整個 0 到 255。實務上不直接取最小值與最大值，而是取第 2 與第 98 百分位——因為少數幾個雜訊點或死像素就足以把最小值與最大值拉到兩端，使拉伸完全失效。圖 5-4 的月球影像就是典型：它的最小值是 0、最大值是 255，看起來已經用滿了整個範圍，但 98% 的像素其實擠在 78 到 129 之間，所以它看起來一片灰濛濛。改用百分位之後，對比立刻拉開。



上排的兩條轉換都是折線，也就是分段線性轉換：對比拉伸把 98% 像素所佔的那一小段（本例 78~129）撐滿 0~255，把你關心的那一段打亮。中下排把同一張影像拆成八個位元平面。請注意一個反直覺的結果：這張月球影像的像素幾乎都落在 78~129，所以最高位元（位元 7）只有 2.4% 的像素是 1，單獨清掉它的 RMSE 是 19.7，反而比清掉位元 6 的 62.9 還小。「高位元一定比較重要」只在像素填滿整個動態範圍時才成立。右下角是累積的代價：只留最高 4 個位元，RMSE 為 8.5;要壓到 1 以下得留 7 個位元。

圖 5-4　分段線性轉換與位元平面切片：資訊並不是平均分布在八個位元上

第二是灰階切片(Gray-level Slicing):只把某一段灰階打亮，其餘保持原樣（或壓暗）。這在醫學與工業檢測中很常見——例如 CT 影像中骨骼落在某個 HU 區間，把那一段挑出來就等於做了一次粗略的分割。事實上，若把「其餘一律壓到 0」這個極端版本寫下來，它就是第 9 章的門檻法。換句話說，灰階切片是連接「影像強化」與「影像分割」的那一步:同樣一條折線，目的從「看得更清楚」變成「把東西切出來」。

還有一種特別的切法叫位元平面切片(Bit-plane Slicing):把 8 位元的灰階影像，按照二進位的每一位拆成八張二值影像。這件事之所以值得做，是因為它讓你看見「資訊藏在哪裡」。一般的說法是「高位元帶走結構，低位元只剩雜訊」，圖 5-4 的實測大致支持這個說法——單獨清掉位元 1 只造成 RMSE 1.4，而清掉位元 6 造成 RMSE 62.9，差了四十餘倍。

但同一張圖也揭露了這句話的例外，值得特別留意。這張月球影像的像素幾乎都落在 78 到 129 之間，而 128 正好是位元 7 的門檻，所以全圖只有 2.4% 的像素其位元 7 是 1。結果是：單獨清掉最高位元只造成 RMSE 19.7，反而比清掉位元 6 的 62.9 小得多。「最高位元最重要」這個直覺，只在像素分布填滿整個動態範圍時才成立;一旦影像偏居一隅，最重要的那一位就會往下移。這也反過來說明了為什麼對比拉伸值得做——它不只是讓影像好看，更是讓後續的每一個位元都真正攜帶資訊。

位元平面切片在工程上有三個直接的用途。其一是壓縮的直觀理解：圖 5-4 右下角量到，只保留最高 4 個位元，重建 RMSE 為 8.5，要壓到 1 以下必須保留 7 個位元——這就是第 11 章談量化與位元分配時的具體依據。其二是數位浮水印：把訊息藏在最低位元，因為改動它造成的 RMSE 只有 0.7，人眼完全看不出來。其三是感測器診斷：若某一個位元平面出現規則的條紋或固定圖樣，那通常是 ADC 或資料傳輸出了問題，而不是場景本身的紋理。

5.2 直方圖處理

5.2.1 讀懂直方圖

影像直方圖統計每個灰階值出現的像素個數，是影像的「亮度身分證」。曝光不足的影像，直方圖擠在左側;過曝擠在右側;對比不足則集中在中間一小段;對比良好的影像，分布通常較為寬廣。



圖 5-5　四種曝光狀況的影像與直方圖

直方圖是影像分析的第一個診斷工具，它的用途貫穿全書：攝影師看它判斷曝光;工程師據此決定該做什麼處理;第 7 章用平坦區域的直方圖形狀來辨識雜訊型態(鐘形是高斯、兩端尖峰是椒鹽);第 9 章的 Otsu 門檻分割更是直接建立在直方圖之上——它假設前景與背景各形成一個峰，並自動找出最佳的分界點。養成「拿到影像先看直方圖」的習慣，你會少走很多彎路。

5.2.2 直方圖等化

直方圖等化(Histogram Equalization)的目標是讓灰階分布盡可能均勻，以擴展對比。其核心是把累積分布函數(CDF)當作轉換函數：先統計各灰階的機率 $p(r_k) = n_k / MN$，再計算累積機率並乘上 $L-1$ 得到映射值 $s_k = (L-1)\cdot\Sigma\, p(r_j)$。

直觀理解：某個灰階若聚集大量像素，CDF 在該處陡升，映射後鄰近灰階被拉開，對比自然增強;像素稀少的區段則被壓縮。換句話說，等化是把「有限的灰階資源」分配給「像素多的地方」——這與第 8 章色度降取樣、第 11 章 JPEG 量化把資源分配給人眼在意之處，是同一種資源分配的思維。

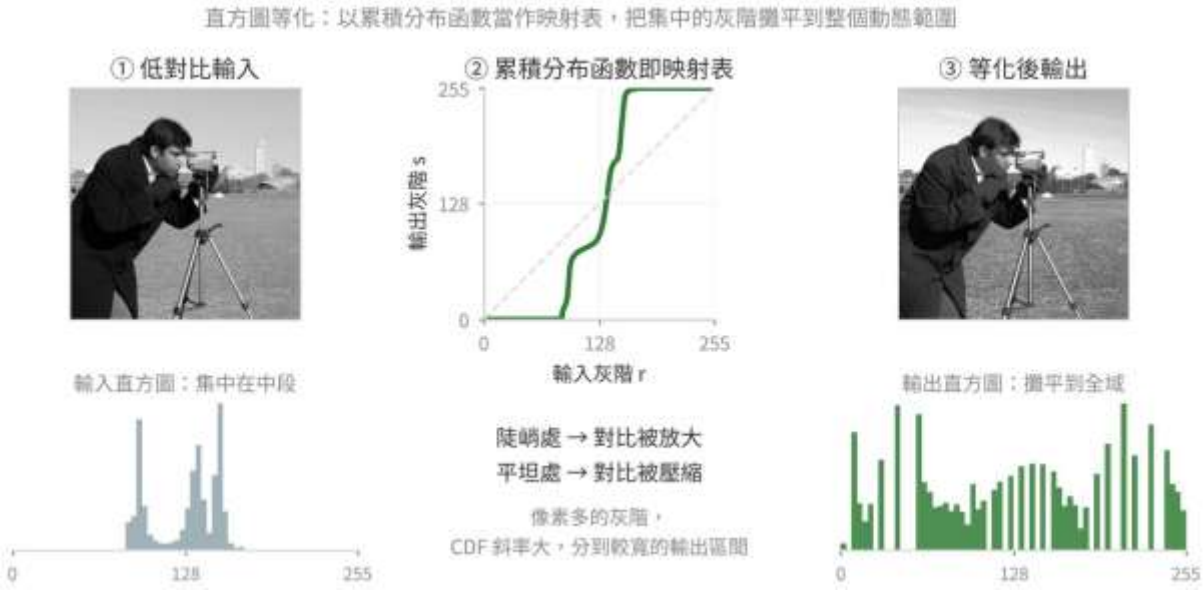


圖 5-6　直方圖等化的三個步驟

手算範例(L = 8，共 64 像素)：設灰階 0 至 7 的像素數為 8、10、16、14、8、5、2、1。累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000;乘以 7 並四捨五入，得到映射 0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。原本擠在中低灰階的像素被推展到整個範圍，對比明顯提升——這就是等化的全部祕密。

但等化也有代價：它是「全自動、不可控」的，而且會放大平坦區域的雜訊(因為雜訊造成的微小灰階差異也被拉開了)。此外，映射後某些原本不同的灰階可能合併成同一階(如上例的 5、6、7 都映射到 7)，造成細節喪失。這解釋了為什麼實務上很少直接用全域等化，而多採用下一節的 CLAHE。

5.2.3 CLAHE：局部才是王道

全域等化有個致命弱點：它用整張影像的統計量決定映射。若影像同時有明亮窗

戶與陰暗室內，等化結果往往顧此失彼——這正是第 8 章「洋裝效應」與第 9 章打光不均問題在灰階世界的翻版。

CLAHE(對比度受限的自適應直方圖等化)的解法有二：其一，把影像切成小格(例如 8×8)，各自等化後再以雙線性內插消除格線接縫，如此亮區與暗區各自依自己的統計量調整;其二，設定對比限制(clipLimit)，把直方圖中過高的長條「削頂」並把超出的部分均攤到其他灰階，避免雜訊被暴力放大——這個「削頂」正是 CLAHE 名稱中 Contrast Limited 的由來，也是它優於一般自適應等化的關鍵。

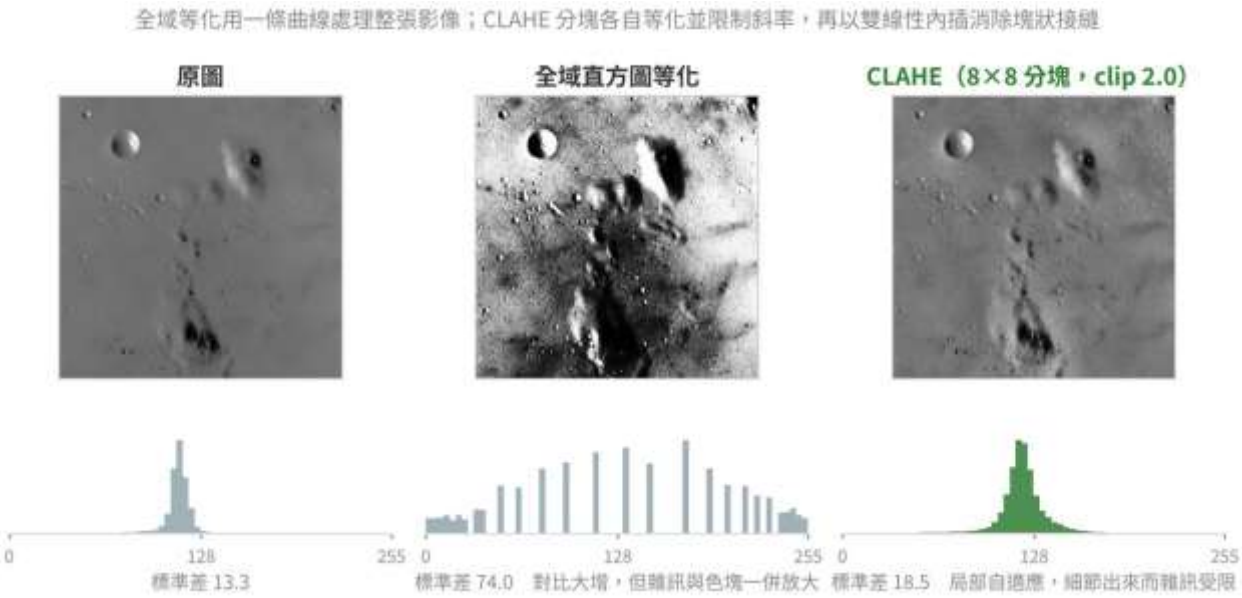


圖 5-7　全域直方圖等化與 CLAHE 的比較

實務上，CLAHE 幾乎總是比全域等化好用，是內視鏡影像、AOI 檢測、夜間監控的常客。本書後續多章的前處理(第 9 章分割前、第 16 章訓練資料準備)都會用到它。

程式 5-2 直方圖診斷、等化與 CLAHE

```
import cv2, numpy as np

gray = cv2.imread("lowcontrast.jpg", cv2.IMREAD_GRAYSCALE)

# 先看直方圖:診斷是第一步
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
print("最亮/最暗像素值:", gray.max(), gray.min())
print("集中區間佔比:", (hist[80:160].sum() / gray.size))

eq = cv2.equalizeHist(gray)                    # 全域等化
```

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
cl = clahe.apply(gray)                          # 局部等化(建議)

# --- 彩色影像的正確做法:只等化亮度通道,別碰色彩 ---
bgr = cv2.imread("lowcontrast_color.jpg")
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)      # L=亮度, a,b=色彩
l, a, b = cv2.split(lab)
l = clahe.apply(l)
result = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

# 錯誤示範:對 BGR 三通道分別等化 → 色偏
wrong = cv2.merge([cv2.equalizeHist(c) for c in cv2.split(bgr)])
```

程式最後兩段是常見考點：若對 BGR 三個通道分別等化，各通道的映射函數不同，色彩比例會被破壞，產生詭異的色偏。正確做法是轉到亮度與色彩分離的色彩空間(LAB 或 YCbCr)，只處理亮度通道——第 8 章會完整說明這些色彩空間的設計理由。

5.2.4 推導：為什麼累積分布函數剛好是對的映射

上一小節說「用累積分布函數當映射表」，但為什麼偏偏是它?這件事在連續的情形下可以嚴格證明，值得花兩段把它看清楚，因為同一套「以累積分布做變數變換」的手法，在第 14 章的機率校正與第 15 章的資料正規化中還會再出現。

設輸入灰階 r 已正規化到 [0, 1]，其機率密度為 p(r)。定義映射 $s = T(r) = \int_0^r p(w)\,dw$，也就是 r 的累積分布函數。由微積分基本定理，$dT/dr = p(r)$。再由機率密度的變數變換公式，輸出的密度為 $q(s) = p(r) \cdot |dr/ds| = p(r) / p(r) = 1$，對所有 $s \in [0, 1]$ 都成立——輸出正是均勻分布。換句話說，不論輸入的直方圖長什麼樣，只要拿它自己的累積分布去映射，輸出的直方圖就是平的。本書以四十萬個樣本的 Beta 分布實測驗證：映射後二十個等寬區間的佔比全為 0.0500，與理論值分毫不差。

但數位影像是離散的，這個完美的結論會打折扣，原因有二。其一，灰階只有 256 階，映射後必須四捨五入回整數，原本不同的輸入灰階可能被捨入到同一個輸出灰階——合併之後就再也分不開，所以輸出的直方圖只會「盡量平」而不可能全平。其二，由此可推得一個常被忽略的性質：等化後影像的相異灰階數，永遠不會超過等化前的相異灰階數。等化能拉開對比，卻不會無中生有地創造出新的灰階層次——這

也是為什麼對一張本來就只有少數幾階的影像做等化，結果會是刺眼的色塊而不是柔順的漸層。

5.2.5 直方圖匹配：指定你要的分布

直方圖等化只有一個目標：把分布拉成均勻。但「均勻」未必是你要的。有時你想要的是「跟昨天那批影像長得一樣」，有時你想要的是一個中間亮、兩端少的常態分布。直方圖匹配(Histogram Matching，又稱直方圖規定化 Specification)就是把等化推廣一步：由你指定目標分布，演算法負責湊過去。

作法完全建立在 5.2.4 節的推導之上，只是用了兩次。設來源影像的累積分布為 F，目標分布的累積分布為 G。第一步，把來源用它自己的 CDF 等化：$s = F(r)$，此時 s 是均勻分布的。第二步，問「目標分布中，哪一個灰階的累積機率等於 s」，也就是取反函數 $z = G^{-1}(s)$。兩步合起來就是 $z = G^{-1}(F(r))$。實作上不需要真的求反函數——只要把 G 的 256 個累積值排成一張表，對每一個 s 用二分搜尋找最接近的位置即可，整個對照表 256 筆，建完之後同樣用 cv2.LUT 一次套用。

圖 5-8 用一個指定的常態分布（中心 128、標準差 45）做了實測。衡量「有多接近目標」用的是 KS 距離，也就是兩條累積分布曲線的最大垂直差距。來源影像的 KS 距離是 0.495;做完等化之後降到 0.252——它確實改善了，但它追的是均勻分布而不是這條常態，所以只能走到一半;做直方圖匹配則降到 0.067，是等化的四分之一。從 CDF 曲線圖上看得更清楚：等化把曲線拉成一條直線，匹配則把它貼到你指定的那一條上。

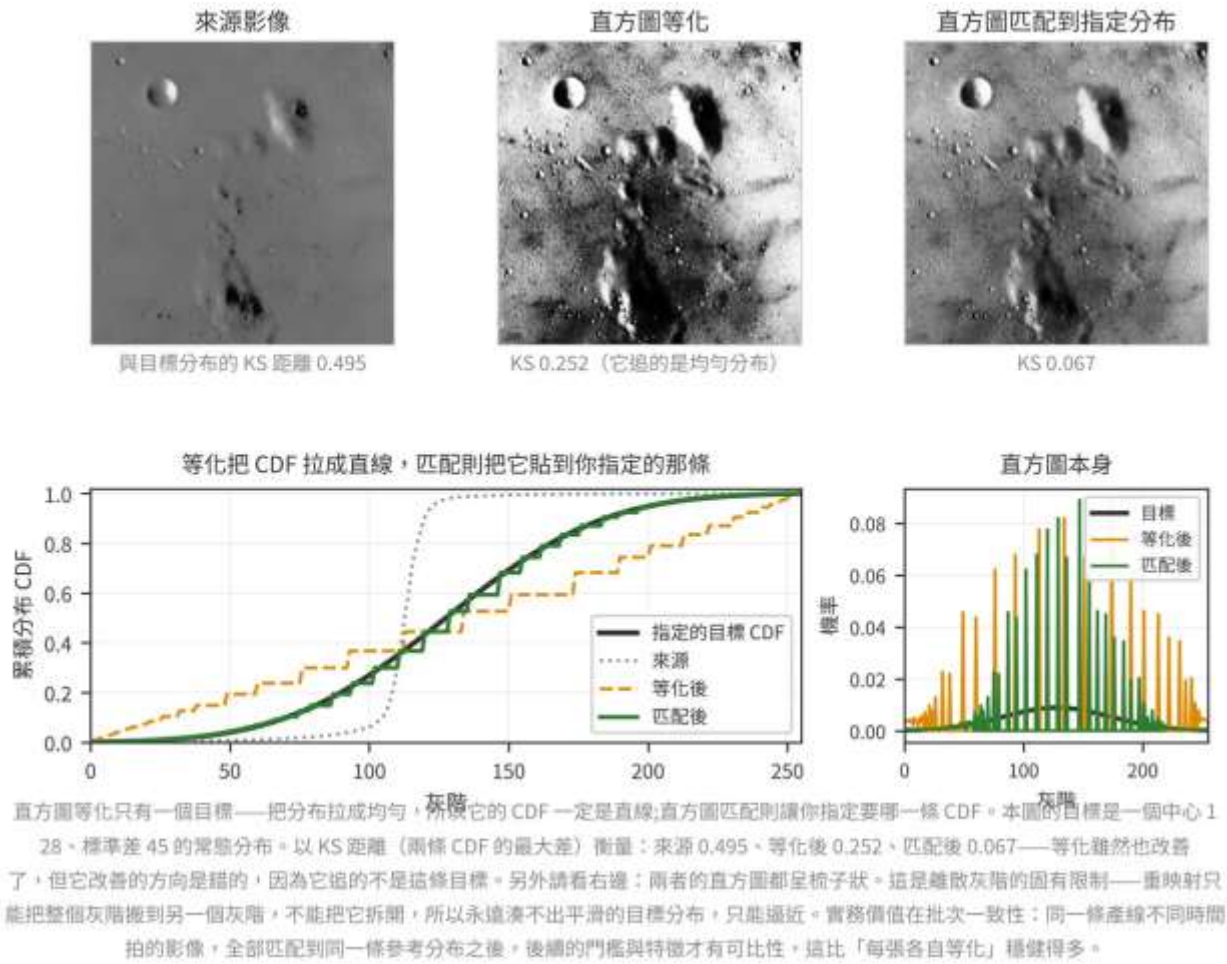


圖 5-8　直方圖匹配：等化把 CDF 拉成直線，匹配則貼到你指定的那條

同一張圖還暴露了一個等化與匹配共同的限制，值得一併記住。請看右側的直方圖：等化後與匹配後的分布都呈現梳子狀，一根一根的尖峰之間是空的。原因是灰階是離散的——重映射只能把「原本是 100 的像素」整批搬到「現在是 137」，不能把它們拆成一半去 136、一半去 138。所以輸出的直方圖永遠是來源直方圖的搬運結果，只能逼近目標而不可能真的等於目標。這也解釋了 5.2.4 節末尾那句話：對一張本來就只有少數幾階的影像做等化，得到的是刺眼的色塊而不是柔順的漸層。

直方圖匹配最有價值的場合是批次一致性。想像一條產線，上午與下午的環境光不同，拍出來的影像整體偏亮或偏暗。若每一張各自做等化，每張的映射表都不一樣，等化後的灰階值彼此不可比，下游用固定門檻做判斷就會失準。改成「全部匹配到同一張參考影像的分布」，所有影像就被拉到同一個基準上，門檻、特徵、甚至第 15 章

的模型輸入都有了可比性。這與第 8 章用灰卡做白平衡是同一個思路：與其讓每一張各自聰明，不如讓全部對齊到同一個已知的標準。

5.3 空間濾波：卷積與相關

5.3.1 定義與差異

空間濾波把一個小矩陣(稱為濾波器核、遮罩或 kernel)滑過影像，在每個位置與鄰域對應相乘後加總，作為輸出像素值。



圖 5-9　空間濾波的逐格計算

這裡有兩個容易混淆的名詞：相關(Correlation)是直接對應相乘相加;卷積(Convolution)則先把核旋轉 180 度再做相同運算。對於對稱核(如高斯)，兩者結果完全相同，所以日常常被混用;但對非對稱核就會差一個翻轉。理論上要求卷積的原因是它滿足交換律與結合律，並對應第 6 章的卷積定理。實務提醒：OpenCV 的 cv2.filter2D 執行的其實是相關，需要真正卷積時得自己先翻轉核。



圖 5-10　相關與卷積的差別：核的 180° 翻轉

一個值得先建立的連結：第 15 章的卷積神經網路，其「卷積層」做的正是這件事——差別在於，本章的核是人設計的(高斯核、Sobel 核)，而 CNN 的核是從資料中學出來的。當你在第 15 章看到 Conv2D 時，請記得它的數學核心就在這一節。

5.3.2 邊界處理

核滑到影像邊緣時，一部分會伸出界外，必須決定「界外的值是什麼」。常見策略：補零(BORDER_CONSTANT，簡單但邊緣會變暗)、複製邊緣像素(BORDER_REPLICATE)、鏡射(BORDER_REFLECT,OpenCV 預設之一，通常視覺效果最自然)、環繞(BORDER_WRAP)。



圖 5-11　五種邊界處理方式的效果

這個選擇在單張照片上不痛不癢，但在三種情況下會造成問題：拼接時(第 13 章)

接縫處會出現亮度不連續;批次分塊處理大圖時(第 17 章 AOI 大圖分割檢測)每塊的邊緣都有瑕疵;以及第 6 章的頻率域處理——DFT 隱含「影像週期性延拓」的假設，等同於 BORDER_WRAP，這會在頻譜上產生十字狀的假亮線。理解邊界處理，能幫你診斷許多莫名其妙的邊緣瑕疵。

5.3.3 可分離濾波與運算量

若一個 m×m 的核可以拆成一個垂直向量與一個水平向量的外積，就稱為可分離(Separable)。此時先做一次水平一維卷積、再做一次垂直一維卷積，結果與二維卷積相同，但每個像素的乘法次數從 m^2 降為 2m。

以 15×15 的高斯核為例：225 次乘法降為 30 次，快了七倍以上;若是 51×51，則從 2601 降為 102，快了二十五倍。高斯核與均值核都是可分離的，OpenCV 的 GaussianBlur 內部就這麼做。

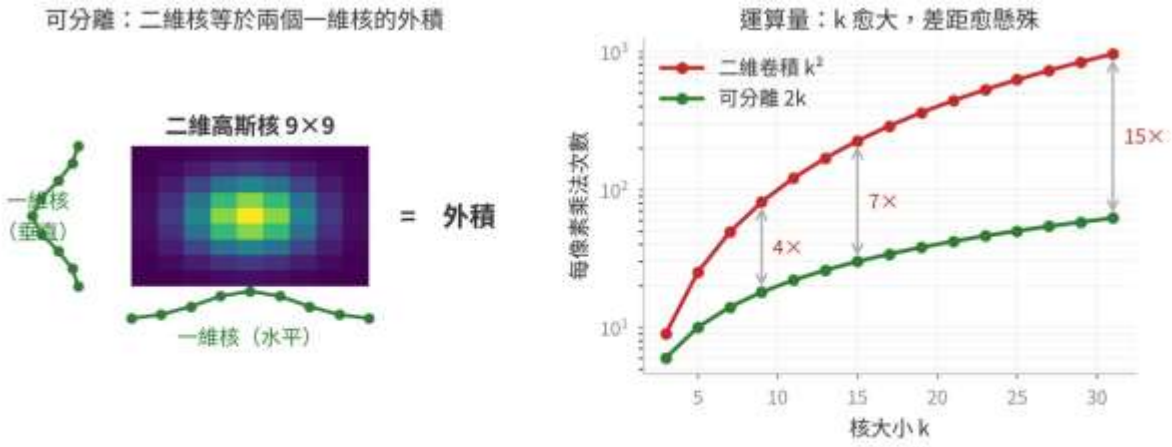


圖 5-12　可分離濾波：原理與運算量

這個技巧是即時影像處理能在 Jetson 或手機上跑得動的關鍵之一，也預告了第 6.5.2 節的工程判準：當核大到連可分離都不划算時，就該轉到頻率域了。三者(空間域直接卷積、可分離卷積、頻率域相乘)構成一個依核大小選擇的完整光譜。

5.3.4 推導：高斯核為什麼可分離，中值為什麼不行

可分離不是巧合，而是高斯函數的一個代數性質。二維高斯核為 $G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$。由於指數相加等於指數相乘，可以直接拆開：$\exp(-(x^2 + y^2) / 2\sigma^2) = \exp(-x^2 / 2\sigma^2) \cdot \exp(-y^2 / 2\sigma^2)$，因此 $G(x, y) = g(x) \cdot g(y)$，其中 g 是一維高斯。二維核恰好是兩個一維核的外積。

有了這個分解，卷積就能拆成兩次一維運算：$(f * G)(x, y) = \Sigma_v g(v) \cdot [\Sigma_u f(x - u, y - v) \cdot g(u)]$。括號內是一次水平方向的一維卷積，括號外是一次垂直方向的一維卷積。原本每個像素要做 k^2 次乘法，現在只要 $k + k = 2k$ 次。本書以 9×9 的高斯核實測，二維卷積與可分離實作的輸出最大差異為 5.6×10^{-16}，純粹是浮點誤差——兩者在數學上完全等價。均值核同樣可分離，因為所有權重都是 $1/k^2 = (1/k) \times (1/k)$，也是一個外積。

中值濾波則不可分離，而且這不是「還沒有人想到好方法」，而是原理上就不行。中值是一個排序統計量，先取各列的中值再取這些中值的中值，一般並不等於整個鄰域的中值。舉一個 3×3 的反例：[[1,5,9],[8,8,0],[0,2,0]]，九個數排序後的中值是 2;但三列的中值分別為 5、8、0，再取中值得到 5——答案不同。這就是為什麼 cv2.medianBlur 在核變大時速度掉得特別快，也是實務上中值濾波很少使用大核的原因。

5.4 平滑濾波：對付雜訊

1. 均值濾波(Box Filter)：鄰域取算術平均。最快最簡單，但邊緣與細節同樣被抹平，且方形核容易產生方向性偽影(從第 6 章的觀點看，它的頻率響應是有振盪旁瓣的 sinc 型低通，這正是偽影的來源)。
2. 高斯濾波(Gaussian Blur)：以高斯函數加權，中心權重最大、離得愈遠愈小。平滑效果自然，是最常用的平滑濾波;參數為核大小與標準差 sigma,sigma 愈大愈模糊。它同時是第 9 章 Canny 邊緣偵測的第一步、第 11 章小波多尺度與第 13 章 SIFT 尺度空間的基礎——高斯是整本書出現最頻繁的濾波器。

3. 中值濾波(Median Blur)：取鄰域的中位數而非平均。對椒鹽雜訊(隨機黑白點)有奇效——因為極端值不會影響中位數;且比均值更能保留邊緣銳利度。缺點是運算較慢，且大核心會讓影像產生塊狀感。它是非線性運算，因此沒有對應的頻率響應(第 6.6.2 節會說明這個限制的意義)。
4. 雙邊濾波(Bilateral Filter)：同時考慮「空間距離」與「像素值差異」兩個權重，只平均那些「離得近且顏色也相近」的鄰居，因此能在去雜訊的同時保住邊緣。代價是運算量大;它正是手機美顏「磨皮但不糊掉五官」的原理，也是第 13 章車牌偵測前處理的常用選擇。

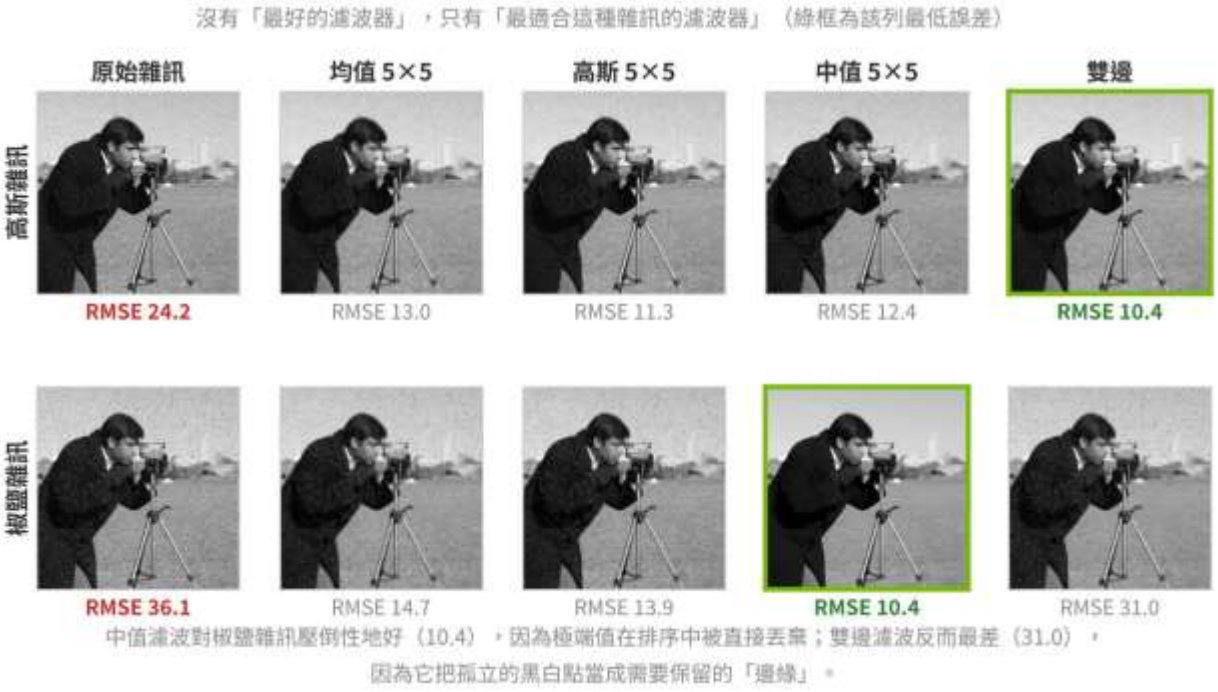


圖 5-13　四種平滑濾波對兩種雜訊的表現

濾波器	擅長的雜訊	邊緣保留	速度	OpenCV 函式
均值	高斯雜訊(普通)	差	最快	cv2.blur
高斯	高斯雜訊(佳)	差	快	cv2.GaussianBlur
中值	椒鹽雜訊(極佳)	中等	中等	cv2.medianBlur
雙邊	高斯雜訊(佳)	優異	慢	cv2.bilateralFilter

表 5-1 四種平滑濾波比較

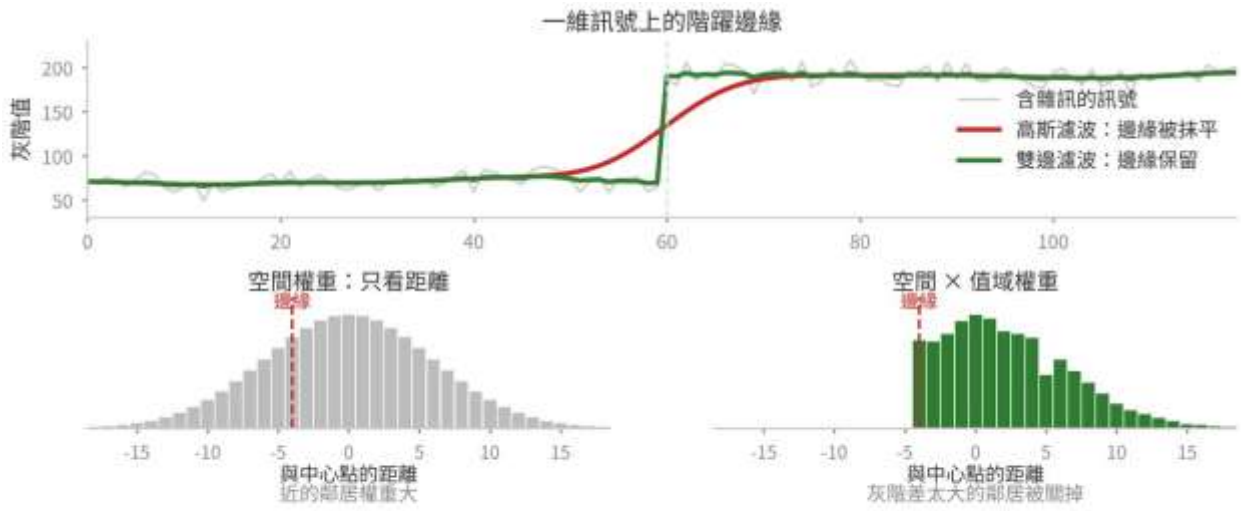


圖 5-14　雙邊濾波為什麼能保留邊緣

程式 5-3 四種平滑濾波與效能比較

```
import cv2, numpy as np

img = cv2.imread("noisy.jpg")

mean   = cv2.blur(img, (5, 5))
gauss  = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)
median = cv2.medianBlur(img, 5)              # 核大小須為奇數
bil    = cv2.bilateralFilter(img, d=9,
                             sigmaColor=75, sigmaSpace=75)

# 實驗建議:在原圖上分別加高斯雜訊與椒鹽雜訊,
# 用第 3 章的 cv2.PSNR 量化四種濾波的還原成效
clean = cv2.imread("clean.jpg")
for name, out in [("均值", mean), ("高斯", gauss),
                  ("中值", median), ("雙邊", bil)]:
    print(f"{name}: PSNR = {cv2.PSNR(clean, out):.2f} dB")

# 驗證可分離濾波的效能(5.3.3 節)
import time
k = 31
t0 = time.time(); cv2.GaussianBlur(img, (k, k), 0)
print(f"高斯(可分離){time.time()-t0:.4f} 秒")
kernel = np.ones((k, k), np.float32) / (k * k)
t0 = time.time(); cv2.filter2D(img, -1, kernel)
print(f"filter2D(二維){time.time()-t0:.4f} 秒")
```

5.4.1 推導：中值濾波的崩潰點，以及為什麼均值救不了椒鹽

本節說「中值濾波對椒鹽雜訊有奇效，因為極端值不會影響中位數」。這句話對，但它其實有一個精確的名字與一個精確的數字，而且那個數字可以直接告訴你「這張

影像還有沒有救」。

統計學把這個性質叫做崩潰點(Breakdown Point):一個估計量最多能被多少比例的壞樣本污染，而仍然維持在合理的範圍內。平均值的崩潰點是 0%——把 25 個鄰居中的任何一個改成 255，平均值就被拉走 (255 – 原值)/25，只要那個壞樣本夠極端，平均值可以被推到任何地方。中位數的崩潰點則是 50%:只要超過一半的鄰居仍然是好的，排序之後的正中央那一個必定落在好樣本的範圍內，不管壞樣本有多極端、有多少個（只要不過半）。這就是「極端值不會影響中位數」的嚴格版本。

把這個 50% 從「單一視窗」推廣到「整張影像」，就得到一個可以驗證的預測。設椒鹽雜訊的污染率為 p，濾波核為 5×5，共 25 個樣本。每個鄰居獨立地以機率 p 變成壞樣本，所以一個視窗中的壞樣本數服從二項分布 B(25, p)。該視窗失效的條件是壞樣本超過一半，也就是大於 12 個。於是「失效視窗的比例」就是 P(X > 12)，X ~ B(25, p)。當 p = 0.5 時，由二項分布對 12.5 的對稱性，這個機率恰好是 50%——也就是說，污染率一旦達到一半，就有一半的視窗會給出錯誤的答案。

圖 5-15 把這條理論曲線與實測的 RMSE 畫在一起，兩者完全吻合。污染率 40% 以前，失效視窗不到 15%，RMSE 幾乎不動（由 10.1 緩升到 13.5）;到 50% 時 RMSE 為 19.3;越過 50% 之後失效視窗急速增加，RMSE 在 60% 時跳到 31.9。對照組更能說明問題：同樣是 20% 的污染率，中值濾波的 RMSE 只有 11.2，均值濾波已經是 23.2——均值不但沒有去掉雜訊，還把每一個黑白點抹成一片灰斑，把污染從一個像素擴散到整個鄰域。

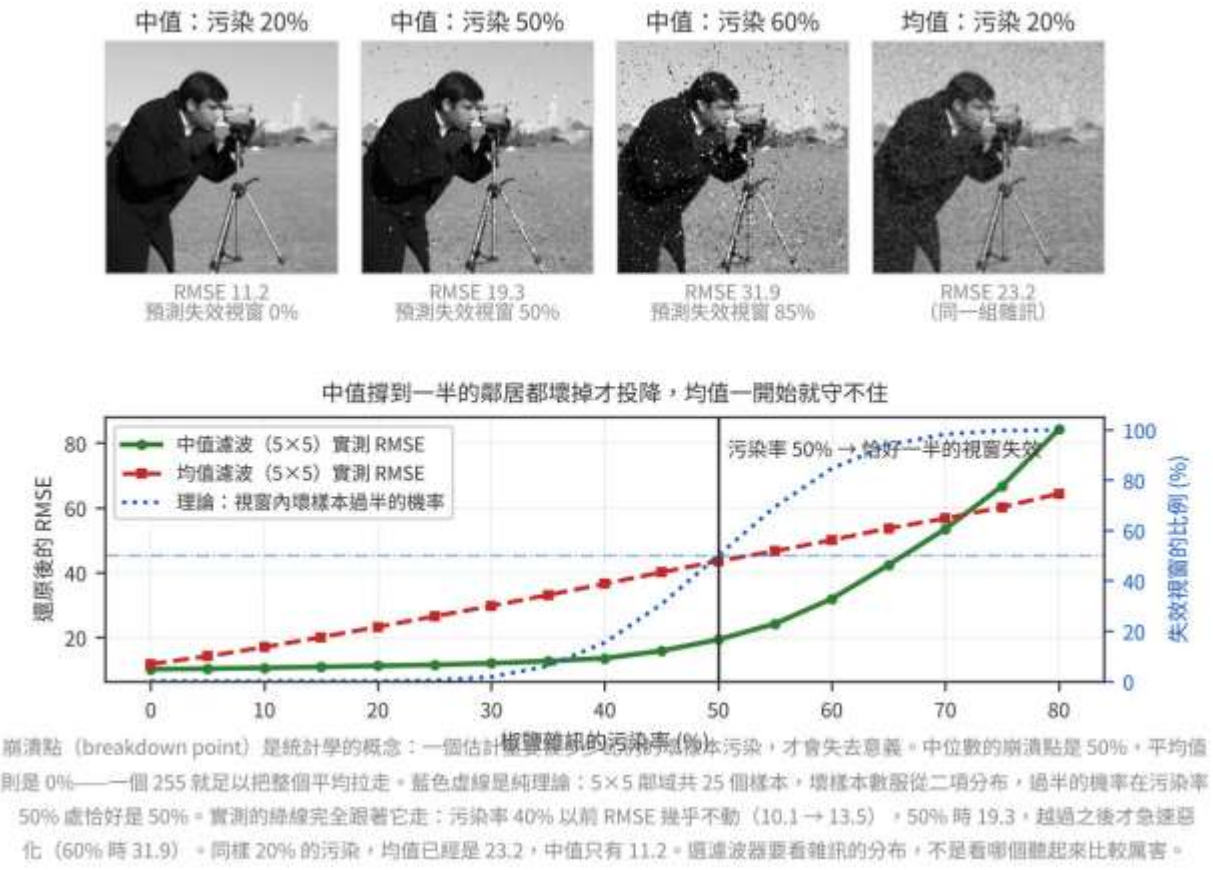


圖 5-15　中值濾波的崩潰點：理論曲線與實測 RMSE 完全吻合

這個推導帶出三個實務判斷。第一，核大小不是愈大愈好。崩潰點 50% 與核大小無關（中位數的性質），但大核會讓更多不相干的像素參與排序，模糊掉真正的細節，所以應該選「剛好能跨過雜訊團塊」的最小核。第二，若污染率真的超過 50%，換更大的核或做兩次中值都救不回來——那時該做的是回頭檢查感測器或傳輸線路，而不是加強演算法。第三，也是最重要的：選濾波器要看雜訊的分布形狀，不是看哪個名字聽起來厲害。椒鹽雜訊是離群值問題，要用穩健估計量（中位數）;高斯雜訊沒有離群值，平均反而是最佳的線性估計。第 7 章會把「先診斷雜訊型態、再選方法」這條原則完整展開。

5.5 銳化濾波：強調細節

平滑靠平均，銳化則靠微分：影像中的邊緣就是灰階劇烈變化之處，而微分對這種變化最敏感。這也預告了第 9 章——邊緣偵測用的是同一套數學，只是目的從「強

調邊緣」變成「找出邊緣的位置」。

5.5.1 Laplacian 銳化

拉普拉斯運算子(Laplacian)是二階微分的離散近似，常見核為中心 -4(或 -8)、四鄰為 1 的 3×3 矩陣。單獨使用 Laplacian 得到的是「邊緣圖」，把它加回原圖才是銳化：$g = f - c\cdot\nabla^2 f$。



圖 5-16　Laplacian 銳化的四個步驟

二階微分的特性是各向同性(不分方向)、定位精準，但對雜訊極度敏感——因為微分兩次會把雜訊放大得更兇。這正是第 9.3.2 節 LoG(高斯拉普拉斯)要先做高斯平滑的原因，也是本節結尾「先去雜訊再銳化」原則的根據。

5.5.2 Unsharp Masking 與 High-Boost

更常用的路線是遮罩銳化(Unsharp Masking)，名字有點反直覺——它其實來自傳統暗房技術：先把影像模糊化，用原圖減去模糊版得到「細節層」(即被模糊抹掉的高頻)，再把細節層加權加回原圖。公式為 $g = f + k\cdot(f - \text{blur}(f))$。

當 $k = 1$ 稱為 Unsharp Masking，$k > 1$ 則稱為 High-Boost Filtering，能同時提亮並強化細節。從第 6 章的頻率觀點看，這正是「高頻強調濾波」——本章與第 6 章做的是同一件事，只是一個用空間域的減法實現、一個用頻率域的加權實現。這種「同

一操作的兩張面孔」是理解影像處理的關鍵心法。

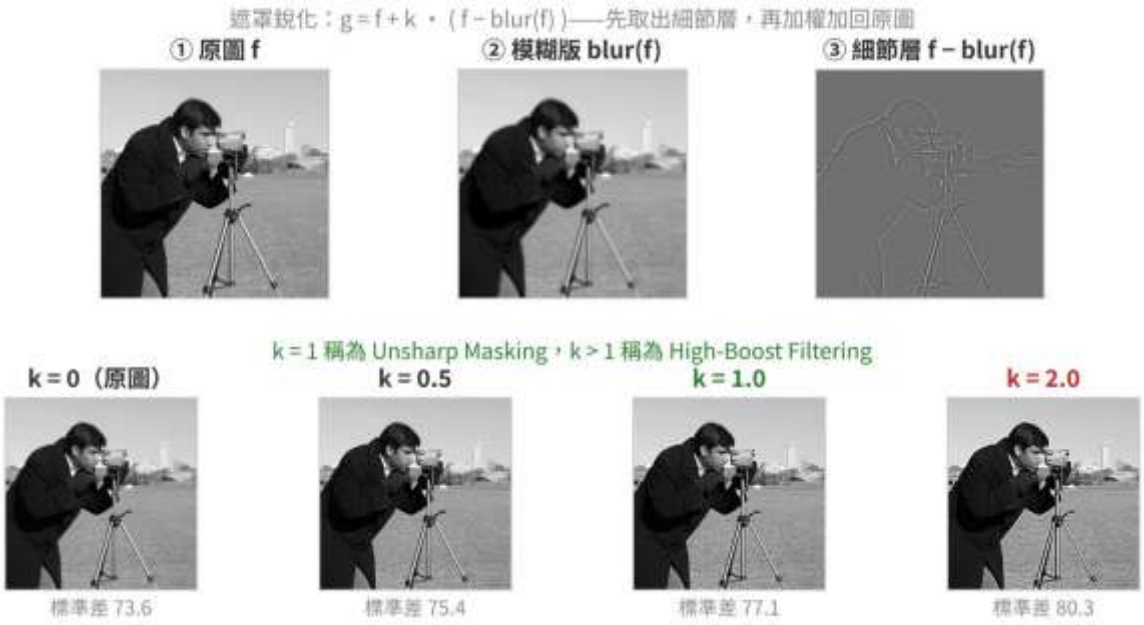


圖 5-17　遮罩銳化的原理與 k 值的影響

5.5.3 銳化的代價

銳化有三個必須知道的代價。第一，過衝(overshoot):k 太大時邊緣兩側會出現白邊與黑邊，俗稱光暈，看起來不自然——這與第 6 章的振鈴效應同源。第二，雜訊放大：高頻成分中雜訊佔了很大比重，銳化會一併放大它們。第三，無中生有的錯覺：銳化並沒有把模糊掉的資訊找回來，它只是把邊緣兩側的對比拉大，製造出「更清楚」的視覺印象——真正的還原是第 7 章的反卷積。

因此有一條實務鐵則：銳化之前要先適度去雜訊，順序不能顛倒。若先銳化再去雜訊，雜訊已經被放大，再平滑只會連同細節一起抹掉。這個「順序決定成敗」的原則，在第 9 章的 Canny(先平滑再求梯度)中會再次出現。

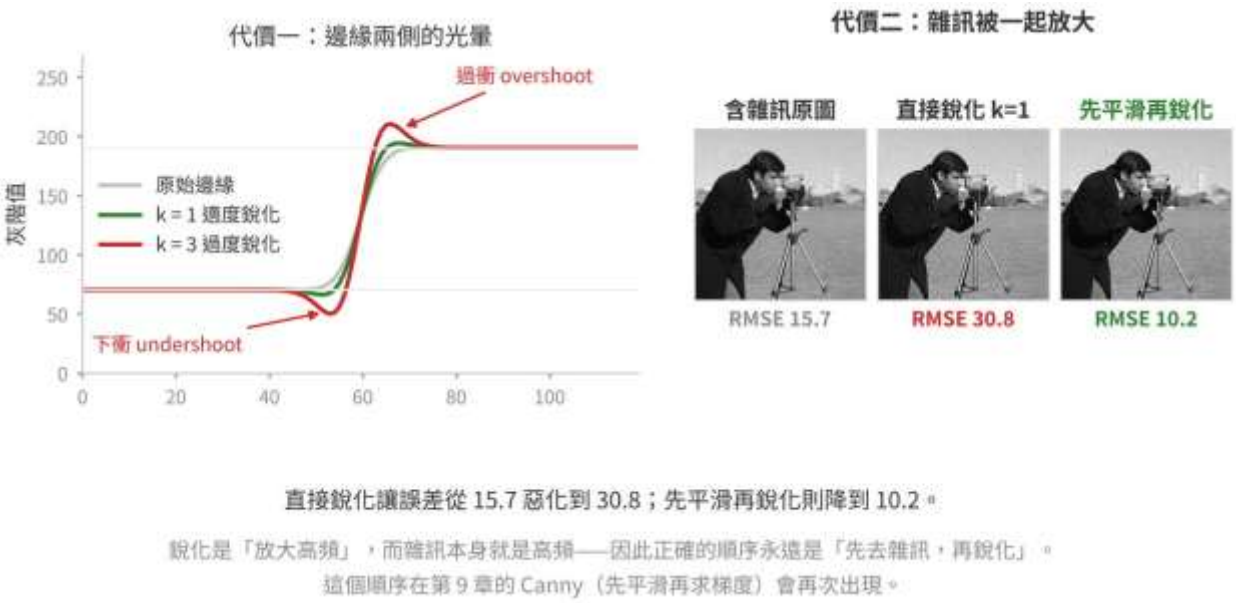


圖 5-18　銳化的兩個代價：光暈與雜訊放大

程式 5-4 銳化與正確的處理順序

```
import cv2, numpy as np

img = cv2.imread("soft.jpg")

# --- 方法一:Laplacian 銳化 ---
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
sharp1 = cv2.convertScaleAbs(img - 0.5 * lap)

# --- 方法二:Unsharp Masking / High-Boost(實務首選) ---
blur = cv2.GaussianBlur(img, (0, 0), sigmaX=3)
k = 1.5                                    # k>1 即 High-Boost
sharp2 = cv2.addWeighted(img, 1 + k, blur, -k, 0)

# --- 正確順序:先去雜訊,再銳化 ---
denoised = cv2.bilateralFilter(img, 9, 50, 50)   # 保邊去雜訊
blur2 = cv2.GaussianBlur(denoised, (0, 0), 3)
good = cv2.addWeighted(denoised, 2.0, blur2, -1.0, 0)

# --- 錯誤順序:先銳化再去雜訊(雜訊已被放大) ---
bad = cv2.bilateralFilter(sharp2, 9, 50, 50)

cv2.imwrite("sharp_correct.jpg", good)
cv2.imwrite("sharp_wrong.jpg", bad)
# 並排比較:錯誤順序的結果細節較糊、雜點殘留較多
```

5.5.4 推導：遮罩銳化其實是一個高通濾波

遮罩銳化的公式 $g = f + k \cdot (f - f * G)$ 看起來像是「兩個步驟」，但把它整理一下會發現，它其實只是「一次卷積」，而且是一個我們到第 6 章才會正式命名的東西。

把式子展開：$g = (1 + k) \cdot f - k \cdot (f * G)$。由於 f 本身可以寫成 $f * \delta$（δ 是單位脈衝，即中心為 1、其餘為 0 的核），整個式子可以合併成 $g = f * [(1 + k) \cdot \delta - k \cdot G]$。也就是說，遮罩銳化等於用單一個核 $(1 + k)\delta - kG$ 做一次卷積。這個核的權重總和為 $(1 + k) - k = 1$（因為高斯核的總和為 1），所以影像的整體亮度不變——這與 5.3 節說的「平滑核總和為 1」是同一個道理。

更有意思的是它在頻率域的樣子（第 6 章會給出完整工具）。這個核的轉移函數為 $H = (1 + k) - k \cdot \hat{G}$，而高斯是典型的低通濾波：在零頻處 $\hat{G} = 1$，頻率愈高 $\hat{G}$ 愈趨近 0。代入可得：零頻處 $H = (1 + k) - k = 1$，高頻處 $H \to (1 + k)$。本書以 $\sigma = 3$ 的高斯核實測，$k = 1$ 時 H 由 1.0000 平滑地升到 2.0000，$k = 2$ 時升到 3.0000，與理論完全吻合。

這解釋了三件事。第一，為什麼它叫 High-Boost——低頻原封不動通過（增益為 1），高頻被放大到 $(1 + k)$ 倍。第二，為什麼雜訊會被放大：雜訊的能量集中在高頻，正好落在增益最大的區段。第三，為什麼第 6 章要花整章談頻率域——空間域裡看似兩個獨立步驟的操作，到了頻率域只是一條增益曲線，設計與分析都變得直觀得多。

5.6 多影格合成：以軟體突破硬體限制

本節把叫獸開講的核心技術展開，它也是計算攝影 (Computational Photography)這個領域的縮影。

5.6.1 為什麼平均能去雜訊

假設雜訊是隨機且零均值的(高斯雜訊符合此假設)，那麼對同一場景拍 N 張、逐像素平均後：訊號部分因為每張都相同，平均後不變;雜訊部分因為隨機且正負相消，其標準差降為原來的 N 平方根分之一。平均 4 張，雜訊減半;平均 16 張，雜訊剩四分之一;平均 100 張，剩十分之一。

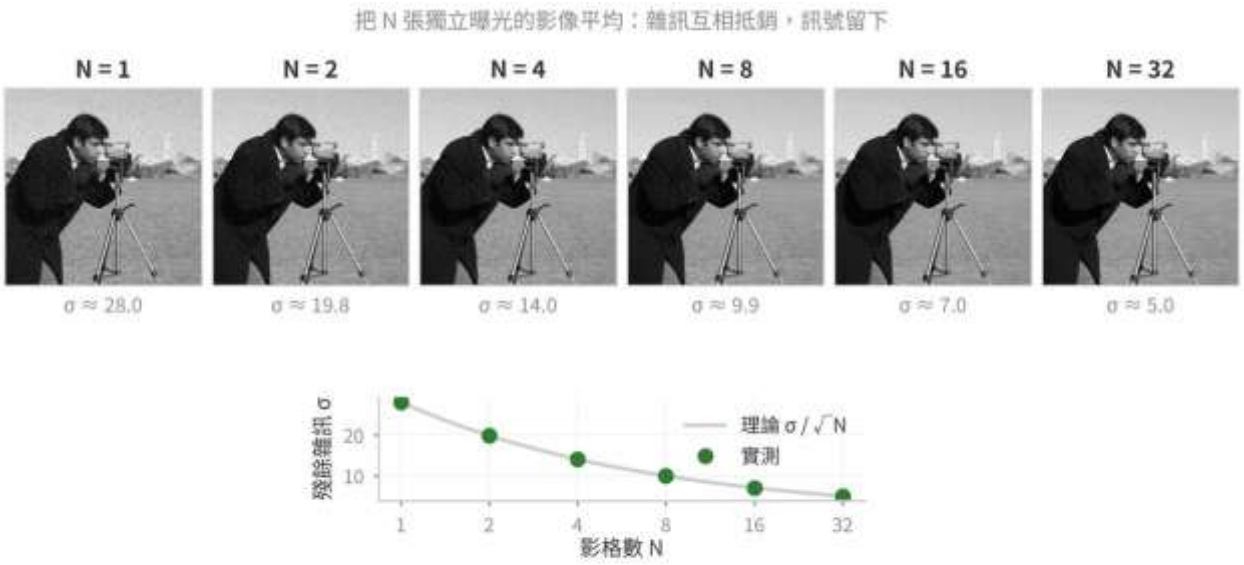


圖 5-19　多影格平均：雜訊隨 1/√N 下降

這正是叫獸開講思考題的答案，也解釋了「訊噪比隨曝光時間平方根上升」的物理直覺——更長的曝光就是收集更多光子，而更多的獨立樣本平均起來，隨機波動自然變小。天文攝影界稱這個技術為「曝光堆疊」，已使用數十年。

5.6.2 多影格平均與空間平均的關鍵差異

這裡有一個極重要的對照，值得細細體會：5.4 節的空間平滑，平均的是「同一時間的不同像素」——但鄰近像素的訊號未必相同(邊緣兩側就完全不同)，因此平均會模糊邊緣。而多影格平均，平均的是「不同時間的同一個像素」——訊號理論上完全相同，只有雜訊在變。

結論是：多影格平均能在幾乎不損失任何細節的前提下降低雜訊，這是單一影格去雜訊永遠做不到的。這也是為什麼所有嚴肅的低光攝影、天文攝影、以及第 7 章的醫學影像，都盡可能採用多影格方案。

5.6.3 對齊：多影格方案的真正難點

但前提是「同一個像素」——手持拍攝必然有位移與旋轉，若不對齊就直接平均，

結果是一張模糊的照片。因此對齊才是多影格合成的真正技術核心，也是 Night Sight 所謂 robust merge 的重點。



圖 5-20　對齊是多影格合成的真正難點

對齊的方法本書後續都會學到：第 6.7.2 節的相位相關法(利用傅立葉的平移性質，快速且對亮度變化免疫)、第 13 章的特徵匹配加 RANSAC(能處理旋轉與透視變化)、以及第 12 章的光流。而 Night Sight 更進一步採用分塊對齊，並丟棄運動過大的區塊——因為場景中可能有移動的人或車，強行對齊會產生鬼影。

這條技術鏈完整地展示了本書各章如何協作：第 5 章提供合成的原理、第 6 或 13 章提供對齊的工具、第 4 章提供轉換的數學、第 3 章提供品質的評估。一個看似單純的「夜景模式」，背後是整本書的知識。

程式 5-5 窮人版夜景模式

```
import cv2, numpy as np, glob

# --- 窮人版夜景模式:多影格平均 ---
files = sorted(glob.glob("burst/*.jpg"))      # 連拍的一疊暗景
ref = cv2.imread(files[0])
acc = np.zeros_like(ref, dtype=np.float32)
n = 0

ref_gray = cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY).astype(np.float32)
win = cv2.createHanningWindow(ref_gray.shape[::-1], cv2.CV_32F)

for f in files:
```

```
    img = cv2.imread(f)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # 對齊:相位相關法求位移(第 6 章)
    (dx, dy), _ = cv2.phaseCorrelate(ref_gray, gray, win)
    M = np.float32([[1, 0, -dx], [0, 1, -dy]])       # 第 4 章仿射
    aligned = cv2.warpAffine(img, M, img.shape[1::-1])

    acc += aligned.astype(np.float32)
    n += 1

merged = (acc / n).astype(np.uint8)

# 後處理:提亮 + 局部對比 + 補回銳利度
merged = gamma_correct(merged, 2.0)                  # 5.1 節
lab = cv2.cvtColor(merged, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
l = cv2.createCLAHE(2.0, (8, 8)).apply(l)            # 5.2 節
merged = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)

blur = cv2.GaussianBlur(merged, (0, 0), 2)
merged = cv2.addWeighted(merged, 1.5, blur, -0.5, 0)  # 5.5 節
cv2.imwrite("night_sight_poor_man.jpg", merged)

print(f"合併 {n} 張,理論雜訊降為 1/{np.sqrt(n):.1f}")
```

5.6.4 推導：1/√N 從哪裡來，又在什麼時候失效

多影格平均的 1/√N 法則值得完整推導一次，因為它的三個前提，每一個在實務上都會被打破，而知道是哪一個被打破，才知道該怎麼補救。

設第 i 張影格的某個像素為 $f_i = s + n_i$，其中 s 是不變的訊號、n_i 是雜訊。假設 n_i 為零均值（$E[n] = 0$）、變異數相同（$\mathrm{Var}[n] = \sigma^2$）、且各影格之間互相獨立。N 張平均後的值為 $(1/N)\cdot\Sigma f_i = s + (1/N)\cdot\Sigma n_i$。訊號 s 原封不動保留;雜訊項的變異數為 $\mathrm{Var}[(1/N)\cdot\Sigma n_i] = (1/N^2)\cdot\Sigma\,\mathrm{Var}[n_i] = (1/N^2)\cdot N\sigma^2 = \sigma^2/N$。開根號後，標準差為 $\sigma/\sqrt{N}$。這就是 1/√N 的來源——要讓雜訊減半需要四張，減到四分之一需要十六張。

三個前提各自對應一種實務上的失效模式。其一，零均值：感測器的暗電流會在每張影格加上同一個正偏移，它不是零均值，平均再多張也不會消失——解法是先拍一張蓋上鏡頭的暗影格再相減。其二，互相獨立：固定圖樣雜訊（每個像素的增益與偏移差異）在每張影格中完全相同，平均不但無法消除，反而會因為隨機雜訊下降而

相對變得更明顯——解法是平場校正。其三，訊號不變：手持拍攝會讓場景在影格間位移，此時「同一個像素」對應到的已經不是同一處景物，平均就會順帶模糊訊號——解法是對齊，也就是 5.6.3 節的主題。

把這三件事放在一起看，就能理解為什麼一套可用的夜景模式遠不只是「相加除以 N」：它必須先扣掉不隨機的部分（暗電流、固定圖樣），再對齊會動的部分（手震、物體移動），最後才輪到平均。理論給出的 1/√N 是上限，工程的任務是盡量逼近它。

5.7 應用案例

5.7.1 案例一：工業影像前處理管線

情境：AOI 檢測系統要在打光不均的工件表面找出刮痕。若直接做第 9 章的門檻分割，亮區的背景會被誤判為前景、暗區的瑕疵則被漏掉。

標準管線：轉灰階 → 用 CLAHE 拉平局部對比(5.2.3 節)→ 用雙邊濾波或中值濾波去除雜點但保住瑕疵邊緣(5.4 節)→ 適度銳化強化瑕疵的邊緣對比(5.5 節)→ 交給第 9 章分割。

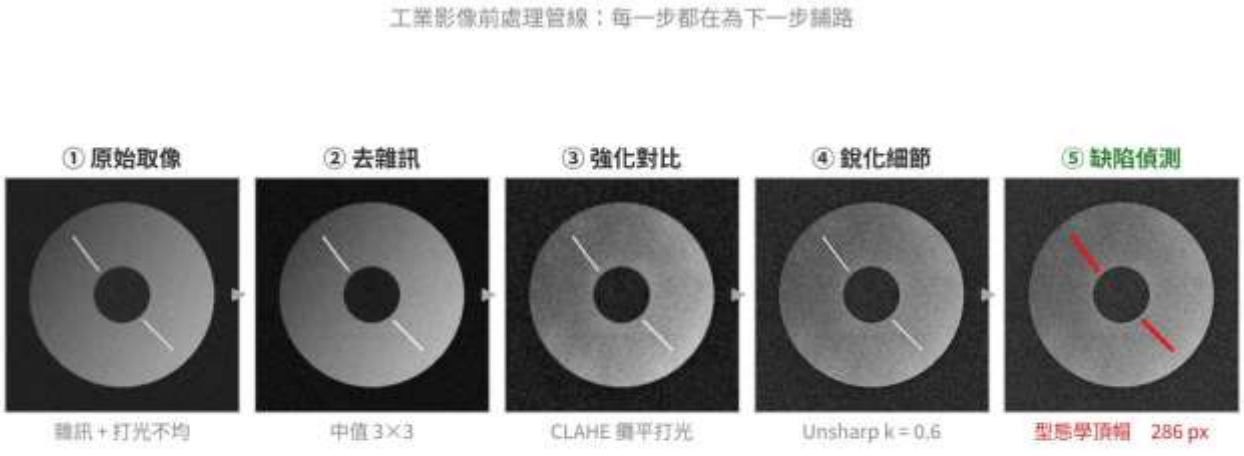


圖 5-21　工業影像前處理管線的五個步驟

這個案例的關鍵教訓是：前處理的目標不是「讓影像好看」，而是「讓後續演算法好做」。有時候一張經過強化的影像在人眼看來反而怪異(對比過強、色彩失真)，但只要它讓分割與量測更準確，就是成功的前處理。這也是第 5 章與第 7 章在判準上的又一個差異——強化的成敗，取決於下游任務的表現。

5.7.2 案例二：醫學影像的視窗化與強化

情境：承第 3.7.2 節，CT 影像以 12 位元儲存(4096 階)，但螢幕只能顯示 256 階。醫師需要依觀察目標選擇「視窗」：看肺部用一組視窗中心與寬度、看骨骼用另一組、看軟組織再一組。

視窗化本質上就是本章的灰階轉換——把 12 位元中的某一段線性拉伸到完整的顯示範圍，超出範圍的一律設為全黑或全白。這說明了一個重要觀念：同一份資料，不同的轉換會揭示不同的資訊。這也是為什麼高位元深度的原始資料必須完整保存——它承載了顯示時尚未取用的資訊。

醫學應用也有其特殊的謹慎：過度的強化(尤其是銳化)可能製造出不存在的邊緣、或讓微小病灶的邊界看起來比實際清楚，誤導判讀。這與第 7 章「還原 vs. 腦補」、第 16 章的技術倫理是同一個主題：在高風險領域，「看起來更清楚」與「更接近真實」必須嚴格區分。

5.7.3 案例三：計算攝影與手機夜景

情境：承叫獸開講，現代手機的相機模組在物理上很受限——鏡頭小、感光元件小、進光量少。但拍出來的照片卻常常勝過某些專業相機的單張直出，原因就在計算攝影：用軟體與運算補上硬體的不足。

這個案例最能說明本章的價值定位。回顧全書的三種思路：硬體不足時，可以換更好的硬體(貴)、可以用數學還原(第 7 章，但受物理上限所限)、也可以「多拍幾張再合成」(本章 5.6 節)。第三條路往往最務實——它把「空間上的限制」轉換成「時間上

的成本」，而時間通常比硬體便宜。這個「換一種資源來解決問題」的思維，值得學生反覆體會。

5.7.4 案例四：訓練資料的前處理與正規化

情境：承第 14 至 16 章，訓練深度學習模型前，所有影像都要經過統一的前處理。這裡本章的技術扮演兩個角色。

其一是正規化：把像素值從 0 到 255 縮放到 0 到 1 或標準化為零均值單位變異數。這在數學上是最單純的點運算(本章 5.1 節)，卻對訓練的收斂速度有顯著影響——因為神經網路的梯度下降對輸入尺度相當敏感。

其二是條件統一：若訓練資料來自不同的光照條件(有些在晴天拍、有些在陰天拍)，用 CLAHE 或白平衡統一它們，能減少模型需要學習的變異、提升泛化能力。但這裡有個微妙的取捨——若推論時的影像也會有同樣的光照變化，那麼「保留變化並用資料增強讓模型適應」可能比「強行統一」更好。這正是第 7.7.3 節與第 15 章討論過的「還原 vs. 資料增強」抉擇，在前處理階段的體現。

5.8 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
對數轉換	第 6 章	顯示傅立葉頻譜前的動態範圍壓縮
Gamma 與線性空間	第 3、7、8 章	RAW 為線性;還原與白平衡須留意非線性
直方圖	第 7、9 章	雜訊型態診斷;Otsu 自動門檻的基礎
CLAHE	第 9、10、16 章	分割前處理;AOI 打光不均;訓練資料統一
卷積與濾波核	第 6、15 章	卷積定理;CNN 的卷積層即可學習的核
高斯濾波	第 9、11、13 章	Canny 首步;小波多尺度;SIFT 尺度空間

中值與雙邊濾波	第 6、7、13 章	非線性故無頻率響應;去雜訊工具箱;車牌前處理
可分離濾波	第 6、17 章	運算量取捨;邊緣裝置的即時性
Unsharp / High-Boost	第 6、7 章	等同頻率域高頻強調;與反卷積的根本差異
多影格合成	第 6、12、13 章	相位相關與特徵匹配對齊;視訊的時間維度

表 5-2 第 5 章與全書其他章節的觀念連結

程式實作 Lab 5

Lab 5-1 灰階轉換與直方圖診斷

目標：建立「先診斷、再開藥」的習慣。(a)準備三張影像：曝光不足、過曝、對比不足，各自繪製直方圖並描述其形狀特徵;(b)對每張套用適當的處理(Gamma、對數、等化)，說明你的選擇理由;(c)實作程式 5-1 的 LUT 版 Gamma 校正，與「逐像素用 np.power 計算」比較執行時間(呼應第 2 章向量化);(d)驗證 Gamma 編碼的非線性：對一張 RAW(或以 γ=1/2.2 模擬的線性影像)與其 JPEG 版本，比較「兩個區域的亮度比值」是否相同;(e)結論：寫出一張決策流程圖——看到什麼樣的直方圖，該用哪種轉換?

Lab 5-2 等化、CLAHE 與彩色陷阱

目標：掌握對比強化的正確做法。(a)手算驗證 5.2.2 節的範例(L=8、64 像素)，再用程式驗證你的答案;(b)對一張「同時有明亮窗戶與陰暗室內」的影像，比較全域等化與 CLAHE 的結果，說明前者為何顧此失彼;(c)調整 CLAHE 的 clipLimit(1、2、4、10)與 tileGridSize(4×4、8×8、16×16)，觀察各自的影響並找出最佳組合;(d)彩色陷阱實驗：對同一張彩色影像，分別用「BGR 三通道各自等化」與「轉 LAB 只等化 L 通道」，並排比較色偏程度;(e)觀察等化在平坦區域放大雜訊的現象，並思考如何緩解。

Lab 5-3 濾波器擂台

目標：為第 7 章的去雜訊工具箱打底。(a)對一張乾淨影像分別加入高斯雜訊與椒鹽雜訊;(b)用均值、高斯、中值、雙邊四種濾波各處理一次，製作 2×4 的 PSNR 與 SSIM 比較表(用第 3 章 Lab 3-3 建立的評估函式);(c)寫出結論：哪種雜訊該配哪種濾波器?為什麼中值對椒鹽近乎完美、對高斯卻無特別優勢?(d)實測可分離濾波：比較 GaussianBlur 與同尺寸 filter2D 的執行時間，核大小從 3 到 51，繪製曲線;(e)銳化順序實驗：比較「先去雜訊再銳化」與「先銳化再去雜訊」的結果，以 PSNR 與目視雙重驗證。

Lab 5-4 窮人版夜景模式

目標：重現叫獸開講的核心技術。(a)用手機或 USB 相機以固定姿勢連拍同一暗景 16 張;(b)先不對齊直接平均，計算雜訊標準差並驗證思考題：平均 16 張後是否約降為四分之一?(c)手持連拍(必然有晃動)，比較「直接平均」與「先用相位相關對齊再平均」的清晰度差異;(d)以程式 5-5 完成完整管線：對齊平均 → Gamma 提亮 → CLAHE 局部對比 → Unsharp 補銳度;(e)以第 3 章的 PSNR/SSIM 比較「單張」、「多張平均」、「完整管線」三者相對於長曝光參考照的差距;(f)進階挑戰：在畫面中放一個移動的物體，觀察直接平均會產生什麼鬼影，並思考 Night Sight 的「分塊對齊並丟棄運動區塊」如何解決它。

Lab 5-5 邊界、可分離與效能實測

目標：把 5.3 節的三個工程細節實際量出來。(a)對同一張影像以 31×31 的高斯核模糊，分別用 constant、replicate、reflect、reflect_101、wrap 五種邊界處理，把影像四個角落放大並列(重現圖 5-11)，說明哪一種在你的應用上最合適;(b)計算這張影像有多少比例的像素受邊界處理影響，與理論值比對;(c)以 cv2.filter2D 與 cv2.sepFilter2D 實作同一個均值濾波，對 k = 3 到 31 各量測十次取平均，畫出時間對 k 的曲線;(d)你會發現 k 很大時 filter2D 的時間不再成長——查閱文件說明原因，並

連結到第 6 章的卷積定理;(e)驗證可分離性：以 cv2.getGaussianKernel 取出一維核，計算其外積，與 cv2.getGaussianKernel 的二維版本比較是否相同。

Lab 5-6 建立你自己的前處理管線

目標：把 5.7.1 節與圖 5-21 的管線套到你自己拍的影像上，並且用數字證明每一步都有幫助。(a)拍攝一組有「打光不均」與「可見雜訊」的照片(刻意用側光、調高 ISO 或在昏暗處拍攝)，物件可以是零件、硬幣或電路板，並刻意製造一處可見的瑕疵;(b)依「去雜訊 → 強化對比 → 銳化」的順序建立管線，每一步都存下中間結果;(c)為每一步定義一個量化指標——例如平坦區的標準差(衡量雜訊)、瑕疵區與周圍背景的灰階差(衡量對比)，逐步記錄數值變化;(d)刻意把順序顛倒(先銳化再去雜訊)，比較指標與目視結果，驗證 5.5.3 節的論點;(e)把最後的結果交給第 10 章的型態學頂帽運算找出瑕疵，記錄偵出面積與良品的誤報面積;(f)寫下一段三百字的檢討：哪一步的收益最大?哪一步其實可以省略?

本章重點整理

- 空間域處理框架 g = T[f];鄰域為單一像素時稱點運算(灰階轉換)，擴大鄰域即空間濾波。本章屬影像強化(主觀、讓影像好看或好用)，與第 7 章的還原(客觀、逼近原始)判準不同。
- 灰階轉換：負片(醫學影像)、對數(壓縮動態範圍、顯示頻譜)、Gamma 冪次法則(γ>1 提亮暗部);螢幕 γ=2.2 的歷史使 JPEG 非線性，精確光度計算須先轉回線性;以 LUT 查表實作最有效率。
- 直方圖是診斷工具，也是第 7 章雜訊辨識與第 9 章 Otsu 門檻的基礎;等化以累積分布函數為映射，但會放大雜訊且不可控;CLAHE 分格局部等化並以 clipLimit 削頂抑制雜訊，實務上優於全域等化;彩色影像應只等化亮度通道。

- 卷積需將核旋轉 180 度，相關則否(OpenCV filter2D 實為相關);CNN 的卷積層即可學習的濾波核。邊界處理影響拼接、分塊與頻域結果;可分離濾波將 m^2 次乘法降為 2m 次，是即時處理的關鍵。
- 平滑濾波：均值(快但有偽影)、高斯(自然，全書最常用)、中值(專治椒鹽、非線性)、雙邊(保邊緣、磨皮原理)。
- 銳化基於微分：Laplacian 為二階微分(對雜訊極敏感);Unsharp Masking 以原圖減模糊得細節層再加回，k>1 即 High-Boost，等同頻率域的高頻強調。銳化會過衝與放大雜訊，且未真正找回資訊，必須先去雜訊再銳化。
- 多影格平均使雜訊標準差降為 N 平方根分之一;相較空間平均(同時刻不同像素，會模糊邊緣)，多影格平均(不同時刻同一像素)幾乎不損失細節，但需先對齊(第 6、12、13 章的技術)。

習題

1. 點運算與鄰域運算的差別是什麼?各舉兩個例子。本章的「強化」與第 7 章的「還原」在判準上有何根本不同?

   > *提示與參考解答：點運算的輸出只取決於該像素自己的值(如負片、Gamma 校正、直方圖等化);鄰域運算則取決於周圍一片像素(如高斯模糊、中值濾波、銳化)。強化追求主觀的「看起來更好或更適合後續處理」，如 Night Sight 刻意壓黑陰影以營造夜的氛圍;還原則是客觀的逆問題，先建立劣化模型再反推最接近原始的估計，可用 PSNR、SSIM 量化驗收。一句話：強化是化妝，還原是手術。*

2. Gamma 校正中，γ = 2.2 與 γ = 0.45 分別使影像變亮或變暗?為什麼 JPEG 的像素值不是線性正比於光強度?這在什麼情況下會造成問題?

   > *提示與參考解答：公式 s = c·r^γ 中，實作上常以 1/γ 為指數(如程式 5-1);依課文定義，γ>1 提亮暗部、γ<1 壓暗。JPEG 非線性的原因：早期映像管螢幕的亮度響應近似 γ=2.2 的冪次曲線，影像儲存時預先做了反向的 Gamma 編碼來抵消。問題發生在精確光度量測時——直接用 JPEG 像素值計算「亮度比值」會得到錯誤答案，須先反 Gamma 轉回線性空間(第 7 章還原、第 8 章白平衡皆須留意)。*

3. 為什麼顯示傅立葉頻譜前通常要取對數?這利用了對數轉換的什麼特性?

   提示與參考解答：因為頻譜的直流分量(整張影像的平均亮度)數值往往比其他成分大好幾個數量級，不取對數的話整張頻譜圖只看得到中央一個亮點、其餘全黑。對數轉換大幅拉伸暗部、壓縮亮部，能壓縮極大的動態範圍，讓微弱的高頻成分也看得見。這是對數轉換「適合處理動態範圍極大影像」的最佳實例。

4. 某 L = 8 的影像共 64 像素，各灰階像素數為 8、10、16、14、8、5、2、1。請計算直方圖等化後的映射表，並說明會發生什麼細節損失。

   提示與參考解答：累積機率依序為 0.125、0.281、0.531、0.750、0.875、0.953、0.984、1.000;乘以 7(L-1)並四捨五入得映射：0→1、1→2、2→4、3→5、4→6、5→7、6→7、7→7。細節損失：原本不同的灰階 5、6、7 都映射到 7，合併成同一階，這三階之間的差異永久消失。這說明等化雖擴展了整體對比，卻可能犧牲高灰階區的細節。

5. 說明 CLAHE 相較全域等化的兩項改良，並解釋 clipLimit 的作用與名稱由來。

   提示與參考解答：改良一：分格局部等化(如 8×8)，各格依自己的統計量調整，再以雙線性內插消除接縫，解決「同時有明亮窗戶與陰暗室內時全域等化顧此失彼」的問題。改良二：對比限制——設定 clipLimit 把直方圖中過高的長條削頂，並把超出部分均攤到其他灰階，避免平坦區的雜訊被暴力放大。名稱 Contrast Limited 正來自這個削頂機制。

6. 對彩色影像的 B、G、R 三通道分別做直方圖等化，會產生什麼問題?正確做法為何?

   提示與參考解答：三個通道的映射函數各不相同，原本的色彩比例會被破壞，產生詭異的色偏。正確做法：轉到亮度與色彩分離的色彩空間(LAB 或 YCbCr)，只對亮度通道(L 或 Y)做等化或 CLAHE，色彩通道保持不變，再轉回 BGR。第 8 章會說明這些色彩空間的設計理由。

7. 卷積與相關的差別是什麼?什麼情況下兩者結果相同?CNN 的卷積層與本章的空間濾波有何異同?

   提示與參考解答：卷積先把核旋轉 180 度再做對應相乘相加，相關則直接相乘相加。對稱核(如高斯)兩者結果相同。理論上要求卷積是因為它滿足交換律與結合律，

並對應第 6 章的卷積定理;OpenCV 的 filter2D 實際執行的是相關。CNN 卷積層的數學運算與本章相同，關鍵差異在於：本章的核是人設計的(高斯、Sobel),CNN 的核是從資料中學出來的(第 15 章)。

8. 一個 11×11 的可分離高斯核，相對於直接二維卷積，每個像素可省下多少次乘法?這個技巧為什麼對第 17 章重要?

   提示與參考解答：二維卷積每像素需 11×11 = 121 次乘法;可分離則先水平再垂直各做一維，需 11 + 11 = 22 次，省下 99 次，約快 5.5 倍。核愈大差距愈懸殊(51×51 時從 2601 降為 102)。對第 17 章重要，是因為邊緣裝置(樹莓派、Jetson)算力有限，即時處理必須錙銖必較;而當核大到連可分離都不划算時，就該考慮轉到第 6 章的頻率域。

9. 影像同時受到椒鹽雜訊污染且需保留邊緣，你會選哪一種平滑濾波?為什麼中值濾波對椒鹽近乎完美，對高斯雜訊卻無特別優勢?

   提示與參考解答：選中值濾波。因為椒鹽雜訊是極端值(0 或 255)，排序後落在最前或最後，永遠不會被選為中位數，故能完全去除且不模糊邊緣。對高斯雜訊則無特別優勢：高斯雜訊是圍繞真值的小幅隨機起伏，並非極端值，取中位數與取平均的效果相近，而均值濾波在此情況下反而是統計上的最佳線性估計。

10. 說明 Unsharp Masking 的公式與步驟。為什麼銳化應該在去雜訊之後執行?過度銳化會出現什麼現象?銳化有沒有真正「找回」被模糊掉的資訊?

    提示與參考解答：公式 $g = f + k\cdot(f - \text{blur}(f))$：先模糊、原圖減模糊得細節層(高頻)、再加權加回。順序理由：高頻成分中雜訊佔比重，銳化會放大雜訊;若先銳化再去雜訊，雜訊已被放大，再平滑只會連同細節一起抹掉。過度銳化出現過衝(邊緣兩側的白邊黑邊，俗稱光暈，與第 6 章振鈴同源)。銳化並未找回資訊，它只是把邊緣兩側的對比拉大製造「更清楚」的視覺印象;真正的還原是第 7 章的反卷積。

11. 為什麼多影格平均能降低雜訊?平均 25 張後雜訊標準差降為原來的幾分之一?它與空間平滑相比有什麼根本優勢?

    提示與參考解答：若雜訊隨機且零均值，平均 N 張後訊號不變、雜訊正負相消，標準差降為原來的 N 平方根分之一;平均 25 張則降為五分之一。根本優勢：空間平滑平均的是「同一時刻的不同像素」，而鄰近像素的訊號未必相同(邊緣兩側就不同)，故會模糊細節;多影格平均的是「不同時刻的同一像素」，訊號理論上完全相同，因

此能在幾乎不損失細節的前提下降噪。

12. (綜合題)承叫獸開講：請說明 Night Sight 用到了本章與本書哪些技術，並解釋為什麼「對齊」是多影格方案的真正難點。

    提示與參考解答：本章技術：多影格平均去雜訊(5.6 節)、Gamma 與色調曲線提亮(5.1 節)、局部對比增強(5.2 節)、銳化補回細節(5.5 節)。其他章節：對齊用第 6.7.2 節相位相關法或第 13 章特徵匹配、第 4 章的幾何轉換做補償、第 3 章的 PSNR/SSIM 評估、第 8 章的白平衡(Night Sight 用學習式演算法)。對齊是難點的原因：手持必有位移旋轉，不對齊直接平均會得到模糊照片;而場景中可能有移動的人車，強行整體對齊會產生鬼影，故 Night Sight 採用分塊對齊並丟棄運動過大的區塊(robust merge)。

13. 承圖 5-1：單點、鄰域與全域運算的差別在於「輸出像素看到多少輸入」。直方圖等化為什麼歸類為全域運算而不是單點運算?CLAHE 又該歸在哪一類?

    提示與參考解答：直方圖等化在「套用映射表」的那一步確實是逐像素進行的，但那張映射表本身來自整張影像的累積分布函數——換一張影像，表就不同。決定映射的資訊來自全域，因此歸為全域運算。CLAHE 介於兩者之間：它把影像切成小塊，每塊各自依自己的直方圖建表，所以是「局部全域」運算;若把一個分塊視為一個鄰域，也可以理解為大尺度的鄰域運算。這個分類的實益在於判斷「同樣的參數換一張影像還能不能用」——單點運算可以，全域運算不行。

14. 承圖 5-2：對數轉換與 $\gamma < 1$ 的 Gamma 曲線都能提亮暗部，兩者的差別在哪裡?什麼場合非用對數不可?

    提示與參考解答：兩者的曲線都是上凸的，但對數在靠近 0 的地方斜率趨於無限大，對極暗的值提升極為劇烈;Gamma 的斜率雖大卻有限，提亮較為溫和可控。因此一般影像的提亮多用 Gamma(可用一個參數連續調整)，而當輸入的動態範圍橫跨數個數量級時就非用對數不可——最典型的是第 6 章的傅立葉頻譜，直流分量往往比高頻大上數萬倍，不取對數的話整張頻譜只會看到中央一個亮點。

15. 計算題：3×3 高斯核為 (1/16)·[[1,2,1],[2,4,2],[1,2,1]]。某影像區塊的灰階為 [[120,140,135],[128,200,142],[118,133,126]]，請計算卷積後輸出的中心像素值。並說明核內權重總和為什麼必須是 1。

提示與參考解答：逐項相乘後相加：(120×1 + 140×2 + 135×1 + 128×2 + 200×4 + 142×2 + 118×1 + 133×2 + 126×1) / 16 = 2385 / 16 = 149.06，四捨五入為 149。權重總和必須為 1 的理由是「亮度守恆」：若總和大於 1，整張影像會整體變亮;小於 1 則整體變暗。對均值核與高斯核這類平滑濾波，我們只想重新分配鄰域內的能量、不想改變平均亮度，所以要正規化。反之，微分型的核(如 Sobel、Laplacian)權重總和為 0，因為它們量測的是「變化量」，平坦區應該輸出 0。

16. 承圖 5-9：若濾波核改成 Sobel 的 [[−1,0,1],[−2,0,2],[−1,0,1]]，相關與卷積的結果差在哪裡?為什麼在深度學習的文獻中這兩個詞常被混用卻不出問題?

    提示與參考解答：這個核左右反對稱，翻轉 180° 後正負號整個顛倒，因此相關與卷積的輸出互為相反數——梯度的方向會相反，若據此判斷「由亮到暗」或「由暗到亮」就會完全弄錯。深度學習不出問題的原因是：CNN 的核是「學出來的」而不是人訂的，框架實際做的是相關運算，網路只要在訓練中把核學成翻轉後的樣子，效果完全等價。換句話說差異被訓練吸收了，所以名詞混用無害;但在本章這種「核由人指定」的場合就必須分清楚。

17. 承圖 5-11：為什麼補零(constant)會讓邊界變暗?對一張 512×512 的影像使用 31×31 的核，有多少比例的像素會受到邊界處理方式的影響?(請計算)

    提示與參考解答：補零等於假設影像之外一片漆黑。靠近邊界的像素，其鄰域有一部分落在影像外並以 0 參與平均，拉低了結果，因此邊界形成一圈暗帶。計算：31×31 的核半徑為 15，只要像素距離任一邊界不足 15 就會受影響。不受影響的區域為 $(512 - 30)^2 = 482^2 = 232{,}324$ 像素;全圖 $512^2 = 262{,}144$ 像素;受影響者 29,820 像素，佔 11.4%。核愈大、影像愈小，這個比例愈高——這也是為什麼在小影像或 ESP32-CAM 的低解析度影像上，邊界處理的選擇特別重要。

18. 計算題：承圖 5-12。請分別計算 k = 5、11、21、31 的核，以二維卷積與可分離實作的每像素乘法次數與加速倍率。為什麼核再更大時，改到頻率域做會更划算?

    提示與參考解答：二維為 k^2 次、可分離為 $2k$ 次，加速倍率 $k/2$：k=5 為 25 對 10(2.5 倍)、k=11 為 121 對 22(5.5 倍)、k=21 為 441 對 42(10.5 倍)、k=31 為 961 對 62(15.5 倍)。可分離實作的成本隨 k 線性成長，二維則是平方成長。但頻率域的成本幾乎與核大小無關——依卷積定理(第 6 章)，空間域的卷積等於頻率域的逐

點相乘，成本由 FFT 決定，約為 O(MN log MN)。因此存在一個交叉點，核大於該點後頻率域勝出;OpenCV 的 filter2D 內部就會在核夠大時自動改用這條路徑。

19. 承圖 5-13：為什麼雙邊濾波對椒鹽雜訊的表現反而最差(RMSE 31.0，比均值的 14.7 還糟)?請從它的值域權重解釋。

    提示與參考解答：雙邊濾波的值域權重會把「灰階與中心差很多」的鄰居關掉，它的設計假設是「灰階差很大 = 跨過了邊緣 = 不該平均」。但椒鹽雜訊正是一群孤立的極端值(0 或 255)。當中心點本身就是一個椒鹽點時，所有正常鄰居都因灰階差太大而被關掉，幾乎只剩自己，於是那個錯誤的極端值被原封不動地保留下來。換句話說，雙邊濾波忠實地執行了「保邊」，只是它把雜訊誤認為邊緣。這提醒我們：濾波器的優劣取決於它的假設與實際雜訊型態是否相符。

20. 承圖 5-14：雙邊濾波有空間權重與值域權重兩項。若把值域的標準差 σ_r 設得非常大(例如 1000)，雙邊濾波會退化成什麼?設得非常小又會如何?

    提示與參考解答：σ_r 很大時，值域權重 $\exp(-\Delta^2/2\sigma_r^2)$ 對任何灰階差都接近 1，等於沒有作用，雙邊濾波退化為單純的高斯濾波——保邊能力消失。反之 σ_r 很小時，只有灰階幾乎相同的鄰居才有權重，絕大多數鄰居被關掉，輸出幾乎等於輸入本身——濾波幾乎沒有發生，雜訊原封不動。因此 σ_r 應該設在「雜訊振幅」與「真實邊緣落差」之間：大於雜訊才壓得掉雜訊，小於邊緣落差才留得住邊緣。這也說明了為什麼雙邊濾波的參數需要依雜訊強度調整，不存在萬用值。

21. 承圖 5-18：本章實測，對含雜訊的影像直接銳化會讓 RMSE 由 15.7 惡化到 30.8，先平滑再銳化則降到 10.2。請解釋這三個數字背後的機制。

    提示與參考解答：15.7 是雜訊本身造成的誤差。直接銳化時，銳化的本質是「放大高頻」，而雜訊正是最典型的高頻成分，於是雜訊被連同細節一起放大，誤差幾乎翻倍到 30.8。先平滑再銳化則是先把高頻雜訊壓下去，再放大剩下的(以中低頻為主的)結構，誤差降到 10.2——比原本的雜訊影像還好，因為平滑本身就降低了誤差。結論是順序決定成敗：「先去雜訊，再銳化」。同樣的順序在第 9 章的 Canny(先高斯平滑再求梯度)與第 7 章的還原流程中都會再出現。

22. 計算題：承圖 5-19。若單張影像的雜訊標準差為 24，要把雜訊降到 6 以下，至少需要平均幾張?若每張曝光 1/15 秒，總曝光時間是多久?這對手持拍攝意味著什麼?

提示與參考解答：多影格平均使雜訊以 1/√N 下降，故需 24/√N ≤ 6，即 √N ≥ 4、N ≥ 16，至少 16 張。總曝光時間為 16 × (1/15) ≈ 1.07 秒。對手持拍攝而言，一秒鐘已經遠超過人手能保持不動的時間(一般約 1/30 秒)，因此這 16 張之間必然存在數個到數十個像素的位移，不先對齊就直接平均只會得到一張模糊的照片(見圖 5-20)。這正是夜景模式的真正難點不在「平均」而在「對齊」的原因。

23. 承圖 5-20：八張影格未對齊直接平均的 RMSE 為 18.9，對齊後為 9.5。為什麼未對齊的平均會「雜訊降了，但影像變糊」?這與空間平滑的效果有何異同?

提示與參考解答：平均仍然對每個像素位置累加了八個獨立的雜訊樣本，所以雜訊照樣以 1/√N 下降——這部分有效。但因為影格之間有位移，同一個像素位置在不同影格中對應到場景的不同位置，於是相鄰的真實結構也被一起平均了，等同於做了一次空間模糊，訊號因此損失。與空間平滑的相同之處在於結果都是「模糊」;不同之處在於空間平滑的模糊程度由核大小決定、可控，而未對齊平均的模糊程度由手震幅度決定、不可控，且在畫面各處不一致。對齊之後，平均就只作用在雜訊上，這才是多影格相對於空間平滑的根本優勢。

24. 承圖 5-21 與 5.7.1 節：工業前處理管線為「去雜訊 → 強化對比 → 銳化 → 偵測」。若把「銳化」提到「去雜訊」之前，結果會如何?若把「強化對比」整個拿掉，缺陷偵測會遇到什麼困難?

提示與參考解答：銳化提前會把雜訊放大(見圖 5-18)，後續的去雜訊既要處理原有雜訊、又要處理被放大的雜訊，往往得加大核，結果連缺陷的細節也一併抹掉——兩頭落空。拿掉強化對比則會遇到打光不均的問題：工件右側因為離光源遠而偏暗，若用一個全域門檻去找亮線缺陷，左側會因為整體偏亮而大量誤判，右側的缺陷則因為整體偏暗而漏判。CLAHE 的作用正是把各處的局部對比拉到同一個水準，讓後續可以用一組參數處理整張影像。

25. (綜合題)設計題：某產線的工件表面有方向性的拉絲紋理，缺陷是淺淺的凹痕，而且打光從左側來，導致右側偏暗。請設計一條前處理管線，說明每一步的選擇理由，以及你會如何驗證每一步真的有幫助。

提示與參考解答：本題無標準答案，評分重點在理由與驗證方式。一條合理的管線：(1)先處理打光不均——用 CLAHE 做局部對比均化，或估計背景打光場後相除;(2)去雜訊要選保邊的方法，因為凹痕本身就是淺的，均值或大核高斯會直接抹掉

> *它，宜用雙邊或小核中值;(3)方向性紋理是干擾——可考慮沿紋理方向做異向性平滑，或改到頻率域用方向性帶阻濾波把紋理的週期成分濾掉(第 6 章);(4)凹痕在側光下通常呈現暗線，宜用型態學黑帽運算而非頂帽(第 10 章);(5)最後以面積與長寬比篩選候選區域。驗證方式最關鍵：每一步都要有量化指標，例如缺陷區與背景的對比(CNR)、良品的誤報率、不良品的漏檢率，並且在固定的一批標註樣本上逐步比較——「看起來比較清楚」不是工程上可接受的驗收標準。*

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 3(空間域強化之標準參考)。
- Levoy, M., & Pritch, Y., Night Sight: Seeing in the Dark on Pixel Phones, Google Research Blog, Nov. 2018.(本章叫獸開講之技術來源)
- Liba, O., et al., Handheld Mobile Photography in Very Low Light, ACM TOG, 2019.(Night Sight 完整論文;運動測光與學習式白平衡)
- Hasinoff, S. W., et al., Burst Photography for High Dynamic Range and Low-Light Imaging on Mobile Cameras, ACM TOG, 2016.(HDR+ 原始論文)
- Zuiderveld, K., Contrast Limited Adaptive Histogram Equalization, Graphics Gems IV, 1994.(CLAHE 原始文獻)
- Tomasi, C., & Manduchi, R., Bilateral Filtering for Gray and Color Images, ICCV, 1998.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 6 章 頻率域影像處理

本章學習目標

- 以「訊號是多個弦波疊加」的觀點理解傅立葉轉換，說明空間頻率在影像中的意義。
- 計算與顯示二維離散傅立葉轉換(DFT)的頻譜，並正確執行頻譜中心化與對數壓縮。
- 解讀頻譜圖：由亮線方向、亮點位置反推影像中的紋理、週期干擾與方向性結構。
- 解釋振幅譜與相位譜各自攜帶的資訊，說明為何相位對影像結構更關鍵。
- 設計並實作理想、Butterworth 與高斯的低通、高通、帶通與帶阻濾波器，理解振鈴效應的成因。
- 以陷波濾波去除週期性雜訊與摩爾紋，並以相位相關法完成影像對齊。
- 陳述卷積定理，並據以判斷同一個濾波工作該在空間域或頻率域執行。
- 說明頻率域觀點與取樣定理、空間濾波、影像還原、特徵擷取、影像壓縮之間的連結。

叫獸開講|傅立葉：一篇被退稿的論文，兩百年後撐起你的每一張 JPEG

1807 年 12 月，一位法國數學家把一份關於熱傳導的長篇手稿送進巴黎的法蘭西學院。他叫 Jean-Baptiste Joseph Fourier(傅立葉)，履歷相當奇特：革命年代差點上斷頭台，曾隨拿破崙遠征埃及，回國後被派去當省長，而這篇改變世界的論文，是他在管理行政事務之餘寫出來的。手稿的核心主張大膽到近乎狂妄：任何函數，哪怕是不連續的、有稜有角的，都可以表示成一系列正弦與餘弦函數的疊加。

審查委員的陣容堪稱豪華：拉普拉斯(Laplace)、拉格朗日(Lagrange)、蒙日(Monge)、拉克魯瓦(Lacroix)——三位都是傅立葉在高等師範學院的老師，蒙日還是他遠征埃及的同袍。然而，論文被擋下了。史料記載，雖然拉普拉斯、拉克魯瓦與蒙日傾向支持出版，拉格朗日堅決反對，因為傅立葉處理三角級數的方式

與他自己在 1750 年代所主張的大不相同;審查者也批評證明不夠嚴謹、不夠一般化。1811 年，科學院以熱傳導為題舉辦論文獎，傅立葉提交修訂版並奪得首獎，卻仍附帶「缺乏嚴謹性」的評語。傅立葉認為批評並不公平，繼續修改;完整的《熱的解析理論》直到 1822 年才出版——那時拉格朗日已經過世。

兩百年後回看這場爭論：拉格朗日的質疑在數學上並非全無道理(級數收斂的嚴格條件確實要等後人補上)，但傅立葉的直覺是對的，而且對得驚人。今天，傅立葉轉換是 JPEG 影像壓縮、MP3 音訊、MRI 醫學造影、5G 通訊、地震波分析共同的數學心臟。你手機裡每一張照片、每一通電話，都在替這篇被退稿的論文作證。

這一章要教的觀念，說穿了就是換一個角度看世界：空間域是「實體」——你看到的是一個個像素的明暗;頻率域是「思想」——你看到的是這張影像由哪些「變化的快慢」組成。有些問題在空間域怎麼看都無解(例如布滿全圖的週期性條紋雜訊)，換到頻率域卻一眼就能揪出元凶，伸手把它拔掉。學會在兩個域之間自由切換，是影像工程師的成年禮。

而且這一章的影響力遠不止於本章：第 3 章的取樣定理、第 5 章每一個濾波器、第 7 章的影像還原、第 11 章的 JPEG 壓縮與小波、第 13 章的特徵擷取，甚至第 15 章卷積神經網路為什麼有效——背後都站著傅立葉。6.6 節會把這些線索一次串起來。

思考題：如果拉格朗日當年沒有擋下這篇論文，傅立葉分析會更早成熟嗎?從這段歷史看，學術審查的「保守」是阻力還是必要的品質關卡?

6.1 從弦波到影像：傅立葉的基本觀念

6.1.1 一維直覺：從聲音開始

先想像聲音：一個和弦聽起來是單一的聲響，但它其實是好幾個不同頻率的音同時響起。訓練有素的耳朵能「聽出」其中有 Do、Mi、Sol 三個音，以及各自的響度—

—傅立葉分析做的就是這件事的數學版本。任何週期訊號都可以分解成不同頻率的正弦與餘弦之和;非週期訊號則以積分形式的傅立葉轉換處理。轉換的結果不再以「時間」或「位置」為軸，而是以「頻率」為軸，稱為頻譜(Spectrum)。

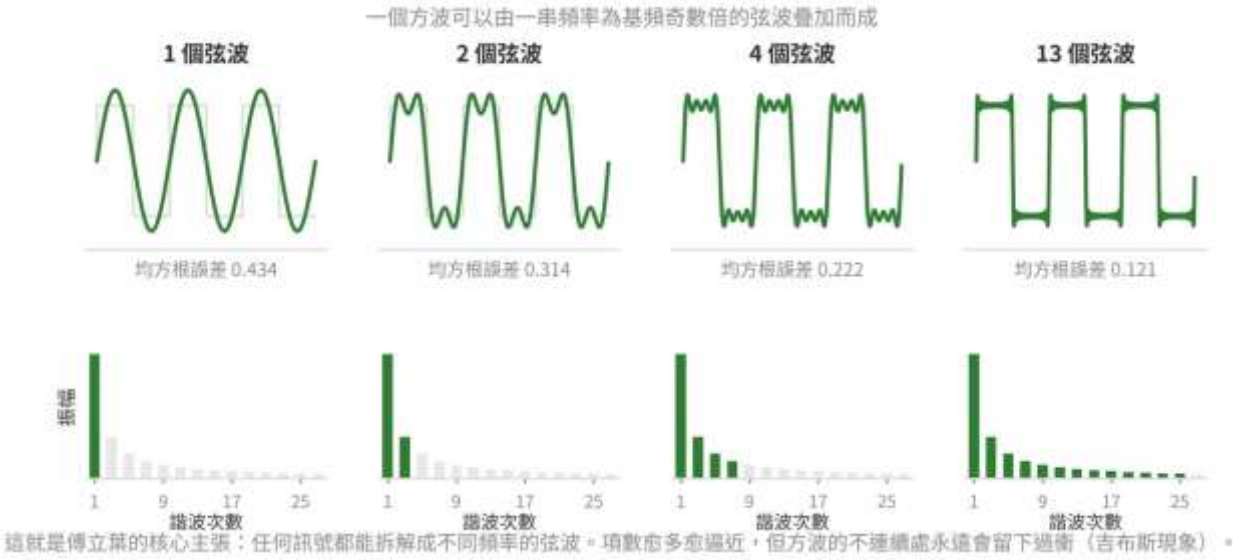


圖 6-1　方波的弦波疊加：項數愈多愈逼近

這個「同一個東西，兩種表示法」的觀念極為重要，值得用一個比喻鞏固：同一首曲子，可以用錄音波形表示(時域)，也可以用五線譜表示(頻域)。兩者資訊完全等價、可以互相轉換，但適合的用途不同——想剪掉開頭三秒，看波形最方便;想把整首曲子升一個八度，看樂譜最方便。影像處理也是如此：想裁切一塊區域，用空間域;想去除一組固定頻率的干擾，用頻率域。

6.1.2 影像中的「頻率」是什麼

影像沒有時間軸，它的頻率指的是空間頻率——灰階隨位置變化的快慢，單位可以想成「每單位距離有幾個明暗週期」。低頻對應緩慢變化的成分：平坦的牆面、漸層的天空、物體的大塊區域、整體亮度;高頻對應劇烈變化的成分：邊緣、細節紋理、文字筆畫，以及雜訊。



圖 6-2　同一張影像拆成低頻成分與高頻成分

這個對應關係是整章的鑰匙，也是所有頻率域操作的設計邏輯：想模糊影像，就削弱高頻(低通濾波);想強調邊緣，就保留或放大高頻(高通濾波);想抓出特定粗細的紋理，就只留下某個頻帶(帶通濾波);想去除規律的條紋干擾，就在頻譜上找到那幾個異常亮點，精準消除(帶阻或陷波濾波)。四種濾波器，四種目的，全部都是同一句話的不同版本：決定哪些「變化的快慢」可以留下來。

6.1.3 基底影像：每個頻率成分長什麼樣子

二維傅立葉轉換把影像分解成一組「基底影像」的加權和，而每一張基底影像，都是一張斜條紋圖。頻率座標 (u, v) 決定這張條紋圖的三個性質：條紋的疏密由 u 與 v 的合成大小(離頻譜中心的距離)決定，離中心愈遠、條紋愈密;條紋的方向由 u 與 v 的比例(相對於中心的角度)決定，且條紋方向與頻譜上該點的方位垂直;而係數的振幅與相位，則決定這張條紋圖在原圖中占多少比重、以及擺放的位移。

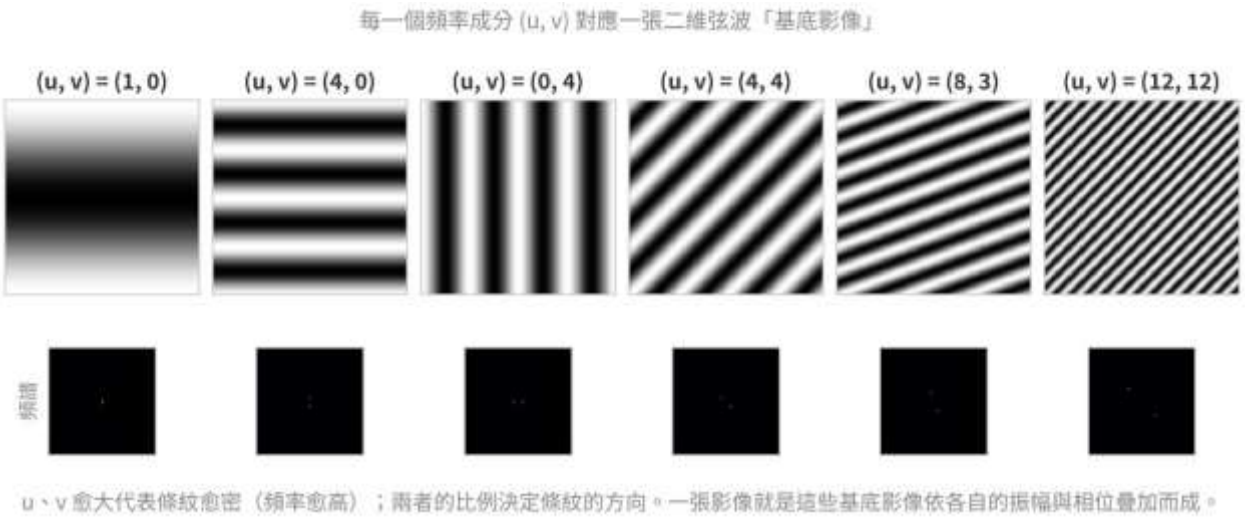


圖 6-3　六個頻率成分 (u, v) 對應的基底影像與其頻譜

一張影像，就是無數張這種條紋圖疊加而成——這正是傅立葉當年主張的「任何函數都能用弦波疊加表示」在二維的版本。建議學生務必在 Lab 6-1 中親手驗證：先用 np.sin 合成一張純條紋圖，看它的頻譜是否真的只有兩個對稱亮點;再把條紋轉 45 度，看亮點是否跟著轉。這個實驗做過一次，對頻譜的直覺就永遠建立起來了。

順帶一提「兩個對稱亮點」的由來：實數影像的傅立葉轉換具有共軛對稱性，F(-u, -v) 是 F(u, v) 的共軛複數，因此頻譜圖必然以中心點對稱。這個性質不只是數學細節——6.4 節設計陷波濾波器時，忘了處理對稱點是最常見的錯誤。

6.1.4 本章在全書中的位置

頻率域是本書的樞紐觀念之一，它向前解釋了第 3、5 章的許多「為什麼」，向後支撐了第 7、11、13、15 章的核心方法。閱讀時建議隨時對照：

- 與第 3 章(取樣與量化):Nyquist 取樣定理本身就是頻域敘述——取樣頻率不足時，高頻成分在頻譜上「摺疊」回低頻區偽裝成假訊號，這就是混疊。摩爾紋是它最常見的視覺表現。
- 與第 5 章(空間域強化)：每一個空間濾波核都有一張「頻率面孔」。均值濾波是粗糙的低通、高斯濾波是平滑的低通、拉普拉斯是高通、Unsharp Masking 是高頻強調。理解這一點，調參就不再是盲目試誤。

- 與第 7 章(影像還原)：劣化模型 g = h * f + n 在頻率域是 G = H·F + N，卷積變成乘法，還原變成除法;而退化函數 H 本身就是一個低通濾波器。
- 與第 9 章(影像分割):Canny 的第一步高斯平滑是低通(壓制雜訊)，第二步梯度運算是高通(找邊緣)——一個演算法內同時用到兩種濾波，順序不能顛倒。
- 與第 11 章(小波與正交轉換):DCT 是 JPEG 的核心，與 DFT 同宗;小波則補上傅立葉最大的弱點——它只告訴你「有哪些頻率」，不告訴你「頻率出現在哪裡」。
- 與第 13 章(特徵擷取):Gabor 濾波器是帶通加方向選擇的紋理特徵萃取器;SIFT 的高斯尺度空間本質上是一組低通濾波器;相位相關法則直接用頻域相位做影像對齊。
- 與第 15 章(深度學習):CNN 的卷積核在訓練後，常自發演化成邊緣偵測器與方向性帶通濾波器——與人工設計的濾波器驚人地相似。
- 與第 17 章(邊緣部署):FFT 的運算成本、記憶體需求與 GPU 加速可行性，是即時系統設計必須評估的項目。

6.2 二維離散傅立葉轉換(DFT)與 FFT

6.2.1 定義與快速演算法

對 M×N 的數位影像 f(x, y)，二維離散傅立葉轉換定義為 F(u, v) = Σx Σy f(x, y)·exp[-j2π(ux/M + vy/N)]，反轉換則加上 1/(MN) 的係數與正號指數。直接依定義計算的複雜度是 O(M²N²)，對一張 1024×1024 的影像大約需要一兆次運算，根本跑不動;1965 年 Cooley 與 Tukey 提出的快速傅立葉轉換(FFT)利用了指數項的對稱性與週期性，以分而治之的方式把複雜度降到 O(MN·log(MN))——同樣一張影像降到約兩千萬次運算，快了五萬倍。這個演算法讓頻率域處理從理論走進實用，被公認為二十世紀最重要的演算法之一。

實作上還有一個效率細節：FFT 在長度為 2 的冪次時效率最高，因此有些函式庫

會建議把影像補零到 2 的冪次尺寸。OpenCV 提供 cv2.getOptimalDFTSize() 來查詢最佳尺寸，對即時系統而言值得使用。

6.2.2 頻譜的中心化與顯示

直接轉換得到的頻譜，零頻率(直流分量，代表整張影像的平均亮度)位於左上角，四個角落各占一部分低頻，不易觀察。慣例是做頻譜中心化(fftshift)，把零頻率移到影像正中央，如此一來「愈靠近中心愈低頻、愈往外愈高頻」，直覺得多。

另一個必要步驟是取對數。這裡正好呼應第 5 章的對數轉換：直流分量的數值往往比其他成分大上好幾個數量級(因為自然影像的能量絕大部分集中在低頻)，不取對數的話，整張頻譜圖只會看到中央一個亮點，其餘全黑，什麼資訊都讀不到。取對數壓縮了動態範圍，讓微弱的高頻成分也看得見——這正是第 5 章所說「對數轉換適合處理動態範圍極大的影像」的最佳實例。

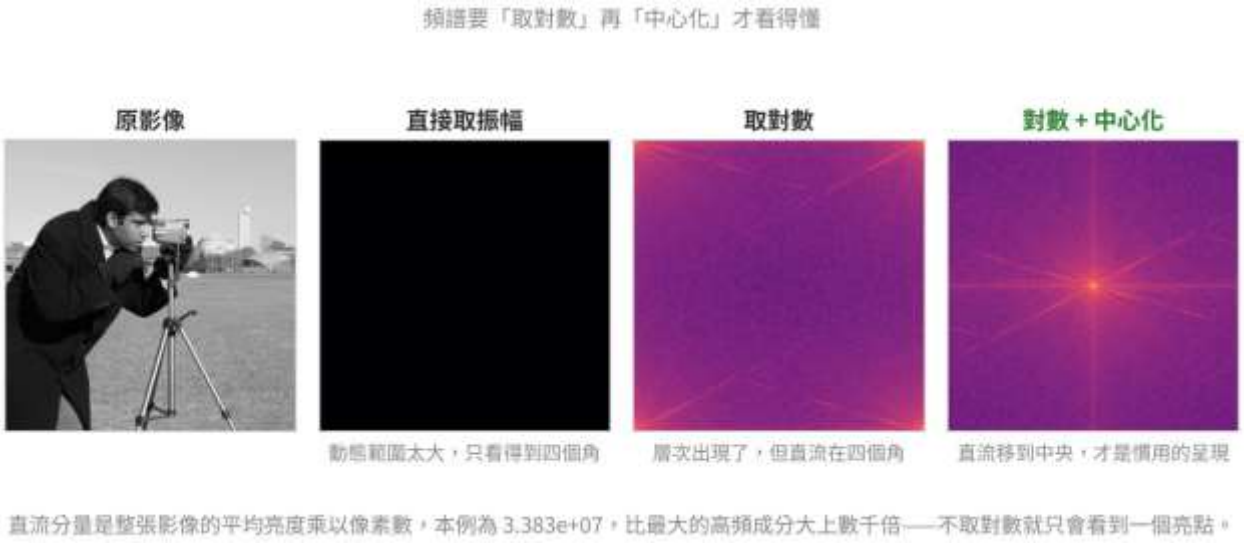


圖 6-4　頻譜的正確顯示：取對數與中心化的四個階段

程式 6-1 計算與顯示二維頻譜

```
import cv2
import numpy as np

gray = cv2.imread("building.jpg", cv2.IMREAD_GRAYSCALE)

F = np.fft.fft2(gray)              # 二維 FFT
Fshift = np.fft.fftshift(F)        # 零頻率移到中央
```

```
magnitude = np.abs(Fshift)          # 振幅譜
spectrum = 20 * np.log(magnitude + 1)          # 取對數才看得見
spectrum = cv2.normalize(spectrum, None, 0, 255,
                         cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("spectrum.png", spectrum)

# 反轉換:確認可以完整還原
back = np.fft.ifft2(np.fft.ifftshift(Fshift))
back = np.abs(back).astype(np.uint8)
print("還原誤差:", np.max(cv2.absdiff(gray, back)))   # 應接近 0
```

6.2.3 頻譜判讀指南

會算頻譜只是第一步，會「讀」頻譜才是真功夫。以下是常見的對應關係，建議做成隨身小抄：

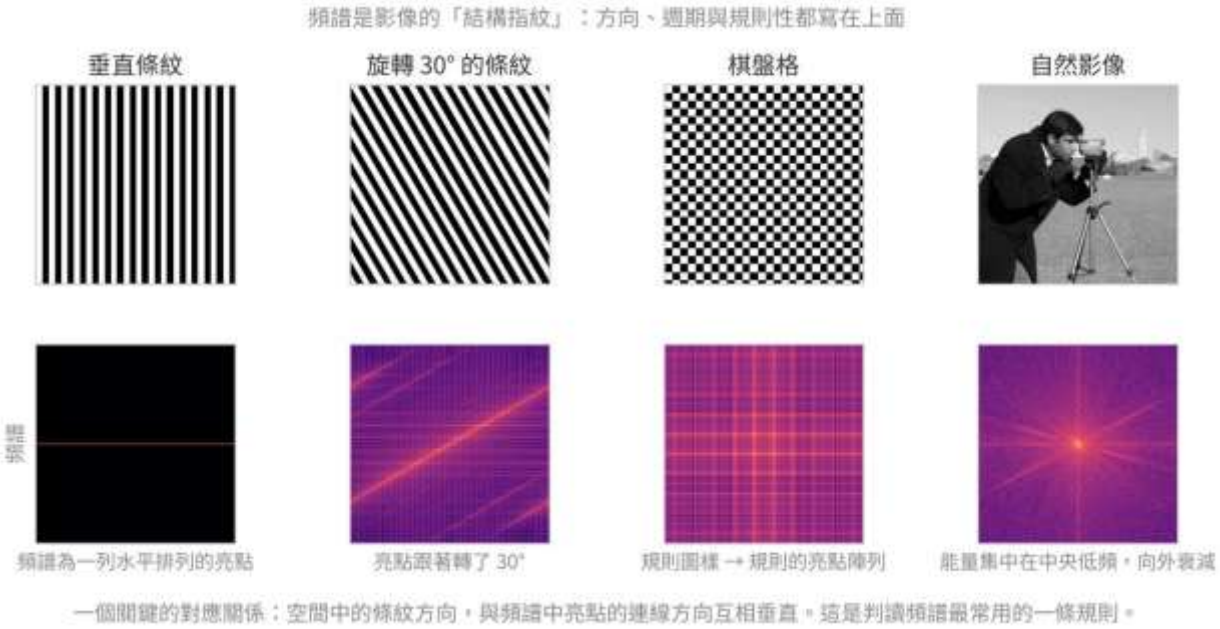


圖 6-5　四種影像的頻譜：方向、週期與規則性的指紋

影像中的內容	頻譜上的表現	相關章節
整體亮度(平均值)	中心的直流分量，通常最亮	第 5 章 灰階轉換
大面積平坦區、漸層	能量集中在中心附近的低頻	第 5 章 平滑濾波
銳利邊緣、細節、文字	高頻區有明顯能量	第 9 章 邊緣偵測
垂直方向的條紋或欄杆	沿水平方向延伸的亮線	本章 6.3.4 方向濾波
規律的週期性干擾	對稱的孤立亮點	本章 6.4 陷波濾波

隨機雜訊	全頻域均勻分布的底噪	第 7 章 影像還原
失焦或運動模糊	高頻能量被壓低，或出現暗紋	第 7 章 反卷積
影像被旋轉	整個頻譜同步旋轉相同角度	第 4 章 幾何轉換
影像被平移	振幅譜不變，只有相位改變	本章 6.7.2 相位相關

表 6-1 頻譜判讀對照表

最後兩列特別值得玩味，它們是傅立葉轉換的兩個重要性質：旋轉性質(影像旋轉，頻譜同步旋轉)與平移性質(影像平移，振幅譜完全不變，資訊全部藏在相位裡)。這兩個性質不只是理論趣味——6.7.2 節的相位相關對齊法，正是直接建立在平移性質上的實用演算法。

6.2.4 振幅與相位：哪個更重要?

傅立葉轉換的結果是複數，可拆成振幅譜(每個頻率成分的強度)與相位譜(每個成分的位移)。有一個經典實驗能揭示兩者的分工：取 A、B 兩張照片，用 A 的振幅配 B 的相位重建影像，結果看起來像什麼?答案是——像 B。

這說明相位攜帶了影像的結構資訊(邊緣在哪裡、輪廓如何排列、物體位於何處)，而振幅主要反映各頻率的能量分布(整體的「質感」與對比強弱)。直覺上可以這樣理解：振幅告訴你「有哪些條紋圖參與疊加、各占多少比重」，相位告訴你「這些條紋圖要對齊在哪裡」——而讓邊緣在正確位置浮現的，正是眾多條紋圖在該處剛好同相疊加的結果。

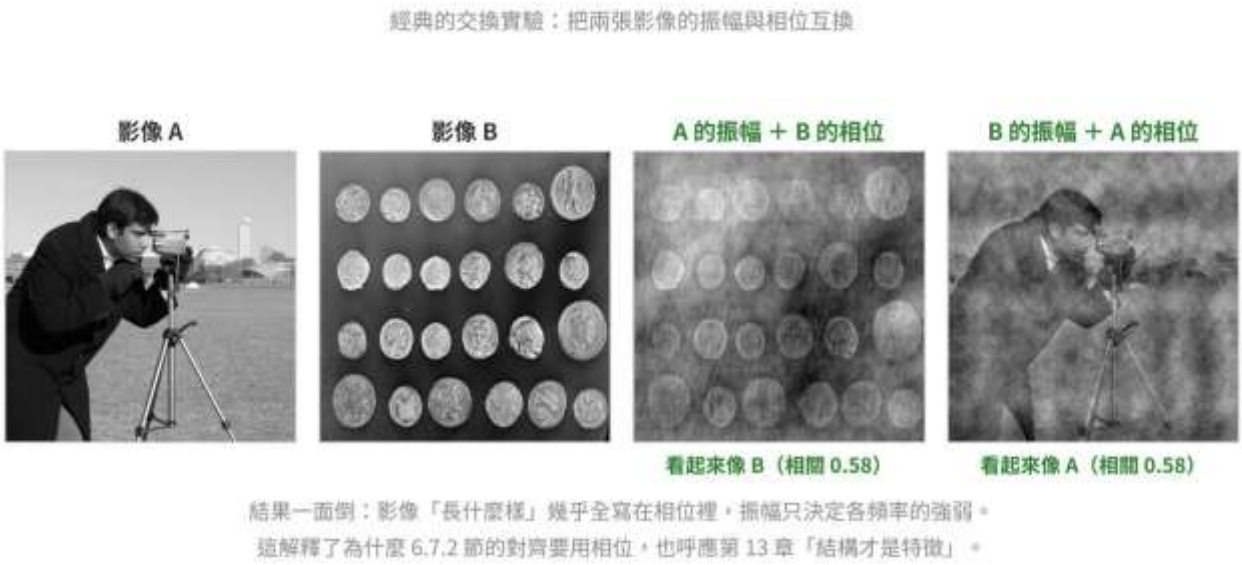


圖 6-6　振幅與相位的交換實驗：結構寫在相位裡

這個發現有深遠的實務意義：許多影像處理與辨識演算法因此特別重視相位資訊;本章 6.7.2 節的相位相關法、以及某些對光照變化強健的特徵描述子，都是這個觀念的產物。

程式 6-2 振幅與相位的經典交換實驗

```
import cv2, numpy as np

a = cv2.imread("cat.jpg", 0).astype(np.float32)
b = cv2.imread("car.jpg", 0).astype(np.float32)
b = cv2.resize(b, (a.shape[1], a.shape[0]))

Fa, Fb = np.fft.fft2(a), np.fft.fft2(b)

# A 的振幅 + B 的相位
mixed = np.abs(Fa) * np.exp(1j * np.angle(Fb))
out = np.abs(np.fft.ifft2(mixed))
cv2.imwrite("amp_a_phase_b.png",
            cv2.normalize(out, None, 0, 255, cv2.NORM_MINMAX))

# 對照組:只保留相位(振幅全設為 1)
phase_only = np.exp(1j * np.angle(Fa))
po = np.abs(np.fft.ifft2(phase_only))
cv2.imwrite("phase_only.png",
            cv2.normalize(po, None, 0, 255, cv2.NORM_MINMAX))
# 觀察:只有相位仍能看出輪廓,只有振幅則什麼都看不出來
```

6.2.5 DFT 的隱藏假設：週期性與邊界效應

這是一個容易被忽略、卻經常在實作中咬人的細節。DFT 隱含假設影像是週期性

延拓的：它認為影像的右邊界緊接著左邊界、下邊界緊接著上邊界，像貼壁紙一樣無限重複。但真實照片的左右邊界通常亮度差異很大，拼接處就出現一道人造的劇烈跳變——這道假邊緣會在頻譜上產生沿水平與垂直方向的十字亮線，經常被初學者誤讀為「影像中有強烈的橫豎紋理」。

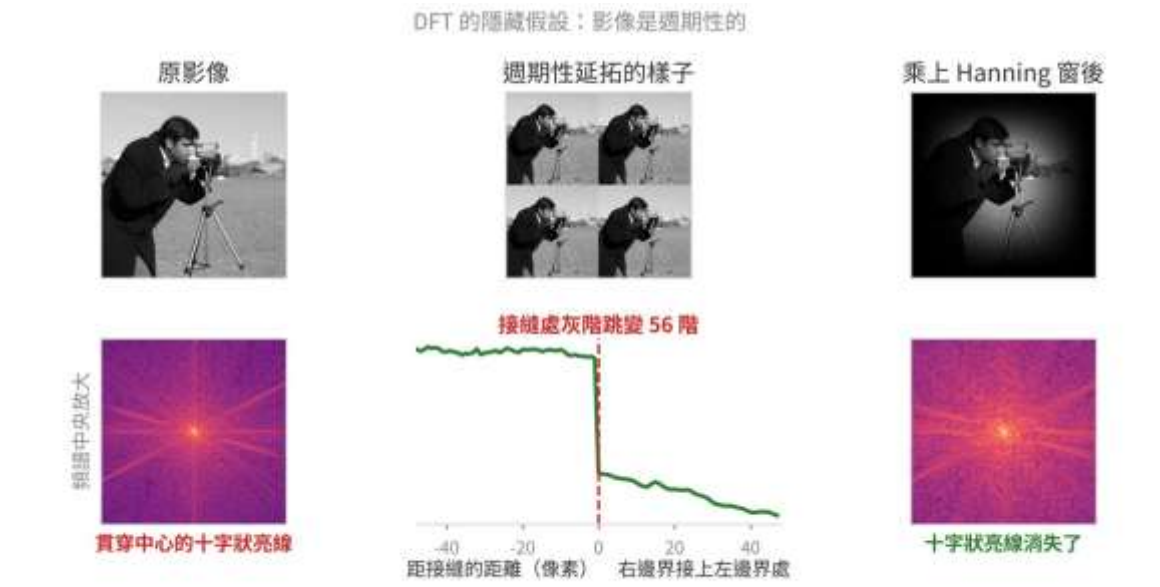


圖 6-7　DFT 的週期性假設、十字狀假亮線與 Hanning 加窗

解決方式有二：一是加窗(Windowing)，在做 FFT 前把影像乘上一個邊緣衰減到零的視窗函數(如 Hanning 窗)，消除邊界跳變;二是補零與鏡射填充，把影像擴大後再轉換。在做精確的頻譜分析或第 6.7.2 節的相位相關對齊時，加窗幾乎是必要步驟。

6.2.6 推導：平移只改變相位，不改變振幅

6.2.4 節說「影像長什麼樣幾乎全寫在相位裡」，而 6.7.2 節的相位相關法更是直接拿相位來求位移。這兩件事背後是同一個性質，值得推導一次，因為它只有兩行。

設影像 f(x, y) 的 DFT 為 F(u, v)。把影像平移 (Δx, Δy) 之後，新影像為 f(x − Δx, y − Δy)。代入 DFT 的定義並做變數代換 x′ = x − Δx、y′ = y − Δy，指數項可以拆出一個與 x′、y′ 無關的常數因子，結果為 F(u, v) · exp[−j2π(uΔx + vΔy) / N]。這個因子的絕對值恆為 1，因此振幅 |F| 完全不變，改變的只有相位——而且相位的改變量與頻

率 (u, v) 成正比。

本書以一張 512×512 的影像做精確的整數循環位移實測：位移前後的振幅最大相對差為 5.5×10^{-17}（純浮點誤差），而相位差與理論值 $-2\pi \cdot v \cdot \Delta x / N$ 的最大偏離為 1.0×10^{-13}。兩者都在機器精度之內，性質成立得乾乾淨淨。

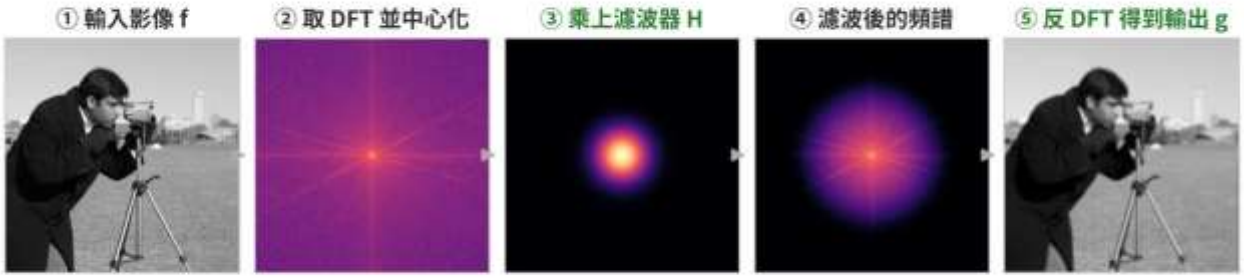
這個性質有三個直接的推論。其一，只看振幅無法分辨「同一個場景拍在不同位置」——這解釋了圖 6-6 交換實驗的結果。其二，若把兩張只差一個平移的影像相除（在頻率域），商的相位恰好是一條線性相位斜面，斜率就是位移量，反轉換後會得到一個脈衝——這正是相位相關法（圖 6-19）。其三，由於相位的變化量與 (u, v) 成正比，高頻成分對位移最敏感，這也是為什麼對齊演算法在紋理豐富的影像上比在平坦影像上準確得多。

6.3 頻率域濾波：低通、高通、帶通

6.3.1 濾波的基本流程

頻率域濾波的標準流程是五個步驟：對影像做 FFT;將頻譜中心化;把頻譜乘上一個濾波器轉移函數 H(u, v);反中心化;做反 FFT 取實部。整個過程可寫成 $g = \mathrm{IFFT}[H \cdot \mathrm{FFT}(f)]$。設計濾波器，就是設計 H 這張「遮罩」——它決定哪些頻率通過、哪些被削減。

頻率域濾波的五個步驟：所有設計上的選擇都集中在第 ③ 步



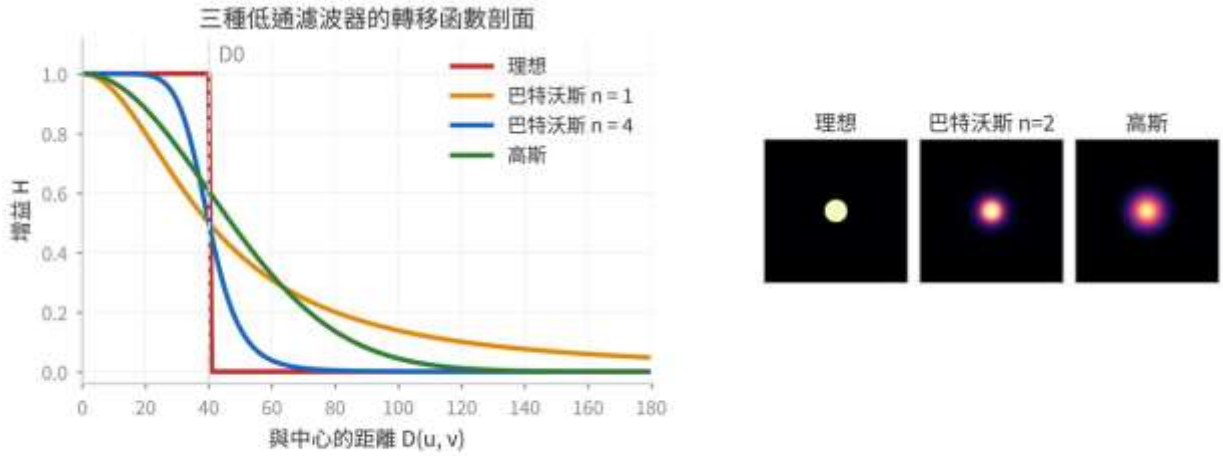
步驟 ① ② ④ ⑤ 永遠相同，不同的濾波器只差在 H 長什麼樣子——這正是頻率域的魅力：把「設計濾波器」變成「畫一條曲線」。

圖 6-8　頻率域濾波的五個步驟

請特別注意這裡的「乘上」二字：在頻率域，濾波是逐點相乘，這是最單純的運算;而同一件事在空間域是卷積，要滑動視窗、逐點相乘再加總。同一個操作，在兩個域的複雜度天差地別——這正是 6.5 節卷積定理的精髓，也是頻率域方法最重要的工程價值。

6.3.2 低通濾波器的三種設計

1. 理想低通濾波器(Ideal)：在截止頻率 D0 內完全通過、之外完全阻斷，像用剪刀直接剪掉。看似完美，卻會產生嚴重的振鈴效應(Ringing)：影像邊緣附近出現一圈圈同心漣漪。原因是「頻率域的圓形方波」對應「空間域的類 sinc 函數」，而 sinc 有無限延伸的振盪旁瓣——這是卷積定理的直接後果，也是本章最重要的反面教材：數學上最乾脆的做法，工程上往往最糟糕。
2. Butterworth 低通濾波器：轉移函數 H(u,v) = 1 / [1 + (D(u,v)/D0)^(2n)]，以階數 n 控制過渡的陡峭程度。n = 1 時極為平滑、幾乎無振鈴;n 增大則逐漸逼近理想濾波器，振鈴隨之出現。它提供了「銳利度」與「振鈴」之間的連續旋鈕，實務上很受歡迎;n = 2 是常用的預設值。
3. 高斯低通濾波器：H(u,v) = exp[-D²(u,v) / (2D0²)]。高斯函數的傅立葉轉換仍是高斯函數，沒有負值旁瓣，因此完全不產生振鈴，過渡最平滑。代價是截止不夠俐落——想擋掉的高頻總會漏一點過去。但在多數應用中這反而是優點，因為自然影像本來就沒有明確的頻率邊界。



三者的差別只在「從通過到阻斷」的那一段有多陡。理想濾波器在 D0 處垂直落下，看似完美，代價見圖 6-10。

圖 6-9　理想、Butterworth、高斯三種低通的轉移函數剖面

三者的取捨可以歸納成一句話：頻率域截得愈乾脆，空間域的副作用愈嚴重。這是不確定性原理在影像處理中的體現——一個函數不可能在兩個域同時都很「窄」。第 11 章談小波時，我們會再次遇到這個根本限制。



理想低通在頻率域是一個硬邊的圓盤，其空間域對應是帶無限振盪旁瓣的類 sinc 函數，卷積後在邊緣附近產生同心漣漪，即振鈴效應。高斯的傅立葉轉換仍是高斯、沒有負旁瓣，因此完全無振鈴——這是實務上幾乎總是選高斯的理由。

圖 6-10　振鈴效應的量化：階躍邊緣上的過衝與下衝

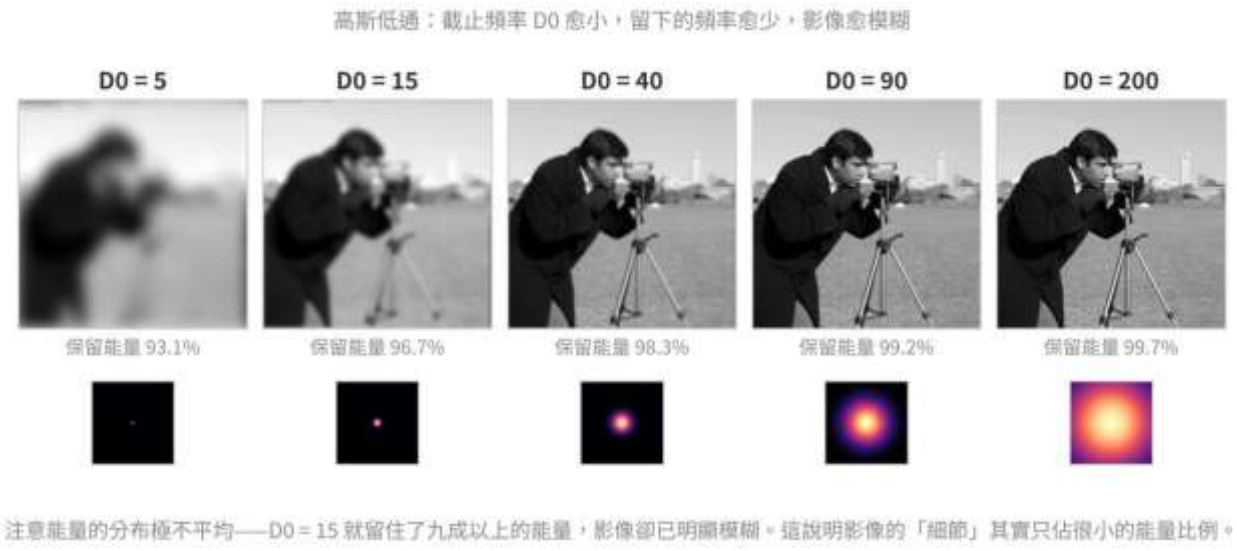


圖 6-11　截止頻率 D0 的掃描：保留能量與視覺清晰度不同步

6.3.3 高通濾波與高頻強調

高通濾波器的轉移函數是低通的補集：Hhp(u,v) = 1 - Hlp(u,v)。單純的高通濾波會把直流分量也濾掉，結果是一張平均亮度為零的影像——邊緣浮現但整體呈灰黑，大部分區域資訊喪失。

實務上更常用的是高頻強調濾波(High-Frequency Emphasis):H(u,v) = a + b·Hhp(u,v)，其中 a 保留部分低頻(通常取 0.5 至 1),b 控制高頻的增益(通常大於 1)。這樣既強調了邊緣，又保留了原本的亮度層次。請把它與第 5 章的 High-Boost 銳化 g = f + k·(f - blur(f)) 並列比較：兩者做的是完全相同的事，只是一個在空間域用減法實現、一個在頻率域用加權實現。這種「同一個操作的兩張面孔」在本章反覆出現，是理解影像處理的關鍵心法。

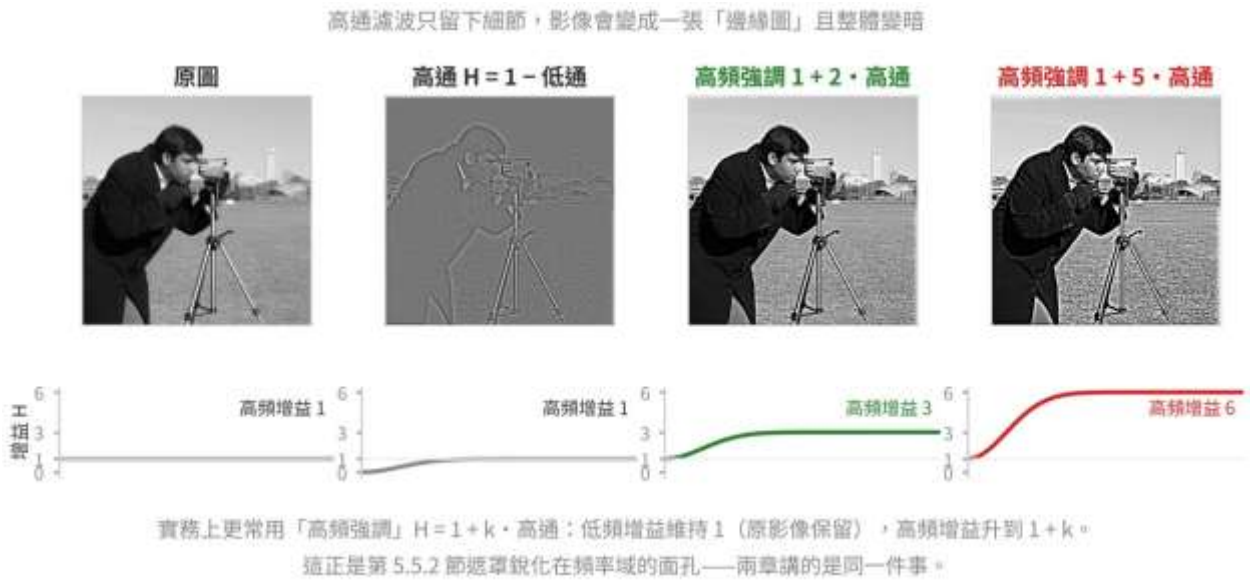


圖 6-12　高通與高頻強調：轉移函數與增益曲線

另一個經典應用是同態濾波(Homomorphic Filtering)，用於同時壓縮動態範圍並增強對比。它的理論基礎是：影像可視為「照度」與「反射率」的乘積，照度變化緩慢(低頻，對應打光不均)，反射率變化劇烈(高頻，對應物體本身的紋理)。做法是先取對數把乘法變成加法，再用一個「低頻壓抑、高頻放大」的濾波器分別處理，最後取指數還原。這對第 17 章 AOI 中「打光不均勻的工件表面」特別有效——它能在拉平打光差異的同時，強化瑕疵的紋理對比。

程式 6-3 低通、高通與高頻強調濾波

```
import cv2, numpy as np

gray = cv2.imread("building.jpg", 0).astype(np.float32)
M, N = gray.shape
u = np.arange(M).reshape(-1, 1) - M // 2
v = np.arange(N).reshape(1, -1) - N // 2
D = np.sqrt(u**2 + v**2)          # 到頻譜中心的距離

D0 = 50                           # 截止頻率

H_ideal  = (D <= D0).astype(np.float32)              # 理想低通
H_butter = 1 / (1 + (D / D0)**(2 * 2))               # n=2 Butterworth
H_gauss  = np.exp(-(D**2) / (2 * D0**2))             # 高斯低通

Fshift = np.fft.fftshift(np.fft.fft2(gray))

def apply_filter(Fshift, H):
    out = np.fft.ifft2(np.fft.ifftshift(Fshift * H))
    return np.clip(np.abs(out), 0, 255).astype(np.uint8)
```

```
for name, H in [("ideal", H_ideal), ("butter", H_butter),
                ("gauss", H_gauss)]:
    cv2.imwrite(f"lowpass_{name}.png", apply_filter(Fshift, H))
    # 比較 ideal 邊緣的同心漣漪(振鈴)與 gauss 的平順

# --- 高通:取補集 ---
cv2.imwrite("highpass.png", apply_filter(Fshift, 1 - H_gauss))

# --- 高頻強調:a + b*Hhp,對照第 5 章的 High-Boost ---
a, b = 0.5, 2.0
H_emph = a + b * (1 - H_gauss)
cv2.imwrite("emphasis.png", apply_filter(Fshift, H_emph))
```

比較項目	理想	Butterworth	高斯
過渡帶	無(垂直截斷)	可調(階數 n)	平滑
振鈴效應	嚴重	n 大時出現	無
截止俐落度	最佳	中等偏佳	較差
空間域對應	sinc(有振盪旁瓣)	介於兩者之間	高斯(無旁瓣)
適用場合	教學示範	一般工程應用	要求無偽影者

表 6-2 三種頻率域濾波器比較

6.3.4 帶通與方向性濾波：通往特徵擷取的橋

低通與高通都是以「距離中心多遠」為唯一標準，屬於各向同性(isotropic)濾波器——不管方向，只管頻率高低。但影像中的資訊往往是有方向性的：布料的織紋、木材的紋理、晶圓上的線路、道路上的車道線，都有明確的走向。這時就需要更精緻的工具。

帶通濾波器(Bandpass)只讓某個頻率區間通過，以 Hbp = Hlp(D0 大) - Hlp(D0 小) 的方式構造。它的用途是「挑出特定粗細的結構」：想找出寬度約十像素的刮痕，就設計一個中心頻率對應該尺度的帶通濾波器，比單純的高通乾淨得多，因為它同時排除了更細的雜訊與更粗的背景起伏。

方向性濾波器則進一步在頻譜上只保留某個角度扇形區域的成分。因為「頻譜上

的方位」對應「影像中條紋的垂直方向」，只要在頻譜上留下水平方向的扇形，就能萃取出影像中的垂直條紋。這正是織品瑕疵檢測、晶圓線路檢查的常用手法——把規律的背景紋理當成「已知頻率」濾掉，剩下的異常就是瑕疵(見 6.7.3 節案例)。

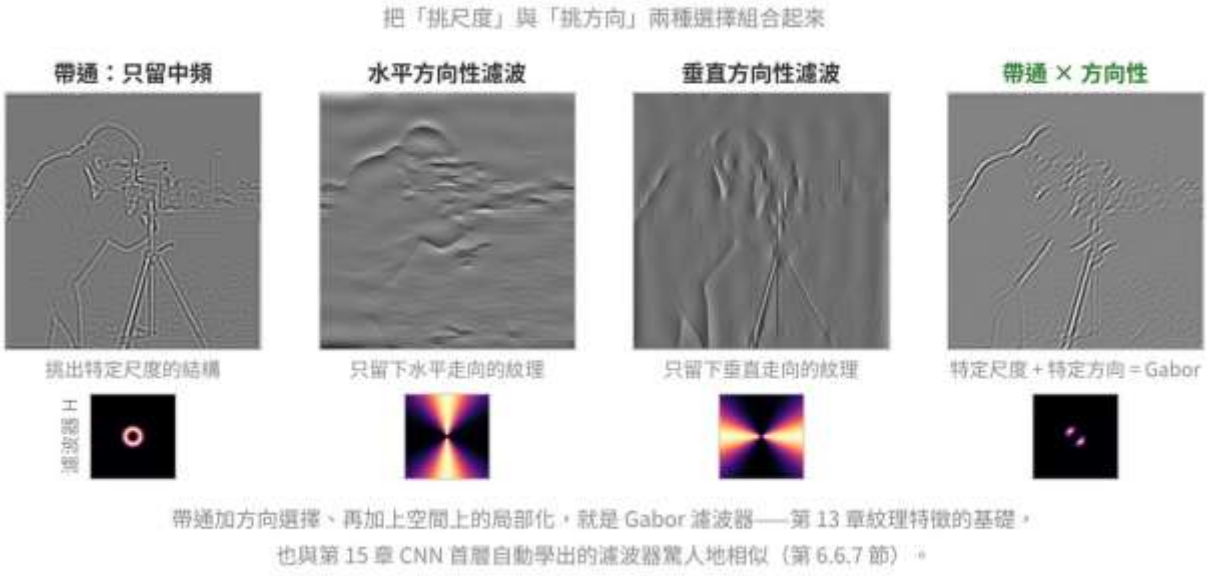


圖 6-13　帶通與方向性濾波：挑尺度，也挑走向

把「帶通」與「方向選擇」兩者結合，再加上空間域的局部化(用高斯視窗限制作用範圍)，就得到了著名的 Gabor 濾波器——第 13 章特徵擷取的重要工具，也是早期人臉辨識與紋理分類的主力。有趣的是，神經科學研究發現哺乳動物視覺皮層 V1 區的簡單細胞，其感受野特性與 Gabor 濾波器高度相似;而第 15 章會看到，CNN 第一層的卷積核在訓練後，也常自發演化出類似的方向性帶通結構。人腦、人工設計、機器學習，三條路走到了同一個答案——這是本書最耐人尋味的觀察之一。

程式 6-4 帶通與方向性濾波

```
import cv2, numpy as np

M, N = gray.shape
u = np.arange(M).reshape(-1, 1) - M // 2
v = np.arange(N).reshape(1, -1) - N // 2
D = np.sqrt(u**2 + v**2)
theta = np.arctan2(u, v)                    # 頻譜上每點的方位角

# --- 帶通:保留 D0_low ~ D0_high 之間的頻率 ---
D_low, D_high = 20, 60
H_bp = (np.exp(-(D**2) / (2 * D_high**2)) -
        np.exp(-(D**2) / (2 * D_low**2)))
```

```
# --- 方向性濾波:只保留 ±15 度扇形(萃取垂直條紋) ---
ang = np.abs(np.degrees(theta))
H_dir = ((ang < 15) | (ang > 165)).astype(np.float32)
H_dir = cv2.GaussianBlur(H_dir, (0, 0), 3)   # 平滑邊界避免振鈴

Fshift = np.fft.fftshift(np.fft.fft2(gray))
cv2.imwrite("bandpass.png", apply_filter(Fshift, H_bp))
cv2.imwrite("directional.png", apply_filter(Fshift, H_bp * H_dir))
```

6.3.5 推導：理想低通為什麼一定會振鈴

圖 6-10 量到理想低通在階躍邊緣上造成 23.0 的過衝與 23.0 的下衝，而高斯是 0.0。這不是參數沒調好，而是數學上躲不掉的結果。

關鍵在於卷積定理（6.5 節）：頻率域的乘法等於空間域的卷積，所以「頻率域乘上 H」等於「空間域與 h 卷積」，其中 h 是 H 的反傅立葉轉換。理想低通的 H 是一個半徑 D0 的圓盤，而圓盤的反轉換是所謂的 jinc 函數（一維情形則是 sinc）——它以 1/r 的速度緩慢衰減，並且有無限多個正負交替的旁瓣。把這樣一個帶負旁瓣的核與階躍邊緣卷積，邊緣一側被負旁瓣壓到低於原值、另一側被正旁瓣推到高於原值，就形成對稱的過衝與下衝，並沿著邊緣向外震盪，在二維上看就是一圈圈同心漣漪。

高斯為什麼免疫?因為高斯函數的傅立葉轉換仍然是高斯——這是高斯獨有的性質。既然 H 是高斯，h 也是高斯，而高斯恆為正、沒有任何旁瓣，卷積後自然不可能產生過衝。Butterworth 則介於兩者之間：階數 n 愈大邊緣愈陡、愈接近理想濾波器，振鈴也愈明顯（本章 Lab 6-5 會把這條曲線實際畫出來）。

退一步看，這其實是「不確定性原理」在影像上的體現：一個函數不可能同時在空間域與頻率域都是緊緻的。理想低通在頻率域緊緻到極致（硬邊圓盤），代價就是在空間域無限延伸;高斯是在兩個域都相對集中的最佳折衷，這也是它在本書中反覆出現的根本理由。

6.4 帶阻與陷波濾波：狙擊週期性雜訊

這是頻率域最無可取代的應用場景。當影像受到規律性干擾——掃描舊報紙時的

網點、翻拍螢幕的摩爾紋、電源干擾造成的橫紋、感光元件的固定圖案雜訊、機械振動造成的條紋——這些干擾在空間域遍布全圖，用第 5 章的平滑濾波去除必然連細節一起犧牲。但在頻率域中，週期性干擾只集中在少數幾個孤立的亮點上(因為它本身就是幾個特定頻率的弦波)，把那幾點壓下去，反轉換回來，干擾消失而細節完好無損。

工具有二：帶阻濾波器(Bandreject)阻擋以中心為圓心、特定半徑範圍的環狀頻帶，適合干擾頻率已知且呈環狀分布者;陷波濾波器(Notch)則只在頻譜上指定的幾個小圓區域歸零，精準狙擊特定的干擾點。

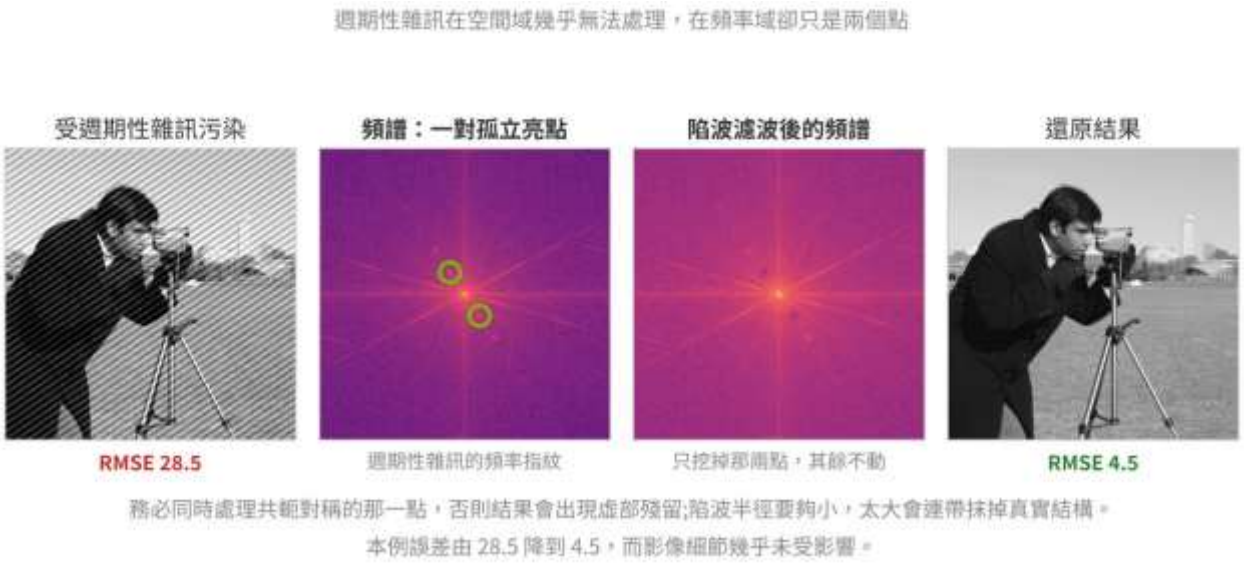


圖 6-14　陷波濾波移除週期性干擾：RMSE 由 28.5 降到 4.5

三個實作要點必須牢記：第一，務必同時消除共軛對稱點(承 6.1.3 節)，否則反轉換會出現虛部殘留與明顯偽影;第二，陷波的半徑不宜過大，否則會連帶損失鄰近的正常頻率成分，在影像上表現為局部模糊;第三，陷波區域的邊界應做平滑處理(如乘上高斯剖面)，避免硬截斷造成振鈴——這與 6.3.2 節理想濾波器的教訓完全一致。

程式 6-5 陷波濾波去除摩爾紋(含自動偵測)

```
import cv2, numpy as np

gray = cv2.imread("moire.jpg", 0).astype(np.float32)
M, N = gray.shape
Fshift = np.fft.fftshift(np.fft.fft2(gray))

# 步驟一:先看頻譜,用肉眼(或程式)找出異常亮點座標
cy, cx = M // 2, N // 2
```

```
notches = [(cy - 40, cx + 55)]      # 依實際觀察填入

# 步驟二:在每個亮點及其對稱點畫上平滑的抑制區
H = np.ones((M, N), np.float32)
r = 8
for (py, px) in notches:
    cv2.circle(H, (px, py), r, 0, -1)
    cv2.circle(H, (2 * cx - px, 2 * cy - py), r, 0, -1)  # 對稱點!
H = cv2.GaussianBlur(H, (0, 0), 2)   # 平滑邊界,抑制振鈴

out = np.abs(np.fft.ifft2(np.fft.ifftshift(Fshift * H)))
cv2.imwrite("moire_removed.png", out.astype(np.uint8))

# 進階:自動偵測干擾亮點
mag = 20 * np.log(np.abs(Fshift) + 1)
local_max = cv2.dilate(mag, np.ones((9, 9), np.uint8))
peaks = (mag == local_max) & (mag > mag.mean() + 3 * mag.std())
peaks[cy-30:cy+30, cx-30:cx+30] = False      # 排除中心低頻
print("偵測到的干擾點:", np.argwhere(peaks)[:10])
```

6.5 卷積定理：兩個域的橋梁

6.5.1 定理與推論

卷積定理是連結第 5 章與本章的核心命題：空間域的卷積，等於頻率域的逐點相乘;反過來，空間域的相乘等於頻率域的卷積。以符號表示：f * h 的傅立葉轉換等於 F · H。

這條定理解釋了本章許多現象，值得逐條對照：為什麼理想低通會振鈴(頻域的圓形方波，其空間域對應是振盪的類 sinc 函數);為什麼高斯模糊如此溫和(高斯的轉換仍是高斯，無旁瓣);為什麼第 7 章的模糊還原可以寫成頻率域的除法(卷積退化為乘法，反運算自然是除法);為什麼第 5 章可分離濾波能省下大量運算(因為二維高斯可拆成兩個一維高斯的外積，對應頻域也可拆解)。

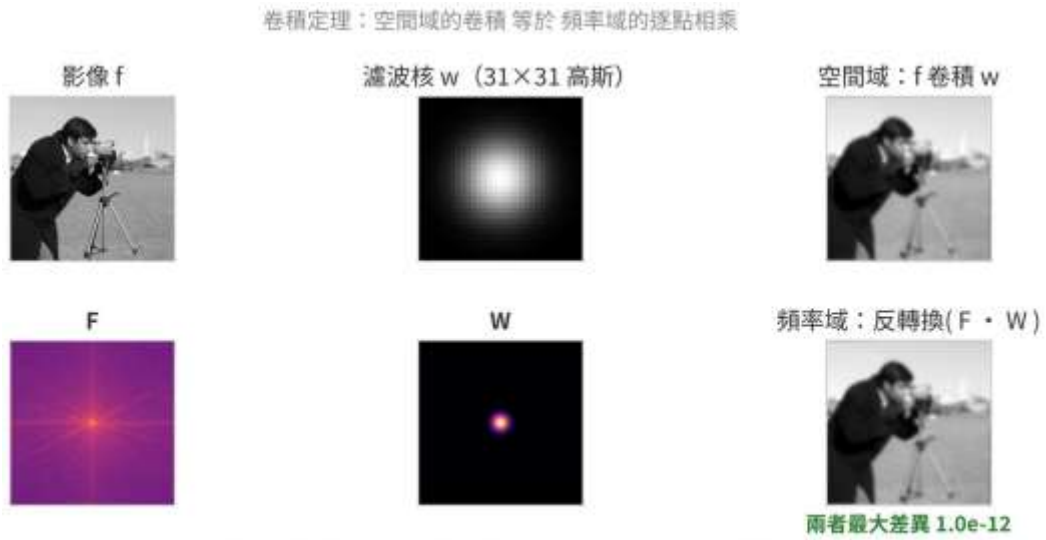


圖 6-15　卷積定理的實測驗證：兩條路徑差異僅 1.0 × 10⁻¹²

6.5.2 該用哪個域?一個工程判準

卷積定理帶來一個實務問題：同一件事(卷積)有兩條路可走，該走哪一條?答案取決於核的大小。空間域卷積的成本正比於核的面積(m^2);而頻率域路線的成本主要是兩次 FFT，約為 O(MN·log(MN))，與核大小完全無關。

因此判準是：小核在空間域做比較快，大核在頻率域做比較划算。以一張 1024×1024 的影像為例，3×3 卷積在空間域約需九百萬次乘法，而走頻率域要付出約兩千萬次運算的固定成本——顯然不划算。但若核放大到 51×51，空間域需要約二十七億次乘法，頻率域的成本卻分毫未增，差距超過百倍。

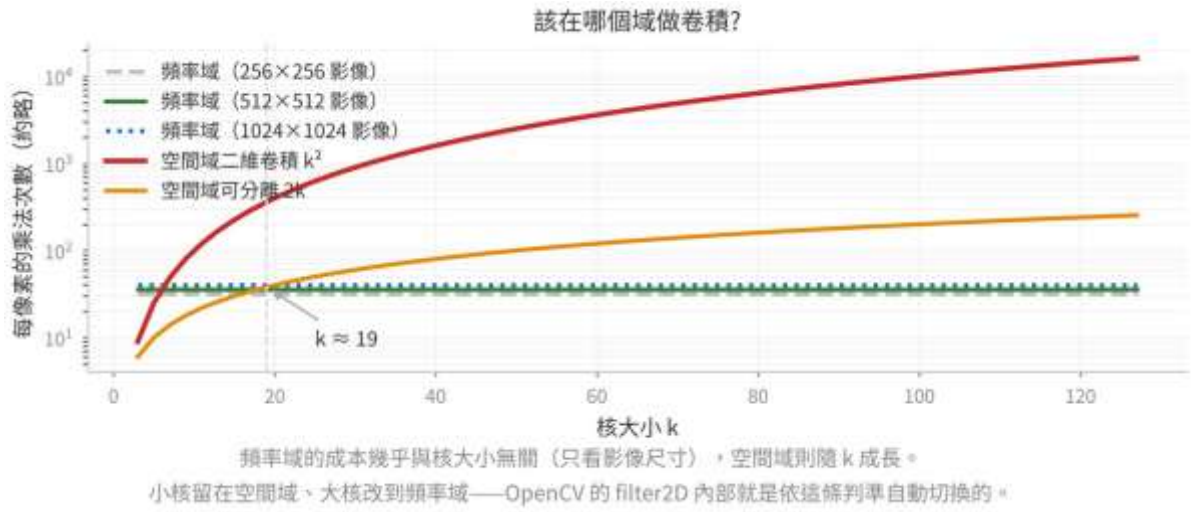


圖 6-16　空間域與頻率域的成本交叉點

這就是為什麼 OpenCV 的 GaussianBlur、filter2D 對小核直接在空間域運算，而像第 7 章維納濾波這類「退化函數與影像同尺寸」的操作，只能在頻率域進行。實務上還有第三個考量：GPU 對空間域卷積的平行化極為友善(第 15 章的 CNN 正是靠這一點)，因此在 Jetson 等有 GPU 的平台上，空間域的優勢區間會比純 CPU 更寬——這是第 17 章部署時需要實測的工程細節。

6.5.3 推導：卷積定理的離散證明

卷積定理是本章——其實是本書前半部——的樞紐，它讓第 5 章的每一個空間濾波器都能翻譯成一條頻率曲線。它的離散版證明不長，而且只用到一個技巧：交換求和順序。

設 g 為 f 與 w 的循環卷積，即 $g(x) = \Sigma_m f(m) \cdot w(x - m)$。對 g 取 DFT：$G(u) = \Sigma_x g(x) \cdot \exp(-j2\pi ux/N) = \Sigma_x \Sigma_m f(m) \cdot w(x - m) \cdot \exp(-j2\pi ux/N)$。交換兩個求和的順序，並令 $n = x - m$（即 $x = m + n$）：$G(u) = \Sigma_m f(m) \cdot \exp(-j2\pi um/N) \cdot \Sigma_n w(n) \cdot \exp(-j2\pi un/N)$。右邊兩個求和恰好各自是 f 與 w 的 DFT，因此 $G(u) = F(u) \cdot W(u)$。二維的情形完全照搬，只是多一個維度。

本書以一張 512×512 的影像與一個 31×31 的高斯核實測兩條路徑：直接做空間卷積，以及取 DFT、逐點相乘、再反轉換。兩者的最大差異為 1.0×10^{-12}，純粹是浮點累積誤差（圖 6-15）。

證明中有一個常被忽略的細節：這裡的卷積是「循環卷積」，也就是假設訊號週期性延拓（6.2.5 節）。若要用 FFT 計算一般的線性卷積，必須先對影像與核各自補零到至少 $(M + K - 1)$ 的長度，否則一端捲出去的部分會從另一端捲回來，造成所謂的循環混疊——這在實作上是最常見的錯誤之一。另外值得一提的是：中值濾波不是卷積，因此它沒有對應的「頻率面孔」，卷積定理對它完全無效（這正是表 6-3 中它缺席的原因）。

6.6 頻率域觀點與全書的觀念連結

本節把散落在各章的線索一次串起來。建議讀者在讀完全書後回頭重讀一次——屆時每一條連結都會有更深的體會。

6.6.1 與第 3 章：取樣定理與混疊的頻域解釋

第 3 章介紹 Nyquist 取樣定理時，說「取樣頻率必須至少是訊號最高頻率的兩倍」。這句話的完整意義只有在頻率域才說得清楚：取樣的動作在頻域上會使原訊號的頻譜以取樣頻率為間隔無限複製。若取樣頻率夠高，這些複製品彼此分離，可以用低通濾波器完整取回原訊號;若取樣頻率不足，相鄰的複製品就會重疊，高頻成分「摺疊」進低頻區，偽裝成原本不存在的低頻訊號——這就是混疊(Aliasing)。

摩爾紋正是它最常見的視覺表現：細密的條紋(高頻)在取樣不足時，摺疊成了粗大的波紋(低頻)。這也解釋了為什麼「縮小影像前應先做低通模糊」——先把超過新取樣率一半的高頻成分濾掉，就不會有東西可以摺疊回來。OpenCV 的 INTER_AREA 內插(第 4 章)之所以適合縮小，正是因為它內建了這個平均(低通)的動作。

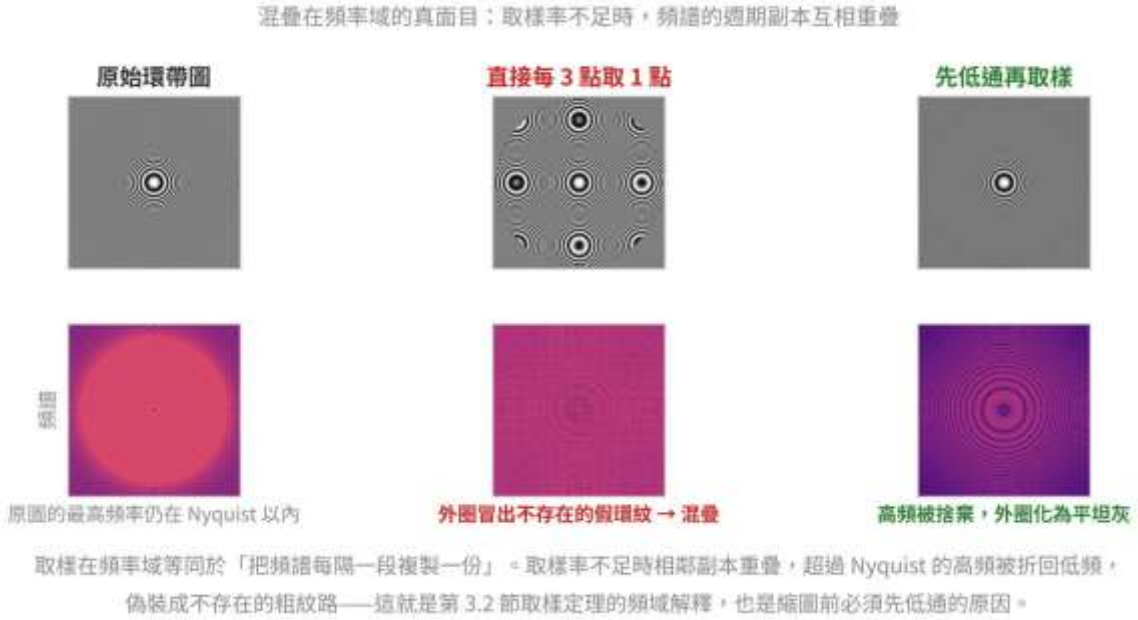


圖 6-17　環帶圖示範混疊：假環紋只出現在外圈

6.6.2 與第 5 章：每個空間濾波器的頻率面孔

第 5 章的所有濾波器，在本章都能看到它們的另一張臉。這個對照表值得學生自己動手驗證一遍(見 Lab 6-4):

空間域(第 5 章)	頻率域面孔	觀察重點
均值濾波(方形核)	二維 sinc 型低通	有振盪旁瓣，故平滑效果不佳且有方向性偽影
高斯濾波	高斯型低通	無旁瓣，sigma 愈大則頻域愈窄、影像愈模糊
拉普拉斯運算子	高通	響應隨頻率平方增長，故對雜訊極敏感
Sobel 梯度(第 9 章)	方向性高通	水平 Sobel 對垂直邊緣響應強，即方向選擇性
Unsharp / High-Boost	高頻強調	等同 6.3.3 節的 a + b·Hhp,k 對應 b
中值濾波	無對應(非線性)	不滿足線性與位置不變性，無法以轉移函數描述

表 6-3 空間域濾波器與其頻率域面孔

最後一列特別重要：卷積定理只適用於線性位置不變系統。中值濾波、雙邊濾波、非局部平均都是非線性的，沒有轉移函數可言——這也解釋了它們為何能做到線性濾波做不到的事(例如中值濾波能去除椒鹽雜訊而不模糊邊緣)。線性系統的分析工具強大，但也劃下了能力的上限。

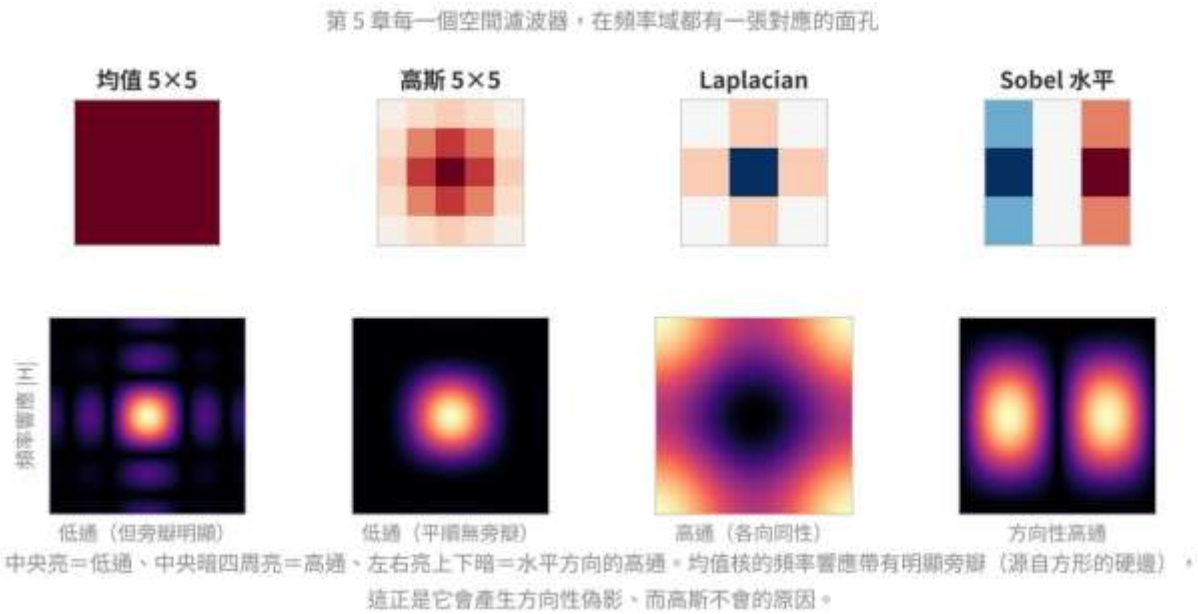


圖 6-18　四個空間濾波核的「頻率面孔」

6.6.3 與第 7 章：模糊是低通，還原是除法

第 7 章的劣化模型 g = h * f + n，在本章的語言中就是 G = H·F + N。所有形式的模糊——失焦、運動、大氣擾動、鏡頭像差——在頻域上做的都是同一件事：把高頻振幅壓低。因此退化函數 H 本質上就是一個低通濾波器，而影像還原就是「受控的高通增強」。

這個觀點也預告了還原的物理上限：運動模糊的 H 是 sinc 函數，在某些頻率上恰好等於零，那些頻率的資訊被乘以零而徹底消失，再高明的演算法也救不回來。理解頻率域，才能分辨「演算法不夠好」與「資訊已經死亡」的差別。

6.6.4 與第 9 章：分割演算法裡的雙重濾波

第 9 章的 Canny 邊緣偵測是一個絕佳的教學案例，因為它在一個演算法內同時用到兩種相反的濾波：第一步的高斯平滑是低通(壓制雜訊，避免微分放大雜訊);第二步的梯度運算是高通(找出灰階劇烈變化處)。順序不能顛倒——先微分再平滑，雜訊已經被放大，再平滑也救不回來。

這也解釋了 Canny 的 sigma 參數為何如此關鍵：sigma 太小，低通不足，雜訊產

生大量假邊緣;sigma 太大，低通過度，弱邊緣連同雜訊一起被抹平。在頻率域的語言中，sigma 就是在決定「訊號與雜訊的分界線畫在哪個頻率」。

6.6.5 與第 11 章：傅立葉的極限與小波的誕生

傅立葉轉換有一個根本的弱點：它是全域的。頻譜告訴你「這張影像含有哪些頻率」，卻不告訴你「這些頻率出現在影像的哪個位置」。一張左半邊平坦、右半邊佈滿細紋的影像，與一張細紋均勻分布的影像，可能有相似的振幅譜。

這個弱點正是第 11 章小波轉換的存在理由：小波以「有限長度的小波形」為基底，同時提供頻率與位置的資訊，這就是多解析度分析。而 JPEG 使用的 DCT 則採取另一種折衷：把影像切成 8×8 小區塊，各自做轉換——用「分區塊」的方式勉強取得局部性，代價就是高壓縮率下惱人的方塊效應。從 DFT 到 DCT 到小波的演進，是一部「如何同時掌握頻率與位置」的奮鬥史。

6.6.6 與第 13 章：頻率域就是特徵

特徵擷取與頻率域的關係比多數人想像的更深。三個具體連結：

- Gabor 濾波器(承 6.3.4 節)是帶通加方向選擇加空間局部化的組合，直接用於紋理分類與早期人臉辨識，其響應強度就是一種特徵。
- SIFT 的尺度空間是由一系列不同 sigma 的高斯模糊構成，本質上是一組截止頻率遞減的低通濾波器;而相鄰兩層相減得到的高斯差(DoG)，正是一個帶通濾波器。SIFT 尋找極值的動作，就是在「頻率與位置」的三維空間中找特徵點。
- 相位相關法(6.7.2 節)利用傅立葉的平移性質做影像對齊，是一種完全在頻域運作的匹配方法，對亮度變化與雜訊比空間域的模板匹配更強健。

6.6.7 與第 15 章：CNN 學到了什麼濾波器

卷積神經網路的名字就洩露了它與本章的血緣。訓練完成後把第一層的卷積核視覺化，會發現它們自發演化成兩類：一類是不同顏色的低頻色塊(相當於低通)，另一

類是不同方向與粗細的條紋偵測器(相當於方向性帶通，與 Gabor 濾波器高度相似)。人工設計了數十年的濾波器，被梯度下降重新發明了一次。

近年也有研究從頻譜角度分析神經網路的行為，例如觀察到網路傾向先學會低頻成分再逐漸擬合高頻細節的現象;也有研究指出對抗樣本的擾動常集中在高頻。這些發現提醒我們：即使進入深度學習時代，頻率域仍是理解影像模型的重要透鏡。

6.7 應用案例

6.7.1 案例一：摩爾紋與週期性雜訊移除

這是本章最經典的案例。情境：用手機翻拍電腦螢幕或電視，拍出的照片佈滿波浪狀的彩色紋路。成因是螢幕的像素網格(高頻週期結構)與相機感光元件的像素網格(取樣格點)相互干涉，產生了低頻的拍頻——正是 6.6.1 節所說的混疊。

處理流程：計算頻譜 → 目視或程式找出干擾造成的孤立亮點 → 設計陷波濾波器(記得對稱點與平滑邊界)→ 反轉換。成效驗收建議用第 3 章的 PSNR 與 SSIM，與「單純高斯模糊」的做法對照——數據會清楚顯示：對付週期雜訊，頻率域完勝空間域，因為前者只移除了那幾個頻率，後者則是全圖細節陪葬。

延伸應用相同：老報紙掃描的網點(印刷半色調網屏)、電源干擾造成的橫紋、CMOS 感光元件的固定圖案雜訊、掃描器機構震動造成的週期條紋，都適用同一套流程。這也是第 7 章影像還原中，唯一不需要動用反卷積就能完美解決的劣化類型。

6.7.2 案例二：相位相關法做影像對齊與運動估計

這個案例展示了頻率域的另一種威力：不做濾波，而是做「比對」。理論基礎是 6.2.3 節提到、並在 6.2.6 節完成推導的平移性質——影像平移時，振幅譜完全不變，所有位移資訊都編碼在相位中。

做法是：把兩張影像各自做 FFT，計算它們的交叉功率譜(一張的頻譜乘上另一張

的共軛，再除以其大小做正規化，只保留相位差)，再做反 FFT。若兩張影像之間只有平移關係，結果會是一個尖銳的脈衝，脈衝的位置就直接給出位移量，精度可達次像素等級。

這個方法的優勢在於：對亮度整體變化免疫(因為只用相位)、對雜訊相當強健、而且是全域搜尋一次算完，不像空間域的模板匹配需要逐位置滑動比對。應用場合極廣：第 5 章夜景模式的多影格對齊、影像穩定(防手震)、第 12 章的運動估計、衛星影像的時序配準、以及第 13 章影像拼接的初始位移估計。OpenCV 直接提供 cv2.phaseCorrelate 函式。



圖 6-19　相位相關法：一次全域運算求出位移

實作提醒：務必先加窗(6.2.5 節)。因為 DFT 假設影像週期性延拓，未加窗時邊界跳變會在相關圖上產生強烈的十字狀假響應，可能蓋過真正的峰值。

程式 6-6 相位相關法對齊影像

```
import cv2, numpy as np

a = cv2.imread("frame1.png", 0).astype(np.float32)
b = cv2.imread("frame2.png", 0).astype(np.float32)

# 加窗:消除 DFT 週期性延拓造成的邊界假響應
win = cv2.createHanningWindow(a.shape[::-1], cv2.CV_32F)

# OpenCV 內建相位相關,回傳次像素精度的位移與峰值信心值
(dx, dy), response = cv2.phaseCorrelate(a, b, win)
```

```
print(f"位移量:dx={dx:.3f}, dy={dy:.3f} 像素,信心值={response:.3f}")

# --- 手動實作,理解原理 ---
Fa = np.fft.fft2(a * win)
Fb = np.fft.fft2(b * win)
R = Fa * np.conj(Fb)
R /= (np.abs(R) + 1e-8)            # 正規化:只保留相位差
corr = np.fft.fftshift(np.abs(np.fft.ifft2(R)))
peak = np.unravel_index(np.argmax(corr), corr.shape)
print("峰值位置(相對中心即為位移):", peak)

# 應用:多影格對齊後平均去雜訊(第 5 章夜景模式)
M = np.float32([[1, 0, dx], [0, 1, dy]])          # 第 4 章仿射矩陣
b_aligned = cv2.warpAffine(b, M, b.shape[::-1])
merged = ((a + b_aligned) / 2).astype(np.uint8)
```

6.7.3 案例三：AOI 紋理瑕疵檢測

第三個案例最貼近本學程的產業場域，也是頻率域在工業界最重要的應用之一。情境：檢測織品、金屬網、晶圓線路、印刷電路板等「具有規律紋理」的產品表面瑕疵。

傳統的空間域做法很棘手：背景本身就佈滿高對比的規律紋理，任何門檻分割或邊緣偵測都會被背景淹沒。但換到頻率域，問題突然變得優雅：規律的背景紋理是週期性的，因此在頻譜上表現為少數幾個明確的亮點(位置由紋理的疏密與方向決定);而瑕疵——刮痕、缺線、汙點、破洞——是打破規律的異常，能量散布在其他頻率上。

於是策略反轉了：不是「找出瑕疵」，而是「濾掉正常」。設計一個陷波濾波器，把代表正常紋理的那幾個亮點移除，反轉換後，規律的背景消失殆盡，剩下的殘差影像中，瑕疵便成為畫面上唯一顯著的區域，再用第 9 章的門檻分割與第 10 章的連通元件標記即可定位與計數。

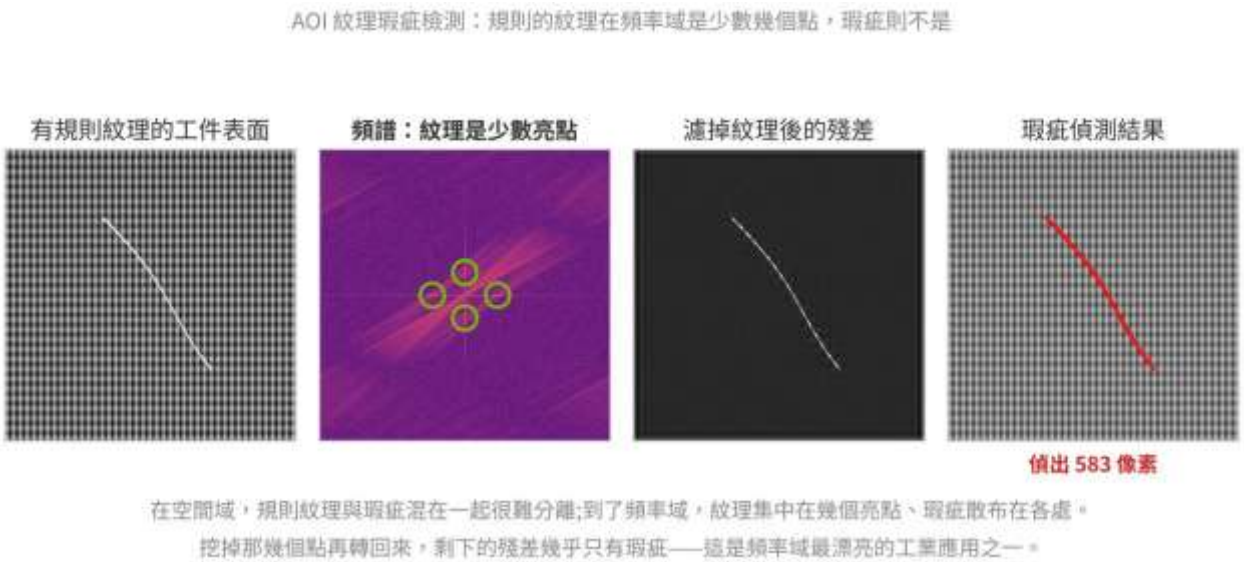


圖 6-20　頻率域 AOI：濾掉正常紋理，殘差中只剩瑕疵

這個方法的工程優點是：不需要大量瑕疵樣本訓練(相對於第 16 章的深度學習方案)，而且對「沒見過的新瑕疵類型」天生有效——因為它偵測的是「異常」而非「特定瑕疵」。缺點是隻適用於背景高度規律的產品，且對取像品質要求高(打光不均會產生額外的低頻成分干擾判讀，此時可先用 6.3.3 節的同態濾波處理)。實務上，它常與深度學習方案互補使用。

程式 6-7 頻率域紋理瑕疵檢測

```
import cv2, numpy as np

fabric = cv2.imread("fabric_defect.png", 0).astype(np.float32)
M, N = fabric.shape
cy, cx = M // 2, N // 2

F = np.fft.fftshift(np.fft.fft2(fabric))
mag = 20 * np.log(np.abs(F) + 1)

# 步驟一:自動找出代表「正常紋理」的週期性亮點
local_max = cv2.dilate(mag, np.ones((11, 11), np.uint8))
peaks = (mag == local_max) & (mag > mag.mean() + 2.5 * mag.std())
peaks[cy-15:cy+15, cx-15:cx+15] = False        # 保留直流與低頻

# 步驟二:陷波濾掉這些「正常」頻率
H = np.ones((M, N), np.float32)
for (py, px) in np.argwhere(peaks):
    cv2.circle(H, (int(px), int(py)), 10, 0, -1)
H = cv2.GaussianBlur(H, (0, 0), 3)

# 步驟三:反轉換,殘差中只剩異常
residual = np.abs(np.fft.ifft2(np.fft.ifftshift(F * H)))
```

```
residual = cv2.normalize(residual, None, 0, 255,
                         cv2.NORM_MINMAX).astype(np.uint8)

# 步驟四:門檻分割定位瑕疵(第 9、10 章)
_, defect = cv2.threshold(residual, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
defect = cv2.morphologyEx(defect, cv2.MORPH_OPEN,
                          np.ones((3, 3), np.uint8))
n, labels, stats, _ = cv2.connectedComponentsWithStats(defect)
print(f"偵測到 {n - 1} 處疑似瑕疵")
```

6.7.4 案例四：JPEG 壓縮中的頻率思維

最後一個案例，說明頻率域如何撐起你手機裡的每一張照片。JPEG 的核心邏輯是：人眼對低頻(整體形狀與亮度)敏感，對高頻(細微紋理)遲鈍，因此可以大膽丟棄高頻資訊。

流程是：先轉到 YCbCr 色彩空間(第 8 章)並對色度降取樣 → 切成 8×8 區塊 → 對每個區塊做離散餘弦轉換(DCT，傅立葉家族的近親，只用餘弦項因此結果為實數)→ 用量化表把高頻係數除以較大的數再取整數，許多高頻係數因此變成零 → 以熵編碼壓縮這些零。品質參數控制的正是量化表的縮放倍率。

這也解釋了 JPEG 的典型瑕疵：高壓縮率時，8×8 區塊各自量化的差異在邊界處顯現，形成方塊效應;而銳利邊緣附近的高頻被截斷，產生類似振鈴的蚊狀雜訊——沒錯，又是 6.3.2 節那個「頻域硬截斷造成空間域振盪」的老問題。第 11 章會完整拆解 DCT 與量化表，並比較小波為何能避免方塊效應。

6.8 本章與全書的觀念連結總表

相關章節	共用的觀念	頻率域提供的洞見
第 3 章 數位影像基礎	取樣定理、混疊、摩爾紋	取樣使頻譜週期性複製，不足則重疊摺疊成混疊
第 4 章 幾何轉換	旋轉、平移;縮小前的低通	影像旋轉則頻譜同步旋轉;平移只改相位

第 5 章 空間域強化	平滑、銳化、可分離濾波	每個空間核都有頻率面孔;High-Boost 即高頻強調
第 7 章 影像還原	劣化模型、反卷積	模糊即低通、還原即除法;H 的零點代表資訊死亡
第 9 章 影像分割	Canny 的平滑與梯度	低通與高通在同一演算法中依序作用，順序不可顛倒
第 11 章 小波轉換	DCT、多解析度分析	傅立葉無位置資訊，催生 DCT 分區塊與小波
第 13 章 特徵擷取	Gabor、SIFT 尺度空間、匹配	DoG 即帶通;相位相關法以平移性質做對齊
第 15 章 深度學習	卷積核、頻譜偏差	CNN 首層自發學成方向性帶通，近似 Gabor
第 17 章 邊緣部署	運算成本、GPU 加速	小核用空間域、大核用頻率域;GPU 會改變分界點

表 6-4 第 6 章與全書其他章節的觀念連結

程式實作 Lab 6

Lab 6-1 頻譜偵探

目標：培養「看頻譜說故事」的能力。(a)分別對下列影像計算並顯示頻譜：純色塊、單一方向的條紋、旋轉 45 度的條紋、棋盤格、自然風景照、文字掃描頁，對照表 6-1 說明每張頻譜的特徵與影像內容的關係;(b)自己合成一張影像：用 np.sin 產生單一頻率的條紋圖，計算其頻譜，驗證是否只有兩個對稱亮點;(c)改變條紋的疏密與角度，觀察亮點如何在頻譜上移動，驗證「疏密對應距中心距離、方向對應方位角」;(d)把一張照片旋轉 30 度後重算頻譜，驗證旋轉性質;把同一張照片平移 50 像素，比較振幅譜是否不變——這就是 6.7.2 節相位相關法的理論基礎;(e)挑戰題：對同一張照片分別「加窗」與「不加窗」計算頻譜，觀察十字亮線的有無，理解 6.2.5 節的邊界效應。

Lab 6-2 摩爾紋清除工作坊

目標：完成一次真實的週期雜訊移除。(a)用手機翻拍電腦螢幕或電視，取得帶有明顯摩爾紋的照片;(b)計算頻譜，找出干擾造成的孤立亮點並記錄座標;(c)以程式 6-5 設計陷波濾波器移除干擾，記得處理對稱點並平滑邊界;(d)以第 3 章的 PSNR 與 SSIM 比較「陷波濾波」與「單純高斯模糊」兩種做法相對於直接截圖(乾淨參考影像)的品質——用數據證明：對付週期雜訊，頻率域完勝空間域;(e)分別測試陷波半徑 r = 3、8、20 三種設定，說明過大的半徑為何造成局部模糊;(f)挑戰題：完成程式 6-5 中的自動偵測函式，讓程式自己找出所有干擾亮點，不需人工輸入座標。

Lab 6-3 相位相關對齊與多影格去雜訊

目標：實作 6.7.2 節的案例，並與第 5 章的夜景模式串接。(a)拍攝或合成一組僅有平移差異的影像對，以程式 6-6 求出位移量，與真值比較誤差(應可達次像素等級);(b)刻意調整其中一張的整體亮度(乘上 0.7 或加上 30)，重新計算——驗證相位相關法對亮度變化的免疫力，並與 cv2.matchTemplate 的結果對照;(c)加入不同強度的高斯雜訊，繪製「雜訊強度對定位誤差」曲線，比較兩種方法的強健度;(d)實戰：手持連拍同一暗景 10 張(必然有輕微晃動)，以相位相關逐張對齊後平均，與「未對齊直接平均」比較清晰度與 PSNR——這正是 Google Night Sight 所謂 robust alignment 的簡化版;(e)討論：若影像之間除了平移還有旋轉與縮放，此法還適用嗎?可以如何擴充?

Lab 6-4 濾波器的兩張臉

目標：親手驗證表 6-3，把第 5 章與第 6 章徹底打通。(a)取 3×3 均值核、5×5 均值核、sigma=1 與 sigma=3 的高斯核、拉普拉斯核、水平 Sobel 核，分別補零到 256×256 後做 FFT，顯示各自的頻率響應;(b)對照觀察：哪些是低通?哪些是高通?哪些有方向性?均值核的旁瓣在哪裡?(c)驗證「sigma 愈大、頻域愈窄」的反比關係，並解釋這如何對應到「影像愈模糊」;(d)實測卷積定理：用同一個 15×15 高斯核，分別

以 cv2.filter2D(空間域)與 FFT 相乘(頻率域)處理同一張影像，比較兩者結果的最大差異值(應接近零)與執行時間;(e)把核尺寸從 3×3 逐步加大到 101×101，繪製「核大小對兩種方法執行時間」的曲線，找出交叉點——用你自己的數據驗證 6.5.2 節的工程判準。

Lab 6-5 振鈴效應的量化實驗

目標：把圖 6-10 的現象變成一組可以畫出曲線的數據。(a)產生一張只有一條垂直階躍邊緣的合成影像;(b)實作理想、Butterworth（n = 1, 2, 4, 8）與高斯三類低通濾波器，固定 D0 = 30 各做一次，取出通過邊緣的那一列剖面;(c)對每個剖面量測過衝與下衝的絕對值，整理成表;(d)把 Butterworth 的階數 n 當橫軸、振鈴量當縱軸畫圖，觀察 n 增大時曲線如何逼近理想濾波器的值，並說明這對應了什麼直覺;(e)把同一組濾波器用在一張真實照片上，統計有多少像素超出 [0, 255]，驗證課文中 1,626 比 0 的對照;(f)寫下你的結論：在什麼情況下你會願意忍受振鈴?

Lab 6-6 頻率域的工業檢測

目標：重現 6.7.3 節與圖 6-20 的方法，但用你自己拍的影像。(a)找一個具有規則紋理的物件（紗窗、織品、金屬網、瓦楞紙、印刷網點皆可），在固定打光下拍攝一張「良品」;(b)人為製造一處瑕疵（抽掉一根線、壓出摺痕、畫一道細線），在相同條件下再拍一張;(c)對兩張影像分別計算頻譜，比較差異——你應該會發現規則紋理在兩張上幾乎一樣，這正是「濾掉正常」策略能成立的前提;(d)寫一段程式自動找出紋理對應的峰值（記得先加窗、先排除低頻），設計陷波濾波器並反轉換;(e)對良品與不良品套用同一組參數，記錄不良品的偵出面積與良品的誤報面積——良品誤報必須接近零，否則參數不可用;(f)討論：若把這套方法換成深度學習，需要多少標註資料?在你的場景中，哪一種比較務實?

本章重點整理

- 影像的空間頻率描述灰階變化的快慢：低頻對應平坦區域與整體亮度，高頻對應邊緣、細節與雜訊;二維傅立葉的每個基底都是一張斜條紋圖，疏密對應距中心距離、方向對應方位角。
- 二維 DFT 以 FFT 計算，複雜度由 $O(M^2N^2)$ 降為 $O(MN \cdot \log MN)$;顯示頻譜必須先 fftshift 中心化並取對數壓縮動態範圍;實數影像的頻譜具共軛對稱性。
- 相位譜攜帶影像的結構資訊，振幅譜反映能量分布;影像平移時振幅譜不變、資訊全在相位，此性質即相位相關對齊法的基礎;影像旋轉則頻譜同步旋轉。
- DFT 隱含週期性延拓假設，邊界跳變會造成十字狀假亮線，精確分析前應加窗處理。
- 理想濾波器截止俐落但有嚴重振鈴(頻域硬截斷對應空間域 sinc 振盪);Butterworth 以階數 n 調控;高斯無振鈴。高通轉移函數為 1 減低通，實務多用高頻強調 $a + b \cdot Hhp$，等價於第 5 章的 High-Boost。
- 帶通濾波挑出特定尺度的結構，方向性濾波挑出特定走向的紋理;兩者結合並加上空間局部化即 Gabor 濾波器，是第 13 章紋理特徵的基礎。
- 週期性雜訊在頻譜上呈孤立亮點，以帶阻或陷波濾波精準移除;務必同時處理共軛對稱點、控制陷波半徑、平滑陷波邊界。
- 卷積定理：空間域卷積等於頻率域相乘。小核在空間域計算較快，大核則轉頻率域較划算;非線性濾波(中值、雙邊、非局部平均)無轉移函數可言。
- 頻率域是全書的樞紐觀念：解釋取樣定理與混疊(第 3 章)、賦予空間濾波器另一張面孔(第 5 章)、界定影像還原的物理上限(第 7 章)、催生 DCT 與小波(第 11 章)、支撐 Gabor 與 SIFT(第 13 章)、並在 CNN 的卷積核中重現(第 15 章)。

習題

1. 請以自己的話說明「影像的高頻成分」代表什麼，並舉出三種會產生高頻的影像內容與兩種只含低頻的影像內容。

   提示與參考解答：高頻代表灰階隨位置劇烈變化的成分。高頻內容：銳利邊緣、細密紋理(織品、髮絲)、文字筆畫、雜訊。低頻內容：漸層的天空、大面積平坦的牆面、失焦的背景。可對照表 6-1。

2. 為什麼顯示頻譜前需要 fftshift 與取對數?各解決什麼問題?取對數這個動作與第 5 章的哪個概念相同?

   提示與參考解答：fftshift 把零頻率從左上角移到中央，使「愈近中心愈低頻」符合直覺。取對數壓縮動態範圍：自然影像能量集中於低頻，直流分量比其他成分大數個數量級，不取對數則只看得到中央一個亮點。這正是第 5 章對數轉換「拉伸暗部、壓縮亮部」的應用。

3. 一張影像若含有水平的規律條紋，其頻譜上的亮點會沿哪個方向分布?為什麼?若條紋變得更細密，亮點會如何移動?

   提示與參考解答：沿垂直方向分布。因為頻譜上的方位對應影像中條紋的垂直方向：水平條紋代表灰階沿垂直方向週期變化，故頻率分量沿垂直軸。條紋愈細密代表頻率愈高，亮點會愈往外(離中心愈遠)移動。

4. 說明傅立葉轉換的旋轉性質與平移性質，並各舉一個實際應用。

   提示與參考解答：旋轉性質：影像旋轉 θ 角，頻譜同步旋轉 θ 角。應用：由頻譜上亮線的方向估計文件掃描的傾斜角以自動校正。平移性質：影像平移時振幅譜完全不變，位移資訊全部編碼在相位。應用：相位相關法做影像對齊與運動估計(6.7.2 節)。

5. 用 A 的振幅搭配 B 的相位重建影像，結果較像哪一張?這說明了什麼?若只保留相位(振幅全設為 1)會看到什麼?

   提示與參考解答：較像 B。說明相位攜帶影像的結構資訊(邊緣位置、輪廓排列)，振幅主要反映各頻率的能量分布(整體質感與對比)。只保留相位仍能看出輪廓與物體形狀;只保留振幅則幾乎什麼都認不出來。

6. 什麼是 DFT 的週期性延拓假設?它會在頻譜上造成什麼假象?如何避免?

提示與參考解答：DFT 假設影像左右、上下邊界相接、無限重複。真實照片邊界亮度不連續，產生人造的劇烈跳變，在頻譜上形成沿水平與垂直方向的十字亮線，易被誤讀為影像有強烈橫豎紋理。避免方式：做 FFT 前乘上 Hanning 等視窗函數(加窗)，或以鏡射填充擴大影像。

7. 說明振鈴效應的成因，並解釋為何高斯濾波器不會產生振鈴。這個現象與「不確定性原理」有何關聯?

   提示與參考解答：理想濾波器在頻域是硬截斷的圓形方波，依卷積定理其空間域對應是類 sinc 函數，具有無限延伸的振盪旁瓣，與影像卷積後在邊緣附近產生同心漣漪。高斯函數的傅立葉轉換仍是高斯，無負值旁瓣，故無振鈴。關聯：函數不可能在空間域與頻率域同時都很窄——頻域截得愈乾脆，空間域的擴散與振盪愈嚴重。

8. 寫出 Butterworth 低通濾波器的轉移函數，說明階數 n 增大時的行為變化，並指出 n 趨近無限大時會退化成什麼。

   提示與參考解答：H(u,v) = 1 / [1 + (D/D0)^(2n)]。n 小時過渡平滑、幾乎無振鈴但截止不俐落;n 增大則過渡帶變陡、截止更俐落，但振鈴開始出現。n 趨近無限大時退化為理想低通濾波器(在 D0 處垂直截斷)。

9. 高通濾波與高頻強調濾波有何差異?後者與第 5 章的 High-Boost 銳化在數學上是什麼關係?

   提示與參考解答：純高通會濾掉直流分量，結果平均亮度為零、整體灰黑。高頻強調 H = a + b·Hhp 保留部分低頻(a)並放大高頻(b)，兼顧亮度層次與邊緣強調。它與第 5 章 g = f + k·(f - blur(f)) 做的是完全相同的事：一個在空間域用減法實現，一個在頻率域用加權實現，k 對應 b。

10. 什麼是帶通濾波?若要偵測影像中寬度約 10 像素的刮痕，為什麼帶通比單純的高通更合適?

    提示與參考解答：帶通只讓某個頻率區間通過，構造方式為兩個不同截止頻率的低通相減。針對特定寬度的刮痕，對應的是特定尺度(頻率)，帶通能同時排除更細的雜訊(更高頻)與更粗的背景起伏(更低頻)，訊噪比遠優於單純高通——高通會把所有雜訊一併留下。

11. Gabor 濾波器由哪三個要素組合而成?它與人類視覺系統及 CNN 有什麼耐人尋味的關聯?

    提示與參考解答：三要素：帶通(選擇尺度)、方向選擇(選擇走向)、空間局部化(以高斯視窗限制作用範圍)。關聯：神經科學研究發現哺乳動物視覺皮層 V1 區簡單細胞的感受野特性與 Gabor 高度相似;而 CNN 第一層卷積核經訓練後也常自發演化出類似的方向性帶通結構——人腦、人工設計、機器學習三條路走到同一個答案。

12. 為什麼設計陷波濾波器時必須同時消除共軛對稱點?若忘記會發生什麼?陷波半徑過大又會有什麼副作用?

    提示與參考解答：實數影像的頻譜滿足共軛對稱 F(-u,-v) = F(u,v)，只消除單邊會破壞對稱性，反轉換後出現虛部殘留與明顯偽影。半徑過大會連帶移除鄰近的正常頻率成分，在影像上表現為該區域的局部模糊或細節損失。此外邊界應平滑處理，否則硬截斷會造成振鈴。*

13. 摩爾紋為什麼適合用頻率域處理而非空間域平滑?請從兩者的作用範圍說明，並解釋摩爾紋與第 3 章混疊的關係。

    提示與參考解答：摩爾紋是週期性干擾，在頻譜上只佔少數幾個孤立點，陷波濾波只移除那幾個頻率、其餘細節完好;空間域平滑則是無差別壓制所有高頻，細節必然陪葬。與混疊的關係：螢幕的像素網格(高頻週期結構)與相機感光元件的取樣格點相互干涉，高頻成分摺疊回低頻區，形成肉眼可見的粗大波紋。

14. 陳述卷積定理。一張 1024×1024 的影像要做 51×51 的模糊，你會選空間域還是頻率域?請估算兩者的運算量並說明理由。

    *提示與參考解答：卷積定理：f * h 的傅立葉轉換等於 F·H，即空間域卷積等於頻率域逐點相乘。空間域成本約 1024×1024×51×51 ≈ 27 億次乘法;頻率域主要成本為兩次 FFT，約 O(MN log MN) ≈ 2 千萬次運算，與核大小無關。應選頻率域，差距超過百倍。若核為 3×3 則反之，空間域較快。*

15. 為什麼中值濾波在表 6-3 中沒有對應的「頻率面孔」?這說明了卷積定理的適用範圍為何?

    提示與參考解答：中值濾波是非線性運算，不滿足線性與位置不變性，無法以轉移函數 H(u,v) 描述。卷積定理只適用於線性位置不變(LSI)系統。這也解釋了非線性濾

波為何能做到線性濾波做不到的事——例如中值濾波去除椒鹽雜訊而不模糊邊緣。

16. 傅立葉轉換的根本弱點是什麼?第 11 章的 DCT 與小波分別如何回應這個弱點?

提示與參考解答：弱點是全域性：頻譜只告訴你「有哪些頻率」，不告訴你「頻率出現在哪個位置」。DCT 的回應是把影像切成 8×8 區塊各自轉換，以分區塊勉強取得局部性，代價是高壓縮率下的方塊效應。小波的回應是以「有限長度的小波形」為基底，同時提供頻率與位置資訊，即多解析度分析。

17. 承 6.7.3 節的 AOI 案例：為什麼檢測規律紋理產品的瑕疵時，策略是「濾掉正常」而非「找出瑕疵」?這個方法相對深度學習方案有何優缺點?

提示與參考解答：因為規律背景紋理是週期性的，在頻譜上僅佔少數明確亮點，容易精準移除;而瑕疵是打破規律的異常，能量散布於其他頻率。濾掉正常後，殘差影像中瑕疵成為唯一顯著區域。優點：不需大量瑕疵樣本訓練，對未見過的新瑕疵類型天生有效。缺點：僅適用於背景高度規律的產品，且對取像品質與打光均勻度要求高(可先用同態濾波改善)。

18. 說明相位相關法的原理與三項優勢，並解釋為何實作時務必先加窗。

提示與參考解答：原理：依平移性質，兩張只有平移差異的影像其相位差編碼了位移量;計算正規化交叉功率譜後反轉換，峰值位置即為位移，可達次像素精度。三項優勢：對整體亮度變化免疫(只用相位)、對雜訊強健、一次算完不需逐位置滑動比對。加窗原因：DFT 的週期性延拓假設使邊界跳變在相關圖上產生十字狀假響應，可能蓋過真正的峰值。

19. (綜合題)請說明頻率域觀點如何解釋以下三件事：(a)縮小影像前為什麼要先模糊;(b)Canny 為什麼要先高斯平滑再算梯度;(c)運動模糊為什麼有時完全救不回來。

提示與參考解答：(a)縮小即降取樣，依取樣定理，超過新取樣率一半的高頻會摺疊成混疊;先低通模糊濾掉這些成分就無物可摺疊，故 INTER_AREA 內建平均動作。(b)梯度是高通運算會放大雜訊，高斯平滑是低通先壓制雜訊;順序顛倒則雜訊已被放大，再平滑也無法還原。(c)運動模糊的退化函數是 sinc 函數，在某些頻率恰好等於零，該頻率的資訊被乘以零而徹底消失，屬於資訊死亡而非演算法不足。

20. 計算題：承圖 6-11。一張 512×512 的影像以高斯低通處理，D0 = 15 時保留了 96.7% 的能量，D0 = 40 時為 98.3%。為什麼能量如此集中，影像在 D0 = 15 時卻已明顯模糊?請從「能量」與「資訊」的差別說明。

    提示與參考解答：自然影像的能量絕大多數集中在低頻，因為影像中面積最大的是緩慢變化的區域（天空、牆面、衣服）。但人眼判斷「清不清楚」靠的是邊緣與紋理，而這些正是高頻——它們的能量佔比極小（不到 4%），感知上的重要性卻極高。所以「保留 96.7% 的能量」與「保留 96.7% 的視覺資訊」完全是兩回事。這個落差正是 JPEG 能大幅壓縮的基礎（第 11 章）：丟掉人眼不敏感的高頻，位元數大減而主觀品質變化不大;反過來也解釋了為什麼評估影像品質不能只看均方誤差（第 3.5 節）。

21. 計算題：承圖 6-10。理想低通在階躍邊緣上造成 23.0 的過衝與 23.0 的下衝，並使 1,626 個像素超出 [0, 255]；高斯則兩者皆為 0。請解釋這組數字的來源，並說明「超出範圍的像素」在工程上為什麼是實際問題。

    提示與參考解答：理想低通在頻率域是一個硬邊的圓盤，硬邊在另一個域必然對應無限延伸的振盪——圓盤的反轉換是帶無限多個正負旁瓣的類 sinc 函數。把它與階躍邊緣卷積，負旁瓣會把邊緣一側壓到 0 以下、正旁瓣把另一側推到 255 以上，於是出現對稱的過衝與下衝。高斯的傅立葉轉換仍是高斯，恆為正、沒有旁瓣，因此完全不振盪。工程上的麻煩在於：8 位元影像無法儲存負值或大於 255 的值，這 1,626 個像素必然被截斷（clip），截斷是不可逆的失真，會在後續的門檻化（第 9 章）或特徵擷取（第 13 章）產生假邊緣。這是「看起來只是美觀問題，實際上會污染下游」的典型例子。

22. 計算題：承圖 6-16。一張 1024×1024 的影像，FFT 的成本約為 $2 \cdot \log_2(1024^2) \approx 40$ 次乘法 / 像素。請問核要多大時，可分離的空間域實作才會輸給頻率域?若核不可分離呢?同樣的問題換成 512×512 的影像，答案會怎麼變?

    提示與參考解答：可分離實作為 $2k$ 次 / 像素，令 $2k > 40$ 得 $k > 20$，即核大於約 21×21 時頻率域才划算。不可分離時為 k^2 次 / 像素，令 $k^2 > 40$ 得 $k > 6.3$，即大約 7×7 以上就該考慮頻率域。換成 512×512 的影像時，FFT 成本降為 $2 \cdot \log_2(512^2) = 36$，交叉點分別落在 $k > 18$ 與 $k > 6$。兩個觀察值得記住：其一，影像愈大，頻率域相對愈不划算（因為 FFT 成本隨影像變大而上升，空間域卻只看核大小）;其

二，核是否可分離會讓交叉點差上三倍，所以「這個核可不可分離」是選擇實作路徑時的第一個問題。

23. 承圖 6-3 與圖 6-5：若一張 512×512 的影像中有週期為 8 像素、方向為 30° 的條紋，它在中心化後的頻譜上會出現在哪裡?請說明距中心的半徑與方向，並推導出通式。

    提示與參考解答：半徑等於影像邊長除以空間週期：512 / 8 = 64 像素。通式為 r = N / T，其中 N 是影像邊長、T 是條紋的空間週期——週期愈短（條紋愈密）則 r 愈大，符合「高頻離中心遠」的直覺。方向則與條紋方向互相垂直：條紋線走向 30°，其空間頻率向量指向 30° + 90° = 120°，因此亮點位於 120° 方向的半徑 64 處，以及共軛對稱的 300° 方向。記住這兩條規則（半徑看週期、方向差 90°），幾乎所有頻譜判讀題都能直接作答。

24. 承圖 6-17：以環帶圖示範混疊時，為什麼假環紋出現在「外圈」而不是中央?本例的環帶圖其瞬時頻率為 r / 560（單位為週期 / 像素）。若把降取樣倍率由 3 改成 2，假環紋的起始半徑會變成多少?(請計算)

    提示與參考解答：環帶圖的頻率由中心向外遞增，中央是低頻、外圈是高頻。混疊只發生在「超過新取樣率的 Nyquist 頻率」的成分上，因此只有外圈會出問題，中央完全不受影響。計算：降取樣 f 倍後的 Nyquist 頻率為 1/(2f)。令 r / 560 = 1/(2f) 得起始半徑 r = 560 / (2f)。f = 3 時 r = 93;f = 2 時 r = 140。也就是降取樣倍率愈小，受影響的範圍愈往外退、混疊愈輕微——這與第 3.2 節「提高取樣率可避免混疊」是同一件事的兩種說法。

25. (綜合設計題)承圖 6-20：某產線的織品有規則的經緯紋理，瑕疵是斷線，呈現為一條細長的暗線。請設計一套頻率域的檢測流程，說明你如何自動找出要挖掉的亮點、陷波半徑如何決定，以及這個方法在什麼情況下會失效。

    提示與參考解答：本題無標準答案，評分重點在流程的完整性與失效分析。一條合理的流程：(1)先加窗消除邊界造成的十字狀亮線（圖 6-7），否則它會干擾峰值偵測;(2)計算頻譜，排除中心低頻區（半徑小於某門檻）後，找出振幅最大的數個峰值及其共軛對稱點——經緯紋理通常各貢獻一對;(3)陷波半徑取「足以涵蓋峰值的主瓣」但不超過相鄰峰值距離的一半，實務上可由峰值振幅衰減到一半的寬度決定;(4)反轉換後取殘差的絕對值，以統計門檻（如平均加五倍標準差）二值化，再用第 10

章的連通元件依長寬比與面積篩選出細長的瑕疵。失效情況至少有四種：紋理不夠規則（手工織品、拉伸變形）時峰值會糊成一片;瑕疵本身恰好具有與紋理相同的週期;影像有明顯的透視變形使紋理週期隨位置改變;以及打光不均造成的低頻起伏被誤判為瑕疵（應先做第 5 章的 CLAHE 處理）。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 4(頻率域處理之標準參考)。
- Fourier, J. B. J., Théorie analytique de la chaleur, Paris, 1822.(1807 年手稿遭拉格朗日反對，本章叫獸開講之史料)
- MacTutor History of Mathematics, Biography: Joseph Fourier, University of St Andrews.
- Cooley, J. W., & Tukey, J. W., An Algorithm for the Machine Calculation of Complex Fourier Series, Math. Comp., 1965.(FFT 原始論文)
- Oppenheim, A. V., & Lim, J. S., The Importance of Phase in Signals, Proc. IEEE, 1981.(相位重要性之經典文獻)
- Kuglin, C. D., & Hines, D. C., The Phase Correlation Image Alignment Method, Proc. IEEE Int. Conf. Cybernetics and Society, 1975.(相位相關法原始論文)
- Daugman, J. G., Uncertainty Relation for Resolution in Space, Spatial Frequency, and Orientation Optimized by Two-Dimensional Visual Cortical Filters, JOSA A, 1985.(Gabor 與視覺皮層)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 3(影像處理與頻域分析)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書
編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 7 章 影像還原

本章學習目標

- 區分影像強化與影像還原的目標差異，寫出影像劣化的數學模型並說明其病態性。
- 說明影像還原與第 5 章空間濾波、第 6 章低通/高通濾波在觀念上的一致性與差異。
- 辨識高斯、椒鹽、均勻、Poisson、週期性等雜訊型態，並以直方圖與平坦區統計量估計雜訊參數。
- 運用排序統計濾波、自適應局部降噪濾波與非局部平均處理不同型態的雜訊。
- 說明逆濾波為何失敗，推導維納濾波與約束最小平方濾波如何加以修正。
- 以維納濾波還原模糊與含雜訊的影像，並以 PSNR 與 SSIM 量化成效。
- 評估影像還原對後續邊緣偵測、特徵擷取與物件偵測的影響。
- 認識反覆式反卷積與 AI 影像修復的原理，並思辨「還原」與「腦補」的界線。

叫獸開講|馬蓋先重生記與哈伯望遠鏡的太空救援

先說一個溫暖的故事。1985 年開播的影集《百戰天龍》(MacGyver)，陪伴了一整個世代的台灣觀眾——那位不用槍、只靠一把瑞士刀和滿腦子物理化學知識就能脫困的馬蓋先，大概是許多工程師最早的偶像。然而，三十多年前的類比訊號母帶畫質模糊、雜訊滿佈，以今天的螢幕觀看幾乎不忍卒睹。近年來，透過 AI 影像修復技術，那段經典的片頭曲被重製成 4K 畫質：雜訊被抹去、邊緣重新銳利、色彩重獲飽和，馬蓋先以前所未有的清晰度重返螢幕。這是「影像還原」最貼近生活的示範。

再說一個驚心動魄的故事。1990 年 4 月，人類史上最昂貴的科學儀器之一——哈伯太空望遠鏡升空。所有人屏息等待它傳回宇宙深處的照片，結果收到的卻是一張張模糊的影像。工程師追查後發現一個令人心碎的錯誤：主鏡的研磨形狀有偏差，產生球面像差，使得點光源超過八成的光線被散射成半徑二至三角秒的光暈。一個微小到以微米計的加工誤差，讓十幾億美元的望遠鏡近乎失明。

修復需要太空人上去動手術，但那是三年後(1993 年 12 月)的事。在那之前的三年，天文學家怎麼辦?他們選擇了另一條路：既然模糊的成因(點擴散函數，PSF)可以被測量與建模，那就用數學把影像「解模糊」回來。維納濾波、Richardson-Lucy 反覆演算法、最大熵法等一系列反卷積技術被大規模投入，竟然真的從模糊資料中救回了大量可用的科學成果。1993 年太空人裝上 COSTAR 校正光學系統與新的 WFPC2 相機後，硬體問題才根本解決——但那三年間，是影像還原技術替人類守住了通往宇宙的窗口。

兩個故事，一今一古，正好對應本章的兩條路線：傳統的數學還原(第 7.5 節的逆濾波、維納濾波與約束最小平方)靠的是「理解劣化的物理過程，再反推回去」;現代的 AI 修復(第 7.8 節)靠的是「從海量影像中學會世界長什麼樣子，然後補上」。前者誠實但受限，後者驚豔但危險——因為它補出來的細節，可能從來不存在。

思考題：如果法庭上的監視器畫面經過 AI 超解析度處理，把模糊的車牌「還原」成清晰的號碼，這個證據可以採信嗎?請從本章兩條路線的差異思考。

7.1 影像還原的定位、模型與病態性

7.1.1 還原與強化的差別

第 5、6 章談的是影像強化(Enhancement)：目標是讓影像「看起來更好」，判準相當主觀——把夜景調亮、把邊緣變銳利，好不好看你說了算。本章的影像還原(Restoration)則是客觀的逆問題：先建立劣化的數學模型，再設法反推出最接近原始

影像的估計。它問的不是「怎樣比較好看」，而是「原本長什麼樣子」，因此可以用第 3 章介紹的 PSNR、SSIM 等指標客觀驗收。

這個差別在工程實務上非常關鍵。舉例來說：同樣是讓模糊的影像變清楚，第 5 章的 Unsharp Masking 只是「把邊緣兩側的對比拉大」，製造出銳利的視覺印象，原本被模糊掉的細節並沒有回來;而本章的反卷積則是「依據模糊的數學模型，把被抹平的高頻成分算回去」。前者是化妝，後者是手術。也因此，還原方法需要更多的先驗知識(退化函數、雜訊統計量)，而且在資訊確實已經完全消失的情況下，它也無能為力——這是所有還原技術的物理上限。



圖 7-1　強化讓影像看起來清楚，還原讓影像真的接近原貌

比較項目	影像強化(第 5、6 章)	影像還原(本章)
目標	看起來更好、更適合後續處理	逼近原始未劣化的影像
判準	主觀(人眼、下游任務表現)	客觀(MSE、PSNR、SSIM)
需要的知識	幾乎不需要先驗知識	退化函數 h、雜訊統計特性
代表方法	直方圖等化、Gamma、平滑與銳化	維納濾波、約束最小平方、反覆式反卷積

數學性質	正問題(直接運算)	逆問題(常為病態問題)

表 7-1 影像強化與影像還原的比較

7.1.2 劣化模型

標準的劣化模型是：g(x, y) = h(x, y) * f(x, y) + n(x, y)，其中 f 是原始影像，h 是退化函數(以卷積作用，例如失焦模糊、運動模糊、光學系統的點擴散函數 PSF),n 是加性雜訊，g 是我們實際拿到的影像。依第 6 章的卷積定理，在頻率域可寫成 G = H·F + N——空間域的卷積變成乘法，還原問題於是變成「已知 G 與 H，如何解出 F」的代數問題。這正是頻率域方法在本章大放異彩的原因，也再次印證第 6 章的核心訊息：換一個域，難題就換一副面孔。

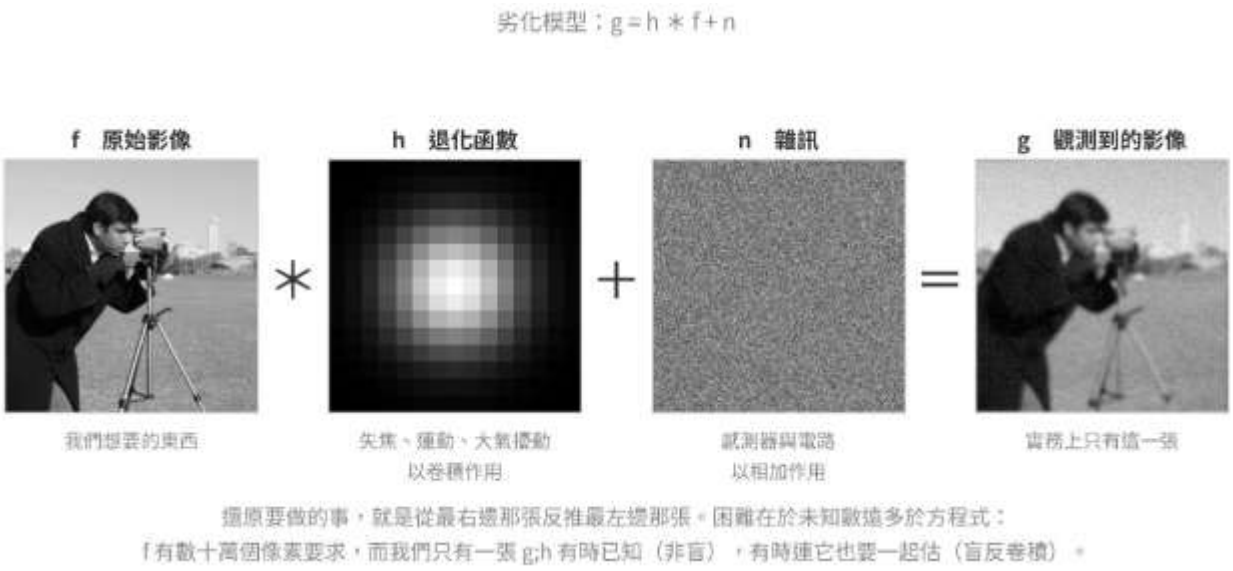


*圖 7-2　劣化模型 g = h * f + n 的四個組成*

請留意這個模型隱含的三個假設，以及它們在真實系統中可能失效的情形：第一，退化是線性且位置不變的(整張影像用同一個 h)。實務上鏡頭邊緣的模糊往往比中心嚴重，哈伯望遠鏡的 PSF 甚至隨時間與位置變動，這時就需要分區塊處理或更複雜的模型。第二，雜訊是加性的且與訊號獨立。這對高斯雜訊成立，但對 Poisson 雜訊(第 7.2 節)並不成立，因為它的強度隨訊號變化。第三，雜訊是零均值的。若感測器有直流偏移(dark current)，必須先做暗電流校正再進入還原流程。

依 h 是否已知，還原問題分為兩類難度：若 h 已知(例如哈伯望遠鏡的 PSF 可由觀測標準星測得、運動模糊的方向與長度可由拍攝參數推算)，稱為非盲反卷積(Non-blind Deconvolution);若 h 未知，必須同時估計 h 與 f，稱為盲反卷積(Blind Deconvolution)。後者困難得多——因為「模糊的影像 = 清楚的影像卷積大模糊核」與「稍微模糊的影像卷積小模糊核」可以得到完全相同的觀測結果，解不唯一，必須靠額外的先驗假設(例如自然影像的梯度分布特性)才能收斂到合理解。

7.1.3 為什麼是病態問題

數學上，一個問題被稱為適定問題(Well-posed)，必須滿足三個條件：解存在、解唯一、解對資料連續依賴(輸入的微小擾動只造成輸出的微小變化)。影像還原問題最常違反的是第三條：退化函數 h 是低通性質的，它把高頻成分壓到接近零;還原時要把這些成分放大回來，等於乘上一個極大的倍數，而觀測資料中不可避免的微小雜訊也會被同等放大。輸入端萬分之一的擾動，可能在輸出端變成鋪天蓋地的雪花——這就是病態(Ill-posed)。

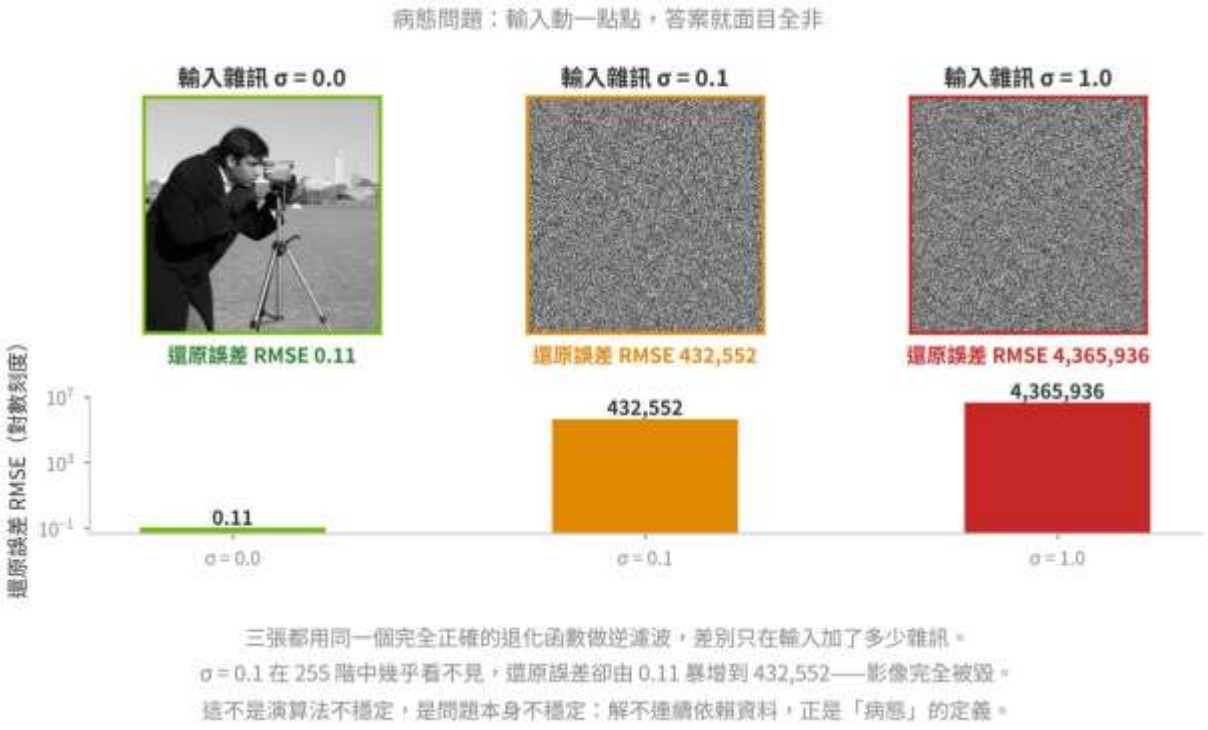


圖 7-3　病態問題：輸入動一點點，答案就面目全非

解決病態問題的通用策略叫正則化(Regularization)：在「忠實還原資料」之外，額外加上對解的約束，例如「影像應該是平滑的」、「影像的梯度應該是稀疏的」。維納濾波中的雜訊訊號比項、約束最小平方法中的平滑度項、以及深度學習模型從資料學到的先驗，本質上都是正則化的不同形式。理解這一點，你就抓住了整章的骨架：所有還原方法的差別，只在於「用什麼方式告訴演算法：合理的影像應該長什麼樣子」。

7.1.4 本章在全書中的位置

影像還原是一個承先啟後的樞紐章節，它幾乎與本書每一篇都有連結，建議讀者在閱讀本章時隨時回顧與前瞻：

- 與第 3 章(數位影像基礎)：還原成效的驗收工具 MSE、PSNR、SSIM 來自第 3 章;而 RAW 檔保留的高位元深度線性資料，是還原演算法最理想的輸入——8 位元 JPEG 已經過壓縮與 Gamma 編碼，細節與線性關係都已受損。
- 與第 5 章(空間域強化)：中值、高斯、雙邊濾波在本章重新登場，但角色從「讓影像好看」變成「依雜訊模型抑制雜訊」;而夜景模式的多張平均，正是本章「以增加觀測次數對抗隨機雜訊」的實例。
- 與第 6 章(頻率域處理)：劣化模型在頻率域是乘法，還原則是除法;退化函數是低通濾波器，還原則相當於受控的高通增強;週期性雜訊的移除直接沿用第 6 章的陷波濾波。7.4 節會完整展開這條主線。
- 與第 9 章(影像分割)：雜訊會讓 Canny 產生大量假邊緣，模糊則會讓邊緣定位偏移。還原品質直接決定分割品質——7.7 節有量化實驗。
- 與第 13 章(特徵擷取):Harris、SIFT、ORB 都依賴局部灰階變化。模糊會抹平角點的響應值、雜訊會製造大量假特徵點，兩者都使匹配率下降。

- 與第 15、16 章(深度學習與 YOLO)：訓練資料若乾淨、測試場景卻模糊含噪，模型準確率會顯著下滑。還原可作為推論前的前處理，或反過來——把劣化當成資料增強來訓練更強健的模型。
- 與第 17 章(邊緣部署)：還原演算法的運算成本必須納入即時系統的預算。維納濾波需要兩次 FFT,Richardson-Lucy 需要數十次反覆，在 Jetson 上是否跑得動?這是工程決策。

7.1.5 推導：病態到什麼程度，可以用一個數字衡量

7.1.3 節說還原是病態問題，但「病態」聽起來像個形容詞。其實它有一個精確的度量，叫做條件數（condition number），而且在我們的情形下它算起來特別簡單。

先把問題寫清楚。在頻率域，劣化是逐頻率的乘法：$G(u, v) = H(u, v) \cdot F(u, v) + N(u, v)$。也就是說，這個看似龐大的問題其實被拆成了數十萬個互不相干的小問題，每個頻率各自解一個一元一次方程式。對單一頻率而言，解是 $F = (G - N) / H$，而我們不知道 N，只能算 G / H。於是誤差 $= N / H$：輸入端的誤差 N 被放大了 $1/|H|$ 倍。

條件數定義為「最壞情況下的放大倍率除以最好情況下的放大倍率」，在這裡就是 $|H|$ 的最大值除以最小值。以本章使用的高斯失焦為例，$|H|$ 的最大值是 1（直流分量，因為 PSF 的總和為 1），最小值在高頻處可以小到 1e-10 以下，條件數因此達到 1e10 的量級。這個數字的意思很具體：輸入資料中相對大小為 1e-16 的擾動（雙精度浮點數的機器精度），在輸出端可以被放大到 1e-6 的相對誤差;而影像頻譜的數值本身就有上萬的量級，乘起來就是圖 7-15 中那個「一個雜訊都沒加、結果卻跑到正負一千七」的現象。

這個推導帶出三個結論。第一，病態與否完全由 H 決定，與你用什麼演算法無關——換一台精度更高的電腦只是把災難往後推，不會消除它。第二，既然放大倍率是 $1/|H|$，那麼補救的方向就很明確：不要在 $|H|$ 很小的地方做除法。半徑受限逆濾波（7.5.1 節）用的是「乾脆不除」，維納濾波（7.5.2 節）用的是「在分母加一個常數

讓它不會太小」，兩者都是同一個想法的不同實作。第三，條件數也告訴你哪些模糊比較好救：|H| 的最小值愈大（在你關心的頻帶內），條件數愈小，問題愈良性。這是 7.4.1 節「H 的零點就是資訊死亡」的量化版本。

7.2 雜訊模型、辨識與參數估計

7.2.1 常見雜訊型態

1. 高斯雜訊(Gaussian)：機率密度為常態分布，由平均值與標準差兩個參數決定。來源包括感測器熱雜訊、電子電路雜訊、放大器雜訊。它是最常見也最常被假設的雜訊模型，一方面因為它確實普遍，另一方面因為中央極限定理——大量獨立小擾動的總和趨近常態分布。直方圖呈鐘形。

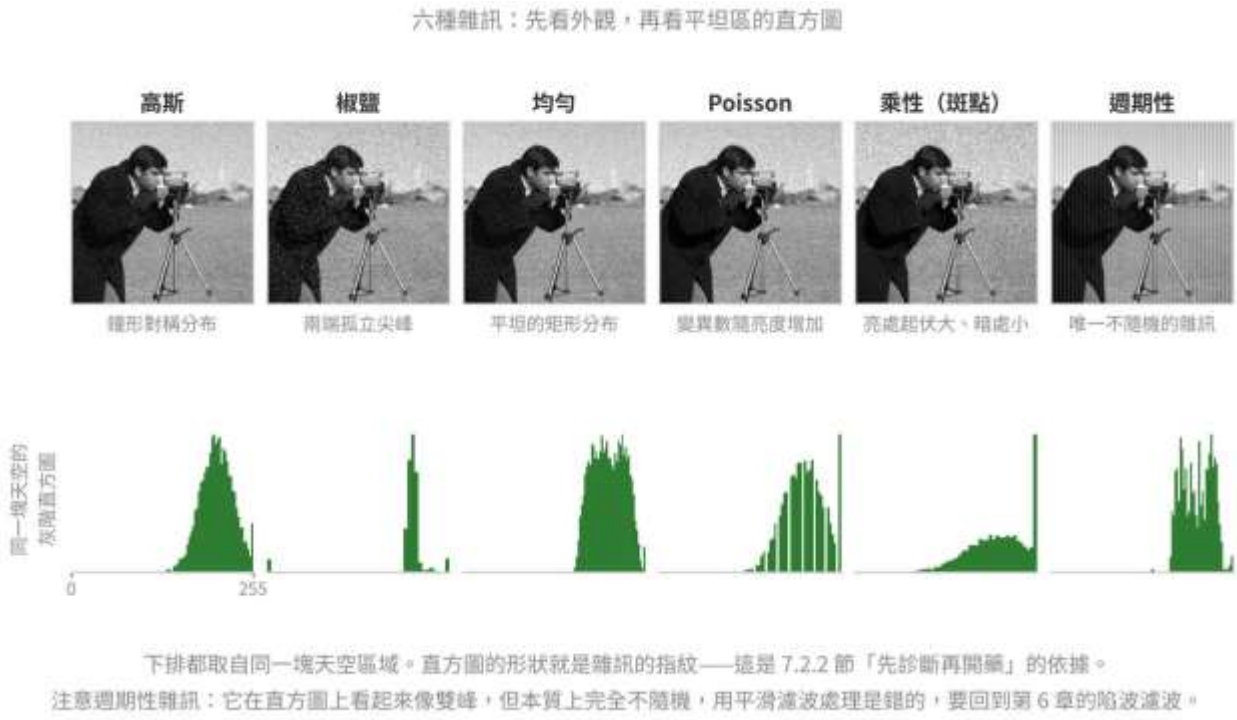


圖 7-4　六種雜訊的外觀與其平坦區直方圖

2. 椒鹽雜訊(Salt-and-Pepper)：又稱脈衝雜訊，像素以某機率隨機變成全黑(0，胡椒)或全白(255，鹽)。來源是訊號傳輸位元錯誤、類比數位轉換失誤、感測器壞點。直方圖在兩端出現孤立尖峰，中間分布不受影響——這是它與高斯雜訊最容易區分的特徵。

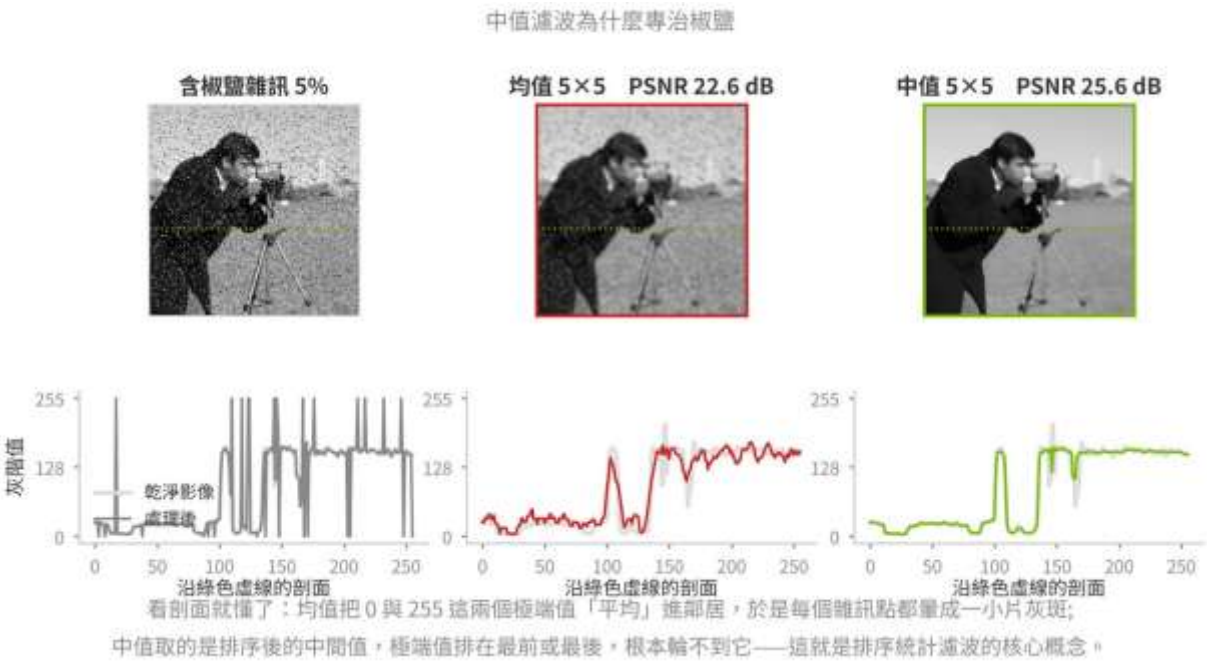


圖 7-9　中值濾波為什麼專治椒鹽：看剖面就懂

3. 均勻雜訊(Uniform)：在某區間內等機率分布。自然界較少見，但量化誤差(第 3 章)的統計特性接近均勻分布，因此在分析位元深度不足造成的失真時會用到。



圖 7-10　自適應濾波在每個位置用了多少力

4. Poisson 雜訊(散粒雜訊，Shot Noise)：源自光子到達感測器的隨機性，其變異數等於平均值——換言之，強度與訊號本身相關：亮處雜訊的絕對值大、暗

處小，但暗處的「相對雜訊」(訊噪比)反而更差。這正是第 5 章夜景模式要對付的主角，也是低光攝影、醫學影像(X 光光子計數)、天文攝影的主要雜訊來源。

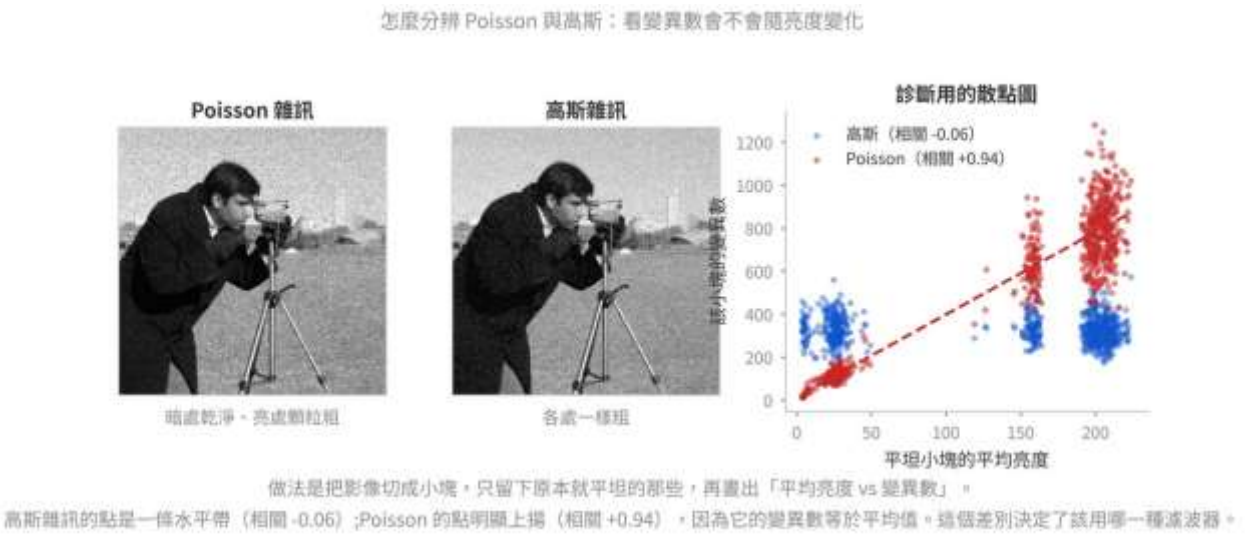


圖 7-5　以變異數是否隨亮度變化分辨 Poisson 與高斯

5. 乘性雜訊與斑點雜訊(Speckle)：雜訊與訊號相乘而非相加，常見於超音波影像、合成孔徑雷達(SAR)。處理策略通常是先取對數把乘法變成加法，再套用加性雜訊的方法，最後取指數還原。



圖 7-11　非局部平均：不問遠近，只問像不像

6. 週期性雜訊(Periodic)：來自電源干擾、機械振動、掃描機構的規律誤差。它是唯一「不隨機」的雜訊，因此不該用平滑濾波處理——正確做法是回到第 6 章的頻率域，以陷波濾波精準移除，詳見 7.4.3 節。

7.2.2 雜訊辨識與參數估計

還原工作的第一步永遠是診斷。實務技巧是：找一塊本該平坦均勻的區域(例如純色背景、牆面、天空、工件的平整表面)，把該區域的像素值取出來分析。因為原始訊號在該區近乎常數，所觀察到的變異幾乎全部來自雜訊。

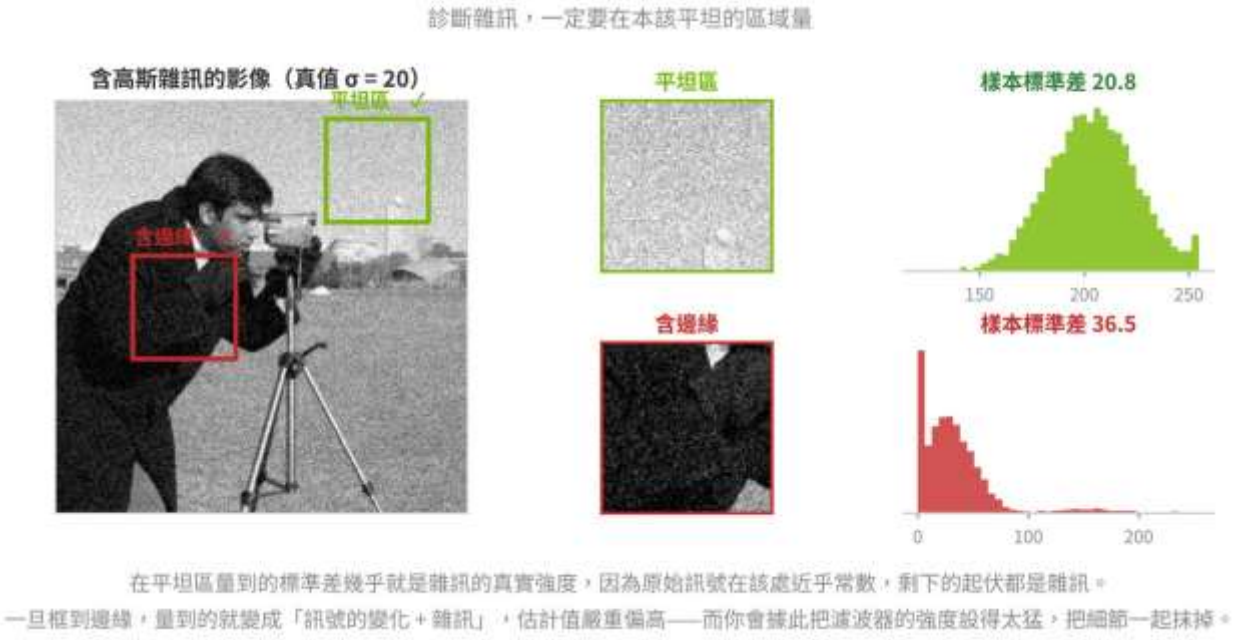


圖 7-6　診斷雜訊一定要在本該平坦的區域量

依直方圖形狀判讀：鐘形對稱分布指向高斯雜訊;兩端出現孤立尖峰、中央分布正常指向椒鹽雜訊;平坦的矩形分布指向均勻雜訊;若變異數隨區域平均亮度成正比增加(需比較亮區與暗區兩塊平坦區)，則指向 Poisson 雜訊。判斷正確，才能選對濾波器——用中值濾波去對付高斯雜訊，或用均值濾波去對付椒鹽雜訊，都是事倍功半的常見錯誤。

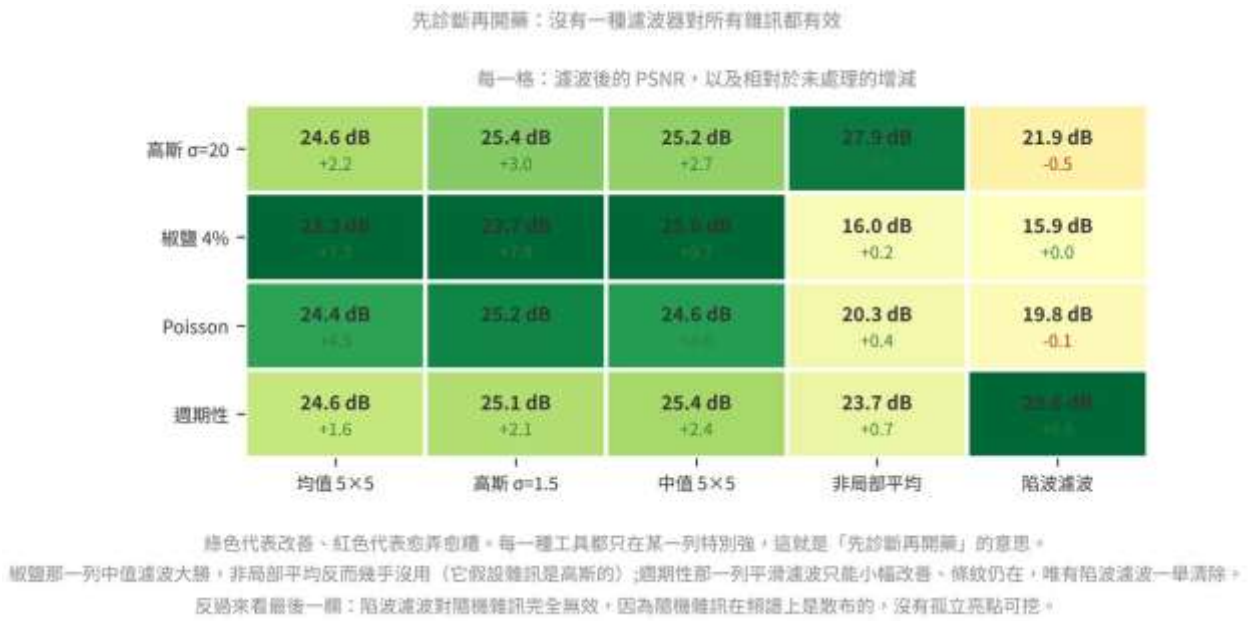


圖 7-8　四種雜訊與五種濾波器的配對矩陣

參數估計則更進一步：高斯雜訊的標準差可直接由平坦區的樣本標準差估得，這個數值是設定維納濾波 K 值、非局部平均 h 參數的重要依據。另一個不需要人工選區的實用技巧，是利用高通濾波的殘差來估計雜訊：對影像做拉普拉斯運算，由於自然影像的高頻能量遠小於雜訊，殘差的統計量可作為雜訊強度的粗估。

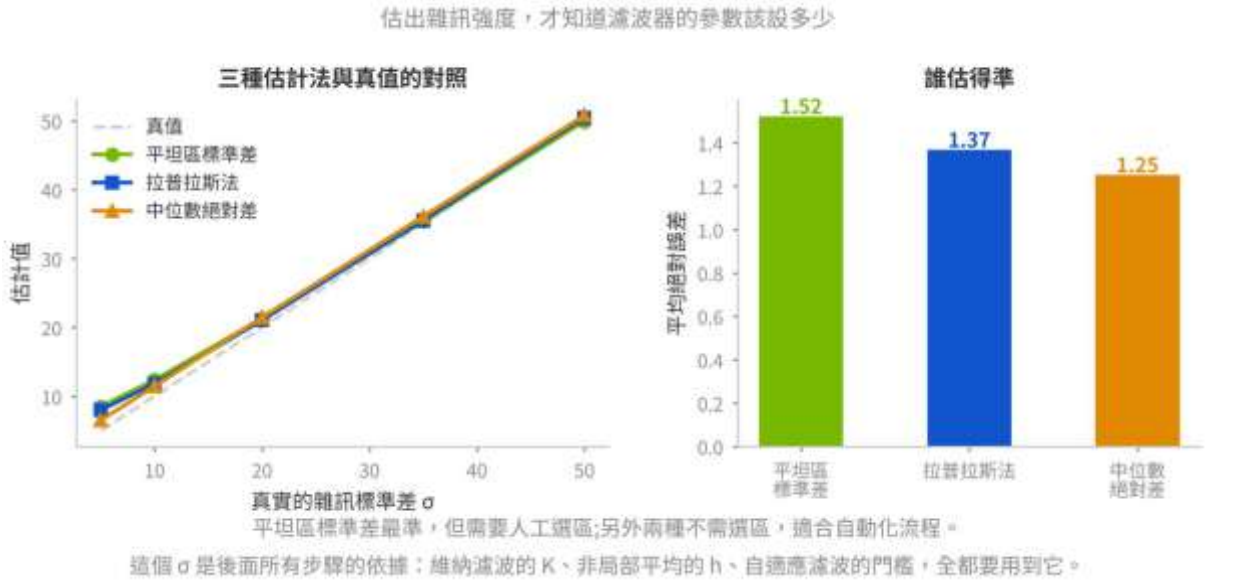


圖 7-7　三種雜訊強度估計法與真值的對照

程式 7-1 產生雜訊影像與估計雜訊參數

```
import cv2, numpy as np

img = cv2.imread("clean.jpg", 0).astype(np.float32)
```

```
# --- 加入高斯雜訊 ---
sigma = 20
gauss = np.random.normal(0, sigma, img.shape)
noisy_g = np.clip(img + gauss, 0, 255).astype(np.uint8)

# --- 加入椒鹽雜訊(各 2%) ---
noisy_sp = img.copy().astype(np.uint8)
p = 0.02
rnd = np.random.rand(*img.shape)
noisy_sp[rnd < p] = 0            # 胡椒(黑)
noisy_sp[rnd > 1 - p] = 255      # 鹽(白)

# --- 加入 Poisson 雜訊(強度隨訊號變化) ---
noisy_po = np.random.poisson(img).clip(0, 255).astype(np.uint8)

# --- 雜訊參數估計 ---
# 方法一:取平坦 ROI 直接算標準差
roi = noisy_g[10:60, 10:60].astype(np.float32)   # 選一塊平坦背景
print("ROI 估計 sigma:", roi.std())

# 方法二:以拉普拉斯殘差估計(免選區)
lap = cv2.Laplacian(noisy_g.astype(np.float32), cv2.CV_32F)
sigma_est = np.sqrt(np.pi / 2) * np.abs(lap).mean() / 6
print("拉普拉斯法估計 sigma:", sigma_est)
```

7.3 空間域去雜訊：從排序統計到非局部平均

當劣化只有雜訊、沒有模糊(即 h 為單位脈衝)時，問題退化為純粹的去雜訊，在空間域處理往往最直接有效。第 5 章介紹的均值、高斯、中值濾波在此重新登場，但這次我們關心的不是「好不好看」，而是「哪一種在對應的雜訊模型下，統計上最接近原始值」。

7.3.1 均值濾波家族

算術平均是高斯雜訊的最佳線性估計：若鄰域內的原始訊號近乎常數，平均 m 個獨立同分布的雜訊樣本，雜訊標準差降為原來的 m 的平方根分之一。這正是第 5 章夜景模式「連拍平均」的數學根據，差別只在於：夜景模式平均的是「不同時間的同一個像素」(訊號完全相同，理想)，而空間平均的是「同一時間的不同像素」(訊號未必相同，因此會模糊邊緣)。這個對照值得學生細細體會——它說明了為什麼多影格去雜訊優於單一影格去雜訊。

幾何平均能在去雜訊的同時比算術平均保留更多細節;諧波平均對鹽雜訊(白點)與高斯雜訊有效，但無法處理胡椒雜訊;反諧波平均則以階數 Q 控制：Q 為正時消除胡椒雜訊，Q 為負時消除鹽雜訊，Q 為零時退化為算術平均——但用錯正負號會使結果更糟，實務上必須先完成 7.2 節的診斷。

7.3.2 排序統計濾波家族

這一族不做算術，而是把鄰域內的像素排序後挑選特定名次的值：中值濾波取中位數，對椒鹽雜訊近乎完美，因為極端值排在最前或最後，永遠不會被選中;最大值濾波取最大值，適合去除胡椒雜訊(黑點)，但會讓亮區膨脹;最小值濾波取最小值，適合去除鹽雜訊(白點);中點濾波取最大與最小的平均，對高斯與均勻雜訊表現不錯;alpha 修剪均值濾波先刪去最大與最小的若干個值再平均，能同時應付混合了高斯與脈衝的雜訊，是實務上很實用的折衷。

這裡也埋著一個與第 10 章的伏筆：最大值與最小值濾波，其實就是型態學的膨脹(Dilation)與侵蝕(Erosion)運算。同一個運算，在本章是「去雜訊工具」，在第 10 章是「形狀操作工具」——影像處理中許多概念都有這種一體多面的性格。

7.3.3 自適應濾波：讓濾波器懂得看場合

固定核濾波的根本缺陷是「一視同仁」：它在平坦區與邊緣區施加同樣強度的平滑，結果不是雜訊沒去乾淨，就是細節被抹掉。自適應濾波的核心思想是：先看看局部的統計特性，再決定要不要動手、動多重的手。

7. 自適應局部降噪濾波器：比較「全域雜訊變異數」與「局部變異數」。若局部變異數接近雜訊變異數，代表這裡是平坦區，雜訊佔主導，應大幅平滑;若局部變異數遠大於雜訊變異數，代表這裡有邊緣或紋理，應盡量保留原值。它的行為可以理解成：平坦處當均值濾波用，邊緣處幾乎不作用。
8. 自適應中值濾波：固定核中值濾波在雜訊密度高時會失效(鄰域內超過一半都是雜訊，中位數本身就是雜訊)，且會抹平細節。自適應版本分兩階段：第一階段

判斷「中位數本身是不是雜訊(等於 0 或 255)」，若是則擴大視窗重試，直到找到有效的中位數或達到視窗上限;第二階段判斷「中心像素是不是雜訊」，若不是就原值保留、完全不動。如此一來，乾淨的像素受到完整保護，只有雜訊點被替換，細節保留能力遠優於固定中值濾波，在雜訊密度達 20% 以上時差距尤其懸殊。

7.3.4 非局部平均：跳脫鄰近的假設

以上所有方法都建立在同一個假設上：相鄰的像素值應該相似。非局部平均(Non-Local Means,NLM)徹底顛覆了這個假設，改為在整張影像中尋找「周圍紋理相似的區塊」來加權平均——不管它離得多遠。它利用的是自然影像的自相似性：磚牆上的每一塊磚、毛衣上的每一道織紋、草地上的每一叢草，在影像各處反覆出現。

具體做法：對每個待處理像素，取一個小的比對視窗(template window，例如 7×7)，在一個較大的搜尋範圍(search window，例如 21×21)內尋找相似的視窗，依相似度給權重再加權平均。因為參與平均的是「內容相同」的像素而非「位置相鄰」的像素，邊緣與紋理得以完整保留，去雜訊能力顯著優於傳統濾波，是深度學習出現前的最強方法之一。代價是運算量大——這也是它在即時系統中較少使用、卻在離線修復(如老照片、醫學影像)中大放異彩的原因。

程式 7-2 五種去雜訊方法的客觀比較

```
import cv2, numpy as np

clean = cv2.imread("clean.jpg", 0)
noisy = cv2.imread("noisy_gaussian.png", 0)

# --- 傳統濾波 ---
mean_f  = cv2.blur(noisy, (5, 5))
gauss_f = cv2.GaussianBlur(noisy, (5, 5), 1.5)
med_f   = cv2.medianBlur(noisy, 5)
bil_f   = cv2.bilateralFilter(noisy, 9, 75, 75)

# --- 非局部平均(h 應依估得的雜訊 sigma 設定) ---
nlm = cv2.fastNlMeansDenoising(noisy, None, h=20,
                               templateWindowSize=7,
                               searchWindowSize=21)
```

```
# --- 客觀比較(第 3 章工具) ---
for name, out in [("原始含噪", noisy), ("均值", mean_f),
                  ("高斯", gauss_f), ("中值", med_f),
                  ("雙邊", bil_f), ("NLM", nlm)]:
    print(f"{name:8s} PSNR = {cv2.PSNR(clean, out):.2f} dB")
```

7.4 頻率域觀點：還原與濾波的血緣關係

這一節是本章與第 6 章的接榫，也是理解所有反卷積方法的鑰匙。請先記住一句話：模糊就是低通濾波，還原就是受控的高通增強。

7.4.1 退化函數是低通濾波器

回想第 6 章：影像的高頻成分對應邊緣與細節，低頻對應平坦區域與整體亮度。任何形式的模糊——失焦、運動、大氣擾動、鏡頭像差——在頻率域上做的事都一樣：把高頻成分的振幅壓低。以高斯模糊為例，它的轉移函數 H 本身也是高斯形狀，中心(低頻)為 1，愈往外(高頻)愈趨近於零。運動模糊的 H 則更兇殘：它是一個 sinc 函數，在某些特定頻率上直接等於零——那些頻率的資訊被完全抹除，再高明的演算法也救不回來。

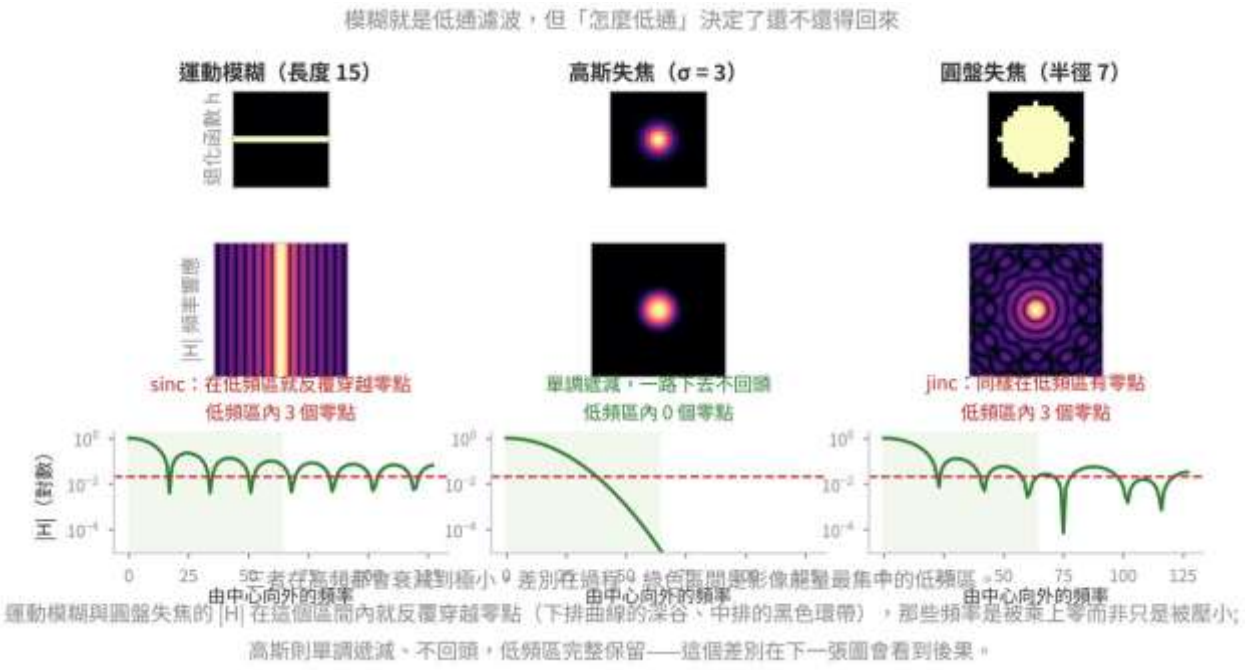


圖 7-12　三種退化函數的頻率響應與它們的零點

這解釋了一個常令初學者困惑的現象：為什麼有些模糊影像還原後很漂亮，有些

卻無論如何都救不回來?答案就在 H 的零點：資訊被乘以零的地方，是真正的資訊死亡，不是演算法不夠好。

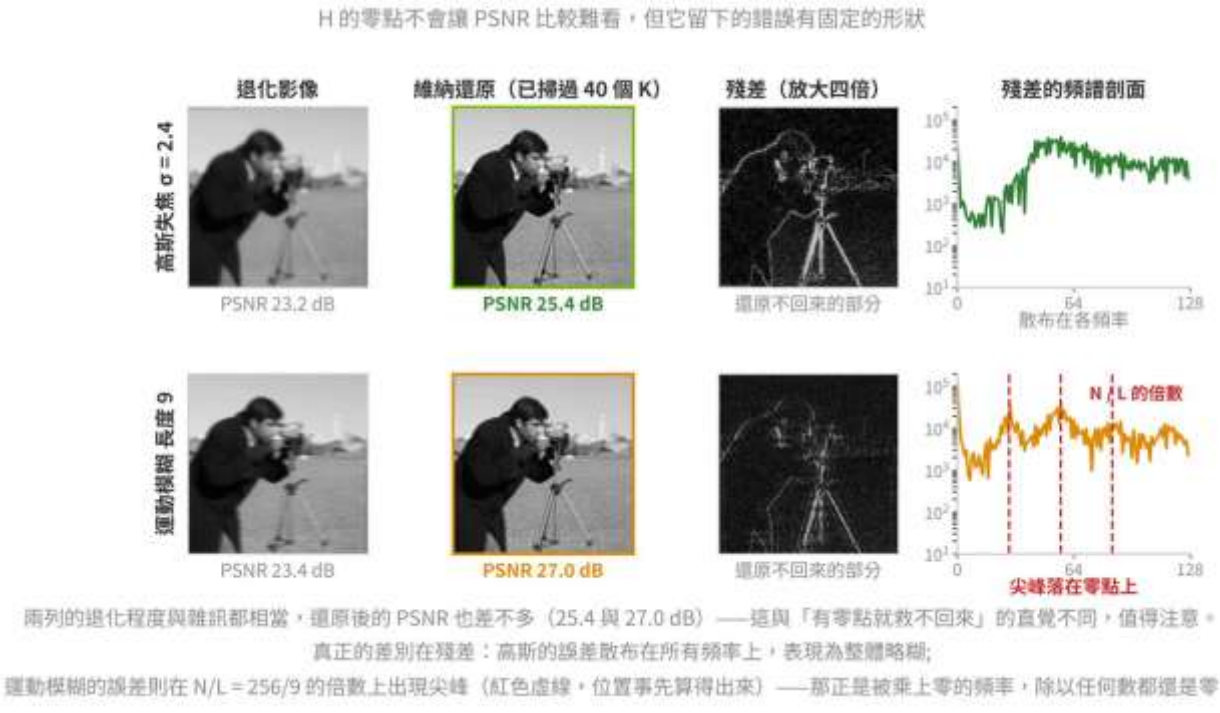


圖 7-13　H 的零點留下的殘差有固定的形狀

7.4.2 還原是受控的高通增強

既然模糊是把高頻乘上一個小數，還原自然就是把它除回來——也就是對高頻施加巨大的增益。這在概念上與第 6 章的高通濾波、第 5 章的 High-Boost 銳化是同一件事：三者都在提升高頻。差別在於：

- 第 5 章的 Unsharp Masking：增益由使用者主觀設定的 k 決定，與造成模糊的物理過程無關。
- 第 6 章的高頻強調濾波：增益由濾波器形狀決定，同樣不管實際的退化函數是什麼。
- 本章的反卷積：增益由退化函數 H 精確決定——H 在哪個頻率壓得多，就在哪個頻率補得多。這是「量身訂做」的高通增強。

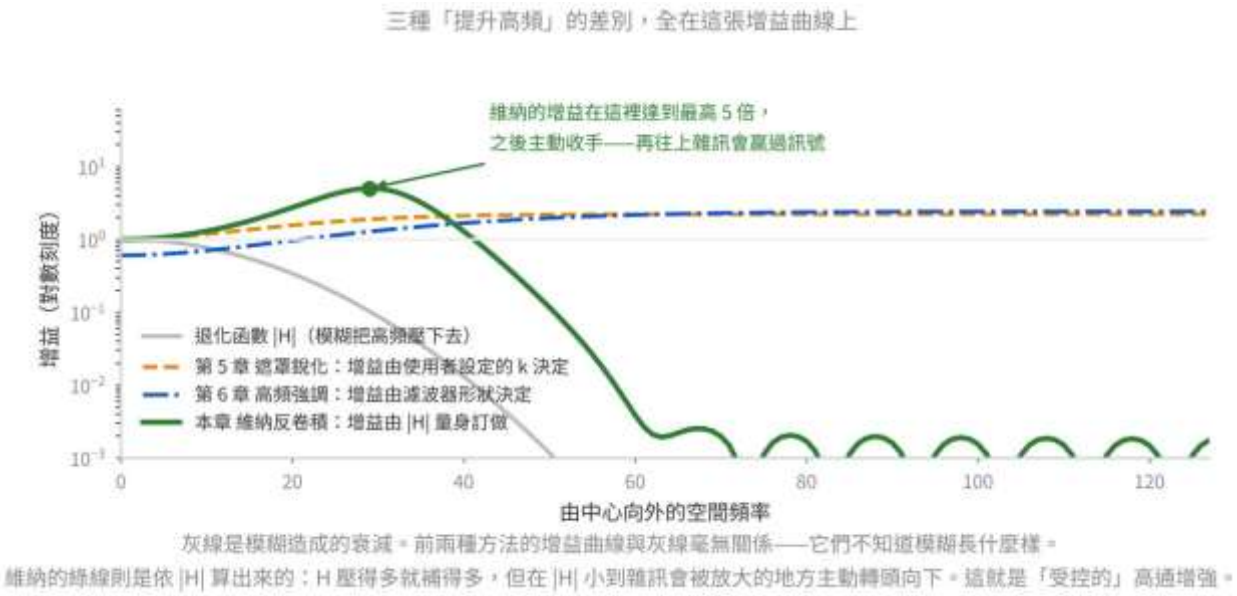


圖 7-14　三種高頻增強的增益曲線對照

而三者共享同一個宿命：雜訊主要分布在高頻，因此任何提升高頻的動作，都必然同時放大雜訊。這是影像處理中最根本的矛盾之一——銳利與乾淨不可兼得。第 5 章用「先去雜訊再銳化」的順序來緩解，第 6 章用平滑的濾波器形狀避免振鈴，而本章的維納濾波與約束最小平方，則是把這個矛盾寫成數學式，直接求出最佳的折衷點。

7.4.3 週期性雜訊：直接沿用第 6 章

週期性雜訊是本章唯一「不需要動用反卷積」的情況。因為它在頻率域上表現為少數幾個孤立亮點，只要以第 6 章的陷波濾波(記得處理共軛對稱點)把那幾點壓下，反轉換即可乾淨移除，且細節毫髮無傷。反之，若誤用空間域的平滑濾波去對付它，必然是全圖細節陪葬。這個例子最能說明「選對域比選對演算法更重要」。

程式 7-3 觀察退化函數的頻率響應

```
import cv2, numpy as np

# 觀察三種退化函數在頻率域的樣貌(理解還原難度)
N = 256

# 高斯模糊 PSF
k = cv2.getGaussianKernel(15, 3); psf_gauss = k @ k.T

# 水平運動模糊 PSF(長度 15)
psf_motion = np.zeros((15, 15), np.float32)
psf_motion[7, :] = 1.0 / 15
```

```
for name, psf in [("gauss", psf_gauss), ("motion", psf_motion)]:
    pad = np.zeros((N, N), np.float32)
    pad[:psf.shape[0], :psf.shape[1]] = psf
    H = np.fft.fftshift(np.fft.fft2(pad))
    mag = np.abs(H)
    print(f"{name}: |H| 最小值 = {mag.min():.6f}")
    # 運動模糊的最小值接近 0 → 該頻率資訊完全喪失,無法還原
    spec = 20 * np.log(mag + 1e-8)
    cv2.imwrite(f"H_{name}.png",
        cv2.normalize(spec, None, 0, 255,
                      cv2.NORM_MINMAX).astype(np.uint8))
```

7.5 反卷積：逆濾波、維納濾波與約束最小平方

7.5.1 逆濾波為何失敗

既然頻率域中 G = H·F + N，最直覺的解法就是兩邊除以 H:F̂ = G / H。若完全沒有雜訊(N = 0)，這個逆濾波確實能完美還原。但真實世界中 N 從不為零，展開後得到：

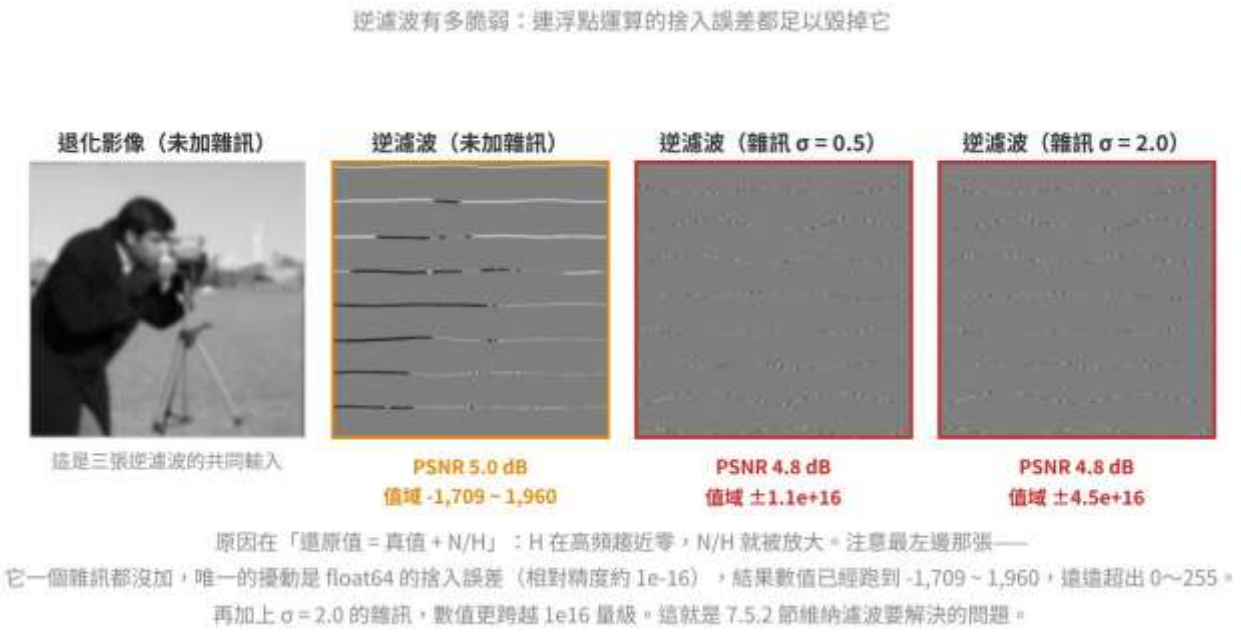


圖 7-15　逆濾波有多脆弱：連捨入誤差都足以毀掉它

$$\hat{F}(u,v) = F(u,v) + N(u,v) / H(u,v)$$

問題就出在第二項。承 7.4.1 節，退化函數 H 在高頻通常趨近於零，當 H 很小時，N/H 會被放大到天文數字，雜訊完全淹沒訊號;若 H 恰好等於零，更是直接除以零。這正是 7.1.3 節所說病態性的具體表現：除法在數學上合法，在工程上災難。

有一個簡單的補救叫半徑受限逆濾波：既然災難發生在高頻，那就只在以頻譜中心為圓心、半徑 D0 之內執行除法，半徑之外一律不動(或設為零)。這相當於「逆濾波 + 低通濾波」的組合，能得到堪用的結果，但 D0 的選擇完全靠試誤，而且截斷會帶來第 6 章的振鈴效應。這個粗糙的做法提示了正確方向：我們需要一個「隨頻率自動調整還原力道」的機制——那就是維納濾波。

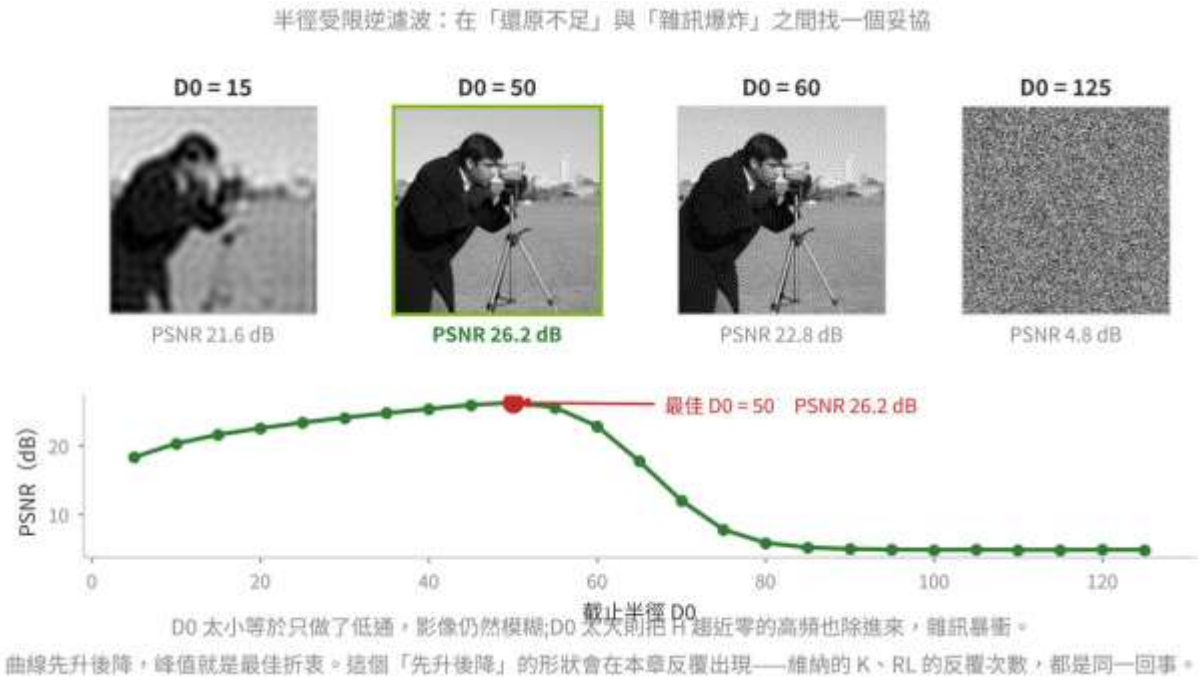


圖 7-16　半徑受限逆濾波的 D0 掃描

7.5.2 維納濾波

維納濾波(Wiener Filter)由 Norbert Wiener 提出，以最小均方誤差為準則，在「還原銳利度」與「抑制雜訊放大」之間取得最佳平衡。其轉移函數為：

$$\hat{F}(u,v) = \left[H^*(u,v) / \left(|H(u,v)|^2 + S_n(u,v)/S_f(u,v) \right) \right] \cdot G(u,v)$$

其中 H* 是 H 的共軛複數，Sn 與 Sf 分別是雜訊與原始影像的功率譜。關鍵在分母多出來的 Sn/Sf 項(常以常數 K 近似，即雜訊對訊號的功率比)：當某頻率的訊號夠強(Sf 大、K 相對小)時，該項趨近 0，公式退化為逆濾波，盡力還原;當訊號微弱而雜訊相對強時，該項變大，壓抑該頻率的還原力道，避免雜訊爆炸。一句話總結：維納濾波是「知道何時該放棄」的逆濾波。

把它與 7.4.2 節對照更能看出巧妙之處：維納濾波在低頻(訊噪比高)做接近完全的高通增強，在高頻(訊噪比低)自動收手，等於內建了一個「依訊噪比自動調整的低通保護」。這也解釋了為什麼維納濾波的結果通常比原圖銳利、卻又不像逆濾波那樣佈滿雪花。

實務上 Sn 與 Sf 很難精確得知，因此常把 Sn/Sf 直接當成可調參數 K:K 太小則雜訊被放大、影像顆粒粗糙;K 太大則還原不足、影像仍然模糊。7.2.2 節估計出的雜訊標準差，是設定 K 的重要參考起點。調參的過程，就是在銳利與乾淨之間找甜蜜點——這個尋找的過程，正是 Lab 7-2 的核心。

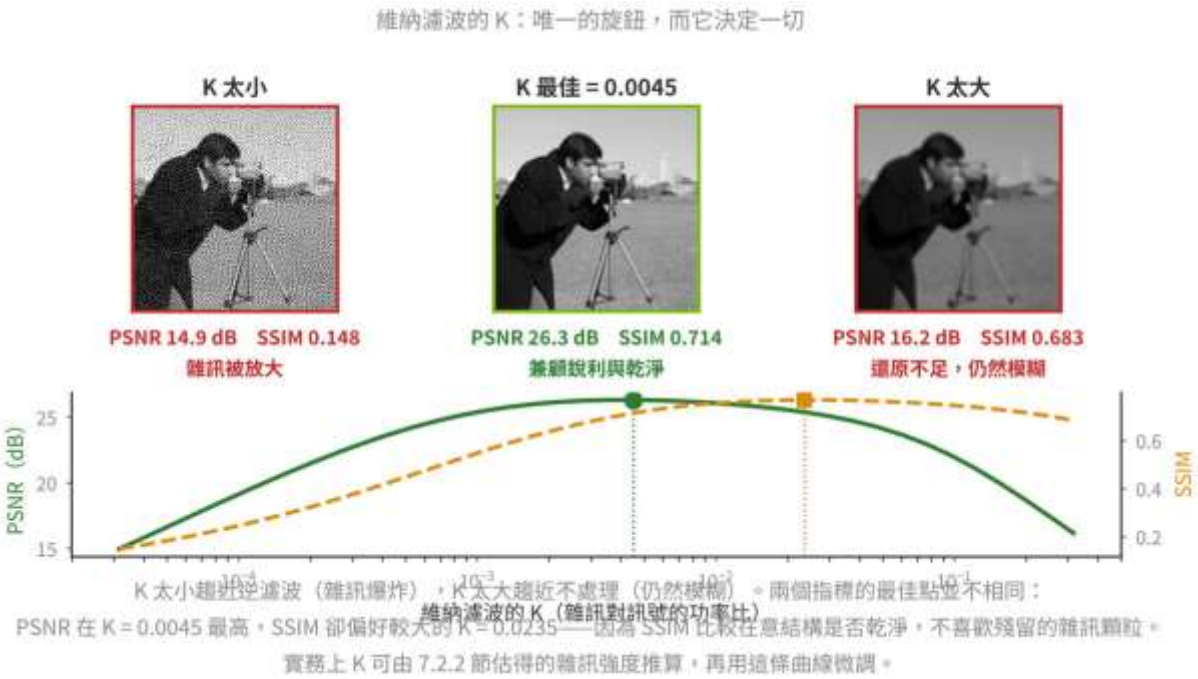


圖 7-17　維納濾波的 K：唯一的旋鈕，而它決定一切

程式 7-4 維納濾波反卷積實作

```
import cv2, numpy as np

def wiener_filter(img, psf, K=0.01):
    """維納濾波反卷積。img:退化影像;psf:退化函數;K:雜訊訊號比"""
    img = img.astype(np.float32)
    # 把 PSF 填補到與影像同尺寸並轉到頻率域
    psf_pad = np.zeros_like(img)
    kh, kw = psf.shape
    psf_pad[:kh, :kw] = psf
    psf_pad = np.roll(psf_pad, -kh // 2, axis=0)
    psf_pad = np.roll(psf_pad, -kw // 2, axis=1)
```

```
    H = np.fft.fft2(psf_pad)
    G = np.fft.fft2(img)

    # 維納公式:H* / (|H|^2 + K)
    W = np.conj(H) / (np.abs(H) ** 2 + K)
    F_hat = W * G

    out = np.abs(np.fft.ifft2(F_hat))
    return np.clip(out, 0, 255).astype(np.uint8)

# --- 實驗:製造模糊 + 雜訊,再還原 ---
clean = cv2.imread("clean.jpg", 0)
psf = cv2.getGaussianKernel(15, 3)
psf = psf @ psf.T                          # 15x15 高斯 PSF

blurred = cv2.filter2D(clean, -1, psf)
noisy = np.clip(blurred + np.random.normal(0, 5, clean.shape), 0, 255)

print("未還原(模糊+雜訊)PSNR:",
      cv2.PSNR(clean, noisy.astype(np.uint8)))

for K in [0.0001, 0.001, 0.01, 0.1]:
    restored = wiener_filter(noisy, psf, K)
    print(f"K={K}: PSNR = {cv2.PSNR(clean, restored):.2f} dB")
    cv2.imwrite(f"wiener_K{K}.png", restored)
```

7.5.3 約束最小平方濾波

維納濾波有個實務門檻：它需要知道 Sn/Sf，而這兩個功率譜通常都不知道。約束最小平方濾波(Constrained Least Squares)提供了另一條路：它只需要知道雜訊的平均值與變異數(比功率譜容易估計得多)，並把「還原結果應該平滑」這件事直接寫進最佳化目標。其轉移函數為：

$$F(u,v) = [\, H^*(u,v) / (\, |H(u,v)|^2 + \gamma \cdot |P(u,v)|^2 \,) \,] \cdot G(u,v)$$

其中 P 是拉普拉斯運算子的頻率響應(第 5 章的二階微分核),γ 是調節參數。比較兩式可以發現：維納濾波在分母加的是常數 K，約束最小平方加的則是隨頻率變化的 $\gamma|P|^2$。因為拉普拉斯在高頻的響應大，這等於「頻率愈高、抑制愈強」——把「不要產生劇烈震盪的結果」這個先驗知識，寫成了數學。這正是 7.1.3 節所說正則化的教科書範例：γ 就是正則化強度，調大則結果平滑但模糊，調小則銳利但可能出現雜訊與振鈴。



圖 7-18　維納濾波與約束最小平方的對照

7.5.4 反覆式反卷積：Richardson-Lucy

以上都是一次算完的線性方法，另有一類反覆逼近的非線性方法，代表是 Richardson-Lucy 演算法(Richardson 1972、Lucy 1974 分別在光學與天文領域獨立提出)。它假設 Poisson 雜訊模型(因此特別適合光子計數的天文與醫學影像)，從一個初始估計出發，每次反覆用「觀測影像除以目前估計的模糊結果」來修正估計，並保證每次結果都非負(符合光強度的物理意義，這是線性方法做不到的優點——維納濾波的輸出可能出現負值)。

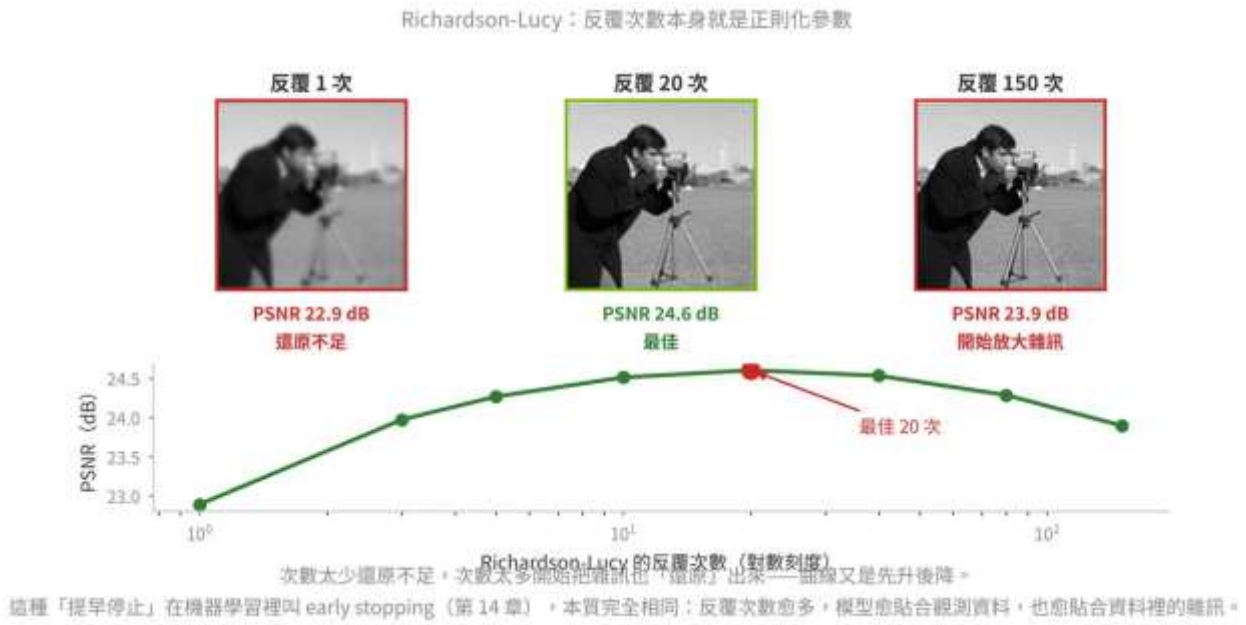


圖 7-19　Richardson-Lucy：反覆次數本身就是正則化參數

這正是叫獸開講中拯救哈伯望遠鏡的主力演算法之一。實務上有個關鍵抉擇：反覆次數。次數太少還原不足;次數太多則開始放大雜訊，影像出現不自然的顆粒與塊狀結構。天文學界處理哈伯早期影像時，經驗上約 50 次反覆能明顯改善影像品質、削減散射光暈又保留原始結構;超過 100 次後，物體的形狀反而開始崩壞、出現人造的塊狀質感。「知道何時停手」是所有反覆式還原演算法的共同課題，學界也發展出以廣義交叉驗證等準則自動決定停止時機的方法。

7.5.5 推導：維納濾波的公式是怎麼來的

7.5.2 節直接給出了維納濾波的轉移函數，但那個分母裡的 Sn/Sf 看起來頗為神祕。其實整條公式可以從一個很單純的要求推出來：我想要一個線性濾波器，使還原結果與真實影像的均方誤差最小。推導只需要三步，而且不需要任何高等數學。

第一步，設定問題。我們要找一個轉移函數 M(u, v)，使還原結果為 $\hat{F} = M \cdot G$。目標是讓誤差的期望值 $E[|\hat{F} - F|^2]$ 最小。由於每個頻率彼此獨立（這正是 7.1.5 節提到的好處），可以對每一個頻率各自求極小，問題瞬間變得容易。

第二步，展開誤差。把 $G = H \cdot F + N$ 代入：$\hat{F} - F = M \cdot (H \cdot F + N) - F = (M \cdot H - 1) \cdot F$

+ M·N。假設訊號與雜訊不相關（這是標準假設，意思是雜訊不會剛好跟影像內容同步變化），誤差的平方期望值就是兩項的和：$|M·H - 1|^2 \cdot Sf + |M|^2 \cdot Sn$，其中 Sf 與 Sn 分別是訊號與雜訊的功率譜。這個式子已經把取捨寫得很清楚了：第一項是「還原不足」的代價（M·H 偏離 1 多少），第二項是「放大雜訊」的代價。M 太小則第一項大，M 太大則第二項大。

第三步，求極小。把上式對 M 微分並令導數為零，整理後得到 $M = H^* / (|H|^2 + Sn/Sf)$，其中 H^* 是 H 的共軛複數。這就是 7.5.2 節的維納濾波公式。分母裡的 Sn/Sf 不再神祕——它就是「雜訊功率對訊號功率的比值」，也就是訊噪比的倒數。

公式到手之後，兩個極端情形特別值得檢查，因為它們印證了圖 7-14 的增益曲線。當某個頻率的雜訊可忽略（$Sn/Sf \to 0$）時，$M \to H^*/|H|^2 = 1/H$，退化成逆濾波——完全信任資料，全力還原。當雜訊遠大於訊號（Sn/Sf 很大）時，分母被這一項主導，$M \to 0$，也就是乾脆放棄這個頻率。維納濾波的全部智慧就在這兩個極端之間的自動切換，而切換的依據是每一個頻率各自的訊噪比。這也解釋了為什麼實務上把 Sn/Sf 簡化成常數 K 仍然好用：即使忽略了訊噪比隨頻率的變化，$|H|^2$ 這一項仍然會讓濾波器在 H 小的地方自動收手。

7.6 應用案例

7.6.1 案例一：MRI 醫學影像去噪

維納濾波在醫學影像領域有長期的應用傳統。以磁振造影(MRI)為例：掃描時間愈長，訊號累積愈多、訊噪比愈好，但病人必須維持不動、費用也愈高;工程上因此常以較快的掃描搭配後處理去噪。將維納濾波套用於含雜訊的 MRI 切片影像後，以 PSNR 量化比較處理前後的差異，是評估演算法成效的標準做法——處理後 PSNR 若提升數個 dB，即代表雜訊被有效抑制而結構資訊得以保留。

醫學影像的特殊要求也值得注意，而且是本案例最重要的教學點：過度平滑可能

抹去微小的病灶，把腫瘤的早期徵兆一起「濾掉」。更麻煩的是，這種錯誤在 PSNR 上幾乎看不出來——一個直徑十個像素的病灶，佔整張影像的比例微乎其微，把它抹平對全域 MSE 的影響小到可以忽略，PSNR 甚至可能因為背景更乾淨而上升。這說明了全域指標的根本盲點：它衡量的是平均表現，而醫療關心的是最壞情況。因此醫療應用的去噪往往寧可保守，並且必須經過臨床驗證與放射科醫師的視覺評估，不能只看 PSNR 好看就上線。

7.6.2 案例二：哈伯太空望遠鏡的 PSF 反卷積

叫獸開講中的哈伯救援，是「非盲反卷積」的教科書級案例，值得從工程角度再拆解一次。它成功的關鍵有三：

9. 退化函數可測量：天文學家可以觀測遙遠的恆星——那是完美的點光源，它在感光元件上留下的光斑，就是點擴散函數 PSF 的直接量測值。此外也可從光學設計反推理論 PSF，理論 PSF 的優點是不含雜訊。有了 h，問題從困難的盲反卷積降級為可解的非盲反卷積。
10. 雜訊模型明確：天文影像的主要雜訊是光子計數的 Poisson 雜訊，正好符合 Richardson-Lucy 的假設前提，演算法與物理模型高度契合。
11. 願意接受不完美：天文學家的目標不是「拍出好看的照片」，而是「量出正確的亮度與位置」。即使還原後的影像仍有殘餘偽影，只要科學量測的誤差在可接受範圍內，任務就算成功。

這個案例也暴露了實務的複雜性：哈伯的 PSF 並非位置不變(視野不同區域的 PSF 不同)，也非時間不變(太空船抖動、望遠鏡結構受熱脹冷縮影響，PSF 會在數十分鐘的尺度上緩慢改變)。這直接違反 7.1.2 節劣化模型的第一個假設，因此實務上必須分區塊、分時段建模——理論模型與工程現實之間的落差，永遠是工程師的主戰場。

7.6.3 案例三：產線運動模糊還原與下游任務

第三個案例最貼近本學程的場域。想像一條輸送帶，工件以每秒 0.5 公尺的速度

通過檢測站，相機曝光時間 2 毫秒——工件在曝光期間移動了 1 毫米。若影像的空間解析度是每像素 0.05 毫米，這代表每個點被抹成了 20 個像素長的線段：典型的水平運動模糊。結果是：第 9 章的邊緣偵測找不到清楚的工件輪廓，第 13 章的特徵匹配比對不上模板，第 16 章的 YOLO 信心值大幅下降，整條檢測線的誤判率飆升。

這個問題有三個層次的解法，依序是工程師應該考慮的優先順序：

12. 光學與機構層(最優先)：縮短曝光時間並加強打光(第 17 章的 AOI 光源設計)，或改用全域快門(Global Shutter)相機、加裝頻閃光源在極短時間內凍結畫面。這是治本之道——沒有模糊，就不需要還原。

13. 演算法層(次之)：若模糊無法避免，則運動模糊的退化函數其實是已知的!模糊方向由輸送帶方向決定，模糊長度等於「速度乘以曝光時間再除以像素尺寸」。編碼器的速度訊號與相機的曝光參數都是現成的資料，可以即時計算出 PSF，再用維納濾波還原。這是非盲反卷積在工業現場的漂亮應用。



圖 7-20　由產線參數推算運動模糊的 PSF 並還原

14. 學習層(最後)：若前兩者都受限，可考慮把運動模糊當成資料增強加入訓練集(第 15 章)，讓 YOLO 直接學會辨識模糊的工件。這是「與其還原資料以配合模型，不如訓練模型以適應資料」的思路轉換。

本案例的教學價值在於：影像還原不是孤立的技術練習，而是整個系統設計中的一環。一個好的視覺工程師，會先問「這個模糊能不能從源頭消除」，而不是一頭栽進演算法。

程式 7-5 由產線參數推算 PSF 並還原運動模糊

```
import cv2, numpy as np

def motion_psf(length, angle_deg):
    """產生運動模糊 PSF:length 為模糊長度(像素),angle 為方向"""
    psf = np.zeros((length, length), np.float32)
    psf[length // 2, :] = 1.0
    M = cv2.getRotationMatrix2D((length / 2 - 0.5, length / 2 - 0.5),
                                angle_deg, 1.0)          # 第 4 章的旋轉
    psf = cv2.warpAffine(psf, M, (length, length))
    return psf / psf.sum()

# --- 由產線參數推算 PSF(非盲反卷積的關鍵) ---
speed_mm_s   = 500.0    # 輸送帶速度(mm/s),來自編碼器
exposure_s   = 0.002    # 曝光時間(s),來自相機設定
pixel_mm     = 0.05     # 每像素代表的實際尺寸(mm),來自第 4 章相機校正

blur_len = int(round(speed_mm_s * exposure_s / pixel_mm))
print("推算模糊長度(像素):", blur_len)          # → 20

psf = motion_psf(blur_len, angle_deg=0)           # 水平運動

blurred = cv2.imread("part_blurred.png", 0)
restored = wiener_filter(blurred, psf, K=0.005)   # 沿用程式 7-4
cv2.imwrite("part_restored.png", restored)

# --- 驗證還原對下游任務的效益(第 9、13 章) ---
for name, im in [("模糊", blurred), ("還原", restored)]:
    edges = cv2.Canny(im, 80, 160)
    orb = cv2.ORB_create(1000)
    kps = orb.detect(im, None)
    print(f"{name}: 邊緣像素數 = {int(edges.sum() / 255)},"
          f" ORB 特徵點數 = {len(kps)}")
```

7.7 影像還原對後續處理的影響

影像還原很少是最終目的，它幾乎總是某個更大任務的前處理步驟。因此評估還原的價值，除了看 PSNR，更應該看它讓下游任務進步了多少。以下三個連結值得學生實際動手量化：

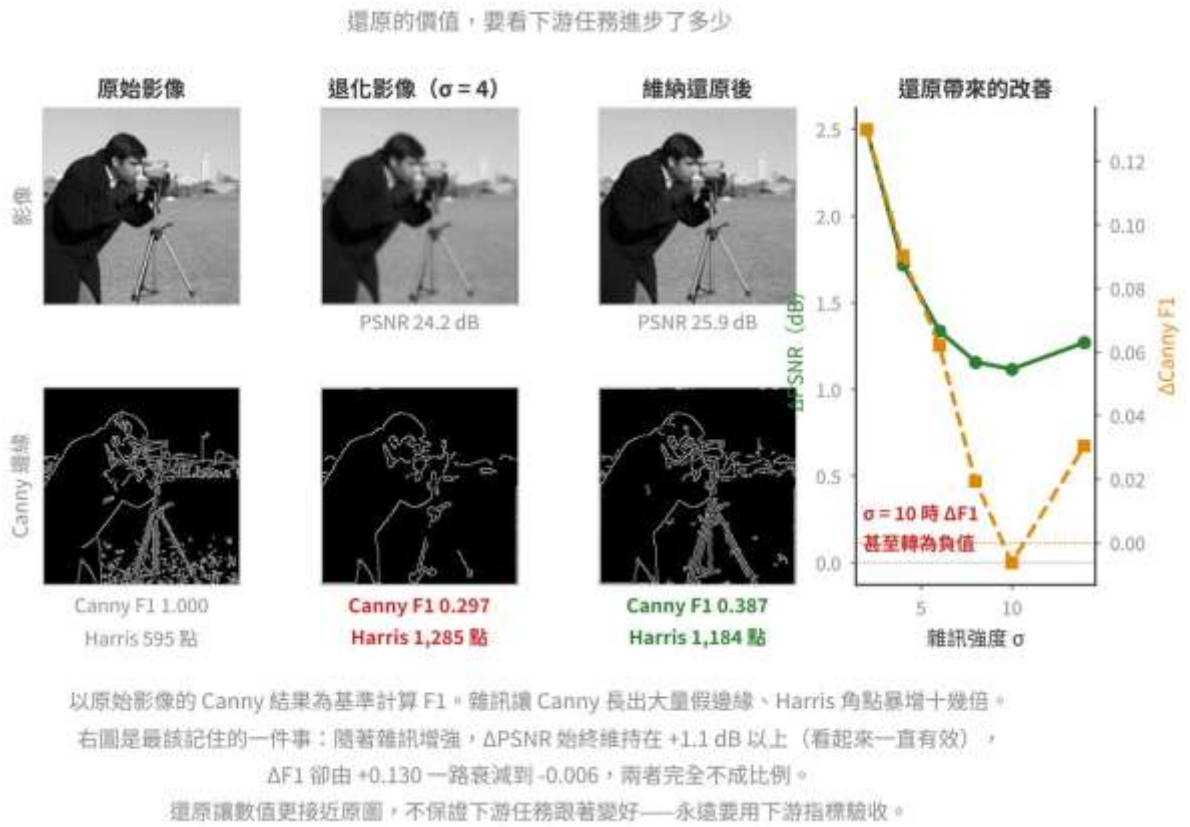


圖 7-21　還原的價值要看下游任務進步了多少

7.7.1 對邊緣偵測的影響(第 9 章)

Canny 邊緣偵測的第一步就是高斯平滑，原因正是「梯度運算會放大雜訊」——雜訊造成的局部劇烈變化，在微分後比真實邊緣還強。若影像雜訊嚴重，Canny 會產生大量細碎的假邊緣;若影像模糊，則邊緣的梯度峰值變得平坦，非極大值抑制難以精確定位，邊緣位置會產生偏移。良好的還原能同時改善這兩個問題，但過度平滑的還原(K 設太大)則會讓弱邊緣完全消失。這裡有一個實務準則：先想清楚下游要的是「完整的邊緣」還是「乾淨的邊緣」，再回頭調還原參數。

7.7.2 對特徵擷取的影響(第 13 章)

Harris 角點偵測依賴局部灰階在兩個方向上的變化量，模糊會直接降低角點響應值，使得許多真實角點落到門檻之下而被漏掉;雜訊則會在平坦區製造大量虛假的高響應點。SIFT 的情況更微妙：它本身就是在多尺度的高斯模糊金字塔上找極值，因此對輕微模糊有一定的耐受性，但嚴重模糊會使得特徵被偵測在錯誤的尺度上，描述子跟

著失真，匹配率大幅下降。ORB 使用的 FAST 角點偵測依賴中心像素與圓周上像素的亮度比較，對雜訊特別敏感——這也是程式 7-5 用 ORB 特徵點數作為還原成效指標的原因。

7.7.3 對深度學習模型的影響(第 15、16 章)

這是近年研究相當熱門的題目，結論可以歸納成三點。第一，分布偏移是主因：模型在乾淨影像上訓練，遇到模糊或含噪的測試影像時，準確率往往顯著下滑，因為輸入的統計特性已經偏離訓練分布。第二，還原不一定總是有幫助：如果還原演算法引入了訓練時未曾見過的偽影(如振鈴、過度銳化的邊緣光暈)，模型的表現可能不升反降——這是一個違反直覺但務必記住的陷阱。第三，資料增強常是更穩健的解法：在訓練時就加入模糊、雜訊、低光等劣化樣本，讓模型自己學會不變性，通常比在推論時做還原更可靠，運算成本也更低(不必為每一影格影像付出還原的代價)。

這三點合起來給出一個重要的工程判斷原則：若你能控制訓練流程，優先用資料增強;若你面對的是已經訓練好、無法重訓的模型，才考慮在前端加上還原模組，而且務必實測它是否真的提升了端到端的準確率，而不是想當然爾。

7.7.4 推導：為什麼雜訊對微分運算特別致命

7.7.1 節說「梯度運算會放大雜訊」，這是 Canny 要先做高斯平滑的理由。但放大多少?這件事可以算，而且答案會讓你對「先去雜訊再找邊緣」這條鐵則有完全不同的感受。

先看一階微分。離散的梯度運算是相鄰像素相減：$d = g(x+1) - g(x)$。設兩個像素的雜訊各自獨立、標準差皆為 σ，則相減之後雜訊的變異數相加，新的標準差為 $\sqrt{2}\cdot\sigma \approx 1.41\sigma$。乍看之下只放大了四成，似乎不嚴重。但關鍵不在雜訊本身變大多少，而在訊號與雜訊的比值：平坦區域的真實梯度接近零，微分後那裡只剩下 1.41σ 的純雜訊;而一條弱邊緣的梯度可能只有 3σ 到 5σ。也就是說，微分把「訊號很大、雜訊很

小」的局面，變成了「兩者同一個量級」。

二階微分更誇張。拉普拉斯核 [1, −2, 1] 的係數平方和為 1 + 4 + 1 = 6，因此輸出雜訊的標準差是 √6·σ ≈ 2.45σ。若用二維的五點拉普拉斯（中心 −4、四鄰各 1），係數平方和為 16 + 4 = 20，雜訊標準差變成 √20·σ ≈ 4.47σ。而真實的二階微分在平坦區與緩慢漸變區同樣接近零——於是第 5 章的拉普拉斯銳化在含雜訊影像上會產生滿屏顆粒，原因就在這個 4.47 倍。

高斯平滑的作用則恰好相反。對一個標準差為 σ 的雜訊做歸一化的加權平均，輸出雜訊的標準差會縮小為 σ 乘上「權重的平方和開根號」。以 sigma = 1 的高斯核為例，這個係數約為 0.4;sigma = 2 時約為 0.28。把它與微分的放大倍率相乘，就得到整條管線的淨效果——這正是 Canny 的 sigma 參數在調的東西：sigma 太小，微分的放大贏過平滑的壓制，假邊緣滿天飛;sigma 太大，弱邊緣的梯度被平滑掉而低於門檻，真邊緣漏掉。

最後把這個推導接回本章。7.4.2 節說「還原是受控的高通增強」，而高通增強與微分在頻率域是同一類操作——兩者都在放大高頻，也就都在放大雜訊。這解釋了為什麼本章的每一個方法（半徑受限、維納的 K、RL 的反覆次數）最後都在做同一件事：決定高頻要放大到哪裡為止。而圖 7-21 的實驗更進一步顯示，這個決定該由下游任務來做，不該由 PSNR 來做。

7.8 AI 影像修復：從還原到生成

近年來，深度學習徹底改寫了影像還原的版圖。以卷積神經網路或 Transformer 訓練的去噪、去模糊、超解析度模型(如 SRCNN、ESRGAN、Real-ESRGAN 及各式擴散模型)，在視覺品質上遠勝傳統方法，老照片修復、影片重製、手機的超解析變焦都已是成熟商品——叫獸開講中馬蓋先片頭的 4K 重生，正是這類技術的成果。

從 7.1.3 節正則化的觀點看，深度學習的突破其實很好理解：傳統方法的先驗知識

是人工設計的(「影像應該平滑」、「梯度應該稀疏」)，而深度模型是從數百萬張自然影像中「學」出先驗——它知道人臉長什麼樣、磚牆的紋理如何、樹葉的邊緣有多銳利。當這個先驗遠比人工設計的更貼近真實世界，還原品質自然大幅躍升。

影像補繪(Inpainting)則處理「缺一塊」的問題：移除照片中的路人、修補老照片的裂痕、去除浮水印。傳統方法(OpenCV 的 cv2.inpaint，提供 Telea 與 Navier-Stokes 兩種演算法)從破損區域邊緣向內推進，以周圍的紋理與梯度填補，適合小面積修補;生成式 AI 則能「想像」出整片合理的內容，面積再大也填得出來。

7.8.1 還原，還是腦補?

這裡必須劃下一條重要的界線。傳統反卷積是在「解方程式」：它只使用觀測資料與已知的物理模型，還原出的每一個細節，理論上都來自原始影像中殘存的資訊。生成式 AI 則是在「猜答案」：它根據訓練資料學到的先驗知識，產生一個「看起來最合理」的高解析度版本。兩者的差異在娛樂用途上無關緊要——馬蓋先的臉多了幾根不存在的頭髮不影響懷舊;但在司法鑑識、醫學診斷、科學量測上就性命攸關：AI 放大的車牌號碼可能是憑空編造的，AI 補全的病灶邊界可能誤導手術。

曾有廣為流傳的案例顯示，某些人臉超解析度模型在放大低解析度的人臉時，會產生與原本族裔特徵不符的結果——因為模型輸出的是「訓練資料中最常見的臉」，而不是「這個人的臉」。這提醒我們：生成模型的偏誤會忠實地反映訓練資料的偏誤，而使用者往往因為結果「看起來很清楚」就誤以為它很準確。清晰度不等於正確性，這是 AI 時代影像工程師必須內化的第一戒律。

身為工程師，你必須永遠能回答一個問題：這個像素的資訊，是「找回來的」還是「編出來的」?在交付任何經 AI 處理的影像時，誠實標註處理流程，是專業倫理的一部分。這也是叫獸開講思考題的核心。

7.9 本章與全書的觀念連結總表

相關章節	共用的觀念	本章的延伸或差異
第 3 章 數位影像基礎	MSE、PSNR、SSIM;RAW 高位元深度	作為還原成效的客觀驗收工具;RAW 為最佳還原輸入
第 5 章 空間域強化	平滑濾波、銳化、多影格平均	由主觀好看轉為依雜訊模型的統計最佳估計
第 6 章 頻率域處理	低通、高通、陷波濾波;卷積定理	模糊即低通、還原即受控高通;週期雜訊直接沿用陷波
第 9 章 影像分割	Canny 的高斯平滑、梯度運算	雜訊生假邊緣、模糊使邊緣定位偏移
第 10 章 二值影像處理	最大值/最小值濾波	即型態學的膨脹與侵蝕，一體多面
第 13 章 特徵擷取	Harris、SIFT、ORB 的局部灰階變化	模糊降低角點響應、雜訊製造假特徵
第 15、16 章 深度學習	資料分布、資料增強	還原作為前處理 vs. 劣化作為增強的取捨
第 17 章 邊緣部署	運算成本、即時性	FFT 與反覆式演算法的算力預算評估

表 7-2 第 7 章與全書其他章節的觀念連結

程式實作 Lab 7

Lab 7-1 雜訊診斷與濾波器配對

目標：建立「先診斷、再開藥」的工作習慣。(a)以程式 7-1 對同一張乾淨影像分別加上高斯、椒鹽與 Poisson 雜訊;(b)取平坦區域繪製直方圖，驗證三種雜訊的分布特徵，並用兩種方法估計高斯雜訊的標準差，比較估計值與真值;(c)對三張雜訊影像分別套用均值、高斯、中值、雙邊、非局部平均五種濾波，製作 3×5 的 PSNR 與 SSIM 比較表;(d)寫出結論：哪種雜訊該配哪種濾波器?為什麼中值濾波對高斯雜訊沒有特別優

勢，對椒鹽卻近乎完美?(e)挑戰題：自行實作自適應中值濾波，在椒鹽雜訊密度 5%、20%、40% 三種情況下與固定核中值濾波比較，繪製「雜訊密度對 PSNR」曲線。

Lab 7-2 維納濾波與醫學影像去噪

目標：重現 7.6.1 節的 MRI 去噪實驗流程。(a)取公開的 MRI 或 CT 切片影像(或以任意灰階醫學影像替代)，加入已知強度的高斯雜訊作為模擬退化影像;(b)以程式 7-4 的維納濾波處理，掃描參數 K 從 0.0001 到 0.5，繪製「K 對 PSNR」的曲線，找出最佳 K 值;(c)將最佳結果與非局部平均去噪比較 PSNR 與 SSIM，並附上局部放大截圖;(d)在原圖中人工加入一個直徑約十像素的低對比小圓點模擬病灶，重跑一次流程，觀察 PSNR 最佳的參數是否會把它抹掉——驗證 7.6.1 節提到的「全域指標盲點」;(e)討論題：PSNR 最高的結果，在醫師眼中是否就是最好的影像?為什麼醫學影像處理不能只看數字?

Lab 7-3 產線運動模糊還原與下游任務評估

目標：完成 7.6.3 節的完整工程流程，並量化還原對下游任務的效益。(a)取一張清晰的工件或零件照片，以程式 7-5 的 motion_psf 製造已知長度與角度的運動模糊，再加入少量高斯雜訊;(b)假裝你不知道真實參數，改由「模擬的產線參數」(自訂速度、曝光時間，並用第 4 章相機校正得到的像素尺寸)推算 PSF，以維納濾波還原;(c)比較三種影像——原始、模糊、還原——在下列三項下游任務的表現：Canny 邊緣像素數與視覺完整度、ORB 特徵點數與對原圖的匹配成功率、以及(若已學到第 16 章)YOLO 偵測的信心值;(d)刻意把推算的模糊長度設錯正負 30%，觀察還原品質如何惡化，體會「PSF 估計準確度」的重要性;(e)分組討論題：承 7.6.3 節的三個層次，若你是這條產線的工程師，實際會怎麼做?請列出成本、可行性與效益的評估。

Lab 7-4 反卷積三兄弟的參數掃描

目標：親手畫出三條「先升後降」的曲線，把正則化這個抽象概念變成你量過的數據。(a)取一張乾淨影像，以已知的高斯 PSF 製造模糊，再加入已知強度的高斯雜

訊;(b)實作半徑受限逆濾波，掃描 D0 由 5 到 125，畫出 D0 對 PSNR 的曲線並找出峰值;(c)實作維納濾波，掃描 K 由 1e-5 到 1，同時畫出 PSNR 與 SSIM 兩條曲線——確認兩者的最佳點不在同一個地方，並解釋為什麼;(d)實作 Richardson-Lucy，掃描反覆次數 1 到 200，畫出曲線並找出最佳次數;(e)把三條曲線放在同一張圖上（橫軸各自正規化到 0 至 1），觀察它們的形狀有多相似;(f)改變雜訊強度重做一次，記錄「雜訊愈大，最佳參數往哪個方向移動」，並解釋這個趨勢;(g)寫下結論：如果只能用一句話告訴學弟妹怎麼調這些參數，你會說什麼?

Lab 7-5 退化函數估錯的代價

目標：理解非盲反卷積「非盲」兩個字的份量。(a)以已知長度 L 與角度 θ 的運動模糊 PSF 製造退化影像;(b)用正確的 L 與 θ 做維納還原，記錄 PSNR 作為基準;(c)固定 θ 正確，把 L 由正確值的一半掃到兩倍，畫出「L 的誤差對 PSNR」的曲線;(d)固定 L 正確，把 θ 由正確值掃到正負 30 度，畫出對應的曲線——比較兩條曲線的陡峭程度，判斷哪一個參數估錯的代價比較大;(e)觀察估錯時的還原影像有什麼特徵（提示：留意有沒有方向一致的條紋或鬼影），並說明你能不能從還原結果本身看出參數估錯了;(f)進階：試著只從模糊影像的頻譜反推 L（提示：利用習題 21 的通式，量測頻譜暗帶的間距），與真值比較;(g)寫下你的結論：在產線上，你會把力氣花在「取得準確的 PSF 參數」還是「改進還原演算法」?

Lab 7-6 還原到底有沒有幫到下游

目標：重現圖 7-21 的實驗，並建立「用下游指標驗收前處理」的工作習慣。(a)取一張乾淨影像，計算它的 Canny 邊緣作為基準答案，並記錄 Harris 角點數;(b)製造一組退化影像，雜訊強度由小到大共六級;(c)對每一級都做維納還原（每一級都要重新掃描最佳 K，不可共用同一個 K——想想為什麼）;(d)對退化影像與還原影像分別計算 Canny 的 F1 分數（以步驟 a 的結果為基準）與 Harris 角點數;(e)畫出「雜訊強度對 ΔPSNR」與「雜訊強度對 ΔF1」兩條曲線，找出 ΔF1 開始失效的位置;(f)把下游任務

換成第 13 章的 ORB 特徵匹配（以還原影像對乾淨影像做匹配，記錄成功匹配的點數），重做一次;(g)最後寫一段兩百字的建議書，對象是產線主管：在什麼條件下值得加上還原模組、在什麼條件下不值得，以及你的判斷依據是什麼。這一題的目的不是演算法，而是工程判斷。

本章重點整理

- 影像強化追求主觀好看且不需先驗知識，影像還原則基於劣化模型 g = h * f + n 客觀反推原始影像;頻率域形式為 G = H·F + N。
- 還原是逆問題，常為病態問題(解不連續依賴資料);解法的共通策略是正則化——以先驗知識約束解的形態。
- 模糊即低通濾波，還原即受控的高通增強;與第 5 章 High-Boost、第 6 章高頻強調同屬提升高頻，差別在於增益是否依退化函數量身訂做。提升高頻必然放大雜訊，是本領域的根本矛盾。
- 雜訊型態：高斯(鐘形)、椒鹽(兩端尖峰)、均勻、Poisson(變異數等於平均值)、乘性/斑點、週期性;以平坦區直方圖辨識，以樣本標準差或拉普拉斯殘差估計參數。
- 去雜訊工具：均值家族(算術、幾何、諧波、反諧波)、排序統計家族(中值、最大、最小、alpha 修剪)、自適應濾波(局部降噪、自適應中值)、非局部平均(利用自相似性)。
- 逆濾波 $\hat{F} = G/H$ 因 H 趨近零時放大雜訊而失敗;維納濾波在分母加入常數 K(雜訊訊號比)，約束最小平方則加入 $\gamma|P|^2$(拉普拉斯正則化)，兩者皆為病態問題的正則化解。
- Richardson-Lucy 為反覆式非線性反卷積，假設 Poisson 雜訊並保證非負;反覆次數過多會放大雜訊，哈伯影像經驗上約 50 次為佳，超過 100 次影像開始崩壞。

- 還原品質直接影響下游：雜訊使 Canny 產生假邊緣、模糊使邊緣定位偏移與特徵點漏檢;對深度學習模型，資料增強常比推論時還原更穩健，且還原引入的偽影可能反而降低準確率。
- AI 修復與生成式補繪效果驚人，但屬於「依學到的先驗猜測」而非「依觀測資料還原」;清晰度不等於正確性，司法、醫療、科學量測場合必須嚴格區分並誠實標註處理流程。

習題

1. 影像強化與影像還原的目標與驗收方式有何不同?各舉一例，並說明兩者對先驗知識的需求差異。

   提示與參考解答：強化追求主觀視覺效果(如直方圖等化讓夜景變亮)，不需知道劣化原因;還原追求逼近原始影像(如維納濾波去除已知的運動模糊)，需要退化函數與雜訊統計量。驗收上強化多靠人眼或下游任務表現，還原則可用 PSNR、SSIM 客觀量化。可參考表 7-1。

2. 寫出空間域與頻率域的影像劣化模型，說明各符號的物理意義，並列出這個模型的三個隱含假設。

   提示與參考解答：空間域 $g = h * f + n$;頻率域 $G = H \cdot F + N$。f 為原始影像、h 為退化函數(PSF)、n 為加性雜訊、g 為觀測影像。三個假設為：退化線性且位置不變、雜訊為加性且與訊號獨立、雜訊零均值。實務上鏡頭邊緣模糊較嚴重違反第一項，Poisson 雜訊違反第二項，感測器暗電流違反第三項。

3. 什麼是適定問題的三個條件?影像還原最常違反哪一個?這在工程上造成什麼後果?

   提示與參考解答：解存在、解唯一、解對資料連續依賴。還原最常違反第三條：退化函數在高頻趨近零，還原時要乘上極大倍數，觀測資料中的微小雜訊被同等放大，造成輸出佈滿雪花。應對策略為正則化。

4. 什麼是盲反卷積?為什麼它比非盲反卷積困難?請從解的唯一性說明。

   提示與參考解答：盲反卷積指退化函數 h 未知，需同時估計 h 與 f。困難在於解不唯

> *一：「清楚影像配大模糊核」與「稍模糊影像配小模糊核」可產生相同的觀測結果，必須靠額外先驗(如自然影像梯度分布的稀疏性)才能收斂到合理解。*

5. 如何從直方圖判斷一張影像受到的是高斯雜訊、椒鹽雜訊還是均勻雜訊?請描述完整的診斷步驟。

> *提示與參考解答：步驟：先選取一塊本該平坦均勻的區域(純色背景、天空、工件平整面)，取出該區像素繪製直方圖。鐘形對稱分布為高斯;中央分布正常但兩端出現孤立尖峰為椒鹽;矩形平坦分布為均勻。若比較亮區與暗區，變異數隨平均亮度成正比增加則為 Poisson。*

6. Poisson 雜訊與高斯雜訊最大的差別是什麼?這個特性對低光攝影有何影響?與第 5 章的夜景模式有何關聯?

> *提示與參考解答：Poisson 雜訊的變異數等於平均值，強度隨訊號變化(訊號相關)，不符合「加性且獨立」的假設。低光時訊號弱，雖然雜訊絕對值小，但訊噪比更差。第 5 章夜景模式以多影格平均提升訊噪比，平均 N 張後雜訊標準差降為原來的 N 平方根分之一，正是對抗此類隨機雜訊的標準做法。*

7. 自適應中值濾波相較固定核中值濾波的兩項改良是什麼?在什麼情況下差距最明顯?

> *提示與參考解答：第一，若中位數本身是雜訊(0 或 255)則擴大視窗重試，解決高密度雜訊下中位數失效的問題;第二，若中心像素本身不是雜訊則原值保留，完全保護乾淨像素與細節。雜訊密度愈高(20% 以上)差距愈明顯，因為固定核在此時中位數常常也是雜訊。*

8. 非局部平均利用了自然影像的什麼特性?請舉出兩種特別適合它發揮的影像內容，以及一種不適合的情況。

> *提示與參考解答：利用自相似性：相似的紋理區塊在影像各處反覆出現。適合磚牆、織品紋理、規律的建築立面、草地等重複性高的內容。不適合的情況：內容高度獨特無重複結構的影像，或需要即時處理的場合(運算量大)。*

9. 推導 F̂ = G/H 在含雜訊時的展開式，說明逆濾波失敗的數學原因，並說明「半徑受限逆濾波」如何補救及其副作用。

> *提示與參考解答：將 G = H·F + N 代入得 F̂ = F + N/H。H 在高頻趨近零時 N/H 發*

散。半徑受限逆濾波只在頻譜中心半徑 D0 內執行除法，相當於逆濾波加低通，但 D0 靠試誤決定，且頻域的硬截斷會造成第 6 章所述的振鈴效應。

10. 比較維納濾波與約束最小平方濾波的轉移函數，說明兩者分母的差異代表什麼樣的先驗知識。

    提示與參考解答：維納分母為 $|H|^2 + K$(常數，代表雜訊訊號功率比);約束最小平方分母為 $|H|^2 + \gamma|P|^2$,P 為拉普拉斯的頻率響應，高頻響應大，等於「頻率愈高抑制愈強」，把「還原結果不應劇烈震盪」的平滑先驗寫進數學。兩者都是正則化，差別在正則化項是否隨頻率變化。

11. 維納濾波公式中的 K 值調大或調小，分別對還原結果造成什麼影響?應如何決定起始值?

    提示與參考解答：K 太小接近逆濾波，還原銳利但雜訊被放大、影像顆粒粗糙;K 太大則過度抑制高頻，還原不足、影像仍模糊。起始值可由 7.2.2 節估得的雜訊標準差推估雜訊功率，再對照影像本身的功率;實務上以掃描 K 值並觀察 PSNR 曲線找極大值(見 Lab 7-2)。

12. Richardson-Lucy 演算法有哪兩項相對於線性方法的優點?為什麼反覆次數不是愈多愈好?

    提示與參考解答：優點：假設 Poisson 雜訊模型，適合光子計數影像(天文、醫學);每次反覆保證結果非負，符合光強度物理意義(維納濾波輸出可能為負)。反覆次數過多會放大雜訊，產生不自然的顆粒與塊狀結構;哈伯經驗約 50 次為佳，超過 100 次形狀開始崩壞。

13. 為什麼說「模糊是低通濾波、還原是高通增強」?本章的反卷積與第 5 章的 Unsharp Masking、第 6 章的高頻強調濾波，三者有何異同?

    提示與參考解答：模糊把高頻振幅壓低即低通;還原要把高頻補回即高通增強。三者相同處：都提升高頻、都會放大雜訊。相異處：Unsharp Masking 的增益由使用者主觀設定的 k 決定;高頻強調由濾波器形狀決定;反卷積的增益由退化函數 H 精確決定，是量身訂做的補償。可參考 7.4.2 節與表 7-2。

14. 影像還原對第 9 章的 Canny 邊緣偵測與第 13 章的 ORB 特徵擷取分別有何影響?請說明雜訊與模糊各自造成的問題。

提示與參考解答：雜訊：梯度運算放大雜訊，Canny 產生大量細碎假邊緣;ORB 的 FAST 角點依賴中心與圓周像素亮度比較，雜訊會製造大量虛假特徵點。模糊：梯度峰值變平坦，非極大值抑制難以定位，邊緣位置偏移;角點響應值下降，真實角點落到門檻之下被漏檢，匹配率下降。

15. 對一個已訓練好的 YOLO 模型，在推論前加上影像還原模組，是否一定會提升準確率?請說明三種可能情況。

提示與參考解答：不一定。情況一：還原減輕分布偏移，準確率提升。情況二：還原引入訓練時未見過的偽影(振鈴、過度銳化光暈)，反而使輸入更偏離訓練分布，準確率下降。情況三：還原的運算成本使系統無法即時處理，實際部署效益為負。較穩健的做法是在訓練階段以劣化樣本做資料增強，見 7.7.3 節。

16. 承 7.6.3 節，若產線工件的運動模糊長度為 20 像素，身為工程師你會依什麼順序考慮解法?請說明每個層次的具體做法。

提示與參考解答：順序為光學機構層、演算法層、學習層。光學機構層(治本)：縮短曝光時間並加強打光、改用全域快門相機、加裝頻閃光源凍結畫面。演算法層：由編碼器速度、曝光時間與像素尺寸推算 PSF，以維納濾波做非盲反卷積。學習層：將運動模糊作為資料增強加入訓練集，讓模型直接適應。

17. (思辨題)監視器畫面經 AI 超解析度處理後在法庭上作為證據，你贊成嗎?請以本章「還原 vs. 腦補」的區分，提出至少三點論證。

提示與參考解答：本題無標準答案，評分重點在論證品質。可能論點包括：傳統反卷積只使用觀測資料與物理模型，細節來自殘存資訊;生成式模型輸出「最合理的猜測」，可能產生原本不存在的細節;模型的訓練資料偏誤會反映在輸出上(如人臉超解析度改變族裔特徵);清晰度不等於正確性，而清晰的影像對陪審團有過度的說服力;可主張區分「可採信的還原」與「僅供參考的重建」，並要求揭露完整處理流程與演算法。

18. 承圖 7-3 與圖 7-15：逆濾波在「完全沒有外加雜訊」的情況下，為什麼還是會失敗?本書實測的結果是：一個雜訊都沒加，還原結果的數值已經跑到 −1,709 到 1,960。請解釋這個現象的來源，並說明它對「病態問題」這個概念的意義。

提示與參考解答：來源是浮點運算本身的捨入誤差。雙精度浮點數的相對精度約為 1e-16，也就是說每一次運算都會引入這個量級的誤差;而逆濾波要做的是除以 H，在高頻處 |H| 可能小到 1e-10 以下，於是 1e-16 的誤差被放大成 1e-6 的相對誤差，乘上影像頻譜本身的大數值後，絕對誤差就達到上千。它對「病態」的意義非常直接：7.1.3 節說病態的定義是「解不連續依賴資料」，而這個實驗顯示所謂的「資料擾動」不必來自感測器——光是計算機表示數字的有限精度就已經足夠。換句話說，這個問題在數學上就無解，不是換一台更好的電腦、用更高的精度就能解決的。唯一的出路是正則化：放棄「完全忠實於資料」，改成「在忠實與穩定之間取平衡」——這正是維納濾波在做的事。

19. 承圖 7-8 的濾波器配對矩陣：請說明為什麼(a) 非局部平均在椒鹽雜訊上幾乎毫無作用;(b) 中值濾波對高斯雜訊的效果不如非局部平均;(c) 陷波濾波對隨機雜訊是負的，對週期性雜訊卻一舉領先。

    提示與參考解答：(a) 非局部平均的運作方式是「找出與目標區塊相似的區塊再加權平均」，而相似度是以區塊間的均方差計算。椒鹽雜訊會在區塊中造成 0 與 255 的極端值，使均方差被少數幾個像素主導，導致相似的區塊被判定為不相似、不相似的被判定為相似——權重算錯了，平均自然沒有意義。它的假設前提是加性高斯雜訊。(b) 中值濾波取的是排序後的中間值，對高斯雜訊而言這只用到了一個樣本的資訊，而非局部平均用的是數百個相似區塊的加權平均，依大數法則雜訊被壓低得多;此外中值濾波會抹掉細線與角點，額外損失細節。(c) 陷波濾波的原理是「在頻譜上挖掉少數孤立亮點」。隨機雜訊的能量均勻散布在整個頻譜上，沒有孤立亮點可挖，挖掉的反而是影像本身的頻率成分，所以是負的;週期性雜訊則恰好相反，它的全部能量集中在少數幾個點上，挖掉就乾淨了，而影像的其餘頻率完好無損。這整張表的訊息是：每一種工具都只在一種情況下最強，所以 7.2.2 節的診斷不是可有可無的前置作業，而是決定成敗的那一步。

20. 承圖 7-10：自適應局部降噪濾波的核心公式是輸出 = g − (σn² / σL²) × (g − μL)，其中 σn² 是雜訊變異數、σL² 與 μL 是局部變異數與局部平均。請說明這個比值在平坦區與邊緣處各是多少，以及為什麼公式要寫成這個形式。若 σn² 估得太大會怎樣?

    提示與參考解答：在平坦區，原始訊號幾乎是常數，局部變異數 σL² 主要來自雜訊，因此 σL² ≈ σn²，比值接近 1，輸出 ≈ g − (g − μL) = μL，也就是完全交給局部

平均，全力平滑。在邊緣或紋理處，σ_L^2 遠大於 σ_n^2，比值趨近 0，輸出 $\approx g$，也就是幾乎原樣保留。公式寫成這個形式的巧妙之處在於：它不是用一個門檻去硬性切換兩種行為，而是讓平滑的強度隨著「這裡有多少真實訊號」連續變化，因此不會在門檻附近產生突兀的接縫。若 σ_n^2 估得太大（例如在邊緣上量雜訊，如圖 7-6 的錯誤示範），比值會在所有地方都接近 1，濾波器退化成普通的均值濾波，邊緣也被抹掉——這就是為什麼 7.2.2 節的參數估計必須做對。實作上通常還會加一道保險：把比值夾在 0 到 1 之間，避免 σ_L^2 小於 σ_n^2 時出現大於 1 的比值而把影像推過頭。

21. 承圖 7-12 與 7-13：運動模糊的頻率響應在哪些頻率上等於零?請推導通式。本書實測模糊長度 L = 9、影像邊長 N = 256 時，殘差的頻譜尖峰落在 28.4、56.9、85.3，請驗證這組數字。

    提示與參考解答：長度 L 的均勻運動模糊，其 PSF 在空間域是一個寬度 L 的矩形。矩形的傅立葉轉換是 sinc 函數，而 $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$ 在 x 為非零整數時等於零。換算到離散頻率座標：$H(u)$ 在 $u \cdot L/N$ 為非零整數時為零，也就是 $u = N \cdot k / L$（$k = 1, 2, 3, \ldots$）。驗證：$N = 256$、$L = 9$ 時，$u = 256/9 = 28.4$、$512/9 = 56.9$、$768/9 = 85.3$——與圖 7-13 量到的殘差尖峰位置完全吻合。這個通式有兩個實務用途。其一是診斷：若你不知道模糊長度，可以從模糊影像的頻譜上找出暗帶的間距反推 L，這是盲反卷積估計 PSF 的入門做法。其二是預警：L 愈大，零點愈密集、落在愈低的頻率上，代表愈多的低頻資訊被徹底抹除;L = 9 時第一個零點在 28，L = 30 時就掉到 8.5，影像能量最集中的區域都被挖空了。

22. 承圖 7-13：本書實測發現，在退化程度相當的條件下，有零點的運動模糊還原後的 PSNR（27.0 dB）反而高於無零點的高斯失焦（25.4 dB）。這與「有零點就救不回來」的直覺相反，請解釋為什麼，並說明兩者真正的差別在哪裡。

    提示與參考解答：關鍵在於「零點」與「整體衰減」是兩件事。高斯的 $|H|$ 沒有零點，但它單調遞減且衰減得非常快，到了中頻就已經壓到極小——換句話說它把「大部分頻率」都衰減得很嚴重，每一個頻率都只剩一點點資訊可以放大。運動模糊的 sinc 則是在零點之間仍然保持相當的響應，只有零點那幾條窄帶被完全抹除;其餘頻帶的資訊保存得相對完整，還原起來反而容易。所以 PSNR 比較高是合理的。真正的差別在殘差的形狀：高斯的還原誤差均勻散布在所有頻率，表現為整體略糊;運動模糊的誤差集中在 N/L 的倍數上（圖 7-13 右側的尖峰），表現為有固定方向與週期的結構性偽影。這個差別在工程上很重要：均勻的糊通常只是不好看，但有

結構的偽影可能被下游的邊緣偵測或特徵擷取誤判為真實結構。這題的教訓是：不要把課本上的定性說法當成定量結論，實際量一次往往會發現事情比口訣複雜。

23. 承圖 7-16、7-17、7-19：半徑受限逆濾波的 D0、維納濾波的 K、Richardson-Lucy 的反覆次數，三條曲線都是「先升後降」。請說明這三個參數在數學上扮演的是同一個什麼角色，並各舉一個其他領域的對應例子。

    提示與參考解答：三者扮演的都是正則化強度（regularization strength），也就是在「忠實於觀測資料」與「解要夠平滑、夠穩定」之間的權衡旋鈕。往一邊走是完全相信資料，結果被雜訊主導（過度擬合）;往另一邊走是過度約束，結果過於平滑而丟失真實細節（欠擬合）。最佳點永遠在中間，所以曲線必然先升後降。三者的對應關係：D0 以「只相信低頻」的方式限制解的複雜度;K 以「在分母加一個常數」的方式限制增益的上限;RL 的反覆次數則是以「還沒來得及擬合雜訊就停手」的方式達到同樣效果。其他領域的例子：機器學習的 L2 正則化係數（第 14 章）、神經網路訓練的提早停止（early stopping，第 15 章）、以及第 5 章銳化強度 k 的選擇。認出這個共通結構很有價值：一旦你知道某個參數扮演的是正則化角色，你就知道該怎麼調它——掃一遍、畫出曲線、取峰值，而不是憑感覺微調。

24. 承圖 7-21：本書實測發現，隨著雜訊增強，維納還原的 ΔPSNR 始終維持在 +1 dB 以上，ΔCanny F1 卻由 +0.130 一路衰減到 −0.006。請解釋為什麼兩者會脫鉤，並說明這對「還原要不要做」這個決策有什麼啟示。

    提示與參考解答：脫鉤的原因在於兩個指標衡量的是不同的東西。PSNR 衡量的是逐像素的數值誤差總量，而維納濾波的目標函數正是最小均方誤差——它在設計上就是為了讓這個數字變好，所以當然一直有效。Canny 的 F1 衡量的則是「邊緣位置是否正確」，它對兩件事特別敏感：殘留的雜訊顆粒會產生假邊緣，而過度平滑會讓弱邊緣的梯度落到門檻之下而漏掉。雜訊強時，維納濾波為了壓制雜訊必須用較大的 K，結果是數值誤差變小（PSNR 進步）但邊緣也被一起抹平（F1 退步），兩者於是反向。對決策的啟示有三點。其一，驗收指標必須與實際任務一致：若下游是邊緣偵測，就該用 F1 調參數，而不是用 PSNR。其二，還原不是免費的——它要花運算時間（第 17 章的預算問題），若換來的下游改善接近零甚至為負，就不該做。其三，更根本的做法往往是回到源頭：與其事後還原，不如改善打光、縮短曝光、換感測器（7.6.3 節把這一點列為最優先）。

25. (綜合設計題)某醫院要導入一套 MRI 影像後處理流程，目的是縮短掃描時間（代價是訊噪比下降），再以演算法補回品質。請設計這套流程的驗收方案：(a) 你會用哪些指標?為什麼不能只用 PSNR?(b) 如何設計實驗才能發現「演算法把微小病灶一起濾掉」這種錯誤?(c) 這套系統在臨床上線前，你認為最大的風險是什麼?(d) 若改用深度學習的去噪模型，你的驗收方案要增加哪些項目?

    提示與參考解答：本題無標準答案，評分重點在風險意識的完整性。(a) 至少三類指標：全域指標（PSNR、SSIM）作為初篩;局部最壞值指標（例如 SSIM 差異圖的最小值、誤差最大的前 1% 區域的平均誤差），因為 7.6.1 節的核心風險正是「平均很好但局部災難」;以及任務指標——由放射科醫師對同一批案例做盲測判讀，比較診斷結論的一致性。不能只用 PSNR 的理由見第 3.5 節與圖 3-16：抹掉一小塊的 PSNR 可能比整體加亮還好看。(b) 關鍵是要有「已知答案」的測試案例。做法是在乾淨影像上人工植入已知大小、已知對比度的微小病灶模擬物，再加入雜訊、跑完整條流程，檢查病灶是否仍可偵測、其對比度衰減了多少;並把病灶的尺寸與對比度逐步調小，找出「這套流程的偵測下限」。這個下限必須明確寫進規格與操作手冊。(c) 最大的風險通常不是演算法本身，而是信任的轉移：一旦醫師習慣了「處理後的影像看起來很乾淨」，就可能降低對原始影像的檢視，而演算法的失效模式恰好是「看起來很乾淨但東西不見了」——這種錯誤沒有警訊。對策是流程上強制保留並可隨時調閱原始影像，以及在處理後的影像上明確標註處理方式與版本。(d) 深度學習模型要額外驗收三件事：分布外表現（訓練資料沒見過的機型、序列、病灶型態）;幻覺風險——模型可能生成訓練資料中常見但本例不存在的結構（7.8.1 節的核心問題），這在醫學影像上是最危險的失效模式;以及可追溯性——模型版本、訓練資料來源、更新紀錄都必須納入醫材品質系統。一句話總結：傳統方法的錯誤通常「看得出來」（偽影、過度平滑），生成式方法的錯誤則「看不出來」，驗收的重點因此完全不同。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 5(影像還原與重建)。
- ESA/Hubble, History: The Spherical Aberration Problem.(本章叫獸開講之史料來源)

- Hanisch, R. J., & White, R. L., The Restoration of HST Images and Spectra II, STScI, 1994.
- Richardson, W. H., Bayesian-Based Iterative Method of Image Restoration, JOSA, 1972;Lucy, L. B., An Iterative Technique for the Rectification of Observed Distributions, AJ, 1974.
- Buades, A., Coll, B., & Morel, J.-M., A Non-Local Algorithm for Image Denoising, CVPR, 2005.(非局部平均原始論文)
- Hunt, B. R., The Application of Constrained Least Squares Estimation to Image Restoration by Digital Computer, IEEE Trans. Computers, 1973.
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 10(計算攝影與影像還原)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

第 8 章 色彩影像處理

本章學習目標

- 說明三原色理論與人眼感色機制，理解 RGB 加色與 CMY 減色系統的差異。
- 描述 HSL 與 HSV 色彩模型的圓柱、圓錐與雙圓錐幾何詮釋，並說明兩者的差異。
- 比較 YCbCr 與 CIE Lab 色彩空間的設計目的與應用場合。
- 正確使用 OpenCV 進行色彩空間轉換，避開 BGR 與 HSV 數值範圍的陷阱。
- 運用虛擬色彩將灰階或非可見光資料轉為易於判讀的彩色影像，並理解色階選擇的專業考量。

- 以灰階世界法等演算法實作白平衡，並完成基於 HSV 的顏色分割與物件追蹤。
- 理解色彩恆常性對機器視覺的啟示，並據以設計不受光源干擾的檢測系統。

叫獸開講|藍黑還是白金?一件洋裝如何撕裂全世界

2015 年 2 月，一張洋裝照片被貼上社群網站，附帶一句求助：「這件洋裝到底是白金色還是藍黑色?」接下來的 48 小時，這個問題席捲全球，朋友吵架、情侶決裂、名人與政治人物紛紛表態，連視覺科學家都被捲進戰局。最詭異的是：雙方都不是在說謊，也不是視力有問題——他們看著同一堆像素，大腦卻給出截然不同的答案。而這件洋裝的真實顏色是藍色與黑色。

科學界的反應非常迅速。同年，麻省理工學院的 Rosa Lafer-Sousa、Katherine Hermann 與 Bevil Conway 在《Current Biology》發表了研究：他們對 1401 人做了自由回答(而非二選一)的調查，確認多數人確實只看到白金或藍黑兩種答案，少數人看到藍棕;而且回報白金的比例在年長者與女性中較高。這證實了「分歧是真實的」，不是社群媒體提問方式造成的假象。

關鍵詞叫做色彩恆常性(Color Constancy)：同一顆蘋果，在正午陽光下、在黃昏夕照下、在螢光燈下，反射到你眼睛的光譜其實差異極大，但你始終覺得它是紅色的。這是因為大腦會自動推測「現在的光源是什麼顏色」，再把光源的影響扣掉，還原出物體本身的顏色——這就是人腦內建的自動白平衡。

問題在於，這張洋裝的照片曝光過度、背景線索模糊，光源資訊變得極度曖昧，大腦只能仰賴各自的「先驗假設」來猜。研究團隊做了一個漂亮的驗證：把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就會趨於一致，並且與被暗示的光源方向吻合。換句話說——看到白金的人，大腦假設場景被偏藍的日光或陰影照亮，於是扣掉藍色成分;看到藍黑的人，大腦假設是偏黃的室內人造光，於是扣掉黃色成分。

那為什麼不同人的假設不同?一個廣受討論的假說是生活作息：紐約大學的

Pascal Wallisch 在 2017 年的《Journal of Vision》發表了一項超過一萬三千人的大規模研究，發現自認是「雲雀」(早起、白天活動)的人較常看到白金，而「夜貓子」(習慣人造光下活動)的人較常看到藍黑，且呈現劑量反應的關係。不過也要誠實說明：Lafer-Sousa 團隊自己的資料對這個假說只提供了微弱的支持，學界至今仍在討論確切的機制。這也是科學該有的樣子——一個引人入勝的假說，需要更多證據來檢驗。

這個全民事件對影像處理學生有三重啟示。第一，你「看到」的顏色不是物理量，而是大腦處理過的結果——同一組 RGB 數值，可以被解讀成完全不同的顏色。第二，電腦視覺系統必須做同一件事：本章 8.5 節的白平衡演算法，就是在教機器扣掉光源的影響，而 Night Sight 的學習式白平衡(第 5 章)正是這條路線的現代版本。第三，也是最重要的：在工業檢測與機器人視覺中，如果光源不穩定，顏色判讀就會像這件洋裝一樣不可靠。這正是第 17 章 AOI 系統要不惜血本控制打光的原因。

思考題：如果要設計一台「絕不會被光源騙倒」的顏色檢測系統，你會採取什麼策略?(提示一：控制光源;提示二：在畫面中放一張已知顏色的參考色卡;提示三：改用對光源較不敏感的特徵)

8.1 色彩的基礎：從人眼到 RGB

8.1.1 三原色理論

人眼視網膜有三種錐狀細胞，分別對長波(紅)、中波(綠)、短波(藍)敏感。任何顏色的光進入眼睛，都被化約成三個數值的組合送進大腦——這就是三原色理論的生理基礎，也是為什麼數位影像用三個通道就能表現幾乎所有顏色。

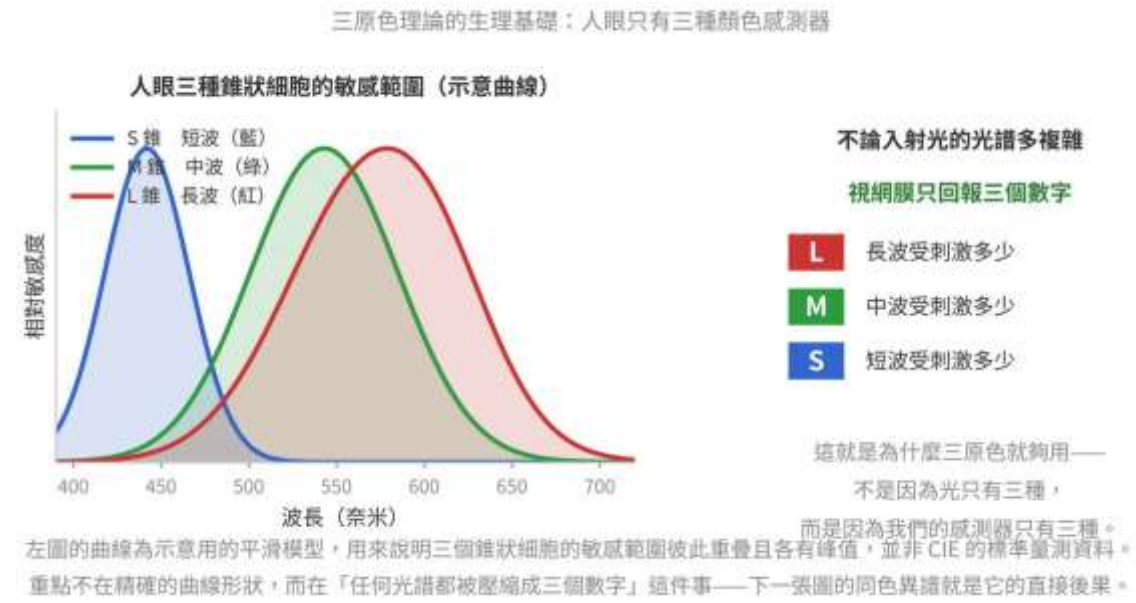


圖 8-1　三種錐狀細胞的敏感範圍：任何光譜都被壓成三個數字

值得深思的是同色異譜(Metamerism)現象：兩束光譜完全不同的光，只要刺激三種錐狀細胞的比例相同，人眼就會覺得它們是同一個顏色。這既是彩色顯示技術得以成立的原因(螢幕只用紅綠藍三種光源，就能模擬幾乎所有顏色)，也是色彩管理困難的根源——在某個光源下看起來相同的兩塊布料，換到另一個光源下可能顏色明顯不同，這在紡織與印刷業是實際的品管難題。

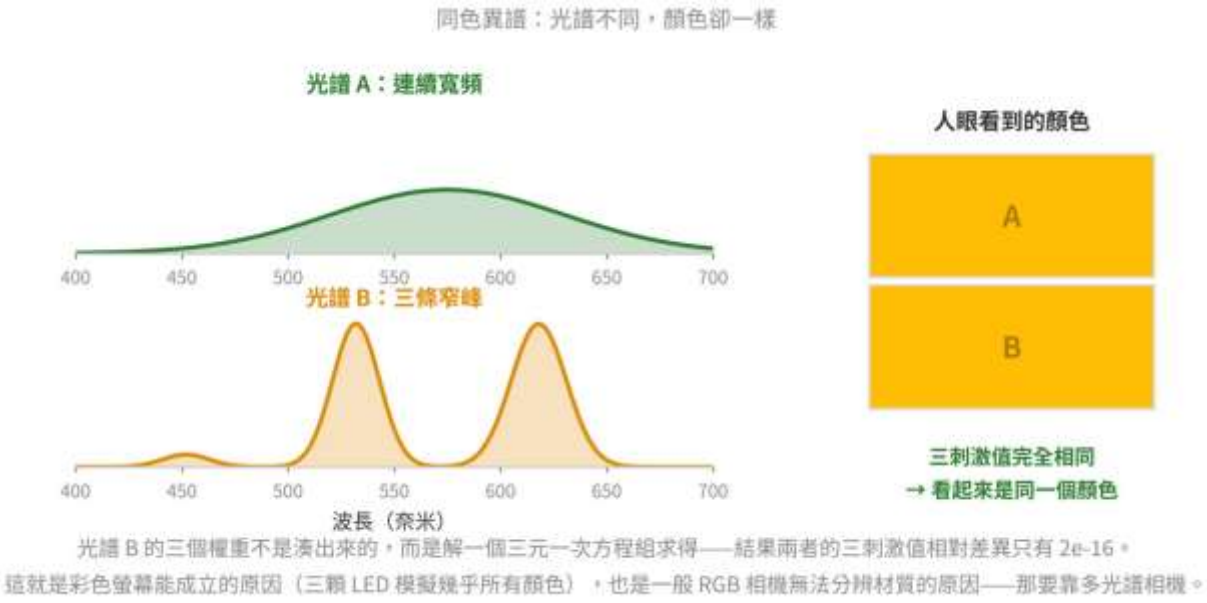


圖 8-2　同色異譜：光譜完全不同，三刺激值卻一模一樣

這也對機器視覺有一個重要啟示：一般 RGB 相機與人眼一樣，只記錄三個數值，

因此無法分辨同色異譜的物體。若要真正區分材質(例如判斷兩塊看似同色的塑膠是否為同一種材料)，就需要記錄完整光譜的多光譜或高光譜相機——這在農產品分級、材料檢測、遙測中相當常見。

8.1.2 加色與減色系統

RGB 是加色系統：光源相加，紅加綠得黃，三色全開得白，適用於自發光的裝置——螢幕、投影機、LED。CMY(青、洋紅、黃)是減色系統：顏料吸收特定波長，疊愈多愈暗，三色相加理論上得黑(實務上因顏料不純而加入黑色 K，成為印刷用的 CMYK)。



圖 8-3　加色系統與減色系統：疊愈多愈亮，還是疊愈多愈暗

同一張照片在螢幕上鮮豔、印出來卻黯淡，正是因為兩個系統的色域(可表現的顏色範圍)不同。這也解釋了為什麼專業的印刷流程需要色彩管理與打樣——這正是 8.3.2 節 CIE Lab 存在的理由。

8.1.3 RGB 的侷限

RGB 對機器友善(直接對應感光元件與螢幕)，卻對人類不直觀。兩個具體的痛點：

第一，調整困難。請問「把這個顏色變深一點但色相不變」該怎麼改 RGB 三個數

值?答案是三個都要按比例調整，很不方便;若只改其中一個，色相就跑掉了。

第二，也是對機器視覺更致命的：同一個物體在陰影中與陽光下，RGB 三個值全部大幅改變，難以判斷「還是不是同一個顏色」。舉例來說，一顆紅色蘋果在陽光下可能是 (220, 40, 40)，在陰影中變成 (90, 15, 15)——數值差了兩倍以上，但它顯然是同一顆蘋果。這使得「用 RGB 數值範圍來偵測特定顏色」在光線變化的環境中極不可靠。

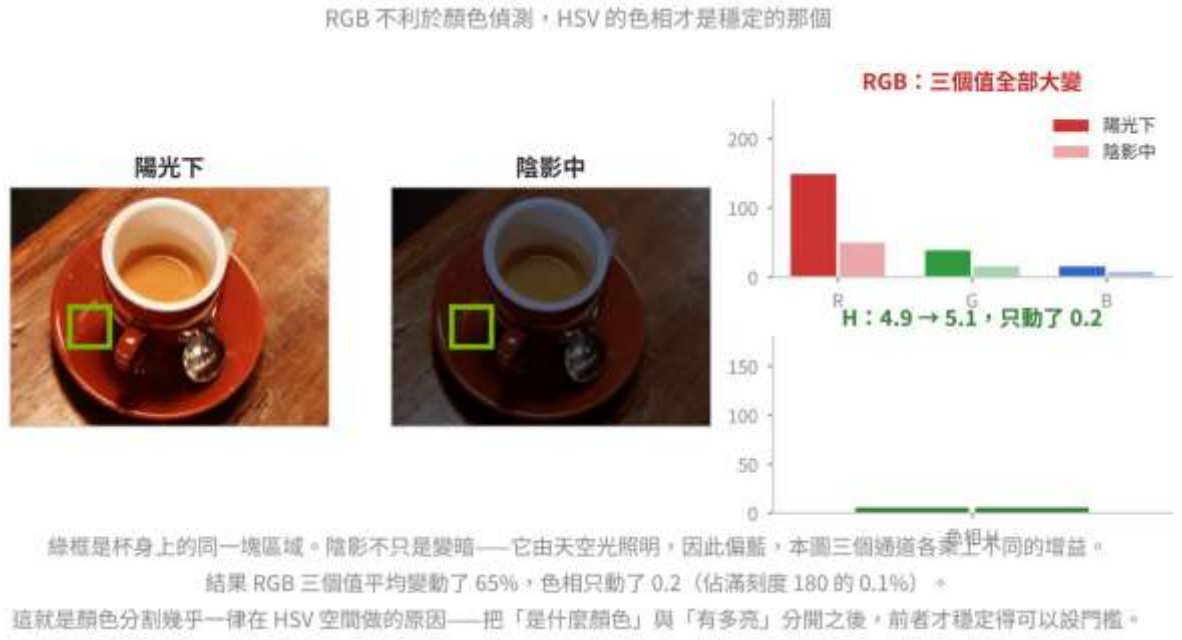


圖 8-4　同一顆蘋果進了陰影：RGB 全變，色相幾乎不動

這兩個痛點，催生了下一節的 HSL 與 HSV。

8.2 HSL 與 HSV：符合人類直覺的色彩模型

HSV(色相 Hue、飽和度 Saturation、明度 Value)與 HSL(色相、飽和度、亮度 Lightness)把「是什麼顏色」與「有多亮、多鮮豔」分離開來，更貼近人類描述顏色的方式：一個「深一點的紅」只需要調降 V 或 L，色相 H 保持不變。

對機器視覺而言，這個分離帶來一個關鍵好處：承 8.1.3 節，陽光下與陰影中的紅蘋果，雖然 RGB 差異巨大，但它們的 H(色相)卻相當接近——變的主要是 V(明度)。因此「以 H 為主要判準、對 V 放寬門檻」的顏色分割，遠比在 RGB 空間設範圍來得

穩健。這正是 8.6 節顏色追蹤的核心策略。



圖 8-5　把一張照片拆成 H、S、V 三個通道

8.2.1 幾何詮釋：圓柱、圓錐與雙圓錐

這兩個模型有三種常見的幾何視覺化方式，各自對應不同的理解角度：



圖 8-6　圓柱、圓錐與雙圓錐：同一組數字的三種幾何說法

1. 圓柱模型：最直接的表示法。繞著圓周一圈是色相 H(0 度紅、120 度綠、240 度藍);離中心軸的半徑是飽和度 S(中心為灰，愈外圈愈鮮豔);沿著軸的高度是 V 或 L。缺點是它「浪費」了很多空間：當 V=0 時整層都是黑色，無論 H 與 S 為何，但圓柱仍然畫出一整個圓面。
2. 圓錐模型(HSV)：把圓柱底部收縮成一個點，正好對應「明度為零時只有黑色」的事實。頂面圓心為白色，頂面外緣是最飽和的純色。這是 HSV 最貼切的幾何。
3. 雙圓錐模型(HSL)：上下各一個圓錐，尖端分別是白(L=1)與黑(L=0)，最飽和的顏色出現在中間高度 L=0.5 的赤道圓周上。這反映了 HSL 的設計：亮度 L 是對稱的，從黑經過純色到白。

HSV 與 HSL 的關鍵差異就在這裡：HSV 的 V=1 頂面同時包含白色與所有純色(把白色和純紅都視為「最亮」);HSL 則把純色放在中間，只有 L=1 才是白色。

實務上的選擇原則：做顏色分割與追蹤時 HSV 較常用，因為「色相加飽和度」的門檻設定較直覺，且 V 通道直接對應「有多亮」便於放寬;而做顏色調整介面(如繪圖軟體的調色盤)則 HSL 較符合使用者對「亮度」的期待——把滑桿拉到最上面應該是白色，而非刺眼的純色。

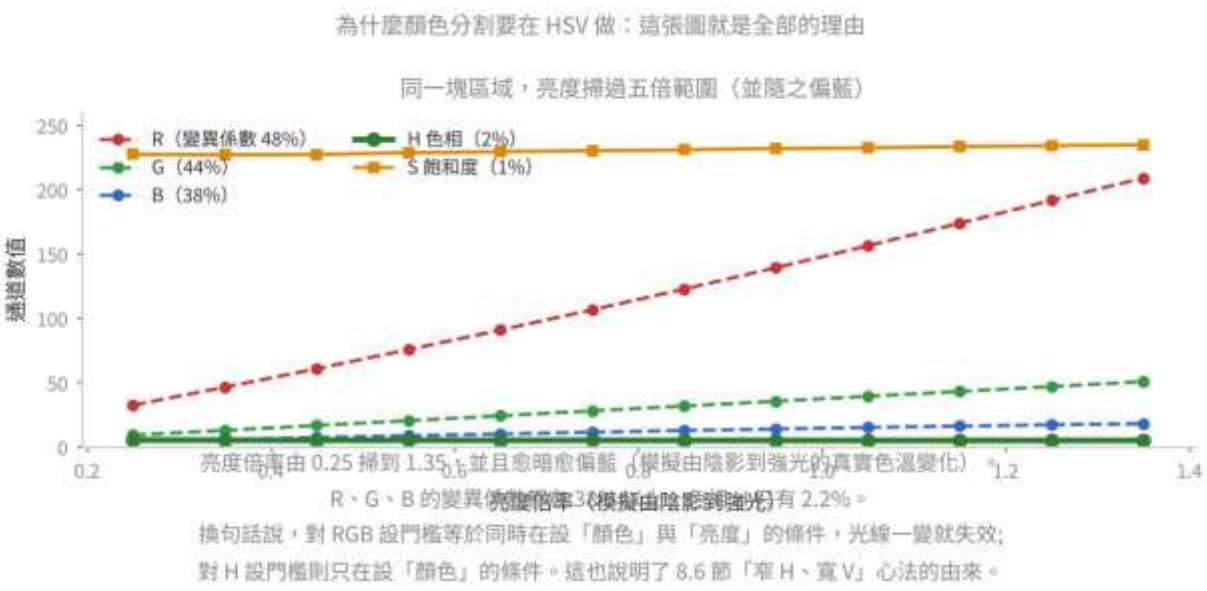


圖 8-7　亮度掃過五倍範圍：哪個通道撐得住

8.2.2 OpenCV 的 HSV 數值範圍陷阱

這是初學者的第二大地雷(第一大是第 2 章的 BGR)。理論上色相 H 的範圍是 0 至 360 度，但 OpenCV 為了塞進 8 位元的 uint8(上限 255)，把 H 除以二，範圍變成 0 至 179;S 與 V 則是 0 至 255。因此網路上查到的「紅色 H 約為 0 或 360」，在 OpenCV 中要換算成 0 或 179。

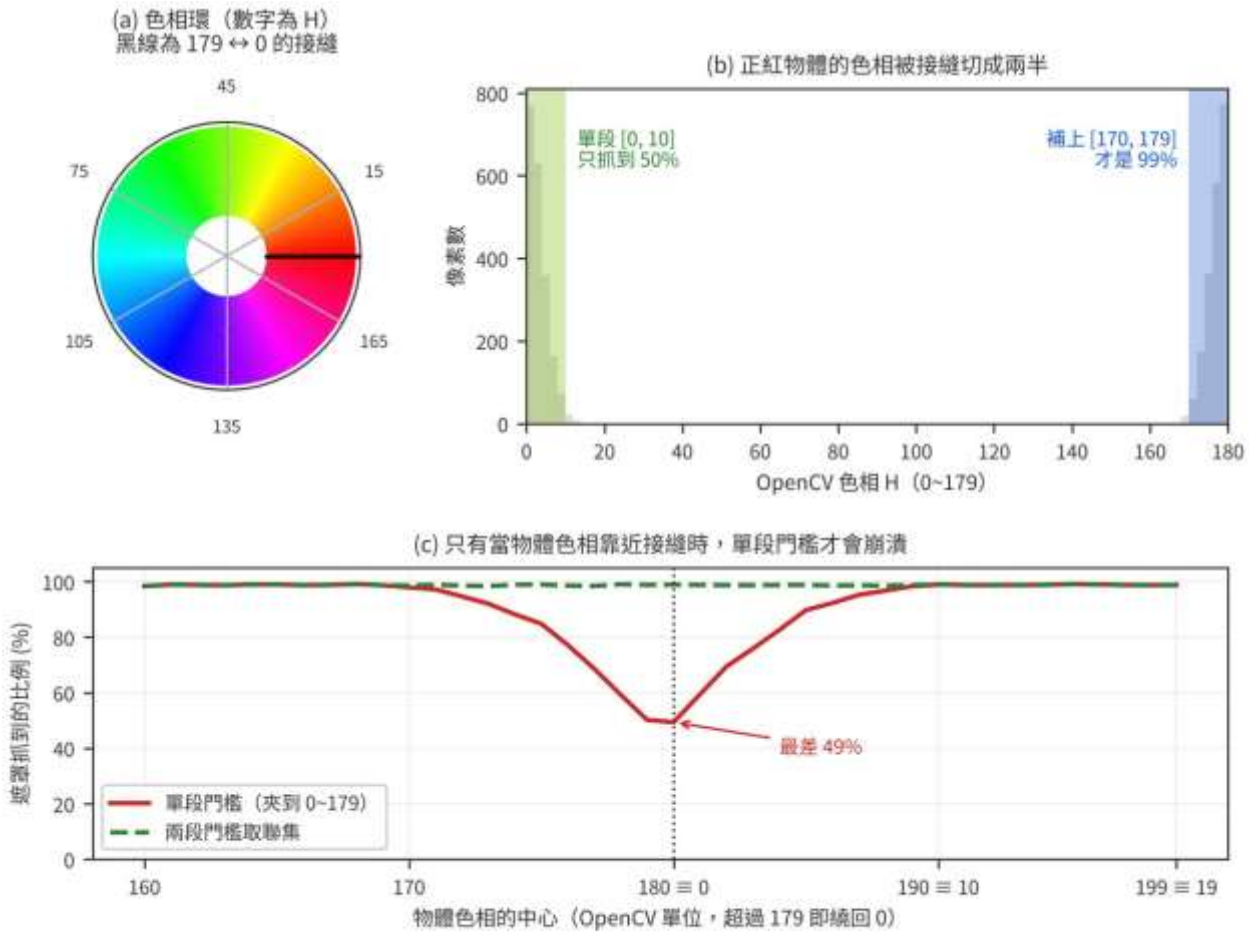
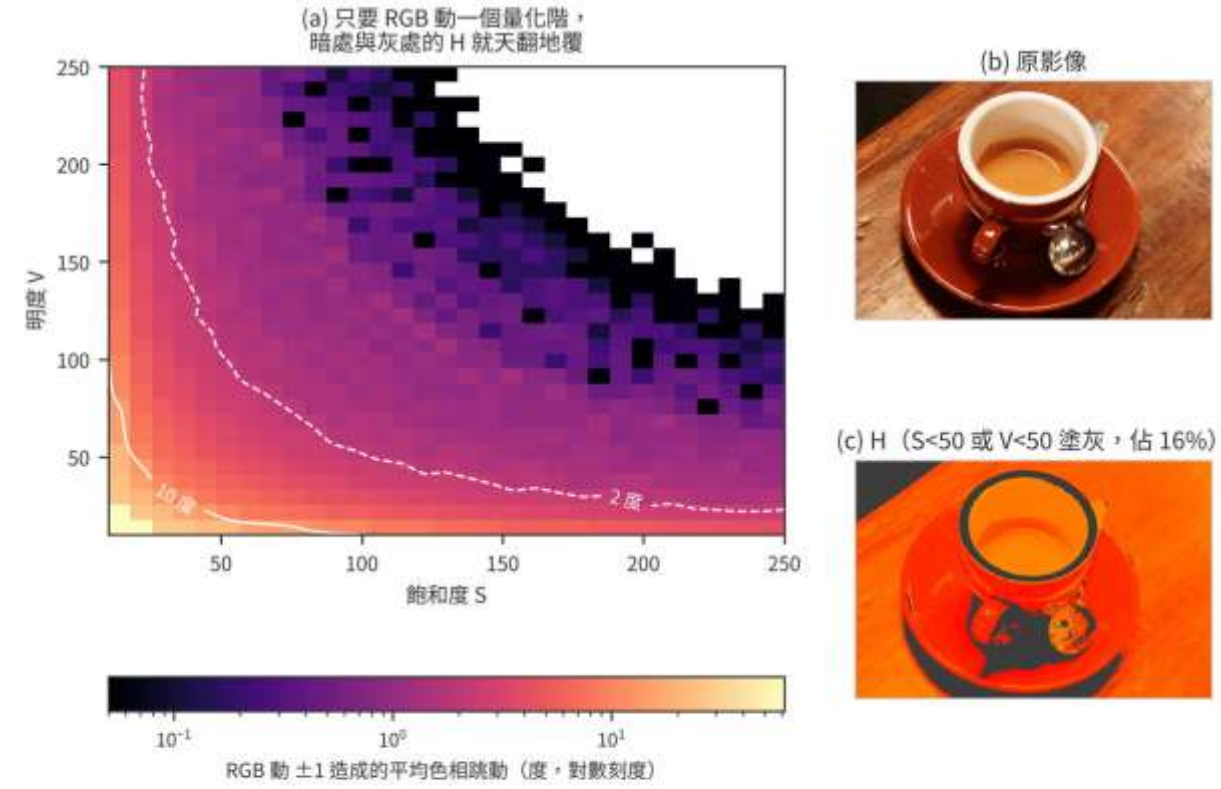


圖 8-8　OpenCV 的 H 只有 0 到 179，而紅色正好卡在接縫上

另一個必須注意的實務細節：紅色的色相跨越 0 度的接縫，因此偵測紅色時通常需要設定兩段區間再取聯集。這個「環狀資料的接縫問題」在處理角度類資料時普遍存在(第 12 章光流的方向、第 13 章 SIFT 的方向直方圖也都要處理)，值得記在心裡。

還有一個常被忽略的陷阱：當顏色很暗或很不飽和時(接近黑、白、灰),H 值變得

極不穩定甚至無意義——想像圓錐的尖端，所有色相在那裡都收縮成同一點。因此顏色分割時務必同時設定 S 與 V 的下限，把這些「色相無意義」的區域排除，否則會出現大量誤判。



實測同一張照片：對每個像素的 BGR 加上 −1~+1 的隨機擾動後，S<50 或 V<50 的區域色相平均跳動 2.4 度，其餘區域只跳動 0.4 度（相差 5.4 倍）；更極端的 S<25 或 V<25 則跳動 3.9 度。這就是顏色分割必須同時設定 S 與 V 下限的量化理由——否則演算法會把一堆「其實沒有顏色」的像素當成有效的色相來比對。

圖 8-9　暗處與灰處的色相：RGB 動一階就天翻地覆

程式 8-1 HSV 轉換與顏色門檻

```
import cv2, numpy as np

bgr = cv2.imread("robot.jpg")
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

# OpenCV:H 為 0~179、S 與 V 為 0~255
h, s, v = cv2.split(hsv)
print("H 範圍:", h.min(), h.max())

# --- 偵測藍色:單一區間即可 ---
#   注意 S 與 V 的下限:排除過暗與過灰的區域(H 在那裡無意義)
blue = cv2.inRange(hsv, (100, 120, 60), (130, 255, 255))

# --- 偵測紅色:跨越 0 度接縫,需兩段聯集 ---
red1 = cv2.inRange(hsv, (0, 120, 60), (10, 255, 255))
red2 = cv2.inRange(hsv, (170, 120, 60), (179, 255, 255))
```

```
red = cv2.bitwise_or(red1, red2)

# --- 驗證 8.1.3 節的說法:同一物體在明暗下的 RGB 與 HSV 變化 ---
bright = np.uint8([[[40, 40, 220]]])            # BGR 的亮紅
dark   = np.uint8([[[15, 15, 90]]])             # 同色但變暗
for name, px in [("亮", bright), ("暗", dark)]:
    hsv_px = cv2.cvtColor(px, cv2.COLOR_BGR2HSV)[0, 0]
    print(f"{name}: BGR={px[0,0]}, HSV={hsv_px}")
# 觀察:BGR 三值都大幅改變,但 H 幾乎不變 → 故以 H 為判準較穩健

cv2.imwrite("mask_red.png", red)
```

8.2.3 推導：RGB 轉 HSV 的公式是怎麼生出來的

幾乎每一本教科書都會列出 RGB 轉 HSV 的公式，而幾乎每一個學生看完都只記得「有一堆 if」。其實那些 if 不是硬湊出來的，它們是一個很單純的幾何操作留下的痕跡。只要把這件事想清楚，公式就不必背了。

先把 RGB 想成一個邊長為 1 的立方體，黑在原點 (0, 0, 0)，白在對角 (1, 1, 1)。連接黑與白的那條對角線上，三個分量永遠相等，所以那整條線就是灰階——這條線叫做中性軸。現在把這個立方體拿起來，讓中性軸對著你的視線方向，從白色那一端往黑色看過去。你看到的會是一個正六邊形：紅、黃、綠、青、藍、洋紅六個頂點依序排在圓周上，灰色落在正中央。HSV 的全部內容就是用極座標描述這張投影圖，再加上一個「離黑色多遠」的高度。

三個分量因此各有明確的幾何意義。明度 V = max(R, G, B)：這是立方體上「最接近白色的那一面」的座標，所以 V = 1 的那一層同時含有白色與所有最飽和的純色——這正是 8.2.1 節說 HSV 圓錐頂面既是白又是純色的原因。飽和度 S = (max − min) / max：分子 max − min 衡量三個分量彼此差多少，完全相等時為零（落在中性軸上，就是灰）；除以 max 是為了讓同一個色相在不同亮度下有相同的 S。色相 H 則是投影圖上的角度，而算角度的標準技巧就是先看「哪一個分量最大」，決定自己落在六邊形的哪一個 60 度扇區，再用另外兩個分量的差去算扇區內的偏移量。

於是那三個 if 的身分就清楚了：它們只是在問「現在人在哪一個扇區」。若 R 最

大，H = 60 × ((G − B) / (max − min) mod 6);若 G 最大，H = 60 × ((B − R) / (max − min) + 2);若 B 最大，H = 60 × ((R − G) / (max − min) + 4)。式子裡的 +2 與 +4 不過是「跳過前面兩個扇區」（每個扇區 120 度，除以 60 就是 2），而 mod 6 正是色相環繞回原點的那一刀——圖 8-8 的接縫問題，源頭就在這個 mod。

最後看分母。三條式子的分母都是 max − min，也就是 S 的分子。當一個顏色接近灰色時 max − min 趨近於零，分母趨近於零，H 的計算結果就會對輸入的微小變動極度敏感。這不是程式寫得不好，而是幾何上的必然：在六邊形的正中央，「角度」這個概念本來就沒有定義。圖 8-9 量到的那個「RGB 動一階、色相跳好幾十度」的現象，到這裡終於有了解釋——它就是一個除以接近零的數的典型症狀，和 7.1.5 節談的條件數是同一件事。

8.3 YCbCr 與 CIE Lab

8.3.1 YCbCr：為壓縮與傳輸而生

YCbCr 把亮度 Y 與兩個色差通道 Cb(藍色差)、Cr(紅色差)分開。它的設計動機非常務實：人眼對亮度細節的敏感度遠高於色彩細節，因此可以對 Cb、Cr 大幅降取樣(常見的 4:2:0 取樣就是把色度解析度砍成四分之一)，而人眼幾乎察覺不出差異。



圖 8-10　為什麼砍色度不心疼，砍亮度會出事

這是 JPEG 壓縮與所有數位視訊標準的第一步——第 11 章談 JPEG 時會完整展開。它也是數位電視與類比電視相容的關鍵：黑白電視只讀 Y 通道即可正常顯示。

值得留意這個「把資源分配給人眼在意之處」的設計思想，它在本書中反覆出現：第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、本節的色度降取樣、第 11 章 JPEG 對高頻係數用較大的量化步長。三者都是同一種智慧——不是所有資訊都同等重要，把位元花在刀口上。

8.3.2 CIE Lab：為量測色差而生

CIE Lab 由國際照明委員會制定，目標是感知均勻(Perceptually Uniform)：在這個空間中，兩個顏色的歐氏距離大致正比於人眼感受到的色差大小。

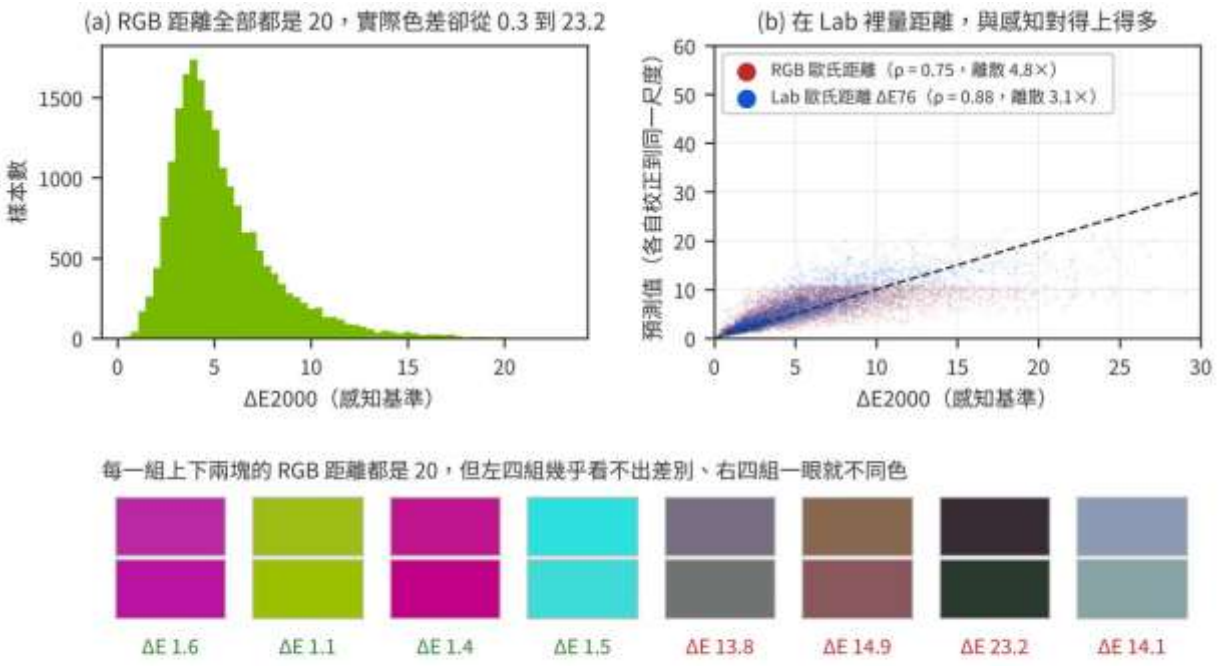


圖 8-11　感知均勻的意思：同樣的 RGB 距離，色差可以差 70 倍

這在 RGB 中是做不到的——RGB 空間裡同樣的數值差距，在不同區域造成的視覺差異可能天差地別(人眼對綠色的細微變化極為敏感，對藍色則相對遲鈍)。Lab 的 L 為亮度(0 黑到 100 白),a 為綠紅軸，b 為藍黃軸。

應用場合：印刷與紡織的品管(以 ΔE 色差值判定是否合格，通常 ΔE 小於 1 表示人眼幾乎無法分辨)、色彩校正、以及第 5 章介紹過的技巧——對彩色影像做 CLAHE 時，轉到 Lab 只處理 L 通道，色彩才不會失真。

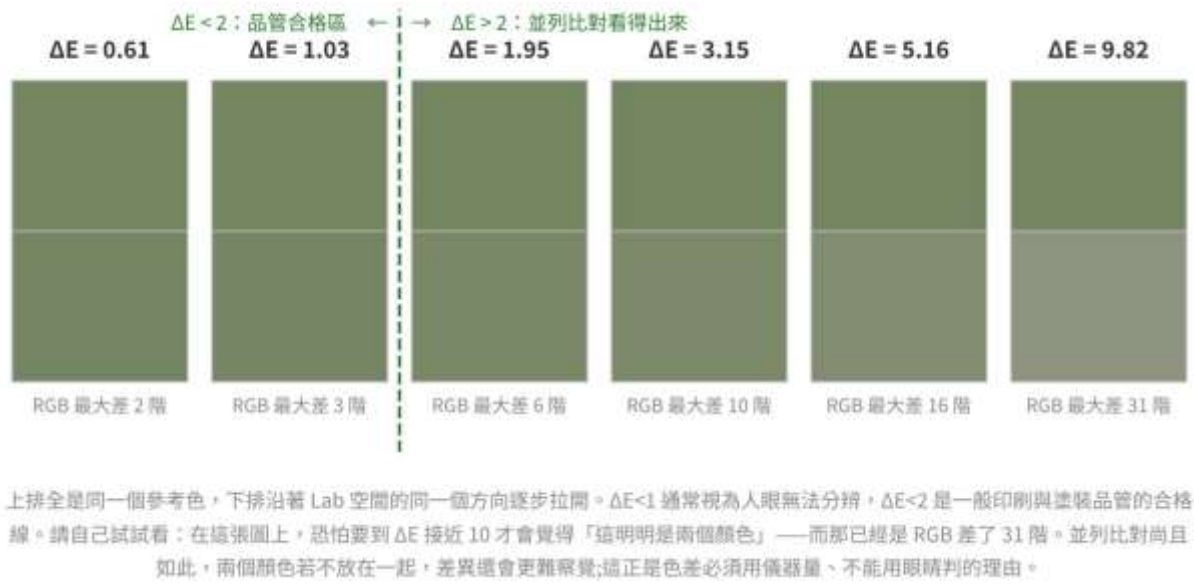


圖 8-12　ΔE 要多大才看得出來：一條親自檢驗的色差尺

色彩空間	設計目的	通道意義	典型應用
RGB	對應感光元件與螢幕	紅、綠、藍強度	顯示、儲存
HSV	符合人類對顏色的描述	色相、飽和度、明度	顏色分割與追蹤
HSL	亮度對稱、便於調色	色相、飽和度、亮度	調色盤、繪圖軟體
YCbCr	利用人眼對色度不敏感以壓縮	亮度、藍色差、紅色差	JPEG、視訊編碼
CIE Lab	感知均勻，可量測色差	亮度、綠紅軸、藍黃軸	品管色差、色彩校正

表 8-1 常用色彩空間比較

8.3.3 推導：0.299、0.587、0.114 這三個數字從哪裡來

把彩色轉成灰階時，幾乎所有的程式庫都在算 Y = 0.299R + 0.587G + 0.114B。這三個係數看起來很隨意，加起來剛好是 1，而且綠色的權重是藍色的五倍以上。它們既不是誰拍腦袋決定的，也不是實驗湊出來的——它們是從一組顏色座標解一個線性方程式解出來的。

出發點是這樣一個要求：我們想要一個亮度通道，使得「白色的亮度等於 1」，而且這個亮度盡可能對應人眼感受到的明暗。CIE 1931 系統裡的 Y 分量正是為此設計

的（它的定義就是用人眼的明視覺光譜效率曲線加權積分而來）。所以問題變成：已知螢幕三個原色各自的 CIE 座標，請問要用多少比例混合，才能得到 Y？

動手算。以類比電視時代的 NTSC 原色為例，三個原色的色度座標分別是紅 (0.67, 0.33)、綠 (0.21, 0.71)、藍 (0.14, 0.08)，而參考白點為 (0.310, 0.316)。色度座標的定義是 $x = X/(X+Y+Z)$、$y = Y/(X+Y+Z)$，因此每個原色的 XYZ 可以寫成 $(x/y,\ 1,\ (1-x-y)/y)$ 乘上一個待定的縮放係數。把三個待定係數排成一個向量，要求「三個原色全開時總和等於白點的 XYZ」，就得到一個三元一次方程組。解出來之後，每個原色的 Y 分量就是我們要的權重，而答案是 0.2990、0.5864、0.1146——四捨五入到三位小數，就是課本上那三個數字。總和為 1 不是巧合，而是「白色的 Y 等於 1」這個條件直接要求的結果。想親手驗算的讀者，這個三元一次方程組用 numpy 的 linalg.solve 三行就解得出來，把原色座標換成別的標準也立刻得到那個標準的係數。

這個推導帶出兩個常被忽略的實務重點。第一，係數會隨原色而變。現代的高畫質電視採用 Rec. 709 原色，同樣的算法解出來是 0.2126、0.7152、0.0722;超高畫質的 Rec. 2020 又是另外一組 0.2627、0.6780、0.0593。OpenCV 的 cvtColor 預設仍用 0.299 那一組，所以若你的流程混用了不同標準，灰階值會有百分之幾的差異——這在一般應用無傷大雅，在色彩量測或影像比對上卻可能造成系統性偏差。

第二，綠色權重特別大，是因為人眼的錐狀細胞在中波段最敏感（圖 8-1），而不是因為綠色比較重要。這一點把本書的三條線索串了起來：第 3 章的拜爾陣列讓綠色佔一半、本節的亮度係數讓綠色佔六成、8.3.1 節的色度降取樣把資源留給 Y——三個看似無關的設計，背後是同一句話：把位元花在人眼在意的地方。

8.4 虛擬色彩：讓看不見的資訊現形

虛擬色彩(Pseudocolor，又稱偽彩色)是把灰階或單一數值的資料，依照一張色彩對照表(Colormap)映射成彩色影像。

它的目的不是「還原真實顏色」——這些資料本來就沒有顏色——而是利用人眼分辨顏色的能力遠強於分辨灰階的特性：人類大約只能分辨數十階灰度，卻能分辨數萬種顏色。把資料的細微差異映射到色彩，就能讓原本看不出來的結構一目了然。

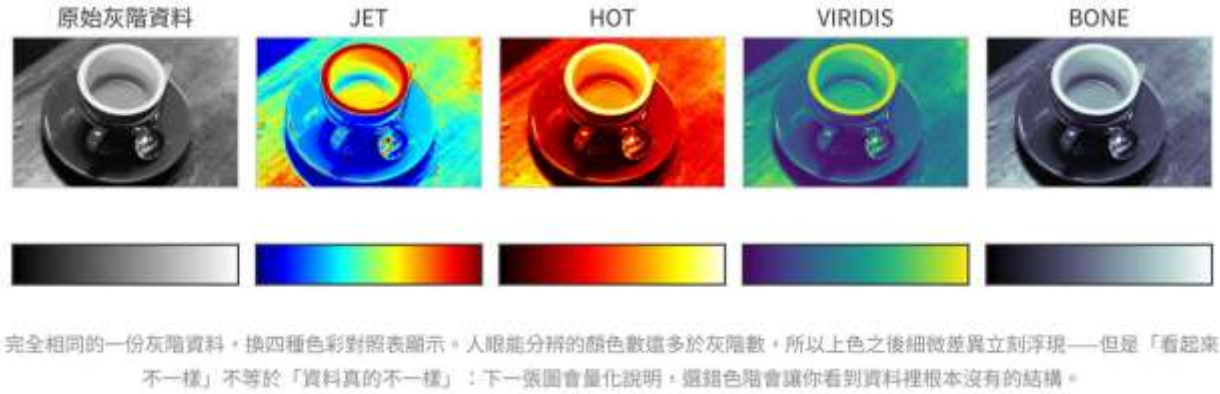


圖 8-13　同一份灰階資料，四種色階

應用場景極廣：熱影像儀把溫度映射成藍到紅的色階;醫學影像用色彩標示血流速度或代謝強度;衛星遙測把近紅外波段染成紅色以突顯植被;深度相機(第 17 章)把距離映射成彩虹色;第 12 章的稠密光流用色相表示運動方向;第 15、16 章的神經網路熱力圖也是靠虛擬色彩顯示「AI 在看哪裡」。

8.4.1 色階選擇的專業考量

經典的 JET(彩虹色)雖然搶眼，但在科學視覺化領域近年頗受批評，原因有三：其一，它的亮度變化不是單調遞增的(中間的黃綠色比兩端更亮)，容易在資料中製造出實際不存在的「邊界」，誤導判讀;其二，對色盲使用者不友善(約百分之八的男性有紅綠色盲);其三，轉成黑白列印後資訊完全喪失。

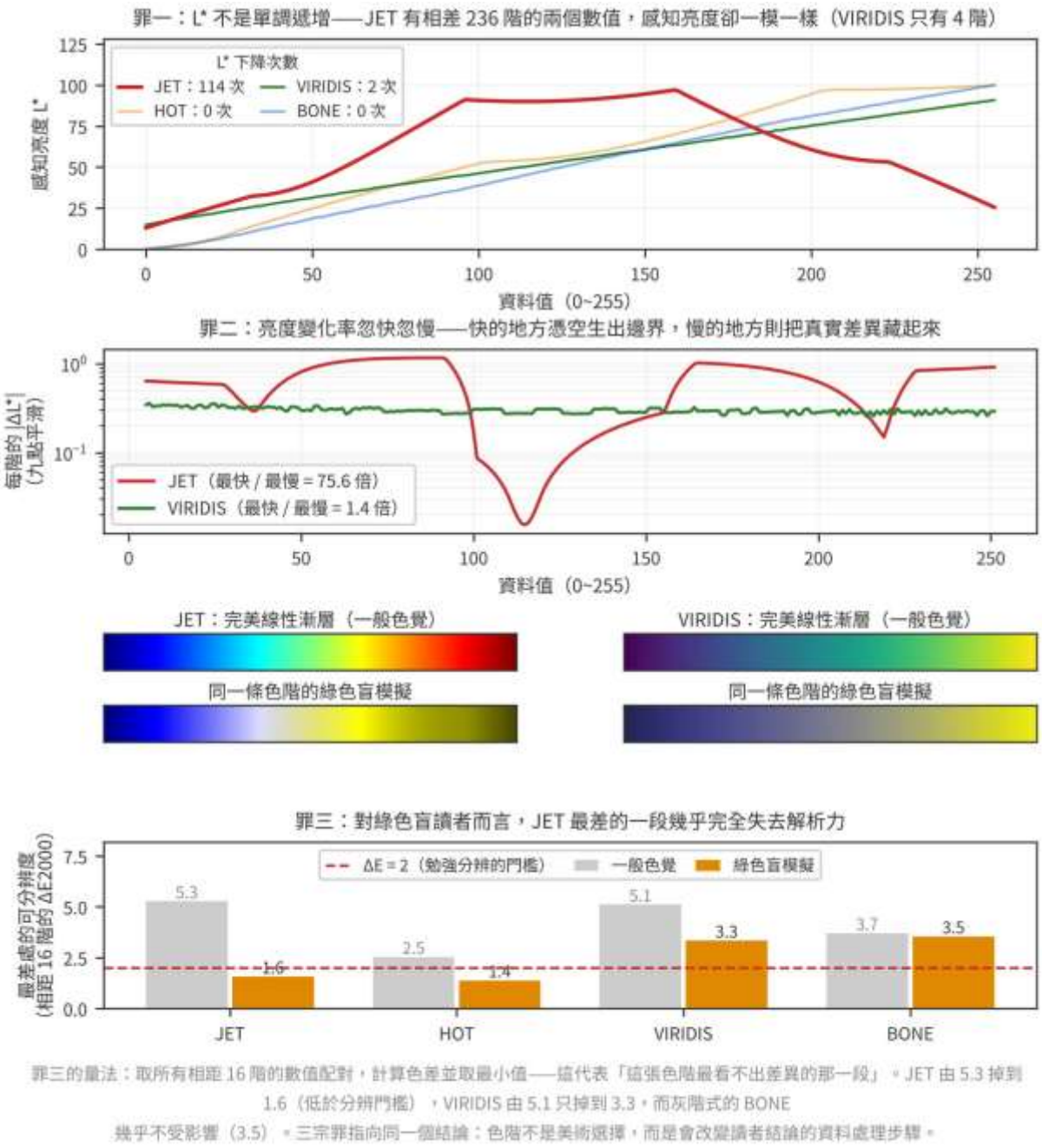


圖 8-14　JET 的三宗罪，每一宗都量得出來

VIRIDIS、MAGMA 等感知均勻的色階已成為新的推薦選項——它們的亮度單調遞增、色盲友善、且黑白列印後仍保有層次。在論文與報告中選擇 colormap，也是一種專業素養;這個考量與 8.3.2 節 CIE Lab 的「感知均勻」是同一種思想在視覺化上的應用。

程式 8-2 虛擬色彩與色階的檢驗

```
import cv2
```

```
gray = cv2.imread("thermal_raw.png", cv2.IMREAD_GRAYSCALE)

# 常用色彩對照表
jet     = cv2.applyColorMap(gray, cv2.COLORMAP_JET)      # 經典彩虹(慎用)
hot     = cv2.applyColorMap(gray, cv2.COLORMAP_HOT)      # 熱影像風格
viridis = cv2.applyColorMap(gray, cv2.COLORMAP_VIRIDIS)  # 感知均勻,推薦
bone    = cv2.applyColorMap(gray, cv2.COLORMAP_BONE)     # 醫學影像風格

# 檢驗色階是否「亮度單調遞增」:轉回灰階看是否仍為漸層
import numpy as np
ramp = np.tile(np.arange(256, dtype=np.uint8), (50, 1))
for name, cm in [("jet", cv2.COLORMAP_JET),
                 ("viridis", cv2.COLORMAP_VIRIDIS)]:
    colored = cv2.applyColorMap(ramp, cm)
    back = cv2.cvtColor(colored, cv2.COLOR_BGR2GRAY)
    diff = np.diff(back[0].astype(np.int16))
    print(f"{name}: 亮度單調遞增? {np.all(diff >= 0)}")
    # JET 會出現 False → 這正是它造成假邊界的原因

cv2.imwrite("pseudo_viridis.png", viridis)
```

8.5 白平衡與色彩校正

回到叫獸開講的核心問題：如何讓機器扣掉光源的影響?這正是白平衡(White Balance)要解決的事，也是人腦色彩恆常性的工程版本。

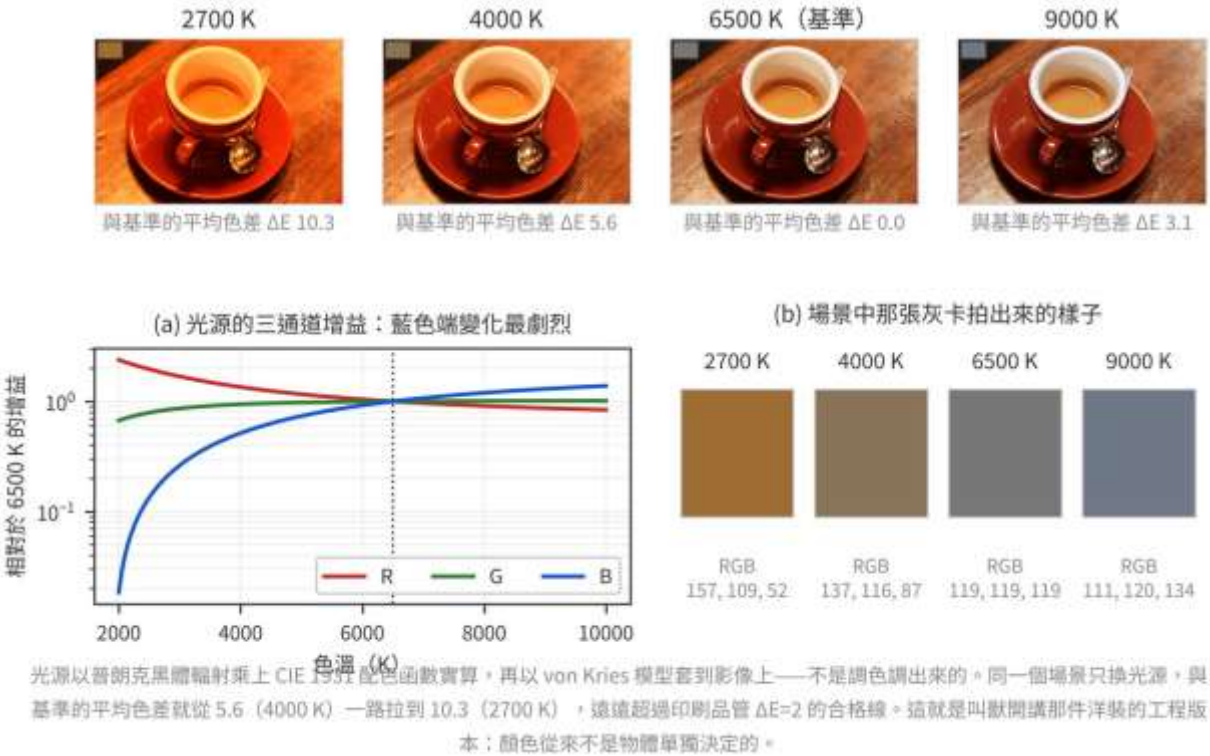
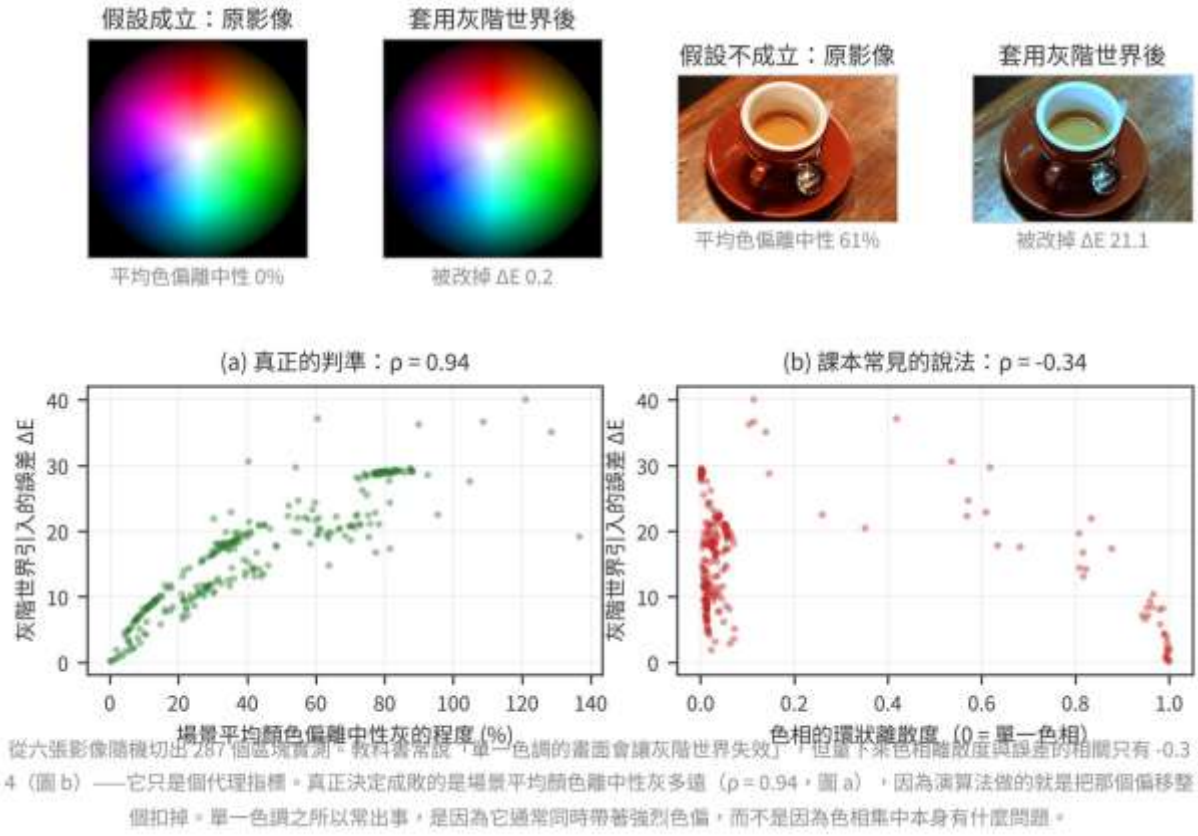


圖 8-15　只換光源，同一個場景就換了顏色

8.5.1 灰階世界假設

最簡單的演算法是灰階世界假設(Gray World)：假設整個場景的平均顏色應該是中性灰。若實測的平均偏藍，代表光源偏藍，於是把藍通道整體調降、紅通道調升。

它在色彩豐富的場景中效果不錯，但遇到「整片草地」、「整面紅牆」、「大片藍天」這類單一色調的畫面就會失效——因為前提假設不成立，演算法會誤以為光源偏綠(或偏紅、偏藍)，硬是把草地校正成灰色。這是一個很好的提醒：所有演算法都建立在假設之上，而工程師的責任是判斷假設是否成立。



從六張影像隨機切出 287 個區塊實測。教科書常說「單一色調的畫面會讓灰階世界失效」，但量下來色相離散度與誤差的相關只有 -0.34（圖 b）——它只是個代理指標。真正決定成敗的是場景平均顏色離中性灰多遠（ρ = 0.94，圖 a），因為演算法做的就是把那個偏移整個扣掉。單一色調之所以常出事，是因為它通常同時帶著強烈色偏，而不是因為色相集中本身有什麼問題。

圖 8-16　灰階世界什麼時候會失效：真正的判準不是色相

8.5.2 完美反射假設與色卡校正

第二種是完美反射假設(White Patch / Retinex)：找出影像中最亮的像素，假定它應該是純白，據此計算各通道的增益。它對「畫面中確實有白色物體」的場景很準，但遇到過曝或高光反射就會誤判——因為過曝的像素已經飽和，其色彩資訊已經喪失。

工業場合最可靠的做法則是第三種：在畫面中放置標準色卡(如 24 色的 ColorChecker)，以已知的參考色計算完整的色彩校正矩陣。這不只是白平衡，而是把整個色彩響應校正到標準——這也是叫獸開講思考題的標準答案之一，更是農產品分級、印刷打樣、醫學影像等對顏色準確度有要求的場合的標準流程。

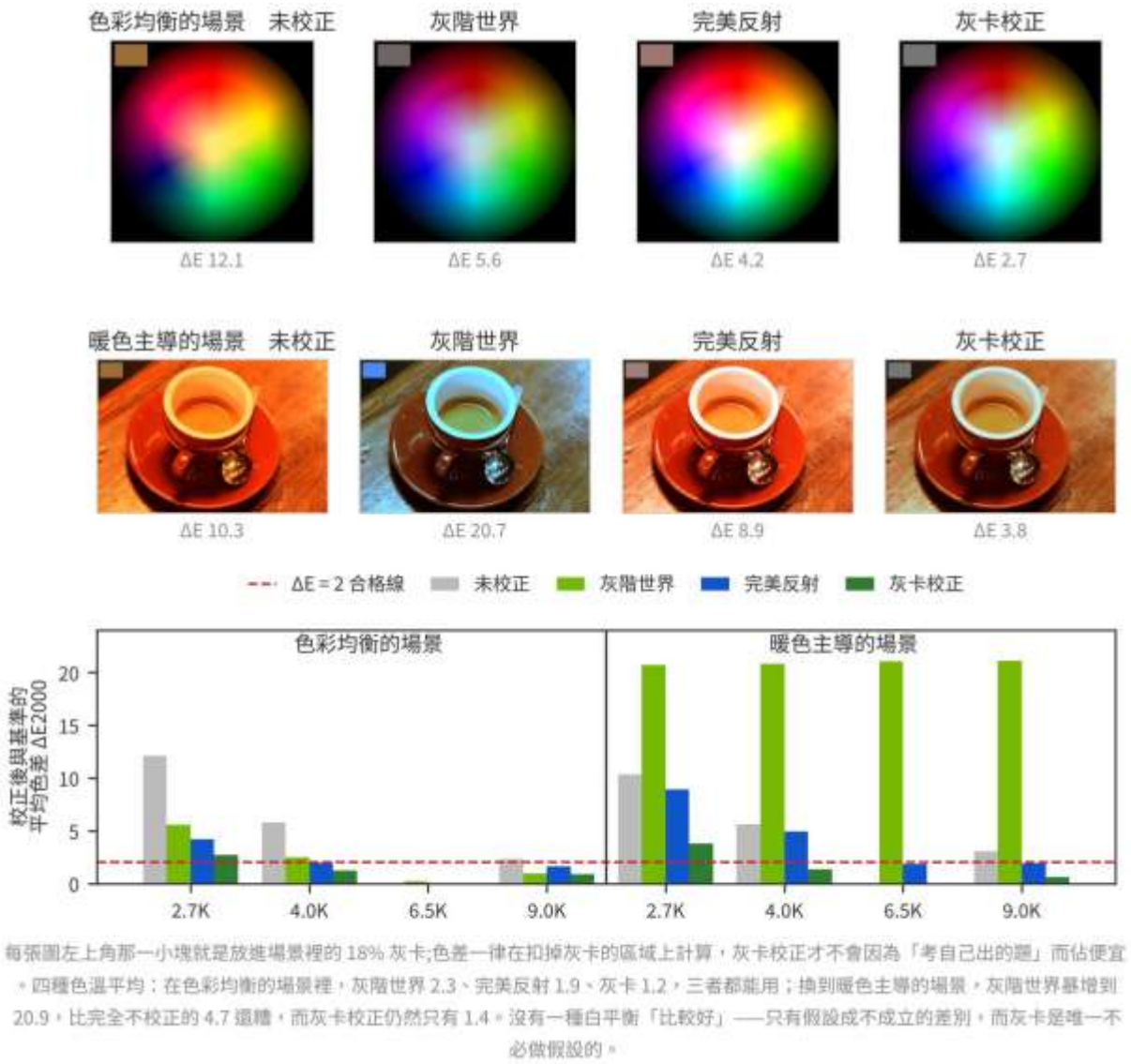


圖 8-17　三種白平衡 × 兩種場景 × 四種色溫的完整成績

至於現代的第四種路線：第 5 章 Night Sight 的學習式白平衡——用神經網路從大量標註資料中學習「什麼樣的影像看起來白平衡正確」。它在傳統方法失效的困難場景(如夜景、單色調畫面)表現優異，但也繼承了深度學習的侷限(第 15 章)：遇到訓練分布外的場景可能失準，且難以解釋。

8.5.3 推導：灰階世界為什麼會失效，以及失效多少

8.5.1 節說灰階世界在單一色調的畫面中會失效。這句話對，但不夠精確——圖 8-16 實測了 287 個真實影像區塊，發現色相的集中程度與誤差的相關係數只有 −0.34，而「場景平均顏色離中性灰多遠」的相關係數高達 0.94。這一小節把這件事算清楚。

先把演算法寫成數學式。設場景中每個像素的反射率為 $\rho(x)$，光源在三個通道的強度為 L，則相機拍到的值為 $I(x) = L \odot \rho(x)$（$\odot$ 表示逐通道相乘）。對整張影像取平均，得到 $\text{mean}(I) = L \odot \text{mean}(\rho)$。灰階世界做的事，是把每個通道除以該通道的平均值，也就是輸出 $I(x) / \text{mean}(I) = \rho(x) / \text{mean}(\rho)$。請注意這個結果：$L$ 被完整地消掉了。這代表灰階世界其實是一個「無論光源是什麼都一定成立」的恆等式——它永遠會把光源除乾淨。

那問題出在哪裡？出在最後那個 $\text{mean}(\rho)$。我們要的答案是 $\rho(x)$ 本身，拿到的卻是 $\rho(x) / \text{mean}(\rho)$。只有當場景的平均反射率在三個通道上相等（也就是 $\text{mean}(\rho)$ 正比於 $(1, 1, 1)$，場景平均真的是中性灰）時，這個除法才只是整體縮放，不改變顏色。否則殘留的誤差就是 $\text{mean}(\rho)$ 本身的色偏，與光源完全無關。這解釋了圖 8-17 裡一個乍看詭異的結果：在暖色主導的場景中，灰階世界在四種色溫下的誤差都是 21 左右，幾乎不隨光源變化——因為那個誤差根本不是光源造成的，而是場景自己的色偏被當成光源扣掉了。

誤差的大小也因此可以預估。殘留的通道增益是 $\text{mean}(\rho)$ 各通道與其平均的比值，所以「場景平均顏色偏離中性灰的百分比」幾乎就是誤差的直接度量——這正是圖 8-16(a) 那條相關係數 0.94 的曲線。單一色調的場景之所以常出事，是因為它通常同時帶著強烈色偏;但一張色相集中卻剛好接近中性的照片（例如灰階的雪景加一點淡藍）並不會讓灰階世界失效，而一張色相很分散卻整體偏暖的照片（例如黃昏的市集）反而會。把判準從「色相散不散」改成「平均離不離中性」，你才真的知道什麼時候可以用它。

最後是工程上的收尾。既然誤差來自 mean(ρ) 的未知色偏，那麼補救的方向只有兩個：一是讓 mean(ρ) 變成已知——在畫面中放一張反射率已知的灰卡，用它來算增益，這就是 8.5.2 節的色卡校正，也是圖 8-17 中唯一在兩種場景都穩定的方法;二是乾脆不讓 L 變動——鎖住光源，問題就不存在。這兩條路都呼應 7.6.3 節的層次原則：能在源頭解決的，不要留到演算法去補。

程式 8-3 兩種白平衡演算法與其失效情境

```
import cv2, numpy as np

def gray_world(img):
    """灰階世界白平衡:假設場景平均為中性灰"""
    result = img.astype(np.float32)
    avg_b, avg_g, avg_r = (result[:,:,0].mean(),
                           result[:,:,1].mean(),
                           result[:,:,2].mean())
    avg_gray = (avg_b + avg_g + avg_r) / 3
    result[:,:,0] *= avg_gray / avg_b
    result[:,:,1] *= avg_gray / avg_g
    result[:,:,2] *= avg_gray / avg_r
    return np.clip(result, 0, 255).astype(np.uint8)

def white_patch(img, percentile=99):
    """完美反射假設:以最亮的像素作為白色參考"""
    result = img.astype(np.float32)
    for c in range(3):
        ref = np.percentile(result[:,:,c], percentile)  # 取分位數較穩健
        result[:,:,c] *= 255.0 / max(ref, 1)
    return np.clip(result, 0, 255).astype(np.uint8)

img = cv2.imread("yellowish_indoor.jpg")
cv2.imwrite("wb_grayworld.jpg", gray_world(img))
cv2.imwrite("wb_whitepatch.jpg", white_patch(img))

# 驗證失效情境:對一張「整片草地」的影像套用灰階世界
grass = cv2.imread("grass_field.jpg")
cv2.imwrite("wb_grass_fail.jpg", gray_world(grass))
# 觀察:草地被硬生生校正成灰綠色 → 假設不成立時演算法失效
```

8.6 顏色分割與物件追蹤

把前面所學組合起來，就得到機器人視覺最基礎也最實用的技能：用顏色找出目標物並持續追蹤。

標準流程五步驟：轉 HSV(利用色相對光照變化較穩健)→ 以 inRange 產生二值遮

罩 → 用型態學運算(第 10 章)清除雜點與補洞 → 找輪廓取最大者(第 9 章)→ 計算中心座標並輸出控制訊號。這正是循跡機器人、色球追蹤、簡易分揀系統的骨幹。

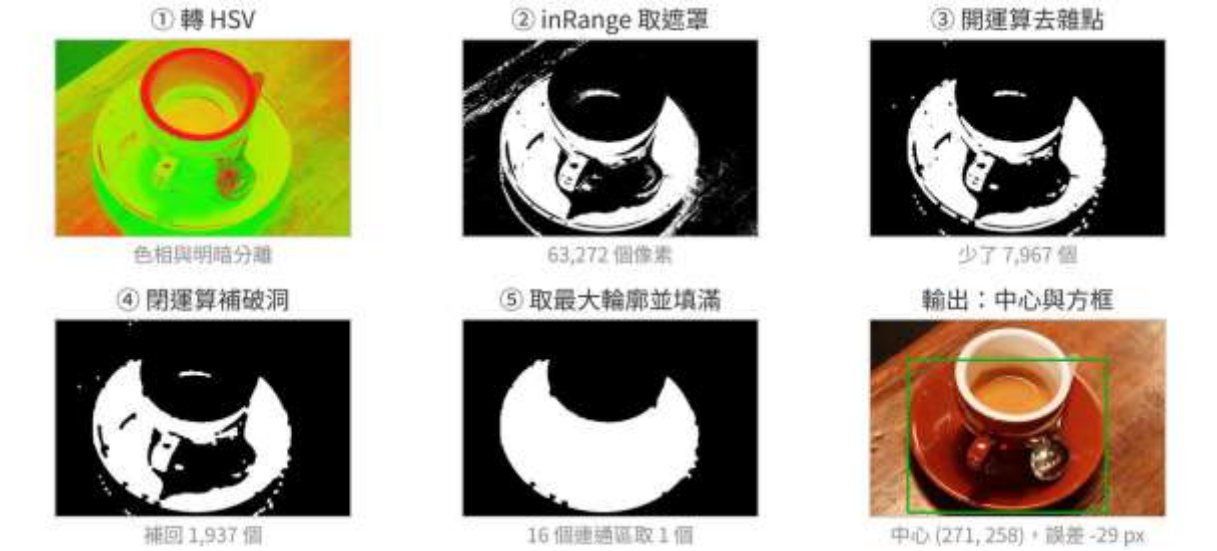


圖 8-18　顏色分割五步驟，每一步的像素數都在上面

參數設定的實務心法：對 H 設窄門檻(因為色相是主要判準)、對 S 與 V 設寬門檻但要有下限(容許明暗變化，同時排除色相無意義的暗處與灰處)。這個「窄 H、寬 V」的原則，直接源自 8.1.3 與 8.2 節對 RGB 與 HSV 的比較。

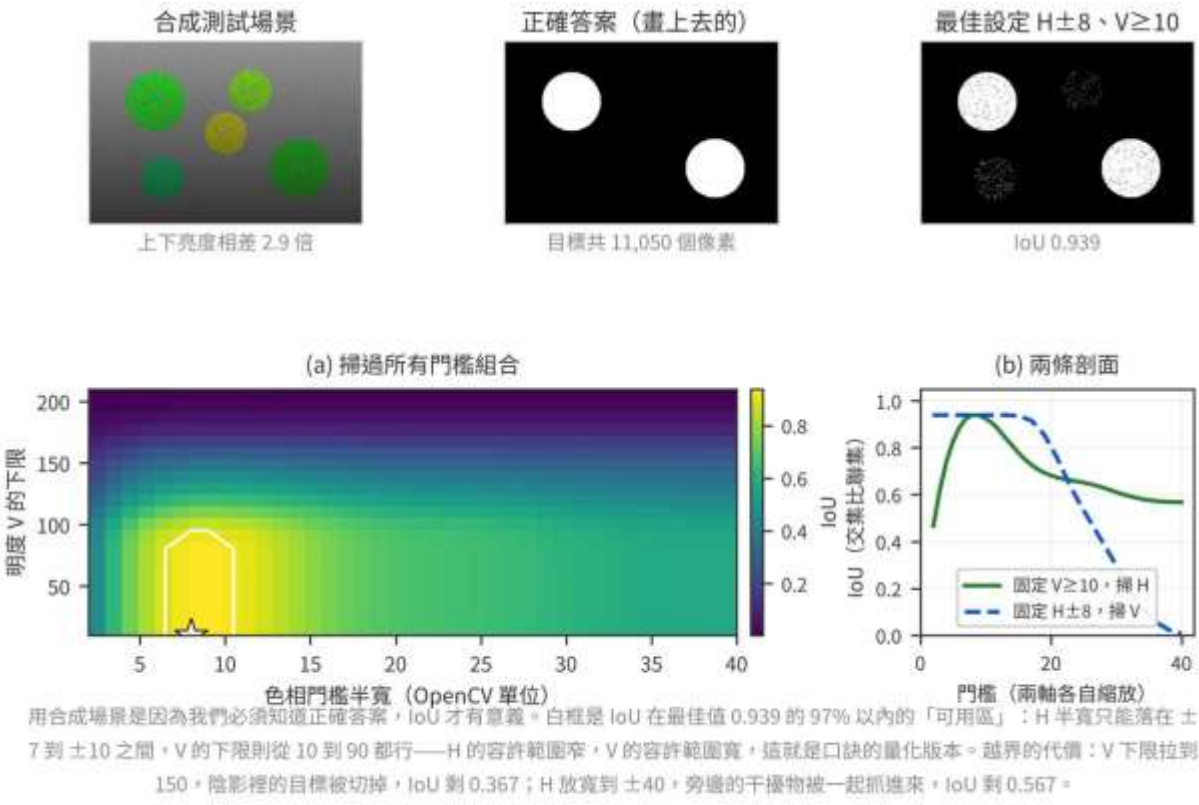


圖 8-19　「窄 H、寬 V」的量化版本：掃過所有門檻組合

程式 8-4 HSV 顏色追蹤與機器人控制訊號

```
import cv2, numpy as np

cap = cv2.VideoCapture(0)          # 或 ESP32-CAM 的串流網址(第 2 章)
kernel = np.ones((5, 5), np.uint8)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    frame = gray_world(frame)                  # 白平衡前處理(8.5 節)
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # 窄 H、寬 V,但 S 與 V 有下限
    mask = cv2.inRange(hsv, (35, 100, 60), (85, 255, 255))   # 綠色物件

    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)    # 去雜點
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)   # 補破洞

    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
    if cnts:
        c = max(cnts, key=cv2.contourArea)
        if cv2.contourArea(c) > 500:
            x, y, w, h = cv2.boundingRect(c)
            cx, cy = x + w // 2, y + h // 2
```

```
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)

            # 機器人控制訊號:依偏差量決定轉向
            err = cx - frame.shape[1] // 2
            cmd = "左轉" if err < -30 else ("右轉" if err > 30 else "直行")
            cv2.putText(frame, f"{cmd} (err={err})", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    cv2.imshow("tracking", frame)
    if cv2.waitKey(1) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

8.7 應用案例

8.7.1 案例一：農產品的成熟度分級

情境：番茄、香蕉、水果的成熟度可由顏色判斷，是雲林在地農業自動化最直接的應用。做法：以 HSV 或 Lab 空間萃取顏色特徵(如色相的平均值與分布)，建立成熟度與顏色的對應關係，再以第 14 章的分類器或簡單的門檻規則分級。

這個案例最能凸顯叫獸開講的教訓：光源是成敗的關鍵。同一顆番茄在陰天與晴天、上午與傍晚拍攝，顏色可能落到不同的等級。因此實務上的必要措施是——建立封閉的檢測箱、使用穩定的 LED 光源、並在每次拍攝時納入標準色卡校正。若做不到這些(如田間直接拍攝)，就必須用第 15 章的資料增強讓模型學會適應光線變化，或改用對光源較不敏感的特徵。

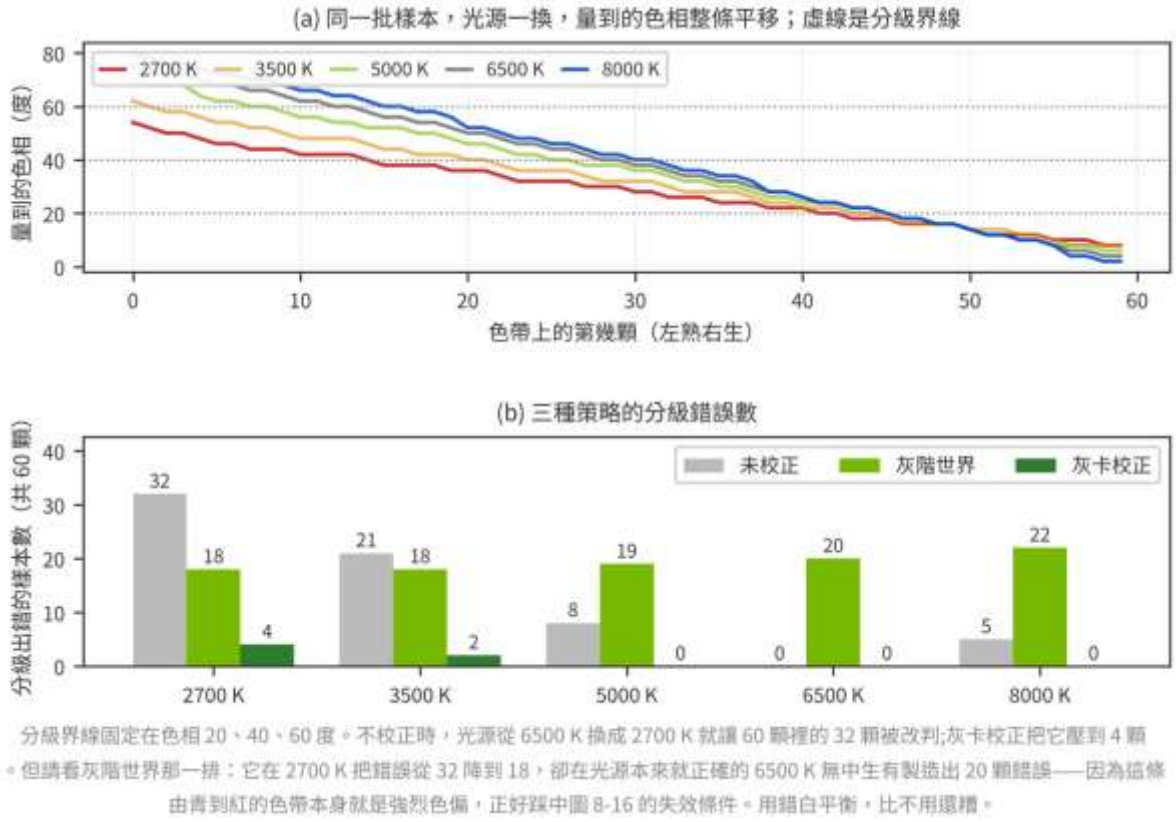


圖 8-20　成熟度分級：光源一換，六十顆裡三十二顆被改判

8.7.2 案例二：醫學與科學的虛擬色彩

情境：熱影像儀顯示體溫分布、超音波都卜勒顯示血流方向與速度、功能性磁振造影顯示腦部活動強度。這些資料本身都是單一數值(溫度、速度、訊號強度)，沒有顏色，必須透過虛擬色彩才能讓人判讀。

這裡有一個攸關判讀正確性的專業考量，呼應 8.4.1 節：色階的選擇會直接影響醫師「看到什麼」。用 JET 彩虹色階顯示的溫度分布，可能因為亮度非單調而讓人誤以為某處有明顯的邊界或異常;而選用感知均勻的色階，則能讓資料的真實漸變忠實呈現。此外，色階的數值範圍(對應第 5.7.2 節的視窗化)同樣關鍵——同一份資料用不同的範圍映射，可以讓病灶明顯或完全隱形。這再次呼應第 7、16 章的倫理主題：視覺化的選擇不是中性的。

8.7.3 案例三：工業 AOI 的光源設計

情境：檢測電子零件的顏色標記、判斷線材的色碼、辨識產品標籤。這類任務在

受控環境下極為可靠，關鍵完全在於「控制變因」。

工程上的標準做法包括：使用高演色性、色溫穩定的 LED 光源(廉價 LED 的光譜不連續，會使某些顏色偏移);設計光源的方向與擴散方式以避免高光反射(反光處會過曝，色彩資訊喪失);以遮光罩隔絕環境光的干擾;定期以標準色卡驗證色彩響應是否漂移(LED 隨使用時間會衰減與偏色)。

這個案例是叫獸開講思考題的完整答案：與其設計一個「能對抗任何光源」的聰明演算法，不如從源頭消除光源的變異——這在可控的產線環境中，永遠是更務實、更可靠的工程選擇。這也呼應第 7.6.3 節「三個層次的解法應以光學機構層為優先」的原則。

8.7.4 案例四：色彩在深度學習中的角色

情境：訓練第 15、16 章的模型時，色彩相關的決策比想像中更多。

第一，輸入格式：大多數預訓練模型期待 RGB 順序，而 OpenCV 讀進來是 BGR(第 2 章)——這個順序錯誤是常見的 bug，會使準確率莫名下降。第二，資料增強：隨機調整色相、飽和度、亮度(第 16 章 YOLO 的 hsv_h、hsv_s、hsv_v 參數)是標準做法，目的正是讓模型對光源變化不敏感——這是「用學習取代白平衡」的思路。第三，是否轉灰階：若任務與顏色無關(如形狀辨識)，轉灰階能減少三分之二的計算量;但若顏色是關鍵線索(如成熟度、警示標誌)，則絕不能丟棄。

這些決策說明了一件事：即使進入深度學習時代，對色彩的理解仍然是工程師必備的判斷力來源。模型不會替你決定該用哪個色彩空間、該不該做白平衡——那是你的工作。

8.8 本章與全書的觀念連結總表

本章觀念	相關章節	如何延伸

BGR 順序	第 2、15、16 章	OpenCV 的歷史包袱;深度模型期待 RGB
拜爾陣列與去馬賽克	第 3、4 章	每像素僅記錄一色，需內插補齊
只處理亮度通道	第 5 章	彩色影像做 CLAHE 應轉 Lab 只等化 L
色度降取樣	第 11 章	JPEG 第一步的 YCbCr 與 4:2:0
虛擬色彩	第 10、12、15 章	距離轉換視覺化;稠密光流;神經網路熱力圖
白平衡與色彩恆常性	第 5、15 章	Night Sight 的學習式白平衡;資料增強取代白平衡
HSV 顏色分割	第 9、10、12 章	分割的顏色依據;型態學清理;CamShift 追蹤
光源控制	第 7、9、17 章	AOI 取像設計;打光不均的處理;治本優於治標

表 8-2 第 8 章與全書其他章節的觀念連結

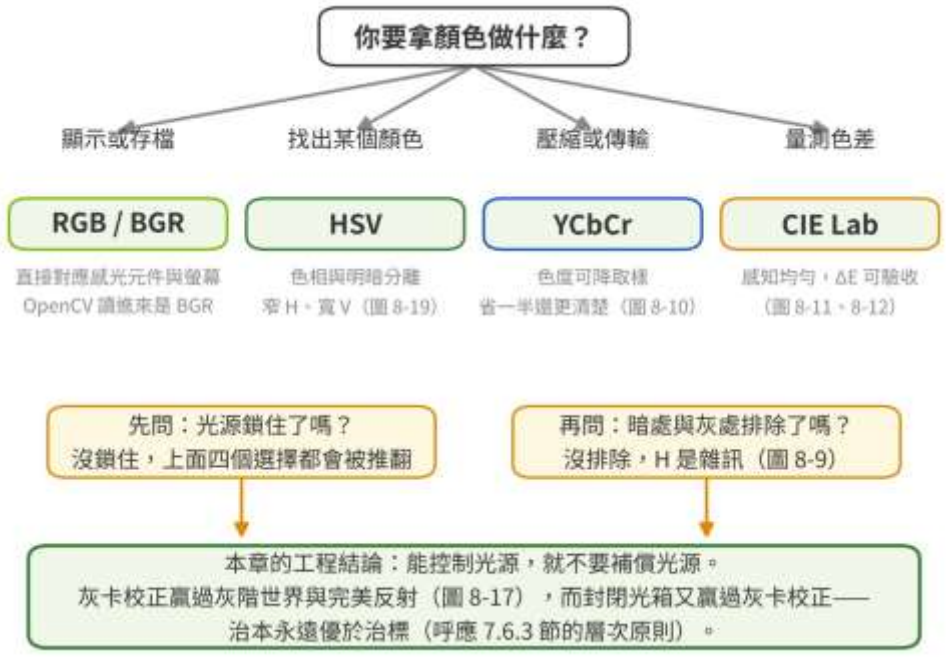


圖 8-21　第 8 章的決策地圖

程式實作 Lab 8

Lab 8-1 色彩空間探索器

目標：建立對色彩空間的直覺，並打造整學期最常用的調參工具。(a)寫一支程式，以 cv2.createTrackbar 建立六個滑桿(H、S、V 的上下界)，即時顯示遮罩與被遮罩後的結果;(b)用它找出教室中五種不同顏色物件的 HSV 範圍，製成表格;(c)驗證 8.1.3 節的論述：記錄同一物件在明亮處與陰影中的 RGB 與 HSV 值，比較哪些通道變化大、哪些穩定;(d)討論：為什麼顏色分割通常對 H 設窄門檻、對 V 設寬門檻?為什麼 S 與 V 必須設下限?(e)把這支工具存成共用程式，第 9、10、12 章的 Lab 都會用到。

Lab 8-2 洋裝效應實測

目標：親身驗證叫獸開講的現象。(a)取一個顏色明確的物件(如紅色馬克杯)，分別在窗邊日光、日光燈、白熾燈、以及手機的暖色螢幕光下拍攝;(b)量測同一位置的 RGB 與 HSV 值，製成表格，計算各通道的變化幅度;(c)套用灰階世界與完美反射兩種白平衡，比較校正後的數值是否收斂;(d)在畫面中放入一張白紙作為參考，以「白紙應為中性灰」為準則手動校正，比較與自動演算法的差異;(e)討論：這個實驗對「用顏色做工業檢測」有什麼啟示?

Lab 8-3 虛擬色彩與色階評估

目標：培養視覺化的專業素養。(a)取一張灰階影像(熱影像、深度圖或一般照片)，分別以 JET、HOT、VIRIDIS、BONE 四種色階顯示;(b)執行程式 8-2 的檢驗，量化各色階的亮度是否單調遞增，找出 JET 在哪個區段違反單調性;(c)把四張結果轉成黑白列印(或轉灰階)，比較哪些色階仍保有層次;(d)模擬色盲觀看：把結果的紅綠通道混合，觀察哪些色階仍可判讀;(e)結論：為一份要投稿的技術報告選擇色階，你會怎麼選?請說明理由。

Lab 8-4 循跡機器人的顏色偵測

目標：完成可上機的顏色追蹤模組。(a)以程式 8-4 為基礎，改用 ESP32-CAM 的串流網址作為輸入(第 2 章);(b)加入白平衡前處理，比較開啟與關閉時在不同光源下的穩定度;(c)輸出控制訊號：依目標物中心與畫面中心的偏差量，印出轉向指令與偏差角度;(d)量測系統的處理速度(FPS)，並思考：若改用第 16 章的 YOLO 偵測，速度與穩定度會如何變化?各適合什麼場合?(e)挑戰題：同時追蹤兩種顏色的物件並標示兩者的距離，或設計一個「顏色遙控」——用不同顏色的卡片對機器人下指令。

Lab 8-5 色溫對檢測的影響實測

目標：親手量出光源對顏色判讀的影響，並驗證白平衡的補救效果。(a)取三種不同色溫的光源（例如暖白燈泡、日光燈、手機手電筒），在同一位置拍攝同一組色卡或同一批水果，相機務必關閉自動白平衡與自動曝光，並記錄每次的快門與 ISO;(b)量測同一個色塊在三種光源下的 RGB 與 HSV，並以 skimage.color.deltaE_ciede2000 計算兩兩之間的色差;(c)分別套用灰階世界、完美反射與灰卡校正，重新計算色差，做出一張類似圖 8-17 的長條圖;(d)把其中一次拍攝換成「畫面中塞滿同一種顏色」的構圖，重做 (c)，觀察灰階世界是否如圖 8-16 所預測的那樣失效;(e)寫出 300 字的結論：若你是產線工程師，你會選哪一種策略，為什麼。

Lab 8-6 色階的專業檢驗工具

目標：做出一支可以重複使用的色階評估工具，培養視覺化的專業判斷力。(a)寫一個函式，輸入任一 OpenCV 或 matplotlib 的 colormap，輸出三項指標：L* 的下降次數、L* 每階變化率的最大值與最小值之比、以及綠色盲模擬後相距 16 階的最小色差;(b)綠色盲模擬請實作 Viénot, Brettel & Mollon (1999) 的方法——sRGB 先轉線性、再轉 LMS，令 M 通道由 L 與 S 線性重建，最後轉回 sRGB;(c)用這支工具評比 JET、HOT、VIRIDIS、BONE、TURBO 五種色階，做出一張類似圖 8-14 的對照表，並檢查你的結果是否與本書一致;(d)找一張你自己報告或論文裡用過的熱圖，用這支工具檢驗它的色階，若不合格就換掉並比較前後的可讀性;(e)進階：把工具包成命令列程式，加

入「黑白列印模擬」（轉灰階後看是否仍保有層次），放進你的個人工具箱。

本章重點整理

- 人眼三種錐狀細胞是三原色理論的生理基礎;同色異譜使不同光譜可呈現相同顏色，是彩色顯示成立的原因，也使一般 RGB 相機無法區分材質(需多光譜相機)。
- RGB 為加色系統(螢幕),CMY(K) 為減色系統(印刷)，色域不同;RGB 對機器友善但對人不直觀，且同一物體在明暗下三通道全變，不利顏色偵測。
- HSV/HSL 分離「色相」與「明暗鮮豔度」;幾何上可用圓柱、圓錐(HSV)、雙圓錐(HSL)詮釋，HSL 的純色位於 L=0.5 的赤道。色相對光照變化較穩健，故顏色分割以 H 為主要判準。
- OpenCV 的 HSV 範圍為 H:0~179、S/V:0~255;偵測紅色需跨 0 度接縫以兩段區間取聯集;暗處與灰處的 H 無意義，必須設定 S 與 V 的下限。
- YCbCr 為壓縮而生(色度可降取樣),CIE Lab 為感知均勻而生(可量測色差 ΔE);「把資源分配給人眼在意之處」的思想貫穿拜爾陣列、色度降取樣與 JPEG 量化。
- 虛擬色彩利用人眼辨色優於辨灰的特性;JET 因亮度非單調而可能製造假邊界且對色盲不友善，建議改用 VIRIDIS 等感知均勻色階。
- 白平衡是色彩恆常性的工程版本：灰階世界(假設場景平均為灰，單色調畫面失效)、完美反射(以最亮處為白，過曝時誤判)、標準色卡校正(最可靠)、學習式白平衡(現代路線)。工業場合應優先從源頭控制光源。

習題

1. 什麼是同色異譜?它為什麼是彩色顯示技術得以成立的基礎?它對機器視覺有什麼限制上的啟示?

提示與參考解答：同色異譜指兩束光譜完全不同的光，只要刺激三種錐狀細胞的比例相同，人眼就覺得是同一顏色。它讓螢幕只用紅綠藍三種光源就能模擬幾乎所有顏色。啟示：一般 RGB 相機與人眼一樣只記錄三個數值，因此無法分辨同色異譜的物體(如兩塊看似同色但材質不同的塑膠);要真正區分材質需使用記錄完整光譜的多光譜或高光譜相機。

2. 說明加色系統與減色系統的差異，並解釋為何螢幕上鮮豔的照片印出來常變暗淡。

提示與參考解答：加色系統(RGB)是光源相加，三色全開得白，用於自發光裝置(螢幕、投影機、LED)。減色系統(CMY)是顏料吸收特定波長，疊愈多愈暗，三色相加理論得黑(實務加 K 成 CMYK)，用於印刷。變暗淡的原因：兩系統的色域(可表現的顏色範圍)不同，螢幕能顯示的某些鮮豔顏色超出印刷色域，只能以最接近的顏色代替。

3. 為什麼說 RGB 不利於顏色偵測?請以「同一物體在陽光下與陰影中」說明，並解釋 HSV 如何改善。

提示與參考解答：同一顆紅蘋果在陽光下可能是 (220,40,40)、陰影中變成 (90,15,15)，三個通道都大幅改變，以 RGB 數值範圍偵測極不可靠。HSV 把「是什麼顏色」(H)與「有多亮」(V)分離：明暗變化主要影響 V，而 H 相當穩定。因此以 H 為主要判準、對 V 放寬門檻的分割策略，遠比在 RGB 空間設範圍穩健。

4. 請比較 HSV 的圓錐模型與 HSL 的雙圓錐模型，說明最飽和的純色分別位於幾何體的哪個位置，以及兩者各適合什麼應用。

提示與參考解答：HSV 圓錐：底部尖端為黑，頂面圓心為白、頂面外緣為最飽和的純色(白與純色同在 V=1)。HSL 雙圓錐：上下尖端分別為白與黑，最飽和的純色在中間高度 L=0.5 的赤道圓周。應用：HSV 適合顏色分割與追蹤(H 加 S 門檻直覺、V 便於放寬);HSL 適合調色盤與繪圖軟體(把滑桿拉到最上應為白色，符合使用者對「亮度」的期待)。

5. OpenCV 的色相 H 為什麼是 0 到 179 而非 0 到 359?偵測紅色時要如何處理?為什麼還必須設定 S 與 V 的下限?

提示與參考解答：因為要塞進 8 位元 uint8(上限 255)，把 0 至 360 度除以二成為 0 至 179。紅色的色相跨越 0 度接縫，須設兩段區間(如 0~10 與 170~179)再取聯

集。必須設 S 與 V 下限的原因：當顏色很暗或很不飽和(接近黑白灰)時，H 值極不穩定甚至無意義(圓錐尖端所有色相收縮成一點)，不排除這些區域會造成大量誤判。

6. YCbCr 的 4:2:0 降取樣為何幾乎不影響觀感?這個設計思想在本書還出現在哪兩個地方?

   提示與參考解答：因為人眼對亮度細節的敏感度遠高於色彩細節，把色度解析度砍成四分之一，人眼幾乎察覺不出。同樣的「把資源分配給人眼在意之處」思想還出現在：第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、第 11 章 JPEG 對人眼不敏感的高頻係數用較大的量化步長。

7. 什麼是「感知均勻」?CIE Lab 為何需要它?為什麼對彩色影像做 CLAHE 時應轉到 Lab 只處理 L 通道?

   提示與參考解答：感知均勻指在該色彩空間中，兩顏色的歐氏距離大致正比於人眼感受到的色差大小。RGB 做不到——同樣的數值差距在不同區域造成的視覺差異天差地別。Lab 為此設計，使色差可量化(ΔE)，用於印刷紡織品管與色彩校正。做 CLAHE 只處理 L 通道的原因：若對 BGR 三通道分別等化，各通道映射函數不同，色彩比例被破壞而產生色偏;Lab 把亮度與色彩分離，只調 L 不動 a、b，色彩即可保持。

8. 虛擬色彩的目的是什麼?為什麼近年 JET 色階受到批評?請列出三個理由。

   提示與參考解答：目的：把沒有顏色的資料(溫度、深度、訊號強度)映射成彩色，利用人眼辨色能力遠強於辨灰(數萬種顏色 vs. 數十階灰度)的特性，讓細微差異一目了然。JET 受批評的三個理由：一、亮度非單調遞增(中間黃綠色比兩端亮)，會製造實際不存在的假邊界而誤導判讀;二、對色盲使用者不友善(約 8% 男性有紅綠色盲);三、轉黑白列印後資訊喪失。建議改用 VIRIDIS 等感知均勻色階。

9. 說明灰階世界白平衡的假設與失效情境。完美反射假設又在什麼情況下誤判?

   提示與參考解答：灰階世界假設整個場景的平均顏色應為中性灰，若實測偏藍就判定光源偏藍並反向校正。失效情境：整片草地、整面紅牆、大片藍天等單一色調畫面，假設不成立，演算法會把草地硬校正成灰綠色。完美反射假設以影像中最亮的像素為純白計算增益，在過曝或有高光反射時誤判——因為過曝像素已飽和，色彩資訊喪失，無法作為可靠的白色參考。

10. 顏色分割時，為什麼通常對 H 設定較窄的門檻，而對 V 設定較寬的門檻?

提示與參考解答：因為 H(色相)是「是什麼顏色」的主要判準，同一物體在不同光照下 H 相當穩定，設窄門檻可精確鎖定目標顏色、排除其他顏色。而 V(明度)會隨光照、陰影、角度大幅變化，設寬門檻才能容許這些變化不致漏檢。但 V(與 S)仍須設下限，以排除色相無意義的過暗與過灰區域。

11. 承叫獸開講：什麼是色彩恆常性?為什麼同一張洋裝照片會讓人看到截然不同的顏色?研究者用什麼方法驗證了這個解釋?

    提示與參考解答：色彩恆常性：大腦會自動推測當下的光源顏色並扣除其影響，還原物體本身的顏色(人腦內建的自動白平衡)。洋裝照片曝光過度、背景線索模糊，光源資訊極度曖昧，大腦只能依各自的先驗假設來猜：假設偏藍日光則扣掉藍色而看到白金，假設偏黃人造光則扣掉黃色而看到藍黑。驗證方法：研究團隊把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就趨於一致，且與被暗示的光源方向吻合。

12. (綜合設計題)請為一條農產品分級產線設計「不會被光源騙倒」的顏色檢測方案，列出至少四項具體措施並說明理由。

    提示與參考解答：一、控制光源：建立封閉檢測箱、使用高演色性且色溫穩定的 LED，隔絕環境光——從源頭消除變異，治本優於治標(呼應第 7.6.3 節的層次原則)。二、放置標準色卡：每次拍攝納入 ColorChecker，計算色彩校正矩陣，補償光源漂移與感測器差異。三、選對色彩空間：以 HSV 的 H 或 Lab 的 a、b 為判準，對明度放寬，降低光照敏感度。四、光學設計：調整光源方向與擴散以避免高光反射(反光處過曝會喪失色彩資訊)。五、定期驗證：LED 會隨使用時間衰減偏色，應定期以色卡檢查色彩響應是否漂移。六、備援策略：若無法完全控制光源(如田間)，則以第 15 章的資料增強讓模型適應光線變化。

13. OpenCV 的色相只有 0 到 179，而一個正紅色物體的色相中心正好落在 0。若你用 cv2.inRange(hsv, (0, 120, 60), (10, 255, 255)) 取遮罩，大約會抓到這個物體的多少比例?為什麼程式不會報錯?

    提示與參考解答：大約只會抓到一半。物體的色相會因為量測雜訊散布在 0 的兩側，而落在 170~179 的那一半完全不在區間內。圖 8-8 的實測是 50%，把門檻中心掃過 160 到 199 後最差為 49%。程式不會報錯，因為 inRange 的語意只是「在不在這個區間」，它不知道資料是環狀的;這類錯誤只會表現為遮罩莫名其妙地少一

塊，debug 時極容易被誤判成門檻設太緊。正解是改用兩段區間取聯集，或把色相先旋轉 90 再比對。

14. 為什麼顏色分割時一定要同時設定 S 與 V 的下限?請從 RGB 轉 HSV 的公式解釋，並估計在很暗或很不飽和的區域中，RGB 只變動一個量化階會造成多大的色相跳動。

    提示與參考解答：色相的三條公式分母都是 max(R,G,B) − min(R,G,B)，也就是飽和度的分子。顏色愈接近灰或愈暗，這個分母愈接近零，微小的輸入變動就被放大成巨大的角度變化（8.2.3 節的推導）。圖 8-9 的實測：對一張真實照片的每個像素加上 −1 到 +1 的隨機擾動，S<50 或 V<50 的區域平均跳動 2.4 度，其餘區域只跳 0.4 度;更極端的 S<25 或 V<25 則跳 3.9 度。在 S、V 都很小的角落，單一量化階就足以讓色相跳過數十度。

15. 一張 1920×1080 的彩色影像，若採用 4:2:0 色度降取樣，資料量會變成原來的幾分之幾?若反過來把亮度降取樣而保留完整色度，資料量又是多少?哪一種划算?為什麼?

    提示與參考解答：4:2:0 把 Cb、Cr 的解析度各砍成四分之一，資料量為 (1 + 0.25 + 0.25)/3 = 50%。反過來砍亮度則是 (0.25 + 1 + 1)/3 = 75%。降色度不但省得多（省 50% vs 省 25%），品質還更好：圖 8-10 在同一張照片上量到 39.7 dB 對 28.5 dB，相差 11.2 dB。原因是細節幾乎全部記在亮度通道裡——該圖量到 Y 的平均梯度為 55，而 Cb、Cr 只有 13 與 16，約為 Y 的三成;砍色度砍掉的多半是本來就不存在的東西。

16. 什麼叫「感知均勻」?請設計一個實驗來檢驗 RGB 空間是不是感知均勻的，並說明你會用什麼當作「人眼感受」的基準。

    提示與參考解答：感知均勻指空間中的歐氏距離正比於人眼感受到的色差。實驗設計：隨機取大量顏色對，令它們在 RGB 中的歐氏距離固定（例如都等於 20），再計算每一對的 ΔE2000（CIE 現行標準，可當作感知基準）。若 RGB 是感知均勻的，所有 ΔE 應該差不多;圖 8-11 的實測結果是 ΔE 從 0.4 一路散到 24，相差六十餘倍，所以 RGB 顯然不是。進一步可比較兩種量尺對 ΔE2000 的預測力：RGB 歐氏距離的 Spearman 相關為 0.75，Lab 歐氏距離為 0.88。注意 ΔE2000 本身也是模型而非真理，誠實的說法是「Lab 比 RGB 接近人眼」，不是「Lab 完美」。

17. 某紡織廠的驗收標準寫著「色差 ΔE 必須小於 2」。請說明這個數字在 RGB 上大約對應多少階的差異，以及為什麼這個標準不能改成「RGB 三通道各差不超過 5 階」。

    提示與參考解答：圖 8-12 沿著 Lab 空間的一個方向實測：ΔE 約 2 時 RGB 最大差約 6 階，ΔE 約 5 時約 16 階，ΔE 約 10 時約 31 階。但這個對應關係完全取決於顏色落在色空間的哪裡——同樣差 5 階，在某些顏色上 ΔE 不到 1，在另一些上卻超過 10（圖 8-11）。因此「RGB 差不超過 5 階」在不同色號上代表寬嚴差距極大的標準，無法作為統一的驗收門檻。這正是 CIE Lab 與 ΔE 存在的理由：它讓一個數字在所有顏色上代表同一件事。

18. 為什麼科學視覺化領域近年普遍不建議使用 JET 色階?請列出三個理由，並各自說明「要怎麼量」才能證明它。

    提示與參考解答：(一)亮度非單調。量法：把色階的 256 階轉成 CIE Lab 取 L，數 L* 下降的次數——圖 8-14 量到 JET 有 114 次，VIRIDIS 只有 2 次;更嚴重的後果是 JET 有相差 236 階的兩個數值感知亮度完全相同，黑白列印後它們會變成同一個灰。(二)假邊界。量法：看 L* 每階的變化率，JET 最快處是最慢處的 76 倍，VIRIDIS 只有 1.4 倍;變化率快的地方會讓平滑的資料看起來有一條邊。(三)對色盲不友善。量法：做綠色盲模擬後，計算相距 16 階的數值配對中最小的色差——JET 由 5.3 掉到 1.6（低於分辨門檻），VIRIDIS 只由 5.1 掉到 3.3。*

19. 請用普朗克黑體輻射與 CIE 配色函數，說明為什麼色溫從 6500 K 降到 2700 K 時，藍色通道的變化遠大於紅色通道。這對工業檢測的光源選用有什麼啟示?

    提示與參考解答：黑體輻射的光譜峰值隨溫度下降而往長波長移動（維因位移定律），因此低色溫光源在短波段的能量急遽減少。圖 8-15 實算的三通道增益顯示：由 6500 K 到 2700 K，紅色增益約變為 2.2 倍，而藍色增益掉到約 0.22 倍——藍色的相對變化大了一個量級。啟示：(一)檢測藍色或青色標記時，光源色溫的漂移影響最大，必須優先鎖定;(二)廉價 LED 的光譜不連續，其等效色溫會隨老化漂移，應選用高演色性、色溫規格明確的光源並定期以色卡複驗。

20. 灰階世界白平衡的輸出可以寫成 I(x)/mean(I)。請由此推導它在什麼條件下會給出正確答案，並解釋為什麼「單一色調的畫面會失效」這句話並不精確。

    提示與參考解答：設 I(x) = L ⊙ ρ(x)，則 mean(I) = L ⊙ mean(ρ)，故輸出為

ρ(x)/mean(ρ)——光源 L 一定會被完全消掉。因此正確與否只取決於 mean(ρ) 是否為中性灰;若不是，殘留的就是場景自身的色偏，與光源無關。圖 8-16 以 287 個真實影像區塊實測：「平均顏色偏離中性灰的程度」與誤差的 Spearman 相關高達 0.94，而「色相的環狀離散度」只有 –0.34。單一色調之所以常出事，是因為它通常同時帶著強烈色偏，而不是因為色相集中本身有問題;一張色相集中卻接近中性的照片（如淡藍的雪景）其實不會失效。

21. 在圖 8-17 的實測中，灰階世界在暖色主導的場景裡誤差高達 ΔE 21，比完全不做白平衡的 ΔE 4.7 還糟。請解釋為什麼會這樣，並說明這對「要不要在產線上開啟自動白平衡」的決策有何意義。

提示與參考解答：因為灰階世界的誤差來自場景自身的色偏（上一題的推導），而不是來自光源;場景愈偏離中性，它造成的傷害愈大。在暖色主導的場景中，它把「這個場景本來就偏橘」誤判成「光源偏橘」，於是硬把木頭校成了灰。決策意義：自動白平衡是一個「做了假設」的演算法，假設不成立時它會主動製造誤差;在場景內容固定且偏離中性的產線上（例如全部都是紅色番茄），應該關閉自動白平衡，改用灰卡或色卡做一次性的固定校正，並把增益參數鎖死。圖 8-17 中灰卡校正在兩種場景都維持在 ΔE 1.5 以內。

22. 為什麼顏色分割的實務心法是「對 H 設窄門檻、對 V 設寬門檻」?如果有人反過來做，會發生什麼事?請用可量化的方式說明。

提示與參考解答：H 是「是什麼顏色」的主要判準且對光照相對穩定，放寬會把鄰近色相的干擾物一起抓進來;V 會隨光照、陰影、角度大幅變化，設窄會把陰影裡的目標切掉。圖 8-19 在一個已知正確答案的合成場景上掃過所有門檻組合：IoU 在最佳值 97% 以內的「可用區」中，H 半寬只能落在 ±7 到 ±10 之間（約佔掃描範圍的一成），V 的下限卻從 10 到 90 都行（約佔四成）。越界的代價：V 下限拉到 150，陰影裡的目標被切掉，IoU 由 0.94 掉到 0.37;H 放寬到 ±40，干擾物被抓進來，IoU 掉到 0.57。

23. 圖 8-18 的第五步「取最大輪廓並填滿」之後，像素數反而由 57,242 增加到 76,324。這是程式寫錯了嗎?請說明原因，並討論什麼情況下不該做這一步。

提示與參考解答：不是錯誤。目標（紅色托盤）是一個環形，填滿外輪廓等於把中間那只杯子一併算進遮罩，所以面積會變大。要不要填，完全取決於下游任務：若

目的是取得「整組杯盤」的位置與方框以便機械手抓取，填滿是正確的;若目的是量測托盤本身的面積或判斷環形是否完整（例如檢測墊圈的缺口），填滿就會把你要找的缺陷抹掉，必須改用 RETR_CCOMP 或 RETR_TREE 保留內輪廓。這題的要點是：形態學與輪廓處理沒有通用的「正確做法」，只有「與下游任務一致的做法」。

24. 一條番茄分級產線以色相劃分三個等級（界線在 20、40、60 度）。若工廠的照明從 6500 K 換成 2700 K，大約會有多少比例的樣本被改判?請提出兩種補救方案並比較其效果。

提示與參考解答：圖 8-20 的實測顯示，60 顆樣本中有 32 顆（約 53%）被改判——因為量到的色相整條平移了。補救方案一是灰階世界：它把 2700 K 的錯誤由 32 顆降到 18 顆，但在光源本來就正確的 6500 K 反而無中生有製造出 20 顆錯誤（因為番茄色帶本身就是強烈色偏，踩中圖 8-16 的失效條件）。方案二是在畫面中放一張灰卡：錯誤由 32 顆降到 4 顆，且在其餘色溫下都是 0 到 2 顆。結論是方案二明顯較佳;但最佳解其實是第三種——建立封閉光箱把光源鎖死，讓問題根本不發生（呼應 7.6.3 節的層次原則）。

25. (綜合設計題)你要為一條電子零件產線設計色碼辨識系統，需分辨十種顏色的線材，允許的誤判率低於萬分之一。請提出完整的設計，涵蓋光源、色彩空間、門檻策略、校正程序與驗收指標，並說明每一項的理由與對應的量化依據。

提示與參考解答：一、光源：封閉光箱、高演色性 LED、色溫規格明確並記錄批號;理由是圖 8-15 顯示只換光源就能造成 ΔE 10 以上的偏移，而圖 8-20 顯示這足以改判過半樣本。二、校正：每批開工前以標準色卡拍一張，計算並鎖定色彩校正矩陣，關閉相機的自動白平衡與自動曝光;理由是圖 8-17 顯示灰卡校正是唯一在不同場景都穩定的方法（ΔE 1.5 以內），而自動白平衡在色偏場景會主動製造誤差。三、色彩空間：量測與判別用 CIE Lab（感知均勻，ΔE 可直接當門檻），即時分割用 HSV（運算快、門檻直覺）。四、門檻策略：窄 H 寬 V，V 下限只用來排除色相無意義的暗處（圖 8-9、8-19）;十種顏色應先量測彼此的 ΔE，確認最接近的一對也大於 5，否則要求更換線材色號。五、驗收指標：以每種顏色各 1000 個樣本建立混淆矩陣，要求最小類間 ΔE、類內 ΔE 標準差、以及跨班別（早中晚班）的重複性;萬分之一的誤判率需要類間距離至少為類內標準差的四倍以上。六、長期監控：每日

記錄色卡的量測值，一旦漂移超過 ΔE 2 即觸發光源更換——預防永遠比補償便宜。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6(彩色影像處理)。
- Lafer-Sousa, R., Hermann, K. L., & Conway, B. R., Striking Individual Differences in Color Perception Uncovered by "the Dress" Photograph, Current Biology, 25(13), 2015.(本章叫獸開講之科學來源)
- Wallisch, P., Illumination Assumptions Account for Individual Differences in the Perceptual Interpretation of a Profoundly Ambiguous Stimulus, Journal of Vision, 17(4), 2017.(逾一萬三千人的雲雀/夜貓子研究)
- Fairchild, M. D., Color Appearance Models, 3rd ed., Wiley, 2013.
- Ebner, M., Color Constancy, Wiley, 2007.(白平衡演算法之專書)
- OpenCV 官方文件：Color conversions(cv2.cvtColor 完整轉換代碼表)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 9 章 影像分割

本章學習目標

- 說明影像分割在視覺流程中的定位，區分基於不連續性與基於相似性的兩大分割策略。
- 運用全域、Otsu 與自適應門檻法完成灰階影像的二值化，並判斷各自的適用時機。

- 推導並實作 Sobel、Laplacian of Gaussian 與 Canny 邊緣偵測，理解其中的低通與高通運算。
- 以霍夫轉換偵測影像中的直線與圓，說明「參數空間投票」的核心思想。
- 運用區域成長與分水嶺法完成基於相似性的分割，並以標記控制避免過度分割。
- 比較傳統分割與深度學習語意分割的差異，並說明分割成效對後續量測與辨識的影響。

叫獸開講|霍夫轉換：從次原子粒子軌跡到你家的車道偏離警示

1950 年代，粒子物理學面臨一個「幸福的煩惱」。1952 年，Donald Glaser 發明了氣泡室(Bubble Chamber)——讓帶電粒子通過過熱液體時，沿途激發出一串微小氣泡，形成可拍攝的軌跡，這項發明讓他拿下 1960 年諾貝爾物理獎。問題是：氣泡室每天產生成千上萬張底片，每一張都佈滿縱橫交錯的粒子軌跡，而當年只能靠人工用眼睛一條一條描出這些軌跡、量測曲率以推算粒子動量。資料產出的速度，遠遠超過人力分析的速度。

在密西根大學與 Glaser 共事的物理學家 Paul Hough，決定讓機器來做這件苦差事。他要解決的核心難題是：如何讓電腦自動從一堆散亂的亮點中，找出「這些點排成了一條直線」?他的靈感堪稱神來一筆：與其在影像空間裡辛苦地嘗試各種直線去湊點，不如把問題翻轉到「參數空間」。影像中的每一個亮點，都對應到參數空間中的一條曲線(代表所有可能通過該點的直線);若許多亮點共線，它們對應的曲線就會在參數空間中交會於同一點——於是「找直線」這個困難的幾何問題，變成了「找參數空間中的交會熱點」這個單純的投票統計問題。1962 年，Hough 以「辨識複雜圖樣的方法與裝置」為題取得美國專利。

這個為高能物理而生的方法，經過 Rosenfeld 在 1969 年引入主流電腦視覺、以及 Duda 與 Hart 在 1972 年改用極座標表示法(解決了垂直線斜率無限大的問題)

之後，成為影像處理最經典的工具之一，至今累積超過兩千五百篇相關論文。而它最貼近你我生活的化身，就在汽車裡：當代車輛的車道偏離警示系統，核心正是用霍夫轉換即時偵測路面上的車道線;當系統發現車輛正在無預警地偏離兩條偵測到的直線，便發出警報。一個誕生於粒子加速器的數學，六十年後守護著高速公路上每一位駕駛的安全。

這個故事的啟示，是本章的核心心法：分割的本質，常常是一次「觀點的翻轉」。把「找直線」翻轉成「投票統計」，把「切割區域」翻轉成「淹水分界」，把「哪裡是邊界」翻轉成「哪裡不連續」——換一個空間、換一種提問方式，難題就迎刃而解。這與第 6 章「換到頻率域」、第 7 章「換到參數空間解逆問題」的精神一脈相承。

思考題：霍夫轉換把「找直線」變成「在參數空間找熱點」。這個「投票」的思想非常強大——你能想到生活中還有哪些問題，也可以用「每個證據都投一票、票多者當選」的方式來解決?

9.1 什麼是影像分割

9.1.1 分割的定位：承上啟下的轉捩點

影像分割(Image Segmentation)的目標，是把影像劃分成若干個有意義的區域——把前景從背景中分離、把每個物件圈出來、把不同的組織或材質區分開。它是影像處理流程中一個關鍵的轉捩點：在它之前(第 1 到 8 章)，我們處理的是「像素」，做的是強化、還原、色彩調整這類「像素進、像素出」的低階運算;從分割開始，我們的關注點從「像素」升級到「區域」與「物件」，踏入了中階視覺的領域，並為第 13 章的特徵擷取、第 16 章的物件偵測鋪路。



圖 9-1　分割站在低階與中階視覺的交界上

這個轉變的重要性怎麼強調都不為過。回顧第 1 章提過的低、中、高階視覺三層架構：分割正是連接低階(像素)與高階(語意)的橋梁。一張影像若沒有經過分割，電腦看到的永遠只是一大片數字;唯有把「這一塊像素屬於同一個物件」界定出來，後續的「這個物件是什麼」才有意義。

9.1.2 兩大策略：不連續與相似

影像分割的方法雖然繁多，但都可以歸入兩大哲學：

1. 基於不連續性(Discontinuity)：著眼於「區域之間的界線」。它假設不同區域的交界處，像素值會發生劇烈變化，因此設法找出這些劇變的位置——這就是邊緣偵測(9.3 節)與霍夫轉換(9.4 節)的思路。用一句話說：找出「哪裡變了」，變的地方就是邊界。
2. 基於相似性(Similarity)：著眼於「區域內部的一致性」。它假設同一個區域內的像素，在某種性質(灰階、顏色、紋理)上是相似的，因此設法把相似的像素聚集成群——這就是門檻法(9.2 節)、區域成長與分水嶺法(9.5 節)的思路。用一句話說：找出「哪些像素是一夥的」，一夥的就是同一區域。

這兩種策略是一體兩面：區域的邊界，正是相似性中斷之處;而相似的內部，正是被邊界包圍的範圍。實務上，強健的分割系統常常兩者並用——先用邊緣界定大致輪

廓，再用相似性填補區域，或反之。理解這個二分法，就掌握了本章的骨架。

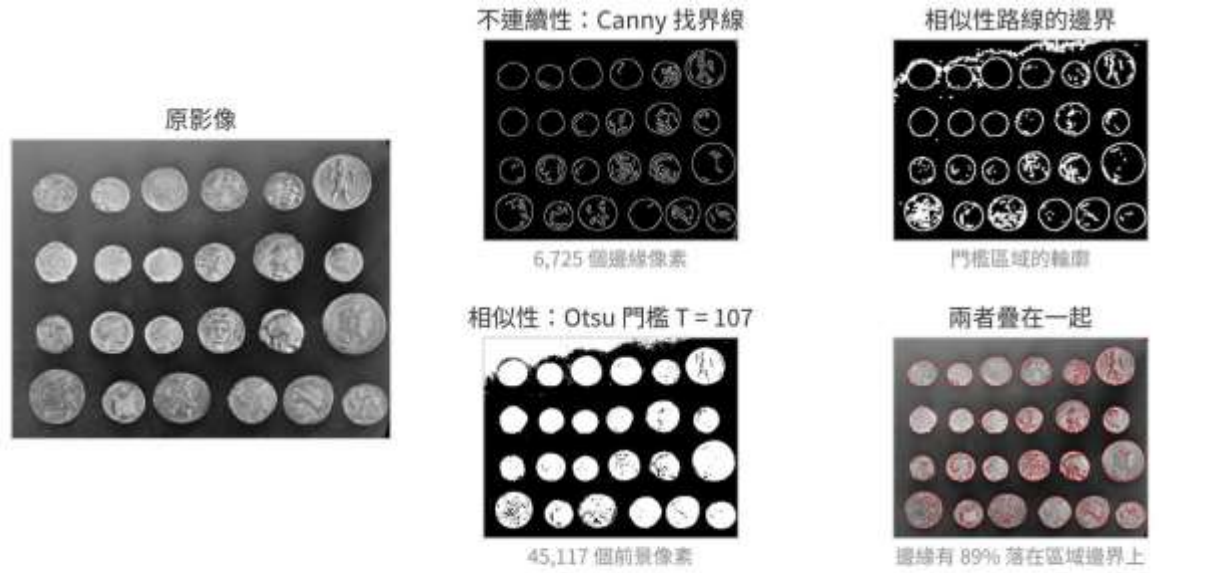


圖 9-2　同一張影像，兩條路線，指的是同一條界線

9.1.3 本章在全書中的位置

分割是承先啟後的樞紐，幾乎用到前八章的所有工具，也支撐著後續各章：

- 與第 5 章(空間域強化)：分割前的品質決定分割的成敗。對比不足的影像要先做直方圖等化或 CLAHE，否則門檻分不開前景背景;雜訊會讓邊緣偵測產生大量假邊，因此 Canny 的第一步就是第 5 章的高斯平滑。
- 與第 6 章(頻率域)：邊緣是高頻、平坦區是低頻;Canny 內含「先低通平滑、再高通求梯度」的雙重濾波，正是第 6.6.4 節詳述的觀念。分割在本質上就是在區分影像中不同頻率特性的區域。
- 與第 7 章(影像還原)：還原品質直接影響分割——雜訊使邊緣偵測產生假邊、模糊使邊緣定位偏移。第 7.7 節已量化過這個連結。
- 與第 8 章(色彩處理)：彩色影像的分割常在 HSV 空間以顏色為依據(第 8.6 節的顏色分割)，或轉到 Lab 空間計算感知色差。

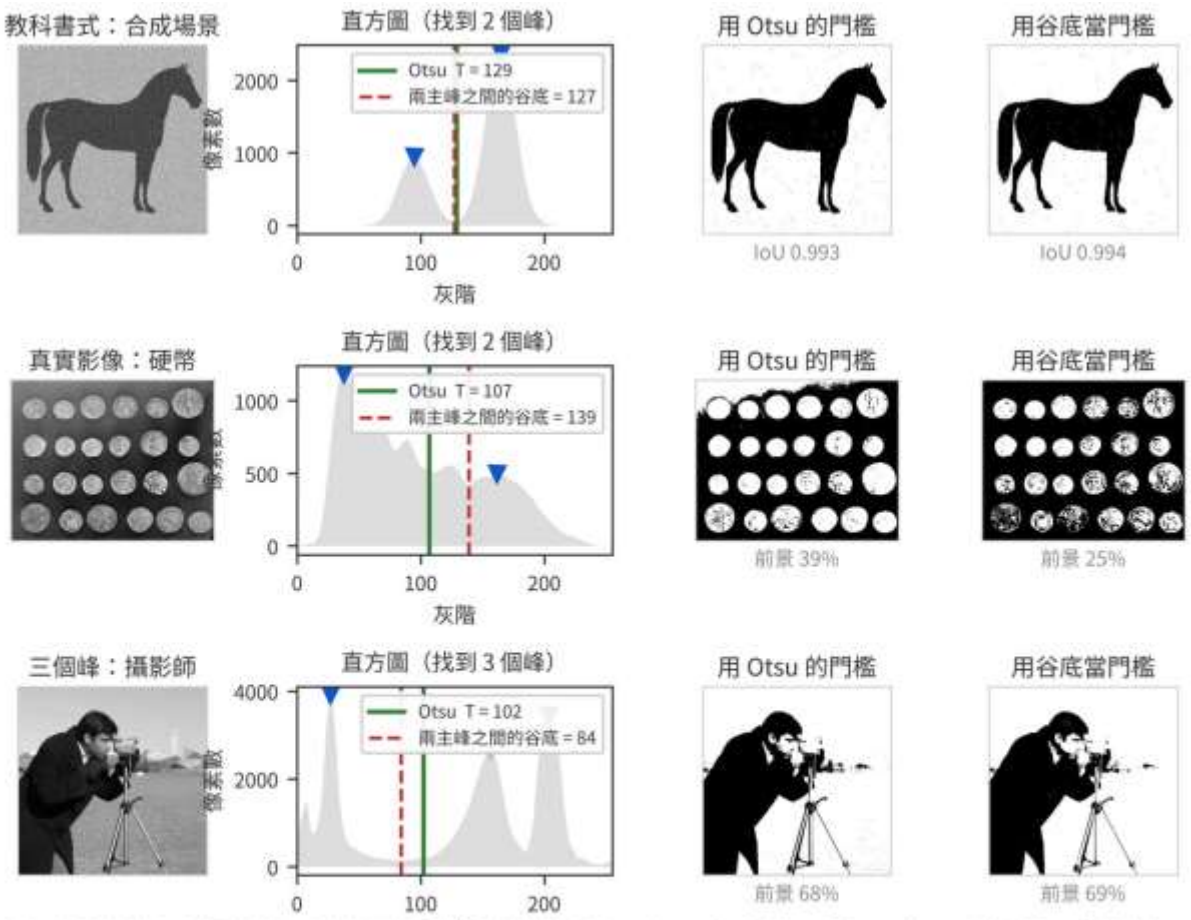
- 與第 10 章(二值影像處理)：門檻分割的輸出是二值影像，緊接著就用型態學清理雜點、連通元件標記來計數與量測——第 10 章是本章的直接下游。
- 與第 13 章(特徵擷取)：分割出的輪廓可計算形狀特徵(面積、周長、圓形度、Hu 矩)，供物件辨識使用。
- 與第 16 章(物件偵測與分割):YOLO 的實例分割、U-Net 的語意分割，是本章傳統方法的深度學習後繼者;9.6 節會說明兩者的分工與互補。
- 與第 17 章(邊緣部署)：傳統分割運算量小、不需訓練資料，在算力受限或樣本稀少的場合仍是首選——這是它在深度學習時代依然重要的原因。

9.2 門檻法：最簡單的分割

9.2.1 全域門檻

門檻法(Thresholding)是最直觀的分割：選一個灰階值 T 作為分界，大於 T 的像素歸為前景(設為白)、小於 T 的歸為背景(設為黑)。當前景與背景的灰階差異明顯、打光均勻時，一個固定的全域門檻就能漂亮地切開兩者。它的關鍵完全繫於 T 的選擇——選對了乾淨俐落，選錯了不是前景殘缺就是背景混入。

如何選 T?最理想的情況是影像的直方圖呈現雙峰(bimodal)：前景與背景各自形成一個峰，兩峰之間的谷底就是理想門檻。這也把我們帶回第 5 章——直方圖不只是診斷曝光的工具，更是選擇分割門檻的依據。



第一排是刻意造出來的理想情形，兩個峰正好落在設時設定的灰階 95 與 165 上，谷底 127 與 Otsu 的 129 只差 2 階——課本畫的就是這一排。第二排是真實影像：谷底在 139，Otsu 卻選 107，差了 32 階。因為 Otsu 最大化的是類間變異數，這個量會把每一類的像素數與分散程度一起算進去，不是單純找最低點。而且看右邊兩格就知道，Otsu 偏離谷底的方向是對的：Otsu 的門檻把硬幣完整留下，谷底那個門檻反而把硬幣表面的刻紋整片挖空。「Otsu 就是找谷底」只在兩類勢均力敵時才成立，而且谷底也未必是比較好的答案。第三排有 3 個峰，Otsu 仍然照給一個數字而且不會警告你——演算法永遠會回答，但答案不一定有意義。

圖 9-3　雙峰、谷底與 Otsu：課本畫的那一排只是三種情形之一

9.2.2 Otsu 自動門檻

人工看直方圖選門檻既主觀又難以自動化。日本學者大津展之(Nobuyuki Otsu)於 1979 年提出的方法，能自動找出最佳門檻：它遍歷所有可能的門檻值，對每一個門檻把像素分成前景與背景兩群，計算兩群的「類間變異數」(between-class variance)，選擇使類間變異數最大的那個門檻。直覺上，最好的門檻應該讓兩群「各自內部愈集中、彼此之間離得愈遠」——類間變異數最大，正對應這個目標。Otsu 法有一個漂亮的性質：最大化類間變異數，等價於最小化類內變異數，而前者的計算可以用遞推快速完成，因此效率極高。

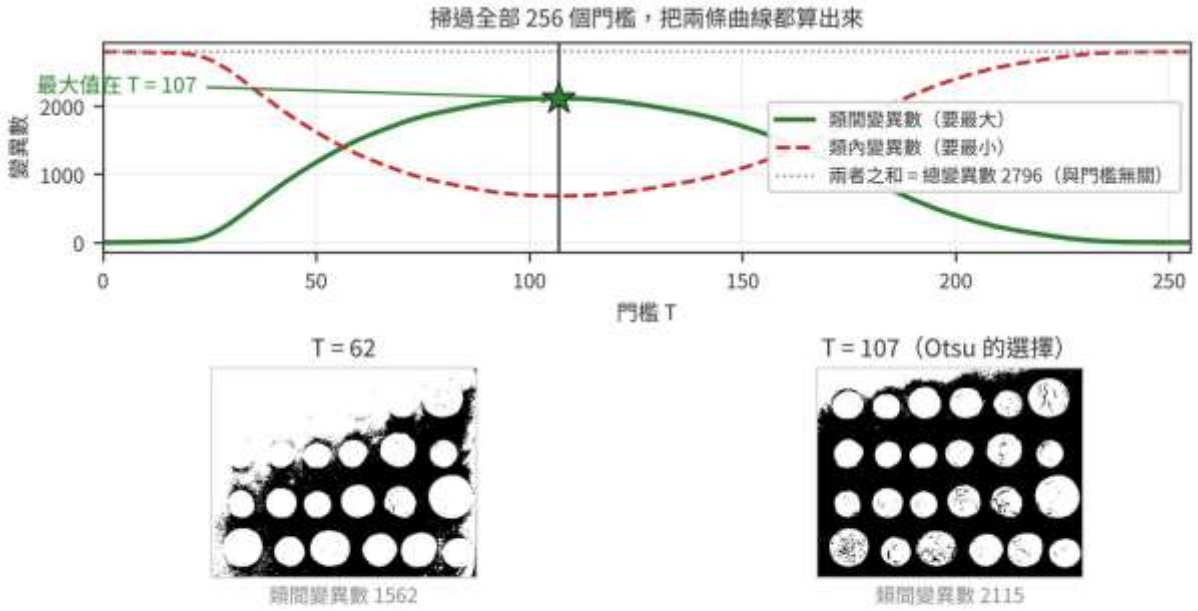


圖 9-4　Otsu 到底在最大化什麼：把 256 個門檻通通算一遍

Otsu 法的前提是直方圖確實雙峰。若前景只佔極小面積(直方圖被背景的單峰主宰)、或前景背景灰階重疊嚴重，Otsu 就會失準——這時要嘛先做前處理拉開對比，要嘛改用下一節的自適應門檻。

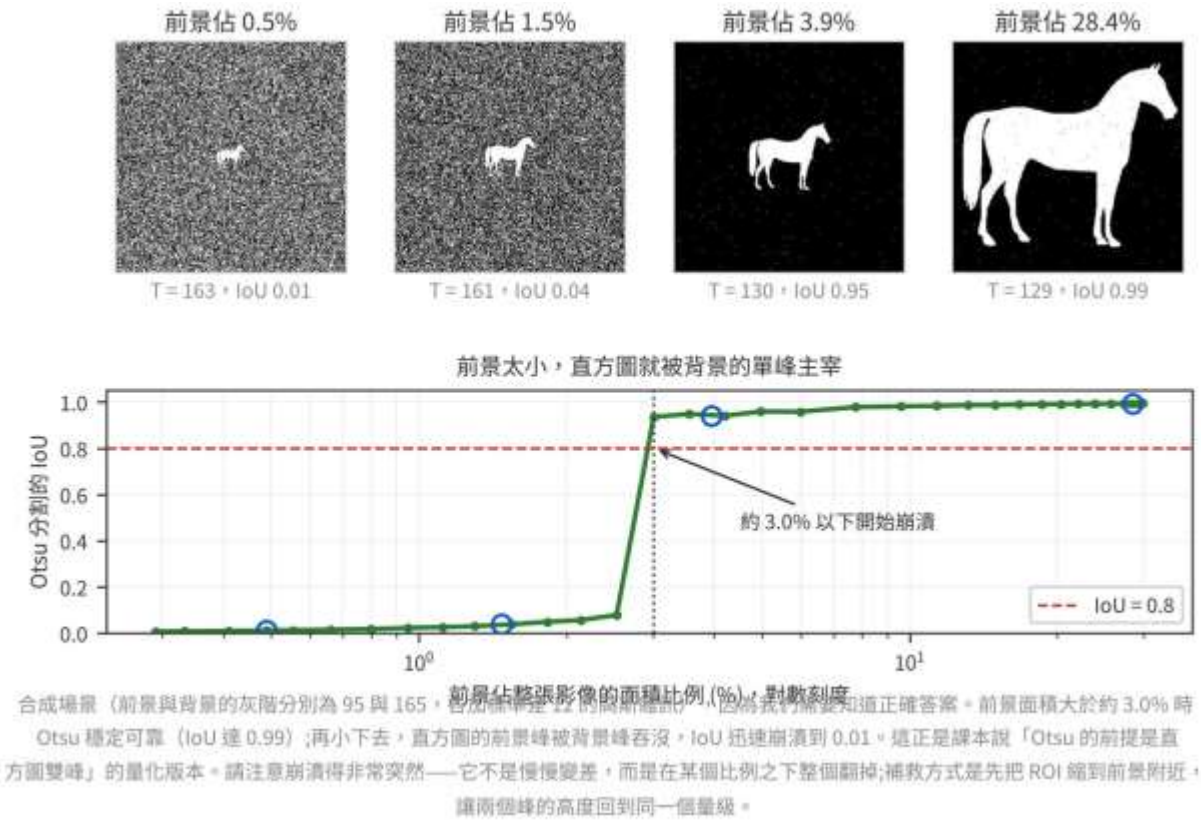
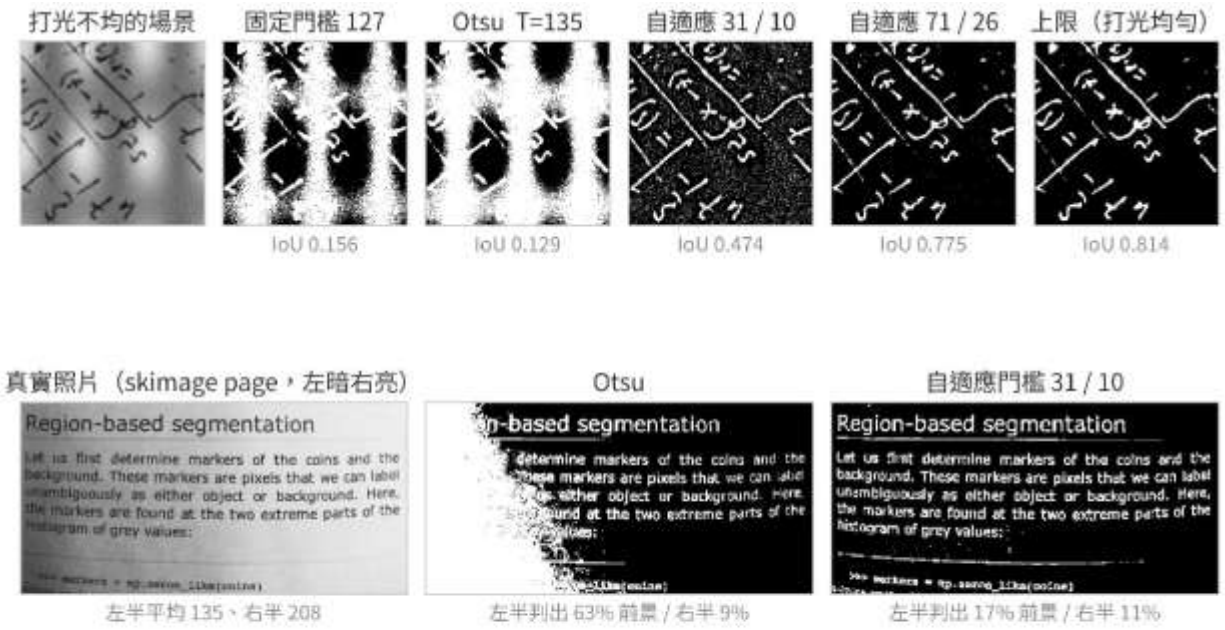


圖 9-5　前景小到什麼程度，Otsu 就會崩潰

9.2.3 自適應門檻

全域門檻有個致命弱點：它假設整張影像的最佳門檻都一樣。但真實場景常有打光不均——一張紙的左半邊在陽光下、右半邊在陰影中，任何單一門檻都會顧此失彼(這正是第 8 章「洋裝效應」在灰階世界的翻版)。自適應門檻(Adaptive Threshold)的解法是：不用一個全域門檻，而是為每個像素、依據它周圍鄰域的局部統計量(平均值或高斯加權平均)各自計算一個門檻。如此一來，亮區用高門檻、暗區用低門檻，打光不均的問題迎刃而解。文件掃描 App 把泛黃、有陰影的紙張轉成黑白分明的掃描檔，靠的就是自適應門檻。



上排是合成場景，所以可以算 IoU：固定門檻 0.16、Otsu 0.13、程式 9-1 預設參數的自適應門檻 0.47、調過參數的 0.77。請特別注意最後一格——就算把打光弄成完全均勻、再用事後挑的最佳門檻 143，IoU 也只有 0.81;剩下的損失來自筷頭模糊與雜訊，跟門檻法一點關係都沒有。調過參數的自適應門檻已經拿到這個上限的 95%。先量出上限，才知道一個方法還有多少改善空間;否則很容易把物理上做不到的事怪到演算法頭上。下排是真實照片，沒有正確答案，改量左右半邊各判出多少前景：文字分布大致均勻，Otsu 卻判出 63% 對 9%，自適應門檻則是 17% 對 11%。

圖 9-6　一條水平線救不了一條斜坡——順便量出門檻法的上限

程式 9-1 三種門檻法

```
import cv2

gray = cv2.imread("document.jpg", cv2.IMREAD_GRAYSCALE)

# --- 全域固定門檻 ---
_, fixed = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# --- Otsu 自動門檻(回傳自動選出的 T) ---
T, otsu = cv2.threshold(gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu 選出的門檻:", T)

# --- 對付雜訊:先高斯模糊再 Otsu(第 5 章前處理) ---
blur = cv2.GaussianBlur(gray, (5, 5), 0)
_, otsu2 = cv2.threshold(blur, 0, 255,
                         cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 自適應門檻:對付打光不均 ---
adap = cv2.adaptiveThreshold(gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, blockSize=31, C=10)

cv2.imwrite("th_fixed.png", fixed)
cv2.imwrite("th_otsu.png", otsu)
cv2.imwrite("th_adaptive.png", adap)
```

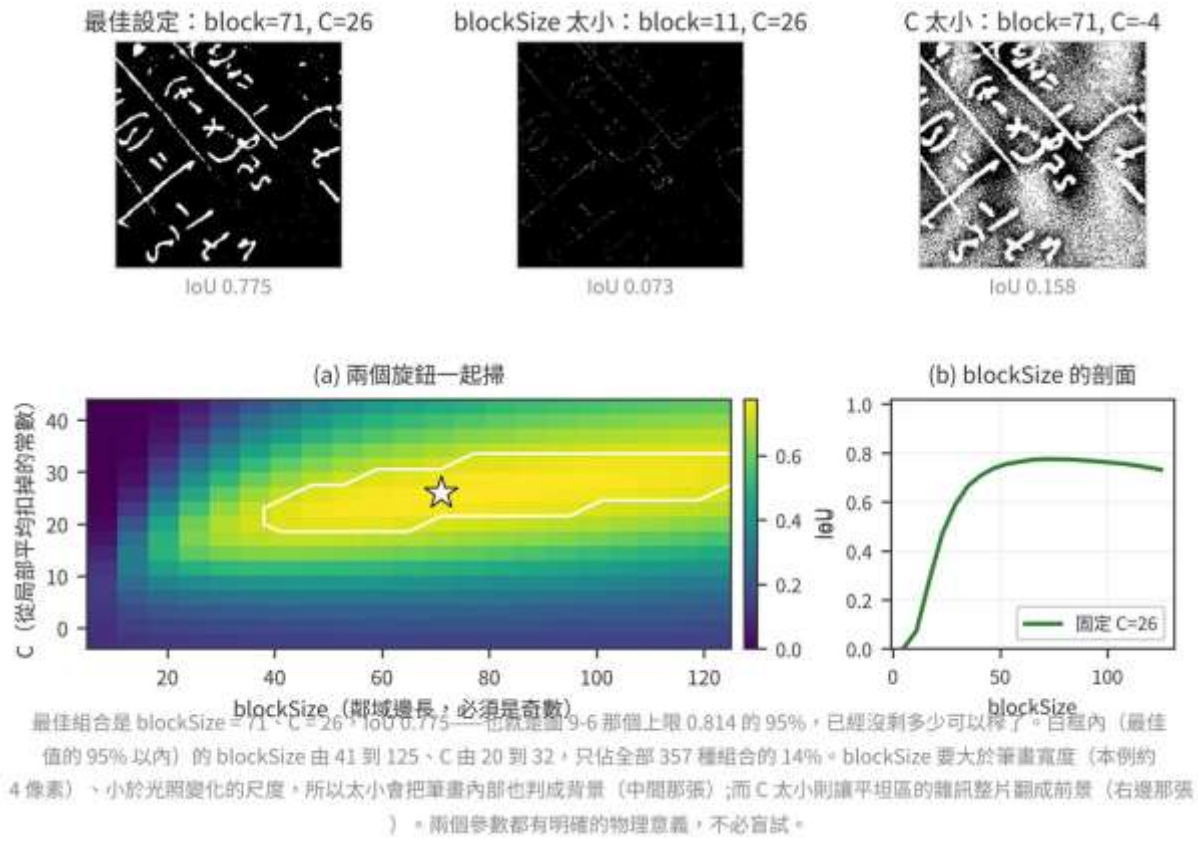


圖 9-7　自適應門檻的兩個旋鈕各有明確的物理意義

9.2.4 推導：Otsu 為什麼能在一瞬間試完 256 個門檻

9.2.2 節說 Otsu「遍歷所有可能的門檻值」。乍聽之下這是一件昂貴的事：每換一個門檻，就要把整張影像重新分成兩群、重算兩群的平均與變異數。一張一千萬像素的影像乘上 256 個門檻，那是二十幾億次運算。但 Otsu 實際上只要掃一次直方圖就算完了，而且與影像的大小完全無關。這一節說明為什麼。

第一個關鍵是：所有的計算都只需要直方圖。把灰階 i 的像素比例記為 p(i)。門檻取 T 時，背景群是灰階 0 到 T，前景群是 T+1 到 255，於是背景的比例 w0(T) = p(0) + p(1) + … + p(T)，背景的平均 m0(T) = [0·p(0) + 1·p(1) + … + T·p(T)] / w0(T)。請注意這兩個式子的分子都是「累積和」：w0(T) 是 p 的前綴和，m0 的分子是 i·p(i) 的前綴和。前綴和可以用遞推一次算完——w0(T) = w0(T−1) + p(T)，只要一次加法。前景那一群則由全體減去背景得到，w1(T) = 1 − w0(T)，而 m1(T) = [mT − Σ i·p(i)] / w1(T)，其中 mT 是整張影像的平均。所以掃過 256 個門檻的總成本是 256 次加法與

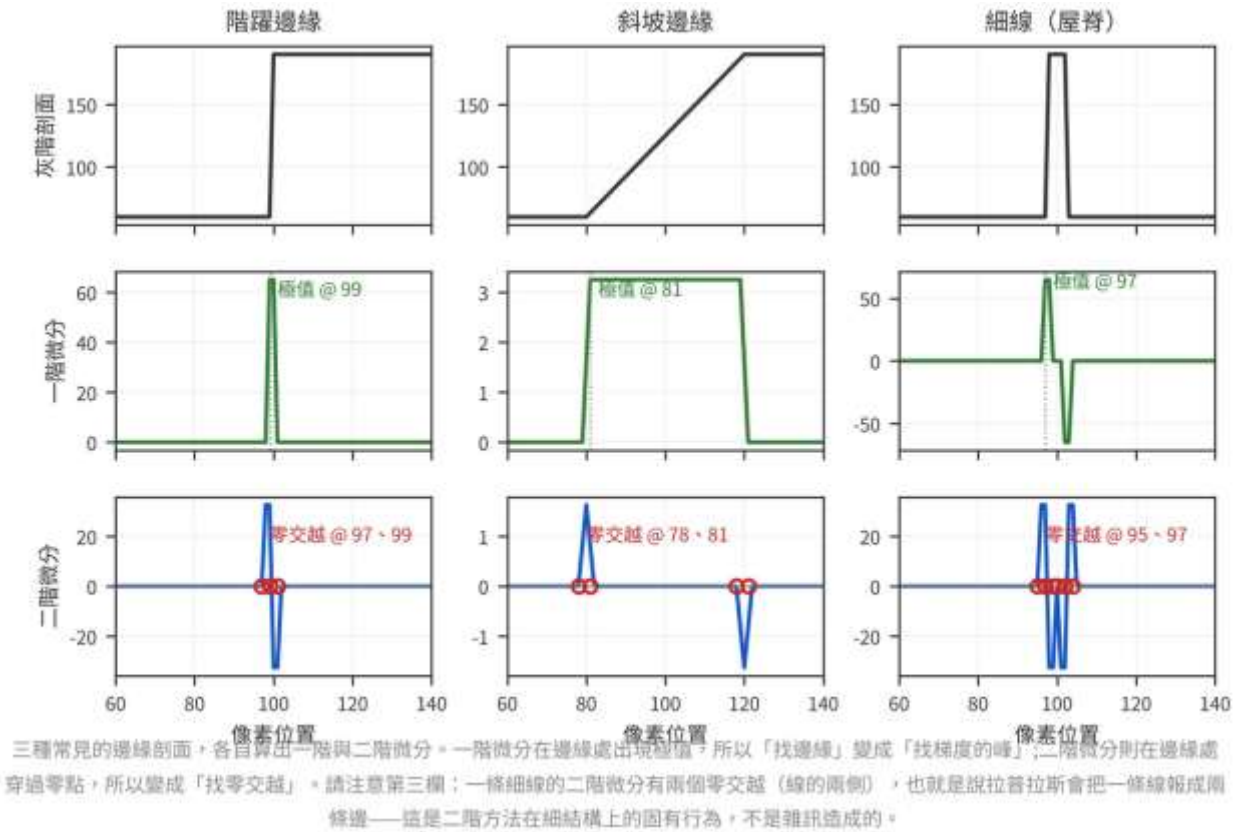
256 次除法，而建直方圖本身是 O(N)。這就是為什麼 cv2.threshold 加上 THRESH_OTSU 幾乎不花時間。

第二個關鍵是那個漂亮的恆等式。把整張影像的總變異數記為 σ^2，它可以被分解成兩項：$\sigma^2 = \sigma^2_{within}(T) + \sigma^2_{between}(T)$，其中 $\sigma^2_{between}(T) = w0 \cdot w1 \cdot (m0 - m1)^2$。圖 9-4 把這兩條曲線都實際畫了出來，它們的和確實是一條與門檻無關的水平線（該圖量到的總變異數是 2796）。這個恆等式的意義非常實用：既然兩者相加是常數，那麼「讓類內變異數最小」與「讓類間變異數最大」就是同一件事。而類間變異數只需要 w0、w1、m0、m1 四個已經算好的量，類內變異數卻要回頭掃每一個像素——Otsu 選了便宜的那一個。

第三件事是這個推導順便揭露的一個常見誤解。$\sigma^2_{between} = w0 \cdot w1 \cdot (m0 - m1)^2$ 這個式子裡，除了「兩群離得多遠」的 $(m0 - m1)^2$ 之外，還乘上了「兩群各佔多少」的 $w0 \cdot w1$。這一項在 $w0 = w1 = 0.5$ 時最大，也就是說 Otsu 內建一個偏好：它傾向把影像切成面積相當的兩半。圖 9-3 第二排量到的現象就是這麼來的——硬幣影像的直方圖谷底在 139，Otsu 卻選了 107，因為往下移可以讓兩群的面積更接近。所以「Otsu 就是自動找谷底」這句話並不正確;更精確的說法是「Otsu 在找一個能把影像切得最開、同時兩邊份量又不會太懸殊的門檻」。圖 9-5 的實驗則是這個偏好的反面：當前景只佔 3% 以下時，$w0 \cdot w1$ 這一項逼著門檻往背景那邊靠，分割就整個垮掉。

9.3 邊緣偵測

邊緣是影像中灰階劇烈變化之處，通常對應物體的輪廓、表面的轉折或深度的不連續。從第 6 章的頻率觀點看，邊緣就是高頻;而偵測高頻變化的數學工具，是微分。



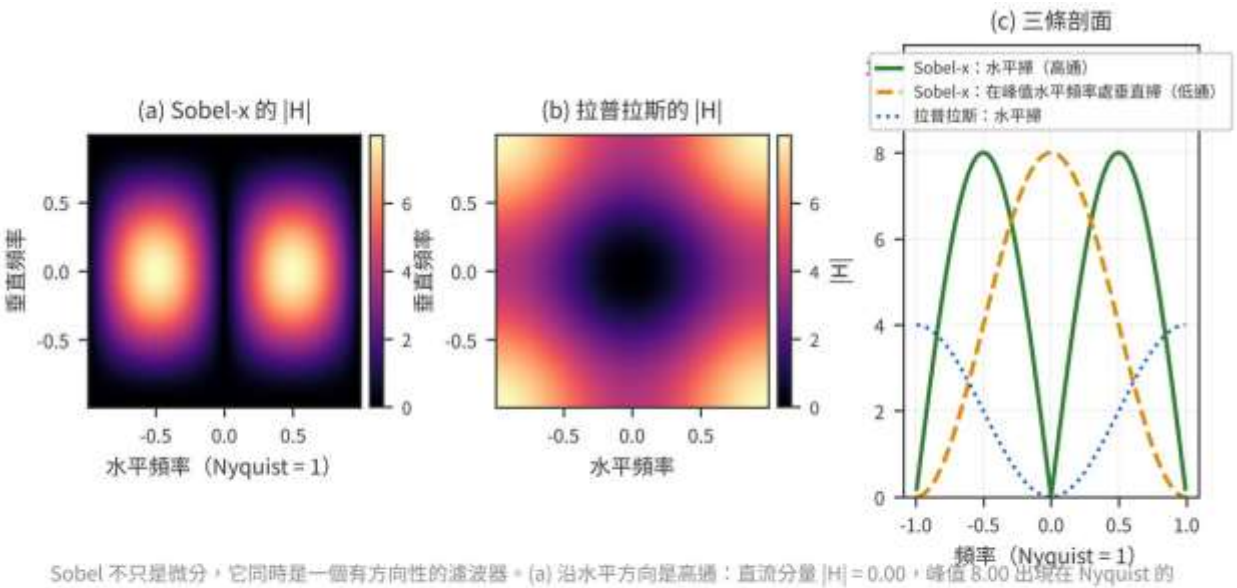
三種常見的邊緣剖面，各自算出一階與二階微分。一階微分在邊緣處出現極值，所以「找邊緣」變成「找梯度的峰」；二階微分則在邊緣處穿過零點，所以變成「找零交越」。請注意第三欄：一條細線的二階微分有兩個零交越（線的兩側），也就是說拉普拉斯會把一條線報成兩條邊——這是二階方法在細結構上的固有行為，不是雜訊造成的。

圖 9-8　一階微分找峰，二階微分找零交越

9.3.1 梯度與一階微分：Sobel

影像的梯度是一個向量，由水平方向與垂直方向的偏微分組成，其大小代表灰階變化的劇烈程度、方向代表變化最快的方向(垂直於邊緣走向)。在離散的數位影像上，微分以差分近似，並包裝成卷積核。最常用的是 Sobel 運算子：它在計算某方向差分的同時，對垂直方向做加權平均，兼具「求梯度」與「抗雜訊」兩個功能——這也呼應第 6 章的觀點：Sobel 是一個帶有平滑效果的方向性高通濾波器。取得水平梯度 Gx 與垂直梯度 Gy 後，梯度大小為兩者平方和開根號，對它做門檻化即可得到邊緣圖。

相關的運算子還有 Scharr(比 Sobel 更精確的梯度近似，適合小核)、Prewitt(較簡單的差分)。它們的共同侷限是：對雜訊敏感(微分會放大雜訊)，且邊緣往往偏粗、需要後續細化。



Sobel 不只是微分，它同時是一個有方向性的濾波器。(a) 沿水平方向是高通：直流分量 |H| = 0.00，峰值 8.00 出現在 Nyquist 的 0.50 倍處。沿垂直方向卻是低通——固定在同一個水平頻率，垂直頻率走到 Nyquist 的一半時響應由 8.00 掉到 4.05（剩 51%），走到 Nyquist 則恰好歸零。這不是巧合：Sobel 可分解為水平的 [−1, 0, 1] 與垂直的 [1, 2, 1]，而 [1, 2, 1] 的頻率響應 2 + 2 cos(ω) 在 ω = π 處正是 0。這正是 9.3.1 節說「Sobel 在求差分的同時對垂直方向做加權平均」的頻率域說法，也是它比單純的差分抗雜訊的原因。(b) 拉普拉斯是各向同性的高通，沒有平滑，所以更怕雜訊。

圖 9-9　Sobel 是一個有方向性的濾波器：一邊高通、一邊低通

9.3.2 二階微分：Laplacian 與 LoG

一階微分在邊緣處產生極值，二階微分則在邊緣處產生零交越(zero-crossing)——由正轉負的那一點，正是邊緣的精確位置。拉普拉斯運算子(Laplacian)是二階微分的離散近似，定位精準且各向同性(不分方向)，但對雜訊極度敏感(第 5、6 章都提過，二階微分把雜訊放大得比一階更兇)。

解法是先平滑再微分：把高斯平滑與拉普拉斯結合，就是高斯拉普拉斯(Laplacian of Gaussian,LoG)，又因其剖面形似而俗稱「墨西哥帽」。LoG 先用高斯壓制雜訊(低通)，再用拉普拉斯找零交越(高通)——又是一個「低通加高通」的組合，與第 6 章的敘述完全一致。LoG 也是第 13 章 SIFT 特徵偵測的理論前身(SIFT 用高斯差 DoG 來近似 LoG)。

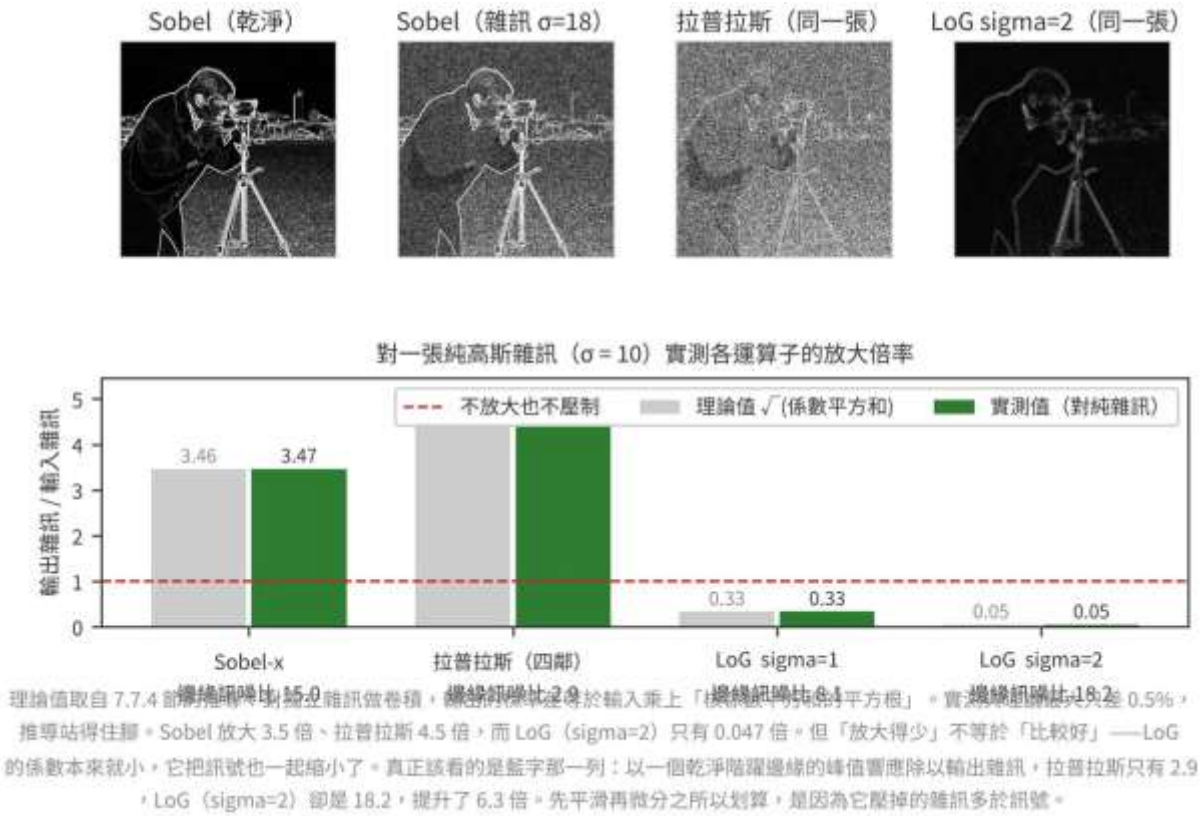


圖 9-10　微分把雜訊放大了多少：理論值與實測值對照

9.3.3 Canny：邊緣偵測的黃金標準

1986 年 John Canny 提出的邊緣偵測器，以「好的偵測、好的定位、單一響應」三個準則為設計目標，至今仍是最廣泛使用的邊緣偵測方法。它是一條精心設計的四步驟管線，每一步都呼應本書前面的觀念：

3. 高斯平滑：先用高斯濾波器壓制雜訊(第 5 章低通)，避免後續微分放大雜訊。sigma 的大小決定了對雜訊的容忍度與細節的保留度，是最關鍵的參數。
4. 計算梯度：以 Sobel 求出每個像素的梯度大小與方向(第 9.3.1 節)。
5. 非極大值抑制(Non-Maximum Suppression)：沿著梯度方向，只保留局部最大的像素，把粗邊緣細化成單像素寬的線。這是 Canny 邊緣俐落的關鍵。
6. 雙門檻與遲滯連接(Hysteresis)：設高、低兩個門檻。高於高門檻的必定是邊緣(強邊);低於低門檻的必定不是;介於兩者之間的(弱邊)，只有在「連接到強邊」

時才保留。這個巧妙的設計，既能避免雜訊造成的假邊，又能保住真實但較弱的邊緣不致斷裂。

Canny 的成功，正是「把多個簡單觀念(平滑、微分、抑制、連接)精心組合成一條管線」的典範——這也是叫獸開講「觀點翻轉」之外，工程上另一種創造價值的方式：巧妙的組合。

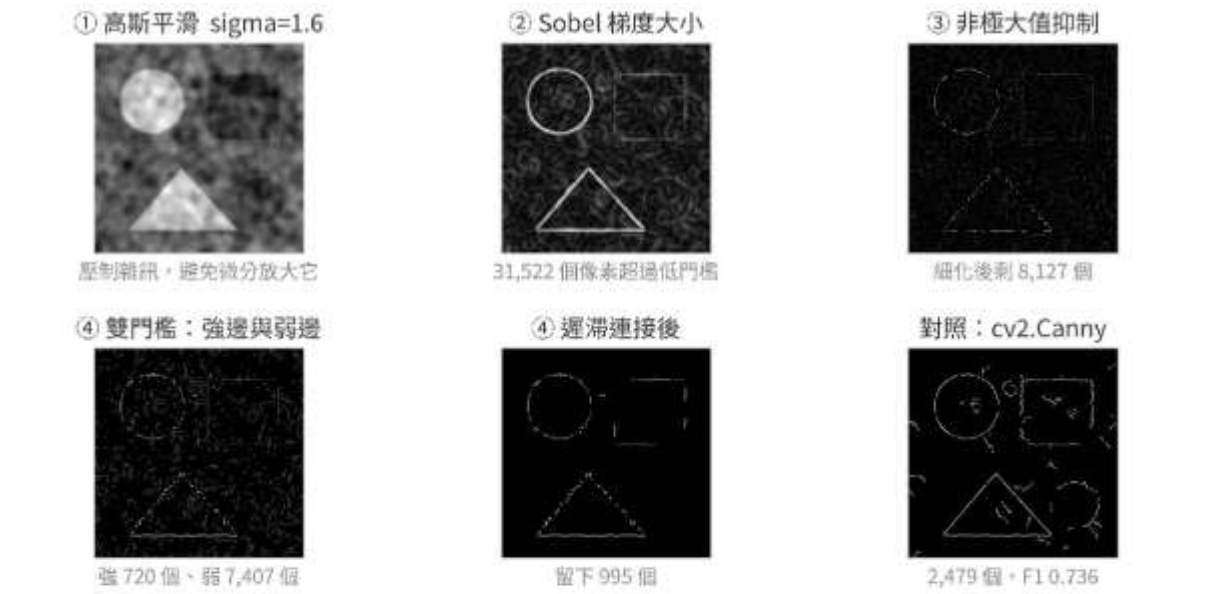


圖 9-11　Canny 的四個步驟，每一步的像素數都在上面

程式 9-2 Sobel、LoG 與 Canny 邊緣偵測

```
import cv2, numpy as np

gray = cv2.imread("parts.jpg", cv2.IMREAD_GRAYSCALE)

# --- Sobel 一階梯度 ---
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
sobel = cv2.convertScaleAbs(mag)

# --- Laplacian of Gaussian(先平滑再二階微分) ---
blur = cv2.GaussianBlur(gray, (3, 3), 0)
log = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
log = cv2.convertScaleAbs(log)

# --- Canny(黃金標準) ---
edges = cv2.Canny(gray, threshold1=80, threshold2=160)
```

```
# 自動決定門檻:以中位數為基準(實務常用技巧)
v = np.median(gray)
lo = int(max(0, 0.66 * v))
hi = int(min(255, 1.33 * v))
edges_auto = cv2.Canny(gray, lo, hi)

cv2.imwrite("edge_sobel.png", sobel)
cv2.imwrite("edge_log.png", log)
cv2.imwrite("edge_canny.png", edges)
```

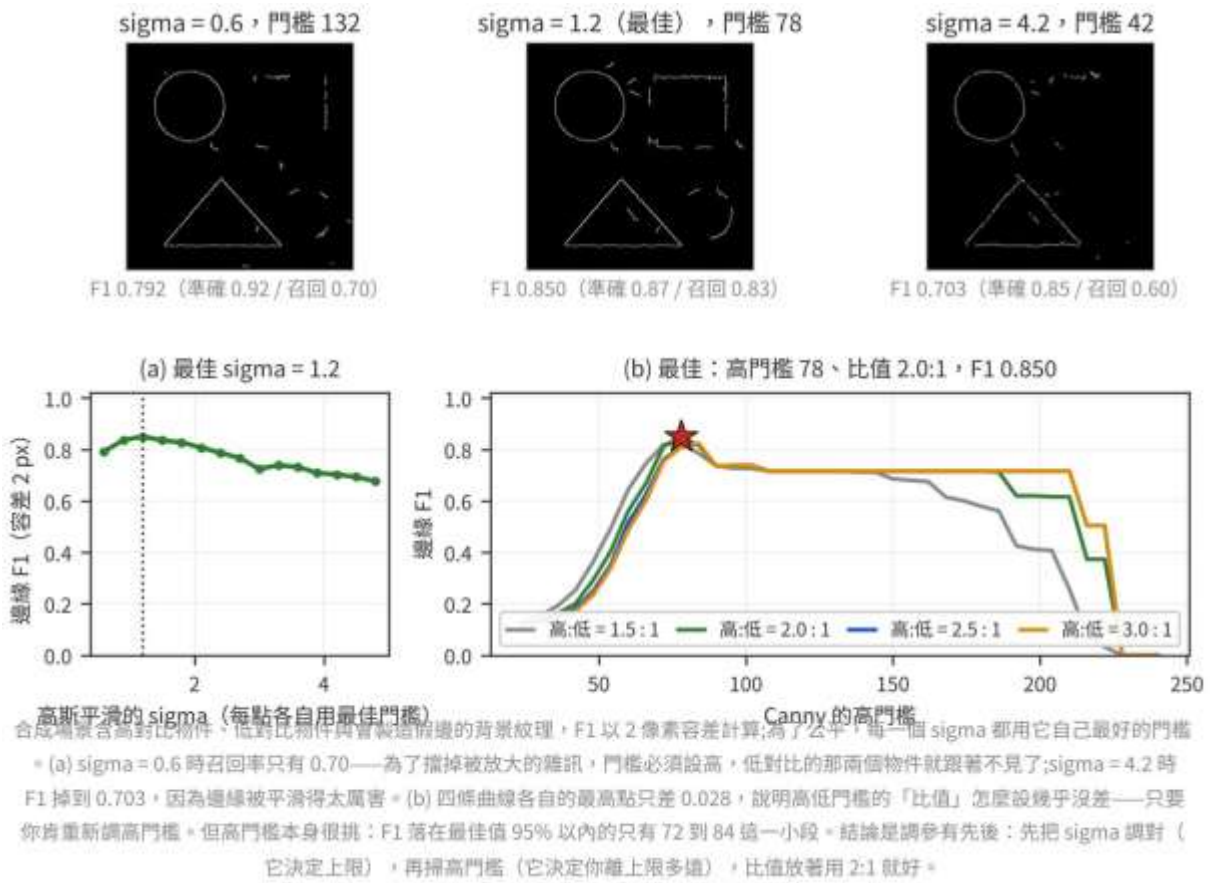


圖 9-12　Canny 的三個旋鈕：sigma 決定上限，門檻決定你離上限多遠

9.3.4 推導：非極大值抑制為什麼需要梯度方向

9.3.3 節把非極大值抑制描述成「沿著梯度方向，只保留局部最大的像素」。很多人讀過就跳過去了，但這一步其實是 Canny 四個步驟裡最精巧的一個，而且「為什麼非得沿梯度方向不可」值得算清楚。

先看問題出在哪裡。一條真實的邊緣經過高斯平滑之後，不會是一條一像素寬的線，而是一道有寬度的斜坡。對寬度為 w 的斜坡求梯度，得到的是一個寬度也大約是 w 的隆起——若直接用門檻切，一條邊緣會變成好幾個像素寬的粗帶子，而且寬度還

隨門檻高低而變。這對後續的霍夫轉換（每個像素投一票）與輪廓追蹤都是災難。我們要的是「把這道隆起壓成它的脊線」。

關鍵的幾何事實是：梯度向量永遠垂直於邊緣的走向。這可以直接從定義看出來——沿著邊緣走，灰階不變，所以方向導數為零;而梯度與某方向的內積就是該方向的方向導數，因此梯度必定垂直於邊緣。於是「跨過邊緣」的方向就是梯度方向，而隆起的剖面正好出現在這個方向上。非極大值抑制做的事就是：在每個像素處沿著梯度方向往前後各看一格，若自己不是三者中最大的就歸零。剩下的那些點，正是隆起的脊。

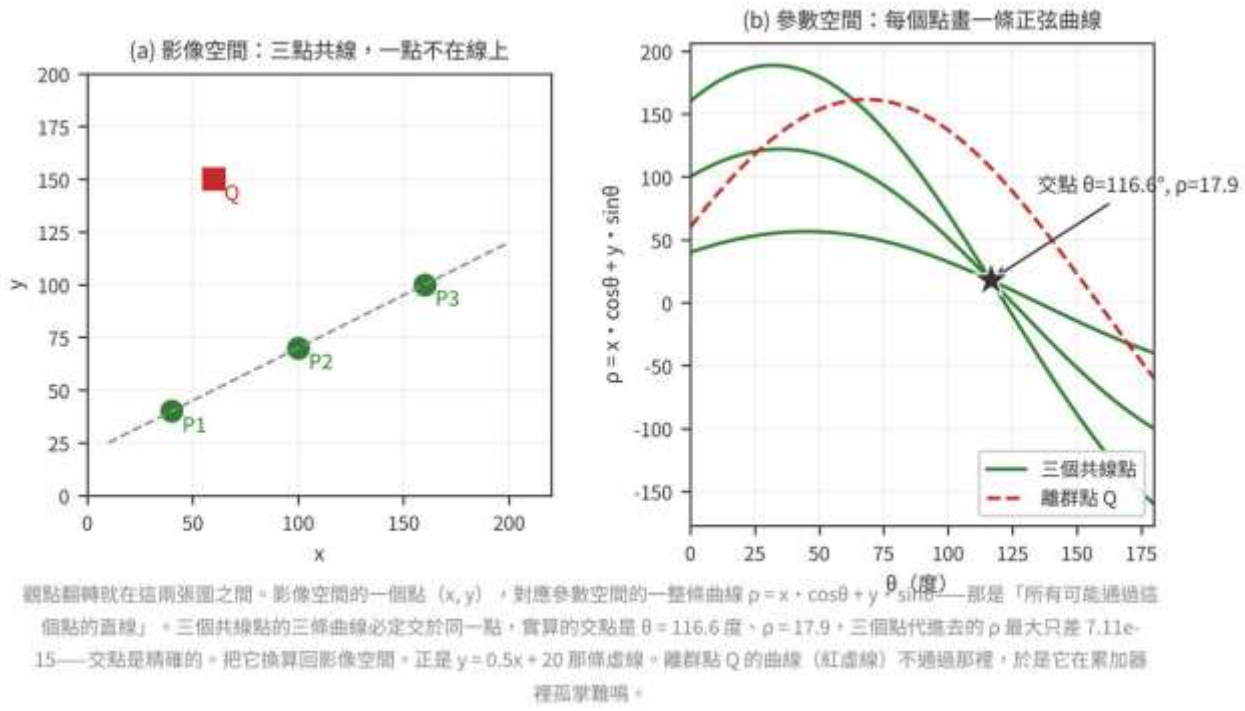
方向為什麼要量化成四個？因為離散影像上「往前走一格」只有八個鄰居，對應四條軸線：0 度（水平）、45 度、90 度、135 度。實作上把梯度角度除以 45 再四捨五入取模 4，就決定了要跟哪兩個鄰居比。這個量化會帶來最多 22.5 度的方向誤差，代價是邊緣位置可能偏半個像素;想更精準的話可以沿梯度方向做線性內插，取出兩個次像素位置的梯度值再比較，這也是多數高品質實作的做法。

效果有多大，可以直接量。圖 9-11 在同一張合成影像上實測：梯度圖中超過低門檻的像素有 31,522 個，經過非極大值抑制之後只剩 8,127 個，一口氣少掉 74%，而邊緣的位置完全沒有移動。換句話說，這一步丟掉的全是「同一條邊緣的重複報告」。也正因為輸出是單像素寬，下一步的雙門檻與遲滯連接才能用「連通性」來判斷弱邊的去留——若邊緣是粗的，連通性就失去意義了。四個步驟環環相扣，少一個都不行。

9.4 霍夫轉換：參數空間的投票

9.4.1 直線偵測的核心思想

邊緣偵測告訴我們「哪些像素在邊緣上」，但它給的是一堆散亂的邊緣點，並沒有告訴我們「這些點排成了一條直線」。霍夫轉換(Hough Transform)接手這個工作，而它的思想正是叫獸開講中那次精彩的觀點翻轉。



觀點翻轉就在這兩張圖之間。影像空間的一個點 (x, y)，對應參數空間的一整條曲線 ρ = x · cosθ + y · sinθ——那是「所有可能通過這個點的直線」。三個共線點的三條曲線必定交於同一點，實算的交點是 θ = 116.6 度、ρ = 17.9，三個點代進去的 ρ 最大只差 7.11e-15——交點是精確的。把它換算回影像空間，正是 y = 0.5x + 20 那條虛線。離群點 Q 的曲線（紅虛線）不通過那裡，於是它在累加器裡孤掌難鳴。

圖 9-13　霍夫轉換的觀點翻轉：一個點變成一條正弦曲線

關鍵在於視角互換。在影像空間中，一條直線由通過它的所有點定義;霍夫轉換反過來問：對於影像中的每一個邊緣點，有哪些直線可能通過它?答案是無限多條，這些直線的參數在「參數空間」中構成一條曲線。於是每個邊緣點都在參數空間投下一條曲線(等於為所有可能通過它的直線各投一票)。如果影像中真的有一條直線，那麼這條線上的所有點，投出的曲線都會通過參數空間中同一個點——那個點的參數，正是這條直線的參數。於是「找直線」變成「在參數空間找得票最高的點」。

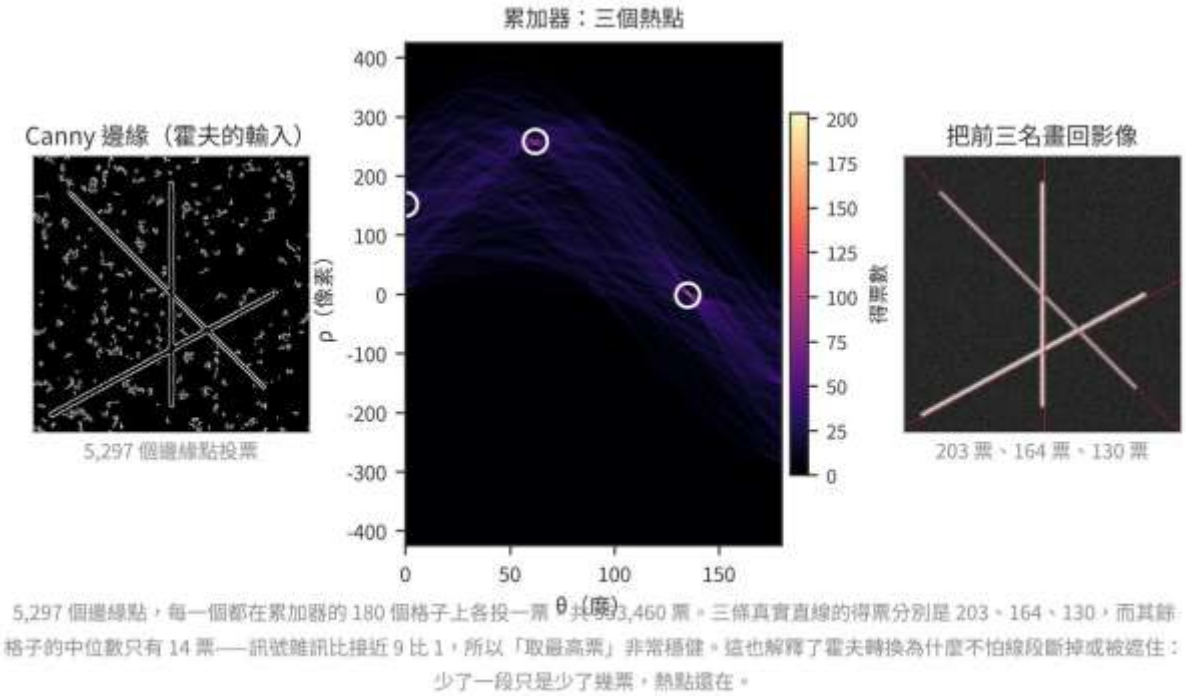
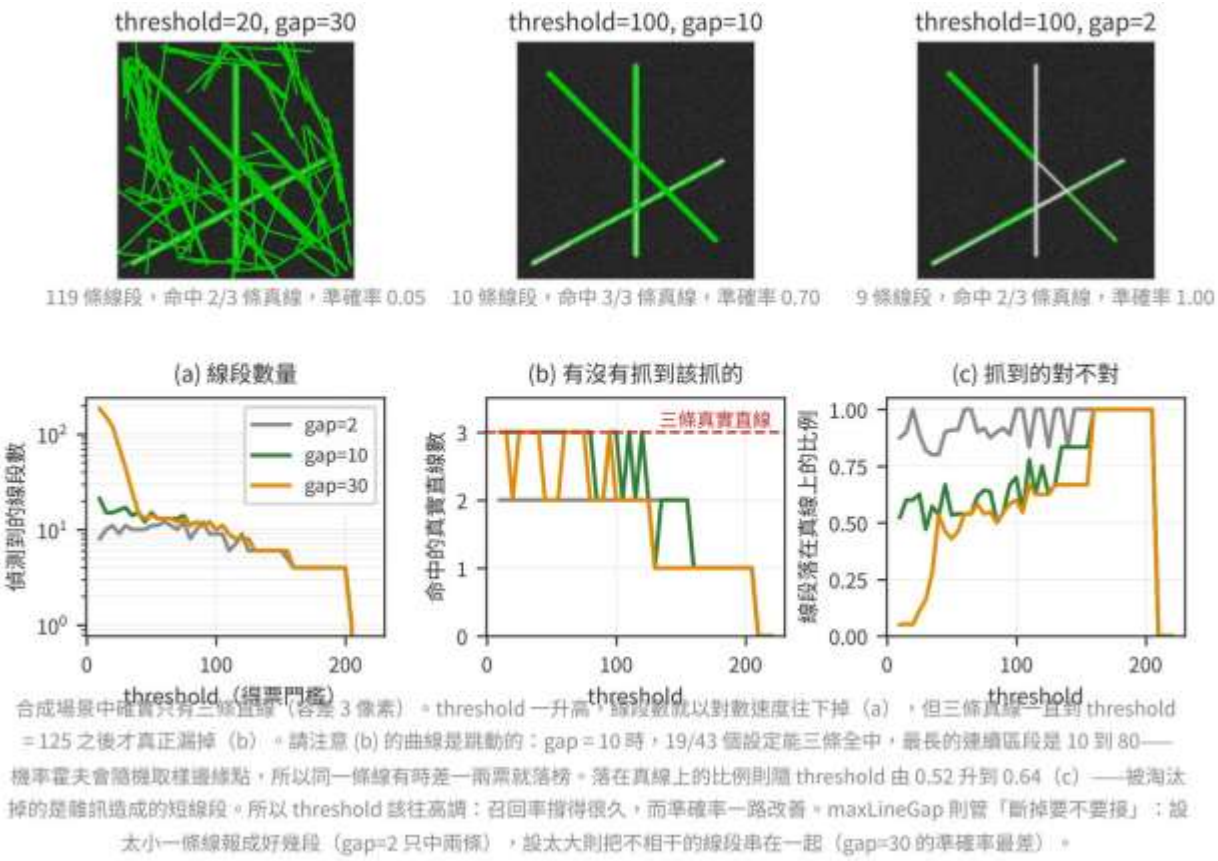


圖 9-14　累加器裡的三個熱點，以及它們的票數

實務上不用斜截式 y = mx + b，因為垂直線的斜率是無限大無法表示。改用 Duda 與 Hart 在 1972 年引入的極座標式：ρ = x·cosθ + y·sinθ，其中 ρ 是原點到直線的垂直距離、θ 是該垂線的角度。每個邊緣點在 (ρ, θ) 參數空間中對應一條正弦曲線，共線的點其正弦曲線交會於一點。參數空間被離散化成一個個格子(稱為累加器，accumulator)，每條曲線經過的格子就加一票，最後票數超過門檻的格子就是偵測到的直線。



合成場景中確實只有三條直線（容差 3 像素）。threshold 一升高，線段數就以對數速度往下掉（a），但三條真線一直到 threshold = 125 之後才真正漏掉（b）。請注意 (b) 的曲線是跳動的：gap = 10 時，19/43 個設定能三條全中，最長的連續區段是 10 到 80——機率霍夫會隨機取樣邊緣點，所以同一條線有時差一兩票就落榜。落在真線上的比例則隨 threshold 由 0.52 升到 0.64（c）——被淘汰掉的是雜訊造成的短線段。所以 threshold 該往高調：召回率撐得很久，而準確率一路改善。maxLineGap 則管「斷掉要不要接」：設太小一條線報成好幾段（gap=2 只中兩條），設太大則把不相干的線段串在一起（gap=30 的準確率最差）。

圖 9-15　機率霍夫的三個旋鈕各管什麼

9.4.2 圓偵測與霍夫梯度法

同樣的投票思想可以推廣到任何能用少數參數描述的形狀。圓需要三個參數：圓心 (a, b) 與半徑 r，因此累加器是三維的。若直接對三維空間投票，運算與記憶體成本都很高。

OpenCV 採用的霍夫梯度法(Hough Gradient Method)是一個聰明的加速：它利用一個幾何事實——圓上每一點的梯度方向都指向(或背離)圓心。因此不必盲目地為所有可能的圓心投票，而是讓每個邊緣點只沿著自己的梯度方向投票，票數自然集中到真正的圓心;先在二維空間定出圓心，再回頭決定半徑。這把三維投票拆解成兩個較低維的步驟，大幅提升效率。它廣泛用於偵測圓形物件——瞳孔、硬幣、球、儀表指針的軸心、工件上的圓孔。

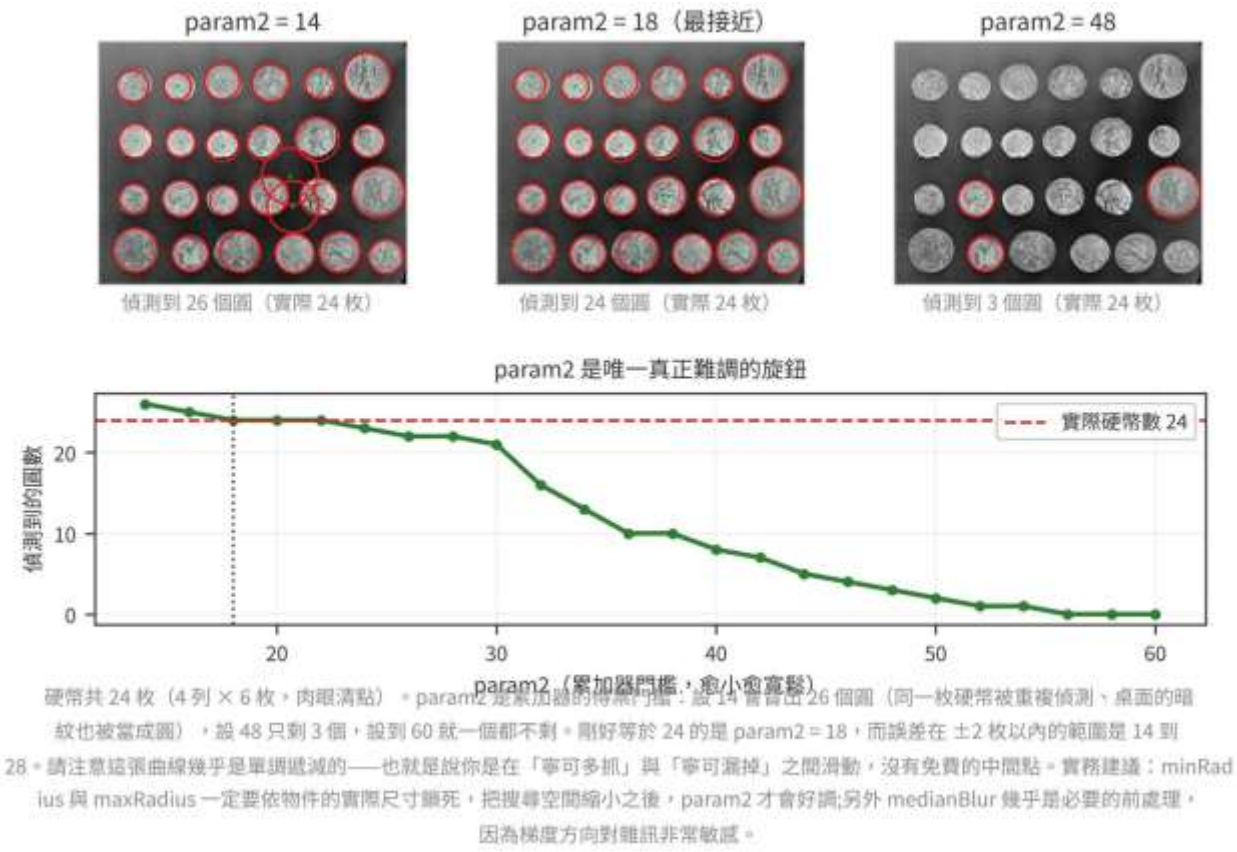


圖 9-16　param2 是霍夫圓偵測唯一真正難調的旋鈕

程式 9-3 霍夫直線與圓偵測

```
import cv2, numpy as np

gray = cv2.imread("road.jpg", cv2.IMREAD_GRAYSCALE)
edges = cv2.Canny(gray, 80, 160)                # 霍夫的輸入是邊緣圖

# --- 標準霍夫直線(回傳 rho, theta) ---
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)

# --- 機率霍夫直線(回傳線段端點,實務更常用) ---
segments = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50,
                           minLineLength=50, maxLineGap=10)
img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
if segments is not None:
    for x1, y1, x2, y2 in segments[:, 0]:
        cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

# --- 霍夫梯度法偵測圓 ---
coins = cv2.imread("coins.jpg", cv2.IMREAD_GRAYSCALE)
coins = cv2.medianBlur(coins, 5)                # 先去雜訊很重要
circles = cv2.HoughCircles(coins, cv2.HOUGH_GRADIENT, dp=1,
            minDist=30, param1=100, param2=30,
            minRadius=10, maxRadius=80)
if circles is not None:
    for a, b, r in np.uint16(np.around(circles))[0]:
        cv2.circle(img, (a, b), r, (0, 0, 255), 2)
```

```
cv2.imwrite("hough_lines.png", img)
```

9.4.3 推導：累加器要多大，投票要花多少錢

霍夫轉換的思想很美，但它是一個會吃記憶體的方法。9.4.2 節說「若直接對三維空間投票，運算與記憶體成本都很高」，這一節把那個「高」算成具體的數字，你才知道什麼時候可以用、什麼時候要換方法。

先算直線。極座標式 ρ = x·cosθ + y·sinθ 裡，θ 的範圍是 0 到 180 度，ρ 的範圍則是從 −D 到 +D，其中 D 是影像的對角線長度。若 θ 取 1 度一格、ρ 取 1 像素一格，累加器就是 2D × 180 格。以圖 9-14 的 300×300 場景為例，D = 425，累加器是 850 × 180 = 153,000 格，用 int32 存不到 0.6 MB，完全不是問題。就算換成 1080p 的畫面，D = 2203，累加器 4406 × 180 = 793,080 格，也只有 3 MB。投票的成本則是「邊緣點數 × θ 的格數」：圖 9-14 量到 5,297 個邊緣點，乘上 180 就是 953,460 次累加——這在現代處理器上是毫秒等級。

再算圓，差別立刻出現。圓需要三個參數（圓心 a、b 與半徑 r），累加器是三維的 H × W × R。以 coins 影像（303 × 384）為例，若半徑限定在 12 到 36 之間（R = 25），累加器是 303 × 384 × 25 = 2,908,800 格，已經是直線的 16 倍。若不限定半徑，讓 r 從 1 跑到 192，就是 22,339,584 格、89 MB，而且每個邊緣點要投的票也從 180 票暴增到數萬票。這就是為什麼 cv2.HoughCircles 的 minRadius 與 maxRadius 不是「可選參數」而是「必填參數」——不鎖住半徑，這個方法在實務上根本跑不動。

霍夫梯度法的加速就建立在一個幾何事實上：圓上任何一點的梯度方向，必定通過圓心。因此不必讓每個邊緣點對所有可能的 (a, b, r) 投票，只要沿著自己的梯度方向畫一條射線，對線上的點投票即可。這把三維的投票降成二維（先只找圓心），圓心確定之後再用「到圓心的距離直方圖」決定半徑。代價是它對梯度方向的準確度非常敏感——這也是為什麼程式 9-3 在偵測圓之前一定要先做 medianBlur：梯度方向被雜訊擾動幾度，射線就射偏了，票就投到別處去。圖 9-16 的實驗顯示，即使前處理做

對了，param2（累加器門檻）仍然是唯一難調的旋鈕：在 24 枚硬幣的影像上，誤差 ±2 枚以內的 param2 只有 14 到 28 這一小段。

最後是一個推廣的原則。霍夫轉換能處理任何「可以用少數參數描述」的形狀，但成本隨參數個數指數成長：直線兩個參數、圓三個、橢圓五個。橢圓的累加器在實務上已經大到不可行，所以現實中偵測橢圓多半改用「先找輪廓、再最小平方擬合」（第 13 章的 cv2.fitEllipse），而不是硬投票。判斷原則因此很清楚：參數少於三個時霍夫很好用，三個時要想辦法降維（像霍夫梯度法那樣），超過三個就該換一條路了。

9.5 基於區域的分割

9.5.1 區域成長

區域成長(Region Growing)體現了「基於相似性」的策略。它從一個或多個種子點出發，檢查種子的鄰居：若鄰居與種子(或目前區域)的性質夠相似(灰階差在容許範圍內)，就把鄰居併入區域，再從新併入的像素繼續向外檢查，像墨水在紙上暈開一樣，直到再也沒有相似的鄰居可併入。它的優點是能保證區域的連通性、對定義明確的區域效果好;缺點是結果高度依賴種子的選擇與相似度門檻，且對雜訊敏感。

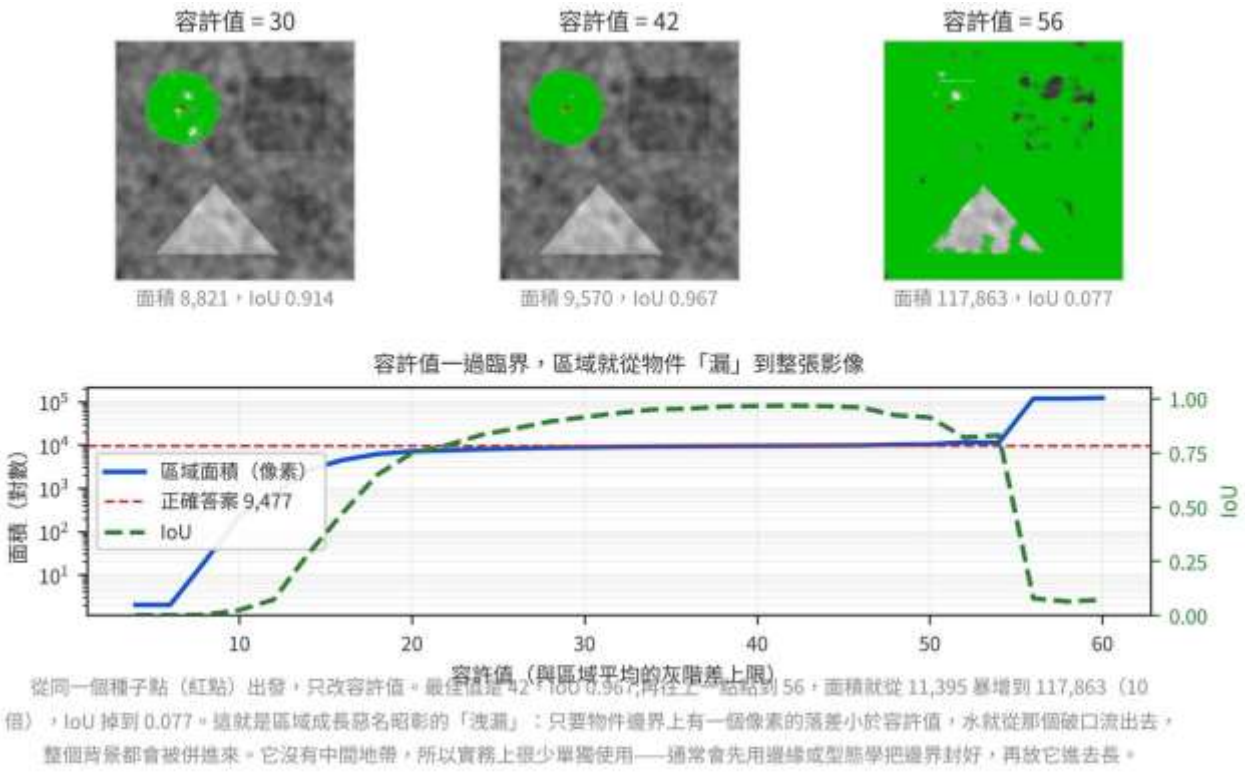
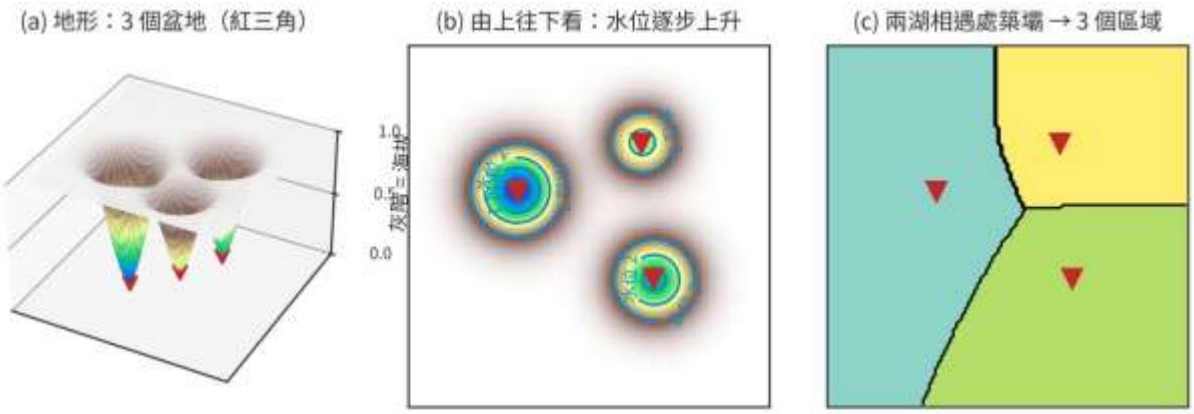


圖 9-17　區域成長的洩漏：容許值差一點，面積差十倍

9.5.2 分水嶺法

分水嶺法(Watershed)有一個極為優美的比喻。把灰階影像想像成一片地形：灰階值就是海拔，亮處是山峰、暗處是山谷(通常對梯度影像操作，則邊緣是山脊、區域內部是盆地)。現在想像從每個盆地的最低點開始注水，水位緩緩上升，不同盆地的水各自成湖;當兩個湖的水即將相遇時，在交界處築起一道堤壩——這些堤壩，就是分割的邊界。「分水嶺」之名由此而來。



把灰階值當成海拔：暗處是盆地，亮處是山脊。從每個局部極小值開始注水。水位慢慢升高;當兩個湖快要相連時，就在交界處築一道壩，那道壩就是分割邊界。本圖的地形有 3 個局部極小值，分水嶺就得到 3 個區域——這正是問題所在：區域數完全由局部極小值的個數決定，而真實影像的梯度圖上，雜訊會製造成千上萬個局部極小值。下一張圖會量給你看那個數字有多誇張。

圖 9-18　把灰階當地形：注水、相遇、築壩

分水嶺法天生擅長分開「相互接觸、黏在一起」的物件——這是門檻法最頭痛的情況(兩個相黏的細胞或零件，門檻分割會把它們當成一個)。但它有一個惡名昭彰的問題：過度分割(over-segmentation)。因為影像中每一個微小的局部極小值(常由雜訊造成)都會形成一個獨立的盆地，結果把影像切得支離破碎。

解法是標記控制的分水嶺(Marker-Controlled Watershed)：不讓水從所有局部極小值自由湧出，而是由使用者(或前處理演算法)指定少數幾個「確定是前景」與「確定是背景」的標記，只從這些標記處注水。這徹底馴服了過度分割，是實務上真正可用的版本。它的標準流程會用到第 10 章的型態學運算(侵蝕求前景標記、膨脹求背景標記、距離轉換分離相黏物件)，因此本節也是通往下一章的橋梁。

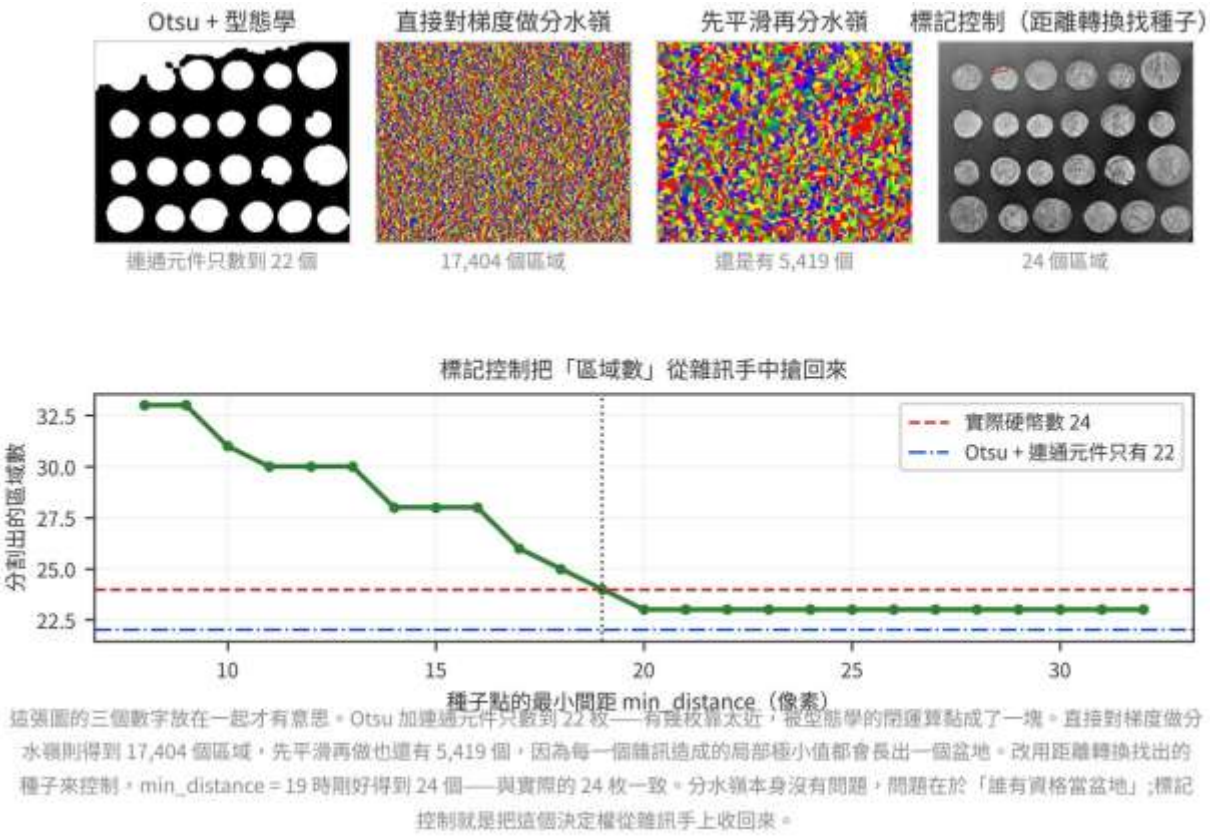


圖 9-19　一萬七千個區域與二十四枚硬幣：標記控制的威力

程式 9-4 標記控制的分水嶺分割

```
import cv2, numpy as np

img = cv2.imread("touching_coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Otsu 二值化(第 9.2 節)
_, thresh = cv2.threshold(gray, 0, 255,
            cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# 2. 型態學去雜點 + 確定背景(第 10 章)
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# 3. 距離轉換 → 確定前景(分開相黏物件的關鍵)
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

# 4. 未知區域 = 背景 - 前景
unknown = cv2.subtract(sure_bg, sure_fg)

# 5. 標記並執行分水嶺
n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
```

```
markers[unknown == 255] = 0
markers = cv2.watershed(img, markers)
img[markers == -1] = [0, 0, 255]           # 分界線標紅

print("分離出的物件數:", n - 1)
cv2.imwrite("watershed.png", img)
```

9.6 從傳統分割到深度學習語意分割

本章至此介紹的都是傳統方法：它們基於人工設計的規則(門檻、梯度、投票、淹水)，不需要訓練資料，運算量小，可解釋性強。但它們有一個共同的天花板：它們分割的是「低階特徵的一致性」，而不是「語意」。門檻法能把亮的和暗的分開，卻不知道分出來的是貓還是狗;霍夫轉換能找出直線，卻不理解那是車道線還是欄杆。

深度學習帶來了語意分割(Semantic Segmentation)的躍進：為影像中的每一個像素賦予一個類別標籤(這是路面、這是行人、這是天空)。2015 年的 U-Net(原為醫學影像設計)以其對稱的編碼器-解碼器結構與跳接(skip connection)，成為語意分割的里程碑;此後的發展延伸出實例分割(Instance Segmentation，不僅分類每個像素，還區分同類的不同個體，例如畫面中的三個人各自獨立)，代表如 Mask R-CNN;近年更有 SAM(Segment Anything Model)這類基礎模型，能對幾乎任何物件進行提示式分割。第 16 章會實作 YOLO 的分割功能。

那麼傳統方法過時了嗎?並沒有。兩者是互補而非取代的關係：



圖 9-20　傳統方法與深度學習的分工象限

比較面向	傳統分割(本章)	深度學習語意分割
依據	人工規則(灰階、梯度、形狀)	從標註資料學到的語意特徵
訓練資料	不需要	需要大量像素級標註(成本高)
運算量	小，CPU 即可即時	大，通常需要 GPU
可解釋性	高，每步驟皆可理解	低，黑箱
語意理解	無，只分低階特徵	有，能辨識類別
適用場合	受控環境、規律背景、樣本稀少	複雜場景、需要語意、資料充足

表 9-1 傳統分割與深度學習語意分割的比較

工程判斷原則：在受控的工業環境(打光穩定、背景單純)，傳統方法往往又快又準又省——一個 AOI 檢測站用 Otsu 加連通元件就能數清零件，何必動用需要標註數千張影像、還得配 GPU 的深度模型?反之，在開放世界的複雜場景(自駕車看到的街景)，語意的理解無可取代，深度學習才是正解。成熟的工程師懂得在兩者間依場景選擇，甚至混用——例如用深度模型做粗略的語意定位，再用傳統方法做精確的邊界細化與量測。

9.7 應用案例

9.7.1 案例一：零件計數與尺寸量測

這是機器視覺最經典的入門任務，也是本章前五節的綜合演練。情境：一盤散落的螺絲或墊圈，要自動數出數量並量測每一個的尺寸。標準流程串起了本書多章：先用第 5 章的前處理(灰階、CLAHE)改善對比 → 用 9.2 節的 Otsu 門檻二值化 → 用第 10 章的型態學去除雜點、填補孔洞 → 用連通元件標記或輪廓偵測數出物件並計算每個的面積、周長、最小外接矩形。若零件相互接觸，則改用 9.5 節的分水嶺分開它們。這個管線是無數工廠檢測線的骨幹。

9.7.2 案例二：車道線偵測

呼應叫獸開講，這是霍夫轉換最著名的應用。情境：從車前攝影機的影像中即時偵測車道線。完整流程：轉灰階並用第 8 章的顏色資訊強化黃白線 → 用 9.3 節的 Canny 找出邊緣 → 設定「感興趣區域」(ROI，只保留路面梯形區域，排除天空與周邊干擾，這用到第 2 章的 ROI 概念)→ 用 9.4 節的機率霍夫轉換 HoughLinesP 找出線段 → 依斜率把線段分成左右車道並各自擬合成一條線 → 判斷車輛是否偏離。

這個案例也適合討論傳統方法的極限：在標線清晰、光線良好的高速公路上，霍夫轉換又快又可靠;但遇到標線磨損、雨天反光、強烈陰影、彎道(直線模型失效)時就力有未逮——這正是為什麼當代自駕系統改用深度學習來做車道線與可行駛區域的語意分割。從霍夫轉換到深度學習，一部車道偵測的演進史，恰好就是本章 9.1 到 9.6 節的縮影。

9.7.3 案例三：醫學影像的病灶分割

分割在醫療領域至關重要：從 X 光、CT、MRI 中把腫瘤、器官、血管的範圍精確圈出，是後續量測體積、規劃手術、追蹤療效的基礎。傳統上，醫師或工程師會用區域成長(從醫師點選的種子開始擴張)、門檻法(對造影劑增強的區域)、或分水嶺法來輔

助分割。

但醫學影像分割也最能凸顯分割品質的重量。一個腫瘤邊界若分割得太大，可能導致切除過多健康組織;太小則可能殘留病灶。這裡有兩個貫穿本書的呼應：其一，第 7 章談過的還原品質直接影響此處——雜訊與模糊會讓邊界偵測失準;其二，第 7 章「還原 vs. 腦補」的倫理警覺在此同樣適用——若用深度學習模型自動分割病灶，模型「腦補」出的邊界是否可信?正因如此，醫學影像的自動分割結果通常僅作為輔助，最終仍須醫師確認。U-Net 當年正是為醫學影像分割而生，如今深度學習與傳統方法在此領域廣泛並用：深度模型提供快速的初步分割，傳統方法(如主動輪廓、分水嶺)做精細的邊界調整。

9.7.4 案例四：視訊中的移動物件分割

前三個案例都是分割單張影像，第四個案例引入時間維度，銜接第 12 章。情境：固定的監視攝影機，要自動框出畫面中移動的人或車。核心思想是背景相減(Background Subtraction)：先學習一個「沒有移動物件時」的背景模型，再把每一影格與背景相減，差異顯著之處就是前景(移動物件)。

這其實是一種特殊的分割：它不靠灰階、顏色或邊緣，而是靠「時間上的變化」來區分前景與背景。得到前景遮罩後，同樣要用第 10 章的型態學清理雜點、用連通元件標記框出各個物件。OpenCV 提供 MOG2、KNN 等背景相減器，能自適應地更新背景模型(應付光線漸變、樹葉搖動等干擾)。這個案例是智慧監控、人流計數、交通流量分析的基礎，第 12 章會深入。

9.8 本章與全書的觀念連結總表

相關章節	共用的觀念	分割中的角色
第 5 章 空間域強化	直方圖、CLAHE、高斯平滑	分割前處理;Canny 首步的平滑;門檻依直方圖選

第 6 章 頻率域	低通、高通、邊緣即高頻	Canny 內含低通平滑加高通求梯度的雙重濾波
第 7 章 影像還原	雜訊與模糊的影響	雜訊生假邊、模糊使邊緣定位偏移
第 8 章 色彩處理	HSV 顏色分割、Lab 色差	彩色影像常以顏色為分割依據
第 10 章 二值影像	型態學、連通元件、距離轉換	門檻輸出的直接下游;分水嶺標記的前處理
第 12 章 視訊處理	背景相減、移動偵測	以時間變化做前景分割
第 13 章 特徵擷取	輪廓、形狀特徵、SIFT	分割出的輪廓供形狀分析;LoG 為 SIFT 前身
第 16 章 物件偵測	語意分割、實例分割	U-Net、Mask R-CNN 為傳統分割的深度學習後繼

表 9-2 第 9 章與全書其他章節的觀念連結

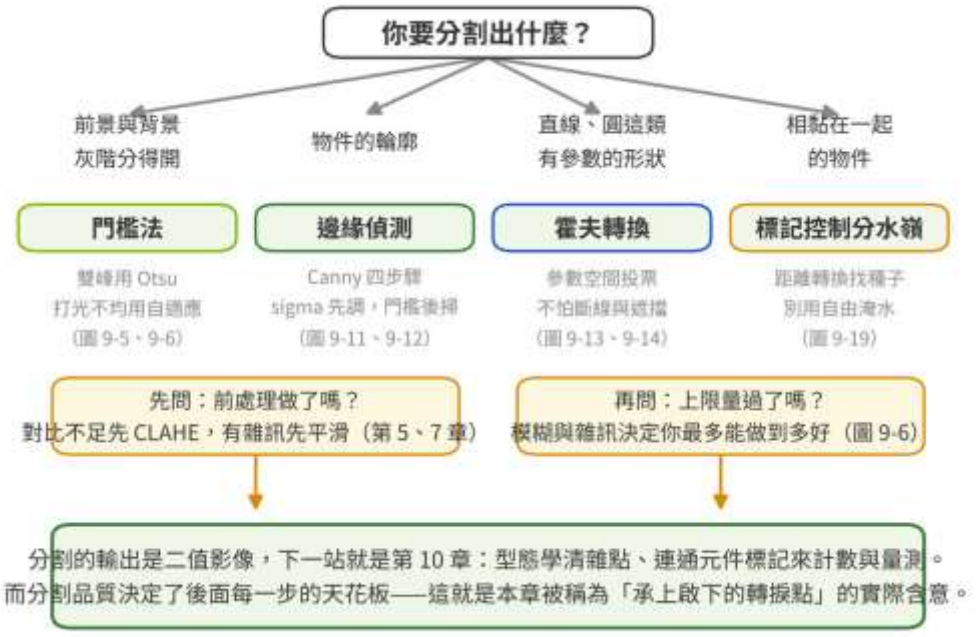


圖 9-21　第 9 章的決策地圖

程式實作 Lab 9

Lab 9-1 三種門檻法的擂台

目標：親手體會各門檻法的適用時機。(a)準備三張影像：對比良好且打光均勻的、打光不均(一側有陰影)的、以及含雜訊的;(b)對每張分別套用全域固定門檻、Otsu、自適應門檻，製成 3×3 的結果對照表;(c)畫出每張影像的直方圖，說明 Otsu 選出的門檻落在哪裡、為什麼在打光不均的影像上失準;(d)對含雜訊影像，比較「直接 Otsu」與「先高斯模糊再 Otsu」的差異;(e)結論：寫出一張決策流程圖——拿到影像後，如何依直方圖形狀與打光狀況選擇門檻法?

Lab 9-2 邊緣偵測參數探索器

目標：掌握 Canny 的參數行為。(a)以 cv2.createTrackbar 建立可即時調整 Canny 高低門檻與高斯 sigma 的介面;(b)觀察並記錄：sigma 太小、太大分別發生什麼事(對照第 6 章「sigma 決定訊號與雜訊的分界頻率」);高低門檻的比例(常建議 1:2 或 1:3)如何影響邊緣的完整與雜訊;(c)實作程式 9-2 的「中位數自動門檻」，與手動最佳值比較;(d)把 Sobel、LoG、Canny 三者的結果並排，說明為何 Canny 的邊緣最俐落(提示：非極大值抑制與遲滯連接)。

Lab 9-3 車道線偵測器

目標：完成叫獸開講的實戰。(a)取一段行車影像(或單張路面照片)，依 9.7.2 節流程：Canny → ROI 遮罩 → HoughLinesP;(b)依斜率把線段分為左右車道，各自用最小平方法擬合成一條直線並延伸繪製;(c)在畫面標出車道中心與車輛中心的偏差，模擬車道偏離警示;(d)測試極限：找出標線磨損、陰影、彎道的影像，記錄霍夫轉換在什麼情況下失效;(e)討論題：承(d)，為什麼當代自駕系統要改用深度學習語意分割?請對照表 9-1 說明。

Lab 9-4 相黏零件的分水嶺分割

目標：解決門檻法的痛點。(a)拍攝或取得一張多個相互接觸的零件(或硬幣、豆子)影像;(b)先用 Otsu 二值化，用連通元件標記計數，觀察相黏的物件如何被錯數成一個;(c)實作程式 9-4 的標記控制分水嶺，比較計數的正確性;(d)調整距離轉換的門檻(0.5、0.7、0.9 倍最大值)，觀察對分離效果的影響;(e)挑戰題：若不用標記控制、直接對梯度影像做分水嶺，會發生什麼?請截圖說明「過度分割」現象。

Lab 9-5 分割方法的擂台與上限量測

目標：建立「先量上限、再比方法」的工程習慣。(a)自製一個合成測試場景：取一張二值遮罩（可用 skimage.data.horse 或自己畫幾何圖形）當作正確答案，渲染成灰階後加上可控的高斯模糊與高斯雜訊，再乘上一個平滑的打光不均場;(b)寫一個 iou(pred, gt) 函式，並對固定門檻、Otsu、自適應門檻三種方法評分;(c)量出上限——把打光場移除、用事後搜尋的最佳固定門檻，記下這個分數，它代表模糊與雜訊造成的不可回復損失;(d)掃描自適應門檻的 blockSize 與 C，畫出 IoU 熱圖，標出落在上限 95% 以內的可用區;(e)把模糊的 sigma 由 0.5 調到 3.0 重做一次，畫出「上限 vs 模糊程度」的曲線，並寫出 300 字的結論：在你的場景裡，改善鏡頭與改善演算法各能帶來多少回報。

Lab 9-6 相黏物件的計數：三種方法的誠實比較

目標：完整走過本章後半的工具鏈，並學會不用標註也能驗證的技巧。(a)拍攝或取得一張含多個相互接觸物件的影像（硬幣、豆子、螺絲、細胞皆可），並以肉眼清點出正確的數量;(b)方法一：Otsu + 型態學 + 連通元件標記，記下數到幾個，並把相黏而被數成一個的位置圈出來;(c)方法二：直接對梯度影像做分水嶺（不給標記），記下區域數——你應該會得到一個大得荒謬的數字，想想為什麼;(d)方法三：距離轉換找局部極大值當種子、再做標記控制分水嶺，並把 min_distance 由 5 掃到 40，畫出「區域數 vs min_distance」曲線，在圖上標出正確答案那條水平線;(e)進階：若物件大小不一，固定的 min_distance 會失效——請改用 h-maxima（先把距離圖減去 h

再取區域最大值）或依物件尺寸自適應的種子策略，並比較兩者的穩健度;(f)寫出結論：在你的影像上，哪一種方法最穩定?若要部署到產線，你會怎麼設計一個「數錯了會自己發現」的檢查機制?

本章重點整理

- 影像分割把影像劃分為有意義的區域，是從「像素」到「區域/物件」的轉捩點;兩大策略為基於不連續性(邊緣、霍夫)與基於相似性(門檻、區域成長、分水嶺)，兩者是一體兩面。
- 門檻法：全域固定門檻適合對比良好且打光均勻者;Otsu 以最大化類間變異數自動選門檻，前提是直方圖雙峰;自適應門檻依局部統計量逐像素定門檻，專治打光不均。
- 邊緣偵測基於微分：Sobel 為帶平滑的一階梯度(方向性高通);Laplacian 為二階微分找零交越但對雜訊敏感，LoG 先平滑再微分改善之;Canny 以高斯平滑、梯度、非極大值抑制、雙門檻遲滯四步驟成為黃金標準。
- 霍夫轉換將「找形狀」翻轉為「參數空間投票」：直線用極座標 $\rho = x\cdot\cos\theta + y\cdot\sin\theta$ 避免斜率無限大，共線點的正弦曲線交會於累加器同一格;霍夫梯度法利用梯度指向圓心的性質加速圓偵測。
- 區域成長從種子出發併入相似鄰居;分水嶺法以「地形淹水」分開相黏物件，但易過度分割，須用標記控制(結合第 10 章的型態學與距離轉換)馴服。
- 深度學習語意分割(U-Net)、實例分割(Mask R-CNN)、基礎模型(SAM)能理解語意，但需大量標註與算力;傳統方法在受控環境、樣本稀少時仍是首選，兩者互補而非取代。

習題

1. 影像分割為什麼被稱為從低階視覺到中階視覺的轉捩點?請以「像素」與「區域」的概念說明。

提示與參考解答：分割之前(第 1 到 8 章)處理的是「像素進、像素出」的低階運算(強化、還原、色彩);分割把「屬於同一物件的像素」界定出來，關注點從像素升級到區域與物件，進入中階視覺，並為高階的物件辨識(這是什麼)提供基礎。沒有分割，電腦看到的只是一片數字。

2. 請比較「基於不連續性」與「基於相似性」兩大分割策略，各舉兩種方法，並說明為何說兩者是一體兩面。

提示與參考解答：不連續性著眼區域間的界線(灰階劇變處)，方法有邊緣偵測、霍夫轉換。相似性著眼區域內的一致性(灰階/顏色/紋理相似)，方法有門檻法、區域成長、分水嶺。一體兩面：區域的邊界正是相似性中斷之處，相似的內部正是被邊界包圍的範圍;強健系統常兩者並用。

3. Otsu 法的目標函數是什麼?它有什麼前提假設?在什麼情況下會失準，該如何補救?

提示與參考解答：目標是找出使「類間變異數」最大(等價於類內變異數最小)的門檻，讓前景背景兩群各自集中、彼此遠離。前提是直方圖雙峰。失準情況：前景面積極小(直方圖被背景單峰主宰)、前景背景灰階重疊嚴重。補救：先做前處理拉開對比，或改用自適應門檻。

4. 為什麼打光不均的影像不能用單一全域門檻?自適應門檻如何解決?這與第 8 章的哪個現象相呼應?

提示與參考解答：打光不均使同一物體在亮區與暗區的灰階不同，任何單一門檻都會顧此失彼(亮區背景被誤判為前景，或暗區前景被誤判為背景)。自適應門檻為每個像素依其鄰域局部統計量各自計算門檻，亮區用高門檻、暗區用低門檻。這是第 8 章「洋裝效應/色彩恆常性」在灰階世界的翻版——局部脈絡決定判讀。

5. 說明梯度大小與方向的幾何意義。Sobel 運算子除了求梯度，還兼具什麼功能?從第 6 章的觀點，Sobel 是什麼濾波器?

提示與參考解答：梯度大小代表灰階變化的劇烈程度，方向指向變化最快的方向(垂直於邊緣走向)。Sobel 在求某方向差分時，對垂直方向做加權平均，兼具抗雜訊(平滑)功能。從頻率域看，Sobel 是帶有平滑效果的方向性高通濾波器。

6. 一階微分與二階微分在邊緣處各有什麼表現?為什麼 LoG 要先做高斯平滑?

提示與參考解答：一階微分在邊緣處產生極值(峰);二階微分在邊緣處產生零交越(由正轉負處即邊緣)。二階微分對雜訊比一階更敏感，故 LoG 先用高斯平滑(低通)壓制雜訊，再用拉普拉斯(高通)找零交越，是「低通加高通」的組合。

7. 請依序說明 Canny 邊緣偵測的四個步驟，並指出每一步對應本書前面的哪個觀念。

提示與參考解答：一、高斯平滑(第 5 章低通，壓制雜訊避免微分放大);二、Sobel 計算梯度(9.3.1 節);三、非極大值抑制(沿梯度方向只留局部最大，細化成單像素邊);四、雙門檻與遲滯連接(強邊必留、弱邊僅在連接強邊時保留，兼顧去假邊與防斷裂)。sigma 是最關鍵參數。

8. 霍夫轉換的核心思想是什麼?為什麼直線偵測要用極座標式 ρ = x·cosθ + y·sinθ 而非斜截式 y = mx + b?

提示與參考解答：核心是觀點翻轉：影像空間中每個邊緣點，對應參數空間中一條曲線(所有可能通過它的直線)，共線的點其曲線交會於一點，故「找直線」變成「參數空間找得票最高的格子」(投票/累加器)。用極座標是因為斜截式無法表示垂直線(斜率無限大)，極座標式對任何方向的直線都有界。

9. 霍夫梯度法如何加速圓偵測?它利用了什麼幾何性質?

提示與參考解答：圓需三參數(圓心 a、b 與半徑 r)，直接投票是三維、成本高。霍夫梯度法利用「圓上每點的梯度方向指向圓心」的性質，讓每個邊緣點只沿自己的梯度方向投票，票數集中到真正圓心;先在二維定出圓心，再回頭決定半徑，把三維投票拆成兩個低維步驟。

10. 分水嶺法的「淹水」比喻是什麼?它擅長解決什麼門檻法的痛點?又有什麼副作用，如何解決?

提示與參考解答：把灰階(或梯度)影像視為地形，灰階為海拔，從各盆地最低點注水，兩湖相遇處築壩即為分割邊界。它擅長分開「相互接觸黏在一起」的物件(門檻法會當成一個)。副作用是過度分割(每個雜訊造成的局部極小值都成一個盆地)。解決：標記控制分水嶺，只從指定的前景/背景標記注水，標記由型態學與距離轉換(第 10 章)產生。

11. 傳統分割與深度學習語意分割最根本的差異是什麼?請各舉一個「該用傳統方法」與「該用深度學習」的場景並說明理由。

    提示與參考解答：根本差異：傳統方法分割「低階特徵的一致性」不懂語意、不需訓練、運算小、可解釋;深度學習分割「語意」能辨識類別但需大量標註與算力、可解釋性低。該用傳統：受控工業環境(打光穩定、背景單純)的 AOI 零件計數，又快又省。該用深度學習：自駕車的開放街景，需要理解「這是行人、這是路面」的語意。可對照表 9-1。

12. (綜合題)請設計一條「散落零件自動計數與尺寸量測」的完整處理管線，列出每一步用到的技術與對應章節，並說明若零件相互接觸該如何處理。

    提示與參考解答：管線：一、前處理——灰階化、CLAHE 增強對比(第 5 章);二、二值化——Otsu 門檻(9.2 節)，必要時先高斯模糊去雜訊;三、清理——型態學開運算去雜點、閉運算填孔(第 10 章);四、標記量測——連通元件標記或輪廓偵測，計算面積、周長、最小外接矩形(第 10、13 章)。若零件相黏：改用標記控制分水嶺(9.5 節)，以距離轉換分離，避免被錯數為一個物件。

13. 「Otsu 就是自動找直方圖的谷底」——這句話在什麼條件下成立?請由類間變異數的公式說明，並估計在前景背景面積懸殊時門檻會往哪邊偏。

    提示與參考解答：類間變異數為 $w0 \cdot w1 \cdot (m0 - m1)^2$，除了「兩群離多遠」之外還乘上「兩群各佔多少」的 $w0 \cdot w1$，而 $w0 \cdot w1$ 在兩群面積相等時最大。因此只有在前景與背景面積相當時，Otsu 的解才會落在谷底;面積懸殊時門檻會被推向「讓兩群面積更接近」的方向，也就是往大的那一群的方向移動。圖 9-3 的實測：硬幣影像的谷底在 139，Otsu 卻選 107，差了 32 階——而且從二值化結果看，Otsu 的選擇反而比較好。

14. 一個檢測任務的前景物件只佔畫面的 1%。直接用 Otsu 會發生什麼事?請提出兩種補救方式，並說明為什麼有效。

    提示與參考解答：直方圖幾乎完全被背景的單峰主宰，前景那個峰小到看不見，Otsu 會把門檻切在背景分布的內部，分割結果幾乎全錯。圖 9-5 的實測顯示，前景面積低於約 3% 時 IoU 由 0.99 急遽崩潰到 0.01，而且是突然崩掉、不是慢慢變差。補救一：先縮小 ROI（用粗定位把視野框到物件附近），讓兩個峰的高度回到同一個量級。補救二：改用自適應門檻或局部對比的方法，讓每個像素依鄰域決定門

檻，不受全域分布比例的影響。

15. 自適應門檻的 blockSize 與 C 各該怎麼選?請說明它們的物理意義，並描述設錯時各會出現什麼症狀。

    提示與參考解答：blockSize 是估計「局部背景亮度」所用的鄰域邊長，必須大於你要抓的前景結構寬度（否則前景自己會把局部平均拉高，造成筆畫內部被判成背景、只剩輪廓），又必須小於光照變化的空間尺度（否則等於退化成全域門檻）。C 是從局部平均扣掉的常數，相當於要求前景比鄰域平均再暗（或亮）多少，用來排除平坦區的雜訊;C 太小會讓平坦區的雜訊整片翻成前景。圖 9-7 在合成文件上掃過兩個參數：最佳為 blockSize 71、C 26，而落在最佳值 95% 以內的組合只佔全部 357 種的 14%。

16. 什麼是「效能上限」?為什麼在評估一個分割方法之前應該先把它量出來?請以圖 9-6 的數據說明。

    提示與參考解答：上限是指在同樣的成像條件（模糊、雜訊、量化）下，任何方法都不可能超過的成績。量法：把干擾因素之一（此例為打光不均）移除，再用事後挑的最佳參數，得到的分數就是上限。圖 9-6 的實測：把打光弄成完全均勻、再用最佳固定門檻，IoU 也只有 0.81，剩下的損失來自鏡頭模糊與雜訊。調過參數的自適應門檻拿到 0.78，已經是上限的 95%。若不先量上限，看到 0.78 很容易以為還有很大的改善空間，於是把時間花在演算法上;事實上真正該改的是鏡頭與曝光。

17. 一階微分與二階微分在「階躍邊緣」與「細線」上的表現各是什麼?為什麼說拉普拉斯會把一條線報成兩條邊?

    提示與參考解答：階躍邊緣的一階微分出現單一極值，二階微分出現一個零交越，兩者都正確指出邊緣位置。但一條細線（屋脊）的剖面是「升上去再降下來」，它的二階微分在線的兩側各有一個零交越（圖 9-8 第三欄實際畫出 @95 與 @97），所以拉普拉斯會輸出兩條邊而不是一條線。這不是雜訊造成的，而是二階方法在細結構上的固有行為;要找線狀結構應改用專門的線偵測器（如 Frangi 濾波）或先做骨架化。

18. 從頻率域的觀點，Sobel 是什麼樣的濾波器?請說明它為什麼比單純的中央差分 [−1, 0, 1] 更能抗雜訊。

    提示與參考解答：Sobel-x 可分解為水平的 [−1, 0, 1] 與垂直的 [1, 2, 1]，因此它在

水平方向是高通（直流分量的響應為 0，峰值出現在 Nyquist 的 0.5 倍處），在垂直方向卻是低通。圖 9-9 實測：固定在峰值水平頻率上，垂直頻率走到 Nyquist 的一半時，響應由 8.00 掉到 4.05（剩 51%），走到 Nyquist 時恰好歸零——因為 [1, 2, 1] 的頻率響應 2 + 2cos(ω) 在 ω = π 處為零。這個垂直方向的平均正是抗雜訊的來源：單純的中央差分沒有這一項，雜訊會原封不動地被放大。

19. 請由「核係數平方和」推導各邊緣運算子的雜訊放大倍率，並說明為什麼「放大得少」不等於「比較好」。

    提示與參考解答：對獨立雜訊做卷積，輸出標準差等於輸入乘上核係數平方和的平方根（第 7.7.4 節）。Sobel 的係數平方和為 12，倍率 √12 ≈ 3.46;五點拉普拉斯為 20，倍率 √20 ≈ 4.47。圖 9-10 的實測與理論最大只差 0.5%。但 LoG（sigma=2）的倍率只有 0.047，卻不能就此說它比較好——它的係數本來就小，把訊號也一起縮小了。該比的是輸出的訊噪比：以乾淨階躍邊緣的峰值響應除以輸出雜訊，拉普拉斯只有 2.9，LoG（sigma=2）是 18.2，提升 6.3 倍。先平滑再微分之所以划算，是因為它壓掉的雜訊多於訊號。

20. Canny 的非極大值抑制為什麼必須沿著梯度方向進行?它實際上丟掉了多少像素?為什麼這一步是後面遲滯連接能夠運作的前提?

    提示與參考解答：梯度向量永遠垂直於邊緣走向（沿邊緣走灰階不變，方向導數為零），所以「跨過邊緣」的方向就是梯度方向，梯度隆起的剖面正好落在這個方向上。沿此方向與前後兩格比較，只留最大者，就把隆起壓成脊線。圖 9-11 實測：梯度圖中超過低門檻的 31,522 個像素，抑制後只剩 8,127 個，少掉 74%，而邊緣位置不變。這一步是遲滯連接的前提，因為遲滯是用「弱邊有沒有連到強邊」來判斷的;若邊緣是好幾像素寬的粗帶子，連通性就失去鑑別力了。

21. 調 Canny 的參數時，sigma、高門檻、高低門檻比三者的重要性順序是什麼?請以可量化的方式說明，並給出一條實務上的調參流程。

    提示與參考解答：圖 9-12 在一個含高對比物件、低對比物件與背景紋理的合成場景上量測（每個 sigma 都用它自己最好的門檻，才算公平）。結果：四種高低比（1.5:1 到 3:1）各自的最佳 F1 只差 0.028，所以比值幾乎不重要;但高門檻很挑，F1 落在最佳值 95% 以內的只有 72 到 84 這一小段。sigma 則決定了上限本身：sigma 0.6 時召回率只有 0.70（為擋雜訊必須把門檻設高，低對比物件跟著不見），sigma 4.2 時 F1 掉到 0.70（邊緣被平滑掉）。流程：一、先依雜訊程度與最小結構尺寸決

定 sigma;二、固定比值 2:1，掃高門檻;三、必要時再微調比值。

22. 霍夫轉換的累加器需要多大?請分別估計一張 1080p 影像上的直線累加器，以及一張 303×384 影像上半徑不限的圓累加器，並說明這對實務的啟示。

提示與參考解答：直線累加器為 2D × 180 格（D 為對角線長）。1080p 的 D = 2203，累加器 4406 × 180 = 793,080 格，int32 約 3 MB，完全可行。圓累加器為 H × W × R:303 × 384 若半徑不限（1 到 192）就是 22,339,584 格、約 89 MB，而且每個邊緣點要投的票從 180 暴增到數萬。啟示：cv2.HoughCircles 的 minRadius 與 maxRadius 實質上是必填的——把半徑限在 12 到 36，累加器降到 2,908,800 格才跑得動。降維的辦法就是霍夫梯度法：利用「圓上每點的梯度方向通過圓心」，讓每個邊緣點只沿自己的梯度方向投票，先定圓心再定半徑;代價是對梯度方向極度敏感，所以程式 9-3 一定要先做 medianBlur。更一般的原則是成本隨參數個數指數成長：兩個參數很好用，三個要想辦法降維，超過三個就該改用輪廓擬合。

23. 什麼是區域成長的「洩漏」?請說明它的成因，並以可量化的方式描述它發生時有多突然。

提示與參考解答：區域成長從種子出發，把與區域夠相似的鄰居併進來。只要物件邊界上有任何一個像素的灰階落差小於容許值，就形成一個破口，區域會從那裡流進背景，然後整張影像都被併入。圖 9-17 的實測：最佳容許值 42（IoU 0.967），但容許值升到 56 時面積由 11,395 暴增到 117,863（10 倍），IoU 掉到 0.077。它沒有中間地帶，這也是區域成長很少單獨使用的原因;實務上會先用邊緣或型態學把邊界封好（第 10 章），再放它進去長。

24. 分水嶺法的「過度分割」有多嚴重?請說明成因，並說明標記控制為什麼能解決它。

提示與參考解答：分水嶺從每一個局部極小值開始注水，所以分割出的區域數完全等於局部極小值的個數;而真實影像的梯度圖上，雜訊會製造出成千上萬個局部極小值。圖 9-19 在 24 枚硬幣的影像上實測：直接對梯度做分水嶺得到 17,404 個區域，先做 sigma=2 的高斯平滑再做仍有 5,419 個。標記控制的作法是不讓水從所有局部極小值自由湧出，而是用距離轉換找出「每個物件的中心」當作指定的種子;本例在 min_distance = 19 時得到剛好 24 個區域，與實際硬幣數一致。重點是分水嶺本身沒有問題，問題在於「誰有資格當盆地」——標記控制就是把這個決定權從雜訊手上收回來。

25. (綜合設計題)請為一條「散落零件自動計數與尺寸量測」的產線設計完整管線，列出每一步的技術、對應章節與參數決定方式，並說明如何處理零件相互接觸、以及如何驗收這套系統。

    提示與參考解答：一、光源與成像：封閉光箱、背光（讓零件成為高對比剪影），關閉自動曝光與自動白平衡（第 8 章）——這一步決定了後面所有步驟的上限。二、前處理：灰階化，必要時 CLAHE 拉對比（第 5 章）;若有脈衝雜訊先用中值濾波（第 7 章）。三、二值化：背光下直方圖是乾淨雙峰，用 Otsu;若打光仍有梯度則改自適應門檻（圖 9-6、9-7）。四、清理：型態學開運算去雜點、閉運算補洞（第 10 章）。五、分離相黏零件：距離轉換 + 標記控制分水嶺（圖 9-19）;關鍵參數 min_distance 應依零件的最小半徑決定，不要盲試。六、計數與量測：連通元件標記（第 10 章）取面積、外接矩形、最小外接圓;以實際尺寸標定像素尺寸（第 4 章）。七、驗收：先量上限——取一批已知答案的樣本，用人工標註算出 IoU 與計數誤差的上限（圖 9-6 的作法），再要求系統達到上限的某個百分比;同時記錄跨班別、跨批次的重複性，一旦漂移就回頭檢查光源而不是調參數。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 10(影像分割之標準參考)。
- Hough, P. V. C., Method and Means for Recognizing Complex Patterns, US Patent 3,069,654, 1962.(霍夫轉換原始專利，本章叫獸開講之史料來源)
- Duda, R. O., & Hart, P. E., Use of the Hough Transformation to Detect Lines and Curves in Pictures, Comm. ACM, 1972.(極座標表示法)
- Canny, J., A Computational Approach to Edge Detection, IEEE TPAMI, 1986.(Canny 邊緣偵測原始論文)
- Otsu, N., A Threshold Selection Method from Gray-Level Histograms, IEEE Trans. SMC, 1979.
- Ronneberger, O., Fischer, P., & Brox, T., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI, 2015.

- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 10 章 二值影像處理

本章學習目標

- 說明二值影像的意義與來源，理解型態學處理建立在集合論與結構元素之上。
- 推導並實作侵蝕、膨脹兩個基本運算，以及斷開、閉合兩個組合運算。
- 運用型態學梯度、頂帽與黑帽轉換解決邊緣萃取與不均勻背景校正問題。
- 操作 Hit-or-Miss、細線化、骨架化與孔洞填補等進階型態學技術。
- 以連通元件標記完成物件計數，並計算距離轉換供分割與量測使用。
- 理解 Haar 級聯分類器如何以積分影像與級聯結構達成即時人臉偵測，並在 ESP32-CAM 上部署。

叫獸開講|2001 年，相機學會了找人臉：Viola-Jones 的百萬分之一秒

在深度學習席捲電腦視覺的十一年前，有一個演算法就已經讓相機學會即時「看見」人臉了。2001 年，微軟研究院的 Paul Viola 與 Michael Jones 在頂級會議 CVPR 上發表了一篇論文《Rapid Object Detection using a Boosted Cascade of Simple Features》，提出了史上第一個能在普通電腦上即時執行的人臉偵測器。在當年一台 700MHz 的 Pentium III 處理器上，它的速度比同等準確度的其他方法快上數十甚至數百倍——這在當時是不可思議的成就。

Viola-Jones 的成功，靠的是三個環環相扣的巧思。第一是 Haar-like 特徵：它不看單一像素，而是計算「相鄰的矩形區域之間的亮度差」——例如，人臉上「眼睛區域通常比臉頰暗」，這個簡單的明暗關係就是一個特徵。第二是積分影

像(Integral Image)：一張影像預先算好「每個點左上方所有像素的累加和」之後，任何一個矩形區域的像素總和，都只需要查表做三次加減法就能得到，與矩形大小完全無關——這讓成千上萬個 Haar 特徵能以驚人的速度計算。第三是級聯分類器(Cascade)：把分類器串成一連串關卡，前面的關卡用極少的特徵、極快的速度，把「明顯不是臉」的區域(佔畫面絕大部分)迅速淘汰;只有通過前關的少數候選，才交給後面更精密(也更耗時)的關卡仔細檢查。這個「快速排除、集中火力」的設計，是它能即時執行的關鍵。

把這三招放在本章的脈絡下看，你會發現它們與二值影像處理的精神一脈相承。Haar 特徵計算的是矩形區域的加總與相減，本質上就是對區域做集合式的運算;積分影像是一種「預先累加」的加速技巧，和本章連通元件標記中「一次掃描就完成統計」的思路異曲同工;而級聯「逐步淘汰、由粗到精」的策略，更是型態學處理中「先粗略清理、再精細處理」的工程智慧。Viola-Jones 內建於 OpenCV 多年(即著名的 Haar Cascade)，至今仍是資源受限裝置(如本章要部署的 ESP32-CAM)上人臉偵測的實用選擇。

當然，它也有明顯的侷限：它最擅長偵測正面、端正的人臉，遇到側臉、大角度傾斜或嚴重遮擋就力有未逮;這些正是第 16 章深度學習物件偵測器要克服的。但 Viola-Jones 的歷史地位無可撼動——它證明了「精心設計的簡單特徵 + 聰明的加速結構 + 機器學習的特徵篩選」可以創造奇蹟，這個信念直到今天仍深深影響著整個領域。

思考題：級聯分類器「用快速的關卡先淘汰明顯不對的候選，把運算集中在少數可能的目標上」——這個「由粗到精、快速排除」的思想，在生活中還有哪些應用?(想想：履歷篩選、垃圾郵件過濾、機場安檢)

10.1 二值影像與型態學的基礎

10.1.1 什麼是二值影像

二值影像(Binary Image)是每個像素只有兩種可能值的影像：前景(通常設為 1 或 255，顯示為白)與背景(通常設為 0，顯示為黑)。它是第 9 章分割的直接產物——門檻法、Otsu、邊緣偵測的輸出，幾乎都是二值影像。二值影像的資訊量雖然遠低於灰階或彩色影像，卻正是它的價值所在：當我們只關心「物件在哪裡、形狀如何、有幾個」時，把影像化約成非黑即白，能讓後續的形狀分析變得極為高效。

但分割產生的二值影像往往並不完美：前景區域中可能有小孔洞(雜訊或反光造成)、背景中散布著孤立的小白點(雜訊)、本該分開的物件黏在一起、本該相連的物件斷成幾截。本章的型態學運算，就是清理與修整這些缺陷、並從中萃取形狀資訊的工具箱——它是第 9 章分割與第 13 章特徵擷取之間承先啟後的關鍵環節。

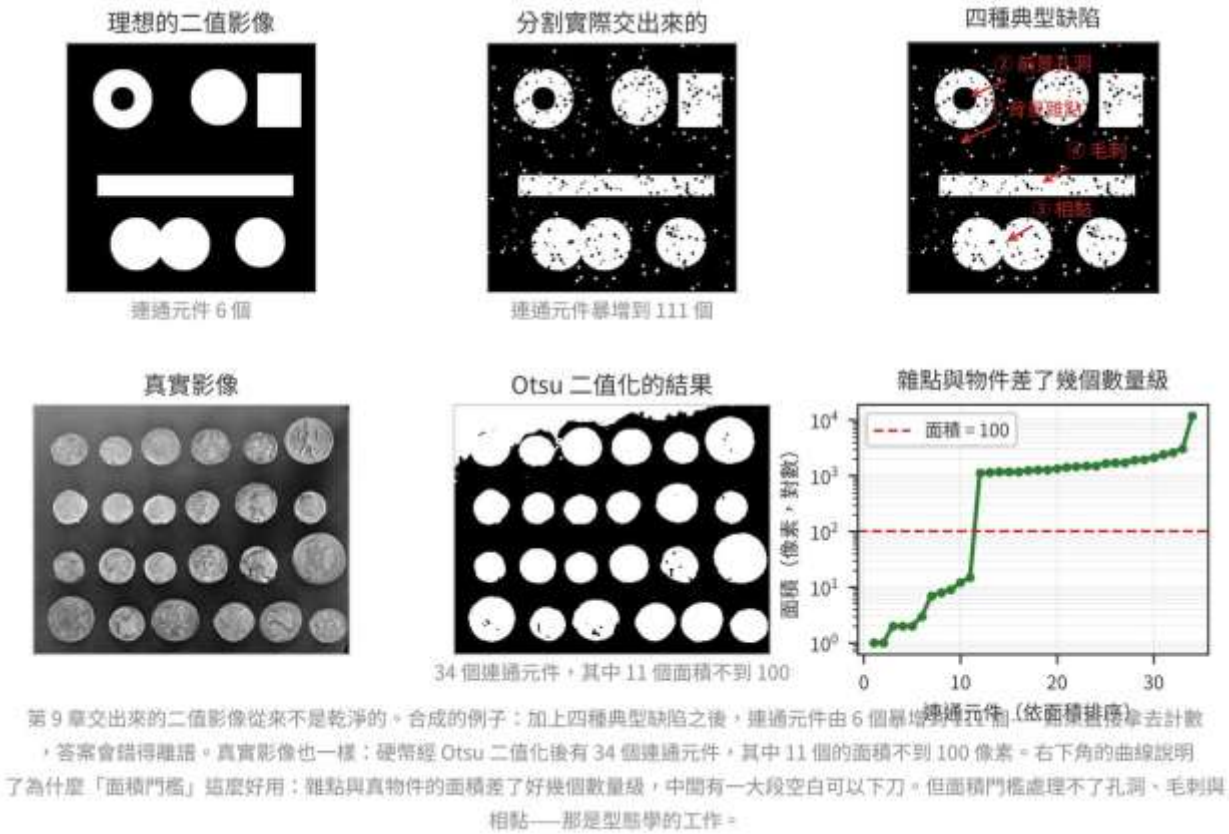


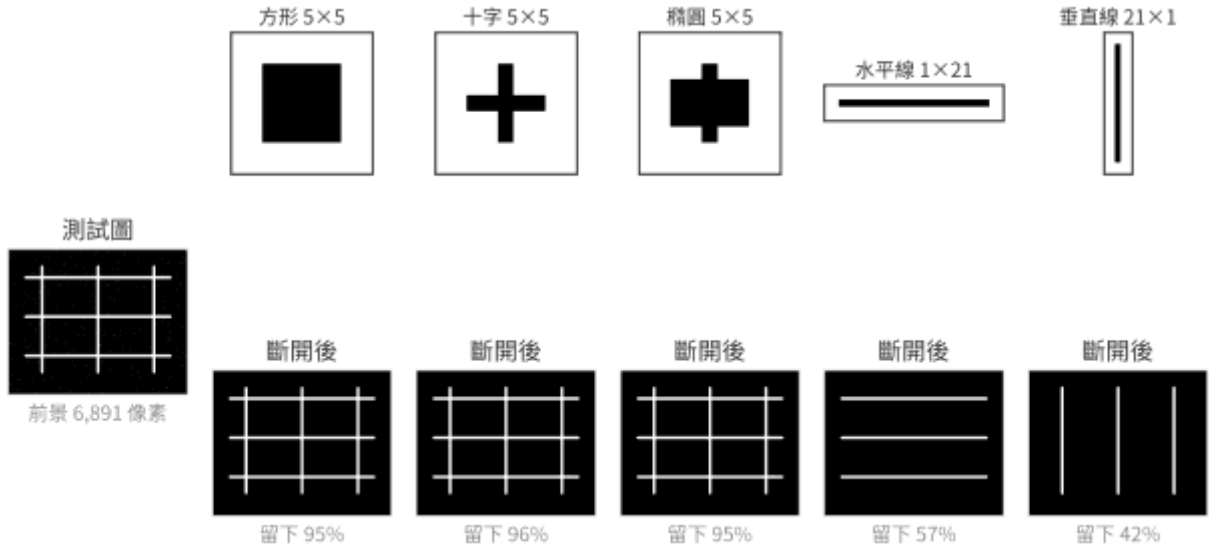
圖 10-1　分割交出來的二值影像長什麼樣：四種典型缺陷

10.1.2 型態學與結構元素

數學型態學(Mathematical Morphology)由 Georges Matheron 與 Jean Serra 在 1960 年代創立於法國，原本用於分析礦石與地質切片的形狀，其理論基礎是集合論。

核心觀念是：把二值影像的前景視為一個點集合，用一個稱為結構元素(Structuring Element)的小形狀去「探測」這個集合，依據結構元素與前景的貼合關係，決定輸出。

結構元素相當於空間濾波的「核」，但作用方式是集合運算而非數值運算。它可以是任意形狀——方形、十字形、圓形、線形——形狀的選擇會直接影響處理結果：用水平線形結構元素，會強化水平結構、削弱垂直結構;用圓形結構元素，則各方向一視同仁。結構元素還有一個「原點」(通常在中心)，決定它在影像上對齊的方式。選對結構元素的形狀與大小，是型態學處理的關鍵藝術。



同一張圖、同一個運算（斷開），只換結構元素的形狀，結果天差地別。方形、十字、橢圓這三個小的結構元素都只是去雜點，前景分別留下 95%、96%、95%;而 1×21 的水平線只留下水平線條（57%），21×1 的垂直線只留下垂直線條（42%）。這就是型態學與空間濾波最大的差別：濾波的核決定「怎麼加權」，結構元素決定「要找什麼形狀」。表格線萃取、PCB 線路分離、條碼定位，用的都是這一招。

圖 10-2　結構元素的形狀決定你探測到什麼

程式 10-1 建立結構元素

```
import cv2, numpy as np

# 建立不同形狀的結構元素
rect  = cv2.getStructuringElement(cv2.MORPH_RECT,    (5, 5))  # 方形
cross = cv2.getStructuringElement(cv2.MORPH_CROSS,   (5, 5))  # 十字
ellip = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))  # 橢圓

# 也可手動指定形狀,例如水平線(專門處理水平結構)
hline = np.zeros((1, 15), np.uint8); hline[:] = 1

print(cross)   # 觀察十字結構元素的形狀
```

10.2 兩個基本運算：侵蝕與膨脹

10.2.1 侵蝕

侵蝕(Erosion)的規則是：把結構元素滑過影像，只有當結構元素完全被包含在前景之內時，其原點對應的位置才保留為前景，否則設為背景。直觀的效果是：前景物件會「縮水」、邊界向內收縮，小於結構元素的細節(細小突起、細線、孤立雜點)會被完全消滅。

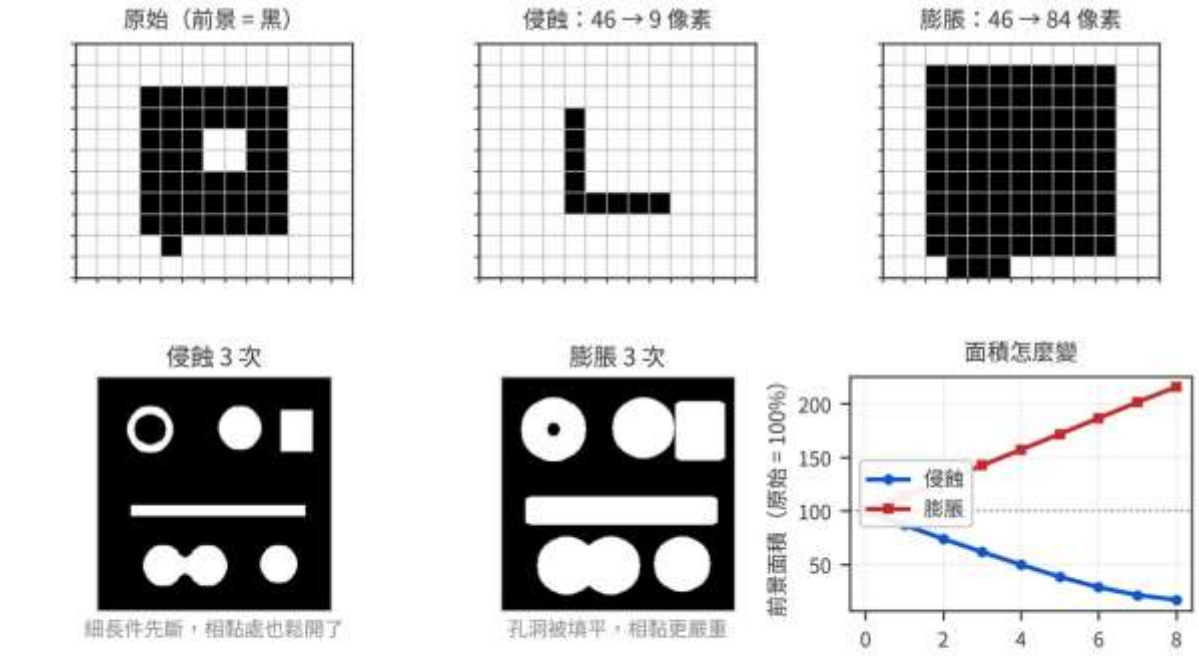


圖 10-3　侵蝕與膨脹：從一個小矩陣看清楚定義

侵蝕的典型用途：去除背景中的孤立雜點、斷開兩個以細頸相連的物件、縮小過度膨脹的前景。一個重要觀察(呼應第 7 章)：對灰階影像而言，侵蝕等同於在鄰域取最小值——這正是第 7 章提過的最小值濾波。同一個運算，在第 7 章是「去除鹽雜訊」，在這裡是「腐蝕形狀」，再次印證影像處理概念的一體多面。

10.2.2 膨脹

膨脹(Dilation)是侵蝕的對偶運算，規則是：只要結構元素與前景有任何交集，其

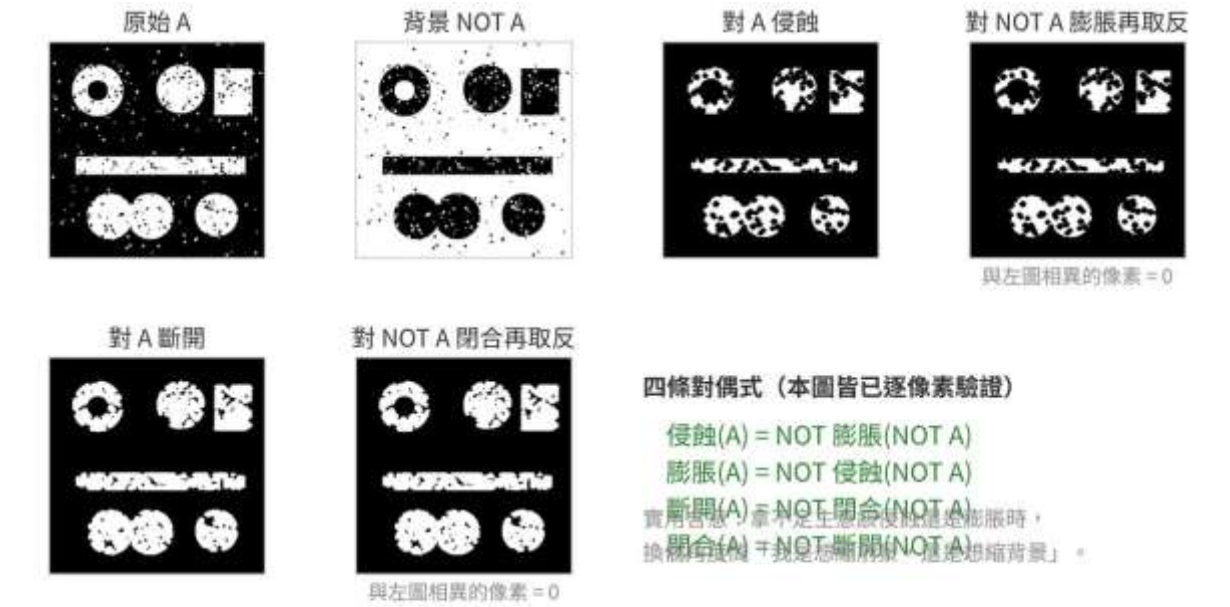
原點對應的位置就設為前景。效果與侵蝕相反：前景物件會「長大」、邊界向外擴張，小的孔洞與裂縫會被填補、斷裂的部分可能重新相連。



圖 10-4　侵蝕就是最小值濾波，膨脹就是最大值濾波——但別只做一半

膨脹的典型用途：填補前景內的小孔洞、連接斷裂的線段或字元、放大前景以便後續處理。同樣呼應第 7 章：對灰階影像而言，膨脹等同於在鄰域取最大值，即最大值濾波(去除胡椒雜訊)。

侵蝕與膨脹是一對對偶運算：對前景侵蝕，等於對背景膨脹。理解這個對偶性，能幫助你在「該侵蝕還是該膨脹」之間快速做出判斷——問自己是想縮前景還是縮背景即可。



對偶性是型態學最省力的一條性質：對前景做的每一個運算，都等於對背景做對應的運算再取補集。本圖把兩條式子逐像素比對：「對 A 侵蝕」與「對背景膨脹再取反」相異的像素是 0 個；「對 A 斷開」與「對背景閉合再取反」相異的像素是 0 個。這表示你永遠只需要記住其中一半。實務上更重要的是它提醒你：前景與背景是對稱的，「哪一邊是前景」只是你自己的約定——若把二值影像反過來看（前景當背景），斷開與閉合的角色就互換了。

圖 10-5　對偶性：對前景侵蝕，就是對背景膨脹

程式 10-2 侵蝕與膨脹

```
import cv2, numpy as np

binary = cv2.imread("binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

eroded  = cv2.erode(binary, kernel, iterations=1)    # 侵蝕:縮水
dilated = cv2.dilate(binary, kernel, iterations=1)   # 膨脹:長大

# iterations 控制重複次數,等效於放大結構元素
eroded3 = cv2.erode(binary, kernel, iterations=3)

cv2.imwrite("eroded.png", eroded)
cv2.imwrite("dilated.png", dilated)
```

10.2.3 推導：為什麼侵蝕就是取最小值，以及對偶性從哪裡來

10.2.1 節提過一句「對灰階影像而言，侵蝕等同於在鄰域取最小值」，很多人把它當成一個巧合記起來。其實這不是巧合，而是同一個定義在不同值域上的樣子，把它推一遍，型態學的許多性質就會一次說得通。

先把二值的定義寫成集合式。設前景為集合 A，結構元素為 B，侵蝕的定義是「把 B 平移到位置 z，若 B 整個落在 A 之內，就保留 z」:侵蝕(A, B) = 所有「把 B 平

移到該處之後，B 完全落在 A 之內」的位置所成的集合。現在把二值影像看成一個只取 0 或 1 的函數 f，「B 整個落在前景內」等價於「B 覆蓋到的每一個位置 f 都是 1」，也就是「B 覆蓋範圍內 f 的最小值是 1」。把這句話裡的 1 換成任意灰階，就得到灰階版本的侵蝕：輸出等於結構元素覆蓋範圍內輸入的最小值。膨脹的推導完全對稱，只是把「完全包含」換成「有交集」，「最小值」換成「最大值」。圖 10-4 把 cv2.erode 與 scipy 的 minimum_filter 逐像素比對，最大差異是 0——不是相近，是同一個運算。

對偶性則來自一個更簡單的觀察：「B 完全落在 A 內」與「B 完全不碰到 A 的補集」是同一句話。把它寫成式子就是「對 A 侵蝕 = 對 A 的補集膨脹之後再取補集」，其中膨脹所用的結構元素要先翻轉 180 度。若結構元素對原點對稱（方形、十字、橢圓都是），這個翻轉可以省略。圖 10-5 把這條式子以及它的斷開閉合版本都逐像素驗證過，相異的像素數都是 0。這條性質的實用價值是省一半的記憶：你只需要熟悉侵蝕，膨脹就是「對背景做侵蝕」。

順著這條線，還有四條代數律值得記住，因為它們會直接影響你怎麼寫程式。第一，侵蝕與膨脹都是遞增的：A 含於 C 則結果也維持包含關係。第二，膨脹滿足結合律：先用 B1 膨脹再用 B2 膨脹，等於直接用「B1 膨脹 B2」所得到的那個結構元素膨脹一次。這條就是 iterations 參數的理論基礎——用 3×3 的結構元素膨脹兩次，等於用 5×5 膨脹一次。第三，侵蝕與膨脹都可以分解：一個 k×k 的方形結構元素可以拆成「1×k 的水平」接「k×1 的垂直」，運算量由 k^2 降到 2k——這正是第 5.3.3 節可分離濾波的型態學版本。第四，侵蝕對交集、膨脹對聯集具有分配律，但反過來只有包含關係而非等號;這個不對稱是後面許多「順序不能換」的根源，10.3.3 節會接著談。

最後是一個實務推論。既然灰階侵蝕就是取最小值，那麼它對「鹽」（白點）特別有效、膨脹對「胡椒」（黑點）特別有效。但圖 10-4 的實測提醒我們不要只做一半：對含鹽雜訊的影像只做侵蝕，RMSE 反而由 20.9 升到 20.9 以上，因為白點雖然消失

了，整張影像也跟著被壓暗。要做完整的斷開（蝕完再脹回來）才降到 10.1，而這個 10.1 幾乎等於「對一張完全乾淨的影像做同樣斷開」的 10.1——也就是說雜訊已經被清光，剩下的全是運算本身的平滑代價。這也是為什麼下一節要談組合運算。

10.3 兩個組合運算：斷開與閉合

把侵蝕與膨脹依不同順序組合，得到兩個極為實用的運算。它們的巧妙之處在於：能在「去除雜訊/填補缺陷」的同時，大致維持物件的原始大小——因為一縮一脹相互抵消。

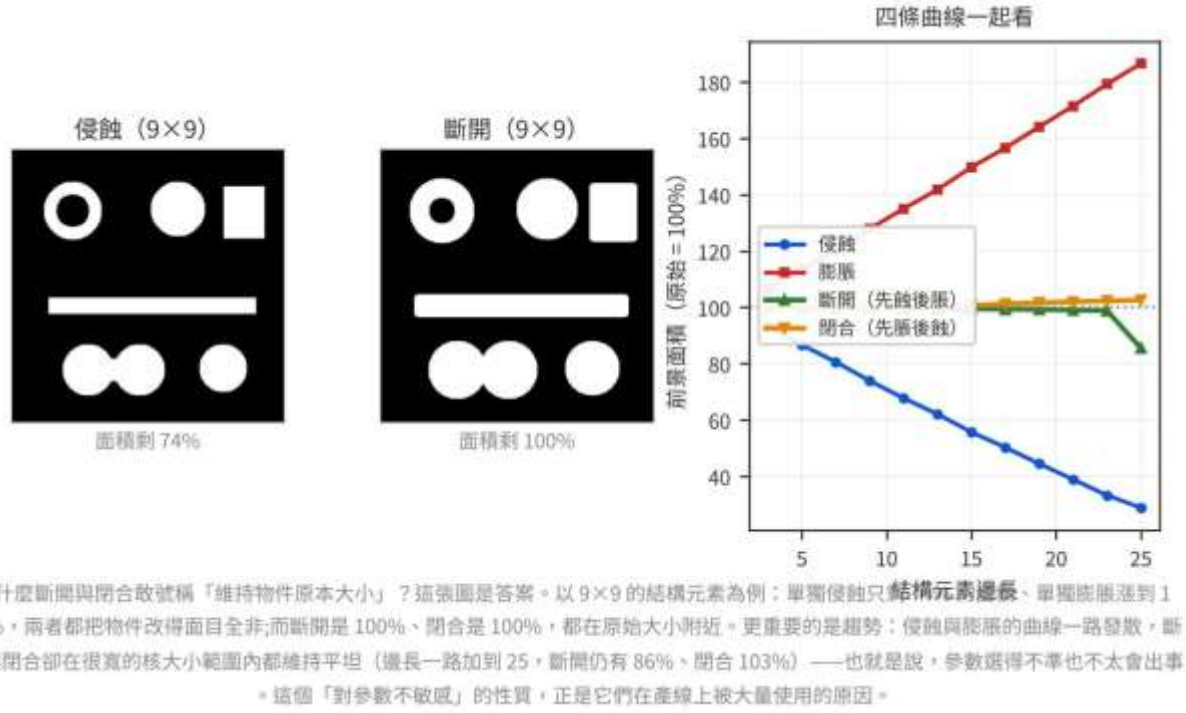


圖 10-6　一縮一脹為什麼能保住物件原本的大小

10.3.1 斷開

斷開(Opening)是「先侵蝕、後膨脹」。先侵蝕消滅了小於結構元素的雜點與細頸，後續的膨脹再把倖存的物件恢復到接近原本的大小。淨效果是：去除背景中的孤立小白點、平滑物件的外部輪廓、斷開細頸相連的物件——而主體幾乎不變。口訣：斷開「開」掉背景的雜訊。

10.3.2 閉合

閉合(Closing)是「先膨脹、後侵蝕」。先膨脹填補了小於結構元素的孔洞與裂縫，後續的侵蝕再把物件收回原本的大小。淨效果是：填補前景內的小孔洞、連接鄰近的斷裂、平滑物件的內部輪廓——而主體幾乎不變。口訣：閉合「閉」合前景的破洞。

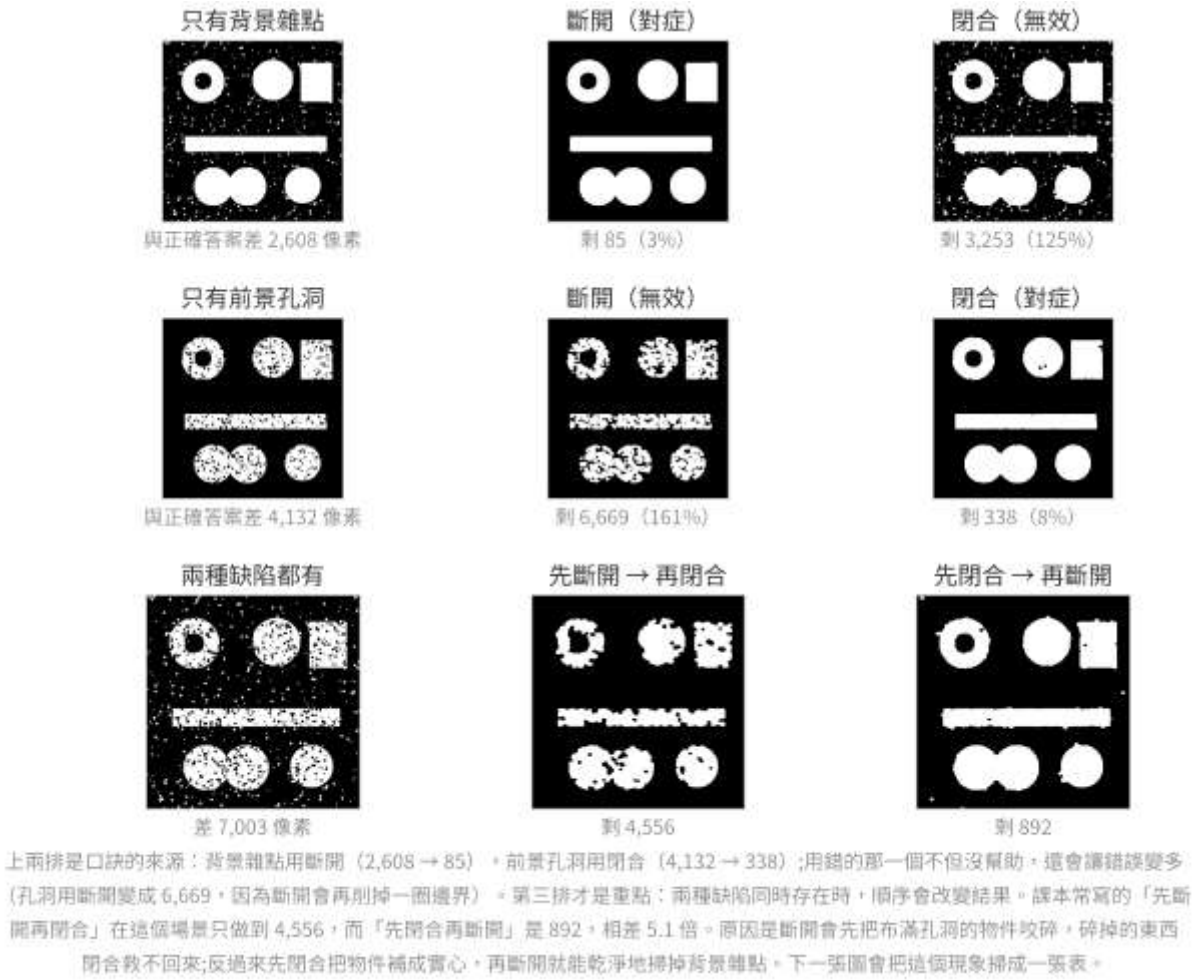
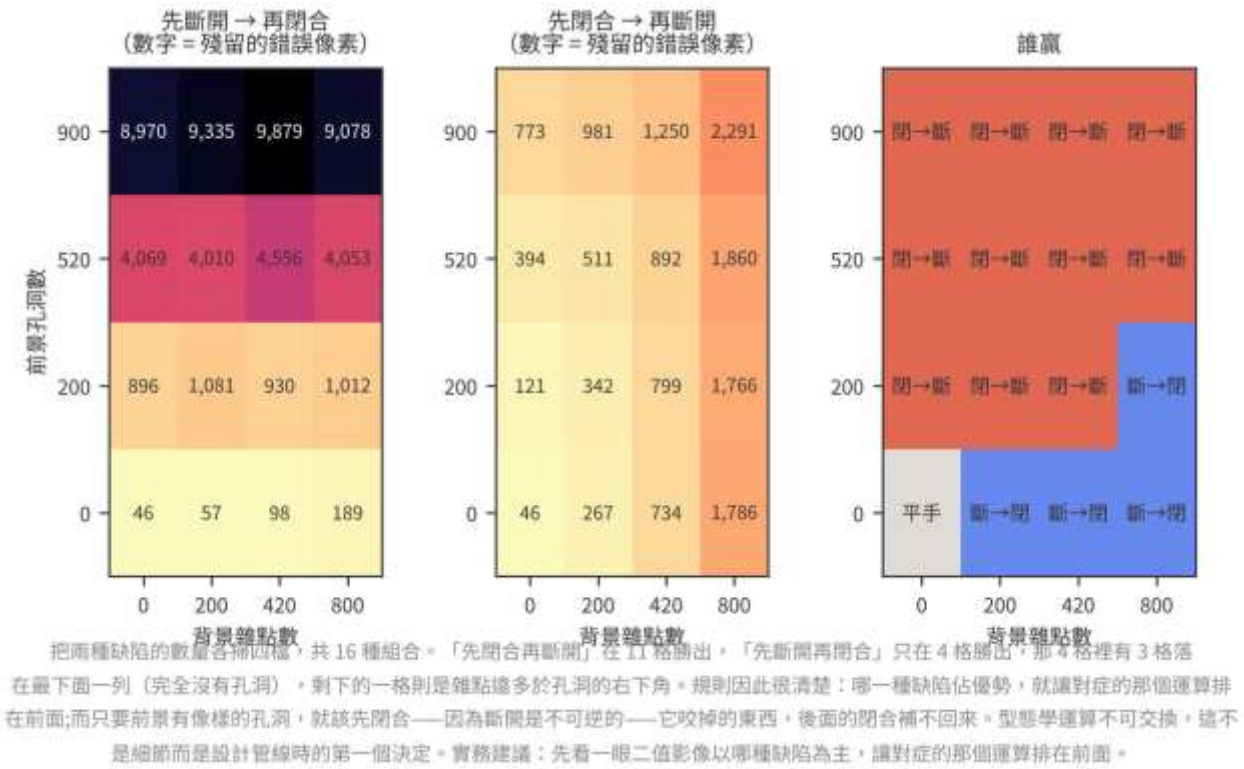


圖 10-7　斷開與閉合各治各的病，而且順序會改變結果

斷開與閉合是實務上最常用的型態學運算，尤其在第 9 章分割之後的清理階段。標準流程往往是：Otsu 二值化 → 斷開去除背景雜點 → 閉合填補前景孔洞 → 連通元件標記計數。第 9 章程式 9-4 的分水嶺前處理，用的正是這套組合。



把兩種缺陷的數量各掃四檔，共 16 種組合。「先閉合再斷開」在 11 格勝出，「先斷開再閉合」只在 4 格勝出，那 4 格裡有 3 格落在最下面一列（完全沒有孔洞），剩下的一格則是雜點遠多於孔洞的右下角。規則因此很清楚：哪一種缺陷佔優勢，就讓對症的那個運算排在前面;而只要前景有像樣的孔洞，就該先閉合——因為斷開是不可逆的——它咬掉的東西，後面的閉合補不回來。型態學運算不可交換，這不是細節而是設計管線時的第一個決定。實務建議：先看一眼二值影像以哪種缺陷為主，讓對症的那個運算排在前面。

圖 10-8　順序之爭：哪一種缺陷多，就先治哪一種

程式 10-3 斷開與閉合

```
import cv2, numpy as np

noisy = cv2.imread("noisy_binary.png", cv2.IMREAD_GRAYSCALE)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

opening = cv2.morphologyEx(noisy, cv2.MORPH_OPEN,  kernel)  # 去背景雜點
closing = cv2.morphologyEx(noisy, cv2.MORPH_CLOSE, kernel)  # 填前景孔洞

# 實務常見:先斷開再閉合,一次清乾淨
clean = cv2.morphologyEx(noisy, cv2.MORPH_OPEN, kernel)
clean = cv2.morphologyEx(clean, cv2.MORPH_CLOSE, kernel)

cv2.imwrite("opening.png", opening)
cv2.imwrite("closing.png", closing)
```

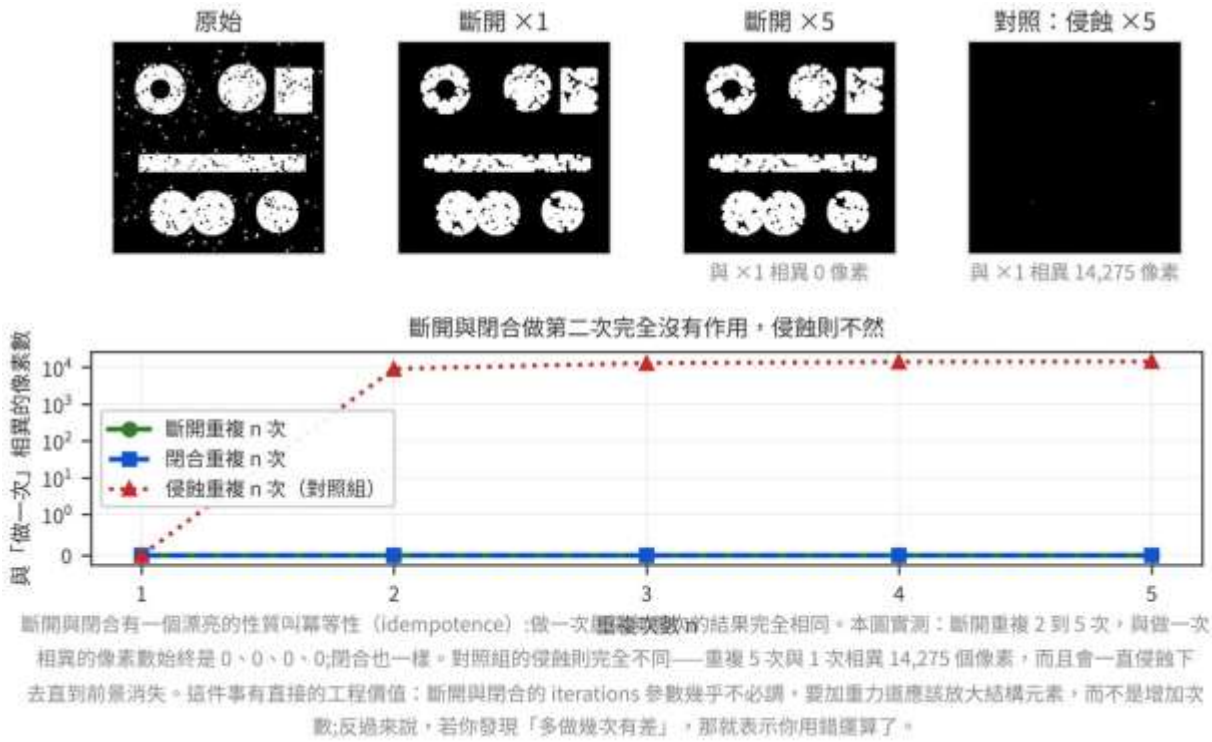


圖 10-9　冪等性：斷開與閉合做第二次完全沒有作用

10.3.3 推導：冪等性，以及為什麼順序不能交換

斷開與閉合不只是「侵蝕加膨脹」的隨意組合，它們滿足兩條很強的性質，而這兩條性質決定了你該怎麼用它們。第一條是冪等性，第二條是不可交換性——前者讓你少調一個參數，後者逼你多想一個決定。

先看冪等性。斷開的定義是：先對 A 用 B 侵蝕，再把結果用 B 膨脹。它有三個基本性質：結果一定含於原影像（遞減性）、原影像變大結果也變大（遞增性）、以及最關鍵的「做兩次等於做一次」——對斷開的結果再斷開一次，結果不變。直觀的理解是：斷開的結果可以被刻畫成「所有能完整塞進 A 裡的 B 的聯集」。既然輸出已經是這些 B 的聯集，再斷開一次時，每一個 B 當然仍然塞得進去，於是什麼都不會改變。閉合的推導對稱，只是把「塞進前景」換成「塞進背景」。

圖 10-9 把這件事實測出來：斷開重複 2、3、4、5 次，與做一次相異的像素數全部都是 0;閉合也一樣。對照組的侵蝕則完全不同——重複五次與一次相差一萬四千多個像素，而且會一直侵蝕到前景消失。工程上的結論很直接：斷開與閉合的

iterations 參數不必調，設 1 就好;要加重力道請放大結構元素，而不是增加次數。反過來說，若你發現「多做幾次結果有差」，那就表示你呼叫的其實不是斷開或閉合（例如用錯成 erode）。

再看不可交換性，這是更容易出事的一條。斷開再閉合與閉合再斷開是兩個不同的運算，而課本常寫的「先斷開再閉合，一次清乾淨」只在某些情況下成立。圖 10-8 把兩種缺陷的數量各掃四檔、共十六種組合實測：「先閉合再斷開」在十一格勝出，「先斷開再閉合」只在四格勝出，而那四格裡有三格是「完全沒有孔洞」的情形。原因不難理解：斷開是遞減運算，它咬掉的東西後面補不回來。若前景布滿孔洞，先斷開會把物件咬成碎片，之後的閉合只能把碎片連成更奇怪的形狀;反過來先閉合把物件補成實心，再斷開就能乾淨地掃掉背景雜點。

把這兩條性質合起來，得到一條可以直接寫進規格書的原則：先看一眼你的二值影像以哪一種缺陷為主，讓對症的那個運算排在前面;只要前景有像樣的孔洞，就先閉合。順序是設計管線時的第一個決定，不是最後一個微調。順帶一提，把斷開與閉合交替使用、並逐次放大結構元素的作法有個名字叫交替序列濾波（Alternating Sequential Filter），在雜訊尺度分布很廣的場合比單次運算穩健得多，代價是運算時間隨級數線性增加。

10.4 進階型態學運算

10.4.1 型態學梯度、頂帽與黑帽

1. 型態學梯度(Gradient)：膨脹減去侵蝕，得到物件的輪廓。它是另一種邊緣偵測方式，與第 9 章的 Sobel、Canny 殊途同歸，但基於形狀運算而非微分。

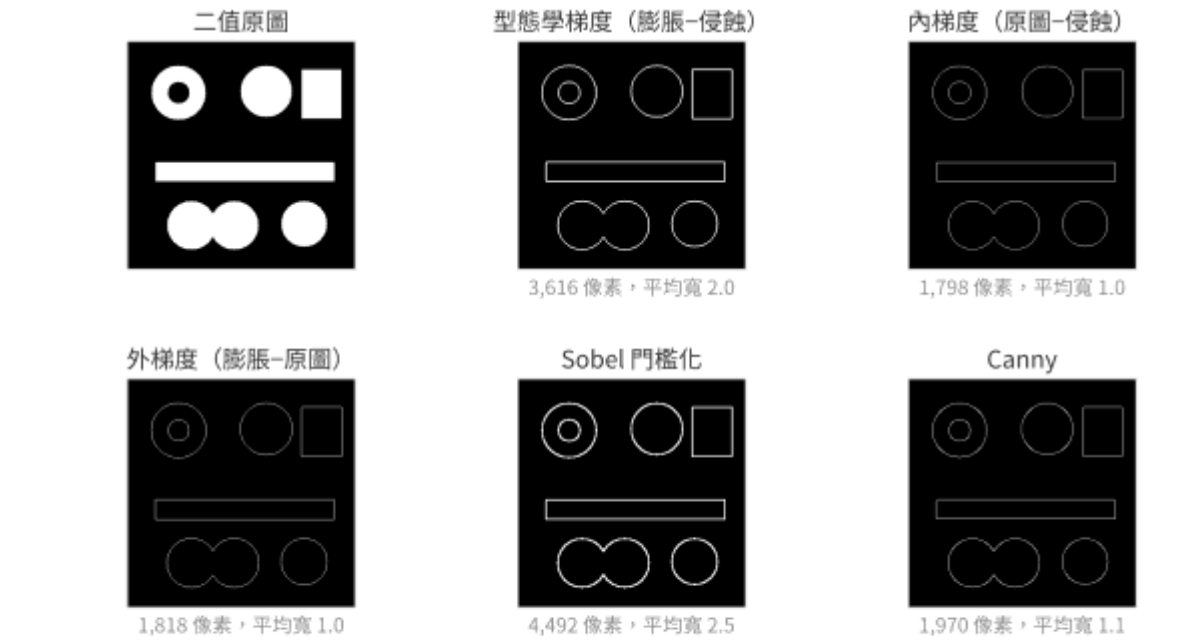


圖 10-10　型態學梯度：內梯度、外梯度與微分法的對照

2. 頂帽轉換(Top Hat)：原圖減去斷開的結果，萃取出「比周圍背景亮、且小於結構元素」的細節。它的殺手級應用是校正不均勻的背景照明——這正是第 8 章「洋裝效應」與第 9 章打光不均問題的另一種解法：用一個大的結構元素做斷開，得到的是緩慢變化的背景照明，原圖減去它，就抽出了均勻照明下的前景細節。

3. 黑帽轉換(Black Hat)：閉合的結果減去原圖，萃取出「比周圍背景暗、且小於結構元素」的細節，例如亮背景上的暗刮痕、暗字。

頂帽與黑帽在 AOI 檢測中極為實用：當工件表面因曲面或打光而有漸變的亮度背景，直接門檻分割會失敗，先做頂帽或黑帽拉平背景，瑕疵便無所遁形。這與第 6.3.3 節的同態濾波是解決同一問題(不均勻照明)的不同途徑——一個在空間域用形狀運算，一個在頻率域用對數濾波。

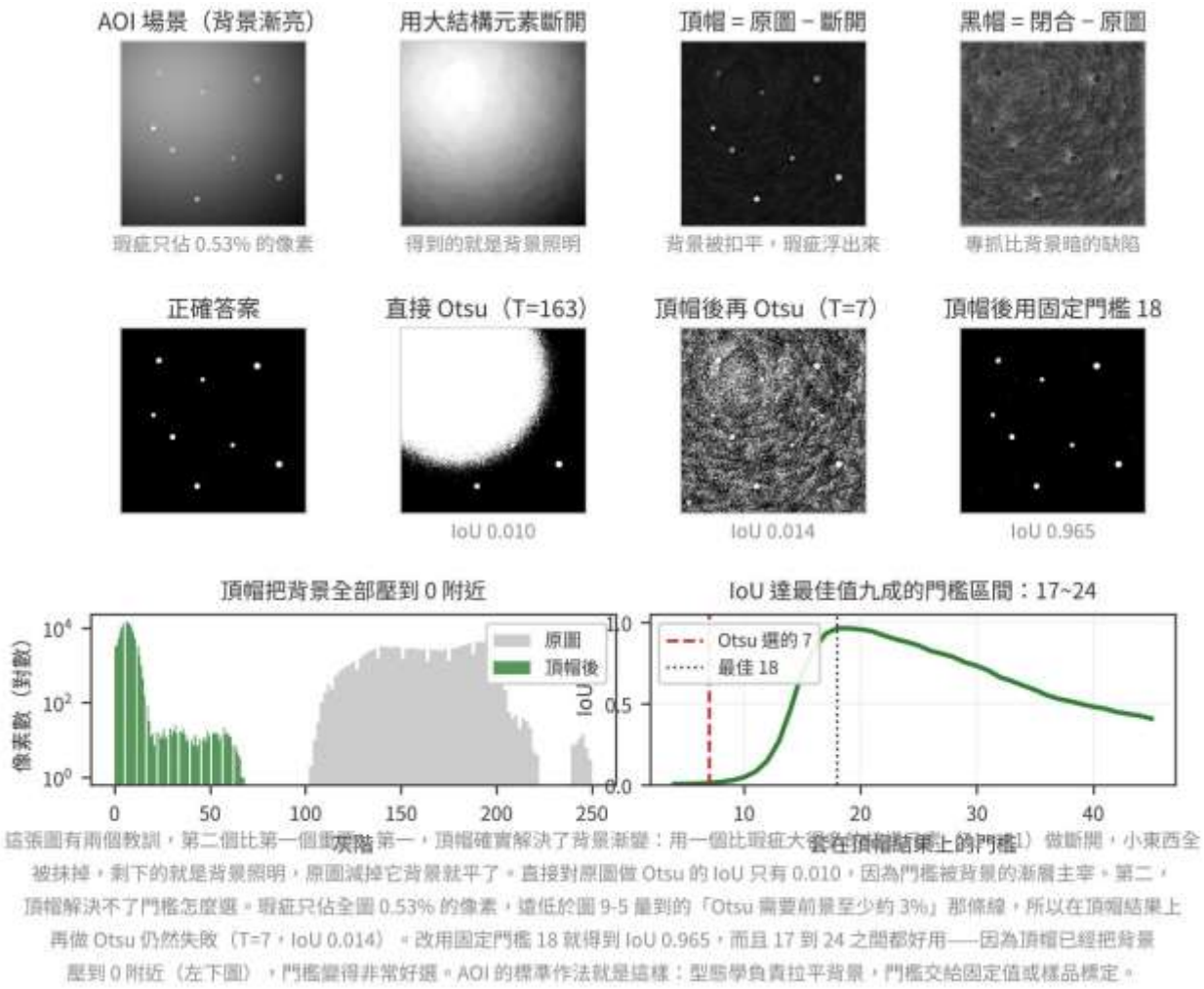


圖 10-11　頂帽拉平了背景，但門檻還是得自己選

10.4.2 Hit-or-Miss、細線化與骨架化

Hit-or-Miss 轉換是型態學的形狀偵測工具：它同時檢查「前景該在的地方是否是前景」與「背景該在的地方是否是背景」，只有兩者都吻合才命中。它能偵測特定的局部形狀，例如角點、線的端點、孤立點，是細線化與骨架化的理論基礎。

細線化(Thinning)反覆剝除物件的邊界像素，但保留其連通性與端點，最終把物件細化成單像素寬的線。骨架化(Skeletonization)則把物件化約成其「中軸」——想像從物件的每個邊界點同時向內燃燒，火線相遇熄滅之處的軌跡就是骨架。這兩者在指紋辨識(把指紋脊線細化)、文字辨識(把筆畫細化以分析結構)、電路板走線分析、道路網路萃取中不可或缺。

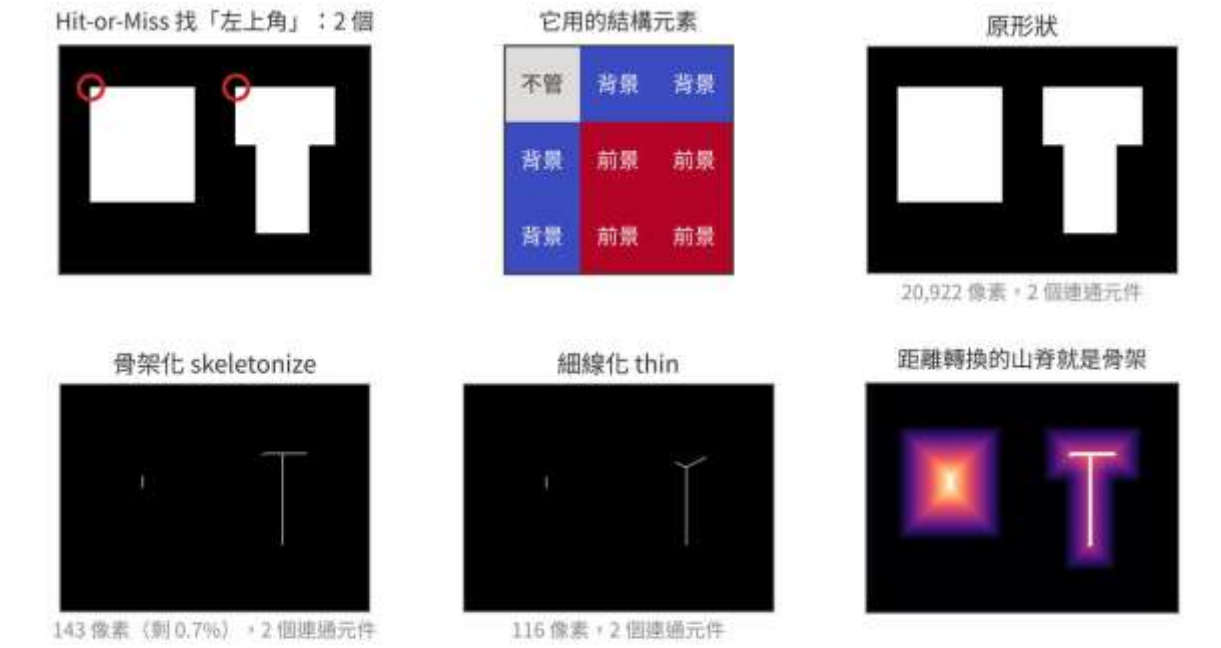
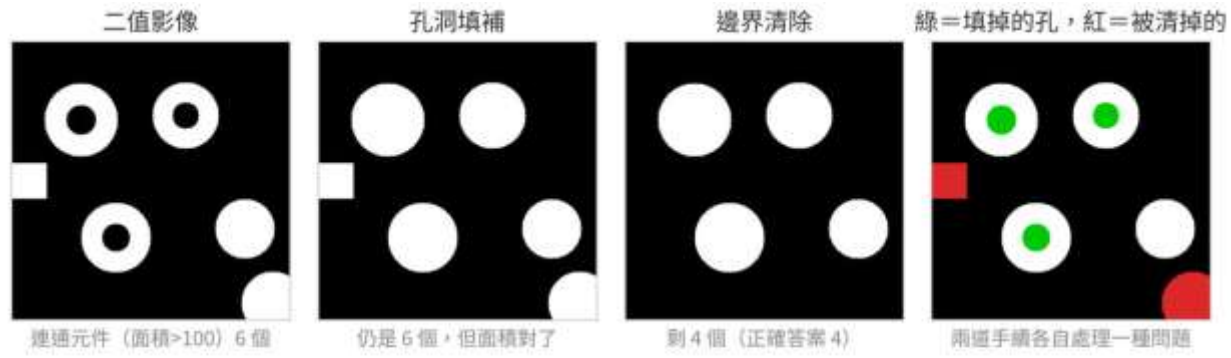


圖 10-12　Hit-or-Miss 找形狀，骨架化保住拓撲

10.4.3 孔洞填補與邊界清除

孔洞填補(Hole Filling)以型態學重建的方式，把前景物件內部完全封閉的背景區域(孔洞)填成前景。它在「數清楚有幾個實心物件」時很重要——一個有孔洞的墊圈，若不填補孔洞，連通元件標記可能會被孔洞干擾。邊界清除(Border Clearing)則移除接觸到影像邊界的物件，因為這些物件通常是「被畫面切到、資訊不完整」的，在量測與計數時應予排除——例如量測顯微鏡下的細胞尺寸時，只計算完整落在視野內的細胞。



計數之前通常要先做兩件事。孔洞填補把物件內部封閉的背景補成前景:本圖的墊圈若不填補，量到的面積是 4,624 像素，填補後是 5,525，差了 16%——若你的驗收標準是面積，這個誤差足以讓良品被判成不良。邊界清除則丟掉所有碰到影像邊緣的物件，因為它們被畫面切掉了一部分，面積與形狀都不可信;做完之後連通元件由 6 個降到 4 個，與正確答案 4 一致。順序是固定的：先填補再清除。反過來做，碰邊的物件會先被清掉，它內部的孔洞反而留下來變成假物件。

圖 10-13　孔洞填補與邊界清除：計數之前的兩道手續

10.5 連通元件標記與距離轉換

10.5.1 連通元件標記

連通元件標記(Connected Component Labeling)是把二值影像中「每一個相連的前景區域」各自賦予一個獨立編號的過程——這正式回答了「畫面中有幾個物件」這個問題。它建立在第 3 章介紹的連通性概念上：採用 4-連通或 8-連通，計數結果可能不同(第 3 章已強調過此點)。

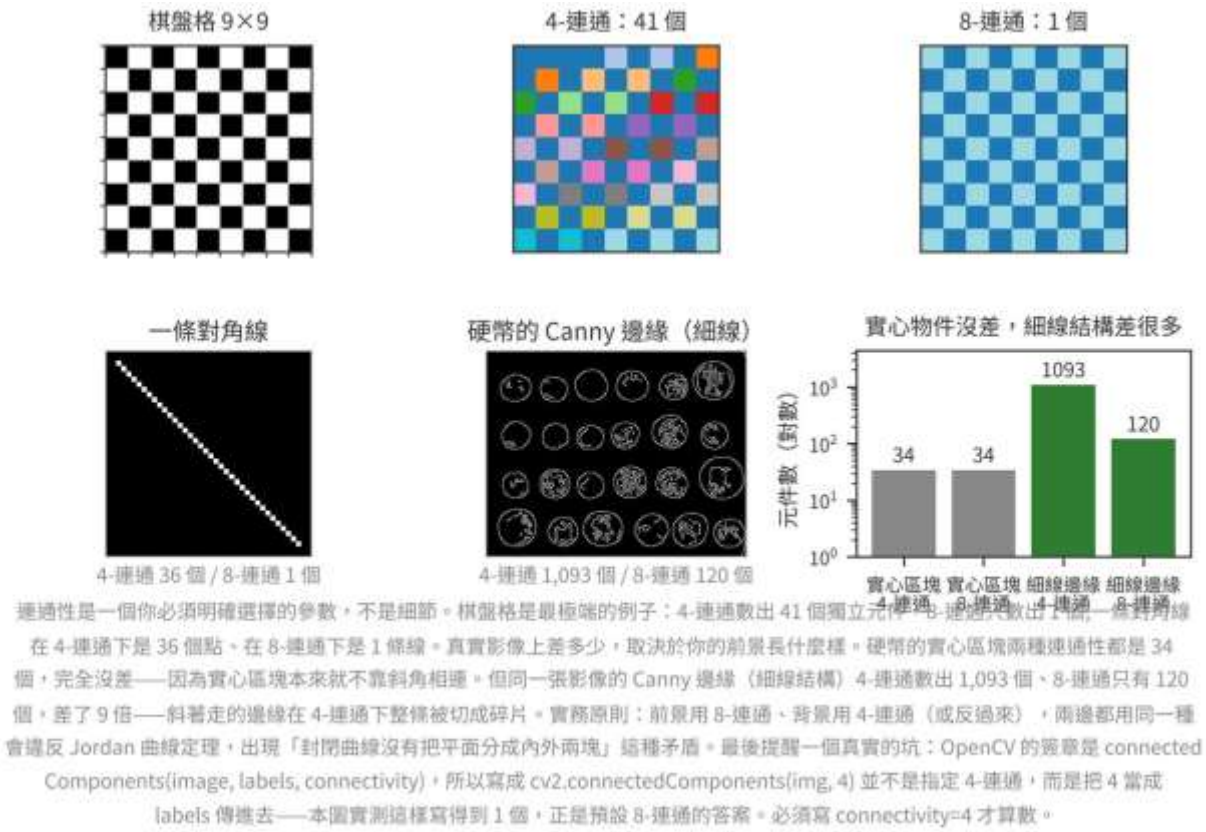


圖 10-14　4-連通還是 8-連通，細線結構上差了九倍

OpenCV 的 connectedComponentsWithStats 不僅標記，還一併回傳每個元件的統計資訊：面積(像素數)、外接矩形(位置與大小)、質心座標。這一步直接把「一堆白色像素」轉化為「一份物件清單」，是所有計數與量測應用的核心——第 9 章的零件計數、細胞計數，骨幹都在這裡。搭配面積門檻，還能輕鬆濾除殘餘的小雜點(面積過小者不計)。

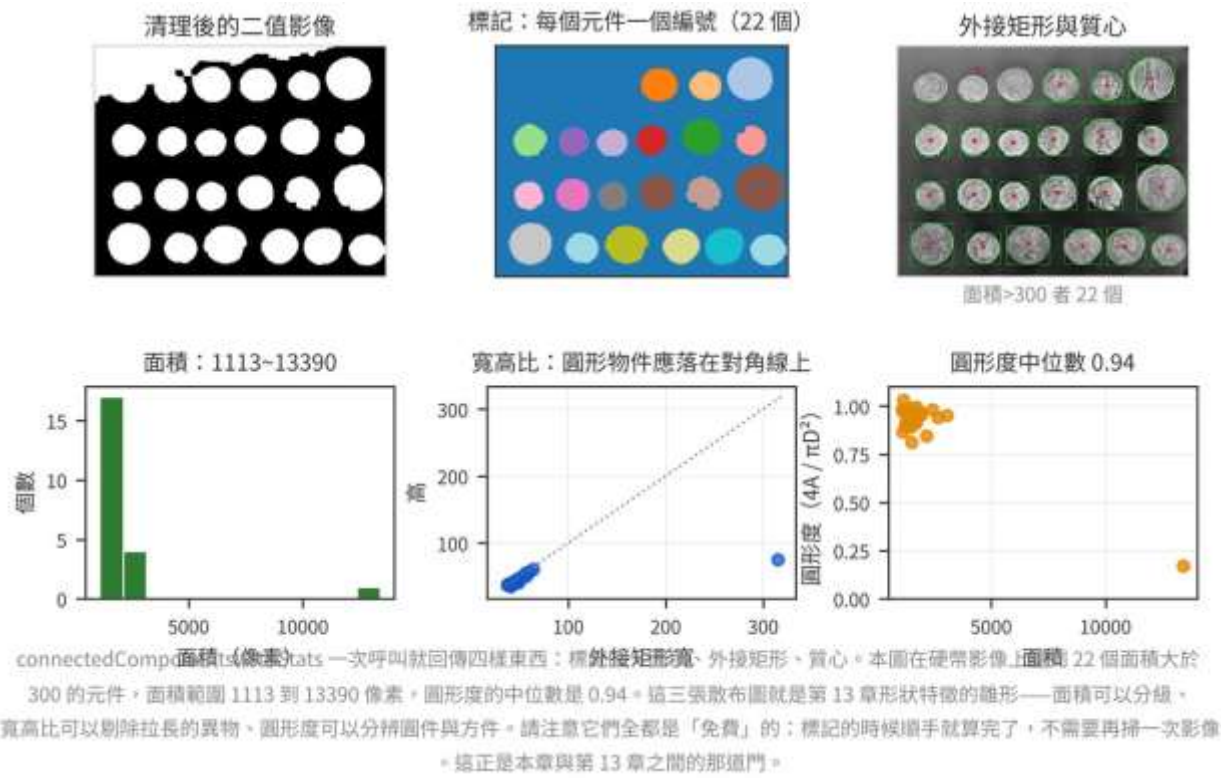


圖 10-15　連通元件標記一次給你的四樣東西

10.5.2 距離轉換

距離轉換(Distance Transform)為二值影像中的每一個前景像素，計算它到最近的背景像素的距離。結果是一張灰階圖：物件中心離背景最遠(值最大、最亮)，邊界處值最小(接近零)。它用到的正是第 3 章介紹的距離度量(歐氏、D4、D8)。

距離轉換有兩個關鍵應用。其一是求骨架：距離轉換的「山脊」(局部最大值的連線)正是物件的中軸，提供了骨架化的另一條途徑。其二是分離相黏物件：這正是第 9 章程式 9-4 分水嶺分割的核心步驟——對兩個相黏的圓形物件做距離轉換，每個圓的中心會各自形成一個亮點(局部最大)，以此作為分水嶺的「確定前景」標記，就能把黏在一起的物件正確分開。至此，第 9 章分水嶺流程中那段「距離轉換」的程式碼，其原理便完全清晰了。

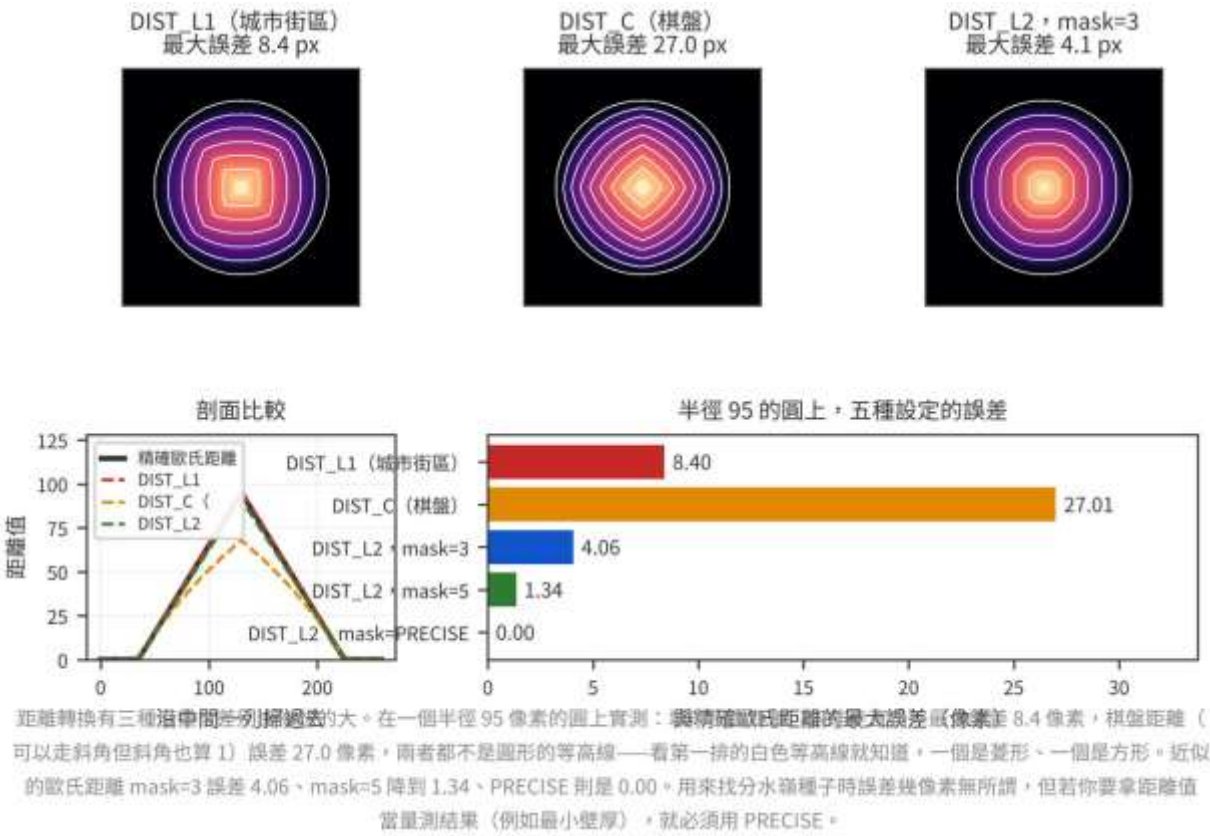


圖 10-16　距離轉換的三種度量：等高線是菱形、方形還是圓形

程式 10-4 連通元件標記與距離轉換

```
import cv2, numpy as np

binary = cv2.imread("parts.png", cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(binary, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# --- 連通元件標記 + 統計 ---
n, labels, stats, centroids = cv2.connectedComponentsWithStats(binary)
print("物件數(扣除背景):", n - 1)

output = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
for i in range(1, n):                          # 0 是背景,跳過
    area = stats[i, cv2.CC_STAT_AREA]
    if area < 50:                              # 面積門檻:濾除雜點
        continue
    x = stats[i, cv2.CC_STAT_LEFT]
    y = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]
    cx, cy = centroids[i]
    cv2.rectangle(output, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.circle(output, (int(cx), int(cy)), 3, (0, 0, 255), -1)
    print(f"物件 {i}: 面積={area}, 質心=({cx:.1f}, {cy:.1f})")

# --- 距離轉換(分離相黏物件的關鍵) ---
```

```
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
dist_vis = cv2.normalize(dist, None, 0, 255,
                         cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("distance.png", dist_vis)
cv2.imwrite("labeled.png", output)
```

10.5.3 推導：連通元件標記怎麼只掃兩遍就完成

「把每一個相連的前景區域各給一個編號」聽起來像是要反覆搜尋，但標準演算法只掃描影像兩遍，而且與物件的數量、形狀都無關。它用到的資料結構叫聯集-尋找（Union-Find），是演算法課的經典題目，也是理解「為什麼計數這麼快」的關鍵。

第一遍由上到下、由左到右掃過每個前景像素，只看它「已經掃過的鄰居」——以 8-連通為例，就是左、左上、上、右上四個。分三種情況處理：若四個鄰居都是背景，給這個像素一個全新的標籤;若鄰居只有一種標籤，就沿用它;若鄰居出現兩種以上不同的標籤，表示這幾個原本以為不同的區域其實是同一個，於是沿用其中最小的，並把「這些標籤是等價的」記錄下來。第二遍再掃一次，把每個像素的暫時標籤換成它所屬等價類的代表。

等價關係的記錄就是 Union-Find 做的事。它用一個陣列存每個標籤的「父節點」，find 沿著父節點往上找到根，union 把兩棵樹接起來。若只做這兩件事，最壞情況會退化成一條長鏈;但加上兩個標準優化——路徑壓縮（find 的時候把沿途節點直接接到根上）與按秩合併（永遠把矮的樹接到高的樹下）——之後，m 次操作的總成本是 $O(m\,\alpha(n))$，其中 α 是反阿克曼函數，在任何實際會遇到的輸入規模下都小於 5，可以當成常數。於是整個演算法的複雜度就是 $O(N)$，N 是像素數。

這個「單次掃描順手把統計算完」的特性，正是 connectedComponentsWithStats 能夠免費回傳面積、外接矩形與質心的原因。面積是計數、外接矩形是四個極值、質心是座標的累加和除以面積——三者都可以在第一遍掃描時邊走邊累加，合併等價類時再把兩份統計加起來即可，完全不需要再掃一次影像。圖 10-15 用這些「免費」的統計量畫出三張散布圖，那其實已經是第 13 章

形狀特徵的雛形了。

最後是兩個必須提醒的實務細節。第一，連通性一定要明寫。OpenCV 的簽章是 connectedComponents(image, labels, connectivity)，所以寫成 cv2.connectedComponents(img, 4) 並不是指定 4-連通，而是把 4 當成 labels 參數傳進去，連通性仍然是預設的 8——圖 10-14 實測這個寫法得到的正是 8-連通的答案。必須寫成 connectivity=4 才算數，而且這個錯誤不會報錯。第二，前景與背景應該用不同的連通性（前景 8、背景 4，或反過來）。兩邊都用同一種會違反 Jordan 曲線定理，出現「一條封閉曲線沒有把平面分成內外兩塊」這種矛盾。圖 10-14 也量到，同一張硬幣影像的實心區塊兩種連通性都是 34 個，但它的 Canny 邊緣圖在 4-連通下是 1,093 個、8-連通下只有 120 個——差了九倍。前景愈細，這個選擇愈要命。

10.6 Haar 級聯分類器與人臉偵測

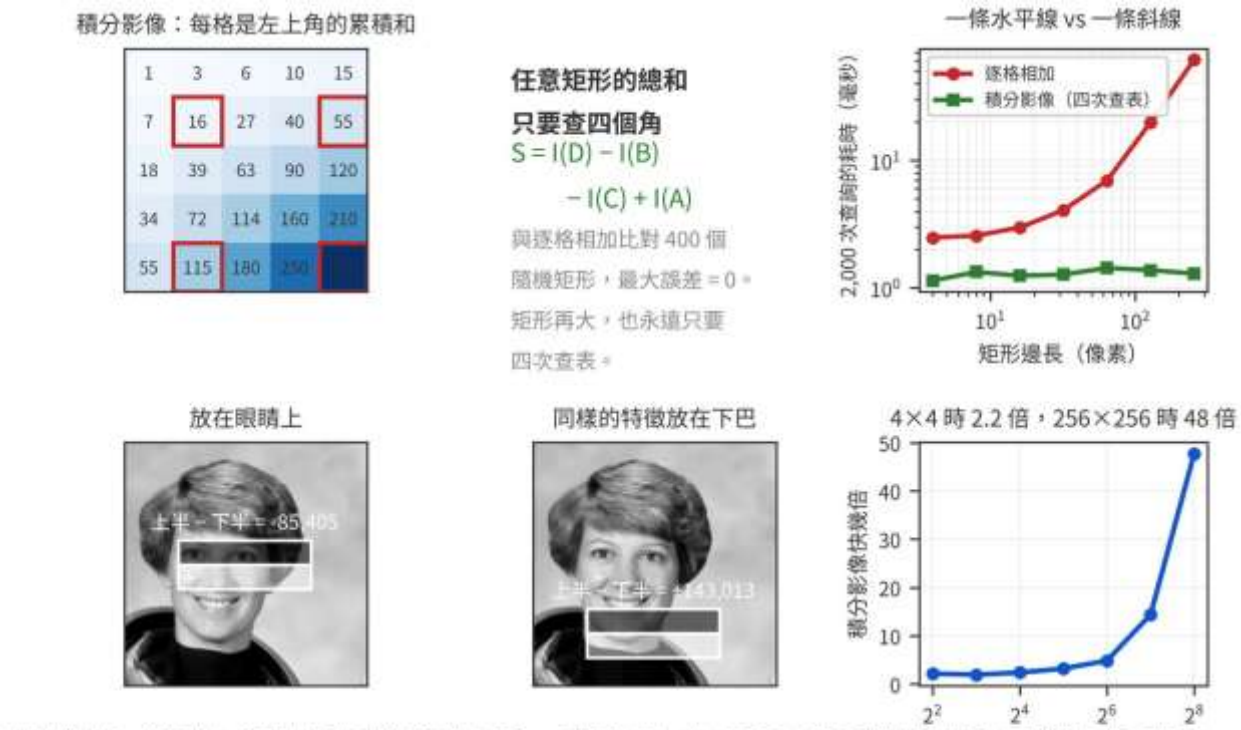
本節把叫獸開講的 Viola-Jones 演算法落實成可執行的技術，它也是本章「二值/區域思維」通往第 15、16 章「機器學習物件偵測」的橋梁。

10.6.1 三大支柱回顧

承叫獸開講，Viola-Jones 由三個技術支柱撐起。第一，Haar-like 特徵：計算相鄰矩形區域的亮度差，捕捉人臉的明暗規律(如眼睛比臉頰暗、鼻樑比兩側亮)。第二，積分影像：預先計算每點左上方所有像素的累加和，使任意矩形區域的總和只需查表三次加減即可求得，與區域大小無關——這是它能高速計算海量特徵的關鍵。第三，AdaBoost 與級聯：一個偵測視窗中可能的 Haar 特徵多達十八萬個，AdaBoost 從中篩選出最有鑑別力的少數;再把分類器串成級聯，前段快速淘汰非人臉區域，後段精細確認，達成即時偵測。

值得一提的是，積分影像的技巧與本章 10.5 節連通元件標記「單次掃描完成統計」的精神相通，都是「用預先計算換取查詢時的常數時間」——這是演算法設計中極重

要的空間換時間思維，學生應細細體會。



積分影像的每一格存的是「從左上角到這裡的所有像素總和」，建表只要掃一遍。有了它，任意矩形的總和都只要查四個角、做三次加減——本圖以 400 個隨機矩形驗證，與逐格相加的最大誤差是 0。代價是固定的，所以矩形愈大賺愈多：邊長 4 時快 2.2 倍，邊長 256 時快 48 倍。Haar-like 特徵就是幾個矩形總和的加減。本圖的雙矩形特徵問的是「上半比下半暗多少」：放在眼睛帶上得到 -85,405（眼窩比顴骨暗，所以是負的），同一個特徵原封不動移到下巴卻是 +143,013（正的）。一正一負，光靠這一個特徵就能排除掉不少位置——這就是級聯第一關在做的事。這個「預先累加換取查詢時的常數時間」的手法，與 10.5 節連通元件「單次掃描完成統計」是同一種思維。

圖 10-17　Haar-like 特徵與積分影像：矩形再大也只要四次查表

10.6.2 OpenCV 實作

OpenCV 內建訓練好的 Haar 級聯模型(XML 檔)，涵蓋正面人臉、眼睛、微笑、全身等，可直接載入使用。

程式 10-5 Haar 級聯人臉與眼睛偵測

```
import cv2

# 載入 OpenCV 內建的正面人臉 Haar 級聯模型
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_eye.xml")

img = cv2.imread("group_photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)   # 只需灰階

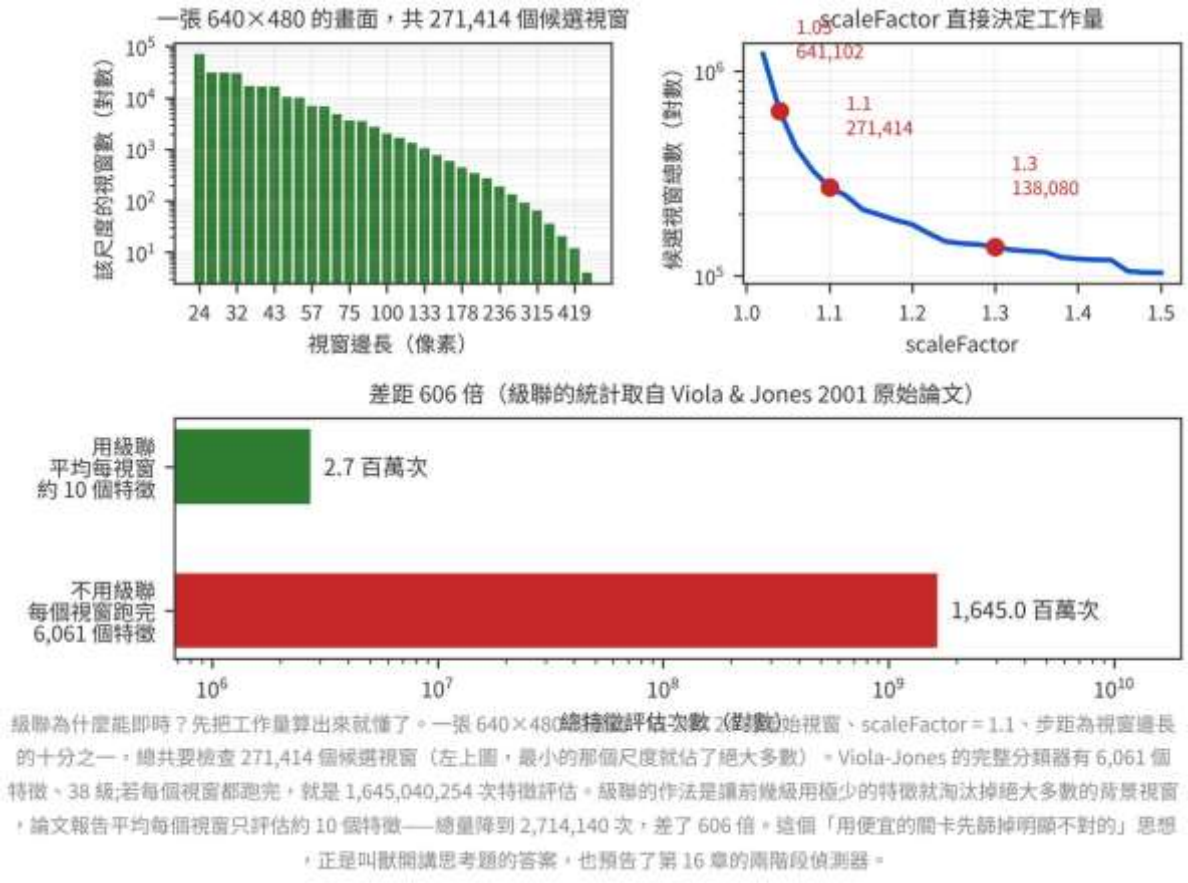
# detectMultiScale:在多種尺度下滑動視窗偵測
faces = face_cascade.detectMultiScale(gray,
```

```
            scaleFactor=1.1,       # 每次縮放比例(影響速度與漏檢)
            minNeighbors=5,        # 需多少鄰近偵測才算數(抑制誤報)
            minSize=(30, 30))      # 最小人臉尺寸

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    roi_gray = gray[y:y+h, x:x+w]              # 第 2 章的 ROI
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(img[y:y+h, x:x+w], (ex, ey),
                      (ex+ew, ey+eh), (255, 0, 0), 2)

print("偵測到人臉數:", len(faces))
cv2.imwrite("faces_detected.jpg", img)
```

兩個參數的調校心法：scaleFactor 愈接近 1，搜尋的尺度愈細密、愈不易漏檢，但速度愈慢;minNeighbors 愈大，愈能抑制誤報(要求同一位置有多個尺度都偵測到才確認)，但也可能漏掉真實人臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 的雙門檻、第 16 章物件偵測的信心門檻，是同一類工程取捨。



級聯為什麼能即時？先把工作量算出來就懂了。一張 640×480 的畫面，scaleFactor = 1.1、步距為視窗邊長的十分之一，總共要檢查 271,414 個候選視窗（左上圖，最小的那個尺度就佔了絕大多數）。Viola-Jones 的完整分類器有 6,061 個特徵、38 級;若每個視窗都跑完，就是 1,645,040,254 次特徵評估。級聯的作法是讓前幾級用極少的特徵就淘汰掉絕大多數的背景視窗，論文報告平均每個視窗只評估約 10 個特徵——總量降到 2,714,140 次，差了 606 倍。這個「用便宜的關卡先篩掉明顯不對的」思想，正是叫獸關講思考題的答案，也預告了第 16 章的兩階段偵測器。

圖 10-18　級聯為什麼能即時：先把候選視窗數算出來

10.6.3 三種辨識架構與 ESP32-CAM 部署

回顧第 2 章介紹的三種 AI 辨識架構——端上運算、雲端運算、近端邊緣運算——Viola-Jones 的輕量特性(不需 GPU、記憶體需求小)使它成為端上運算的理想選擇。事實上，ESP32-CAM 的官方韌體就內建了人臉偵測與辨識功能，正是基於這類輕量演算法。

這帶出一個重要的部署思考：在 ESP32-CAM 這樣算力極為有限的裝置上，深度學習的 YOLO(第 16 章)難以即時執行，而 Viola-Jones 這類傳統方法反而遊刃有餘。這再次印證第 9 章的判斷原則：方法沒有絕對優劣，只有適不適合。當任務是「在低功耗感測節點上偵測正面人臉」時，一個 2001 年的演算法可能比 2020 年代的深度模型更務實。第 17 章會深入這個部署決策。



圖 10-19　OpenCV 5 的現實：Haar 級聯的 API 已經不在了

10.6.4 現況提醒：程式 10-5 需要 OpenCV 4.x

這一節有一個必須講清楚的版本問題。本書插圖的產生環境是 OpenCV 5.0.0（opencv-python 5.0.0.93），在這個版本上實測：cv2.CascadeClassifier 這個類別已經不存在，cv2.data.haarcascades 指向的目錄裡也不再附帶任何 XML 模型檔，連

cv2.HOGDescriptor（行人偵測的傳統作法）也一併消失了。也就是說，程式 10-5 照著打會直接拋出 AttributeError，而不是跑出一個空的偵測結果。圖 10-19 把實測的結果整理成一張表。

要怎麼辦，看你的目的。若你要照著程式 10-5 把 Haar 級聯跑一遍（建議初學者這樣做，因為它最能體會 detectMultiScale 兩個參數的影響），請安裝 OpenCV 4.x:pip install "opencv-python<5"。Haar 級聯與內建的 XML 模型在 4.x 中都還在，ESP32-CAM 端上的函式庫也多半是以 4.x 為基礎移植的，所以 Lab 10-3 完全不受影響。

若你用的是現行版本而目的只是「把人臉框出來」，可以改用 cv2.FaceDetectorYN（YuNet）。它是一個很小的卷積網路，模型檔只有數百 KB，需要另外下載;在精度與對側臉、遮擋的容忍度上都明顯勝過 Haar，速度在一般 CPU 上也足以即時。搭配 cv2.FaceRecognizerSF（SFace）還能做人臉特徵比對。這兩個類別在 OpenCV 5 中都還在，本書實測可用。

最後要強調的是：本節真正要教的東西不受這個變動影響。積分影像讓任意矩形的總和降為四次查表（圖 10-17）、級聯用便宜的關卡先淘汰絕大多數候選視窗（圖 10-18），這兩個思想不但沒有過時，還直接出現在第 16 章的兩階段偵測器與第 17 章的邊緣部署策略裡。API 會變，演算法的想法不會。這也提醒你一件事：教科書上的程式碼有保存期限，讀程式庫的版本變更紀錄（CHANGELOG）是工程師的基本功，而不是等到跑出錯誤才回頭查。

10.7 應用案例

10.7.1 案例一：PCB 與工件的瑕疵前處理

情境：AOI 系統要檢測印刷電路板上的斷路、短路、缺件。分割(第 9 章)後得到的二值影像，線路邊緣常有毛刺、線路中有雜點造成的假斷點。處理流程：先用斷開去

除毛刺與孤立雜點 → 用閉合填補線路上因反光造成的假斷點 → 用型態學梯度萃取乾淨的線路輪廓 → 用細線化把線路化約成單像素骨架，便於分析走線的連通性與交叉點。若背景照明不均，則在分割前先用頂帽轉換拉平背景。這整套流程展示了型態學作為「分割與量測之間的清理工序」的核心價值。

10.7.2 案例二：細胞與顆粒計數

情境：顯微鏡影像中要自動計數細胞或顆粒，且它們常相互接觸。這是本章與第 9 章的完整綜合：Otsu 二值化 → 斷開去雜點 → 孔洞填補(把細胞內的亮點填實)→ 邊界清除(排除被視野邊緣切到的不完整細胞)→ 對相黏的細胞做距離轉換，取局部極大作為標記 → 分水嶺分割分開它們 → 連通元件標記計數並量測每個細胞的面積。生物醫學、水質檢測、材料科學的顆粒分析，骨幹都是這條管線。

10.7.3 案例三：即時人臉偵測門禁

情境：用 ESP32-CAM 打造一個低成本的門禁裝置，偵測到人臉時觸發後續動作(拍照上傳、開門、記錄)。這是叫獸開講與第 2 章三架構的實戰整合：ESP32-CAM 在端上以 Haar 級聯即時偵測人臉(端上運算，低延遲、保護隱私)，偵測到後再把影像上傳雲端做進一步的身分辨識(雲端運算，借助強大算力)——這正是第 2 章「近端/雲端混合架構」的體現：輕量的偵測放在端上、耗算力的辨識放在雲端，兼顧即時性與準確度。Lab 10-3 會實作這個系統的偵測端。

10.7.4 案例四：車牌字元分割

情境：車牌辨識(第 13 章會完整處理)的前段，需要把車牌上的每個字元切開。流程：定位並裁切出車牌區域 → 轉灰階、Otsu 二值化 → 用閉合連接字元中斷裂的筆畫(如數字 8 因印刷不良而斷開)→ 用斷開去除邊框雜點 → 連通元件標記，每個連通區域就是一個字元 → 依面積、長寬比濾除非字元的雜訊區域 → 依 x 座標排序，得到字元序列。這條「二值化 → 型態學清理 → 連通元件分割字元」的流程，是所有文字/字元辨識(OCR)的共通前處理，也直接銜接第 13 章的特徵擷取與第 15 章的字元分類。



圖 10-20　車牌字元分割：本章工具的綜合演練

10.8 本章與全書的觀念連結總表

相關章節	共用的觀念	二值處理中的角色
第 3 章 數位影像基礎	連通性、距離度量	連通元件標記與距離轉換的理論基礎
第 6 章 頻率域	不均勻照明校正	頂帽轉換與同態濾波是同一問題的兩種解法
第 7 章 影像還原	最大值/最小值濾波	灰階膨脹即最大值濾波、侵蝕即最小值濾波
第 9 章 影像分割	二值化、分水嶺	型態學是分割輸出的直接清理工序;距離轉換供分水嶺標記
第 13 章 特徵擷取	輪廓、形狀特徵、車牌	連通元件提供物件清單;字元分割接特徵擷取

第 15、16 章 深度學習	Haar 級聯、物件偵測	Viola-Jones 是深度學習偵測器的前輩
第 17 章 邊緣部署	三種辨識架構、ESP32-CAM	輕量的 Haar 級聯適合端上運算

表 10-1 第 10 章與全書其他章節的觀念連結

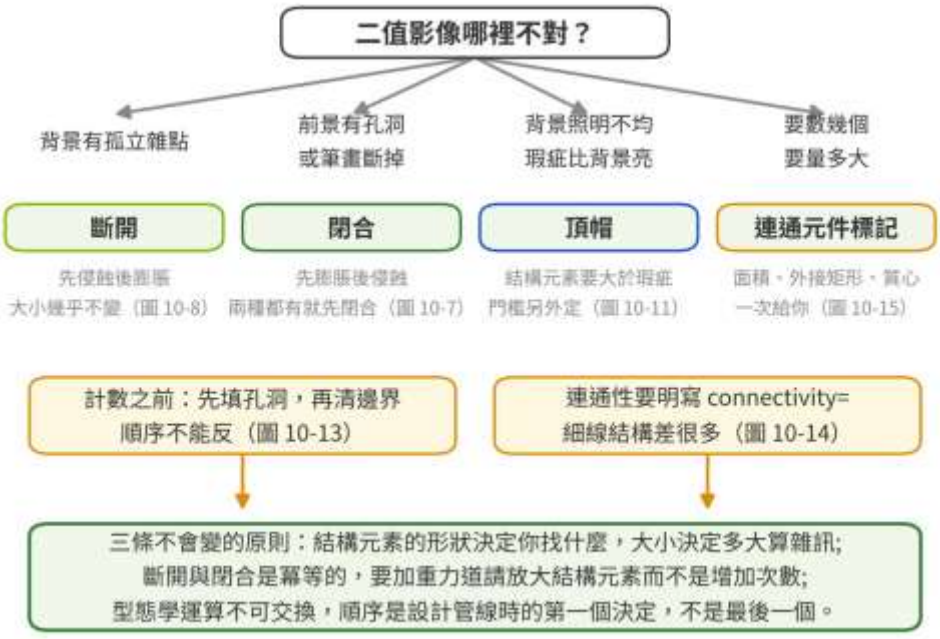


圖 10-21　第 10 章的決策地圖

程式實作 Lab 10

Lab 10-1 型態學運算探索器

目標：建立對型態學運算的直覺。(a)取一張含雜訊的二值影像(可對乾淨二值圖加椒鹽雜訊);(b)以 cv2.createTrackbar 建立可調整結構元素形狀(方/十字/橢圓)、大小、運算類型(侵蝕/膨脹/斷開/閉合)與 iterations 的即時介面;(c)實驗並記錄：去除背景孤立雜點該用哪個運算?填補前景小孔該用哪個?斷開細頸相連的兩物件該用哪個?(d)驗證對偶性：對前景侵蝕，與對背景膨脹(先反轉、膨脹、再反轉)，結果是否相同?(e)驗證與第 7 章的關聯：對灰階影像做膨脹，結果是否等同於最大值濾波?

Lab 10-2 完整計數量測管線

目標：整合第 9、10 章完成一條可用的計數量測系統。(a)拍一張散落的螺絲、鈕扣或豆子照片;(b)依序完成：CLAHE 前處理(第 5 章)→ Otsu 二值化(第 9 章)→ 斷開去雜點 → 閉合填孔 → 孔洞填補 → 邊界清除 → 連通元件標記;(c)以面積門檻濾除雜點，框出每個物件、標示質心與編號、印出總數與各自面積;(d)進階：若物件相黏，加入距離轉換與分水嶺(第 9 章程式 9-4)正確分開，比較加入前後的計數差異;(e)量測校準：放一個已知尺寸的參考物(如硬幣)，換算出每像素代表的實際毫米數(呼應第 4 章相機校正)，輸出每個物件的實際尺寸。

Lab 10-3 ESP32-CAM 人臉偵測門禁

目標：實作叫獸開講與 10.6.3 節的端上人臉偵測。(a)先在電腦上以程式 10-5 跑通 Haar 級聯人臉偵測，調校 scaleFactor 與 minNeighbors，記錄「漏檢」與「誤報」如何隨參數變化;(b)改以 ESP32-CAM 的串流網址作為輸入來源(第 2、8 章);(c)當偵測到人臉持續數影格，觸發動作(存檔、亮 LED、印出時間戳記);(d)量測 FPS，比較「電腦端」與「ESP32-CAM 端上」的效能差異;(e)討論題：承第 2 章三架構——這個門禁若要加上「辨識是誰」的功能，偵測與辨識應各自放在端上、近端還是雲端?請畫出你的系統架構圖並說明理由。

Lab 10-4 型態學的代數性質實測

目標：把本章的幾條性質從「課本說」變成「我量過」。(a)寫一個 diff(a, b) 函式回傳兩張二值影像相異的像素數;(b)驗證對偶性：確認「對前景侵蝕」與「對背景膨脹再取反」的差為 0，斷開與閉合的對應關係也做一次;(c)驗證冪等性：把斷開重複 1 到 10 次，畫出「與做一次的差異」對次數的曲線，再用侵蝕做同一件事當對照組;(d)驗證結合律：比較「3×3 膨脹兩次」與「5×5 膨脹一次」，說明為什麼相等（提示：3×3 膨脹 3×3 會得到什麼）;(e)驗證可分離性：比較 k×k 方形結構元素與「1×k 接 k×1」的結果與耗時，把 k 由 3 掃到 31，畫出兩條時間曲線;(f)寫出 300 字的結論：這五條性質各自讓你在寫程式時少做了什麼決定。

Lab 10-5 順序之爭：自己掃一張表

目標：親手確認型態學運算不可交換，並找出自己場景的最佳順序。(a)取一張乾淨的二值影像當正確答案（可用幾何圖形合成），寫兩個函式分別加入「背景雜點」與「前景孔洞」，數量可調;(b)寫一個 err(m) 函式回傳與正確答案相異的像素數;(c)把雜點數與孔洞數各掃四到五檔，對每一種組合分別計算「先斷開再閉合」與「先閉合再斷開」的誤差，做成一張熱圖（參考圖 10-8）;(d)標出每一格的勝者，看看你的結論與本書的「只要有孔洞就先閉合」是否一致;(e)進階：加入交替序列濾波（結構元素由小到大，斷開閉合交替），看它能不能在整張表上都贏過固定順序，並記錄它多花了多少時間;(f)把你的熱圖與結論寫成一頁報告——這就是產線參數規格書的雛形。

Lab 10-6 從二值影像到量測報告

目標：把第 9、10 兩章串成一條能交差的完整管線。(a)拍一張散落的墊圈、螺帽或鈕扣照片（最好有背光，若無則請記錄光源條件）;(b)依序完成：灰階化 → CLAHE 或頂帽拉平背景（第 5 章、10.4 節）→ Otsu 或自適應門檻（第 9 章）→ 型態學清理（順序請用 Lab 10-5 的結論）→ 孔洞填補 → 邊界清除 → connectedComponentsWithStats;(c)用一枚已知外徑的標準件做標定，把像素換算成毫米，輸出每一件的外徑、面積與圓形度;(d)重複拍攝同一盤件十次（中間不要移動），計算每一項量測值的標準差——這是你的重複性;(e)刻意改變一個條件（移動光源、換曝光、把件稍微挪動），再拍十次，比較兩組的差異，判斷你的系統對什麼最敏感;(f)寫出一份含「量測值、重複性、已知的失效條件」三節的報告。請注意第三節最重要：一套說不出自己什麼時候會壞的系統，不能上線。

本章重點整理

- 二值影像每像素僅前景/背景兩值，多為第 9 章分割的產物;型態學以集合論為基礎，用結構元素探測前景形狀，結構元素的形狀與大小是處理關鍵。

- 侵蝕使前景縮水、消滅小細節(對灰階即最小值濾波);膨脹使前景長大、填補小孔(對灰階即最大值濾波);兩者為對偶運算。
- 斷開(先侵蝕後膨脹)去除背景雜點且維持主體大小;閉合(先膨脹後侵蝕)填補前景孔洞且維持主體大小;是分割後清理的常用組合。
- 型態學梯度求輪廓;頂帽萃取亮細節並校正不均勻背景(與同態濾波同功);黑帽萃取暗細節;Hit-or-Miss 偵測特定形狀，為細線化與骨架化的基礎。
- 連通元件標記回答「有幾個物件」並提供面積、外接矩形、質心等統計;距離轉換為每個前景像素計算到最近背景的距離，用於求骨架與分離相黏物件(分水嶺標記)。
- Viola-Jones 以 Haar 特徵、積分影像、AdaBoost 級聯三支柱達成即時人臉偵測，輕量而適合 ESP32-CAM 等端上部署;擅長正面人臉，側臉與遮擋則需第 16 章的深度學習方法。

習題

1. 二值影像通常從何而來?為什麼把影像化約成非黑即白反而有價值?分割產生的二值影像常有哪些缺陷?

   > *提示與參考解答：多來自第 9 章的分割(門檻、Otsu、邊緣偵測)。價值在於：當只關心物件的位置、形狀、數量時，二值化讓形狀分析極為高效。常見缺陷：前景內有小孔洞(反光/雜訊)、背景有孤立小白點、該分開的物件相黏、該相連的物件斷裂——這些正是型態學要清理的。*

2. 什麼是結構元素?它與空間濾波的核有何異同?為什麼選擇不同形狀的結構元素會影響結果?

   > *提示與參考解答：結構元素是型態學用來探測前景的小形狀，相當於濾波的核，但作用是集合運算而非數值加權。形狀(方/十字/圓/線)會影響結果：例如水平線形結構元素會強化水平結構、削弱垂直結構;圓形則各方向一視同仁。選對形狀與大小是型態學的關鍵。*

3. 說明侵蝕與膨脹的規則與效果，並指出它們與第 7 章哪兩種濾波等價。

> *提示與參考解答：侵蝕：結構元素完全落在前景內時原點才保留，效果是前景縮水、消滅小細節，等價於灰階的最小值濾波(去鹽雜訊)。膨脹：結構元素與前景有交集即設為前景，效果是前景長大、填補小孔，等價於最大值濾波(去胡椒雜訊)。*

4. 斷開與閉合各是什麼順序的組合?為什麼它們能在去除缺陷的同時大致維持物件原本大小?各自的口訣是什麼?

> *提示與參考解答：斷開=先侵蝕後膨脹，去除背景雜點(「開」掉背景雜訊);閉合=先膨脹後侵蝕，填補前景孔洞(「閉」合前景破洞)。維持大小的原因：一縮一脹(或一脹一縮)相互抵消，主體回到接近原始尺寸，只有小於結構元素的缺陷被永久去除或填補。*

5. 頂帽轉換如何校正不均勻的背景照明?它與第 6 章的哪個技術解決同一問題?兩者作用的域有何不同?

> *提示與參考解答：頂帽=原圖減去斷開結果。用大結構元素做斷開會得到緩慢變化的背景照明，原圖減去它即抽出均勻照明下的前景亮細節。它與第 6.3.3 節的同態濾波解決同一問題(不均勻照明)：頂帽在空間域用形狀運算，同態濾波在頻率域用對數加高低頻分離。*

6. 細線化與骨架化的目的是什麼?各舉一個應用。骨架化與距離轉換有何關聯?

> *提示與參考解答：目的是把物件化約成單像素寬的線或中軸，以分析其拓撲結構。細線化應用：指紋脊線細化、文字筆畫分析。骨架化應用：電路走線分析、道路網路萃取。關聯：距離轉換的山脊(局部最大值連線)正是物件中軸，提供求骨架的另一途徑。*

7. 連通元件標記回答什麼問題?它建立在第 3 章的什麼概念上?connectedComponentsWithStats 除了標記還提供哪些資訊?

> *提示與參考解答：回答「畫面中有幾個相連的前景區域(物件)」。建立在第 3 章的連通性(4-連通或 8-連通，選擇不同計數可能不同)。額外資訊：每個元件的面積、外接矩形(位置與大小)、質心座標，可直接用於計數與量測，並以面積門檻濾除雜點。*

8. 距離轉換如何幫助分開相黏的物件?請說明它在第 9 章分水嶺流程中扮演的角色。

提示與參考解答：對相黏的圓形物件做距離轉換，每個物件的中心離背景最遠、形成一個局部極大亮點。取這些亮點作為分水嶺的「確定前景」標記，由此注水，即可在兩物件的接縫處築起分界，正確分開它們。這就是第 9 章程式 9-4 中距離轉換那一步的原理。

9. Viola-Jones 的三大技術支柱是什麼?各自解決了什麼問題?

   提示與參考解答：一、Haar-like 特徵：以相鄰矩形亮度差捕捉人臉明暗規律(如眼比頰暗)。二、積分影像：預先累加，使任意矩形區域總和查表三次加減即得、與大小無關，解決海量特徵的計算速度。三、AdaBoost 級聯：從十八萬個特徵篩選少數有效者，並串成級聯前段快速淘汰非人臉、後段精細確認，達成即時偵測。

10. 積分影像的「預先計算換取常數時間查詢」思想，與本章哪個技術相通?這屬於哪一類演算法設計思維?

    提示與參考解答：與連通元件標記的「單次掃描完成統計」相通，也與部分實作中的累加思想一致。這屬於「空間換時間」的演算法設計思維：預先付出計算與儲存成本，換取後續查詢時的極高速度。

11. detectMultiScale 的 scaleFactor 與 minNeighbors 各控制什麼?調大或調小如何影響「漏檢」與「誤報」?這與第 9、16 章的哪些取捨同類?

    提示與參考解答：scaleFactor 是每次尺度縮放比例：愈接近 1 搜尋愈細密、愈不漏檢但愈慢。minNeighbors 是確認一個偵測所需的鄰近偵測數：愈大愈能抑制誤報但可能漏掉真臉。這種「漏檢 vs. 誤報」的權衡，與第 9 章 Canny 雙門檻、第 16 章物件偵測的信心門檻同類。

12. (綜合題)為什麼在 ESP32-CAM 上偵測人臉，2001 年的 Viola-Jones 可能比 2020 年代的 YOLO 更合適?請以第 2 章三架構與第 9 章「方法沒有絕對優劣」的觀點論證。

    提示與參考解答：ESP32-CAM 算力與記憶體極為有限，YOLO 等深度模型難以即時執行;Viola-Jones 輕量、不需 GPU，適合端上運算(低延遲、保護隱私、不需連網)。以第 2 章三架構看，輕量偵測放端上最務實;若需辨識身分再上傳雲端。呼應第 9 章：方法沒有絕對優劣，只有適不適合任務與硬體——在低功耗節點偵測正面人臉的場景，舊演算法反而勝出。

13. 為什麼說「對灰階影像而言，侵蝕就是最小值濾波」不是巧合而是必然?請由侵蝕的集合定義推導出來。

    提示與參考解答：二值侵蝕要求「結構元素覆蓋範圍內每一點都是前景」，也就是「覆蓋範圍內的值最小者仍然是 1」。把這句話裡的 1 換成任意灰階，就得到「輸出 = 覆蓋範圍內輸入的最小值」，正是最小值濾波。膨脹的推導對稱：把「完全包含」換成「有交集」、「最小值」換成「最大值」。圖 10-4 把 cv2.erode 與 scipy 的 minimum_filter 逐像素比對，最大差異為 0。

14. 一張影像只有鹽雜訊（白點）。只做侵蝕會發生什麼事?請以 RMSE 說明，並指出正確的作法。

    提示與參考解答：侵蝕確實會消滅白點（取最小值），但它同時把每個像素都換成鄰域的最小值，整張影像跟著變暗。圖 10-4 實測：RMSE 由 20.9 不降反升。正確作法是做完整的斷開（蝕完再膨脹回來），RMSE 降到 10.1。更值得注意的是：對一張完全乾淨的影像做同樣的斷開，RMSE 已經是 10.1——兩者幾乎相等，表示雜訊已經被清光，剩下的全是運算本身的平滑代價。這個「先量上限再評估方法」的作法與圖 9-6 是同一套思路。

15. 請寫出侵蝕與膨脹的對偶關係，並說明它在實務上怎麼幫你省力。

    提示與參考解答：「對 A 侵蝕」等於「對 A 的補集膨脹之後再取補集」（結構元素需翻轉 180 度，但對稱的結構元素可省略）;斷開與閉合之間也有同樣的對偶關係。圖 10-5 把兩條式子逐像素驗證，相異的像素數都是 0。省力之處有二：一是只需要熟悉其中一半，另一半用「換個角度看背景」即可推得;二是當你猶豫該侵蝕還是膨脹時，可以改問「我是想縮前景，還是想縮背景」。

16. 膨脹滿足結合律，這對 cv2.erode/dilate 的 iterations 參數有什麼含意?它與第 5 章的可分離濾波有什麼關聯?

    提示與參考解答：結合律表示「用 B1 膨脹再用 B2 膨脹」等於「用 B1 膨脹 B2 所得的結構元素膨脹一次」，所以用 3×3 膨脹兩次等於用 5×5 膨脹一次——iterations 與放大結構元素是等效的。關聯：k×k 的方形結構元素可以拆成 1×k 的水平接 k×1 的垂直，運算量由 k^2 降到 $2k$，這正是第 5.3.3 節可分離濾波在型態學上的對應物。

17. 什麼是冪等性?斷開與閉合有，侵蝕沒有。這個差別對參數調校有什麼直接的影響?

    提示與參考解答：冪等性指「做一次與做無數次結果相同」。斷開的結果可以刻畫成「所有能完整塞進前景的結構元素的聯集」，既然輸出已經是這些結構元素的聯集，再斷開一次當然不會改變。圖 10-9 實測：斷開與閉合重複 2 到 5 次，與做一次相異的像素數都是 0;侵蝕重複 5 次則相差一萬四千多個像素。影響：斷開與閉合的 iterations 設 1 即可，要加重力道請放大結構元素;若你發現「多做幾次有差」，那表示你呼叫的其實不是斷開或閉合。

18. 一張二值影像同時有背景雜點與前景孔洞。「先斷開再閉合」與「先閉合再斷開」哪一個比較好?為什麼?

    提示與參考解答：型態學運算不可交換，兩者是不同的運算。圖 10-8 把兩種缺陷的數量各掃四檔、共 16 種組合實測：「先閉合再斷開」在 11 格勝出，「先斷開再閉合」只在 4 格勝出，而那 4 格裡有 3 格是「完全沒有孔洞」的情形。原因是斷開具有遞減性且不可逆——若前景布滿孔洞，先斷開會把物件咬成碎片，之後的閉合補不回來;先閉合則把物件補成實心，再斷開就能乾淨地掃掉背景雜點。原則：哪種缺陷佔優勢就讓對症的運算排在前面，只要前景有像樣的孔洞就先閉合。

19. 承第 5 題的頂帽轉換：為什麼它的結構元素必須比瑕疵大很多?做完頂帽之後，為什麼還是不能直接套 Otsu?

    提示與參考解答：用一個比瑕疵大得多的結構元素做斷開，小東西全被抹掉，剩下的就是緩慢變化的背景照明;原圖減去它，背景就被扣平。結構元素若比瑕疵小，瑕疵本身也會在斷開時存活下來，相減之後就消失了。圖 10-11 的實測回答了第二問：該圖的瑕疵只佔全圖 0.53% 的像素，遠低於圖 9-5 量到的「Otsu 需要前景至少約 3%」那條線，所以在頂帽結果上再做 Otsu 仍然失敗（IoU 0.014）。改用固定門檻 18 則得到 IoU 0.965，而且 17 到 24 之間都好用——因為頂帽已把背景壓到 0 附近，門檻變得非常好選。分工是：型態學負責拉平背景，門檻交給固定值或樣品標定。

20. 型態學梯度與第 9 章的 Sobel、Canny 有什麼不同?什麼叫內梯度與外梯度，各該在什麼時候用?

    提示與參考解答：型態學梯度 = 膨脹減侵蝕，基於形狀的集合運算而非微分。圖

10-10 實測：3×3 結構元素下型態學梯度的輪廓平均寬約 2.0 像素（向內外各長一格），內梯度（原圖減侵蝕）與外梯度（膨脹減原圖）各為 1.0，Canny 為 1.1。內梯度取得的是物件這一側的邊界，外梯度取得的是外側——這個「想要哪一側」的選擇權是微分型邊緣偵測給不了的。選用原則：已經二值化的影像用型態學梯度（一行寫完，且結構元素的形狀還能挑方向），灰階影像用微分法（定位較準）。

21. 計數之前為什麼要做孔洞填補與邊界清除?兩者的先後順序可以交換嗎?

提示與參考解答：孔洞填補把物件內部封閉的背景補成前景，否則像墊圈這種環形物件量到的面積會少掉孔的部分——圖 10-13 實測為 4,624 對 5,525 像素，差了 16%，若驗收標準是面積，這個誤差足以讓良品被判成不良。邊界清除丟掉所有碰到影像邊緣的物件，因為它們被畫面切掉了一部分，面積與形狀都不可信;該圖做完之後元件數由 6 降到 4，與正解一致。順序不能交換：必須先填補再清除。反過來做，碰邊的物件會先被清掉，它內部的孔洞反而留下來變成假物件。

22. 4-連通與 8-連通會造成多大的差別?為什麼前景與背景應該用不同的連通性?另外，cv2.connectedComponents(img, 4) 這樣寫對嗎?

提示與參考解答：差別取決於前景的形狀。圖 10-14 實測：硬幣的實心區塊兩種連通性都是 34 個，完全沒差;但同一張影像的 Canny 邊緣（細線結構）4-連通數出 1,093 個、8-連通只有 120 個，差了九倍——斜著走的邊緣在 4-連通下整條被切成碎片。前景與背景必須用不同的連通性，否則會違反 Jordan 曲線定理，出現「封閉曲線沒有把平面分成內外兩塊」的矛盾。最後一問：寫錯了。OpenCV 的簽章是 connectedComponents(image, labels, connectivity)，所以 4 被當成 labels 傳進去，連通性仍是預設的 8，而且不會報錯。必須寫 connectivity=4。

23. 連通元件標記為什麼只要掃描兩遍?它用到什麼資料結構?為什麼面積、外接矩形、質心是「免費」的?

提示與參考解答：第一遍由上到下、由左到右，只看已掃過的鄰居，分三種情況給標籤（全新／沿用／合併並記錄等價）;第二遍把暫時標籤換成等價類的代表。等價關係用聯集-尋找（Union-Find）維護，加上路徑壓縮與按秩合併後，m 次操作的成本是 O(m α(n))，α 是反阿克曼函數，實務上小於 5，可視為常數，所以整體複雜度是 O(N)。統計量之所以免費，是因為面積是計數、外接矩形是四個極值、質心是座標累加和除以面積，三者都能在第一遍邊走邊累加，合併等價類時把兩份統計相加即可，不需要再掃一次影像。

24. 積分影像為什麼能加速 Haar-like 特徵的計算?加速比與矩形大小有什麼關係?級聯結構又省下了多少運算?

    提示與參考解答：積分影像的每一格存「從左上角到該處的所有像素總和」，建表只要掃一遍;之後任意矩形的總和都只要查四個角、做三次加減，成本與矩形大小無關。圖 10-17 實測：邊長 4 時快約 2 倍，邊長 256 時快約 45 倍——矩形愈大賺愈多，因為逐格相加的成本隨面積成長而查表是常數。級聯方面，圖 10-18 實算：一張 640×480 的畫面以 24×24 起始視窗、scaleFactor 1.1 掃描，共有 271,414 個候選視窗;Viola-Jones 的完整分類器有 6,061 個特徵，若每個視窗都跑完是 16.5 億次評估。論文報告平均每視窗只評估約 10 個特徵，總量降到 271 萬次，差了約 600 倍。

25. (綜合設計題)請為一條「墊圈外觀檢測與計數」的產線設計完整的二值影像處理管線，列出每一步的技術、參數決定方式與驗收指標，並說明若換成 OpenCV 5 需要注意什麼。

    提示與參考解答：一、成像：背光讓墊圈成為高對比剪影，關閉自動曝光（第 8 章），固定工作距離以便像素與毫米的標定（第 4 章）。二、二值化：背光下直方圖乾淨雙峰，用 Otsu;若仍有照明梯度，改用頂帽拉平背景再套固定門檻（圖 10-11）。三、清理：先看缺陷以哪種為主——墊圈內孔容易因反光產生破洞，所以通常先閉合再斷開（圖 10-8）;iterations 設 1，力道用結構元素大小控制（圖 10-9）。四、計數前處理：先孔洞填補（讓面積正確）再邊界清除（丟掉被畫面切到的件），順序不可反（圖 10-13）。五、量測：connectedComponentsWithStats 取面積、外接矩形、質心，並以 connectivity=8 明寫;內孔直徑可由「填補前後的面積差」反推，或用距離轉換取最大值（需用 DIST_MASK_PRECISE，圖 10-16）。六、驗收：以已知尺寸的標準件標定像素與毫米的換算，記錄面積、圓形度、內外徑的統計分布，要求跨班別重複性優於公差的三分之一;一旦漂移先檢查光源與鏡頭，而不是調參數。七、版本：若環境是 OpenCV 5，本章的型態學與連通元件 API 全部不受影響，但若管線中有用到 Haar 級聯（例如同時做人臉或特定圖樣偵測），該 API 已被移除，需改裝 OpenCV 4.x 或改用其他偵測器（10.6.4 節）。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 9(型態學影像處理之標準參考)。
- Viola, P., & Jones, M., Rapid Object Detection using a Boosted Cascade of Simple Features, CVPR, 2001.(本章叫獸開講之史料來源)
- Viola, P., & Jones, M., Robust Real-Time Face Detection, IJCV, 57(2), 2004.
- Serra, J., Image Analysis and Mathematical Morphology, Academic Press, 1982.(數學型態學經典)
- Soille, P., Morphological Image Analysis: Principles and Applications, 2nd ed., Springer, 2003.
- OpenCV 官方教學：Morphological Transformations 與 Cascade Classifier。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 11 章 小波與正交轉換

本章學習目標

- 說明正交轉換的共同思想，理解「換一組基底表示影像」為何有利於壓縮與分析。
- 闡述傅立葉轉換「有頻率、無位置」的根本弱點，說明短時傅立葉與小波如何回應。
- 推導 Haar 小波與多解析度分析，實作二維離散小波轉換的多層分解。
- 說明離散餘弦轉換(DCT)的原理，完整拆解 JPEG 壓縮的五個步驟與方塊效應成因。

- 比較 JPEG 與 JPEG 2000(小波)的差異，解釋小波為何避免方塊效應。
- 運用小波於影像去雜訊、融合與浮水印，並認識其在深度學習時代的定位。

叫獸開講|FBI 的指紋危機：三千五百萬張卡片與小波的救援

1990 年代初，美國聯邦調查局(FBI)面臨一個幾乎要把系統壓垮的難題。它的檔案庫裡堆放著超過三千五百萬張紙本指紋卡，還有大量待分類的積壓，而每天湧入的指紋比對請求又不斷增加。唯一的出路是全面數位化——但這裡藏著一個駭人的數字：以當年的技術把整個指紋庫掃描存檔，將產生大約一千一百四十兆位元組(超過一千 TB)的影像資料。光是儲存就是天文數字，更別提用當年的數據機傳輸一張指紋卡要花上好幾分鐘。不壓縮，數位化根本不可能。

最直覺的選擇是用現成的 JPEG 壓縮。但 FBI 與國家標準暨技術研究院(NIST)的早期測試發現一個致命問題：JPEG 在高壓縮率下會產生「方塊效應」——影像被切成 8×8 的小方塊各自處理，壓得愈狠，方塊之間的接縫就愈明顯，像一面貼歪的磁磚牆。對一般照片這或許可以忍受，但對指紋是災難：指紋辨識靠的是脊線的細微特徵點(minutiae)——分岔、端點、間距，而 JPEG 的方塊效應與高頻細節流失，恰恰會破壞或偽造這些用於法律鑑識的關鍵特徵。一個被壓縮演算法「抹掉」或「無中生有」的特徵點，在法庭上可能就是天壤之別。

FBI 找上了洛斯阿拉莫斯國家實驗室。由 FBI 的 Tom Hopper 與洛斯阿拉莫斯的 Jonathan Bradley、Chris Brislawn 領軍的團隊，最終採用了一套基於小波的方案，稱為「小波純量量化」(Wavelet Scalar Quantization,WSQ)，並在 1993 年定為國家標準。與 JPEG 把影像硬切成方塊不同，小波轉換是對整張影像做多解析度分解，不存在方塊邊界，因此從根本上避免了方塊效應。WSQ 能在 20:1 的壓縮率下，達到人眼幾乎無法察覺的品質損失，同時保留鑑識級的脊線細節。三千五百萬張指紋卡的數位化危機，就這樣被一個當時還很年輕的數學工具——小波——化解了。

這個故事點出了本章的核心。第 6 章我們讚嘆傅立葉轉換的威力，但也埋下一個伏筆：它是「全域」的，只告訴你影像含有哪些頻率，卻不告訴你這些頻率出現在哪裡。JPEG 用「切成小方塊」來勉強取得局部性，代價就是方塊效應。而小波提供了一個更優雅的答案：它能同時告訴你「有哪些頻率」與「頻率在哪裡」——這就是多解析度分析。本章要講的，正是一場「如何同時掌握頻率與位置」的數學奮鬥史，以及它如何撐起你每天使用的影像壓縮。

思考題：JPEG 為了取得「局部性」而把影像切成 8×8 方塊，結果付出了方塊效應的代價。小波用不切塊的方式達到局部性。這讓你想到什麼工程上的通則?(提示：當一個系統為了某個好處而做出粗暴的妥協時，往往存在更精巧的替代方案)

11.1 正交轉換的共同思想

11.1.1 換一組基底看世界

本章與第 6 章共享一個深刻的思想：把影像從一組基底轉換到另一組基底表示。第 6 章的傅立葉轉換，是把影像表示成一堆「不同頻率的弦波(斜條紋圖)」的加權和;本章的離散餘弦轉換與小波轉換，只是換用不同的基底函數，但精神完全一致——都是在問「這張影像，用這組積木來拼，各需要多少?」

為什麼要換基底?因為在一個好的基底下，影像的資訊會變得「稀疏」——絕大部分的能量集中在少數幾個係數上，其餘係數接近於零。而稀疏正是壓縮的黃金鑰匙：只要記住那少數重要的係數、把大量接近零的係數丟棄或粗略記錄，就能用極少的資料重建出接近原貌的影像。這是所有轉換式壓縮(JPEG、JPEG 2000)的共同原理。自然影像在空間域(像素域)並不稀疏(每個像素都攜帶資訊)，但轉換到頻率域或小波域後就變得非常稀疏——這就是轉換的價值。

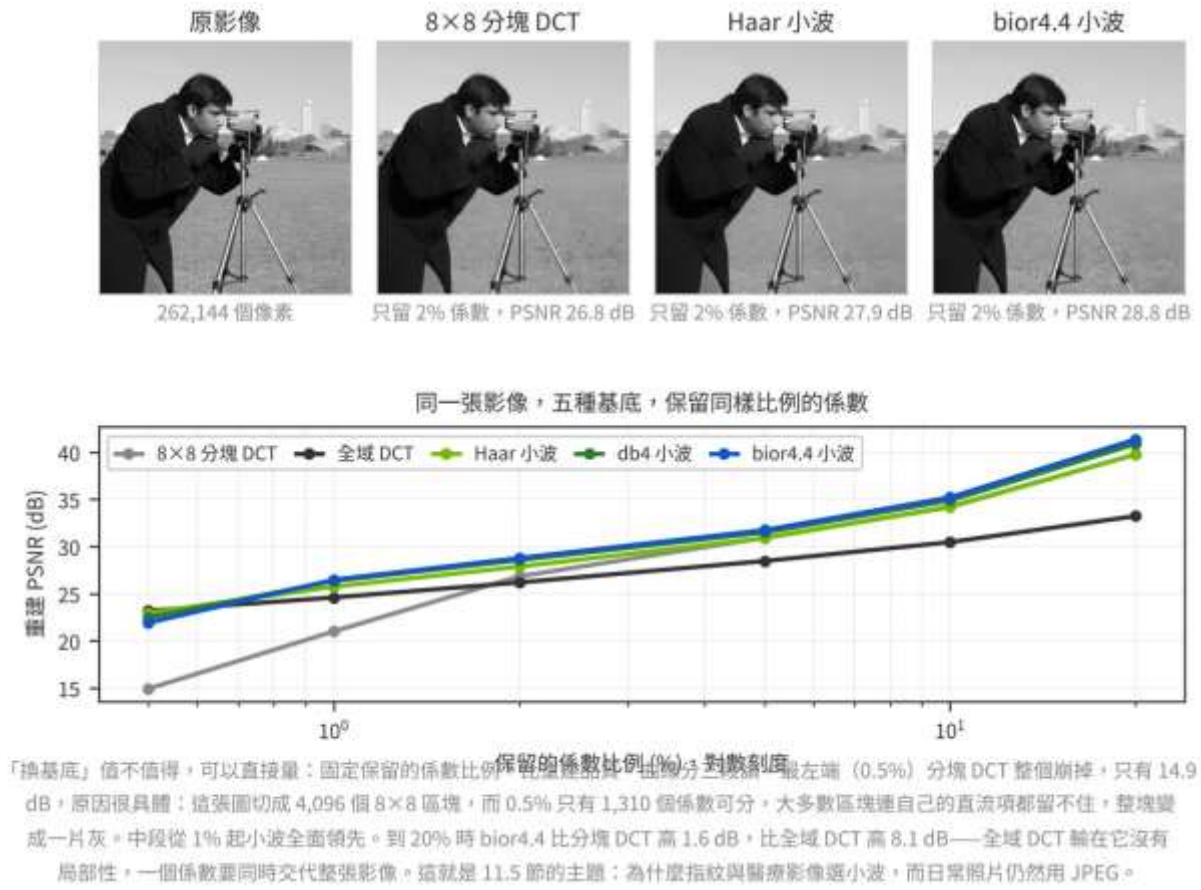
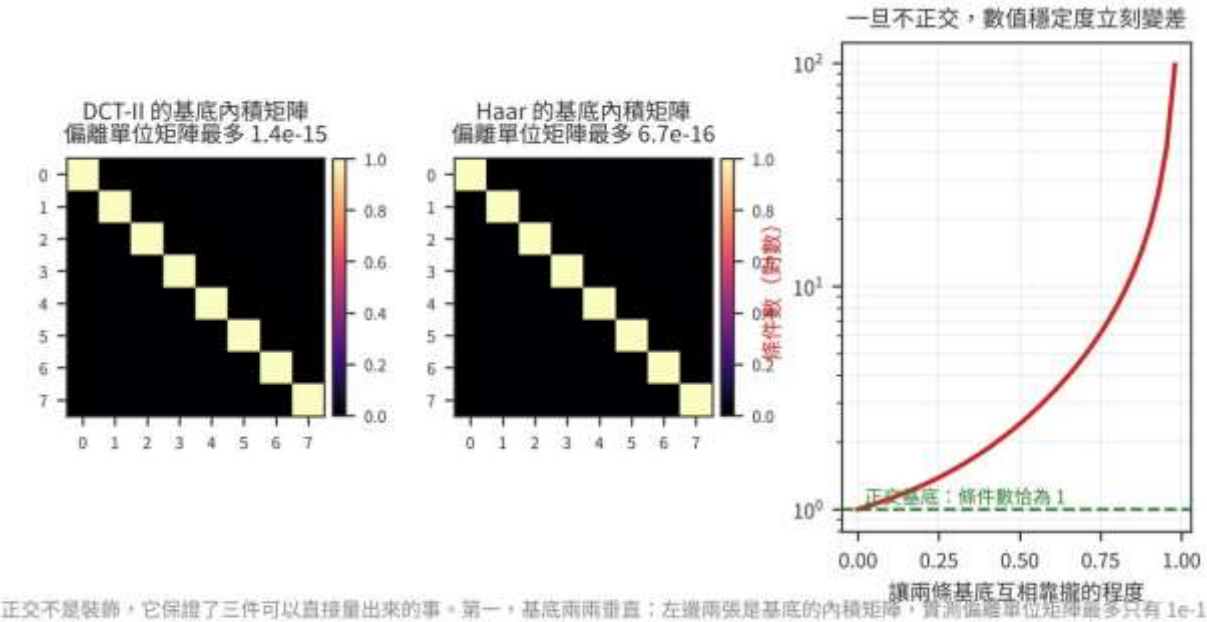


圖 11-1　三種基底看同一張影像：稀疏程度決定壓縮的本錢

11.1.2 正交的意義

這些轉換之所以稱為「正交」(Orthogonal)，是因為它們的基底函數彼此正交——用向量的語言說，任兩個基底函數的內積為零，彼此獨立、互不重疊。正交性帶來三個好處：轉換可逆(能無失真地變回原影像)、能量守恆(轉換前後總能量不變，即 Parseval 定理，這讓我們能在轉換域中判斷丟棄某係數會損失多少能量)、以及係數之間互不干擾(改動一個係數不影響其他係數的意義)。第 6 章的傅立葉、本章的 DCT 與正交小波，都具備這些性質。



正交不是裝飾，它保證了三件可以直接量出來的事。第一，基底兩兩垂直；左邊兩張是基底的內積矩陣，實測偏離單位矩陣最多只有 1e-15（DCT）與 7e-16（Haar），也就是浮點誤差的量級。第二，能量守恆（Parseval 定理）：一段 4096 點的隨機訊號，時域能量 4088.2，DCT 域 4088.2，Haar 域 4088.2，三者相同——所以「丟掉小係數」造成的誤差可以直接由被丟掉的係數平方和算出來，不必重建回去再比對。第三，數值穩定：正交基底的條件數恰為 1，而右圖讓兩條基底逐漸靠攏，條件數就一路竄到 98;基底愈接近平行，係數求解與反轉換就愈不穩定，輸入的一點點誤差會被放大同樣的倍數（這正是 7.1.5 節條件數的老問題）。

圖 11-2　正交性的三個好處：可逆、能量守恆、係數互不干擾

11.2 傅立葉的弱點與小波的動機

11.2.1 有頻率，無位置

第 6.6.5 節已經預告過傅立葉轉換的根本弱點，這裡完整展開。傅立葉轉換用「無限延伸的弦波」作為基底——每一個弦波都從負無窮延伸到正無窮，在整張影像上均勻存在。這帶來一個後果：當你在頻譜上看到某個頻率成分很強，你知道「這張影像含有這個頻率」，但你完全不知道「這個頻率來自影像的哪個位置」。

用一個聽覺的比喻：傅立葉分析一首樂曲，能告訴你整首曲子用到了哪些音符，卻無法告訴你這些音符出現的先後順序——它把樂譜壓扁成了一張「音符清單」。對於平穩(statistics 不隨位置改變)的訊號，這不成問題;但影像恰恰是高度非平穩的：左半邊可能是平滑的天空(只有低頻)，右半邊可能是茂密的樹葉(充滿高頻)。傅立葉把這兩者的頻率混在一起報告，喪失了寶貴的位置資訊。

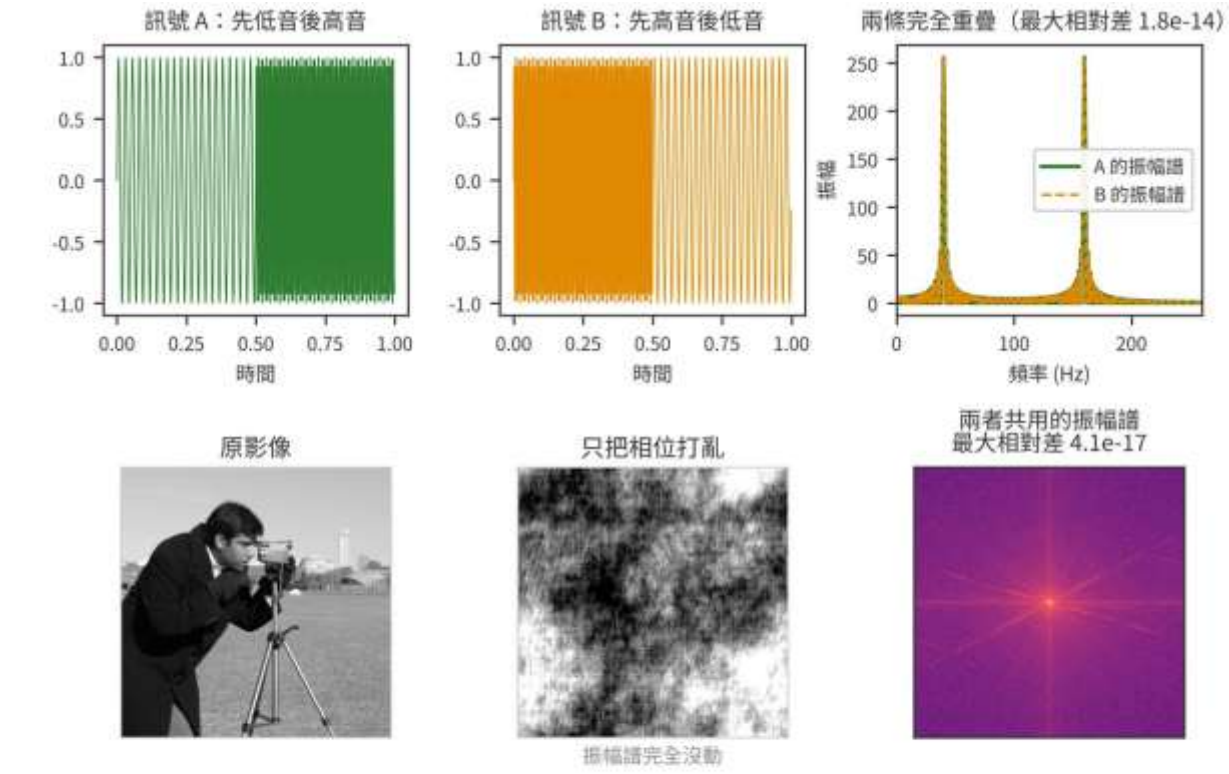
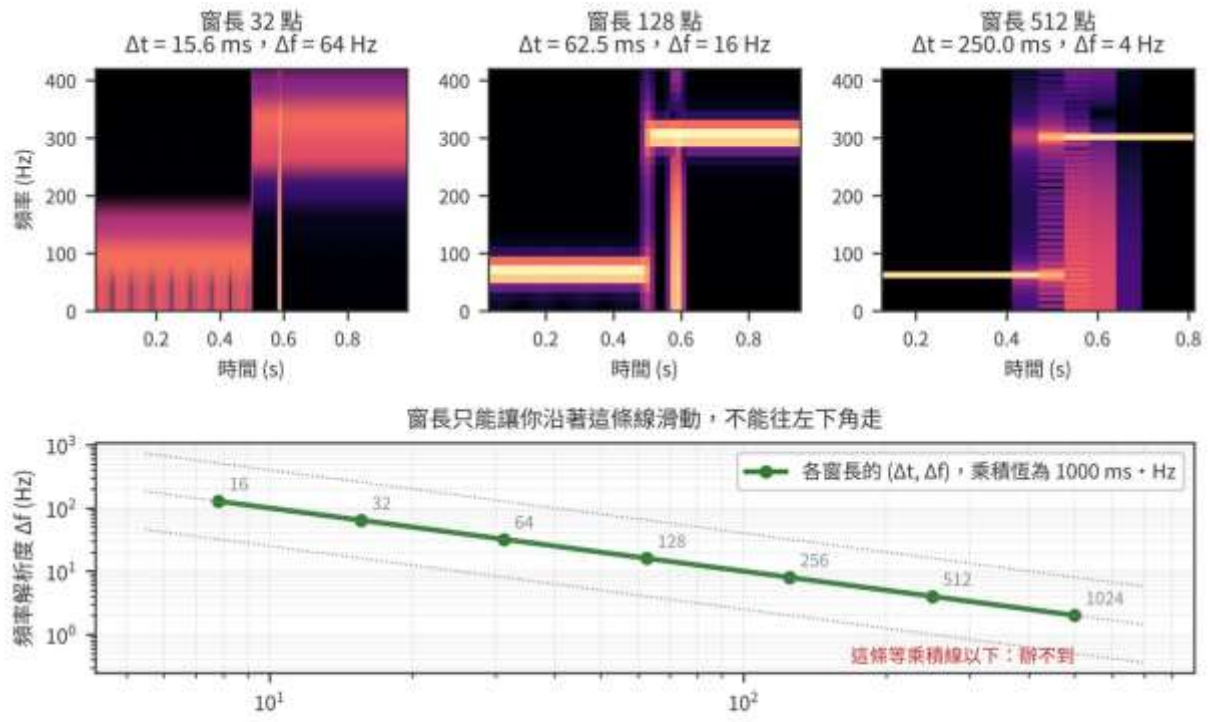


圖 11-3　有頻率、無位置：兩段訊號的頻譜完全相同

11.2.2 短時傅立葉與測不準

第一個補救是短時傅立葉轉換(STFT)：把影像切成小塊，對每一塊各自做傅立葉轉換，如此就能知道「某頻率出現在某塊」。JPEG 的 8×8 分塊 DCT 正是這個思路的體現。但 STFT 有一個先天的困境：窗口大小固定。窗口切得大，頻率分辨得準、但位置模糊;窗口切得小，位置定得準、但頻率模糊。你無法讓兩者同時精準——這就是訊號處理版的測不準原理(第 6.3.2 節提過「函數不可能在兩個域同時很窄」)。JPEG 的方塊效應，本質上就是固定窗口大小付出的代價。



短時傅立葉把訊號切成小塊各自轉換，於是「某頻率出現在某塊」就得到答案；但窗口一旦固定，解析度就被鎖死——而且兩者是此消彼長：窗長 32 點時時間看得清（Δt = 15.6 ms），頻率卻糊成 64 Hz 一格;窗長 512 點時剛好相反（Δt = 250 ms、Δf = 4 Hz）。上排三張圖請對照著看：第一張找得到那個短暫的敲擊但分不清兩個音高，第三張分得清音高卻把敲擊抹掉了。下圖說明這不是調參的問題：Δt 與 Δf 的乘積固定為 1000 ms · Hz，窗長只能讓你沿著那條線滑動。這就是測不準原理在訊號處理中的版本。小波的突破不在打破這個下界，而在「不同頻率用不同的窗口」。

圖 11-4　測不準：窗口愈短，位置愈準而頻率愈糊

11.2.3 小波的洞見：讓窗口大小自己調整

小波的天才之處，在於它放棄了「固定窗口」。它的觀察是：低頻成分變化緩慢，需要長窗口才能看清(犧牲位置換頻率精度);高頻成分變化劇烈且短暫，需要短窗口才能定位(犧牲頻率換位置精度)。小波用一個「有限長度的小波形」(mother wavelet)作為基底，透過縮放(scaling，拉長或壓縮波形以對應不同頻率)與平移(translation，移動波形以對應不同位置)，自動實現「低頻用長窗、高頻用短窗」的自適應分析。

這正是多解析度分析(Multi-Resolution Analysis)的精髓：在粗的尺度上看大輪廓(低頻、低空間解析度)，在細的尺度上看小細節(高頻、高空間解析度)，不同尺度各司其職。它同時提供了頻率與位置資訊，徹底回應了傅立葉的弱點——這也是小波能避免方塊效應的根本原因：它從一開始就不需要把影像切成方塊。

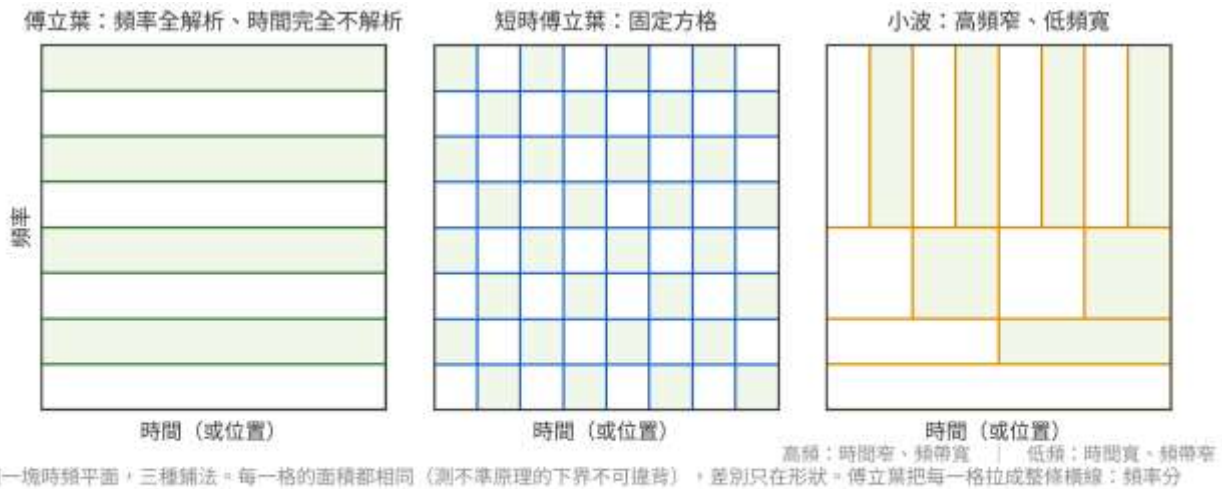


圖 11-5　時頻平面的三種鋪法：傅立葉、短時傅立葉、小波

11.3 Haar 小波與多解析度分析

11.3.1 最簡單的小波：Haar

Haar 小波是最早、也最簡單的小波，早在 1909 年就由 Alfréd Haar 提出(比小波理論的正式成熟早了數十年)。它的一維運算簡單到可以手算：對相鄰的一對數值，計算它們的平均(得到「概略」資訊，對應低頻)與差值的一半(得到「細節」資訊，對應高頻)。例如數列 [8, 4]：平均是 6、差的一半是 2，於是 [8, 4] 被表示成 [6, 2]——其中 6 是概略、2 是細節。要還原也容易：8 = 6 + 2、4 = 6 - 2。這個「平均與差分」的配對，就是最基本的小波分解。

對整個數列反覆施行：先兩兩配對得到一半長度的概略序列與一半長度的細節序列，再對概略序列繼續兩兩配對……如此層層遞進，就是多解析度分解。每往上一層，概略的解析度減半(看得更粗略)，而每一層的細節被分別保留。這解釋了小波「多解析度」之名的由來，也說明了它與傅立葉的根本不同：傅立葉一次把訊號攤平成頻譜，小波則是一層一層地剝出不同尺度的細節。

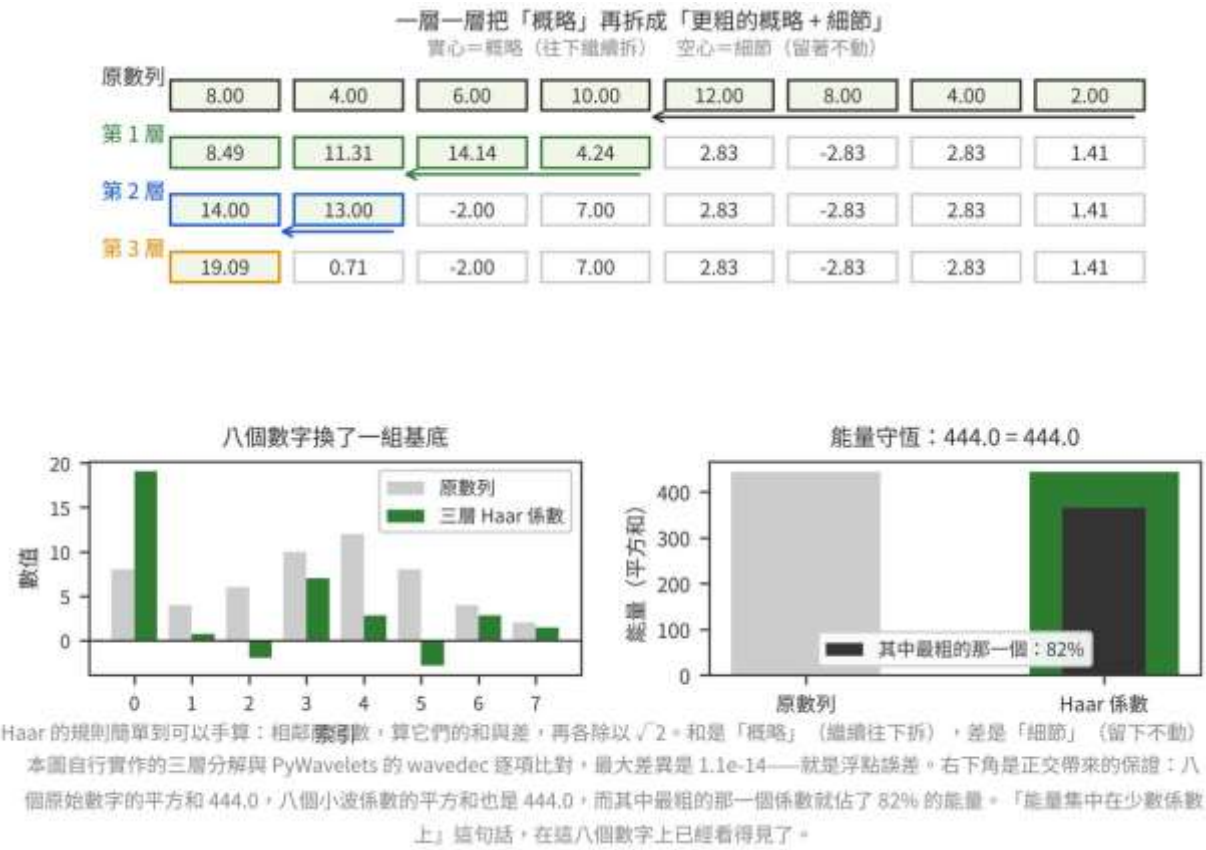


圖 11-6　Haar 小波的一維手算：平均與差分，層層遞進

11.3.2 二維小波轉換與四個子帶

把一維小波推廣到二維影像的標準做法是：先對每一列做一維小波轉換(水平方向)，再對每一行做一維小波轉換(垂直方向)。一層轉換後，影像被分解成四個子帶(subband)，各佔原影像四分之一大小：

1. LL(低-低)子帶：水平垂直都取概略，是原影像的縮小版(縮略圖)，集中了絕大部分能量。
2. LH(低-高)子帶：水平概略、垂直細節，凸顯水平方向的邊緣。
3. HL(高-低)子帶：水平細節、垂直概略，凸顯垂直方向的邊緣。
4. HH(高-高)子帶：兩方向都取細節，凸顯對角線方向的細節與雜訊。

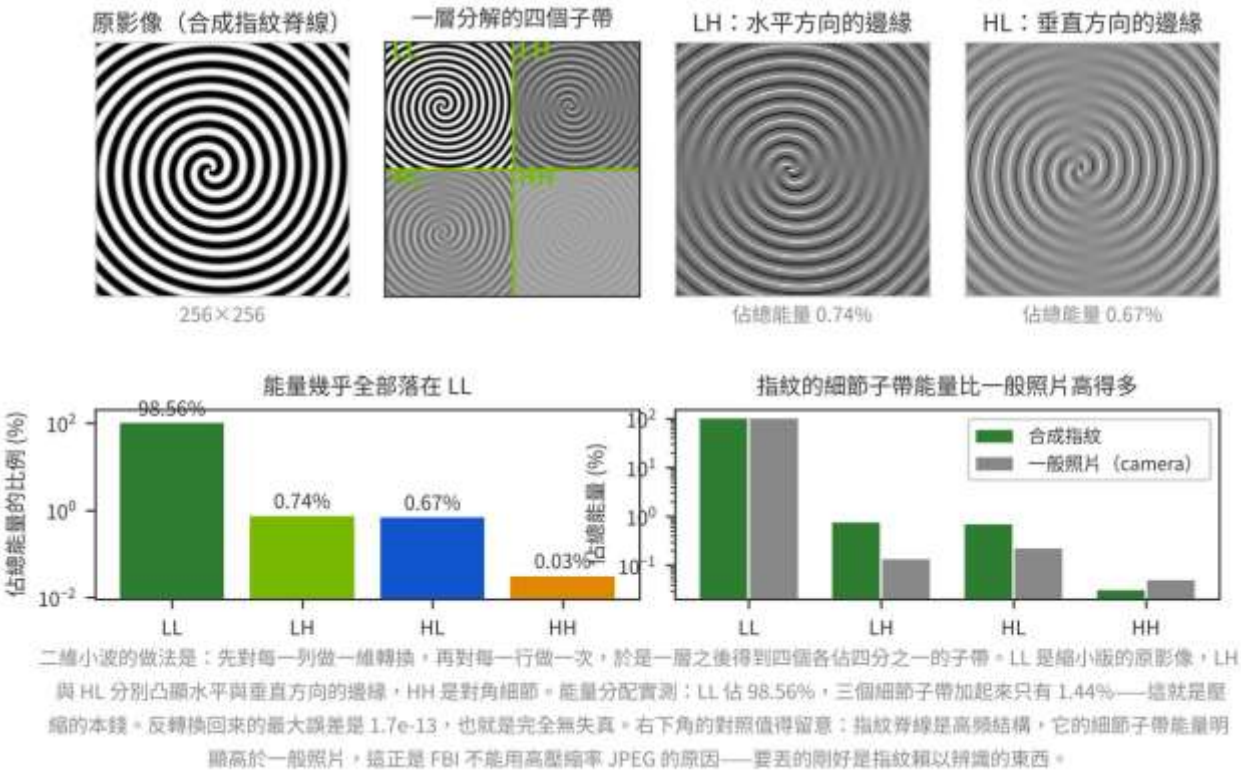


圖 11-7　二維一層分解的四個子帶

多層分解時，只對 LL 子帶(縮略圖)繼續往下分解，形成一個由粗到細的金字塔結構。這個結構的美妙之處在於：大部分能量集中在最頂層的小小 LL 子帶，而其他子帶的係數大多接近於零(自然影像的細節是稀疏的)——這正是 11.1.1 節說的「稀疏性」，壓縮的機會就藏在這裡。

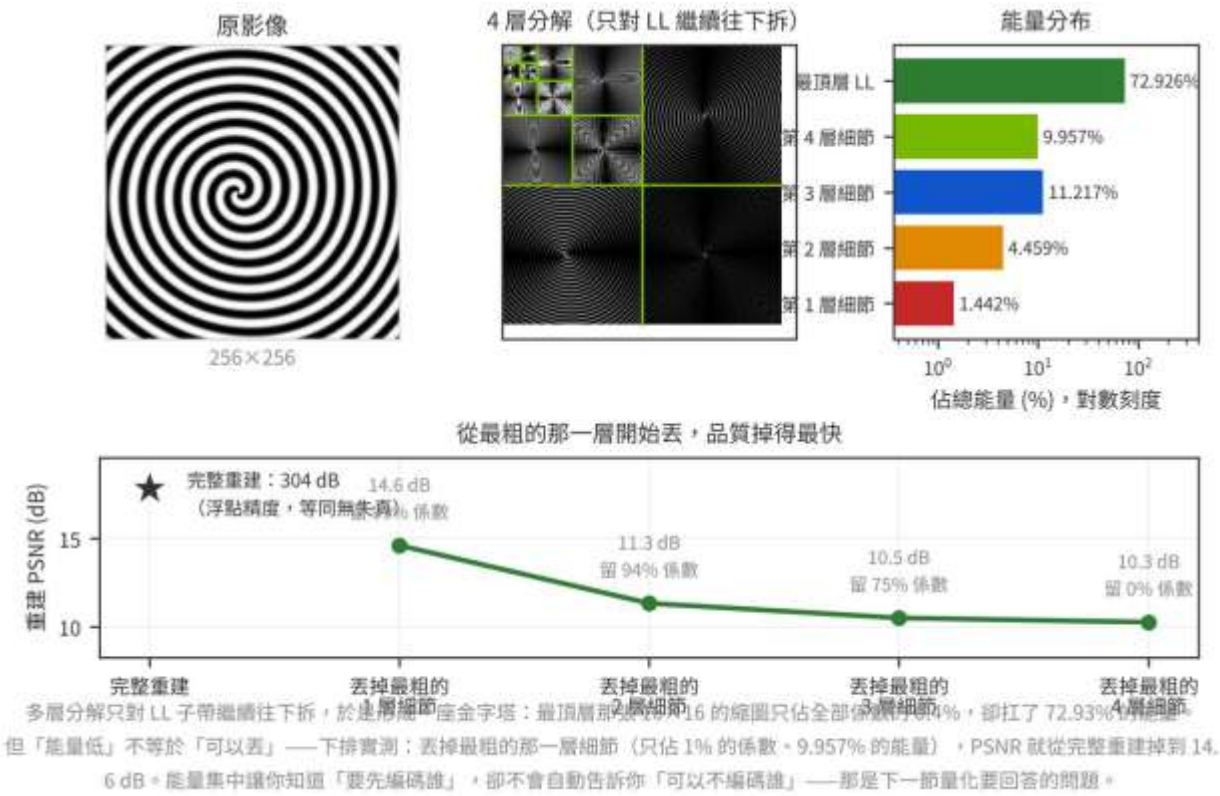


圖 11-8　多層分解的金字塔：只對 LL 續分，係數總數不變

程式 11-1 安裝 PyWavelets

```
# 安裝小波處理套件
pip install PyWavelets
```

程式 11-2 二維小波分解與重建

```
import cv2, numpy as np, pywt

gray = cv2.imread("fingerprint.jpg", cv2.IMREAD_GRAYSCALE).astype(float)

# --- 單層二維 Haar 小波轉換 ---
coeffs = pywt.dwt2(gray, "haar")
LL, (LH, HL, HH) = coeffs
print("四個子帶尺寸:", LL.shape)     # 各為原圖的四分之一

# 把四個子帶拼成一張圖觀察
def norm(x):
    return cv2.normalize(x, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
top = np.hstack([norm(LL), norm(LH)])
bot = np.hstack([norm(HL), norm(HH)])
cv2.imwrite("wavelet_subbands.png", np.vstack([top, bot]))

# --- 多層分解(三層) ---
coeffs_multi = pywt.wavedec2(gray, "haar", level=3)
arr, slices = pywt.coeffs_to_array(coeffs_multi)
cv2.imwrite("wavelet_pyramid.png", norm(arr))

# --- 反轉換:確認可完整還原 ---
```

```
recon = pywt.idwt2(coeffs, "haar")
print("還原最大誤差:", np.abs(gray - recon).max())    # 接近 0
```

除了 Haar，實務上常用的小波還有 Daubechies(db 系列，由 Ingrid Daubechies 於 1988 年提出，是小波理論成熟的里程碑)、Biorthogonal(雙正交，JPEG 2000 採用)等。不同的小波在「平滑度」與「支撐長度」上各有取捨，選擇適合訊號特性的小波，是小波應用的一門學問——FBI 的 WSQ 正是精心挑選了適合指紋脊線特性的小波。

11.3.3 推導：Haar 的 1/√2 為什麼不能省

很多教科書把 Haar 分解寫成「平均與差的一半」——概略 $a = (x_1+x_2)/2$、細節 $d = (x_1-x_2)/2$，本章 11.3.1 節也是這樣介紹的，因為它最好手算。但真正的 Haar 小波轉換用的是另一組係數：$a = (x_1+x_2)/\sqrt{2}$、$d = (x_1-x_2)/\sqrt{2}$。差一個 $\sqrt{2}$ 看似小事，其實決定了這個轉換算不算「正交」。

先驗算能量。取 $[x_1, x_2] = [8, 4]$。用 1/2 的版本：$a = 6$、$d = 2$，轉換後的能量是 $6^2 + 2^2 = 40$，而原始能量是 $8^2 + 4^2 = 80$——能量少了一半。用 $1/\sqrt{2}$ 的版本：$a = 12/\sqrt{2} \approx 8.485$、$d = 4/\sqrt{2} \approx 2.828$，能量是 $72 + 8 = 80$，與原始完全相同。

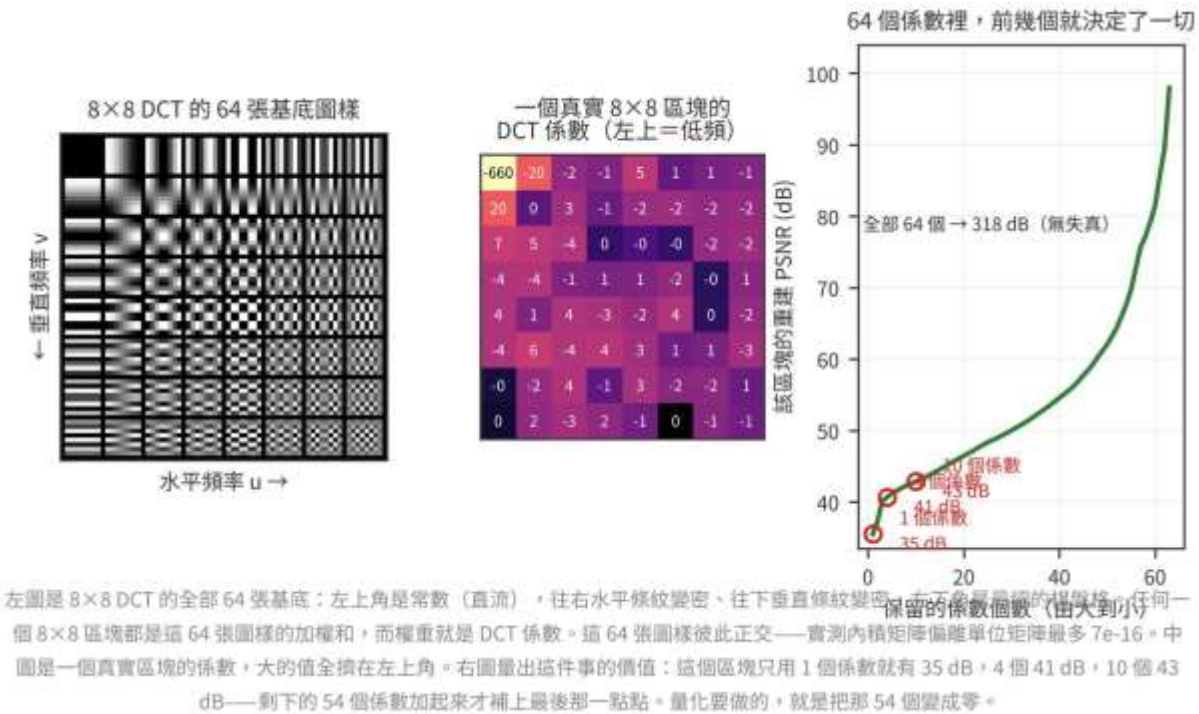
為什麼會這樣?把一層 Haar 寫成矩陣乘法 $y = H\,x$，其中 $H = c\cdot[[1, 1], [1, -1]]$，$c$ 是待定的常數。正交矩陣的定義是 $H^{\mathsf{T}}H = I$。計算 $H^{\mathsf{T}}H = c^2\cdot[[2, 0], [0, 2]] = 2c^2\cdot I$，要讓它等於單位矩陣，必須 $2c^2 = 1$，也就是 $c = 1/\sqrt{2}$。換句話說，$1/\sqrt{2}$ 不是美觀的選擇，而是「正交」這個條件唯一允許的值。

這件事為什麼重要?回到 11.1.2 節列的三個好處。正交保證了 Parseval 定理成立：轉換前後總能量相等。而壓縮的整套邏輯都建立在這條定理上——我們之所以敢說「丟掉小係數只損失一點點」，是因為係數的平方和就是影像的能量，丟掉一個大小為 ε 的係數，造成的重建誤差能量恰好也是 ε^2，不多也不少。若轉換不正交，係數的大小與它對重建的貢獻就脫鉤了：一個看起來很小的係數可能在反轉換時被放大成一片明顯的失真，「挑大的留下」這個策略也就失去了依據。

實務上 PyWavelets 的 pywt.dwt2(gray, "haar") 用的正是 1/√2 的正規化版本。所以如果你照著 11.3.1 節手算出 [6, 2]，再拿去和程式輸出比對，會發現程式給的是 [8.485, 2.828]——兩者差一個 √2 倍，都不是錯的，只是前者方便心算、後者保有正交性。要判斷手上的實作屬於哪一種，最快的方法就是算一次能量：若轉換前後的平方和相等，它就是正交版本。

11.4 離散餘弦轉換與 JPEG 壓縮

在小波稱霸之前(以及在絕大多數日常照片中至今)，影像壓縮的主力是離散餘弦轉換(DCT)與建立其上的 JPEG。理解 JPEG，是理解影像壓縮不可跳過的一課，它也把第 3、6、8 章的許多概念串在一起。



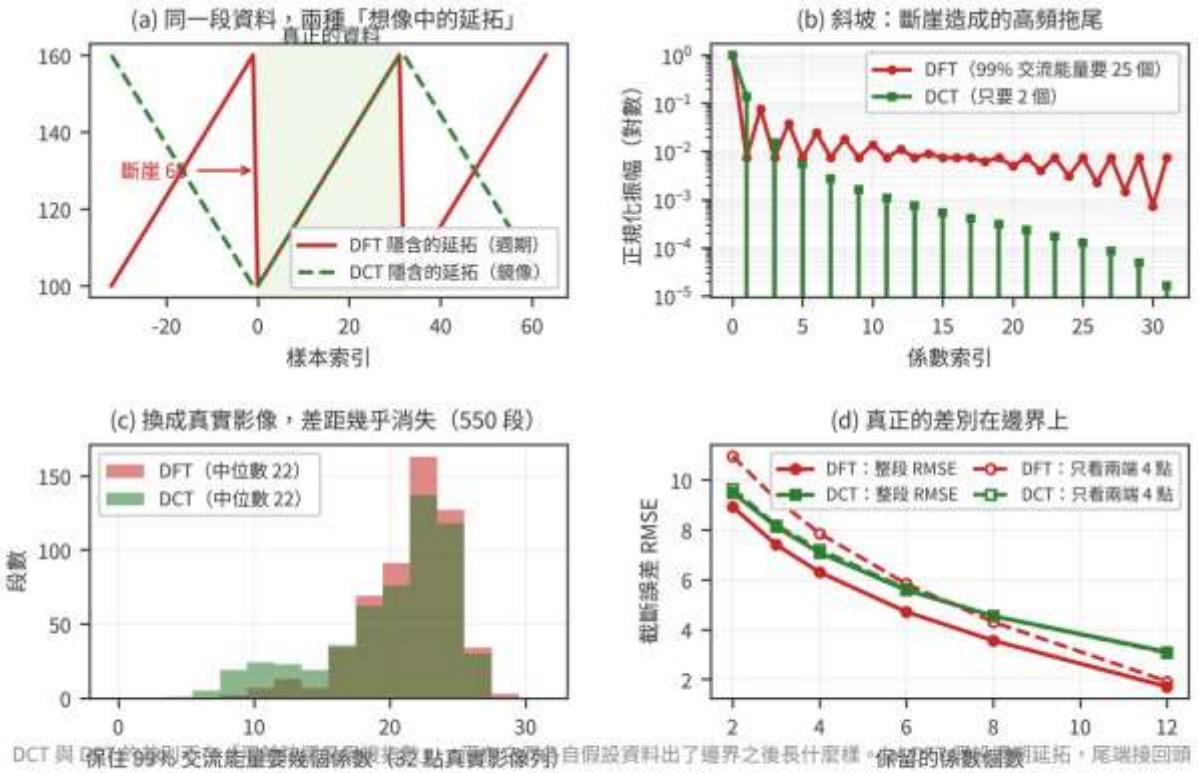
左圖是 8×8 DCT 的全部 64 張基底：左上角是常數（直流），往右水平條紋變密、往下垂直條紋變密，左上角是最低頻的模樣。任何一個 8×8 區塊都是這 64 張圖樣的加權和，而權重就是 DCT 係數。這 64 張圖樣彼此正交——實測內積矩陣偏離單位矩陣最多 7e-16。中圖是一個真實區塊的係數，大的值全擠在左上角。右圖量出這件事的價值：這個區塊只用 1 個係數就有 35 dB，4 個 41 dB，10 個 43 dB——剩下的 54 個係數加起來才補上最後那一點點。量化要做的，就是把那 54 個變成零。

圖 11-9　DCT 的 64 個基底影像

11.4.1 離散餘弦轉換

DCT 是傅立葉家族的近親，差別在於它只使用餘弦函數作為基底。這帶來一個實用的好處：對實數影像做 DCT 得到的係數也是實數(不像傅立葉會產生複數)，處理起來更簡便。更重要的是，DCT 有優異的能量集中特性：對自然影像的區塊做 DCT，能

量會高度集中在少數幾個低頻係數(左上角)上，而高頻係數(右下角)大多接近零——這種稀疏性正是壓縮的基礎(呼應 11.1.1 節)。



DCT 與 DFT 的比較：(a) 同一段資料，兩種「想像中的延拓」——DFT 假設資料出了邊界之後長什麼樣，保留的誤差都來自延拓，尾端接回頭端，中間出現一道高度 60 的斷崖;DCT 假設鏡像延拓，接縫完全連續（斷崖 0）。(b) 斷崖就是高頻：同一段斜坡 DFT 要 25 個係數才保得住 99% 的交流能量，DCT 只要 2 個。但 (c) 換成真實影像的 32 點列，兩者的中位數都是 22——課本常說的「DCT 比較會集中能量」，在自然影像上其實只是微幅領先。真正的差別在 (d)：只保留 2 個係數時，DFT 的整段誤差反而略低，但兩端 4 點的誤差是 11.0，DCT 只有 9.7。那道斷崖的振鈴全擠在邊界上，而分塊編碼恰恰是把一堆邊界拼在一起——這才是 JPEG 選 DCT 的實質理由（外加一個很現實的好處：DCT 的輸出是實數，不必做複數運算，也不必存虛部）。

圖 11-10　DCT 比 DFT 更能集中能量?真正的差別在邊界

11.4.2 JPEG 壓縮的五個步驟

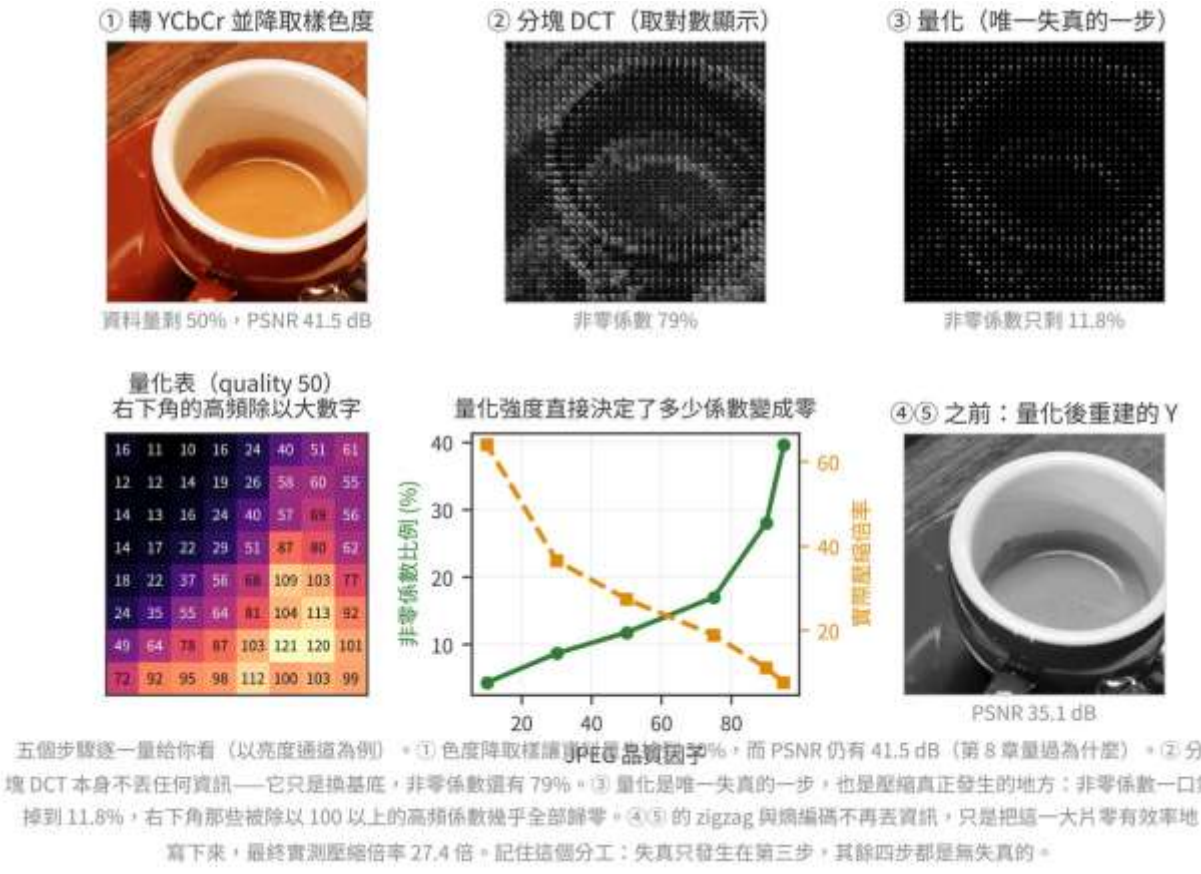
JPEG 是一條精心設計的管線，每個步驟都呼應本書前面的概念：

5. 色彩空間轉換與降取樣：先把 RGB 轉為 YCbCr(第 8 章)，分離亮度 Y 與色度 Cb、Cr;因為人眼對色度不敏感，對 Cb、Cr 做 4:2:0 降取樣(解析度砍四分之一)，這一步就先省下一半資料且幾乎無感——這是第 8 章「人眼對色度遲鈍」的直接應用。
6. 分塊與 DCT：把每個通道切成 8×8 的小區塊，對每一塊各自做 DCT。切塊正是 11.2.2 節短時傅立葉的思路——用分塊取得局部性，但也埋下方塊效應的種子。

7. 量化(Quantization)：這是 JPEG 唯一「失真」的步驟，也是壓縮的核心。用一張「量化表」把每個 DCT 係數除以對應的量化步長再取整數——高頻係數用較大的步長(除以大數字)，許多因此變成零。品質參數(quality)控制的正是量化表的縮放：品質愈低、量化步長愈大、被歸零的高頻愈多、檔案愈小、失真愈大。這一步利用了人眼對高頻細節較不敏感的特性，大膽丟棄「看不太出來」的資訊。

8. Zigzag 掃描與遊程編碼：量化後，8×8 區塊的右下角(高頻)佈滿了零。以 zigzag(之字形)順序掃描，把二維區塊排成一維序列，能讓這些零連續排在一起，再用遊程編碼(把「連續 20 個零」記成一個符號)大幅壓縮。

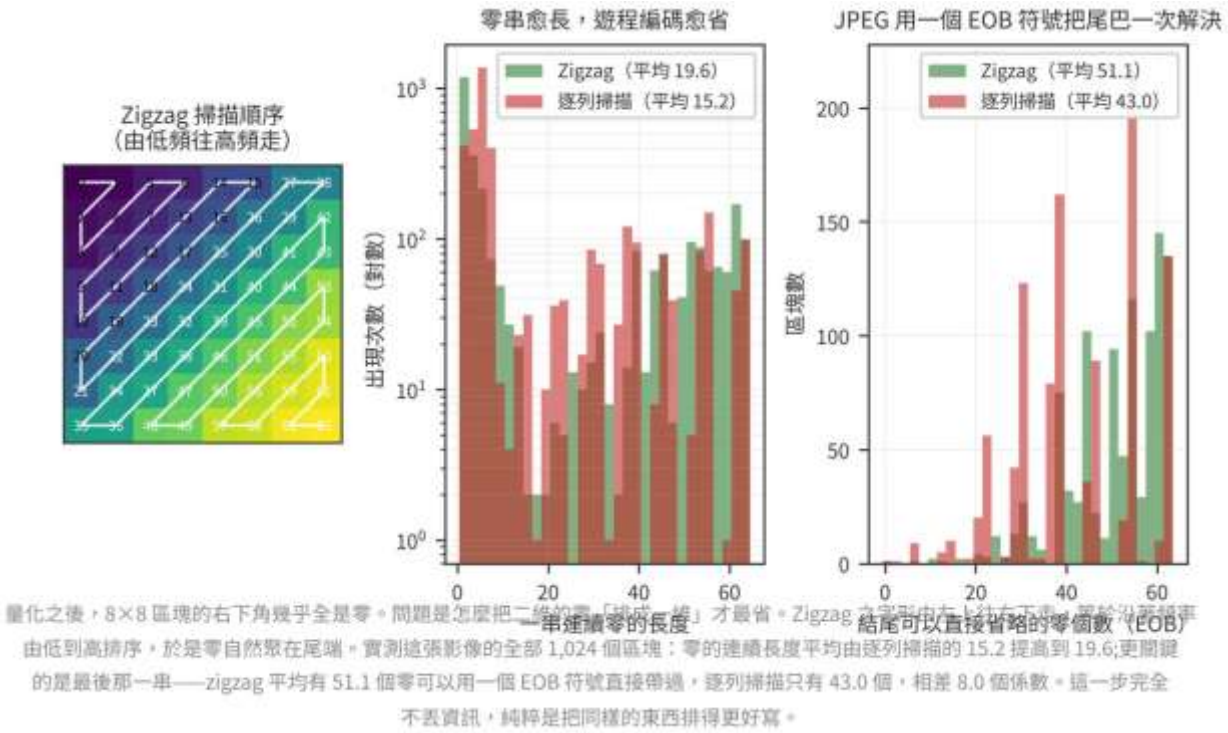
9. 熵編碼(Entropy Coding)：最後用 Huffman 編碼(或算術編碼)對結果做無失真壓縮，把出現頻繁的符號用較短的碼表示。這一步不丟資訊，純粹榨乾統計冗餘。



五個步驟逐一量給你看（以亮度通道為例）。① 色度降取樣讓資料量剩 50%，而 PSNR 仍有 41.5 dB（第 8 章量過為什麼）。② 分塊 DCT 本身不丟任何資訊——它只是換基底，非零係數還有 79%。③ 量化是唯一失真的一步，也是壓縮真正發生的地方：非零係數一口氣掉到 11.8%，右下角那些被除以 100 以上的高頻係數幾乎全部歸零。④⑤ 的 zigzag 與熵編碼不再丟資訊，只是把這一大片零有效率地寫下來，最終實測壓縮倍率 27.4 倍。記住這個分工：失真只發生在第三步，其餘四步都是無失真的。

圖 11-11　JPEG 五個步驟的實際數據流

解壓縮則是完全反向的過程。整條管線中，只有第三步「量化」是不可逆的失真來源——這也是「反覆存檔 JPEG 會愈存愈糊」這個說法的由來。不過這句話需要補上條件：若每次都用同一個品質設定、而且影像沒有被裁切或移動，再量化其實近乎不動（因為係數已經落在量化格點上，這個運算是冪等的）;真正會累積失真的，是每次改變品質設定、或在存檔之間裁切縮放而打亂了 8×8 的對齊。11.4.3 節會完整推導，圖 11-14 則用二十個世代的實測驗證這件事。



量化之後，8×8 區塊的右下角幾乎全是零。問題是怎麼把二維的零「攤成一維」才最省。Zigzag 自左上角開始，沿對角線一格一格往右下走，再折回來，由低到高排序，於是零自然聚在尾端。實測這張影像的全部 1,024 個區塊：零的連續長度平均由逐列掃描的 15.2 提高到 19.6;更關鍵的是最後那一串——zigzag 平均有 51.1 個零可以用一個 EOB 符號直接帶過，逐列掃描只有 43.0 個，相差 8.0 個係數。這一步完全不丟資訊，純粹是把同樣的東西排得更好寫。

圖 11-12　Zigzag 掃描：把二維的零聚成一維的連續零

程式 11-3 DCT 量化與 JPEG 品質實驗

```
import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)

# --- 手動示範 JPEG 的核心:8x8 區塊 DCT 與量化 ---
# JPEG 標準亮度量化表(quality≈50)
Q = np.array([
    [16,11,10,16,24,40,51,61], [12,12,14,19,26,58,60,55],
    [14,13,16,24,40,57,69,56], [14,17,22,29,51,87,80,62],
    [18,22,37,56,68,109,103,77], [24,35,55,64,81,104,113,92],
    [49,64,78,87,103,121,120,101],[72,92,95,98,112,100,103,99]],
    dtype=np.float32)

block = gray[:8, :8] - 128            # 置中
dct_block = cv2.dct(block)            # 8x8 DCT
```

```
quantized = np.round(dct_block / Q)   # 量化(失真在此發生)
print("非零係數數量:", np.count_nonzero(quantized), "/ 64")

# 反量化 + 反 DCT 還原
recovered = cv2.idct(quantized * Q) + 128

# --- 實用:直接用不同品質存 JPEG,觀察方塊效應 ---
img = cv2.imread("photo.jpg")
for q in [90, 50, 10]:
    cv2.imwrite(f"jpeg_q{q}.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, q])
    back = cv2.imread(f"jpeg_q{q}.jpg")
    print(f"品質 {q}: PSNR = {cv2.PSNR(img, back):.2f} dB")
    # 放大 q10 的結果,可清楚看到 8x8 方塊邊界
```

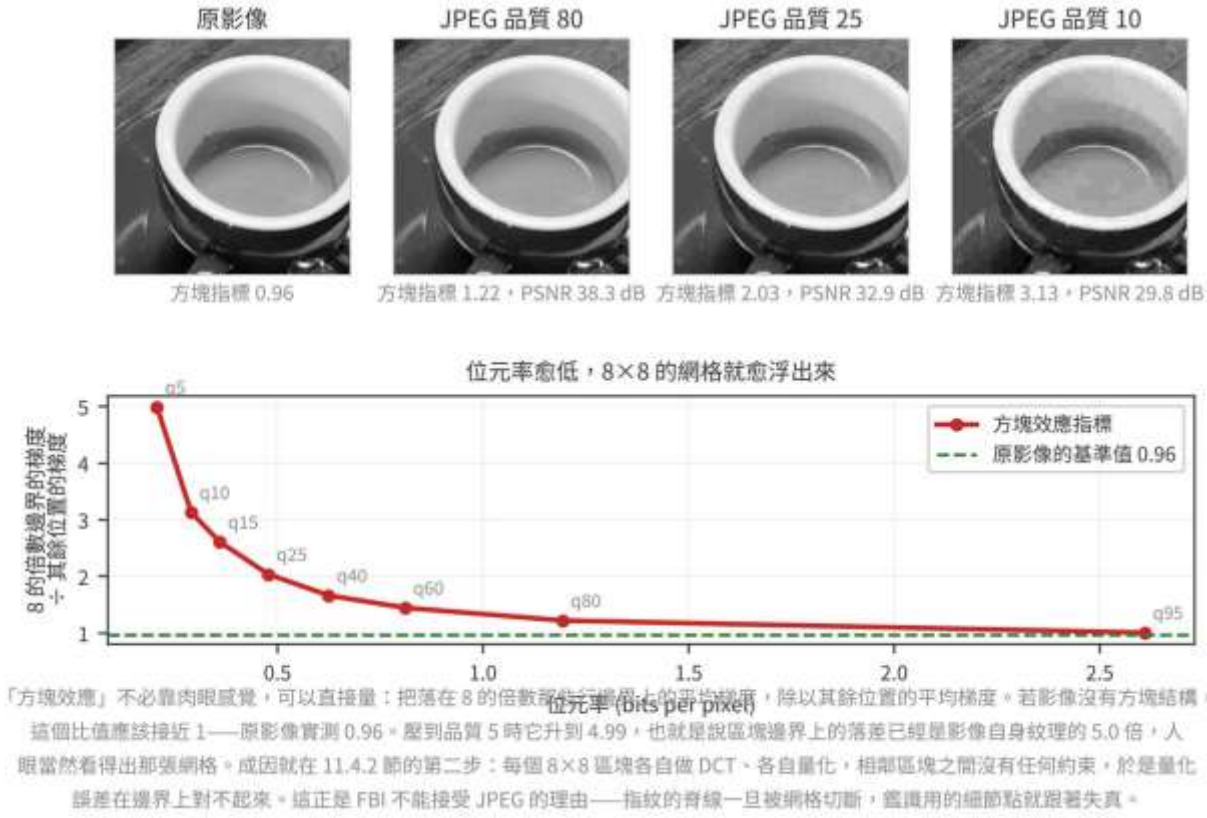


圖 11-13　方塊效應：量化步長變大，格線就浮現

11.4.3 推導：再量化為什麼近乎冪等——「愈存愈糊」的條件

「JPEG 每存一次就糊一次」是流傳極廣的說法，本章 11.4.2 節也提過。但這句話需要補上條件，否則它是錯的。先把量化這個動作寫清楚：編碼時算 $q = \text{round}(F / Q)$，解碼時算 $F' = q \times Q$。合起來看，一次「存檔再打開」做的事是 $F \to \text{round}(F/Q) \times Q$，也就是把係數 F 吸附到最近的 Q 的整數倍上。

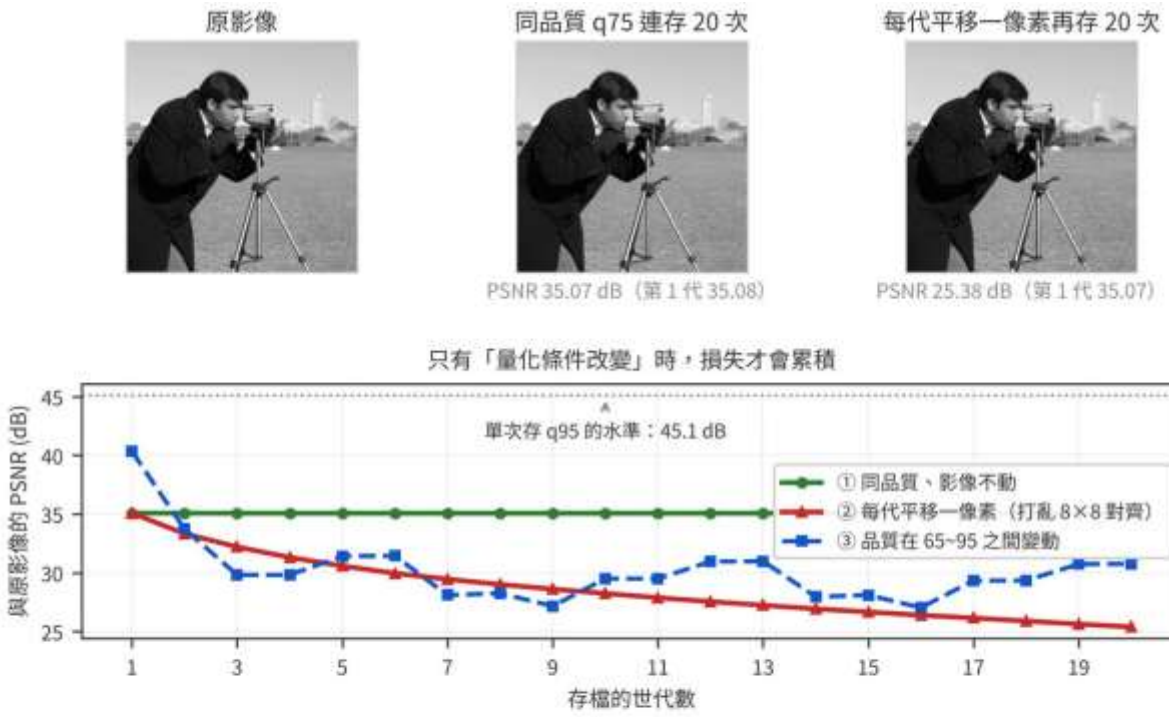
關鍵問題：如果把 F' 再存一次會怎樣?算一下 $\text{round}(F'/Q) \times Q$。因為 F' 已經是 Q

的整數倍，F'/Q 本來就是整數，round 不會改變它，再乘回 Q 自然還是 F'。用數學的話說，這個運算滿足 T(T(F)) = T(F)，它是冪等的（idempotent）——與第 10 章開運算、閉運算的冪等性是同一個概念。所以在 DCT 係數這一層，第二次以後的存檔完全不再損失任何資訊。

這個推論可以用實測檢驗。以同一個品質 q75 連續存 20 次，PSNR 從第 1 代的 35.08 dB 到第 20 代的 35.07 dB，幾乎原地不動。之所以不是分毫不差，是因為解碼還要做反 DCT、取整數、再截斷到 0~255 這三道手續，這些才是微小擾動的來源——第 2 代與第 1 代之間只有約 0.5% 的像素有差異，之後就收斂了。

那麼什麼時候才真的會「愈存愈糊」?答案是：當量化的條件改變時。第一種情況是每次改變品質設定，等於每次換一張不同的量化表 Q，把係數搬到另一個格點上，冪等性立刻失效。圖 11-14 的實測中，品質在 65~95 之間輪流變動存 20 代後只剩 30.8 dB，而且那一代其實是用最高的 q95 存的——單次用 q95 可以達到 45.1 dB。這說明已經丟掉的資訊，事後調高品質救不回來。

第二種情況更常見也更隱蔽：在存檔之間裁切、縮放或旋轉影像。這些操作會讓 8×8 的切割線落在不同的像素位置上，同一塊內容每次被切進不同的區塊、配到不同的鄰居，自然每次都要重新量化一次。圖 11-14 中「每代平移一個像素」的那條線在 20 代之內掉了 9.7 dB，正是這個效應。對實務的建議因此很具體：編修影像時用無失真格式（PNG、TIFF）當工作檔，只在最後輸出時存一次 JPEG;若必須反覆存 JPEG，至少要固定品質設定，而且不要在中途裁切或縮放。



「JPEG 反覆存檔會愈存愈糊」這句話需要補上條件。綠線是情境①：同一個品質連存 20 次，PSNR 從 35.08 dB 到 35.07 dB，幾乎原地不動。原因很乾淨：第一次量化之後，係數已經落在量化格點上，再除以同一張量化表、再取整數就是它自己——近乎冪等（之所以不是完全相等，是因為反轉換後還要取整數並截斷到 0~255，第 2 代只有 1,237 個像素、也就是 0.5% 與第 1 代不同，之後就收斂了）。真正會累積損失的是另外兩條線：②每代平移一像素，8×8 的切割線每次落在不同位置，20 代掉了 9.7 dB；③每代換一個品質，係數每次被搬到不同的格點上。③ 的第 20 代其實是用 q95 存的，單次用 q95 可以達到 45.1 dB，它卻只有 30.8 dB——這說明已經丟掉的資訊，事後把品質調高是救不回來的。結論：反覆存檔本身不可怕，可怕的是中間裁切、縮放或改變品質設定。

圖 11-14　反覆存檔真的會愈存愈糊嗎：三種情境的實測

11.5 小波壓縮與 JPEG 2000

理解了 JPEG 的方塊之痛，小波壓縮的優勢就不言自明。JPEG 2000 是小波壓縮的標準化成果，它用二維離散小波轉換取代了 JPEG 的分塊 DCT。關鍵差異在於：小波轉換作用於整張影像(或大的圖塊)，不把影像硬切成獨立的 8×8 方塊，因此從根本上不存在方塊邊界——高壓縮率下，JPEG 2000 的影像是整體變得柔和模糊，而非碎裂成磁磚，視覺上遠優於同壓縮率的 JPEG。

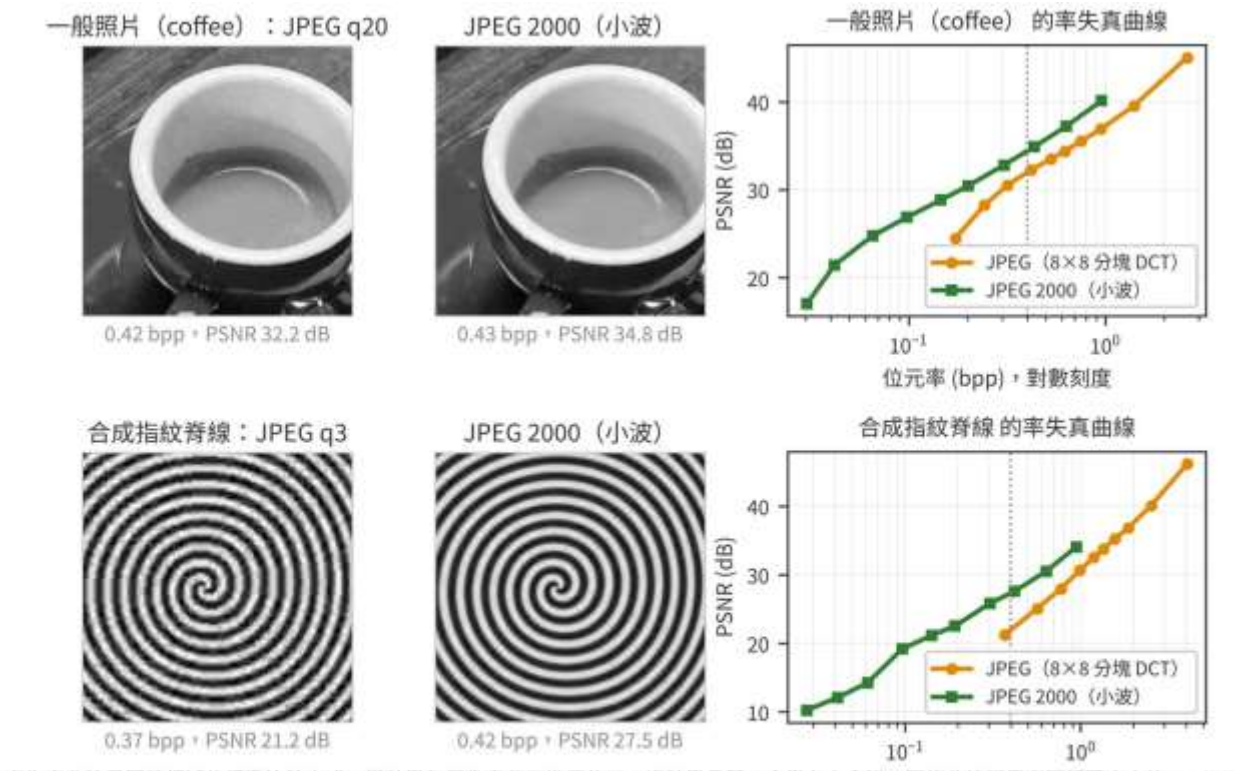
小波壓縮的一般流程與 WSQ、JPEG 2000 一致：對影像做多層小波分解 → 對各子帶的係數做量化(利用能量集中於 LL 子帶、其餘子帶稀疏的特性)→ 熵編碼。除了避免方塊效應，小波壓縮還有一個附加優點：天然支援漸進式傳輸與多解析度存取——先傳最頂層的 LL(縮略圖)就能顯示模糊的全貌，隨著更多子帶傳入逐步變清晰，非常

適合網路傳輸與大圖瀏覽。

比較面向	JPEG(DCT)	JPEG 2000 / WSQ(小波)
轉換單位	8×8 獨立方塊	整張影像多解析度分解
高壓縮率瑕疵	方塊效應、蚊狀雜訊	整體柔化模糊，無方塊
漸進式傳輸	有限(progressive 模式)	天然支援，多解析度
運算複雜度	較低，普及極廣	較高
典型應用	一般照片、網頁、相機	指紋(WSQ)、醫療、電影數位母帶

表 11-1 JPEG 與小波壓縮的比較

那為什麼今天絕大多數照片仍是 JPEG?因為 JPEG「夠好、夠快、夠普及」——它的運算簡單、硬體支援完善、相容性無敵。JPEG 2000 雖然技術更優，卻因複雜度較高與相容性問題，主要用在對品質要求極高的專業領域(醫療影像、電影數位母帶、指紋鑑識)。這是工程世界的常態：最好的技術不一定勝出，「足夠好加上生態系」往往才是贏家。這個觀察值得每一位工程師記在心裡。



率失真曲線是壓縮領域的標準比較方式：橫軸是每個像素花了幾個位元，縱軸是品質，愈靠左上愈好。兩條曲線都內插到同一個位元率 0.40 bpp 再比：一般照片上 JPEG 2000 比 JPEG 高 +2.4 dB（31.9 → 34.3），指紋脊線上高 +5.4 dB（21.8 → 27.3）。差距在高頻密集的指紋上明顯更大，因為分塊 DCT 的量化誤差會沿著 8×8 的格線堆積（圖 11-13），而脊線正好被那些格線切斷。這就是 WSQ 與 JPEG 2000 選小波的量化依據，也說明了為什麼日常照片用 JPEG 就夠——那個差距在一般照片上小得多，而 JPEG 的編碼速度與相容性遠勝一籌。

圖 11-15　率失真曲線：JPEG 與 JPEG 2000 的實測對決

11.6 小波的其他應用

11.6.1 小波去雜訊

小波提供了一種優雅的去雜訊思路，與第 7 章的方法互補。核心觀察是：自然影像的重要結構(邊緣、輪廓)在小波分解後，會集中成少數幾個「大係數」;而雜訊則均勻散布成大量「小係數」。因此去雜訊的策略非常直觀——小波門檻化(Wavelet Thresholding)：把小波係數中絕對值小於某門檻的通通歸零(視為雜訊)，保留大係數(視為訊號)，再反轉換。

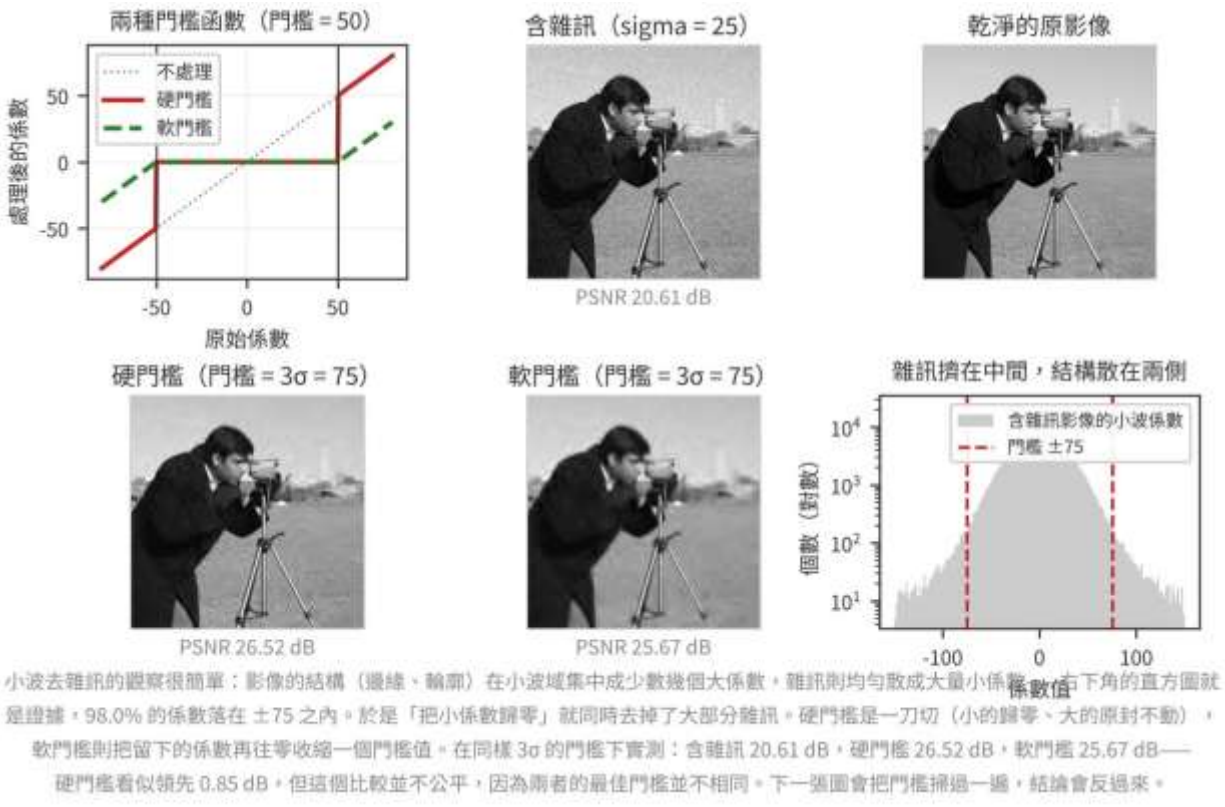
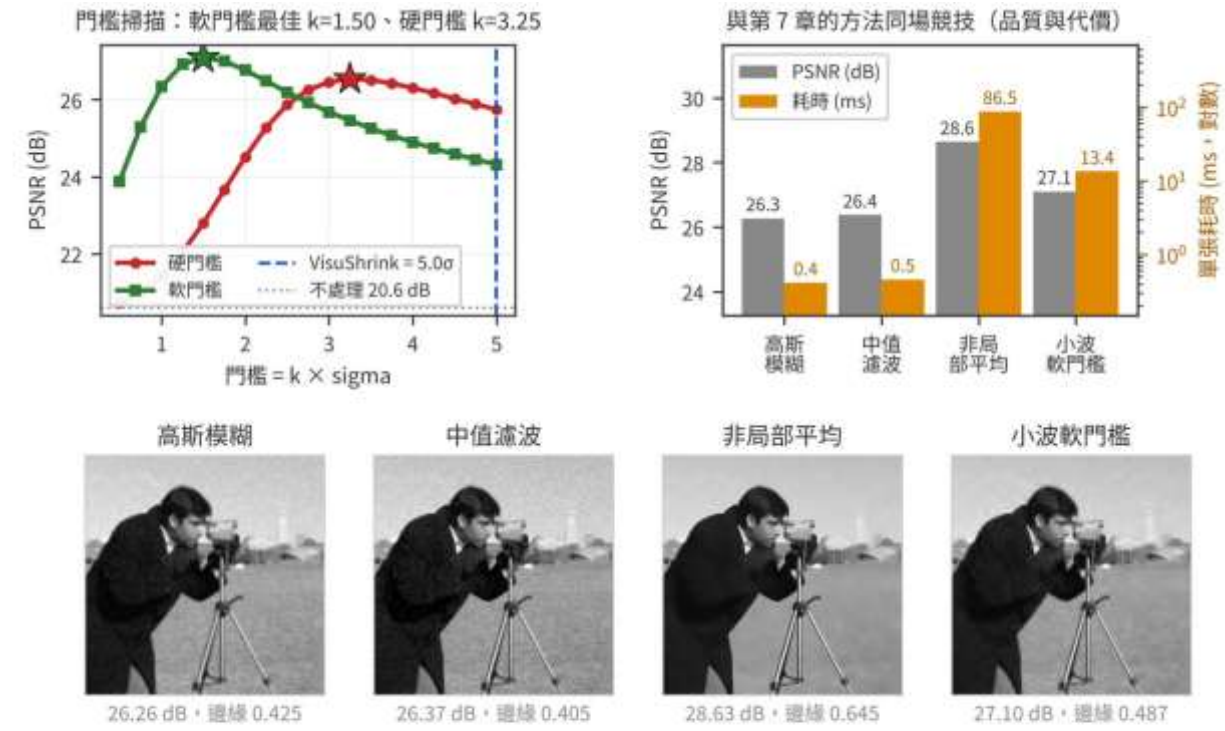


圖 11-16　硬門檻與軟門檻：兩種收縮函數與去雜訊結果

這個方法之所以強大，是因為它在「同時掌握頻率與位置」的小波域中操作：它能區分「邊緣附近的高頻(該保留)」與「平坦區的高頻(是雜訊，該去除)」——這是第 7 章純空間域或第 6 章純頻率域方法難以做到的。它與第 7 章的非局部平均、維納濾波各擅勝場，共同構成去雜訊的工具箱。

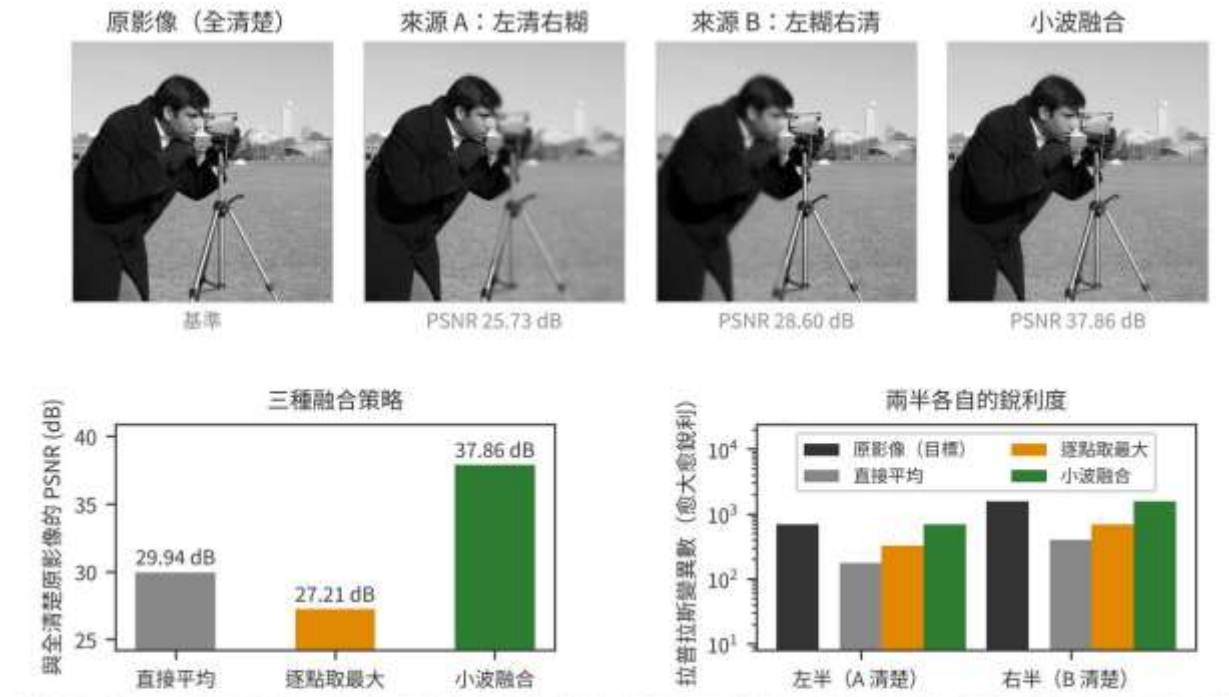


門檻值不能隨便挑。左上掃描 k（門檻 = k×sigma）：軟門檻在 k=1.50 達到最佳 27.10 dB，硬門檻在 k=3.25 達到 26.54 dB——各自調到最好之後，軟門檻反而勝出，這修正了上一張圖的印象。教科書常引用的 VisuShrink 門檻在這裡相當於 5.0σ，只有 24.32 dB，比最佳值低 2.78 dB——它的設計目標是「幾乎不留任何雜訊」，代價是也抹掉細節。右上與下排是誠實的對照：PSNR 最高的不是小波，是非局部平均的 28.63 dB，邊緣保留 0.645 也是它最高。但它要 86.5 ms，是小波的 6.5 倍，而且只會做去雜訊這一件事。小波的價值不在單項奪冠，而在同一套分解可以接著做壓縮、融合與浮水印（接下來三張圖）——這是「換一個基底」帶來的通用性。

圖 11-17　門檻值怎麼挑，以及與第 7 章方法的同場競技

11.6.2 影像融合

影像融合(Image Fusion)把多張影像的資訊合併成一張。典型應用：把一張對焦近物、一張對焦遠物的照片融合成「全景深」清晰影像;把紅外線影像與可見光影像融合以兼得夜視與細節;醫學上把 CT(骨骼)與 MRI(軟組織)融合。小波是影像融合的常用工具：先把各影像做小波分解，再依規則合併各子帶的係數(例如細節子帶取絕對值較大者，代表該處較清晰)，最後反轉換得到融合影像。

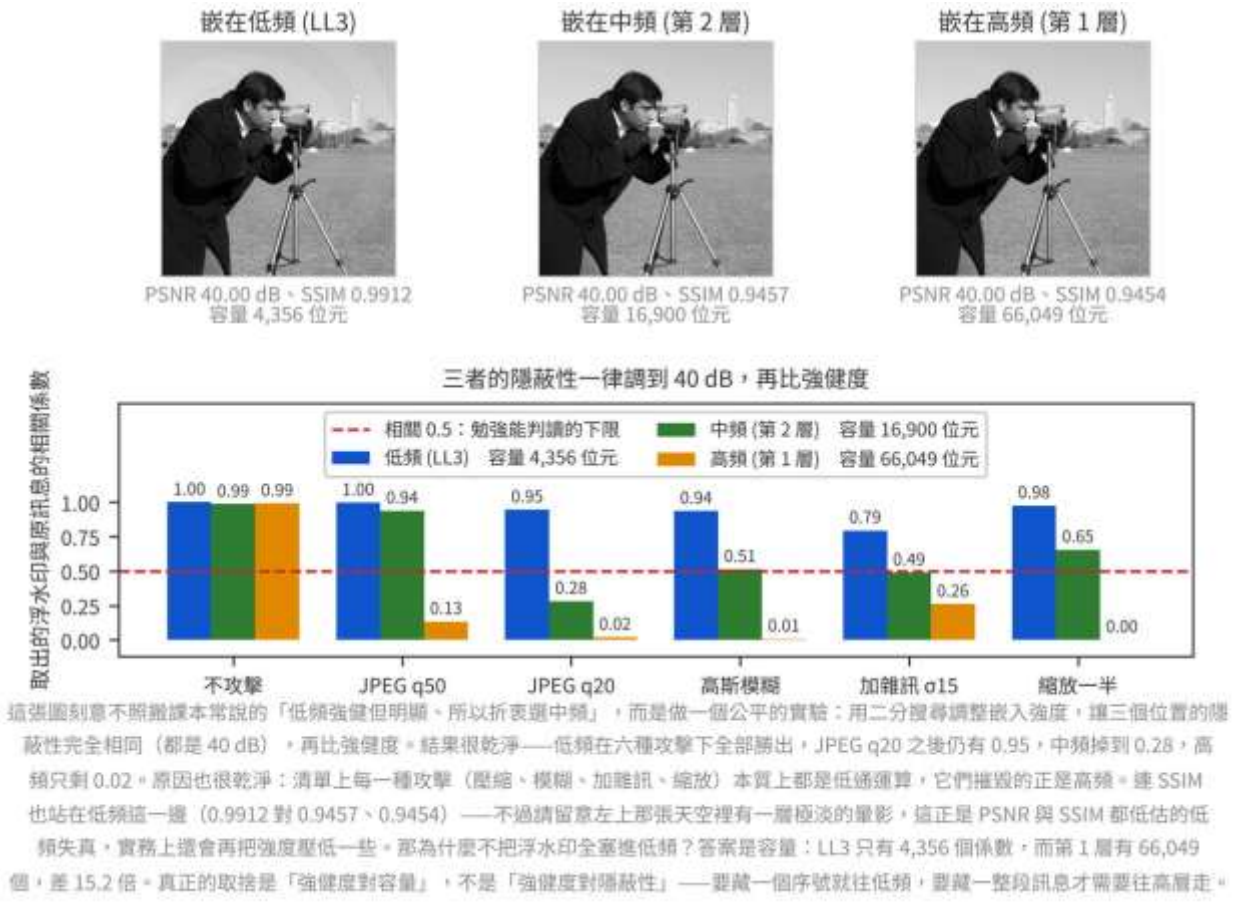


多焦點融合的任務：兩張照片各有一半清楚，要合成一張全部清楚的。本圖的兩張來源由同一張清晰影像人工製造（左右各模糊一半），所以有「正確答案」可以比對。直接平均只有 29.94 dB——兩邊都半糊，這是最糟的做法，卻也最常被想到。逐點取最大 27.21 dB，更差，因為它會把亮點誤判成細節。小波融合 37.86 dB：它在每個子帶的每個位置，比較兩張影像誰的細節係數較強就選誰，等於在「頻率 × 位置」的網格上逐格做選擇——這正是小波同時握有兩種資訊才做得到的事。右下角是驗證：融合後左半的銳利度 691（原影像 691）、右半 1579（原影像 1576），兩半都回到了原影像的水準。

圖 11-18　多焦點影像融合：在頻率與位置的網格上逐格選擇

11.6.3 數位浮水印

數位浮水印(Watermarking)把識別資訊(版權標記、序號)隱藏進影像。把浮水印嵌入小波域的中頻係數，是常見的強健做法：它既不像嵌入低頻那樣容易被肉眼察覺，也不像嵌入高頻那樣容易被壓縮或濾波破壞，能在影像被 JPEG 壓縮、縮放後依然存活。這在版權保護與影像鑑識中很重要。不過這個「中頻最好」的通則值得用實驗檢驗一次：若把三個位置的嵌入強度調到隱蔽性完全相同再比較，低頻其實在各種攻擊下全面勝出，因為壓縮、模糊、縮放本質上都是低通運算。低頻真正的限制是容量——它的係數個數比最細的子帶少了十幾倍。換句話說，真正的取捨是「強健度對容量」，圖 11-19 有完整的對照。



這張圖刻意不照嚴謹本常說的「低頻強健但明顯、所以折衷選中頻」，而是做一個公平的實驗：用二分搜尋調整嵌入強度，讓三個位置的隱蔽性完全相同（都是 40 dB），再比強健度。結果很乾淨——低頻在六種攻擊下全部勝出，JPEG q20 之後仍有 0.95，中頻掉到 0.28，高頻只剩 0.02。原因也很乾淨：清單上每一種攻擊（壓縮、模糊、加雜訊、縮放）本質上都是低通運算，它們摧毀的正是高頻。連 SSIM 也站在低頻這一邊（0.9912 對 0.9457、0.9454）——不過請留意左上那張天空裡有一層極淡的暈影，這正是 PSNR 與 SSIM 都低估的低頻失真，實務上還會再把強度壓低一些。那為什麼不把浮水印全塞進低頻？答案是容量：LL3 只有 4,356 個係數，而第 1 層有 66,049 個，差 15.2 倍。真正的取捨是「強健度對容量」，不是「強健度對隱蔽性」——要藏一個序號就往低頻，要藏一整段訊息才需要往高層走。

圖 11-19　浮水印嵌在哪一層：等隱蔽性下的強健度與容量

11.6.4 推導：軟門檻為什麼要再收縮一個門檻值

11.6.1 節說小波去雜訊是「把小於門檻的係數歸零」，這是硬門檻。軟門檻多做一件看起來很奇怪的事：留下來的大係數也要再往零減去一個門檻值 T，寫成公式是 soft(w) = sign(w)·max(|w| – T, 0)。明明判定它是訊號了，為什麼還要削它一刀?

理由要從「被觀測到的係數是什麼」說起。影像加了標準差 σ 的高斯雜訊之後，因為小波轉換是正交的（這正是 11.3.3 節推導的 1/√2 在保證的事），雜訊在小波域仍然是標準差 σ 的高斯雜訊，而且均勻散布在每一個係數上。所以觀測到的係數是 w = s + n，其中 s 是真正的訊號、n 是雜訊。我們想要的是 s，手上只有 w。

現在看硬門檻做了什麼。當 |w| > T 時它原封不動地保留 w，也就是把 s + n 當成 s 用——雜訊 n 整個被留了下來。門檻只擋掉了「純雜訊造成的假訊號」，卻沒有處理「真訊號上附帶的雜訊」。軟門檻的 –T 正是針對這一項：既然每個留下的係數都

夾帶著雜訊，就統一把它往零拉回一段，平均而言可以抵消掉一部分雜訊的貢獻。

代價是引入偏差（bias）。對一個真實強度很大的係數——比如一條強邊緣，w = 300、T = 75——軟門檻會把它變成 225，硬是削掉了四分之一，邊緣因此變淡。所以軟門檻是拿「偏差」去換「變異數」：係數整體變得平滑穩定，但系統性地偏小。硬門檻反過來，不帶偏差卻保留了全部雜訊，而且在門檻附近有跳躍，反轉換後會在影像上留下細小的偽影。

既然兩者的取捨不同，它們的最佳門檻自然也不同，比較時必須各自調到最好才公平。圖 11-16 用同樣的 3σ 門檻比較，硬門檻（26.52 dB）看似勝過軟門檻（25.67 dB）;但圖 11-17 把門檻掃過一遍之後，軟門檻在 1.50σ 達到 27.10 dB，反而勝過硬門檻在 3.25σ 的 26.54 dB。軟門檻的最佳門檻明顯較低，原因就在上一段：它本來就會額外削掉一截，門檻再設高就削過頭了。這也提醒一件更一般的事——比較兩個方法時，若只在同一組參數下較量，得到的往往是參數的高下，不是方法的高下。

11.7 小波在深度學習時代的定位

進入深度學習時代，壓縮與特徵擷取的許多工作逐漸被神經網路接手，小波是否過時了?答案與第 9、10 章一致：它轉變了角色，但並未退場。

三個觀察值得學生留意。其一，小波仍是專業壓縮的中流砥柱：指紋(WSQ)、醫療影像、電影數位母帶仍大量使用小波，因為這些領域對「無方塊、可鑑識」的要求無可妥協。其二，小波與深度學習正在融合：近年出現不少「小波卷積網路」的研究，把小波的多解析度分解作為神經網路的一層，結合兩者的長處——用小波提供的多尺度結構，幫助網路更有效率地學習。其三，多解析度的思想無所不在：第 13 章 SIFT 的尺度空間、卷積網路中的多層降取樣(每一層看到的都是更粗尺度的特徵)，骨子裡都與小波「由粗到細、多尺度分析」的精神相通。

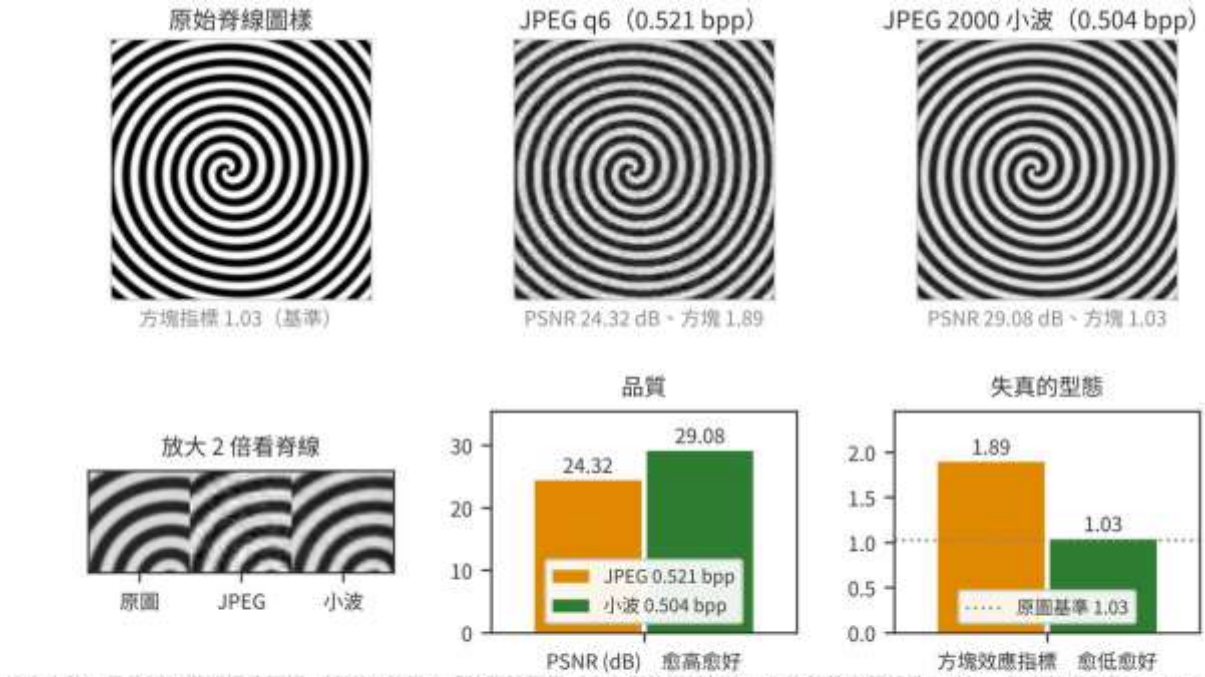
換言之，小波作為一個「具體演算法」或許在日常應用中不如深度學習搶眼，但

它所承載的「多解析度分析」思想，已經深深融入現代電腦視覺的血液。理解小波，就是理解這個貫穿全書(第 6 章頻率、第 9 章尺度、第 13 章 SIFT、第 15 章 CNN)的多尺度思維。

11.8 應用案例

11.8.1 案例一：指紋壓縮與 WSQ

呼應叫獸開講，這是小波壓縮的旗艦案例。情境：執法機構需要儲存與傳輸海量的 500 ppi 指紋影像，且不能損失用於鑑識的脊線細節。WSQ 的流程：對整張指紋影像做多層離散小波分解(精選適合指紋脊線頻率特性的小波)→ 對各子帶做純量量化(依子帶的重要性分配不同的量化精度，脊線所在的頻帶量化得細)→ Huffman 熵編碼。在 20:1 壓縮率下，人眼幾乎無法察覺品質損失，關鍵的特徵點得以保留。這個案例完美展示了小波「無方塊、保細節」相對 JPEG 的決定性優勢——在鑑識這種細節攸關人命的場合，方塊效應是不可接受的。



把本章的工具放回叫獸開講的現場。這裡用的是合成的脊線圖樣（真實指紋屬於個資，但脊線的空間特性一致）。兩者都壓到約 0.5 bpp、也就是約 16 倍：JPEG 用了 0.521 bpp 換到 24.32 dB，小波用更少的 0.504 bpp 換到 29.08 dB——位元數較少卻贏了 4.76 dB。更要命的是失真的「型態」：JPEG 的方塊指標 1.89 遠高於原圖基準 1.03，而小波的 1.03 幾乎沒變，左下角放大圖也看得見 JPEG 把脊線切成一段一段。對鑑識而言，脊線的分岔點與端點就是比對依據，一旦被 8×8 網格切斷就可能生出假的細節點——這是精度問題，更是證據可信度的問題。FBI 選小波（WSQ）不是因為它時髦，而是因為 JPEG 的失真型態正好落在最不能失真的地方。

圖 11-20　WSQ 的理由：同一位元率下，失真的型態才是關鍵

11.8.2 案例二：多焦點影像融合

情境：用顯微鏡或微距鏡頭拍攝時，景深很淺，一張照片只能拍清楚某個深度的物體。要得到「全部清晰」的影像，可拍攝一系列對焦在不同深度的照片，再融合。小波做法：對每張照片做小波分解 → 在每個位置，細節子帶取「絕對值最大」的係數(絕對值大代表該處有清晰的邊緣細節，即該張在此處對焦準確)→ 反轉換得到處處清晰的融合影像。這在顯微攝影、微距攝影、工業檢測(檢查有高低起伏的工件)中極為實用。

11.8.3 案例三：小波去雜訊對照第 7 章

情境：同一張含高斯雜訊的影像，分別用第 7 章的方法(高斯、中值、非局部平均、維納)與本章的小波門檻化去雜訊，比較成效。小波法的特色在於它對「邊緣的保留」通常較佳——因為它能在小波域中區分「邊緣的高頻(大係數，保留)」與「雜訊的高頻(小係數，歸零)」，而空間域的高斯模糊會無差別地抹平兩者。以 PSNR 與 SSIM 量化，並附上邊緣區域的局部放大，學生能清楚看到不同方法的取捨。這個案例把第 7 章與第 11 章的去雜訊工具箱完整地擺在一起比較。

11.8.4 案例四：壓縮率與品質的權衡實驗

情境：對同一張影像，分別以不同壓縮率存成 JPEG 與 JPEG 2000(或用 PyWavelets 模擬小波壓縮)，繪製「壓縮率對 PSNR/SSIM」的率失真曲線(rate-distortion curve)，並在高壓縮率處並排比較兩者的視覺瑕疵。學生會看到：低壓縮率時兩者差異不大，但當壓縮率推到極高(如 50:1),JPEG 碎裂成明顯的方塊，而小波僅是整體柔化——這正是叫獸開講中 FBI 捨 JPEG 就小波的量化證據。這個案例也是理解「率失真」這個壓縮領域核心概念的最佳實作。

11.9 本章與全書的觀念連結總表

相關章節	共用的觀念	本章的延伸或差異
第 3 章 數位影像基礎	位元深度、PSNR/SSIM、檔案格式	壓縮的品質以 PSNR/SSIM 量化;RAW 對照壓縮格式
第 5 章 空間域強化	平滑去雜訊	小波門檻化是另一種去雜訊途徑
第 6 章 頻率域	換基底、頻率、測不準	小波補上傅立葉缺的位置資訊(多解析度)
第 7 章 影像還原	去雜訊工具箱	小波門檻化與維納、NLM 互補，較能保邊緣
第 8 章 色彩處理	YCbCr、色度降取樣	JPEG 第一步即 YCbCr 轉換與 4:2:0 降取樣
第 9 章 影像分割	尺度空間	多解析度分析與 LoG/DoG 的尺度思想相通
第 13 章 特徵擷取	SIFT 尺度空間	SIFT 的多尺度金字塔與小波多解析度同源
第 15 章 深度學習	多層降取樣、多尺度特徵	CNN 的層級降取樣與小波多解析度精神相通

表 11-2 第 11 章與全書其他章節的觀念連結

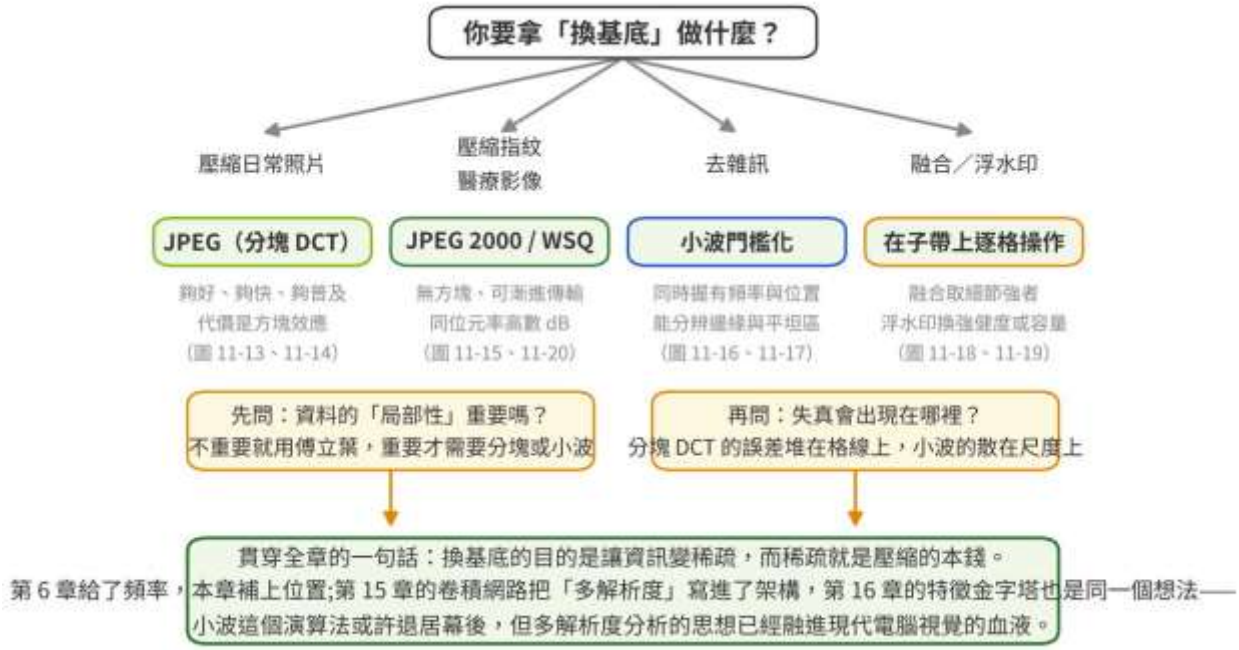


圖 11-21　第 11 章的決策地圖

程式實作 Lab 11

Lab 11-1 手算 Haar 小波

目標：用最簡單的方式理解小波。(a)不用任何函式庫，對一維數列 [8, 4, 6, 10, 12, 8, 4, 2] 手算(或用 Python 純運算)一層 Haar 分解，得到 4 個概略值與 4 個細節值，並驗證能無失真還原;(b)繼續對概略序列做第二、三層分解，觀察多解析度結構;(c)把最小的細節係數歸零後還原，計算與原數列的誤差——體會「丟棄小係數」的壓縮思想;(d)用 PyWavelets 的 pywt.wavedec 驗證你的手算結果。

Lab 11-2 影像的小波金字塔

目標：視覺化二維小波分解。(a)以程式 11-2 對一張影像(建議用指紋圖或紋理豐富的圖)做單層分解，把 LL、LH、HL、HH 四個子帶拼成一張圖顯示，說明每個子帶凸顯了什麼方向的結構;(b)做三層分解，顯示金字塔結構，觀察能量如何集中在頂層 LL;(c)統計各子帶中「絕對值小於某門檻」的係數比例，驗證細節子帶的稀疏性;(d)只保留最大的 5%、10%、25% 係數，其餘歸零後重建，計算各自的 PSNR——這就是

小波壓縮的核心實驗。

Lab 11-3 JPEG 方塊效應解剖

目標：親眼見證叫獸開講的方塊效應。(a)以程式 11-3 對一張影像用品質 90、50、30、10 分別存成 JPEG，計算各自的 PSNR 與檔案大小，繪製率失真曲線;(b)把品質 10 的結果放大到像素級，清楚標出 8×8 方塊邊界與蚊狀雜訊;(c)手動實作單一 8×8 區塊的 DCT 與量化，統計量化後非零係數的數量，理解壓縮如何發生;(d)找一張含銳利邊緣(如文字)的影像，觀察高壓縮率下邊緣附近的蚊狀雜訊——說明它與第 6 章振鈴效應的關聯。

Lab 11-4 小波去雜訊擂台

目標：把第 7 章與第 11 章的去雜訊方法同場競技。(a)對一張乾淨影像加入已知強度的高斯雜訊;(b)分別用高斯模糊、中值、非局部平均(第 7 章)與小波門檻化(本章)去雜訊;(c)以 PSNR 與 SSIM 製作比較表，並附上「邊緣區域局部放大」的對照圖;(d)特別觀察：小波法在「邊緣銳利度」與「平坦區乾淨度」的平衡上表現如何?為什麼它能同時做到(提示：在小波域區分邊緣高頻與雜訊高頻)?(e)討論：什麼情況下你會選小波去雜訊，什麼情況下選非局部平均?

Lab 11-5 反覆存檔的真相

目標：親手推翻（或補上條件給）一個流傳很廣的說法，並練習設計對照實驗。(a)取一張灰階影像，用同一個品質 q75 連續「存檔再讀回」20 次，每一代都與最初的原影像計算 PSNR，畫出世代—PSNR 曲線;(b)統計第 1 代與第 2 代之間有多少像素不同，佔全部像素的百分比是多少?試從 round(F/Q)×Q 的冪等性解釋這個數字為什麼這麼小，以及它為什麼不是零（提示：反 DCT 之後還要取整數與截斷到 0~255）;(c)改為每一代存檔前先把影像平移一個像素（且不移回），與同樣平移量的原影像比對 PSNR，重畫曲線;(d)再改為每一代輪流使用 90、80、70、85、75、95、65 的品質，重畫曲線。注意第 20 代其實是用 q95 存的，請與「對原影像單次存 q95」

的 PSNR 比較，並解釋這個落差說明了什麼;(e)把三條曲線畫在同一張圖上，寫出你會給攝影同好的一句具體建議。

Lab 11-6 浮水印的公平擂台

目標：學會設計「控制變因」的比較實驗——這是本章最重要的方法論訓練。(a)以 db2 做 3 層分解，分別把同一個 ±1 的二值浮水印嵌入 LL3（低頻）、第 2 層與第 1 層的細節子帶（中頻、高頻），嵌入方式為「係數 += α × 浮水印 × 該子帶係數絕對值的平均」;(b)先用固定的 α 比較三者，記錄各自與原影像的 PSNR——你應該會發現三者差距極大，這樣比較強健度是不公平的;(c)改用二分搜尋調整每個位置的 α，讓三者的 PSNR 都等於 40 dB;(d)對三張浮水印影像施加六種攻擊：不攻擊、JPEG q50、JPEG q20、高斯模糊 σ=1.2、加高斯雜訊 σ=15、縮放到一半再放大回來。每次都取出浮水印並與原訊息計算相關係數，畫成分組長條圖;(e)你的結果支持還是推翻「浮水印應該嵌在中頻」這個常見建議?請同時報告三個子帶的係數個數（容量），並用「強健度對容量」重新表述這個取捨;(f)觀察低頻那張影像的平坦區（例如天空），你看得到 PSNR 與 SSIM 都沒反映出來的淡暈影嗎?這說明了什麼?

本章重點整理

- 正交轉換的共同思想是「換一組基底表示影像」，使能量集中於少數係數(稀疏)，而稀疏是壓縮的黃金鑰匙;正交性帶來可逆、能量守恆、係數互不干擾三大好處。
- 傅立葉的根本弱點是「有頻率、無位置」(全域基底);短時傅立葉以固定窗口取得局部性但受測不準限制;小波以縮放與平移實現「低頻用長窗、高頻用短窗」的自適應多解析度分析，同時掌握頻率與位置。
- Haar 小波以「相鄰值的平均與差分」做分解，可手算;二維小波一層產生 LL(縮略圖)、LH、HL、HH 四個子帶，多層時只續分 LL，形成能量集中於頂層的金字塔。

- DCT 只用餘弦基底、係數為實數、能量集中;JPEG 五步驟為 YCbCr 降取樣、8×8 分塊 DCT、量化(唯一失真來源)、zigzag 與遊程編碼、熵編碼;反覆存檔只有在「改變品質設定」或「中途裁切縮放」時才會累積失真，同品質且不動影像的再存檔近乎冪等（11.4.3 節）。
- 小波壓縮(JPEG 2000、WSQ)作用於整張影像不切塊，從根本避免方塊效應，並天然支援漸進式與多解析度存取;但 JPEG 因夠好、夠快、夠普及而仍是日常主流。
- 小波應用：門檻化去雜訊(能區分邊緣高頻與雜訊高頻，較保邊緣)、多焦點/多模態影像融合、中頻域數位浮水印;其「多解析度」思想延續到 SIFT 尺度空間與 CNN 的層級降取樣。

習題

1. 正交轉換的共同思想是什麼?為什麼「換基底使能量稀疏」有利於壓縮?自然影像在空間域是否稀疏?

   提示與參考解答：共同思想是把影像從一組基底轉換到另一組基底表示。換到好的基底(頻率域、小波域)後能量集中於少數係數(稀疏)，只需記住少數重要係數、丟棄大量近零係數即可重建，故利於壓縮。自然影像在空間域不稀疏(每個像素都攜帶資訊)，但轉換後變稀疏——這正是轉換的價值。

2. 「正交」是什麼意思?它為轉換帶來哪三個好處?

   提示與參考解答：正交指基底函數彼此內積為零、互相獨立不重疊。三個好處：轉換可逆(能無失真變回原影像)、能量守恆(Parseval 定理，轉換前後總能量不變，可判斷丟棄係數的能量損失)、係數互不干擾(改一個不影響其他)。

3. 傅立葉轉換的根本弱點是什麼?用一個生活化的比喻說明。短時傅立葉如何補救，又受什麼限制?

   提示與參考解答：弱點是「有頻率、無位置」：基底是無限延伸的弦波，只知道有哪些頻率、不知道頻率來自哪個位置。比喻：分析樂曲只能列出用到哪些音符，卻不知先後順序。短時傅立葉把影像切塊各自轉換以取得位置，但受測不準限制：窗

口大則頻率準位置模糊，窗口小則位置準頻率模糊，無法兼得。

4. 小波如何同時掌握頻率與位置?請說明「低頻用長窗、高頻用短窗」背後的觀察。

   提示與參考解答：小波用有限長度的小波形為基底，透過縮放(對應頻率)與平移(對應位置)分析。背後觀察：低頻成分變化緩慢，需長窗才看得清(換取頻率精度);高頻成分變化劇烈短暫，需短窗才能定位(換取位置精度)。小波自動實現這種自適應，即多解析度分析。

5. 對數對 [8, 4] 做一層 Haar 小波分解，寫出概略值與細節值，並驗證還原。

   提示與參考解答：概略值(平均)= (8+4)/2 = 6;細節值(差的一半)= (8-4)/2 = 2。表示為 [6, 2]。還原：8 = 6+2、4 = 6-2，無失真。

6. 二維小波一層分解產生哪四個子帶?各凸顯什麼?多層分解時只對哪個子帶繼續分解，為什麼?

   提示與參考解答：LL(縮略圖，集中能量)、LH(水平邊緣)、HL(垂直邊緣)、HH(對角細節與雜訊)。多層時只續分 LL，因為能量集中在 LL，其餘子帶已稀疏;續分 LL 可得到由粗到細的金字塔，大部分能量集中在最頂層小 LL，其餘近零——這就是壓縮的機會。

7. 請完整列出 JPEG 壓縮的五個步驟，並指出哪一步是唯一的失真來源。另外：「反覆存檔 JPEG 會愈存愈糊」在什麼條件下成立、什麼條件下不成立?

   提示與參考解答：一、YCbCr 轉換與色度 4:2:0 降取樣;二、8×8 分塊做 DCT;三、量化(唯一失真來源);四、zigzag 掃描與遊程編碼;五、Huffman 熵編碼。量化把係數除以量化步長取整，不可逆。但再量化是冪等的：若每次都用同一張量化表且影像未被移動，係數已在格點上，round(F/Q)×Q 不會再改變它，實測連存 20 次幾乎無損。只有改變品質（換了量化表）或在存檔之間裁切縮放（打亂 8×8 對齊）才會累積失真。詳見 11.4.3 節與圖 11-14。

8. JPEG 的方塊效應從何而來?小波壓縮(JPEG 2000/WSQ)為什麼能避免它?

   提示與參考解答：方塊效應源於 JPEG 把影像切成獨立的 8×8 方塊各自 DCT 與量化，高壓縮率下相鄰方塊的量化差異在邊界顯現。小波轉換作用於整張影像(不切塊)，不存在方塊邊界，高壓縮率下只是整體柔化模糊而非碎裂，故避免方塊效應。

9. 承叫獸開講：FBI 為什麼不用 JPEG 而選擇小波(WSQ)來壓縮指紋?這對「技術選型」有何啟示?

   提示與參考解答：指紋辨識靠脊線的細微特徵點(分岔、端點),JPEG 的方塊效應與高頻流失會破壞或偽造這些鑑識關鍵特徵，法律上不可接受。小波無方塊、能保脊線細節，在 20:1 壓縮下無感知損失。啟示：技術選型須看應用的核心需求——一般照片能容忍方塊，鑑識則不能;沒有最好的壓縮，只有最適合任務的壓縮。

10. 小波門檻化如何去雜訊?為什麼它在保留邊緣方面常優於空間域的高斯模糊?

    提示與參考解答：自然影像的重要結構在小波域集中成少數大係數，雜訊散布成大量小係數;把小於門檻的小係數歸零(視為雜訊)、保留大係數(視為訊號)，再反轉換。它在「同時掌握頻率與位置」的小波域操作，能區分「邊緣附近的高頻(保留)」與「平坦區的高頻(雜訊，去除)」;高斯模糊則無差別抹平所有高頻，故傷邊緣。

11. 為什麼技術更優的 JPEG 2000 至今仍未取代 JPEG 成為日常主流?這反映了什麼工程通則?

    提示與參考解答：JPEG 運算簡單、硬體支援完善、相容性無敵、「夠好夠快夠普及」;JPEG 2000 複雜度較高、相容性較差，主要用於醫療、電影母帶、指紋等高要求專業領域。通則：最好的技術不一定勝出，「足夠好加上成熟生態系」往往才是贏家(與第 9、10 章「適合勝於最優」呼應)。

12. (綜合題)小波的「多解析度分析」思想，如何呼應第 9 章的尺度空間、第 13 章的 SIFT 與第 15 章的 CNN?

    提示與參考解答：多解析度=由粗到細、多尺度分析。第 9 章 LoG/DoG 在不同尺度找邊緣;第 13 章 SIFT 建高斯尺度金字塔在多尺度找特徵點;第 15 章 CNN 透過層級降取樣，每層看到更粗尺度的特徵。它們與小波「不同尺度各司其職」的精神同源——多尺度思維貫穿全書，是現代電腦視覺的核心觀念之一。

13. 承第 2 題的能量守恆與第 5 題的 [8, 4]：對它做一層 Haar 分解，若用「平均與差的一半」寫成 [6, 2]，轉換前後的能量各是多少?改用 1/√2 的正規化版本呢?由此說明為什麼正交性是 Parseval 定理的前提。

    提示與參考解答：原始能量 $8^2 + 4^2 = 80$。[6, 2] 的能量 = 36 + 4 = 40，少了一半，不滿足 Parseval。1/√2 版本為 [12/√2, 4/√2] ≈ [8.485, 2.828]，能量 = 72 + 8

= 80，與原始相同。把一層 Haar 寫成 H = c·[[1,1],[1,−1]]，HᵀH = 2c²·I，要等於單位矩陣必須 c = 1/√2——1/√2 是正交性唯一允許的值。壓縮之所以敢說「丟掉小係數只損失一點點」，正是因為正交保證了「係數的平方和 = 影像能量」，丟掉大小為 ε 的係數造成的重建誤差能量恰好是 ε²。詳見 11.3.3 節的推導。

14. 測不準原理在短時傅立葉上的具體意義是什麼?若取樣率 fs = 1000 Hz、窗口長度 w = 64 個樣本，時間解析度 Δt 與頻率解析度 Δf 各是多少?兩者的乘積與窗口長度有何關係?

    提示與參考解答：Δt = w / fs = 64/1000 = 0.064 秒;Δf = fs / w = 1000/64 ≈ 15.6 Hz。乘積 Δt·Δf = (w/fs)·(fs/w) = 1，與窗口長度完全無關——這就是測不準的具體形式：窗口拉長換來頻率精度，卻等量地犧牲時間精度，兩者的乘積是一個無法突破的常數。圖 11-4 用三個不同窗長驗證了這一點。小波的回應不是突破這個下限（做不到），而是重新分配：低頻用長窗（頻率準）、高頻用短窗（位置準），讓每個頻段拿到它最需要的那種精度。

15. 二維小波做 N 層分解後，一共有幾個子帶?若原影像為 512×512，做 3 層 db2 分解，係數的總數大約是多少?這對「壓縮」意味著什麼?

    提示與參考解答：N 層分解有 3N + 1 個子帶（每層產生 LH、HL、HH 三個，最後再加一個最頂層的 LL）。3 層就是 10 個子帶。係數總數與原像素數大致相同（邊界延拓會讓實際數字略多一些，例如 512×512 的 db2 三層分解約為 530×530 的量級）。關鍵認識：小波轉換本身「不壓縮任何東西」——它是可逆的換基底，係數不會變少。真正帶來壓縮的是轉換之後的稀疏性：能量集中到最頂層那個小小的 LL，其餘九個子帶的係數大多接近零，量化與熵編碼才有東西可榨。把「轉換」與「壓縮」分開理解，是讀懂 JPEG 與 JPEG 2000 的關鍵。

16. 教科書常說「DCT 比 DFT 更能集中能量」。請說明這個說法成立的真正原因，並指出它在什麼情況下差別很小、什麼情況下差別很大。

    提示與參考解答：原因在邊界，不在基底本身。DFT 隱含假設訊號是週期延拓的，若訊號左右兩端的值差很多，延拓後會在接縫處產生一個人造的跳階，這個不存在的邊緣必須靠大量高頻係數才能描述，能量因此被攤開。DCT 改用「鏡射延拓」，接縫處連續，就沒有這個假邊緣。所以差別的大小取決於訊號兩端的落差：對一條線性斜坡（兩端落差最大），圖 11-10 實測 DFT 需要 25 個係數才能涵蓋 99% 的交

流能量，DCT 只要 2 個;但對自然影像中隨機取出的 32 點列，兩者的中位數都是 22 個，幾乎分不出高下。這也解釋了 JPEG 為什麼選 DCT——它切成 8×8 的小塊，每一塊的邊界到處都是，邊界效應被放大了數千倍。

17. JPEG 的量化表會隨品質因子縮放。已知縮放倍率 s 的定義為：品質 q < 50 時 s = 5000/q，q ≥ 50 時 s = 200 − 2q，量化步長 Q' = floor((Q × s + 50) / 100)（下限為 1）。若標準亮度表的某個位置 Q = 16，請算出 q = 90、q = 50、q = 10 時的實際步長，並說明壓縮率與品質為何不是線性關係。

提示與參考解答：q = 90 → s = 200 − 180 = 20，Q' = floor((320+50)/100) = 3;q = 50 → s = 100，Q' = floor((1600+50)/100) = 16（即標準表本身）;q = 10 → s = 500，Q' = floor((8000+50)/100) = 80。從 q90 到 q50 步長變 5.3 倍，從 q50 到 q10 再變 5 倍，但 q 的區間一邊是 40、一邊也是 40——可見品質數字與實際量化強度之間是倒數關係，不是線性。這就是為什麼 q95 與 q90 的檔案大小差很多，q20 與 q10 的視覺差異卻遠比數字差距來得劇烈。

18. Zigzag 掃描為什麼能提升壓縮率?若某個 8×8 區塊量化後只有左上角 6 個係數非零，以列優先（row-major）掃描與以 zigzag 掃描，各自能得到多長的尾端連續零?這與 EOB（End of Block）標記有什麼關係?

提示與參考解答：量化後的非零係數集中在左上角（低頻），zigzag 以之字形由左上往右下掃，使頻率由低到高大致單調遞增，因此所有的零會聚集在序列尾端。列優先掃描則每掃完一列就跳回最左邊（回到低頻），零與非零交錯出現，無法形成長串。有了尾端的長串零，編碼器只要放一個 EOB 標記就能代表「後面全是零」，不必逐一記錄——這是 JPEG 壓縮率的主要來源之一。圖 11-12 用實際區塊比較了兩種掃描的游程長度。

19. 「反覆存檔 JPEG 會愈存愈糊」這句話在什麼條件下成立?什麼條件下不成立?請用量化的數學式說明，並指出實務上該如何避免累積失真。

提示與參考解答：存檔再打開在係數層面做的是 F → round(F/Q)×Q，也就是把係數吸附到最近的 Q 的整數倍上。若第二次用同一張量化表 Q，因為 F' 已是 Q 的整數倍，F'/Q 是整數、round 不改變它，所以 T(T(F)) = T(F)——這個運算是冪等的（同第 10 章開閉運算的冪等性）。圖 11-14 實測：同品質 q75 連存 20 次，PSNR 僅由 35.08 dB 變成 35.07 dB。不成立的條件有二：一是每次改變品質（換了量化

表），實測在 65~95 間變動存 20 代後只剩 30.8 dB，而該代其實是用 q95 存的（單次 q95 可達 45.1 dB）——已丟的資訊救不回來;二是存檔之間裁切、縮放或旋轉，8×8 的切割線錯位，實測 20 代掉了 9.7 dB。實務建議：工作檔用 PNG/TIFF，最後才輸出一次 JPEG;若必須反覆存，至少固定品質且不在中途裁切縮放。詳見 11.4.3 節。

20. 要比較 JPEG 與 JPEG 2000 的優劣，為什麼不能直接比「同一個品質設定下的 PSNR」?正確的比較方式是什麼?請說明率失真曲線的讀法。

    提示與參考解答：兩種編碼器的「品質」參數定義完全不同（JPEG 是量化表縮放倍率，JPEG 2000 是目標壓縮比），同樣標 50 並不代表花了同樣多的位元，比出來的是參數對照表，不是壓縮效率。正確做法是畫率失真曲線：橫軸是位元率（bpp = 檔案位元數 ÷ 像素數），縱軸是失真指標（PSNR 或 SSIM），讓兩者在「同樣的位元預算」下較量。由於兩條曲線的取樣點不會剛好對齊，報告差距時還要先把兩條曲線內插到同一個 bpp，才能說出「在 0.40 bpp 處相差幾 dB」這種有意義的句子。圖 11-15 即依此繪製。讀法：曲線愈靠左上愈好（同位元率下失真更小，或同失真下位元更少）。

21. 軟門檻為什麼要把留下來的大係數也往零收縮一個門檻值?這樣做的好處與代價各是什麼?為什麼軟門檻的最佳門檻通常比硬門檻低?

    提示與參考解答：觀測到的係數是 w = s + n（訊號加雜訊）。硬門檻在 |w| > T 時原封不動保留 w，等於把 s + n 當成 s，雜訊整個留了下來;軟門檻的 −T 就是用來抵消這一項，平均而言可以去掉一部分附著在訊號上的雜訊。代價是偏差：一個 w = 300 的強邊緣在 T = 75 時會變成 225，邊緣因此變淡。所以軟門檻是拿偏差換變異數。既然它本來就會額外削掉一截，門檻再設高就削過頭，最佳門檻自然較低。圖 11-17 實測：軟門檻在 1.50σ 達到 27.10 dB，硬門檻在 3.25σ 達到 26.54 dB。詳見 11.6.4 節的推導。

22. VisuShrink 建議的門檻是 T = σ·√(2·ln N)，其中 N 是係數個數。對一張 512×512 的影像，這個門檻相當於幾倍的 σ?實測顯示它的去雜訊效果明顯低於最佳門檻，這是否代表 VisuShrink 是錯的?

    提示與參考解答：N = 512×512 = 262,144，ln N ≈ 12.48，√(2 × 12.48) ≈ 5.0，所以 T ≈ 5.0σ。圖 11-17 實測這個門檻只得到 24.32 dB，比最佳的 27.10 dB 低了

2.78 dB。但這不代表它錯，而是它在最佳化另一個目標：VisuShrink 的設計準則是「以高機率讓純雜訊的係數全部被歸零」（N 個高斯變數的最大值約為 σ√(2 ln N)），追求的是視覺上的乾淨，不是 PSNR 最大。代價是連弱的真實細節也一併抹掉。這是工程上很常見的一課：先確認指標衡量的是不是你真正要的東西，再去談哪個方法比較好。

23. 在多焦點影像融合中，「直接平均」「逐點取最大」「小波逐子帶取細節較強者」三種策略的成效如何?為什麼前兩者會失敗?

提示與參考解答：圖 11-18 以人工製造的來源（同一張清晰影像左右各模糊一半，故有正確答案可比對）實測：直接平均 29.94 dB、逐點取最大 27.21 dB、小波融合 37.86 dB。直接平均失敗是因為它無差別地把清楚與模糊的部分各取一半，兩邊都變半糊;逐點取最大更差，因為「比較亮」與「比較清楚」是兩回事，它會把亮點誤判成細節，還可能破壞原本的明暗關係。小波融合的做法是在每個子帶的每個位置比較兩張影像誰的細節係數較強就選誰，等於在「頻率 × 位置」的網格上逐格做選擇——這正是小波同時握有兩種資訊才做得到的事。驗證：融合後左右兩半的拉普拉斯變異數都回到了原影像的水準。

24. 課本常說浮水印要嵌在中頻，理由是「低頻太明顯、高頻不耐壓縮」。若把三個位置的嵌入強度調到隱蔽性完全相同再比較強健度，結果會是什麼?由此說明浮水印設計真正的取捨是什麼。

提示與參考解答：圖 11-19 用二分搜尋讓三者的 PSNR 都等於 40 dB 再比較，結果低頻在六種攻擊下全部勝出：JPEG q20 之後低頻仍有 0.945，中頻 0.283，高頻只剩 0.020;連 SSIM 也站在低頻這邊（0.9912 對 0.9457、0.9454）。原因是清單上每一種攻擊（壓縮、模糊、加雜訊、縮放）本質上都是低通運算，它們摧毀的正是高頻。那為什麼不全塞低頻?因為容量：LL3 只有 4,356 個係數，第 1 層有 66,049 個，差 15.2 倍。所以真正的取捨是「強健度對容量」，而不是「強健度對隱蔽性」——藏一個序號就往低頻，藏一整段訊息才需要往高層走。補充：PSNR 與 SSIM 都低估低頻失真的可見度（平坦區會出現淡暈影），實務上仍會把低頻的嵌入強度再壓低一些。

25. (綜合題)要量化「方塊效應」有多嚴重，可以計算影像在 8 的倍數欄位上的平均梯度，再除以其他欄位的平均梯度。為什麼要做這個除法?若直接用「8 的倍數欄位的平均梯度」當指標會有什麼問題?

    提示與參考解答：直接用絕對值會被影像內容干擾——一張本來就布滿細節的影像，所有欄位的梯度都大，8 的倍數欄位自然也大，但那不是方塊效應。除以其他欄位的平均梯度等於做了正規化，衡量的是「格線位置的不連續程度是否異常高於一般位置」，與影像整體的紋理強弱無關。判讀時要看它與原圖基準的落差：圖 11-20 中原圖的指標是 1.03（格線位置並無特殊之處），JPEG 壓到 0.521 bpp 後升到 1.89，而小波在更低的 0.504 bpp 下仍是 1.03。這個設計思路值得記住：好的指標要能把「想測的效應」與「資料本身的性質」分開，否則測到的只是資料長什麼樣子。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6-7、8(小波、多解析度與影像壓縮)。
- Bradley, J. N., Brislawn, C. M., & Hopper, T., The FBI Wavelet/Scalar Quantization Standard for Gray-Scale Fingerprint Image Compression, 1993.(本章叫獸開講之史料來源)
- Daubechies, I., Ten Lectures on Wavelets, SIAM, 1992.(小波理論經典)
- Mallat, S., A Theory for Multiresolution Signal Decomposition: The Wavelet Representation, IEEE TPAMI, 1989.
- Wallace, G. K., The JPEG Still Picture Compression Standard, Comm. ACM, 1991.
- Taubman, D., & Marcellin, M., JPEG2000: Image Compression Fundamentals, Standards and Practice, Springer, 2002.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書
編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 12 章 視訊與動態影像處理

本章學習目標

- 說明視訊的本質(連續影格)與時間維度為影像處理帶來的新機會與新挑戰。
- 以 OpenCV 讀取、處理、寫出視訊與即時攝影機串流，理解影格率與編解碼器。
- 運用影格差分與背景相減完成移動偵測，並以型態學清理前景遮罩。
- 闡述光流的概念，區分稀疏光流(Lucas-Kanade)與稠密光流(Farnebäck)。
- 實作物件追蹤，區分「偵測」與「追蹤」的差異，並串接第 8、9、10、16 章。
- 認識視訊穩定、動作辨識與時序深度學習，理解時間維度在機器人視覺中的角色。

叫獸開講|一場關於奔馬的爭論，如何意外催生了電影

故事要從一個看似無聊的問題說起：一匹馬全速奔跑時，會不會有那麼一瞬間，四隻蹄子同時離地?在 1870 年代，這個問題無法回答——馬腿動得太快，人眼根本看不清。傳統繪畫總把奔馬畫成「飛躍式」：四腿向前後極力伸展、如同騰空飛翔。但這真的是馬奔跑的樣子嗎?沒有人能證明。

加州的鐵路大亨、前州長 Leland Stanford(後來史丹佛大學的創辦人)對這個問題著了迷。1872 年，他請來英國攝影師 Eadweard Muybridge 來解決它。流傳最廣的版本說，Stanford 為此下了兩萬五千美元的賭注——不過要誠實說，不少歷史學家認為這個「賭局」缺乏一手史料佐證，很可能是後人加油添醋的傳說。無論賭注是真是假，可以確定的是：這是一項需要當時最頂尖攝影技術的艱鉅挑戰，而 Stanford 願意持續投入重金。

經過六年的技術摸索(期間 Muybridge 還一度因槍殺妻子的情夫而中斷工作、受審),1878 年 6 月，他終於在史丹佛的帕羅奧圖農場架設起一整排相機——沿著跑道平行排列，相機的快門連接絆線，當名為「Sallie Gardner」的母馬奔馳而過、依序碰斷絆線，便觸發一台台相機，以大約二十五分之一秒的間隔連續拍下一整組動作。照片沖洗出來，答案揭曉：確實有一個瞬間，馬的四蹄全部離地——但不是傳統繪畫想像的伸展飛躍姿，而是四腿收攏於腹下的那一刻。從此，「飛躍式奔馬」的錯誤畫法在藝術界銷聲匿跡。

但這個故事真正的遺產遠不止於此。當 Muybridge 把這一系列靜態照片依序快速播放，觀眾看到的竟是一匹「活生生奔跑的馬」——史上第一次，連續的靜態影像創造出了「動態」的錯覺。這組名為《奔跑的馬》(Sallie Gardner at a Gallop)的照片，被公認為電影的源頭。一場關於馬蹄的爭論，意外開啟了整個電影工業。而它揭示的原理，正是本章的基石：視訊，本質上就是一疊「快速翻頁的照片」;而所謂的「動態」，是我們的視覺系統把一連串靜態影格腦補連接起來的結果。

這也帶出了視訊處理的核心洞見。前十一章，我們把每一張影像當成孤立的個體來處理;但視訊多了一個維度——時間。相鄰的影格之間有著極強的關聯(這一影格和上一影格通常只差一點點)，這個「時間上的連續性」既是新的資訊來源(可以偵測移動、追蹤物件、估計速度)，也是新的挑戰(要即時處理、要在影格與影格之間維持一致)。學會利用時間這個維度，是本章的目標，也是機器人「看懂動態世界」的關鍵。

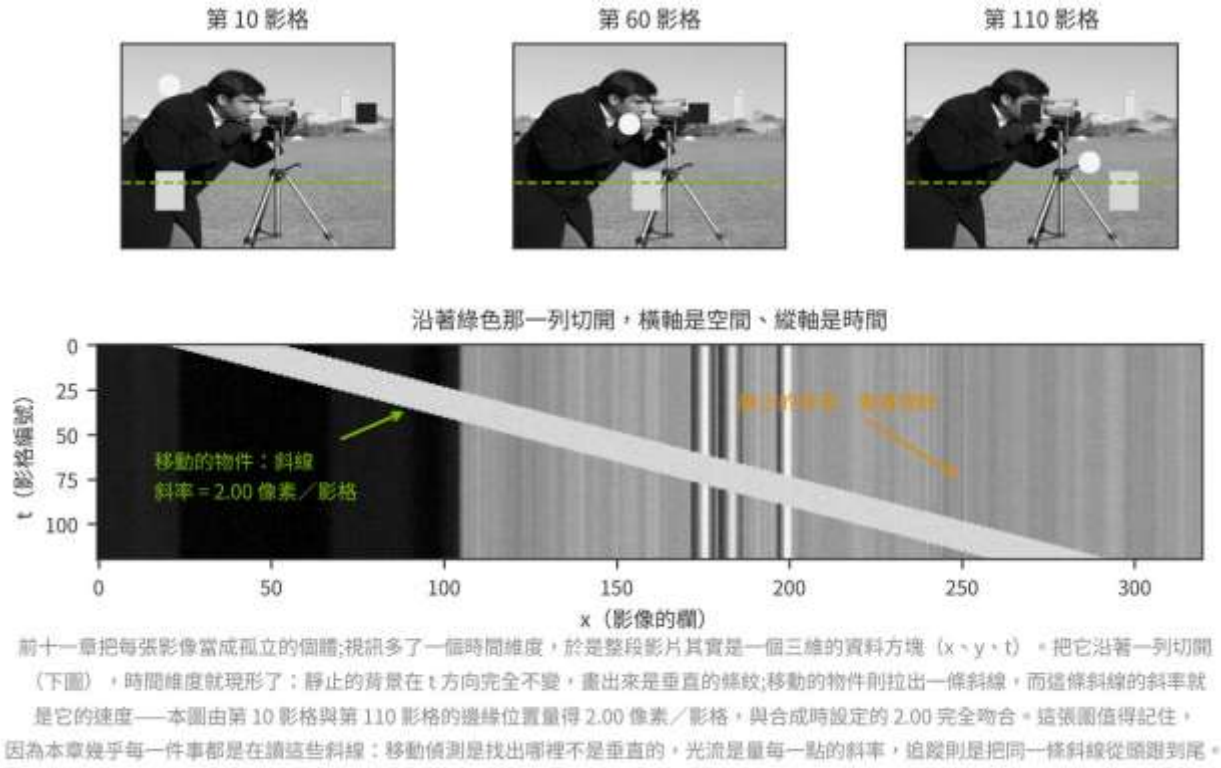
思考題：Muybridge 用「一排相機依序觸發」來把時間切成一格一格。今天的手機用單一感光元件、以固定影格率連續讀取來達成同樣的事。想一想：如果影格率不夠高(取樣時間的間隔太大)，拍攝高速旋轉的電風扇或車輪，會看到什麼奇怪的現象?這與第 3、6 章的哪個概念有關?

12.1 視訊的本質：多了一個時間維度

12.1.1 從影像到視訊

一張影像是二維的(寬、高)，而視訊多了第三個維度——時間。視訊是由一連串影格(frame)以固定的時間間隔排列而成，每一個影格都是一張完整的影像。播放的速度稱為影格率(frame rate)，單位是每秒影格數(fps)：電影通常是 24 fps，電視是 25 或 30 fps，而流暢的遊戲與高速攝影可達 60 fps 以上。當影格切換得夠快(一般認為超過每秒十餘格)，人類的視覺暫留就會把離散的影格「腦補」成連續的動態——這正是叫獸開講中 Muybridge 的發現，也是所有動態影像的生理基礎。

這個「時間軸」的加入，徹底改變了影像處理的格局。它同時帶來了機會與挑戰，理解這個雙面性，是掌握本章的關鍵。



前十一章把每張影像當成孤立的個體;視訊多了一個時間維度，於是整段影片其實是一個三維的資料方塊（x、y、t）。把它沿著一列切開（下圖），時間維度就現形了：靜止的背景在 t 方向完全不變，畫出來是垂直的條紋;移動的物件則拉出一條斜線，而這條斜線的斜率就是它的速度——本圖由第 10 影格與第 110 影格的邊緣位置量得 2.00 像素／影格，與合成時設定的 2.00 完全吻合。這張圖值得記住，因為本章幾乎每一件事都是在讀這些斜線：移動偵測是找出哪裡不是垂直的，光流是量每一點的斜率，追蹤則是把同一條斜線從頭跟到尾。

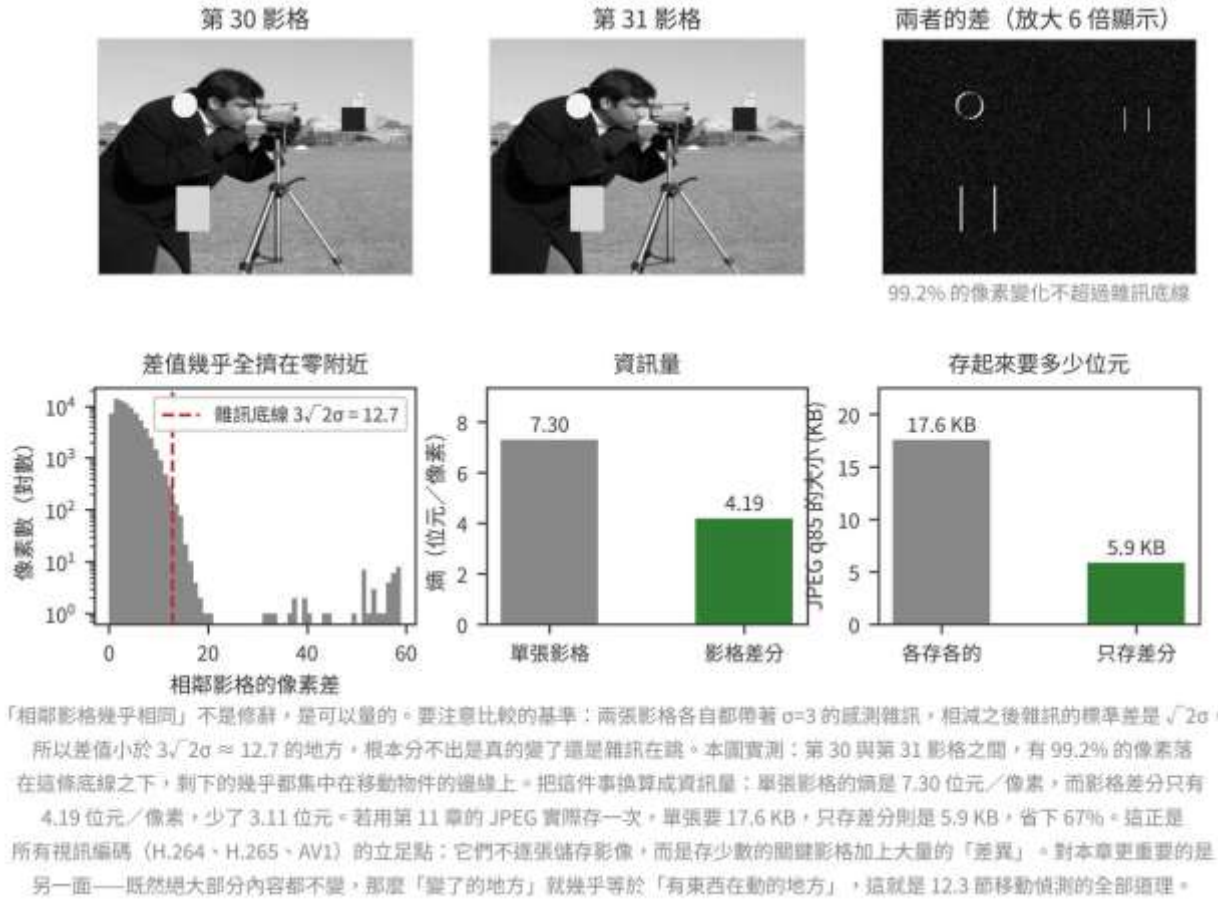
圖 12-1　把視訊切開來看：靜止是垂直條紋，移動是斜線

12.1.2 時間維度帶來的機會

相鄰影格之間有極強的時間關聯性——這一影格和上一影格，通常絕大部分內容

都相同，只有移動的物件或變化的區域不同。這個「幾乎不變」的特性，是一座資訊金礦：

- 偵測移動：把兩影格相減，不變的背景相消為零，只剩下移動的物件浮現(12.3 節)。單張影像永遠做不到這件事——「移動」本身就是一個跨越時間的概念。
- 追蹤物件：知道一個物件在這一影格的位置，由於下一影格它不會跳太遠，可以有效率地在附近找到它、串起它的軌跡(12.5 節)。
- 估計運動與速度：量測物件在連續影格間的位移，除以時間，就得到速度——這是機器人閃避障礙、預測行人動向的基礎(12.4 節光流)。
- 提升品質：多影格之間的冗餘可用於去雜訊(第 5 章夜景模式的多影格平均)、超解析度、視訊穩定。



「相鄰影格幾乎相同」不是修辭，是可以量的。要注意比較的基準：兩張影格各自都帶著 σ=3 的感測雜訊，相減之後雜訊的標準差是 $\sqrt{2}\sigma$，所以差值小於 $3\sqrt{2}\sigma \approx 12.7$ 的地方，根本分不出是真的變了還是雜訊在跳。本圖實測：第 30 與第 31 影格之間，有 99.2% 的像素落在這條底線之下，剩下的幾乎都集中在移動物件的邊線上。把這件事換算成資訊量：單張影格的熵是 7.30 位元／像素，而影格差分只有 4.19 位元／像素，少了 3.11 位元。若用第 11 章的 JPEG 實際存一次，單張要 17.6 KB，只存差分則是 5.9 KB，省下 67%。這正是所有視訊編碼（H.264、H.265、AV1）的立足點：它們不逐張儲存影像，而是存少數的關鍵影格加上大量的「差異」。對本章更重要的是另一面——既然絕大部分內容都不變，那麼「變了的地方」就幾乎等於「有東西在動的地方」，這就是 12.3 節移動偵測的全部道理。

圖 12-2　時間冗餘有多大：相鄰影格的差、熵與位元數

12.1.3 時間維度帶來的挑戰

但時間維度也帶來三個新挑戰：其一是即時性——視訊以每秒數十影格的速度湧入，處理演算法必須跟得上，否則就會掉影格或延遲，這對第 17 章的邊緣部署是嚴苛的考驗(一個在單張影像上跑得很好、但每一影格要一秒的演算法，對 30 fps 的視訊毫無用處)。其二是時間一致性——處理結果在影格與影格之間必須穩定，若每一影格獨立處理，偵測框可能會抖動、閃爍，追蹤編號可能會跳號。其三是資料量——視訊的資料量是影像的數十倍，儲存與傳輸都仰賴第 11 章的壓縮技術(視訊編碼更進一步利用了「影格間冗餘」，只記錄影格與影格之間的差異)。

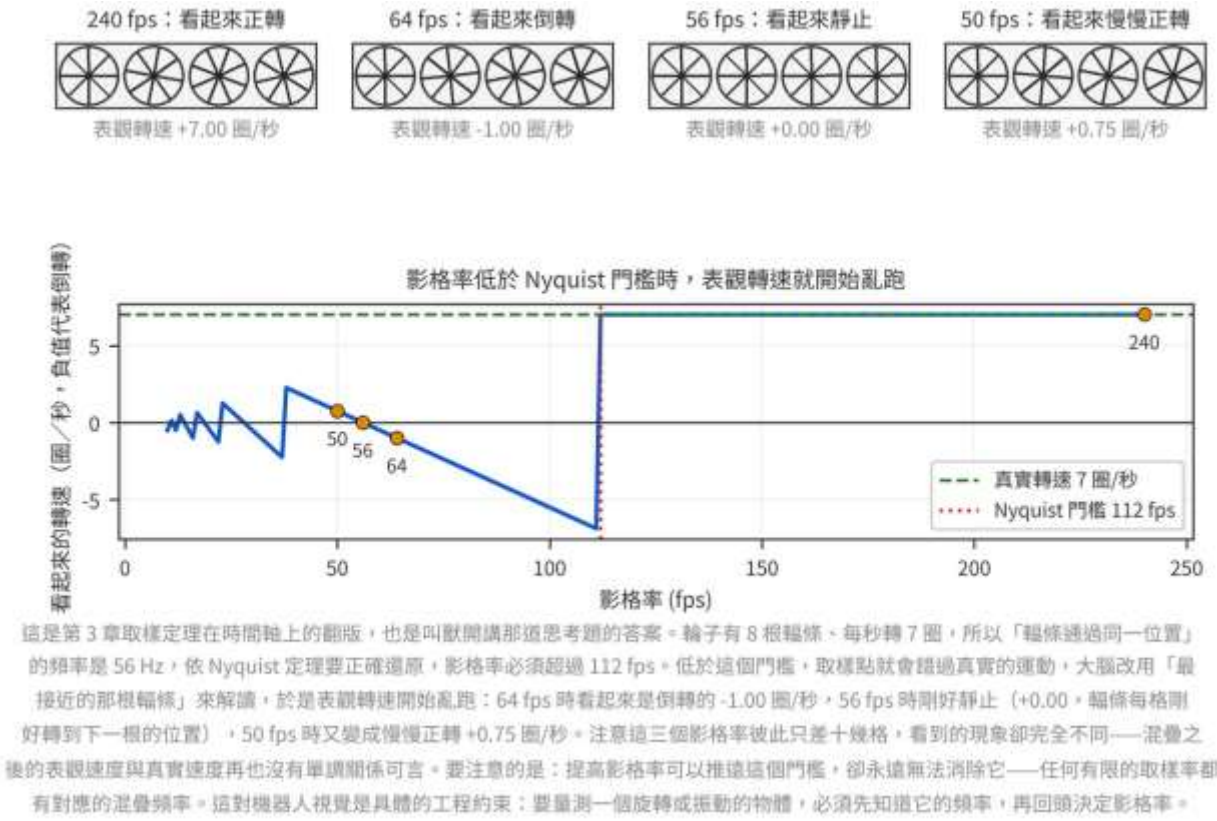


圖 12-3　時間混疊：影格率不足時，車輪會倒轉

12.2 視訊的讀取、處理與寫出

OpenCV 以 VideoCapture 讀取視訊(檔案或即時攝影機)，以 VideoWriter 寫出。核心觀念很簡單：視訊處理就是一個迴圈——逐影格讀取、把每一影格當成一張影像套用前面十一章的任何技術、再逐影格輸出或顯示。



圖 12-4　每一影格只有幾毫秒：各章演算法的實測耗時

程式 12-1 視訊讀取、處理與寫出的基本框架

```
import cv2

# 開啟視訊來源:檔名 → 讀檔;數字 0 → 預設攝影機;網址 → 串流
cap = cv2.VideoCapture("traffic.mp4")   # 或 0,或 ESP32-CAM 串流網址

# 查詢視訊屬性
fps    = cap.get(cv2.CAP_PROP_FPS)
width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"影格率={fps}, 尺寸={width}x{height}")

# 準備寫出(fourcc 指定編解碼器)
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("output.mp4", fourcc, fps, (width, height))

while True:
    ok, frame = cap.read()      # 逐影格讀取
    if not ok:                  # 讀完或讀取失敗
        break

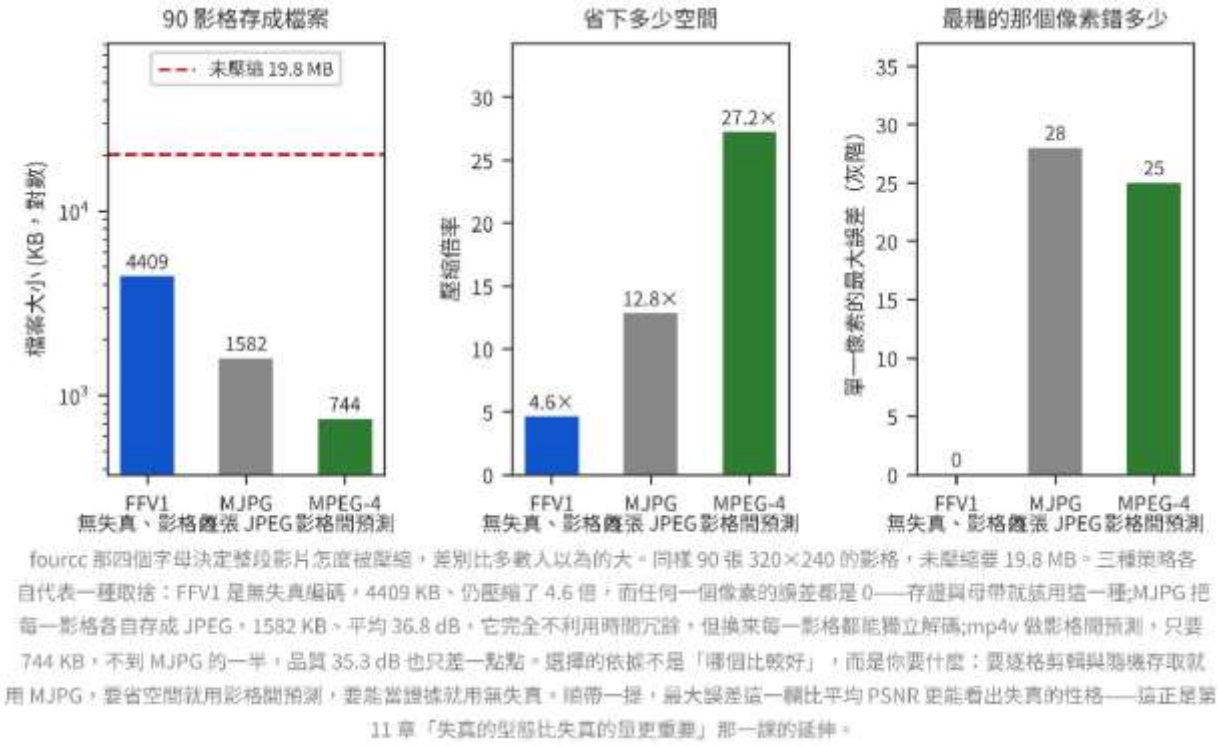
    # === 把每一影格當成一張影像:套用前 11 章的任何技術 ===
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 80, 160)          # 第 9 章
    result = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

    out.write(result)           # 寫出這一影格
    cv2.imshow("video", result)
    if cv2.waitKey(1) == 27:    # ESC 離開;waitKey 值影響播放速度
```

```
        break

cap.release()                       # 務必釋放資源
out.release()
cv2.destroyAllWindows()
```

幾個實務要點：第一，fourcc 是編解碼器的四字元代碼(如 mp4v、XVID、H264)，它決定輸出檔的壓縮方式，選錯會導致無法寫出或檔案過大——這直接關聯第 11 章的壓縮。第二，cap.read() 回傳的成功旗標一定要檢查，它是判斷「視訊是否讀完」的依據。第三，cv2.waitKey(n) 的 n 值(毫秒)控制每一影格之間的等待，影響顯示的播放速度;處理即時攝影機時通常設 1。第四，務必呼叫 release() 釋放資源，否則攝影機或檔案可能被鎖住。

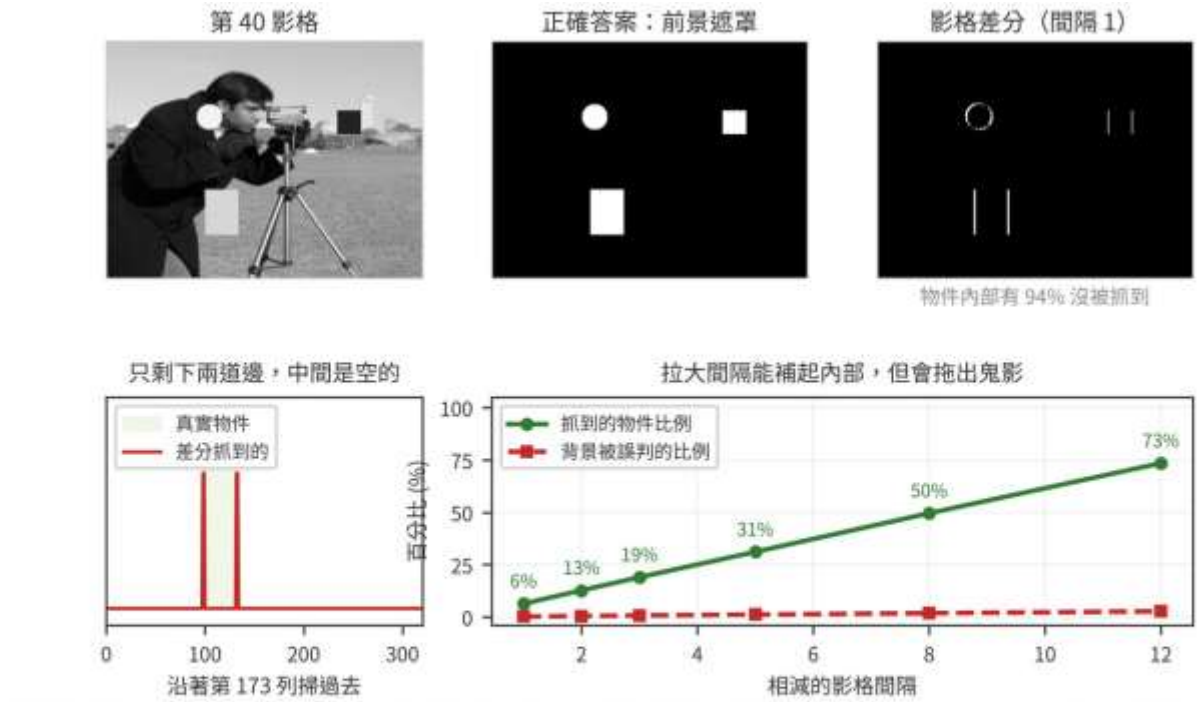


fourcc 那四個字母決定整段影片怎麼被壓縮，差別比多數人以為的大。同樣 90 張 320×240 的影格，未壓縮要 19.8 MB；三種策略各自代表一種取捨：FFV1 是無失真編碼，4409 KB，仍壓縮了 4.6 倍，而任何一個像素的誤差都是 0——存證與母帶就該用這一種;MJPG 把每一影格各自存成 JPEG，1582 KB、平均 36.8 dB，它完全不利用時間冗餘，但換來每一影格都能獨立解碼;mp4v 做影格間預測，只要 744 KB，不到 MJPG 的一半，品質 35.3 dB 也只差一點點。選擇的依據不是「哪個比較好」，而是你要什麼：要逐格剪輯與隨機存取就用 MJPG，要省空間就用影格間預測，要能當證據就用無失真。順帶一提，最大誤差這一欄比平均 PSNR 更能看出失真的性格——這正是第 11 章「失真的型態比失真的量更重要」那一課的延伸。

圖 12-5　編解碼器的三種策略：無失真、逐張、影格間預測

12.3 移動偵測：影格差分與背景相減

移動偵測是視訊處理最基礎也最實用的任務，它完美體現了「時間維度是資訊金礦」的觀念。核心思想只有一句：不變的是背景，變化的是前景。

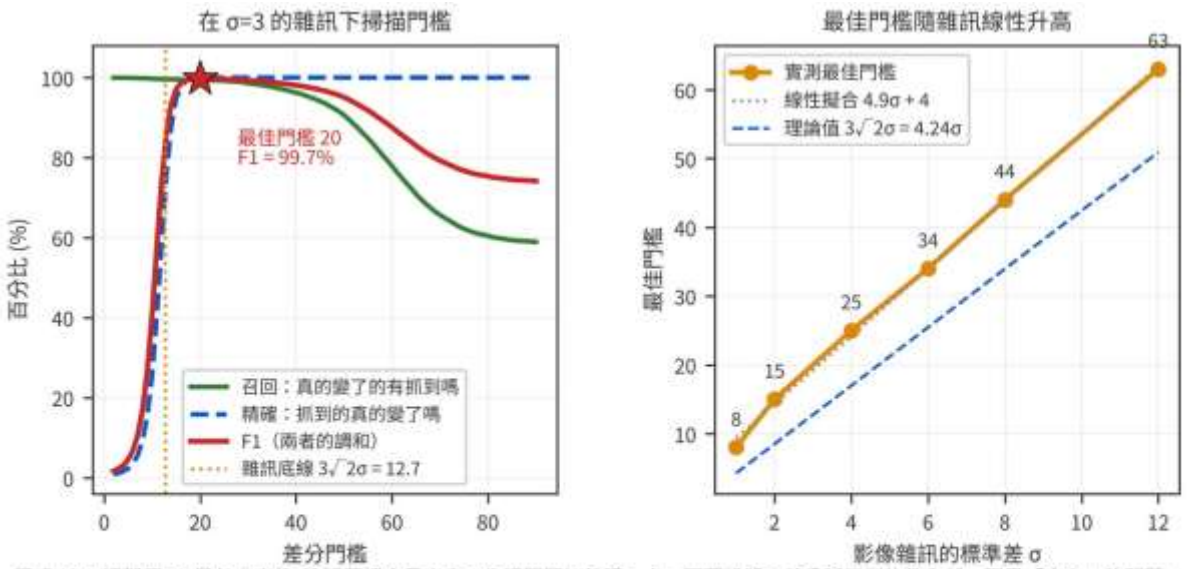


影格差分會得到「空心」的物件，這是初學者最常遇到的困惑，原因其實很單純：相減只會在「這一格與上一格不同」的地方留下數值。一個顏色均勻的物件，往右移動兩個像素之後，它的內部仍然蓋在同樣的灰階上，相減就是零——只有前緣（露出新背景）與後緣（蓋住舊背景）會留下痕跡。本圖實測：間隔 1 影格時，真實物件的面積有 94% 完全沒被抓到。把間隔拉大到 12 影格，抓到的比例從 6% 升到 73%，但背景的誤判率也從 0.2% 升到 2.7%——那些是物件「曾經在、現在不在」的位置，也就是鬼影。所以影格差分適合回答「有沒有東西在動」，卻不適合回答「那個東西長什麼樣、有多大」。要得到完整的物件，需要的是下一節的背景相減。

圖 12-6　影格差分為什麼是空心的

12.3.1 影格差分

最簡單的方法是影格差分(Frame Differencing)：把相鄰兩影格相減，取絕對值。靜止的背景在兩影格中相同，相減後為零(黑);移動的物件在兩影格中位置不同，相減後留下非零的差異(亮)。再對差值做門檻化(第 9 章)，就得到移動區域的遮罩。它的優點是極簡、極快、能自動適應緩慢的光線變化;缺點是：只能偵測到「正在移動」的部分，物件若暫停就消失;且移動物件的內部(顏色均勻處)前後影格相同，會被誤判為靜止，只留下物件的「邊緣輪廓」——這稱為「鬼影」或空心問題。



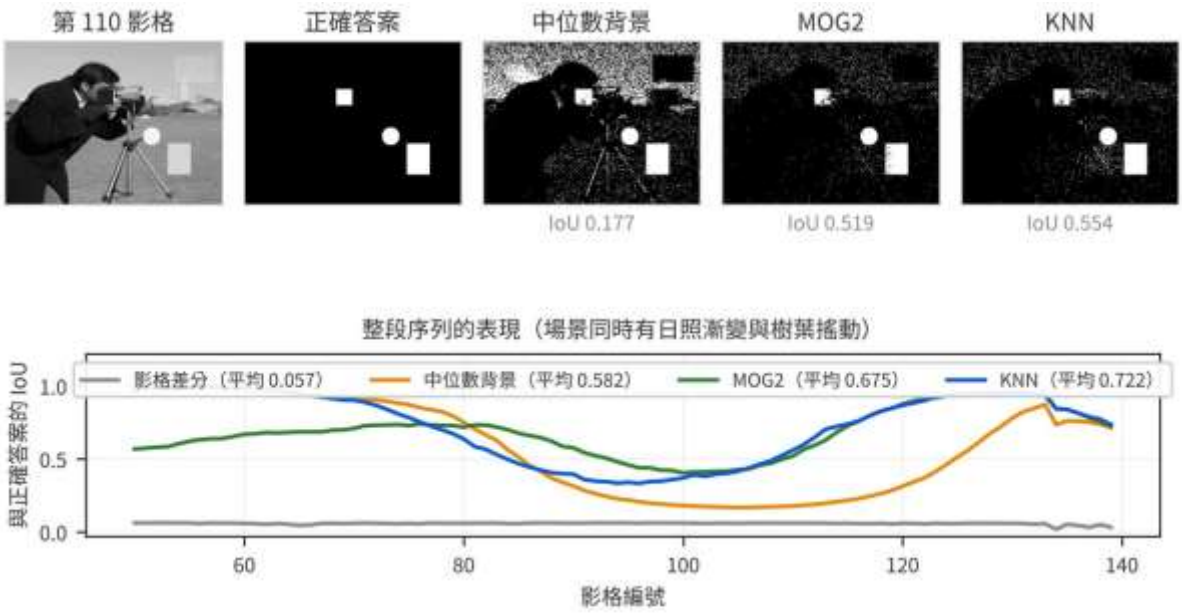
程式 12-2 裡那個 25 是怎麼來的？這張圖把它量出來。先把問題切乾淨：上一張圖說明了差分抓不到物件內部，那是「空心」的問題，不是門檻的問題。所以本圖的正確答案改用「乾淨序列中真的改變了的像素」（同一組合成參數、不加雜訊再跑一次），這樣量到的才是門檻真正在做的事：把真實變化與雜訊分開。結果：在 σ=3 的雜訊下最佳門檻是 20，F1 達到 99.7%（召回 99%、精確 100%）。右圖把雜訊從 σ=1 掃到 σ=12，最佳門檻從 8 升到 63，擬合出來是 4.9σ，與理論的 $3\sqrt{2}\sigma \approx 4.24\sigma$ 同一個量級（兩張含雜訊影格相減後，雜訊的標準差是 $\sqrt{2}\sigma$，取三倍即為可靠的底線）。這給了一條可操作的規則：門檻不該寫死，而該取一塊靜止區域量出 σ，再乘上約四倍。這也解釋了為什麼同一組參數換一台相機、換一個光照就失效。

圖 12-7　差分門檻怎麼挑：最佳門檻約為雜訊的四倍

12.3.2 背景相減

更強大的方法是背景相減(Background Subtraction)：先建立一個「純背景」的模型(沒有任何移動物件時的畫面)，再把每一影格與這個背景模型相減，差異顯著處就是前景。相較影格差分，它能偵測到完整的物件(而非只有邊緣)，且暫停的物件也不會消失。

關鍵難點在於「背景模型」如何建立與維護——真實場景的背景並非一成不變：光線會漸變(日照移動)、樹葉會搖動、水面會波動、攝影機會輕微震動。因此優秀的背景相減器必須是自適應的：它持續、緩慢地更新背景模型，把「長期穩定存在」的東西學習為背景，把「突然出現、快速移動」的東西判定為前景。OpenCV 提供的 MOG2(高斯混合模型)與 KNN 背景相減器就是這類自適應方法，它們為每個像素維護一個統計模型，能應付上述干擾，還能偵測並標記陰影。



這張圖把四種做法放在同一段序列上較量，而且場景刻意加了兩種真實世界的干擾：日照漸變（整體亮度緩慢起伏）與樹葉搖動（右上角一小塊持續抖動）。評分用 IoU——偵測到的前景與正確答案的交集除以聯集，1.0 代表完全吻合。影格差分平均只有 0.057，因為它永遠是空心的（上一張圖已經解釋過）。中位數背景 0.582：它能抓到完整物件，但背景模型一旦建好就不再更新，日照一變就整片誤判。MOG2 0.675 與 KNN 0.722 則持續緩慢更新模型，把「長期穩定存在」的學成背景、「突然出現」的判為前景，因此能同時應付漸變與抖動。本例中以 KNN 最好。這就是「自適應」三個字的具體意義：不是演算法比較聰明，而是它承認背景本來就會變。但也請注意三條曲線在中段同時明顯下滑——那正是日照變化最快的時候。自適應不等於免疫：模型更新需要時間，當場景變得比學習速度還快，誤判就無可避免。

圖 12-8　四種移動偵測在同一段序列上的正面交鋒

程式 12-2 影格差分與背景相減移動偵測

```
import cv2, numpy as np

cap = cv2.VideoCapture("surveillance.mp4")

# --- 方法一:影格差分 ---
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 方法二:自適應背景相減器(實務首選) ---
backsub = cv2.createBackgroundSubtractorMOG2(
            history=500, varThreshold=16, detectShadows=True)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

while True:
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 影格差分
    diff = cv2.absdiff(prev_gray, gray)
    _, motion = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
    prev_gray = gray

    # 背景相減 → 前景遮罩
```

```
    fgmask = backsub.apply(frame)

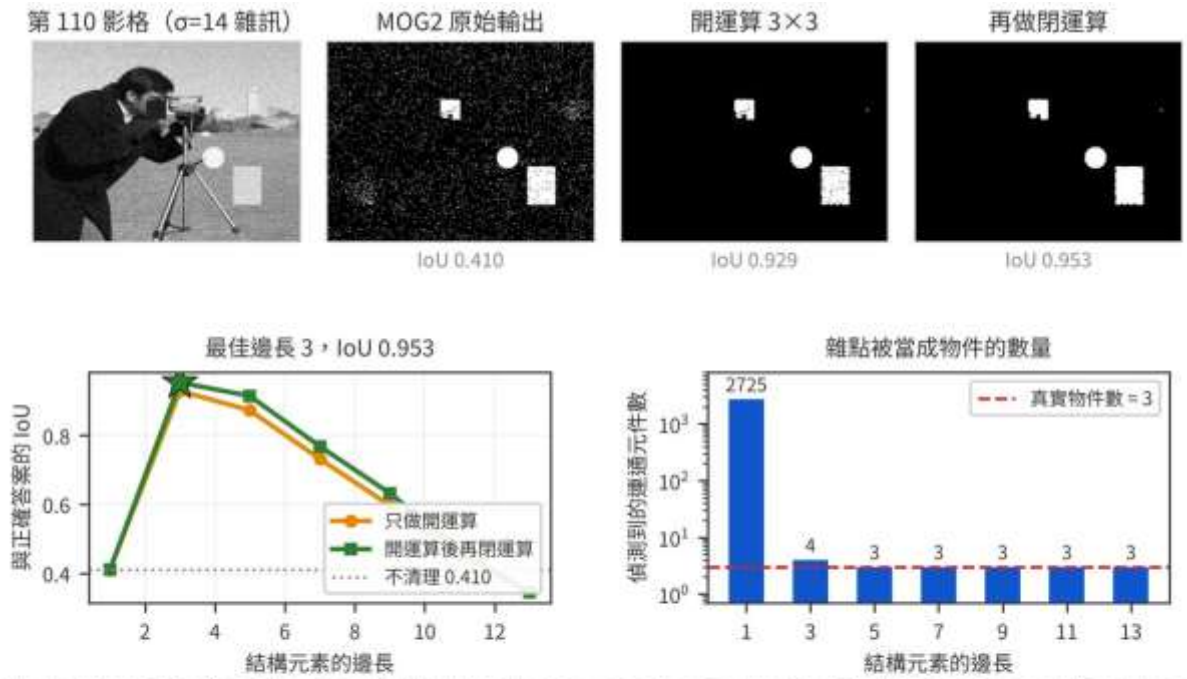
    # 型態學清理(第 10 章):去雜點、補破洞
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel)
    fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_CLOSE, kernel)

    # 連通元件框出移動物件(第 10 章)
    cnts, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
    for c in cnts:
        if cv2.contourArea(c) > 500:          # 面積門檻濾雜訊
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    cv2.imshow("motion", frame)
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

這段程式是本書多章的綜合：背景相減得到前景 → 第 10 章的型態學清理雜點與破洞 → 第 10 章的連通元件與第 9 章的輪廓框出物件。它是智慧監控、人流計數、交通流量分析、入侵偵測的骨幹，也正是第 9.7.4 節「視訊中的移動物件分割」的完整實作。



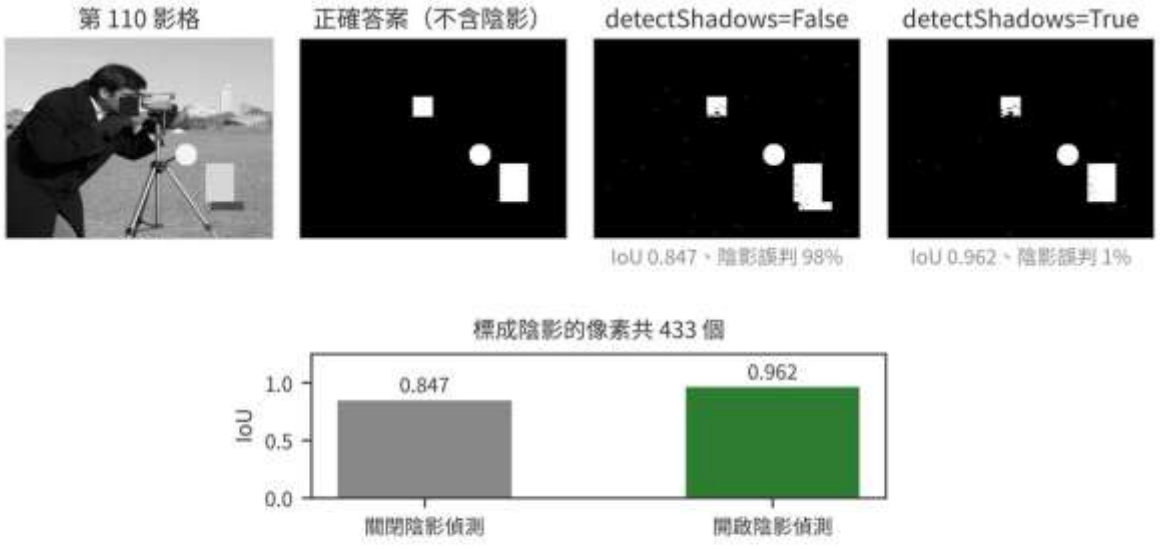
背景相減的輸出幾乎不可能直接用——它布滿雜點、物件內部有破洞。這正是第 10 章型態學派上用場的地方，而且順序有講究：先開運算（侵蝕再膨脹）去掉比結構元素小的雜點，再閉運算（膨脹再侵蝕）補起物件內部的破洞。本圖在 σ=14 的高雜訊下實測：未清理的 IoU 只有 0.410，開運算後升到 0.929，再閉運算達到 0.953。右圖是更具體的理由——未清理時連通元件多達 2725 個，而真實物件只有 3 個，也就是說若不清理就直接數物件，會得到荒謬的結果;邊長到 3 時才收斂到接近正確的數量。但也別把結構元素開得太大：邊長過大時 IoU 反而下降，因為小的物件連同雜點一起被侵蝕掉了。結構元素的大小應該由「最小的目標物件」決定，而不是由雜訊決定。

圖 12-9　型態學清理：接回第 10 章的開運算與閉運算

12.3.3 推導：背景模型的三個現實

背景相減在投影片上看起來很乾淨：建一個背景模型，相減，門檻化，完成。實際接上攝影機之後，會有三件事立刻找上門，而且都不是調參數就能繞過的。這一節把它們攤開來講，因為理解它們，比記住 MOG2 有哪些參數有用得多。

第一件是陰影。陰影跟著物件一起移動，所以背景相減理所當然地把它判為前景，框出來的物件因此比真實的大上一截——人流計數可能把一個人算成兩個，車速估計會算錯車尾的位置。陰影的判定靠的是一個很聰明的觀察：陰影只是讓像素「變暗」，但照到的還是同一塊地面，所以紋理與色度都沒變，只有亮度等比例下降。於是只要檢查「目前的像素值除以背景模型的像素值」是否落在一個固定區間內（例如 0.5 到 0.95）、且色度沒有明顯偏移，就能把它標成陰影。OpenCV 的 MOG2 把這件事做成 detectShadows 參數，開啟後輸出就變成三態：0 是背景、255 是前景、127 是陰影。這裡有一個很容易踩到的坑——程式裡必須寫 mask == 255，若沿用舊習慣寫 mask > 0，陰影會被當成前景，這個功能等於沒開。



陰影是移動偵測最常見的誤判來源，而且它有一個很討厭的性質：陰影會跟著物件一起移動，所以背景相減理所當然地把它判為前景，結果框出來的物件比真實大上一截（人流計數會把一個人算成兩個，車速估計會算錯車尾的位置）。MOG2 的 detectShadows 參數就是為此設計：它利用「陰影只是變暗、紋理不變」這個特性——若某個像素與背景模型的比值落在固定範圍內、且色度沒變，就標成陰影（輸出值 127 而非 255）。本圖實測：關閉時 IoU 0.847、陰影區有 98% 被誤判;開啟並濾掉 127 之後，IoU 升到 0.962、陰影誤判降到 1%，共有 433 個像素被正確標為陰影。要記得輸出值是三態的：0 是背景、255 是前景、127 是陰影，所以程式裡必須寫 mask == 255 而不是 mask > 0，否則這個功能等於沒開。

圖 12-10　陰影：跟著物件一起移動的那一塊誤判

第二件是停住的物件。很多教科書會說「背景相減的優點是暫停的物件不會消失」，這句話只對了一半。自適應的背景模型會持續把「長期穩定存在」的東西學成背景，所以一個物件只要停得夠久，它就會被吸收進背景，然後憑空消失。問題只在於「夠久」是多久——而這由學習率決定，也就是第三件事。

第三件是學習率，它是這三者的樞紐。MOG2 更新背景模型的方式大致是指數移動平均：$B \leftarrow (1-\alpha)\cdot B + \alpha\cdot I$，其中 α 就是學習率，I 是目前的影格。把這條式子展開，n 格之前的那一影格對現在的背景模型貢獻了 $\alpha(1-\alpha)^n$ 的權重，所以模型的「記憶長度」大約是 $1/\alpha$ 格——$\alpha = 0.01$ 記得住百來格，$\alpha = 0.05$ 就只記得住二十格左右。這一個數字同時決定了兩件相反的事：學得多快（暖身要幾格才把背景學乾淨），以及忘得多快（物件停住之後幾格會被吸收）。

圖 12-11 把這個取捨量了出來。學習率 0.05 時，背景模型只要 6 格就學乾淨了，但物件一停住，2 格之內就被吃掉;學習率 0.001 時，物件停住後可以撐 102 格不被吸收，代價是背景模型到第 170 格都還沒學乾淨。兩條曲線的形狀是對稱的，因為它們本來就是同一個 α 的兩面。

這個取捨決定了系統能不能用，而且答案完全取決於你要偵測什麼。要偵測「有沒有人經過」，學習率可以大一點，讓光線變化很快被吸收;但要偵測「有沒有人在門口徘徊」或「包裹被遺留在月台上」，學習率就必須非常小，甚至要另外維護一個更新極慢的長期背景模型，拿它與短期模型比對——短期模型認為是背景、長期模型認為是前景的區域，就是「最近才出現並停住」的東西。所以遺留物偵測在實務上是一個獨立的題目，而不是把背景相減的參數調一調就能解決。

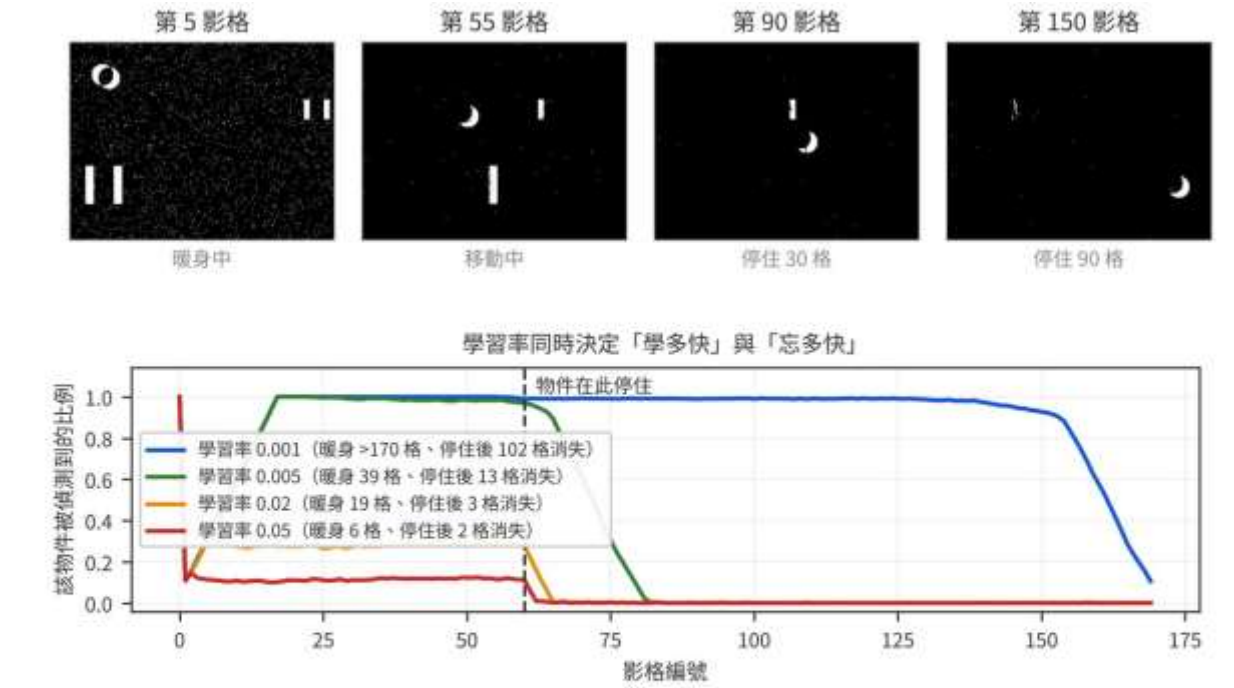


圖 12-11　學習率同時決定「學多快」與「忘多快」

12.4 光流：估計像素的運動

移動偵測只回答「哪裡有東西在動」，光流(Optical Flow)則更進一步，回答「每個點往哪個方向、以多快的速度移動」——它為影像中的點估計出運動向量。這是機器人理解動態世界的核心工具：知道了場景中每個點的運動，就能推算自身的移動、偵測逼近的障礙、預測物件的軌跡。

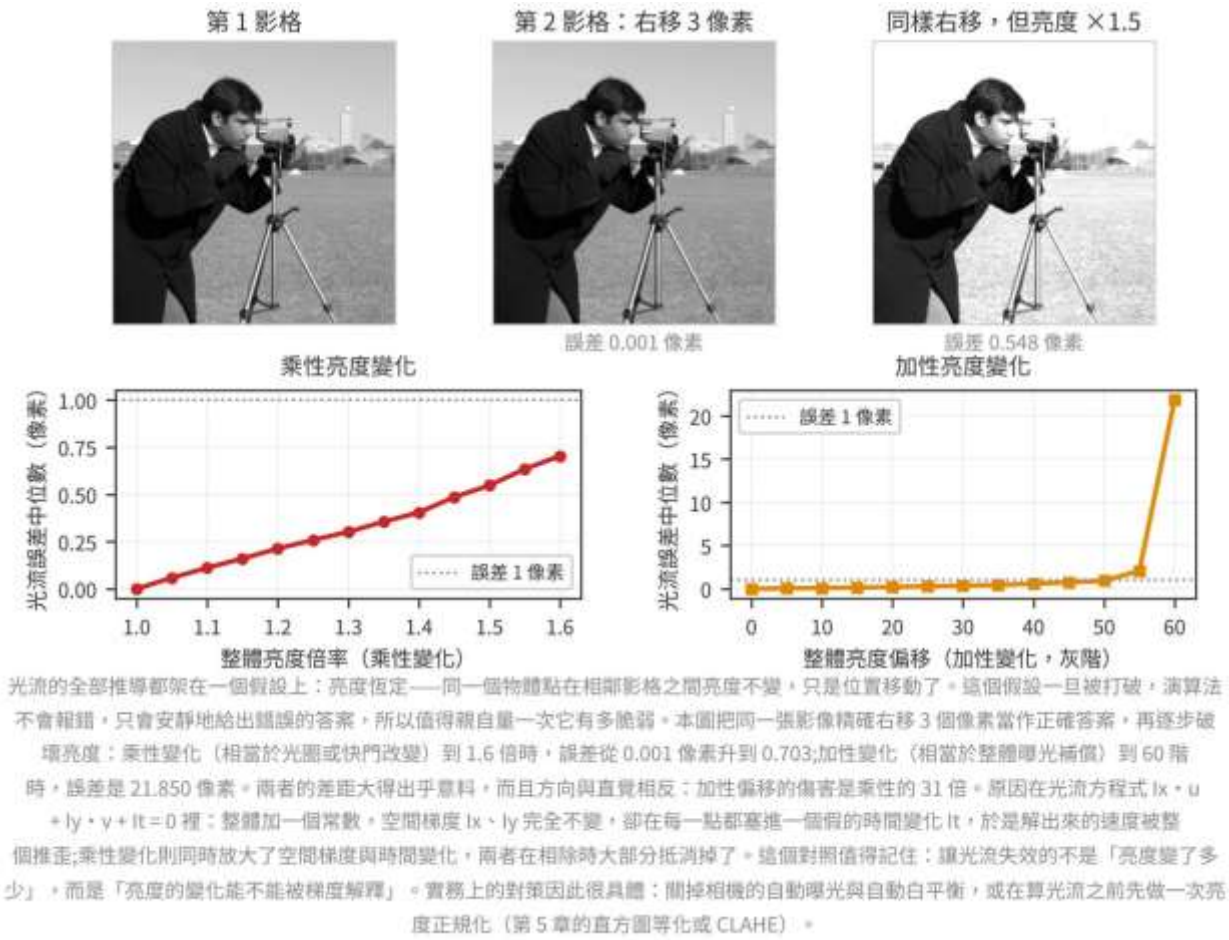
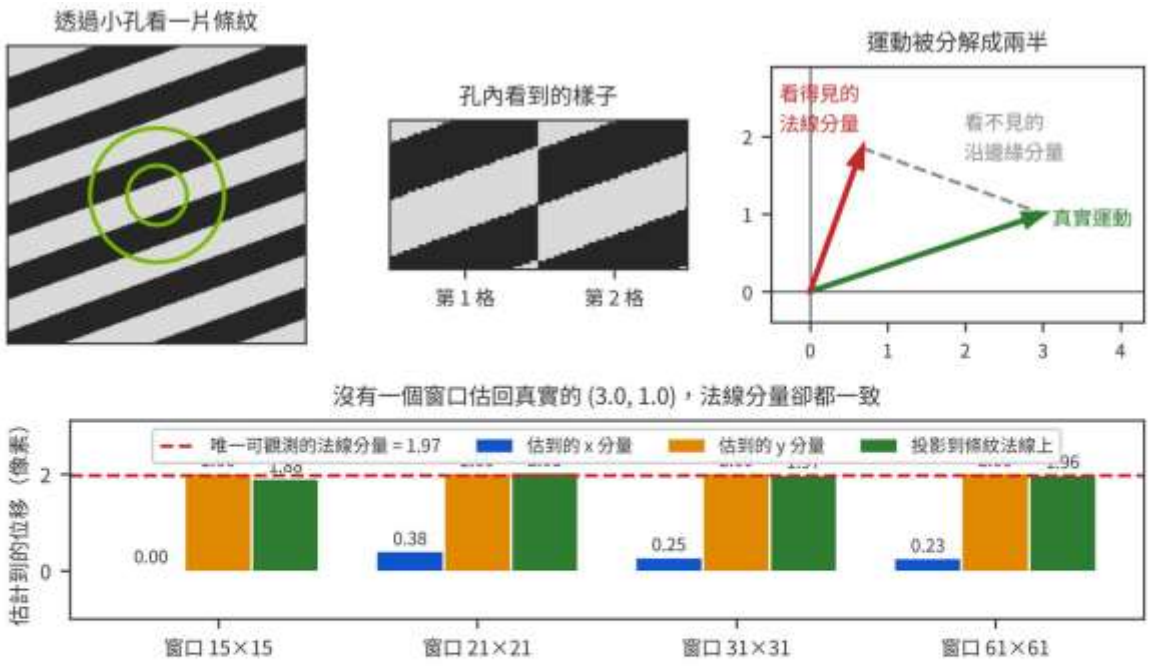


圖 12-12　亮度恆定假設垮掉時，光流不會報錯，只會算錯

12.4.1 基本假設與孔徑問題

光流建立在一個核心假設上：亮度恆定——同一個物體點在相鄰兩影格之間，亮度不變，只是位置移動了。由這個假設可推導出光流方程式。但這裡有一個著名的困境：孔徑問題(Aperture Problem)。透過一個小孔觀察一條移動的直線邊緣，你只能看出它「垂直於邊緣方向」的運動分量，而沿著邊緣方向的運動則無從察覺(就像只透過吸管看一根滑動的長棍，無法判斷它沿長軸滑了多少)。因此，單看一個像素的局部資訊，無法唯一決定它的運動——必須引入額外的假設(例如鄰近像素運動相似)才能求解。

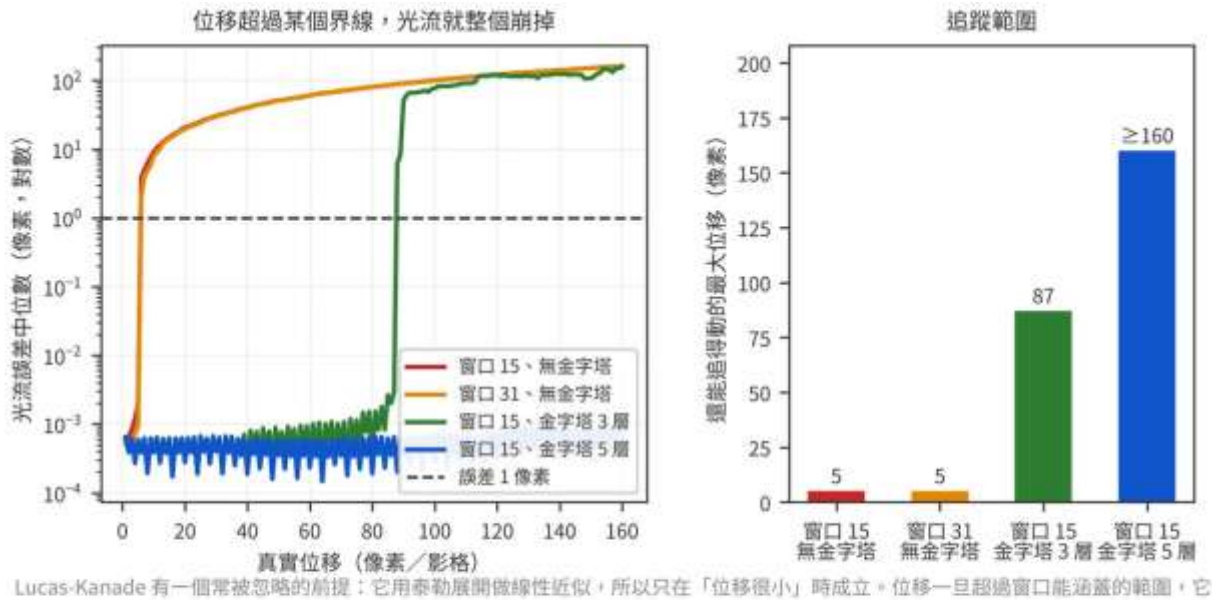


孔徑問題是光流最根本的限制：它不是演算法不夠好，而是資訊真的不在畫面裡。透過一個小孔看一片平行條紋移動，你只能看出條紋「垂直於自己」移動了多少;沿著條紋方向滑動多少，畫面一模一樣，因此無從得知。本圖的實測把這件事演得很清楚。同一對影格，只是改變 Lucas-Kanade 的窗口大小，估出來的位移就分別是 (0.00, 2.00)、(0.38, 2.00)、(0.25, 2.00)、(0.23, 2.00)——沒有一個等於真實的 (3.0, 1.0)，而且彼此也不一致。但把每一個都投影到條紋的法線方向，結果是 1.88、2.01、1.97、1.96，全部落在 1.97 附近。換句話說，演算法並沒有算錯：這些答案全都同樣正確，因為它們產生的畫面完全相同。真正被決定的只有法線分量，沿邊緣的那一半是任意的，LK 只是從無限多組解中挑了一組交給你。這直接解釋了 12.4.2 節為什麼要挑角點：角點同時含有兩個方向的梯度，兩個未知數才有兩條獨立的方程式。邊緣上的點只能定出一個分量，平坦區則一個都定不出來——這和第 13 章判斷「哪些點適合當特徵」是同一套道理。

圖 12-13　孔徑問題：每個窗口的答案都不同，法線分量卻一致

12.4.2 稀疏光流：Lucas-Kanade

Lucas-Kanade 法計算稀疏光流：它不追蹤每一個像素，而是只追蹤少數「適合追蹤的特徵點」(通常用第 13 章的 Shi-Tomasi 角點——因為角點在兩個方向都有明顯變化，能繞過孔徑問題)。它假設一個小鄰域內的所有像素運動相同，藉此求解每個特徵點的運動向量。它高效、精準，適合追蹤特定的關鍵點，是視訊穩定、運動估計、視覺里程計(第 17 章機器人定位)的常用工具。

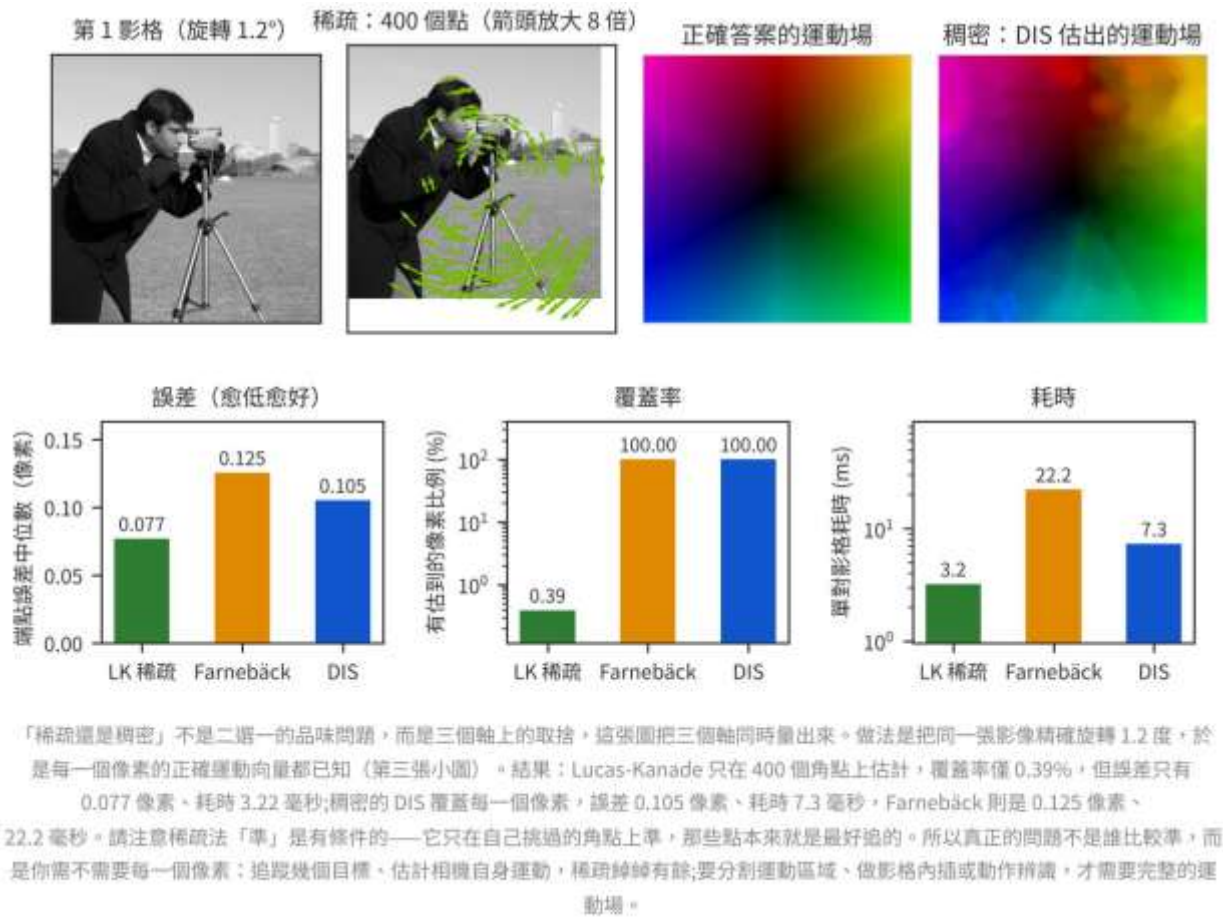


Lucas-Kanade 有一個常被忽略的前提：它用泰勒展開做線性近似，所以只在「位移很小」時成立。位移一旦超過窗口能涵蓋的範圍，它不是變得不準，而是完全失效。本圖把同一張影像精確平移已知的距離當正確答案，量出誤差中位數開始超過 1 像素的那個界線：窗口 15、無金字塔為 5 像素、窗口 31、無金字塔為 5 像素、窗口 15、金字塔 3 層為 87 像素、窗口 15、金字塔 5 層為 ≥160 像素（≥ 代表掃到本圖上限 160 像素仍未崩潰）。兩件事值得記住。第一，把窗口從 15 放大到 31 對追蹤範圍幾乎沒有幫助（5 對 5 像素）——因為限制來自線性近似的有效範圍，而那由影像本身的紋理尺度決定，不是把窗口開大就能改變的。第二，金字塔才是真正的解法——它先在縮小 8 倍的影像上估出粗略位移（此時 87 像素的位移看起來只剩幾個像素），再逐層放大修正，等於把「大位移」化約成一連串「小位移」。這又是一次多解析度分析（第 11 章）的勝利。實務上的推論很直接：若你的目標移動很快，與其調大窗口，不如增加金字塔層數，或者乾脆提高影格率。

圖 12-14　Lucas-Kanade 追得動多快：窗口沒用，金字塔才有用

12.4.3 稠密光流：Farnebäck

稠密光流(如 Farnebäck 法)則為影像中的每一個像素都計算運動向量，得到一張完整的「運動場」。它資訊更豐富(能看出整個場景的運動結構)，但運算量遠大於稀疏光流。常用第 8 章的虛擬色彩把運動場視覺化：用色相(H)表示運動方向、飽和度或明度表示運動速度，於是整個畫面的運動一目了然。稠密光流用於動作辨識、視訊分割、慢動作生成(影格內插)等需要全域運動資訊的任務。



「稀疏還是稠密」不是二選一的品味問題，而是三個軸上的取捨，這張圖把三個軸同時量出來。做法是把同一張影像精確旋轉 1.2 度，於是每一個像素的正確運動向量都已知（第三張小圖）。結果：Lucas-Kanade 只在 400 個角點上估計，覆蓋率僅 0.39%，但誤差只有 0.077 像素、耗時 3.22 毫秒;稠密的 DIS 覆蓋每一個像素，誤差 0.105 像素、耗時 7.3 毫秒，Farnebäck 則是 0.125 像素、22.2 毫秒。請注意稀疏法「準」是有條件的——它只在自己挑過的角點上準，那些點本來就是最好追的。所以真正的問題不是誰比較準，而是你需不需要每一個像素：追蹤幾個目標、估計相機自身運動，稀疏綽綽有餘;要分割運動區域、做影格內插或動作辨識，才需要完整的運動場。

圖 12-15　稀疏與稠密：誤差、覆蓋率、耗時三個軸上的取捨

程式 12-3 稀疏與稠密光流

```
import cv2, numpy as np

cap = cv2.VideoCapture("walking.mp4")
ok, prev = cap.read()
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)

# --- 稀疏光流:先找適合追蹤的角點(第 13 章 Shi-Tomasi) ---
pts = cv2.goodFeaturesToTrack(prev_gray, maxCorners=100,
        qualityLevel=0.3, minDistance=7)

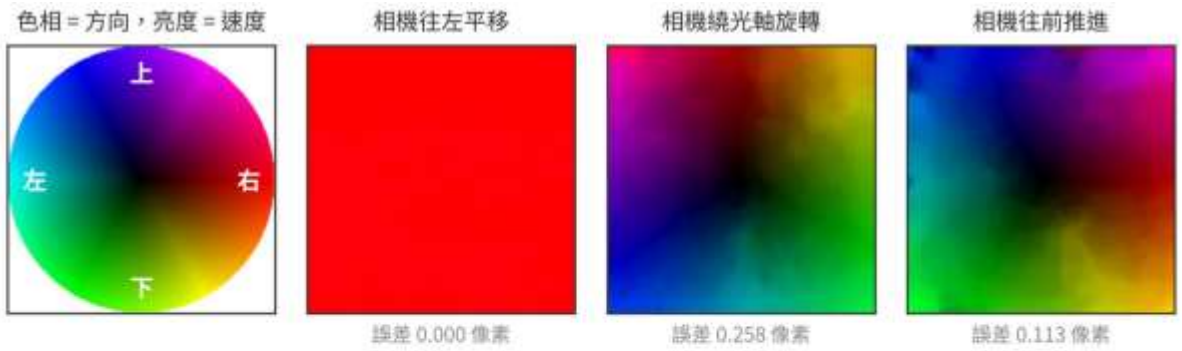
while True:
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Lucas-Kanade 稀疏光流:追蹤這些點到新位置
    new_pts, status, err = cv2.calcOpticalFlowPyrLK(
        prev_gray, gray, pts, None)
    for (nx, ny), (ox, oy) in zip(new_pts[status==1],
                                  pts[status==1]):
        cv2.line(frame, (int(nx),int(ny)), (int(ox),int(oy)),
                 (0,255,0), 2)
```

```
        cv2.circle(frame, (int(nx),int(ny)), 3, (0,0,255), -1)

    # --- 稠密光流:每個像素的運動,用色彩視覺化 ---
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
            0.5, 3, 15, 3, 5, 1.2, 0)
    mag, ang = cv2.cartToPolar(flow[...,0], flow[...,1])
    hsv = np.zeros_like(frame)
    hsv[...,0] = ang * 180 / np.pi / 2      # 方向 → 色相(第 8 章)
    hsv[...,1] = 255
    hsv[...,2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    flow_vis = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    cv2.imshow("sparse", frame)
    cv2.imshow("dense", flow_vis)
    prev_gray = gray
    pts = new_pts[status==1].reshape(-1,1,2)
```



稠密光流每個像素有兩個數字，沒辦法直接畫成灰階圖，所以慣例是借用第 8 章的 HSV：把運動的方向編成色相、大小編成亮度。最左邊是對照用的色輪——往右是青色、往下是藍紫、往左是紅橙、往上是綠。學會讀它之後，整個場景的運動結構一眼就能看出來：相機往左平移時，整張圖是單一顏色（所有像素同方向同速）；繞光軸旋轉時，顏色繞著中心轉一圈，而且中心最暗（轉軸上不動）、邊角最亮（半徑最大、線速度最快）；往前推進時，顏色從中心向外放射，像素都在往外跑，中心那個不動的點就是「擴張焦點」（Focus of Expansion），它指出相機正在往哪裡前進——這是視覺里程計與障礙閃避的關鍵線索，也是 12.7.4 節機器人案例的核心。所以運動場的圖樣本身就帶著幾何意義：均勻色塊代表平移，漩渦代表旋轉，放射狀代表前進或後退。

圖 12-16　運動場的色彩怎麼讀：平移、旋轉、前進的三種圖樣

12.4.4 推導：孔徑問題的數學形式，與金字塔為什麼有效

先把光流方程式寫出來。亮度恆定假設說的是：同一個物體點在相鄰影格間亮度不變，也就是 I(x, y, t) = I(x + u, y + v, t + 1)。對右邊做一階泰勒展開並消去 I(x, y, t)，得到 $I_x \cdot u + I_y \cdot v + I_t = 0$，其中 I_x、I_y 是空間梯度、I_t 是時間梯度。這條式子就是整個光流理論的起點，而它一眼就看得出問題：一條方程式，兩個未知數（u 與 v）。

這正是孔徑問題的數學形式。單一個像素提供的約束，只能把解限制在一條直線上——也就是只定出運動在梯度方向（邊緣的法線方向）的分量，沿著邊緣的那一個

分量完全沒有被約束。圖 12-13 用實測演了這件事：同一對影格，只改變 Lucas-Kanade 的窗口大小，估出來的位移分別是 (0.00, 2.00)、(0.38, 2.00)、(0.25, 2.00)、(0.23, 2.00)，沒有一個等於真實的 (3.0, 1.0);但把每一個都投影到條紋的法線方向，答案都是 1.96 左右。演算法沒有算錯——這些解產生的畫面完全相同，是資訊本身不足以分辨。

Lucas-Kanade 的補救是加上一個假設：一個小窗口內的所有像素具有相同的運動。於是窗口內的 N 個像素各給一條方程式，兩個未知數就有 N 條方程式，可以用最小平方法解。寫成矩陣形式，要解的是一個 2×2 的方程組，而那個 2×2 矩陣正是第 13 章 Harris 角點偵測用的同一個結構張量（由 I_x^2、I_xI_y、I_y^2 在窗口內的和組成）。這個矩陣可逆的條件，就是「兩個特徵值都夠大」，也就是角點的定義。所以 12.4.2 節說「要挑角點來追蹤」並不是經驗法則，而是這個方程組有唯一解的必要條件——平坦區兩個特徵值都接近零（矩陣奇異，一個分量都解不出來），邊緣上一個大一個接近零（只解得出法線分量）。

還有第二個限制，來自那次泰勒展開：一階近似只在位移很小的時候成立。位移一旦超過影像紋理的尺度，線性近似就完全失效——注意是失效，不是變得不準。直覺上會想「把窗口開大一點不就好了」，但圖 12-14 的實測說不行：窗口從 15 放大到 31，可追蹤的最大位移都是 5 個像素，一點也沒有改善。原因是限制來自近似的有效範圍，而那由影像本身的紋理尺度決定，把窗口開大只是納入更多同樣失效的方程式。

真正的解法是金字塔，而它的想法正是第 11 章的多解析度分析。先把兩張影格都縮小 2^n 倍，在最粗的那一層上，原本 80 個像素的位移看起來只剩幾個像素，線性近似重新成立，於是可以估出一個粗略的位移;把這個結果放大一倍當作下一層的起點，只需要再估一個小小的修正量;逐層下來，就等於把一個「大位移」化約成一連串「小位移」。圖 12-14 的實測：同樣的窗口 15，加上 3 層金字塔之後可追蹤的位移從 5 像

素跳到 87 像素，5 層則超過本圖掃描的 160 像素。實務上的推論很直接——若目標移動很快，與其調大窗口，不如增加金字塔層數（maxLevel），或者乾脆提高影格率把位移變小。

```
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

12.5 物件追蹤

12.5.1 偵測與追蹤的差別

這是視訊分析中一組容易混淆卻至關重要的概念。偵測(Detection)是在單一影格中「找出物件在哪裡」，每一影格獨立進行，彼此無關;追蹤(Tracking)則是在連續影格中「維持同一個物件的身分」，把它在時間軸上的位置串成一條軌跡，並給它一個穩定的編號(ID)。

兩者為什麼要分工?因為各有優劣。偵測(尤其是第 16 章的 YOLO)準確但通常較耗算力，且它不知道「這一影格的人」和「上一影格的人」是不是同一個。追蹤則利用時間連續性，運算快(只需在上一影格位置附近搜尋)，還能維持身分——但它可能會「跟丟」(物件被遮擋、快速移動時)或「跟錯」(兩個物件交錯時身分互換)。

實務上最強健的方案是兩者結合，稱為「偵測式追蹤」(Tracking-by-Detection)：每隔幾影格用偵測器(YOLO)重新定位所有物件(校正累積誤差、發現新物件)，中間的影格則用輕量的追蹤演算法快速更新位置並維持 ID。著名的 SORT、DeepSORT 演算法就是這個框架，廣泛用於多物件追蹤(MOT)——這也是第 16 章 YOLO 追蹤功能的底層邏輯。

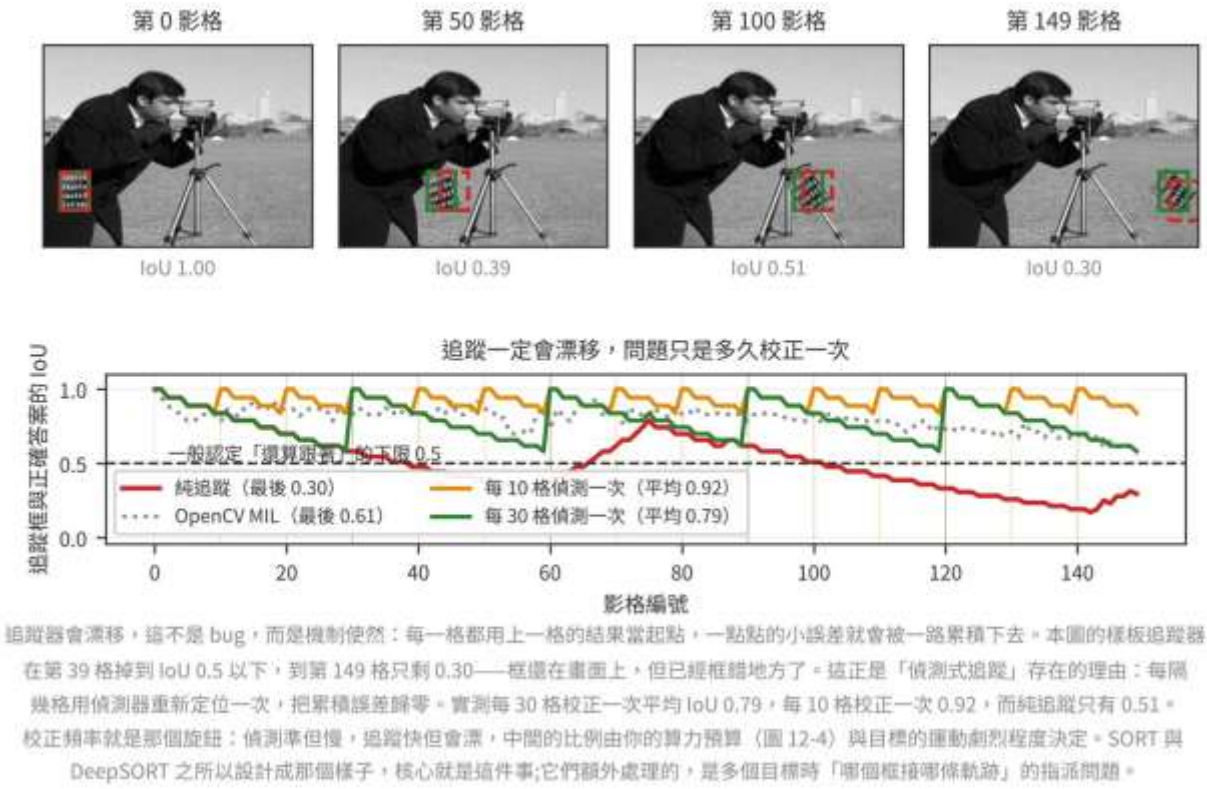


圖 12-17　追蹤一定會漂移，問題只是多久校正一次

12.5.2 追蹤演算法

傳統追蹤器種類繁多：MeanShift 與 CamShift 基於顏色直方圖(第 8 章)，追蹤顏色特徵明顯的物件;KCF、CSRT 等相關濾波器類追蹤器在速度與準確度間各有取捨(CSRT 較準、KCF 較快);卡爾曼濾波器(Kalman Filter)則從運動模型預測物件下一刻的位置，能在短暫遮擋時「猜測」物件的去向，是追蹤系統的經典組件。OpenCV 的 tracking 模組提供了這些追蹤器的現成實作。這裡要補一則現況提醒：KCF、CSRT、MOSSE 這些追蹤器一直住在 opencv_contrib 的 tracking 模組裡，而 OpenCV 5 的主套件(pip install opencv-python)並不包含它們，直接呼叫 cv2.TrackerCSRT_create() 會得到 AttributeError。主套件目前提供的是 TrackerMIL(傳統法)與 TrackerNano、TrackerVit、TrackerDaSiamRPN(以深度學習為基礎，需另外下載模型檔)。要用 KCF/CSRT 請改裝 opencv-contrib-python。在自己的環境上確認最快的方式是執行一行：print([x for x in dir(cv2) if x.startswith("Tracker")])。這類模組搬家在 OpenCV 的版本之間很常見，養成先查再

寫的習慣比記住 API 重要。

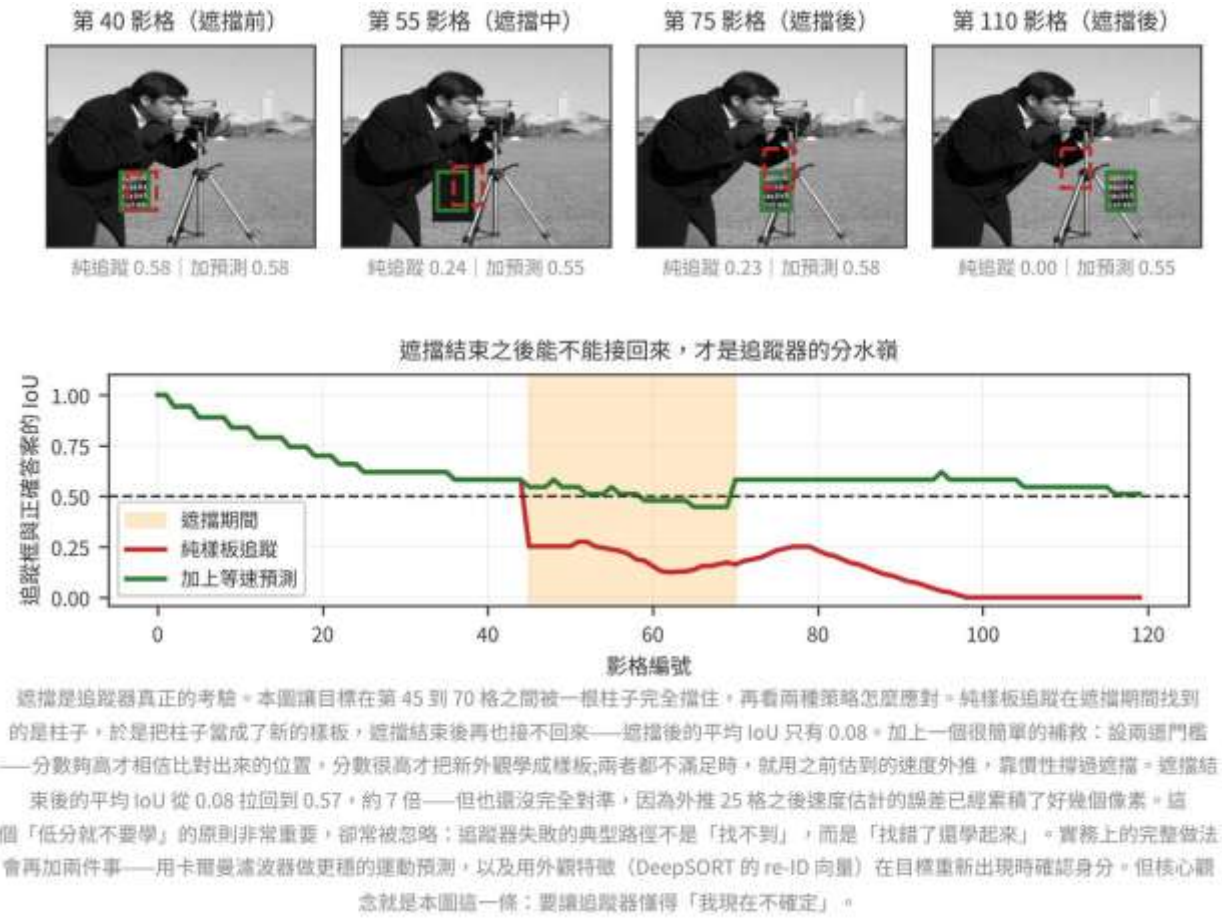


圖 12-18　遮擋：能不能接回來，才是追蹤器的分水嶺

程式 12-4 單物件追蹤

```
import cv2

cap = cv2.VideoCapture("target.mp4")
ok, frame = cap.read()

# 第一影格:框選要追蹤的物件(可手動或用第 16 章 YOLO 偵測)
bbox = cv2.selectROI("select", frame, False)

# 建立追蹤器(CSRT 較準,KCF 較快)
tracker = cv2.TrackerCSRT_create()
tracker.init(frame, bbox)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    success, bbox = tracker.update(frame)   # 在新影格中更新位置
    if success:
        x, y, w, h = [int(v) for v in bbox]
```

```
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    else:
        cv2.putText(frame, "Lost!", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

    cv2.imshow("tracking", frame)
```

12.5.3 推導：偵測多久該校正一次

追蹤器會漂移，這不是 bug，而是機制使然。追蹤的每一格都以上一格的結果為起點，所以每一格引入的小誤差不會被修正，只會被帶進下一格繼續累積。若每一格的位置誤差是獨立同分布、標準差為 σ 的隨機量，n 格之後累積誤差的標準差大約是 σ√n（隨機漫步）;而若誤差有系統性的偏向（例如樣板每格都往移動方向多學了一點背景），那就是 n·σ 的線性累積，更快。圖 12-17 的實測屬於後者：IoU 近乎線性地下滑，第 39 格就跌破 0.5，到第 149 格只剩 0.30——框還在畫面上，但已經框錯地方了。

偵測式追蹤的作用，就是每隔 k 格把累積誤差歸零一次。由此可以寫出一個粗略但好用的估算：若追蹤的漂移速率是每格 d 個像素，而你能容忍的最大偏移是 D 個像素，那麼校正週期 k 就該滿足 $k \le D / d$。圖 12-17 實測每 30 格校正一次的平均 IoU 是 0.79，每 10 格校正一次是 0.92，而純追蹤只有 0.51。另一邊是成本：偵測器（例如第 16 章的 YOLO）每次都要完整跑一遍，所以平均每格的運算量大約是「偵測成本 ÷ k + 追蹤成本」。k 就是那個旋鈕，把準確度與算力接在一起——這也是為什麼圖 12-4 的耗時實測值得放在本章的前面。

遮擋是另一個獨立的失敗模式，而且它的致命之處不在「找不到」，而在「找錯了還學起來」。當目標被擋住，比對分數最高的位置往往落在遮擋物上;若追蹤器照例把那裡的影像更新成新樣板，它就改追遮擋物了，遮擋結束後再也接不回來。圖 12-18 的實測：純樣板追蹤在遮擋結束後的平均 IoU 只剩 0.09。

補救的原則可以寫成一句話：要讓追蹤器懂得「我現在不確定」。具體做法是設兩道門檻——分數夠高才相信比對出來的位置，分數非常高才把新外觀學進樣板;兩者

都不滿足時，就不要更新任何東西，改用先前估到的速度把位置外推下去，靠慣性撐過遮擋。圖 12-18 實測這個簡單的補救讓遮擋後的平均 IoU 從 0.09 回到 0.57，約七倍;但也還沒完全對準，因為外推 25 格之後，速度估計的誤差已經累積了好幾個像素。

這正好指出實務系統還要補上的兩件事，也是 SORT 與 DeepSORT 的設計重點。其一是用卡爾曼濾波器取代土法外推：它用一個運動模型（通常是等速模型）持續預測下一刻的位置，並且會同時維護這個預測的不確定度，遮擋愈久、搜尋範圍就自動放得愈寬。其二是用外觀特徵確認身分：DeepSORT 為每個目標存一個再識別（re-ID）向量，目標重新出現時用它來確認「這是不是原來那一個」，而不是只看位置接不接得上。多目標時還有第三件事——「哪個偵測框該接哪一條軌跡」是一個指派問題，通常用匈牙利演算法求解。但這些都建立在本節這一條原則上：先承認自己不確定，才談得上接回來。

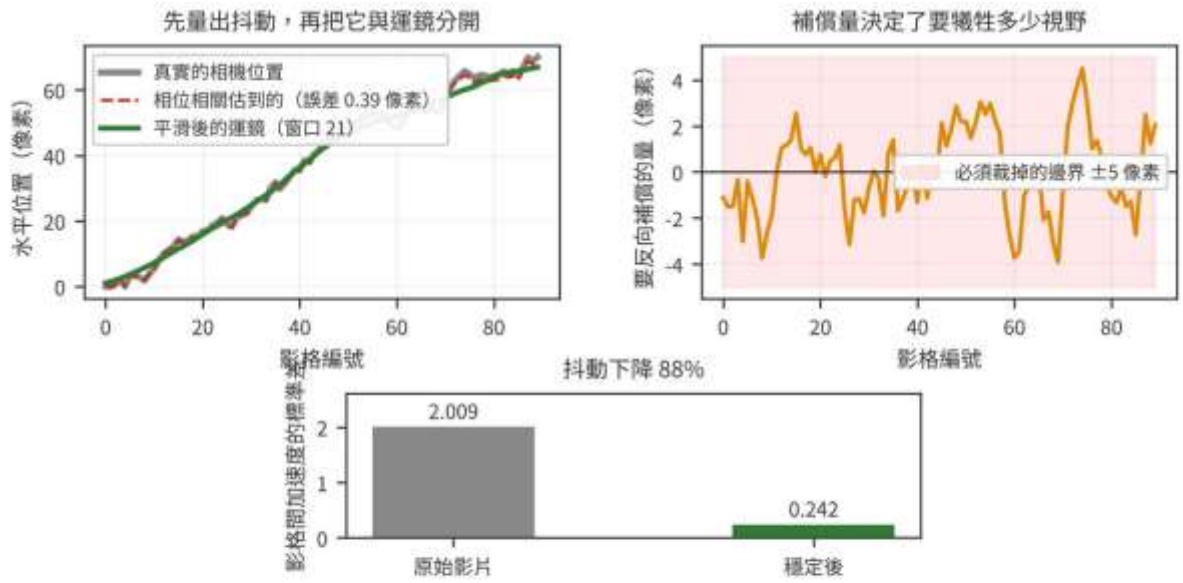
```
    if cv2.waitKey(30) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

12.6 進階主題：穩定、動作辨識與時序深度學習

12.6.1 視訊穩定

手持或車載攝影機的畫面總是晃動的。視訊穩定(Video Stabilization)的原理是：估計相鄰影格之間因抖動造成的整體運動(用第 6.7.2 節的相位相關法，或第 12.4 節的光流，或第 13 章的特徵匹配)，再反向補償這個運動(用第 4 章的幾何轉換把每一影格「拉回」穩定位置)，最後裁掉補償後露出的黑邊。這串起了第 4、6、12、13 章——是本書多項技術的綜合應用。



視訊穩定是本章與前面好幾章的綜合，流程只有三步，但每一步都有講究。第一步「量」：用第 6 章的相位相關估出相鄰影格的整體位移，本例的估計誤差中位數只有 0.39 像素。第二步「分」——這一步最關鍵：不是把所有運動都消掉，而是要把「刻意的運鏡」與「不要的手震」分開。做法是對軌跡做時間上的低通濾波（本圖用窗口 21 的移動平均），平滑後的那條就是攝影師想要的運鏡，兩者的差就是該補償掉的抖動。若不做這一步、直接把位移全部歸零，畫面會變成完全靜止，運鏡也沒了。第三步「補」：對每一影格做反向的幾何轉換（第 4 章），實測影格間加速度的標準差從 2.009 降到 0.242，下降 88%。代價寫在右圖：補償會把畫面推出邊界，所以必須裁掉 ±5 像素的邊框。這就是手機「超級防手震」會讓視野變窄的原因——穩定度與視野是同一條線上的兩端。

圖 12-19　視訊穩定的三步：量、分、補

12.6.2 動作辨識

動作辨識(Action Recognition)要回答「影片中的人在做什麼」(走路、揮手、跌倒)。它與單張影像分類的根本差別在於：動作是時間性的——單看一張「手舉起來」的照片，無法分辨是在打招呼還是要打人，必須看時間序列。傳統方法結合外觀特徵與光流(運動資訊);深度學習方法則用能處理時間維度的網路(如 3D 卷積、雙流網路、或後來的時序 Transformer)。應用包括：照護場景的跌倒偵測、運動分析、人機協作的手勢指令、異常行為監控。

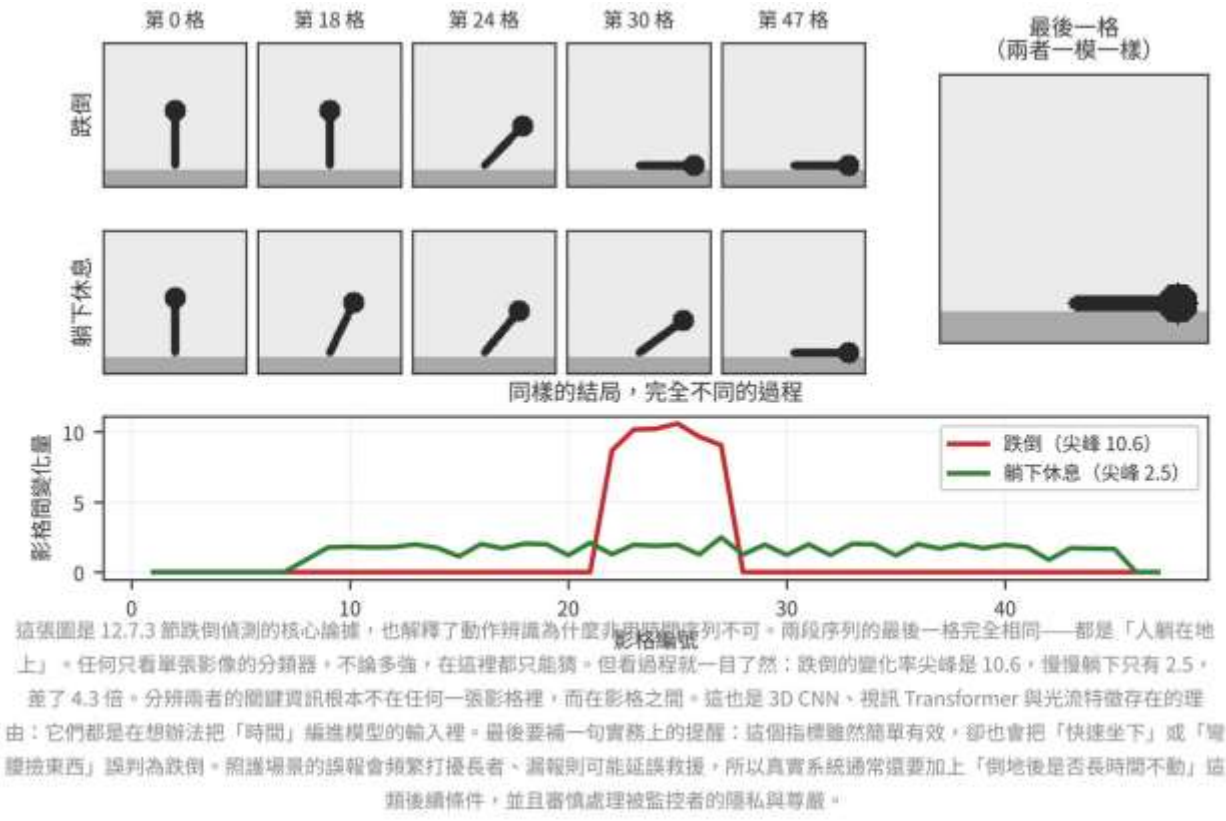


圖 12-20　同樣的結局，完全不同的過程：動作必須看時間序列

12.6.3 時序深度學習與時間一致性

進入深度學習，處理時間維度成為一個核心課題。早期用循環神經網路(RNN/LSTM)處理影格序列;3D CNN 把卷積從二維空間擴展到「空間加時間」的三維;近年的視訊 Transformer 則以注意力機制捕捉長距離的時間關聯。這些方法讓機器不只看懂單一影格，更能理解「事件的發展」。

但無論用什麼方法，視訊深度學習都要面對 12.1.3 節提過的時間一致性挑戰：若對每一影格獨立套用模型，結果會閃爍抖動(這一影格偵測到、下一影格又消失)。解決之道包括：在時間軸上做平滑、用追蹤維持身分、或設計本身就考慮時間關聯的模型。這也是機器人視覺與靜態影像辨識最大的不同——機器人活在連續的時間裡，它需要的不是一張張孤立的判斷，而是對「正在發生什麼」的連貫理解。

12.7 應用案例

12.7.1 案例一：智慧監控與人流計數

情境：在店門口或走廊上方架設攝影機，自動統計進出的人數。流程：背景相減得到移動前景(12.3 節)→ 型態學清理、連通元件框出每個人(第 10 章)→ 用追蹤器維持每個人的 ID(12.5 節)→ 在畫面中畫一條虛擬計數線，當某人的軌跡越過此線，依方向計數(進或出)。關鍵在於追蹤——若只靠偵測，同一個人可能被重複計數;有了穩定的 ID，才能確保「一個人只算一次」。這是零售分析、場館人流管理、防疫人數管制的實用系統。

12.7.2 案例二：車流量與車速偵測

情境：道路監控攝影機自動統計車流量與估計車速。流程：背景相減或第 16 章 YOLO 偵測車輛 → 追蹤每輛車的軌跡 → 由軌跡在連續影格的位移，結合第 4 章相機校正得到的「像素對應實際距離」的比例，以及已知的影格率(位移除以時間即速度)，換算出每輛車的實際速度。這串起了第 4 章(相機校正把像素換算成公尺)、第 12 章(追蹤取得位移)、以及影格率(提供時間基準)——是一個把「像素運動」轉換成「物理速度」的漂亮範例，也是智慧交通、區間測速的基礎。

12.7.3 案例三：照護場景的跌倒偵測

情境：在長者居家或病房中，自動偵測跌倒並發出警示。這是動作辨識(12.6.2 節)的重要應用，也凸顯了「時間維度不可或缺」——單看一張「人躺在地上」的照片，無法分辨是跌倒還是正常躺下休息;必須看時間序列：一個「快速的、由站立到倒地的垂直運動」才是跌倒的特徵。傳統做法可結合人體姿態(第 16 章 YOLO Pose)的重心高度變化與運動速度來判斷;深度學習則用時序模型直接學習跌倒的動作模式。此案例同時觸及第 16 章的姿態估計，展示了視訊、姿態、時序三者的整合。

這裡也值得提醒一個貫穿全書的價值：照護應用直接關係人的安危與尊嚴，系統的誤報(把正常動作誤判為跌倒，頻繁打擾)與漏報(真跌倒卻沒偵測到)都有嚴重後果，且涉及被監控者的隱私。這類系統的設計，技術之外更需要對使用者處境的體貼——

這是工程倫理的一環。

12.7.4 案例四：機器人的視覺里程計

情境：一台自主移動機器人(或無人機)需要知道自己移動了多少、轉了多少，即使沒有 GPS。視覺里程計(Visual Odometry)的核心正是本章的光流與追蹤：機器人前進時，攝影機看到的靜態場景會「反向流動」;透過追蹤場景中的特徵點(第 13 章)、分析它們在連續影格間的運動(12.4 節光流)，就能反推出攝影機自身的運動——這是機器人「自我定位」的視覺基礎，也是第 17 章機器人視覺整合的重要前置技術。這個案例揭示了一個深刻的觀點：同樣是「場景中的運動」，換個角度看，就成了「觀察者自身的運動」——運動是相對的，而機器人正是利用這個相對性來認識自己在世界中的位置。

12.8 本章與全書的觀念連結總表

相關章節	共用的觀念	視訊處理中的角色
第 3 章 數位影像基礎	取樣定理	影格率不足造成時間混疊(車輪倒轉錯覺)
第 4 章 幾何轉換	仿射/透視轉換、相機校正	視訊穩定的運動補償;像素換算實際速度
第 5 章 空間域強化	多影格平均	夜景模式即時間維度的去雜訊
第 6 章 頻率域	相位相關法	視訊穩定的影格間運動估計
第 8 章 色彩處理	HSV、顏色追蹤、虛擬色彩	稠密光流視覺化;CamShift 顏色追蹤
第 9、10 章 分割/二值	門檻、型態學、連通元件	前景遮罩的清理與物件框選
第 11 章 小波	壓縮	視訊編碼進一步利用影格間冗餘

第 13 章 特徵擷取	角點、特徵匹配	Shi-Tomasi 角點供稀疏光流追蹤
第 16 章 物件偵測	YOLO、姿態估計	偵測式追蹤;跌倒偵測結合姿態與時序
第 17 章 邊緣部署	即時性、視覺里程計	視訊處理的算力預算;機器人自我定位

表 12-1 第 12 章與全書其他章節的觀念連結

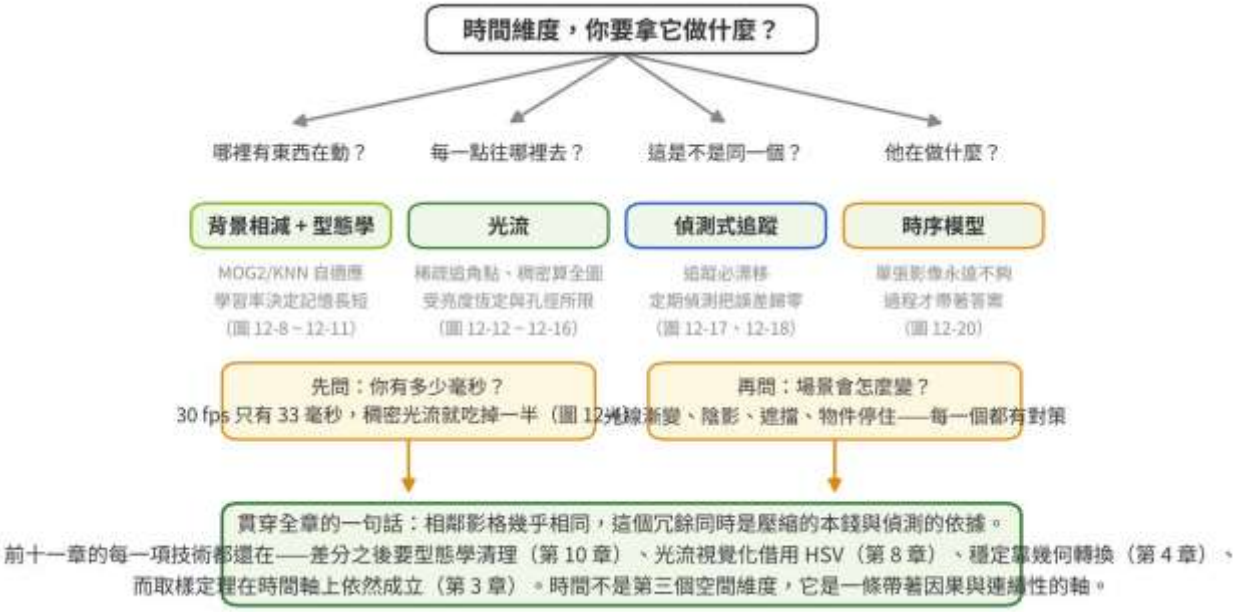


圖 12-21　第 12 章的決策地圖

程式實作 Lab 12

Lab 12-1 視訊處理管線

目標：熟悉視訊讀寫框架。(a)以程式 12-1 為基礎，讀取一段視訊或即時攝影機，對每一影格套用前面各章的至少三種技術(如 CLAHE、Canny、HSV 顏色分割)，並排顯示原始與處理結果;(b)在畫面上即時顯示影格率(用 cv2.getTickCount 計時)，觀察不同處理的耗時;(c)把結果寫成輸出檔，嘗試不同的 fourcc 編解碼器，比較輸出檔的大小與品質;(d)討論題：哪一種處理最耗時?若要在 30 fps 即時執行，每一影格的處理時間預算是多少毫秒?

Lab 12-2 移動偵測監控系統

目標：實作 12.7.1 節的監控系統。(a)以程式 12-2 對一段監控影片做背景相減，比較「影格差分」與「MOG2 背景相減」在偵測完整物件、應對暫停物件、抵抗光線變化上的差異;(b)加入型態學清理與面積門檻，穩定地框出移動物件;(c)畫一條虛擬計數線，實作越線計數(需簡單追蹤以避免重複計數);(d)測試干擾：讓畫面中有搖動的樹葉或漸變的光線，觀察 MOG2 如何自適應;(e)挑戰題：加入陰影偵測(detectShadows)，避免把物體的影子也框進去。

Lab 12-3 光流視覺化

目標：直觀理解光流。(a)以程式 12-3 對一段有明顯運動的影片(如行走的人、車流)，同時顯示稀疏光流(特徵點的運動軌跡)與稠密光流(色彩運動場);(b)解讀稠密光流的色彩圖：不同顏色代表什麼方向?為什麼靜止的背景是黑色?(c)拍攝或找一段「攝影機自己在移動」的影片(如手持前進)，觀察整個畫面的光流呈現什麼模式——這與 12.7.4 節的視覺里程計有何關聯?(d)驗證孔徑問題：對著一條緩慢移動的直線邊緣，觀察光流估計是否只捕捉到垂直於邊緣的分量。

Lab 12-4 物件追蹤與偵測式追蹤

目標：體會偵測與追蹤的分工。(a)以程式 12-4 用 CSRT 追蹤器追蹤一個手選的物件，測試它在物件被短暫遮擋、快速移動、與其他物件交錯時是否跟丟或跟錯;(b)比較 KCF 與 CSRT 在速度與準確度上的差異(量測 FPS);(c)概念實作偵測式追蹤：每隔 N 影格「重新框選」(模擬偵測器校正)，中間影格用追蹤器更新，觀察這樣是否比純追蹤更穩健;(d)討論題：承第 16 章，若把手動框選換成 YOLO 自動偵測，並對多個物件各自維持 ID，你會如何設計?(這正是 DeepSORT 的思路)

Lab 12-5 自己做一段有正確答案的測試序列

目標：學會「造一個有正確答案的資料集」——這是本章最重要的方法論訓練。真實影片沒有標準答案，調參數只能靠眼睛看，但只要序列是自己合成的，每一格的

物件位置與遮罩都已知，就能真的算出誤差。(a)寫一個函式，把一張靜態影像當背景，讓一個方塊、一個圓以已知的速度移動，同時回傳每一格的前景遮罩當正確答案;(b)把雜訊加在最後一步，並讓函式接受一個亂數種子，使得同一個種子配不同的雜訊強度會得到「幾何完全對齊」的乾淨版與含雜訊版（這一點在後面會很有用）;(c)加上三種真實世界的干擾選項：整體亮度緩慢起伏（日照漸變）、某一小塊持續隨機抖動（樹葉搖動）、物件底下跟著一塊變暗的區域（陰影）;(d)再加一個選項讓物件在指定影格停住不動;(e)用你的序列重做圖 12-7 的門檻掃描：用乾淨版算出「真的改變了的像素」當正確答案，掃描門檻並畫出召回率、精確率與 F1，找出最佳門檻;(f)把雜訊強度從 σ=1 掃到 σ=12，畫出最佳門檻對 σ 的關係，並與理論值 3√2σ 比較。你得到的係數是多少?

Lab 12-6 偵測式追蹤與遮擋

目標：親手量出追蹤的漂移，並體會「讓追蹤器承認自己不確定」的威力。(a)沿用 Lab 12-5 的序列產生器，但把目標換成有紋理的影像區塊（例如從另一張照片裁一小塊）。想一想：為什麼顏色均勻的方塊不適合用來測試追蹤器?（提示：回頭看 12.4.4 節的孔徑問題）;(b)用 cv2.matchTemplate 寫一個最小的樣板追蹤器：每一格在上一格位置附近搜尋，找到後更新樣板。讓目標同時做次像素位移與緩慢自轉，再畫出 IoU 對影格的曲線，記錄它在第幾格跌破 0.5;(c)加入「每 k 格用正確答案重新定位一次」模擬偵測式追蹤，對 k = 10、30 各畫一條曲線，比較平均 IoU。你應該會看到鋸齒狀的曲線——解釋每一個鋸齒的形狀;(d)估算：若你的偵測器要 40 毫秒、追蹤器要 2 毫秒，在 30 fps 的預算下 k 最小可以設到多少?(e)讓目標在中途被一根柱子完全擋住 25 格。先用原本的追蹤器跑一次，觀察遮擋結束後能不能接回來;(f)改成兩道門檻：分數夠高才相信位置、分數非常高才更新樣板，兩者都不滿足時用先前估到的速度外推。比較兩者在遮擋後的平均 IoU，並說明外推能撐多久、為什麼會慢慢失準;(g)選作：把等速外推換成卡爾曼濾波器（cv2.KalmanFilter），觀察它在遮擋期間如何同時給出預測位置與不確定度。

本章重點整理

- 視訊是以固定影格率排列的連續影格，多了「時間」維度;視訊處理的基本框架是逐影格讀取、當成影像處理、逐影格輸出的迴圈。時間維度帶來偵測移動、追蹤、估計速度、多影格提質的機會，也帶來即時性、時間一致性、資料量三大挑戰。
- 移動偵測基於「背景不變、前景變化」：影格差分極簡但只偵測到移動邊緣、暫停物件會消失;背景相減能偵測完整物件，MOG2/KNN 等自適應方法可應對光線漸變與搖動干擾，但自適應是雙面刃——物件只要停得夠久就會被學進背景而消失，撐多久由學習率決定（12.3.3 節、圖 12-11）。
- 光流估計每個點的運動向量，基於亮度恆定假設，受孔徑問題限制;Lucas-Kanade 為稀疏光流(追蹤角點),Farnebäck 為稠密光流(每像素運動場，常以虛擬色彩視覺化)。
- 偵測是單一影格找物件、追蹤是跨影格維持身分;偵測式追蹤(SORT/DeepSORT)結合兩者——定期偵測校正、中間追蹤維持 ID;傳統追蹤器有 MeanShift/CamShift(顏色)、KCF/CSRT(相關濾波)、卡爾曼濾波(運動預測)。
- 視訊穩定綜合光流/特徵匹配估計抖動、以幾何轉換反向補償;動作辨識需時間序列(單張影像無法分辨動作);時序深度學習(3D CNN、視訊 Transformer)須處理時間一致性以免結果閃爍。
- 視訊處理串起全書：第 3 章取樣(時間混疊)、第 4 章幾何(穩定與速度換算)、第 8 章色彩(光流視覺化)、第 9/10 章(前景清理)、第 13 章(角點追蹤)、第 16 章(偵測式追蹤、跌倒偵測)、第 17 章(視覺里程計)。

習題

1. 視訊相較於單張影像，多了什麼維度?這個維度帶來哪些機會與哪些挑戰?

提示與參考解答：多了「時間」維度。機會：相鄰影格高度關聯，可偵測移動(影格相減)、追蹤物件、估計速度、多影格提質去雜訊。挑戰：即時性(須跟上每秒數十影格)、時間一致性(結果不可影格間抖動閃爍)、資料量(數十倍於影像，仰賴壓縮與影格間編碼)。

2. 承叫獸開講的思考題：用影格率不足的相機拍攝高速旋轉的車輪，會看到車輪倒轉或靜止的錯覺，為什麼?這與第 3、6 章的什麼概念相關?

提示與參考解答：這是時間維度的混疊(時間 aliasing)。車輪旋轉是一個高頻的週期運動，影格率(取樣頻率)不足以滿足 Nyquist 定理時，取樣點恰好錯過真實運動，大腦把離散樣本腦補成較低頻的運動(倒轉或靜止)。這與第 3 章取樣定理、第 6 章混疊(高頻摺疊成低頻)同源，只是發生在時間軸上。

3. 視訊處理的基本框架是什麼?請說明 VideoCapture、逐影格迴圈、VideoWriter 三者的角色，以及 fourcc 與 release 的重要性。

提示與參考解答：框架是一個迴圈：VideoCapture 逐影格讀取來源(檔案/攝影機/串流)→ 每一影格當成影像套用任何影像處理技術 → VideoWriter 逐影格寫出。fourcc 指定輸出的編解碼器(決定壓縮方式，選錯無法寫出),release 釋放攝影機/檔案資源以免被鎖住。cap.read 的成功旗標用於判斷視訊是否讀完。

4. 影格差分與背景相減有何差異?各自的優缺點是什麼?為什麼影格差分會產生「空心」的移動物件?

提示與參考解答：影格差分：相鄰兩影格相減，極簡極快、自動適應光線，但只偵測到移動的邊緣，暫停物件會消失，且物件內部顏色均勻處前後影格相同、相減為零，故只留輪廓(空心/鬼影)。背景相減：與一個學習得到的背景模型相減，能偵測完整物件，且在物件短暫停住時仍看得見(這點勝過影格差分)，但需自適應維護背景模型以應對光線與搖動干擾(MOG2/KNN)。要注意「暫停物件不會消失」只對了一半：自適應模型會把長期靜止的東西學成背景，實測在學習率 0.05 時物件停住 2 格就被吸收、0.001 時可撐 102 格——撐多久完全由學習率決定(12.3.3 節、圖 12-11)。

5. 為什麼優秀的背景相減器必須是「自適應」的?它要應對哪些真實場景的干擾?

提示與參考解答：因為真實背景並非一成不變：日照移動使光線漸變、樹葉搖動、水面波動、攝影機輕微震動。自適應背景相減器持續緩慢更新背景模型，把「長期

穩定存在」的學為背景、「突然出現快速移動」的判為前景，並可偵測陰影，才能在這些干擾下正確運作。

6. 光流的核心假設是什麼?什麼是孔徑問題?它為什麼使單一像素的運動無法唯一決定?

提示與參考解答：核心假設是亮度恆定：同一物體點在相鄰影格間亮度不變、只是位置移動。孔徑問題：透過小孔(局部)觀察移動的直線邊緣，只能看出垂直於邊緣方向的運動分量，沿邊緣方向的運動無從察覺。故單看一個像素的局部資訊無法唯一決定運動，須引入額外假設(如鄰域運動相似)才能求解。

7. 稀疏光流與稠密光流有何差別?Lucas-Kanade 為什麼要選角點來追蹤?

提示與參考解答：稀疏光流(Lucas-Kanade)只追蹤少數適合追蹤的特徵點，高效精準;稠密光流(Farnebäck)為每個像素計算運動向量，資訊豐富但運算量大，常以虛擬色彩視覺化。Lucas-Kanade 選角點(Shi-Tomasi)是因為角點在兩個方向都有明顯灰階變化，能繞過孔徑問題、運動可唯一決定。

8. 「偵測」與「追蹤」有何根本差別?為什麼「偵測式追蹤」結合兩者會更強健?

提示與參考解答：偵測是在單一影格找出物件位置、影格間獨立無關;追蹤是跨影格維持同一物件的身分(ID)並串成軌跡。偵測準但耗算力、不知影格間是否同一物件;追蹤快且維持身分但可能跟丟或跟錯。偵測式追蹤(SORT/DeepSORT)定期用偵測器校正並發現新物件，中間影格用輕量追蹤維持 ID，兼得準確、效率與身分一致性。

9. 為什麼動作辨識(如跌倒偵測)必須用時間序列，而不能只看單張影像?

提示與參考解答：動作是時間性的：單看一張「人躺在地上」的照片無法分辨是跌倒還是正常躺下休息;單看「手舉起」無法分辨是打招呼還是攻擊。必須看時間序列——例如「快速的、由站立到倒地的垂直運動」才是跌倒特徵。故需結合光流/姿態的時間變化，或用 3D CNN、視訊 Transformer 等時序模型。

10. 什麼是視訊深度學習的「時間一致性」問題?若對每一影格獨立套用模型會發生什麼?如何緩解?

提示與參考解答：時間一致性指處理結果在影格與影格之間應穩定連貫。若每一影格獨立套用模型，結果會閃爍抖動(偵測框跳動、物件時有時無、ID 跳號)。緩解：在時間軸上平滑結果、用追蹤維持身分、或設計本身考慮時間關聯的模型(3D

CNN、時序 Transformer)。這是機器人視覺與靜態辨識最大的不同。

11. 視訊穩定的原理是什麼?它綜合了本書哪幾章的技術?

    提示與參考解答：原理：估計相鄰影格因抖動造成的整體運動，再反向補償、拉回穩定位置，最後裁掉黑邊。綜合：第 6 章相位相關法或第 12 章光流或第 13 章特徵匹配(估計運動)、第 4 章幾何轉換(反向補償)、第 12 章視訊框架。是多章技術的整合應用。

12. (綜合題)請設計一個「道路車速偵測」系統，說明如何從影片算出每輛車的實際速度(km/h)，並指出用到本書哪幾章的技術。

    提示與參考解答：流程：一、偵測車輛(背景相減或第 16 章 YOLO);二、追蹤每輛車的軌跡並維持 ID(第 12 章);三、量測車在連續影格間的像素位移;四、用第 4 章相機校正得到「每像素對應的實際距離(公尺)」把像素位移換算成實際位移;五、除以時間(位移影格數除以影格率得秒數)得速度，再換算 km/h。用到第 4 章(校正/像素換算)、第 12 章(偵測、追蹤、影格率提供時間基準)、第 16 章(YOLO 偵測)。

13. 把一段視訊沿著某一列切開，得到一張以 x 為橫軸、t 為縱軸的切片圖。靜止的背景與移動的物件在這張圖上各自長什麼樣子?若某條斜線在 100 格之內橫跨了 200 欄，該物件的速度是多少?

    提示與參考解答：靜止的背景在時間方向完全不變，畫出來是垂直的條紋;移動的物件則拉出一條斜線。斜率就是速度：200 欄 ÷ 100 格 = 2 像素 / 影格。圖 12-1 用這個方法量出來的速度與合成設定值完全吻合。這張切片圖值得記住，因為本章幾乎每一件事都是在讀這些斜線：移動偵測是找出哪裡不是垂直的，光流是量每一點的斜率，追蹤則是把同一條斜線從頭跟到尾。

14. 要說「相鄰影格幾乎相同」，該用什麼基準來比?若兩張影格各自帶著標準差 σ 的雜訊，相減之後雜訊的標準差是多少?再把單張影格與影格差分各自算熵，為什麼差分的熵低得多?這與 H.264、H.265 這類視訊編碼的設計有什麼關係?

    提示與參考解答：兩張獨立含雜訊的影格相減，雜訊的變異數相加，所以標準差變成 √2σ。取三倍即 3√2σ ≈ 4.24σ 作為「真的變了」的底線。σ=3 時這個底線是 12.7，而不是 3——若用 3 當判準，會把大量純粹的雜訊跳動當成真實變化。圖 12-2 實測在這條底線之下的像素佔 99.2%，剩下的幾乎都集中在移動物件的邊緣。一

般化的教訓：任何「有沒有變化」的判斷，門檻都必須從雜訊水準推出來，而不是寫死一個數字。至於資訊量：差值幾乎全部擠在零附近（分布極尖），而單張影格的灰階分布平坦得多，所以差分的熵低很多。圖 12-2 實測：單張影格 7.30 位元／像素，影格差分 4.19 位元／像素。用 JPEG 實際存一次，單張 17.6 KB、只存差分 5.9 KB，省下 67%。這正是視訊編碼的立足點：它們不逐張儲存影像，而是存少數關鍵影格（I 影格）加上大量的「與預測的差異」（P、B 影格）。注意反過來看的另一面——既然絕大部分內容都不變，那麼「變了的地方」就幾乎等於「有東西在動的地方」，這就是移動偵測的全部道理。壓縮與偵測，用的是同一個事實。

15. 30 fps 的視訊，每一影格的處理預算是多少毫秒?若某演算法在 320×240 上要 15 毫秒，換到 1920×1080 大約要多久?這對演算法選擇有什麼影響?

    提示與參考解答：30 fps 的預算是 1000/30 ≈ 33.3 毫秒，而且這還得包含讀取、解碼、顯示與寫出。耗時大致隨像素數成正比：(1920×1080)/(320×240) = 27 倍，所以約 15 × 27 ≈ 405 毫秒，遠超預算。圖 12-4 的實測顯示大部分前處理（灰階、模糊、CLAHE、Canny、型態學）都在 1 毫秒以內，但稠密光流是另一個量級。這解釋了為什麼實務上「稀疏光流 + 偶爾偵測」遠比「每一影格都算稠密光流」常見。實務上的對策還包括：縮小解析度處理、只在感興趣區域處理、或隔幾格才算一次。

16. MJPG 與 mp4v 這兩種編碼的根本差別是什麼?各自適合什麼場合?若要把影片當作法律證據保存，該選哪一種?

    提示與參考解答：MJPG 把每一影格各自存成 JPEG，完全不利用時間冗餘;mp4v 會做影格間預測，只記錄影格之間的差異。圖 12-5 實測同樣 90 張影格：MJPG 1582 KB、mp4v 744 KB，不到一半，而品質只差 1.5 dB。但 MJPG 的每一影格都能獨立解碼，隨機存取與逐格剪輯都方便，所以監控與工業相機常用它;mp4v 則必須從關鍵影格開始解。法律證據應選無失真編碼（如 FFV1）：圖 12-5 實測它仍能壓縮 4.6 倍，而且任何一個像素的誤差都是 0。注意「最大誤差」這個指標比平均 PSNR 更能看出失真的性格——這是第 11 章「失真的型態比失真的量更重要」那一課的延伸。

17. 設計一個實驗來決定影格差分的門檻。正確答案該怎麼定義才不會把「空心」的問題混進來?實測顯示最佳門檻與雜訊 σ 是什麼關係?

    提示與參考解答：關鍵在把兩件事分開。若用「完整的物件遮罩」當正確答案，量

到的其實是空心問題（召回率上限只有 6%），不是門檻的問題。正確做法是用同一組合成參數、不加雜訊再跑一次，以「乾淨序列中真的改變了的像素」當正確答案，這樣量到的才是門檻真正在做的事：把真實變化與雜訊分開。圖 12-7 這樣做之後，σ=3 時最佳門檻是 20，F1 達到 99.7%;把 σ 從 1 掃到 12，最佳門檻從 8 升到 63，擬合出約 4.9σ，與理論的 3√2σ ≈ 4.24σ 同一個量級。可操作的規則：取一塊靜止區域量出 σ，門檻設為約四倍。

18. 背景相減的輸出為什麼幾乎不可能直接使用?型態學的開運算與閉運算該用什麼順序、結構元素該多大?請說明結構元素過大時會發生什麼事。

提示與參考解答：輸出會布滿雜點、物件內部有破洞。順序是先開運算（侵蝕再膨脹）去掉比結構元素小的雜點，再閉運算（膨脹再侵蝕）補起內部破洞。圖 12-9 在 σ=14 的高雜訊下實測：未清理的 IoU 只有 0.410，開運算後 0.929，再閉運算 0.953。更具體的理由是連通元件數——未清理時多達 2725 個，而真實物件只有 3 個，也就是說若不清理就直接數物件，會得到荒謬的結果。但結構元素過大時 IoU 反而下降（邊長 13 時只剩 0.345），因為小的物件連同雜點一起被侵蝕掉了。結構元素的大小應由「最小的目標物件」決定，而不是由雜訊決定。

19. MOG2 的 detectShadows 是怎麼判斷陰影的?它的輸出有幾種值、程式裡若寫成 mask > 0 會發生什麼事?另外，MOG2 的背景更新大致是 B ← (1−α)·B + α·I，請說明模型的「記憶長度」約為多少影格，以及為什麼同一個 α 同時決定了「學多快」與「忘多快」。若要偵測「被遺留在月台上的包裹」，α 該怎麼設?

提示與參考解答：判斷依據是「陰影只讓像素變暗，但照到的還是同一塊地面，所以紋理與色度都沒變、只有亮度等比例下降」。因此檢查「目前像素值除以背景模型像素值」是否落在固定區間（例如 0.5~0.95）且色度沒有明顯偏移，就能標成陰影。輸出是三態：0 背景、255 前景、127 陰影。若寫成 mask > 0，127 會被當成前景，這個功能等於沒開。正確寫法是 mask == 255。圖 12-10 實測：關閉時 IoU 0.847、陰影區有 98% 被誤判;開啟並正確濾掉 127 之後 IoU 升到 0.962、陰影誤判降到 1%。詳見 12.3.3 節。至於學習率：把遞迴式展開，n 格之前的影格貢獻 $\alpha(1-\alpha)^n$ 的權重，所以記憶長度約為 1/α 格：α=0.01 記得住百來格，α=0.05 只記得住二十格左右。學得快與忘得快是同一個 α 的兩面，無法分開調。圖 12-11 實測：α=0.05 時暖身只要 6 格，但物件停住後 2 格就被吃掉;α=0.001 時物件可以撐

102 格，代價是背景模型到第 170 格都還沒學乾淨。遺留物偵測需要極小的 α，而更好的做法是同時維護一個長期與一個短期模型，取「短期認為是背景、長期認為是前景」的區域——那就是「最近才出現並停住」的東西。

20. 光流的亮度恆定假設被破壞時，演算法會報錯嗎?乘性亮度變化（如光圈改變）與加性亮度變化（如曝光補償）哪一個傷害比較大?請從光流方程式解釋，並說明實務上的對策。

提示與參考解答：不會報錯，只會安靜地給出錯誤答案——這比報錯更危險。圖 12-12 實測顯示加性變化的傷害遠大於乘性：亮度乘以 1.6 時誤差只有 0.70 像素，但整體加 60 階時誤差高達 21.85 像素。從 $I_x \cdot u + I_y \cdot v + I_t = 0$ 看：整體加一個常數，空間梯度 I_x、I_y 完全不變，卻在每一點都塞進一個假的時間變化 I_t，解出來的速度因此被整個推歪;乘性變化則同時放大了空間梯度與時間變化，兩者在相除時大部分抵消掉了。對策：關掉相機的自動曝光與自動白平衡，或在算光流之前先做亮度正規化（第 5 章的直方圖等化或 CLAHE）。

21. 請寫出光流方程式，並用它說明孔徑問題為什麼不是「演算法不夠好」。Lucas-Kanade 用什麼假設補上缺少的那條方程式，這個補救與第 13 章的 Harris 角點有什麼關係?又，為什麼把窗口從 15 放大到 31 幾乎無助於追蹤更大的位移，金字塔卻有效?

提示與參考解答：由亮度恆定假設做一階泰勒展開得 $I_x \cdot u + I_y \cdot v + I_t = 0$——一條方程式、兩個未知數，所以單一像素只能定出梯度方向（邊緣法線）的分量，沿邊緣的分量完全沒有被約束。這是資訊本身不足，不是演算法的缺陷。圖 12-13 的實測：改變窗口大小得到四組完全不同的位移，但投影到條紋法線方向後全部一致，因為這些解產生的畫面完全相同。Lucas-Kanade 補上的假設是「小窗口內所有像素運動相同」，於是 N 個像素給 N 條方程式，用最小平方法解一個 2×2 方程組。那個 2×2 矩陣正是 Harris 用的結構張量，它可逆的條件就是「兩個特徵值都夠大」，也就是角點的定義。所以「挑角點來追蹤」不是經驗法則，而是方程組有唯一解的必要條件。詳見 12.4.4 節。至於金字塔：限制來自泰勒展開的一階近似，它的有效範圍由影像本身的紋理尺度決定，把窗口開大只是納入更多同樣失效的方程式。圖 12-14 實測：窗口 15 與窗口 31 的可追蹤位移都是 5 像素，完全沒有改善。金字塔的做法是先把兩張影格縮小 2^n 倍，在最粗的那一層上原本 80 像素的位移看起來只剩幾個像素，線性近似重新成立;估出粗略位移後放大一倍當作下一層的起點，逐層

只需估一個小修正量，等於把「大位移」化約成一連串「小位移」。實測：加上 3 層金字塔後可追位移從 5 跳到 87 像素，5 層超過 160 像素。這正是第 11 章多解析度分析的再一次勝利。實務推論：目標移動很快時，與其調大窗口，不如增加 maxLevel 或提高影格率。

22. 稠密光流的運動場常用 HSV 視覺化。請說明色相與亮度各編碼什麼，並描述「相機往左平移」「繞光軸旋轉」「往前推進」三種情況各會呈現什麼圖樣。什麼是擴張焦點?它有什麼用?

提示與參考解答：色相編碼運動方向、亮度編碼運動大小（第 8 章的 HSV）。往左平移：整張圖是單一顏色（所有像素同方向同速）。繞光軸旋轉：顏色繞著中心轉一圈，中心最暗（轉軸上不動）、邊角最亮（半徑最大、線速度最快）。往前推進：顏色從中心向外放射，中心那個不動的點就是擴張焦點（Focus of Expansion），它指出相機正在往哪裡前進。擴張焦點是視覺里程計與障礙閃避的關鍵線索，也是 12.7.4 節機器人案例的核心。一般化的讀法：均勻色塊代表平移、漩渦代表旋轉、放射狀代表前進或後退。

23. 追蹤器為什麼一定會漂移?若漂移速率是每格 d 像素、可容忍偏移是 D 像素，偵測的校正週期 k 該怎麼估?請說明這個旋鈕連接了哪兩件事。

提示與參考解答：追蹤的每一格都以上一格為起點，所以每格引入的誤差不會被修正，只會被帶進下一格累積。若誤差是獨立隨機的，n 格後約累積 $\sigma\sqrt{n}$;若有系統性偏向（例如樣板每格都多學了一點背景），則是 $n\cdot\sigma$ 的線性累積。校正週期應滿足 $k \le D/d$。圖 12-17 實測：純追蹤第 39 格就跌破 IoU 0.5，末格只剩 0.30;每 30 格校正一次平均 0.79，每 10 格校正一次 0.92。成本面：平均每格運算量約為「偵測成本 ÷ k + 追蹤成本」。所以 k 把準確度與算力接在一起——這也是圖 12-4 的耗時實測要放在前面的原因。詳見 12.5.3 節。

24. 追蹤器在遮擋時失敗的典型路徑是什麼?為什麼說問題不在「找不到」而在「找錯了還學起來」?請提出一個最小的補救方案並說明它的極限。

提示與參考解答：遮擋時比對分數最高的位置往往落在遮擋物上，若追蹤器照例把那裡更新成新樣板，它就改追遮擋物了，遮擋結束後再也接不回來。圖 12-18 實測：純樣板追蹤在遮擋結束後的平均 IoU 只剩 0.09。最小補救是設兩道門檻——分數夠高才相信位置，分數非常高才更新樣板;兩者都不滿足時不更新任何東西，改用

先前估到的速度外推，靠慣性撐過遮擋。實測這讓遮擋後的平均 IoU 從 0.09 回到 0.57，約七倍。極限是：外推 25 格之後速度估計的誤差已經累積了好幾個像素，所以還沒完全對準。實務上要再補兩件事——卡爾曼濾波器（維護預測的不確定度，遮擋愈久搜尋範圍愈寬）與外觀再識別向量（DeepSORT 的 re-ID，確認重新出現的是不是原來那一個）。

25. (綜合題)為什麼跌倒偵測不能只看單張影像?請設計一個最簡單的時序指標來分辨「跌倒」與「慢慢躺下」，說明它會在什麼情況下誤判，以及照護場景還要補上什麼考量。

    提示與參考解答：兩者的最後一格完全相同——都是「人躺在地上」，任何只看單張影像的分類器在這裡都只能猜。分辨的資訊不在任何一張影格裡，而在影格之間。最簡單的指標是影格間變化量（或身體主軸角度的變化率）：圖 12-20 實測跌倒的變化率尖峰是 10.6，慢慢躺下只有 2.5，差了 4.3 倍。誤判情況：「快速坐下」「彎腰撿東西」「突然低頭」都可能產生高變化率。所以真實系統還要加上後續條件，例如「倒地後是否長時間不動」、身體與地面的角度、以及事後的姿態維持時間。照護場景的額外考量：誤報會頻繁打擾長者、漏報可能延誤救援，兩者後果都嚴重;且涉及被監控者的隱私與尊嚴，應審慎設計（例如只在本地運算、不上傳影像、或改用不成像的感測器）。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018.(相關章節)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 9(視訊處理與運動)。
- Muybridge, E., The Horse in Motion (Sallie Gardner at a Gallop), 1878, Library of Congress.(本章叫獸開講之史料來源)
- Lucas, B. D., & Kanade, T., An Iterative Image Registration Technique with an Application to Stereo Vision, IJCAI, 1981.

- Farnebäck, G., Two-Frame Motion Estimation Based on Polynomial Expansion, SCIA, 2003.
- Bewley, A., et al., Simple Online and Realtime Tracking (SORT), ICIP, 2016;Wojke, N., et al., DeepSORT, ICIP, 2017.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 13 章 特徵擷取

本章學習目標

- 說明「特徵」在電腦視覺中的意義，理解好的特徵應具備重複性、獨特性與不變性。
- 推導 Harris 與 Shi-Tomasi 角點偵測，說明角點為何是良好的追蹤與匹配基準。
- 闡述 SIFT 的尺度不變原理與 ORB 的工程取捨，說明兩者的授權與效能差異。
- 運用 BF 與 FLANN 進行特徵匹配，並以比值測試與 RANSAC 去除錯誤匹配。
- 完成影像拼接、模板比對與物件定位，並實作車牌偵測的特徵前處理。
- 運用 HOG、LBP 與形狀特徵描述整塊區域與物件外形，並說明三類手工特徵的分工與選用時機。
- 理解手工特徵與深度學習特徵的關係，說明本章在傳統與現代視覺之間的橋梁地位。

叫獸開講|SIFT 的二十年專利長跑：一個演算法的解禁之日

2020 年 3 月，全球電腦視覺社群悄悄迎來一個值得慶祝的日子。開源函式庫

OpenCV 的開發者們，終於可以名正言順地把一個名叫 SIFT 的演算法，從那個帶著警示意味的「非自由」(non-free)模組，搬進人人可自由使用的主庫。這一天，許多工程師等了將近二十年。

故事要從 1999 年說起。加拿大英屬哥倫比亞大學的 David Lowe 教授，發表了尺度不變特徵轉換(Scale-Invariant Feature Transform，簡稱 SIFT)。這是一個劃時代的成就：在 SIFT 之前，電腦視覺一直被一個根本難題困擾——當同一個物體以不同的大小、不同的角度、不同的光線出現時，電腦很難認出「這是同一個東西」。SIFT 優雅地解決了這個問題，它能在影像中找出一些獨特的「關鍵點」，而這些關鍵點對縮放、旋轉都保持不變，對光線變化與雜訊也相當強健——就像替影像中的每個特徵蓋上一枚無論如何變形都認得出來的「指紋」。SIFT 迅速成為物件辨識、影像拼接、3D 重建的基石，是深度學習興起前最重要的視覺演算法之一。

但 SIFT 有一道緊箍咒：它被英屬哥倫比亞大學申請了專利(美國專利 6,711,293,1999 年 3 月提出臨時申請，2004 年正式授予)。這意味著在商業產品中使用 SIFT 需要授權。因此，儘管 OpenCV 收錄了 SIFT 的實作，卻只能把它放進一個特別的「非自由」模組，並附上明確警示：使用前請自行確認專利與授權問題。全球工程師只能一邊讚嘆它的威力，一邊小心翼翼地繞道而行。這道緊箍咒也催生了替代品：2011 年，OpenCV 團隊自己提出了 ORB(Oriented FAST and Rotated BRIEF)，一個完全免費、速度更快、專為「沒有專利負擔」而設計的方案——工程界對專利限制的回應，往往就是造一個自由的輪子。

專利的保護期是二十年。2020 年 3 月，SIFT 的專利終於到期。消息傳開，社群一片歡騰，OpenCV 隨即在 4.4.0(以及 3.4.11)版本中，把 SIFT 正式移入自由的主庫。一個沉睡了二十年的經典，終於解除封印，回到每一位工程師都能自由取用的工具箱中。

這個故事給了我們兩個超越技術的啟示。其一，技術選型不只看效能，還要

看授權：在 SIFT 被專利綁住的年代，免費的 ORB 儘管精度稍遜，卻因為「可以自由商用」而被廣泛採用——工程決策從來不是在真空中做的。其二，好的基礎研究會持續發光：即使深度學習已經興起，SIFT 所奠定的「尺度不變、局部特徵」思想，依然深深影響著今天的視覺演算法。這一章，我們就來認識這些為影像蓋「指紋」的經典方法。

思考題：在 SIFT 被專利保護的二十年間，ORB 這個「免費替代品」蓬勃發展。你能想到其他領域中，因為某個技術被專利或授權限制，而催生出自由替代方案的例子嗎?(提示：作業系統、資料庫、影音編碼)

13.1 什麼是好的特徵

13.1.1 從像素到特徵：再一次的抽象躍升

回顧全書的旅程：第 1 到 8 章我們處理「像素」，第 9、10 章我們把像素聚成「區域與物件」，而本章要再往上一階——從影像中萃取出精簡而有代表性的「特徵」(Feature)。所謂特徵，是影像中一小塊「值得注意、容易辨認、可重複找到」的局部結構。與其比對成千上萬個像素，不如只比對數十個關鍵特徵——這讓影像的比對、辨識、對齊變得高效而強健。

這個抽象躍升的價值，可以用一個生活比喻理解：要向朋友描述一個人，你不會逐一報告他臉上每個毛孔的位置與顏色(像素層級)，而是說「他有濃眉、鷹勾鼻、左臉有顆痣」(特徵層級)——用少數幾個獨特的標記，就足以精準辨認。電腦視覺的特徵擷取，做的正是同一件事：為影像找出它最具辨識度的「標記」。

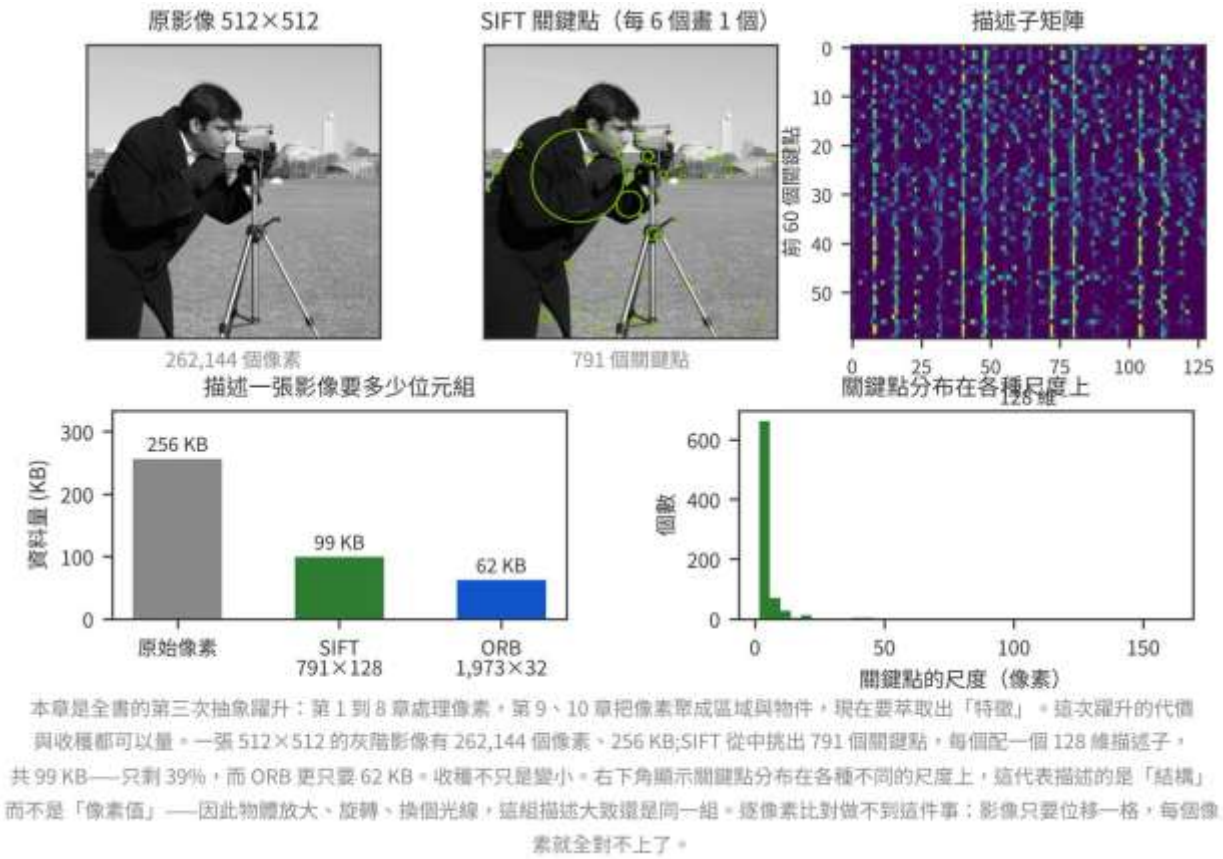


圖 13-1　從像素到特徵：資料量少了，能做的事卻多了

13.1.2 好特徵的三個條件

並非影像中任何一點都適合當特徵。一個理想的特徵應具備三個性質：

1. 重複性(Repeatability)：同一個場景在不同條件下(不同視角、光線、雜訊)拍攝，同一個特徵都應該能被再次偵測到。若一個特徵這次找得到、下次就消失，它就無法用於比對。
2. 獨特性(Distinctiveness)：特徵應該與眾不同、容易與其他特徵區分。一片平坦的牆面、一段筆直的邊緣都不夠獨特(牆上每一點都長得一樣，邊緣上的點沿邊緣滑動也分不出來)，而角點、紋理豐富的斑塊則獨一無二。
3. 不變性(Invariance)：特徵的描述應該對各種變化保持穩定——尺度不變(物體放大縮小仍認得)、旋轉不變(物體轉了角度仍認得)、光照不變(明暗改變仍認得)。不變性愈強，特徵愈能在惡劣條件下發揮作用。

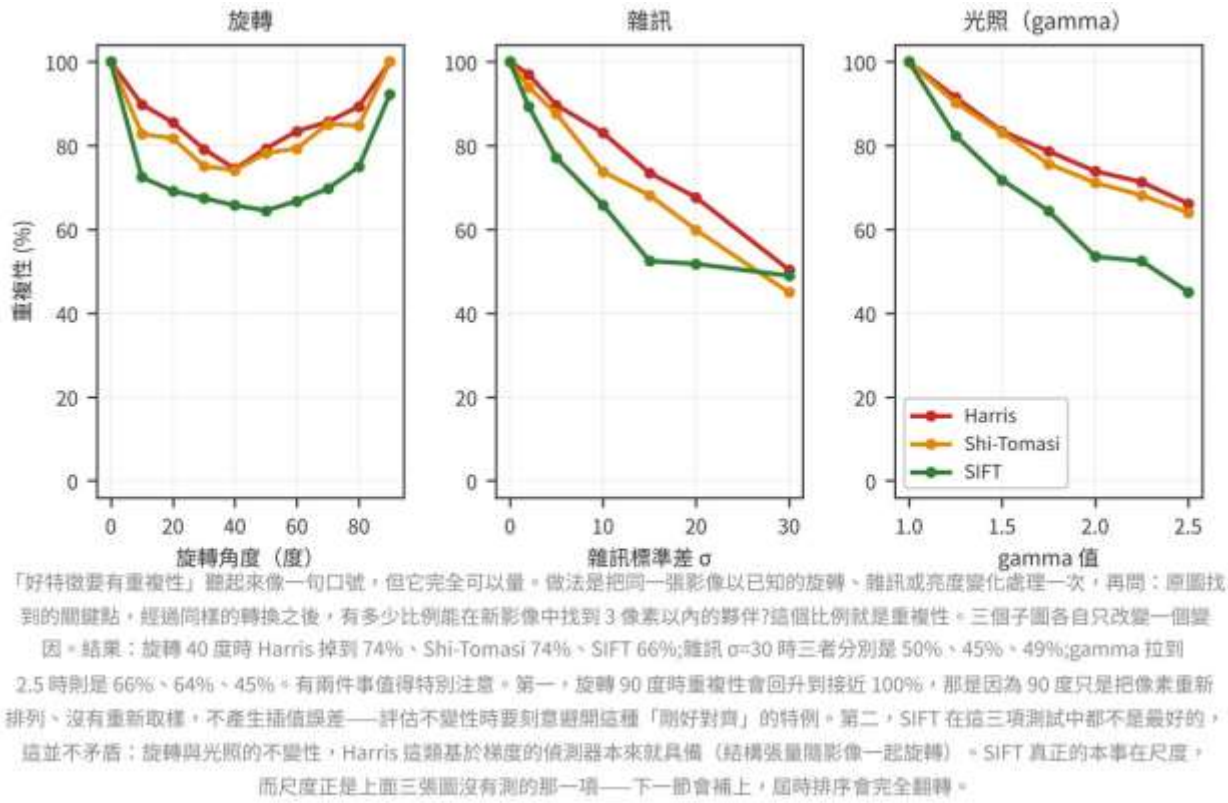


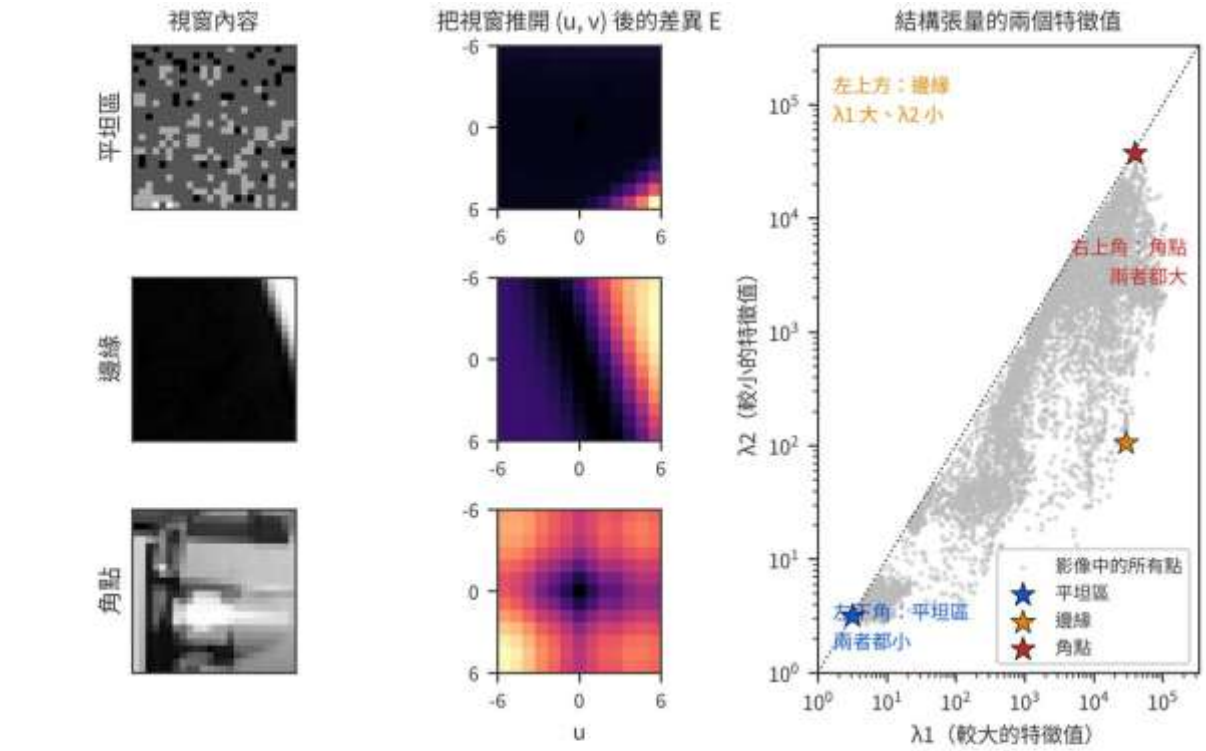
圖 13-2　重複性是可以量的：旋轉、雜訊、光照三項實測

這三個條件也解釋了為什麼「角點」是特徵偵測的寵兒。承接第 12 章的孔徑問題：平坦區域在任何方向移動都看不出變化(無法定位);邊緣只能定出垂直於邊緣的方向(一維模糊);唯有角點在兩個方向都有明顯變化，能被唯一地定位——它同時滿足重複性與獨特性。這就是下一節 Harris 角點偵測的出發點。

13.2 角點偵測：Harris 與 Shi-Tomasi

13.2.1 Harris 角點的直覺

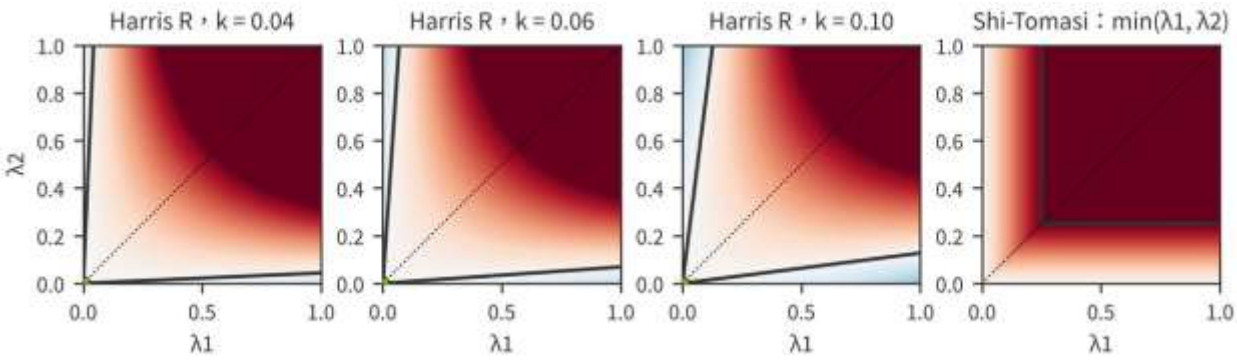
Harris 角點偵測(1988 年由 Chris Harris 與 Mike Stephens 提出)的核心直覺非常優雅：拿一個小視窗在影像上滑動，觀察視窗內的灰階如何變化。有三種情況——若視窗在平坦區域移動，無論往哪個方向，視窗內容都幾乎不變;若視窗沿著邊緣移動(平行於邊緣)，內容不變，但垂直於邊緣移動則劇烈變化;若視窗位於角點，則往任何方向移動，內容都會顯著改變。「往任何方向移動都變化很大」正是角點的數學特徵。



Harris 的直覺是「拿一個小視窗在影像上推來推去，看灰階變化多大」，中間那一欄就把這件事直接畫出來：把視窗往各個方向推 (u, v)，記錄推開之後與原位置的平方差 E。平坦區推到哪裡都幾乎沒變化（整片暗）；邊緣只有垂直於邊的方向會變（一條亮帶）；角點則往任何方向推都劇烈變化（中間一個明顯的坑）。右邊把這件事濃縮成兩個數字——結構張量的兩個特徵值 λ1、λ2，它們正是 E 這個二次曲面在兩個主軸上的陡峭程度。本圖的三個位置不是用眼睛指的，而是由量測挑出來的：λ1 最小者為平坦區（3.2, 3.1，比值 1.0）、λ1 大而比值最高者為邊緣（29120, 105，比值 276）、λ2 最大者為角點（39345, 37386，比值 1.1）。散點圖顯示絕大多數像素都擠在左下角（平坦）與左上方（邊緣），真正兩者都大的點少之又少——這就是「角點稀少而珍貴」的量化版本。也請留意：這個 2×2 矩陣與第 12 章光流解方程式用的是同一個矩陣，它可逆的條件就是角點的定義。

圖 13-3　平坦、邊緣、角點：自相關曲面與結構張量的兩個特徵值

Harris 把這個直覺量化成一個「結構張量」(從視窗內的梯度統計而來)，其兩個特徵值反映了兩個主要方向上的變化強度：兩者都小是平坦區、一大一小是邊緣、兩者都大則是角點。Harris 巧妙地用一個不需實際計算特徵值的響應函數 R 來判斷，計算高效。這裡也再次呼應第 9 章：梯度(Sobel)是這一切的基礎——角點偵測建立在灰階變化(梯度)的分析之上。



兩個判準都想回答同一個問題——「λ1 與 λ2 是不是都夠大」——但用的方式不同，而這個差別有實際後果。Harris 不去解特徵值（1988 年算特徵值太貴），而是用 $R = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2 = \det(M) - k \cdot \mathrm{trace}(M)^2$ 來近似，因為行列式與跡可以直接從矩陣元素算出來。前三張圖是 R 在 (λ1, λ2) 平面上的樣子，黑線是 R = 0 的分界：k 愈大，被判為角點的區域就愈窄、愈挑剔。要注意 k 有一個上限 0.25——因為在 λ1 = λ2 的對角線上 $R = \lambda^2(1 - 4k)$，一旦 k > 0.25，連「完美的角點」R 都會變成負的，整張圖再也找不到角點。最右邊是 Shi-Tomasi：它直接取 min(λ1, λ2)，判準就是「較小的那個夠不夠大」，沒有 k 要調，分界線也單純得多。實測兩者在同一張影像上各取前 200 個角點，有 80% 落在彼此 3 像素以內——大同小異，但 Shi-Tomasi 少一個要調的參數，這就是它成為 goodFeaturesToTrack 預設的原因。

圖 13-4　兩種判準：Harris 的 R 與 Shi-Tomasi 的 min(λ1, λ2)

13.2.2 Shi-Tomasi 的改良

1994 年，Jianbo Shi 與 Carlo Tomasi 在論文《Good Features to Track》中提出改良：與其用 Harris 的響應函數，不如直接取「兩個特徵值中較小者」作為角點強度——只要較小的特徵值夠大，就保證兩個方向都有足夠變化。這個判準在實務上(尤其是用於第 12 章的光流追蹤)表現更穩定，因此 OpenCV 的 goodFeaturesToTrack 函式預設採用 Shi-Tomasi。名稱「適合追蹤的好特徵」也點明了它與第 12 章光流的血緣。

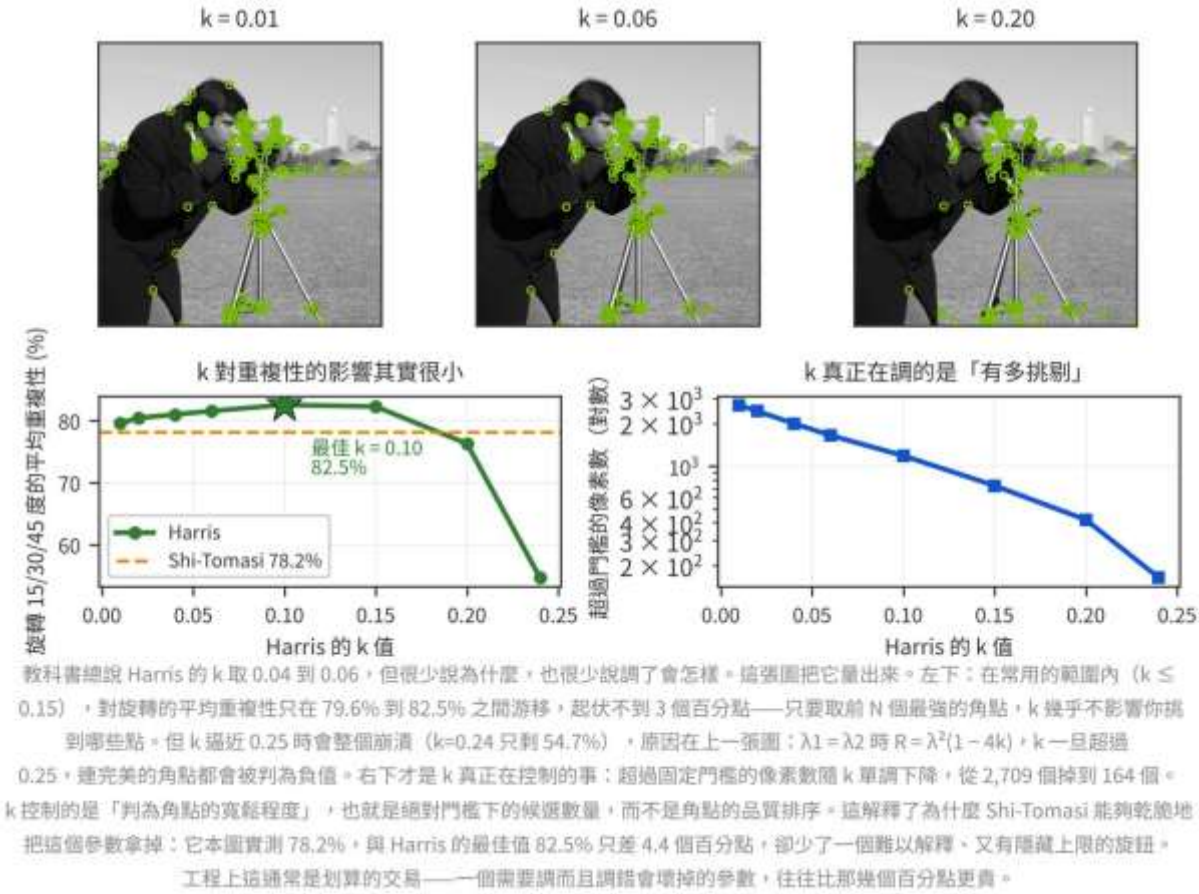


圖 13-5　k 值到底在調什麼：挑剔程度，而不是角點品質

程式 13-1 Harris 與 Shi-Tomasi 角點偵測

```
import cv2, numpy as np

img = cv2.imread("building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# --- Harris 角點 ---
harris = cv2.cornerHarris(np.float32(gray), blockSize=2,
                          ksize=3, k=0.04)
img_h = img.copy()
img_h[harris > 0.01 * harris.max()] = [0, 0, 255]   # 標紅

# --- Shi-Tomasi(適合追蹤,第 12 章光流的輸入) ---
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100,
            qualityLevel=0.01, minDistance=10)
img_s = img.copy()
for c in np.int0(corners):
    x, y = c.ravel()
    cv2.circle(img_s, (x, y), 4, (0, 255, 0), -1)

cv2.imwrite("harris.png", img_h)
cv2.imwrite("shitomasi.png", img_s)
```

角點偵測的一個侷限是：它不具備尺度不變性。一個在原尺寸下是「尖銳角點」

的結構，尺度一變，同一個固定大小的視窗框住的就是完全不同的內容。這裡要補一個容易被略過的細節：真正垮掉的其實是「描述」而不是「偵測」。實測顯示角點縮小之後多半仍偵測得到、位置也大致還在，但描述子必須知道「要用多大的鄰域來描述這個點」，而固定大小的鄰域在縮放後框住的是不同的結構，於是匹配就失效了(圖 13-6 實測：縮到三分之一時，Harris 配固定區塊描述子的匹配正確率只剩 15%，ORB 57%，而 SIFT 仍有 82%)。要處理「同一物體以不同大小出現」的問題，就需要下一節登場的主角——SIFT。

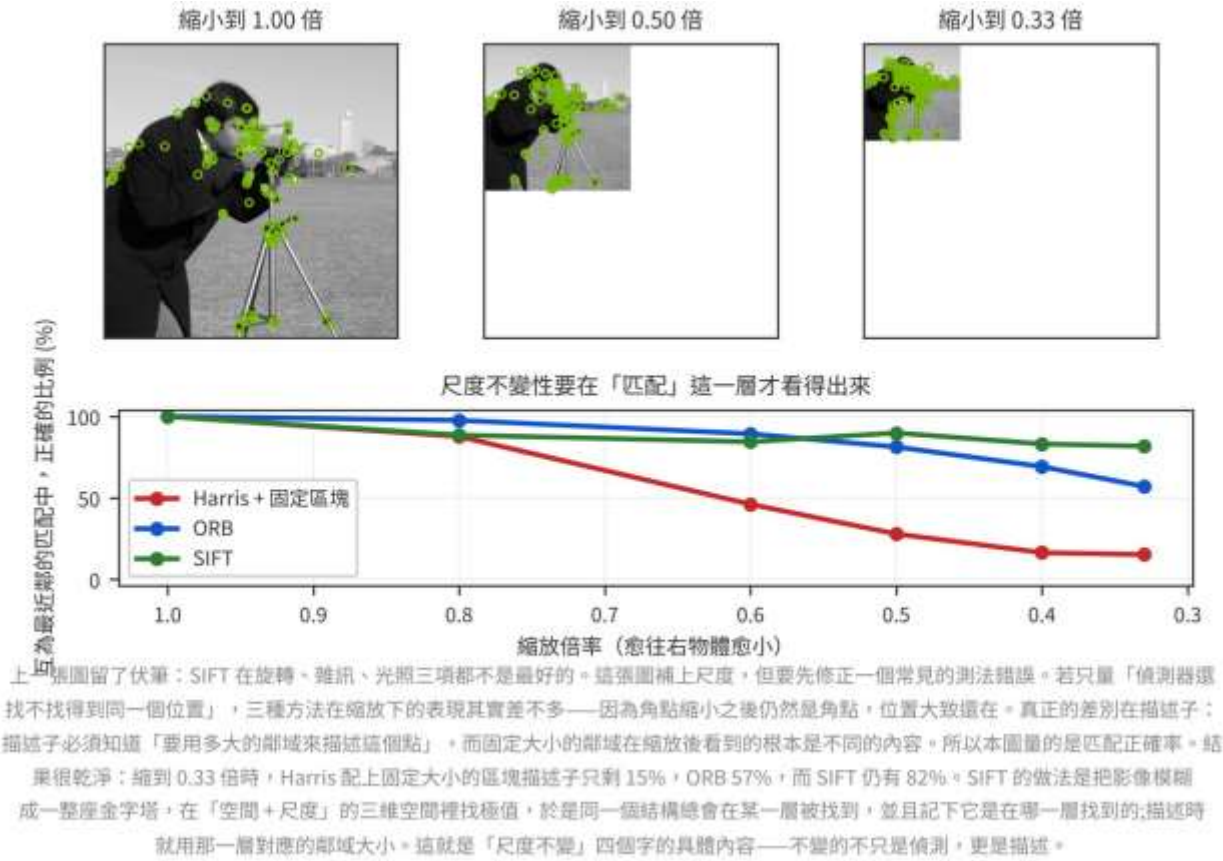


圖 13-6　尺度不變性要在「匹配」這一層才看得出來

13.2.3 推導：Harris 的 R 函數從哪裡來，以及 k 的隱藏上限

13.2.1 節說 Harris 把「視窗往各方向推、看灰階變化多大」量化成一個結構張量，這一節把中間的推導補上，因為它會解釋兩件事：$R = \det(M) - k \cdot \mathrm{trace}(M)^2$ 這個奇怪的式子從哪裡來，以及那個 k 為什麼不能亂調。

先把直覺寫成數學。把視窗往 (u, v) 推之後與原位置的平方差是 $E(u, v) = \Sigma\, w(x, y)\cdot[I(x+u, y+v) - I(x, y)]^2$。對位移很小的情況做一階泰勒展開，$I(x+u, y+v) - I(x, y) \approx I_x\cdot u + I_y\cdot v$，於是 $E(u, v) \approx [u\ v]\cdot M\cdot[u\ v]^T$，其中 M 是那個 2×2 的結構張量，由 I_x^2、I_xI_y、I_y^2 在視窗內的加權和組成。換句話說，E 是一個二次曲面——圖 13-3 中間那一欄畫的就是它。

M 是對稱半正定矩陣，所以可以對角化，兩個特徵值 λ1、λ2 正是這個二次曲面在兩個主軸方向上的陡峭程度。於是判斷規則很清楚：兩者都小是平坦區（推到哪裡都不變）、一大一小是邊緣（只有一個方向會變）、兩者都大才是角點。圖 13-3 的實測也印證了這一點：平坦區的 (λ1, λ2) 是 (3.2, 3.1)，邊緣是 (29120, 105)，比值高達 276 倍，而角點是 (39345, 37386)，兩者都大且比值只有 1.1。

問題是 1988 年算特徵值太貴。Harris 的巧思是：不必真的解出 λ1 與 λ2，只要用它們的兩個對稱函數就夠了——$\det(M) = \lambda1\lambda2$、$\mathrm{trace}(M) = \lambda1 + \lambda2$，而這兩個量可以直接從矩陣元素算出來，不需要開根號。於是定義 $R = \lambda1\lambda2 - k(\lambda1 + \lambda2)^2$：平坦區兩者都小，R 接近零;邊緣時 $\lambda2 \approx 0$，det 接近零而 trace 很大，R 變成大的負值;只有兩者都大時 R 才是大的正值。用一個不必開根號的式子，就把三種情況分開了。

那個 k 呢?教科書一律說取 0.04 到 0.06，但很少說它有一個數學上的上限。把 $\lambda1 = \lambda2 = \lambda$（最理想的角點）代進去：$R = \lambda^2 - k(2\lambda)^2 = \lambda^2(1 - 4k)$。一旦 $k > 0.25$，括號內就變成負的——連最完美的角點都會得到負的響應，整張影像再也找不到任何角點。圖 13-5 的實測正好驗證：k 從 0.01 到 0.15 之間，對旋轉的重複性只在 79.6% 到 82.5% 之間游移，起伏不到 3 個百分點;但 k 推到 0.24 時就崩到 54.7%。

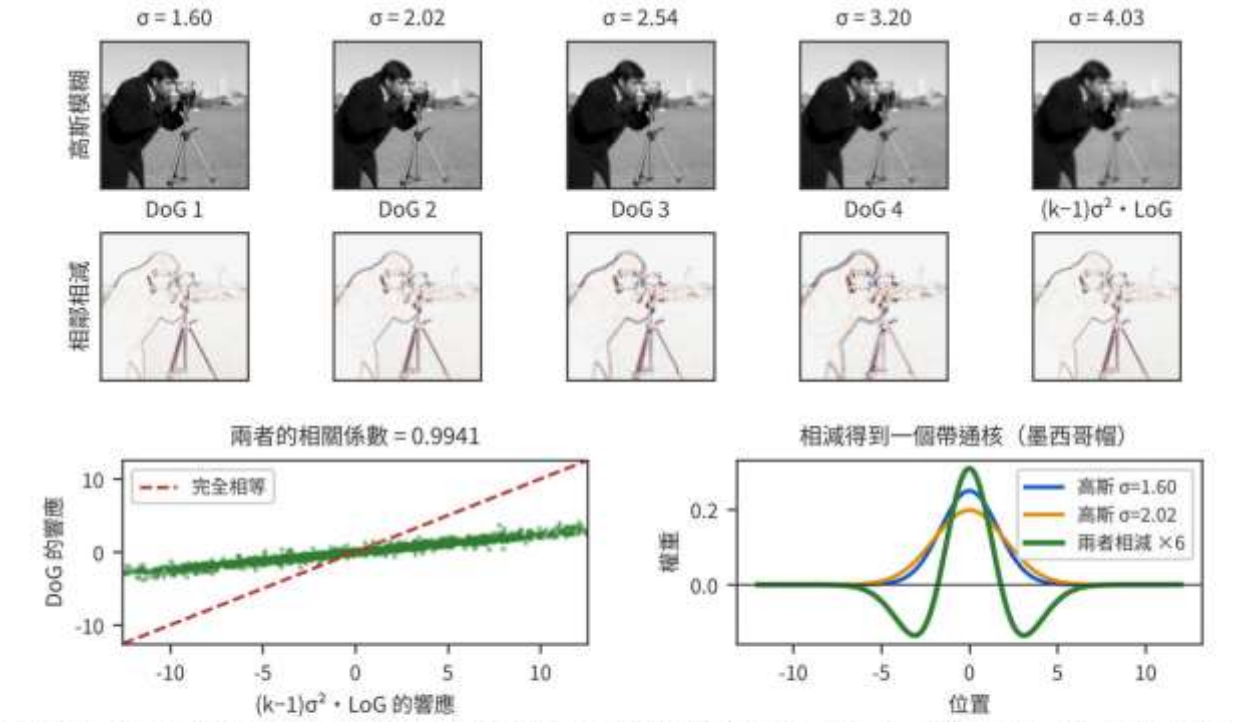
這也說明了 k 真正在控制什麼。它不改變角點的強弱排序（所以「取前 N 個最強的角點」幾乎不受影響），它改變的是「在固定門檻下有多少像素被判為角點」——圖 13-5 右下實測，k 從 0.01 到 0.24，超過門檻的像素從 2,709 個降到 164 個。理解了這一點，就會明白 Shi-Tomasi 的取捨：它直接取 $\min(\lambda1, \lambda2)$，判準的意義一目了

然、沒有隱藏的上限、也沒有參數要調，代價是真的要解一次特徵值。以今天的算力，這個代價早就不值一提——這是演算法設計中很常見的一幕：當年為了省算力而做的近似，在算力變便宜之後，反而成了需要被解釋的負擔。

13.3 尺度不變特徵：SIFT 與 ORB

13.3.1 SIFT 的四個步驟

承叫獸開講，SIFT 的偉大之處在於它同時解決了尺度與旋轉不變性。它的完整流程可以拆成四步，每一步都巧妙地呼應了本書前面的概念：



SIFT 的第一步是建一座尺度金字塔，而它找極值用的不是影像本身，是相鄰兩層的「高斯差」（DoG）。為什麼可以這樣？右下角給了直覺：兩個寬度略不同的高斯相減，得到一個中間正、周圍負的帶通核——也就是第 9 章的墨西哥帽（LoG）。嚴格一點說，熱傳導方程式給出 $\partial G/\partial\sigma = \sigma\cdot\nabla^2 G$，把左邊的微分換成差分就得到 $G(k\sigma)-G(\sigma)\approx(k-1)\sigma^2\cdot\nabla^2 G$。本圖把兩邊實際算出來比對：相關係數高達 0.9941（左下散點圖）。這個近似為什麼重要？因為 LoG 要做二階微分、運算貴，而高斯模糊無論如何都要做（建金字塔本來就要），相減則幾乎不花錢。SIFT 等於用一個「順手」的運算換到了昂貴的帶通濾波器——這是演算法設計上非常漂亮的一步，也是第 11 章多解析度分析與第 9 章邊緣偵測在本章交會的地方。

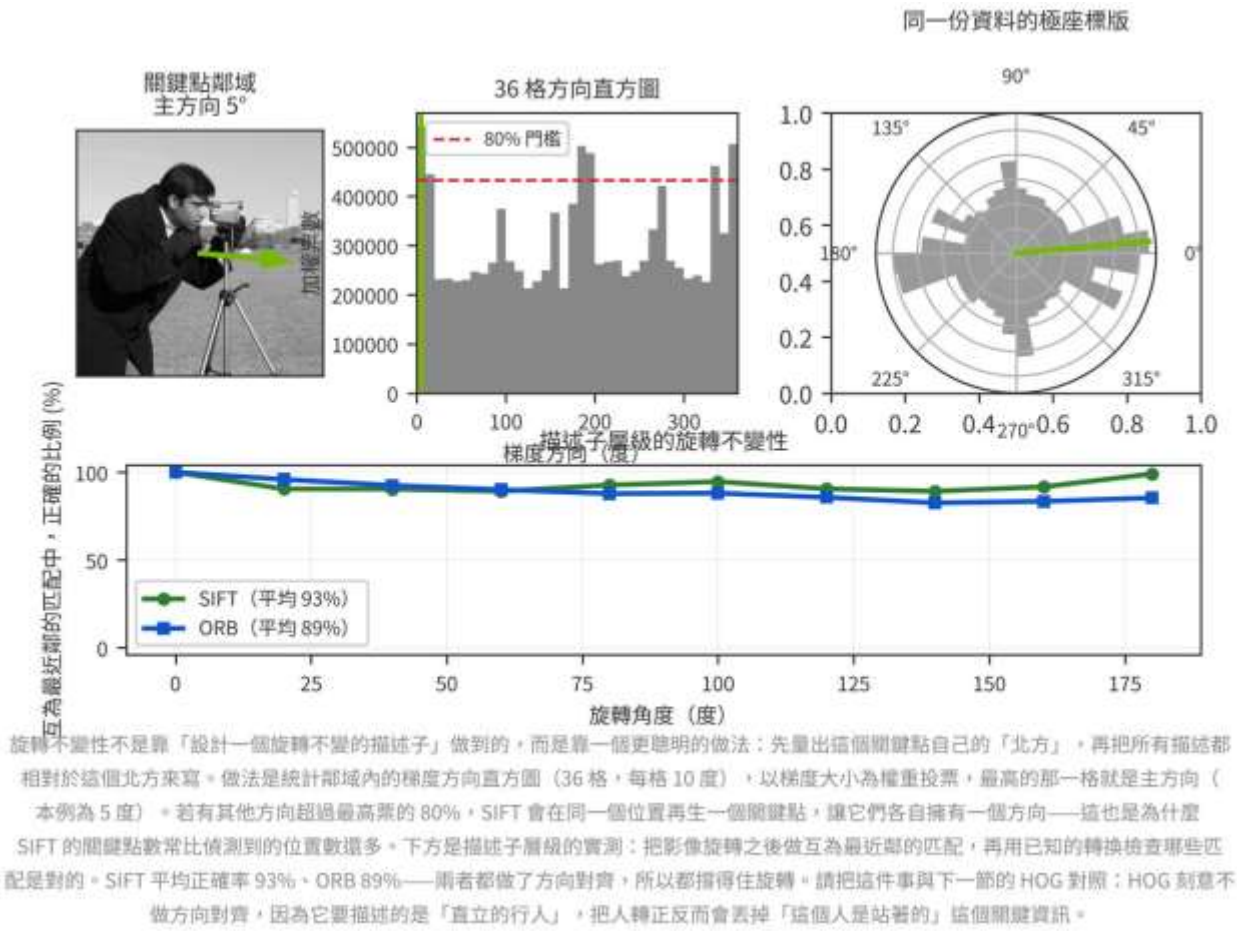
圖 13-7　尺度空間與 DoG：為什麼相減就能近似 LoG

4. 尺度空間極值偵測：把影像用一系列愈來愈強的高斯模糊處理，建構出一個「尺度金字塔」(這正是第 11 章多解析度分析的思想!)。相鄰兩層相減得到「高斯差」(DoG)，它是第 9 章 LoG 的高效近似，也是一個帶通濾波器(第 6 章)。

在這個「空間加尺度」的三維空間中尋找極值點——一個點若在自己的尺度與相鄰尺度中都是局部極值，它就是候選關鍵點。關鍵在於：因為在多個尺度中搜尋，無論物體以什麼大小出現，都能在對應的尺度層被找到，這就實現了尺度不變性。

5. 關鍵點精確定位：剔除對比度太低(易受雜訊干擾)與位於邊緣上(定位不穩)的候選點，留下真正穩定的關鍵點。
6. 方向指派：分析關鍵點周圍的梯度方向分布，把最主要的方向指派給該關鍵點作為「基準方向」。之後所有的描述都相對於這個基準方向來計算——這就實現了旋轉不變性(無論物體轉了多少度，基準方向會跟著轉，描述保持一致)。
7. 描述子生成：在關鍵點周圍取一個區塊，依基準方向對齊後，統計區塊內各子區域的梯度方向直方圖，組成一個 128 維的向量——這就是該關鍵點的「指紋」(描述子)。這個向量經過正規化以抵抗光照變化，最終得到一個對尺度、旋轉、光照都穩健的特徵描述。

值得回味的是，SIFT 幾乎是本書前半的集大成：高斯模糊(第 5 章)、多尺度(第 11 章)、DoG/帶通(第 6、9 章)、梯度(第 9 章)、直方圖(第 5 章)、正規化(第 5 章)——這些看似分散的工具，在 SIFT 中被組織成一個優雅的整體。這也說明了為什麼要循序學完前面各章：它們是理解進階演算法的積木。



旋轉不變性不是靠「設計一個旋轉不變的描述子」做到的，而是靠一個更聰明的做法：先量出這個關鍵點自己的「北方」，再把所有描述都相對於這個北方來寫。做法是統計鄰域內的梯度方向直方圖（36 格，每格 10 度），以梯度大小為權重投票，最高的那一格就是主方向（本例為 5 度）。若有其他方向超過最高票的 80%，SIFT 會在同一個位置再生一個關鍵點，讓它們各自擁有一個方向——這也是為什麼 SIFT 的關鍵點數常比偵測到的位置數還多。下方是描述子層級的實測：把影像旋轉之後做互為最近鄰的匹配，再用已知的轉換檢查哪些匹配是對的。SIFT 平均正確率 93%、ORB 89%——兩者都做了方向對齊，所以都撐得住旋轉。請把這件事與下一節的 HOG 對照：HOG 刻意不做方向對齊，因為它要描述的是「直立的行人」，把人轉正反而會丟掉「這個人是站著的」這個關鍵資訊。

圖 13-8　方向指派：先量出自己的北方，再相對於它來描述

13.3.2 ORB：免費而快速的替代方案

SIFT 雖強，卻有兩個現實問題：受專利保護(叫獸開講的核心)、且運算較慢(128 維浮點描述子，計算與比對都吃資源)，不適合即時或資源受限的場合。2011 年 OpenCV 團隊提出的 ORB，正是為了解決這兩點。

ORB 是兩個既有技術的聰明融合：用 FAST 角點偵測器(極快)找關鍵點，並加上方向資訊使其具旋轉不變性;用 BRIEF 二進位描述子(以一連串「這個點比那個點亮嗎」的是非比較，產生一串 0 與 1)來描述特徵，並加以旋轉校正。二進位描述子的好處是：比對時用漢明距離(算兩串位元有幾位不同)，可以用 CPU 指令極速完成，遠快於 SIFT 的浮點運算。ORB 完全免費、速度可比 SIFT 快數十倍，雖然在尺度變化極大或精度要求極高的場合略遜 SIFT，但在多數即時應用(尤其是機器人、手機、樹莓派 / Jetson 等第 17 章的邊緣平台)中是務實首選。

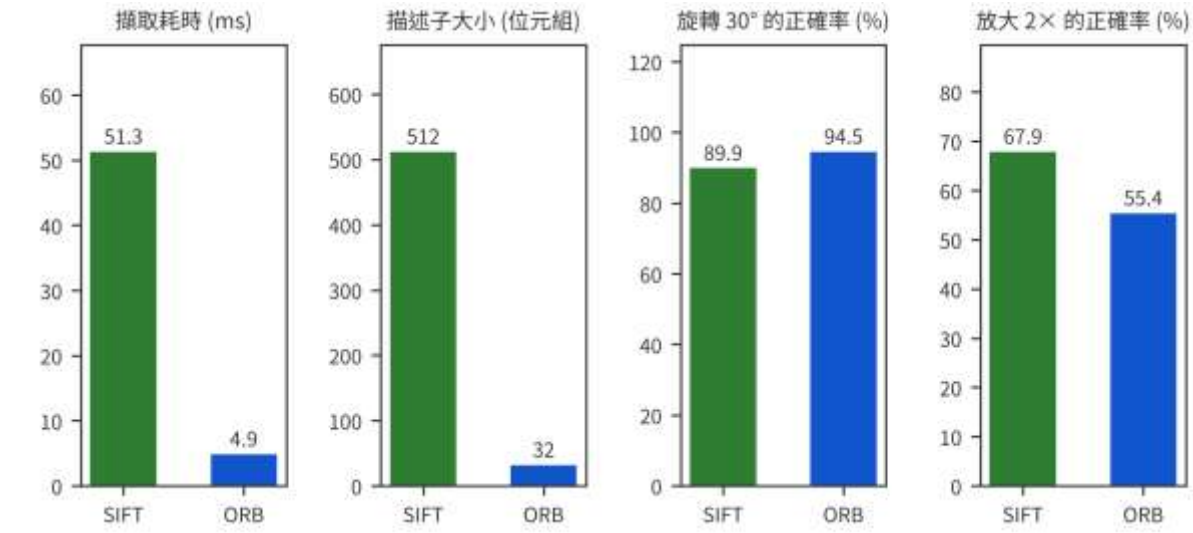


表 13-1 列了 SIFT 與 ORB 的差異，這張圖把它量出來。在同一張 512×512 影像、同樣上限 800 個特徵的條件下：ORB 擷取只要 4.9 毫秒，SIFT 要 51.3 毫秒，快了 10.6 倍;描述子一個是 32 位元組的二進位碼，一個是 512 位元組的浮點向量，差 16 倍。右邊兩格是代價，但結果可能和你預期的不同。旋轉 30 度時 ORB 的匹配正確率 94.5% 反而略高於 SIFT 的 89.9%——ORB 也會先量主方向再旋轉它的取樣樣式（rBRIEF），所以旋轉這一項它並不吃虧，「SIFT 一定比較準」這句話並不成立。真正的分水嶺在尺度：放大 2 倍時 SIFT 有 67.9%，ORB 只剩 55.4%，因為 ORB 的金字塔層數遠少於 SIFT，尺度跨度一大就跟不上（這與上一張圖的結論一致）。所以選擇的依據很清楚：即時系統、嵌入式裝置、尺度變化不大（例如同一台固定相機的連續影格），選 ORB;要跨尺度比對、拼接差異很大的照片、或精度優先，選 SIFT。再加上叫獸閱讀那一課——2020 年之前還要考慮專利，而這正是 ORB 當年能站穩腳步的真正原因。

圖 13-9　SIFT 與 ORB 的四項實測對照

比較面向	SIFT	ORB
授權	原有專利(2020 年到期)	完全免費開源
描述子	128 維浮點向量	二進位字串(漢明距離比對)
速度	較慢	快數十倍，可即時
尺度不變性	優異	中等(尺度變化大時較弱)
適用場合	高精度離線比對、拼接	即時、行動與邊緣裝置

表 13-1 SIFT 與 ORB 的比較

程式 13-2 SIFT 與 ORB 特徵擷取

```
import cv2

img = cv2.imread("object.jpg", cv2.IMREAD_GRAYSCALE)

# --- SIFT(專利已於 2020 到期,可自由使用) ---
sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(img, None)
print("SIFT 關鍵點數:", len(kp_sift),
      "描述子維度:", des_sift.shape)      # (N, 128)
```

```
# --- ORB(免費、快速) ---
orb = cv2.ORB_create(nfeatures=500)
kp_orb, des_orb = orb.detectAndCompute(img, None)
print("ORB 關鍵點數:", len(kp_orb),
      "描述子維度:", des_orb.shape)        # (N, 32) 二進位

# 畫出關鍵點(含尺度與方向)
vis = cv2.drawKeypoints(img, kp_sift, None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("keypoints.png", vis)
```

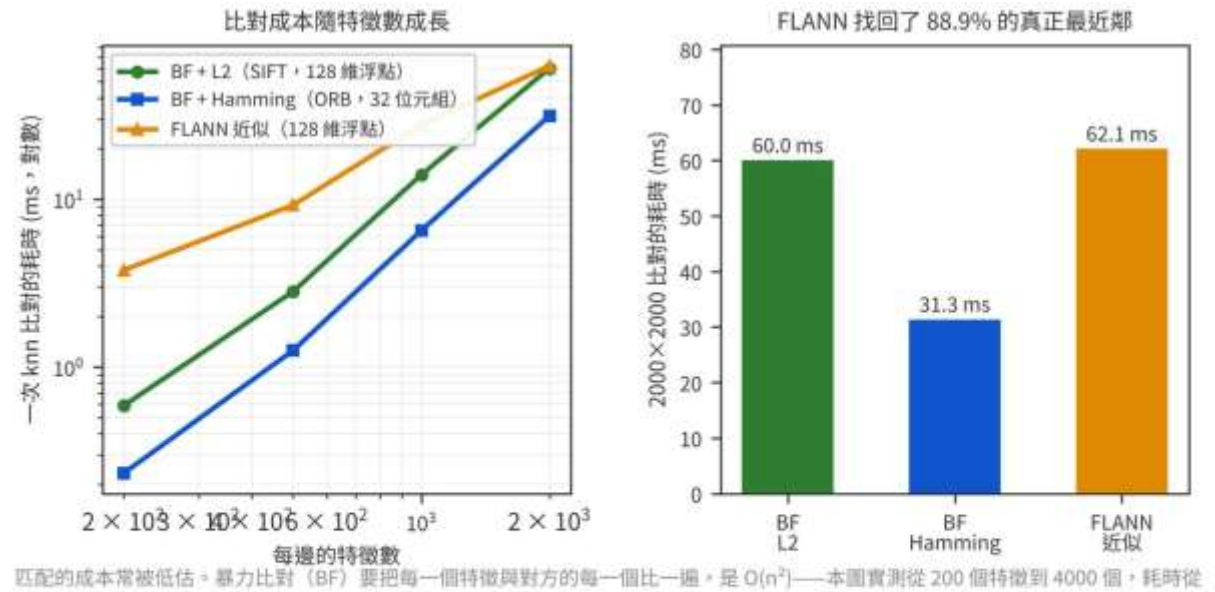
13.4 特徵匹配與外點去除

偵測出兩張影像各自的特徵與描述子後，下一步是匹配(Matching)：找出「這張影像的哪個特徵，對應到那張影像的哪個特徵」。這是影像拼接、物件辨識、3D 重建的關鍵一步。

13.4.1 匹配器：BF 與 FLANN

最直接的方法是暴力匹配(Brute-Force,BF)：把第一張影像的每個描述子，與第二張影像的所有描述子逐一比對距離，取最近的作為匹配。距離的計算方式取決於描述子類型——SIFT 的浮點描述子用歐氏距離，ORB 的二進位描述子用漢明距離。BF 準確但當特徵量大時很慢。

FLANN(快速近似最近鄰函式庫)則以近似的方式加速搜尋：它建立特殊的資料結構(如 KD 樹)，不保證找到絕對最近的匹配，但速度快得多，適合大量特徵的場合。BF 與 FLANN 的取捨，又是一次「準確 vs. 速度」的工程權衡——與第 9 章門檻、第 10 章偵測參數、第 12 章追蹤器選擇一脈相承。

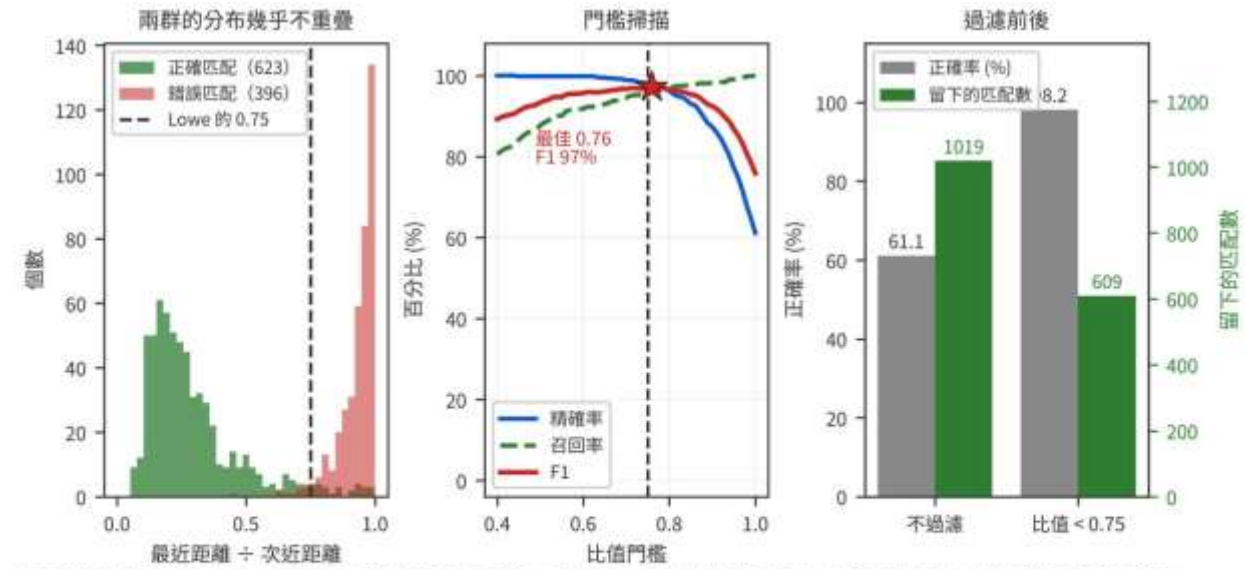


匹配的成本常被低估。暴力比對（BF）要把每一個特徵與對方的每一個比一遍，是 $O(n^2)$——本圖實測從 200 個特徵到 4000 個，耗時從 0.6 毫秒漲到 60 毫秒。ORB 的二進位描述子在這裡展現真正的威力：Hamming 距離只是互斥或再數 1，CPU 有專門的指令，2000×2000 時只要 31.3 毫秒，而 L2 要 60.0 毫秒，快了 2 倍。注意這已經包含了「32 位元組對 512 位元組」的資料量差距，兩個因素是疊加的。FLANN 走另一條路：用 kd 樹做近似搜尋，2000×2000 時 62.1 毫秒。但它是近似的——實測在真實的 SIFT 描述子上，它找回了 88.9% 的真正最近鄰。（這個實驗一定要用真實描述子：隨機的高維向量彼此幾乎等距，近似搜尋的召回率會低到毫無代表性，量到的是維度災難而不是 FLANN 的本事。）觀念要記住：FLANN 用一點正確率換速度，而且特徵數愈多愈划算;精度優先、特徵數又不多的場合，仍該用 BF。

圖 13-10　比對的成本：BF、漢明距離與 FLANN 近似搜尋

13.4.2 比值測試：過濾模糊匹配

原始匹配充滿錯誤——因為並非每個特徵都在另一張影像中有真正的對應(可能被遮擋、或根本不存在)。David Lowe 在 SIFT 論文中提出了一個極為有效的過濾法：比值測試(Ratio Test)。對每個特徵，找出它在另一張影像中最近與次近的兩個匹配;若「最近距離」明顯小於「次近距離」(例如小於 0.75 倍)，代表這個匹配獨一無二、可信;若兩者接近，代表這個特徵在對方影像中有多個相似的候選、難以區分，應該捨棄。這個簡單的比值，能濾掉大量不可靠的匹配。



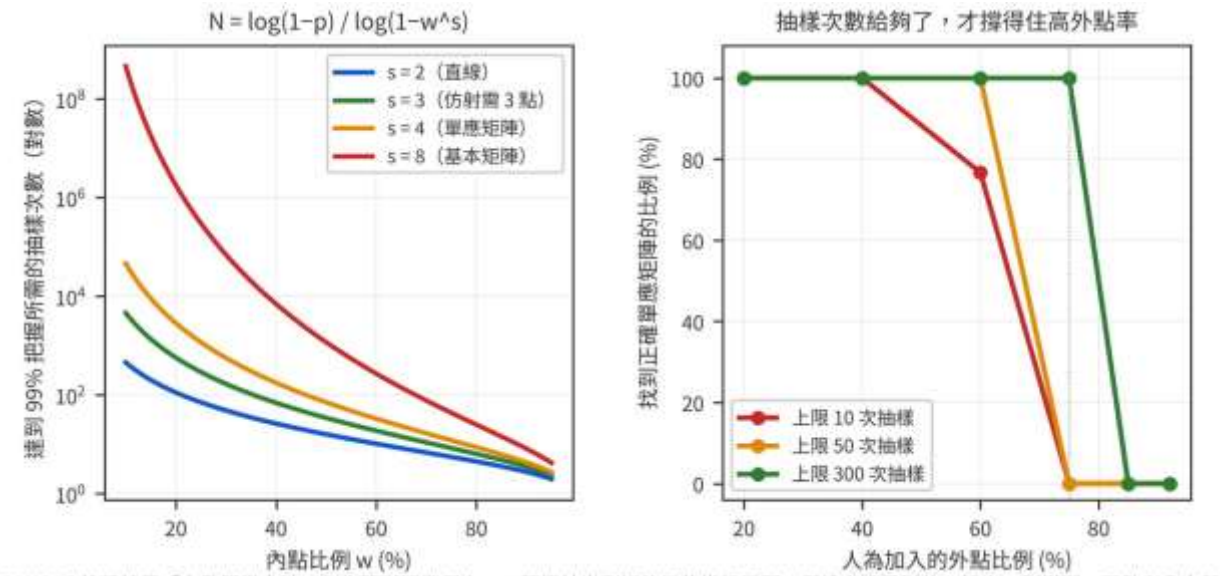
Lowe 比值測試的巧妙之處，在於它不看「最近的距離有多近」，而看「最近的比次近的近多少」。理由是：如果一個特徵在對方影像中真的有唯一的對應，那麼最近鄰應該明顯勝過第二名;若第一名與第二名難分軒輊，代表這個特徵長得很普通（例如重複的紋理、磚牆、樹葉），配哪一個都不可靠。左圖是實測的證據：本圖用已知的單應矩陣造出一對影像，所以每一個匹配是對是錯都查得出來。兩群的分布幾乎不重疊——錯誤匹配的比值大多擠在 0.9 以上。中圖掃描門檻：最佳值在 0.76（F1 97%），而 Lowe 論文建議的 0.75 給出精確率 98.2%、召回率 96.0%，相當接近最佳。右圖是這一刀的效果：原始 1019 個匹配中只有 61.1% 是對的，過濾後剩 609 個，正確率升到 98.2%。一個不用訓練、只比兩個數字的規則就能做到這件事，這是整個電腦視覺裡性價比最高的技巧之一。

圖 13-11　比值測試：為什麼要看「第二名」

13.4.3 RANSAC：在錯誤中找出真相

即使經過比值測試，仍會有一些「看起來可信、實則錯誤」的匹配(稱為外點，outliers)。RANSAC(隨機抽樣一致法)是對抗外點的利器，其思想極為漂亮：與其相信所有資料，不如反覆隨機抽取少數幾組匹配，假設它們是正確的、據此計算出一個幾何模型(例如第 4 章的透視轉換矩陣)，再看有多少其他匹配符合這個模型;重複多次，「獲得最多支持者」的模型就是正確答案，而不符合它的匹配就是外點，予以剔除。

RANSAC 的精神與第 9 章霍夫轉換異曲同工——都是「投票」：霍夫讓每個邊緣點投票給可能的直線，RANSAC 讓每個匹配投票給可能的幾何模型，得票最高者勝出。這種「用多數證據對抗少數雜訊」的思想，是穩健估計的核心，在充滿雜訊與錯誤的真實資料中至關重要。



RANSAC 的精神是「少數服從多數，但先隨機問幾個人」：隨機抽出剛好夠用的幾組匹配、假設它們是對的、算出一個模型，再數有多少其他匹配同意這個模型;重複多次，票最高的勝出。要抽幾次才夠?這有閉式解。若內點比例是 w、建一個模型最少要 s 組匹配，那麼一次抽樣全中的機率是 w^s，連抽 N 次都沒全中的機率是 (1−w^s)^N，要讓成功機率達到 p，就需要 N = log(1−p) / log(1−w^s)。左圖把這條式子畫出來，重點在縱軸是對數：單應矩陣要 4 組點（s = 4），內點 50% 時只要 71 次，內點 20% 時要 2876 次，內點 10% 時暴增到 46049 次。右圖是實測，而且刻意把抽樣次數的上限調低，好讓這條公式吃得到苦頭。上限 10 次只撐得到外點 40%、上限 50 次只撐得到外點 60%、上限 300 次只撐得到外點 75%。三條曲線崩潰的位置，正好對應左圖那條理論曲線穿過各自上限的地方——外點 60% 時理論需要 178 次，所以上限 10 次的那條已經開始掉（77%）;外點 85% 時理論需要 9094 次，三條就全軍覆沒了。兩個實務上的推論：一是 s 要愈小愈好（這是為什麼能用仿射就別用單應）;二是先做比值測試把內點比例拉高，RANSAC 會輕鬆非常多——這兩步是搭配使用的，不是二選一。

圖 13-12　RANSAC 要抽幾次才夠：一條有閉式解的公式

程式 13-3 特徵匹配、比值測試與 RANSAC 物件定位

```
import cv2, numpy as np

img1 = cv2.imread("box.jpg", cv2.IMREAD_GRAYSCALE)        # 要找的物件
img2 = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)      # 場景

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# --- BF 匹配 + 比值測試 ---
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)     # 取最近兩個
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:     # Lowe 比值測試
        good.append(m)
print("比值測試後的匹配數:", len(good))

# --- RANSAC 求單應性矩陣,剔除外點 ---
if len(good) > 10:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("RANSAC 判定的內點數:", int(mask.sum()))
```

```
# 在場景中畫出物件的位置(第 4 章透視轉換)
h, w = img1.shape
corners = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1,1,2)
projected = cv2.perspectiveTransform(corners, H)
scene = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)
cv2.polylines(scene, [np.int32(projected)], True, (0,255,0), 3)
cv2.imwrite("object_located.png", scene)
```

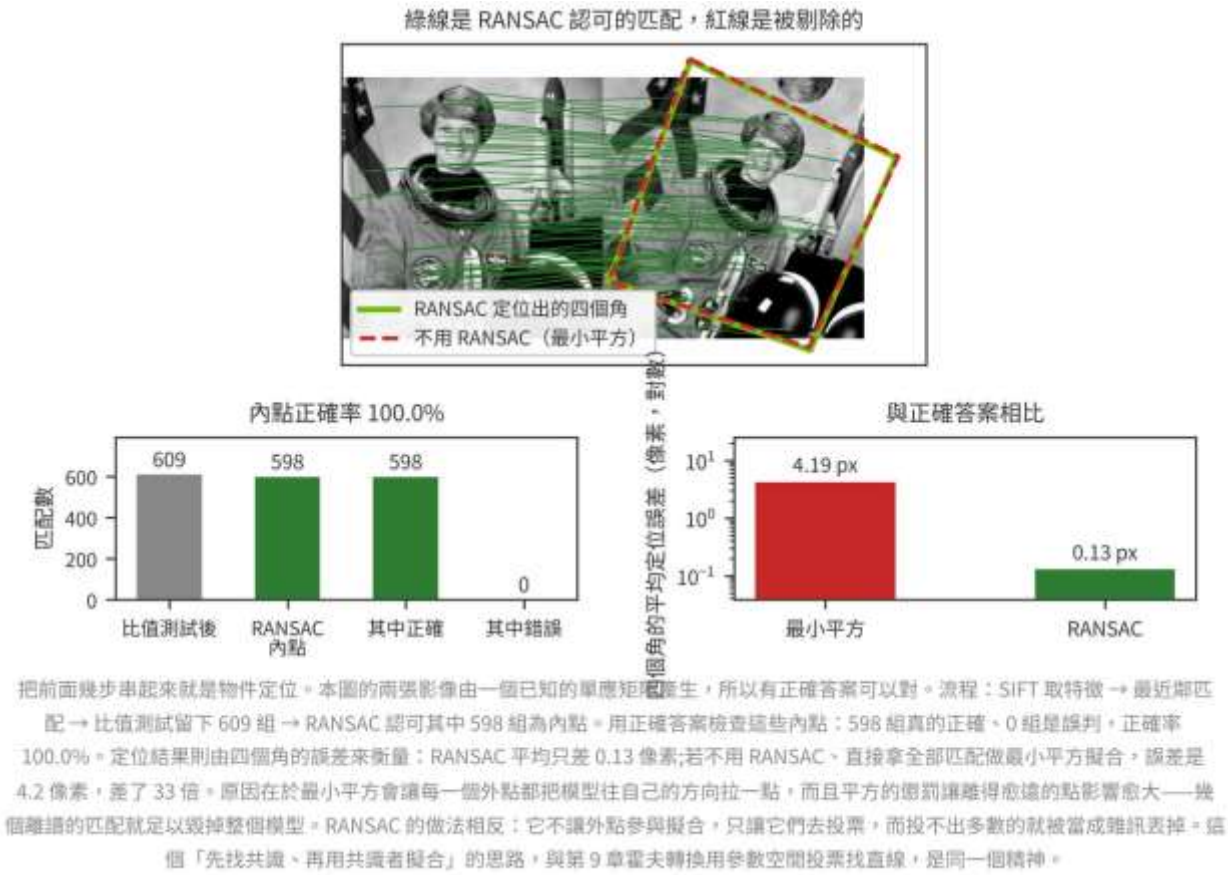


圖 13-13　RANSAC 實戰：用了與不用，定位誤差差三十倍

13.4.4 推導：RANSAC 要抽幾次才夠

RANSAC 是隨機演算法，所以它不保證成功，只保證「成功的機率」。好消息是這個機率可以精確算出來，而且算式很短，值得親手推一次。

設內點（正確匹配）佔全部匹配的比例為 w，而建立一個模型最少需要 s 組匹配（直線要 2 點、仿射轉換要 3 點、單應矩陣要 4 點、基本矩陣要 8 點）。一次隨機抽樣時，抽到的 s 組全是內點的機率是 w^s（匹配數夠多時可視為獨立）。那麼「這一次沒有全中」的機率是 1 – w^s，「連抽 N 次都沒有全中」的機率就是 (1 – w^s)^N。我們希望至少成功一次的機率達到 p（例如 0.99），於是 1 – (1 – w^s)^N ≥ p，兩

邊取對數整理即得 N ≥ log(1 – p) / log(1 – w^s)。

這條式子的性質值得細看，因為它決定了實務上的每一個選擇。第一，N 對 w 極度敏感，而且是指數式的。以單應矩陣（s = 4、p = 0.99）為例：內點 90% 時只要 4 次，內點 50% 時要 71 次，內點 30% 時要 566 次，內點 20% 時要 2,876 次，內點 10% 時暴增到 46,049 次。圖 13-12 左圖把這條曲線畫出來，縱軸必須用對數才畫得下。

第二，N 對 s 也極度敏感。同樣在內點 50% 的情況下，s = 2 只要 16 次、s = 4 要 71 次、s = 8 則要 1,177 次。這給了一條很實用的原則：能用自由度低的模型就別用高的。若已知兩張影像只差旋轉與平移，就別去解 8 個自由度的單應矩陣，用 4 個自由度的相似轉換，RANSAC 會輕鬆非常多，而且不容易被外點誘導出一個「扭曲但碰巧符合」的解。

第三，這條式子也解釋了比值測試為什麼必須先做。圖 13-11 的實測：原始匹配的正確率只有 61.1%，經過比值測試之後升到 98.2%。代進公式，w 從 0.61 變成 0.98，所需抽樣次數從 29 次降到 2 次。比值測試與 RANSAC 不是二選一，而是一前一後的搭配——前者便宜地把 w 拉高，後者才有辦法在合理的次數內收斂。

實務上還有一個細節：真正的實作（包括 OpenCV）不會把 N 寫死，而是「邊做邊更新」。每找到一個更好的模型，就用它的內點數重新估計 w，再據此下修 N。所以你設的 maxIters 其實是一個上限，不是實際次數。圖 13-12 右圖刻意把這個上限調低，好讓理論吃得到苦頭：上限 10 次時外點 60% 就開始失手（成功率 77%），上限 50 次能撐到外點 60%，上限 300 次則撐到外點 75%。三條曲線崩潰的位置，正好對應理論曲線穿過各自上限的地方。這也提醒一件事——當你的 RANSAC 常常失敗，該先檢查的是內點比例，而不是一味調高迭代次數。

13.5 區域描述子與形狀特徵：HOG、LBP 與幾何量測

到這裡為止，本章的特徵都屬於同一種類型：在影像中找出少數幾個「點」，為每個點生成描述子，再跨影像配對這些點。這類關鍵點特徵(keypoint features)回答的是一個很特定的問題——「這兩張影像裡的哪一點是同一點?」它是拼接、定位、追蹤與 SLAM 的基礎。但真實世界還有另一大類問題：「這一塊區域是什麼?」這是行人嗎?這片金屬表面有刮痕嗎?這個零件的外形合格嗎?此時我們根本不需要知道點與點的對應關係，而需要一個能把整塊區域濃縮成一串數字、並且能直接餵給分類器的描述。

本節補上這一塊：三種最重要的區域與形狀描述子——HOG、LBP 與以輪廓為基礎的幾何特徵。它們既是深度學習之前電腦視覺的主力，也是第 14 章傳統機器學習的標準輸入;更重要的是，前面第 8、9、10 章多次預告的「紋理特徵」與「形狀特徵」，答案都在這裡。

13.5.1 兩種特徵觀：對應問題與辨識問題

關鍵點特徵與區域描述子的差別，可以從三個角度看清楚。

8. 取樣是稀疏還是稠密:關鍵點特徵是稀疏(sparse)的——它只在影像中「值得注意」的少數位置取值，一張照片可能只產生幾百到幾千個關鍵點，其餘區域一概不管。區域描述子則是稠密(dense)的——它在一個固定視窗內逐格取樣、全面覆蓋，不挑位置，因為我們要描述的是「整塊」而非「某點」。
9. 是否需要配對:關鍵點特徵的價值在配對，所以必須先經過匹配、比值測試與 RANSAC(13.4 節)。區域描述子不配對，而是把向量直接交給分類器(第 14 章的 SVM、kNN)，或是用直方圖距離比較兩塊區域像不像。
10. 描述的對象是灰階分布還是幾何:HOG 與 LBP 描述的仍是視窗內的灰階/梯度分布;形狀特徵則更進一步，描述一個「已經被切出來的物件」的幾何性質——它的輸入不是灰階影像，而是第 9 章分割、第 10 章型態學清理與連通元件標記之後得到的二值遮罩或輪廓。

這個分類也解釋了為什麼本節要放在這個位置：關鍵點特徵處理「對應問題」，區域與形狀特徵處理「辨識問題」，而一條完整的工業視覺管線，兩者往往都要用上。



13.5 節開頭說本章有「兩種特徵觀」，這張圖把差別畫出來。左邊是前四節的世界：偵測器在影像中挑出少數值得注意的位置，本例 241 個關鍵點，其餘區域一概不管。中間是接下來的世界：不挑位置，把視窗切成固定的網格，本例 32×32 = 1024 個細胞，每一格都要算。每一格都算進最後的向量。差別不只是「挑不挑位置」，而是要解的問題根本不同。關鍵點特徵要解的是對應問題——這張照片的這個點，是不是那張照片的那個點?所以它必須配對，也因此需要比值測試與 RANSAC。區域描述子要解的是辨識問題——這塊區域裡有沒有行人?它不配對，而是把一個長度固定的向量直接交給分類器（第 14 章）。長度固定這件事很關鍵：分類器要求每個樣本的維度一致，而關鍵點的數量每張影像都不同，沒辦法直接餵。一個 64×128 的行人視窗，HOG 給出的就是固定的 3,780 維。

圖 13-14　兩種特徵觀：稀疏取樣找對應，稠密取樣做辨識

13.5.2 HOG：方向梯度直方圖

2005 年，法國 INRIA 研究院的 Navneet Dalal 與 Bill Triggs 在 CVPR 發表了方向梯度直方圖(Histogram of Oriented Gradients,HOG)。在深度學習興起之前，「HOG + SVM」幾乎就是行人偵測的同義詞，也是汽車主動安全系統最早能真正上路的視覺技術之一。

HOG 的核心洞見只有一句話：一個物體的外形，主要寫在「邊緣方向的分布」裡，而不在絕對亮度裡。一個直立的人，不論穿深色或淺色衣服、不論在陽光下或陰影中，他的輪廓都會呈現同一套方向模式——肩線偏水平、軀幹兩側偏垂直、雙腿之間有一道縱向間隙。只要把這套方向模式量化下來，就抓住了「人形」。

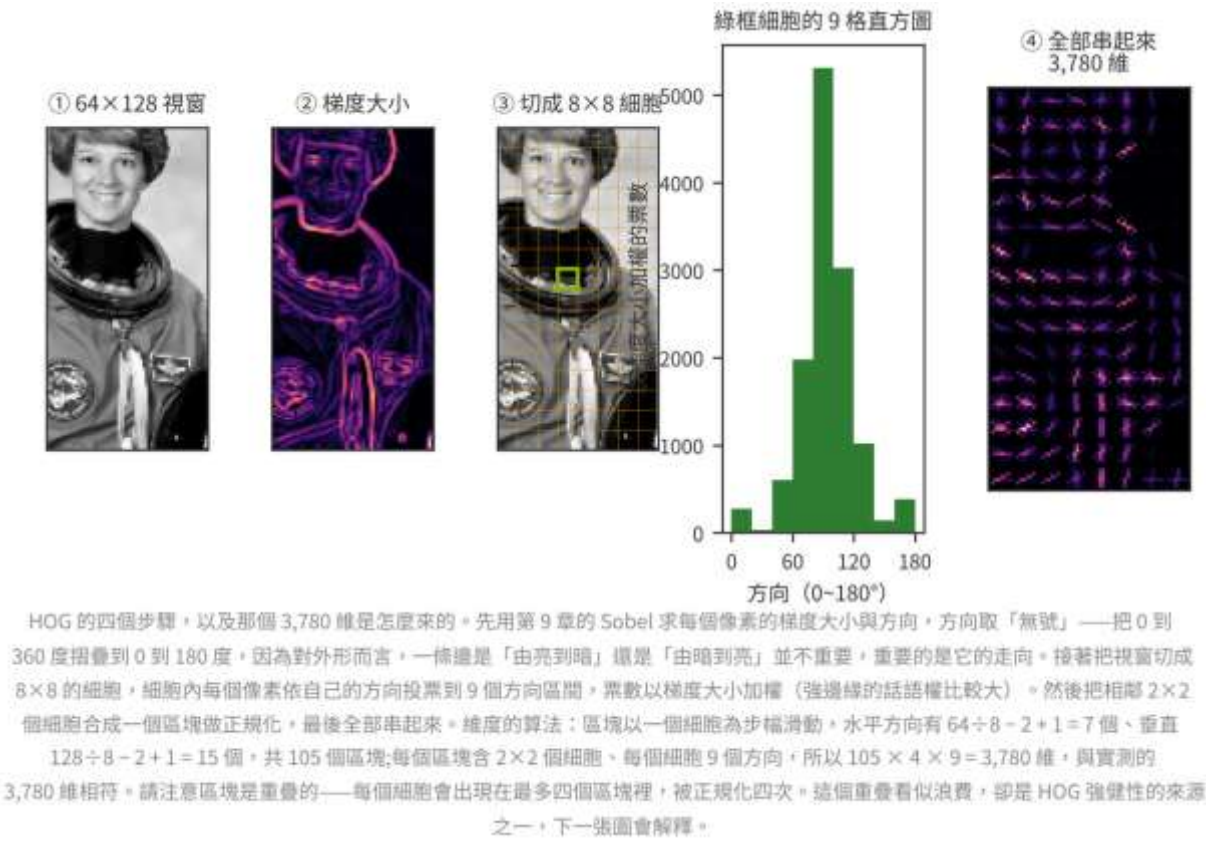
HOG 的計算分成四步，每一步都用到本書前面的工具：

11. 梯度計算:先用第 9 章的 Sobel 算子求每個像素的梯度大小與方向。注意 HOG 一般使用「無號方向」，把 0 到 360 度摺疊到 0 到 180 度——因為我們在意的是邊緣的走向，而不在意它是由亮到暗還是由暗到亮。

12. 細胞直方圖:把影像切成不重疊的小細胞(cell,典型為 8×8 像素)。每個細胞內的所有像素，依自己的方向投票到 9 個方向區間(每區間 20 度)，投票的權重是該像素的梯度大小——邊緣愈強，發言權愈大。這一步正是第 5 章直方圖思想的再次運用：用統計分布取代逐點數值，因此對細胞內的小幅位移與變形天生容忍。

13. 區塊正規化:把相鄰的 2×2 個細胞合成一個區塊(block,即 16×16 像素)，對區塊內 4 個直方圖串成的 36 維向量做正規化(常用 L2-Hys:L2 正規化後裁切再正規化)。這一步是 HOG 強健性的關鍵——局部正規化讓描述子對光照強弱與對比變化幾乎無感，呼應第 5 章「先正規化再比較」的原則。區塊以半個區塊(8 像素)為步幅滑動，彼此重疊，讓同一個細胞在不同的正規化脈絡下被重複描述，進一步提升穩定度。

14. 串接成向量:把所有區塊的正規化向量首尾相接，就是整個視窗的 HOG 描述子。



HOG 的四個步驟，以及那個 3,780 維是怎麼來的。先用第 9 章的 Sobel 求每個像素的梯度大小與方向，方向取「無號」——把 0 到 360 度摺疊到 0 到 180 度，因為對外形而言，一條邊是「由亮到暗」還是「由暗到亮」並不重要，重要的是它的走向。接著把視窗切成 8×8 的細胞，細胞內每個像素依自己的方向投票到 9 個方向區間，票數以梯度大小加權（強邊緣的話語權比較大）。然後把相鄰 2×2 個細胞合成一個區塊做正規化，最後全部串起來。維度的算法：區塊以一個細胞為步幅滑動，水平方向有 64÷8 − 2 + 1 = 7 個、垂直 128÷8 − 2 + 1 = 15 個，共 105 個區塊;每個區塊含 2×2 個細胞、每個細胞 9 個方向，所以 105 × 4 × 9 = 3,780 維，與實測的 3,780 維相符。請注意區塊是重疊的——每個細胞會出現在最多四個區塊裡，被正規化四次。這個重疊看似浪費，卻是 HOG 強健性的來源之一，下一張圖會解釋。

圖 13-15　HOG 的四個步驟，與那個 3,780 維是怎麼來的

以行人偵測的標準視窗 64×128 像素計算：水平方向可放 64÷8−2+1 = 7 個區塊，垂直方向 128÷8−2+1 = 15 個區塊，共 105 個區塊;每區塊 4 個細胞、每細胞 9 個方向，於是維度為 105×4×9 = 3,780 維。這個數字值得記住——它說明了手工特徵的代價：為了描述一個小小的 64×128 視窗，我們產生了近四千維的向量，而且這套規則是人訂的，換一個任務就得重新調整。

HOG 與 SIFT 的關係很值得玩味：兩者骨子裡都是「梯度方向直方圖」。差別在於 SIFT 在關鍵點上稀疏取樣，並且把直方圖對齊到關鍵點的主方向以取得旋轉不變性;HOG 在固定視窗上稠密取樣，而且刻意不做方向對齊——因為行人就是直立的，方向資訊本身就是重要線索，對齊反而會把它丟掉。這也直接說明了 HOG 的侷限：它不具旋轉不變性，而尺度問題則靠「影像金字塔 + 滑動視窗」在外層解決(第 11 章的多解析度思想)。

完整的 HOG 偵測器長這樣：建立影像金字塔，在每一層用固定大小的視窗逐格

滑動，對每個視窗算 HOG 並丟給一個線性 SVM(第 14 章)判斷「是/不是行人」，最後以非極大值抑制合併重疊的框。這條管線的每一個零件，在第 16 章的 YOLO 裡都還在——只是「人工設計的 HOG + SVM」被換成了「自動學出的卷積特徵 + 神經網路輸出頭」，而且不再需要滑動視窗。理解了 HOG，再看 YOLO 就不會覺得它是魔法。

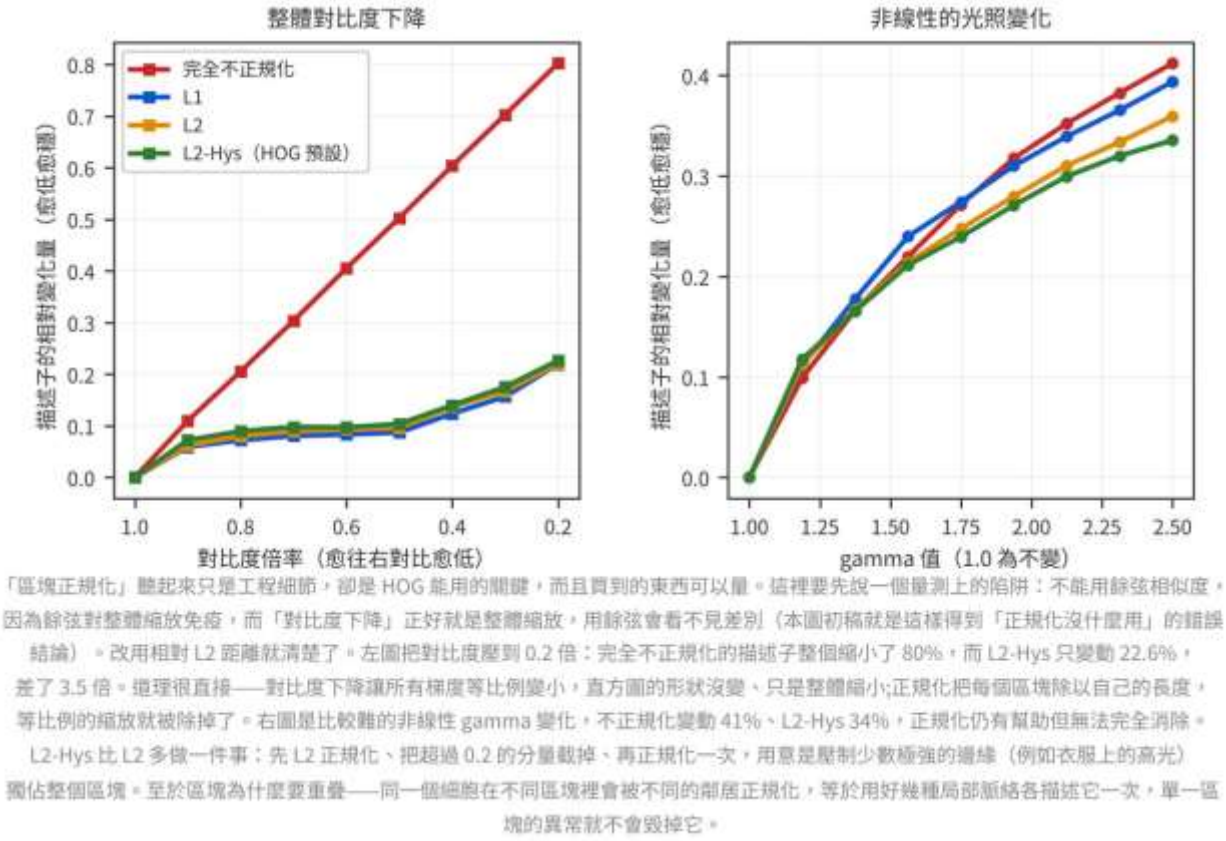


圖 13-16　區塊正規化買到了什麼：對比度一降就見真章

13.5.3 LBP：區域二值模式與紋理描述

如果 HOG 描述的是「外形」，那麼區域二值模式(Local Binary Pattern,LBP)描述的就是「表面」。它由芬蘭奧盧大學的 Timo Ojala 等人於 1996 年提出、2002 年在 IEEE TPAMI 上完成理論化，是紋理分析領域影響最深遠的方法之一，至今仍是表面瑕疵檢測(AOI)最具性價比的工具。

LBP 的做法樸素到令人意外：取一個 3×3 鄰域，以中心像素的值當門檻，八個鄰居只問一個問題——「你比中心亮嗎?」亮(大於或等於)記 1，暗記 0。依固定順序把

八個答案串成一個八位元二進位數，就得到 0 到 255 之間的一個碼。整張影像每個位置都算一次，再統計這些碼的直方圖，這個直方圖就是該區域的紋理描述。

這個樸素的設計帶來一個極為珍貴的性質：對單調遞增的灰階變換完全不變。因為 LBP 只比較大小、不使用數值差，只要影像的灰階經過任何「愈亮仍愈亮」的對應(例如 Gamma 校正、線性調亮、整體降對比)，每一組比較的結果都不會翻轉，LBP 碼一模一樣。本書以嚴格遞增的灰階對應表實測驗證：變換前後的 LBP 影像完全相同;即使改用會產生併階的直方圖等化，仍有 99.2% 的像素碼不變。這正是 LBP 在光照多變的產線與戶外場景下格外可靠的原因。

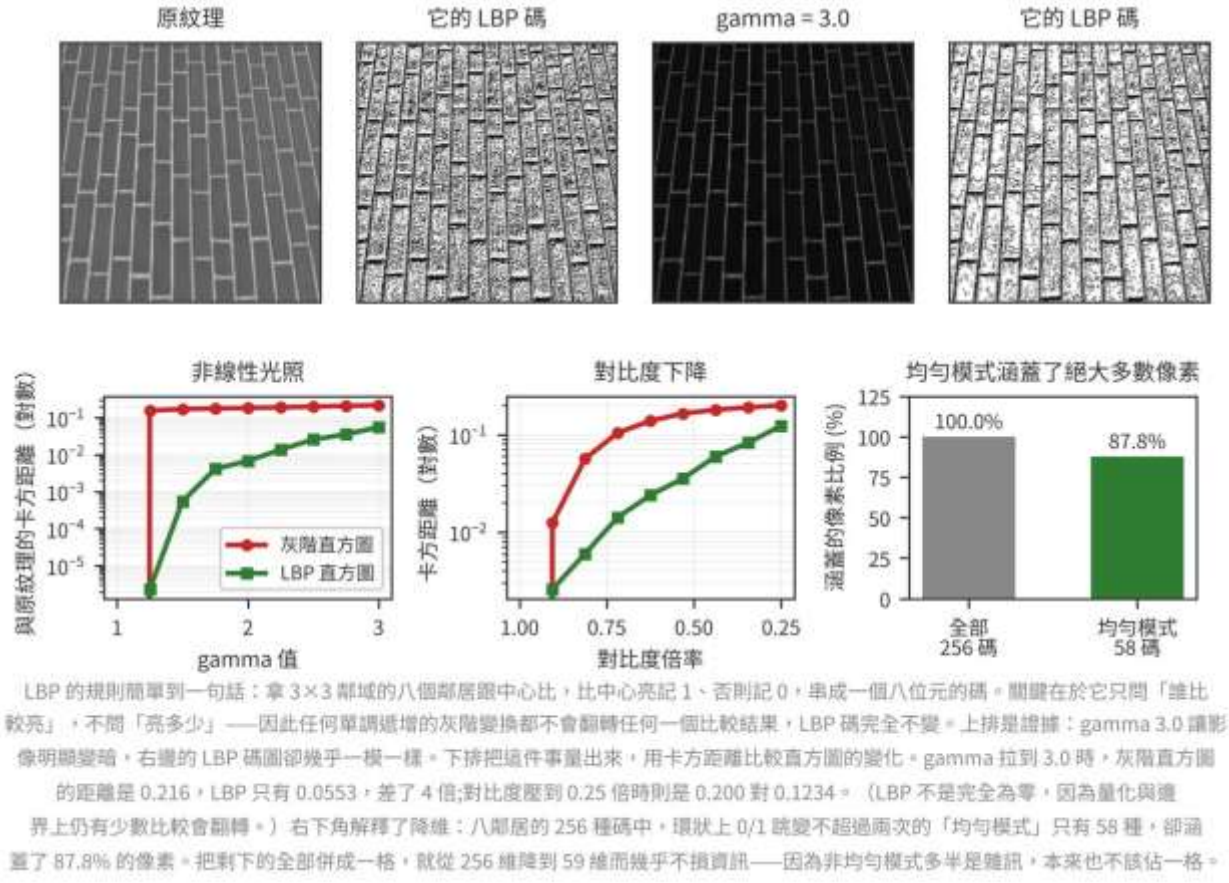
256 維的直方圖偏大，而且其中許多碼其實很罕見。Ojala 等人進一步發現，把八個位元看成一個環，若環上「0 與 1 的跳變次數」不超過兩次，稱為均勻模式(uniform pattern)。直覺上，均勻模式對應的正是自然影像裡真正有意義的局部結構——平坦面(全 0 或全 1)、邊緣、線端、角點;而跳變頻繁的碼多半來自雜訊。在八鄰居的情況下，256 個碼中共有 58 個均勻碼(僅佔 22.7%)，卻能覆蓋自然紋理中約九成的像素。於是標準做法是：58 個均勻碼各給一個區間、其餘全部併為一個「非均勻」區間，共 59 維——維度降到不足四分之一，資訊幾乎不損。

LBP 還有兩個常見的延伸。其一是旋轉不變 LBP：把八位元的環往同一方向循環位移，取所有位移中數值最小者作為代表，於是整塊紋理轉動時碼不變(八鄰居時可壓到 10 維)。其二是多尺度 LBP(P,R)：改成在半徑 R 的圓周上等距取 P 個點，非整數座標處用第 4 章的雙線性內插取值;把 LBP(8,1)、LBP(16,2)、LBP(24,3) 的直方圖串接起來，就能同時描述細、中、粗三種尺度的紋理。

與第 6 章的 Gabor 濾波器對照，可以看出兩條不同的路：Gabor 從頻率域出發，用「特定頻率 + 特定方向」的帶通濾波器去量測紋理的週期與走向;LBP 從空間域出發，用局部大小比較去量測紋理的微結構。Gabor 的物理意義漂亮但運算昂貴(一組濾波器往往要數十次卷積)，LBP 只有比較與查表，便宜到可以在最廉價的嵌入式處理器上即

時執行——這也是第 17 章在談邊緣部署時值得記住的取捨。

在應用上，LBP 的經典戰場有三個：表面瑕疵檢測(把工件表面切成網格，逐格算 LBP 直方圖，與良品樣本比較距離，異常格即為可疑瑕疵)、材質與紋理分類(布料、木紋、路面)，以及人臉辨識(把人臉切成若干區塊、各算直方圖再串接，是深度學習之前的主流方法之一)。第 14 章的農產品與零件分級案例所說的「以紋理特徵判斷表面瑕疵」，用的就是 LBP。



LBP 的規則簡單到一句話：拿 3×3 鄰域的八個鄰居跟中心比，比中心亮記 1、否則記 0，串成一個八位元的碼。關鍵在於它只問「誰比較亮」，不問「亮多少」——因此任何單調遞增的灰階變換都不會翻轉任何一個比較結果，LBP 碼完全不變。上排是證據：gamma 3.0 讓影像明顯變暗，右邊的 LBP 碼圖卻幾乎一模一樣。下排把這件事量出來，用卡方距離比較直方圖的變化。gamma 拉到 3.0 時，灰階直方圖的距離是 0.216，LBP 只有 0.0553，差了 4 倍;對比度壓到 0.25 倍時則是 0.200 對 0.1234。（LBP 不是完全為零，因為量化與邊界上仍有少數比較會翻轉。）右下角解釋了降維：八鄰居的 256 種碼中，環狀上 0/1 跳變不超過兩次的「均勻模式」只有 58 種，卻涵蓋了 87.8% 的像素。把剩下的全部併成一格，就從 256 維降到 59 維而幾乎不損資訊——因為非均勻模式多半是雜訊，本來也不該佔一格。

圖 13-17　LBP 的光照不變性：只問誰比較亮，不問亮多少

13.5.4 形狀特徵：從輪廓到 Hu 不變矩

第三類特徵的前提與前兩類不同：它假設物件已經被切出來了。輸入是第 9 章門檻化或分割、再經第 10 章型態學清理與連通元件標記之後得到的二值遮罩或輪廓。既然已經知道「物件在哪」，接下來要問的就是「它長什麼樣」。

最基本的量測有三個：面積 A(遮罩內的像素數)、周長 P(輪廓的長度)，以及外接

矩形的寬 w 與高 h。但這三個量都帶著單位、都隨拍攝距離改變，不能直接拿來比較。真正好用的是由它們組合出來的無因次比值——因為比值會把尺度約掉，天生就對距離遠近與影像縮放不敏感：

15. 圓形度(circularity)4πA/P²:衡量輪廓有多「緊緻」。完美圓形為 1.000，正方形為 0.785，3:1 的長矩形為 0.589;值愈小，代表輪廓愈狹長或愈曲折。這是區分「圓孔/圓形零件」與「長條狀物件」最快的一刀。
16. 長寬比(aspect ratio)w/h:最直觀的外形指標。13.7 節車牌偵測用「約 3:1 到 4:1」篩選候選區域，用的就是這個量。
17. 填充率(extent)A/(w·h):物件填滿其外接矩形的比例。實心矩形接近 1，十字形或 L 形零件則明顯偏低。
18. 凸實度(solidity)A/A 凸包:物件面積與其凸包面積之比。完整的凸形物件接近 1，一旦邊緣有缺角、崩邊或凹口就會明顯下降——這使它成為瑕疵檢測中最便宜的一道判準。
19. 等效直徑 √(4A/π):與該物件同面積的圓的直徑，便於把不規則物件換算成一個「代表尺寸」。

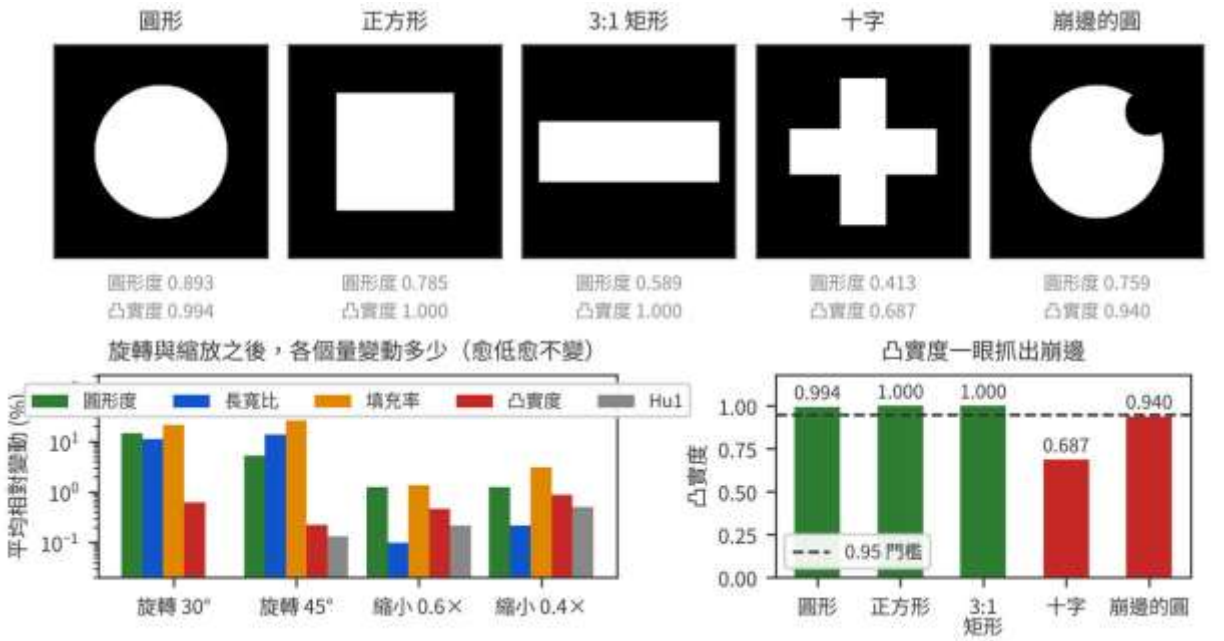
要更進一步、要能真正「辨認形狀」，就需要矩(moments)。空間矩定義為 m(p,q) = Σ x^p y^q I(x,y)，它把物件的質量分布對座標做加權統計。由此可得質心 (m10/m00, m01/m00);若把座標原點平移到質心再計算，得到的中心矩 μ(p,q) 自然就與物件的位置無關——取得平移不變性。再把中心矩除以適當次方的 μ00，得到正規化中心矩 η(p,q) = μ(p,q) / μ00^((p+q)/2+1)，尺度的影響也被約掉了——取得尺度不變性。

剩下的旋轉不變性最難。1962 年，胡明寬(Ming-Kuei Hu)在一篇經典論文中，以二階與三階的正規化中心矩巧妙組合出七個量，證明它們在平移、縮放與旋轉之下都保持不變——這就是著名的 Hu 不變矩。由於七個值的數量級差異極大(動輒跨越十

幾個數量級)，實務上一律先取 $-\text{sign}(h)\cdot\log_{10}|h|$ 壓縮動態範圍，再拿來比較或餵給分類器。

本書以一個不規則五邊形實測驗證了這些性質：把它平移之後，七個值完全相同;放大 1.8 倍或旋轉 37 度，在取對數的尺度下前四項的差異都不超過 0.02。還有一個有趣的性質值得記住：第七個不變矩在鏡像(左右翻轉)時會變號而絕對值不變，因此它可以用來區分一對互為鏡像的形狀——例如左旋與右旋的螺絲、左右手的手套。

Hu 不變矩也有清楚的侷限。高階矩對雜訊與輪廓的量化誤差相當敏感，對高度對稱的形狀還會出現數值接近零而不穩定的情況，所以實務上多半只採用前三到四項。若需要更強的形狀辨識能力，常見的升級路線有二：改用輪廓的傅立葉描述子(把輪廓視為一維週期訊號做 DFT，正是第 6 章的思想用在輪廓上)，或者乾脆把遮罩交給 CNN(第 15 章)去學。OpenCV 的 matchShapes 函式即以 Hu 矩為基礎，可直接回報兩個輪廓的形狀距離，值愈小愈相似。



形狀特徵的設計重點是「無因次」——用比值把單位約掉，量到的才是形狀本身，而不是拍攝距離。圓形度 $4\pi A/P^2$ 就是典型：面積與周長的平方都是長度平方，相除後單位消失。實測本圖五個形狀：圓形 0.893、正方形 0.785、3:1 矩形 0.589、十字 0.413，正方形與 3:1 矩形與理論值 0.785、0.589 完全相符，但圓形量到 0.893 而不是理論的 1.000——這不是公式錯了，是離散化的後果：圓在像素網格上是鋸齒狀的，沿著鋸齒走一圈量到的周長比真正的圓周長了約 6%，而周長在分母又被平方，誤差就被放大了。這是所有以周長為分母的指標共通的毛病，實務上若要精確，應先對輪廓做多邊形近似或平滑再量。左下是不變性的實測：把每個形狀旋轉與縮小之後，各個量變動多少。縮小 0.4 倍時，圓形度變動 1.2%、凸實度 0.9%、而 Hu1 變動 0.5%。要注意長寬比對旋轉完全不是不變量（旋轉 45 度時變動 14%），因為它用的是「與座標軸對齊」的外接矩形——這正是為什麼車牌偵測在斜拍時要先做透視校正。右下示範凸實度的用途：完整的凸形物件接近 1，而崩邊的圓只有 0.940，一刀就切出來了。瑕疵檢測常用它而不用圓形度，因為小缺口對周長的影響有限，對凸包面積的影響卻很直接。

圖 13-18　形狀特徵：哪些量真的不變，哪些只是看起來不變

13.5.5 三類手工特徵的選用指南

把本章至此介紹的特徵整理成一張表，可以看清它們各自的位置與適用場合。

表 13-2 三類手工特徵的比較

比較面向	關鍵點描述子	區域描述子	形狀特徵
代表方法	Harris、SIFT、ORB	HOG、LBP、Gabor	圓形度、凸實度、Hu 不變矩
取樣方式	稀疏:少數關鍵點	稠密:固定視窗全覆蓋	以單一物件為單位
輸出形式	每個點一個描述子	每個視窗一個向量	每個物件一組數值
主要不變性	尺度、旋轉	光照(LBP 最強)	平移、尺度、旋轉
需先分割嗎	不需要	不需要	需要:第 9、10 章的輸出

典型用途	拼接、定位、追蹤	行人偵測、紋理與瑕疵	零件量測與幾何篩選
下游接什麼	RANSAC 幾何求解	第 14 章的分類器	門檻篩選或分類器

三者並非互斥，實務上常常串成一條管線。以一條零件外觀檢測線為例：先用第 9 章門檻化與第 10 章型態學取出候選物件 → 用形狀特徵(面積、圓形度、凸實度)做幾何篩選，一秒之內淘汰明顯不合格與誤檢的區塊 → 對通過的物件逐格算 LBP 直方圖判斷表面是否有刮痕與鏽蝕 → 再加上第 8 章的顏色直方圖判斷色差 → 把這些向量串接起來交給第 14 章的 SVM 做最終分級。這正是第 14 章分級案例所用的特徵組合，也是台灣機械與電子業產線上最常見的做法：便宜、可解釋、樣本需求少。當產品換線時只要重新訓練分類器，不必重新標註上萬張照片。

最後要強調的是，這三類特徵並沒有被深度學習淘汰。在樣本稀少、算力受限、或必須向客戶解釋「為什麼判定不良」的場合，手工特徵往往比端到端的神經網路更合適——一個「凸實度低於 0.95 即判定崩邊」的規則，品保工程師看得懂、也願意簽名;而這恰好是 13.8 節要繼續討論的主題。

程式 13-4 HOG 與 LBP 特徵擷取

```
import cv2, numpy as np
from skimage.feature import local_binary_pattern

img = cv2.imread("person.jpg", cv2.IMREAD_GRAYSCALE)

# === HOG:方向梯度直方圖 ===
# 注意:cv2.HOGDescriptor 只存在於 OpenCV 4.x,
#      OpenCV 5 已移除其 Python 介面,直接呼叫會拋 AttributeError,
win = cv2.resize(img, (64, 128))      # 標準行人視窗
# hog = cv2.HOGDescriptor(    # 以下一律註解,改用後面的 skimage 版本
#         _winSize=(64, 128), _blockSize=(16, 16),
#         _blockStride=(8, 8), _cellSize=(8, 8),
#         _nbins=9)
# f_hog = hog.compute(win).ravel()
# print("HOG 維度:", f_hog.shape)  # (3780,)=105x4x9

# skimage 版本:結果完全等價,並可輸出方向場視覺化圖
from skimage.feature import hog as sk_hog
f2, vis = sk_hog(win, orientations=9,
                 pixels_per_cell=(8, 8),
                 cells_per_block=(2, 2),
                 block_norm="L2-Hys", visualize=True)
```

```
print("HOG 維度:", f2.shape)   # (3780,)=105x4x9,見 13.5.6 節

# === 內建的 HOG + SVM 行人偵測器:同樣只存在於 OpenCV 4.x ===
# det = cv2.HOGDescriptor()  # 5.x 需改裝 opencv-contrib-python
# det.setSVMDetector(
#     cv2.HOGDescriptor_getDefaultPeopleDetector())
# boxes, _ = det.detectMultiScale(img,       # 影像金字塔
#                                 winStride=(8, 8),
#                                 scale=1.05)
# print("偵測到的行人數:", len(boxes))

# === LBP:區域二值模式(紋理) ===
P, R = 8, 1                       # 8 鄰居、半徑 1
lbp = local_binary_pattern(img, P, R,
                           method="nri_uniform")
hist, _ = np.histogram(lbp, bins=59, range=(0, 59),
                       density=True)
print("均勻模式直方圖維度:", hist.shape)      # (59,)

# 旋轉不變版本:八鄰居時只剩 10 維
lbp_ri = local_binary_pattern(img, P, R,
                              method="uniform")
h_ri, _ = np.histogram(lbp_ri, bins=P + 2,
                       range=(0, P + 2), density=True)
print("旋轉不變 LBP 維度:", h_ri.shape)      # (10,)
```

程式 13-5 形狀特徵與 Hu 不變矩

```
import cv2, numpy as np, math

# 輸入:第 9 章分割 + 第 10 章型態學清理後的遮罩
mask = cv2.imread("part.png", cv2.IMREAD_GRAYSCALE)
_, bw = cv2.threshold(mask, 0, 255,
          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)

for c in cnts:
    A = cv2.contourArea(c)            # 面積
    if A < 200:                       # 濾掉雜點
        continue
    P = cv2.arcLength(c, True)        # 周長
    x, y, w, h = cv2.boundingRect(c)
    hull = cv2.convexHull(c)

    circ  = 4 * math.pi * A / (P * P)   # 圓形度 圓=1
    ratio = w / float(h)                # 長寬比
    ext   = A / float(w * h)            # 填充率
    sol   = A / cv2.contourArea(hull)   # 凸實度
    dia   = math.sqrt(4 * A / math.pi)  # 等效直徑
    print(f"面積{A:7.0f} 圓形度{circ:.3f} "
          f"長寬比{ratio:5.2f} 填充率{ext:.3f} "
          f"凸實度{sol:.3f} 等效直徑{dia:6.1f}")
```

```
    # --- Hu 不變矩:平移/尺度/旋轉不變 ---
    hu = cv2.HuMoments(cv2.moments(c)).ravel()
    # 取對數壓縮動態範圍;高階項不穩,實務多取前 3~4 項
    hu_log = [-np.sign(v) * math.log10(abs(v))
              if abs(v) > 1e-20 else 0.0 for v in hu]
    print("   log Hu:", np.round(hu_log, 3))

# 以 Hu 矩直接比較兩個輪廓的形狀(值愈小愈相似)
if len(cnts) >= 2:
    d = cv2.matchShapes(cnts[0], cnts[1],
                        cv2.CONTOURS_MATCH_I1, 0)
    print("形狀距離:", round(d, 4))
```

13.5.6 推導：HOG 的維度怎麼算，以及區塊為什麼要重疊

HOG 的 3,780 維常被當成一個要背的數字，其實它完全可以推出來，而推導的過程會順便解釋兩個設計決定。

先算區塊數。視窗 64×128，細胞 8×8，所以水平有 64÷8 = 8 行細胞、垂直有 128÷8 = 16 列細胞。區塊由 2×2 個細胞組成，並以「一個細胞」為步幅滑動，因此水平方向能放 8 − 2 + 1 = 7 個區塊，垂直方向 16 − 2 + 1 = 15 個，共 7 × 15 = 105 個區塊。每個區塊含 4 個細胞、每個細胞 9 個方向，所以總維度是 105 × 4 × 9 = 3,780。圖 13-15 用 skimage 實際算出來的也正是 3,780 維，兩者相符。

注意步幅是「一個細胞」而不是「一個區塊」，這表示區塊是重疊的——相鄰兩個區塊共用一半的細胞。若不重疊，區塊數會變成 4 × 8 = 32 個，維度只有 1,152 維。多出來的 2,628 維買到了什麼?答案是：同一個細胞會出現在最多四個區塊裡，被四組不同的鄰居各正規化一次。萬一某個區塊裡剛好有一道強光或一條高反差的邊，它會壓低該區塊裡其他細胞的相對權重;但同一個細胞在另外三個區塊裡仍有正常的表現，所以單一區塊的異常不會毀掉這個細胞的描述。用冗餘換強健，是很典型的工程交易。

再看正規化本身。圖 13-16 的實測必須先避開一個陷阱：不能用餘弦相似度來比較，因為餘弦對整體縮放免疫，而「對比度下降」正好就是整體縮放，用餘弦會完全看不見差別。改用相對 L2 距離之後結論就清楚了：把對比度壓到 0.2 倍時，完全不正規化的描述子整個縮小了 80%，而 L2-Hys 只變動 22.6%，差了 3.5 倍。道理很直接

——對比度下降讓所有梯度等比例變小，直方圖的形狀其實沒變、只是整體縮小，而正規化把每個區塊除以自己的長度，等比例的縮放就被除掉了。

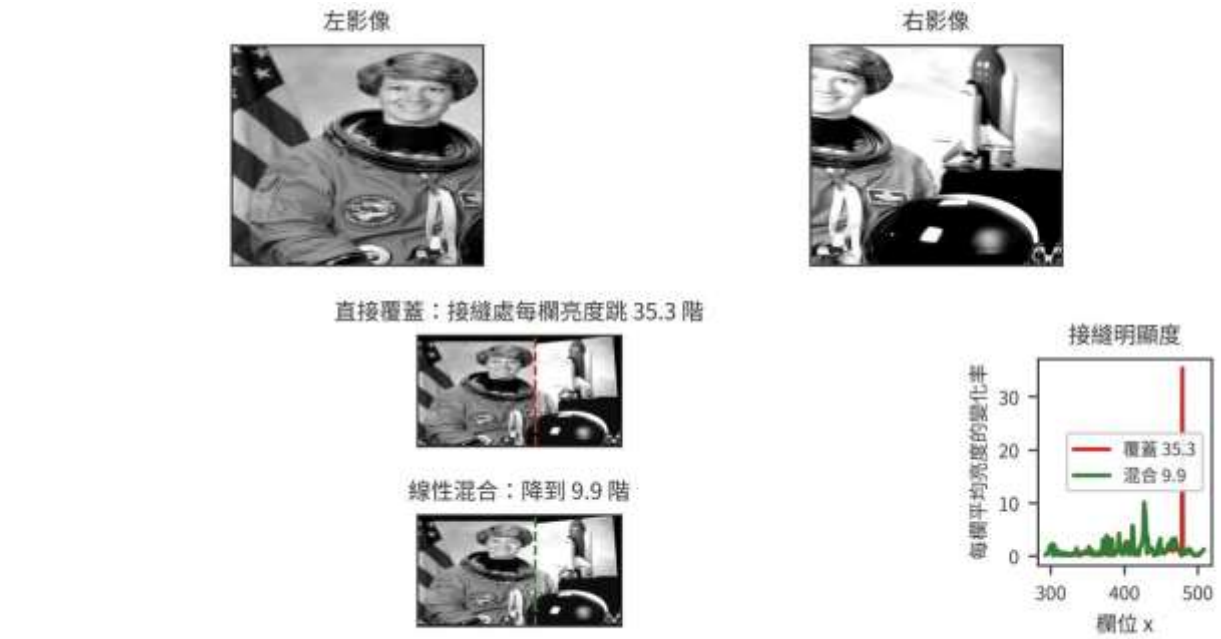
L2-Hys 這個名字裡的 Hys 是 hysteresis（遲滯）：先做 L2 正規化，把任何超過 0.2 的分量截斷成 0.2，再做一次 L2 正規化。用意是壓制少數極強的梯度獨佔整個區塊的描述——例如衣服上的一塊高光或背景的一根電線桿，它們的梯度可能比人的輪廓強上好幾倍，若不截斷，整個區塊的描述就會被它主導，人的外形反而被稀釋掉。這個「把過強的證據打折」的想法，與第 5 章 CLAHE 的對比度限制如出一轍。

最後補一個與 SIFT 的對照，因為兩者的骨幹其實是同一個：梯度方向直方圖。三個關鍵差異是：取樣上 SIFT 稀疏（只在關鍵點）、HOG 稠密（固定網格全覆蓋）;方向上 SIFT 先量主方向再對齊、HOG 刻意不對齊;輸出上 SIFT 每個關鍵點一個 128 維向量、數量不固定，HOG 每個視窗一個固定長度的向量。其中「HOG 為什麼不做方向對齊」最值得想一想：因為它要辨識的是「站著的行人」，把每個區塊都轉正，等於丟掉了「這個人是直立的」這個最重要的線索。不變性不是愈多愈好——該對什麼不變，是由任務決定的。

13.6 應用：影像拼接、模板比對與物件定位

13.6.1 影像拼接(全景圖)

全景照片是特徵匹配最迷人的應用。原理：對兩張有重疊區域的照片各自擷取特徵 → 匹配並用 RANSAC 求出它們之間的透視轉換(第 4 章)→ 把一張影像依此轉換「拉」到與另一張對齊的位置 → 融合重疊區域(消除接縫)。手機相機的「全景模式」、Google 街景的環景圖，骨幹都是這條流程。它綜合了本章(特徵匹配)、第 4 章(透視轉換)、第 6 章(相位相關可做初始對齊)、第 11 章(多頻帶融合可消接縫)。

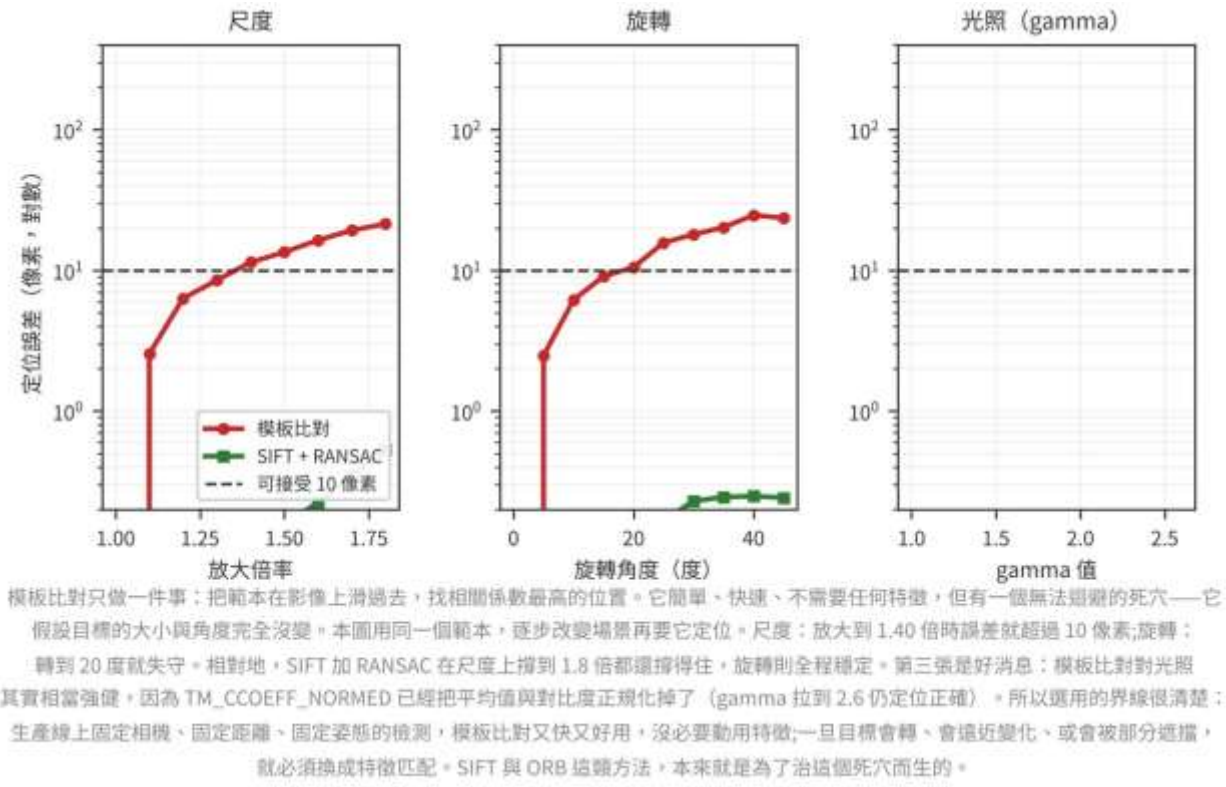


影像拼接把本章前面所有東西串成一條線，而且每一步都不能省。先用 SIFT 在兩張影像各取特徵、做最近鄰匹配，比值測試留下 220 組，RANSAC 再認可其中 209 組為內點，並由它們解出一個單應矩陣（第 4 章的透視轉換，8 個自由度，所以最少要 4 組點）。把右圖依這個矩陣扭到左圖的座標系，兩張就對齊了。但對齊不等於拼好——最後一步的「融合」常被初學者忽略。中間上方是直接覆蓋。本圖刻意讓右影像的曝光高一些（增益 1.22、偏移 14，模擬兩次拍攝的自動曝光差異），於是即使幾何完全正確，接縫處仍留下一條明顯的亮度斷層，實測這道階梯讓每欄平均亮度在一瞬之內跳了 35.3 階。中間下方做了最簡單的線性混合——在重疊區讓權重由 1 平滑降到 0。量的時候要小心：混合會把斷層攤開到整個重疊區，若只量接縫那幾欄，會得到「混合沒有用」的錯誤結論（本圖初稿就犯了這個錯），而且不能量局部梯度——重疊區本來就有高對比的影像內容，量到的會是內容而不是接縫。改成對整欄取平均再看變化率，影像內容被平均掉，剩下的就是曝光不一致造成的那道階梯。結果：尖峰從一欄跳 35.3 階降到 9.9 階，而且不再是一根尖刺，而是攤平在整個重疊區的一片小丘。實務上還有更好的做法：找一條沿著「兩張差異最小」的路徑來切（接縫雕刻），或在多個頻帶上分別混合（拉普拉斯金字塔融合，又是第 11 章的多解析度）。但觀念是同一個：讓過渡發生在人眼不容易察覺的地方。

圖 13-19　影像拼接：對齊之後，還有融合這一步

13.6.2 模板比對與其侷限

模板比對(Template Matching)是最樸素的「找物件」方法：拿一個小範本(template)在大影像上逐位置滑動，計算每個位置的相似度，相似度最高處就是物件所在。OpenCV 的 matchTemplate 提供此功能。它簡單直接，但侷限明顯：對尺度與旋轉極度敏感——範本一旦與目標的大小或角度不符就會失敗。這恰恰反襯出特徵匹配(SIFT/ORB)的價值：後者的尺度與旋轉不變性，正是為了克服模板比對的死穴而生。何時用哪個?目標大小角度固定(如生產線上位置固定的工件)用模板比對即可;目標會變大變小變角度，就需要特徵匹配。



模板比對只做一件事：把範本在影像上滑過去，找相關係數最高的位置。它簡單、快速、不需要任何特徵，但有一個無法迴避的死穴——它假設目標的大小與角度完全沒變。本圖用同一個範本，逐步改變場景再要它定位。尺度：放大到 1.40 倍時誤差就超過 10 像素;旋轉：轉到 20 度就失守。相對地，SIFT 加 RANSAC 在尺度上撐到 1.8 倍都還撐得住，旋轉則全程穩定。第三張是好消息：模板比對對光照其實相當強健，因為 TM_CCOEFF_NORMED 已經把平均值與對比度正規化掉了（gamma 拉到 2.6 仍定位正確）。所以選用的界線很清楚：生產線上固定相機、固定距離、固定姿態的檢測，模板比對又快又好用，沒必要動用特徵;一旦目標會轉、會遠近變化、或會被部分遮擋，就必須換成特徵匹配。SIFT 與 ORB 這類方法，本來就是為了治這個死穴而生的。

圖 13-20　模板比對的死穴：尺度與旋轉，以及它其實不怕的東西

13.6.3 物件定位

程式 13-3 已示範了完整的物件定位：給定一張「目標物件」的參考圖與一張「場景」圖，透過特徵匹配加 RANSAC，不僅能判斷場景中「有沒有」該物件，還能精確框出它「在哪裡、以什麼角度出現」——即使物件被部分遮擋(SIFT 論文指出，少至三個正確匹配的特徵就足以定位物件)。這是深度學習物件偵測(第 16 章)普及之前，工業界物件辨識與定位的主力方法，至今在樣本稀少、不便訓練深度模型的場合仍有一席之地。

13.7 案例深探：車牌偵測

車牌偵測是一個絕佳的綜合案例，它把本書多章的技術串成一條完整的實用管線，也是您在課程中反覆演練的經典題目。完整的車牌辨識(License Plate Recognition,LPR)分為「偵測定位」與「字元辨識」兩大階段，本節聚焦前者(偵測)，字元辨識則銜接第 10 章(字元分割)與第 15 章(字元分類)。

13.7.1 傳統車牌偵測管線

傳統方法充分利用車牌的視覺特性——矩形、高對比、字元密集(高頻紋理豐富)。典型流程：

20. 前處理：轉灰階、以第 5 章的方法增強對比、去雜訊。
21. 邊緣與紋理分析：車牌區域因字元密集而有大量垂直邊緣。用第 9 章的 Sobel 求垂直梯度，車牌區域會呈現密集的邊緣響應，凸顯出候選區域。
22. 型態學處理：用第 10 章的閉合運算把密集的邊緣連成一整塊矩形區域，再用斷開去除雜點。
23. 候選區域篩選：用第 10 章的連通元件標記找出候選區塊，依車牌的長寬比(約 3:1 到 4:1)、面積、填充率等幾何條件過濾掉不像車牌的區域。
24. 透視校正：若車牌是斜拍的，用第 4 章的透視轉換把它「拉正」成正面矩形，方便後續字元分割。

這條管線是第 4、5、9、10 章的完美綜合，展示了傳統影像處理如何以「一連串簡單步驟的巧妙串接」解決真實問題——這正是叫獸開講中 Canny、Viola-Jones 都體現過的工程智慧。

13.7.2 特徵匹配在車牌辨識中的角色

特徵匹配(本章主題)在車牌辨識中也有用武之地：例如以特徵匹配比對車牌上的地區標誌或特殊符號、或在多影格影片中追蹤同一張車牌(結合第 12 章)。而在現代系統中，整個偵測階段愈來愈多改用第 16 章的 YOLO——直接訓練一個「車牌偵測器」，一步到位框出車牌，對傾斜、模糊、光線變化的強健度遠勝傳統管線。這再次呼應全書的主軸：傳統方法(可解釋、不需訓練、樣本少也能用)與深度學習(強健、需資料與算力)各擅勝場，成熟的工程師懂得依場景選擇，甚至混用。

程式 13-6 傳統車牌偵測管線

```
import cv2, numpy as np
```

```
img = cv2.imread("car.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. 前處理(第 5 章)
gray = cv2.bilateralFilter(gray, 11, 17, 17)   # 保邊去雜訊

# 2. 垂直邊緣:車牌字元密集,垂直邊緣豐富(第 9 章)
edges = cv2.Sobel(gray, cv2.CV_8U, 1, 0, ksize=3)
_, edges = cv2.threshold(edges, 0, 255,
           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3. 型態學:把密集邊緣連成矩形塊(第 10 章)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (17, 5))
closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)

# 4. 連通元件 + 幾何條件篩選候選車牌(第 10 章)
cnts, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)
for c in cnts:
    x, y, w, h = cv2.boundingRect(c)
    ratio = w / float(h)
    if 2.5 < ratio < 5.0 and w > 80:      # 車牌長寬比約 3~4:1
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
        plate = gray[y:y+h, x:x+w]        # 裁出車牌供字元分割(第 10 章)

cv2.imwrite("plate_detected.png", img)
```

13.8 從手工特徵到學習特徵

本章介紹的 Harris、SIFT、ORB 與 HOG、LBP、形狀特徵，都屬於手工設計特徵(hand-crafted features)——它們是人類專家憑藉對影像的深刻理解，精心設計出來的偵測與描述規則。在深度學習興起前的二十年，手工特徵是電腦視覺的絕對主流，SIFT 更是其中的皇冠。

2012 年後，深度學習帶來了典範轉移(這正是第 15 章的主題)：卷積神經網路不再依賴人類設計特徵，而是從海量資料中自動學習特徵。但這裡有一個深刻且美麗的連結——第 6.6.7 節已經預告過：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣偵測器與方向性帶通濾波器，與人工設計的 Gabor、SIFT 的梯度方向分析驚人地相似!換言之，深度學習並沒有推翻手工特徵的智慧，而是「重新發明」並「自動化」了它——網路透過梯度下降，自己學會了人類專家花了數十年才設計出的東西，並且能組合出更複雜、更抽象的高層特徵。

因此，學習本章絕非學習「過時的技術」。理由有三：其一，理解手工特徵，才能真正理解 CNN 學到了什麼、為什麼有效——手工特徵是深度特徵的「白箱版本」。其二，在樣本稀少、算力受限、需要可解釋性的場合(許多工業與機器人場景)，手工特徵至今仍是最務實的選擇。其三，現代許多系統是混合式的：用深度學習做高層理解、用手工特徵(如 ORB)做即時的幾何對齊與定位(例如許多 SLAM 系統的核心仍是 ORB 特徵)。手工特徵與學習特徵不是取代關係，而是各據所長、相互補充——這也是本章作為「傳統視覺」與「現代視覺」橋梁的意義所在。

13.9 本章與全書的觀念連結總表

相關章節	共用的觀念	特徵擷取中的角色
第 4 章 幾何轉換	透視轉換、單應性	RANSAC 求轉換做物件定位與影像拼接
第 5 章 空間域強化	高斯模糊、直方圖、正規化	SIFT 尺度空間、方向直方圖、描述子正規化
第 6 章 頻率域	帶通、Gabor、DoG	SIFT 的 DoG 即帶通;Gabor 為紋理特徵
第 8 章 色彩處理	顏色直方圖、紋理	LBP 判斷表面瑕疵;顏色直方圖為區域描述子
第 9 章 影像分割	梯度、Sobel、LoG	角點與 SIFT 建立於梯度;DoG 近似 LoG
第 10 章 二值影像	型態學、連通元件	車牌偵測的候選區域生成與篩選
第 11 章 小波	多解析度分析	SIFT 尺度金字塔與小波多尺度同源
第 12 章 視訊處理	光流、追蹤、視訊穩定	Shi-Tomasi 供光流追蹤;特徵匹配做穩定
第 14 章 機器	SVM、特徵工程	HOG/LBP/形狀特徵即傳統機器學習的輸

學習		入
第 15、16 章 深度學習	CNN 卷積核、物件偵測	CNN 首層重現手工特徵;YOLO 取代傳統偵測
第 17 章 邊緣部署	即時性、SLAM	ORB 輕量，為機器人 SLAM 的核心特徵

表 13-3 第 13 章與全書其他章節的觀念連結

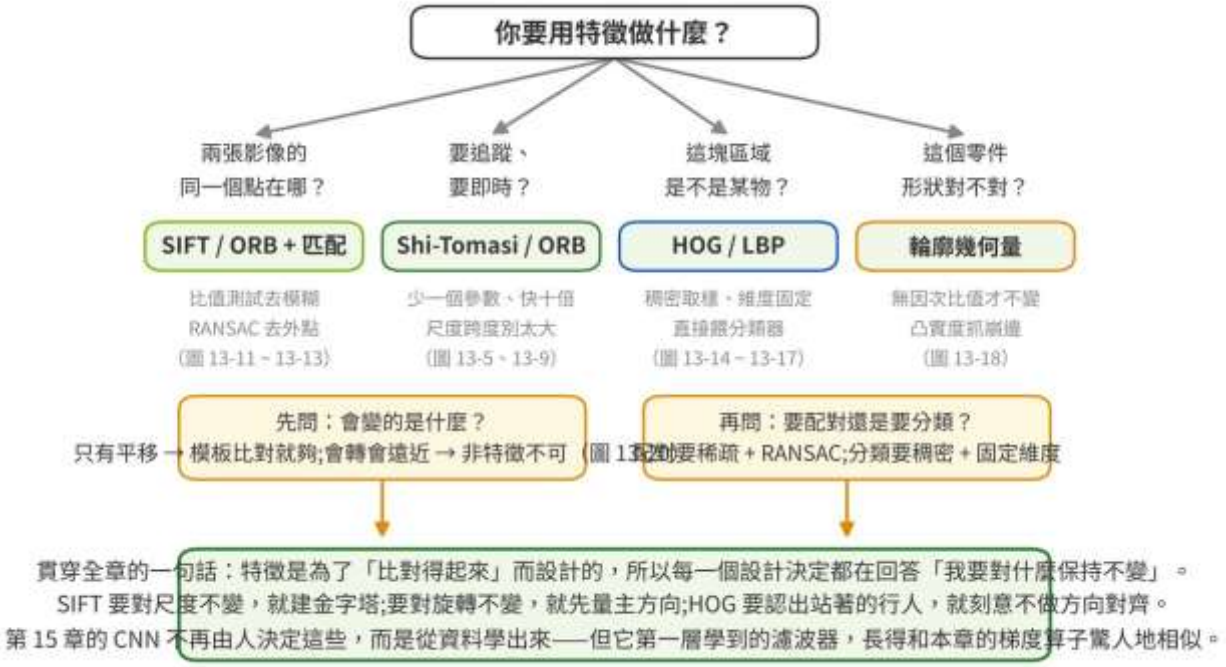


圖 13-21　第 13 章的決策地圖

程式實作 Lab 13

Lab 13-1 角點偵測與特徵比較

目標：建立對特徵的直覺。(a)對同一張影像分別套用 Harris、Shi-Tomasi、SIFT、ORB，比較各自偵測到的關鍵點分布——它們都偏好落在什麼樣的區域?(b)把影像放大兩倍、旋轉 45 度、調暗，重新偵測，量化比較哪種方法的「重複性」最好(同一特徵在變換後仍被偵測到的比例);(c)驗證角點偵測不具尺度不變性：放大影像後，Harris 角點是否消失?SIFT 是否仍找得到?(d)討論：為什麼平坦區與邊緣上偵測不到角點?這與第 12 章孔徑問題有何關聯?

Lab 13-2 物件定位器

目標：實作程式 13-3 的完整物件定位。(a)拍一張小物件(如書本封面、商標)作為「目標」，再拍幾張把它放進雜亂場景(不同角度、部分遮擋)的照片作為「場景」;(b)用 SIFT/ORB 加 BF 匹配加比值測試加 RANSAC，在場景中框出目標的位置與方向;(c)測試極限：目標旋轉多少、遮擋多少、縮放多少時開始失敗?(d)比較 SIFT 與 ORB 的定位成功率與速度(量測 FPS);(e)討論：比值測試的門檻(0.7、0.75、0.8)如何影響匹配的數量與品質?RANSAC 剔除了多少外點?

Lab 13-3 全景圖拼接

目標：做出自己的全景照片。(a)手持相機水平平移，拍攝三到五張有重疊(建議重疊三成以上)的照片;(b)相鄰兩張擷取特徵、匹配、RANSAC 求透視轉換，把後一張拉到與前一張對齊，逐步拼接;(c)處理接縫：比較「直接覆蓋」與「重疊區漸變融合」的視覺差異(進階可用第 11 章的多頻帶融合);(d)直接用 OpenCV 的 cv2.Stitcher 對照你的手工結果;(e)討論：什麼情況下拼接會失敗(重疊太少、場景無紋理、有移動物件)?

Lab 13-4 HOG 與 LBP 的手感實驗

目標：親手驗證兩種區域描述子的性質。(a)對一張含行人的照片，用程式 13-4 輸出 HOG 的方向場視覺化圖，觀察人體輪廓是否真的在方向場中浮現;(b)用內建的 HOG+SVM 偵測器跑同一張照片，再把照片旋轉 30 度後重跑，說明偵測率為何明顯下降——這驗證了 HOG 不具旋轉不變性;(c)對一張紋理照片(布料、木紋或金屬表面)計算 59 維均勻模式 LBP 直方圖;(d)把同一張照片做 Gamma 校正與線性調亮後重算 LBP，比較兩個直方圖的差異，驗證 LBP 對單調灰階變換的不變性;(e)取兩種不同材質各五張照片，以直方圖距離(如卡方距離)判斷「同材質是否真的比較近」。

Lab 13-5 零件形狀量測與幾何篩選

目標：建立一條最便宜的瑕疵篩選管線。(a)拍攝一批同款零件(或以硬幣、墊片、螺帽代替)，刻意混入數個有缺角、崩邊或變形的樣本;(b)以第 9 章門檻化加第 10 章型

態學取出每個物件的輪廓;(c)用程式 13-5 計算面積、圓形度、長寬比、填充率、凸實度與等效直徑，整理成一張表;(d)觀察哪一個指標對「缺角」最敏感(提示：先看凸實度)，訂出一組門檻值，計算你的篩選規則在這批樣本上的誤判率;(e)對同一個零件分別拍攝正放、放大與旋轉 45 度三張照片，計算 Hu 不變矩並比較前四項，驗證其不變性;(f)以 cv2.matchShapes 比較良品與不良品的形狀距離，討論這個單一數值能否取代你在 (d) 訂出的多條規則。

Lab 13-6 車牌偵測管線

目標：實作 13.7 節的傳統車牌偵測。(a)以程式 13-6 為基礎，對車輛照片完成「前處理→垂直邊緣→型態學→連通元件→幾何篩選」的完整管線，框出車牌;(b)調校型態學結構元素的尺寸與長寬比門檻，提高偵測率、降低誤報;(c)對斜拍的車牌，用第 4 章透視轉換拉正;(d)串接第 10 章：對拉正後的車牌做二值化與連通元件，分割出各個字元;(e)討論：傳統管線在什麼情況下失敗(強光、髒污、傾斜過大)?對照第 16 章，YOLO 車牌偵測會如何改善這些問題?各自的成本與適用場景為何?

本章重點整理

- 特徵是影像中值得注意、易辨認、可重複找到的局部結構;好特徵須具重複性、獨特性、不變性(尺度/旋轉/光照)。角點在兩方向都有變化，能唯一定位，是理想特徵(呼應第 12 章孔徑問題)。
- Harris 角點以視窗滑動的灰階變化與結構張量特徵值判斷：兩者都大即角點;Shi-Tomasi 取較小特徵值作強度，更適合追蹤，為 goodFeaturesToTrack 的預設;兩者皆不具尺度不變性。
- SIFT 四步驟(尺度空間 DoG 極值、關鍵點定位、方向指派、128 維描述子)達成尺度與旋轉不變，是第 5、6、9、11 章工具的集大成;ORB(FAST+BRIEF)免費、快數十倍、二進位描述子用漢明距離比對，適合即時與邊緣裝置。

- 匹配：BF 準確、FLANN 快速近似;Lowe 比值測試(最近/次近距離比)過濾模糊匹配;RANSAC 以「投票」思想反覆抽樣建模、剔除外點，精神與霍夫轉換一致。
- 應用：影像拼接(特徵匹配+透視轉換+融合)、模板比對(簡單但不具尺度旋轉不變性)、物件定位(匹配+RANSAC，可抗遮擋);車牌偵測綜合第 4、5、9、10 章，是傳統管線的典範。
- Harris/SIFT/ORB 為手工特徵;CNN 首層自動學出的濾波器與之高度相似，深度學習是「重新發明並自動化」了手工特徵。兩者互補：手工特徵可解釋、樣本少可用、即時(SLAM 用 ORB)，學習特徵強健、能學高層語意。
- HOG 以「無號梯度方向直方圖 + 區塊 L2-Hys 正規化」描述外形:8×8 細胞、9 個方向、2×2 細胞成區塊且重疊滑動;64×128 視窗得 105×4×9 = 3,780 維。與 SIFT 同為梯度方向直方圖，但稠密取樣且刻意不對齊方向，故不具旋轉不變性，尺度靠影像金字塔加滑動視窗解決;HOG+SVM 是 YOLO 之前行人偵測的標準答案。
- LBP 以 3×3 鄰域「比中心亮記 1」串成八位元碼，其直方圖即紋理描述。因只比大小不比數值，對單調遞增的灰階變換完全不變，光照強健性極佳;取跳變次數不超過兩次的 58 個均勻模式加一個併合區間，可由 256 維降到 59 維而幾乎不損資訊;另有旋轉不變與多尺度 LBP(P,R) 延伸。運算極廉，是表面瑕疵檢測的性價比首選。
- 形狀特徵以第 9、10 章切出的輪廓為輸入，用無因次比值取得尺度不變性:圓形度 $4\pi A/P^2$(圓 1.000、正方形 0.785、3:1 矩形 0.589)、長寬比、填充率、凸實度(對缺角最敏感)、等效直徑。矩的三級遞進為:中心矩得平移不變 → 正規化中心矩得尺度不變 → Hu 的七個組合得旋轉不變;實務取對數並只用前三到四項，第七項在鏡像時變號可區分左右形狀。

習題

1. 什麼是影像的「特徵」?為什麼用少數特徵比對比逐像素比對更好?請用一個生活比喻說明。

   提示與參考解答：特徵是影像中值得注意、易辨認、可重複找到的局部結構(如角點、獨特斑塊)。用少數特徵比對高效且強健，避免比對成千上萬像素。生活比喻：描述一個人不會逐一報告每個毛孔，而是說「濃眉、鷹勾鼻、左臉有痣」——用少數獨特標記就能精準辨認。

2. 好的特徵應具備哪三個條件?各舉一個「不符合」該條件的反例。

   提示與參考解答：重複性(不同條件下都能再次偵測到;反例：只在特定光線下出現的反光點)、獨特性(與眾不同易區分;反例：平坦牆面上的點，每點都一樣)、不變性(對尺度/旋轉/光照穩定;反例：Harris 角點放大後消失，不具尺度不變性)。

3. 為什麼角點是理想的特徵，而平坦區與邊緣不是?這與第 12 章的孔徑問題有何關聯?

   提示與參考解答：角點在兩個方向都有明顯灰階變化，能被唯一定位(重複+獨特)。平坦區任何方向移動都無變化，無法定位;邊緣沿邊緣方向移動無變化，只能定出垂直方向(一維模糊)。這正是第 12 章孔徑問題的體現——局部只能看出垂直於邊緣的運動/位置，唯有角點無此歧義。

4. 說明 Harris 角點的核心直覺，以及結構張量兩個特徵值的三種組合分別代表什麼。

   提示與參考解答：直覺：小視窗在影像上滑動，觀察灰階變化——平坦區任何方向都不變、邊緣僅垂直方向變、角點任何方向都劇烈變。結構張量兩特徵值：都小=平坦區，一大一小=邊緣，都大=角點。Harris 用不需實際算特徵值的響應函數 R 高效判斷。

5. Shi-Tomasi 相對 Harris 做了什麼改良?為什麼它更適合用於第 12 章的光流追蹤?

   提示與參考解答：Shi-Tomasi 直接取兩個特徵值中較小者作為角點強度，只要較小者夠大就保證兩方向都有足夠變化，判準更穩定。因此在追蹤場合表現更好，OpenCV 的 goodFeaturesToTrack 預設採用它，論文名〈Good Features to Track〉即點明與光流的血緣——這些角點是 Lucas-Kanade 稀疏光流理想的追蹤

點。

6. SIFT 如何達成尺度不變性與旋轉不變性?請分別說明對應的步驟。

   提示與參考解答：尺度不變性：建構高斯尺度金字塔(多解析度)，在「空間+尺度」三維空間找 DoG 極值——物體無論多大都能在對應尺度層被找到。旋轉不變性：分析關鍵點周圍梯度方向分布，指派主方向為基準，之後所有描述相對此基準計算——物體轉多少度，基準跟著轉，描述保持一致。

7. SIFT 的四個步驟分別用到本書前面的哪些概念?這說明了什麼學習上的道理?

   提示與參考解答：高斯模糊(第 5 章)、多尺度金字塔(第 11 章)、DoG 帶通/近似 LoG(第 6、9 章)、梯度(第 9 章)、方向直方圖(第 5 章)、描述子正規化抗光照(第 5 章)。說明 SIFT 是前面各章工具的集大成;循序學完基礎章節，是理解進階演算法的積木——基礎不牢，進階難懂。

8. ORB 為什麼比 SIFT 快?它由哪兩個技術融合而成?二進位描述子如何加速比對?

   提示與參考解答：ORB 融合 FAST 角點偵測器(極快)加方向資訊，與 BRIEF 二進位描述子(一連串「這點比那點亮嗎」的是非比較，產生 0/1 字串)加旋轉校正。二進位描述子比對時用漢明距離(算兩串位元有幾位不同)，可用 CPU 指令極速完成，遠快於 SIFT 的 128 維浮點歐氏距離運算。

9. 承叫獸開講：為什麼在 2020 年之前，免費的 ORB 儘管精度稍遜 SIFT 卻被廣泛採用?這對「技術選型」有何啟示?

   提示與參考解答：因為 SIFT 受專利保護(US 6,711,293,2020 年到期)，商業使用需授權;ORB 完全免費可自由商用。啟示：技術選型不只看效能，還要看授權、成本、生態系——工程決策不在真空中做，一個「夠好且無授權負擔」的方案常勝過「最強但受限」的方案(呼應第 11 章 JPEG vs. JPEG 2000)。

10. 說明 Lowe 比值測試的原理。為什麼「最近距離明顯小於次近距離」代表匹配可信?

    提示與參考解答：對每個特徵找出另一影像中最近與次近兩個匹配，若最近距離明顯小於次近(如小於 0.75 倍)則匹配可信，否則捨棄。原理：若一個特徵在對方影像中有唯一明確的對應，最近的會遠比次近的近;若最近與次近相當，代表有多個相似候選、難以區分，很可能匹配錯誤，應捨棄。

11. RANSAC 如何從充滿錯誤的匹配中找出正確的幾何模型?它與第 9 章的哪個演算法精神相同?

    提示與參考解答：RANSAC 反覆隨機抽取少數匹配、假設正確、據此算出幾何模型(如透視轉換)，再統計有多少其他匹配符合此模型;重複多次，得票最高(支持者最多)的模型勝出，不符者為外點剔除。與第 9 章霍夫轉換精神相同——都是「投票」：用多數證據對抗少數雜訊，是穩健估計的核心。

12. 模板比對的最大侷限是什麼?特徵匹配如何克服?各自適合什麼場合?

    提示與參考解答：模板比對對尺度與旋轉極度敏感，範本一旦與目標大小/角度不符就失敗。特徵匹配(SIFT/ORB)具尺度與旋轉不變性，正是為克服此死穴而生。場合：目標大小角度固定(如產線上位置固定的工件)用模板比對即可;目標會變大小變角度，需用特徵匹配。

13. (綜合題)請設計一條傳統車牌偵測管線，列出每一步用到本書哪一章的技術，並說明相對於第 16 章 YOLO 方案的優缺點。

    提示與參考解答：管線：前處理(第 5 章：灰階、去雜訊、增強)→ 垂直邊緣分析(第 9 章 Sobel，車牌字元密集邊緣多)→ 型態學閉合連成矩形塊、斷開去雜(第 10 章)→ 連通元件標記加長寬比/面積/填充率篩選(第 10 章)→ 透視校正拉正(第 4 章)→ 字元分割(第 10 章)。相對 YOLO：傳統管線可解釋、不需訓練資料、樣本少也能用、算力低;但對強光、髒污、大傾斜較不強健。YOLO 強健但需大量標註與 GPU。依場景與資源選擇或混用。

14. HOG 的四個計算步驟分別是什麼?為什麼第二步要用「直方圖」而不是直接保留每個像素的梯度方向?

    提示與參考解答：四步為(1)以 Sobel 求梯度大小與無號方向(0~180 度)、(2)在 8×8 細胞內以梯度大小為權重投票到 9 個方向區間、(3)把 2×2 細胞合為區塊並做 L2-Hys 正規化(區塊重疊滑動)、(4)串接成向量。用直方圖的理由是容忍度：直方圖只記錄「方向的分布」而不記錄「哪個像素是哪個方向」，因此細胞內的小幅位移、姿態與體型差異都不會改變統計結果，這與第 5 章用直方圖描述整體亮度分布的思想完全一致。

15. 一個 64×128 的行人視窗，在細胞 8×8、區塊 2×2 細胞、步幅 8 像素、9 個方向的設定下，HOG 描述子有幾維?請寫出計算過程，並說明這個數字反映了手工特徵的什麼代價。

    提示與參考解答：水平 64÷8−2+1 = 7、垂直 128÷8−2+1 = 15，共 105 個區塊;105×(2×2)×9 = 3,780 維。代價有二：其一是維度膨脹——描述一個小視窗就要近四千個數字，計算與儲存都不便宜;其二是這套規則(細胞大小、方向數、正規化方式)全由人訂，換一個任務往往要重新試誤調參，而第 15 章的 CNN 正是把這份工作交給資料與最佳化演算法。

16. 為什麼 LBP 對光照變化特別強健?什麼是均勻模式?為什麼取 58 個均勻碼加一個併合區間就能從 256 維降到 59 維而幾乎不損資訊?

    提示與參考解答：LBP 只問「鄰居是否比中心亮」，只比較大小、不使用灰階差值，因此任何單調遞增的灰階變換(Gamma、線性調亮、整體降對比)都不會翻轉比較結果，碼完全不變。均勻模式指八位元環上 0 與 1 的跳變次數不超過兩次的碼，它們對應平坦面、邊緣、線端與角點等真正有意義的局部結構，八鄰居時共 58 個(佔 256 碼的 22.7%)，卻覆蓋自然紋理約九成的像素;跳變頻繁的非均勻碼多半來自雜訊，彼此之間的區別意義不大，故全部併為一個區間即可。

17. 圓形度定義為 $4\pi A/P^2$。請說明它為什麼是無因次量，計算正方形與 3:1 矩形的圓形度，並說明在零件檢測中「凸實度」為何常比圓形度更適合用來發現崩邊。

    提示與參考解答：面積的單位是長度平方、周長的平方也是長度平方，相除後單位互相約掉，故為無因次量，對影像縮放與拍攝距離不敏感。正方形邊長 a:$4\pi\cdot a^2/(4a)^2 = \pi/4 \approx 0.785$;3:1 矩形(300×100):$4\pi\cdot 30000/(2\times 400)^2 \approx 0.589$。圓形度衡量的是整體輪廓的緊緻程度，一個小缺角對面積與周長的影響都很小，訊號容易被淹沒;凸實度 A/A 凸包 則是拿物件與「補滿凹處後的自己」相比，缺角、崩邊與凹口會直接造成凸包面積增加而使比值下降，因此靈敏度高得多。

18. Hu 不變矩如何逐步達成平移、尺度與旋轉不變?第七個不變矩有什麼特殊性質?為什麼實務上通常只使用前三到四項?

    提示與參考解答：三級遞進——把原點平移到質心得到中心矩 $\mu(p,q)$，位置的影響

消失(平移不變);再除以適當次方的 μ00 得到正規化中心矩 η(p,q) = μ(p,q)/μ00^((p+q)/2+1)，尺度的影響消失(尺度不變);Hu(1962)再以二階與三階的 η 組合出七個特定式子，使其在旋轉下亦不變。第七項在鏡像(左右翻轉)時絕對值不變但正負號翻轉，因此可用來區分互為鏡像的形狀(如左旋與右旋螺絲)。只用前三到四項的原因是高階矩由高階統計量構成，對雜訊與輪廓的量化誤差極為敏感，對高度對稱的形狀還會逼近零而數值不穩。

19. (綜合題)請比較 HOG 與 SIFT：兩者的共同骨幹是什麼?三個關鍵差異是什麼?為什麼 HOG 刻意不做方向對齊?並說明第 16 章的 YOLO 保留了這條管線的哪些零件、取代了哪些。

    提示與參考解答：共同骨幹都是「梯度方向直方圖」。差異：(1)取樣——SIFT 稀疏(只在關鍵點)、HOG 稠密(固定視窗全覆蓋);(2)方向對齊——SIFT 對齊關鍵點主方向以取得旋轉不變性、HOG 不對齊;(3)用途——SIFT 用於跨影像配對(對應問題)、HOG 用於視窗分類(辨識問題)。HOG 不對齊是刻意的設計：行人在影像中幾乎總是直立，「肩線偏水平、軀幹兩側偏垂直」本身就是最有價值的判別線索，對齊會把它洗掉。YOLO 保留了「影像金字塔/多尺度、密集特徵、非極大值抑制」這些零件，取代掉的是「人工設計的 HOG 描述子 + 線性 SVM + 滑動視窗」，改為卷積自動學出的特徵加上一次前向傳遞直接回歸所有框。

20. 「好特徵要有重複性」要怎麼量?請設計一個實驗，說明為什麼評估不變性時要避開旋轉 90 度這種角度，以及為什麼 SIFT 在旋轉、雜訊、光照三項測試中都不是最好的，卻仍被稱為劃時代的方法。

    提示與參考解答：做法是把同一張影像以已知的轉換處理一次，再問「原圖找到的關鍵點，經過同樣的轉換之後，有多少比例能在新影像中找到 3 像素以內的夥伴」，這個比例就是重複性。因為轉換已知，正確答案完全可控。要避開 90 度是因為它只是把像素重新排列、沒有重新取樣，不產生插值誤差，重複性會回升到接近 100%，是不具代表性的特例。圖 13-2 實測：旋轉 40 度時 Harris 74%、Shi-Tomasi 74%、SIFT 66%。SIFT 不佔優並不矛盾——旋轉與光照的不變性，Harris 這類基於梯度的偵測器本來就具備（結構張量隨影像一起旋轉）。SIFT 真正的本事在尺度，而那是這三項測試沒有涵蓋的（見下一題）。

21. 要證明「SIFT 具尺度不變性而 Harris 沒有」，該用放大還是縮小來測?該量偵測器還是量匹配?請說明兩個選擇的理由，並寫出你預期的結果。

    提示與參考解答：兩個選擇都有陷阱。第一，要用縮小而不是放大：物體變遠時資訊是真的被聚合掉的，而放大只是內插，並沒有製造出新的尺度內容，測不到重點。第二，要量匹配而不是量偵測。若只量「偵測器還找不找得到同一個位置」，三種方法在縮放下的表現其實差不多，因為角點縮小之後仍然是角點、位置大致還在。真正的差別在描述子——它必須知道「要用多大的鄰域來描述這個點」，而固定大小的鄰域在縮放後看到的根本是不同的內容。圖 13-6 實測匹配正確率：縮到 0.33 倍時，Harris 配固定大小的區塊描述子只剩 15%，ORB 57%，而 SIFT 仍有 82%。所以「尺度不變」不變的不只是偵測，更是描述。

22. SIFT 用「高斯差」(DoG) 而不是直接算 LoG，為什麼這樣做是合理的?請從熱傳導方程式說明這個近似的來源，並說明它為 SIFT 省下了什麼。

    提示與參考解答：兩個寬度略不同的高斯相減，會得到一個中間正、周圍負的帶通核，也就是第 9 章的墨西哥帽（LoG）。嚴格的來源是熱傳導方程式 $\partial G/\partial\sigma = \sigma\cdot\nabla^2 G$，把左邊的微分換成差分即得 $G(k\sigma) - G(\sigma) \approx (k-1)\sigma^2\cdot\nabla^2 G$。圖 13-7 把兩邊實際算出來比對，相關係數高達 0.9941。省下的是運算：LoG 要做二階微分、成本高，而高斯模糊無論如何都要做（建尺度金字塔本來就要），相減則幾乎不花錢。SIFT 等於用一個「順手」的運算換到了昂貴的帶通濾波器，這是演算法設計上非常漂亮的一步。

23. 比值測試為什麼看「最近 ÷ 次近」而不是看「最近的距離夠不夠小」?若要為自己的資料挑門檻，該怎麼做?過濾前後的正確率大約會差多少?

    提示與參考解答：因為「距離夠小」沒有絕對標準——不同影像、不同紋理，描述子距離的尺度完全不同。而「最近比次近近多少」是相對量，自動適應了這個尺度。背後的道理：若一個特徵在對方影像中真的有唯一對應，最近鄰應該明顯勝過第二名;若第一名與第二名難分軒輊，代表這個特徵長得很普通（重複紋理、磚牆、樹葉），配哪一個都不可靠。挑門檻的做法是用已知的幾何轉換造一對影像，這樣每個匹配是對是錯都查得出來，再掃描門檻看精確率與召回率。圖 13-11 實測：原始 1,019 個匹配只有 61.1% 正確;門檻 0.75 留下 609 個，正確率升到 98.2%、召回率 96.0%;最佳門檻落在 0.76，與 Lowe 論文建議的 0.75 幾乎一致。

24. RANSAC 要抽幾次才夠?請推導 N = log(1−p) / log(1−w^s)，並用它說明三件事：(a) 為什麼比值測試必須先做;(b) 為什麼「能用仿射就別用單應」;(c) 當 RANSAC 常常失敗時，該先檢查什麼。

    提示與參考解答：設內點比例為 w、建模最少需 s 組匹配。一次抽樣全中的機率是 w^s，連抽 N 次都沒全中的機率是 (1−w^s)^N，要讓成功機率達 p，需 1 − (1−w^s)^N ≥ p，取對數整理即得該式。(a) 圖 13-11 實測比值測試把正確率從 61.1% 拉到 98.2%，代入公式（s=4、p=0.99），所需抽樣從 29 次降到 2 次——兩者是一前一後的搭配，不是二選一。(b) N 對 s 極度敏感：內點 50% 時 s=2 只要 16 次、s=4 要 71 次、s=8 要 1,177 次。自由度低的模型不只快，也不容易被外點誘導出「扭曲但碰巧符合」的解。(c) 該先檢查內點比例，而不是一味調高迭代次數——N 對 w 是指數式敏感（內點 20% 要 2,876 次、10% 要 46,049 次），把 w 拉高永遠比把 N 調高划算。詳見 13.4.4 節。

25. (綜合題)請比較 SIFT 與 HOG：兩者的共同骨幹是什麼?三個關鍵差異是什麼?為什麼 HOG 刻意不做方向對齊——這對「不變性是不是愈多愈好」有什麼啟示?

    提示與參考解答：共同骨幹都是「梯度方向直方圖」。三個差異：(1) 取樣——SIFT 稀疏（只在關鍵點，數量每張不固定）、HOG 稠密（固定網格全覆蓋，每個視窗維度固定）;(2) 方向——SIFT 先量主方向再對齊以取得旋轉不變，HOG 刻意不對齊;(3) 用途——SIFT 要解對應問題所以必須配對（比值測試 + RANSAC），HOG 要解辨識問題所以直接把固定長度的向量餵給分類器。「維度固定」這件事很關鍵：分類器要求每個樣本的維度一致，而關鍵點的數量每張影像都不同，沒辦法直接餵。HOG 不做方向對齊，是因為它要辨識的是「站著的行人」，把每個區塊都轉正等於丟掉了「這個人是直立的」這個最重要的線索。啟示：不變性不是愈多愈好。每多一分不變性，就少一分鑑別力——該對什麼不變，是由任務決定的，不是由演算法的優劣決定的。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 11(特徵擷取)。

- Lowe, D. G., Distinctive Image Features from Scale-Invariant Keypoints, IJCV, 60(2), 2004.(SIFT;US Patent 6,711,293，本章叫獸開講之史料來源)
- Harris, C., & Stephens, M., A Combined Corner and Edge Detector, Alvey Vision Conference, 1988.
- Shi, J., & Tomasi, C., Good Features to Track, CVPR, 1994.
- Rublee, E., et al., ORB: An Efficient Alternative to SIFT or SURF, ICCV, 2011.
- Fischler, M. A., & Bolles, R. C., Random Sample Consensus (RANSAC), Comm. ACM, 1981.
- Dalal, N., & Triggs, B., Histograms of Oriented Gradients for Human Detection, CVPR, 2005.
- Ojala, T., Pietikäinen, M., & Mäenpää, T., Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns, IEEE TPAMI, 24(7), 2002.
- Hu, M.-K., Visual Pattern Recognition by Moment Invariants, IRE Transactions on Information Theory, 8(2), 1962.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 14 章 機器學習基礎

本章學習目標

- 說明機器學習的定義與典範轉移，區分監督式、非監督式與強化學習。
- 理解影像分類的完整流程：資料、特徵、模型、訓練、評估。

- 認識重要的影像資料集(MNIST、CIFAR、COCO、ImageNet/ILSVRC)及其歷史角色。
- 操作 kNN 與 SVM 兩種經典分類器，理解其原理與適用場合。
- 正確切分訓練/驗證/測試集，以準確率、混淆矩陣、精確率與召回率評估模型。
- 診斷過度擬合與擬合不足，並理解偏差-變異權衡在機器學習中的核心地位。

叫獸開講|MNIST：讀懂支票的神經網路與七萬個手寫數字

1980 年代末，一位名叫 Yann LeCun 的年輕研究者來到美國紐澤西州的貝爾實驗室——那個誕生過電晶體、資訊理論、C 語言、Unix 的傳奇之地。他手上的任務很實際：讓機器學會辨認信封上手寫的郵遞區號，好讓美國郵政的信件自動分揀。手寫數字千變萬化，每個人的「7」都不一樣，傳統那套「人工設計規則」的方法在這裡撞牆了——你永遠寫不完「什麼樣算是 7」的規則。

LeCun 走了一條當時還很非主流的路：與其教電腦規則，不如給它大量範例，讓它自己「學」。1989 年，他和同事打造出第一個以反向傳播訓練的卷積神經網路 LeNet，成功辨識美國郵政的手寫郵遞區號。這個系統不斷改良，到 1990 年代初已經進化到能讀取銀行支票上的金額。這不是實驗室的玩具：搭載這套技術的支票辨識系統於 1990 年代初商業化部署，進入美國與歐洲的自動櫃員機;到了 1990 年代末，全美國約有百分之十以上的支票是由 LeCun 團隊的系統自動判讀的。一個學術演算法，悄悄地每天處理著數以百萬計的真實金錢交易。

但要訓練與比較這些「會學習」的系統，需要一套公平的考卷。LeCun 與同事 Corinna Cortes、Chris Burges 於是整理出了 MNIST 資料集：七萬張手寫數字影像(六萬張用於訓練、一萬張用於測試)，每張都是 28×28 像素的灰階圖，標好了 0 到 9 的正確答案。它的來源有個有趣的細節：MNIST 混合自美國國家標準局的兩個資料庫——一個由人口普查局員工書寫(字跡工整、較容易)，一個由高中生

書寫(字跡潦草、較困難)，刻意混合是為了讓訓練與測試的難度一致、結論才可靠。這個看似瑣碎的設計考量，其實蘊含了本章要教的重要觀念：資料的品質與切分方式，往往比演算法本身更決定成敗。

MNIST 從此成為機器學習界的「Hello World」——幾乎每一個學習視覺辨識的人，第一個訓練的模型都是 MNIST 手寫數字分類器，您在課程中的跨框架 MNIST 實作(從 TensorFlow 到 Keras 到 PyTorch)正是這個傳統的延續。它也是無數新演算法的第一塊試金石。一個為了解決郵政與銀行實際問題而生的資料集，意外地推動了整個領域三十多年——這印證了一句在機器學習界流傳的名言：「資料，才是驅動這一切的燃料。」

思考題：LeCun 沒有去寫「什麼樣算是數字 7」的規則，而是給機器看幾萬個 7 的範例讓它自己學。這個「從範例學習」而非「被明確告知規則」的思路，與你過去學程式的方式(工程師寫死每一條 if-else)有什麼根本不同?這個轉變為什麼被稱為「典範轉移」?

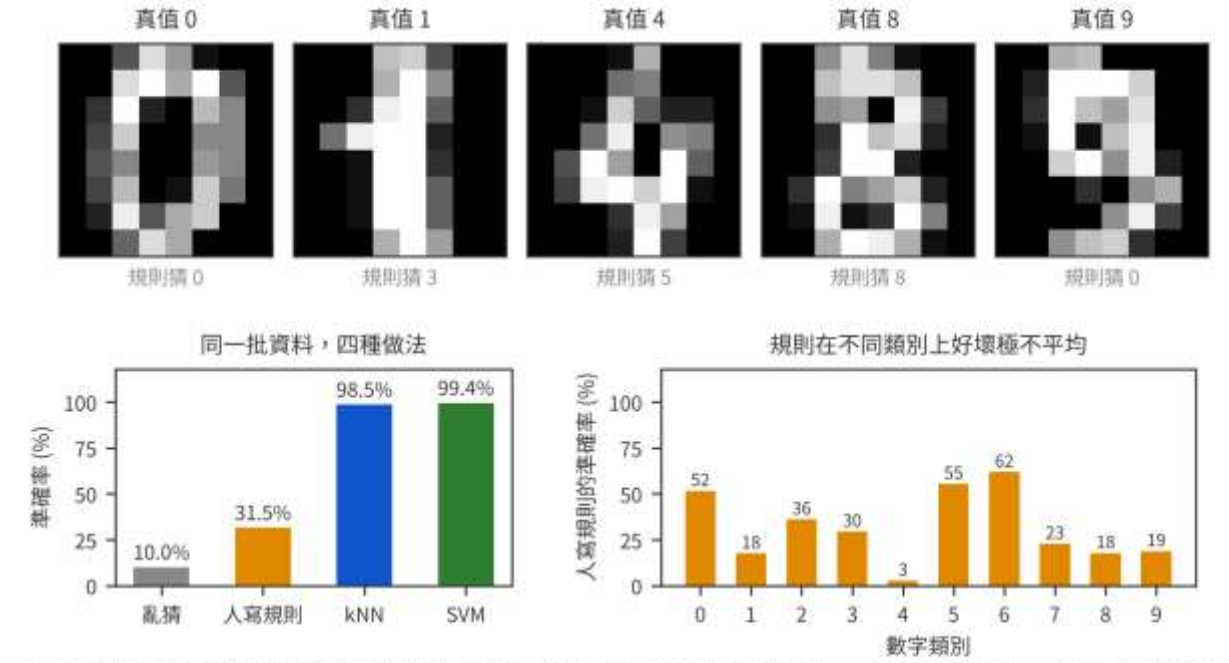
14.1 什麼是機器學習

14.1.1 典範轉移：從寫規則到給範例

回顧本書前十三章，我們解決問題的方式有一個共同點：人類專家分析問題、設計演算法。想去雜訊，我們設計濾波器;想找邊緣，我們設計 Canny;想描述特徵，我們設計 SIFT。這是「傳統程式設計」的典範——工程師明確地把「怎麼做」一步一步寫給電腦。

機器學習代表了一次根本的典範轉移(正是叫獸開講思考題的核心)。它不再由人類寫死規則，而是：給電腦大量的「輸入-輸出」範例，讓它自己從中歸納出規律。用一句話對比：傳統程式是「人給規則，電腦套用到資料上得出答案」;機器學習是「人給資料和答案，電腦自己歸納出規則」。這個轉變之所以重要，是因為有太多問題——例如辨認手寫數字、辨識人臉、理解語音——其規則複雜到人類根本寫不出來，但

範例卻俯拾皆是。既然寫不出規則，那就讓機器從範例中自己學。



這張圖把「典範轉移」四個字變成可以比較的數字。左下第二根是一個認真寫出來的規則式分類器：先取外接框把大小正規化，再數筆劃圍出幾個洞（8 有兩個，0、6、9 各有一個），接著看上下與左右的墨水投影分布、長寬比與整體密度，一共十餘條規則。這些規則都有道理，也都是人看得懂、改得動的，但它在 1,797 張手寫數字上只拿到 31.5%——比亂猜的 10% 好了三倍，卻遠不足以使用。同樣的資料交給 kNN 得到 98.5%、SVM 99.4%，而且程式碼比規則版短得多。右下角透露了規則式方法真正的死穴：它在各類別上的表現極不平均（最高 62%、最低 3%）。每修好一個類別，往往就弄壞另一個，因為所有規則共用同一組門檻。這正是機器學習存在的理由：當規則多到彼此衝突、而你又說不清楚「一個 5 到底長什麼樣」時，與其繼續寫規則，不如給它一萬個例子，讓它自己去歸納那條界線。

圖 14-1　寫規則與給範例：同一個問題，兩種做法

學術上常引用的定義來自 Tom Mitchell：「若一個電腦程式在某任務 T 上、以某效能指標 P 衡量的表現，能隨著經驗 E 而提升，則稱它從經驗 E 中學習。」以 MNIST 為例：任務 T 是辨認數字、效能 P 是辨識準確率、經驗 E 是那六萬張標好答案的訓練影像。看得愈多、學得愈好——這就是「學習」。

14.1.2 三種學習範式

1. 監督式學習(Supervised Learning)：訓練資料同時提供「輸入」與「正確答案(標籤)」，模型學習從輸入預測答案。本章與 MNIST 都屬此類——每張數字影像都附有正確的數字標籤。它又分為分類(輸出是類別，如「這是 3」)與迴歸(輸出是連續值，如「房價是多少」)。影像辨識絕大多數是分類問題。

2. 非監督式學習(Unsupervised Learning)：訓練資料只有輸入、沒有標籤，模型自己發現資料中的結構。例如分群(把相似的影像自動歸為一類，而沒人告訴它類別是什麼)、降維(把高維資料壓縮到低維以便視覺化或去除冗餘)。
3. 強化學習(Reinforcement Learning)：模型(稱為智能體 agent)透過與環境互動、依據行動的獎懲來學習策略，沒有現成的正確答案，只有「做得好不好」的回饋。它是機器人控制、遊戲 AI(如下圍棋的 AlphaGo)、自駕決策的核心，與機器人學程的關聯尤深——不過本章聚焦影像分類，強化學習留待進階課程。

本章與第 15、16 章的深度學習，主軸都是監督式學習中的影像分類與偵測。理解監督式學習的完整流程，是踏入深度學習的必要基礎——事實上，深度學習只是機器學習的一個分支(用「深層神經網路」作為模型)，它遵循的訓練、評估、過度擬合診斷等原則，與本章介紹的完全一致。先打好本章的地基，第 15 章的 CNN 才不會是空中樓閣。

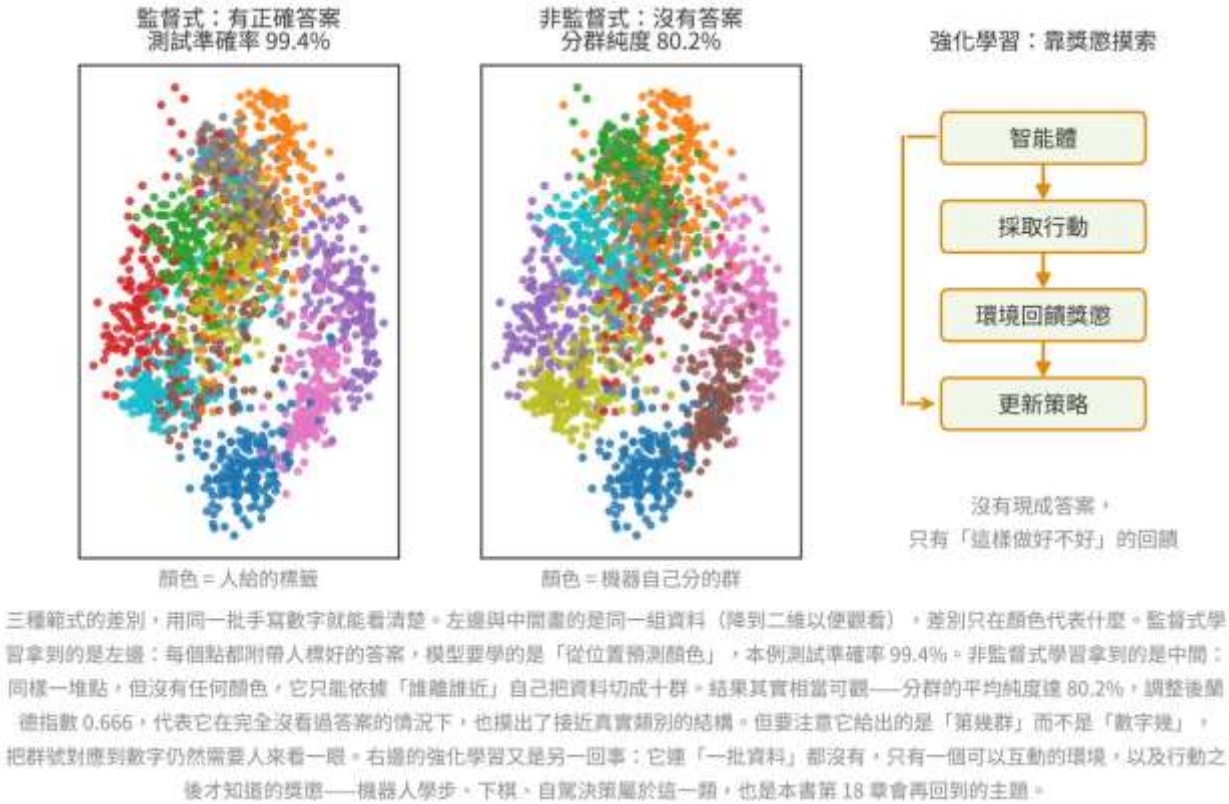


圖 14-2　三種學習範式：有標籤、沒標籤、靠獎懲

14.2 影像分類的完整流程

一個監督式影像分類系統，從資料到可用模型，要走過五個環節。這條流程貫穿本章與後續深度學習各章，務必牢記：

1. 資料(Data)：收集大量影像並標註正確答案。這是整個流程的地基——正如叫獸開講所示，資料的品質與數量往往比演算法更決定成敗。14.3 節專門討論資料集。
2. 特徵(Features)：把原始像素轉換成更有代表性的數值向量。傳統機器學習高度依賴人工設計特徵(第 13 章的 SIFT、HOG、顏色直方圖等)——這一步的好壞直接決定成敗。而深度學習(第 15 章)最大的突破，正是讓模型自動學出特徵，免去了人工設計的負擔。
3. 模型(Model)：選擇一個能從特徵映射到類別的數學模型，如本章的 kNN、SVM，或第 15 章的神經網路。
4. 訓練(Training)：用訓練資料調整模型的參數，使它在訓練資料上的預測盡量正確。
5. 評估(Evaluation)：用「模型沒看過的」測試資料檢驗它的真實表現——這是判斷模型好壞的關鍵，14.5 節詳述。

請特別注意「特徵」這一環，它是理解傳統機器學習與深度學習分野的鑰匙。在傳統流程中，特徵是人工設計的，模型只負責「從特徵到類別」這後半段;而深度學習把「從像素到特徵」與「從特徵到類別」整合進同一個網路，端到端一起學習——這就是第 13.8 節說的「從手工特徵到學習特徵」的典範轉移。本章聚焦傳統流程，正是為了讓你清楚看見：深度學習到底改變了哪一環、又保留了哪一環。

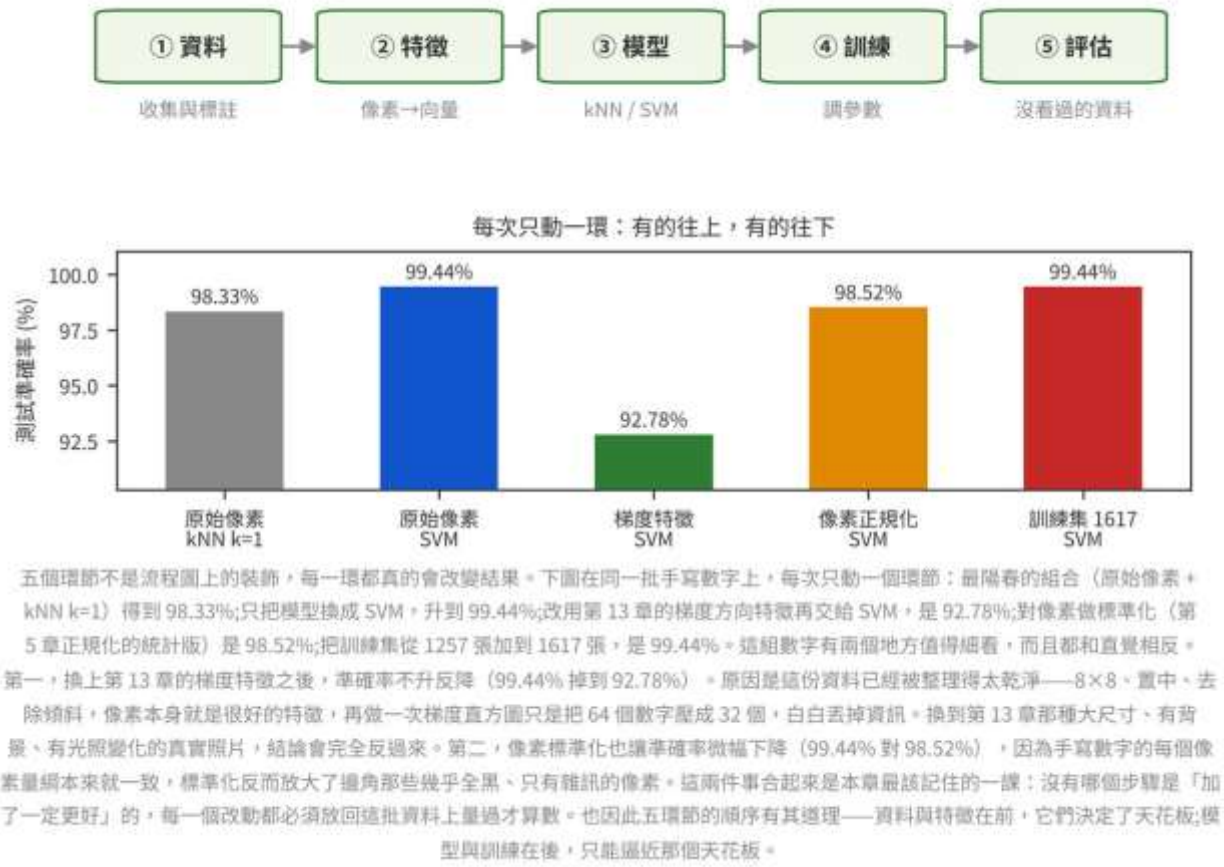
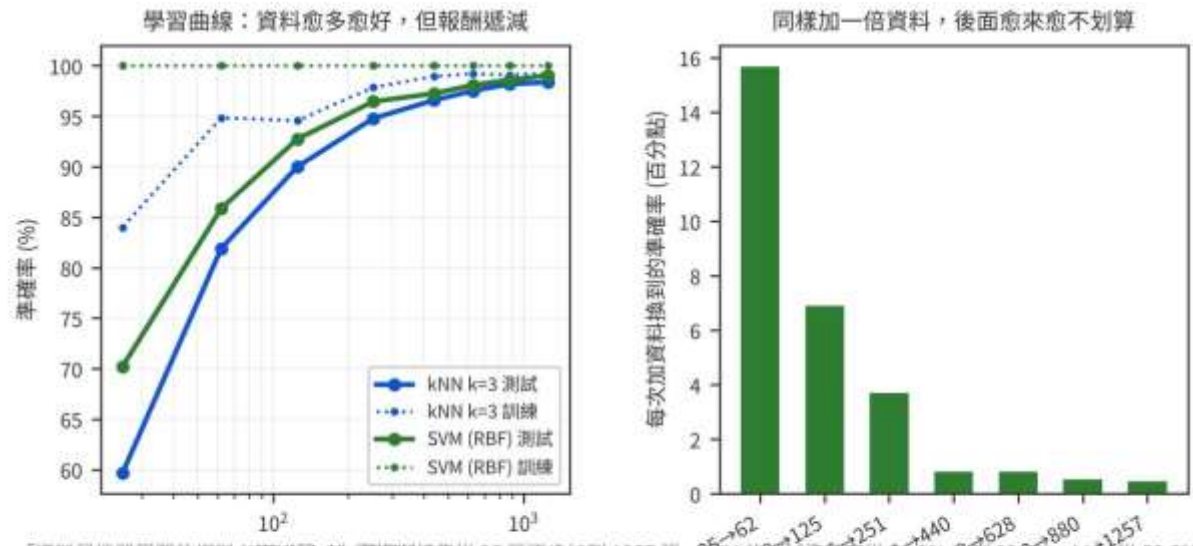


圖 14-3　五個環節：每一環各值多少分

14.3 影像資料集：機器學習的燃料

如果說演算法是引擎，那麼資料就是燃料——沒有燃料，再好的引擎也跑不動。機器學習的發展史，某種程度上就是一部「資料集推動演算法」的歷史。以下四個資料集，不僅是實作練習的素材，更是理解整個領域演進的關鍵座標。



「資料是機器學習的燃料」這句話在這張圖裡被具體化了。訓練集從 25 張逐步加到 1257 張，SVM 的測試準確率從 70.2% 到 98.4%，kNN 從 59.6% 升到 98.4%。注意虛線是訓練準確率：樣本很少時它接近 100%，與測試之間拉開一道大縫——那就是 14.6 節要談的過度擬合，而「加資料」正是最直接的解藥，兩條線隨樣本增加而靠攏。右圖是更實用的視角：每加一批資料換到幾個百分點。從 25 加到 62 張換到 15.7 個百分點，但從 880 加到 1257 張只換到 0.44 個百分點。這個報酬遞減有兩個實務意涵：一是專案初期砸資源標資料非常划算，二是到了曲線平坦處，再標十倍資料也不會有驚喜，該去改善的是特徵、模型，或是資料的「品質」而不是「數量」——例如補齊那些模型一直分錯的困難樣本。

圖 14-4　資料是燃料：訓練集要多大才夠

14.3.1 MNIST：入門的經典

承叫獸開講，MNIST 是七萬張 28×28 灰階手寫數字(六萬訓練、一萬測試)，分 0 到 9 共十類。它體積小、乾淨、易上手，是所有人學習機器學習的「Hello World」，也是您課程中跨框架實作的主角。它的優點(簡單)也是它的侷限：現代模型在 MNIST 上輕鬆達到 99% 以上準確率，已無法區分演算法的優劣，因此僅適合入門教學，不再用於嚴肅的演算法評比。

14.3.2 CIFAR-10 與 CIFAR-100：邁向真實

CIFAR 資料集由多倫多大學整理，是六萬張 32×32 的彩色自然影像。CIFAR-10 分十類(飛機、汽車、鳥、貓、狗、船等),CIFAR-100 則細分為一百類。相較 MNIST 的黑白數字，CIFAR 是彩色的真實世界物體，有複雜的背景與多變的視角，難度大幅提升，是驗證中小型模型的常用基準。從 MNIST 到 CIFAR，反映了資料集「由簡入繁、逼近真實」的演進軌跡。

14.3.3 ImageNet 與 ILSVRC：改變歷史的資料集

ImageNet 是一個劃時代的資料集，由史丹佛大學的李飛飛(Fei-Fei Li)團隊自 2009 年起建構，包含超過一千四百萬張影像、涵蓋兩萬多個類別，以人工標註。基於它舉辦的年度競賽 ILSVRC(ImageNet 大規模視覺辨識挑戰賽)，使用約一千個類別的子集，成為衡量視覺演算法的最高殿堂。

ImageNet 的歷史地位無可取代：2012 年，一個名為 AlexNet 的深度卷積神經網路在 ILSVRC 上以壓倒性差距奪冠，錯誤率大幅領先第二名，一舉引爆了深度學習革命(這正是第 15 章叫獸開講的主角)。可以說，沒有 ImageNet 這個規模空前的資料集，就沒有深度學習的爆發——它完美印證了「資料是燃料」的道理：是李飛飛「與其改進演算法、不如先建立夠大的資料集」的洞見，為 AlexNet 的成功鋪好了跑道。這也呼應本章的核心訊息：資料，常常比演算法更重要。

14.3.4 COCO：走向複雜場景

COCO(Common Objects in Context)由微軟推出，它的特點是「情境中的物件」：一張影像中包含多個物件、彼此有空間關係，且提供了比「這張圖是什麼」更豐富的標註——每個物件的精確位置(邊界框)、像素級輪廓(分割遮罩)、甚至人體關鍵點。因此 COCO 不只用於分類，更是物件偵測、實例分割、姿態估計的標準基準，直接銜接第 16 章的 YOLO。從 ImageNet(單物件分類)到 COCO(多物件偵測與分割)，又是一次「由簡入繁」的演進。

資料集	規模與尺寸	主要任務	歷史角色
MNIST	7 萬張，28×28 灰階	手寫數字分類	機器學習的 Hello World
CIFAR-10/100	6 萬張，32×32 彩色	自然物體分類	中小型模型基準
ImageNet	逾 1400 萬張	大規模分類(ILSVRC)	2012 引爆深度學習革命
COCO	逾 30 萬張	偵測、分割、姿態	複雜場景基準，接第 16 章

表 14-1 重要影像資料集比較

14.4 兩個經典分類器：kNN 與 SVM

14.4.1 k 最近鄰(kNN)

kNN 是最直觀的分類器，它的哲學是「物以類聚」：要判斷一個新樣本屬於哪一類，就看它在特徵空間中最靠近的 k 個鄰居，由這些鄰居「投票」決定——多數鄰居屬於哪一類，新樣本就歸為哪一類。例如 k=5，新樣本最近的五個鄰居中有三個是貓、兩個是狗，就判為貓。

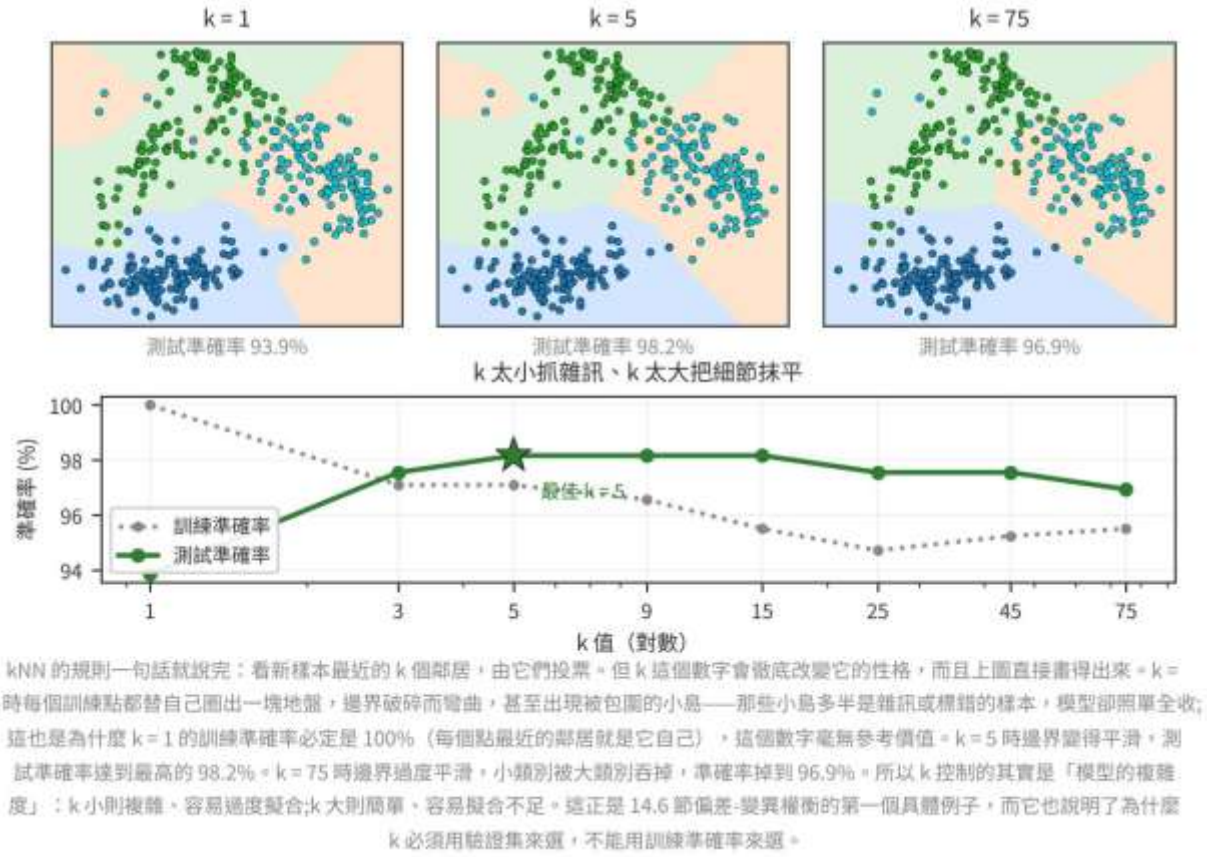
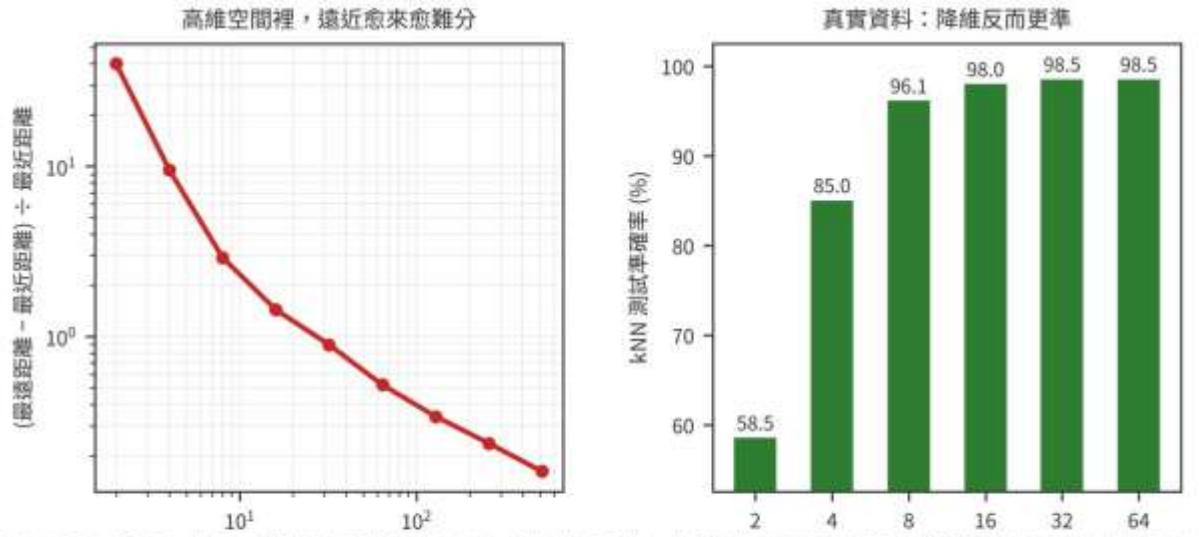


圖 14-5　kNN 的決策邊界與 k 值

kNN 有一個奇特的性質：它「不訓練」——沒有需要學習調整的參數，只是把所有訓練樣本記住，預測時才逐一計算距離找鄰居。這使它實作極簡，但也帶來缺點：預測慢(每次都要與所有訓練樣本比距離)、記憶體耗費大(要存全部訓練資料)、且對特徵的尺度與維度敏感(高維空間中「距離」會失去意義，稱為維度災難)。k 值的選擇是

關鍵：k 太小容易受雜訊影響，k 太大則會把界線模糊掉——這又是一個貫穿全書的取捨。kNN 常作為理解分類概念的第一站，以及新問題的簡單基準線。

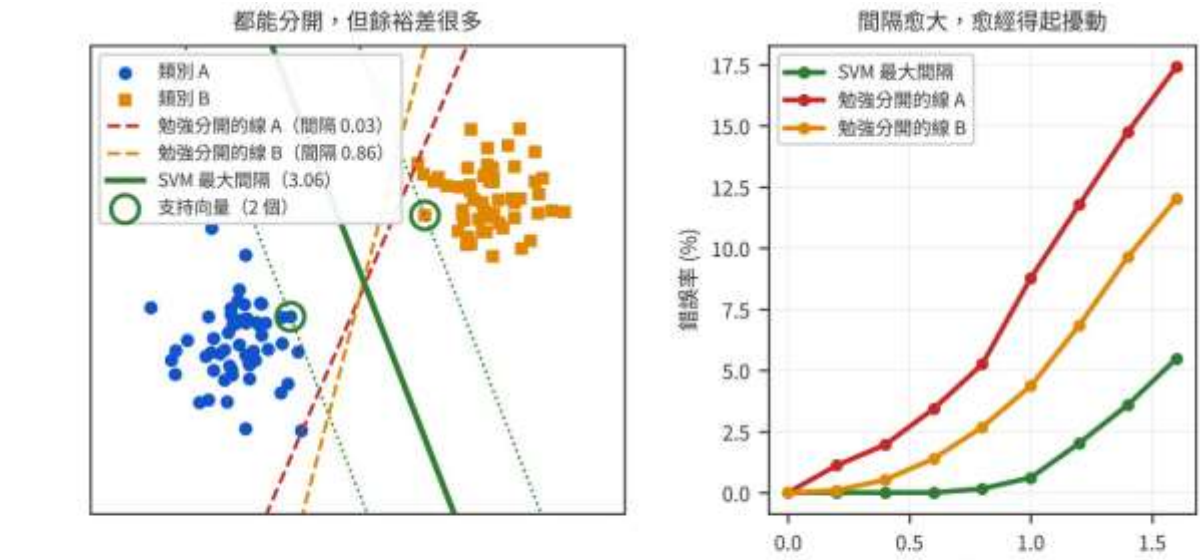


kNN 完全靠「距離」做事，所以當維度本身失去意義，它就失去了依據。左圖是純粹的數學現象：在 d 維單位立方體裡隨機撒 500 個點，再問「最遠的點比最近的點遠多少倍」。二維時這個比值是 40.0，但到了 512 維只剩 0.16——也就是說，所有點到查詢點的距離幾乎一樣遠，「最近的鄰居」與「最遠的鄰居」已經沒什麼差別了。這就是維度災難。右圖是同一件事在真實資料上的後果：把 64 維的手寫數字用 PCA 降到不同維度再跑 kNN，最佳落在 32 維（98.5%），比用完整 64 維的 98.5% 還高。這個結果常讓初學者意外：丟掉資訊反而更準。原因是被丟掉的多半是雜訊方向，保留的則是真正能區分類別的方向，距離因此變得更有意義。這也說明了第 13 章特徵工程的價值——好的特徵不只是「更有代表性」，也是「更低維」。

圖 14-6　維度災難：為什麼「最近」會失去意義

14.4.2 支持向量機(SVM)

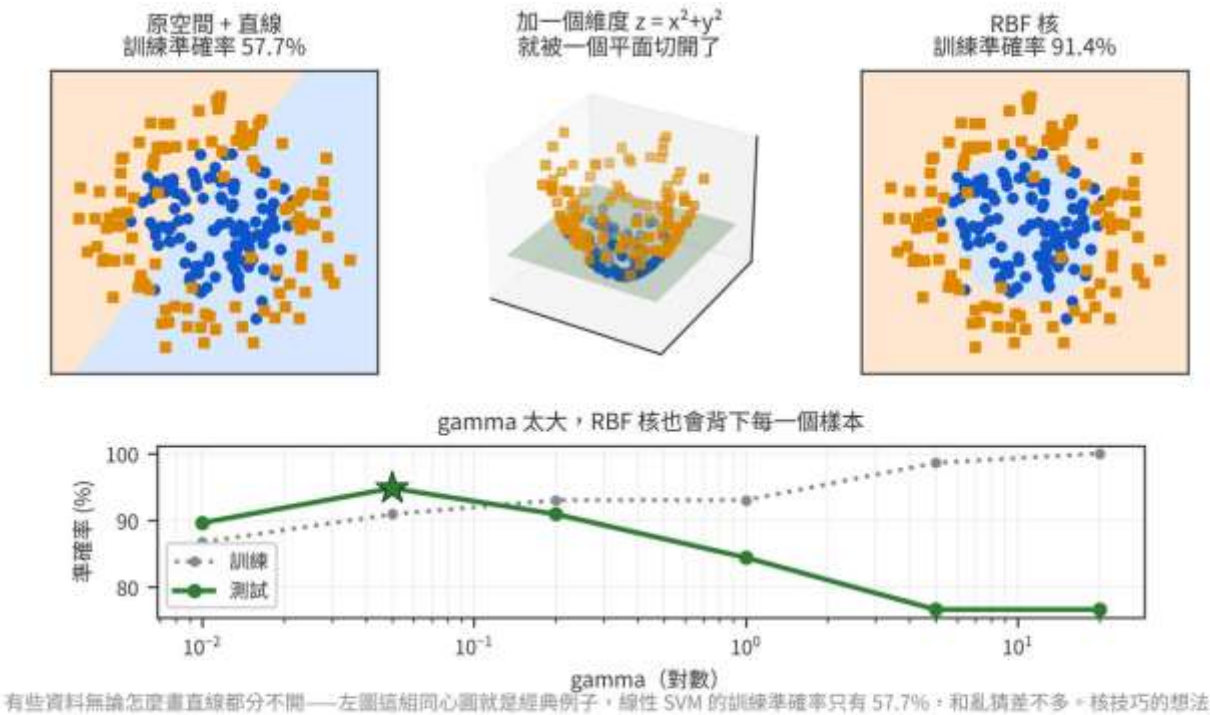
SVM 是深度學習興起前最強大、最受推崇的分類器之一。它的核心思想是找出一條「最好的分界線」(在高維空間中稱為超平面)來分開兩類資料。什麼叫「最好」?SVM 的答案很優雅：不只要能分開，還要讓分界線離兩邊最近的樣本盡可能遠——這個「最大間隔」(maximum margin)原則，讓分類器對新樣本更有容錯能力、泛化更好。那些恰好落在間隔邊界上、決定了分界線位置的關鍵樣本，就稱為「支持向量」，SVM 之名由此而來。



「最大間隔」為什麼有助於泛化？這張圖用實測回答。左圖的兩類資料有無限多條線可以分開，本圖測試資料的擾動強度 σ 都完全正確地分開了訓練資料，但它們緊貼著某一邊的樣本，間隔只有 0.03、0.86；SVM 選的那條線線間隔是 3.06，兩側各留出一條走廊。若只看訓練準確率，三條線一模一樣好，完全分不出高下。差別要在右圖才顯現：把測試樣本加上愈來愈大的擾動（模擬真實世界的拍攝差異、雜訊、個體變化），間隔小的線很快就開始出錯，而最大間隔那條撐得最久。這理很直觀——間隔就是「容錯的空間」，樣本在這條走廊的寬度之內怎麼晃動都還是分對的。順帶一提，決定這條線的只有貼在走廊邊上的那幾個樣本（本例 2 個，即支持向量），其餘樣本就算全部刪掉，答案也不會變——這是 SVM 模型可以很小的原因，也是 14.4.3 節要推導的重點。

圖 14-7　最大間隔：不只分得開，還要分得有餘裕

SVM 還有一個威力強大的祕密武器——核技巧(Kernel Trick)：當資料在原始空間無法用直線分開時(例如一類包在另一類中間)，核技巧能把資料「隱含地」映射到更高維的空間，在那裡它們就變得線性可分了，而且計算上不必真的算出高維座標。這讓 SVM 能處理複雜的非線性分類。在深度學習普及前，「人工設計特徵(如 SIFT/HOG)+ SVM 分類」是影像辨識的黃金組合，至今在樣本較少的場合仍是可靠選擇。



有些資料無論怎麼畫直線都分不開——左圖這組同心圓就是經典例子，線性 SVM 的訓練準確率只有 57.7%，和亂猜差不多。核技巧的想法是：既然在二維分不開，就升到三維看看。中圖把每個點加上第三個座標 $z = x^2 + y^2$，內圈的點 z 小、外圈的點 z 大，於是一個水平的平面就乾淨地把兩類切開了。回到二維看，這個平面就是一個圓。「核技巧」的巧妙在於：它根本不需要真的把資料升上去。SVM 的推導中資料只以內積的形式出現，所以只要能直接算出「升維之後兩點的內積」就夠了，而核函數正是在做這件事——省掉了升維的運算與儲存，甚至可以升到無限維（RBF 核就是）。右圖的 RBF 核把訓練準確率拉到 91.4%。但天下沒有白吃的午餐：下圖掃描 gamma 顯示，gamma 太大時訓練準確率衝到 100%、測試卻掉到 76.6%——核把模型的能力放大了，也把過度擬合的風險一起放大了。

圖 14-8　核技巧：在原空間分不開，升上去就分得開

程式 14-1 kNN 與 SVM 手寫數字分類

```
import cv2, numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# 載入手寫數字(類 MNIST 的小型資料集)
digits = load_digits()
X = digits.data          # 每張影像攤平成特徵向量
y = digits.target        # 標籤 0~9

# 切分訓練/測試(14.5 節的核心觀念)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# --- kNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)              # kNN 的 fit 只是記住資料
pred_knn = knn.predict(X_te)
print("kNN 準確率:", accuracy_score(y_te, pred_knn))

# --- SVM(RBF 核) ---
```

```
svm = SVC(kernel="rbf", C=10, gamma=0.001)
svm.fit(X_tr, y_tr)
pred_svm = svm.predict(X_te)
print("SVM 準確率:", accuracy_score(y_te, pred_svm))
print("SVM 混淆矩陣:\n", confusion_matrix(y_te, pred_svm))
```

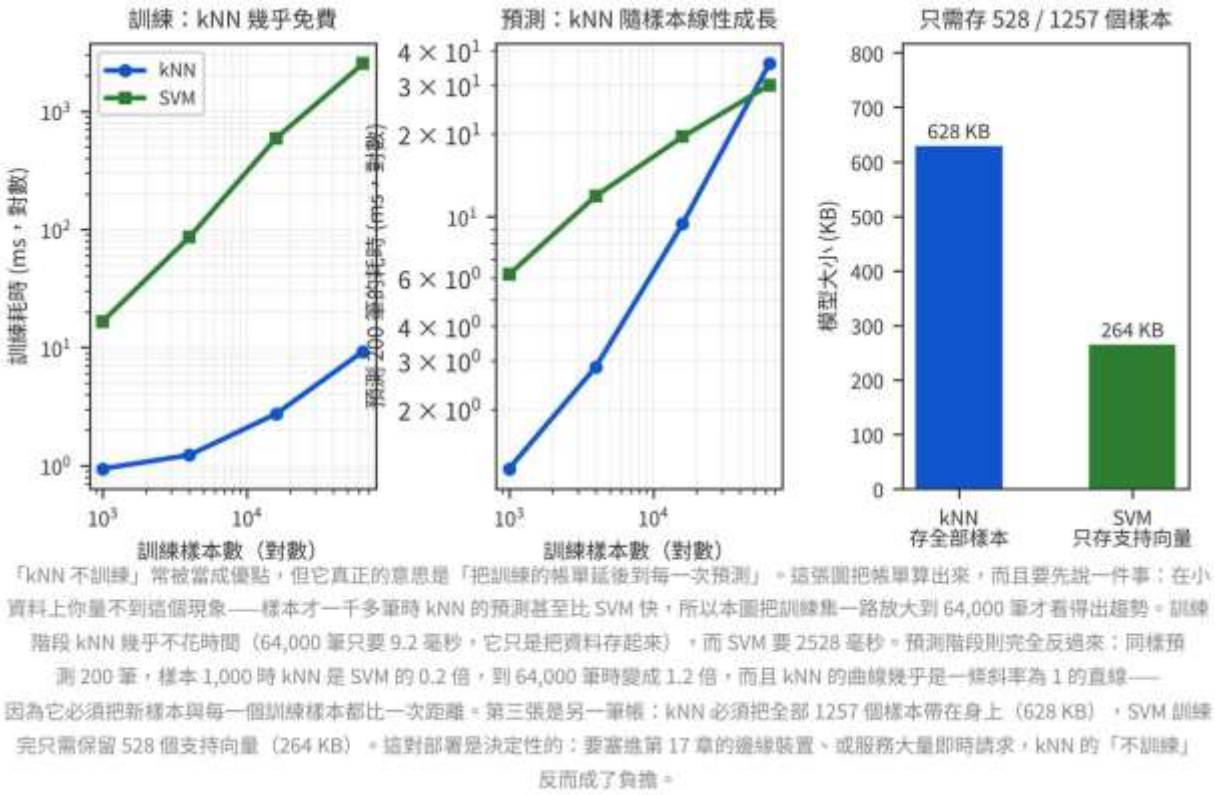


圖 14-9　kNN 的帳單：不訓練，但每次預測都要付錢

14.4.3 推導：最大間隔為什麼等價於最小化 |w|

14.4.2 節說 SVM 要找「離兩邊最近樣本盡可能遠」的分界線，這句話聽起來像個模糊的審美標準。但它其實可以寫成一道有唯一解的最佳化問題，而且推導只需要高中的點到直線距離公式。值得花十分鐘跟著走一遍，因為這是全書第一次完整示範「把一個直覺要求翻譯成數學目標」——第 15 章的損失函數、第 16 章的 IoU 門檻，用的都是同一套思路。

先把分界線寫成一般式。在二維平面上它是一條直線 w1·x1 + w2·x2 + b = 0;在 d 維特徵空間中則是一個超平面，用向量內積寫成 w·x + b = 0，其中 w 是這個超平面的法向量(垂直於它的方向)、b 決定它離原點多遠。任一點 x 到這個超平面的距離，正是高中那條公式的推廣：距離 = |w·x + b| / |w|，其中 |w| = √(w1² + w2² + … + wd²) 是

法向量的長度。

接下來是整個推導最關鍵、也最常被跳過的一步：縮放的自由度。請注意 w·x + b = 0 與 (3w)·x + 3b = 0 畫出來是同一個超平面——把 w 和 b 同時乘上任何非零常數，幾何上什麼都沒變。這表示 (w, b) 這組參數有一個多餘的自由度，我們可以挑一個最方便的縮放，而不損失任何一般性。SVM 選的是：讓離超平面最近的那些樣本恰好滿足 |w·x + b| = 1。這個約定叫做「正規化超平面」，它不是額外的假設，只是把尺規訂死而已。

尺規訂死之後，一切就都量得出來了。設每個樣本的標籤 y 只取 −1 或 +1(SVM 習慣用 ±1 而非 0/1，等一下就知道為什麼)，則「全部樣本都被正確分類、且最近的樣本剛好落在 ±1 上」可以濃縮成一條不等式：對每一筆樣本 (x, y) 都有 y·(w·x + b) ≥ 1。這就是用 ±1 當標籤的好處：正樣本要求 w·x + b ≥ 1、負樣本要求 w·x + b ≤ −1，各自乘上自己的 y 之後，兩者合而為一。

而最近樣本到超平面的距離，依前面的公式就是 |w·x + b| / |w| = 1 / |w|。兩側各有一條這樣的邊界，所以「間隔」(margin，兩條邊界之間的總寬度)等於 2 / |w|。到這裡，那個模糊的審美標準就變成了一個精確的目標：最大化 2 / |w|。

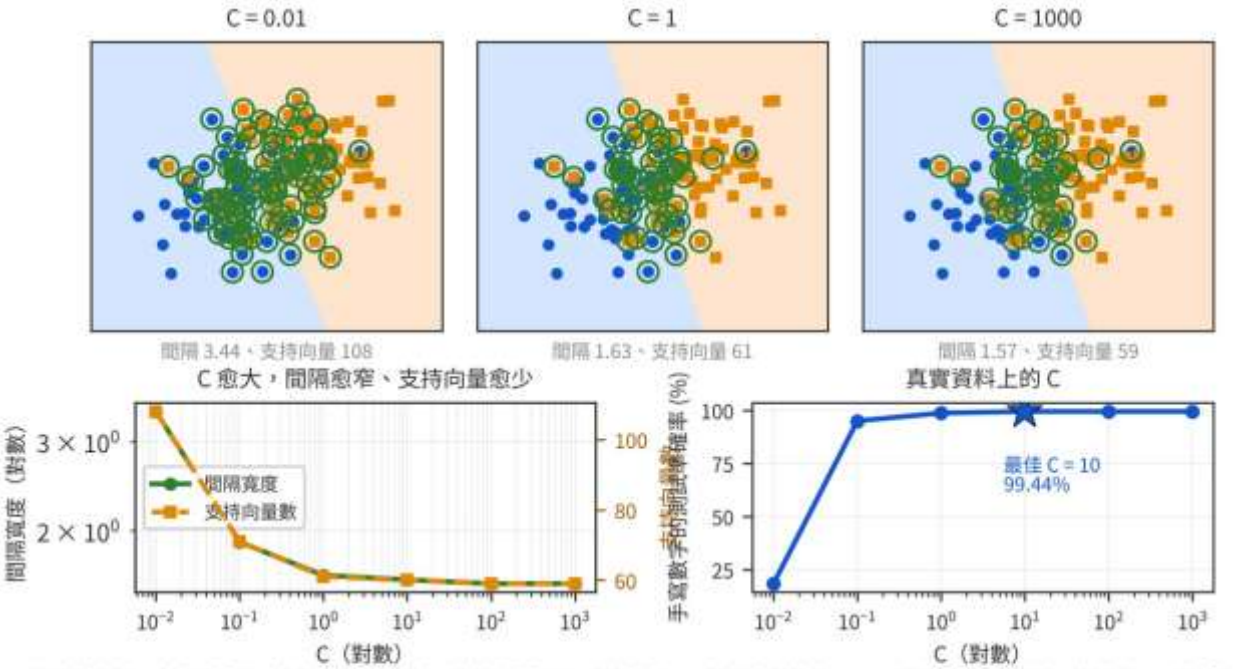
最後是兩次等價轉換。其一，最大化 2 / |w| 等同於最小化 |w|，因為分子是常數，而 |w| > 0 時兩者單調反向。其二，最小化 |w| 等同於最小化 ½·|w|²，因為平方在正數上單調遞增，不改變最小值的位置;加上 ½ 純粹是為了微分之後係數剛好消掉，是數學家的小小潔癖。於是整個問題寫成：

```
最小化    (1/2) · |w|²
受限於    y · (w·x + b) ≥ 1 ,   對每一筆訓練樣本 (x, y)
```

這個形式有三個漂亮的性質，也是 SVM 在 1990 年代席捲學界的原因。第一，目標函數是凸函數、約束是線性的，所以這是一個凸二次規劃問題——它只有一個全域最佳解，不像神經網路那樣會卡在局部極小值;同一批資料跑一百次會得到同一個答案。

第二，用拉格朗日乘子法解它的對偶問題時，會發現最終的 w 只由那些「不等式取等號」的樣本決定，其餘樣本的乘子都是 0——這就是支持向量的數學來源，也解釋了圖 14-7 的實測：110 個樣本中只有 2 個是支持向量。第三，對偶形式中樣本只以內積的形式出現，把內積換成核函數就得到了非線性的 SVM——這正是 14.4.2 節核技巧「不必真的算出高維座標」的根據，也是圖 14-8 能把線性核的 57.7% 拉到 RBF 核 91.4% 的原理。

最後補一個實務上的轉折。上面的推導假設資料完全線性可分，但真實資料幾乎從不如此，只要有一個離群樣本就會讓約束無解。因此實用的 SVM 在每個約束上放寬一個鬆弛量 $\xi \ge 0$，改成 $y\cdot(w\cdot x + b) \ge 1 - \xi$，並在目標函數裡加上懲罰項，變成最小化 $(1/2)\cdot|w|^2 + C\cdot\Sigma\xi$。這個 C 就是 14.4.2 節與圖 14-10 裡那個參數：C 很大時懲罰很重，模型不准犯錯、間隔被壓窄;C 很小時容許犯錯、間隔變寬。圖 14-10 的實測正是這條公式的具體樣貌：C 一路加大，間隔從 3.44 縮到 1.57、支持向量從 108 個減到 59 個。所以 C 不是一個神祕的旋鈕，它就是「間隔」與「訓練誤差」之間的兌換率——而這個兌換率該訂在哪裡，只能靠驗證集決定，不能靠訓練集。



現實的資料不會乖乖分開，總會有幾個標錯或天生就長得像另一類的樣本。C 這個參數決定了 SVM 要用多強硬的態度面對它們：C 是「犯錯的罰金」。罰金低（C = 0.01）時，模型寧願放過幾個難搞的樣本，換取一條又寬又平滑的走廊，此時間隔 3.44、支持向量多達 108 個;罰金高（C = 1000）時，模型會為了避就每一個樣本而把邊界扭過去，間隔縮到 1.57、支持向量只剩 59 個。左下的兩條曲線把這個趨勢量了出來。右下是真實資料上的後果：在手寫數字上掃 C，最佳落在 C = 10（99.44%），兩端都比較差。C 太小是擬合不足（連該學的都沒學），C 太大是過度擬合（連雜訊都背了下來）——這又是一次偏差與變異的拉鋸，而拉鋸的位置必須用驗證集決定，不能憑經驗填一個數字。

圖 14-10　C 參數：間隔要寬，還是不准錯

14.5 資料切分與模型評估

14.5.1 訓練、驗證、測試：為什麼要三分

這是機器學習最重要、也最容易被初學者忽略的觀念：評估模型，絕不能用它訓練時看過的資料。道理很簡單——一個學生若考的是他背過答案的題目，高分不代表他真懂。模型也一樣：它可能只是「記住」了訓練資料，而非「學會」了規律。因此我們把資料切成三份：

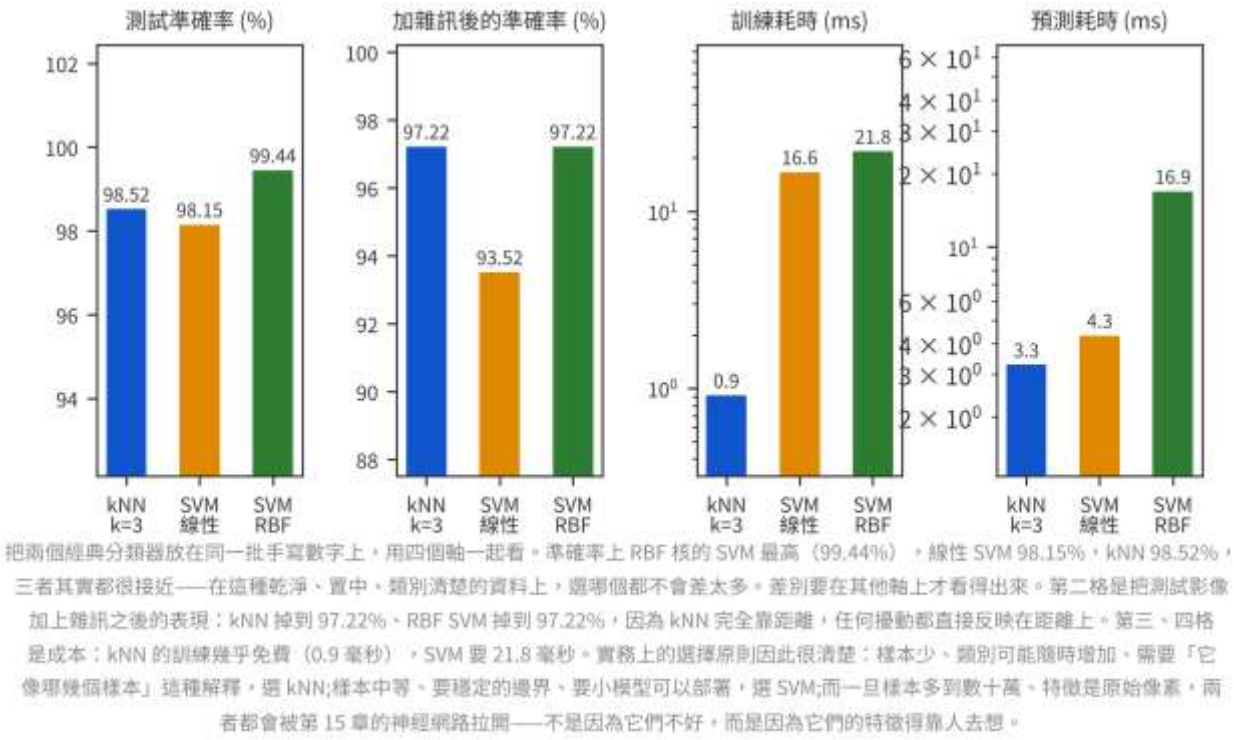
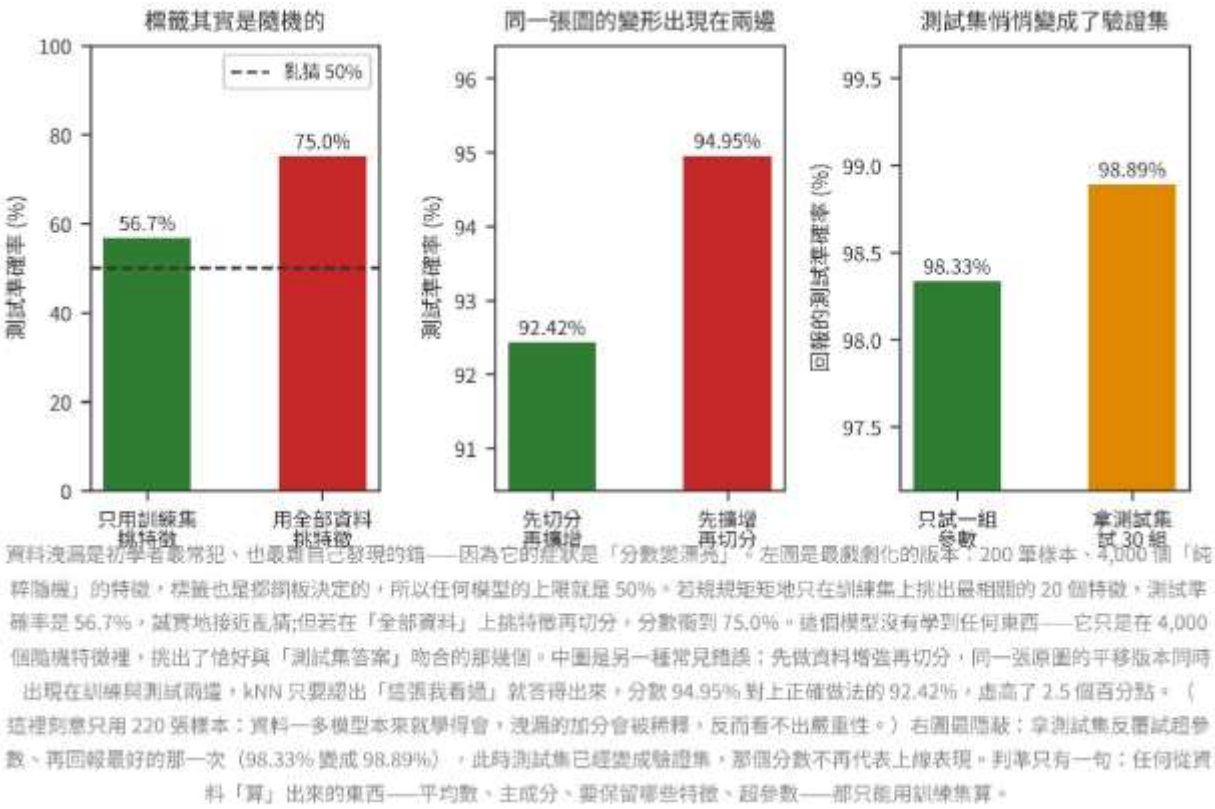


圖 14-11　擂台：kNN 與 SVM 在五個軸上的對照

1. 訓練集(Training Set)：用來調整模型參數，佔最大比例(如 60~80%)。
2. 驗證集(Validation Set)：在訓練過程中用來調整「超參數」(如 kNN 的 k、SVM 的 C)、選擇模型、決定何時停止訓練。它不直接參與參數調整，但會影響我們的決策。
3. 測試集(Test Set)：模型與所有決策都定案後，最後才用它評估——它模擬「模型上線後遇到全新資料」的真實表現。測試集必須嚴格保密，絕不能在開發過程中反覆使用，否則就等於間接讓模型「看過」了它，評估結果會失真、過度樂觀。

這也解釋了叫獸開講中 MNIST 的設計巧思：LeCun 團隊刻意混合「人口普查局員工(易)」與「高中生(難)」兩種來源，並確保訓練集與測試集的難度分布一致——若訓練集全是工整字跡、測試集全是潦草字跡，模型的測試表現會低估它的真實能力，反之則高估。資料切分的公平性，是可靠評估的前提。當資料量較少時，還可用「交叉驗證」(把資料輪流當訓練與驗證，取平均)來更充分地利用資料。

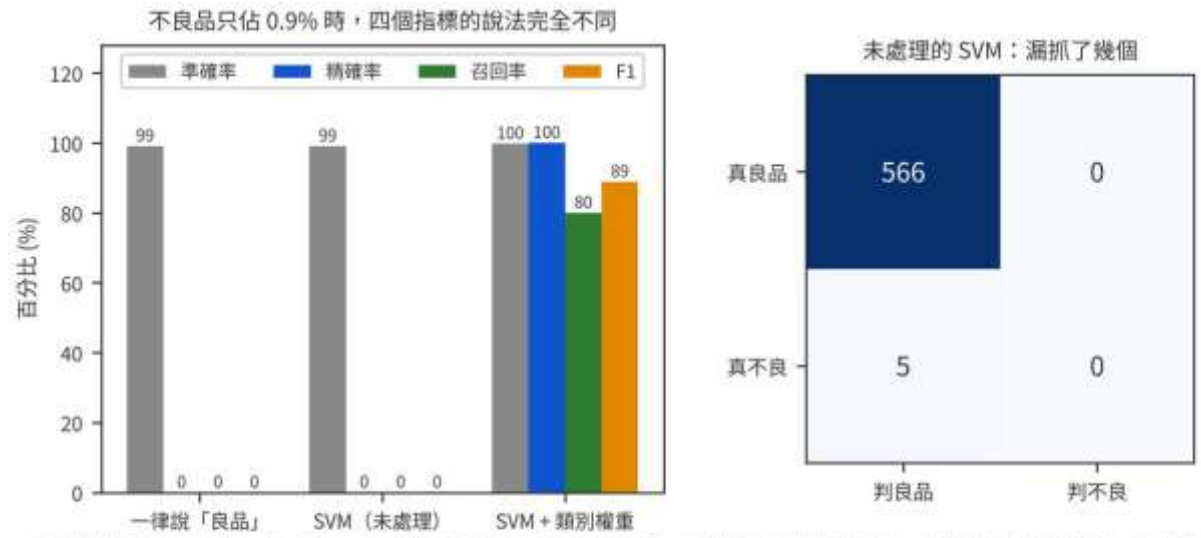


資料洩漏是初學者最常犯、也最難自己發現的錯——因為它的症狀是「分數變漂亮」。左圖是最戲劇化的版本：200 筆樣本、4,000 個「純粹隨機」的特徵，標籤也是擲銅板決定的，所以任何模型的上限就是 50%。若規規矩矩地只在訓練集上挑出最相關的 20 個特徵，測試準確率是 56.7%，誠實地接近亂猜；但若在「全部資料」上挑特徵再切分，分數衝到 75.0%。這個模型沒有學到任何東西——它只是在 4,000 個隨機特徵裡，挑出了恰好與「測試集答案」吻合的那幾個。中圖是另一種常見錯誤：先做資料增強再切分，同一張原圖的平移版本同時出現在訓練與測試兩邊，kNN 只要認出「這張我看過」就答得出來，分數 94.95% 對上正確做法的 92.42%，虛高了 2.5 個百分點。（這裡刻意只用 220 張樣本：資料一多模型本來就學得會，洩漏的加分會被稀釋，反而看不出嚴重性。）右圖區隔敲：拿測試集反覆試超參數、再回報最好的那一次（98.33% 變成 98.89%），此時測試集已經變成驗證集，那個分數不再代表上線表現。判準只有一句：任何從資料「算」出來的東西——平均數、主成分、要保留哪些特徵、超參數——都只能用訓練集算。

圖 14-12　資料洩漏：測試集被偷看會發生什麼事

14.5.2 評估指標：準確率的陷阱

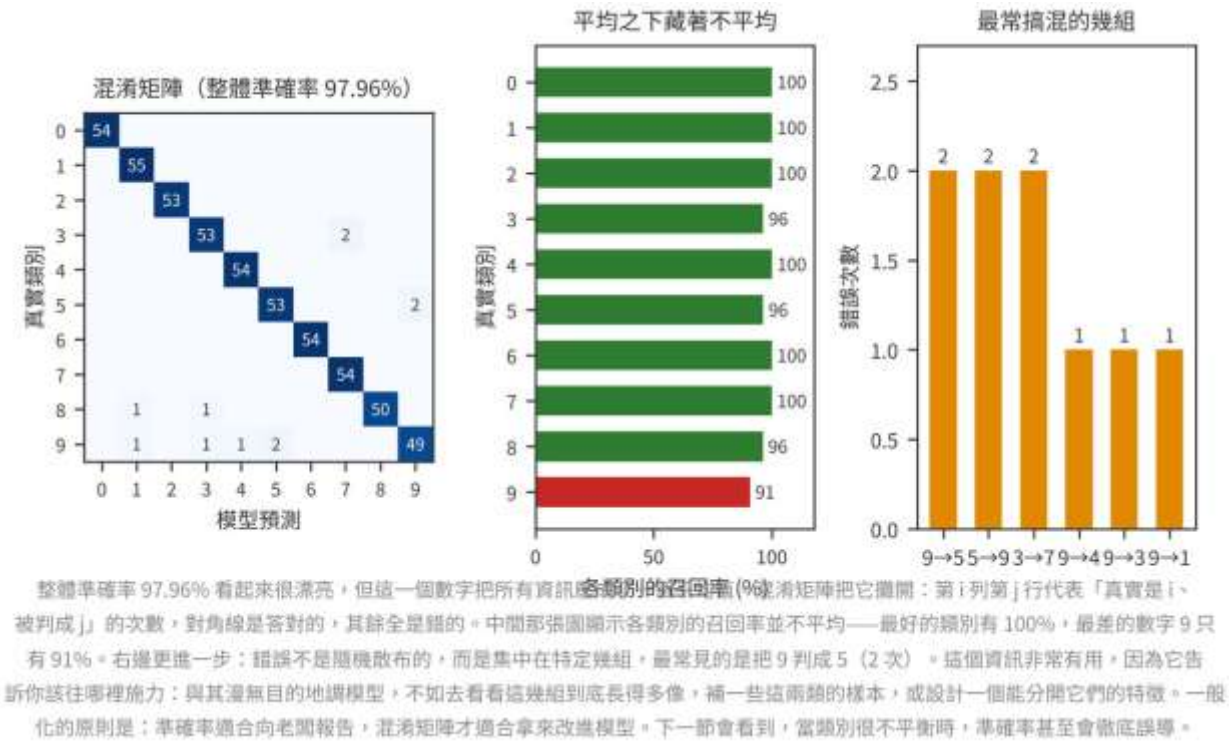
最直觀的指標是準確率(Accuracy)：預測正確的比例。但準確率有一個著名的陷阱——類別不平衡時會嚴重誤導。舉例：某罕見疾病的盛行率是 1%，一個「無論如何都判斷沒病」的模型，準確率高達 99%，卻毫無用處(它抓不出任何一個病人)。因此，只看準確率是危險的，必須輔以更細緻的指標。這裡要補一個比「一律判沒病」更令人不安的事實：不是只有偷懶的基準模型才會這樣，一個認真訓練的分類器在類別極度不平衡時也會做出同樣的選擇，因為「最小化整體錯誤」最快的方法就是放棄少數類。圖 14-13 的實測：不良品只佔 0.86% 時，「一律說良品」與一個未做任何處理的 SVM，兩者的準確率都是 99.12%、召回率都是 0%——也就是說，那個 SVM 花了時間訓練，學到的東西與什麼都不學完全一樣。加上類別權重之後，準確率 99.82%、召回率才升到 80%。



這張圖把「準確率的陷阱」做成實驗。我們把數字 8 當成產線上的不良品，刻意把它的比例壓到 0.9%，模擬真實的品檢場景。第一根柱子是一個什麼都不做的「模型」：一律回答良品。它的準確率高達 99.1%，漂亮得可以寫進簡報——但它的召回率是 0%，一個不良品都沒抓到，完全沒有用。這就是類別不平衡時準確率會誤導的原因：多數類別把平均值整個拉上去了。更值得警惕的是中間那根：一個正常訓練的 SVM，準確率 99.1% 甚至比「一律說良品」還高，但它的召回率同樣是 0%——模型在訓練時發現「全部猜多數類別」就能把損失壓得很低，於是它就真的這麼做了。這不是模型壞掉，是我們給錯了目標。第三根只改一件事：加上類別權重，讓罕見類別的錯誤罰得更重（權重與類別頻率成反比）。召回率立刻升到 80%、精確率 100%，而準確率 99.8% 幾乎沒變——這正說明了準確率對這件事有多麼不敏感。要選哪一個，取決於代價：漏抓一個不良品流到客戶手上，與誤判一個良品重驗一次，這兩件事的損失通常差好幾個數量級。指標不是數學問題，是商業問題。

圖 14-13　準確率的陷阱：類別不平衡

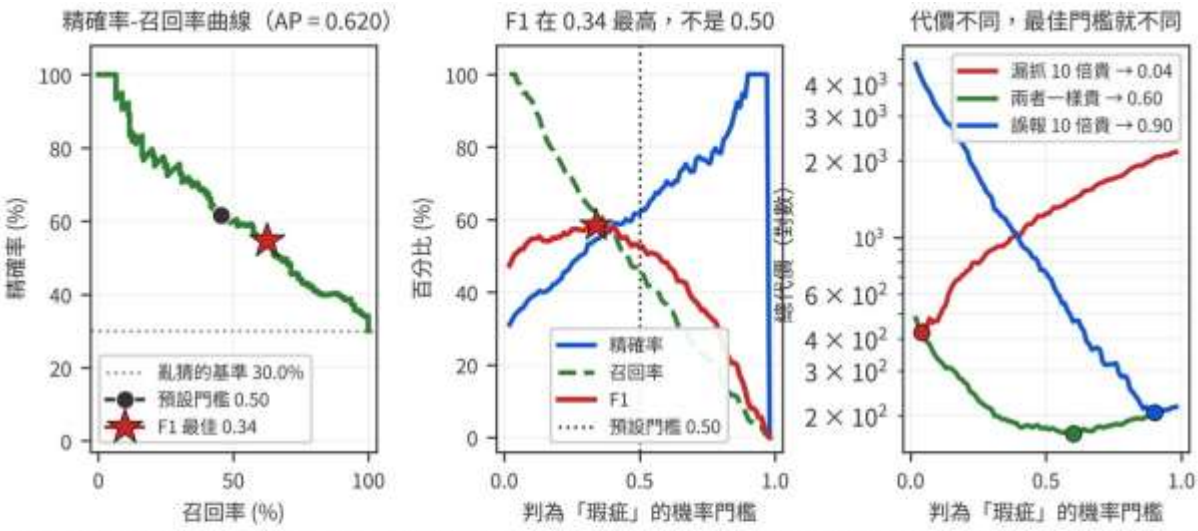
混淆矩陣(Confusion Matrix)是評估的基礎工具：它是一張表格，列出「實際類別」與「預測類別」的所有組合，清楚顯示模型把哪些類別混淆了(例如把 4 誤判成 9)。從混淆矩陣可導出兩個關鍵指標：精確率(Precision，模型說是陽性的裡面，有多少真的是陽性——衡量「不誤報」)與召回率(Recall，所有真正的陽性中，模型抓出了多少——衡量「不漏抓」)。這兩者常有取捨：提高召回(寧可錯抓也不放過)往往降低精確(誤報變多)，反之亦然;F1 分數則是兩者的調和平均，綜合衡量。



整體準確率 97.96% 看起來很漂亮，但這一個數字把所有資訊壓成一個標量。混淆矩陣把它攤開：第 i 列第 j 行代表「真實是 i、被判成 j」的次數，對角線是答對的，其餘全是錯的。中間那張圖顯示各類別的召回率並不平均——最好的類別有 100%，最差的數字 9 只有 91%。右邊更進一步：錯誤不是隨機散布的，而是集中在特定幾組，最常見的是把 9 判成 5（2 次）。這個資訊非常有用，因為它告訴你該往哪裡施力：與其漫無目的地調模型，不如去看看這幾組到底長得多像，補一些這兩類的樣本，或設計一個能分開它們的特徵。一般化的原則是：準確率適合向老闆報告，混淆矩陣才適合拿來改進模型。下一節會看到，當類別很不平衡時，準確率甚至會徹底誤導。

圖 14-14　混淆矩陣：一個數字看不出來的事

選擇哪個指標，取決於應用的代價結構——這是工程判斷，不是數學問題。癌症篩檢重召回(漏抓一個病人的代價，遠大於多做幾次複檢);垃圾郵件過濾重精確(把重要郵件誤判為垃圾的代價，大於偶爾漏掉一封垃圾);這與第 10 章 Viola-Jones 的「漏檢 vs. 誤報」、第 9 章 Canny 雙門檻，是同一類「依代價權衡」的思維，貫穿整本書。還要補上一件實務上最常轉、教科書卻最少提的旋鈕：判定門檻。分類器真正輸出的是一個分數或機率，「判為陽性」是我們拿 0.5 去切出來的，而那個 0.5 與任務的代價毫無關係。門檻調低，召回率上升、精確率下降;調高則反之。圖 14-15 的實測顯示，同一個模型、同一批資料，在三種代價結構下的最佳門檻分別是 0.04、0.60 與 0.90;即使只看 F1，最佳門檻也是 0.34 而非 0.50，差了 6.0 個百分點。換句話說，調門檻不是技術細節，而是把商業判斷寫進系統的那一步，而且這一步只有懂現場的人能做。



分類器真正輸出的是一個分數或機率，「判為瑕疵」是我們拿 0.5 去切出來的——而那個 0.5 從來不是什麼神聖的數字。中圖把門檻從 0.02 掃到 0.98：門檻低時幾乎全判瑕疵，召回率接近 100% 而精確率很低（寧可錯殺）；門檻高時反過來（寧可放過）。兩條線必然交叉，F1 在 0.34 處達到 58.3%，而預設的 0.50 只有 52.3%（精確率 61.6%、召回率 45.4%）——光是轉一下這個旋鈕就多了 6.0 個百分點，一行模型程式都不用改。左圖是同一件事的另一種畫法：精確率-召回率曲線上的每一點就是一個門檻，曲線下的面積（AP = 0.620）因此是一個與門檻無關的總體指標，比任何單一門檻下的準確率都可靠，也是第 16 章物件偵測評分的基礎。右圖才是最該帶走的一課：最佳門檻取決於代價。漏抓的代價是誤報十倍時（瑕疵品流到客戶手上），最佳門檻低到 0.04；兩者一樣貴時是 0.60；誤報十倍貴時（誤判害整批重工）升到 0.90。同一個模型、同一批資料，答案卻完全不同。所以調門檻不是技術細節，而是把商業判斷寫進系統的那一步，而且這一步只有懂現場的人能做，不是調參程式能自動決定的。

圖 14-15　門檻不是 0.5：精確率、召回率與代價

14.5.3 推導：一個準確率該附上多大的誤差範圍

14.5.1 節說資料量少時可以用交叉驗證，圖 14-16 則實測了單次切分有多不穩。這一小節補上背後的算式，讓你能在任何時候自己估出「我報的這個數字，誤差大概多少」——這是一個只需要兩個數字就算得完、卻幾乎沒有人去算的量。

先把問題說清楚。測試集有 n 筆樣本，模型答對了 k 筆，我們報告的準確率是 a = k / n。但我們真正想知道的不是 a，而是這個模型在「所有可能的新樣本」上的真實準確率 p——a 只是 p 的一次抽樣估計。如果把每一筆測試樣本看成一次獨立的伯努利試驗(答對的機率是 p、答錯是 1 − p)，那麼答對的總數 k 服從二項分布，其變異數為 n·p·(1 − p)。

把它除以 n^2 換算成 a 的變異數，再開根號，就得到準確率估計的標準誤：

```
SE(a) = √( p · (1 − p) / n )  ≈  √( a · (1 − a) / n )
約 95% 信賴區間 ：  a − 1.96·SE(a)  ～  a + 1.96·SE(a)
```

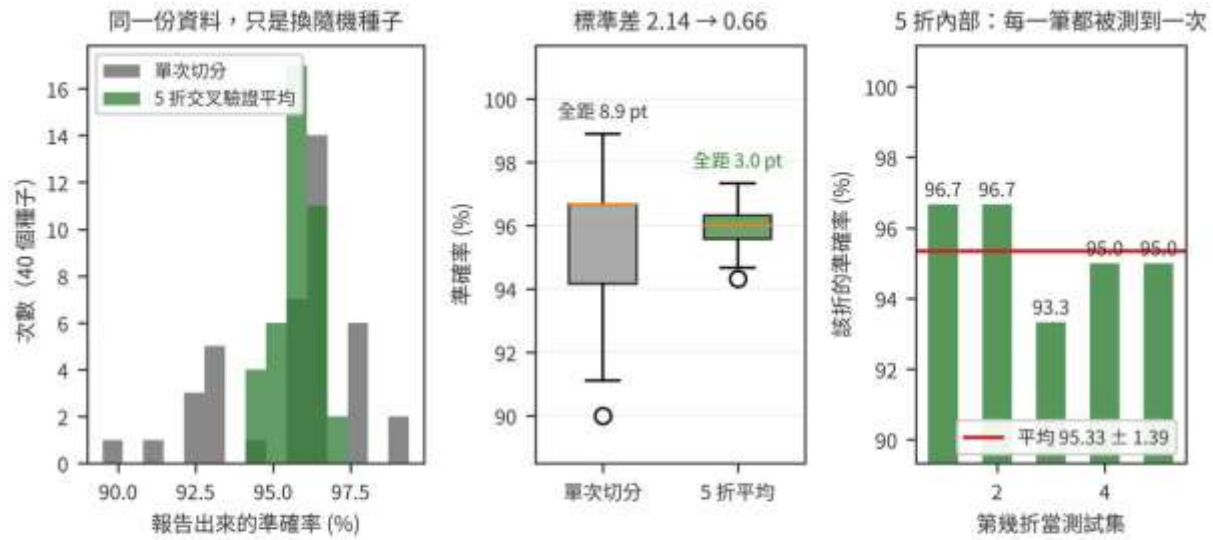
這條公式有三個推論值得記住。其一，標準誤隨 √n 下降：要把誤差砍半，測試

集必須變成四倍大。其二，$p\cdot(1-p)$ 在 $p = 0.5$ 時最大、在 p 接近 0 或 1 時最小，所以同樣大小的測試集，高準確率的模型估得比較準。其三，也是最實用的：拿 1.96 倍的標準誤往兩邊各畫一道，就是那個該寫進報告裡的區間。

代幾個數字就會冒冷汗。測試集 $n = 200$、準確率 $a = 0.90$ 時，$SE = \sqrt{(0.9 \times 0.1 / 200)} \approx 0.0212$，95% 信賴區間是 90% ± 4.2%，也就是 85.8% 到 94.2%。換句話說，在一個 200 筆的測試集上，一個 91% 的模型與一個 89% 的模型根本分不出高下。而論文與驗收報告裡「我們的方法比基準高 1.5 個百分點」這種句子，如果測試集只有兩三百筆，那 1.5 個百分點完全埋在雜訊裡。$n = 2{,}000$ 時 SE 降到 0.67%，區間縮成 88.7% 到 91.3%;$n = 20{,}000$ 時才來到 ±0.42%。這就是為什麼嚴肅的演算法評比，測試集動輒上萬筆。

交叉驗證處理的則是另一個誤差來源。上面的公式只算了「測試集抽樣」的隨機性，卻假設模型本身是固定的;但換一次切分，訓練集也跟著變，學出來的模型就不一樣了。圖 14-16 量到的就是這一層：同一份 300 筆資料、同一個模型設定，光是換 40 個隨機種子，單次切分的結果就從 90.0% 跳到 98.9%，標準差 2.14 個百分點。k 折交叉驗證把資料切成 k 份、輪流拿一份當測試集，每一筆樣本都剛好被測到一次，再把 k 次結果平均;實測顯示這樣得到的估計標準差降到 0.66，穩定了約 3.3 倍。

報告交叉驗證結果時，慣例是寫成「平均 ± 標準差」，例如 95.33% ± 1.39%。要注意這個標準差是「折與折之間」的變異，它同時反映了測試集抽樣與訓練集變動兩種來源，因此通常比上面那條二項分布公式算出來的標準誤大——而這是誠實的，因為它涵蓋了更多真實存在的不確定性。順帶一提，k 折的各折並不獨立(各折的訓練集彼此高度重疊)，所以嚴格說來不能直接把標準差除以 $\sqrt{k}$ 當成標準誤使用;這是統計上一個著名的細節，但在初學階段，只要養成「報平均也要報標準差」的習慣，就已經勝過大多數人了。



「我的模型測試準確率 96.7%」這句話，比多數人以為的更不可靠。做法是用同一份資料（300 筆）、同一個模型，只換 40 個隨機種子，各做兩件事：一次 7:3 切分，以及一次 5 折交叉驗證。灰色那堆是單次切分的結果：從 90.0% 到 98.9%，全距 8.9 個百分點，標準差 2.14。換句話說，只要種子換得好，同一個模型就能「變好」好幾個百分點，而這種「調種子」在論文與驗收報告裡比想像中常見。綠色那堆是 5 折平均：全距縮到 3.0 個百分點，標準差 0.66，也就是穩定了約 3.3 倍。比較模型時問題更嚴重：本圖實測 SVM 與 kNN 在這 40 次切分中，有 13 次 kNN 反而贏——只切一次就下結論，有相當機率選錯模型。右圖是 5 折內部的樣子：每一折輪流當測試集，折與折之間本來就差 3.3 個百分點，平均之後才是 95.33% ± 1.39。實務上的規矩因此很簡單：資料量不大時一律用交叉驗證，而且報告準確率時要附上標準差——沒有誤差範圍的單一數字，在統計上幾乎沒有意義。

圖 14-16　一次切分不可信：交叉驗證把變異壓下來

14.6 過度擬合、擬合不足與偏差-變異權衡

這是機器學習最核心的理論觀念，也是所有實務調校的指南針。它回答一個根本問題：為什麼模型在訓練資料上表現很好，到了真實世界卻不行?

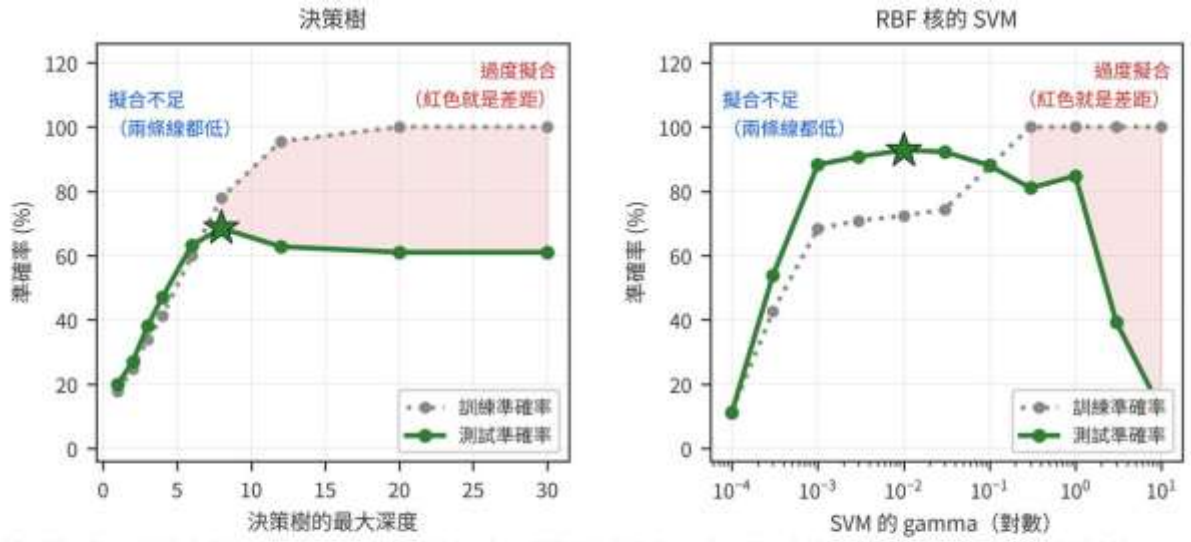
14.6.1 擬合不足與過度擬合

擬合不足(Underfitting)是模型太簡單，連訓練資料的規律都抓不住——訓練表現與測試表現都差。好比用一條直線去擬合明顯彎曲的資料，怎麼畫都不貼。

過度擬合(Overfitting)則相反，是模型太複雜，把訓練資料「背」得太死，連其中的雜訊與偶然細節都當成規律學了進去——訓練表現極好，但一遇到沒看過的測試資料就崩潰。好比一個學生死背了考古題的每個答案，換個題目就不會了。過度擬合是實務中最常見、最需要警惕的問題。

診斷方法很簡單：比較模型在訓練集與驗證集上的表現。兩者都差是擬合不足;訓練好、驗證差且落差很大，就是過度擬合。理想的模型落在中間——複雜度剛好抓住

真實規律，又不至於背下雜訊。

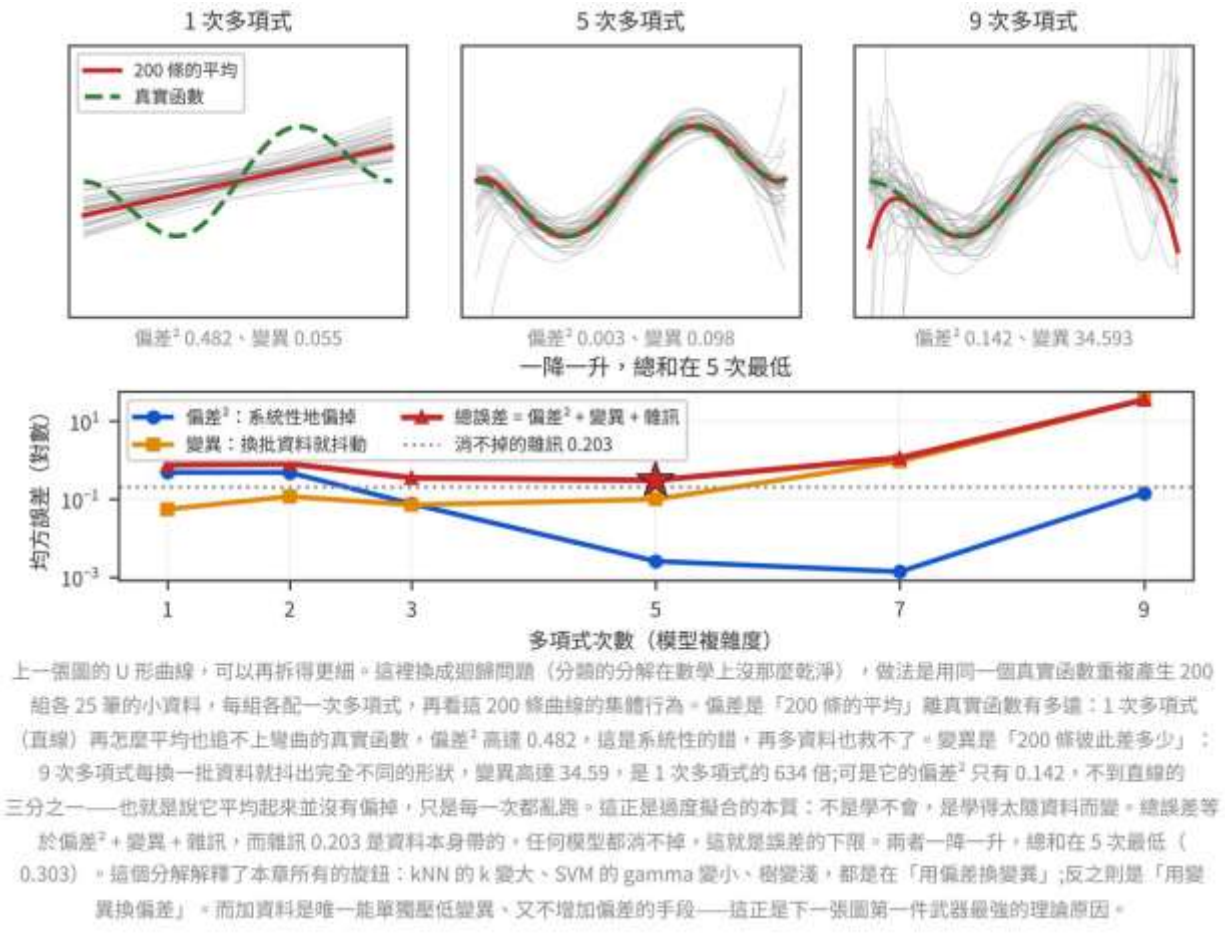


這是機器學習最該畫在白板上的一張圖，而且兩種毫不相干的模型都畫得出同一個形狀。左邊是決策樹，複雜度由深度控制;右邊是 RBF 核的 SVM，複雜度由 gamma 控制（gamma 愈大，每個支持向量的影響範圍愈小、決策邊界愈嚴）。為了貼近現實，訓練標籤裡刻意混入了 30% 的錯誤標註——真實專案的標註從來不是完美的，而這正是過度擬合最愛的養分。兩張圖的灰色虛線都是訓練準確率：它隨複雜度單調上升，最後抵達 100% 與 100%——模型把錯誤的標籤也一起背了下來。綠線是測試準確率，卻是先升後降：決策樹在深度 8 達到 68.5%，之後掉到 61.0%;SVM 在 gamma = 0.01 達到 92.8%，之後崩到 12.0%，幾乎等於亂猜。兩條線之間那塊紅色區域就是「過度擬合的量」，也是本章 14.6 節一切手段想壓掉的東西。最左端則是擬合不足：模型太簡單，連訓練資料都學不好。最佳點在中間，而且只能靠驗證集找——因為訓練準確率在右端是一路上升的，它完全沒有告訴你該停在哪裡。這也是為什麼「訓練準確率 100%」從來不是一個好消息。

圖 14-17　模型複雜度：那條 U 形曲線

14.6.2 偏差-變異權衡

這兩種失敗的背後，是一組深刻的張力：偏差-變異權衡(Bias-Variance Tradeoff)。偏差(Bias)是模型因為太簡單而產生的系統性誤差(對應擬合不足);變異(Variance)是模型因為太複雜、對訓練資料太敏感而產生的不穩定(對應過度擬合)。兩者往往此消彼長：提高模型複雜度會降低偏差、但增加變異;降低複雜度則反之。機器學習調校的藝術，很大程度就是在這兩者間尋找最佳平衡點——這也呼應了全書反覆出現的「取捨」主題。



上一張圖的 U 形曲線，可以再拆得更細。這裡換成迴歸問題（分類的分解在數學上沒那麼乾淨），做法是用同一個真實函數重複產生 200 組各 25 筆的小資料，每組各配一次多項式，再看這 200 條曲線的集體行為。偏差是「200 條的平均」離真實函數有多遠：1 次多項式（直線）再怎麼平均也追不上彎曲的真實函數，偏差² 高達 0.482，這是系統性的錯，再多資料也救不了。變異是「200 條彼此差多少」：9 次多項式每換一批資料就抖出完全不同的形狀，變異高達 34.59，是 1 次多項式的 634 倍;可是它的偏差² 只有 0.142，不到直線的三分之一——也就是說它平均起來並沒有偏掉，只是每一次都亂跑。這正是過度擬合的本質：不是學不會，是學得太隨資料而變。總誤差等於偏差² + 變異 + 雜訊，而雜訊 0.203 是資料本身帶的，任何模型都消不掉，這就是誤差的下限。兩者一降一升，總和在 5 次最低（0.303）。這個分解解釋了本章所有的旋鈕：kNN 的 k 變大、SVM 的 gamma 變小、樹變淺，都是在「用偏差換變異」;反之則是「用變異換偏差」。而加資料是唯一能單獨壓低變異、又不增加偏差的手段——這正是下一張圖第一件武器最強的理論原因。

圖 14-18　偏差與變異：把誤差拆成兩半

14.6.3 對抗過度擬合的武器

對抗過度擬合有幾件常備武器，它們在第 15 章訓練神經網路時會反覆出現：其一，更多資料——資料愈多，模型愈難靠死背過關，這再次呼應「資料是燃料」;其二，資料增強(Data Augmentation)——對訓練影像做旋轉、平移、翻轉、調亮度(全是第 4、5 章的技術!)人工擴充資料的多樣性，這是影像領域對抗過度擬合最有效的手段之一;其三，正則化(Regularization)——限制模型的複雜度，懲罰過大的參數，這與第 7 章影像還原的正則化是同一個數學思想(以約束對抗病態);其四，及早停止——在驗證表現開始變差時就停止訓練，不讓模型繼續往「背誦」的方向走。這四件武器常被並列，但它們的效果其實差了一個量級。圖 14-19 把四件都套在同一個模型與同一套訓練流程上實測(起點：訓練 100%、測試 88.2%)：更多真實資料 97.3%、資料增強 92.3%、正則化 89.2%、及早停止 88.7%，四件一起上 98.5%。成本的排序恰好相反——改參

數免費、寫增強程式便宜、收集標註最貴。因此實務上的建議順序是：正則化與及早停止先試，資料增強次之，收集真實資料最後，但要知道最後那一件才是效果最大的。另外要留意效果不會線性相加(它們壓的是同一個變異)，而且組合使用時，及早停止的輪數必須用該組合自己的驗證曲線重新找。

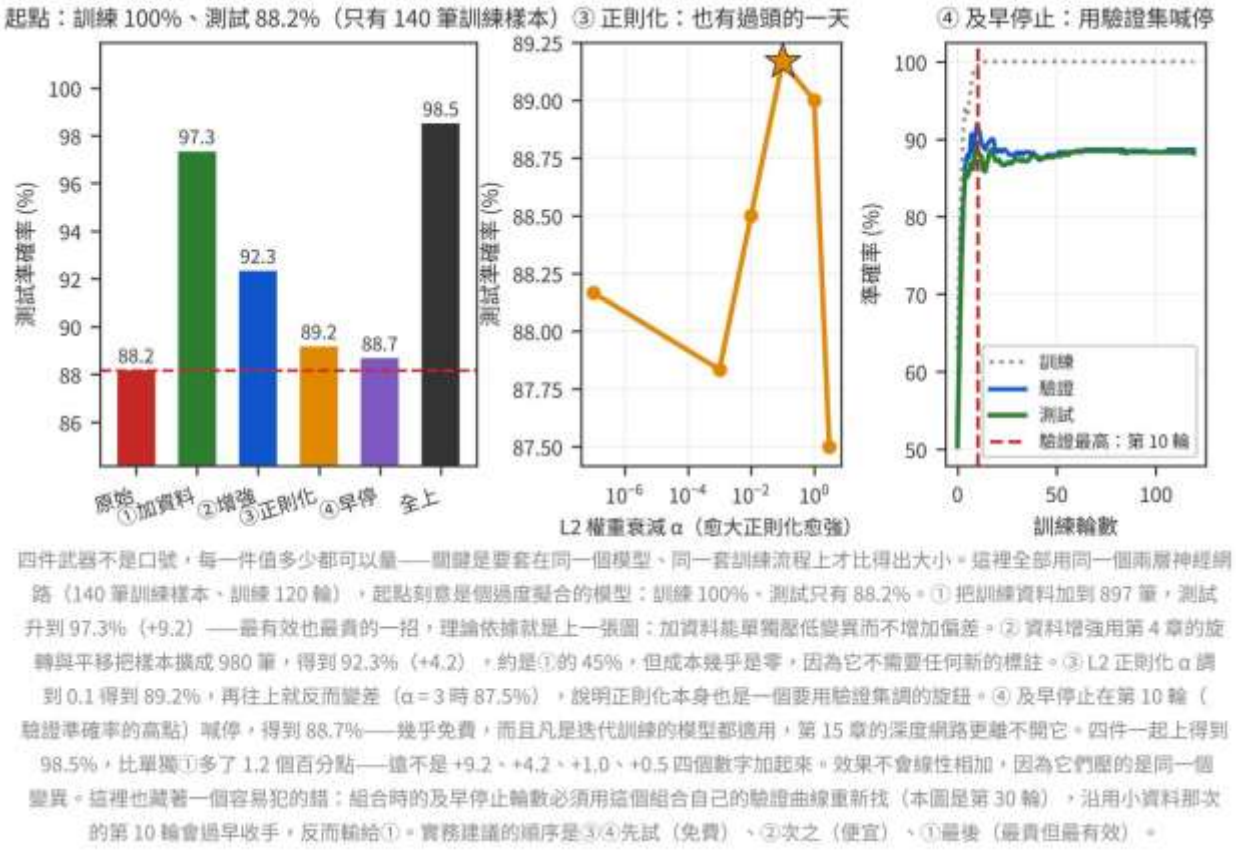


圖 14-19　四件武器，各值多少

請留意這裡的美妙連結：對抗過度擬合的資料增強，用的是第 4、5 章的幾何轉換與影像強化;正則化的思想來自第 7 章。機器學習不是憑空而來的新學問，它站在前面十三章的肩膀上——這正是本教科書把機器學習放在第 14 章、而非一開頭的用意。

14.6.4 推導：偏差-變異分解

14.6.2 節用文字說了偏差與變異此消彼長，圖 14-18 做了實測。這一小節把那條「總誤差 = 偏差² + 變異 + 雜訊」的等式推一遍。它只用到一個技巧——加一項再減一項——卻是整個機器學習理論裡最常被引用的分解。

為了推導乾淨，這裡用迴歸來談(分類問題的分解要麻煩得多，這也是圖 14-18 改用迴歸示範的原因)。設真實世界的關係是 y = f(x) + e，其中 f 是我們想學的真實函數、e 是量測雜訊，假設它平均為 0、變異數為 σ²，且與 x 無關。注意這個 σ² 是資料本身帶的，任何模型都消不掉——它是誤差的下限。

現在的關鍵是：我們的模型 g 是從一筆訓練資料學出來的，換一筆訓練資料就會學出不同的 g。所以 g(x) 是一個隨機變數，隨機性來自「抽到哪一筆訓練資料」。我們關心的是它在某個固定測試點 x 上的期望平方誤差 E[(y − g(x))²]，期望值同時對雜訊與訓練資料取。再令 m(x) = E[g(x)]，也就是「無限多次重抽訓練資料、學出無限多個模型之後的平均預測」——這正是圖 14-18 上排那條紅線的意思(以 200 條曲線的平均來近似)。

展開期望值，先把 y 換成 f(x) + e：

```
E[(y − g)²] = E[(f + e − g)²]
            = E[(f − g)²] + 2·E[e·(f − g)] + E[e²]
            = E[(f − g)²] + 0 + σ²
```

中間那項消失，是因為雜訊 e 與訓練資料無關、且平均為 0，所以 E[e·(f − g)] = E[e] · E[f − g] = 0。剩下的 E[(f − g)²] 就用那個加一項減一項的技巧：在括號裡同時加上與減去平均模型 m：

```
E[(f − g)²] = E[( (f − m) + (m − g) )²]
            = (f − m)² + 2·(f − m)·E[m − g] + E[(m − g)²]
            = (f − m)² + 0 + E[(g − m)²]
```

這一次中間項消失，是因為 m 的定義就是 g 的期望值，所以 E[m − g] = m − E[g] = 0;而 (f − m)² 不含隨機性，可以直接提到期望值之外。把兩段合起來，就得到了那條著名的等式：

```
E[(y − g(x))²]  =  (f(x) − m(x))²  +  E[(g(x) − m(x))²]  +  σ²
   總誤差       =       偏差²       +          變異         + 雜訊
```

三項各有清楚的意思。偏差² 是「平均模型離真實函數多遠」——它衡量模型族本身的表達能力不足，是系統性的錯，再多資料也救不了。變異是「每個模型離平均

模型多遠」——它衡量模型對訓練資料的敏感程度。雜訊則是資料自帶的下限。

圖 14-18 的實測把這三項都量了出來。1 次多項式(直線)配不上彎曲的真實函數，偏差² 高達 0.482、變異只有 0.055;9 次多項式每換一批資料就抖出完全不同的形狀，變異高達 34.59、是直線的 634 倍，但偏差² 只剩 0.142。兩者一降一升，總和在 5 次多項式處最低。而那個消不掉的雜訊項 $\sigma^2 = 0.45^2 = 0.203$，就是圖上那條怎麼樣都跨不過去的灰色虛線。

最後是這個分解的實務價值，它把本章所有的旋鈕串成了一條線。kNN 的 k 變大、SVM 的 gamma 變小、決策樹變淺、正則化係數變大——這些動作全都是「增加偏差、減少變異」;反方向則是「減少偏差、增加變異」。理解這一點，調參就不再是亂試：先診斷自己落在哪一端(訓練測試都差是偏差高、訓練好測試差是變異高)，再往反方向轉。而四件武器中「更多資料」之所以特別珍貴，正是因為它是唯一能單獨壓低變異、卻不增加偏差的手段——圖 14-19 實測它帶來 9.2 個百分點，是另外三件分別帶來的好幾倍。

14.7 應用案例

14.7.1 案例一：手寫數字辨識(傳統流程)

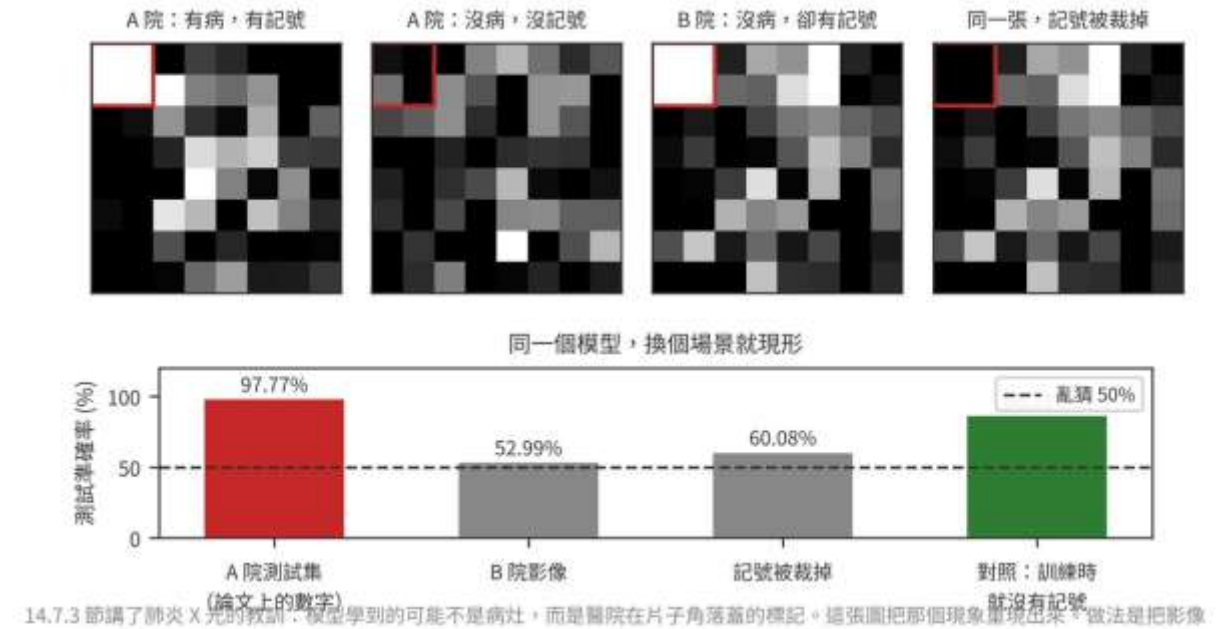
這是呼應叫獸開講、也是您課程 MNIST 實作的傳統版本。情境：辨識 0 到 9 的手寫數字，但不使用深度學習，以凸顯「特徵」這一環的重要。流程：對每張數字影像萃取特徵——可以是簡單的「像素攤平」，也可以是更聰明的 HOG(方向梯度直方圖，第 13 章特徵思想的延伸)→ 用 SVM 或 kNN 訓練分類器 → 以測試集評估、看混淆矩陣哪些數字最容易搞混(通常是 4 與 9、3 與 8、7 與 1)。這個案例讓學生親身體會：同樣的分類器，換用更好的特徵(HOG 優於原始像素)，準確率會顯著提升——這正是第 15 章深度學習要「自動學特徵」的動機所在。

14.7.2 案例二：水果或零件的品質分類

情境：農產分級(將水果依外觀分為特級、良級、次級)或工業零件的良品/不良品分類——這是機器學習在雲林在地農業與製造業最實際的應用。流程：拍攝大量已分級的樣本並標註 → 萃取特徵(第 8 章的顏色直方圖判斷成熟度、第 13 章的形狀特徵判斷外形、紋理特徵判斷表面瑕疵)→ 訓練 SVM 分類器 → 部署到分級生產線。這個案例的價值在於：它示範了前面各章的「特徵工程」如何餵給機器學習模型，是傳統影像處理與機器學習銜接的典型範例，也說明了為什麼在樣本有限的中小企業場景，「特徵工程 + SVM」常比動輒需要上萬標註樣本的深度學習更務實。

14.7.3 案例三：資料切分陷阱的真實教訓

這個案例不寫程式，而是講一個發人深省的故事，凸顯 14.5 節資料切分的重要。曾有研究團隊訓練模型從胸部 X 光辨識肺炎，在測試集上準確率極高，上線後卻表現崩壞。追查發現：訓練資料中，重症病人的 X 光多來自某幾家配備較舊機器的醫院，模型竟然「偷懶」學會了辨識影像角落的醫院標記與機器雜訊特徵，而非真正的病灶——它在資料中找到了一條與答案高度相關、卻毫無醫學意義的「捷徑」。因為測試集也來自同樣的醫院分布，這個捷徑在測試時依然有效，直到面對新醫院的資料才現形。



14.7.3 節講了肺炎 X 光的教訓：模型學到的可能不是病灶，而是醫院在片子角落蓋的標記。這張圖把那個現象重現出來。做法是把影像加重雜訊（讓真正的任務變難），再於左上角蓋一個與標籤有 97% 相關的小方塊，就像某家醫院的重症片則好都用同一台機器拍、帶著同一個記號。訓練與測試都來自這家醫院時，模型拿到 97.77%，論文、簡報、驗收報告都會很好看。但把同一個模型拿去 B 醫院（記號還在、只是與病情無關），準確率掉到 52.99%，幾乎就是丟銅板;把記號整個裁掉也只剩 60.08%——它從頭到尾幾乎沒在看影像本身。對照組最刺眼：訓練時就沒有記號的模型只能老實學影像，拿到 86.23%，比那個漂亮的 97.77% 低了 11.5 個百分點，卻是四根柱子裡唯一真的能用的模型。這件事最可怕的地方在於：如果測試集和訓練集來自同一個偏差分布，那麼再嚴謹的切分、再漂亮的交叉驗證都抓不到它——測試集本身就帶著同樣的捷徑。唯一的解法是換一批真正獨立來源的資料驗收，並且實際去看模型在看影像的哪裡（第 15 章的類別活化圖就是為此而生）。這是工程倫理，不只是技術細節。

圖 14-20　捷徑學習：測試分數很高，卻什麼都沒學到

這個真實教訓有兩層啟示：其一，資料的收集與切分若有偏差(不同類別來自不同來源)，模型會學到「捷徑」而非真正的規律，測試表現會給出虛假的信心。其二，這也呼應了第 7 章「還原 vs. 腦補」與第 9 章醫學影像的倫理提醒——在醫療等高風險領域，一個「準確率很高」的模型未必可信，必須深究它到底學到了什麼。機器學習工程師的責任，不只是把準確率調高，更是理解模型為何做出判斷、資料是否公平。這是技術素養，也是工程倫理。

14.8 本章與全書的觀念連結總表

相關章節	共用的觀念	機器學習中的角色
第 4 章 幾何轉換	旋轉、平移、翻轉	資料增強對抗過度擬合的主要手段
第 5 章 空間域	亮度、對比調整	資料增強的另一組操作

強化		
第 7 章 影像還原	正則化	對抗過度擬合的正則化與病態問題正則化同源
第 8 章 色彩處理	顏色直方圖	品質分類的顏色特徵
第 13 章 特徵擷取	SIFT、HOG、形狀特徵	傳統機器學習的「特徵」環節
第 15 章 深度學習	神經網路、自動特徵	深度學習是機器學習的分支，遵循同樣的訓練評估原則
第 16 章 物件偵測	COCO、mAP	COCO 資料集;評估指標延伸自本章
第 9、10 章 分割/二值	準確、召回、誤報漏檢	評估指標的取捨思維一以貫之

表 14-2 第 14 章與全書其他章節的觀念連結

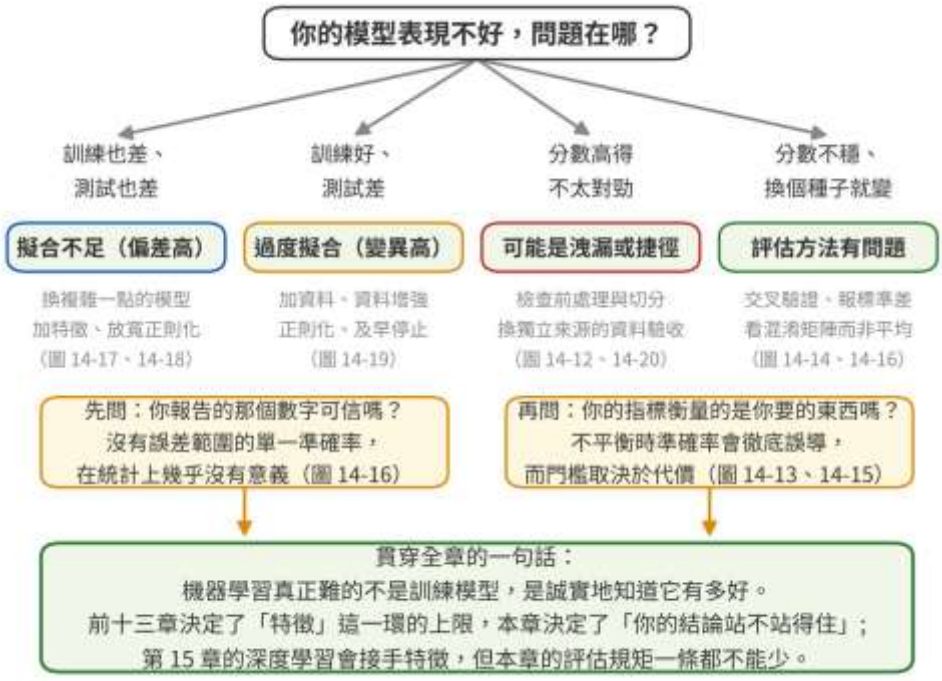


圖 14-21　第 14 章的決策地圖

程式實作 Lab 14

Lab 14-1 分類器擂台：kNN vs. SVM

目標：比較兩種經典分類器。(a)以程式 14-1 在手寫數字資料集上訓練 kNN 與 SVM，比較準確率與預測速度;(b)調整 kNN 的 k 值(1、5、15、50)，觀察準確率如何變化，找出最佳 k，並解釋 k 太小與太大各自的問題;(c)調整 SVM 的 C 與 gamma，觀察對結果的影響;(d)畫出兩者的混淆矩陣，找出最容易被搞混的數字對(如 4 與 9)，並思考為什麼;(e)討論：kNN 為什麼「不訓練」?這帶來什麼優點與缺點?

Lab 14-2 特徵的威力

目標：親身體會「特徵」這一環的決定性。(a)對同一個手寫數字資料集，分別用兩種特徵訓練 SVM：一是「原始像素攤平」，二是「HOG 特徵」;(b)比較兩者的準確率——通常 HOG 會明顯勝出;(c)思考：為什麼更好的特徵能提升同一個分類器的表現?這說明了傳統機器學習中「特徵工程」的重要性;(d)延伸討論：第 15 章的深度學習如何「自動」完成這件事，免去人工設計特徵的負擔?

Lab 14-3 過度擬合的現形與對抗

目標：親眼見證過度擬合。(a)刻意用一個很複雜的模型(如 SVM 的 gamma 設得很大)在少量訓練資料上訓練，記錄訓練準確率(會很高)與測試準確率(會很低)，這就是過度擬合;(b)畫出「訓練集大小 vs. 訓練/測試準確率」的學習曲線，觀察兩條線的落差如何隨資料增加而縮小——驗證「更多資料對抗過度擬合」;(c)加入資料增強(用第 4 章旋轉、第 5 章調亮度擴充訓練集)，觀察測試準確率是否提升;(d)討論：如何從學習曲線判斷一個模型是擬合不足還是過度擬合?各該怎麼補救?

Lab 14-4 品質分類器(整合前面各章)

目標：打造一個結合影像處理與機器學習的實用系統。(a)拍攝或收集兩三類物件的影像(如不同成熟度的水果、良品與不良品的零件)，各數十張並標註;(b)萃取特徵：

用第 8 章顏色直方圖 + 第 13 章形狀/紋理特徵，組成特徵向量;(c)正確切分訓練/測試集，訓練 SVM 分類器;(d)以混淆矩陣、精確率、召回率評估，並依應用場景(如「不良品絕不能放過」)判斷該重召回還是重精確;(e)反思：你的資料收集有沒有落入 14.7.3 節的「捷徑陷阱」?(例如所有良品都在同一背景下拍攝，模型會不會學到背景而非物件?)

Lab 14-5 門檻、代價與不平衡：把商業判斷寫進模型

目標：體會準確率的陷阱，並親手把代價結構轉成判定門檻。(a)用 load_digits 造一個人工的不平衡問題：挑一個數字當「不良品」，只保留十幾筆，其餘全當良品，計算不良品的佔比;(b)訓練一個未做任何處理的 SVM，記錄準確率、精確率、召回率與 F1——你應該會看到很高的準確率搭配 0% 的召回率，並與「一律判良品」這個什麼都沒學的基準比較，確認兩者分數相同;(c)加上 class_weight="balanced" 重訓，觀察四個指標各自怎麼變;(d)改用 predict_proba 取得分數，把門檻從 0.02 掃到 0.98，畫出精確率、召回率、F1 隨門檻變化的三條曲線，找出 F1 最高的門檻，並與預設的 0.5 比較差了幾個百分點;(e)設定三種代價(漏抓 10 倍貴、一樣貴、誤報 10 倍貴)，各自算出總代價隨門檻的曲線與最佳門檻，確認三者不同;(f)寫一段 150 字的說明，向一位不懂機器學習的廠長解釋「為什麼我們不用準確率驗收，以及為什麼門檻要由您來決定」。延伸：改用 average_precision_score 計算 AP，說明它為什麼與門檻無關。

Lab 14-6 誠實評估工作坊：切分、洩漏與捷徑

目標：親手製造三種常見的評估錯誤，再親手修好，把「誠實地知道模型有多好」變成肌肉記憶。(a)單次切分的不穩：固定資料與模型，只換 40 個隨機種子做 7:3 切分，記錄 40 個測試準確率，畫出直方圖，算出全距與標準差;再對同一份資料做 40 次 5 折交叉驗證取平均，比較兩組的標準差差幾倍;(b)排名翻轉：在同樣的 40 次切分中同時評估 SVM 與 kNN，統計「誰比較好」翻轉了幾次，寫下你對「只切一次就選模型」的結論;(c)製造洩漏：取一小批資料，把標籤完全打亂(此時真實準確率上限等於

亂猜)，然後故意在切分前用全部資料做特徵選擇，看測試準確率能虛高到多少;再改成先切分、只在訓練集上做特徵選擇，確認分數掉回亂猜水準;(d) 用 sklearn.pipeline.Pipeline 把前處理與模型包起來重做一次(c)，說明為什麼 Pipeline 從結構上就防止了這個錯誤;(e)製造捷徑：在每張影像的角落蓋一個與標籤高度相關的小方塊，訓練後分別在「同分布測試集」、「記號與標籤無關的影像」、「記號被裁掉的影像」三種情境下評估，並與「訓練時就沒有記號」的對照模型比較;(f)把上述所有數字整理成一頁報告，每個數字都附上標準差或信賴區間。延伸：用 sklearn.model_selection.GroupKFold 示範「同一來源的樣本不可跨越切分」，說明它在(e)那種場景能擋下什麼。

本章重點整理

- 機器學習是典範轉移：從「人寫規則」到「給範例讓機器歸納規則」;分監督式(有標籤，含分類與迴歸)、非監督式(無標籤，分群/降維)、強化學習(獎懲互動)三類，本章與深度學習聚焦監督式影像分類。
- 影像分類五環節：資料、特徵、模型、訓練、評估;傳統機器學習依賴人工設計特徵(第 13 章)，深度學習則自動學特徵——這是兩者的分野。
- 重要資料集：MNIST(入門 Hello World)、CIFAR(彩色自然物體)、ImageNet/ILSVRC(2012 引爆深度學習)、COCO(複雜場景偵測分割);資料是機器學習的燃料，常比演算法更決定成敗。
- kNN 以最近 k 個鄰居投票、不訓練但預測慢;SVM 以最大間隔超平面分類，核技巧處理非線性，是深度學習前的黃金分類器。
- 資料須切分訓練/驗證/測試，測試集嚴格保密以模擬真實表現;準確率在類別不平衡時會誤導，須輔以混淆矩陣、精確率(不誤報)、召回率(不漏抓)，依應用代價權衡。

- 擬合不足(太簡單、訓練測試都差)、過度擬合(太複雜、背下雜訊、訓練好測試差)源於偏差-變異權衡;對抗過度擬合靠更多資料、資料增強(第 4、5 章)、正則化(第 7 章)、及早停止。

習題

1. 機器學習相對於傳統程式設計，發生了什麼「典範轉移」?為什麼有些問題非用機器學習不可?

   提示與參考解答：傳統程式：人寫死規則，電腦套用到資料得答案。機器學習：人給資料和答案，電腦自己歸納出規則。轉移在於「從寫規則到給範例」。有些問題(辨認手寫數字、人臉、語音)的規則複雜到人類根本寫不出來，但範例俯拾皆是，只能讓機器從範例自己學。

2. 請說明監督式、非監督式、強化學習的差別，各舉一個影像或機器人相關的例子。

   提示與參考解答：監督式：資料有輸入與正確標籤，學習預測答案(如 MNIST 數字分類)。非監督式：資料無標籤，自己發現結構(如把相似影像自動分群)。強化學習：智能體與環境互動、依獎懲學策略(如機器人學會抓取物件、AlphaGo 下圍棋)。

3. 影像分類的五個環節是什麼?傳統機器學習與深度學習在哪一個環節產生了根本差異?

   提示與參考解答：五環節：資料、特徵、模型、訓練、評估。根本差異在「特徵」：傳統機器學習依賴人工設計特徵(SIFT、HOG)，模型只做「特徵到類別」;深度學習把「像素到特徵」與「特徵到類別」整合進同一網路，端到端自動學習特徵。

4. 為什麼說「資料是機器學習的燃料」?請以 ImageNet 對深度學習的影響說明。

   提示與參考解答：演算法是引擎、資料是燃料，沒燃料引擎跑不動。ImageNet(李飛飛團隊，逾 1400 萬張影像)提供了規模空前的訓練資料，2012 年 AlexNet 正是靠它在 ILSVRC 奪冠、引爆深度學習革命。沒有 ImageNet 就沒有深度學習爆發—

—是「先建大資料集」的洞見為演算法鋪好跑道，印證資料常比演算法更重要。

5. 承叫獸開講：MNIST 為什麼要混合「人口普查局員工」與「高中生」兩種來源的手寫數字?這與資料切分有什麼關聯?

   提示與參考解答：普查局員工字跡工整(較易)、高中生字跡潦草(較難)，混合是為了讓訓練集與測試集的難度分布一致，結論才可靠。若訓練全易、測試全難會低估模型能力，反之則高估。這正是資料切分公平性的體現——訓練與測試的分布必須一致，評估才可信。

6. kNN 的分類原理是什麼?為什麼說它「不訓練」?它有哪些缺點?

   提示與參考解答：原理：看新樣本在特徵空間最近的 k 個鄰居，由它們投票決定類別(物以類聚)。「不訓練」是因為它沒有需學習調整的參數，只把訓練樣本記住，預測時才算距離找鄰居。缺點：預測慢(每次都要比全部樣本)、記憶體耗費大、對特徵尺度與高維敏感(維度災難)。

7. SVM 的「最大間隔」原則是什麼?為什麼它有助於泛化?什麼是核技巧?

   提示與參考解答：最大間隔：不只要能分開兩類，還要讓分界線(超平面)離兩邊最近樣本盡可能遠;決定分界線的關鍵樣本即支持向量。它有助泛化，因為留有餘裕、對新樣本更有容錯能力。核技巧：當資料在原空間線性不可分時，隱含地映射到更高維空間使其線性可分，且不必真的計算高維座標，讓 SVM 能處理非線性分類。

8. 為什麼評估模型不能用訓練時看過的資料?訓練、驗證、測試三個集合各扮演什麼角色?

   提示與參考解答：用看過的資料評估，無法分辨模型是「學會規律」還是只「背下答案」(如考背過的題)。訓練集調整參數;驗證集調超參數、選模型、決定何時停止;測試集在一切定案後最後評估，模擬上線遇到全新資料的表現，須嚴格保密不可反覆使用，否則評估會過度樂觀。

9. 準確率在什麼情況下會誤導?請以罕見疾病為例，並說明精確率與召回率如何補救。

   提示與參考解答：類別不平衡時誤導：某病盛行率 1%，「一律判沒病」的模型準確率 99% 卻抓不出任何病人。精確率(判為陽性中真陽性的比例，衡量不誤報)與召回率(真陽性中被抓出的比例，衡量不漏抓)能揭示這個問題;癌症篩檢重召回(漏抓代價

大)，垃圾郵件重精確(誤判重要郵件代價大)。

10. 過度擬合與擬合不足各是什麼?如何從訓練與驗證表現診斷?它們與偏差-變異權衡有何關係?

    提示與參考解答：擬合不足：模型太簡單、抓不住規律，訓練與測試都差(高偏差)。過度擬合：模型太複雜、背下雜訊，訓練好但測試崩壞(高變異)。診斷：兩者都差是擬合不足;訓練好、驗證差且落差大是過度擬合。偏差-變異權衡：提高複雜度降偏差但增變異，反之亦然，調校即在兩者間找平衡。

11. 對抗過度擬合有哪四種常備武器?其中資料增強與正則化分別與本書哪幾章相關?

    提示與參考解答：四武器：更多資料、資料增強、正則化、及早停止。資料增強用第 4 章幾何轉換(旋轉、平移、翻轉)與第 5 章影像強化(調亮度、對比)擴充訓練資料多樣性;正則化(限制模型複雜度、懲罰過大參數)與第 7 章影像還原的正則化同源——都是以約束對抗病態/過度擬合。

12. (綜合思辨題)承 14.7.3 節肺炎 X 光的教訓：一個測試準確率很高的模型，為什麼上線後可能崩壞?這給機器學習工程師什麼責任上的啟示?

    提示與參考解答：模型可能學到與答案高度相關、卻無實質意義的「捷徑」(如醫院標記、機器雜訊)，而非真正的病灶。若測試集與訓練集來自同樣的偏差分布，捷徑在測試時仍有效，直到面對新分布才現形。啟示：資料收集與切分若有偏差，模型會學捷徑、測試給出虛假信心;工程師的責任不只是調高準確率，更要理解模型為何做出判斷、資料是否公平——這是技術素養，也是工程倫理(呼應第 7、9 章)。

13. 為什麼「訓練準確率 100%」通常不是一個好消息?請用圖 14-17 的兩條曲線說明，並說明該用什麼來決定模型複雜度。

    提示與參考解答：訓練準確率隨複雜度單調上升，模型複雜到一定程度就會把訓練資料(連同其中的錯誤標註與雜訊)背下來，因此 100% 只代表「背完了」，不代表學會規律。圖 14-17 實測：決策樹深度加到 20 以上時訓練 100%，測試卻從深度 8 的 68.5% 掉到 61.0%;SVM 的 gamma 加大時訓練 100%，測試從 92.8% 崩到 12.0%(十類問題下幾乎等於亂猜)。兩條線之間的落差就是過度擬合的量。決定複雜度只能靠驗證集(或交叉驗證)，因為訓練準確率在過度擬合區是一路上升的，完全沒有提供「該停在哪裡」的訊號。

14. 手上只有 200 筆測試樣本，你的模型測得 90% 準確率。請用 14.5.3 節的公式算出 95% 信賴區間，並說明若另一個模型測得 92%，你能不能宣稱它比較好。

    提示與參考解答：SE = √(0.9 × 0.1 / 200) = √0.00045 ≈ 0.0212，即 2.12 個百分點;95% 區間為 90% ± 1.96 × 2.12% ≈ 90% ± 4.2%，也就是約 85.8% 到 94.2%。92% 落在這個區間內，兩個區間重疊得很厲害，因此不能宣稱較好——這個差距在統計上無法與抽樣雜訊區分。要做出可信的宣稱，有三條路：把測試集加大(SE 隨 √n 下降，要把誤差砍半需四倍樣本)、改用交叉驗證並報告標準差、或在多個隨機種子下重複實驗看排名是否穩定。圖 14-16 的實測提醒：同一份資料只換種子，單次切分的結果就能差 8.9 個百分點。

15. 什麼是「資料洩漏」(data leakage)?請舉出三種常見的洩漏途徑，並說明為什麼洩漏比單純的過度擬合更危險。

    提示與參考解答：資料洩漏指訓練過程中以任何形式用到了測試集的資訊，使測試分數不再代表真實表現。三種常見途徑：(1)前處理先做再切分——標準化、特徵選擇、PCA 若在全部資料上擬合，測試集的統計量就已經流進模型;(2)資料增強先做再切分——同一張原圖的增強版本同時出現在訓練與測試集，等於拿看過的東西考試;(3)反覆用測試集調參——每看一次測試集做一次決定，就洩漏一點點，累積起來就讓測試集失效。洩漏比過度擬合更危險，是因為過度擬合會被誠實的測試集抓到，洩漏卻讓測試集本身失去偵測能力——分數漂亮、問題看不見，直到上線才爆發。圖 14-12 的實測顯示：在完全隨機的標籤上(真實準確率上限 50%)，先做特徵選擇再切分能讓測試準確率虛高到 75%。

16. 承上題：請寫出正確的前處理順序，並說明 scikit-learn 的 Pipeline 為什麼能從結構上防止這類錯誤。

    提示與參考解答：正確順序是「先切分 → 只在訓練集上 fit 前處理器 → 用同一個前處理器 transform 訓練集與測試集」。關鍵是測試集只能被 transform、絕不能參與 fit。Pipeline 把前處理與模型包成一個物件，呼叫 pipeline.fit(X_tr, y_tr) 時，它只會在訓練資料上擬合每一個前處理步驟;在交叉驗證中(cross_val_score(pipeline, X, y, cv=5))，每一折都會重新擬合一次前處理器，確保該折的驗證資料完全沒參與。若不用 Pipeline、自己先做 scaler.fit(X) 再切分，就犯了上題的第一種洩漏。所以 Pipeline 不只是寫法簡潔，它是一種把正確流程寫死在結構裡的防呆設計。

17. 在一條瑕疵檢測產線上，不良品只佔 0.86%。請說明為什麼「一律判良品」的模型能拿到 99.12% 的準確率，以及你該改用哪些指標，又該如何讓模型真的抓到不良品。

    提示與參考解答：準確率只計算「判對的比例」，當某一類佔了 99% 以上，全部判成那一類就自動拿到 99% 以上。圖 14-13 的實測：盛行率 0.86% 時，「一律說良品」的準確率 99.12%，但精確率與召回率都是 0——它一個不良品也沒抓到，對產線毫無價值。更警醒的是，未做處理的 SVM 給出一模一樣的 99.12% 與 0% 召回率，因為最小化整體錯誤的最快方法就是放棄少數類。應改看混淆矩陣、精確率、召回率、F1，以及與門檻無關的 AP(圖 14-15)。讓模型真的抓到不良品的手段包括：設定類別權重(實測 class_weight 讓召回率從 0% 升到 80%、準確率 99.82%)、重抽樣(過採樣少數類或欠採樣多數類)、調整判定門檻、以及在評估時改用對少數類敏感的指標。

18. 為什麼說「0.5 從來不是一個神聖的門檻」?請以漏抓代價是誤報十倍、與誤報代價是漏抓十倍兩種情境，說明最佳門檻會往哪個方向移動，以及這個決定該由誰來做。

    提示與參考解答：分類器輸出的是分數或機率，0.5 只是預設的切點，與任務的代價結構毫無關係。門檻調低會判更多樣本為陽性，召回率上升、精確率下降(寧可錯殺);門檻調高則反之(寧可放過)。漏抓十倍貴時(例如瑕疵品流到客戶手上)，最佳門檻往下移，寧可多攔一些;誤報十倍貴時(例如誤判害整批重工)，最佳門檻往上移。圖 14-15 的實測：同一個模型、同一批資料，三種代價結構下的最佳門檻分別是 0.04、0.60 與 0.90。這個決定不是演算法能自動算出來的，必須由懂現場的人提供兩類錯誤的相對代價——它是把商業判斷寫進系統的那一步，也呼應第 9 章 Canny 雙門檻與第 10 章 Viola-Jones 的同一類思維。

19. 請說明交叉驗證的做法，以及它解決了單次切分的什麼問題。當資料量達到數十萬筆時，還需要交叉驗證嗎?

    提示與參考解答：k 折交叉驗證把資料切成 k 等份，輪流拿一份當驗證集、其餘 k−1 份訓練，共做 k 次，每一筆樣本都剛好被驗證到一次，最後取平均。它解決的是單次切分的高變異：圖 14-16 實測同一份 300 筆資料、只換 40 個隨機種子，單次切分的結果從 90.0% 到 98.9%、標準差 2.14;5 折平均的標準差降到 0.66，穩定

約 3.3 倍。更嚴重的是排名會翻轉——該實驗中 SVM 與 kNN 的勝負在 40 次裡翻轉了 13 次。資料達數十萬筆時，單一次切分出來的測試集本身就夠大(由 14.5.3 節的公式，n = 100,000 時標準誤只有 0.1% 上下)，加上交叉驗證要把訓練成本乘以 k 倍，因此大資料場景通常改用單次固定切分;但分層抽樣、以及「測試集只在最後用一次」的紀律依然不能省。

20. 請用偏差-變異的語言，說明以下四個調整各自往哪個方向移動：(a)kNN 的 k 從 1 調到 50;(b)SVM 的 gamma 加大;(c)決策樹的最大深度變淺;(d)訓練資料從 140 筆加到 897 筆。

    提示與參考解答：(a)k 變大 → 更多鄰居投票、決策邊界變平滑 → 變異下降、偏差上升(k=1 時訓練準確率 100% 是典型的高變異)。(b)gamma 加大 → 每個支持向量影響範圍變小、邊界變皺 → 變異上升、偏差下降(圖 14-17 實測 gamma 過大時測試掉到 12.0%)。(c)樹變淺 → 模型變簡單 → 變異下降、偏差上升。(d)加資料是特例：它單獨壓低變異而不增加偏差，因為模型族沒有改變、只是估計得更穩(圖 14-19 實測 +9.2 個百分點，遠勝其他三件武器)。前三項都是在同一條權衡曲線上左右移動，只有(d)是把整條曲線往下推——這正是「資料是燃料」這句話的理論版本。

21. 對抗過度擬合的四件武器，實測效果分別是多少?請依「成本」與「效果」排出你的實務使用順序，並說明理由。

    提示與參考解答：圖 14-19 在同一個模型與訓練流程上量過四件武器(起點：140 筆訓練樣本，訓練 100%、測試 88.2%)：更多真實資料 97.3%(+9.2)、資料增強 92.3%(+4.2)、正則化 89.2%(+1.0)、及早停止 88.7%(+0.5)，四件一起上 98.5%。成本排序正好相反：正則化與及早停止只是改參數，幾乎零成本;資料增強只需程式(第 4、5 章的旋轉、平移、亮度調整)，不必新標註;收集並標註真實資料最貴。因此實務順序建議是先試正則化與及早停止、再做資料增強、最後才投資收集資料。要注意效果不會線性相加(它們壓的是同一個變異)，而且組合使用時及早停止的輪數必須用該組合自己的驗證曲線重新找——沿用小資料時的輪數會過早收手。

22. 什麼是「捷徑學習」(shortcut learning)?為什麼嚴謹的資料切分與交叉驗證都抓不到它?該怎麼辦?

    提示與參考解答：捷徑學習指模型找到一個與答案高度相關、卻與任務本質無關的特徵(如醫院在片子角落蓋的標記、不同來源的機器雜訊、背景或拍攝條件的差異)，

並用它取代真正該學的規律。切分與交叉驗證抓不到它，是因為這兩者都假設訓練集與測試集來自同一個分布;若偏差存在於整份資料中，測試集就帶著同樣的捷徑，分數依然漂亮。圖 14-20 的實測：加上與標籤 97% 相關的角落記號後，同分布測試集拿到 97.77%，換一家「醫院」掉到 52.99%、把記號裁掉只剩 60.08%(二元任務亂猜是 50%);而訓練時就沒有記號的對照模型只有 86.23%，卻是唯一能用的。對策：用真正獨立來源的資料驗收、刻意做跨來源(跨醫院、跨產線、跨季節)測試、用可解釋性工具(第 15 章的類別活化圖)實際去看模型在看哪裡、並在資料收集階段就避免讓類別與來源綁在一起。

23. (推導題)承 14.4.3 節：請說明為什麼可以自由把 w 與 b 同乘一個常數，以及 SVM 為什麼要把最近樣本的 |w·x + b| 訂為 1。若改訂為 2，最後的目標函數會變成什麼?

    提示與參考解答：$w·x + b = 0$ 與 $(cw)·x + cb = 0$ 對任何 $c \neq 0$ 描述的是同一個超平面，因為把方程式兩邊同乘常數不改變解集合。這表示 (w, b) 有一個多餘的縮放自由度，可以任意訂定而不失一般性。訂為 1 是為了讓約束寫成乾淨的 $y·(w·x + b) \geq 1$，並讓間隔有簡潔的表達式 $2 / |w|$。若改訂為 2，則最近樣本滿足 $|w·x + b| = 2$，距離變成 $2 / |w|$、間隔變成 $4 / |w|$，約束變成 $y·(w·x + b) \geq 2$，目標仍是最小化 $|w|$(只差一個常數倍)。換言之，最佳的那個超平面完全沒變，變的只是 w 的長度單位——這正說明了那個 1 是一個方便的約定，不是數學上的必要條件。

24. (推導題)承 14.6.4 節：在偏差-變異分解中，為什麼交叉項 E[e·(f – g)] 與 E[m – g] 都會消失?若雜訊 e 的平均不是 0 而是某個常數 c，等式會變成什麼?

    提示與參考解答：第一個交叉項消失，是因為雜訊 e 與訓練資料獨立，所以期望值可以拆開成 $E[e] · E[f – g]$，而 $E[e] = 0$。第二個交叉項消失，是因為 m 依定義就是 g 的期望值，所以 $E[m – g] = m – E[g] = m – m = 0$。兩者都用到「某一項的期望為 0」這個事實，但理由不同：前者來自雜訊的假設，後者來自平均模型的定義。若 $E[e] = c \neq 0$，則 $E[e^2] = \sigma^2 + c^2$，且交叉項變成 $2c · E[f – g] = 2c · (f – m)$，等式多出兩項。實務上這表示量測系統有系統性偏移(例如感測器未歸零)，它會被模型吸收成一個常數偏差;正確的做法是先校正資料(第 5 章的影像強化與第 7 章的還原)，而不是讓模型去補償。

25. (綜合實作思辨題)你要為雲林一家柑橘包裝廠做一套分級系統，目前只拿到 300 張已分級的照片，全部在同一個上午、同一條產線拍攝。請規劃完整流程，並指出這份資料最可能埋下哪三個陷阱、各要如何處置。

    提示與參考解答：流程應涵蓋本章五環節——資料(切分策略先訂好)、特徵(第 8 章顏色直方圖判成熟度、第 13 章 HOG/LBP 判外形與表面)、模型(樣本少，SVM 優於深度學習)、訓練(交叉驗證調 C 與 gamma)、評估(混淆矩陣、各級的精確率與召回率、依代價選門檻)。三個陷阱：(1)資料量太少——300 張分三級、再切測試集後每級只剩幾十張，由 14.5.3 節的公式，測試集 60 筆時標準誤高達 ±6 個百分點，任何小幅改善都不可信;處置是用交叉驗證、報標準差，並持續累積資料。(2)捷徑與分布偏差——同一個上午、同一條產線代表光照、背景、相機角度完全一致，模型極可能學到與分級相關的拍攝條件而非果實本身(圖 14-20 的教訓);處置是跨時段、跨天候、跨批次重新收集驗收資料，並做資料增強模擬光照變化(第 5 章)。(3)類別不平衡——次級品通常遠少於良品，準確率會被多數類灌水(圖 14-13);處置是看混淆矩陣與各級召回率、設類別權重、並與廠方確認「漏判次級品」與「誤判良品」的實際代價，據此定門檻(圖 14-15)。最後要留一句工程倫理：上線後仍須持續抽檢回饋，因為柑橘的外觀會隨季節與產區改變，模型會隨時間漂移。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 12(物件辨識與分類基礎)。
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P., Gradient-Based Learning Applied to Document Recognition, Proc. IEEE, 1998.(LeNet 與 MNIST，本章叫獸開講之史料來源)
- Deng, J., et al. (Li, F.-F.), ImageNet: A Large-Scale Hierarchical Image Database, CVPR, 2009.
- Cortes, C., & Vapnik, V., Support-Vector Networks, Machine Learning, 1995.(SVM 原始論文)

- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.
- Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.(機器學習經典教科書)
- 補充教材與課程資訊：@prof(https://page.line.me/prof)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 15 章 深度學習

本章學習目標

- 說明從手工特徵到學習特徵的典範轉移，理解深度學習「端到端」的核心精神。
- 推導神經網路的基本構件：神經元、激勵函數、損失函數與反向傳播。
- 闡述卷積神經網路(CNN)的三大構件：卷積層、池化層、全連接層，理解其為何適合影像。
- 認識經典 CNN 架構譜系(LeNet→AlexNet→VGG→GoogLeNet→ResNet→MobileNet)的演進脈絡。
- 以三大框架(TensorFlow、Keras、PyTorch)實作 MNIST 手寫數字分類。
- 運用遷移學習與資料增強，在小資料上訓練出高效能的影像模型。

叫獸開講|黃仁勳與 AlexNet：兩顆遊戲顯卡掀起的革命

2012 年 9 月，電腦視覺界發生了一場地震。在權威的 ImageNet 影像辨識競賽(ILSVRC)上，多年來各隊都靠著人工設計特徵配上傳統分類器，辛苦地把錯誤率一個百分點、一個百分點地往下磨，當時的最佳成績卡在約 26% 的 top-5 錯誤率。然後，來自多倫多大學的一支隊伍——Alex Krizhevsky、Ilya Sutskever，以

及他們的指導教授、後來被稱為「深度學習教父」的 Geoffrey Hinton——投下了一顆震撼彈：他們的模型把錯誤率一舉壓到 15.3%，足足領先第二名近 11 個百分點。在一個習慣以零點幾個百分點論輸贏的領域，這個差距簡直是不同次元的存在。這個模型，就是後來以第一作者命名的 AlexNet。

AlexNet 憑什麼?它其實是一個八層的卷積神經網路(五層卷積、三層全連接)，概念上並非全新——它的祖先正是第 14 章 LeCun 讀支票的 LeNet。真正讓它脫胎換骨的是三個關鍵：第一，它用了 ReLU 激勵函數，讓深層網路的訓練快了好幾倍;第二，它用了 Dropout 這個新招來對抗過度擬合(第 14 章的老朋友);第三——也是最出人意料的——它是在兩張 NVIDIA GTX 580 上訓練的。沒錯，是遊戲玩家用來跑 3A 大作的那種顯示卡。

這第三點，正是這個故事最迷人的轉折。原本為了「讓遊戲畫面更逼真」而設計的圖形處理器(GPU)，因為天生擅長大量平行的數值運算，竟意外地與神經網路的訓練需求完美契合——訓練神經網路，本質上就是海量的矩陣乘法，而這正是 GPU 的看家本領。AlexNet 團隊用兩張遊戲顯卡，把原本需要數週的訓練壓縮到五、六天，讓「訓練一個夠深的網路」從不切實際變成了可行。這個發現讓一家原本以電玩顯卡聞名的公司——NVIDIA，以及它的創辦人黃仁勳——站上了 AI 時代的浪頭。黃仁勳多次談到，AlexNet 是 NVIDIA 全力轉向 AI 運算的關鍵時刻：公司從此不再只是賣遊戲顯卡，而是打造整個 AI 運算的基礎設施。今天訓練大型語言模型的資料中心裡，滿滿都是 NVIDIA 的晶片——這條路的起點，就是 2012 年那兩張跑 AlexNet 的 GTX 580。

AlexNet 之後，深度學習如野火燎原：Google、Facebook、微軟、百度紛紛重金投入，ImageNet 的冠軍模型愈疊愈深，到 2015 年 ResNet 的錯誤率已低於人類水準。這場革命的三大燃料——大數據(第 14 章的 ImageNet)、強大算力(GPU)、以及深層網路架構——缺一不可。而它給我們的最大啟示，呼應了第 13、14 章一路鋪陳的主軸：深度學習最革命性的地方，不是它「更準」，而是它讓機

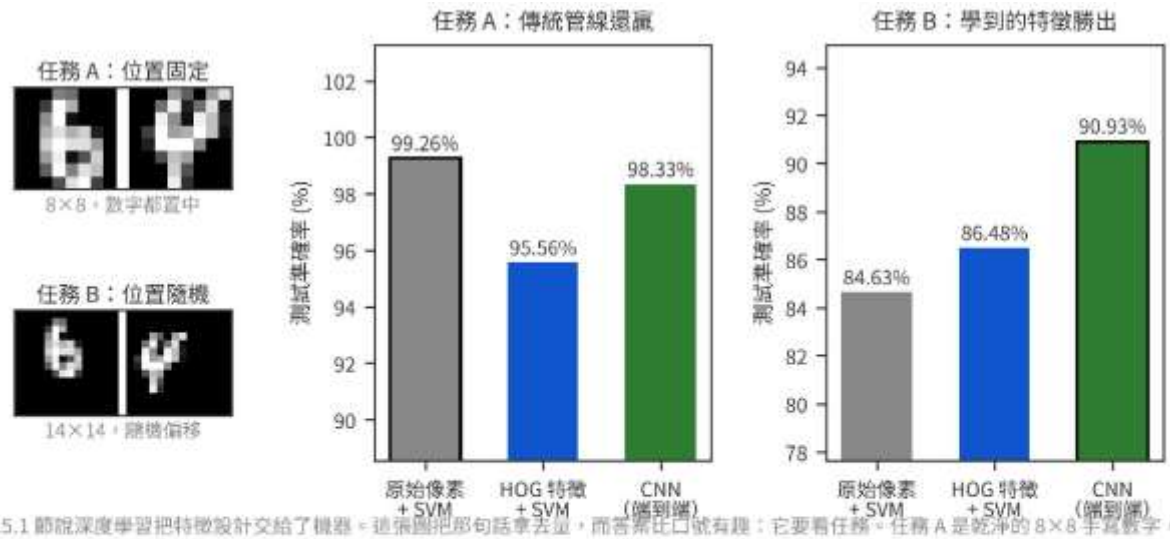
器自己學會了特徵。AlexNet 較低的網路層自動長出了邊緣與顏色的偵測器，較高的層則認得出輪子、狗臉——這些過去要靠人類專家嘔心瀝血設計(SIFT、HOG)的東西，網路從資料中自己學會了。這一章，我們就來理解這場革命的技術核心。

思考題：GPU 原本是為了電玩遊戲的畫面而設計，卻意外成為 AI 革命的引擎。這種「一項技術在原本用途之外找到革命性新應用」的現象，你還能想到哪些例子?這對「基礎技術的投資」有什麼啟示?

15.1 從手工特徵到學習特徵：典範轉移

第 13.8 節與第 14 章已經多次預告這場典範轉移，現在正式展開。回顧傳統的影像辨識流程(第 14.2 節的五環節)：人類專家設計特徵(第 13 章的 SIFT、HOG)，再把特徵餵給分類器(第 14 章的 SVM)。這裡的瓶頸顯而易見——特徵是人設計的，而人的設計能力有限：我們能想出邊緣、角點、梯度方向，但面對「什麼特徵能區分狗和狼」這種問題，人類的直覺很快就見底了。

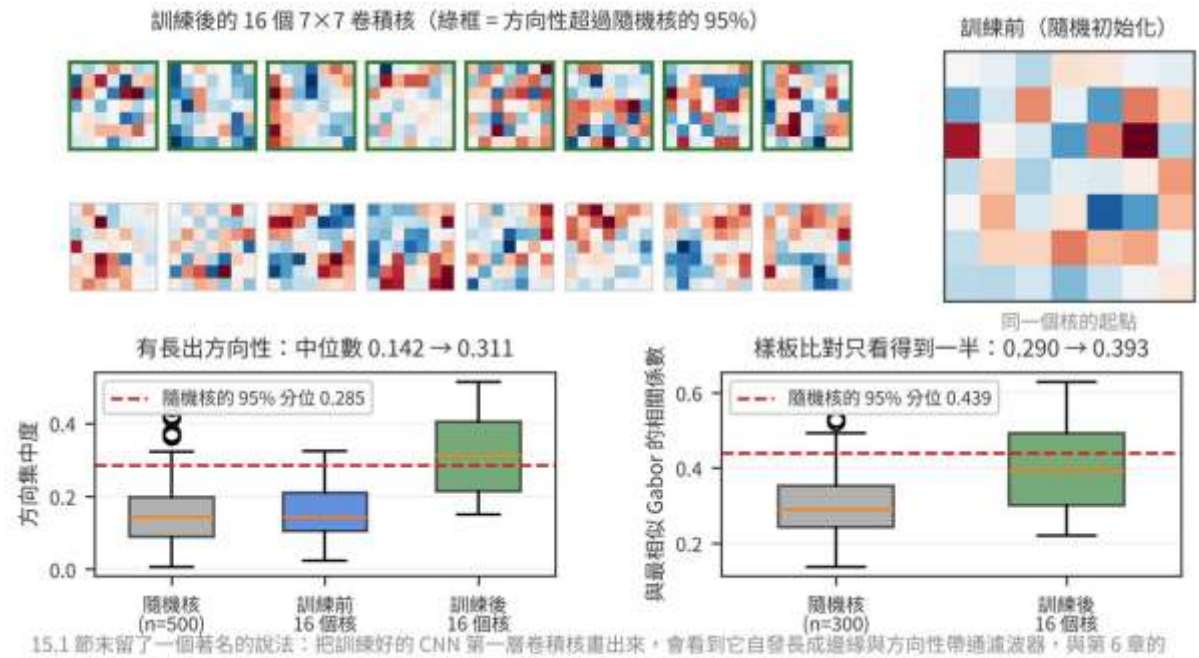
深度學習的核心突破，是把「特徵設計」這件事，從人類手中交給了機器。一個深度神經網路，把「從像素到特徵」與「從特徵到類別」整合進同一個模型，端到端(end-to-end)一起訓練：你只需給它影像與答案，它會自己學出「什麼特徵最有助於分類」。這就是叫獸開講中最關鍵的啟示——AlexNet 的網路層自動長出了邊緣偵測器與物件辨識器，無需人類插手。



15.1 節說深度學習把特徵設計交給了機器。這張圖把那句話拿去量，而答案比口號有趣：它要看任務。任務 A 是乾淨的 8×8 手寫數字，每個數字都好好置中。在這裡，第 13 章的手工特徵配 SVM 拿到 95.56%，連最接素的「原始像素 + SVM」都有 99.26%，而一個 CNN 端到端學完只有 98.33%——傳統管線這贏。任務 B 只改一件事：把同樣的數字擺到 14×14 畫布的隨機位置上，模擬真實世界裡物件不會乖乖置中。原始像素 + SVM 立刻掉到 84.63%，HOG 也只有 86.48%（已用最佳參數參賽，不是稻草人），而同一個 CNN（僅 6,218 個參數）拿到 90.93%。差別來自 15.3.1 節的權重共享：同一組卷積核滑過整張影像，數字出現在哪裡都認得;而攤平成向量的做法，位置一變，每個像素的意義就全變了。所以這一章要先立一個誠實的前提：深度學習不是自動比較好，它贏在「任務夠難、變異夠多、資料夠多」的時候。在小而乾淨的問題上，第 13、14 章的工具往往又快又準又好解釋——會選工具，比會用最新的工具更重要。

圖 15-1　典範轉移：什麼時候「學到的特徵」才真的贏

而這正呼應了第 6.6.7 節與第 13.8 節埋下的伏筆：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣與方向性帶通濾波器，與人工設計的 Gabor、SIFT 驚人地相似。這個說法本書做了實測，結果要分兩半講，因為它常被引用得太滿。圖 15-2 用一個刻意不循環的任務(判斷小圖來自哪一張自然照片，完全沒提到邊緣或方向)訓練 CNN，再用與樣板無關的方式量第一層卷積核的「方向集中度」：同一批核訓練前的中位數 0.142(與隨機核無異)，訓練後升到 0.311，16 個核中有 8 個超過隨機核的 95% 分位(純靠運氣的期望值是 0.8 個)——方向性結構確實是學出來的。但若改用「與 Gabor 樣板的相關係數」來量，訓練後中位數 0.393、隨機核 0.290，只有 6 個核超過隨機核的 95% 分位，差距小得多。換句話說：方向性會自發出現，但「長得像教科書上那張 AlexNet 卷積核圖」是另一回事，那需要 ImageNet 等級的自然影像與資料量，不是隨便一個小 CNN 都能重現。帶著這個分寸再讀下去：深度學習沒有推翻前面十四章的智慧，而是「自動重新發明」了它的一部分，並能組合出人類難以手工設計的、更抽象的高層特徵。這也是為什麼要先學完前面各章：唯有理解手工特徵，才能真正看懂 CNN 到底學到了什麼、為什麼有效。



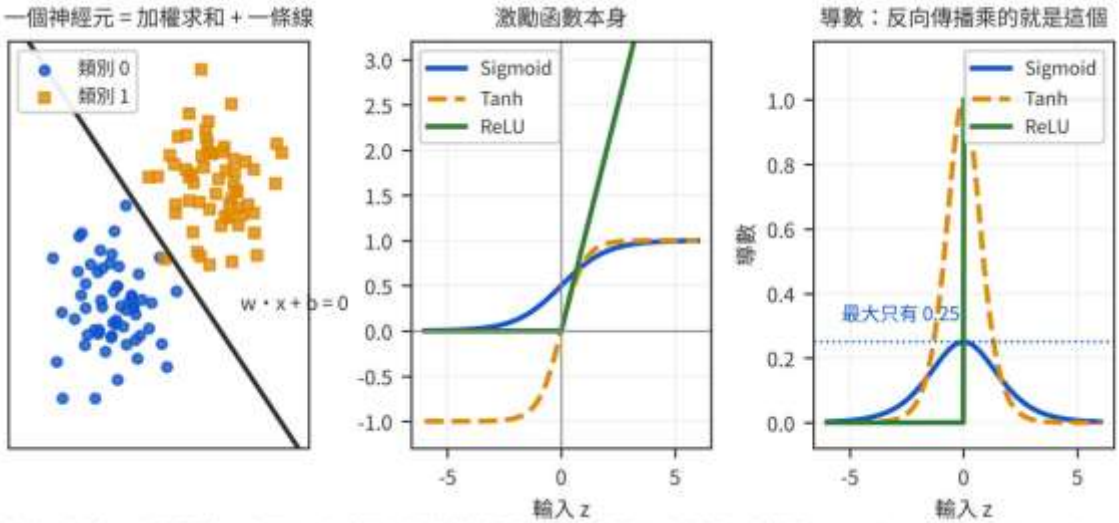
15.1 節末留了一個著名的說法：把訓練好的 CNN 第一層卷積核畫出來，會看到它自發長成邊緣與方向性帶通濾波器，與第 6 章的 Gabor、第 13 章的手工特徵驚人地相似。這張圖把那句話拿去驗，而結果需要分兩半說。實驗設計得刻意不循環：任務是「判斷這塊 24×24 的小圖來自六張自然照片中的哪一張」，完全沒有提到邊緣或方向，影像還先減掉了各自的平均亮度，免得網路靠亮度作弊（測試準確率 88.7%）。好消息在左下：用與樣板無關的方式量「方向集中度」（把核的功率頻譜當角度分布，看它有多集中），同一批核訓練前中位數 0.142（與隨機核的 0.142 無異），訓練後升到 0.311，8/16 個核超過隨機核的 95% 分位（純靠運氣的期望值是 0.8 個）。方向性確實是學出來的，不是初始化就有的。右下則要保守一點：改用「與 Gabor 樣板的相關係數」來量，訓練後中位數從隨機核的 0.290 升到 0.393，只有 6/16 個核超過隨機核的 95% 分位。同一件事，用這個指標看起來弱得多。兩個結論都要講。其一，方向性結構確實會自發出現，這一點與 15.1 節的說法一致，也與第 6 章的 Gabor、第 13 章的手工特徵遙相呼應；其二，「長得像教科書上那張 AlexNet 卷積核圖」是另一回事，那需要 ImageNet 等級的自然影像與資料量，不是隨便一個小 CNN 都能重現——本圖的核有方向，卻還稱不上 Gabor。順帶一提，兩個指標的落差本身就是第 13 章那一課的回馬槍：樣板比對只能看見「長得像我這個樣板」的核，對其他形式的方向性結構是半盲的；換一個不綁樣板的指標，同一個效應就清楚得多。量什麼，往往決定了你會看見什麼。

圖 15-2　第一層真的長出方向性濾波器了嗎

15.2 神經網路的基本構件

15.2.1 神經元與激勵函數

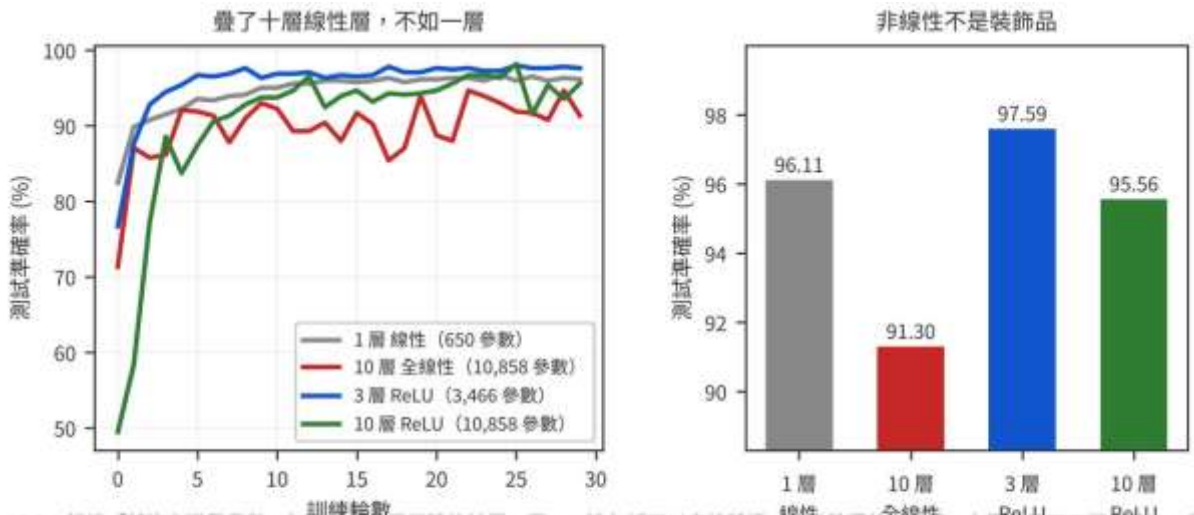
神經網路的靈感來自生物神經元。一個人工神經元做的事很簡單：把多個輸入各自乘上一個權重(weight)、加總、再加上一個偏置(bias)，然後通過一個激勵函數(activation function)輸出。權重與偏置就是神經元「學習」的對象——訓練的過程，就是不斷調整這些數值，讓網路的輸出愈來愈接近正確答案。



15.2.1 節的三句話，一張圖看完。左圖：一個神經元做的事就是把輸入加權求和再加偏置，也就是 $w \cdot x + b$——而 $w \cdot x + b = 0$ 在平面上正是一條直線，在高維空間是一個超平面。這是第 14.4.3 節推導 SVM 時已經見過的同一個式子，差別只在怎麼決定 w 與 b：SVM 解一道有唯一解的最佳化問題，神經網路則靠梯度下降一步步挪。所以單一個神經元的能力只等於一條線性分界，要處理彎曲的邊界，就得靠中間的激勵函數注入非線性，再把許多神經元疊起來。右圖是本章最該記住的一張小圖：三種激勵函數的導數。反向傳播每往回走一層，梯度就要乘上當層激勵函數的導數一次。Sigmoid 的導數最大只有 0.25，而且輸入稍大或稍小就迅速趨近 0;Tanh 最大是 1.0 但兩側一樣會飽和;ReLU 在正半邊的導數固定是 1.0，乘幾層都不會縮小。把 0.25 連乘十層是 0.25 的十次方，大約一兆分之一——這就是「梯度消失」四個字的全部算術，也是 ReLU 取代 Sigmoid 的根本原因。圖 15-9 會把這件事實際量出來。

圖 15-3　一個神經元，與三種激勵函數

激勵函數為網路注入了「非線性」，這一點至關重要：若沒有它，無論疊多少層，整個網路都只是一連串線性運算的組合，等效於單一層，無法學習複雜的規律。常見的激勵函數有：Sigmoid(把輸出壓到 0 與 1 之間，早期常用，但在深層網路會導致梯度消失)、Tanh(類似 Sigmoid 但範圍是 -1 到 1)、以及 ReLU(修正線性單元，負值歸零、正值保留)。ReLU 正是叫獸開講中 AlexNet 的關鍵創新之一——它計算極簡、且大幅緩解了梯度消失問題，讓深層網路的訓練從「幾乎不可能」變得「又快又穩」，至今仍是最主流的選擇。

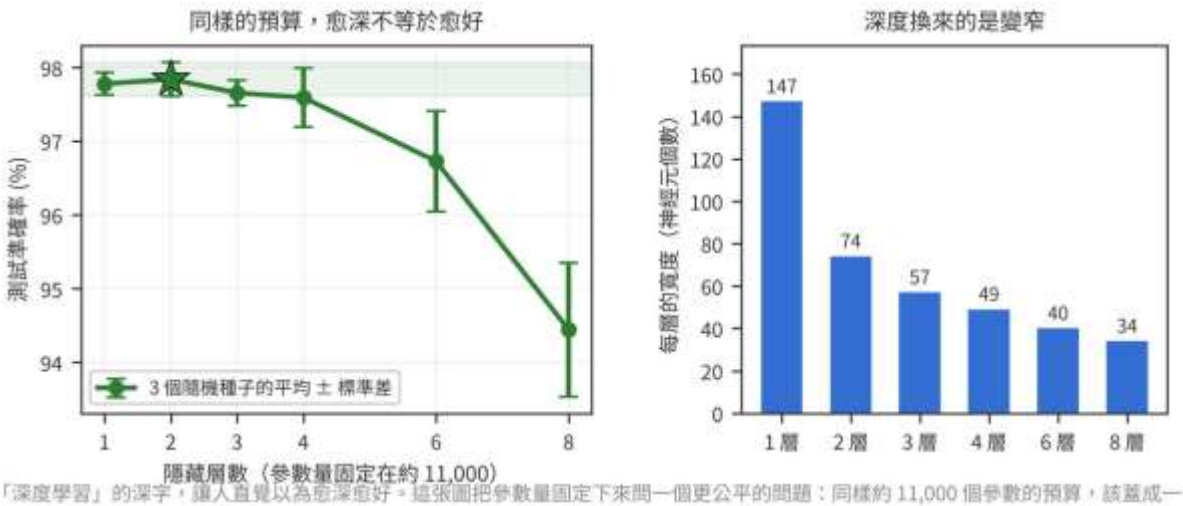


15.2.1 節說「若沒有激勵函數，無論疊多少層都等效於單一層」。這句話可以直接驗證，而且驗得乾淨俐落。左圖訓練了四個網路：一層線性拿到 96.11%，而把它加深到十層全線性（參數量多了 16 倍，10,858 個）卻只有 91.30%——不但沒變好，還更差了。原因是這十層在數學上根本是同一個模型：十個矩陣連乘仍然只是一個矩陣。本圖實際把訓練好的十層權重依序相乘摺成單一個 64×10 矩陣，再比對它與原網路對測試集的輸出，最大差異只有 1.78e-14——這個數量級就是浮點運算的捨入誤差，等於說兩者是同一個函數。既然表達能力相同，多出來的那九層就只是讓最佳化變難而已。把激勵函數加回去，一切就不同了：三層 ReLU 用 3,466 個參數拿到 97.59%，十層 ReLU 95.56%。請注意最後這個數字——十層 ReLU 反而輸給三層 ReLU，這不是打錯字，而是 15.4 節 ResNet 要解決的「退化問題」在這裡提前現身了：深度本身不會自動帶來好處，深到某個程度反而更難訓練。圖 15-16 會把這件事單獨拿出來量。

圖 15-4　把激勵函數拿掉：十層等於一層

15.2.2 多層感知器

把許多神經元排成一層、再把多層堆疊起來，就構成了多層感知器(MLP)：輸入層接收資料、若干隱藏層逐層萃取愈來愈抽象的表示、輸出層給出最終預測(例如十個輸出對應 MNIST 的十個數字)。「深度學習」的「深」，指的正是隱藏層的層數多——層數愈多，網路能表達的函數愈複雜，但也愈難訓練(這正是 AlexNet 之前深層網路難以成功的原因)。

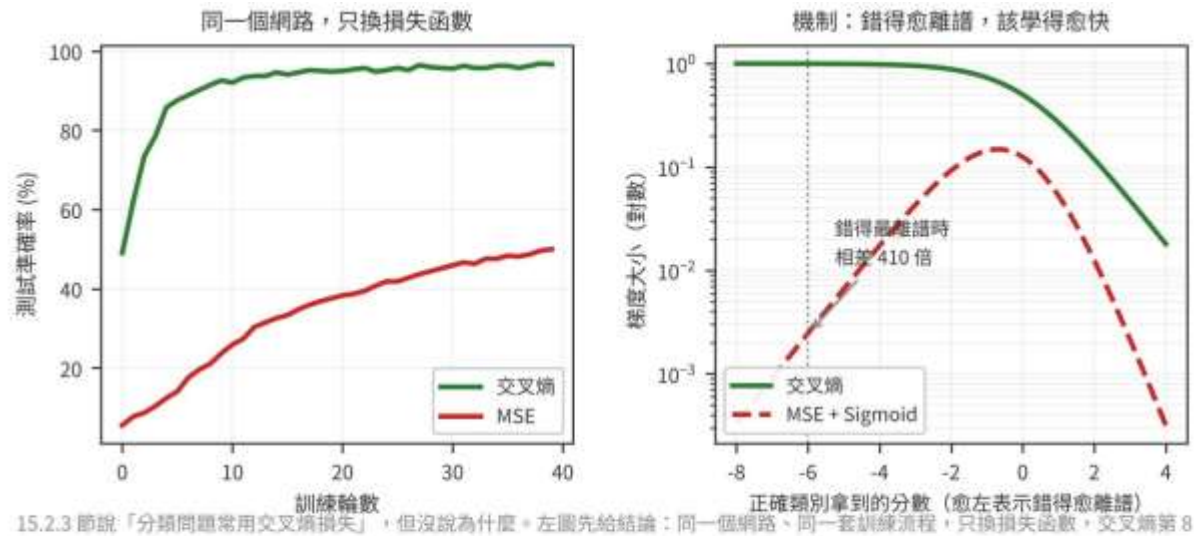


「深度學習」的深字，讓人直覺以為愈深愈好。這張圖把參數量固定下來問一個更公平的問題：同樣約 11,000 個參數的預算，該蓋成一層很寬，還是八層很窄?右圖是換算關係：一層時可以有 147 個神經元，八層時每層只剩 34 個。左圖是結果，而且每個設定都跑了三個隨機種子、附上標準差（第 14.5.3 節的規矩，沒有誤差範圍的單一數字沒有意義）。一層 97.78%、兩層 97.84%、三層 97.65%、四層 97.59%——這四個在誤差範圍內根本分不出高下;而六層掉到 96.73%、八層 94.44% ± 0.91，不但變差，種子之間的波動也明顯變大。所以在這個小任務上，深度不但沒有紅利，還開始收費。這不是說深度沒用——AlexNet 到 ResNet 的歷史正是深度的勝利——而是說深度的紅利要在「任務夠複雜、資料夠多」時才兌現得了，這與圖 15-1 的結論是同一件事。在小資料上硬疊層數，換到的往往只是更難訓練的網路。至於深到幾十層以上為什麼還能繼續變好，答案是 15.4 節的殘差連接，圖 15-16 會實際量給你看。

圖 15-5　同樣的參數量，該花在深度還是寬度

15.2.3 損失函數與梯度下降

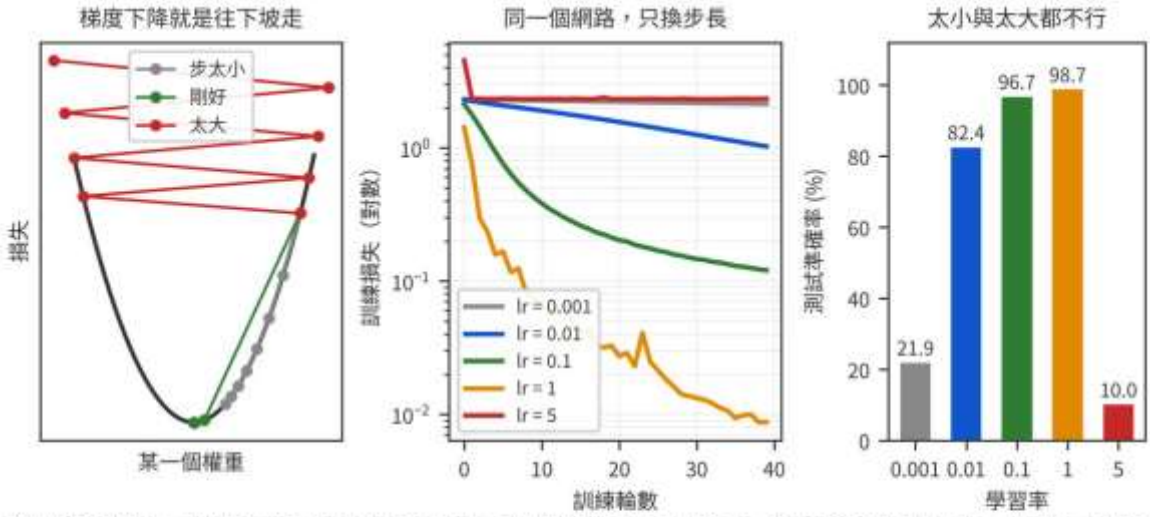
網路怎麼知道自己預測得好不好?靠損失函數(loss function)：它量化「預測」與「正確答案」之間的差距——差距愈大、損失愈高。分類問題常用交叉熵損失(cross-entropy)，迴歸問題常用均方誤差(MSE，正是第 3 章的老朋友)。訓練的目標，就是調整所有權重與偏置，讓損失盡可能小。



15.2.3 節說「分類問題常用交叉熵損失」，但沒說為什麼。左圖先給結論：同一個網路、同一套訓練流程，只換損失函數，交叉熵第 8 輪就突破 90%、最後 96.67%，而均方誤差（MSE，配 Sigmoid 輸出）四十輪只爬到 50.00%，差了 46.7 個百分點。右圖是機制，也是這張圖真正的重點。橫軸是「正確類別拿到的分數」，愈往左代表模型錯得愈離譜；縱軸是損失對該分數的梯度，也就是「這一步會被修正多少」。交叉熵的梯度在左端趨近 1：錯得愈離譜，修正力道愈大，這正是我們要的行為。MSE 配 Sigmoid 卻完全相反——Sigmoid 在兩端飽和（見圖 15-3 右圖，導數趨近 0），梯度還要再乘上這個趨近 0 的導數，於是在分數 −6 這個「錯得最離譜」的位置，交叉熵的梯度是 0.9976、MSE 只有 0.002436，相差 410 倍。換句話說，用 MSE 配 Sigmoid 做分類時，錯得最嚴重的那些樣本學得最慢——這是最糟的組合方式。交叉熵搭配 Softmax 之所以成為分類的標準配備，不是慣例，而是因為兩者的導數恰好把 Sigmoid 的飽和項約掉了。迴歸問題沒有這個飽和結構，MSE 仍然是合理的選擇。

圖 15-6　分類為什麼要用交叉熵

怎麼調?靠梯度下降(gradient descent)。想像損失是一片高低起伏的地形，而我們要走到最低點(最小損失)。梯度指出了「當前位置最陡上坡的方向」，那麼往反方向(下坡)走一小步，損失就會下降一點;反覆走，終能逼近谷底。每一步的大小由「學習率」(learning rate)控制——太大會在谷底附近來回震盪甚至衝過頭，太小則收斂慢得令人絕望，這又是一個貫穿全書的取捨。實務上用「隨機梯度下降」(SGD)及其改良版(如 Adam)，每次只用一小批(mini-batch)資料估計梯度，兼顧效率與穩定。

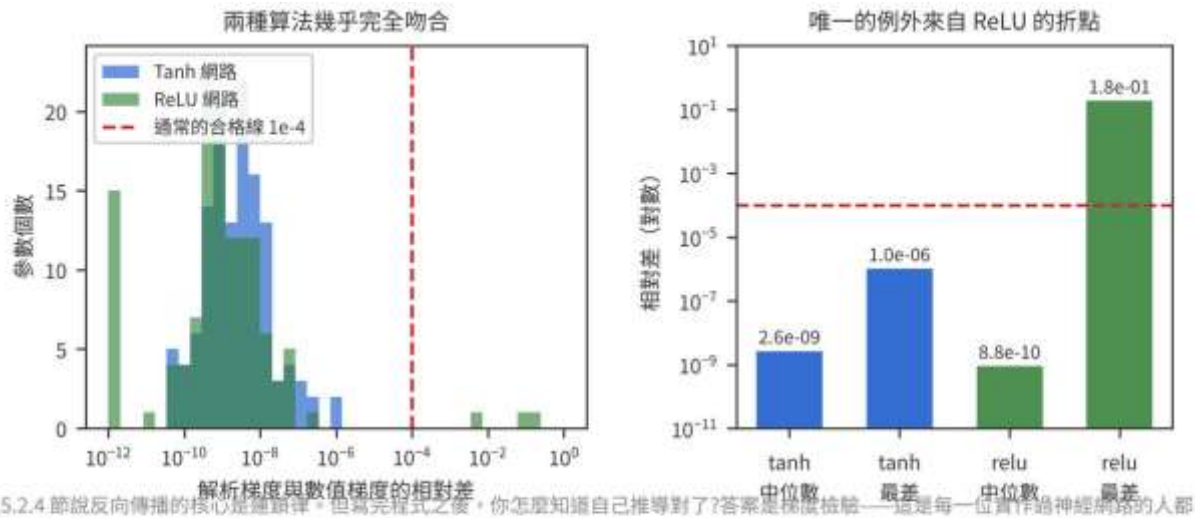


15.2.3 節把損失比喻成一片地形、梯度下降比喻成往下坡走，而學習率就是每一步跨多大。這個比喻可以量得很具體。左圖是一個簡化的碗狀損失：步子太小，走七步還在半山腰;步子剛好，幾步就到谷底;步子太大，每一步都跨過谷底、彈到對面更高的地方，愈走愈遠。中圖與右圖是同一個網路（兩層、ReLU、純 SGD）在五種學習率下的實測。lr = 0.001 時損失四十輪只從 2.32 降到 2.14，測試準確率 21.9%，幾乎等於沒學（十類亂猜是 10%）;lr = 0.01 爬到 82.4%;lr = 0.1 到 96.7%;lr = 1.0 反而最好，98.7%;而 lr = 5.0 直接發散，損失衝到 4.53 之後卡在 2.33 再也下不去，準確率 10.0%——這個數字正好就是十類亂猜。這裡有兩個該帶走的重點。其一，學習率是所有超參數裡最該優先調的那一個，因為它錯了，別的怎麼調都沒用。其二，最佳值沒辦法用背的：本例的 1.0 在多數真實任務上會炸得很難看，而這正是 Adam 這類自適應最佳化器受歡迎的原因——它替每個參數自動調整有效步長，讓人不必在這個旋鈕上耗掉太多時間。本章其餘各圖多半就是用 Adam 訓練的。

圖 15-7　學習率：走太小、剛好、與直接炸掉

15.2.4 反向傳播

最後一塊拼圖是反向傳播(backpropagation)——它回答了「如何算出每一個權重對損失的影響(梯度)」這個關鍵問題。它的核心是微積分的連鎖律：先前向計算得到預測與損失，再從輸出層往輸入層「逆向」逐層傳遞誤差，依連鎖律算出每個權重的梯度。這個演算法讓「調整深層網路裡數百萬個參數」變得可行，是神經網路能夠訓練的數學引擎。LeCun 在 1989 年正是用反向傳播訓練了 LeNet(第 14 章)，而 AlexNet 也建立在同一個基礎上。所幸，今天的深度學習框架(15.5 節)會自動完成反向傳播的所有計算，使用者無需手動推導——但理解它的原理，能幫助你診斷訓練為何失敗(如梯度消失或爆炸)。



15.2.4 節說反向傳播的核心是鏈鎖律。但寫完程式之後，你怎麼知道自己推導對了？答案是梯度檢驗——這是每一位實作過神經網路的人都該會的一招。原理很單純：梯度的定義就是「參數動一點點，損失變多少」，所以把某個權重加上與減去一個極小的 ε，各算一次損失，相減再除以 2ε，就得到了與推導完全無關的數值梯度。拿它跟反向傳播算出來的解析梯度比，兩者該吻合。本圖對一個含卷積層、池化層與全連接層的網路抽查了數百個參數：Tanh 版本的相對差中位數 2.6e-09、ReLU 版本 8.8e-10，都遠低於一般採用的合格線 1e-4——也就是說，本章後面所有數字所依賴的這份 NumPy 實作，梯度是對的。右圖那根突出的柱子則是一個值得認識的陷阱：ReLU 版本有 2.3% 的參數檢驗「失敗」，最差達到 1.8e-01。這不是程式有錯，而是 ReLU 在 0 處有折點、不可微；當某個權重剛好讓 z 落在 0 附近，加減 ε 會讓它跨過折點，此時數值梯度量到的是折點兩側的平均斜率，本來就與解析梯度不同。Tanh 處處平滑，所以完全沒有這個現象。實務上的規矩是：梯度檢驗用平滑的激勵函數做，而且看的是中位數與絕大多數參數，不是單一個最差值。

圖 15-8　反向傳播對不對：用數值梯度驗證

15.2.5 推導：連鎖律如何變成反向傳播

15.2.4 節說反向傳播的核心是微積分的連鎖律。這一小節把那句話展開成可以照著寫程式的式子。值得花時間跟一遍，理由有三個：它解釋了為什麼深層網路訓練得動、為什麼激勵函數的導數那麼要命（圖 15-9 的梯度消失）、以及為什麼 ResNet 的一個加法就能救回深度（圖 15-16）。

先把一層寫清楚。第 L 層收到前一層的輸出 a（上一層的激活值），做兩件事：先算加權和 z = a·W + b，再通過激勵函數得到 out = f(z)。整個網路就是這兩件事交替做很多次，最後把輸出送進損失函數 E。我們要的是 E 對每一個 W 與 b 的偏微分，因為梯度下降要用它來更新。

連鎖律說：若 E 透過 z 影響結果，則 E 對 W 的偏微分等於「E 對 z 的偏微分」乘上「z 對 W 的偏微分」。第二項很好算——z = a·W + b 對 W 微分就是 a。所以真正的關鍵只有第一項，我們給它一個名字，叫做這一層的誤差訊號 d：

```
d = ∂E / ∂z          （這一層加權和的誤差訊號）
```

```
∂E / ∂W = a 的轉置 · d
∂E / ∂b = d 逐列加總
```

於是問題只剩一個：怎麼算出每一層的 d。答案就是「反向」兩個字的由來——從最後一層往回推。最後一層的 d 由損失函數直接給出（Softmax 配交叉熵時，d 恰好等於「預測機率減去正確答案的 one-hot」，乾淨得不可思議，這正是 15.2.3 節那個組合的妙處，圖 15-6 量過它的後果）。而任何一層的 d，都可以由它後面那一層的 d 推出來：

```
d(本層) = ( d(後一層) · W(後一層) 的轉置 ) × f 的導數(本層的 z)
          └── 把誤差從後面傳回來 ──┘    └── 乘上本層激勵的導數 ──┘
```

這條式子就是反向傳播的全部。它說：某一層的誤差，等於把後一層的誤差透過權重「攤回來」，再乘上本層激勵函數的導數。前向計算一次把資料推到底、算出損失，反向再走一次就能拿到所有參數的梯度——總成本大約只是前向的兩倍，而不是「每個參數各試一次」那種與參數量成正比的天文數字。一個有一千萬個參數的網路，若用數值微分逐一估計梯度，每更新一步要做兩千萬次前向運算;反向傳播只要兩次。這就是神經網路能被訓練的全部理由。

現在回頭看那個「乘上本層激勵函數的導數」。每往回一層就乘一次，往回 n 層就連乘 n 次。Sigmoid 的導數最大只有 0.25（圖 15-3 右圖），所以九層之後大約只剩 0.25 的九次方，約四十萬分之一。圖 15-9 實測一個九層網路，最後一層的梯度範數是 5.09e−01，傳到第一層只剩 6.84e−06，相差約三萬七千倍——與這條算式預測的量級一致。ReLU 在正半邊的導數固定是 1，連乘多少次都不縮小，這就是它取代 Sigmoid 的數學理由。

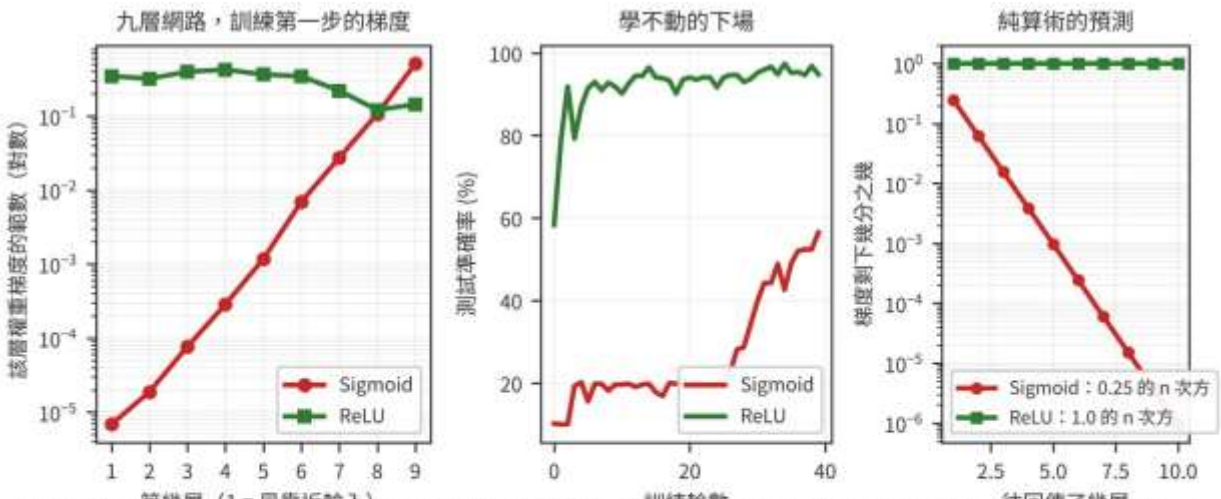


圖 15-3 右圖那條紅線就是這個預測。左圖是實測：一個九層的網路，在訓練的第一步量每一層權重的梯度範數。Sigmoid 版本最後一層的梯度是 5.09e-01，傳到第一層只剩 6.84e-06，相差 74,409 倍。這代表靠近輸入的那幾層幾乎收不到任何學習訊號，它們的權重從頭到尾跟隨機初始化差不多——說是九層，實際上只有最後兩三層在學。換成 ReLU，情況完全不同：各層梯度都在 0.12 到 0.42 之間，第一層甚至比最後一層還大（2.4 倍），訊號一路傳得到底。中圖是後果：同樣訓練四十輪，Sigmoid 版本只有 56.48%，ReLU 版本 94.81%。這就是 2012 年 AlexNet 把 ReLU 當成關鍵配方之一的原因，也是 15.4 節 ResNet 殘差連接要處理的同一個病根——當網路深到連 ReLU 都撐不住時，就需要讓梯度有一條捷徑可走（圖 15-16）。

圖 15-9　梯度消失：0.25 連乘九次會怎樣

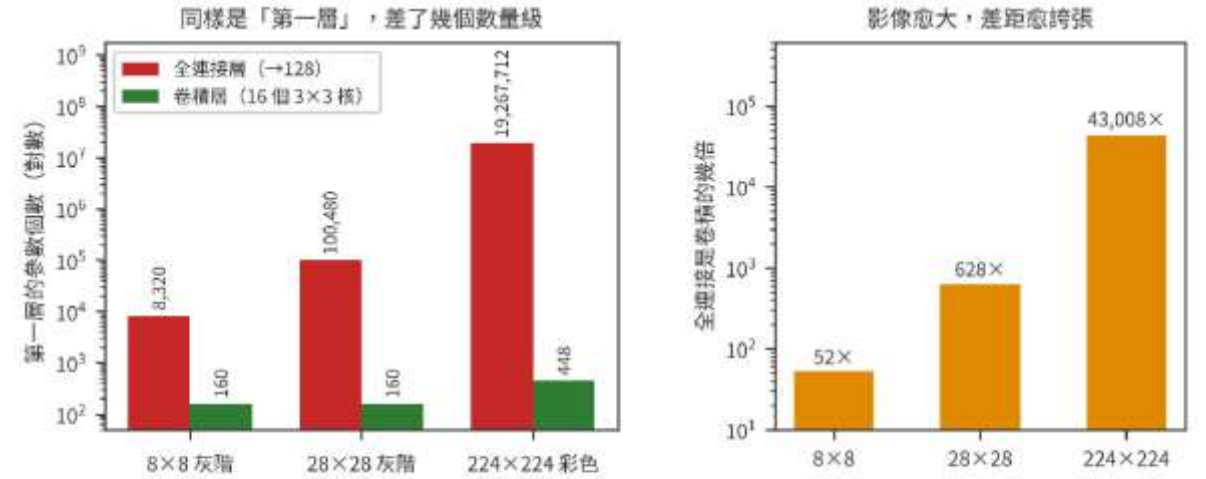
最後一個收穫是殘差連接。把一個區塊寫成 out = x + F(x)，對 x 微分會得到「1 + F 的導數」。那個 1 代表：無論 F 這條路上的導數多小，誤差訊號都有一條直通的路可以原封不動傳回去。連乘因此不再是連乘，而是「每一項都至少有 1」。圖 15-16 的實測正是這條式子的樣子：沒有捷徑的網路第一層梯度隨深度從 4.05e−01 縮到 8.73e−02，有殘差連接的反而從 6.98e−01 長到 1.27e+01。一個加號，就把梯度消失變成了梯度暢通。

實務上要補一句：框架會自動完成這一切（這叫自動微分），所以你幾乎不會親手寫這些式子。但理解它仍然重要——當訓練不動、損失變成 NaN、或深層網路怎麼調都學不起來時，能在腦中跑一遍這條式子的人，才知道該去看激勵函數、學習率、初始化還是架構。順帶一提，驗證自己推導對不對有一個標準做法：梯度檢驗，圖 15-8 示範了它，也示範了它在 ReLU 上會有的那個陷阱。

15.3 卷積神經網路(CNN)

多層感知器有一個致命問題：它把影像「攤平」成一長串像素再處理，徹底破壞

了影像的空間結構(相鄰像素的關聯性)，而且參數量爆炸——一張小小的 224×224 彩色圖攤平就是十五萬個輸入，若第一層隱藏層有一千個神經元，光第一層就要一億五千萬個權重。卷積神經網路(CNN)正是為影像量身打造，優雅地解決了這兩個問題，也是叫獸開講中 AlexNet 的骨幹。

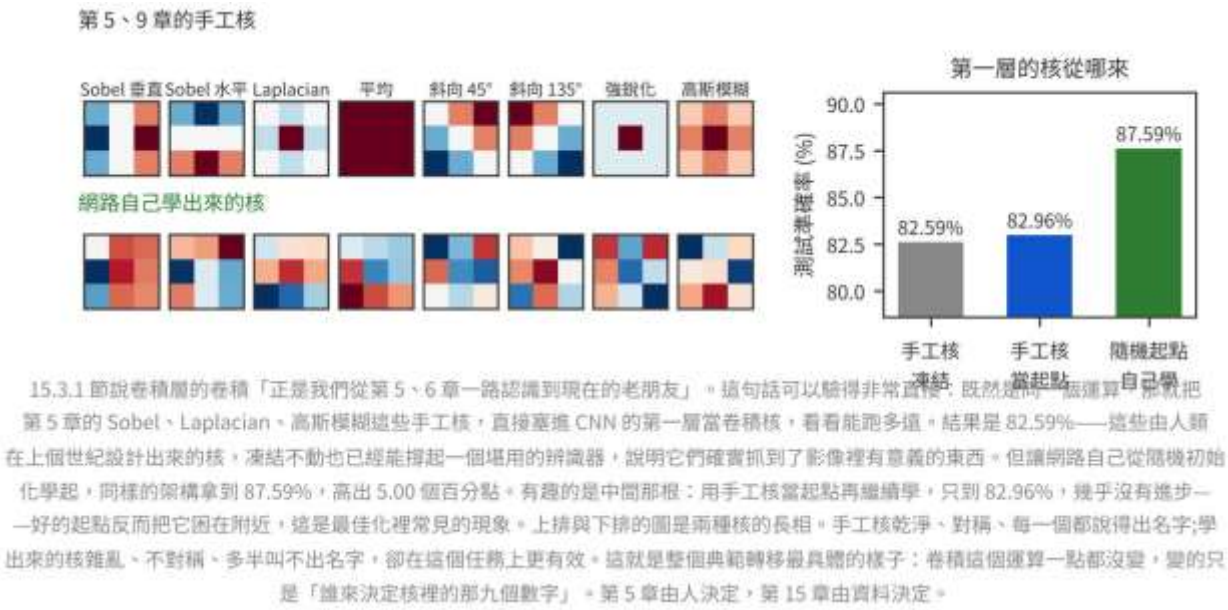


15.3 第一個頭疼的問題：參數量爆炸」(MNIST 手寫數字)換成 ImageNet 的彩色照片，把那個爆炸算成具體數字。一張 224×224 的彩色影像攤平之後是 150,528 個數字。若第一層就換一個 128 個神經元的全連接層，光這一層的權重就有 19,267,712 個;若比照 AlexNet 那樣接 4,096 個神經元，則是 616,562,688 個權重，用 float32 存要 2.5 GB——一層就塞不進任何一張顯示卡。而一個卷積層用 16 個 3×3×3 的核，參數只有 448 個，相差 43,008 倍。關鍵在於卷積層的參數量只跟「核多大、幾個核」有關，與影像多大完全無關——把影像換成 1024×1024，那 448 個參數一個都不會增加。右圖顯示影像愈大、差距愈誇張：8×8 時只差 52 倍，到 224×224 就是 43,008 倍。這是 15.3.1 節「權重共享」最直接的好處，但它還有第二個好處——同一組核滑遍整張影像，也意味著它在哪裡都能得同一個圖案。那一半的故事在圖 15-12。

圖 15-10　參數量爆炸：為什麼影像不能用全連接硬幹

15.3.1 卷積層：本書最熟悉的老朋友

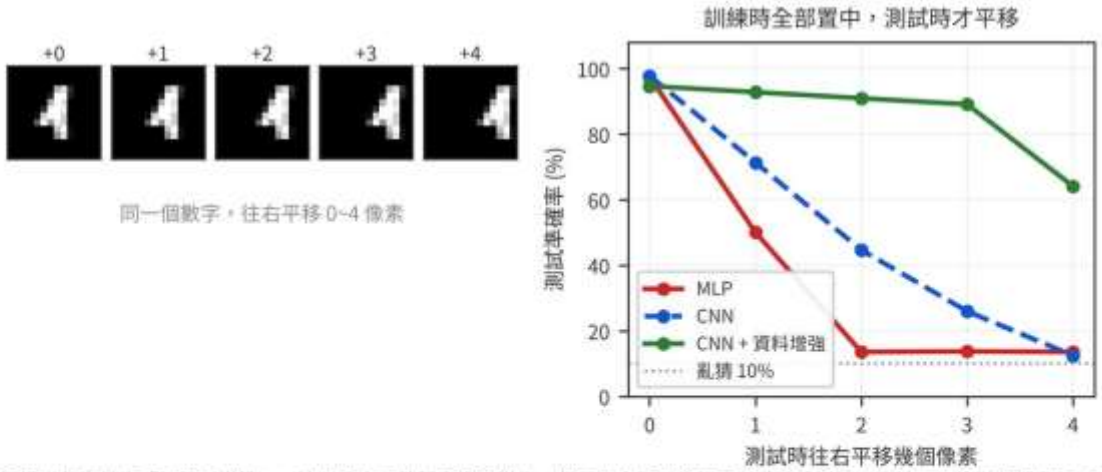
CNN 的核心是卷積層，而「卷積」正是我們從第 5、6 章一路認識到現在的老朋友!卷積層用一組小小的卷積核(filter)在影像上滑動，計算局部區域的加權和——這與第 5 章的空間濾波在數學上是同一件事。差別只有一個、卻是革命性的：第 5 章的濾波核是人設計的(高斯核、Sobel 核)，而 CNN 的卷積核是學出來的——網路透過訓練自己決定核裡的每個數值，以萃取出最有助於任務的特徵。



15.3.1 節說卷積層的卷積「正是我們從第 5、6 章一路認識到現在的老朋友」。這句話可以驗得非常直接。既然是同一個運算，那就把第 5 章的 Sobel、Laplacian、高斯模糊這些手工核，直接塞進 CNN 的第一層當卷積核，看看能跑多遠。結果是 82.59%——這些由人類在上個世紀設計出來的核，凍結不動也已經能撐起一個堪用的辨識器，說明它們確實抓到了影像裡有意義的東西。但讓網路自己從隨機初始化學起，同樣的架構拿到 87.59%，高出 5.00 個百分點。有趣的是中間那根：用手工核當起點再繼續學，只到 82.96%，幾乎沒有進步——好的起點反而把它困在附近，這是最佳化裡常見的現象。上排與下排的圖是兩種核的長相。手工核乾淨、對稱、每一個都說得出名字;學出來的核雜亂、不對稱、多半叫不出名字，卻在這個任務上更有效。這就是整個典範轉移最具體的樣子：卷積這個運算一點都沒變，變的只是「誰來決定核裡的那九個數字」。第 5 章由人決定，第 15 章由資料決定。

圖 15-11　卷積核是誰決定的：手工核 vs 學出來的核

這個設計帶來三個關鍵優勢。其一，局部連接：每個卷積核只看局部區域，符合「影像的意義來自局部結構(邊緣、紋理)」的直覺。其二，權重共享：同一個卷積核滑遍整張影像，用同一組權重——這既大幅減少參數(一個 3×3 核只有 9 個權重，不管影像多大)，又帶來對位置變化的容忍(物件出現在哪裡都能被同一個核偵測到)。這裡要把常見的「平移不變性」這個說法修得精確一點，因為它是本章最容易被過度解讀的四個字。卷積給的其實是「平移等變」——影像平移，特徵圖就跟著平移;要再加上池化的降取樣，才換到一點點「平移容忍」。圖 15-12 實測：三個模型都只用置中的數字訓練，平移 2 個像素時多層感知器崩到 13.52%(十類亂猜是 10%)、CNN 還有 44.63%，卷積確實好得多;但平移 4 個像素時 CNN 也只剩 12.41%，一樣崩了。真正的不變性要靠資料教：同一個 CNN 若訓練時就看過隨機平移的樣本(也就是 15.6.2 節的資料增強)，平移 3 個像素仍有 89.07%，是未增強版本的 3.4 倍。記住這句話：不變性多半是資料教出來的，不是架構免費送的。其三，層級特徵：淺層的卷積核學到邊緣、顏色等低階特徵，深層的則組合出紋理、部件乃至完整物件等高階特徵——這正是叫獸開講中 AlexNet「淺層學邊緣、深層認物件」的由來，也再次呼應第 6、11 章的多解析度、由粗到細的思想。

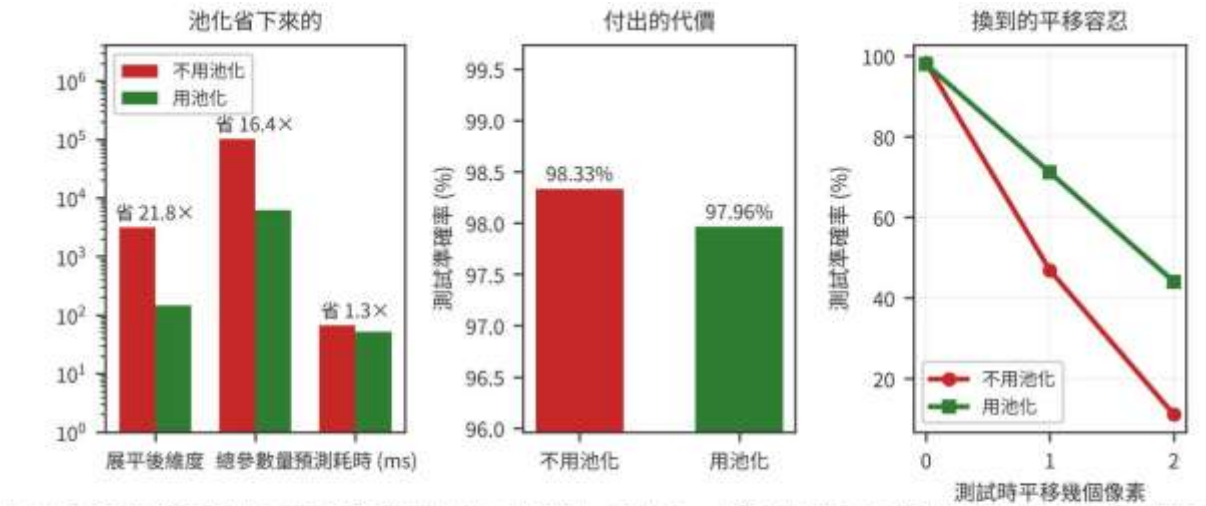


15.3.1 節說權重共享帶來「平移不變性」。這四個字在教科書裡很常見，但量過之後要修正成更精確的說法。實驗：三個模型都只用置中的數字訓練，測試時才把數字往右平移。平移 0 像素時，MLP 97.41%、CNN 97.78%，不相上下。平移 2 個像素，MLP 崩到 13.52%（十類亂猜是 10%），CNN 還有 44.63%——確實好得多，卷積的權重共享真的有用。但請看平移 4 個像素：CNN 也只剩 12.41%，一樣崩了。所以嚴格說，卷積給的是「平移等變」（影像平移，特徵圖跟著平移），加上池化之後換到一點點「平移容忍」，但距離真正的「不變」還很遠。第三條線是補上的那一塊：同一個 CNN，訓練時就讓它看過隨機平移過的樣本（也就是 15.6.2 節的資料增強，用的是第 4 章的幾何轉換）。它在平移 3 個像素時仍有 89.07%，是未增強版本 25.93% 的 3.4 倍。代價是置中時略低一點（94.63% vs 97.78%），因為同樣的容量要分給更多種情況。結論值得記住：不變性不是架構免費送的，多半是資料教出來的。

圖 15-12　權重共享給的是容忍，不是不變

15.3.2 池化層

池化層(pooling)負責「降取樣」——縮小特徵圖的尺寸。最常見的最大池化(max pooling)把每個小區域(如 2×2)只保留最大值，尺寸減半。它有三個作用：減少後續運算量與參數、擴大後續卷積核的「視野」(讓深層能看到更大範圍)、並提供一定的位置容錯(物件稍微移動，池化結果變化不大)。有趣的是，最大池化與最小池化，本質上就是第 10 章的型態學膨脹與侵蝕——又一次一體多面的印證。

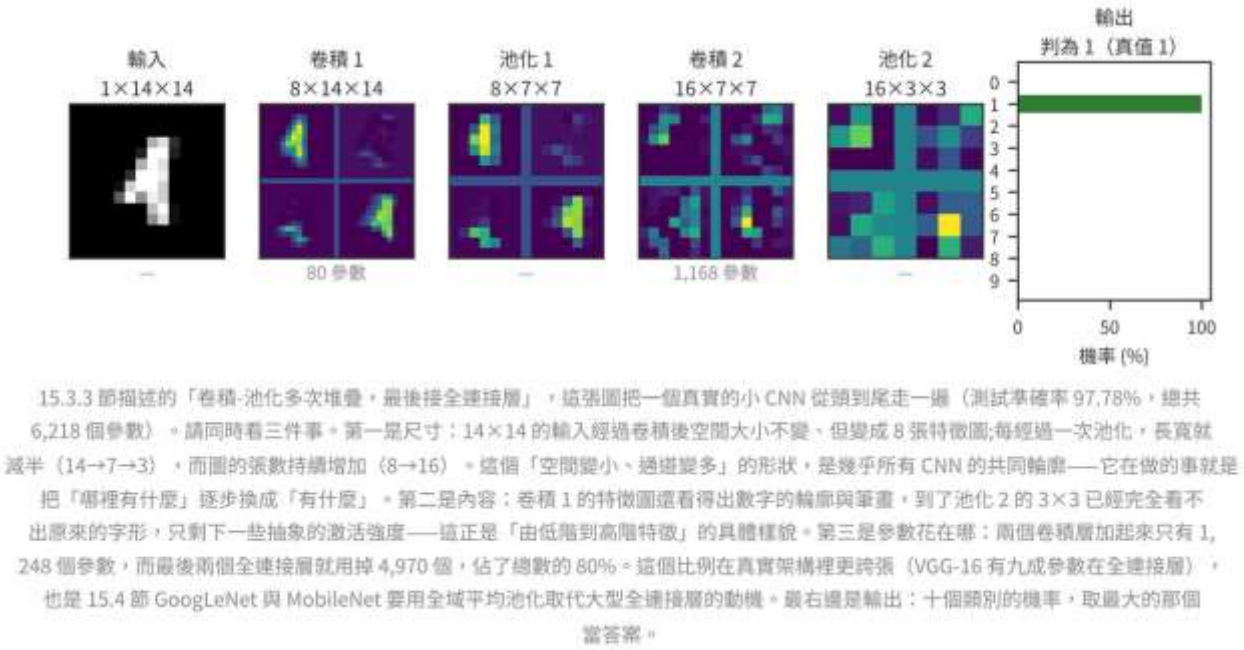


15.3.2 節列了池化的三個作用，這張圖把三個都量出來，也把代價一起量出來——因為工程上沒有白拿的東西。同一個 CNN，唯一的差別是拿掉兩個池化層。拿掉之後，卷積的輸出仍是 14×14，展平後是 3,136 維，接上全連接層使總參數量達到 101,962 個;加回池化，展平後只剩 144 維、總參數 6,218 個，足足少了 16.4 倍，預測也快了 1.3 倍。第二個作用是擴大感受野：3×3 卷積的感受野是 3 個像素，經過一次 2×2 池化之後，下一個 3×3 卷積看到的已經相當於原圖的 10 個像素，再池化一次就是 20 個——這就是「層級特徵」能從邊緣長成形狀再長成物件的原因。第三個作用是平移容忍：右圖顯示平移 2 個像素時，有池化的 44.1%、沒池化的只有 11.1%，差了 4.0 倍。代價寫在中圖：準確率從 98.33% 降到 97.96%，少了 0.37 個百分點。用 0.37 個百分點換 16 倍的參數、1.3 倍的速度與 4.0 倍的平移容忍，這筆交易在幾乎任何實務場合都划算——而這也正是第 17 章要把模型塞進邊緣裝置時最先想到的那類取捨。

圖 15-13　池化層用零點幾個百分點換到了什麼

15.3.3 全連接層與整體結構

一個典型的 CNN，是「卷積層 + 池化層」的多次堆疊(負責萃取由低階到高階的特徵)，最後接上幾層全連接層(第 15.2 節的 MLP，負責把萃取到的特徵映射到最終的類別)。這個「卷積池化萃取特徵、全連接做決策」的結構，從 1998 年的 LeNet 到 2012 年的 AlexNet 一脈相承，也是下一節架構譜系的共同骨架。



15.3.3 節描述的「卷積-池化多次堆疊，最後接全連接層」，這張圖把一個真實的小 CNN 從頭到尾走一遍（測試準確率 97.78%，總共 6,218 個參數）。請同時看三件事。第一是尺寸：14×14 的輸入經過卷積後空間大小不變、但變成 8 張特徵圖;每經過一次池化，長寬就減半（14→7→3），而圖的張數持續增加（8→16）。這個「空間變小、通道變多」的形狀，是幾乎所有 CNN 的共同輪廓——它在做的事就是把「哪裡有什麼」逐步換成「有什麼」。第二是內容：卷積 1 的特徵圖還看得出數字的輪廓與筆畫，到了池化 2 的 3×3 已經完全看不出原來的字形，只剩下一些抽象的激活強度——這正是「由低階到高階特徵」的具體樣貌。第三是參數花在哪：兩個卷積層加起來只有 1,248 個參數，而最後兩個全連接層就用掉 4,970 個，佔了總數的 80%。這個比例在真實架構裡更誇張（VGG-16 有九成參數在全連接層），也是 15.4 節 GoogLeNet 與 MobileNet 要用全域平均池化取代大型全連接層的動機。最右邊是輸出：十個類別的機率，取最大的那個當答案。

圖 15-14　走完一次 CNN：每一層看到什麼、花了多少參數

15.3.4 推導：參數量、感受野與那個「空間變小、通道變多」的形狀

15.3 節用文字說了卷積層省參數、池化層擴大感受野。這兩件事都有簡單的公式，而且算一遍之後，你會發現幾乎所有 CNN 長得像同一個形狀，不是巧合。

先算參數量。一個卷積層若輸入有 Cin 個通道、輸出 Cout 個通道、卷積核是 k×k，則參數量是：

```
卷積層參數 = k × k × Cin × Cout + Cout        （最後一項是偏置）
全連接層參數 = (H × W × Cin) × Cout + Cout
```

關鍵差別一眼可見：卷積層的式子裡沒有 H 與 W。影像多大，卷積層的參數量都一樣——因為同一組核滑遍全圖，這就是「權重共享」的數學形式。圖 15-10 用真實尺寸算過：224×224 的彩色影像攤平是 150,528 個數字，接一個 128 個神經元的全連接層要 19,267,712 個參數;而 16 個 3×3×3 的卷積核只要 448 個，相差 43,008 倍。把影像換成 1024×1024，前者會再膨脹二十倍，後者一個參數都不變。

再算感受野，也就是「輸出上的一個點，看得到原圖多大一塊」。它有一條遞迴式：

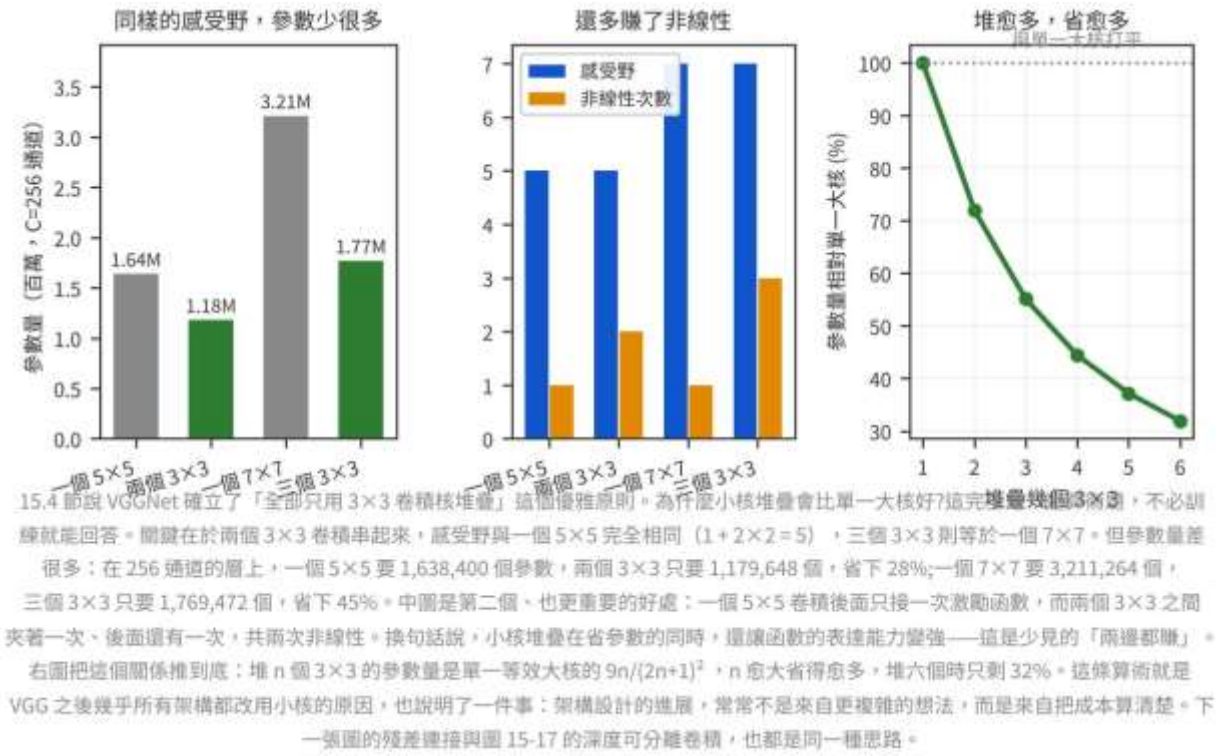
```
RF(新) = RF(舊) + (k − 1) × 跳躍步幅(舊)
```

```
跳躍步幅(新) = 跳躍步幅(舊) × stride
```

代進一個典型的堆疊就很清楚。一開始感受野 1、步幅 1;經過 3×3 卷積，感受野變 3;經過 2×2 池化，感受野變 4、而步幅變成 2;再一個 3×3 卷積，感受野變成 4 + 2×2 = 8;再池化一次，感受野 10、步幅 4。換句話說，每加一次池化，後面每一層的「視野成長速度」就翻一倍。這是池化最被低估的作用：它不只是省計算，更是讓深層能夠「看見整體」的手段。沒有降取樣的話，想讓感受野涵蓋整張 224×224 的影像，用 3×3 卷積要疊一百多層;有了五次池化，十幾層就夠了。

這兩條式子合起來，就解釋了 CNN 那個標準形狀：空間尺寸一路減半（224→112→56→28→14→7），通道數一路加倍（64→128→256→512）。為什麼要同步加倍?因為每次池化都讓特徵圖的面積變成四分之一，若通道數不變，這一層能攜帶的資訊量就掉到四分之一;把通道加倍，總量只掉到一半，比較不會形成瓶頸。同時，深層的特徵愈來愈抽象、需要的種類愈來愈多（淺層只需要少數幾種邊緣，深層要分辨幾百種物件部件），所以通道變多也是語意上的需要。圖 15-14 的小 CNN 走的正是這個形狀：1×14×14 → 8×14×14 → 8×7×7 → 16×7×7 → 16×3×3。

最後一個算術解釋了 15.4 節的兩個架構決策。其一，兩個 3×3 卷積串起來的感受野是 1 + 2×2 = 5，與一個 5×5 相同，但參數量是 2×9 = 18 對 25，只有 72%，而且多了一次非線性——這就是 VGG 全面改用 3×3 的理由（圖 15-15）。其二，標準卷積的 k×k×Cin×Cout 可以拆成「每通道各做一次 k×k」加上「1×1 混合通道」，參數變成 k×k×Cin + Cin×Cout，兩者的比值是 1/Cout + 1/k²，通道數夠多時大約就是 1/k²，也就是省下約 k² 倍——這就是 MobileNet 的深度可分離卷積（圖 15-17）。架構設計看起來像藝術，底層往往就是這幾條式子在算成本。



15.4 節說 VGGNet 確立了「全部只用 3×3 卷積核堆疊」這個優雅原則。為什麼小核堆疊會比單一大核好？這完全可以用一張圖一次講清楚，不必訓練就能回答。關鍵在於兩個 3×3 卷積串起來，感受野與一個 5×5 完全相同（1 + 2×2 = 5），三個 3×3 則等於一個 7×7。但參數量差很多：在 256 通道的層上，一個 5×5 要 1,638,400 個參數，兩個 3×3 只要 1,179,648 個，省下 28%；一個 7×7 要 3,211,264 個，三個 3×3 只要 1,769,472 個，省下 45%。中圖是第二個、也更重要的好處：一個 5×5 卷積後面只接一次激勵函數，而兩個 3×3 之間夾著一次、後面還有一次，共兩次非線性。換句話說，小核堆疊在省參數的同時，還讓函數的表達能力變強——這是少見的「兩邊都賺」。右圖把這個關係推到底：堆 n 個 3×3 的參數量是單一等效大核的 $9n/(2n+1)^2$，n 愈大省得愈多，堆六個時只剩 32%。這條算術就是 VGG 之後幾乎所有架構都改用小核的原因，也說明了一件事：架構設計的進展，常常不是來自更複雜的想法，而是來自把成本算清楚。下一張圖的殘差連接與圖 15-17 的深度可分離卷積，也都是同一種思路。

圖 15-15　VGG 的算術：為什麼全部改用 3×3

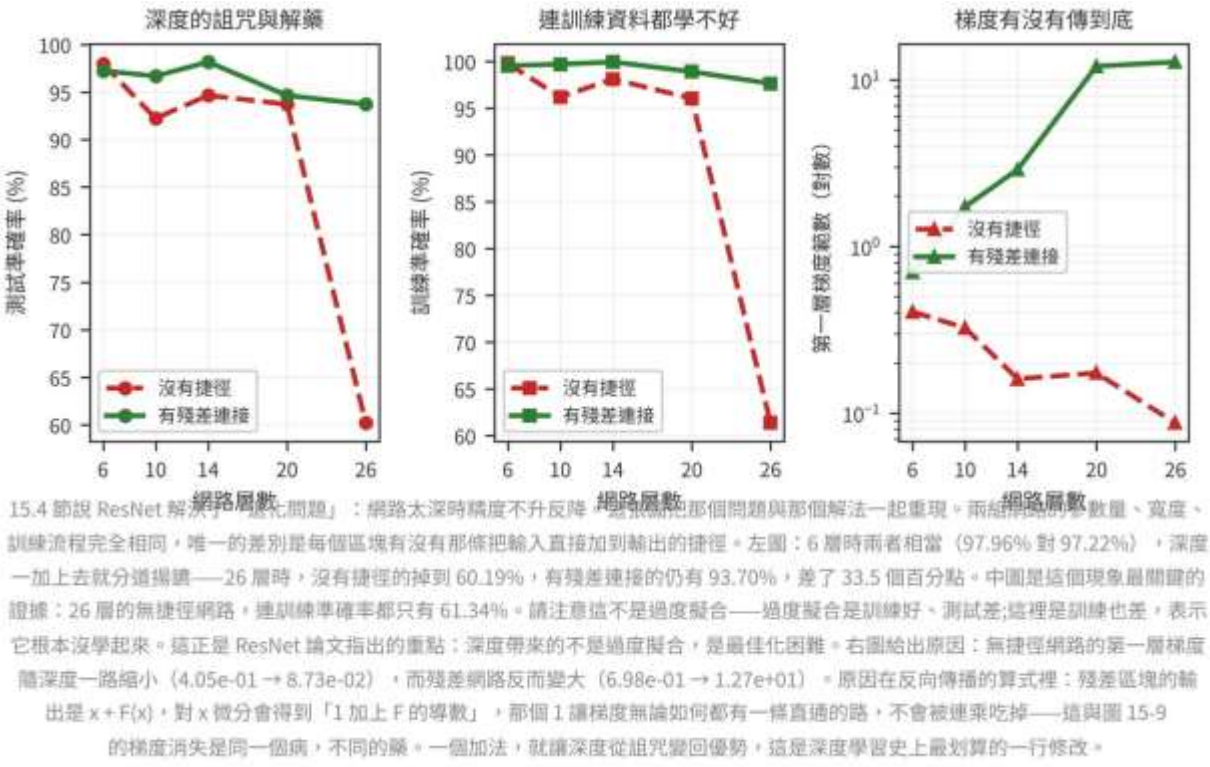
15.4 經典 CNN 架構譜系

自 AlexNet 引爆革命後，CNN 架構快速演進，每一代都針對前一代的瓶頸提出突破。理解這條譜系，就是理解深度學習視覺這十餘年的發展主線。

1. LeNet(1998,LeCun):CNN 的開山始祖，約七層，用於 MNIST 手寫數字辨識(第 14 章)。它奠定了「卷積-池化-全連接」的基本範式，但受限於當年的資料與算力，未能大放異彩。
2. AlexNet(2012,Krizhevsky 等)：叫獸開講的主角，八層。以 ReLU、Dropout、GPU 訓練與資料增強，在 ImageNet 上一舉成名，引爆深度學習革命。它證明了「夠深的網路 + 大數據 + GPU」的威力。
3. VGGNet(2014，牛津)：把網路加深到 16、19 層，並確立了一個優雅的原則——全部只用小小的 3×3 卷積核堆疊。它證明了「深度」本身的價值，結構規整、易於理解，至今仍是教學與遷移學習的常用骨幹。

4. GoogLeNet / Inception(2014,Google)：同年冠軍，提出「Inception 模組」——在同一層並行使用多種尺寸的卷積核(1×1、3×3、5×5)，讓網路自己選擇適合的尺度(呼應多尺度思想);並用 1×1 卷積大幅壓縮參數，以 22 層的深度卻比 AlexNet 更省參數。

5. ResNet(2015，微軟何愷明等)：解決了一個致命瓶頸——網路太深時，梯度難以傳遞、反而更難訓練(退化問題)。它的神來之筆是「殘差連接」(skip connection)：讓資訊可以「跳過」若干層直接往後傳，使得訓練上百層的網路成為可能。ResNet 在 ImageNet 上的錯誤率首度低於人類水準(約 5%)，是深度學習史上又一里程碑，殘差連接的思想至今無所不在(包括 Transformer)。

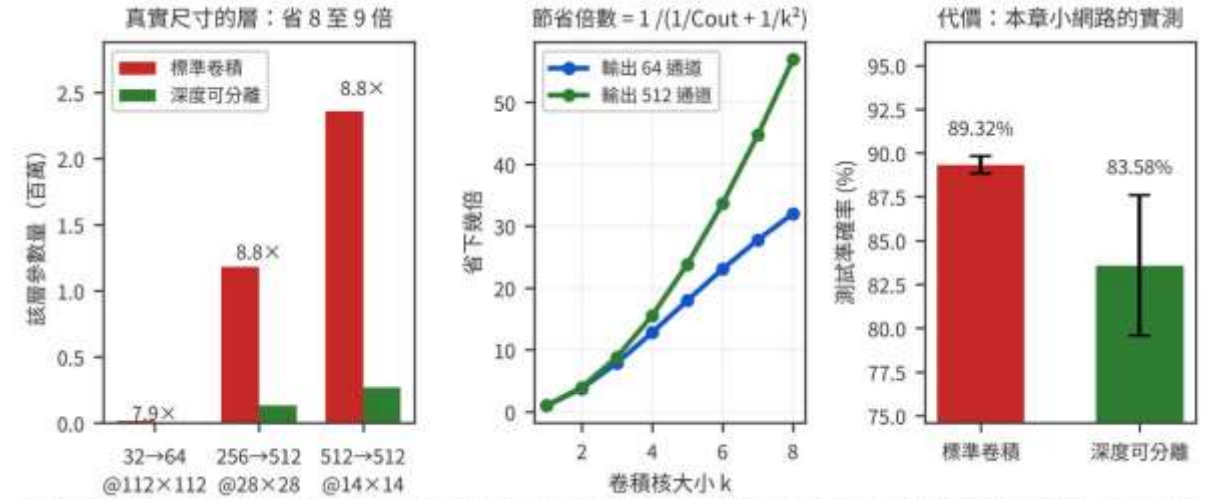


15.4 節說 ResNet 解決了「退化問題」：網路太深時精度不升反降，並把那個問題與那個解法一起重現。兩組網路的參數量、寬度、訓練流程完全相同，唯一的差別是每個區塊有沒有那條把輸入直接加到輸出的捷徑。左圖：6 層時兩者相當（97.96% 對 97.22%），深度一加上去就分道揚鑣——26 層時，沒有捷徑的掉到 60.19%，有殘差連接的仍有 93.70%，差了 33.5 個百分點。中圖是這個現象最關鍵的證據：26 層的無捷徑網路，連訓練準確率都只有 61.34%。請注意這不是過度擬合——過度擬合是訓練好、測試差;這裡是訓練也差，表示它根本沒學起來。這正是 ResNet 論文指出的重點：深度帶來的不是過度擬合，是最佳化困難。右圖給出原因：無捷徑網路的第一層梯度隨深度一路縮小（4.05e-01 → 8.73e-02），而殘差網路反而變大（6.98e-01 → 1.27e+01）。原因在反向傳播的算式裡：殘差區塊的輸出是 x + F(x)，對 x 微分會得到「1 加上 F 的導數」，那個 1 讓梯度無論如何都有一條直通的路，不會被連乘吃掉——這與圖 15-9 的梯度消失是同一個病，不同的藥。一個加法，就讓深度從詛咒變回優勢，這是深度學習史上最划算的一行修改。

圖 15-16　殘差連接：讓深度重新變成優勢

6. MobileNet(2017,Google)：前面的架構愈做愈深、愈準，但也愈龐大、愈耗算力，無法在手機或嵌入式裝置上執行。MobileNet 反其道而行，以「深度可分離卷積」大幅壓縮運算量與模型大小，在可接受的精度損失下，讓 CNN 能在手機、樹莓派、Jetson 等資源受限的邊緣裝置上即時執行——這正是第 17 章邊緣部署的關鍵，也呼應第 9、10、13 章反覆強調的「適合勝於最強」。

這條譜系揭示了一個清晰的張力：一邊是「愈深愈準」的極致追求(VGG、ResNet)，一邊是「夠好且能部署」的務實考量(MobileNet)。成熟的工程師懂得依場景選擇——雲端伺服器追求極致精度，邊緣裝置追求效率，沒有一個架構適用於所有場合。這也是為什麼您的教材要完整涵蓋整條譜系：它們不是「新的取代舊的」，而是各自佔據不同的應用生態位。



15.4 節說 MobileNet 以「深度可分離卷積」大幅縮減計算量。它的做法是把一次卷積拆成兩步：先讓每個輸入通道各自做一次 3×3（只管空間、不混通道），再用 1×1 卷積把通道混合（只管通道、不管空間）。左圖算的是真實架構裡的層：256→512 通道的那一層，標準卷積要 1,179,648 個參數，拆開後只要 133,376 個，省 8.8 倍。中圖給出通式：節省倍數是 $1/(1/\text{Cout} + 1/k^2)$，通道數夠多時大約就是 k^2，所以 3×3 省約九倍、5×5 可省到二十倍以上。右圖是代價，而且必須誠實講：把本章小網路的第二個卷積層換成可分離版本，該層參數從 4,640 降到 688，但測試準確率從 89.32% ± 0.49 掉到 83.58% ± 4.01，少了 5.74 個百分點，而且種子之間的波動大了好幾倍。這個結果不意外：拆開之後表達能力確實變弱了，而在這種小網路上，省下的參數又被全連接層的龐大參數量稀釋掉，所以是賠本生意。MobileNet 的價值要在「大網路、要塞進手機或嵌入式裝置」時才兌現——那正是第 17 章邊緣 AI 的主題。工程上的判準一如既往：先問省下來的資源是不是你真正缺的那一種。

圖 15-17　深度可分離卷積：MobileNet 的那筆帳

表 15-1 經典 CNN 架構譜系

15.5 三大框架與 MNIST 實作

手動實作神經網路(尤其是反向傳播)極為繁瑣，所幸有深度學習框架代勞——它們自動處理梯度計算、GPU 加速、以及大量現成的網路元件。三大主流框架各有特色：

1. TensorFlow(Google,2015)：工業界部署的重鎮，生態完整，尤其擅長大規模生產環境與行動端(TensorFlow Lite 可部署到手機，更受限的微控制器則有 TensorFlow Lite for Microcontrollers，呼應第 17 章)。

2. Keras：原為獨立的高階 API、現已整合進 TensorFlow，以極簡潔的語法著稱，幾行程式就能搭出一個網路，是初學者與快速原型的首選——也是您課程 MNIST 教學的理想起點。
3. PyTorch(Meta,2016)：學術研究的主流，以直觀的「動態計算圖」與貼近 Python 的風格深受研究者喜愛，除錯容易，近年在產業界也快速普及。

您的教材採用「TensorFlow → Keras → PyTorch」的跨框架 MNIST 實作，用意深遠：同一個手寫數字分類任務，用三種框架各做一遍，學生能清楚看見「網路架構的思想是共通的，只是語法不同」——這培養的是不被單一工具綁架的核心理解力。以下以 Keras 示範最簡潔的版本。

程式 15-1 Keras 實作 MNIST CNN 分類器

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. 載入 MNIST(第 14 章的經典資料集)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0   # 正規化到 0~1
x_test  = x_test.reshape(-1, 28, 28, 1) / 255.0

# 2. 建構 CNN(卷積-池化-全連接,15.3 節的結構)
model = keras.Sequential([
    layers.Conv2D(32, 3, activation="relu", input_shape=(28,28,1)),
    layers.MaxPooling2D(),                          # 池化降取樣
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),                               # 攤平接全連接
    layers.Dropout(0.5),                            # 對抗過度擬合(第 14 章)
    layers.Dense(10, activation="softmax")          # 10 類輸出
])

# 3. 編譯:指定損失函數與最佳化器(15.2 節)
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# 4. 訓練
model.fit(x_train, y_train, epochs=5, batch_size=128,
          validation_split=0.1)     # 切出驗證集(第 14 章)

# 5. 評估(用沒看過的測試集,第 14 章)
loss, acc = model.evaluate(x_test, y_test)
```

```
print(f"測試準確率:{acc:.4f}")      # 通常可達 99% 以上
```

請留意這段程式如何綜合了前兩章的觀念：正規化(第 3 章)、卷積與池化(本章)、Dropout 對抗過度擬合(第 14 章)、切驗證集與用測試集評估(第 14 章)、交叉熵損失與 Adam 最佳化器(本章)。短短二十行，是全書多章觀念的結晶。

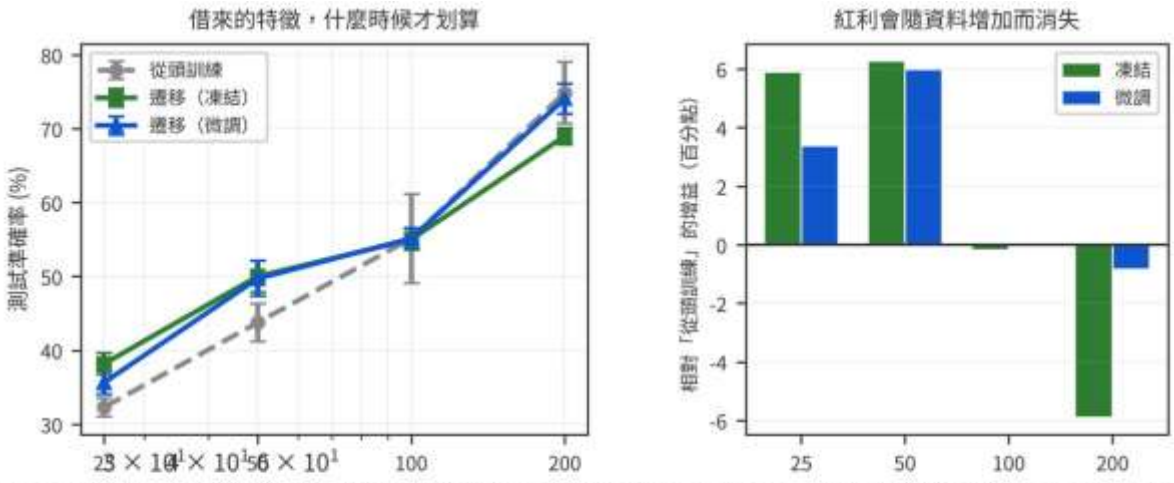
15.6 遷移學習與資料增強

15.6.1 遷移學習：站在巨人的肩膀上

訓練一個像 ResNet 這樣的大型網路，需要 ImageNet 那樣的百萬級資料與大量 GPU 時間——這對絕大多數應用(尤其是資料稀少的中小企業與學生專題)是不切實際的。遷移學習(Transfer Learning)是這個困境的救星，其思想極為漂亮：一個在 ImageNet 上訓練好的網路，它較低的層已經學會了通用的視覺特徵(邊緣、紋理、形狀)——這些特徵對任何影像任務都有用，不必從頭再學。

因此，我們可以「借用」一個預訓練模型：保留它已經學好的特徵萃取層(凍結權重)，只替換並重新訓練最後負責分類的幾層，使其適應我們自己的任務。這樣一來，即使只有幾百張自己的影像，也能訓練出高效能的模型——因為困難的「學特徵」部分已經由巨人(ImageNet 預訓練)完成了，我們只需教會它「把這些特徵對應到我的類別」。這呼應第 14 章「資料是燃料」：遷移學習讓你借用別人燒過的燃料所產生的成果。對雲林在地的農業分級、工業檢測等樣本有限的應用，遷移學習往往是比從頭訓練更務實、更高效的選擇。但這句話有一個必須補上的條件，否則很容易用錯。圖 15-18 把「凍結式遷移」「從頭訓練」「小學習率微調」三條路線放在同一個實驗裡比較：目標任務每類只有 5 張(共 25 張)時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類增加到 40 張(共 200 張)時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。道理不難理解：凍結等於宣告「這些特徵不准改」，資料少時這是有益的約束，資料多到足以自己學出更貼合的特徵時，它就變成了天花板(用第 14.6.4 節的話說，就是用偏差換變異，而這筆交易會隨資料量改變划不

划算)。實務上的標準答案因此是微調：不凍結，但用較小的學習率更新整個網路，讓預訓練權重當成好的起點而非硬性約束;實測它在小資料時接近凍結式、資料變多時又追上從頭訓練，兩邊都不吃虧。一句話：資料愈少，凍結愈划算;資料變多，就該把特徵層解凍——而交叉點在哪裡，只能靠驗證集找。

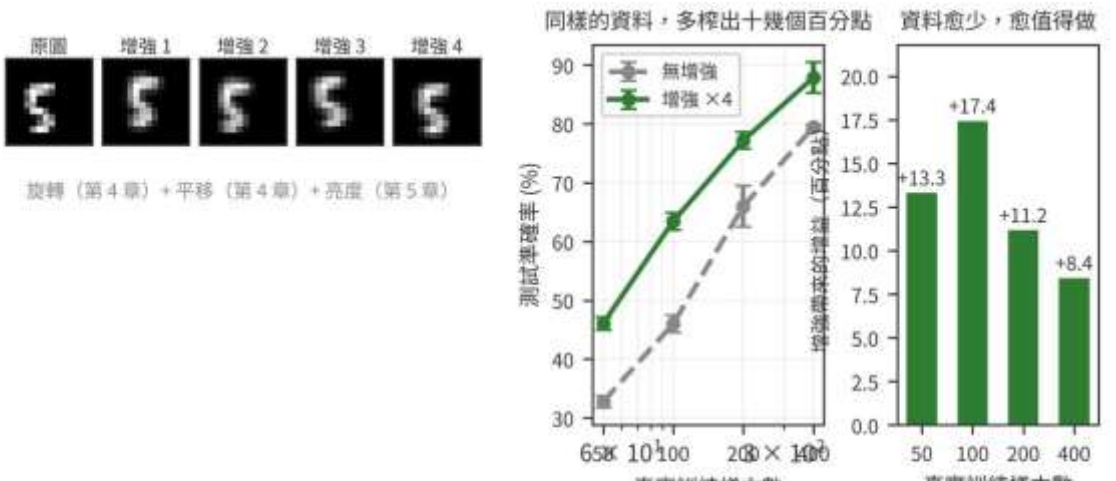


15.6.1 節說遷移學習是預訓練模型的特徵層，只重訓分類層，少量資料就能表現。這張圖用一個乾淨的設計把這句話量出來，並補上它沒說的那一半。來源任務是數字 0 到 4（901 張，預訓練到訓練準確率 100%），目標任務是完全沒看過的數字 5 到 9，樣本數從 25 張變化到 200 張。小資料時遷移學習確實有效：每類 5 張（共 25 張）時，凍結式遷移 38.17%、從頭訓練只有 32.29%，多了 5.88 個百分點;每類 10 張時仍多 6.25 個百分點。但請看右邊兩根：每類 40 張（共 200 張）時，凍結式遷移 68.97% 反而輸給從頭訓練的 74.85%，落後 5.88 個百分點。道理不難理解：凍結等於宣告「這些特徵不准改」，資料少時這是有益的約束，資料多到足以自己學出更貼合的特徵時，它就變成了天花板。第三條線是實務上的標準答案——不凍結、用較小的學習率微調整個網路。它在小資料時接近凍結式（35.64%），資料變多時又追上從頭訓練（74.03%），兩邊都不吃虧。所以 15.6.1 節那句話要補一個條件：資料愈少，凍結愈划算;資料變多，就該把特徵層解凍。這個判斷點只能靠驗證集找（第 14.5 節）。

圖 15-18　遷移學習：什麼時候該借，什麼時候該自己學

15.6.2 資料增強：第 4、5 章的華麗回歸

第 14 章介紹過，資料增強是對抗過度擬合最有效的手段之一，而它用的正是本書前面的技術!對訓練影像做隨機的旋轉、平移、翻轉(第 4 章幾何轉換)、亮度與對比調整(第 5 章空間域強化)、乃至加入雜訊(第 7 章)——每一種變換都產生一張「新」的訓練樣本，人工擴充了資料的多樣性，逼迫模型學習真正穩健的特徵(而非死背某張圖的細節)。



15.6.2 節說資料增強是「第 4、5 章的華麗回歸」，這張圖把那句話的價碼算出來。左圖是效果，右圖就是增強的全部內容。隨機旋轉正負 12 度（第 4 章的幾何轉換）、隨機平移一個多像素（第 4 章）、亮度乘上 0.8 到 1.2 倍（第 5 章的影像強化）。沒有任何新資料、沒有任何新標註，只是把同一張圖換個樣子再看一次。右圖是效果：只有 50 張真實樣本時，不增強的 CNN 只有 32.73%，增強到四倍之後升到 46.04%，多了 13.3 個百分點;100 張時更誇張，增益達到 17.4 個百分點。但請注意最右邊那根：400 張時增益縮到 8.4 個百分點。資料愈多，增強的邊際效益愈小——這與第 14.6.3 節圖 14-19 量到的趨勢完全一致，因為兩者對抗的是同一個東西：變異。這裡值得停下來想一件事：增強用的全是本書第 4、5 章的基本運算，沒有任何深度學習的成分，卻是小資料情境下性價比最高的一招——它便宜、不需要標註、而且立刻見效。代價是必須選對增強方式：對手寫數字做上下翻轉會把 6 變成 9，那不是增強而是加錯標籤。增強要保留「標籤不變」這個前提，這需要的是領域知識，不是演算法。

圖 15-19　資料增強：第 4、5 章技術的華麗回歸

叫獸開講中的 AlexNet 正是資料增強的早期實踐者——它用隨機裁切、水平翻轉、色彩擾動來擴充 ImageNet，顯著降低了過度擬合。這個美妙的連結值得再次強調：深度學習訓練中不可或缺的資料增強，骨子裡就是第 4、5、7 章的影像處理技術。這再次證明，深度學習不是憑空而來的新學問，而是穩穩地站在傳統影像處理的地基之上——這正是本教科書把深度學習放在最後、而非一開頭的深意。

15.6.3 推導：遷移學習與資料增強，為什麼在小資料上特別有效

15.6 節介紹了兩個對付小資料的手段，但沒說它們為什麼有效、又在什麼時候會失效。答案藏在第 14.6.4 節推導過的偏差-變異分解裡，把它拿來重讀一次，兩件事就都說得通了。

回憶那條等式：總誤差 = 偏差² + 變異 + 雜訊。深度學習模型的參數動輒數百萬，表達能力極強，所以偏差很低;但也因此對訓練資料極度敏感，變異很高。換一批訓練資料就學出一個差很多的模型——這正是小資料情境下深度學習最大的敵人。所以對付小資料的手段，目標幾乎都是同一個：壓低變異。

資料增強是最直接的一招。它利用一個領域知識：對影像做小幅旋轉、平移、亮度調整之後，標籤不會改變。於是一張標註可以生出許多訓練樣本，等於告訴模型「這些變化都不該影響你的答案」，把原本要靠資料量才能學到的不變性，直接寫進訓練集裡。圖 15-19 實測：只有 50 張真實樣本時，增強到四倍讓準確率從 32.73% 升到 46.04%，多了 13.3 個百分點;但 400 張時增益縮到 8.4 個百分點。這個遞減完全符合預期——增強出來的樣本彼此高度相關，提供的「新資訊」遠少於等量的真實資料，當真實資料已經夠多，它能壓的變異就所剩無幾了。第 14.6.3 節圖 14-19 在傳統模型上量到的也是同一個趨勢。

這裡要補一個常被忽略的前提：增強必須保持標籤不變。對手寫數字做上下翻轉會把 6 變成 9，那不是增強，是在訓練集裡加錯誤標籤;對醫學影像做左右翻轉可能把心臟換到右邊，造出不存在的病例。選擇增強方式需要的是領域知識，不是演算法——這也是為什麼自動化增強搜尋至今仍是研究題目。

遷移學習壓低變異的方式更強硬：它直接凍結大部分參數。一個凍結了特徵層的模型，可訓練參數可能只剩原來的百分之幾，變異自然大幅下降。它之所以行得通，是因為低層特徵（邊緣、紋理、顏色漸變）對幾乎所有視覺任務都通用——這一點與第 6 章的 Gabor、第 13 章的 SIFT/HOG 是同一個洞見，只是這次的通用特徵是從一百萬張影像上學出來的，而不是人設計的。

但凍結同時也把偏差鎖住了。被凍結的特徵層學不到目標任務特有的東西，這在資料少時無所謂（反正也學不準），資料一多就變成天花板。圖 15-18 把這個交叉點量了出來：每類 5 張時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類 40 張時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。用偏差-變異的話來說：資料少時「少一點變異」值得用「多一點偏差」去換，資料多時這筆交易就不划算了。

實務上的標準做法因此是微調（fine-tuning）：不凍結，但用比較小的學習率更

新整個網路，讓預訓練權重當成一個好的起點而非硬性約束。圖 15-18 的第三條線顯示它兩邊都不吃虧——小資料時接近凍結式，資料變多時追上從頭訓練。常見的折衷還有「分段解凍」：先只訓練分類頭，等它穩定了再解凍高層、最後解凍低層，每解凍一次就把學習率調小一些。

最後把這三章串起來。第 14 章給了偏差-變異這把尺;第 15 章的遷移學習與資料增強，是這把尺在深度學習上的兩個具體用法;而該用哪一個、用到什麼程度，答案永遠來自驗證集，不是來自預設值。深度學習換掉了特徵工程，但沒有換掉第 14 章任何一條評估規矩。

程式 15-2 遷移學習與資料增強

```
from tensorflow import keras
from tensorflow.keras import layers

# --- 資料增強:全是第 4、5、7 章的影像處理技術 ---
augment = keras.Sequential([
    layers.RandomFlip("horizontal"),      # 第 4 章:翻轉
    layers.RandomRotation(0.1),           # 第 4 章:旋轉
    layers.RandomZoom(0.1),               # 第 4 章:縮放
    layers.RandomContrast(0.2),           # 第 5 章:對比
])

# --- 遷移學習:借用 ImageNet 預訓練的 MobileNetV2 ---
base = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,          # 去掉原本的分類層
    weights="imagenet")         # 載入預訓練權重
base.trainable = False          # 凍結:保留學好的通用特徵

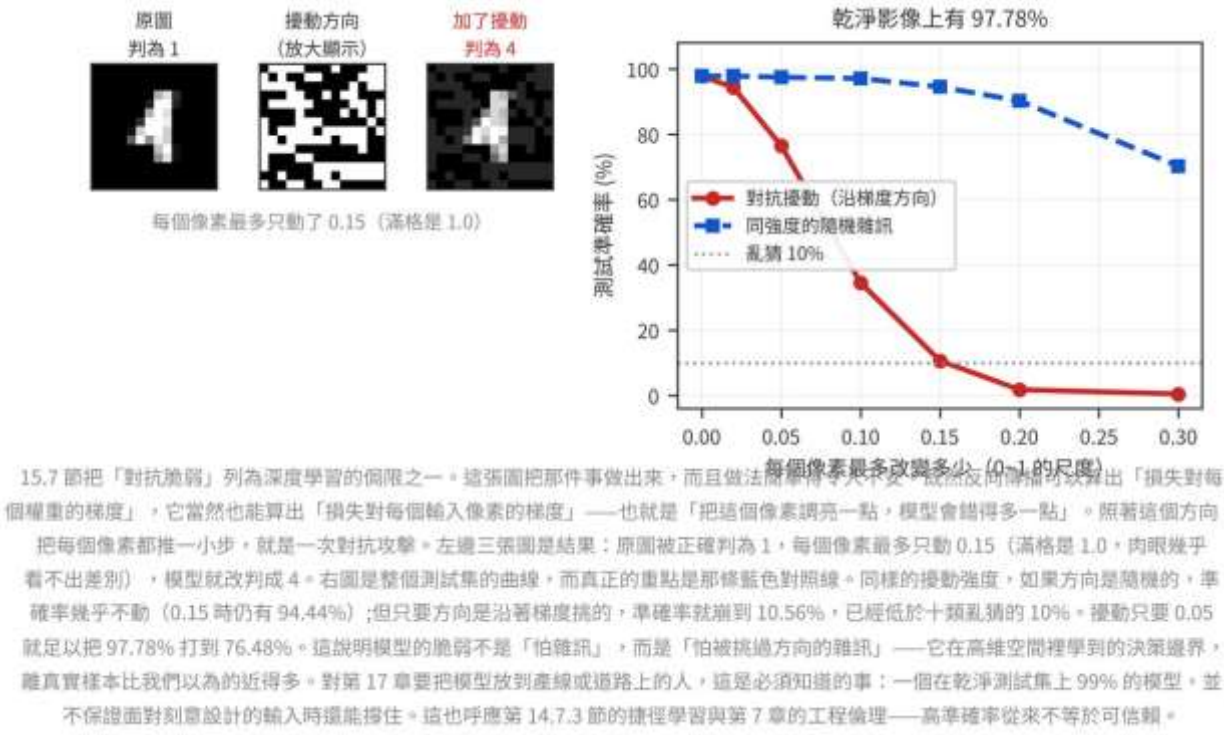
model = keras.Sequential([
    augment,                            # 先做資料增強
    base,                               # 借用的特徵萃取器
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(5, activation="softmax")   # 換成自己的 5 類
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
# 只需少量自己的資料即可訓練出高效能模型
```

15.7 深度學習的侷限與展望

深度學習威力驚人，但絕非萬靈丹。理解它的侷限，是負責任地使用它的前提，也呼應了第 7、14 章一路強調的工程倫理。

第一，資料飢渴：深度學習需要大量標註資料，取得成本高昂;遷移學習與資料增強能緩解，但無法完全消除。第二，算力昂貴：訓練大型模型耗費可觀的電力與硬體，有其環境與成本代價。第三，黑箱難解：深度網路的決策過程難以解釋——它為什麼判斷這是貓?我們往往說不清。這在醫療、司法等高風險領域是嚴重問題(呼應第 14 章的肺炎 X 光捷徑陷阱：模型可能學到與答案相關卻無意義的捷徑，而我們難以察覺)。第四，對抗脆弱性：對影像加入人眼難以察覺的微小擾動，可能讓模型完全誤判——這在安全攸關的應用(如自駕、人臉辨識門禁)中是真實的風險。第五，分布偏移：模型在訓練分布外的資料上表現會急劇下滑(第 7.7.3 節已詳述)。



15.7 節把「對抗脆弱」列為深度學習的侷限之一。這張圖把那件事做出來，而且做法簡單得令人不安。我們反向傳播可以算出「損失對每個權重的梯度」，它當然也能算出「損失對每個輸入像素的梯度」——也就是「把這個像素調亮一點，模型會錯得多一點」。照著這個方向，把每個像素都推一小步，就是一次對抗攻擊。左邊三張圖是結果：原圖被正確判為 1，每個像素最多只動 0.15（滿格是 1.0，肉眼幾乎看不出差別），模型就改判成 4。右圖是整個測試集的曲線，而真正的重點是那條藍色對照線。同樣的擾動強度，如果方向是隨機的，準確率幾乎不動（0.15 時仍有 94.44%）;但只要方向是沿著梯度挑的，準確率就崩到 10.56%，已經低於十類亂猜的 10%。擾動只要 0.05 就足以把 97.78% 打到 76.48%。這說明模型的脆弱不是「怕雜訊」，而是「怕被挑過方向的雜訊」——它在高維空間裡學到的決策邊界，離真實樣本比我們以為的近得多。對第 17 章要把模型放到產線或道路上的人，這是必須知道的事：一個在乾淨測試集上 99% 的模型，並不保證面對刻意設計的輸入時還能撐住。這也呼應第 14.7.3 節的捷徑學習與第 7 章的工程倫理——高準確率從來不等於可信賴。

圖 15-20　對抗脆弱：看不出差別的影像，模型全認錯

展望未來，幾個方向正在改變格局：Transformer 架構(源於自然語言處理)已進入視覺領域(Vision Transformer)，在大資料上表現超越 CNN;自監督學習試圖擺脫對人

工標註的依賴，讓模型從無標籤資料中自學;而生成式 AI(擴散模型、多模態大模型)則模糊了「辨識」與「創造」的界線。但無論架構如何演進，本章的基礎——神經元、卷積、損失、梯度下降、反向傳播、以及對抗過度擬合的智慧——都是理解這一切的根基。技術的表層日新月異，底層的原理歷久彌新。

15.8 本章與全書的觀念連結總表

表 15-2 第 15 章與全書其他章節的觀念連結

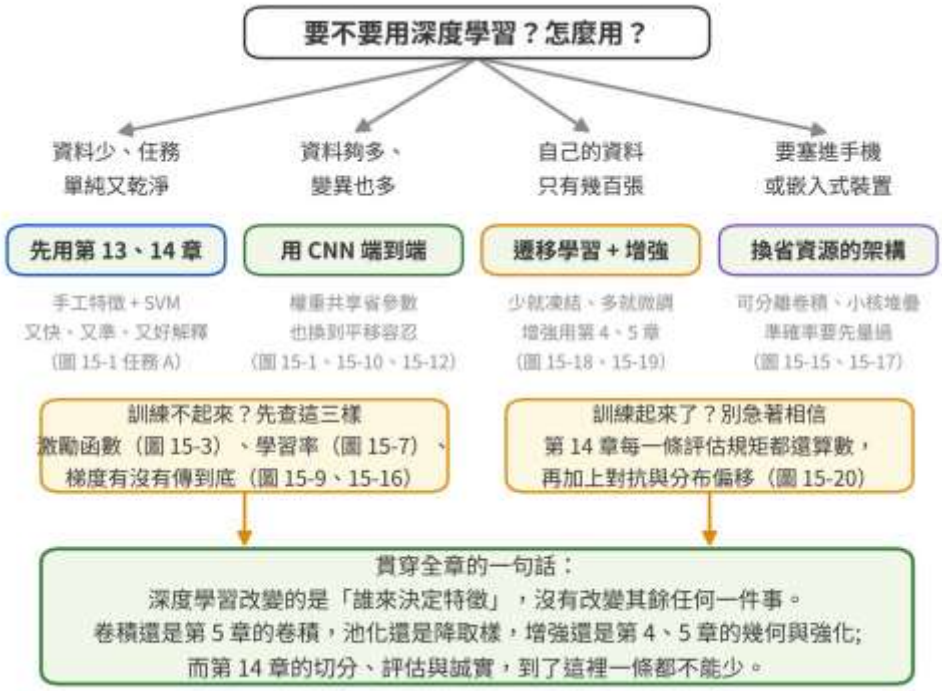


圖 15-21　第 15 章的決策地圖

程式實作 Lab 15

Lab 15-1 跨框架 MNIST 三部曲

目標：體會「架構思想共通、語法各異」。(a)以程式 15-1 用 Keras 訓練 MNIST CNN，達到 99% 以上準確率;(b)用 PyTorch 實作同樣架構的網路，比較兩者的程式風格(Keras 的宣告式 vs. PyTorch 的命令式);(c)用純 TensorFlow(低階 API)實作，體會框架自動化了多少細節;(d)三者訓練同一任務，比較準確率是否一致(應該相近)、程式碼行數與可讀性;(e)討論：為什麼說「不被單一工具綁架的核心理解力」比熟練某個框架

更重要?

Lab 15-2 卷積核視覺化

目標：親眼見證叫獸開講「網路自動學出特徵」的核心。(a)訓練一個 CNN 後，把第一層卷積核視覺化成小圖;(b)觀察它們是否自發演化成邊緣偵測器、方向性條紋(對照第 6 章 Gabor、第 13 章 SIFT);(c)把一張影像餵入網路，視覺化各層的「特徵圖」(feature map)，觀察淺層與深層分別對什麼有反應;(d)討論：這與第 13.8 節「CNN 重新發明手工特徵」的說法是否吻合?為什麼理解手工特徵有助於理解 CNN?

Lab 15-3 遷移學習實戰

目標：用少量資料訓練高效能模型。(a)收集自己的小資料集(如三到五類水果或零件，每類數十張);(b)以程式 15-2 借用 ImageNet 預訓練的 MobileNetV2，凍結特徵層、只訓練分類層;(c)對照組：用同樣的資料從頭訓練一個 CNN，比較兩者的準確率與訓練時間——遷移學習應在小資料上明顯勝出;(d)加入資料增強，觀察測試準確率的提升;(e)討論：對雲林在地的農業或製造業應用，為什麼遷移學習加資料增強常比從頭訓練更務實?

Lab 15-4 過度擬合的深度學習版

目標：在深度學習情境重現第 14 章的過度擬合診斷。(a)訓練一個 CNN，畫出訓練與驗證的準確率/損失隨 epoch 變化的曲線;(b)觀察過度擬合的典型徵兆：訓練準確率持續上升、但驗證準確率停滯甚至下降;(c)分別加入 Dropout、資料增強、及早停止，觀察各自如何改善過度擬合(呼應第 14 章四大武器);(d)討論：深度學習的過度擬合對抗手段，與第 14 章的傳統機器學習有何異同?為什麼深度網路特別容易過度擬合?

Lab 15-5 自己寫一次反向傳播

目標：把 15.2 節的公式變成你自己寫得出來的程式——這是理解深度學習最有效、也最一勞永逸的一堂課。全程只用 NumPy，不需要任何框架，也不需要 GPU。(a)實

作一個 Dense 層，提供 forward(x) 與 backward(dout) 兩個方法，分別對應 z = x·W + b 與 15.2.5 節那兩條式子;(b)實作 ReLU 與 Sigmoid 及其導數，再實作 Softmax 搭配交叉熵損失，確認最後一層的誤差訊號恰好是「預測機率減去 one-hot 答案」;(c)做梯度檢驗：把某個權重加減一個極小的 ε，用損失差除以 2ε 得到數值梯度，與反向傳播的結果比對相對差，確認中位數低於 1e−4（用 Tanh 做，理由見圖 15-8 與習題 17）;(d)用 sklearn 的 load_digits 訓練一個兩層網路，目標是測試準確率超過 95%;(e)把激勵函數全部換成 linear 重訓一次，並把各層權重相乘摺成單一矩陣，驗證它與原網路的輸出差異在浮點誤差等級（重現圖 15-4）;(f)把學習率設成 0.001、0.1、1.0、5.0 各訓練一次，畫出四條損失曲線，找出發散的那一個（重現圖 15-7）。延伸：再加上 Conv2D 與 MaxPool2D 兩層並通過梯度檢驗，你就擁有一個完整的迷你框架了。

Lab 15-6 小資料生存戰

目標：在只有幾百張影像的真實處境下，把能用的手段全部量過一遍，並學會用第 14 章的規矩誠實地報告結果。(a)建立基準線：收集或指定一個三到五類、每類數十張的小資料集（水果成熟度、零件良品與不良品皆可），先用第 13 章的 HOG 或顏色直方圖配 SVM 做出一個傳統基準，記錄準確率與訓練時間;(b)從頭訓練一個小 CNN，與基準比較——若 CNN 輸了，不要修飾結果，這正是圖 15-1 任務 A 的情境，請在報告裡解釋為什麼;(c)資料增強：加入旋轉、平移、亮度與對比調整（第 4、5 章），先確認每一種變換都不會改變標籤，再量出增益，並畫出「增益 vs 真實樣本數」的曲線（重現圖 15-19 的趨勢）;(d)遷移學習：借用一個預訓練模型，分別做「凍結特徵層」與「小學習率微調」兩種版本，在不同樣本數下比較三條曲線（從頭、凍結、微調），找出凍結開始變成天花板的交叉點（圖 15-18）;(e)誠實評估：以上每一個數字都要用交叉驗證跑至少三個隨機種子並附標準差;若兩個方法的誤差範圍重疊，就寫「分不出高下」，不要宣稱贏;(f)穩健性檢查：把測試影像平移幾個像素、改變亮度、或換一個拍攝場景再測一次，看看準確率掉多少，並說明這對上線部署的意義（圖 15-12、15-20）。延伸：挑你最好的模型做一次對抗測試——沿著損失對輸入的梯度方向加上肉

眼幾乎看不見的擾動，記錄準確率崩到多少，然後在報告最後寫三句話：這個模型可以上線嗎?為什麼?還缺什麼?

本章重點整理

- 深度學習的核心典範轉移：把特徵設計從人類手中交給機器，端到端一起學習;CNN 首層自發學出的濾波器與第 6、13 章的手工特徵(Gabor、SIFT)高度相似——深度學習是「自動重新發明」了手工特徵。
- 神經網路構件：神經元(加權求和加激活)、激勵函數(注入非線性，ReLU 為主流且緩解梯度消失)、損失函數(交叉熵/MSE 量化誤差)、梯度下降(往損失下坡走，學習率控制步長)、反向傳播(以連鎖律逆向算梯度，框架自動完成)。
- CNN 三大構件：卷積層(可學習的濾波核，局部連接、權重共享、層級特徵)、池化層(降取樣，即型態學膨脹/侵蝕)、全連接層(做最終決策);專為影像設計，解決 MLP 破壞空間結構與參數爆炸的問題。
- 架構譜系：LeNet(範式開創)→AlexNet(ReLU/Dropout/GPU，引爆革命)→VGG(3×3 加深)→GoogLeNet(Inception 多尺度)→ResNet(殘差連接，低於人類錯誤率)→MobileNet(深度可分離卷積，可邊緣部署);「愈深愈準」與「夠好可部署」兩條路線並存。
- 三大框架 TensorFlow(部署)、Keras(簡潔入門)、PyTorch(研究);跨框架 MNIST 實作培養不被工具綁架的核心理解;程式綜合了正規化、卷積池化、Dropout、驗證測試切分等全書觀念。
- 遷移學習借用預訓練模型的通用特徵、只重訓分類層，少量資料即可高效;資料增強(第 4、5、7 章的影像處理技術)對抗過度擬合。深度學習侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱、分布偏移——負責任使用需正視這些。

習題

1. 深度學習相對於傳統影像辨識(第 13、14 章)，最革命性的改變是什麼?請以「特徵」說明。

   提示與參考解答：最革命性的是把「特徵設計」從人類手中交給機器：傳統流程由人設計特徵(SIFT、HOG)再餵給分類器，受限於人的設計能力;深度學習把「像素到特徵」與「特徵到類別」整合進同一網路端到端學習，自己學出最有助於任務的特徵。如叫獸開講，AlexNet 網路層自動長出邊緣偵測器與物件辨識器。

2. 為什麼激勵函數是必要的?若把所有激勵函數拿掉，深層網路會變成什麼?ReLU 相對 Sigmoid 有何優勢?

   提示與參考解答：激勵函數注入非線性。若拿掉，無論疊多少層都只是一連串線性運算的組合，等效於單一層，無法學複雜規律。ReLU(負值歸零、正值保留)相對 Sigmoid：計算極簡、且大幅緩解梯度消失問題，讓深層網路訓練又快又穩，是 AlexNet 的關鍵創新之一。

3. 用「地形與下坡」的比喻說明梯度下降。學習率太大或太小分別會怎樣?

   提示與參考解答：損失是一片高低起伏的地形，目標是走到最低點(最小損失)。梯度指出最陡上坡方向，往反方向(下坡)走一小步損失就下降，反覆走逼近谷底。學習率控制步長：太大會在谷底附近震盪甚至衝過頭，太小則收斂慢得令人絕望——又一個取捨。

4. 反向傳播解決了什麼問題?它的數學核心是什麼?為什麼理解它仍然重要(即使框架會自動完成)?

   提示與參考解答：它解決「如何算出每個權重對損失的影響(梯度)」的問題，使調整深層網路數百萬參數變得可行，是神經網路能訓練的數學引擎。核心是微積分的連鎖律：前向算損失，再從輸出往輸入逆向逐層傳遞誤差算梯度。框架雖自動完成，但理解它能幫助診斷訓練失敗的原因(如梯度消失或爆炸)。

5. 多層感知器直接處理影像有哪兩個致命問題?CNN 如何分別解決?

   提示與參考解答：問題一：把影像攤平破壞了空間結構(相鄰像素關聯)。問題二：參數量爆炸(小圖攤平就數萬輸入，全連接層權重上億)。CNN 解決：卷積的局部連接保留空間結構;權重共享(同一核滑遍全圖)大幅減少參數並帶來平移不變性。

6. CNN 的卷積層與第 5 章的空間濾波有何相同、有何關鍵不同?這說明了什麼?

提示與參考解答：相同：都是用小核在影像上滑動計算局部加權和，數學上是同一件事(卷積)。關鍵不同：第 5 章的核是人設計的(高斯、Sobel),CNN 的核是訓練學出來的——網路自己決定核裡的數值以萃取最有用的特徵。說明深度學習「自動學特徵」正是建立在傳統卷積之上，是它的自動化與升級。

7. 卷積層的「權重共享」帶來哪兩個好處?池化層有什麼作用?它與第 10 章的什麼運算相同?

   提示與參考解答：權重共享(同一核滑遍全圖用同組權重)：其一大幅減少參數(一個 3×3 核只 9 個權重，不管影像多大)，其二帶來平移不變性(物件在哪都能被同一核偵測)。池化(如最大池化)降取樣：減少運算、擴大深層視野、提供位置容錯。最大/最小池化即第 10 章型態學的膨脹/侵蝕。

8. 請依序說明 CNN 架構譜系從 LeNet 到 MobileNet 各自的關鍵創新，並指出它們回應了什麼瓶頸。

   提示與參考解答：LeNet(1998)開創卷積-池化-全連接範式。AlexNet(2012)以 ReLU/Dropout/GPU 引爆革命，回應「深層難訓練」。VGG(2014)全用 3×3 加深，證明深度價值。GoogLeNet(2014)Inception 多尺度加 1×1 壓縮，回應「更深卻要省參數」。ResNet(2015)殘差連接，回應「太深反而難訓練的退化問題」，錯誤率低於人類。MobileNet(2017)深度可分離卷積，回應「大模型無法在邊緣裝置執行」。

9. ResNet 的「殘差連接」解決了什麼問題?為什麼它讓訓練上百層的網路成為可能?

   提示與參考解答：解決退化問題：網路太深時梯度難以傳遞、反而更難訓練、精度不升反降。殘差連接(skip connection)讓資訊可以「跳過」若干層直接往後傳，梯度也能沿捷徑順暢回傳，避免梯度消失，使訓練上百層網路成為可能。此思想至今無所不在(含 Transformer)。

10. 遷移學習的核心思想是什麼?為什麼它對「資料稀少」的應用特別有價值?

    提示與參考解答：核心思想：預訓練模型較低的層已學會通用視覺特徵(邊緣、紋理、形狀)，對任何影像任務都有用、不必重學;因此保留(凍結)特徵萃取層，只重新訓練最後的分類層以適應自己的任務。對資料稀少的應用特別有價值，因為困難的「學特徵」已由 ImageNet 預訓練的巨人完成，只需少量自己的資料教它「特徵對

應到我的類別」。

11. 資料增強用到了本書前面哪幾章的技術?為什麼說「深度學習站在傳統影像處理的地基上」?

    提示與參考解答：資料增強用第 4 章幾何轉換(旋轉、平移、翻轉、縮放)、第 5 章空間域強化(亮度、對比)、第 7 章(加雜訊)。AlexNet 即以隨機裁切、翻轉、色彩擾動擴充資料。這說明深度學習不是憑空而來：資料增強是傳統影像處理、卷積源於空間濾波、正則化源於第 7 章、訓練評估原則源於第 14 章——深度學習穩穩站在前十四章的地基上，這正是本書把它放在最後的深意。

12. (綜合思辨題)深度學習有哪些主要侷限?為什麼在醫療、司法等高風險領域使用時要格外謹慎?請結合第 7、14 章的觀點。

    提示與參考解答：侷限：資料飢渴、算力昂貴、黑箱難解、對抗脆弱(微小擾動致誤判)、分布偏移(訓練分布外表現急降)。高風險領域謹慎的原因：黑箱使我們難以解釋模型為何判斷，可能學到與答案相關卻無意義的捷徑(第 14 章肺炎 X 光)、或在新分布上崩壞(第 7.7.3 節);清晰不等於正確(第 7 章還原 vs. 腦補)。工程師的責任不只調高準確率，更要理解模型學到什麼、資料是否公平——這是技術素養與工程倫理。

13. 有人說「深度學習一定比傳統影像處理好」。請用圖 15-1 的兩個任務反駁或限縮這句話，並說明深度學習真正的優勢出現在什麼條件下。

    提示與參考解答：這句話不成立，條件才是關鍵。圖 15-1 的任務 A 是乾淨、置中的 8×8 手寫數字，原始像素配 SVM 拿到 99.26%、HOG 配 SVM 95.56%，而端到端的 CNN 只有 98.33%——傳統管線還贏。任務 B 只改一件事：把同樣的數字擺到 14×14 畫布的隨機位置。原始像素 + SVM 立刻掉到 84.63%、HOG（已調到最佳參數）86.48%，而同一個 CNN（6,218 個參數）拿到 90.93%。差別來自權重共享：同一組卷積核滑遍整張影像，物件在哪裡都認得。所以深度學習的優勢出現在「任務夠難、變異夠多、資料夠多」時;在小而乾淨的問題上，第 13、14 章的工具往往又快、又準、又好解釋，而且不需要 GPU。工程上的判準是先看任務與資料，不是先看技術新不新。

14. 為什麼沒有激勵函數的深層網路等於單層?請說明數學理由，並設計一個實驗來驗證這件事。

提示與參考解答：線性映射的合成仍是線性映射——十個矩陣連乘仍然只是一個矩陣，所以十層全線性網路的表達能力與一層完全相同。驗證實驗：訓練一個十層全線性網路，再把各層權重依序相乘摺成單一矩陣 W 與偏置 b，比較原網路的輸出與 x·W + b。圖 15-4 實測兩者最大差異只有 1.78e−14，這個數量級就是浮點捨入誤差，等於說兩者是同一個函數。更有意思的是效能：一層線性拿到 96.11%，十層全線性（參數多 16 倍）只有 91.30%，不進反退，因為表達能力沒變、最佳化卻變難了。加回 ReLU 後，三層拿到 97.59%。結論：非線性不是裝飾品，它是深度之所以有意義的前提。

15. 梯度消失是怎麼產生的?請用連鎖律算出 Sigmoid 九層之後大約剩多少，並說明 ReLU 與殘差連接各自如何處理這個問題。

提示與參考解答：反向傳播每往回一層，誤差訊號就要乘上該層激勵函數的導數一次（15.2.5 節）。Sigmoid 的導數最大只有 0.25，所以九層之後上限約是 0.25 的九次方，大約四十萬分之一。圖 15-9 實測一個九層網路：最後一層梯度範數 5.09e−01，傳到第一層只剩 6.84e−06，相差約三萬七千倍，等於前面幾層幾乎收不到學習訊號;訓練四十輪後只有 56.48%，而 ReLU 版本 94.81%。ReLU 的處理方式是讓正半邊導數固定為 1，連乘不縮小。殘差連接則更徹底：區塊輸出是 x + F(x)，對 x 微分得到「1 加上 F 的導數」，那個 1 讓梯度永遠有一條直通路徑。圖 15-16 實測：無捷徑網路第一層梯度隨深度從 4.05e−01 縮到 8.73e−02，有殘差連接的反而從 6.98e−01 長到 1.27e+01。

16. 分類問題為什麼用交叉熵而不用均方誤差?請說明「錯得愈離譜、學得愈慢」這個現象的成因。

提示與參考解答：MSE 配 Sigmoid 輸出時，梯度要乘上 Sigmoid 的導數，而 Sigmoid 在兩端飽和、導數趨近 0，於是模型「錯得最離譜」的那些樣本反而梯度最小、學得最慢——這是最糟的組合。圖 15-6 量出這個差距：當正確類別的分數是 −6（錯得很離譜）時，交叉熵的梯度是 0.9976，MSE + Sigmoid 只有 0.002436，相差約 410 倍。後果是同一個網路訓練四十輪，交叉熵 96.67%、MSE 只有 50.00%。交叉熵搭配 Softmax 之所以成為標準配備，是因為兩者的導數恰好把飽和項約掉，最終的誤差訊號就是「預測機率減正確答案」，乾淨且不飽和。迴歸問題沒有這個飽和結構，MSE 仍然合理。

17. 什麼是梯度檢驗?請寫出它的做法，並說明為什麼在 ReLU 網路上會有少數參數「檢驗失敗」，那是不是程式有錯。

    提示與參考解答：梯度檢驗用數值微分驗證反向傳播的推導：把某個權重加上與減去一個極小的 ε，各算一次損失，相減再除以 2ε，得到與推導完全無關的數值梯度，再與解析梯度比對相對差，一般以 1e−4 為合格線。圖 15-8 對一個含卷積、池化與全連接層的網路抽查數百個參數：Tanh 版本中位數 2.61e−09、ReLU 版本 8.79e−10，都遠低於合格線。但 ReLU 版本有約 2.3% 的參數「失敗」，最差到 1.85e−01。這不是程式有錯，而是 ReLU 在 0 處有折點、不可微：當權重剛好讓 z 落在 0 附近，加減 ε 會跨過折點，數值梯度量到的是折點兩側的平均斜率，本來就與解析梯度不同。Tanh 處處平滑，完全沒有這個現象。實務規矩：梯度檢驗用平滑激勵函數做，並看中位數與絕大多數參數，而不是單一最差值。

18. CNN 的權重共享帶來「平移不變性」——這個常見說法需要修正成什麼?請以圖 15-12 的實測說明，並說明真正的不變性從哪裡來。

    提示與參考解答：嚴格說，卷積給的是「平移等變」（影像平移，特徵圖跟著平移），加上池化之後換到一點「平移容忍」，但不是不變。圖 15-12 實測（三個模型都只用置中的數字訓練）：平移 2 個像素時，MLP 崩到 13.52%（十類亂猜是 10%）、CNN 還有 44.63%，CNN 確實好得多;但平移 4 個像素時 CNN 也只剩 12.41%，一樣崩了。真正的不變性來自資料：同一個 CNN 若訓練時就看過隨機平移的樣本（資料增強，第 4 章幾何轉換），平移 3 個像素仍有 89.07%，是未增強版本 25.93% 的 3.4 倍。代價是置中時略低（94.63% 對 97.78%），因為同樣的容量要分給更多情況。結論：不變性多半是資料教出來的，不是架構免費送的。

19. 池化層付出什麼代價、換到什麼好處?請用圖 15-13 的四個數字說明這筆交易划不划算。

    提示與參考解答：代價是準確率從 98.33% 降到 97.96%，少了 0.37 個百分點。換到的有三樣：其一，展平後的維度從 3,136 降到 144，總參數從 101,962 降到 6,218，少了約 16 倍;其二，預測時間快約 1.6 倍;其三，平移 2 個像素時的準確率從 11.1% 升到 44.1%，約 4 倍的平移容忍。此外池化還擴大感受野：3×3 卷積感受野是 3，加一次 2×2 池化後，下一個 3×3 卷積看到的相當於原圖 10 個像素，再池化一次是 20 個——這是深層能「看見整體」的關鍵（15.3.4 節）。用 0.37 個百分點換這些，在幾乎任何實務場合都划算，尤其是第 17 章要把模型塞進邊緣裝置時。

20. 為什麼 VGG 要把所有卷積核都改成 3×3?請算出兩個 3×3 與一個 5×5 的感受野與參數量，並說明小核堆疊除了省參數還多賺了什麼。

    提示與參考解答：兩個 3×3 串接的感受野是 1 + 2×(3−1) = 5，與一個 5×5 相同;三個 3×3 則等於一個 7×7。參數量方面（以 256 通道的層計算）：一個 5×5 要 1,638,400 個，兩個 3×3 只要 1,179,648 個，省 28%;一個 7×7 要 3,211,264 個，三個 3×3 只要 1,769,472 個，省 45%（圖 15-15）。多賺的是非線性次數：一個 5×5 後面只接一次激勵函數，兩個 3×3 之間夾一次、後面還有一次，共兩次，所以表達能力更強。通式是：堆 n 個 3×3 的參數量是單一等效大核的 $9n/(2n+1)^2$，n 愈大省愈多。這是少見的「省成本又提升能力」，也是 VGG 之後幾乎所有架構都改用小核的原因。

21. ResNet 要解決的「退化問題」是什麼?它與過度擬合有何根本不同?請用圖 15-16 的訓練準確率說明如何分辨這兩者。

    提示與參考解答：退化問題是指網路加深到一定程度後，精度不升反降。它與過度擬合的根本差別在訓練表現：過度擬合是「訓練好、測試差」;退化是「訓練也差」，表示模型根本沒學起來，是最佳化困難而非泛化問題。圖 15-16 實測：26 層的無捷徑網路訓練準確率只有 61.34%、測試 60.19%，兩者都差——這是退化不是過擬合;同樣 26 層加上殘差連接，訓練 97.61%、測試 93.70%。分辨方法因此很簡單：先看訓練準確率。訓練也上不去，就別往正則化或資料增強的方向調，該檢查的是梯度傳不傳得下去、學習率、初始化與架構（15.2.5 節）。

22. 遷移學習什麼時候該凍結特徵層、什麼時候該解凍?請用圖 15-18 的實測說明，並用偏差-變異的語言解釋為什麼會有交叉點。

    提示與參考解答：資料少時凍結、資料多時解凍。圖 15-18 實測：每類 5 張（共 25 張）時，凍結式遷移 38.17%、從頭訓練 32.29%，遷移領先 5.9 個百分點;但每類 40 張（共 200 張）時，凍結式 68.97% 反而輸給從頭訓練的 74.85%，落後 5.9 個百分點。用偏差-變異解釋（第 14.6.4 節）：凍結大幅減少可訓練參數，因而壓低變異，但同時把偏差鎖住——被凍結的特徵學不到目標任務特有的東西。資料少時「少一點變異」值得用「多一點偏差」去換;資料多到足以自己學出更貼合的特徵時，凍結就變成天花板。實務上的標準答案是微調：不凍結、但用較小的學習率，實測在小資料時接近凍結式、資料變多時追上從頭訓練，兩邊都不吃虧。交叉點在哪裡只能靠驗證集找。

23. 資料增強為什麼有效?它的增益會隨什麼改變?請說明選擇增強方式時最重要的前提。

    提示與參考解答：深度模型參數多、偏差低但變異高，而增強利用「小幅旋轉、平移、亮度變化不改變標籤」這個領域知識，把原本要靠資料量才學得到的不變性直接寫進訓練集，藉此壓低變異（15.6.3 節）。圖 15-19 實測：只有 50 張真實樣本時，增強到四倍讓準確率從 32.73% 升到 46.04%，多 13.3 個百分點;100 張時增益 17.4 個百分點;到 400 張時縮到 8.4 個百分點。增益隨真實資料量增加而遞減，因為增強出的樣本彼此高度相關，提供的新資訊遠少於等量真實資料。最重要的前提是「標籤不變」：對手寫數字做上下翻轉會把 6 變成 9，對醫學影像左右翻轉可能把心臟換到右邊——那不是增強，是加錯誤標籤。選擇增強方式靠的是領域知識，不是演算法。

24. (推導題)什麼是對抗樣本?它為什麼能用反向傳播做出來?請說明「模型怕的不是雜訊，而是被挑過方向的雜訊」這句話的實測根據。

    提示與參考解答：對抗樣本是加了微小、刻意設計的擾動後能讓模型誤判的輸入。既然反向傳播能算出損失對每個權重的梯度，它同樣能算出損失對每個輸入像素的梯度，也就是「這個像素往哪個方向改，模型會錯得更多」;沿著這個方向把每個像素推一小步，就是一次攻擊。圖 15-20 的實測給出那句話的根據：在相同的擾動強度下，若方向是隨機的，準確率幾乎不動（每像素最多改 0.15 時仍有 94.44%）;若方向沿著梯度挑選，準確率崩到 10.56%，已低於十類亂猜。擾動只要 0.05 就足以把 97.78% 打到 76.48%。這說明脆弱性是「方向性」的：模型在高維空間學到的決策邊界，離真實樣本比我們以為的近得多。對第 17 章要把模型放上產線或道路的人，這表示乾淨測試集上的高準確率不等於可信賴，呼應第 14.7.3 節的捷徑學習與第 7 章的工程倫理。

25. (綜合實作思辨題)你要為一家中小型零件廠做表面瑕疵檢測，手上有 300 張已標註影像、一台沒有獨立顯卡的工控電腦，客戶要求「漏檢率愈低愈好」。請規劃完整方案，說明每個決定的依據，並指出三個最可能失敗的地方。

    提示與參考解答：方案應逐項說明依據。(1)技術選擇：300 張屬於小資料，且工控電腦沒有 GPU，因此優先考慮「第 13 章特徵 + SVM」與「遷移學習 + 微調」兩條路線並實測比較，而不是從頭訓練大型 CNN（圖 15-1、15-18）。(2)小資料對策：資料增強必用，且只用保持標籤不變的變換（旋轉、平移、亮度;若瑕疵有方向

性則不可翻轉）——圖 15-19 顯示小資料時可多出十幾個百分點。遷移學習先凍結、資料累積後改微調。(3)部署考量：無 GPU 代表要控制模型大小與延遲，可評估深度可分離卷積與小核堆疊，但必須實測準確率代價（圖 15-17 顯示小網路上換成可分離版本反而掉了 5.7 個百分點），第 17 章詳述。(4)評估：客戶要求「漏檢率愈低愈好」等於要高召回率，因此用混淆矩陣、精確率與召回率，並依代價調整判定門檻，不能只看準確率（第 14.5.2 節、圖 14-13、14-15）;資料量小務必用交叉驗證並報標準差（第 14.5.3 節）。三個最可能失敗的地方：其一，資料偏差與捷徑——300 張若來自同一批次、同一光源，模型可能學到拍攝條件而非瑕疵（第 14.7.3 節、圖 14-20、15-20 的分布偏移），對策是跨批次、跨光源收集驗收資料;其二，類別不平衡——瑕疵品通常遠少於良品，準確率會被灌水（圖 14-13），對策是看召回率、設類別權重、調門檻;其三，樣本數不足以支撐可信的結論——測試集若只有數十張，標準誤可達數個百分點，任何小幅改善都不可信（第 14.5.3 節）。最後應建立上線後的持續抽檢與回饋機制，因為製程與光源會隨時間漂移。

參考資料與延伸閱讀

- Goodfellow, I., Bengio, Y., & Courville, A., Deep Learning, MIT Press, 2016.(深度學習權威教科書)
- Krizhevsky, A., Sutskever, I., & Hinton, G. E., ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.(AlexNet，本章叫獸開講之史料來源)
- LeCun, Y., Bengio, Y., & Hinton, G., Deep Learning, Nature, 2015.
- Simonyan, K., & Zisserman, A., Very Deep Convolutional Networks (VGG), ICLR, 2015;He, K., et al., Deep Residual Learning (ResNet), CVPR, 2016.
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.
- OpenCV、TensorFlow、Keras、PyTorch 官方文件與教學。
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 16 章 物件偵測 YOLO 實戰

本章學習目標

- 區分影像分類、物件偵測、語意分割與實例分割四種視覺任務。
- 說明兩階段與單階段偵測器的差異，理解 YOLO「只看一次」的核心思想。
- 運用 IoU、NMS、mAP 等關鍵概念評估與後處理物件偵測結果。
- 以 Ultralytics YOLO26 完成推論、訓練自訂資料集與模型驗證，並理解免 NMS 端到端推論的意義。
- 操作 YOLO 的分割、姿態估計與追蹤等延伸功能，並串接第 12 章視訊處理。
- 思辨電腦視覺技術的倫理責任，理解工程師在技術應用中的角色。

叫獸開講|YOLO 之父的告別：當創造者選擇放下

2016 年，華盛頓大學的研究生 Joseph Redmon 在頂級會議 CVPR 上發表了一篇論文，提出一個名叫 YOLO(You Only Look Once，「你只需看一次」)的物件偵測演算法。在那之前，主流的偵測方法要分兩階段：先產生大量候選框、再逐一分類，又慢又笨重。Redmon 的想法極為大膽：何不讓神經網路「看一次」整張影像，就直接同時輸出所有物件的位置與類別?這個一氣呵成的設計讓偵測速度快到能即時執行，論文一舉拿下 OpenCV 群眾票選獎，隔年的改良版 YOLO9000 更獲得 CVPR 最佳論文榮譽提名。YOLO 成了物件偵測的代名詞，從自駕車、工廠檢測到手機相機，無所不在。

Redmon 本人也是個充滿個人風格的研究者——他的個人網站與履歷以幽默不羈聞名，論文裡甚至夾雜著玩笑與自嘲。但在 2018 年的 YOLOv3 論文結尾，他寫下了一段不太像技術論文的文字，流露出對自己研究可能被誤用的不安。

2020 年 2 月，這份不安化為行動。當時學術界正在討論 NeurIPS 會議的新規定——要求論文作者加寫一段「廣泛影響」，說明研究可能帶來的正面與負面社會後果。有人質疑：研究者真的有必要因為「廣泛影響」而決定不投稿嗎?就在這場討論中，Redmon 現身分享了自己的親身經歷：他說，他已經停止電腦視覺研究了，因為他看見自己的研究成果帶來的影響——儘管熱愛這份工作，但軍事應用與隱私疑慮已經到了無法忽視的地步。這位親手創造出當代最流行偵測演算法的人，選擇了放下。

這件事在社群中引發了持久的辯論，而雙方都有道理。反對者說：技術本身沒有道德，刀子可以殺人也可以救人，問題出在使用者而非發明者;而且你不做，別人也會做，退出只是讓自己失去影響力。支持者則說：創造者對自己創造物的流向並非全然無責，當一項技術的下行風險遠大於上行效益時，選擇不推進它，本身就是一種負責任的表態。

這一章我們要學的，正是 Redmon 創造的 YOLO——它強大、快速、易用，是您在課程中要親手部署的主力工具。但在按下訓練指令之前，叫獸希望你先記住這個故事。作為工程師，我們手上的技術是中性的，但我們選擇把它用在哪裡、為誰服務、防範什麼樣的濫用，從來就不是中性的。這門課從第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤，一路鋪陳到這裡，談的都是同一件事：技術能力愈強大，對使用者處境的體貼與對後果的想像，就愈是專業素養不可分割的一部分。

思考題：「技術本身沒有道德，問題在使用者」與「創造者對技術的流向也有責任」——你比較認同哪一方?如果你設計的視覺系統可能被用於你不認同的用途，你會怎麼做?(想想：授權條款、使用限制、公開透明、或如 Redmon 一般選擇退出)

16.1 四種視覺任務

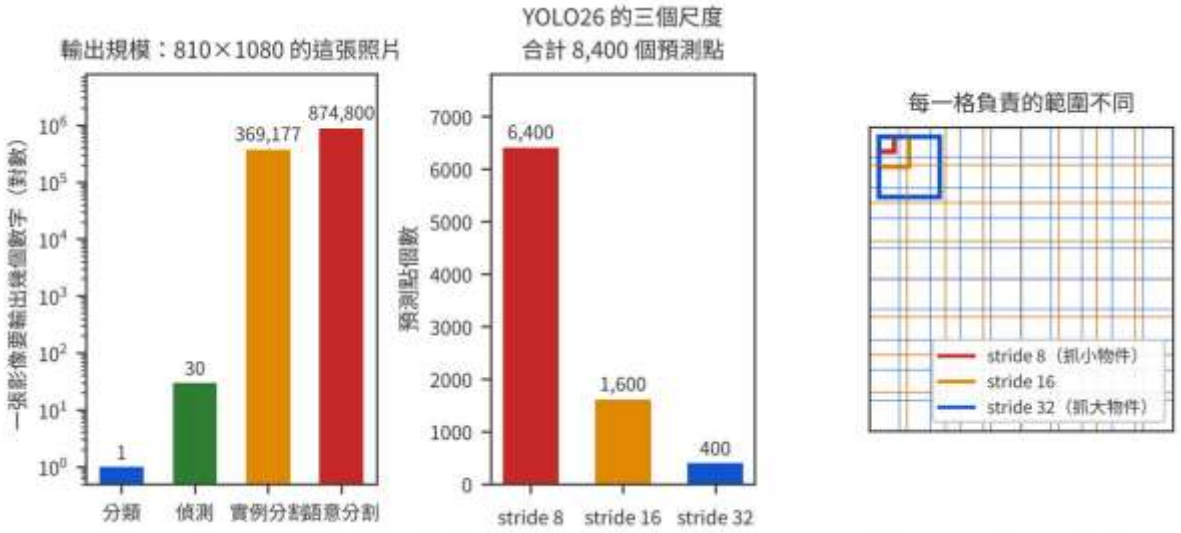
進入 YOLO 之前，必須先分清楚四種常被混淆的視覺任務。它們的差別在於「輸出的精細程度」，難度也依序遞增：

1. 影像分類(Classification)：輸出整張影像屬於哪一類——「這是一張貓的照片」。這是第 14、15 章的主題，只回答「是什麼」，不回答「在哪裡」。
2. 物件偵測(Object Detection)：輸出影像中每個物件的類別與位置(邊界框)——「這裡有一隻貓、那裡有兩個人」。它同時回答「是什麼」與「在哪裡」，是本章的主軸。
3. 語意分割(Semantic Segmentation)：為每一個像素標上類別——「這些像素是路面、那些是天空」。它給出精確的輪廓，但不區分同類的不同個體(三個人會被標成同一片「人」的區域)。這是第 9.6 節介紹過的 U-Net 所擅長的。
4. 實例分割(Instance Segmentation)：結合偵測與分割——不僅標出每個像素的類別，還區分同類的不同個體(三個人各自有獨立的輪廓與編號)。這是最精細、也最困難的任務。



圖 16-1　四種視覺任務：同一張照片，四種輸出

另有一個常見的延伸任務：姿態估計(Pose Estimation)，它偵測的不是物件的框，而是人體(或物體)的關鍵點——頭、肩、肘、腕、髖、膝、踝等骨架節點。它在動作分析、人機協作、以及第 12 章的跌倒偵測中極為重要。現代的 YOLO 已經把偵測、分割、姿態估計、追蹤整合進同一個工具中，這正是它成為業界主力的原因。



左圖把四種任務的輸出規模畫在對數軸上，一眼就看出 1 個數字 vs. 369,177 個數字的物理來源：分類只要吐 1 個數字，偵測要吐 5×6 = 30 個，實例分割要為 369,177 個前景像素各做一次判斷，語意分割更是整張 810×1080 = 874,800 個像素都要。輸出規模差了五個數量級，標註成本也差不多是這個比例——畫一個框幾秒鐘，描一個輪廓要好幾分鐘，這就是為什麼實務上能用框解決的事，很少有人去做分割。中圖與右圖則回答「YOLO 一次看完全圖，那它到底預測了幾次」。把 YOLO26n 的偵測頭直接攤開來看，640×640 的輸入會輸出一個 84×8,400 的張量：84 是每個預測點的內容（4 個框座標 + 80 個 COCO 類別分數），8,400 則是 6,400 + 1,600 + 400——三個解析度的特徵圖，格子邊長分別是 8、16、32 個像素。這就是 16.2.3 節說 v3 引入「多尺度預測」的實體：細的格子多、負責小東西，粗的格子大、負責大東西，而這正是第 11 章影像金字塔的同一個思想（下一張圖會量它救回多少）。所以「只看一次」不是只預測一次，而是 8,400 個預測在同一次前向傳播裡一起算完——真正省下來的是重複的特徵萃取。

圖 16-2　輸出的資訊量，與 YOLO 的 8400 個預測點

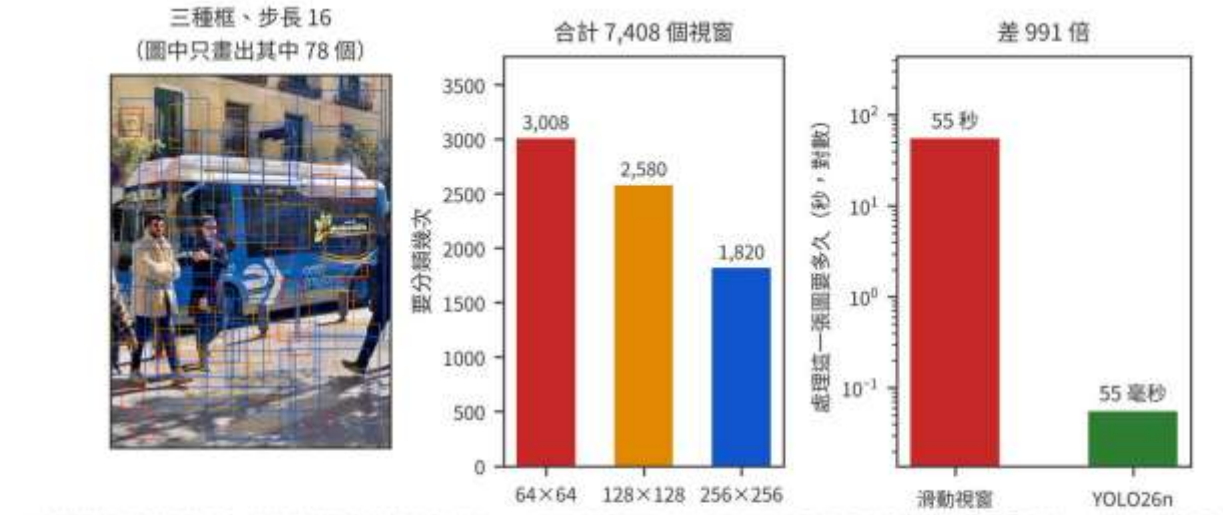
16.2 物件偵測的演進與 YOLO 的核心思想

16.2.1 從傳統方法到深度學習

回顧本書，我們其實已經走過了物件偵測的整部演進史。第 10 章的 Viola-Jones(2001)以 Haar 特徵加級聯分類器，是第一代能即時執行的偵測器，但只擅長正面人臉;第 13 章的特徵匹配加 RANSAC，能定位特定物件但難以泛化到「所有的貓」;而傳統做法的共同侷限是：依賴人工設計特徵(第 15 章已詳述這個瓶頸)。

深度學習時代的偵測器分兩大流派。兩階段偵測器(如 R-CNN 系列)先產生候選區域、再對每個區域分類與微調框，準確度高但速度慢——因為同一張影像要被網路處

理很多次。單階段偵測器(如 YOLO、SSD)則一次前向傳播就同時輸出所有物件的位置與類別，速度快得多，這正是 YOLO 名稱「You Only Look Once」的由來。



16.2 節說單階段偵測器「一次前向傳播就輸出所有結果」，而兩階段與更早的滑動視窗要「同一張影像跑很多次」。「很多次」換算成秒。做法是老派的滑動視窗：拿一個分類器，用 64×64、128×128、256×256 三種框、步長 16 個像素掃過這張 810×1080 的照片，一共要分類 3,008 + 2,580 + 1,820 = 7,408 次。每一次要多久?本圖實測 YOLO26n-cls 分類一塊 224×224 的小圖是 7.4 毫秒，乘起來就是 55 秒——一張圖要跑快一分鐘。同一台機器上，YOLO26n 看完整張圖只要 55 毫秒，快了約 991 倍。差距的來源不是網路比較小（兩者骨幹規模相近），而是滑動視窗把重疊區域的特徵重複算了上萬遍，YOLO 則整張圖只萃取一次特徵，再讓上千個預測點共用這份特徵。這就是「只看一次」真正的意思。要公平的話得補一句：真正的兩階段偵測器（Faster R-CNN 之後）早就不這樣做了——它們同樣只萃取一次特徵，只是多了一個「提候選區域」的步驟，因此慢的是倍數而不是數量級。這張圖量的是最樸素的做法，用來說明「重複計算」的代價有多可怕。

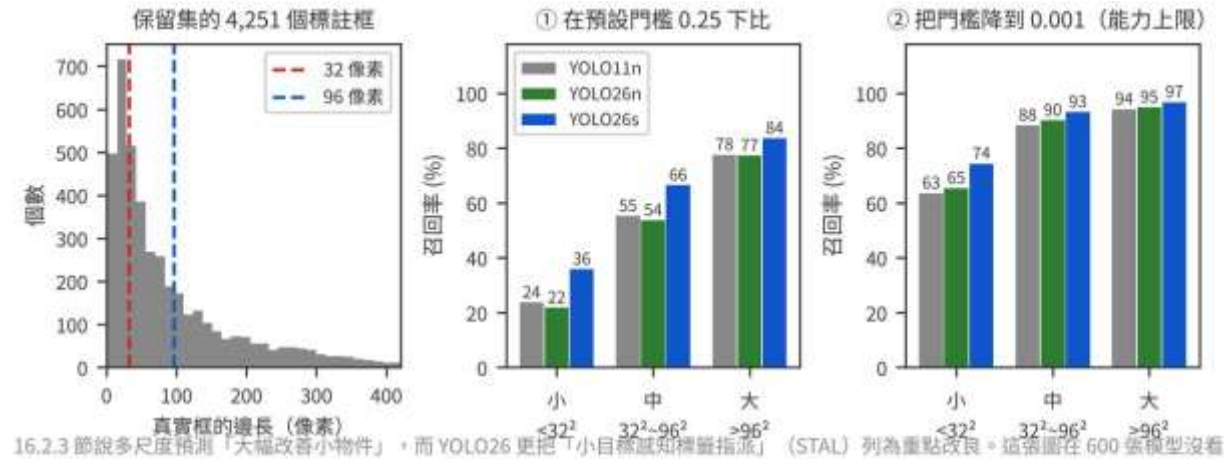
圖 16-3　滑動視窗的帳單：為什麼「只看一次」是關鍵

16.2.2 YOLO 的核心思想

YOLO 的巧思在於把「偵測」重新定義成一個「迴歸問題」：它把影像切成網格，讓網路對每個網格直接預測「這裡有沒有物件、框在哪裡、是什麼類別、信心多高」。整張影像只需通過網路一次，所有預測一氣呵成——這與兩階段方法「先提候選再逐一檢查」的思路截然不同，是一次漂亮的觀點翻轉(呼應第 9 章霍夫轉換的叫獸開講)。

這個設計還有一個額外好處：因為 YOLO 每次都看整張影像，它能利用全域的脈絡資訊(例如「這個區域周圍都是道路，所以這個模糊的物體比較可能是車而非船」)，背景誤判反而較少。而它早期的弱點是對小物件與密集群體較不擅長——這在後續版本中已大幅改善，但必須說清楚「改善」不等於「解決」。圖 16-4 以 YOLO26n 在 600 張模型沒看過的 COCO 影像上實測(信心門檻 0.25)：大物件的召回率有 77%，中

物件 54%，面積小於 32×32 的小物件只剩 22%。就算把信心門檻降到底、問它「究竟找不找得到」，小物件也只有 65%，而大物件已經 95%;換成大一號的 YOLO26s 也只把小物件的上限拉到 74%——換更大的模型救不了小物件。真正有效的是提高輸入解析度或把大圖切塊偵測(圖 16-12)，而這兩招又各有代價。請把這件事記在心裡：當你的應用要抓的是遠方的果實、小螺絲或路口外的行人，預設設定給你的召回率可能只有兩成。



16.2.3 節說多尺度預測「大幅改善小物件」，而 YOLO26 更把「小目標感知標籤指派」（STAL）列為重點改良。這張圖在 600 張模型沒看過的 COCO 影像、4,251 個標註框上量它，順便示範一個很容易踩到的比較陷阱。左圖先看資料：面積小於 32^2 的小物件有 1,331 個、中 1,487 個、大 1,433 個——小物件不是邊緣案例，是常態。中圖①是多數人會做的比較：固定信心門檻 0.25。結果 YOLO26n 的小物件召回率 22% 竟然比上一代的 YOLO11n（24%）還低，中、大物件也都略輸。難道新版反而退步了?右圖②揭穿了答案：把門檻降到 0.001，也就是問「它究竟找不找得到」，YOLO26n 在三個尺寸上全部反超——小 63% → 65%、中 88% → 90%、大 94% → 95%。差別在「信心的刻度」不是「能力」：實測 YOLO26n 命中框的信心中位數是 0.418，YOLO11n 是 0.457——新模型比較保守，同一個 0.25 砍掉了它更多正確的預測。這就是為什麼比較模型要看與門檻無關的 mAP（圖 16-7），而不是在某個固定門檻下比召回率;也是為什麼換模型之後，門檻必須重新調（圖 16-11）。至於小物件本身：即使放到能力上限，YOLO26s 的小物件召回也只有 74%，而在可用的門檻下整批漏掉的 2,059 個框中有 1,042 個是小物件。架構的改良方向是對的，但問題沒有被解決，只是變得比較可以忍受;真正有效的仍是提高輸入解析度（圖 16-12）或把大圖切塊偵測。

圖 16-4　小物件有多難，以及固定門檻比較的陷阱

16.2.3 YOLO 的世代演進

YOLO 從 2016 年至今經歷了快速的世代更迭：v1 到 v3 由 Redmon 本人主導(叫獸開講的主角),v3 引入多尺度預測(呼應第 11 章多解析度思想)大幅改善小物件偵測;此後由社群與不同團隊接棒，陸續推出 v4、v5、v6、v7、v8，再到 YOLO11，以及本章採用的 YOLO26。演進的主軸始終不變：更快、更準、更容易使用，並逐步把分割、姿態估計、追蹤等功能整合進同一套框架。但 YOLO26 這一代有一個特別值得注意的轉折：它第一次讓「不需要非極大值抑制」成為可選的標準做法，把一個從 v1 就跟著

YOLO 的後處理步驟變成可以拿掉的東西。下一小節專門說明這件事與它帶來的連鎖效應。

本章實作採用 Ultralytics 提供的 YOLO26。它把整套流程封裝得極為簡潔——載入模型、推論、訓練自訂資料集都只需幾行程式，同時提供 n(nano)、s、m、l、x 等不同大小的模型，讓使用者依算力需求選擇：nano 版可在第 17 章的邊緣裝置上即時執行，x 版則追求最高精度。這個「同一架構、多種尺寸」的設計，正呼應了第 15 章 MobileNet 所代表的「夠好且能部署」的務實路線。依官方在完整 COCO 驗證集上的數據，YOLO26 五種尺寸的 mAP@0.5:0.95 落在 40.9% 到 57.5% 之間;而本章所有實測數字則取自 COCO val2017 的 600 張隨機子集(刻意避開模型訓練用過的影像，理由見圖 16-18)，因此與官方數字不會完全一致，但兩者的相對關係一致。

16.2.4 YOLO26 帶來的四個改變

每一代 YOLO 都宣稱更快更準，但多半是同一個架構的調校。YOLO26 不太一樣——它動到了四個會改變你寫程式方式的地方。這一小節逐一說明，並指出各自在本章哪一張圖被量過。

其一，也是影響最大的一項：免 NMS 的端到端推論。從 v1 到 YOLO11，偵測器的輸出都是「很多個互相重疊的框」，必須再跑一次非極大值抑制(16.3.2 節)才能得到最終結果。YOLO26 同時訓練兩個預測頭：預設的「一對多」(one-to-many)頭維持舊行為，讓多個預測點各報一個框、再交給 NMS 去重;另一個「一對一」(one-to-one)頭則被訓練成每個物件只報一個框，因此完全不需要 NMS。使用方式只是在推論、驗證或匯出時加上 nms=False:

```
from ultralytics import YOLO
model = YOLO("yolo26n.pt")

# 預設:一對多預測頭,後面接 NMS
r1 = model.predict("street.jpg")

# 端到端:一對一預測頭,不跑 NMS,輸出固定 300 個框
r2 = model.predict("street.jpg", nms=False)
```

```
m  = model.val(data="coco.yaml", nms=False)
model.export(format="onnx", nms=False)
```

兩個頭的差別值得細看。一對多頭的原始輸出是一個 (N, 類別數 + 4, 8400) 的張量——8400 就是 16.2.2 節那三個尺度的預測點總數(圖 16-2);一對一頭的輸出則是 (N, 300, 6)，也就是最多 300 個框、每個框六個數字(四個座標 + 信心 + 類別)。後者不但不必跑 NMS，輸出形狀還是固定的，這對只接受靜態形狀的推論引擎特別友善。準確度要付多少代價?圖 16-10 在 600 張保留影像上實測，兩者的 mAP 幾乎相同;而圖 16-9 更顯示在物件彼此嚴重重疊的場景中，免 NMS 的頭反而明顯較好——因為它不會像 NMS 那樣把「兩個真的疊在一起的物件」誤判為重複而刪掉。官方另外報告在特定伺服器 CPU 上匯出 ONNX 後可快上約四成，但那是特定硬體與格式下的數字，請在自己的平台上重測。

其二，移除分布焦點損失(DFL)。從 YOLOv8 以來，框的邊界是用一組離散的機率分布來回歸的，預測出來的不是「邊界在哪」而是「邊界落在各個位置的機率」，再取期望值得到座標。這個設計精度好，但它讓預測頭多了一段只有框架自己懂的解碼邏輯，匯出到別的推論引擎時經常要手工重寫。YOLO26 把它拿掉，改回直接回歸座標、且不限制回歸範圍。對讀者的實際意義是：匯出變單純了，而這與免 NMS 是同一個方向——讓模型愈接近「一張計算圖就跑得完」，第 17 章的邊緣部署就愈輕鬆。

其三，訓練方法的兩項改良。漸進式損失(ProgLoss)讓訓練過程逐步把重心移到推論時真正會用的那個頭上，避免兩個頭互相拉扯;小目標感知標籤指派(STAL)則調整了「哪些預測點該為某個物件負責」的規則，提高小物件取得正樣本的機會。此外最佳化器改用 MuSGD，是 SGD 與 Muon 的混合體，靈感來自大型語言模型的訓練經驗。這些都是訓練端的改動，使用者不會直接碰到，但 STAL 的效果可以量：圖 16-4 在保留集上比較兩代模型，把信心門檻降到底之後，YOLO26n 的小物件召回率確實高於 YOLO11n。同一張圖也示範了一個重要的陷阱——若在固定的信心門檻下比較，

結論會反過來，因為新模型的信心刻度比較保守。換版本之後，門檻必須重新調。

其四，任務家族再度擴大。除了原有的偵測、實例分割、分類、姿態估計與有向邊界框，YOLO26 另外提供語意分割與單眼深度估計的預訓練權重，權重檔名一律是 yolo26{尺寸}-{任務}.pt 的形式(例如 yolo26n-seg.pt、yolo26s-pose.pt、yolo26n-obb.pt)。16.5.4 節會逐一說明這些新任務各自回答什麼問題。

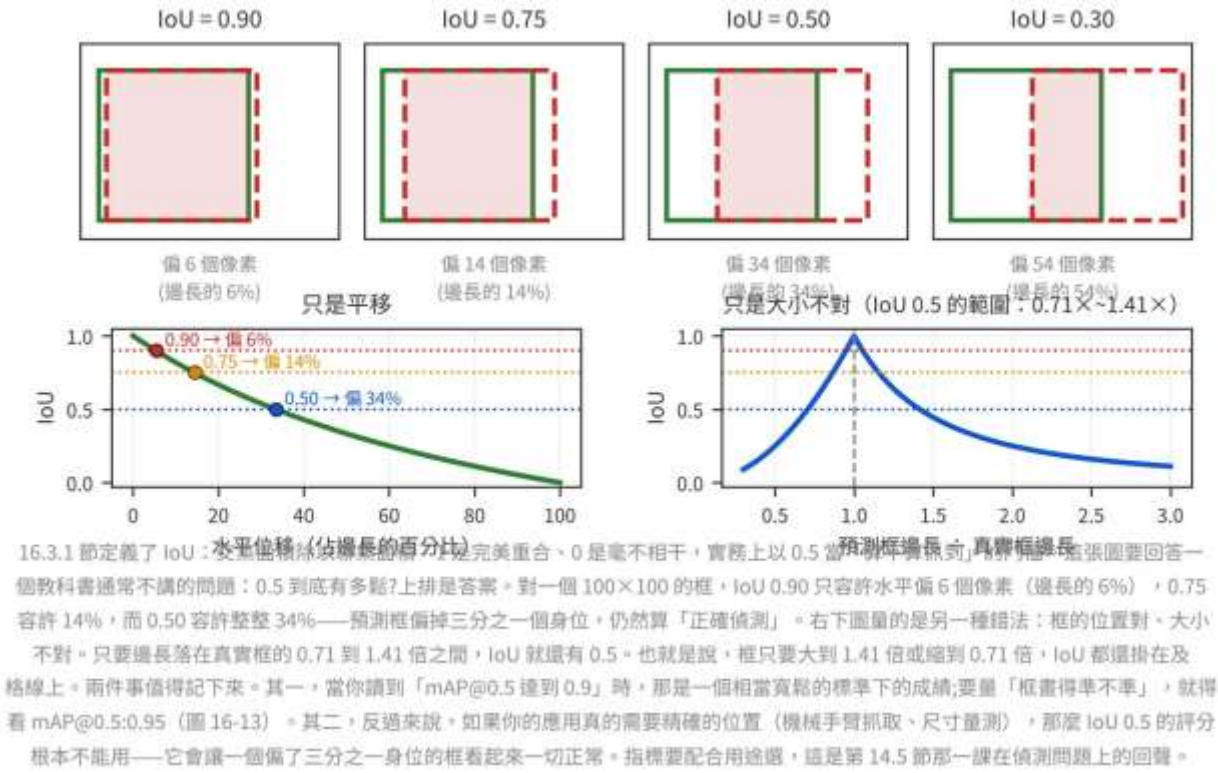
最後補一句使用上的提醒，它比上面任何一項都重要。本章其餘各節的觀念——IoU、NMS、mAP、信心門檻、標註品質、評估的誤差——沒有任何一項因為換版本而失效。免 NMS 拿掉的是一個後處理步驟，不是「框準不準」這個問題;STAL 改善的是小物件的召回率，不是「小物件很難」這個事實。新版本讓你少寫幾行程式、少調一個門檻，但判斷模型好不好的方法完全沒有改變。這也是為什麼本書把方法論放在工具之前。

16.3 物件偵測的關鍵概念

16.3.1 邊界框與 IoU

物件偵測的輸出是邊界框(bounding box)，通常以「左上角座標加寬高」或「中心座標加寬高」表示，並附帶類別與信心分數。要評估一個預測框準不準，需要一個量化的標準——這就是 IoU(Intersection over Union，交集聯集比)：把預測框與真實框的「重疊面積」除以「聯集面積」。IoU 為 1 代表完美重合，為 0 代表完全不重疊。實務上常以 IoU 大於某門檻(如 0.5)判定為「偵測正確」。這裡要補一句很少被講明的話：0.5 這個門檻其實相當寬鬆。對一個 100×100 的框，交集與聯集的算式給出 IoU = (100−d)/(100+d)，令它等於 0.5 得 d ≈ 33——也就是說預測框水平偏掉三分之一個身位，仍然算「正確偵測」;同理，框的邊長只要落在真實框的 0.71 到 1.41 倍之間，IoU 也還有 0.5(圖 16-5 的實測與這兩個算式完全吻合)。所以讀到「mAP@0.5 高達九成」時要意識到那是在問「有沒有找到東西」;要衡量「框畫得準不準」，必須看

mAP@0.5:0.95。若你的應用需要精確位置(機械手臂抓取、尺寸量測)，IoU 0.5 的評分根本不能用——它會讓一個偏了三分之一身位的框看起來一切正常。



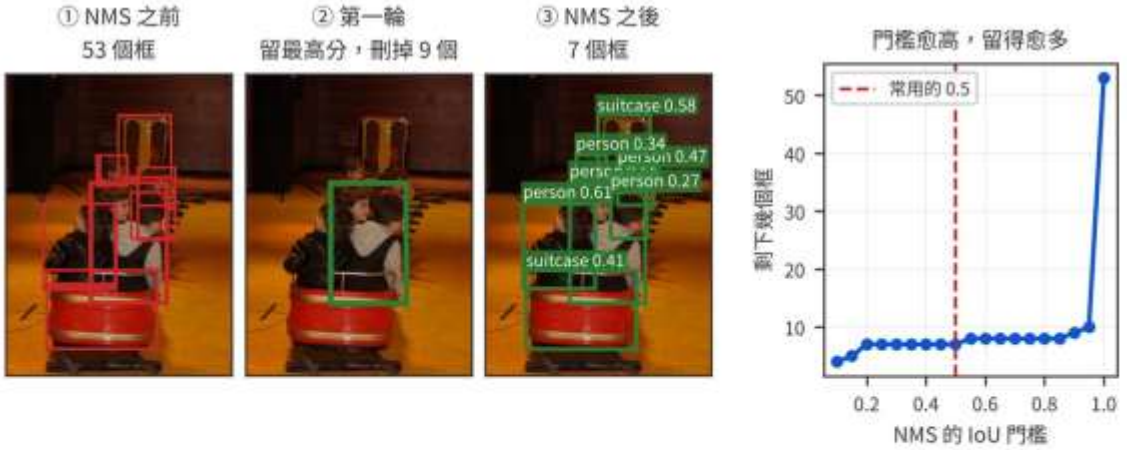
16.3.1 節定義了 IoU：交集面積除以聯集面積，1 是完美重合、0 是毫不相干，實務上以 0.5 當「對得夠準」的門檻。但這張圖要回答一個教科書通常不講的問題：0.5 到底有多鬆？上排是答案。對一個 100×100 的框，IoU 0.90 只容許水平偏 6 個像素（邊長的 6%），0.75 容許 14%，而 0.50 容許整整 34%——預測框偏掉三分之一個身位，仍然算「正確偵測」。右下圖量的是另一種錯法：框的位置對、大小不對。只要邊長落在真實框的 0.71 到 1.41 倍之間，IoU 就還有 0.5。也就是說，框只要大到 1.41 倍或縮到 0.71 倍，IoU 都還掛在及格線上。兩件事值得記下來。其一，當你讀到「mAP@0.5 達到 0.9」時，那是一個相當寬鬆的標準下的成績;要量「框畫得準不準」，就得看 mAP@0.5:0.95（圖 16-13）。其二，反過來說，如果你的應用真的需要精確的位置（機械手臂抓取、尺寸量測），那麼 IoU 0.5 的評分根本不能用——它會讓一個偏了三分之一身位的框看起來一切正常。指標要配合用途選，這是第 14.5 節那一課在偵測問題上的回聲。

圖 16-5　IoU 的幾何：偏一點點，分數掉多少

16.3.2 非極大值抑制(NMS)

偵測器對同一個物件，往往會輸出好幾個高度重疊的框。非極大值抑制(NMS)負責去蕪存菁：先取信心分數最高的框，再把與它 IoU 超過門檻的其他框都刪掉(視為重複)，接著對剩下的框重複這個程序，直到沒有框可處理。這個「保留局部最強、抑制鄰近重複」的思想，你應該覺得眼熟——它正是第 9 章 Canny 邊緣偵測第三步「非極大值抑制」的同名兄弟，兩者的精神完全一致。這種概念的跨章重現，是本書一再強調的：影像處理的核心思想並不多，但它們會以不同面貌反覆出現。不過 NMS 有一個無解的兩難必須先說：它分不清「同一個物件的重複框」與「兩個真的疊在一起的物件」。圖 16-9 在保留集裡找到的實例是兩個並排的同類物件——它們的真實框彼此 IoU 就有 0.66，只要 NMS 門檻設在這個值以下，其中一隻必定被當成重複而刪除，而且是在偵測器正確找到它之後才刪的。人群、貨架、果樹、停車場這些密集場景天

天在發生這件事。實測把門檻從 0.2 調到 0.95，召回率上升而精確率大幅下降，所以這個門檻不是「調到最好」，而是「依場景調」。也正因為如此，YOLO26 的免 NMS 預測頭(16.2.4 節)才特別有價值：它根本不需要這個門檻，而且在物件重疊嚴重的場景中召回率上限比一對多頭高出約六個百分點(圖 16-9)。還要注意一個容易混淆的地方：NMS 的 IoU 門檻與評分用的 IoU 門檻是兩回事，前者問「這兩個預測是不是在講同一個物件」，後者問「這個預測跟標準答案夠不夠接近」;它們剛好同名、也剛好常常都設 0.5，但調錯了哪一個，後果完全不同。



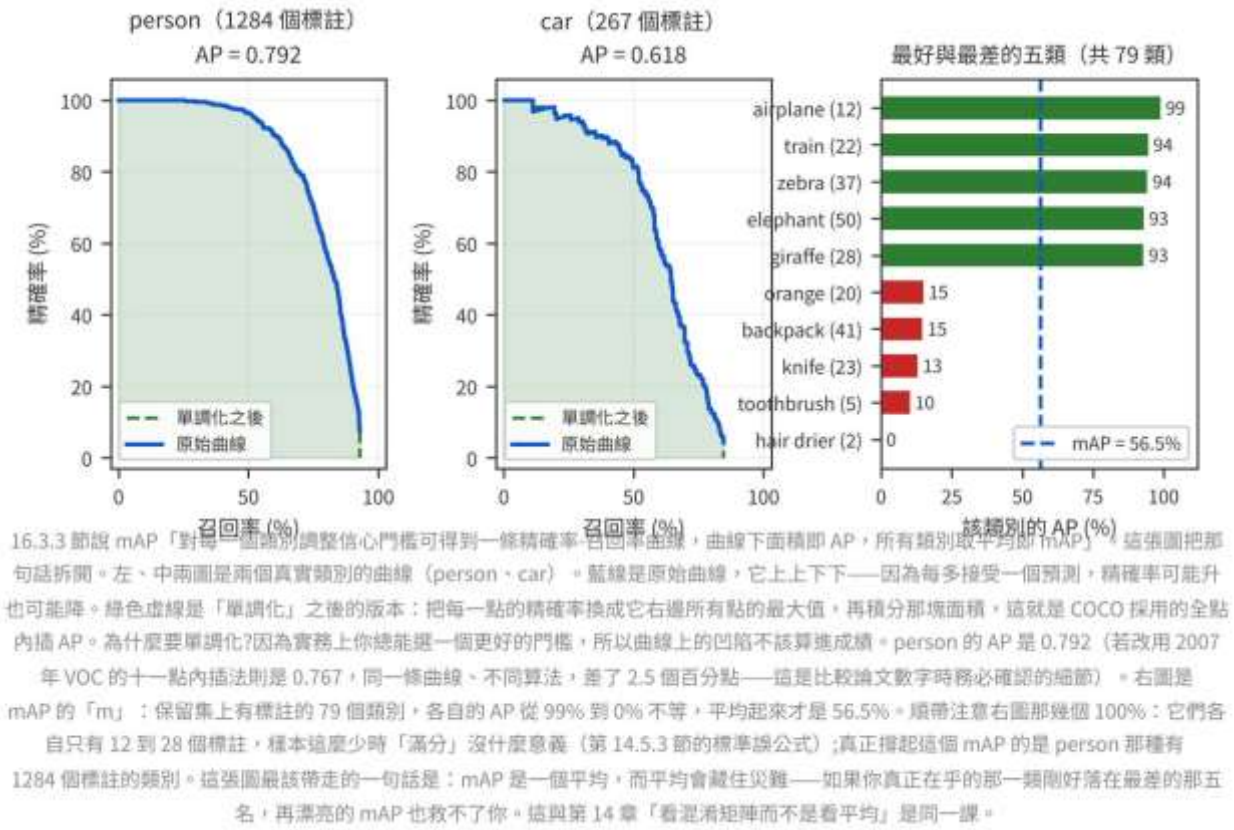
16.3.2 節描述了 NMS 的三個步驟：取信心最高的框、刪掉與它 IoU 超過門檻的其他框、對剩下的重複。這張圖把那三步畫在一張真實的 COCO 影像上。①是偵測器的原始輸出（把 NMS 關掉）：同一批物件上疊了 53 個框，這是因為相鄰的預測點都看見了同一個東西、各自報了一個框。②是第一輪：綠色是全場信心最高的框，灰色那 9 個與它的 IoU 超過 0.5，被視為重複而刪除。③是把這個程序做到底的結果：53 個框收斂成 7 個。右圖是門檻的影響——門檻從 0.1 調到 1.0，保留的框從 4 個增加到 53 個。這裡有個容易混淆的地方值得點破：NMS 的 IoU 門檻與評分用的 IoU 門檻（圖 16-8）是兩回事。前者問「這兩個預測是不是在講同一個物件」，後者問「這個預測跟標準答案夠不夠接近」。它們剛好都叫 IoU、也剛好常常都設 0.5，但調錯了哪一個，後果完全不同。另外請記住 16.3.2 節提到的同名之線：這裡的「非極大值抑制」與第 9 章 Canny 邊緣偵測的第三步是同一個思想——沿著某個方向只留下最強的那一個，把它的鄰居壓掉。

圖 16-6　NMS：一步一步看它刪掉什麼

16.3.3 評估指標：mAP

物件偵測的標準評估指標是 mAP(mean Average Precision，平均精度均值)。它建立在第 14 章的精確率與召回率之上：對每一個類別，調整信心門檻可得到一條「精確率-召回率曲線」，曲線下的面積即為該類別的 AP;把所有類別的 AP 取平均，就是 mAP。常見的寫法如 mAP@0.5(以 IoU 門檻 0.5 判定正確)或 mAP@0.5:0.95(在多個 IoU 門檻下取平均，標準更嚴格，是 COCO 資料集的主要指標)。

請留意這裡與第 14 章的連貫：同樣是「精確率 vs. 召回率」的權衡，同樣是「依應用代價選擇門檻」的工程判斷。一個工安監控系統可能寧可誤報也不能漏報(重召回)，一個自動分揀系統則可能相反(重精確)。指標的選擇從來不只是數學問題。



16.3.3 節說 mAP「對每一個類別調整信心門檻可得到一條精確率-召回率曲線，曲線下面積即 AP，所有類別取平均即 mAP」。這張圖把那句話拆開。左、中兩圖是兩個真實類別的曲線（person、car）。藍線是原始曲線，它上上下下——因為每多接受一個預測，精確率可能升也可能降。綠色虛線是「單調化」之後的版本：把每一點的精確率換成它右邊所有點的最大值，再積分那塊面積。這就是 COCO 採用的全點內插 AP。為什麼要單調化？因為實務上你總能選一個更好的門檻，所以曲線上的凹陷不該算進成績。person 的 AP 是 0.792（若改用 2007 年 VOC 的十一點內插法則是 0.767，同一條曲線、不同算法，差了 2.5 個百分點——這是比較論文數字時務必確認的細節）。右圖是 mAP 的「m」：保留集上有標註的 79 個類別，各自的 AP 從 99% 到 0% 不等，平均起來才是 56.5%。順帶注意右圖那幾個 100%：它們各自只有 12 到 28 個標註，樣本這麼少時「滿分」沒什麼意義（第 14.5.3 節的標準誤公式）；真正撐起這個 mAP 的是 person 那種有 1284 個標註的類別。這張圖最該帶走的一句話是：mAP 是一個平均，而平均會藏住災難——如果你真正在乎的那一類剛好落在最差的那五名，再漂亮的 mAP 也救不了你。這與第 14 章「看混淆矩陣而不是看平均」是同一課。

圖 16-7　AP 就是那塊面積

16.3.4 推導：IoU、NMS 與 mAP 的三道算式

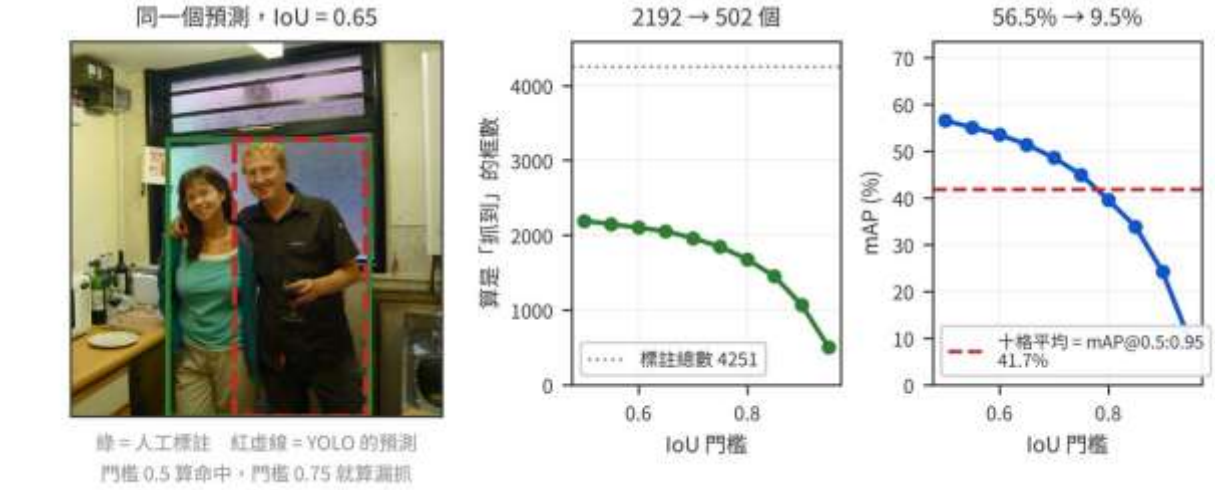
16.3 節用文字介紹了三個關鍵概念，它們其實都只是幾行算術。這一小節把它們寫成可以照著實作的式子——本章所有圖裡的數字，就是用這幾行算出來的，而且與 Ultralytics 官方工具的結果只差一個百分點左右(差異來自群體標註與面積過濾等細節規則)。親手算過一遍，你對那些分數的信任才會建立在理解之上，而不是建立在別人的函式庫之上。

先看 IoU。兩個框各用左上與右下兩點表示為 (x1, y1, x2, y2)，交集矩形的左上角取兩者較大的，右下角取兩者較小的：

```
    ix1 = max(a.x1, b.x1)       iy1 = max(a.y1, b.y1)
```

```
ix2 = min(a.x2, b.x2)       iy2 = min(a.y2, b.y2)
交集 = max(0, ix2 − ix1) × max(0, iy2 − iy1)
聯集 = 面積(a) + 面積(b) − 交集
IoU  = 交集 / 聯集
```

那兩個 max(0, …) 是整段程式唯一需要小心的地方：兩個框完全不相交時，ix2 − ix1 會是負數，少了這道截斷就會算出一個莫名其妙的正面積。另外注意聯集要減掉交集，否則重疊的部分被算了兩次。從這個式子可以立刻讀出兩件圖 16-5 量到的事：其一，IoU 對平移的敏感度與框的大小無關，只與「偏移佔邊長的比例」有關;其二，若兩個同中心的正方形邊長比是 s(s > 1)，交集就是小的那個、聯集是大的那個，於是 IoU = 1/s²，令它等於 0.5 得 s = √2 ≈ 1.41——這正是圖 16-5 實測到的上界 1.41 倍，而縮小的那一側同理是 1/√2 ≈ 0.71。



上一張圖量了 IoU 的幾何，這一張量它對成績的影響。左圖是一個真實的例子：YOLO 對這個物件給出的框與人工標註的 IoU 是 0.65，看起來相當準——但在 IoU 0.5 的門檻下它是命中，在 0.75 的門檻下它就變成一次漏抓加一次誤報。同一個預測、同一張圖，換個門檻就換個身分。中圖把 600 張圖的 4251 個標註框全部算一遍（信心門檻固定 0.25）：門檻 0.5 時有 2192 個算抓到，門檻 0.95 時只剩 502 個，掉到 23%。右圖是 mAP 的同一條線：56.5% 一路降到 9.5%。請注意這條曲線在 0.5 到 0.7 之間還算平緩，過了 0.8 就陡降——那是因為要把框畫到 IoU 0.9 以上，等於要求邊界誤差小於邊長的 5%（圖 16-5），而人工標註本身都未必有這個精度。COCO 的主指標 mAP@0.5:0.95 就是這十格的平均（紅色虛線），本圖實測為 41.7%，遠低於大家習慣引用的 mAP@0.5 的 56.5%。兩個數字都沒有錯，但它們回答的是不同的問題：mAP@0.5 問「有沒有找到東西」，mAP@0.5:0.95 問「框畫得準不準」。報告成績時務必寫清楚是哪一個，否則比較毫無意義。

圖 16-8　IoU 門檻決定什麼算「抓到」

再看 NMS。它是一個貪婪演算法，只有四行：

```
把所有框依信心分數由高到低排序
while 還有框:
    取出分數最高的框，放進保留清單
    把剩下的框中，與它 IoU > 門檻 的全部刪除
```

這裡有一個常被忽略的細節：實務上 NMS 應該「逐類別」進行。一個人站在腳踏車前面，兩個框可能高度重疊，但它們講的是不同的東西，不該互相抑制。Ultralytics 的做法是先把每個框的座標按類別編號平移一段很大的距離(agnostic=False 時)，讓不同類別的框在幾何上永遠不可能相交，再跑一次全域的 NMS——一個很漂亮的小技巧。另外值得估一下成本：最壞情況下每一輪要與剩下的框各算一次 IoU，總共 $O(n^2)$ 次;n 是 NMS 前的框數，而圖 16-6 的例子中 n 是 55，若把信心門檻設成 0.001(算 mAP 時必須這麼做)，n 可能上千，這時 NMS 本身就會變成瓶頸。這也是實務上總會先用信心門檻砍一刀、再限制每張影像最多保留幾個框(Ultralytics 預設 300)的原因。

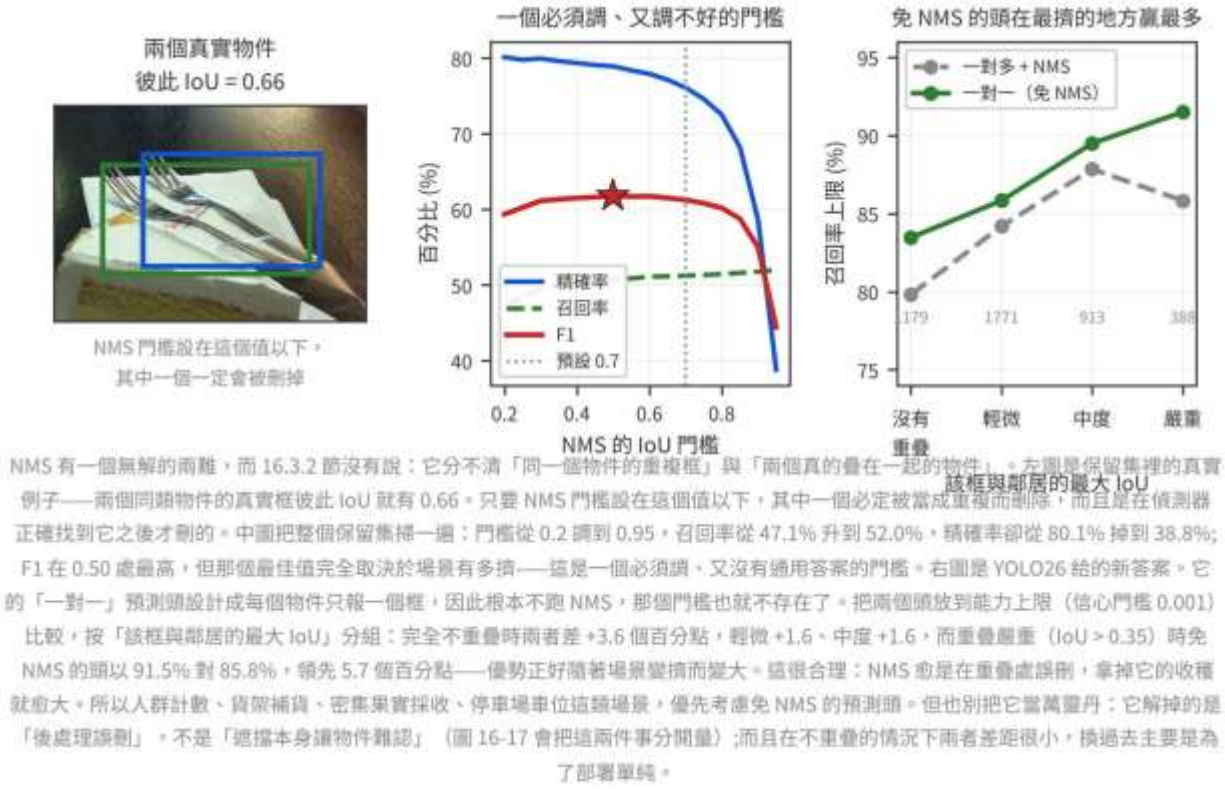


圖 16-9　NMS 的兩難，以及 YOLO26 怎麼把它拿掉

最後是 AP。它分成三步：配對、累積、積分。配對的規則是：把預測依分數由高到低處理，每個預測找 IoU 最大且尚未被認領的同類真實框;IoU 超過門檻才算命中(TP)，否則算誤報(FP)，而且每個真實框最多被認領一次——這條「最多一次」的規則就是為什麼重複的框會變成誤報，也是 NMS 必須存在的理由。接著沿著分數由高

到低累積：

```
召回率 R(k) = 前 k 個預測中的 TP 數 / 真實框總數
精確率 P(k) = 前 k 個預測中的 TP 數 / k
AP = ∫ P dR   (曲線下面積)
```

直接積分會遇到一個問題：精確率曲線是鋸齒狀的，每多接受一個預測，精確率可能升也可能降(圖 16-7 的藍線)。標準做法是先讓它單調不升——把每一點的精確率換成它右邊所有點的最大值：

```
P*(R) = max{ P(R') : R' ≥ R }
AP = Σ (R(i+1) − R(i)) × P*(R(i+1))
```

這個「單調化」的理由是：既然你總能選一個更好的門檻，那麼曲線上的凹陷就不該算進成績。歷史上還有另一種算法——2007 年 PASCAL VOC 的十一點內插法，只在召回率 0、0.1、…、1.0 這十一個點取精確率再平均。兩者用同一條曲線可以差好幾個百分點(圖 16-7 實測 person 類別：全點 0.782、十一點 0.758)，所以比較不同論文的數字時，先確認它們用的是同一種算法。

把 AP 對所有類別取平均得到 mAP;再把 IoU 門檻從 0.50 掃到 0.95(每 0.05 一格)、把十個 mAP 平均起來，就是 COCO 的主指標 mAP@0.5:0.95。圖 16-8 畫出了那十個數字，而它們的差距之大(從 56.5% 到 9.5%)正好說明：沒有指明門檻的 mAP，是一個沒有意義的數字。

表 16-1 物件偵測的關鍵概念

16.4 YOLO 實戰：推論、訓練與驗證

16.4.1 環境安裝與快速推論

Ultralytics 把 YOLO 的使用門檻降到極低。安裝一個套件、載入預訓練模型，幾行程式就能偵測 COCO 資料集(第 14 章)的八十個類別——人、車、貓、狗、瓶子、椅子等日常物件。

程式 16-1 安裝 Ultralytics YOLO

```
# 安裝(建議在虛擬環境中)
pip install ultralytics
```

程式 16-2 YOLO 影像推論

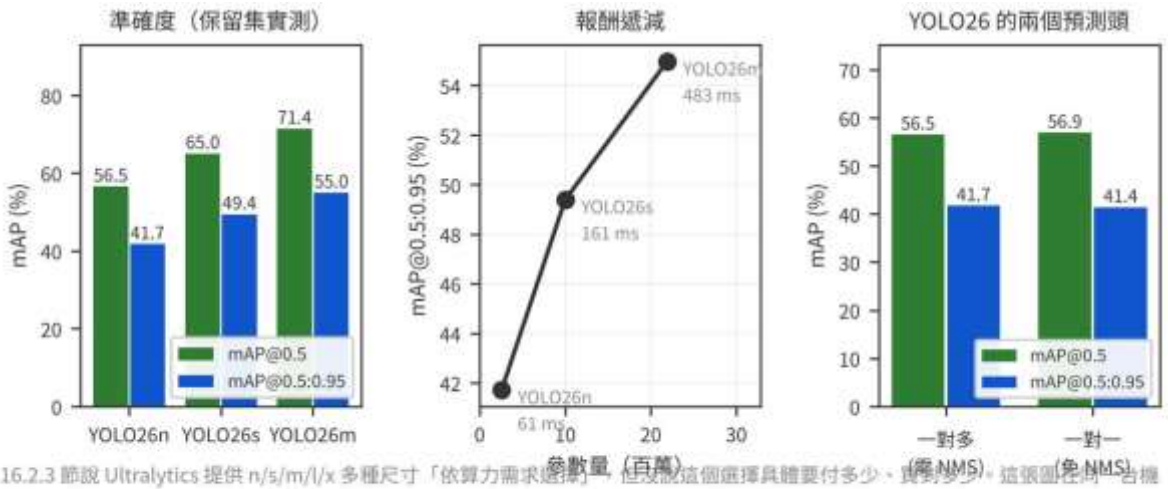
```
from ultralytics import YOLO
import cv2

# 載入預訓練模型(n/s/m/l/x 依算力需求選擇)
model = YOLO("yolo26n.pt")        # nano:最輕量,適合邊緣裝置

# --- 單張影像推論 ---
results = model("street.jpg", conf=0.25, iou=0.45)
#   YOLO26 另有免 NMS 的端到端模式(16.2.4 節):
#   results = model("street.jpg", conf=0.25, nms=False)
#   conf:信心門檻(調高減少誤報、調低減少漏檢)
#   iou :NMS 的 IoU 門檻(用 nms=False 時此參數不生效)

for r in results:
    for box in r.boxes:
        cls  = int(box.cls[0])                  # 類別編號
        name = model.names[cls]                 # 類別名稱
        conf = float(box.conf[0])               # 信心分數
        x1, y1, x2, y2 = box.xyxy[0].tolist()   # 邊界框座標
        print(f"{name}: {conf:.2f} @ ({x1:.0f},{y1:.0f})-({x2:.0f},{y2:.0f})")

    annotated = r.plot()          # 畫出所有偵測框
    cv2.imwrite("detected.jpg", annotated)
```



16.2.3 節說 Ultralytics 提供 n/s/m/l/x 多種尺寸「依算力需求選擇」，但沒說這個選擇具體要付多少。這張圖把兩台機器上量給你看，並且多問一個 YOLO26 才有的問題：要不要 NMS。尺寸方面，從 n 到 m，mAP@0.5 從 56.5% 升到 71.4%，mAP@0.5:0.95 從 41.7% 升到 55.0%，多了 13.3 個百分點;代價是參數從 2.6 百萬變成 21.9 百萬、單張耗時從 61 毫秒變成 483 毫秒（7.9 倍），在這台 CPU 上等於從 16 fps 掉到 2.1 fps。中圖畫出這條報酬遞減曲線：n 到 s 這一步最划算，s 到 m 就明顯平了。右圖是 YOLO26 的新選項。它同時訓練兩個預測頭：預設的「一對多」頭讓多個預測點各報一個框，因此必須靠 NMS 去重（圖 16-6）;而「一對一」頭（推論時加上 nms=False）設計成每個物件只報一個框，完全不需要 NMS，輸出固定是 300 個框。實測兩者的準確度幾乎相同：mAP@0.5 是 56.5% 對 56.9%、mAP@0.5:0.95 是 41.7% 對 41.4%——也就是說，免 NMS 這件事幾乎是白送的。要誠實說明的是：本圖在 CPU 上量不出明顯的速度差，因為 PyTorch 後處理只佔總時間的百分之一上下;Ultralytics 官方報告的「CPU ONNX 快上四成」是在匯出成 ONNX 並在特定伺服器 CPU 上量的，條件與這裡不同，不宜直接套用。免 NMS 真正的價值在別處：整個模型可以匯出成單一張計算圖，不必在 TensorRT、LiteRT 或瀏覽器端另外實作一段後處理;輸出形狀固定，對只吃靜態形狀的推論引擎特別友善;而且少了一個必須依場景調的 NMS 門檻（圖 16-9 會量這件事值多少）。這三點對第 17 章的邊緣部署都是決定性的差別。該怎麼選?規則很簡單：先問應用需要幾 fps，那是一條硬線;在這條線以內挑能跑的最大模型，並且只要部署端不方便寫後處理，就優先用免 NMS 的那個頭。

圖 16-10　模型尺寸與兩個預測頭：要不要 NMS

16.4.2 訓練自訂資料集

真正的價值在於訓練自己的偵測器——偵測您關心的物件(工件瑕疵、農產品、特定零件)。流程分三步：

1. 準備資料：蒐集影像並標註邊界框(常用工具如 LabelImg、Roboflow、CVAT)。標註格式為 YOLO 格式：每張影像對應一個 txt 檔，每行記錄「類別編號 中心 x 中心 y 寬 高」(座標皆正規化到 0 至 1)。務必依第 14 章原則切分訓練/驗證/測試集。
2. 撰寫設定檔：一個 YAML 檔，指定資料集路徑與類別名稱。
3. 訓練與驗證：呼叫 train() 開始訓練(內建資料增強、及早停止等第 14、15 章的機制)，再用 val() 在驗證集上計算 mAP。

程式 16-3 資料集設定檔

```
# data.yaml — 資料集設定檔
path: /home/user/mydata
```

```
train: images/train
val:   images/val
names:
  0: good_part
  1: scratch
  2: crack
```

程式 16-4 訓練自訂 YOLO 模型

```
from ultralytics import YOLO

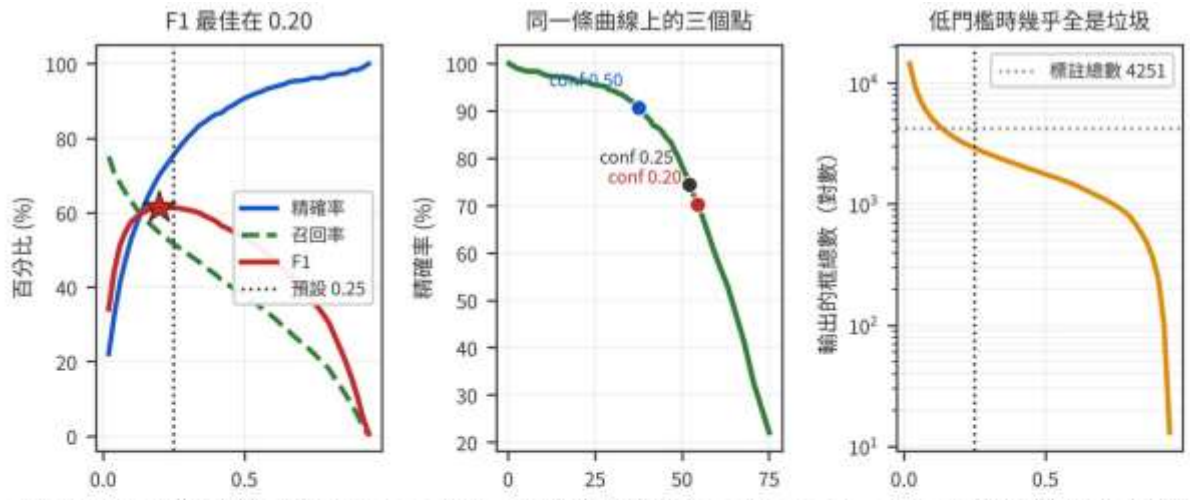
# 從預訓練權重開始訓練 = 遷移學習(第 15 章)
model = YOLO("yolo26n.pt")

results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20,          # 及早停止(第 14 章對抗過度擬合)
    degrees=10,           # 資料增強:旋轉(第 4 章)
    hsv_v=0.4,            # 資料增強:亮度(第 5、8 章)
    fliplr=0.5,           # 資料增強:水平翻轉(第 4 章)
    project="runs", name="defect_v1")

# 驗證:計算 mAP
metrics = model.val()
print("mAP@0.5      :", metrics.box.map50)
print("mAP@0.5:0.95 :", metrics.box.map)

# 匯出為部署格式(第 17 章邊緣部署)
model.export(format="onnx", nms=False)   # 免 NMS:匯出成單一張計算圖
#   也可用 engine(TensorRT)、tflite(LiteRT)、openvino、coreml
```

請注意程式 16-4 中的訓練參數：degrees、hsv_v、fliplr 全是第 4、5、8 章的影像處理技術，在此作為資料增強使用;patience 是第 14 章的及早停止;而「從預訓練權重開始」正是第 15 章的遷移學習。這再次印證全書的主軸——最先進的工具，骨子裡仍是前面各章觀念的組合。



程式 16-2 把 conf 一路掃過，讓你看見「調高減少誤報、調低減少漏報」這句話畫出來，順便說明那條 F1 曲線為什麼長那樣。左圖是三條線隨門檻的變化（600 張保留影像、4251 個標註框、IoU 0.5 判定）：門檻 0.02 時召回率 75.1%、精確率只有 22.1%；門檻 0.94 時精確率升到 100.0%、召回率只剩 0.3%。F1 的最佳點落在 0.20，而預設的 0.25 給出的是精確率 74.4%、召回率 52.2%、F1 61.3%。中圖把同一件事畫成精確率-召回率曲線：調 conf 不會讓模型變好，只是讓你沿著這條曲線移動——這是第 14.5 節那一課在偵測上的翻版。右圖說明為什麼低門檻不能用：門檻 0.02 時輸出 14,469 個框，而真實物件只有 4251 個，也就是說超過 71% 是誤報。那麼 0.25 是怎麼來的？在這份資料上它其實相當接近 F1 的最佳點（0.20，差 0.1 個百分點），所以它不是隨便訂的——但它最佳化的是 F1，也就是「把精確率與召回率看得一樣重」。你的應用多半不是這樣。工安監控寧可誤報也不能漏報，門檻該調低；自動分揀誤判就是整批重工，門檻該調高。還有一個陷阱要記住：算 mAP 時 conf 必須設到極低（本章用 0.001），因為 AP 要涵蓋整條曲線；若先用 0.25 砍掉低分預測，曲線的右端就永遠補不回來，mAP 會被低估。

圖 16-11　信心門檻：那個被當成預設值的 0.25

16.4.3 推導：該標幾張、該訓練多久、該相信幾分

16.4.2 節把訓練自訂偵測器寫成三個步驟，但學生最常問的三個問題它都沒回答：要標幾張才夠？要訓練幾個 epoch？最後測出來的 mAP 可以相信幾分？這三個問題都沒有唯一答案，但都有可以估算的下界，而估算的工具在第 14 章都給過了。

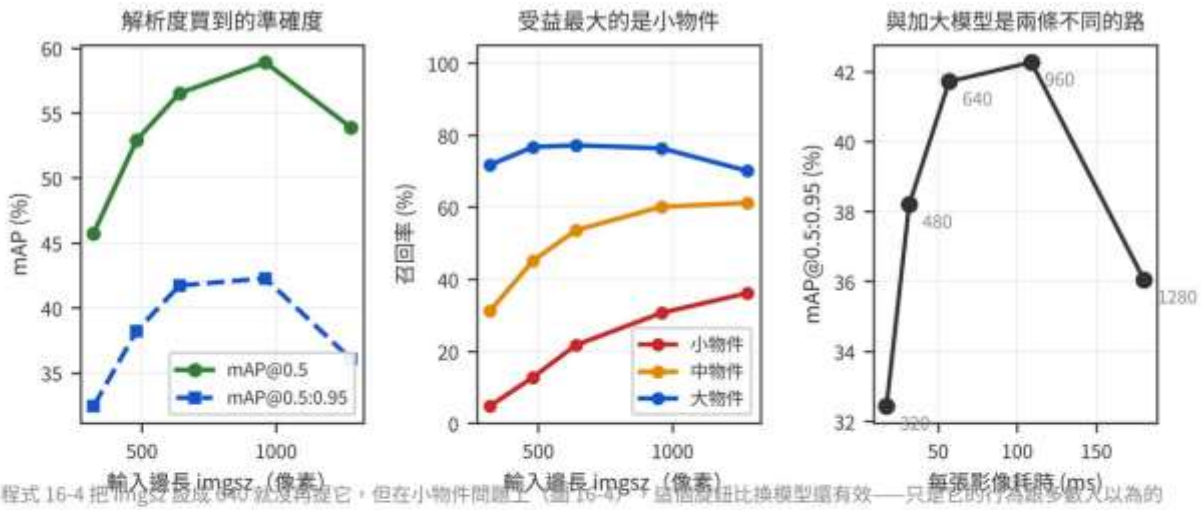
先談最後一個，因為它決定前兩個。第 14.5.3 節推過：在 n 筆測試樣本上量到的比例 a，其標準誤約為 $\sqrt{a(1-a)/n}$。偵測問題的單位不是影像而是「物件」，所以 n 要用標註框數來算。假設你的驗收集有 200 個標註框、量到的召回率是 0.90，那麼標準誤是 $\sqrt{0.9 \times 0.1 / 200} \approx 0.021$，95% 區間大約是 90% ± 4 個百分點。換句話說，用 200 個框去驗收，你分不出 88% 與 92% 的差別。

mAP 的情況更糟，因為它是「先逐類別算 AP、再平均」。一個只有三個標註框的類別，它的 AP 只有少數幾種可能值，平均進去就是在擲骰子——圖 16-18 實測保留集的 79 個類別中，標註數不到四個的那幾類，AP 幾乎都是 0% 或 100%。同一張

圖用重抽樣量出：只用其中 128 張時，mAP@0.5 的標準差有 2.8 個百分點，用滿 600 張才降到 1.1。可操作的規矩是：你真正在乎的每一個類別，驗收集至少要有幾十個標註框;少於十個的類別，不要報它的 AP，改報「抓到幾個、漏掉幾個」這種可以直接數的量。

回到第一個問題：訓練集要標幾張?這要看任務有多難、以及你從哪裡開始。程式 16-4 的第一行就是答案的一半——從 yolo26n.pt 這個 COCO 預訓練權重開始，就是第 15 章的遷移學習。圖 15-18 量過它的規律：資料愈少，借來的特徵愈划算;資料多到足以自己學，凍結反而變成天花板。偵測任務的經驗起點是「每類一百到數百張、每張數個物件」，但真正該做的不是背這個數字，而是畫一條學習曲線：用 25%、50%、100% 的資料各訓練一次，看驗證 mAP 有沒有還在上升。還在上升就代表加資料有用，已經平掉就代表瓶頸在別處(多半是標註品質，見圖 16-13，或是類別本身太難)。

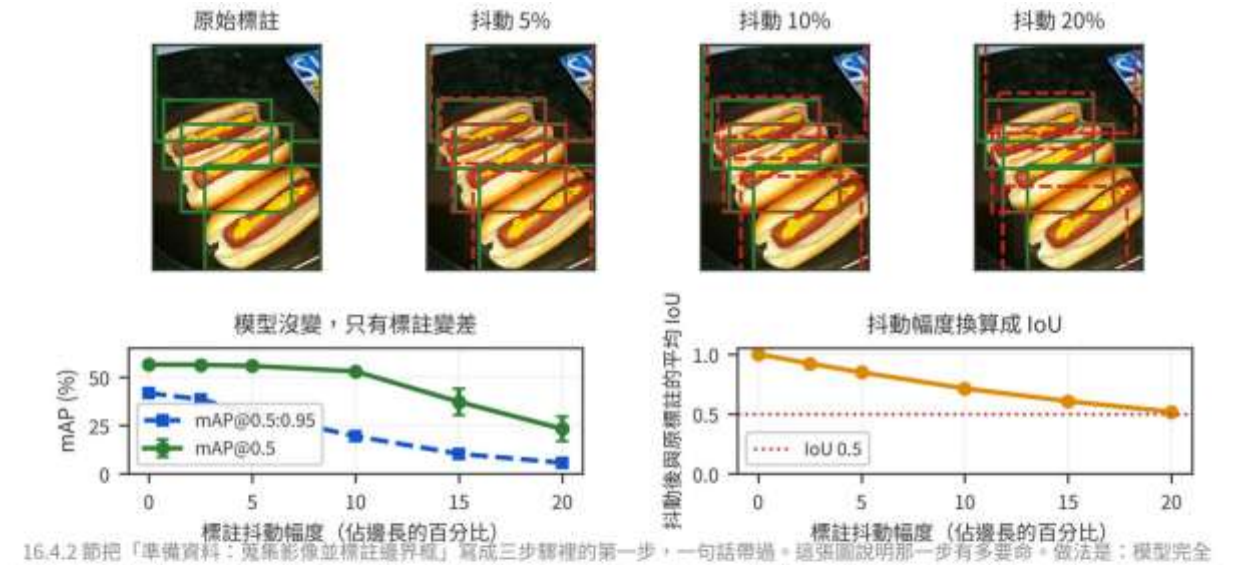
第二個問題：訓練幾個 epoch?程式 16-4 寫 epochs=100、patience=20，這兩個數字要一起讀：真正決定何時停的是 patience，epochs 只是上限。它的意思是「驗證指標連續 20 個 epoch 沒有進步就停」，正是第 14.6.3 節的及早停止。所以 epochs 設大一點並不浪費(大不了提早停)，而 patience 設太小才危險——訓練曲線本來就會上下抖動，patience 太小會在一次正常的抖動中誤判為「不再進步」。



程式 16-4 把 imgsz 設成 640 就沒再提它，但在小物件問題上（圖 16-4），這個旋鈕比換模型還有效——只是它的行為跟多數人以為的不一樣，所以這張圖量了五種解析度。先看中圖，那是符合直覺的部分：輸入從 320 加到 1280，小物件的召回率從 5% 一路升到 36%，翻了 7.5 倍。道理很單純：一個在原圖上只有 20 像素的物件，縮到 320 之後可能只剩幾個像素，連 stride 8 的格子都裝不下它（圖 16-2）；把輸入放大，它才重新變成「看得見」的東西。但同一張圖也說了反直覺的那一半：大物件的召回率在 640 時最高（77%），到 1280 反而掉到 70%。於是左圖的總成績並不是一路向上，而是一條倒 U：mAP@0.5 在 imgsz = 960 達到最高的 58.9%，320 時只有 45.7%，1280 時又掉回 53.9%。原因不難想：YOLO26 的預訓練權重是在 640 上訓出來的，把輸入放大一倍，所有物件在網路眼中都大了一倍，大物件因此被推出了它熟悉的尺度範圍——這就是第 15 章說的分布偏移，只是發生在尺度這個維度上。右圖是代價：耗時從 17 毫秒增加到 180 毫秒，大致隨像素數成正比，所以 1280 是「花了 3.2 倍的時間換到更差的總成績」。三個可以直接拿去用的結論。其一，調 imgsz 與換模型（圖 16-10）是兩條不同的路：加大模型是讓網路更會認，加大輸入是讓小東西還在。其二，推論時的 imgsz 不要離訓練時的值太遠;真的需要高解析度，正確做法是用高解析度「重新訓練」，而不是只在推論時放大。其三，若漏抓集中在小物件而又不能重訓，把大圖切塊、各自以 640 偵測再合併，通常比直接放大整張圖有效。

圖 16-12　解析度：imgsz 這個被忽略的旋鈕

最後補一個偵測特有、而且非常容易踩的陷阱：切分不能只看影像。同一條產線、同一批果實、同一段影片的相鄰影格，彼此極度相似;若把它們隨機打散後再切訓練與驗證集，驗證集裡就會出現「訓練集那張照片的孿生兄弟」，分數會虛高好幾個百分點。這是第 14 章資料洩漏的偵測版本。正確做法是按「來源」切——依拍攝日期、依批次、依影片檔，讓同一個來源的影像整組落在同一邊(scikit-learn 的 GroupKFold 就是為此而生)。這一條比任何調參都重要：切分錯了，後面量到的每一個數字都不必看了。



16.4.2 節把「準備資料：蒐集影像並標註邊界框」寫成三步驟裡的第一步，一句話帶過。這張圖說明那一步有多要命。做法是：模型完全不動，只把人工標註隨機抖一抖（平移與縮放，幅度以邊長的百分比計），再重算一次成績。上排是抖動長什麼樣——抖動 5% 時肉眼幾乎看不出差別。但成績會：抖 5% 時 mAP@0.5 從 56.5% 掉到 55.8%（少 0.7 分），mAP@0.5:0.95 從 41.7% 掉到 31.4%（少 10.3 分）；抖 20% 時更掉到 23.3% 與 5.7%。請特別注意兩條線的斜率差：mAP@0.5:0.95 掉得明顯更快，因為它本來就在量「框準不準」，而標註歪了，準的定義就歪了。右圖把抖動換算成 IoU：抖 10% 時，抖動後的框與原標註平均 IoU 還有 0.71——在 0.5 的門檻下它們「是同一個框」，成績卻已經少了好幾分。三個實務結論。其一，標註品質是成績的天花板，換模型換不回來。其二，比較兩個模型時，兩邊必須用同一份標註，否則你比的是標註員而不是模型。其三，驗收標準要跟著標註精度走——如果你的標註本身只有 IoU 0.9 的一致性，那就不該拿 mAP@0.95 當 KPI。這是第 14 章「誠實地知道模型有多好」在偵測上的具體版本。

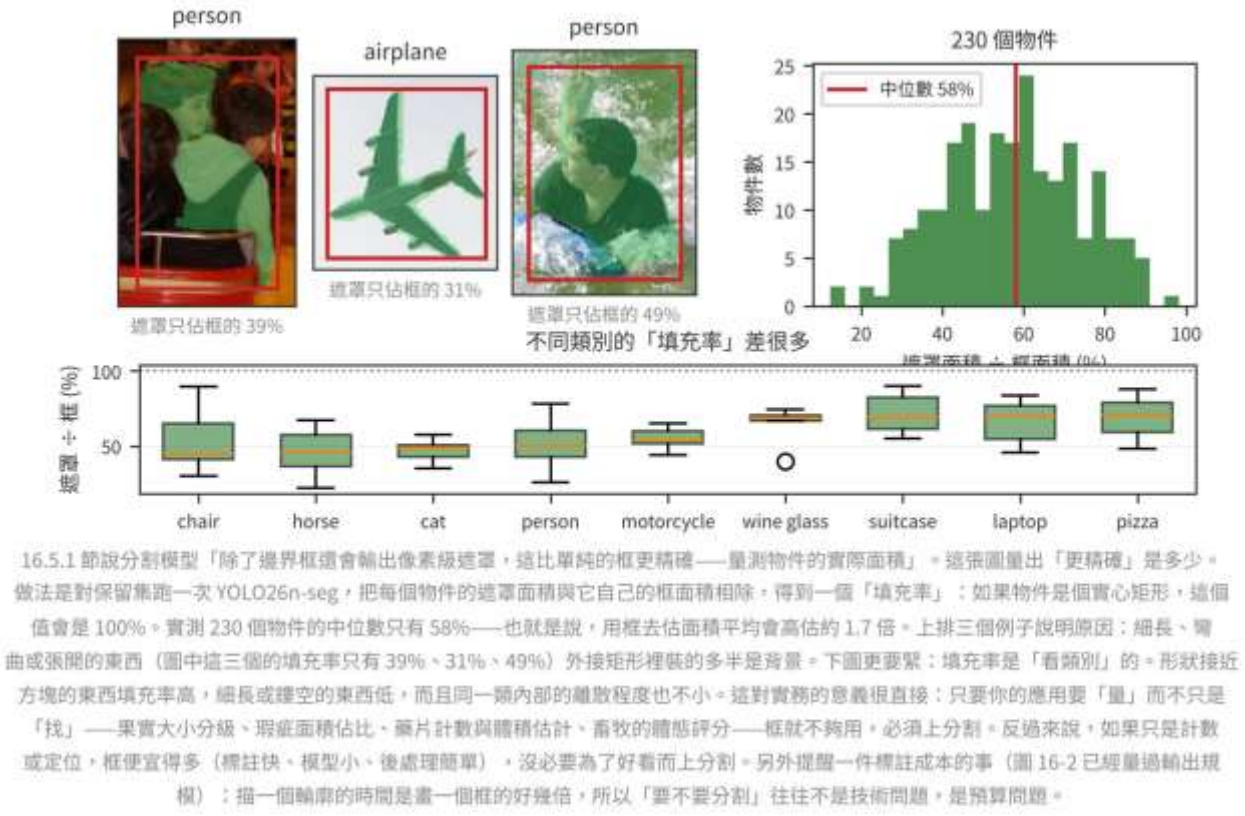
圖 16-13　標註品質：框畫歪一點，成績掉多少

16.5 延伸功能：分割、姿態估計與追蹤

現代 YOLO 已不只是偵測器，而是一套整合的視覺工具箱，只需更換模型權重即可切換任務。

16.5.1 實例分割

使用 seg 模型，YOLO 除了邊界框還會輸出每個物件的像素級遮罩(mask)，達成 16.1 節所述的實例分割。這比單純的框更精確——量測物件的實際面積、計算不規則形狀的尺寸、或做背景去除時，遮罩遠優於矩形框。它也是第 9 章傳統分割方法的深度學習後繼者。

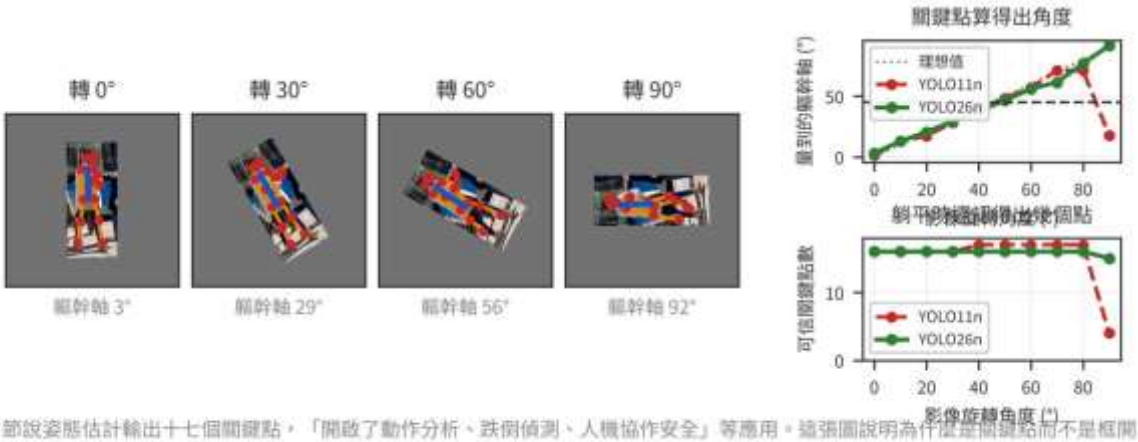


16.5.1 節說分割模型「除了邊界框還會輸出像素級遮罩，這比單純的框更精確——量測物件的實際面積」。這張圖量出「更精確」是多少。做法是對保留集跑一次 YOLO26n-seg，把每個物件的遮罩面積與它自己的框面積相除，得到一個「填充率」：如果物件是個實心矩形，這個值會是 100%。實測 230 個物件的中位數只有 58%——也就是說，用框去估面積平均會高估約 1.7 倍。上排三個例子說明原因：細長、彎曲或張開的東西（圖中這三個的填充率只有 39%、31%、49%）外接矩形裡裝的多半是背景。下圖更要緊：填充率是「看類別」的。形狀接近方塊的東西填充率高，細長或鏤空的東西低，而且同一類內部的離散程度也不小。這對實務的意義很直接：只要你的應用要「量」而不只是「找」——果實大小分級、瑕疵面積估比、藥片計數與體積估計、畜牧的體態評分——框就不夠用，必須上分割。反過來說，如果只是計數或定位，框便宜得多（標註快、模型小、後處理簡單），沒必要為了好看而上分割。另外提醒一件標註成本的事（圖 16-2 已經量過輸出規模）：描一個輪廓的時間是畫一個框的好幾倍，所以「要不要分割」往往不是技術問題，是預算問題。

圖 16-14　框與遮罩：量面積時差多少

16.5.2 姿態估計

使用 pose 模型，YOLO 會輸出人體的關鍵點(通常十七個：鼻、雙眼、雙耳、雙肩、雙肘、雙腕、雙髖、雙膝、雙踝)，形成骨架。這開啟了豐富的應用：動作分析與運動姿勢矯正、人機協作中判斷作業員的手部位置以確保安全、手勢指令控制、以及第 12 章討論過的跌倒偵測——透過追蹤重心高度的急遽變化與骨架的姿態，能比單純的框更可靠地判斷跌倒。

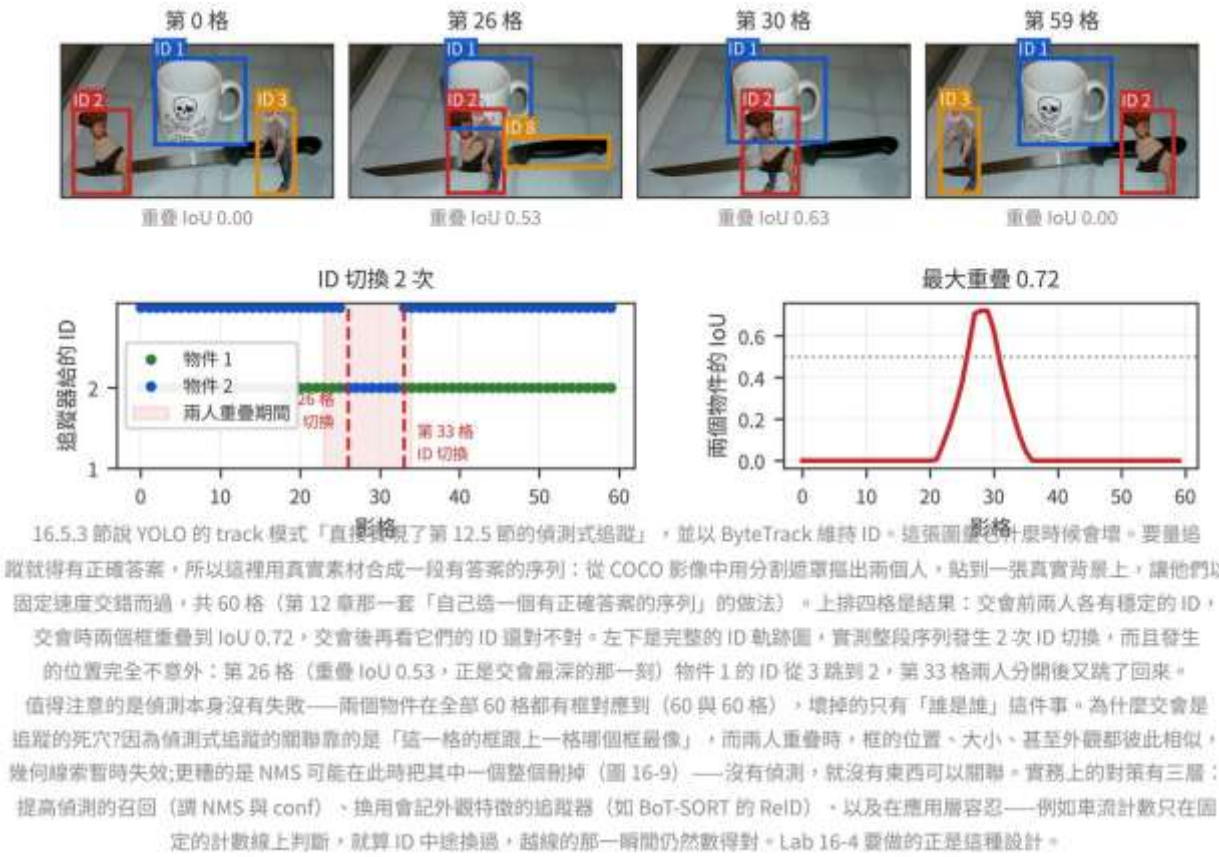


16.5.2 節說姿態估計輸出十七個關鍵點，「開啟了動作分析、跌倒偵測、人機協作安全」等應用。這張圖說明為什麼是關鍵點而不是框開啟了這些應用，並且比較兩代模型在最關鍵的那個角度上的差別。做法是把同一個人的影像逐步旋轉（模擬人倒下），每一步重做一次估計，再取雙肩中點與雙髖中點連成「軀幹軸」（Lab 16-3 要做的事），算它與垂直線的夾角。YOLO26n 量到的角度從 3° 一路跟到 92°，幾乎貼著理想的對角線——它是一個可以直接設門檻的連續量，超過 45° 就報警。相較之下，只看偵測框的寬高比從 0.37 變到 2.51，方向雖對卻容易被污染：人蹲下、彎腰、張開雙臂，寬高比都會變，而軀幹軸不會。這就是「輸出結構」的價值（圖 16-1 量過它的資訊量）。右邊兩張圖則是這一版真正的進步所在。上一代的 YOLO11n 在轉到 90°、也就是人完全躺平時整個崩掉：可信關鍵點從 17/17 掉到 4/17，軀幹軸算出 18°——一個完全錯誤、而且看起來像「站著」的值。YOLO26n 在同一個角度仍有 15/17 個可信關鍵點，軀幹軸 92°，判斷正確。這與官方宣稱姿態估計 AP 提升的方向一致，而且提升的正是最需要它的那一端。話雖如此，16.6.4 節那句「不能單靠一個可能誤判的 AI 模型」依然成立：本圖只旋轉了一個人、一種背景，真實的跌倒還有遮擋、拍攝角度、衣著與光線的變化。正確的系統設計仍是把姿態估計當成第一道預警、要求連續多格一致、並搭配其他感測器——下一步的算術在圖 16-19。

圖 16-15　姿態估計：十七個點，與兩代模型在躺平時的差別

16.5.3 物件追蹤

YOLO 內建追蹤功能，直接實現了第 12.5 節介紹的「偵測式追蹤」(tracking-by-detection)：每一影格用 YOLO 偵測，再以 ByteTrack、BoT-SORT 等演算法把跨影格的偵測結果關聯起來、賦予穩定的 ID。這正是第 12 章所說 SORT/DeepSORT 框架的現代實作。有了穩定的 ID，才能做人流計數、車流分析、軌跡記錄——這些在第 12.7 節的案例中都已鋪陳過，現在終於能用最先進的工具實現。



16.5.3 節說 YOLO 的 track 模式「直接套用了第 12.5 節的偵測式追蹤」，並以 ByteTrack 維持 ID。這張圖最右什麼時候會壞。要量追蹤就得有正確答案，所以這裡用真實素材合成一段有答案的序列：從 COCO 影像中用分割遮罩摳出兩個人，貼到一張真實背景上，讓他們以固定速度交錯而過，共 60 格（第 12 章那一套「自己造一個有正確答案的序列」的做法）。上排四格是結果：交會前兩人各有穩定的 ID，交會時兩個框重疊到 IoU 0.72，交會後再看它們的 ID 還對不對。左下是完整的 ID 軌跡圖，實測整段序列發生 2 次 ID 切換，而且發生的位置完全不意外：第 26 格（重疊 IoU 0.53，正是交會最深的那一刻）物件 1 的 ID 從 3 跳到 2，第 33 格兩人分開後又跳了回來。值得注意的是偵測本身沒有失敗——兩個物件在全部 60 格都有框對應到（60 與 60 格），壞掉的只有「誰是誰」這件事。為什麼交會是追蹤的死穴？因為偵測式追蹤的關聯靠的是「這一格的框跟上一格哪個框最像」，而兩人重疊時，框的位置、大小、甚至外觀都彼此相似，幾何線索暫時失效；更糟的是 NMS 可能在此時把其中一個整個刪掉（圖 16-9）——沒有偵測，就沒有東西可以關聯。實務上的對策有三層：提高偵測的召回（調 NMS 與 conf）、換用會記外觀特徵的追蹤器（如 BoT-SORT 的 ReID）、以及在應用層容忍——例如車流計數只在固定的計數線上判斷，就算 ID 中途換過，越線的那一瞬間仍然數得對。Lab 16-4 要做的正是這種設計。

圖 16-16　偵測式追蹤：ID 在交會時還認得出來嗎

16.5.4 YOLO26 再擴充的任務家族

前面三小節的分割、姿態與追蹤，從 YOLOv8 時代就有了。YOLO26 又補上三種任務與一個特別的變體，它們各自回答一個框回答不了的問題。用法一律是換一個權重檔，介面完全相同。

語意分割(yolo26n-sem.pt)為畫面上每一個像素標類別，但不區分同類的不同個體——這正是 16.1 節區分過的第三種任務，也是第 9.6 節 U-Net 擅長的那一類。它與實例分割的分工很清楚：要數「有幾顆果實、每顆多大」用實例分割;要問「這張空拍圖裡農田、道路、建物各佔多少面積」——個體根本無從數起——就用語意分割。農地分區、道路可行駛區域、瑕疵區域佔比，都屬於後者。

單眼深度估計(yolo26n-depth.pt)從一張普通的彩色影像推估每個像素的相對深度。請特別注意「單眼」與「相對」這兩個限制詞。單眼表示它不靠雙目視差、也不靠光達，而是從透視、遮擋、紋理梯度這些線索推論——這與第 4 章談相機模型時說

過的「單張影像遺失了深度維度」是同一件事，所以它推出來的是一種合理的猜測，不是量測。相對則表示輸出沒有絕對尺度：它能告訴你 A 比 B 近，卻不能直接告訴你 A 距離鏡頭幾公分，除非另外用已知尺寸的物件或其他感測器做標定。用在避障、場景理解、背景虛化這類只需要相對遠近的任務很合適;用在尺寸量測或機械手臂定位就必須格外謹慎，這又是第 7 章「還原 vs. 腦補」的老問題。

有向邊界框(OBB，yolo26n-obb.pt)輸出的框可以旋轉，也就是多了一個角度參數。一般的邊界框永遠與影像的軸平行，遇到斜放的物件時，框裡會塞進大量背景(圖 16-14 量過這個「填充率」問題)。空拍影像中的船隻、停車場中斜停的車輛、輸送帶上隨機旋轉的零件、以及 PCB 上的元件，都是有向邊界框的主場。它同時也直接給出物件的方向，對機械手臂要決定夾爪角度這類任務非常實用——這比第 13 章用主軸或最小外接矩形去估方向省事得多。YOLO26 在這個任務上額外設計了角度損失，並處理了角度在 0 度與 180 度交界處的不連續問題(一個 179 度與一個 −179 度的框其實幾乎一樣，但單純相減會得到很大的誤差)。

最後是 YOLOE-26，一個開放詞彙(open-vocabulary)的變體。前面所有模型都只認得訓練時定義好的類別——COCO 的八十類，或你自己標註的那三類。開放詞彙模型則接受文字提示：你可以直接要求它找「戴安全帽的人」或「生鏽的螺絲」，即使訓練時沒有這些類別。它也接受視覺提示(給一張範例影像)，甚至可以無提示運作。對「類別很多、而且會一直增加」的場景(例如零售貨架上的上千種商品)，這是很有吸引力的方向。代價是準確度仍明顯低於專門訓練的模型，而且提示詞怎麼寫會顯著影響結果。務實的用法是把它當成「快速原型與資料標註的助手」：先用文字提示把可能的目標框出來，再由人工確認、最後仍然訓練一個專用模型上線。

把這幾個任務放在一起看，可以看出一條清楚的脈絡：YOLO 正在從「一個偵測器」變成「一組共用骨幹的視覺預測頭」。對工程實務的意義是，選任務的成本變低了——你可以先用偵測做出原型，發現需要面積就換 seg、需要方向就換 obb、需要

遠近就換 depth，而資料管線、訓練指令與評估流程幾乎不用改。但有一件事沒有變便宜：標註。每換一種任務，標註的形式與成本就完全不同(圖 16-2 量過四種任務的輸出規模差了五個數量級)，而標註品質仍然是成績的天花板(圖 16-13)。所以選任務時真正該問的不是「模型支不支援」，而是「我付得起那個標註成本嗎」。

程式 16-5 分割、姿態估計與追蹤

```
from ultralytics import YOLO
import cv2

# --- 實例分割:輸出像素級遮罩 ---
seg_model = YOLO("yolo26n-seg.pt")
r = seg_model("parts.jpg")[0]
if r.masks is not None:
    print("偵測到物件數:", len(r.masks))
    mask = r.masks.data[0].cpu().numpy()      # 第一個物件的遮罩
    area_px = int(mask.sum())                 # 像素面積(可量測)
    print("面積(像素):", area_px)

# --- 姿態估計:輸出人體 17 個關鍵點 ---
pose_model = YOLO("yolo26n-pose.pt")
r = pose_model("person.jpg")[0]
if r.keypoints is not None:
    kpts = r.keypoints.xy[0]                  # 第一個人的關鍵點座標
    print("鼻子座標:", kpts[0].tolist())
    print("左肩座標:", kpts[5].tolist())

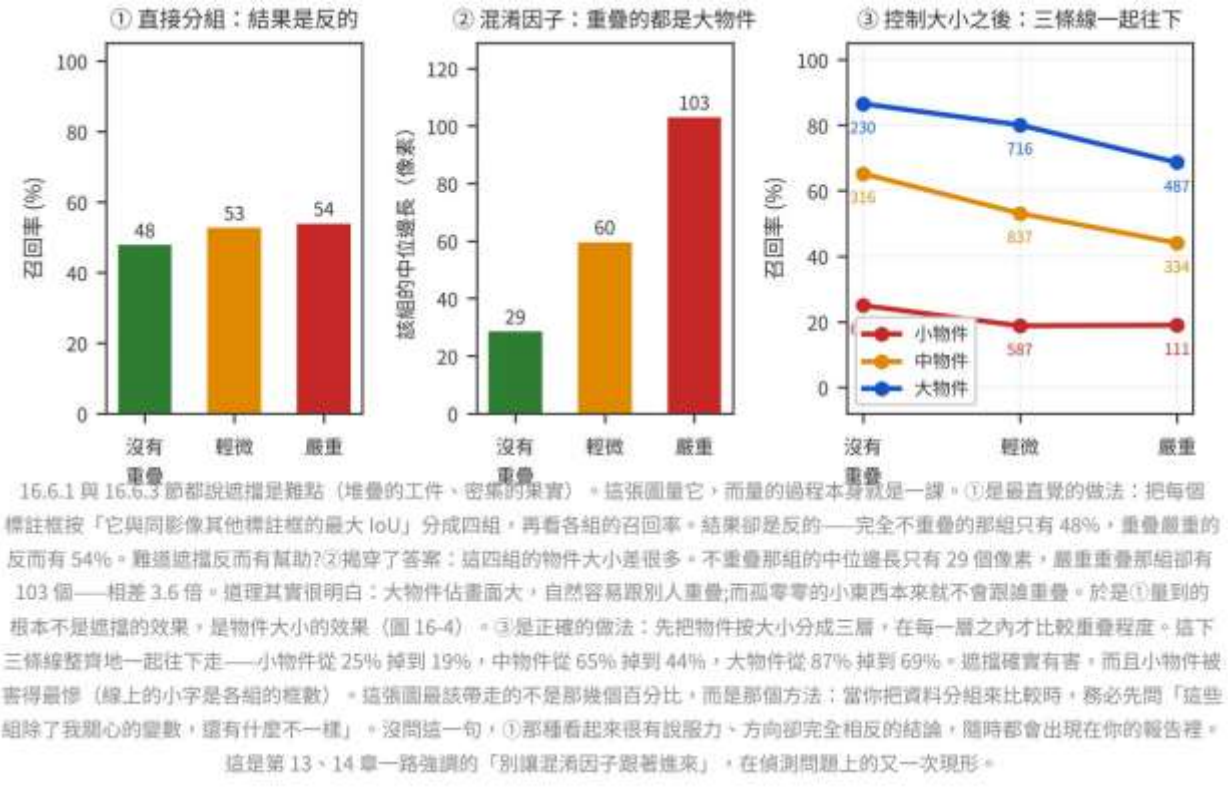
# --- 追蹤:第 12 章的偵測式追蹤 ---
cap = cv2.VideoCapture("traffic.mp4")
while True:
    ok, frame = cap.read()
    if not ok:
        break
    # persist=True 讓 ID 跨影格維持
    res = seg_model.track(frame, persist=True, tracker="bytetrack.yaml")[0]
    if res.boxes.id is not None:
        for box, tid in zip(res.boxes.xyxy, res.boxes.id):
            x1, y1, x2, y2 = box.tolist()
            cv2.putText(frame, f"ID {int(tid)}", (int(x1), int(y1)-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
    cv2.imshow("track", res.plot())
    if cv2.waitKey(1) == 27:
        break
cap.release(); cv2.destroyAllWindows()
```

16.6 應用案例

16.6.1 案例一：AOI 瑕疵偵測

情境：產線上自動偵測工件表面的刮痕、裂紋、缺件。相較第 6 章的頻率域紋理檢測與第 9、10 章的傳統管線，YOLO 的優勢是能處理形態多變、難以用規則描述的瑕疵，且能同時分類多種瑕疵類型。流程：蒐集並標註各類瑕疵影像(通常需數百至數千張)→ 以遷移學習訓練 YOLO → 用 mAP 驗證 → 匯出為 ONNX 或 TensorRT 部署到產線電腦或 Jetson(第 17 章)。

關鍵挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少。對策包括：資料增強(第 4、5 章)、合成瑕疵樣本、或改用只學習「正常」樣貌的異常偵測方法。這也解釋了為什麼第 6 章的傳統方法至今仍有價值——它不需要瑕疵樣本，對「沒見過的新瑕疵」天生有效，常與 YOLO 互補使用。



16.6.1 與 16.6.3 節都說遮擋是難點（堆疊的工件、密集的果實）。這張圖量它，而量的過程本身就是一課。①是最直覺的做法：把每個標註框按「它與同影像其他標註框的最大 IoU」分成四組，再看各組的召回率。結果卻是反的——完全不重疊的那組只有 48%，重疊最重的反而有 54%。難道遮擋反而有幫助?②揭穿了答案：這四組的物件大小差很多。不重疊那組的中位邊長只有 29 個像素，最重重疊那組卻有 103 個——相差 3.6 倍。道理其實很明白：大物件佔畫面大，自然容易跟別人重疊;而孤零零的小東西本來就不會跟誰重疊。於是①量到的根本不是遮擋的效果，是物件大小的效果（圖 16-4）。③是正確的做法：先把物件按大小分成三層，在每一層之內才比較重疊程度。這下三條線整齊地一起往下走——小物件從 25% 掉到 19%，中物件從 65% 掉到 44%，大物件從 87% 掉到 69%。遮擋確實有害，而且小物件被害得最慘（線上的小字是各組的框數）。這張圖最該帶走的不是那幾個百分比，而是那個方法：當你把資料分組來比較時，務必先問「這些組除了我關心的變數，還有什麼不一樣」。沒問這一句，①那種看起來很有說服力、方向卻完全相反的結論，隨時都會出現在你的報告裡。這是第 13、14 章一路強調的「別讓混淆因子跟著進來」，在偵測問題上的又一次現形。

圖 16-17　擁擠：遮擋吃掉多少召回率（以及一個混淆因子）

16.6.2 案例二：交通場景分析

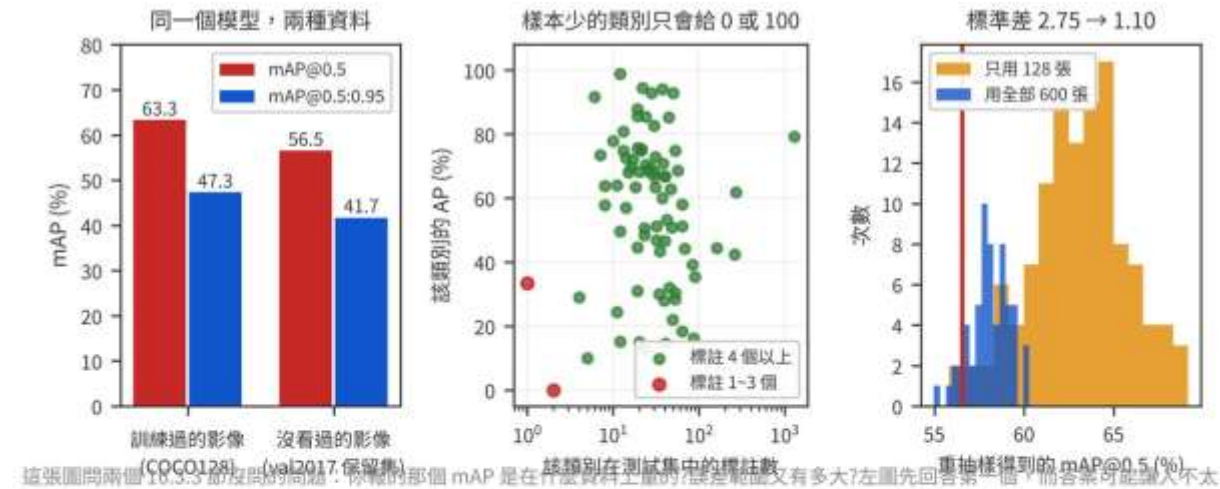
情境：路口監控自動統計車流量、分類車種、偵測違規。這是第 12 章與本章的完

整整合：YOLO 偵測並分類車輛與行人 → 內建追蹤器維持每個目標的 ID → 依軌跡越線計數(第 12.7.1 節)、結合第 4 章相機校正換算實際速度(第 12.7.2 節)、或判斷是否闖紅燈、逆向。這條管線串起了第 4、12、16 章，是智慧交通的標準架構。

16.6.3 案例三：農業與畜牧應用

情境：這是雲林在地產業最有潛力的應用之一。以無人機或固定攝影機搭配 YOLO，可自動計數果樹上的果實以估算產量、偵測病蟲害的葉片斑點、辨識雜草以做精準除草、或在畜牧場中計數與追蹤牲畜、透過姿態估計監控其行為異常(如久臥不起可能是生病徵兆)。

這類應用的工程重點在於：田間環境的光線變化劇烈(第 5、8 章的白平衡與對比強化成為必要的前處理)、物件常相互遮擋(密集果實的偵測是難點)、且需在無網路的田間即時執行(第 17 章的邊緣部署)。這也再次體現本書一貫的訊息：先進的 AI 模型不是孤立運作的，它需要前面各章的影像處理技術一起支撐，才能在真實世界中可靠工作。



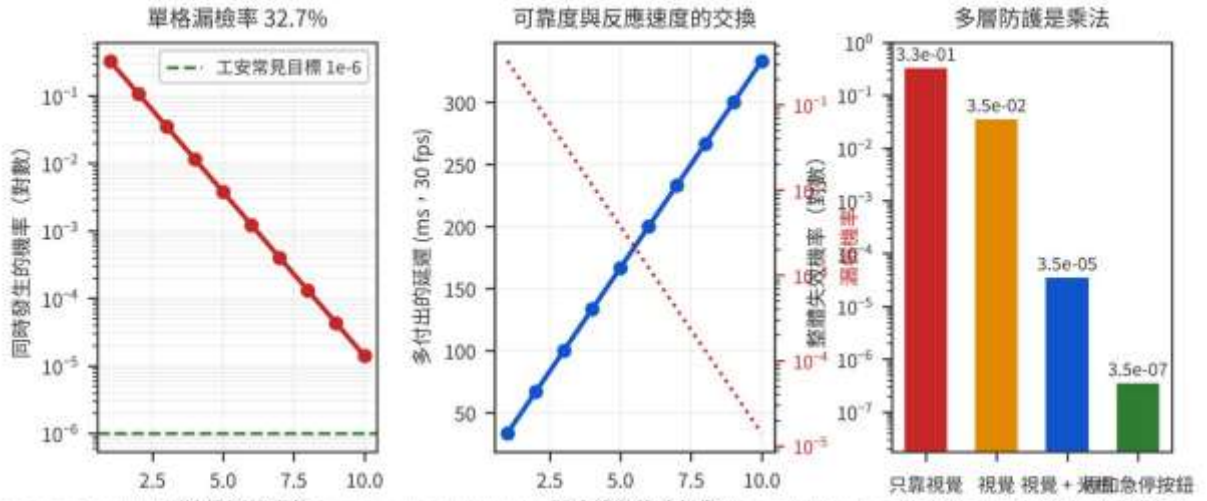
這張圖問兩個你不太舒服的問題：你報的那個 mAP 是在什麼資料上量的？誤差範圍又有多大？左圖先回答第一個，恐怕答案可能讓人不太舒服。同一個 YOLO26n，在 COCO128 上得到 mAP@0.5 63.3%，在 600 張沒看過的 val2017 影像上只有 56.5%，差了 6.8 個百分點；mAP@0.5:0.95 則是 47.3% 對 41.7%。原因很簡單：COCO128 是從 COCO 的「訓練集」裡抽出來的 128 張，模型早就看過它們。這不是 COCO128 的錯——它本來就是給人試跑流程用的小玩具；錯的是拿它當驗收成績。第 14 章講過「評估模型絕不能用它訓練時看過的資料」，這張圖就是那句話在偵測問題上的現場：一個 10 個百分點的落差，足以讓任何選型結論翻盤。中圖與右圖回答第二個問題。保留集上有標註的 79 個類別中，有 2 類的標註數只有 1 到 3 個，其中 1 類的 AP 剛好是 0% 或 100%——那不是能力，是樣本太少時 AP 只有幾種可能值。右圖量整體的不確定度：只用其中 128 張時，重抽樣的 mAP 標準差是 2.75 個百分點、95% 區間 57.4% 到 68.1%；用全部 600 張則降到 1.10——老規矩，誤差大致隨樣本數的平方根下降。三條守則：報 mAP 一定要說清楚測試集的來源與大小；比較模型要用同一份、而且模型都沒看過的資料；以及你真正在乎的那一類要有足夠樣本，否則它的 AP 只是在擲骰子。順帶一提，本章自寫的 mAP 實作在這份保留集上得到 56.52%，與 Ultralytics 官方工具的結果相差約一個百分點，差異來自群體標註與面積過濾等規則——跨工具比較數字時，先確認兩邊算的是同一件事。

圖 16-18　你報的那個 mAP：在哪種資料上量的，誤差又有多大

16.6.4 案例四：人機協作的安全監控

情境：在協作型機器人(cobot)的工作區，以 YOLO 的姿態估計即時追蹤作業員的手部與身體位置，當人員進入機器人的危險作業半徑時，自動減速或停機。這是工業安全的重要應用，也直接連結智慧機器人學程的核心關懷。

但這個案例也必須談責任：安全攸關的系統，不能單靠一個可能誤判的 AI 模型。工程上的正確做法是「多層防護」——視覺系統作為第一道預警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制。呼應叫獸開講與第 15 章談過的深度學習侷限(對抗脆弱、分布偏移)，在人命關天的場合，對 AI 保持審慎、設計冗餘，是工程師的基本責任。



16.6.4 節說人機協作系統不能單靠一個可能誤判的 AI 模型，這張圖把那句話畫成了兩張圖。這張圖把那句話畫成了兩張圖。左圖是單格漏檢率 32.7%（YOLO26n 在 1284 張測試圖上的失效率），右圖是多層防護相乘後的失效率。左圖：在保留集的 1284 個「人」的標註上，YOLO26n 以預設設定單格漏檢 420 個，漏檢率 32.7%。這個數字要公平地讀：COCO 影像裡的「人」有很多是遠處的小身影，而協作機器人工作區看到的是近距離的大目標，圖 16-17 量過大物件的召回率是 94%，對應的漏檢率約 6%。所以真實場域的單格漏檢率會比這裡低，但重點不變——它絕不是零，而機械手臂不會等你。但偵測是逐格進行的，所以第二步是連續多格的要求：若假設各格的漏檢彼此獨立，連續 3 格都漏的機率是 3.50e-02，連續 5 格是 3.74e-03。左圖畫的就是這條指數下降的線。代價在中間：30 fps 下，要求連續 3 格等於多付出 100 毫秒的反應延遲，連續 5 格是 167 毫秒——而機械手臂在這段時間裡還在動。可靠度與反應速度是直接對換的。右圖是 16.6.4 節真正的重點：把獨立的防護層乘起來。視覺連續 3 格、加上實體光柵、再加上急停按鈕，整體失效機率可以壓到 3.5e-07。這裡有兩個必須講清楚的前提，否則這張圖會變成危險的樂觀。其一，「獨立」是關鍵假設，而它常常不成立——連續影格的漏檢高度相關（同一個遮擋會連漏很多格），所以真實的連續漏檢機率遠高於單格機率的三次方。其二，機率相乘只在「各層失效原因不同」時才成立;這正是為什麼安全層要用不同的物理原理（光、壓力、人手），而不是裝三台一樣的相機。這一段算術不是為了讓你放心，是為了讓你知道「我的模型準確率 95%」這句話，在安全系統裡什麼也沒保證。

圖 16-19　安全攸關系統：單一模型的失效率與多層防護

16.7 技術倫理：工程師的責任

本節不談程式，而是承接叫獸開講，把全書零散提及的倫理線索收攏起來——因為 YOLO 這樣強大而易用的工具，正是最需要談責任的地方。

回顧本書的倫理伏筆：第 7 章的「還原 vs. 腦補」提醒我們區分「找回的資訊」與「編造的資訊」，並在司法、醫療場合誠實標註;第 9 章的醫學影像分割強調自動結果僅供輔助、須由醫師確認;第 12 章的跌倒偵測提醒照護應用涉及被監控者的隱私與尊嚴;第 14 章的肺炎 X 光案例揭露資料偏誤如何讓模型學到無意義的捷徑;第 15 章列舉了深度學習的黑箱與對抗脆弱。這些線索指向同一個核心：技術能力愈強，對後果的想像力就愈重要。

物件偵測尤其敏感，因為它的用途橫跨光譜的兩端：同一套人流分析技術，可以用於商場最佳化動線、也可以用於監控特定族群;同一套目標追蹤技術，可以用於搜救

失蹤者、也可以用於軍事鎖定。Redmon 的選擇是退出，但那不是唯一的答案。工程師還有許多實際的作為：在系統設計中內建隱私保護(如只統計數量不儲存人臉、在裝置端處理而非上傳雲端)、對資料來源與模型限制保持透明、在授權條款中明訂禁止的用途、在部署前評估誤判的後果與受影響者、以及——最基本卻最重要的——在被要求做一件你認為不對的事情時，有能力說出自己的疑慮。

叫獸想說的是：這門課教你把技術做出來，但沒有任何一行程式碼能替你決定「該不該做」。那是你作為一個專業工作者、也作為一個人的判斷。而判斷力，和寫程式一樣，是需要練習的——從現在開始，每當你部署一個系統，多問自己一句：「誰會受益?誰可能受害?如果它出錯了，誰要承擔?」

16.7.4 推導：把「安全」與「隱私」寫成可以驗收的數字

16.6.4 與 16.7 節談了多層防護與隱私保護，但倫理若停在態度層次，就只是漂亮話。這一小節把那兩件事變成可以寫進規格書、也可以被稽核的算式。這不是要用數學取代判斷，而是要讓判斷有東西可以討論——一個說不出失效率的安全承諾，和沒有承諾是一樣的。

先看安全。設偵測器在單一影格上漏掉目標的機率是 q。若要求「連續 k 格都偵測不到才算真的沒看到」，且假設各格的漏檢彼此獨立，則整段判斷失效的機率是 q 的 k 次方。圖 16-19 以保留集的 1,284 個「人」實測 $q \approx 0.33$(其中包含大量遠處的小身影;若只看近距離、不被遮擋的大目標，依圖 16-17 的量測 q 約為 0.13)，於是連續 3 格全漏的機率約 0.035、連續 5 格約 0.0037。代價是延遲：在 30 fps 下，多等 k 格就是多 $k/30$ 秒，$k = 3$ 是 100 毫秒、$k = 5$ 是 167 毫秒——而機械手臂在這 167 毫秒裡還在移動。所以 k 不是愈大愈好，它由「可容忍的反應距離」反推：若手臂速度是 v、安全距離是 d，則 $k/30 \times v$ 必須遠小於 d。

接著是多層防護的算術。若有若干道彼此獨立的防護，整體失效機率是各層失效機率的乘積。視覺連續 3 格(0.035)、實體光柵(約 10^{-3})、急停按鈕(約 10^{-2})串起來，

整體約 3.5×10^{-7}，這才接近工安領域常見的目標值。

但這裡有兩個前提必須說破，否則上面那串乘法會變成危險的樂觀。其一，「獨立」幾乎從不成立。連續影格的漏檢高度相關——同一個遮擋、同一個逆光、同一個被 NMS 刪掉的框(圖 16-9)，會讓好幾格一起漏;真實的連續漏檢機率因此遠高於 q 的 k 次方。工程上的對策是把 q 的估計做在「最壞情境」而非平均情境上：量的是逆光、遮擋、極端姿態時的漏檢率。其二，各層必須用不同的物理原理才算獨立。裝三台一模一樣的相機不會讓失效率變成三次方，因為讓第一台失效的霧氣、逆光或污損，會同時讓另外兩台失效。光、壓力、人手——原理不同，才是真正的獨立。

再看隱私。去識別化本身也是一個偵測任務，因此它也有召回率，而「漏掉一張臉」就是一次直接的隱私失效。圖 16-20 實測了這件事：一張有四個人的照片，姿態模型只圍出三個頭部，另一個因為背對鏡頭、臉部關鍵點信心不足而漏掉。所以這類管線的驗收標準不是準確率，而是「最壞情況下有沒有漏」。可操作的設計有三條：把臉部偵測的門檻調到極低，寧可多模糊(此處誤報幾乎沒有代價);偵測不到臉時，直接模糊整個人體框作為保底;以及最根本的一條——在管線的最前端就不保存原始影像，只留下框的座標與時間戳。



16.7 節把「內建隱私保護」列為工程師的實際作為之一，但沒說怎麼做、代價多少。這張圖做一次給你看。做法是把去識別化直接接在偵測管線後面：先用姿態模型的臉部關鍵點（鼻、雙眼、雙耳）圈出頭部範圍，再對那一塊做高斯模糊（第 5 章）。結果是中間那張：3 個頭被遮蔽，只佔畫面的 0.7%，而偵測器之後仍然數得出 4 個人（原圖 4 個）、姿態模型仍然抓得到 39 個可信關鍵點（原圖 41 個）——也就是說，人流計數、區域進出、安全距離、跌倒偵測這些功能完全不受影響。右邊那張是粗暴的做法：把整個人都模糊掉，遮蔽 27.4% 的畫面，偵測到的人數掉到 1 個、可信關鍵點只剩 12 個——功能跟著一起消失。這裡要補一個實測得到的細節：模糊的「強度」也會改變結果。同樣是全身模糊，核小一點時偵測器照樣數得出四個人，核大到足以抹掉輪廓才會失效。換句話說，弱的模糊既保護不了身分、也沒省下什麼，是兩頭落空的設定——這種參數必須實測，不能憑感覺調。這張圖想說的是：隱私保護不是「要或不要」的二選一，而是一個可以設計的取捨。多數監控類應用真正需要的是「有幾個人、在哪裡、往哪走」，而不是「這個人是誰」；把身分資訊在管線的最前端就抹掉，功能幾乎不損失。更進一步的做法是根本不存原始影像——只存框的座標與時間戳，影像處理完就丟。這些都是可以寫進規格書、也可以被稽核的工程決定，比事後宣稱「我們很重視隱私」有意義得多。最後呼應 16.7 節那句話：沒有任何一行程式碼能替你決定該不該做，但一旦決定要做，怎麼做得更負責任，是有技術答案的。

圖 16-20　去識別化：把隱私保護做進管線裡

最後把這兩件事收攏成一個共同的形狀。安全與隱私的工程處理，都是把一個模糊的承諾(「我們很重視」)換成三樣具體的東西：一個可以量的指標、一個明確的最壞情境、以及一個在模型失效時仍然有效的保底機制。第 14 章說機器學習真正難的不是訓練模型，是誠實地知道它有多好;本章要補的下半句是——知道它有多好之後，你還得決定，這樣夠不夠好到可以上線。那個決定沒有公式，但它應該建立在有公式的那一部分之上。

16.8 本章與全書的觀念連結總表

表 16-2 第 16 章與全書其他章節的觀念連結

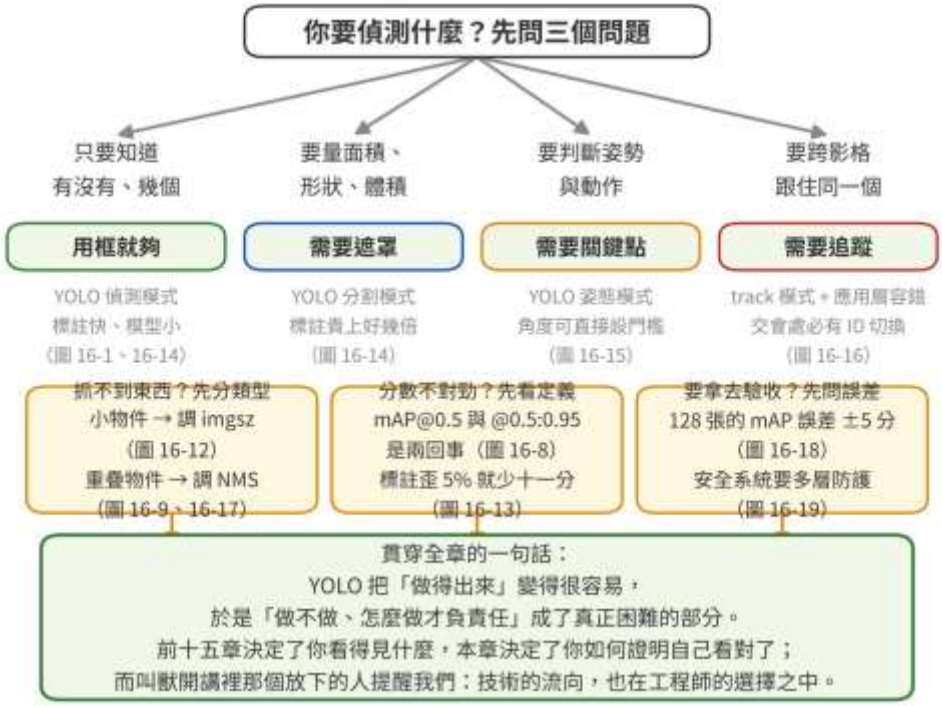


圖 16-21　第 16 章的決策地圖

程式實作 Lab 16

Lab 16-1 YOLO 快速上手

目標：熟悉推論與參數。(a)安裝 Ultralytics，以程式 16-2 對數張照片(街景、室內、多物件場景)推論，觀察 COCO 八十類的偵測結果;(a2)同一張照片改用 nms=False 再跑一次，比較兩個預測頭的框數、信心分數與耗時，並說明免 NMS 的輸出為什麼固定是 300 個框;(b)調整 conf 門檻(0.1、0.25、0.5、0.8)，記錄漏檢與誤報如何變化，對照第 14 章的精確率-召回率權衡;(c)調整 iou 門檻，觀察 NMS 如何影響重疊框的保留;(d)比較 yolo26n 與 yolo26s/m 的偵測結果與推論速度(FPS)，體會精度與速度的取捨;(e)討論：在什麼應用中你會把 conf 調低?什麼應用中會調高?

Lab 16-2 訓練自訂偵測器

目標：完成一個屬於自己的偵測器。(a)選定二到三類物件(如實驗室器材、特定零件、水果)，各拍攝並標註五十張以上影像(建議用 LabelImg 或 Roboflow);(b)依第 14 章原則切分訓練/驗證集，撰寫 data.yaml;(c)以程式 16-4 從預訓練權重開始訓練(遷移

學習)，觀察訓練過程的損失曲線與 mAP 變化;(d)在驗證集上計算 mAP@0.5 與 mAP@0.5:0.95，並找出模型最常誤判的情況;(e)刻意減少訓練資料到二十張重訓，比較 mAP 差異——驗證第 14 章「資料是燃料」;(f)加入或移除資料增強參數，觀察對過度擬合的影響。

Lab 16-3 姿態估計與跌倒偵測

目標：實作第 12 章預告的跌倒偵測。(a)以程式 16-5 的 pose 模型對人物影片做姿態估計，視覺化十七個關鍵點與骨架;(b)計算人體重心(可用髖部關鍵點近似)的高度與其變化速率;(c)設計一個簡單的跌倒判斷規則：重心高度在短時間內急遽下降、且之後維持在低位;(d)測試：正常坐下、蹲下、躺下與真正的跌倒，你的規則能否區分?(e)討論：承第 12.7.3 節，這類照護系統的誤報與漏報各有什麼代價?如何在系統設計中兼顧隱私(例如只傳送骨架資料而非影像)?

Lab 16-4 交通流量分析系統

目標：整合第 12 章與本章，完成完整的視訊分析系統。(a)以 YOLO 的 track 模式對交通影片偵測並追蹤車輛，確認 ID 跨影格穩定;(b)畫一條虛擬計數線，依軌跡越線方向統計雙向車流量;(c)分類統計不同車種(汽車、機車、公車、卡車)的數量;(d)進階：結合第 4 章相機校正，把像素位移換算成實際速度(第 12.7.2 節);(e)量測系統的 FPS，評估在你的硬體上能否即時處理，並思考第 17 章的部署選項(用更小的模型?降低解析度?用 GPU?)。

Lab 16-5 自己寫一次 IoU、NMS 與 mAP

目標：把 16.3 節的三個概念從「看得懂」變成「寫得出來」，並親手驗證自己的實作與官方工具一致。(a)寫一個 iou(a, b) 函式，並用三組手算的例子驗證：完全重合（應為 1）、完全不相交（應為 0）、以及一個 100×100 的框水平偏 33 像素（應約為 0.5）;特別檢查你的程式在不相交時有沒有算出負面積。(b)寫一個 nms(boxes, scores, thr) 函式，對同一張影像先以 iou=1.0 取得未經抑制的原始輸出，再用自己的

NMS 處理，確認結果與 Ultralytics 內建的一致;並畫出「門檻 vs 保留框數」的曲線（對照圖 16-6）。(c)寫一個配對函式：預測依分數由高到低，每個預測找 IoU 最大且尚未被認領的同類真實框，超過門檻才算命中、每個真實框最多認領一次;輸出 TP 標記陣列。(d)由 TP 標記累積出精確率-召回率曲線，實作全點內插 AP 與十一點內插 AP，比較兩者的差距。(e)對整個資料集逐類別計算 AP 再平均得到 mAP@0.5，與 model.val() 的輸出比較，若有差距請找出原因（提示：群體標註、面積過濾、以及低於信心門檻的預測有沒有被算進去）。(f)把 IoU 門檻從 0.50 掃到 0.95 計算十個 mAP 並平均，確認你算得出 mAP@0.5:0.95。延伸：把 conf 從 0.001 改成 0.25 重跑一次 (e)，觀察 mAP 被低估多少，並解釋原因。

Lab 16-6 把一個偵測器誠實地驗收一次

目標：訓練一個自己的偵測器，然後用本章的工具把它的能力與極限量清楚——重點不在分數多高，而在你能說出這個分數可以信幾分。(a)選定一到三類物件，拍攝並標註;切分時務必按「來源」分組（依拍攝日期、批次或場景），不要隨機打散，並在報告中說明你的切分依據。(b)以程式 16-4 從預訓練權重開始訓練，記錄訓練與驗證的 mAP 曲線，確認及早停止有沒有生效。(c)依物件面積分成小、中、大三組，分別計算召回率（對照圖 16-4）;再依「與鄰居的最大 IoU」分組，但必須在每個大小層之內比較（對照圖 16-17，想想為什麼不能直接分組）。(d)掃描信心門檻與 NMS 門檻各十個值，畫出精確率、召回率與 F1 的曲線，依你的應用代價（漏抓貴還是誤報貴）選定一組門檻並說明理由。(e)用重抽樣估計 mAP 的 95% 區間：對測試影像有放回地抽樣一百次、每次重算 mAP，取 2.5% 與 97.5% 分位數;據此說明你的模型與一個基準模型是否真的有差。(f)把人工標註隨機抖動 5% 再重算一次成績，用這個數字說明「標註品質的天花板」在哪裡（對照圖 16-13）。(g)寫一份一頁的驗收報告，內容必須包含：測試集的影像數與標註框數、mAP 及其誤差範圍、逐類別與逐尺寸的召回率、選定的門檻與理由、以及一節「已知失效情境」（列出至少三種你實際觀察到模型會失敗的狀況）。延伸：若你的應用屬於安全攸關類型，依 16.7.4 節的算式估計單格漏

檢率與連續多格的失效機率，並設計一套不依賴視覺的保底機制。

本章重點整理

- 四種視覺任務依精細度遞增：分類(是什麼)、偵測(是什麼加在哪裡)、語意分割(每像素類別、不分個體)、實例分割(每像素類別且分個體);姿態估計則輸出關鍵點骨架。
- 兩階段偵測器先提候選再分類(準但慢)，單階段偵測器一次前向傳播直接輸出所有結果(快);YOLO 把偵測重新定義為迴歸問題，只看一次全圖，並能利用全域脈絡減少背景誤判。
- 關鍵概念：IoU(交集聯集比，判定框準不準)、NMS(保留信心最高框、抑制重疊者，與第 9 章 Canny 同名同理)、mAP(建立於第 14 章精確率與召回率，常用 mAP@0.5 與 mAP@0.5:0.95)。
- Ultralytics YOLO26 提供 n/s/m/l/x 多種尺寸供依算力選擇，並同時訓練一對多(需 NMS)與一對一(免 NMS)兩個預測頭，以 nms=False 切換;它移除了 DFL、改用漸進式損失與小目標感知標籤指派，並擴充出語意分割、單眼深度估計、有向邊界框等任務;訓練自訂資料集三步驟：標註資料(YOLO 格式)、撰寫 data.yaml、train 與 val;訓練參數中的旋轉、亮度、翻轉即第 4、5 章的資料增強，從預訓練權重開始即第 15 章的遷移學習。
- 延伸功能：seg 模型輸出像素級遮罩(可精確量測面積)、pose 模型輸出十七個人體關鍵點(動作分析、跌倒偵測、人機協作安全)、track 模式實現第 12 章的偵測式追蹤並維持穩定 ID。
- 技術倫理：同一套技術可用於截然不同的目的;工程師的實際作為包括內建隱私保護、對限制保持透明、授權中明訂禁止用途、評估誤判後果、以及在必要時表達疑慮。安全攸關系統須設計多層防護，不可單靠可能誤判的 AI 模型。

習題

1. 請區分影像分類、物件偵測、語意分割與實例分割，說明各自的輸出形式與難度差異。

   提示與參考解答：分類：輸出整張影像的類別(是什麼)，不含位置。偵測：輸出每個物件的類別與邊界框(是什麼加在哪裡)。語意分割：每個像素標上類別，有精確輪廓但不分同類個體(三個人是同一片「人」)。實例分割：每像素類別且區分不同個體(三個人各有獨立輪廓與編號)，最精細也最困難。

2. 兩階段與單階段偵測器有何差異?YOLO 的「只看一次」具體是什麼意思?

   提示與參考解答：兩階段(如 R-CNN 系列)先產生候選區域再逐一分類與微調框，準確度高但慢(同一影像被網路處理多次)。單階段(YOLO、SSD)一次前向傳播就同時輸出所有物件的位置與類別。YOLO 把影像切成網格，讓網路對每個網格直接預測有無物件、框的位置、類別與信心，整張影像只需通過網路一次，故名 You Only Look Once。

3. YOLO 每次都看整張影像，這帶來什麼額外好處?它早期的弱點是什麼?

   提示與參考解答：好處：能利用全域脈絡資訊(如「周圍都是道路，這個模糊物體較可能是車而非船」)，背景誤判較少。早期弱點：對小物件與密集群體較不擅長;YOLOv3 引入多尺度預測(呼應第 11 章多解析度)後大幅改善。

4. 什麼是 IoU?它如何用於判定偵測是否正確?

   提示與參考解答：IoU(Intersection over Union)是預測框與真實框的交集面積除以聯集面積。IoU 為 1 代表完美重合、0 代表完全不重疊。實務上以 IoU 超過某門檻(常用 0.5)判定該偵測為正確;也用於 NMS 判斷兩框是否重複。

5. 說明 NMS 的運作步驟。它與第 9 章的哪個步驟同名同理?

   提示與參考解答：步驟：取信心分數最高的框，刪除與它 IoU 超過門檻的其他框(視為重複)，對剩下的框重複此程序直到處理完畢。它與第 9 章 Canny 邊緣偵測第三步的「非極大值抑制」同名同理——都是「保留局部最強、抑制鄰近重複」，體現本書「核心思想以不同面貌反覆出現」的主軸。

6. mAP 是如何計算的?它建立在第 14 章的哪兩個指標之上?mAP@0.5 與 mAP@0.5:0.95 有何差別?

   提示與參考解答：對每個類別，調整信心門檻得到精確率-召回率曲線，曲線下面積

為該類的 AP;所有類別的 AP 取平均即 mAP。它建立於第 14 章的精確率與召回率。mAP@0.5 以 IoU 門檻 0.5 判定正確;mAP@0.5:0.95 在多個 IoU 門檻(0.5 到 0.95)下取平均，標準更嚴格，是 COCO 的主要指標。

7. 訓練自訂 YOLO 模型的三個步驟是什麼?YOLO 標註格式的每一行記錄什麼?

提示與參考解答：三步驟：一、準備並標註資料(用 LabelImg/Roboflow 等，依第 14 章切分訓練/驗證/測試集);二、撰寫 data.yaml 設定檔指定路徑與類別名稱;三、呼叫 train() 訓練、val() 計算 mAP。標註格式每行為「類別編號 中心 x 中心 y 寬 高」，座標皆正規化到 0 至 1。

8. 程式 16-4 的訓練參數 degrees、hsv_v、fliplr、patience 分別對應本書哪幾章的觀念?「從預訓練權重開始訓練」又是什麼?

提示與參考解答：degrees(旋轉)、fliplr(水平翻轉)是第 4 章幾何轉換;hsv_v(亮度)是第 5、8 章;三者皆作為資料增強對抗過度擬合(第 14 章)。patience 是第 14 章的及早停止。從預訓練權重開始訓練即第 15 章的遷移學習——借用已學好的通用特徵，少量資料也能訓練出高效能模型。

9. YOLO 的 seg、pose、track 三種模式各輸出什麼?各舉一個應用。

提示與參考解答：seg：輸出像素級遮罩(實例分割)，應用如精確量測不規則物件面積、背景去除。pose：輸出人體十七個關鍵點骨架，應用如動作分析、跌倒偵測、人機協作安全監控。track：每一影格偵測並跨影格關聯維持穩定 ID(第 12 章偵測式追蹤)，應用如人流計數、車流分析、軌跡記錄。

10. AOI 瑕疵偵測用 YOLO 會遇到什麼資料上的挑戰?為什麼第 6 章的傳統方法至今仍有價值?

提示與參考解答：挑戰是資料不平衡：產線上良品遠多於不良品，瑕疵樣本稀少，難以訓練。對策：資料增強、合成瑕疵樣本、或改用只學正常樣貌的異常偵測。第 6 章的頻率域紋理檢測不需要瑕疵樣本(策略是「濾掉正常」)，對「沒見過的新瑕疵」天生有效，故與 YOLO 互補使用。

11. 為什麼人機協作的安全監控不能單靠 YOLO 姿態估計?應如何設計?

提示與參考解答：因為深度學習模型可能誤判(第 15 章：對抗脆弱、分布偏移、黑箱難解)，而人機協作安全攸關人命。正確做法是多層防護：視覺系統作為第一道預

警，搭配實體光柵、壓力感測墊、緊急停止按鈕等獨立的安全機制，形成冗餘。對 AI 保持審慎、設計冗餘，是工程師的基本責任。

12. (思辨題)承叫獸開講：Redmon 因軍事應用與隱私疑慮退出電腦視覺研究。請說明支持與反對的論點，並提出「除了退出之外」工程師還能有哪些負責任的作為。

    提示與參考解答：本題無標準答案，評分重點在論證品質。反對方：技術本身中性，道德在於使用者;你不做別人也會做，退出反而失去影響力。支持方：創造者對創造物的流向並非全然無責;當下行風險遠大於上行效益時，不推進本身即是負責任的表態。其他作為：系統設計內建隱私保護(只統計數量不存人臉、端上處理不上傳)、對資料來源與模型限制保持透明、授權條款明訂禁止用途、部署前評估誤判後果與受影響者、以及在被要求做不對的事時有能力表達疑慮。

13. 為什麼說「只看一次」真正省下來的是重複的特徵萃取?請用滑動視窗的成本估算說明，並指出現代兩階段偵測器與樸素滑動視窗的差別。

    提示與參考解答：樸素滑動視窗要對每一個候選位置各跑一次分類器，而相鄰視窗大量重疊，等於把同一塊影像的特徵重複算了上萬遍。圖 16-3 實測：對一張 810×1080 的照片用三種框、步長 16 掃過，要分類 7,408 次;以實測的每次 7.4 毫秒估算，共需約 55 秒。同一台機器上 YOLO26n 看完整張圖只要約 55 毫秒，差了約 1,000 倍。YOLO 整張圖只萃取一次特徵，再讓 8,400 個預測點共用這份特徵（圖 16-2）。要公平的是：Faster R-CNN 之後的兩階段偵測器同樣只萃取一次特徵，只是多了「提候選區域」這一步，因此它比單階段慢的是倍數而非數量級;圖 16-3 量的是最樸素的做法，用來凸顯重複計算的代價。

14. YOLO26 的一對多預測頭在 640×640 的輸入下會輸出一個 84×8400 的張量，而一對一預測頭輸出的是 (N, 300, 6)。請說明這兩個數字各自的來源，以及它與「多尺度預測」的關係。

    提示與參考解答：84 = 4 + 80，即每個預測點輸出 4 個框座標與 80 個 COCO 類別分數（採用無錨框設計，不再有物件存在與否的獨立分數）。8400 = 80×80 + 40×40 + 20×20 = 6400 + 1600 + 400，對應三個解析度的特徵圖，格子邊長（stride）分別是 8、16、32 個像素。細的格子多、負責小物件，粗的格子大、負責大物件，這就是 16.2.3 節說 v3 引入多尺度預測的實體，也是第 11 章影像金字塔

思想的延續。可以自己驗證：640/8 = 80、640/16 = 40、640/32 = 20，這也說明 imgsz 為什麼必須是 32 的倍數。

15. IoU 0.5 到底有多鬆?請以一個 100×100 的框為例，算出 IoU 等於 0.5 時容許的水平偏移量，以及容許的邊長倍率範圍。這對「mAP@0.5 很高」這句話有什麼啟示?

    提示與參考解答：水平平移 d 時交集是 (100−d)×100、聯集是 (100+d)×100，故 IoU = (100−d)/(100+d);令它等於 0.5 得 d = 100/3 ≈ 33，即偏掉三分之一個身位仍算「正確偵測」（圖 16-5 實測 33.5%）。同中心、邊長比 s > 1 時，交集是小框、聯集是大框，IoU = 1/s²;令它等於 0.5 得 s = √2 ≈ 1.41，縮小側同理是 1/√2 ≈ 0.71。啟示：mAP@0.5 是一個相當寬鬆的標準，它問的是「有沒有找到東西」;要衡量「框畫得準不準」必須看 mAP@0.5:0.95。圖 16-8 實測同一個模型兩者分別是 56.5% 與 41.7%，差了 14.8 個百分點。若應用需要精確位置（機械手臂抓取、尺寸量測），IoU 0.5 的評分根本不能用。

16. NMS 的 IoU 門檻與評分用的 IoU 門檻有什麼不同?若把 NMS 門檻從 0.7 調到 0.3，精確率與召回率會怎麼變?為什麼?

    提示與參考解答：兩者只是剛好同名。NMS 的門檻問「這兩個預測是不是在講同一個物件」，用於後處理去重;評分的門檻問「這個預測跟標準答案夠不夠接近」，用於計算 TP/FP。NMS 門檻調低會刪掉更多框：輸出變少，誤報跟著減少（精確率上升），但真的疊在一起的兩個物件也會被刪掉一個（召回率下降）。圖 16-9 實測：門檻從 0.2 調到 0.95，召回率上升而精確率大幅下降，F1 在 0.5 附近最高，但那個最佳值完全取決於場景有多擠。該怎麼調取決於場景：稀疏場景（產線一次一件）可以調低;密集場景（人群計數、果實計數）必須調高並接受誤報變多。

17. NMS 有一個無解的兩難，是什麼?請舉一個實際場景，並說明有哪些對策。

    提示與參考解答：NMS 分不清「同一個物件的重複框」與「兩個真的疊在一起的物件」。圖 16-9 的實例：保留集裡兩個同類物件的真實框彼此 IoU 就有 0.66，只要 NMS 門檻設在這個值以下，其中一隻必定被當成重複刪掉——而且是在偵測器正確找到它之後才刪的。實際場景：人群計數、貨架補貨、密集果實採收、停車場車位偵測。對策：（1）提高 NMS 門檻並以其他方式過濾誤報;（2）改用 Soft-NMS（把重疊框的分數降低而非直接刪除）;（3）改用不需要 NMS 的架構（DETR 系列以集合預測與匈牙利配對取代後處理）;（4）在應用層容錯，例如只在固定計數線上

判斷。

18. mAP 的 m 是什麼意思?為什麼「mAP 很高」不保證你在乎的那一類表現好?請以本章實測說明。

    提示與參考解答：m 是 mean，指「對所有類別的 AP 取平均」。平均會藏住個別類別的災難。圖 16-7 實測：保留集上有標註的 79 個類別，AP 從 100%（airplane、train 等）到 0%（scissors、mouse）都有，平均起來是 56.5%;其中 person 的 AP 是 0.792、而最差的幾類是 0。如果你的應用真正在乎的剛好是墊底的那一類，那個 56.5% 完全沒有意義。正確做法是看逐類別的 AP 與混淆情形，這與第 14 章「看混淆矩陣而不是看平均」是同一課。另外要注意樣本數：圖 16-18 顯示標註不到四個的類別，AP 多半不是 0 就是 100，那是雜訊而非能力。

19. 為什麼計算 mAP 時信心門檻必須設到極低（例如 0.001），而實際部署時卻該用 0.25 以上?把兩者搞混會發生什麼事?

    提示與參考解答：AP 是精確率-召回率曲線下的面積，而曲線的高召回率端必須由低分預測提供。若先用 0.25 砍掉低分預測，曲線右端就永遠補不回來，AP 會被系統性低估。部署時則相反：低門檻會產生大量誤報，圖 16-11 實測門檻 0.02 時輸出 14,469 個框，而真實物件只有 4,251 個，超過七成是誤報。搞混的後果有兩種：用高門檻算 mAP 會低估自己的模型、並在與別人比較時吃虧;用低門檻部署則會讓下游被垃圾框淹沒。實務建議：驗證腳本與部署設定分開寫，並在程式碼裡註明理由。

20. 提高輸入解析度 imgsz 是對付小物件最有效的手段之一，但本章實測它在某個點之後反而讓總成績變差。請說明這個現象與它的成因，並提出正確的做法。

    提示與參考解答：圖 16-12 實測，imgsz 從 320 加到 1280 時，小物件召回率從 5% 一路升到 36%，但大物件召回率在 640 時最高（77%）、到 1280 掉到 70%;總成績因此呈倒 U 形，mAP@0.5 在 960 達到最高的 58.9%，1280 時反而掉回 53.9%，而耗時卻增加了好幾倍。成因是分布偏移（第 15 章）：預訓練權重是在 640 上訓練的，把輸入放大一倍，所有物件在網路眼中都大了一倍，大物件因此被推出了它熟悉的尺度範圍。正確做法：（1）推論的 imgsz 不要離訓練值太遠;（2）真的需要高解析度，就用高解析度重新訓練，而不是只在推論時放大;（3）把大圖切塊、各自以訓練解析度偵測再合併座標。

21. 標註品質對成績的影響有多大?為什麼 mAP@0.5 與 mAP@0.5:0.95 對標註誤差的敏感度差這麼多?

    提示與參考解答：圖 16-13 的實驗把模型完全固定，只對人工標註加入隨機平移與縮放。抖動 5%（肉眼幾乎看不出）時，mAP@0.5 從 56.5% 只掉到 55.8%，但 mAP@0.5:0.95 從 41.7% 掉到 31.4%，少了 10.3 個百分點;抖動 20% 時兩者分別掉到 23.3% 與 5.7%。敏感度差異的原因：mAP@0.5 只要求 IoU 超過 0.5，而由圖 16-5，偏 5% 的框 IoU 仍有 0.9 以上，輕鬆過關;mAP@0.5:0.95 本來就在量「框準不準」，標註歪了，準的定義就跟著歪。三個實務結論：標註品質是成績的天花板;比較模型必須用同一份標註;驗收標準要跟著標註精度走，標註一致性只有 IoU 0.9 時就不該拿 mAP@0.95 當 KPI。

22. 在 128 張影像上量到 mAP@0.5 = 56.5%。這個數字的誤差範圍大約多少?如果另一個模型量到 59%，你能說它比較好嗎?

    提示與參考解答：圖 16-18 以重抽樣（bootstrap）量得標準差約 2.8 個百分點、95% 區間約 51% 到 62%。59% 完全落在這個區間內，因此不能說它比較好——差距沒有超過雜訊。要做出可信的宣稱有三條路：把測試集加大（圖 16-18 實測 128 張時標準差 2.8、600 張時 1.1，大致隨樣本數的平方根下降）;用交叉驗證或多次重抽樣並報告區間;或改用配對比較（兩個模型在同一批影像上逐張比較），這樣可以消去影像難度造成的共同變異，檢定力高得多。另外提醒：報 mAP 一定要附測試集的影像數與標註框數。

23. (推導題)承 16.7.4 節：某偵測器對作業員的單格漏檢率是 6%。若要求連續 k 格都偵測不到才判定「沒有人」，請算出 k = 1、3、5 時的失效機率與 30 fps 下的延遲，並說明「各格獨立」這個假設為什麼通常不成立。

    提示與參考解答：假設獨立時失效機率是 0.06 的 k 次方：k = 1 為 6×10^{-2}、k = 3 為 2.16×10^{-4}、k = 5 為 7.78×10^{-7}。30 fps 下延遲分別是 33、100、167 毫秒。「獨立」通常不成立，因為造成漏檢的原因多半是持續性的：同一個遮擋、同一個逆光、同一個被 NMS 刪掉的框（圖 16-9）會讓連續好幾格一起漏;因此真實的連續漏檢機率遠高於單格機率的 k 次方。工程對策：把漏檢率估計做在最壞情境（逆光、遮擋、極端姿態）而非平均情境;並且各防護層要用不同的物理原理（光柵、壓力墊、急停按鈕），裝三台相同的相機並不會讓失效率變成三次方。

24. (推導題)請說明全點內插 AP 的「單調化」步驟在做什麼、為什麼要做，並說明它與 2007 年 VOC 十一點內插法的差別。

    提示與參考解答：單調化是把每一點的精確率換成它右邊所有點的最大值，即 $P^*(R) = \max\{P(R') : R' \geq R\}$，讓曲線變成單調不升的階梯，再積分面積。理由是：精確率曲線本身是鋸齒狀的（每多接受一個預測，精確率可能升也可能降），而既然實務上你總能選一個更好的門檻，曲線上的凹陷就不該算進成績。十一點內插法只在召回率 0、0.1、…、1.0 這十一個點取（單調化後的）精確率再平均，是一種粗糙的取樣;全點內插則把每一段召回率的變化都算進去，是目前 COCO 採用的方式。兩者用同一條曲線可以差好幾個百分點——圖 16-7 實測 person 類別：全點 0.782、十一點 0.758。所以比較論文數字時務必確認算法一致。

25. (綜合實作思辨題)你要為一座雲林的文旦果園做產量估算系統：以無人機拍攝、用 YOLO 計數樹上的果實。請規劃完整流程，指出這個任務最可能遇到的四個技術難題與各自對策，並說明你會如何向果農報告「這套系統有多準」。

    提示與參考解答：流程應涵蓋資料（拍攝規劃、按樹/按日期分組切分）、標註（框或遮罩、標註規範與一致性檢查）、訓練（從預訓練權重遷移學習、資料增強用第 4、5 章技術）、評估（逐類別召回率、計數誤差）、部署（第 17 章邊緣裝置）。四個難題與對策：（1）小物件——遠距離拍攝的果實往往不到 32 像素，圖 16-4 實測這個尺寸的召回率只有兩成;對策是降低飛行高度、提高 imgsz（但注意圖 16-12 的倒 U）、或切塊偵測。（2）遮擋與密集——果實彼此重疊且被葉片遮擋，圖 16-17 實測控制大小後重疊愈嚴重召回率愈低;對策是提高 NMS 門檻、多角度拍攝、並接受「可見果實數」與「實際果實數」之間需要一個校正係數。（3）資料洩漏——同一棵樹、同一天的相鄰影像極度相似，隨機切分會讓驗證分數虛高;對策是按樹、按日期分組切分（GroupKFold）。（4）分布偏移——光線、季節、品種、生長階段都會變，訓練時的晴天中午資料無法代表陰天傍晚;對策是跨時段跨天候收集驗收資料，並規劃上線後的持續抽檢與重訓。報告準確度時：不要只報 mAP。果農在乎的是「這棵樹有幾顆」，所以該報的是計數誤差（例如人工清點 30 棵樹作為基準，報平均絕對誤差與其 95% 區間），並明確說明系統的適用條件與已知失效情境。這呼應第 14 章「指標要衡量你真正要的東西」與本章 16.7 節的透明原則。

參考資料與延伸閱讀

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A., You Only Look Once: Unified, Real-Time Object Detection, CVPR, 2016.(YOLO 原始論文)
- Redmon, J., & Farhadi, A., YOLO9000: Better, Faster, Stronger, CVPR, 2017;YOLOv3: An Incremental Improvement, arXiv, 2018.
- Synced Review, YOLO Creator Joseph Redmon Stopped CV Research Due to Ethical Concerns, Feb. 2020.(本章叫獸開講之史料來源)
- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.(第 14 章亦引用)
- Ultralytics YOLO 官方文件：https://docs.ultralytics.com
- Zhang, Y., et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box, ECCV, 2022.
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 17 章 邊緣 AI 與機器人視覺部署

本章學習目標

- 說明邊緣運算的動機，比較端上、近端邊緣、雲端三種 AI 辨識架構的取捨。
- 認識樹莓派 5 加 AI Camera 與 NVIDIA Jetson Orin Nano Super 兩大實作平台的定位與能力差異，並理解「感測器上推論」與「開放詞彙偵測」各自解決什麼問題。
- 運用量化、剪枝、ONNX、TensorRT 等技術最佳化模型以在邊緣裝置上執行。
- 量測與最佳化推論效能，理解延遲、吞吐量與功耗的權衡。
- 將視覺模組整合進 ROS 2 機器人系統，理解節點與話題的通訊架構。
- 完成一個端到端的機器人視覺部署，並評估系統的可靠度與失效模式。

叫獸開講|二十五美元的賭注：樹莓派如何讓「會看的電腦」變成每個人都買得起的東西

2006 年，劍橋大學聖約翰學院的電腦科學課程主任 Eben Upton 遇到一個他想不通的問題。每年來申請電腦科學的學生愈來愈少，而且來的人幾乎都沒寫過程式。這在他那個年代是不可思議的——1988 年，十歲的 Upton 用零用錢買了一台二手的 BBC Micro，自學 BASIC、自己寫遊戲;那個年代的家用電腦一開機就是程式提示符號，你想玩什麼，常常得自己先把它寫出來。二十年後，年輕人用電腦的時間比以前多得多，卻幾乎沒有人在「對電腦說話」。Upton 的診斷很具體：不是這一代比較笨，而是家裡那台電腦太貴、太重要了——它是爸媽報稅、哥哥姊姊交報告的機器，你怎麼敢拿它亂搞?

他的對策也同樣具體：做一台便宜到弄壞也不心疼的電腦。最早的原型在他的辦公桌上誕生，用的是一顆小小的微控制器，連接電視與鍵盤。接下來幾年，他與幾位同事一邊在 Broadcom 上班、一邊利用晚上與週末把它做出來，並成立了一個慈善基金會，把賣機器的利潤全部投回教育。2012 年 2 月 29 日，樹莓派正式開賣。Upton 原本希望能賣出一千台;他後來說，「一萬台對我們來說就已經是很棒的結果了」。結果開賣當天訂單就把兩家經銷商的網站打掛，到 2015 年初，銷量已超過五百萬台。

但真正和本課程有關的，是接下來發生的事。當一台能跑 Linux 的電腦只要二十五美元，它最先被拿去做的事情之一，就是「看」。2013 年，一位英國的業餘愛好者 Dave Akerman 用樹莓派控制一台相機，綁在高空氣球上送到三萬公尺的高空——連同基金會的吉祥物泰迪熊 Babbage 一起，在平流層拍下照片、再讓它跳傘回到地面。他說得很直白：樹莓派提供了一個便宜的方法，去控制一台畫質不錯的相機，並把影像與遙測資料傳回來。2015 年，兩台裝著相機的樹莓派被送上國際太空站;英國太空人 Tim Peake 在軌道上執行的，是中小學生自己寫的程式——有的量測艙內環境，有的拍攝地球並分析影像。在那之前，「把影像處理程式送上太空」是國家級機構的事;在那之後，它變成一份作業。

這條線一路延伸到 2024 年，而且拐了一個很有意思的彎。樹莓派推出 AI Camera，核心是 Sony 的 IMX500 智慧視覺感光元件——它把神經網路的推論做在感測器晶片裡。請注意這句話的份量：影像不必先傳給處理器，推論就已經完成了，主機拿到的是「畫面裡有什麼」，而不是「畫面」。從「便宜到能讓學生控制相機」，走到「便宜到讓相機自己會看」，中間隔了十二年。而本章要處理的，正是這十二年累積下來的工程問題：當你要把前面十六章的演算法，從有插電、有散熱、有充裕記憶體的筆電，搬到一顆靠電池、會發熱、記憶體以 MB 計的裝置上時，會發生什麼事。

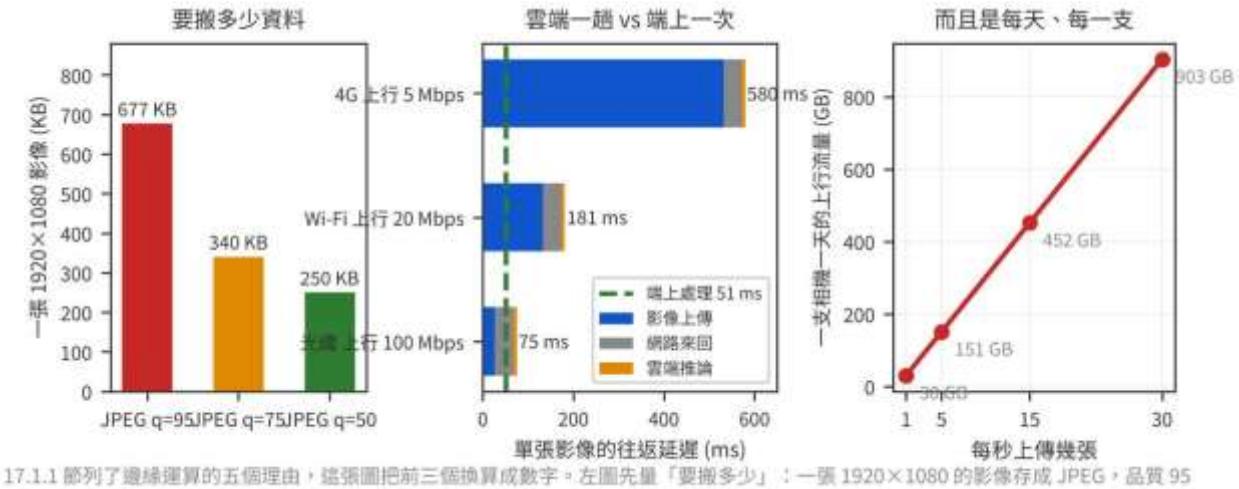
思考題：Upton 認為年輕人不寫程式的原因，不是能力問題，而是「家裡那台電腦太貴重，沒人敢亂搞」。回想你自己學得最快的一次經驗，是不是也發生在一個

「搞壞了也沒關係」的環境裡?這對你設計自己的學習方式——以及第 18 章的期末專題——有什麼啟發?

17.1 為什麼需要邊緣運算

17.1.1 從雲端回到現場

在深度學習興起的頭幾年，主流想法是「把運算放到雲端」：裝置只負責拍照，把影像傳到雲端伺服器，由強大的 GPU 完成辨識，再把結果傳回來。這個架構有明顯的優點——裝置可以很便宜、模型可以很大、更新也方便。但用在機器人與工業現場，它很快就撞牆了。



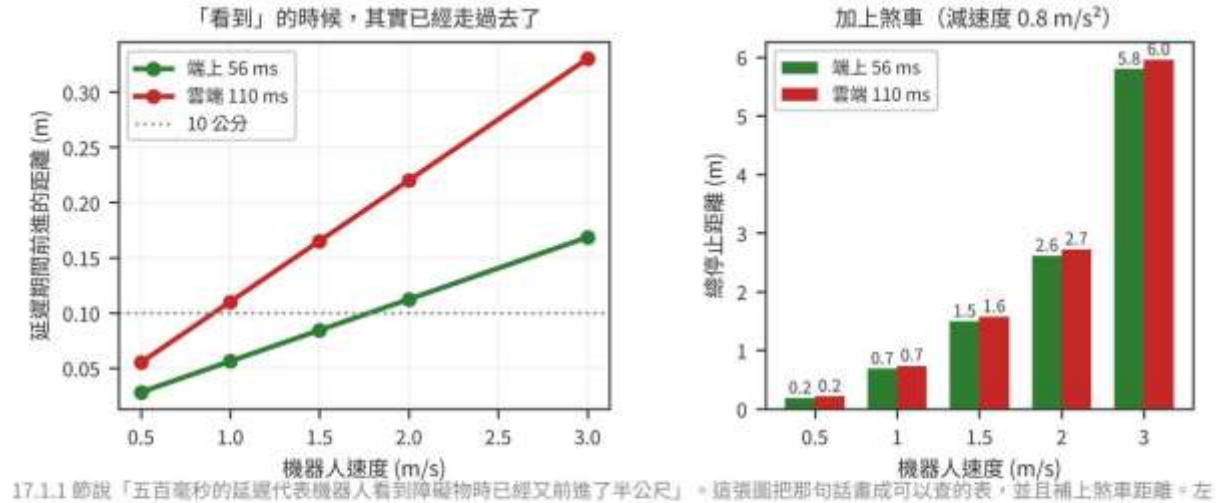
17.1.1 節列了邊緣運算的五個理由，這張圖把前三個換算成數字。左圖先量「要搬多少」：一張 1920×1080 的影像存成 JPEG，品質 95 時 677 KB、品質 75 時 340 KB、品質 50 時 250 KB。壓得更狠當然更省，但第 11 章量過壓縮會吃掉細節，而第 16 章的圖 16-13 又告訴我們細節正是小物件的命脈——這條路並不能無限壓下去。中圖是延遲的帳。以品質 75 的 340 KB 為例，在 4G 上行 5 Mbps 的條件下光是上傳就要 532 毫秒，加上約 40 毫秒的網路來回與 8 毫秒的雲端推論，一趟是 580 毫秒;就算換成 100 Mbps 的光纖，也還要 75 毫秒，因為那 40 毫秒的來回延遲與頻寬無關、壓不下去。綠色虛線是同一張影像在本機用 ONNX 跑 YOLO26n 的實測延遲 51 毫秒——注意它比最好的雲端情境還低，而且不會因為網路塞車而變成十倍。右圖是頻寬與成本：一支相機以 15 fps 持續上傳，一天就是 452 GB;一條產線三十支相機，一個月就是數十 TB。這三張圖合起來說明一件事：把運算搬回現場，買到的不只是速度，還有「不受網路狀況左右」這個性質。至於隱私與可靠性，它們無法畫成長條圖，但道理一樣直接——沒有上傳的影像，就不會外洩;不依賴網路的系統，斷網時仍然看得見。(本圖的頻寬與伺服器時間是常見的假設值，影像大小與端上延遲則是實測;換到你的場域請重新量一次。)

圖 17-1　雲端來回一趟，到底花掉什麼

邊緣運算(Edge Computing)的核心主張是：把運算搬回資料產生的現場。理由有五，每一個都攸關機器人視覺的成敗：

1. 延遲(Latency)：影像上傳、雲端運算、結果回傳，一來一回動輒數百毫秒。對一台以每秒一公尺行進的機器人，五百毫秒的延遲代表它「看到」障礙物時已

經又前進了半公尺——這在避障、人機協作安全上是不可接受的。這裡請記住一個很好用的換算：延遲乘上速度，就是「資訊過期的距離」。圖 17-2 把這條關係畫了出來，而它還帶出一個比延遲更重要的事實——真正的停止距離是反應距離加上煞車距離，前者與速度成一次方、後者與速度成平方。實測以每秒一公尺、減速度 0.8 m/s² 計算：反應距離只有 0.15 公尺，煞車距離卻有 0.62 公尺。所以在低速時延遲是主角，到了高速則是煞車說話;而「把速度減半」往往是整個系統裡最便宜也最有效的旋鈕(推導 17.4.3 會把這條預算完整算一遍)。



17.1.1 節說「五百毫秒的延遲代表機器人看到障礙物時已經又前進了半公尺」。這張圖把那句話畫成可以查的表，並且補上煞車距離。左圖是純粹的感知延遲：以每秒一公尺行進時，端上處理的 56 毫秒等於 6 公分，而雲端來回的 110 毫秒等於 11 公分。速度拉到每秒三公尺（相當於快走到小跑），雲端架構光是在等結果就滑行了 33 公分。右圖加上物理：停下來還需要煞車距離，以 0.8 m/s² 的減速度估算，速度的平方才是主角。每秒一公尺時總停止距離是 0.68 公尺（端上）對 0.73 公尺（雲端）;到了每秒三公尺，兩者是 5.79 與 5.96 公尺。這張圖要傳達三件事。其一，延遲不是抽象的數字，它可以直接換算成公分，而公分是會撞到東西的單位。其二，感知延遲與速度成正比、煞車距離與速度平方成正比，所以高速時真正吃掉距離的是煞車——這表示一味壓低模型延遲會有報酬遞減，限制速度往往比換更快的模型有效得多。其三，反過來說，這張圖也給了你一個設計工具：先訂下「要在多遠之前停住」，再回推可容許的總延遲，這就是 17.4.2 節的延遲預算（圖 17-13 會把整條鏈算完）。

圖 17-2　延遲換算成距離：機器人在反應時間裡走了多遠

2. 頻寬與成本：一路 1080p 30fps 的影像串流，持續上傳的頻寬與雲端運算費用相當可觀;若是一整條產線數十支攝影機，成本更是驚人。
3. 可靠性：工廠、田間、隧道、災難現場往往網路不穩或根本沒有網路。一個「斷網就瞎掉」的機器人，是不可靠的機器人。

4. 隱私：影像是高度敏感的資料。承第 16 章的倫理討論，把人臉影像上傳雲端，與在裝置上處理完只回報「有幾個人」，對隱私的意涵天差地別。端上處理是最有力的隱私保護設計之一。
5. 功耗與自主性：電池供電的機器人與無人機，無法負擔持續高功率的無線傳輸;而自主系統的本質，就是不依賴外部連線也能運作。

17.1.2 三種 AI 辨識架構

這正是第 2 章就介紹過、並在第 10、16 章反覆呼應的三種架構，現在可以完整地整合起來：

表 17-1 三種 AI 辨識架構的比較

關鍵觀念是：這三者不是互斥的選擇，而是可以分層協作的。承第 10.7.3 節的門禁案例：端上用輕量的方式判斷「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把那一瞬間的影像交給後端做「是誰」的精細辨識(低頻、高算力)。這種「端上預篩、後端精算」的分層設計，兼顧了即時性、成本與隱私，是實務上最常見的架構模式。值得注意的是，這條分界線正在往前移動：17.2.1 節的 AI Camera 把第一層的推論直接做進感測器晶片裡，於是「端上預篩」不再需要一顆處理器來做——主機拿到的已經是結果，而不是影像。

17.2 兩大實作平台：樹莓派 5 與 Jetson Orin Nano Super

本書的實作橫跨三個層級，本節正式比較其中兩套的定位——這也是您在課程中希望學生建立的核心判斷力：選對工具，比用最強的工具更重要。這兩個平台剛好代表邊緣 AI 的兩種截然不同的思路：一個把推論推到最前端（感測器裡），另一個把算力集中在機器人身上;它們不是高階與低階的關係，而是兩種不同的系統設計。

17.2.1 樹莓派 5 與 AI Camera：把推論搬進感測器裡

承叫獸開講，樹莓派 5 是一台完整的 Linux 單板電腦（本章以 8 GB 記憶體的版本

為例），能跑 Python、OpenCV、PyTorch 與本書前面所有的程式。真正特別的是搭配它的 AI Camera 模組。它的核心是 Sony IMX500 智慧視覺感光元件：一顆 1,230 萬畫素（4056×3040）的 CMOS 感測器，把神經網路的推論加速器做在同一顆晶片上。換句話說，你不需要外掛任何加速棒或加速卡，感測器自己就會跑模型。

它的運作方式值得仔細看一次，因為它和本書前面所有的做法都不同。傳統的流程是：感測器輸出影像 → 主機解碼與前處理 → 主機跑模型 → 得到結果。IMX500 的流程是：感測器輸出影像，同時附上一包「張量後設資料」（tensor metadata），裡面就是模型的輸出——偵測框、類別、分數。主機只要讀那包後設資料就好。libcamera、Picamera2 與 rpicam-apps 都已經支援這個欄位，所以在程式裡取用它並不複雜。兩條路徑的差別因此不只是快慢，而是「主機要不要參與」：傳統路徑上，解碼與推論這兩件事都得由主機親自做;感測器路徑上，主機只是讀一包很小的後設資料，處理器因此空下來，可以拿去做導航、控制或使用者介面。

這個設計買到的是三樣東西，而且都直接對應 17.1.1 節的五個理由。一是主機的時間：樹莓派的處理器可以拿去跑導航、控制或使用者介面，不必被視覺吃掉。二是頻寬與功耗：影像不必傳出去，連解碼都省了。三是隱私：這是最有力的一種隱私設計——不是承諾不看，而是影像根本沒有離開感測器（承第 16 章的倫理討論與 17.1.1 節）。

它的限制同樣明確，而且全部來自同一件事。感測器晶片上的記憶體額度很小，而且這個額度要同時容納韌體、網路權重與推論時的工作記憶體。由此衍生出三條硬規定：輸入張量最大 640×640;輸入資料型別只收 8 位元整數（int8 或 uint8）——連 FP16 都不支援;能跑的模型清單很短，目前官方工具鏈支援的是 YOLOv8n 與 YOLO11n 這一級，本書第 16 章用的 YOLO26 還不在清單上。換句話說，INT8 在這裡不是一個可以權衡的選項，而是必要條件（這也讓 17.3.1 節與推導 17.3.4 的那筆帳變得具體）。

部署的流程因此多了一步「轉檔」。你在電腦上訓練好模型之後，要用官方的轉換工具鏈把它轉成感測器看得懂的格式，過程中會自動套用訓練後量化（PTQ，見 17.3.1 節與推導 17.3.4）。以 Ultralytics 為例，匯出與部署大致是這樣：

```
# 在電腦上：匯出成 IMX500 的部署格式（會自動做 INT8 量化）
from ultralytics import YOLO
model = YOLO("yolo11n.pt")          # 注意：目前支援到 YOLO11n 這一級
model.export(format="imx", data="coco8.yaml")   # 產生部署用的封裝檔

# 在樹莓派上：安裝韌體與工具
#   sudo apt update && sudo apt full-upgrade
#   sudo apt install imx500-all
#   sudo reboot
# 之後以 Picamera2 讀取影像與「張量後設資料」，
# 主機端不需要再跑一次模型。
```

最後是一個容易被誤解的地方，必須說清楚。感測器上推論並不比較快。感測器上的加速器是為低功耗與小面積設計的，不是為了極速;它跑同一個模型的速度，和一台普通電腦的處理器並沒有拉開差距。它買到的不是速度，是「主機的時間」與「不必搬影像」。所以它最適合的場景是：主機還有別的正事要做，或者主機大部分時間應該在睡覺（圖 17-4 的功耗帳）。若你的系統只有視覺這一件事、而且主機閒著，那麼它帶來的好處就小得多——這又是一次「適合勝於最強」。

17.2.2 Jetson Orin Nano Super：機器人的視覺大腦

Jetson 系列是 NVIDIA 專為邊緣 AI 設計的模組化電腦。本章以 Jetson Orin Nano Super 開發套件為例：它採用 Ampere 架構的 GPU，有 1024 個 CUDA 核心與 32 個張量核心，AI 效能約 67 TOPS，而功耗可以在 7 瓦到 25 瓦之間設定。它跑的是 Linux，支援 CUDA、TensorRT、PyTorch、ROS 2 等完整生態系——本質上是一台「能塞進機器人裡的小型 AI 工作站」。請特別注意那個可調的功耗範圍：它不只是省電的選項，更是一個設計旋鈕，讓同一塊板子能從「靠電池的無人機」一路用到「有插電的檢測站」（圖 17-4 會把它換算成續航時間）。

對機器人視覺而言，Jetson 的意義在於它讓第 15、16 章的深度學習模型能真正「上車」。YOLO 即時偵測、語意分割、姿態估計、視覺 SLAM——這些過去只能在

桌機上跑的任務，現在能在機器人身上以合理的功耗即時執行。呼應第 15 章叫獸開講：當年 AlexNet 證明了 GPU 能訓練深度網路，而 Jetson 把同一套東西變成了機器人身上的一顆心臟。

這一級的算力解鎖了一類新的模型，值得在這裡特別介紹，因為它改變的不只是速度，而是「你能問什麼問題」。本書到第 16 章為止的偵測器，類別都是在訓練時就固定的：COCO 的那八十類，多一類都不行，要加就得重新標註與訓練。開放詞彙偵測（open-vocabulary detection）把這件事翻轉過來——類別變成一段文字，執行時才決定。它的作法是把影像與文字投影到同一個向量空間，比對兩者的相似度;這類模型的代表是 OWL-ViT，而 NVIDIA 的 NanoOWL 專案把它用 TensorRT 最佳化，讓它能在 Jetson Orin 上即時執行。

NanoOWL 還提供一個有趣的「樹狀提示」語法，可以把偵測與分類巢狀地組合起來。例如寫 [a face [a nose, an eye, a mouth]]，它會先找出每一張臉，再在每一張臉的區域裡找鼻子、眼睛與嘴巴;而寫 (indoors, outdoors) 則是把整張影像分類成室內或室外。對機器人來說，這代表你可以在現場用一句話改變它要找的東西，而不必回實驗室重訓一輪。圖 17-3 以本機可執行的開放詞彙模型示範了這件事——同一張照片，只是換了幾個字，找到的東西就完全不同。



圖 17-3　開放詞彙偵測：換一句話，就換一組類別

```
# NanoOWL 的典型用法（需在 Jetson 上，並先建好 TensorRT 引擎）
from nanoowl.owl_predictor import OwlPredictor

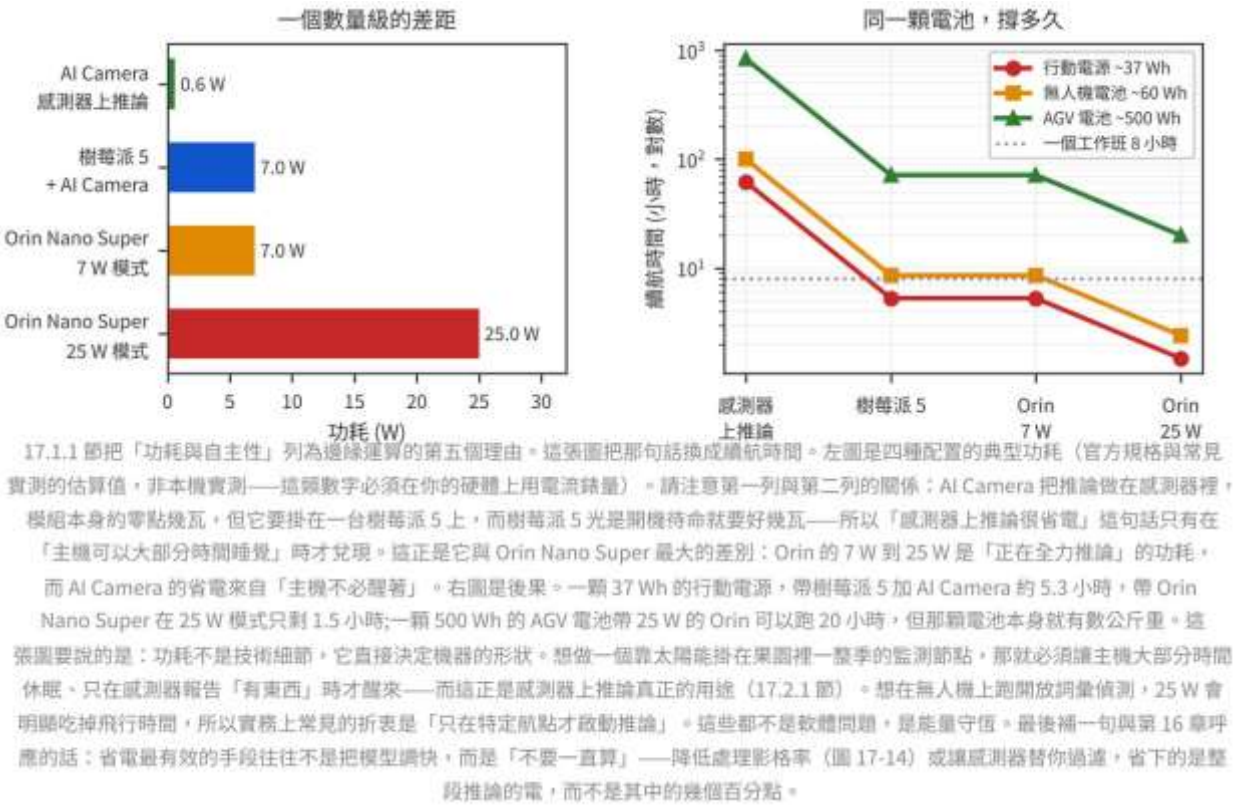
predictor = OwlPredictor(
    "google/owlvit-base-patch32",
    image_encoder_engine="data/owl_image_encoder_patch32.engine")

output = predictor.predict(image=image,
                           text=["a person", "a helmet"],
                           threshold=0.1)

# 樹狀提示（先找臉，再在臉裡找五官）：
#   --prompt="[a face [a nose, an eye, a mouth]]"
```

但開放詞彙不是免費的，而且代價比多數人想的大。它有三筆代價。第一筆最要命：它比對的是文字產生的向量，不是你心裡的概念——單數與複數、加不加冠詞、換一個同義詞，結果都可能天差地別（圖 17-3 的右圖就是一個例子）。這代表提示詞本身就是一個必須在開發集上調的參數。第二筆是速度：它比同級的封閉詞彙模型重，因為每次推論都要同時處理影像與文字兩邊。第三筆是精度：它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。所以它的正確用法是：在專題前期用它快速試出「這個場景裡到底要找什麼」，等類別定下來之後，再訓練一個封閉詞彙的模型來真正上線。

選擇建議：若你的任務是固定的、類別已知、而且希望主機盡量閒著（環境監測、門禁、產線上的單一檢項），樹莓派 5 加 AI Camera 是很划算的選擇——它便宜、省電、影像不出裝置。若你需要即時的深度學習推論、SLAM、多感測器融合，或者需要開放詞彙這類較重的模型，那就得用 Jetson Orin Nano Super 這一級的算力。兩者也常搭配使用：多個 AI Camera 節點分佈在環境中做第一層過濾，Jetson 作為集中處理與決策的大腦——這正是 17.1.2 節三層架構的實體。



17.1.1 節把「功耗與自主性」列為邊緣運算的第五個理由。這張圖把那句話換成續航時間。左圖是四種配置的典型功耗（官方規格與常見實測的估算值，非本機實測——這類數字必須在你的硬體上用電流錶量）。請注意第一列與第二列的關係：AI Camera 把推論做在感測器裡，模組本身的零點幾瓦，但它要掛在一台樹莓派 5 上，而樹莓派 5 光是開機待命就要好幾瓦——所以「感測器上推論很省電」這句話只有在「主機可以大部分時間睡覺」時才兌現。這正是它與 Orin Nano Super 最大的差別：Orin 的 7 W 到 25 W 是「正在全力推論」的功耗，而 AI Camera 的省電來自「主機不必醒著」。右圖是後果。一顆 37 Wh 的行動電源，帶樹莓派 5 加 AI Camera 約 5.3 小時，帶 Orin Nano Super 在 25 W 模式只剩 1.5 小時;一顆 500 Wh 的 AGV 電池帶 25 W 的 Orin 可以跑 20 小時，但那顆電池本身就有數公斤重。這張圖要說的是：功耗不是技術細節，它直接決定機器的形狀。想做一個靠太陽能掛在果園裡一整季的監測節點，那就必須讓主機大部分時間休眠、只在感測器報告「有東西」時才醒來——而這正是感測器上推論真正的用途（17.2.1 節）。想在無人機上跑開放詞彙偵測，25 W 會明顯吃掉飛行時間，所以實務上常見的折衷是「只在特定航點才啟動推論」。這些都不是軟體問題，是能量守恆。最後補一句與第 16 章呼應的話：省電最有效的手段往往不是把模型調快，而是「不要一直算」——降低處理影格率（圖 17-14）或讓感測器替你過濾，省下的是整段推論的電，而不是其中的幾個百分點。

圖 17-4　功耗與續航：自主性的算術

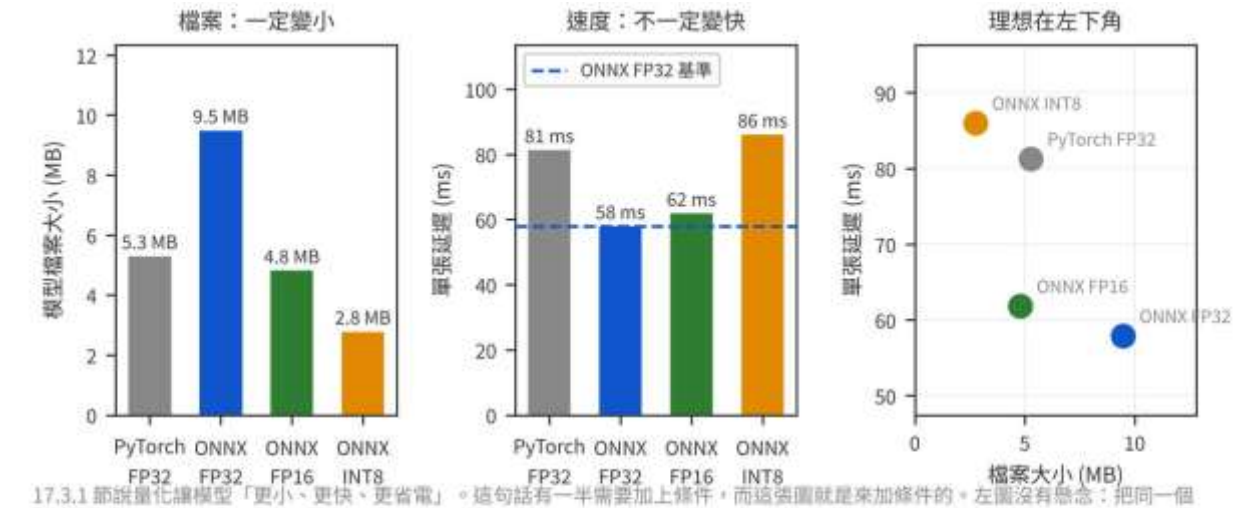
17.3 模型最佳化：讓大模型跑在小裝置上

直接把訓練好的模型丟到邊緣裝置，往往跑不動或跑太慢。模型最佳化是邊緣部署的核心技術，目標是在可接受的精度損失下，大幅降低運算量、記憶體與功耗。

17.3.1 量化

量化(Quantization)是最有效的手段：訓練時模型的權重與運算通常用 32 位元浮點數(FP32)，但推論時未必需要這麼高的精度。把它降到 16 位元浮點(FP16)或 8 位元

整數(INT8)，模型大小可縮為二分之一到四分之一，而精度往往只損失一點點。不過「速度」這一項必須說得更小心，因為它不一定兌現。量化之所以可能加速，靠的是兩條路徑：一是權重少搬一半，若這一層原本被記憶體速度卡住，搬得快就真的算得快;二是硬體有原生的低精度乘加指令(Jetson 的 Tensor Core 對 FP16、許多 NPU 與 DSP 對 INT8)，同一個週期能做更多次乘加。這兩條都要硬體配合，而第二條更挑。圖 17-5 在一顆沒有這些單元的 CPU 上實測：ONNX FP32 是 9.48 MB、33.5 毫秒，FP16 縮到 4.81 MB 但變成 42.6 毫秒，INT8 縮到 2.77 MB 卻慢到 58.7 毫秒——體積一路變小，速度一路變慢。原因是處理器沒有半精度單元時必須把 FP16 轉回 FP32 才能算，而動態 INT8 還要在執行期間統計激活值範圍、並在整數與浮點之間來回轉換，這些額外動作吃掉了省下來的乘法時間。精度那一面則是 FP16 幾乎免費、INT8 要付代價：圖 17-6 實測 mAP@0.5 從 61.61% 降到 59.19%。所以量化該拆成三個獨立的問題來看——裝不裝得下(一定有幫助)、掉多少精度(可以靠靜態量化或量化感知訓練救回)、會不會變快(只有在目標硬體上量過才知道)，推導 17.3.4 會把這三件事的算式各寫一遍。這個「用精度換效率」的思想，你應該覺得眼熟——它正是第 3 章量化(把連續的光強度離散化)與第 11 章 JPEG 量化(把 DCT 係數粗略化)的同一個核心概念，在模型層次的再現。本書反覆出現的觀念，再一次以新的面貌登場。

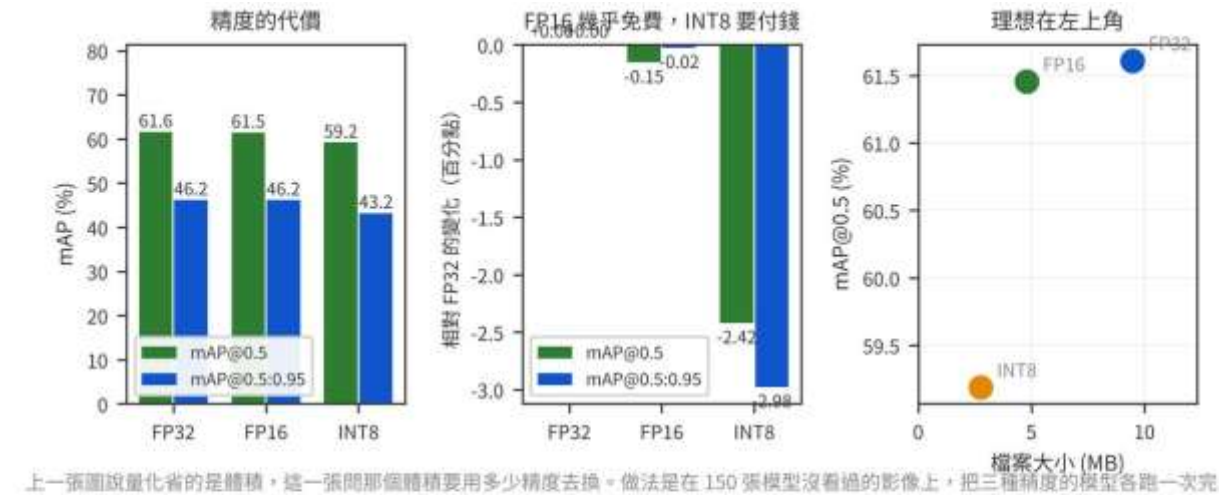


17.3.1 節說量化讓模型「更小、更快、更省電」。這句話有一半需要加上條件，而這張圖就是來加條件的。左圖沒有懸念：把同一個 YOLO26n 從 FP32 降到 FP16 再降到 INT8，檔案從 9.5 MB 變成 4.8 MB 再變成 2.8 MB，大約就是位元數的比例——這部分與第 3 章的影像量化、第 11 章的 JPEG 量化是同一個算術。中圖才是重點。在這台純 CPU 的機器上，FP32 的 ONNX 跑 58 毫秒，FP16 反而變成 62 毫秒，INT8 更慢到 86 毫秒——比沒量化還慢。原因不難理解：這顆 CPU 沒有原生的 FP16 運算單元，推論引擎得先把半精度轉回單精度才算得動，轉換本身就是成本;而動態 INT8 量化要在執行時即時決定縮放係數、並在整數與浮點之間來回搬運。省下的乘法時間被這些額外動作吃光。所以量化真正換到的是什麼?是「體積」與「記憶體頻寬」——這兩項在小裝置上往往才是瓶頸，而且它決定了模型裝不裝得進去。至於速度，要等硬體原生支援那個精度才兌現：Jetson 的 Tensor Core 對 FP16、微控制器與 NPU 對 INT8，都有專門的運算單元，那裡的加速是實打實的。這帶出本章最該記住的工程紀律：任何最佳化都必須在「目標硬體」上量過才算數。在筆電上量到的結論，換到 Jetson 或 IMX500 這類感測器可能完全相反。

圖 17-5　量化：檔案一定變小，速度不一定變快

17.3.2 剪枝與知識蒸餾

剪枝(Pruning)把網路中「貢獻很小」的權重或整個通道移除，讓模型變稀疏、變小。知識蒸餾(Knowledge Distillation)則訓練一個小模型(學生)去模仿大模型(老師)的輸出，讓小模型繼承大模型的能力。兩者都是「把模型變小但盡量保住能力」的策略，常與量化搭配使用。



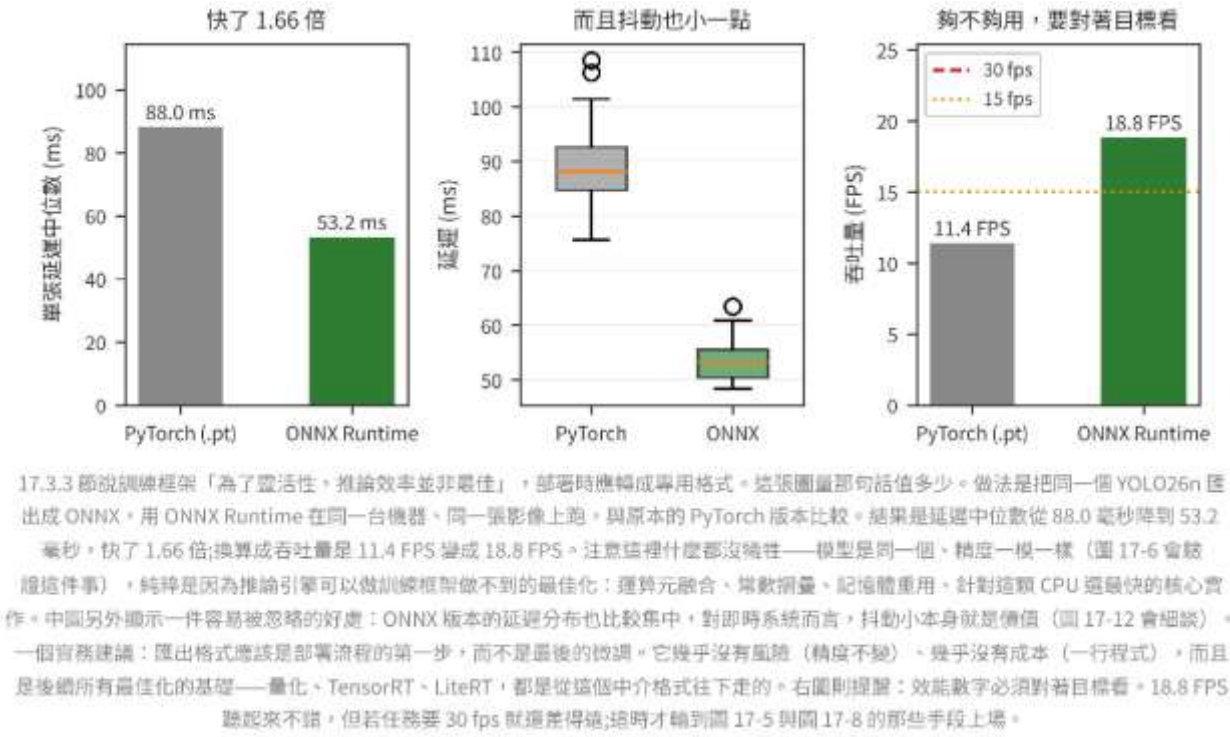
上一張圖說量化省的是體積，這一張問那個體積要用多少精度去換。做法是在 150 張模型沒看過的影像上，把三種精度的模型各跑一次完整評估。FP16 的結果幾乎可以說是免費的：mAP@0.5 從 61.61% 變成 61.45%，mAP@0.5:0.95 從 46.19% 變成 46.17%，兩項都在小數點後第二位打轉，而檔案少了一半。這很合理——半精度浮點仍有十位元的有效位數，而神經網路的權重本來就分布在一個不大的範圍內。INT8 就不一樣了：mAP@0.5 掉到 59.19%（少 2.42 個百分點），mAP@0.5:0.95 掉到 43.22%（少 2.98 個百分點），換來的是檔案只剩 2.8 MB。要不要付這筆錢，取決於應用。八位元整數只有 256 個刻度，要把一層的權重與激活值都塞進這 256 格，靠的是為每一層挑一個縮放係數——挑得好不好，直接決定損失多少。本圖用的是最省事的「動態量化」，只量化權重;實務上若改用「訓練後靜態量化」，拿一小批代表性影像先跑一遍、統計每一層激活值的範圍再定縮放，精度通常可以救回大半。更講究的還有「量化感知訓練」，在訓練時就模擬量化誤差。所以 INT8 掉三個百分點不是定論，是「不花力氣時」的結果。最後提醒一句方法論：這些數字必須在模型沒看過的資料上量（第 16 章圖 16-18），否則量化的損失會被記憶效應蓋過去。

圖 17-6　量化的精度代價

17.3.3 推論引擎：ONNX 與 TensorRT

訓練用的框架(PyTorch、TensorFlow)為了靈活性，推論效率並非最佳。部署時通常會轉換成專用的推論格式：ONNX 是一個開放的中介格式，讓模型能在不同框架與硬體間流通(第 16 章程式 16-4 的 export 就是在做這件事);TensorRT 則是 NVIDIA 針對自家 GPU 的推論最佳化引擎，它會分析模型結構、融合運算層、選擇最佳核心、並套用量化，在 Jetson 上通常能帶來數倍的加速。對更受限的微控制器，則有 TensorFlow Lite for Microcontrollers 等專用的輕量執行環境;而 17.2.1 節的 IMX500 走的是另一條路——它有自己的轉換工具鏈，把模型編成感測器韌體看得懂的格式。值得特別強調的是：單純換成推論引擎，是本章最划算的一招。圖 17-7 實測同一顆 YOLO26n、同一張影像、同一台機器：PyTorch 的 .pt 中位 55.11 毫秒，ONNX Runtime 中位 32.82 毫秒，快了約 1.68 倍——而且精度一模一樣(圖 17-6)。加速來自三件事：把卷積與批次正規化這類推論時固定的線性變換事先融合成單一運算、拿掉自動微分為反向傳播保留的全部紀錄、以及把模型編成靜態圖而不必每一層都回到

Python 問下一步做什麼。代價是失去靈活性——靜態圖不能隨意改結構、除錯也麻煩;但部署時模型已經定案，這個代價等於不存在。換句話說，在開始調解析度、換小模型、或嘗試量化之前，先把模型匯出成部署格式量一次，因為這一步不花精度就能拿到的加速，往後幾步都未必拿得到。



17.3.3 節說訓練框架「為了靈活性、推論效率並非最佳」，部署時應轉成專用格式。這張圖量那句話值多少。做法是把同一個 YOLO26n 匯出成 ONNX，用 ONNX Runtime 在同一台機器、同一張影像上跑，與原本的 PyTorch 版本比較。結果是延遲中位數從 88.0 毫秒降到 53.2 毫秒，快了 1.66 倍;換算成吞吐量是 11.4 FPS 變成 18.8 FPS。注意這裡什麼都沒換——模型是同一個、精度一模一樣（圖 17-6 會驗證這件事），純粹是因為推論引擎可以做訓練框架做不到的最佳化：運算元融合、常數摺疊、記憶體重用、針對這顆 CPU 選最快的核心實作。中圖另外顯示一件容易被忽略的好處：ONNX 版本的延遲分布也比較集中，對即時系統而言，抖動小本身就是價值（圖 17-12 會細談）。一個實務建議：匯出格式應該是部署流程的第一步，而不是最後的微調。它幾乎沒有風險（精度不變）、幾乎沒有成本（一行程式），而且是後續所有最佳化的基礎——量化、TensorRT、LiteRT，都是從這個中介格式往下走的。右圖則提醒：效能數字必須對著目標看。18.8 FPS 聽起來不錯，但若任務要 30 fps 就還差得遠;這時才輪到圖 17-5 與圖 17-8 的那些手段上場。

圖 17-7　換一個推論引擎，就快了一截

架構層面的選擇同樣重要：與其把大模型硬壓小，不如一開始就選用為邊緣設計的架構——第 15 章的 MobileNet、第 16 章 YOLO 的 nano 版本，都是為此而生。這呼應了第 9、10、13 章反覆強調的判準：適合勝於最強。

程式 17-1 模型匯出與效能量測

```
from ultralytics import YOLO
import time, cv2

model = YOLO("yolo26n.pt")        # nano:為邊緣裝置設計

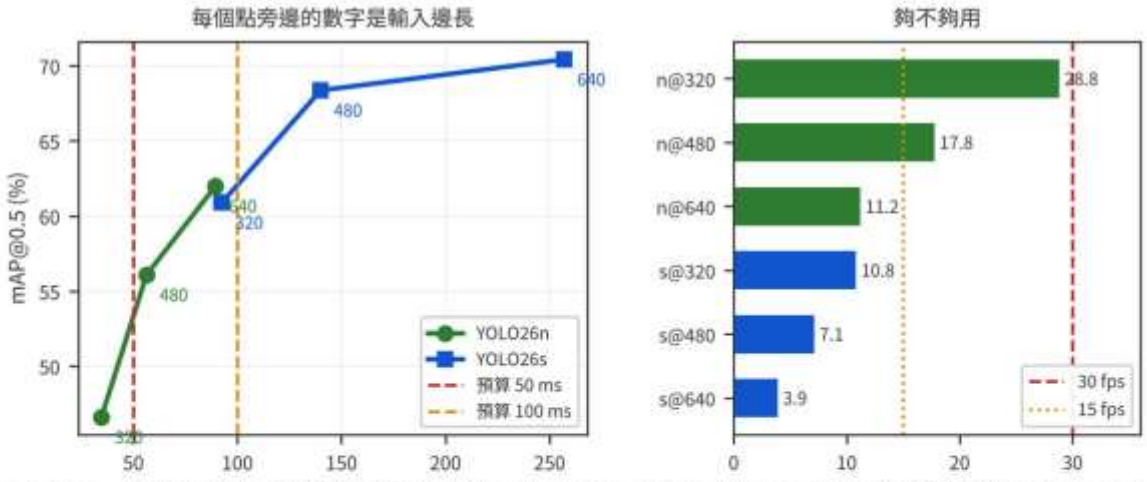
# --- 匯出為部署格式 ---
model.export(format="onnx")                    # 通用中介格式
# model.export(format="engine", half=True)     # TensorRT + FP16(Jetson)
# model.export(format="tflite", int8=True)     # TFLite + INT8(行動/微控)

# --- 量測推論效能(部署前必做) ---
img = cv2.imread("test.jpg")
```

```
model(img)                                  # 暖機一次(第一次含初始化)

N = 50
t0 = time.time()
for _ in range(N):
    model(img, verbose=False)
dt = (time.time() - t0) / N
print(f"平均延遲:{dt*1000:.1f} ms,吞吐量:{1/dt:.1f} FPS")

# 效能不足時的調整順序:
#  1. 換更小的模型(n < s < m < l < x)
#  2. 降低輸入解析度(imgsz=640 → 416 → 320)
#  3. 量化(FP16 / INT8)
#  4. 用 TensorRT 等推論引擎
#  5. 降低處理影格率(不必每一影格都偵測)
```



程式 17-1 給了一份最佳化的順序：先換更小的模型，再降解析度。但這兩招其實是在同一個預算裡互相競爭——前面的章節已經過才知道。這張圖把兩個模型 × 三種輸入邊長共六種組合全部量一遍（延遲在本機實測，mAP 在 150 張保留影像上實測），畫成一條「延遲對準確度」的取捨曲線。關鍵的對照在中段。「大模型配小輸入」的 YOLO26s@320 是 92 毫秒、60.9%；「小模型配大輸入」的 YOLO26n@640 是 89 毫秒、62.0%。兩者差 3 毫秒、1.1 個百分點——幾乎疊在同一個點上。也就是說，在這個預算帶裡，程式 17-1 建議的那兩招（換小模型、降解析度）最後到達的位置差不多，沒有哪一招天生比較划算。往兩端走才有明顯的差異：預算壓到 20 毫秒以內時只剩 YOLO26n@320 可選（46.6%），而預算放寬到 90 毫秒以上時，YOLO26s@480 的 68.4% 把所有 n 系列的組合都甩開。所以沒有一條通用的規則，只有一條你自己量出來的曲線。這也是為什麼程式 17-1 給的是「嘗試的順序」而不是「正確的答案」。實務上的做法是：把這張圖在目標硬體上重畫一次，在圖上拉一條垂直線代表你的延遲預算，取這條線左邊最高的那個點。順帶一提，左圖也解釋了為什麼第 16 章圖 16-12 的解析度曲線會有倒 U：輸入放大到超過訓練尺度之後，準確度不升反降，而延遲還是照樣增加——那是整張圖上最不划算的區域。

圖 17-8　同一個延遲預算，該換小模型還是降解析度

17.3.4 推導：量化在算什麼，以及它為什麼不一定變快

17.3.1 節說量化讓模型「更小、更快、更省電」。這一小節把那句話拆成三件事來算，因為它們兌現的條件並不相同——圖 17-5 的實測就出現了「檔案變小、速度反而變慢」的情況，而理解下面這幾行算術，就知道那不是意外。

先看量化在做什麼。訓練好的權重是一堆 32 位元浮點數，它們實際的分布範圍通

常不大。量化的基本做法是為一組數值（可能是一整層、也可能是一個通道）找出最大與最小值，把這個範圍線性地映射到整數刻度上。以 8 位元無號整數為例，可用的刻度只有 0 到 255 共 256 格：

```
縮放係數 s = (最大值 − 最小值) / 255
零點       z = round(−最小值 / s)
量化       q = clip(round(x / s) + z, 0, 255)
還原       x̂ = s × (q − z)
```

還原回來的 $\hat{x}$ 與原本的 x 之間會差一點，這個誤差最大是半格，也就是 s/2。這裡立刻看出兩件事。其一，誤差與「範圍寬度」成正比：若某一層裡剛好有一個離群的大權重，整個範圍被它撐開，其餘所有權重的精度就一起變差。這正是實務上要逐通道（per-channel）而非逐層決定縮放係數的原因，也是「剪掉離群值」這類校正手段有效的原因。其二，這個做法與第 3 章把連續亮度切成 256 階、第 11 章 JPEG 用量化表除掉高頻係數，是同一個思想：用精度換效率，而且換掉的那部分再也回不來。

接著算「省了多少」。這部分最單純，而且一定兌現：

```
權重所需位元組 = 參數量 × 每個參數的位元組數
  FP32：4 bytes    FP16：2 bytes    INT8：1 byte
```

YOLO26n 有約 257 萬個參數，以 FP32 存放約 9.8 MB、FP16 約 4.9 MB、INT8 約 2.5 MB（圖 17-4 的實測）。體積決定了模型裝不裝得進去，而在微控制器這一級，裝不裝得下往往才是唯一的問題。

最後是「快了多少」，也就是最容易說錯的那一部分。量化之所以可能加速，有兩條完全不同的路徑。第一條是記憶體頻寬：權重少搬一半，若這一層原本是被記憶體速度卡住（memory-bound），那麼搬得快就真的算得快。第二條是運算單元：若硬體有原生的低精度乘加指令（Jetson 的 Tensor Core 對 FP16、許多 NPU 與 DSP 對 INT8），那麼同樣一個週期可以做更多次乘加，速度才會成倍數地提升。

關鍵在於：這兩條路徑都要硬體配合，而且第二條更挑。若處理器沒有原生的 FP16 運算單元，推論引擎只能把半精度轉回單精度再算，轉換本身就是額外的指令;

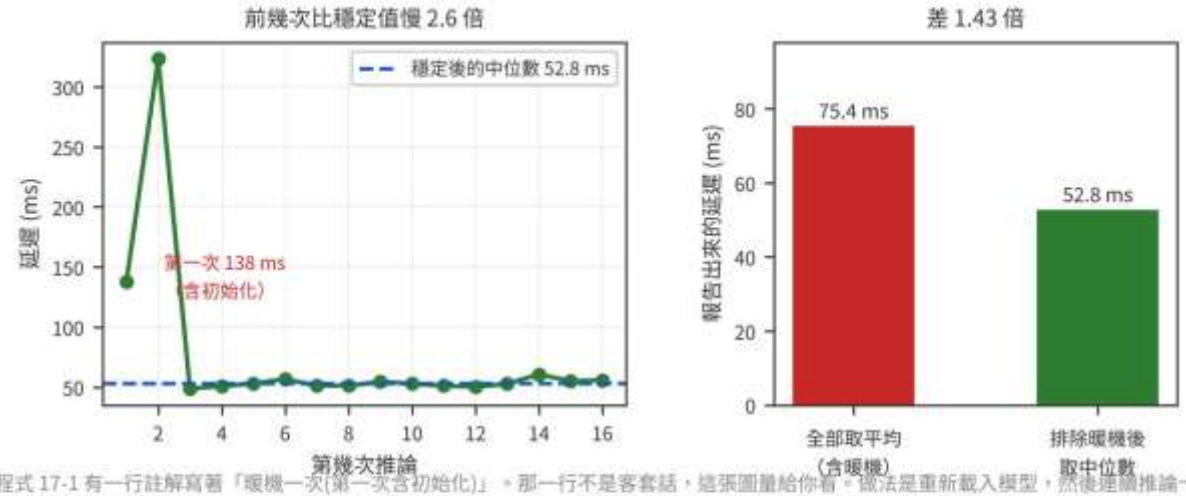
若用的是動態 INT8 量化，還要在執行期間統計激活值的範圍、即時算出縮放係數、並在整數與浮點之間來回轉換。這些額外動作有可能吃掉、甚至超過省下來的乘法時間——圖 17-5 在一顆沒有這些單元的 CPU 上量到的，正是這個結果：FP32 的 ONNX 跑 33 毫秒，FP16 變成 43 毫秒，INT8 更慢到 59 毫秒。

所以量化該怎麼用?把它想成三個獨立的問題。「裝不裝得下」——量化一定有幫助，先量體積。「掉多少精度」——FP16 幾乎免費，INT8 在圖 17-6 的實測掉了約 2.4 個百分點的 mAP@0.5，而這個損失可以靠更講究的做法救回大半：訓練後靜態量化（拿一小批代表性影像先統計每層激活值的範圍）、或量化感知訓練（在訓練時就模擬量化誤差）。「會不會變快」——只有在目標硬體上量過才知道，而且結論可能與筆電上完全相反。這第三點是本章最該養成的習慣：任何最佳化都必須在它最終要執行的那顆晶片上驗證。

17.4 效能量測與系統設計

17.4.1 三個關鍵指標

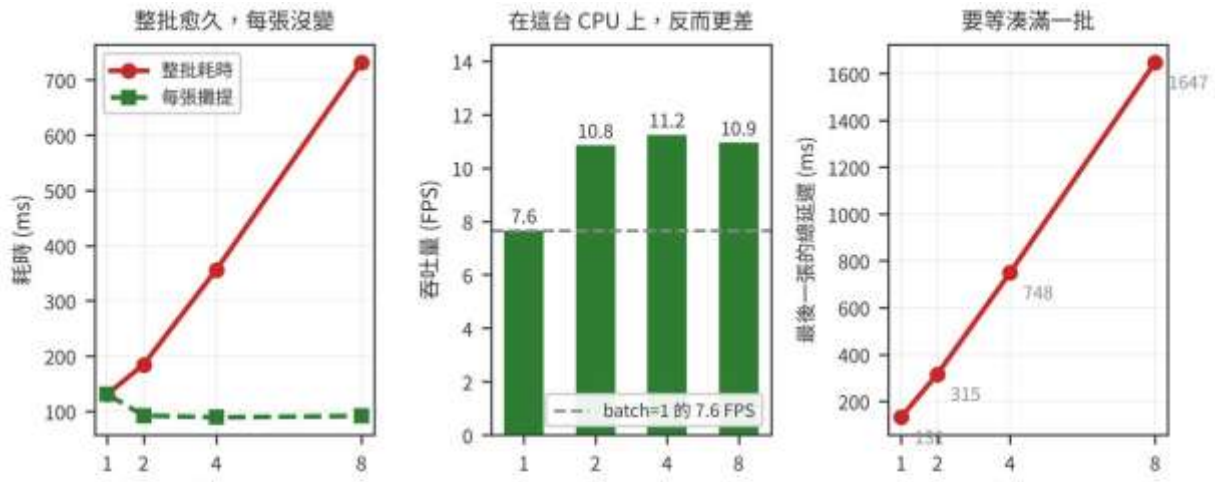
部署前必須量測三個指標，它們常被混淆卻意義迥異：延遲(Latency)是「單張影像從輸入到輸出所需的時間」，決定系統的反應速度，對即時控制最關鍵;吞吐量(Throughput，常以 FPS 表示)是「每秒能處理幾張影像」，決定系統的處理能力;功耗(Power)則決定電池續航與散熱設計。



程式 17-1 有一行註解寫著「暖機一次(第一次含初始化)」。那一行不是客套話，這張圖是給你看。做法是重新載入模型，然後連續推論十六次、每一次都計時。第一次是 138 毫秒，第二次 323 毫秒，要到第四、五次之後才穩定在 52.8 毫秒上下——第一次是穩定值的 2.6 倍。這段額外時間花在哪裡?配置記憶體、把權重從硬碟讀進來並搬到運算裝置、推論引擎依實際的輸入形狀挑選最快的核心實作、以及作業系統的快取預熱。這些都是一次性的，之後不會再發生。右圖是這件事的後果：把十六次全部平均起來會報出 75.4 毫秒，比真正的穩態值高了 1.43 倍。在效能報告裡，這種數字會讓你誤以為需要換更小的模型，而實際上什麼都不必換。正確的量測姿勢有三條：先跑幾次把系統跑熱再開始計時;報告中位數而不是平均數（平均數會被少數極端值拉走，圖 17-12 會說明為什麼）;以及明確寫出你量的是哪一段——只有模型推論，還是包含影像讀取與前處理（圖 17-11）。順帶一提，暖機在部署時也有實務意義：系統啟動後應該先空跑幾張假影像，不要讓第一個真實的請求去承擔初始化成本——那一瞬間剛好可能是機器人開始移動的時候。

圖 17-9　第一次推論不能算：暖機

延遲與吞吐量並不等價：透過批次處理(一次處理多張)可以提高吞吐量，但單張的延遲反而變長——因為要等湊滿一批。這是標準的說法，而它有一個常被省略的前提：硬體必須還有閒置的平行處理能力。GPU 有數千個運算單元，單張影像往往餵不滿，湊成一批才能讓這些單元同時工作;但若核心數少、而推論本身已經把核心用滿，就沒有閒置的平行度可以利用。圖 17-10 在這樣一台機器上實測：batch=1 是 18.5 FPS，batch=8 降到 13.9 FPS——不是原地踏步，是倒退。多出來的更大中間張量、更差的快取命中率與額外的記憶體搬運，讓整體比一張一張處理還慢。所以批次處理的正確判準是先看硬體利用率(GPU 的 occupancy、CPU 的核心負載)，只有在明顯有閒置時才值得試，而且一定要實測。順帶提醒：即使在 GPU 上批次確實有效，對機器人避障這類任務仍然不該用——湊滿一批的等待時間會直接加在延遲上。對機器人避障這種需要即刻反應的任務，低延遲遠比高吞吐量重要;對離線的影片分析，則反之。這又是一個「依應用選指標」的工程判斷，與第 14、16 章的精確率召回率取捨同出一轍。



17.4.1 節說「批次處理可以提高吞吐量，但單張的延遲反而變長」。這句話成立，前半句則要看硬體——這張圖就是那個「要看」。左圖與中圖是實測：批次從 1 加到 8，整批耗時從 131 毫秒線性漲到 732 毫秒，而每張攤提下來不但沒省，反而從 130.8 毫秒變成 91.5 毫秒;吞吐量因此從 7.6 FPS 掉到 10.9 FPS——不是原地踏步，是倒退。原因很直接：批次能提高吞吐量，前提是運算單元還有閒置的平行度可以被填滿。GPU 有數千個運算核心，單張影像往往餵不飽它，這時一次給八張就能把空檔填起來;但這台機器的 CPU 核心數有限，單張推論就已經把它們佔滿了，再堆批次只是排隊，而且多出來的記憶體搬運還要額外付錢，所以吞吐量不升反降。右圖是代價的部分，而且比看起來更糟：要湊滿一批，前面的影像得先等後面的到齊。以每張間隔 131 毫秒的串流估算，批次 8 時第一張影像從抵達到拿到結果要 1647 毫秒，是批次 1 的 12.6 倍。所以結論很清楚：機器人避障、產線剔除這類「要立刻反應」的任務，批次永遠是 1;離線分析一整天的錄影、或在伺服器上同時服務多支相機，才是批次發揮價值的場合——而且前提是你跑在 GPU 上。這也是為什麼本章一再強調「在目標硬體上量」：同一個設定在 CPU 與 GPU 上的結論可以完全相反。

圖 17-10　批次處理：吞吐量不一定買得到

17.4.2 端到端的延遲預算

一個常見的錯誤是：只量測模型推論的時間，就以為那是系統的反應速度。實際上，一個機器人視覺系統的端到端延遲包含：影像擷取(相機曝光與傳輸)、前處理(縮放、正規化，第 4、5 章)、模型推論、後處理(NMS、座標轉換，第 16 章)、決策與通訊、以及致動器的反應時間。模型推論往往只佔其中一部分。

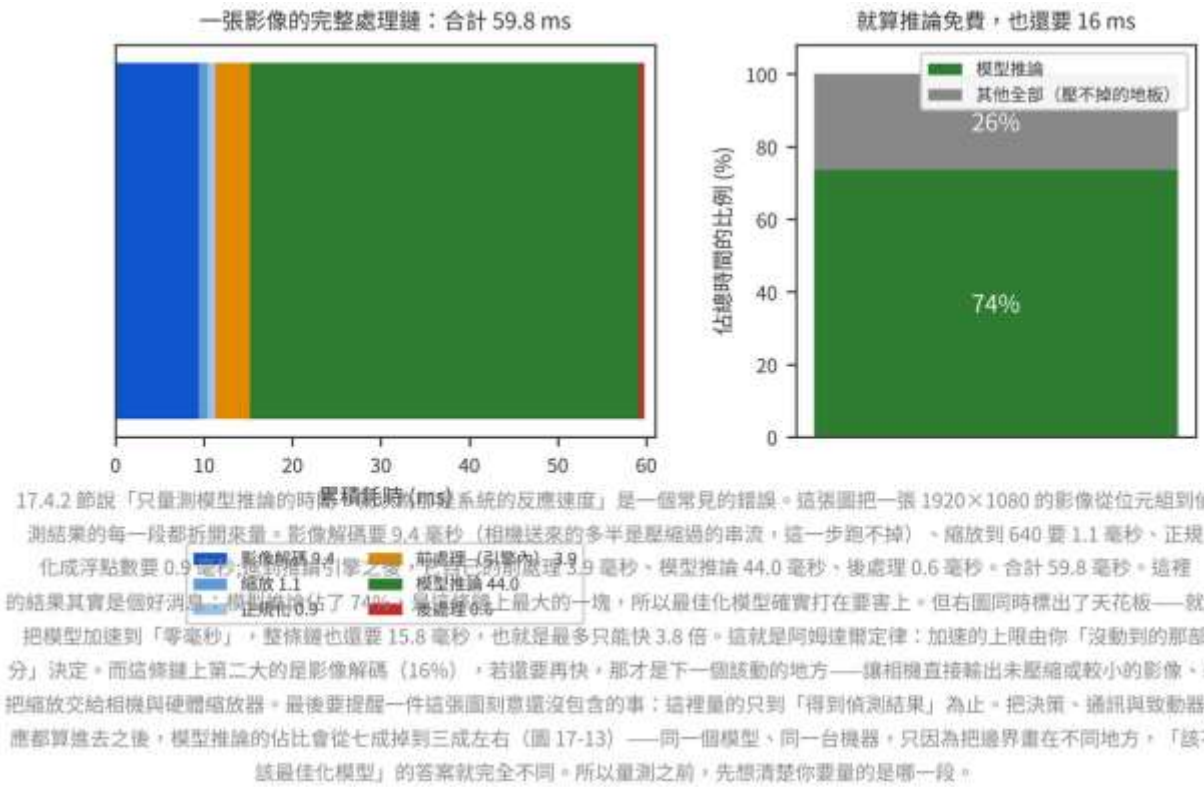


圖 17-11　延遲的分解：模型推論只是其中一段

因此系統設計要做「延遲預算」：先確定任務允許的總延遲(例如機器人以 0.5 公尺/秒行進、要在碰撞前 1 公尺停下，總延遲須小於某個上限)，再回頭分配給各個環節。這種「從需求反推規格」的思維，是工程設計的基本功，也是本章從「會寫演算法」邁向「會設計系統」的關鍵一步。但預算要用哪一個數字去編?多數人的直覺是平均延遲，而那是錯的。延遲是一個有長尾的分布，長尾來自作業系統排程、其他行程搶佔 CPU、快取失效，以及影像內容本身的差異(物件多時後處理較久)。圖 17-12 實測：中位數 35.3 毫秒、第 90 百分位 39.8、第 99 百分位 49.2、最大值 62。若以 40 毫秒為預算，實測有 21.5% 的影格超出——而中位數看起來明明還有餘裕。對串流播放而言偶爾一格遲到只是不順，但對控制迴路而言那一格就是控制訊號遲到，而機器人在那一瞬間仍然以原速移動。所以安全攸關的系統應以第 99 百分位、甚至最大值編預算，並在這之上再留餘裕。還有一件事比選統計量更基本：先把邊界定下來。圖 17-11 量「視覺處理鏈」時，推論佔 69.8%(解碼 8.75 毫秒、推論 29.73 毫秒)，最佳化模型顯然是對的;圖 17-13 把相機曝光、決策、通訊與致動器反應都算進去之後，整

條鏈約 145.9 毫秒，推論的佔比掉到三成，而致動器反應那一段比推論還久。兩個數字都沒錯，差別只在邊界畫在哪裡——而阿姆達爾定律(Amdahl's law)說得很直接：最佳化某一段所能帶來的整體改善，上限就是那一段的佔比。這是本章最該養成的兩個習慣：先定義邊界，再量測;而且所謂「量測」不是在筆電上跑個平均值，而是在它最終要執行的那顆晶片上、用它真正會遇到的影像量出來的。

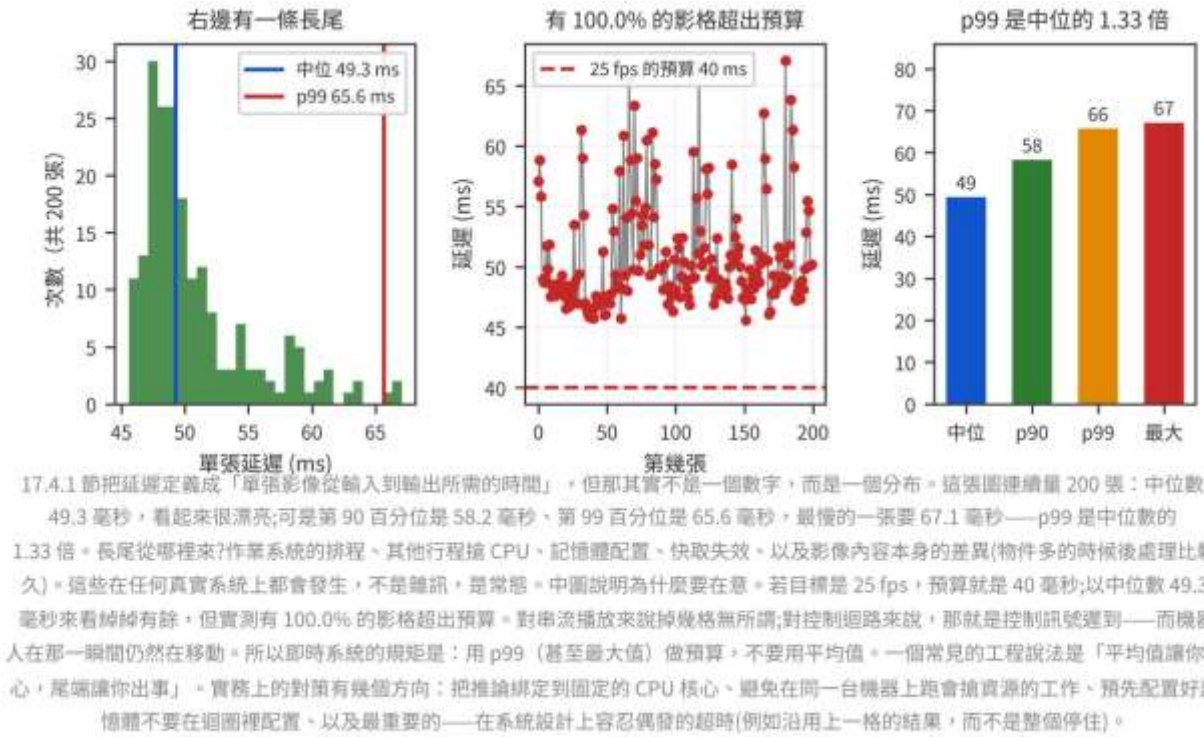


圖 17-12　延遲不是一個數字

17.4.3 推導：延遲預算該怎麼編，又該用哪一個數字

17.4.2 節要我們做延遲預算，但沒說預算該用什麼數字去編。是平均延遲嗎?這一小節說明為什麼不是，並把一條完整的預算算給你看。

先把安全距離寫出來。機器人以速度 v 行進，從「障礙物進入視野」到「輪子真的開始減速」之間有一段總延遲 T，這段期間它以原速前進，走過的距離稱為反應距離;接著以減速度 a 煞停，再走一段煞車距離：

```
反應距離 = v × T
煞車距離 = v² / (2a)
總停止距離 = v × T + v² / (2a)
```

這條式子有一個重要的性質：反應距離與速度成一次方關係，煞車距離與速度成平方關係。所以速度低時延遲是主角，速度高時煞車是主角。圖 17-2 的實測顯示：以每秒一公尺行進、減速度 0.8 m/s² 時，延遲造成的反應距離只有十幾公分，煞車距離卻有 62 公分;這時再花力氣把模型加速十毫秒，對安全距離幾乎沒有貢獻。反過來說，把速度減半，總停止距離會縮到不到三分之一——這是整個系統裡最便宜也最有效的旋鈕。

接著是 T 要怎麼估。它是整條鏈的總和，而不是模型推論那一段：

```
T = 相機曝光與傳輸 + 影像解碼 + 前處理
  + 模型推論 + 後處理 + 決策
  + 通訊 + 致動器反應
```

圖 17-13 把這條鏈量了一遍，總計約 146 毫秒，而模型推論只佔其中約三成。這帶出延遲預算的第一條紀律：先量再改。若不量就開始最佳化模型，很可能動了佔三成的那一段，卻對另外七成束手無策——而本例中致動器反應那一段比推論還久。

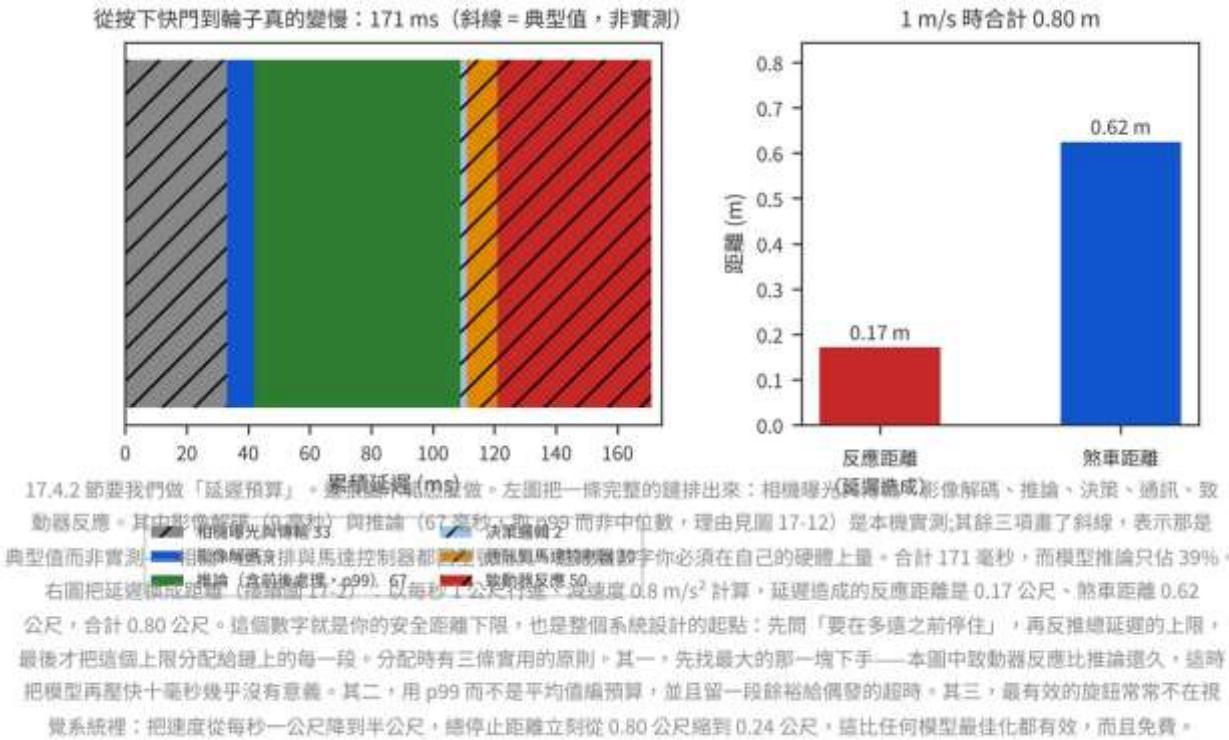
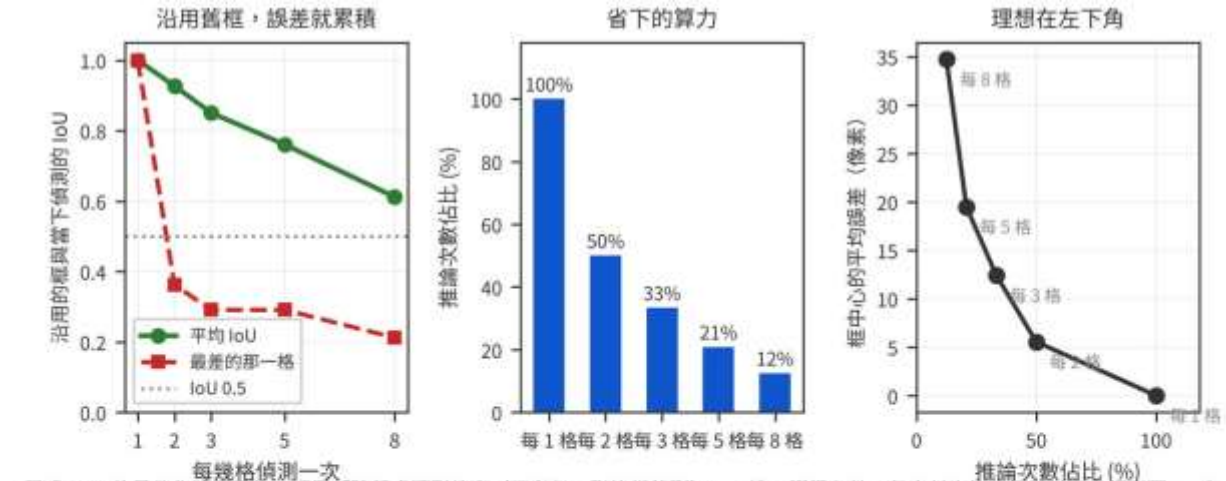


圖 17-13　端到端延遲預算

最後是本小節真正的重點：預算該用哪一個數字。很多人直覺用平均延遲，但那

是錯的。延遲是一個有長尾的分布（圖 17-12 實測：中位數 35 毫秒、第 99 百分位 49 毫秒、最大值 62 毫秒），長尾來自作業系統排程、其他行程搶資源、快取失效與影像內容本身的差異。若用中位數編預算，等於接受「每一百格就有一格超出預算」;對串流播放來說無所謂，但對控制迴路而言，那一格就是控制訊號遲到，而機器人在那一瞬間仍然在移動。所以安全攸關的系統應該用 p99、甚至最大值編預算，並且在這之上再留一段餘裕。

把這些合起來，一份可以實作的預算流程是這樣的。第一步，從任務反推：決定要在多遠之前停住，代進總停止距離的式子，解出可容許的 T。第二步，量測現況：把整條鏈拆開，每一段都量出中位數與 p99。第三步，比對與分配：若 p99 的總和超過 T，從最大的那一段開始改，並重新檢查改完之後的分布（而不是只看平均值有沒有下降）。第四步，設計容錯：即使預算編得再好，偶爾仍會超時，所以系統必須定義「超時了怎麼辦」——沿用上一格的結果（並限制最多幾格）、降速、或停止，這就是 17.7 節失效模式設計的起點。



程式 17-1 的最佳化清單最後一項是「降低處理影格率（不必每一影格都偵測）」。這一招很有效，但它付出的代價與前面幾招不同——它付的不是準確度，是「新鮮度」。這張圖用一段合成的移動序列量它（真實的人物剪貼到真實背景上，每格移動 9 個像素）。做法是每 N 格才真的跑一次偵測，中間的影格沿用上一次的框，再拿沿用的框與「若當下真的偵測會得到什麼」比較。要注意這個比較基準是刻意選的：若改用貼上去的矩形當答案，量到的會混進「偵測框比人體矩形緊一點」這個與跳格無關的系統性差異——那是第 13、16 章一再提醒的混淆因子。結果：每 2 格偵測一次時，沿用的框與當下偵測的平均 IoU 是 0.93、中心誤差 6 個像素;每 5 格一次是 0.76 與 19 個像素;每 8 格一次掉到 0.61 與 35 個像素。更該看的是紅線——最差的那一格：每 8 格偵測一次時最糟只有 0.21，已經低於一般判定為「抓到」的 0.5。中圖是省下來的：每 8 格一次等於只用 12% 的推論次數。右圖把兩者畫在一起，就是一條可以挑的取捨曲線。怎麼挑?關鍵在物件相對於畫面的移動速度。誤差大致是「速度 × 跳過的時間」，所以同樣跳 5 格，慢速的輸送帶幾乎沒影響，高速公路的車輛就會差到離譜。實務上有一個明顯更好的做法：跳格的空檔不要「沿用舊框」，而是用便宜的方法把框推下去——第 12 章的光流或卡爾曼濾波、或第 16 章的追蹤器，成本只有偵測的幾十分之一，卻能把中間那幾格的位置補得相當準。這就是「偵測式追蹤」在邊緣裝置上的真正價值：它不只是為了維持 ID，更是為了省下算力。

圖 17-14　跳格處理：省下算力，付出什麼

17.5 整合進機器人系統：ROS 2

17.5.1 為什麼需要機器人作業系統

一個機器人不只有視覺：它有馬達、輪子、雷射雷達、IMU、機械臂、電池管理，以及導航、定位、路徑規劃、任務決策等軟體模組。如果把所有東西寫成一支巨大的程式，將難以開發、除錯與維護。ROS 2(Robot Operating System 2)提供的正是一套「讓各個模組獨立開發、彼此通訊」的框架。

它的核心概念很簡單：每個功能模組是一個節點(Node)，節點之間透過話題(Topic)以「發布-訂閱」的方式交換訊息。例如相機節點持續發布影像到 /camera/image 話題，視覺辨識節點訂閱它、處理後發布偵測結果到 /detections 話題，導航節點再訂閱偵測結果來避障。各節點可以用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作。這種鬆耦合的設計，讓團隊能分工開發、也讓模組能重複使用。

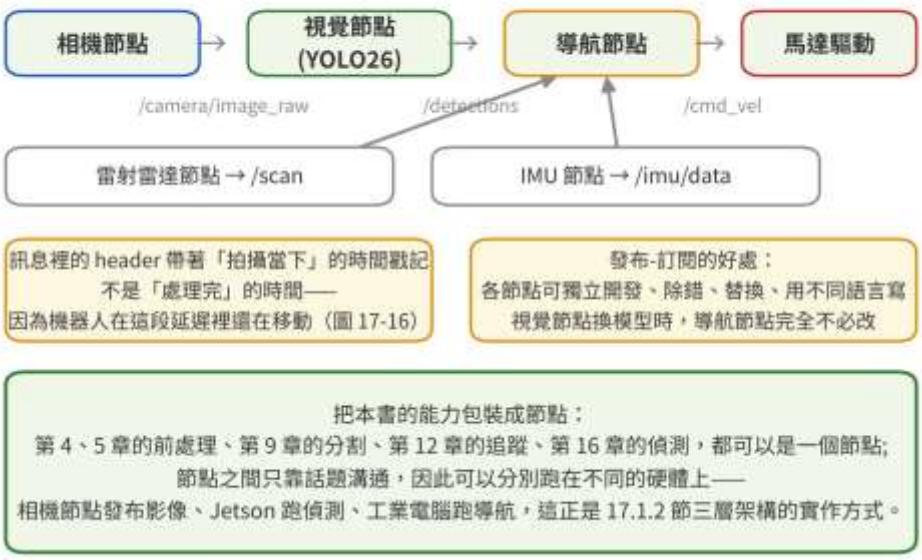


圖 17-15　ROS 2：節點與話題

17.5.2 視覺節點的典型結構

把本課程學到的視覺能力包裝成 ROS 2 節點，是機器人視覺工程師的核心工作。典型結構是：訂閱相機影像話題 → 把 ROS 影像訊息轉成 OpenCV 格式(用 cv_bridge)→ 套用本書的處理流程(前處理、YOLO 推論、後處理)→ 把結果包裝成 ROS 訊息發布出去，供其他節點使用。

程式 17-2 ROS 2 視覺節點骨架

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray
from cv_bridge import CvBridge
from ultralytics import YOLO

class VisionNode(Node):
    def __init__(self):
        super().__init__("vision_node")
        self.bridge = CvBridge()
        self.model = YOLO("yolo26n.engine")   # TensorRT 最佳化版

        # 訂閱相機影像
        self.sub = self.create_subscription(
            Image, "/camera/image_raw", self.on_image, 10)
        # 發布偵測結果
        self.pub = self.create_publisher(
            Detection2DArray, "/detections", 10)

    def on_image(self, msg):
        # ROS 影像訊息 → OpenCV 格式
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")

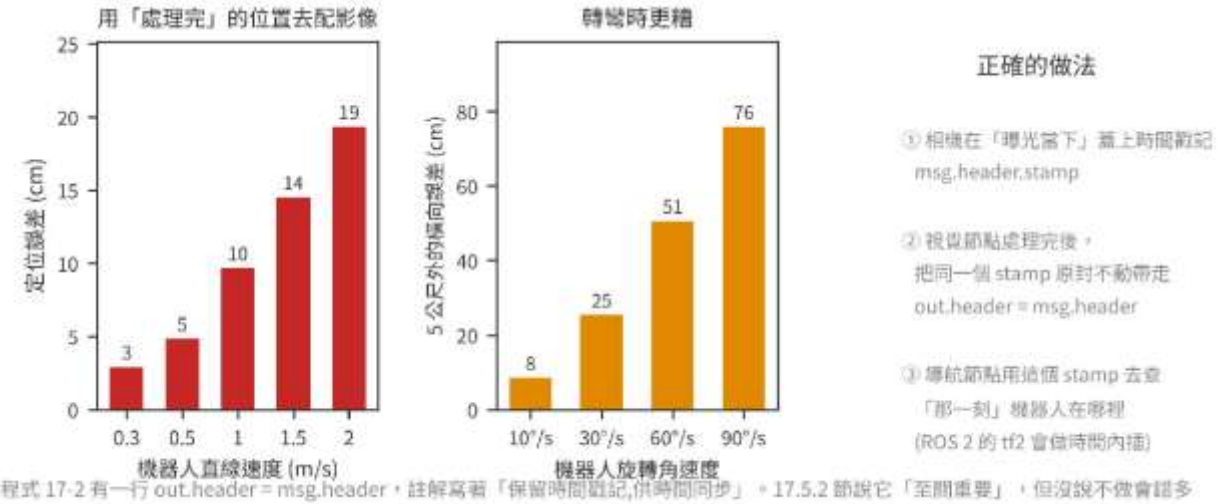
        # 套用本書所學的處理流程
        results = self.model(frame, verbose=False)[0]

        # 包裝成 ROS 訊息發布給導航等其他節點
        out = Detection2DArray()
        out.header = msg.header          # 保留時間戳記,供時間同步
        # ... 填入 results.boxes 的內容 ...
        self.pub.publish(out)

def main():
    rclpy.init()
    rclpy.spin(VisionNode())
    rclpy.shutdown()
```

請留意 header 那一行：保留時間戳記至關重要，因為機器人要把「影像看到什

麼」與「當時機器人在哪裡」對應起來——這需要精確的時間同步，也需要第 4 章的座標轉換(ROS 2 的 tf2 系統正是齊次座標與座標框架的實作，第 4.1.2 節已預告過這個連結)。



程式 17-2 有一行 out.header = msg.header，註解寫著「保留時間戳記,供時間同步」。17.5.2 節說它「至關重要」，但沒說不做會錯多少。這張圖補上。問題是這樣的：影像從曝光到偵測結果出爐，本章實測這段約 97 毫秒（相機傳輸＋解碼＋推論，見圖 17-11）。如果導航節點拿到偵測結果時，用「現在」機器人的位置去反推障礙物在世界座標的哪裡，那它算的是機器人已經往前走了一段之後的位置——障礙物會被放到錯誤的地方。左圖是直線運動的誤差：以每秒一公尺行進時偏 10 公分，每秒兩公尺時偏 19 公分。中圖是旋轉的情況，而且更嚴重：以每秒 60 度轉彎時，5 公尺外的物件橫向偏 51 公分，每秒 90 度時偏 76 公分。原因是角度誤差會被距離放大——這正是第 4 章相機模型那一課：角度的小誤差在遠處就是大位移。右圖是解法，而它幾乎不花力氣：讓時間戳記跟著資料一起流動，在最後需要座標轉換時，用「拍攝那一刻」的機器人姿態。ROS 2 的 tf2 可以依時間戳記內插出那一刻的轉換矩陣，所以你只要「不要把 header 弄丟」就好。這也是為什麼 ROS 2 的訊息幾乎都帶 header——它不是形式，是一個把延遲明確記錄下來的設計。順帶一提，這個誤差無法靠加快模型消除，只能靠正確的時間對齊;模型再快，那段延遲仍然存在。

圖 17-16　時間戳記：用錯時間，就把物件放錯地方

17.6 應用案例

17.6.1 案例一：自主移動機器人的視覺避障

情境：一台自主移動機器人在走廊或廠區中行進，需即時偵測前方障礙物(人、箱子、其他車輛)並避開。系統架構：Jetson 作為主控 → 相機節點發布影像 → 視覺節點以 YOLO 偵測障礙物與其位置 → 結合深度資訊(深度相機或雙目視覺)估計距離 → 發布給導航節點調整路徑。

本案例的工程重點在延遲預算(17.4.2 節)：機器人速度、煞車距離、總延遲三者必須匹配。若量測發現延遲過高，依程式 17-1 的順序最佳化——換更小的模型、降解析度、量化、用 TensorRT。同時要設計失效模式：若視覺節點當機或延遲暴增，機

器人應自動減速或停止，而非盲目前進。這呼應第 16 章人機協作案例的「多層防護」原則。

17.6.2 案例二：智慧鏡頭人臉辨識（樹莓派 5 + AI Camera）

情境：在實驗室、辦公室或社團教室的門口做人臉辨識門禁，要求是即時、不連網也能運作，而且不把任何人的臉傳出去。架構：AI Camera 在感測器上持續執行人臉偵測，只把偵測結果（有沒有人、框在哪裡）以張量後設資料交給樹莓派 5 → 樹莓派僅在「偵測到人臉」時才取出那一小塊影像區域，在本機做身分比對（第 13 章的特徵與第 15 章的嵌入向量）→ 比對成功則開門並記錄時間，失敗則通知管理者。

這個案例的工程重點有三。一是隱私設計：絕大多數的影格連解碼都沒有，主機拿到的只是「有幾張臉」;只有在需要比對的那一瞬間，才有一小塊影像進入主機記憶體，而且比對完即丟、不落地。這正是 17.1.1 節說的「最有力的隱私保護」——不是承諾不看，而是沒有東西可看。二是模型的額度：人臉偵測模型必須塞進感測器上那很小的記憶體額度，而且只能是 INT8，所以這一層只做「有沒有臉」，不做「是誰」。三是分層：「是誰」這個高算力、低頻率的問題留給主機，甚至可以留給後端——這就是 17.1.2 節三層架構最乾淨的一個實例。

延伸與風險：人臉是高度敏感的個人資料，本案例在設計上就該回答第 18 章要問的三個倫理問題——誰受益、誰可能受害、出錯了誰承擔。具體的做法包括：公告蒐集範圍與目的、提供不使用人臉的替代方式（卡片或密碼）、只儲存比對用的特徵向量而非影像、以及設定保存期限並定期刪除。技術上也要誠實面對誤判：門禁屬於「寧可擋錯也不要放錯」的場景，所以門檻應偏向高精確率，並且一定要有人工覆核的通道（承第 16 章的多層防護）。

這是三層架構協作的典範：端上負責「有沒有事情發生」的高頻低算力判斷，近端負責「發生了什麼」的精細辨識，雲端負責長期趨勢分析與遠端存取。成本、頻寬、隱私、即時性四者在此取得平衡——這正是叫獸開講中「五美元革命」讓大規模佈署

成為可能的實際體現。

17.6.3 案例三：農業無人機的田間巡檢

情境：無人機飛越農田，即時辨識作物病害、雜草分布或缺株位置。這個案例把邊緣運算的每個理由都體現得淋漓盡致：田間沒有穩定網路(可靠性)、無人機電池有限(功耗)、需即時調整航線(延遲)、影像資料量龐大(頻寬)。因此運算必須在機上完成——用輕量模型加量化，在 Jetson 或類似的機載電腦上即時推論，只把「哪裡有病害」的座標與少量關鍵影像回傳。

技術上還需整合第 4 章的相機校正(把像素座標換算成實際的田間位置)、第 8 章的色彩處理(戶外光線變化劇烈，白平衡與對比校正是必要前處理)、第 12 章的視訊處理(連續影格的追蹤與拼接)。這個案例最能說明：邊緣部署不是把模型搬過去就好，而是整個系統的重新設計。

17.6.5 案例五：開放語意的現場巡檢（Jetson Orin Nano Super + NanoOWL）

情境：一台巡檢機器人要在廠區或校園裡找出「今天要找的東西」，而那個清單每週都在變——這週要找沒戴安全帽的人，下週要找倒地的推車，再下週要找堆在逃生通道上的紙箱。用第 16 章的做法，每換一次清單就要標註與重訓一輪，現實上根本跟不上。架構：Jetson Orin Nano Super 作為主控 → 相機節點發布影像 → NanoOWL 節點以文字提示執行開放詞彙偵測 → 把偵測結果發布成 ROS 2 訊息 → 決策節點依結果記錄位置與時間並回報。提示詞則從設定檔讀入，現場就能改。

這個案例真正的教學價值在於它的限制，而不是它的炫目。它有三筆代價，每一筆都會直接影響這個系統能不能用。第一，提示詞寫法的影響大得驚人：單數與複數、加不加冠詞、用哪一個同義詞，結果可能天差地別（圖 17-3 的右圖就是這麼來的）。所以提示詞必須當成參數來調，而且要在開發集上調、在另一份測試集上驗（承第 14、18 章）。第二，速度：開放詞彙模型比同級的封閉詞彙模型重，因為每次推論都要同

時處理影像與文字兩邊，這會直接吃掉 17.4.2 節的延遲預算，因此巡檢機器人通常是「停下來看」而不是「邊走邊看」。第三，精度：它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。

務實的做法因此是把它當成第一道篩子，而不是最終的判斷。先用開放詞彙快速找出「可能有問題的畫面」，再把這些畫面交給人工複核、或交給一個針對該類別訓練過的小模型確認。等到某一個項目變成長期固定的檢查項（例如安全帽），就值得為它標註資料、訓練一個封閉詞彙的模型來正式上線。這正是第 18 章「最小可行版本」的精神：先把路走通，再把最爛的那一段換掉。

17.6.4 案例四：產線 AOI 的邊緣部署

情境：產線上每秒通過數個工件，需即時檢測瑕疵並剔除不良品。這是延遲要求最嚴苛的場景之一——工件通過檢測站到剔除機構的時間可能只有數百毫秒，整條處理鏈必須在此之內完成。

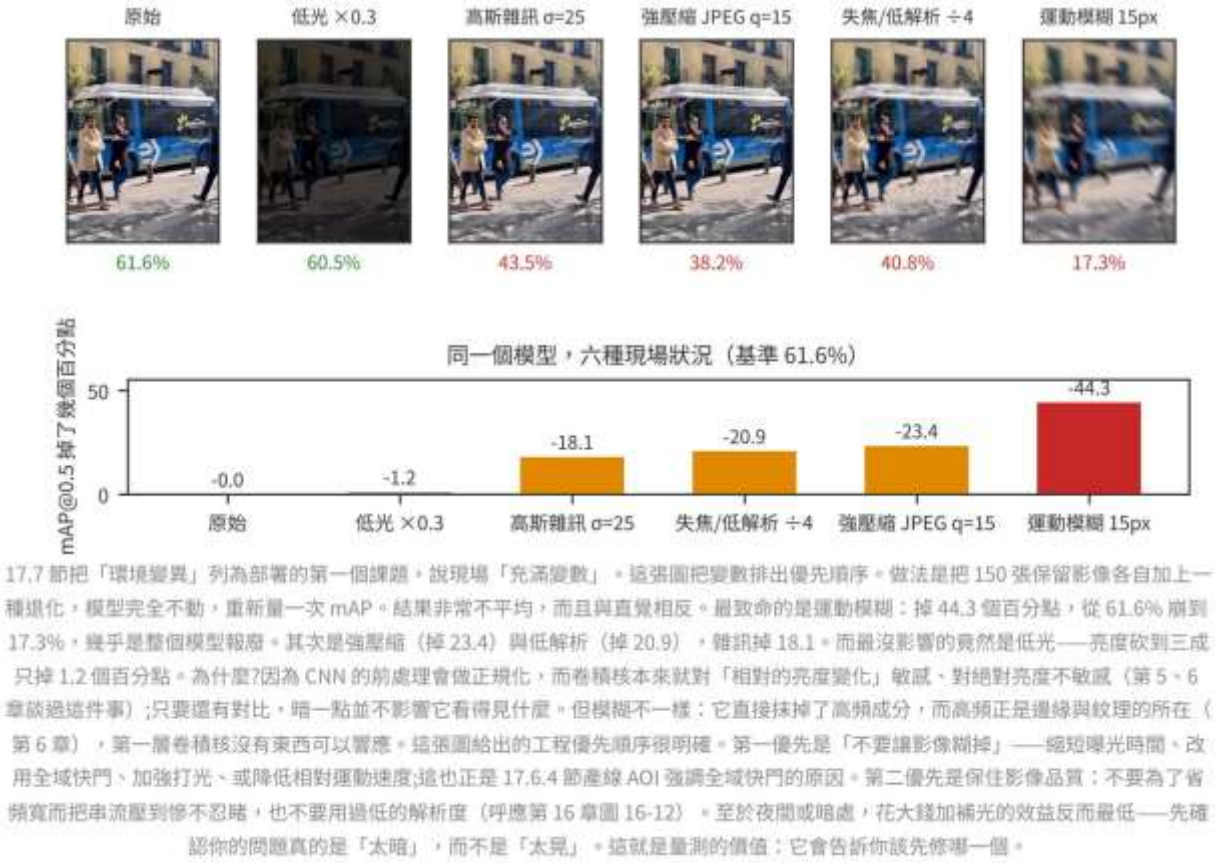
系統設計要點：相機用全域快門避免運動模糊(第 7.6.3 節已詳述)、打光穩定以避免第 8 章的「洋裝效應」、模型用量化與 TensorRT 最佳化、並保留餘裕以應對突發的處理延遲。此外，產線 AOI 常混用傳統與深度學習方法(第 6 章頻率域紋理檢測、第 9、10 章的傳統管線、第 16 章的 YOLO)，因為傳統方法不需瑕疵樣本、運算極輕、可解釋性高——在邊緣裝置上，這些優勢格外珍貴。

17.7 部署的實務課題

把系統從「實驗室能跑」帶到「現場能用」，還有幾個常被低估的課題：

- 環境變異：實驗室的光線、背景、角度都是理想的，現場則充滿變數。這正是第 15 章談過的分布偏移——模型在訓練分布外表現會急劇下滑。對策：在真實環境蒐集訓練資料、大量使用資料增強、並持續監控上線後的表現。不過「充滿變數」這句話太籠統，若要排優先順序就得量。圖 17-17 對同一批模型

沒看過的影像施加五種退化，量 mAP@0.5 的掉幅：低光只掉 1.16 個百分點、雜訊掉 18.11、低解析掉 20.85、JPEG 品質壓到 15 掉 23.42、運動模糊掉 44.34。這個排序是反直覺的：大家最先想到的低光幾乎無害，而運動模糊的破壞力是它的將近四十倍。原因是現代模型訓練時已見過大量曝光不足的影像，而卷積網路主要依賴邊緣與紋理的相對關係，整體變暗並不破壞這些關係;運動模糊卻直接把邊緣與紋理抹掉，刪掉的是模型賴以判斷的資訊本身。工程上的結論很具體：與其加裝補光燈，不如先縮短快門時間(全域快門、提高增益、換更靈敏的感測器，第 7.6.3 節)、或降低機器人與雲台的運動速度。另外 JPEG 壓縮品質與傳輸解析度這兩項是系統設計者可以直接控制的，卻常為了省頻寬被調得太低——實測顯示它們的代價與雜訊相當，絕不是免費的。



17.7 節把「環境變異」列為部署的第一個課題，說現場「充滿變數」。這張圖把變數排出優先順序。做法是把 150 張保留影像各自加上一種退化，模型完全不動，重新量一次 mAP。結果非常不均，而且與直覺相反。最致命的是運動模糊：掉 44.3 個百分點，從 61.6% 崩到 17.3%，幾乎是整個模型報廢。其次是強壓縮（掉 23.4）與低解析（掉 20.9），雜訊掉 18.1。而最沒影響的竟然是低光——亮度砍到三成只掉 1.2 個百分點。為什麼?因為 CNN 的前處理會做正規化，而卷積核本來就對「相對的亮度變化」敏感、對絕對亮度不敏感（第 5、6 章談過這件事）;只要還有對比，暗一點並不影響它看得見什麼。但模糊不一樣：它直接抹掉了高頻成分，而高頻正是邊緣與紋理的所在（第 6 章），第一層卷積核沒有東西可以響應。這張圖給出的工程優先順序很明確。第一優先是「不要讓影像糊掉」——縮短曝光時間、改用全域快門、加強打光、或降低相對運動速度;這也正是 17.6.4 節產線 AOI 強調全域快門的原因。第二優先是保住影像品質：不要為了省頻寬而把串流壓到慘不忍睹，也不要用過低的解析度（呼應第 16 章圖 16-12）。至於夜間或暗處，花大錢加補光的效益反而最低——先確認你的問題真的是「太暗」，而不是「太晃」。這就是量測的價值：它會告訴你該先修哪一個。

圖 17-17　現場不是實驗室：哪一種退化最致命

- 溫度與散熱：邊緣裝置長時間高負載執行會發熱，過熱時會自動降頻，導致效能突然下降。散熱設計與功耗管理是硬體工程的一環。

- 失效模式設計：系統一定會出錯，問題是「出錯時會怎樣」。安全的設計應該是「失效時進入安全狀態」(如停止、警示)，而非繼續盲目運作。這與第 16 章人機協作的多層防護一脈相承。

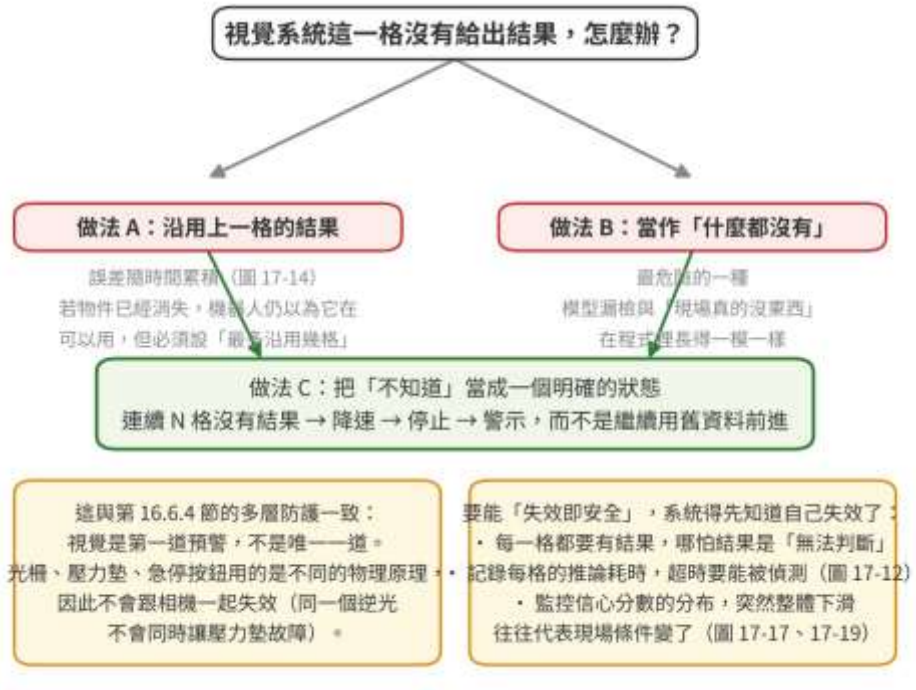


圖 17-18　失效即安全

- 模型更新與維運：部署不是終點。現場條件會改變、新的物件類型會出現，模型需要定期用新資料重訓與更新。如何遠端安全地更新邊緣裝置上的模型，是系統設計的一部分。
- 資料回饋迴圈：上線後蒐集模型出錯的案例、加以標註並納入下一輪訓練，是提升系統的最有效途徑。設計時就應規劃這個迴圈——這也呼應第 14 章「資料是燃料」。

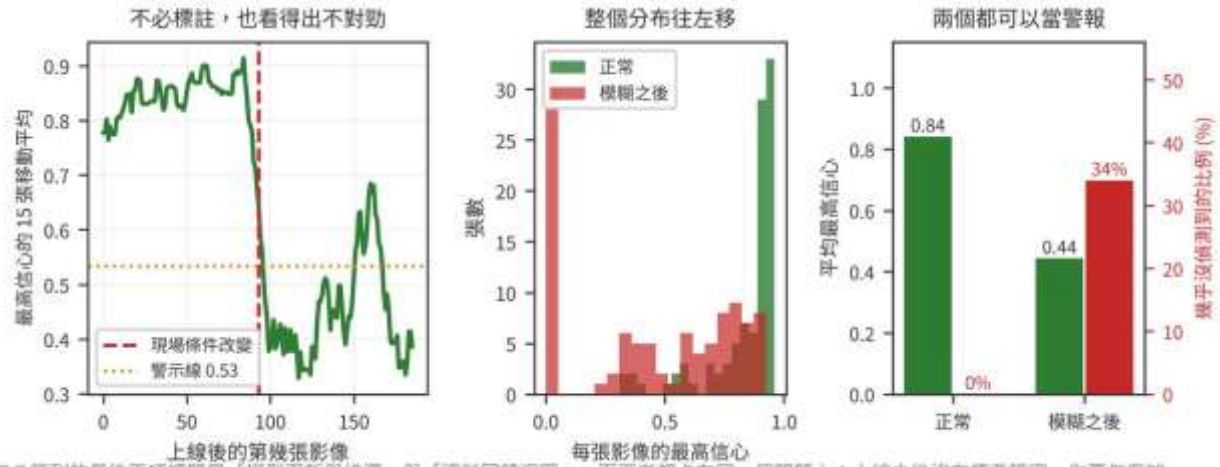
17.7.1 推導：上線之後，怎麼知道模型開始失準了

17.7 節列的最後兩項課題——模型更新與維運、資料回饋迴圈——都卡在同一個問題上：上線之後沒有標準答案，你要怎麼知道模型已經不準了?總不能等客訴才知道。這一小節給三個不需要人工標註就能算的監控訊號，以及它們各自看得見與看不見的東西。

第一個訊號是信心分數的分布。模型對每個偵測都會給一個分數，把「每張影像中最高的那個分數」記錄下來，取移動平均，就得到一條可以畫在儀表板上的曲線。圖 17-19 做了這個實驗：在影像變模糊的那一刻，平均最高信心從 0.84 掉到 0.44，而「幾乎沒偵測到任何東西」的影像比例從 0% 升到 34%。兩者都在沒有任何標註的情況下清楚地指出「出事了」。設定警示線的方法也很單純：上線初期先收集一段正常運作的資料，算出平均值與標準差，把警示線訂在平均減去兩倍標準差。

第二個訊號是輸出的統計量。每張影像偵測到幾個物件、各類別的比例、框的平均大小與位置分布——這些量在穩定的場域中應該是穩定的。一條產線每分鐘通過的工件數大致固定，若偵測到的數量突然少了一成，不是產線慢了就是模型瞎了，兩者都該有人看一眼。這類訊號的好處是它與應用語意直接相關，因此警報比較容易判讀。

第三個訊號是系統層面的：延遲與超時比例。若 p99 延遲開始上升、或超出預算的影格比例從 1% 變成 10%，通常代表硬體在降頻（過熱）、或有別的行程在搶資源。這與模型準不準無關，但同樣會讓系統失效，而且它是少數可以「在壞掉之前」就看出來的徵兆。



17.7 節列的最後兩項課題是「模型更新與維運」與「資料回饋迴圈」，而兩者都卡在同一個問題上：上線之後沒有標準答案，你要怎麼知道模型開始失準了？這張圖示範一個不需要標註的監控訊號——信心分數本身。做法是把 100 張正常影像與 100 張模糊影像串成一段「上線資料流」，只記錄每一張影像中最高的那個信心分數，再取移動平均。左圖的結果一目了然：現場條件改變的那一刻，曲線明顯下沉並跌破警示線，而這一切都不需要任何人工標註。中圖是背後的分布：正常時平均最高信心是 0.84，模糊之後掉到 0.44；右圖另外給一個更穩健的指標——「幾乎沒偵測到任何東西」的影像比例從 0% 升到 34%。要提醒的是，這個訊號只能告訴你「有事發生」，不能告訴你「錯在哪裡」，也不能取代真正的驗收——模型可能自信滿滿地持續犯錯（第 15 章的對抗脆弱、第 16 章的捷徑學習都是這種情況）。所以完整的維運迴圈應該是三層：自動監控信心分布與超時比例（這張圖）、定期抽樣人工複核（真正的驗收）、以及把複核出來的錯誤案例收進下一輪訓練資料。第三層最容易被省略，卻是長期唯一有效的改善途徑——這也呼應第 14 章那句「資料是燃料」：上線之後，你的使用者就是最好的資料來源。

圖 17-19　上線之後：用信心分數察覺現場變了

這三個訊號有一個共同的侷限，必須說清楚：它們只能告訴你「有事發生」，不能告訴你「錯在哪裡」。更糟的是，有一整類失效它們完全看不見——模型自信滿滿地持續犯錯。第 15 章的對抗脆弱、第 16 章的捷徑學習都屬於這一類：信心分數很高、輸出數量正常、延遲也漂亮，但答案是錯的。所以自動監控永遠不能取代定期的人工抽樣複核。

完整的維運迴圈因此有三層。最外層是自動監控（上面三個訊號），它跑得最頻繁、成本最低，負責發現明顯的異常。中間層是定期抽樣複核：每週固定抽一批上線影像人工標註，算出真實的 mAP 或召回率——注意這批資料必須是模型沒看過的（第 16 章圖 16-18），而且樣本數要夠大才報得出有意義的數字（第 14.5.3 節的標準誤）。最內層是資料回饋：把複核出來的錯誤案例標註好、納入下一輪訓練，重訓之後再走一次完整的驗收流程。

最後一個常被忽略的工程課題是「怎麼把新模型送上去」。邊緣裝置往往分散在田間、廠房或車上，遠端更新必須處理三件事：更新失敗要能自動回復到舊版本（否則一次失敗就是一台報廢的機器）、更新過程不能讓系統處於不安全的狀態（機器人應該先停下來再更新）、以及新舊版本要能並行比較一段時間，確認新版在真實場域真的比較好——畢竟離線測試集上的進步，不保證現場也進步。這一整套做下來，部署才算真正完成;而這也說明了為什麼 17.7 節開頭那句話值得再強調一次：部署不是終點，而是另一個循環的起點。

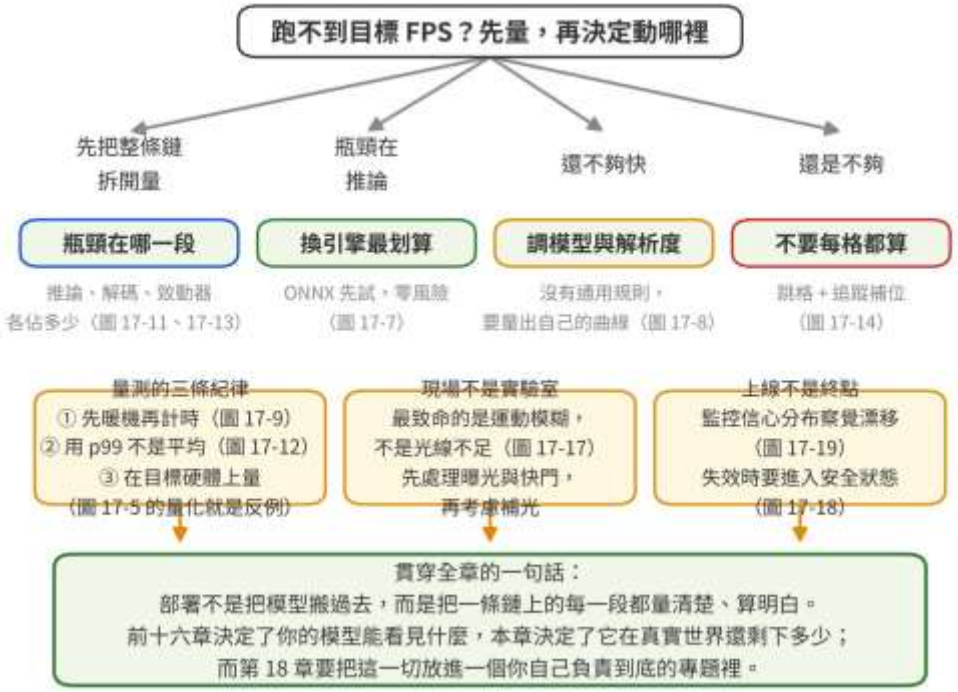


圖 17-20　第 17 章的決策地圖

17.8 本章與全書的觀念連結總表

表 17-2 第 17 章與全書其他章節的觀念連結

程式實作 Lab 17

Lab 17-1 效能量測與最佳化

目標：建立效能工程的實作能力。(a)以程式 17-1 量測 YOLO 在你的裝置上的延遲與 FPS，注意排除第一次推論的初始化時間;(b)依序測試五種最佳化：換小模型(n/s/m)、降解析度(640/416/320)、FP16 量化、ONNX 匯出、跳影格處理，記錄每種對延遲與 mAP 的影響，製成取捨表;(c)畫出「精度 vs. 速度」的散點圖，標出各種組合的位置，找出符合你需求的最佳點;(d)討論：若目標是 30 FPS 即時處理，你會選哪個組合?若目標是最高精度而不在意速度呢?

Lab 17-2 感測器上推論 vs 主機推論

目標：親手比較「感測器上推論」與「主機推論」兩條路徑。(a)在樹莓派 5 上安裝 AI Camera 的韌體與工具(sudo apt install imx500-all)，跑通官方範例並確認能讀

到張量後設資料;(b)路徑一(感測器上)：用 Picamera2 讀取後設資料做即時偵測，記錄主機的 CPU 使用率與每張影像的處理時間;(c)路徑二(主機上)：改用同一台相機輸出的影像，在樹莓派上自己跑一個 YOLO 模型，記錄同樣兩個數字;(d)把兩條路徑的「主機被佔用的時間」與「要搬的資料量」畫成對照表(參考圖 17-3);(e)把模型換成你自己訓練的版本：用 export(format="imx") 轉檔，注意過程會自動做 INT8 量化，並在轉檔前後各量一次 mAP，記錄精度損失(承 17.3.1 節與推導 17.3.4);(f)討論：什麼應用適合感測器上推論?什麼適合主機推論?若你的主機還要同時跑導航與介面，答案會不會不一樣?

Lab 17-3 端到端延遲預算

目標：從系統角度思考。(a)為一個「移動機器人視覺避障」系統做延遲預算：設定機器人速度與煞車距離，推算允許的總延遲上限;(b)實測系統各環節的耗時：影像擷取、前處理、推論、後處理、通訊，製成延遲分解表;(c)找出瓶頸環節並最佳化;(d)設計失效模式：若視覺節點延遲超過門檻或當機，系統該如何反應?寫出你的安全邏輯;(e)討論：為什麼「只量測模型推論時間」是危險的?

Lab 17-4 ROS 2 視覺節點整合(期末專題前導)

目標：把本課程的視覺能力包裝成機器人可用的模組。(a)以程式 17-2 為骨架，建立一個 ROS 2 節點，訂閱相機影像、執行 YOLO 偵測、發布偵測結果;(b)用 ros2 topic echo 驗證訊息正確發布，用 rqt_graph 查看節點與話題的關係圖;(c)加入 header 時間戳記，確認時間同步正確;(d)進階：再寫一個簡單的訂閱節點，依偵測結果印出「前方有障礙物，建議停止」等決策訊息，模擬視覺與導航的協作;(e)這個節點將是第 18 章期末專題的基礎模組，請妥善保存並撰寫使用說明文件。

Lab 17-5 量化與匯出格式的三面取捨

目標：親手量出「更小、更快、更省電」這句話在你的硬體上哪一半成立。(a)把同一顆 YOLO26n 匯出為四種形式：PyTorch 原生 .pt、ONNX FP32、ONNX FP16、

ONNX INT8（FP16 用 onnxconverter_common 的 float16 轉換，INT8 用 onnxruntime.quantization 的 quantize_dynamic）;(b)量體積：記錄四個檔案的大小，與「參數量 × 每參數位元組數」的理論值比對，差異來自哪裡?(c)量速度：每一種各跑 30 次（前 3 次暖機不計），記錄中位數與第 99 百分位，而不只是平均值;(d)量精度：在至少 200 張模型沒看過的影像上算 mAP@0.5 與 mAP@0.5:0.95，注意信心門檻要設得很低（如 0.001），否則量到的是校正差異而非能力差異（第 16 章圖 16-4）;(e)把三組數字畫成一張三面取捨圖（如圖 17-5、17-6），並寫下你的結論：在你這台機器上，哪一種格式該用在哪一種情境?(f)若你有 Jetson 或任何帶 NPU 的開發板，在上面重跑 (c)，並解釋為什麼結論可能與筆電完全相反。討論：若你的 INT8 掉了超過三個百分點的 mAP，試著改用訓練後靜態量化（用 100 張代表性影像做校正）再量一次。

Lab 17-6 上線監控與失效即安全

目標：把 17.7 節與推導 17.7.1 的設計原則寫成可以執行的程式。(a)寫一支監控迴圈，對連續影像即時計算三個訊號：每張影像的最高信心分數（取 30 格移動平均）、「最高信心低於 0.3」的影格比例、以及推論延遲的 p99;(b)先在正常影像上跑一段，記錄三個訊號的平均與標準差，據此訂出警示線（平均減兩倍標準差）;(c)注入三種故障並確認警示有沒有正確觸發：把影像加上高斯模糊（模擬鏡頭失焦或劇烈運動）、把影像亮度壓到極暗、以及在背景另跑一支吃滿 CPU 的程式（模擬行程競爭）。哪一種故障是信心訊號抓到的?哪一種只有延遲訊號抓得到?(d)設計一個會「自信地犯錯」的故障：把影像左右翻轉，或貼上第 16 章的捷徑特徵。你的三個訊號抓得到嗎?為什麼抓不到?(e)實作失效即安全的狀態機：定義正常、降級（降速並提高偵測頻率）、安全停止三種狀態，以及它們之間的轉移條件（含連續沿用舊框的上限格數，見圖 17-14）;用 (c) 的注入故障驗證狀態機會進入正確的狀態，並且在故障移除後能安全地回到正常狀態。(f)寫一頁交付文件：列出每一個警示線的數值與它的根據，以及你建議的人工複核頻率與抽樣方式。討論：(d) 這一題沒有程式解。請說明你會用什麼非自

動化的手段（抽樣複核、跨感測器交叉驗證、與其他模組的一致性檢查）來補上這個缺口。

本章重點整理

- 邊緣運算的五個理由：低延遲(即時控制)、省頻寬與成本、斷網仍可靠、保護隱私(資料不出裝置)、低功耗與自主性;三種架構(端上、近端邊緣、雲端)非互斥，常以「端上預篩、近端精算、雲端分析」分層協作。
- 樹莓派 5 加 AI Camera 把推論做在 IMX500 感測器裡，主機只讀張量後設資料(圖 17-3)，代價是感測器上的記憶體額度很小、只收 INT8、模型清單有限;它買到的不是速度，是主機的時間、頻寬與隱私。Jetson Orin Nano Super(67 TOPS、7–25 W)是內建 GPU 的邊緣 AI 電腦，能即時執行深度模型、ROS 2 與 NanoOWL 這類開放詞彙模型，是機器人的視覺大腦。
- 模型最佳化手段：量化(FP32→FP16/INT8，與第 3、11 章的量化同一思想)、剪枝、知識蒸餾;部署格式 ONNX(通用)、TensorRT(NVIDIA GPU)、TFLite(行動與微控制器);更根本的是選用為邊緣設計的架構(MobileNet、YOLO nano)。
- 三個效能指標：延遲(反應速度，即時控制的關鍵)、吞吐量(FPS，處理能力)、功耗;兩者不等價，批次處理提高吞吐量卻增加延遲。系統設計應做端到端的延遲預算，而非只量測模型推論時間。
- ROS 2 以節點與話題的發布訂閱架構整合機器人各模組;視覺節點的典型結構為訂閱影像、cv_bridge 轉換、處理推論、發布結果，並須保留時間戳記以供時間同步與座標轉換(tf2，呼應第 4 章齊次座標)。
- 部署實務課題：環境變異造成的分布偏移、溫度與散熱降頻、失效模式應設計為「失效即安全」、模型更新與遠端維運、以及蒐集錯誤案例重訓的資料回饋迴圈。

習題

1. 請列出邊緣運算相較純雲端架構的五個理由，並各舉一個機器人或工業場景說明。

   提示與參考解答：一、低延遲：機器人避障需毫秒級反應，雲端一來一回數百毫秒不可接受。二、頻寬與成本：一整條產線數十支攝影機持續上傳成本驚人。三、可靠性：田間、隧道、災難現場網路不穩或無網路。四、隱私：人臉影像在端上處理完只回報數量，遠優於上傳雲端。五、功耗與自主性：電池供電的無人機無法負擔持續高功率傳輸，自主系統本質上不應依賴外部連線。

2. 比較端上、近端邊緣、雲端三種架構的算力、延遲、隱私與斷網可用性。為什麼說它們不是互斥的?

   提示與參考解答：端上(AI Camera 的感測器上推論)：算力受限於感測器晶片上的記憶體額度、延遲最低、隱私最佳、斷網可用。近端(Jetson Orin Nano Super)：算力中高、延遲低、隱私佳、斷網可用。雲端：算力極高、延遲高且受網路影響、隱私需審慎、斷網不可用。三者可分層協作：端上做高頻低算力的預篩(有沒有事發生)、近端做精細辨識(發生了什麼，必要時用開放詞彙問新的問題)、雲端做長期分析與遠端存取。要補一句常被忽略的事：這三層的分界不是固定的。感測器上推論把第一層往前推進了晶片裡(圖 17-3)，於是「預篩」這一層連處理器都不必醒著。

3. 感測器上推論(IMX500)的能力邊界在哪裡?為什麼說它「買到的不是速度」?

   提示與參考解答：邊界：處理器與記憶體極有限，無法執行 YOLO 等深度模型(即使 nano 版也太重)。兩種用法：一是端上做極簡偵測與觸發(移動偵測、Haar 級聯人臉偵測、顏色追蹤，偵測到才拍照或通知);二是作為網路攝影機，串流影像給更強的裝置(電腦或 Jetson)處理。價值不在算力，而在極低成本與功耗讓視覺能佈署到任何角落。

4. 模型量化是什麼?它與第 3 章的影像量化、第 11 章的 JPEG 量化在思想上有何共通?

   提示與參考解答：模型量化把權重與運算從 32 位元浮點降到 16 位元浮點或 8 位元整數，模型縮小、加速、省電，精度僅損失少許。共通思想是「用精度換效率」：第 3 章把連續光強度離散成有限灰階、第 11 章 JPEG 把 DCT 係數除以量化步長取整(丟棄人眼不敏感的高頻)、本章把模型參數的精度降低——都是在可接受的品質損失下換取儲存與運算的大幅節省。

5. ONNX 與 TensorRT 各扮演什麼角色?為什麼訓練框架不適合直接用於部署?

   提示與參考解答：訓練框架(PyTorch、TensorFlow)為靈活性而設計，推論效率非最佳。ONNX 是開放的中介格式，讓模型能在不同框架與硬體間流通。TensorRT 是 NVIDIA 針對自家 GPU 的推論最佳化引擎，會分析模型結構、融合運算層、選最佳核心並套用量化，在 Jetson 上常帶來數倍加速。微控制器則用 TensorFlow Lite for Microcontrollers 等輕量執行環境。

6. 延遲與吞吐量有何差異?為什麼提高吞吐量有時反而增加延遲?各適合什麼應用?

   提示與參考解答：延遲是單張影像從輸入到輸出的時間(反應速度);吞吐量是每秒能處理幾張(處理能力)。批次處理一次處理多張可提高吞吐量，但要等湊滿一批，單張延遲反而變長。機器人避障等即時控制重延遲;離線影片分析重吞吐量。這與第 14、16 章「依應用選指標」的工程判斷同出一轍。

7. 什麼是端到端延遲預算?為什麼「只量測模型推論時間」是危險的?

   提示與參考解答：端到端延遲包含影像擷取、前處理、模型推論、後處理、決策通訊、致動器反應等全部環節，模型推論往往只佔一部分。延遲預算是先確定任務允許的總延遲(如依機器人速度與煞車距離推算)，再回頭分配給各環節。只量測推論時間會嚴重低估實際反應時間，導致系統在真實場景中反應不及。

8. 若量測發現 YOLO 在你的邊緣裝置上跑不到目標 FPS，你會依什麼順序嘗試最佳化?

   提示與參考解答：依程式 17-1 的順序：一、換更小的模型(n < s < m < l < x);二、降低輸入解析度(640→416→320);三、量化(FP16 或 INT8);四、改用 TensorRT 等推論引擎;五、降低處理影格率(不必每一影格都偵測，可搭配第 12 章的追蹤填補中間影格)。每一步都要重新量測延遲與 mAP，評估精度損失是否可接受。

9. ROS 2 的節點與話題是什麼?這種架構帶來什麼好處?

   提示與參考解答：節點是獨立的功能模組(相機、視覺辨識、導航等)，話題是節點間以「發布-訂閱」方式交換訊息的通道。好處：鬆耦合設計讓各模組能獨立開發、除錯與維護，可用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作;模組也能重複使用，團隊可分工。

10. ROS 2 視覺節點為什麼要保留影像訊息的時間戳記(header)?這與第 4 章有什麼關聯?

    提示與參考解答：因為機器人要把「影像看到什麼」與「當時機器人在哪裡」對應起來，需要精確的時間同步(影像有處理延遲，不能用當下的位置)。與第 4 章的關聯：ROS 2 的 tf2 座標轉換系統正是齊次座標與座標框架的實作，把相機座標系的偵測結果轉換到機器人或世界座標系，才能用於導航決策。

11. 部署到真實環境時，有哪些實驗室不會遇到的課題?請至少說明三項及其對策。

    提示與參考解答：一、環境變異造成分布偏移(第 15 章)：現場光線背景角度多變，模型表現下滑;對策是在真實環境蒐集資料、大量資料增強、持續監控上線表現。二、溫度與散熱：長時間高負載發熱會自動降頻導致效能驟降;需散熱設計與功耗管理。三、失效模式：系統必然會出錯，應設計成「失效即安全」(停止、警示)而非盲目運作。另有模型更新與遠端維運、資料回饋迴圈等。

12. (綜合設計題)請為「校園自主巡邏機器人」設計視覺系統架構：列出硬體選擇、模型與最佳化策略、ROS 2 節點劃分、延遲預算、以及失效安全設計，並說明每項決策的理由。

    提示與參考解答：本題為開放設計題，評分重點在決策理由的完整性。參考方向——硬體：Jetson 作為主控(需即時深度學習與 ROS 2)，搭配相機與可能的深度感測器;若需多點監測可加樹莓派 5 配 AI Camera 的節點(感測器上推論，影像不出裝置，正好回應校園的隱私疑慮)。模型：YOLO nano 或小型模型加 FP16/TensorRT 最佳化，兼顧速度與精度。節點劃分：相機節點、視覺辨識節點、定位節點、導航節點、決策節點，以話題通訊並保留時間戳記。延遲預算：依巡邏速度與煞車距離反推總延遲上限，分配給擷取、前處理、推論、後處理、通訊、致動各環節。失效安全：視覺節點逾時或當機則減速停止並警示;多層防護不單靠視覺(可加超音波或碰撞感測器，呼應第 16 章)。另應考慮隱私(校園有大量人員，建議端上處理、不儲存人臉)。

13. 把同一個模型從 PyTorch 的 .pt 換成 ONNX Runtime 執行，什麼都沒改卻快了將近七成。這個加速從哪裡來?請說明訓練框架與推論引擎在設計目標上的差異。

    提示與參考解答：圖 17-7 實測同一顆 YOLO26n、同一張影像、同一台機器：

PyTorch 中位 55.11 毫秒，ONNX Runtime 中位 32.82 毫秒，約 1.68 倍。加速來自三件事。一、圖最佳化：推論引擎會把整個計算圖讀過一遍，把卷積與批次正規化融合成單一運算（這兩者在推論時都是固定的線性變換，可以事先合併）、消去不會用到的分支、並重排運算順序以利快取。二、沒有自動微分的包袱：訓練框架的每一個張量都要保留反向傳播所需的紀錄，推論引擎直接把這整套機制拿掉。三、沒有 Python 解譯器的逐層派遣成本：模型被編成一張靜態圖，執行時不必每一層都回到 Python 問一次「下一步做什麼」。代價是失去靈活性——靜態圖不能隨意改結構、除錯也困難，但部署時模型已經定案，這個代價不存在。結論：匯出成部署格式是本章最划算的最佳化，因為它不損失任何精度（圖 17-6 的 FP32 ONNX 與 PyTorch 的 mAP 完全相同）。

14. 「量化可以讓模型更小、更快、更省電」——這句話在本章的實測中只有一半成立。請說明圖 17-5 量到了什麼，以及為什麼會這樣。

提示與參考解答：圖 17-5 在一顆純 CPU 上量得：ONNX FP32 是 9.48 MB、33.5 毫秒;FP16 是 4.81 MB、42.6 毫秒;INT8 是 2.77 MB、58.7 毫秒。體積確實一路縮小（而且幾乎正好按位元數比例），但速度反而一路變慢，INT8 甚至比 FP32 慢了將近八成。原因是量化的加速要靠硬體配合，而這台機器沒有這個配合。若處理器沒有原生的半精度運算單元，推論引擎必須把 FP16 權重轉回 FP32 才能計算，轉換本身就是額外指令;動態 INT8 量化更要在執行期間統計激活值範圍、即時算縮放係數、並在整數與浮點之間反覆轉換。這些額外動作吃掉了省下來的乘法時間。在有 Tensor Core 的 Jetson（FP16）或有 NPU / DSP 的裝置（INT8）上，結論會完全相反——那裡量化是數倍的加速。本題真正要記住的是方法論：任何最佳化都必須在它最終要執行的那顆晶片上驗證，而不能從規格書或他人的部落格推論。

15. 量化掉多少精度?若必須用 INT8，有哪些做法可以把損失救回來?另外請說明為什麼「逐通道」決定縮放係數比「逐層」好。

提示與參考解答：圖 17-6 在 600 張保留集影像上實測：FP32 的 mAP@0.5 是 61.61%、mAP@0.5:0.95 是 46.19%;FP16 幾乎完全相同（61.45 / 46.17，差異在量測噪聲範圍內）;INT8 降到 59.19 / 43.22，約掉 2.4 與 3.0 個百分點。所以 FP16 可以視為免費，INT8 要付代價。救回來的做法有三：一、訓練後靜態量化——先拿一小批有代表性的影像跑過一遍，統計每一層激活值的真實範圍，而不是在執行期間現場猜;二、量化感知訓練——訓練時就在前向傳播中模擬量化誤差，讓權重自己學會躲開量化不友善的分布;三、混合精度——把對量化特別敏感的層（通常是第一

層與最後的預測頭）留在較高精度。逐通道優於逐層的原因在誤差的算式：量化誤差最大為縮放係數的一半，而縮放係數與該組數值的範圍寬度成正比。一整層裡只要有一個離群的大權重，整層的範圍就被它撐開，其餘所有權重的精度一起變差;改成每個通道各算自己的縮放係數，離群值的影響就被關在它所在的那個通道裡。同理也說明了為什麼「剪掉離群值」這類校正手段有效。

16. 有兩個旋鈕可以換取速度：換小模型，或降低輸入解析度。若延遲預算是 60 毫秒，你會怎麼選?請依圖 17-8 的實測說明，並指出這題的正確答案形式是什麼。

提示與參考解答：圖 17-8 把兩個旋鈕交叉實測（延遲 / mAP@0.5）：n@320 是 16.9 毫秒 / 46.61%，n@480 是 32.5 / 56.08，n@640 是 54.5 / 61.99;s@320 是 51.2 / 60.89，s@480 是 91.8 / 68.36，s@640 是 170.9 / 70.42。在 60 毫秒的預算內，可行的有 n@320、n@480、n@640 與 s@320。關鍵的觀察是 s@320（51 毫秒、60.9%）與 n@640（55 毫秒、62.0%）幾乎落在同一點上——也就是說「小模型看大圖」與「大模型看小圖」在這個預算下打成平手。所以這題沒有「換小模型優先」或「降解析度優先」這種普適的答案;正確的答案形式是一條帕雷托前緣（Pareto frontier）：把兩個旋鈕交叉量過，畫出延遲對精度的散點圖，找出「在同樣延遲下精度最高」的那些組合，再依預算在這條前緣上挑一點。而且這條前緣與資料集、物件大小分布、硬體都有關——若你的物件偏小，解析度這個旋鈕會比模型大小更關鍵（第 16 章圖 16-12）;若物件都很大，降解析度幾乎是免費的。程式 17-1 列的五步最佳化順序是一個合理的起點，而不是一條定律。

17. 在微控制器這一級，「裝不裝得下」往往比「跑多快」更要緊。請說明模型權重所需記憶體的算式，並討論為什麼實際需要的記憶體比權重還多。

提示與參考解答：權重所需位元組 = 參數量 × 每個參數的位元組數，FP32 是 4 位元組、FP16 是 2、INT8 是 1。YOLO26n 約 257 萬個參數，故 FP32 約 9.8 MB、FP16 約 4.9、INT8 約 2.5。但執行時還需要兩塊額外的記憶體。一是激活值：每一層的輸出都要暫存給下一層用，這塊的大小與輸入解析度的平方成正比——把 imgsz 從 640 降到 320，激活值的記憶體掉到四分之一，這是解析度這個旋鈕除了速度之外的第二個好處。二是推論引擎自身的執行期開銷。這個算式在 17.2.1 節的感測器上推論那一層特別重要：IMX500 把推論做在感測器晶片裡，而晶片上的記憶體額度要同時容納韌體、網路權重與推論的工作記憶體，所以它只收 INT8、而且能

跑的模型清單很短。換句話說，那三條硬規定不是刁難，是這個算式的直接後果。所以在這一層要跑的是專為此設計的小模型（MobileNet 的極小版本、YOLO 的 nano 級），而不是想辦法把大模型塞進去。

18. 為什麼量測推論延遲一定要「暖機」?若不暖機，量到的數字會偏差多少?請依實測說明，並指出暖機在真實部署中對應到什麼問題。

提示與參考解答：圖 17-9 實測前五次推論的耗時是 163.6、43.8、36.2、32.0、32.3 毫秒，而穩態中位數是 32.83 毫秒。第一次慢了將近五倍。若把包含前幾次的所有量測取平均，會得到 41.78 毫秒，比真實的穩態值高估了約 27%。第一次慢的原因有：推論引擎要配置記憶體與工作緩衝區、為當前的張量形狀選擇最佳的運算核心（有些引擎會現場試跑幾種實作）、權重第一次被讀進快取、以及可能的即時編譯。正確的做法是先跑幾次不計時的暖機，再開始量測（程式 17-1 的 model(img) 那一行就是在做這件事）。但暖機在真實部署中並不是可以忽略的量測瑕疵，而是一個要設計的問題：機器人剛開機的第一個影格就會慢上好幾倍，若系統在開機後立刻開始移動，那一瞬間的反應能力與穩態完全不同。對策是在系統進入待命狀態時就先用一張假影像把模型跑熱，並且不要在暖機完成前允許機器人移動。同一個道理也適用於切換解析度或切換模型的時刻。

19. 編延遲預算時該用平均延遲、中位數、還是第 99 百分位?請依圖 17-12 的實測說明，並解釋為什麼對控制迴路而言這個選擇是安全問題。

提示與參考解答：圖 17-12 實測同一個模型連續推論的延遲分布：中位數 35.3 毫秒、第 90 百分位 39.8、第 99 百分位 49.2、最大值 62。分布有明顯的長尾，來源是作業系統排程、其他行程搶佔 CPU、快取失效、以及影像內容本身的差異（物件多時後處理較久）。若以 40 毫秒為預算，實測有 21.5% 的影格超出——而中位數看起來明明還有餘裕。這就是為什麼不能用中位數或平均值編預算：它們描述「典型情況」，而即時系統的風險全在尾端。對串流播放來說，偶爾一格遲到只是輕微的不順;但對控制迴路而言，那一格就是控制訊號遲到，而機器人在那一瞬間仍然在以原速移動（推導 17.4.3 的反應距離）。所以安全攸關的系統應以 p99 甚至最大值編預算，並在這之上再留餘裕;同時必須定義「超時了怎麼辦」——沿用上一格的結果（並限制最多連續幾格）、降速、或停止，這就是 17.7 節失效即安全的起點。

20. 同一個系統，圖 17-11 說推論佔七成、圖 17-13 說推論只佔三成。這兩個數字並不矛盾。請說明差異從哪裡來，以及這對「該最佳化哪一段」的判斷有什麼意義。

    提示與參考解答：差別在量測邊界畫在哪裡。圖 17-11 量的是「視覺處理鏈」：影像解碼 8.75 毫秒（20.6%）、模型推論 29.73 毫秒（69.8%），合計 42.59 毫秒。在這個邊界內推論是絕對的主角，最佳化模型是對的。圖 17-13 量的是「完整的機器人反應鏈」：再加上相機曝光與傳輸、決策、通訊、以及致動器反應，總計約 145.9 毫秒，推論佔比掉到約三成，而致動器反應那一段比推論還久。意義有兩層。第一層是阿姆達爾定律（Amdahl's law）的實際後果：最佳化某一段所能帶來的整體改善，上限是那一段的佔比。即使把推論加速到零，圖 17-11 的鏈仍有 12.9 毫秒的地板，圖 17-13 的鏈仍有 100 毫秒以上。第二層更重要：先定義邊界再量測，否則會把力氣花錯地方。若你關心的是「機器人多久才會煞車」，那麼只盯著推論時間最佳化，很可能在佔三成的那一段拚命，卻對另外七成（例如換一顆延遲更低的相機、或縮短通訊的輪詢週期）完全沒動過。

21. 「批次處理可以提高吞吐量，代價是增加單張延遲」——這是教科書的標準說法，但圖 17-10 的實測顯示吞吐量反而下降。為什麼?那個標準說法在什麼條件下才成立?

    提示與參考解答：圖 17-10 實測：batch=1 是 18.5 FPS，batch=8 降到 13.9 FPS。不是原地踏步，是倒退。標準說法成立的前提是「硬體還有閒置的平行處理能力」。GPU 有數千個運算單元，單張影像往往餵不滿，湊成一批可以讓這些單元同時工作，於是吞吐量上升。但本例的 CPU 核心數少、而推論本身已經把核心用滿，沒有閒置的平行度可以利用;這時湊批次不但沒有好處，還多了更大的中間張量、更差的快取命中率與額外的記憶體搬運，於是整體變慢。所以批次處理的正確判準是：先看硬體利用率（GPU 的 occupancy、CPU 的核心負載），只有在明顯有閒置時才值得試，而且要實測。順帶一提，即使在 GPU 上批次有效，對機器人避障這類任務仍然不該用：湊滿一批的等待時間直接加在延遲上，而延遲才是這類任務的關鍵指標（17.4.1 節）。批次處理屬於離線影片分析的世界。

22. 若延遲不夠，可以不必每一影格都偵測——中間的影格沿用上一次的結果。這個做法的代價是什麼?請依圖 17-14 的實測量化，並說明如何設計一個安全的跳格策略。

提示與參考解答：圖 17-14 拿一段有移動物件的影片實測，把「沿用的舊框」與「同一影格上重新偵測得到的框」比較：每格偵測時 IoU 是 1.000、中心誤差 0 像素;每 2 格偵測降到 0.927 / 5.5 像素;每 5 格降到 0.761 / 19.5 像素;每 8 格只剩 0.612 / 34.7 像素。所以代價是「位置資訊逐漸過期」，而且誤差隨跳格數近似線性成長。注意這個誤差與物件的移動速度、以及物件在畫面中的大小有關——快速接近的物件在影像中位移最大，而那正是最危險的情況。安全的跳格策略有三個要素。一、用追蹤器填補空檔：在不偵測的影格上用光流或 KCF 等輕量追蹤器更新框的位置（第 12 章），而不是完全沿用舊框;追蹤的成本遠低於偵測，卻能把誤差壓下來。二、把跳格數與速度掛鉤：機器人慢速時可以多跳，高速時必須每格偵測，或者反過來——高速時先降速再維持偵測頻率。三、設定沿用的上限：若連續 N 格沒有新的偵測結果（例如偵測執行緒卡住），系統必須進入安全狀態，而不是無限期地相信一個越來越舊的框。

23. 現場的影像品質會以各種方式退化。若只能先解決一種，你會先解決哪一種?請依圖 17-17 的實測排序，並說明這個排序為什麼可能反直覺。

提示與參考解答：圖 17-17 對同一批保留集影像施加五種退化，量測 mAP@0.5 的掉幅（百分點）：低光 −1.16、雜訊 −18.11、低解析 −20.85、JPEG 品質 15 −23.42、運動模糊 −44.34。所以答案是先解決運動模糊，它的破壞力是低光的將近四十倍。這個排序反直覺的地方在於「低光」幾乎無害。原因是現代模型在訓練時已經見過大量曝光不足的影像，而且卷積網路主要依賴的是邊緣與紋理的相對關係，整體變暗並不會破壞這些關係（第 8 章的亮度與對比調整也印證這點）;運動模糊卻直接把邊緣與紋理抹掉——它刪除的是模型賴以判斷的資訊本身。工程上的意義很具體：與其加裝補光燈，不如先把快門時間縮短（用全域快門、提高增益、或換一顆更靈敏的感測器，第 7.6.3 節）、或是降低機器人與雲台的運動速度。同理，JPEG 壓縮品質與傳輸解析度這兩項是系統設計者可以直接控制的，卻常常為了省頻寬被調得太低——實測顯示它們的代價與雜訊相當，絕不是免費的。

24. 把推論搬進感測器裡，主機到底省下了什麼?請說明 IMX500 這條路徑與傳統路徑的差別，並解釋為什麼這個設計同時省了頻寬、功耗與隱私。

提示與參考解答：傳統路徑是「感測器輸出影像 → 主機解碼與前處理 → 主機跑模型 → 得到結果」，這段期間主機的處理器幾乎做不了別的事。IMX500 的路徑是「感測器輸出影像，同時附上一包張量後設資料」，裡面就是模型的輸出——偵測框、類別、分數;主機只要讀那包後設資料就好。三個好處都來自同一件事——影像

沒有離開感測器。頻寬：要傳的只是幾個框的座標與分數，而不是一張影格。功耗：省掉傳輸，也省掉主機的解碼與推論，主機因此可以大部分時間休眠（圖 17-4 的續航帳）。隱私：這是最有力的隱私保護設計——不是承諾不看，而是技術上沒有東西可看（17.1.1 節）。但要記住 17.2.1 節的提醒：它買到的不是速度。感測器上的推論並不比主機快，它買到的是「主機的時間」與「不必搬影像」。所以它最適合的場景是主機還有別的正事要做、或者主機大部分時間應該在睡覺;若你的系統只有視覺這一件事而且主機閒著，好處就小得多——這又是一次「適合勝於最強」。

25. （綜合設計題）模型上線之後沒有標準答案，你要怎麼知道它開始失準了?請設計一套監控方案，列出不需要人工標註就能算的訊號、警示線的訂法、以及這套方案「看不見」什麼。

提示與參考解答：本題為開放設計題，評分重點在三層架構是否完整，以及是否誠實地指出侷限。參考方向——第一層自動監控，三個訊號。一、信心分數分布：記錄每張影像最高信心分數的移動平均，以及「幾乎沒偵測到東西」的影像比例。圖 17-19 實測在影像變模糊的那一刻，前者從 0.841 掉到 0.443、後者從 0% 升到 34%，兩者都在零標註的條件下清楚指出異常。二、輸出統計量：每張影像的物件數、各類別比例、框的大小與位置分布。這些在穩定場域中應該穩定，而且與應用語意直接相關（產線每分鐘的工件數少了一成就該有人看）。三、系統指標：p99 延遲與超出預算的影格比例，它們能在硬體降頻或行程競爭「造成失效之前」就看出徵兆。警示線的訂法：上線初期先收集一段正常運作期的資料，算出平均與標準差，警示線訂在平均減去兩倍標準差，並留意季節性（戶外應用的日照、產線的班別）造成的正常波動。第二層定期抽樣複核：每週固定抽一批上線影像人工標註，算出真實的 mAP 或召回率;這批資料必須是模型沒看過的（第 16 章圖 16-18），樣本數也要夠大才報得出有意義的數字（第 14.5.3 節的標準誤）。第三層資料回饋：把複核出的錯誤案例標註後納入下一輪訓練，重訓後再走完整的驗收流程，並與舊版並行比較一段時間。看不見什麼（這是本題的關鍵）：上述訊號只能說「有事發生」，不能說「錯在哪裡」;而且有一整類失效它們完全偵測不到——模型自信滿滿地持續犯錯。第 15 章的對抗脆弱、第 16 章的捷徑學習都屬於這一類：信心高、輸出數量正常、延遲漂亮，但答案是錯的。所以自動監控永遠不能取代定期的人工抽樣複核。另外也要設計更新機制本身的安全性：更新失敗要能自動回復舊版、更新時機器人須先停止、新版要能與舊版並行比較後才全面切換。

參考資料與延伸閱讀

- Raspberry Pi 官方文件與 AI Camera 產品規格：https://www.raspberrypi.com(本章叫獸開講與 17.2.1 節之史料與規格來源)
- NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南：https://developer.nvidia.com/embedded-computing
- ROS 2 官方教學文件：https://docs.ros.org
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.(第 15 章亦引用)
- Jacob, B., et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR, 2018.
- Ultralytics 部署與匯出文件：https://docs.ultralytics.com
- Sony IMX500 智慧視覺感光元件技術資料與模型轉換工具鏈說明。
- NVIDIA Jetson Orin 開發者文件與 TensorRT 最佳化指南：https://developer.nvidia.com/embedded-computing
- NVIDIA NanoOWL 專案（OWL-ViT 的 TensorRT 最佳化與樹狀提示）：https://github.com/NVIDIA-AI-IOT/nanoowl
- Minderer, M., et al., Simple Open-Vocabulary Object Detection with Vision Transformers (OWL-ViT), ECCV, 2022.
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)

影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 18 章 期末專題：4P 實踐

本章學習目標

- 闡述 4P 創意學習(Projects、Passion、Peers、Play)的理念與其在工程教育中的意義。
- 說明創意學習螺旋的五個階段，並以之規劃專題的迭代開發流程。
- 選定一個兼具熱情與可行性的專題題目，並拆解為可執行的里程碑。
- 運用本書十七章的技術，整合出一個端到端的視覺應用系統。
- 進行有效的團隊協作、版本管理與技術文件撰寫。
- 完成專題成果的展示、反思與評估，並規劃後續的學習路徑。

叫獸開講|終身幼兒園：千年來最重要的發明是什麼?

世紀之交，一場研討會上有人拋出了一個問題：過去一千年來，最重要的發明是什麼?與會者陸續給出答案——有人說是印刷術，讓知識得以大量流通;有人說是蒸汽機，啟動了工業革命;有人說是電燈泡，把人類的活動延伸到夜晚。輪到麻省理工學院媒體實驗室的 Mitchel Resnick 教授時，他給了一個讓全場愣住的答案：幼兒園。

這聽起來像個玩笑，但 Resnick 是認真的。他的理由是：一百多年前誕生的幼兒園，是人類教育史上一次根本的創新——它不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解。孩子們用積木蓋城堡、用蠟筆畫故事、在遊戲中學會協商與分享。這個過程，正是創造力的搖籃。

Resnick 觀察到一個令人遺憾的現象：離開幼兒園之後，學校教育就變了樣—

—孩子開始長時間坐在課桌前，填寫習題、聽講、應付考試。更糟的是，近年連幼兒園本身都在「小學化」，積木與手指畫被數學練習卷與識字卡取代。他的主張因此格外鮮明：不該讓幼兒園變得像其他學校，而該讓其他學校——甚至整個人生——變得更像幼兒園。

這不是要大人去玩積木，而是要保留幼兒園那套學習方式的精髓。Resnick 帶領的研究群就叫「終身幼兒園」(Lifelong Kindergarten)，他們把這套精髓提煉成四個以 P 開頭的原則：專題(Projects)、熱情(Passion)、同儕(Peers)、遊玩(Play)——讓人們基於自己的熱情、與同儕協作、以遊玩的精神，投入專題的創作。這個研究群的成果你可能早就用過：他們開發了全球最大的兒童程式學習平台 Scratch，也與樂高合作催生了 LEGO Mindstorms 機器人套件。Resnick 的學術血脈同樣耀眼——他的博士指導教授是建構主義學習理論的先驅 Seymour Papert。

而這，正是這門課從第一章走到現在的隱藏主線。回頭看看：每一章開頭的「叫獸開講」，講的都是一個真實的人、在真實的處境中，因為某種熱情或不甘而做出的創造——傅立葉在行政庶務之餘寫下被退稿的論文、張正友用一張紙取代昂貴的標定物、Muybridge 為了一場關於馬蹄的爭論架起一整排相機、Redmon 在成就的頂點選擇放下、樂鑫的一群工程師讓五美元的晶片改變了創客世界。這些故事不是花絮，它們是想告訴你：技術從來不是冷冰冰的公式，而是活生生的人，帶著熱情與判斷，一步步做出來的。

現在，輪到你了。期末專題不是一份要交出去的作業，而是這門課給你的一次機會：去做一個你真心想做的東西，和你的夥伴一起，帶著玩的心情，把這十七章學到的技術變成一個真實存在的作品。做出來之後，它會是你的——不是老師的、不是課本的，是你的。

思考題：回想你的求學歷程，哪一次學習經驗讓你最投入、學得最深刻?它符合 4P 中的哪幾項?為什麼那次經驗與「為了考試而讀書」感覺如此不同?

18.1 4P 創意學習

4P 是本課程的教學設計骨架，也是期末專題的評量精神。理解它，你才會明白為什麼這門課要這樣上、專題要這樣做。



這是一張說明圖，不是實測。4P 是 18.1 節的四個原則，而它們並不是期末才登場的口號——本書從第 1 章開始就按這個骨架設計：每一章都以 Lab 收尾（Projects）、每一章的叫獸開講講的都是某個人的熱情（Passion）、每一章都要求你把結果與同學比對（Peers）、而幾乎每一個 Lab 都在要求你試參數、看結果、再調整（Play）。這四根柱子裡最常被低估的是熱情，因為它決定了你願意為一個卡住的問題坐多久;最常被誤解的是遊玩，它不是不認真，而是「投入來自好奇而非壓力」。接下來的每一張圖，都會把其中一根柱子換算成可以量的東西。

圖 18-1　4P：這門課其實一直在用的教學骨架

18.1.1 Projects：專題

人們在動手創造具體事物的過程中學得最深。聽講與寫習題能傳遞資訊，但真正的理解來自於「試著做出來卻卡住，然後想辦法解決」的那個當下。當你為了讓機器人認出障礙物而反覆調整參數、當你發現訓練出來的模型在教室裡很準、拿到走廊就失靈——那些挫折時刻學到的東西，遠比課本上的段落深刻。這也是為什麼本書每一章都有 Lab，而整門課以專題收尾：知識要透過創造才能真正內化。

18.1.2 Passion：熱情

這是 4P 中最容易被忽略、卻最關鍵的一項。當人們投入自己真正在乎的題目時，願意付出更多時間、承受更多挫折、學得更多東西。所以選題時，叫獸的建議永遠是：

選一個你自己會想用、會想給家人朋友看的東西。是替阿嬤的果園做病蟲害偵測?是幫社團管理器材借還?是做一個能陪你打電動的機器人?題目不必偉大，但必須是你的。這裡要補一個很實際的理由，而不只是勵志的說法：熱情決定了你願意為一個卡住的問題坐多久，而專題的進度幾乎都是在那些時刻產生的。圖 18-3 量過一個例子：同一條零件計數的管線，照抄預設參數時完全數對的比例只有 8%;五輪調整之後是 90%。那五輪不難，但每一輪都要先看懂上一輪為什麼失敗——而願意去看懂，就是熱情在工程上的樣子。反過來說，一個你不在乎的題目，你會在第 0 輪那個難看的數字出現時就想換題目。

請特別注意：熱情不是「一開始就有」，更多時候是「做著做著長出來的」。若你暫時想不到有熱情的題目，就先動手做一個看起來有趣的，過程中的小成就感會慢慢點燃它。

18.1.3 Peers：同儕

創造很少是孤獨的事。與同伴協作、互相給予回饋、分享彼此的發現，能讓每個人走得更遠。專題採分組進行，不只是為了分攤工作量，更是因為：當你必須向隊友解釋你的做法時，你會發現自己理解上的漏洞;當隊友提出你沒想過的角度時，你的方案會變得更好。

同儕也不限於同組隊友。向其他組請教、參考開源社群的做法、在網路上提問——這些都是同儕學習。承第 13 章 ORB 的故事、第 17 章樹莓派的故事：整個開源與創客文化，本身就是「同儕協作」在全球尺度上的展現。

18.1.4 Play：遊玩

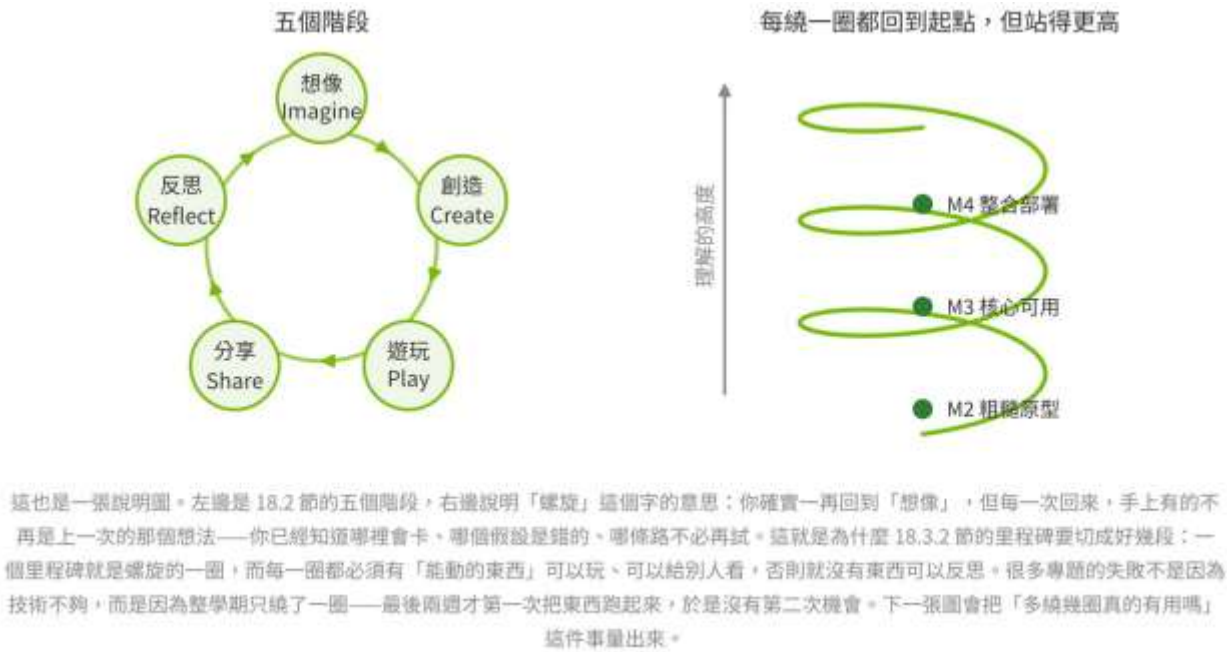
遊玩的精神，指的是願意嘗試、不怕失敗、樂於實驗的心態。它與「不認真」正好相反——遊玩需要高度的投入，只是這份投入來自好奇而非壓力。在專題中，這意味著：大膽試試看那個不確定會不會成功的想法、把出錯的結果當成有趣的線索而非

災難、在做出基本功能後想想「如果加上這個會怎樣」。

工程上，遊玩的精神有一個實際的名字叫「快速原型」：先做一個粗糙但能動的版本，從中學習，再改進。這比在紙上完美規劃三個月然後一次做對，更符合真實的創造歷程。

18.2 創意學習螺旋

Resnick 觀察幼兒園裡孩子玩積木的過程，提煉出一個循環，稱為創意學習螺旋——這也是專題開發最貼近真實的流程模型：



這也是一張說明圖。左邊是 18.2 節的五個階段，右邊說明「螺旋」這個字的意思：你確實一再回到「想像」，但每一次回來，手上有的不再是上一次的那個想法——你已經知道哪裡會卡、哪個假設是錯的、哪條路不必再試。這就是為什麼 18.3.2 節的里程碑要切成好幾段：一個里程碑就是螺旋的一圈，而每一圈都必須有「能動的東西」可以玩、可以給別人看，否則就沒有東西可以反思。很多專題的失敗不是因為技術不夠，而是因為整學期只繞了一圈——最後兩週才第一次把東西跑起來，於是沒有第二次機會。下一張圖會把「多繞幾圈真的有用嗎」這件事量出來。

圖 18-2　創意學習螺旋：為什麼是螺旋而不是圓圈

1. 想像(Imagine)：產生一個想法——「我想做一個能自動辨識回收物的垃圾桶」。
2. 創造(Create)：動手把它做出來，哪怕只是最粗糙的版本。
3. 遊玩(Play)：實際玩玩看、測試它，在各種情況下試探它的行為。
4. 分享(Share)：給別人看、聽取回饋——同學、老師、家人的反應往往能點出你沒發現的問題。

5. 反思(Reflect)：想想哪裡好、哪裡不好、為什麼——然後回到「想像」，展開下一輪。

之所以稱為「螺旋」而非「循環」，是因為每繞一圈，你都回到起點但站在更高的位置：你的想法更成熟、技術更純熟、對問題的理解更深。這正是專題應有的節奏——不要期待一次做對，而要規劃多次迭代。

這個螺旋與工程界的敏捷開發、與您在本書各章 Lab 中反覆經歷的「試參數、看結果、再調整」，骨子裡是同一件事。而它也解釋了為什麼 18.3 節的專題時程要切成多個里程碑：每個里程碑就是螺旋的一圈。「多繞幾圈真的有用嗎」這件事可以量。圖 18-3 拿本章那個零件計數的任務跑了五輪：第 0 輪照抄範例的預設參數，測試集上完全數對的比例是 8%;第 1 輪把全域門檻換成自適應門檻，跳到 83%;五輪之後是 90%。這張圖還順便說明了兩件不太中聽的事。第一，幾乎所有的進步都來自同一個改動——後面四輪加起來只移動了幾個百分點。卡住的時候該問的往往是「我這一步的假設對不對」，而不是「這個數字再調大一點會不會好一些」。第二，最後兩輪在開發集上還在進步，測試集卻退了一點點;你每在開發集上挑一次最好的參數，就把一點點開發集的運氣寫進了系統裡。所以螺旋要繞，但每一圈的成績要報在一份從頭到尾不碰的測試集上。

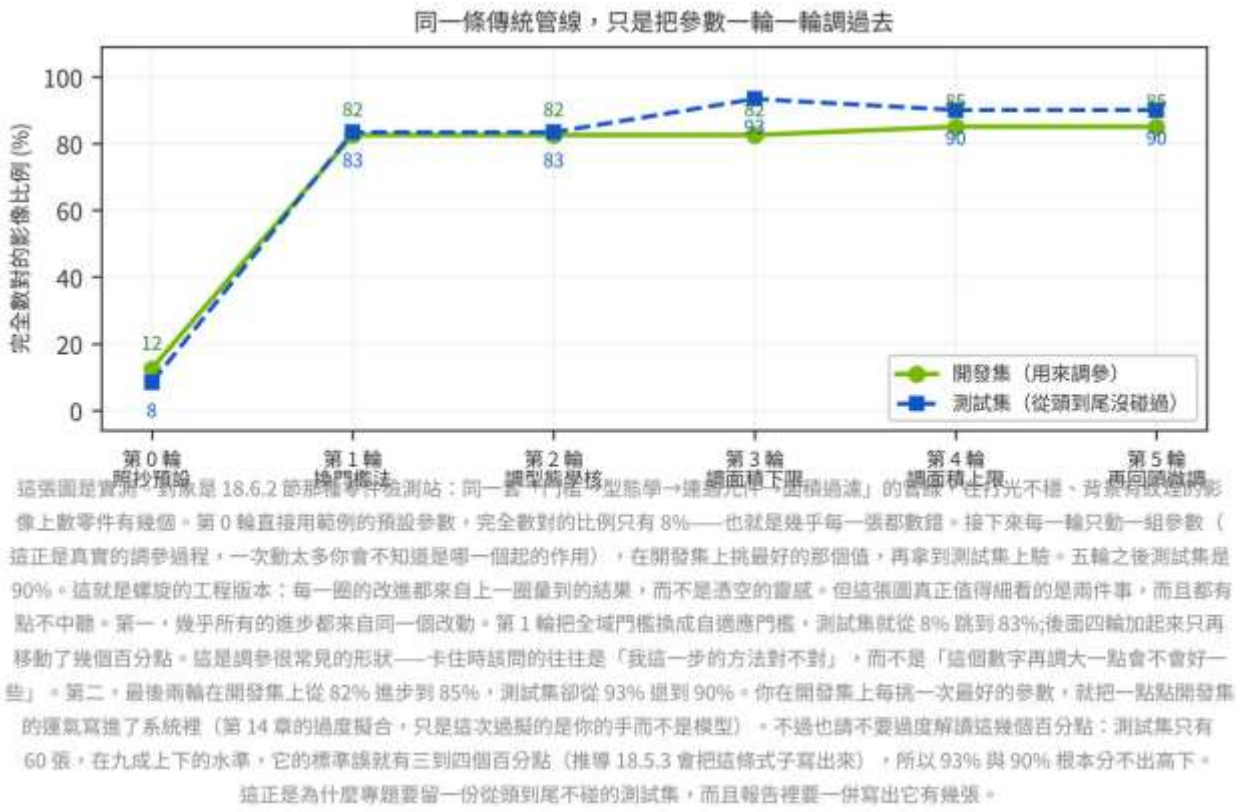


圖 18-3　多繞幾圈真的有用嗎：五輪調參的實測

18.3 專題的規劃與執行

18.3.1 選題：熱情與可行性的交集

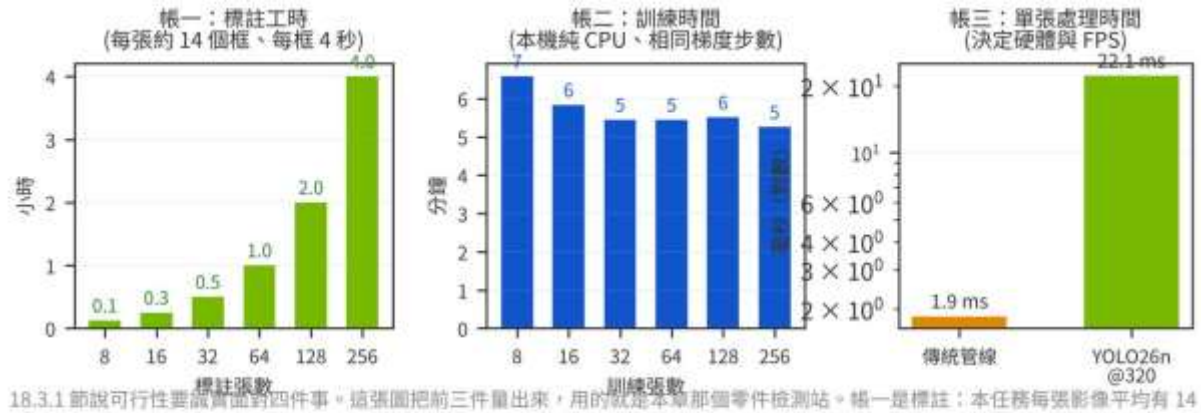
好的題目落在兩個圓的交集：一邊是「你真心想做」(Passion)，另一邊是「在時間與資源內做得出來」(可行性)。只有熱情而不可行，會做到一半崩潰;只有可行而無熱情，會做得索然無味、成果平庸。



圖 18-4　選題：熱情與可行性的交集

可行性的評估要誠實面對四件事：時間(一學期實際能投入多少小時?)、資料(需要多少標註影像?蒐集得到嗎?)、硬體(有沒有相機、Jetson、機構?)、技術落差(有多少是你沒學過、需要現學的?)。一個實用的技巧是設定「最小可行版本」：先定義一個「一定做得出來」的核心功能，確保專題有底線;行有餘力再往上加。這比訂一個宏大目標卻做不完要好得多。它的投資報酬率可以直接算。圖 18-9 在同一批現場影像上比了三條路線：照抄預設值的傳統管線零準備、寫完就能跑，平均數錯 3.20 個零件;同一條管線花約半天調過參數，誤差降到 0.28 個;而標註 256 張再訓練的偵測器，準備成本約四小時，誤差 1.20 個。也就是說，在「數有幾個」這件事上，半天的調參打敗了四小時的標註加訓練，而且快了約二十倍。要公平地補充：偵測器在這個實驗裡只拿到很小的訓練預算，而且它能做傳統管線完全做不到的事（判斷零件好壞，圖 18-13）。但這正是最小可行版本的價值所在——它幾乎免費，卻立刻告訴你三件事：這個任務的基準線在哪裡、它到底難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。至於可行性四件事裡的「資料」，也有一個比直覺可靠的估法：圖 18-5 量過本章這個任務每張影像平均有 14.1 個框要畫，以熟手每框四秒計算，標 256 張約四小時;若你的題目要框遮擋的果實

或畫出瑕疵輪廓，一張十分鐘都有可能。這通常是專題裡最被低估的一筆帳。



18.3.1 節說可行性要試實面對四件事。這張圖把前三件量出來，用的就是本章那個零件檢測站。帳一是標註：本任務每張影像平均有 14 個零件要框，以熟手每框 4 秒估算，標 128 張要 2.0 小時、256 張要 4.0 小時。聽起來不多，但請注意這是「一個人、一種零件、框很好畫」的情況;換成要框遮擋的果實或分割的瑕疵區域，一張十分鐘都有可能。這通常是專題裡最被低估的一筆帳。帳二是訓練：在本機純 CPU 上，每一次訓練約 5 到 7 分鐘，而你不會只訓練一次——調一次參數就要重跑一次。帳三是硬體：傳統管線 1.9 毫秒，YOLO26n 在 320 解析度下 22 毫秒，差了約 12 倍;這個數字決定你要不要 Jetson、要不要降解析度（第 17 章）。第四本帳「技術落差」沒有畫在圖上，因為它因人而異——但它有一個很好的量法：把你的系統拆成模組，逐一問「這個我寫得出來嗎」，答案是否的那幾個，就是你要在 M2 之前先試掉的風險。

圖 18-5　可行性的四本帳：把「做得完嗎」換算成數字

那麼資料到底要多少才夠?這個問題有一個比直覺可靠的答案，而且答案和多數人想的不一樣。圖 18-6 在本章這個零件檢測站上做了一條學習曲線：相同的模型、相同的梯度步數，只有標註張數不同。結果是「找得到零件」這件事八張就有九成以上，而「判斷它是良品還是瑕疵」始終落後二十個百分點上下——難的從來不是定位而是分類，這也解釋了為什麼很多專題「框抓得很準、就是判錯」。更值得注意的是曲線的形狀：瑕疵那一類從八張爬到三十二張之後就停了，三十二、六十四、一百二十八、兩百五十六張的成績彼此分不出高下。所以「我要標幾張」這個問題，問法本身就有點問題。

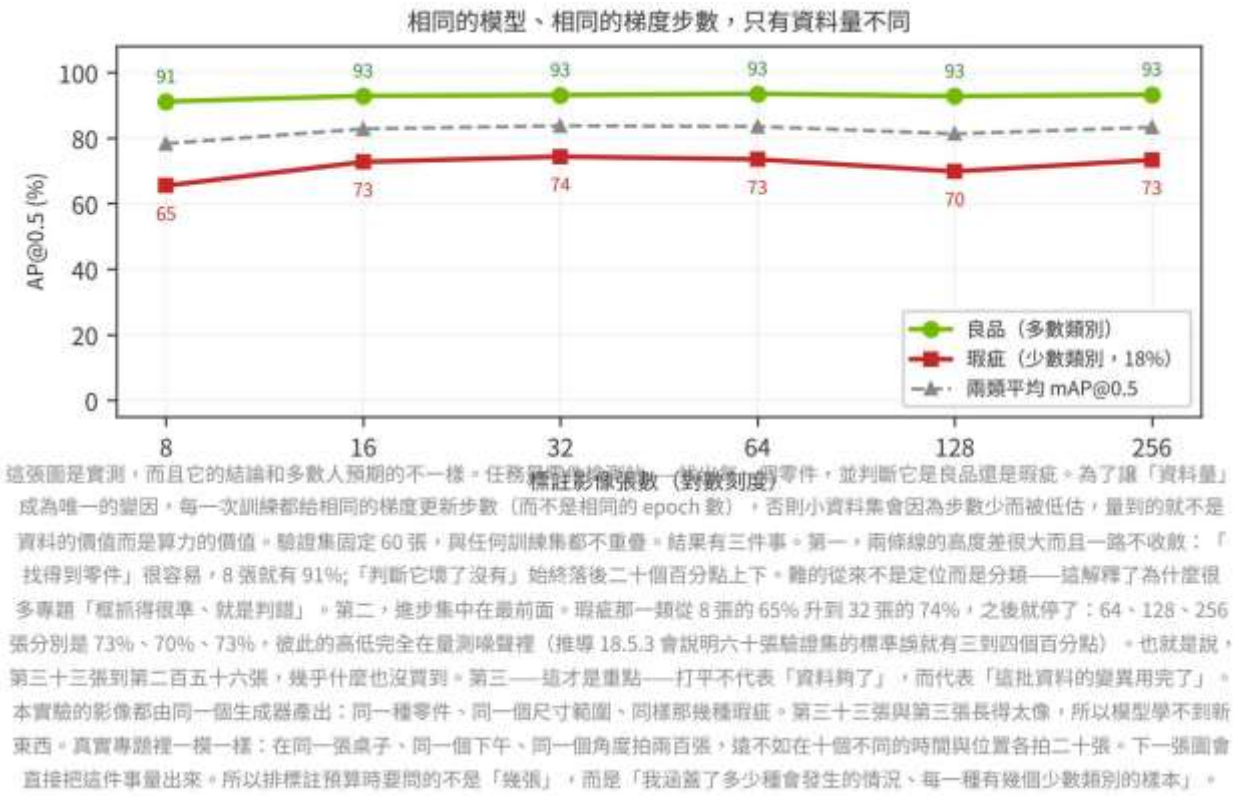
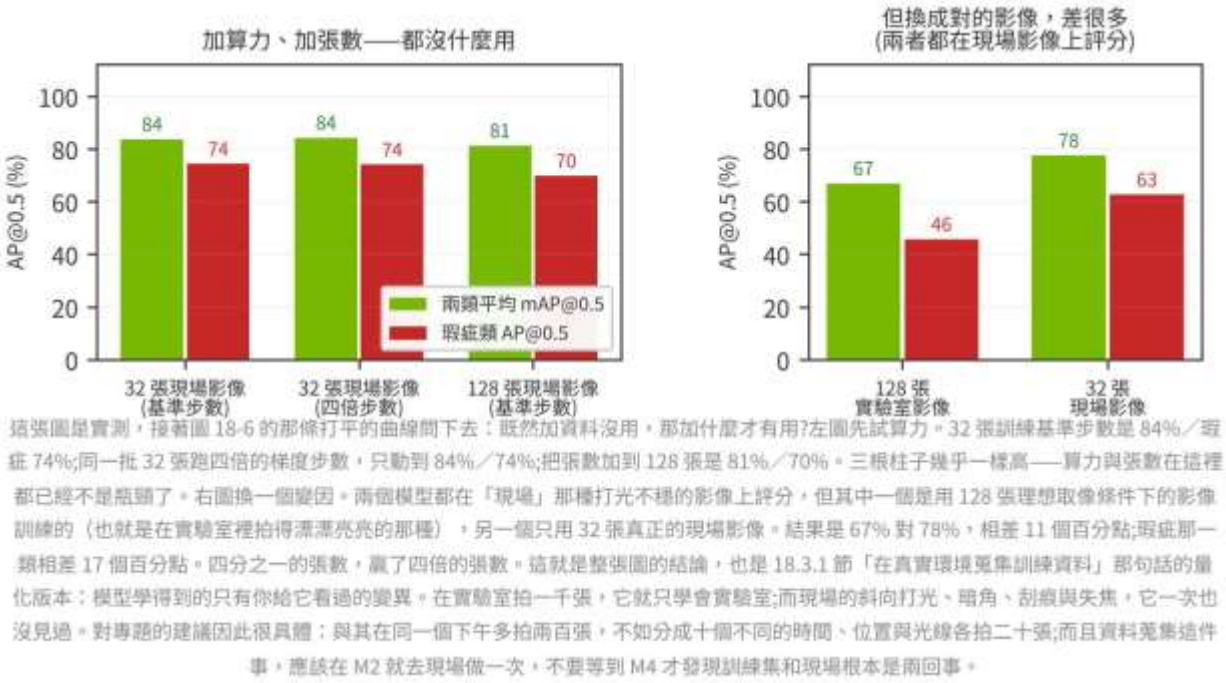


圖 18-6　要標多少張才夠：學習曲線

打平不代表資料夠了，而代表這批資料的變異用完了。圖 18-7 把這件事直接量出來：同一批三十二張影像多跑四倍的訓練步數，成績幾乎不動;張數加到一百二十八張，成績也幾乎不動;但若把訓練影像換成與使用現場相同條件的影像，mAP 就從 66.9% 升到 77.6%，瑕疵類更高出十七個百分點——而那只用了四分之一的張數。道理很單純：模型學得到的只有你給它看過的變異，在實驗室拍一千張，它就只學會實驗室。所以可行性評估裡的「資料」那一欄，該寫的不是「我能拍幾張照片」，而是「我涵蓋得到幾種光線、角度與距離，每一種少數類別各有幾個樣本」。這也直接決定了排程：資料蒐集要在 M2 就去現場做一次，不要等到 M4 整合時才發現訓練集和現場是兩回事。



這張圖是實測，接著圖 18-6 的那條打平的曲線問下去：既然加資料沒用，那加什麼才有用?左圖先試算力。32 張訓練基準步數是 84%／瑕疵 74%;同一批 32 張跑四倍的梯度步數，只動到 84%／74%;把張數加到 128 張是 81%／70%。三根柱子幾乎一樣高——算力與張數在這裡都已經不是瓶頸了。右圖換一個變因。兩個模型都在「現場」那種打光不穩的影像上評分，但其中一個是用 128 張理想取像條件下的影像訓練的（也就是在實驗室裡拍得漂漂亮亮的那種），另一個只用 32 張真正的現場影像。結果是 67% 對 78%，相差 11 個百分點;瑕疵那一類相差 17 個百分點。四分之一的張數，贏了四倍的張數。這就是整張圖的結論，也是 18.3.1 節「在真實環境蒐集訓練資料」那句話的量化版本：模型學得到的只有你給它看過的變異。在實驗室拍一千張，它就只學會實驗室;而現場的斜向打光、暗角、刮痕與失焦，它一次也沒見過。對專題的建議因此很具體：與其在同一個下午多拍兩百張，不如分成十個不同的時間、位置與光線各拍二十張;而且資料蒐集這件事，應該在 M2 就去現場做一次，不要等到 M4 才發現訓練集和現場根本是兩回事。

圖 18-7　打平之後，該加算力還是加變異

選題方向的建議：從你熟悉的生活場域出發——雲林在地的農業(果實計數、病害辨識、雜草偵測)、製造業(工件檢測、儀表判讀、安全監控)、校園生活(教室人數統計、器材管理、無障礙輔助)、或個人興趣(運動姿勢分析、寵物監控、遊戲互動)。在地與生活化的題目，往往最能激發熱情，也最容易取得資料。

18.3.2 里程碑規劃

把一學期的專題切成四到五個里程碑，每個里程碑對應創意學習螺旋的一圈——都要有可展示的成果，而非只是「進度報告」：

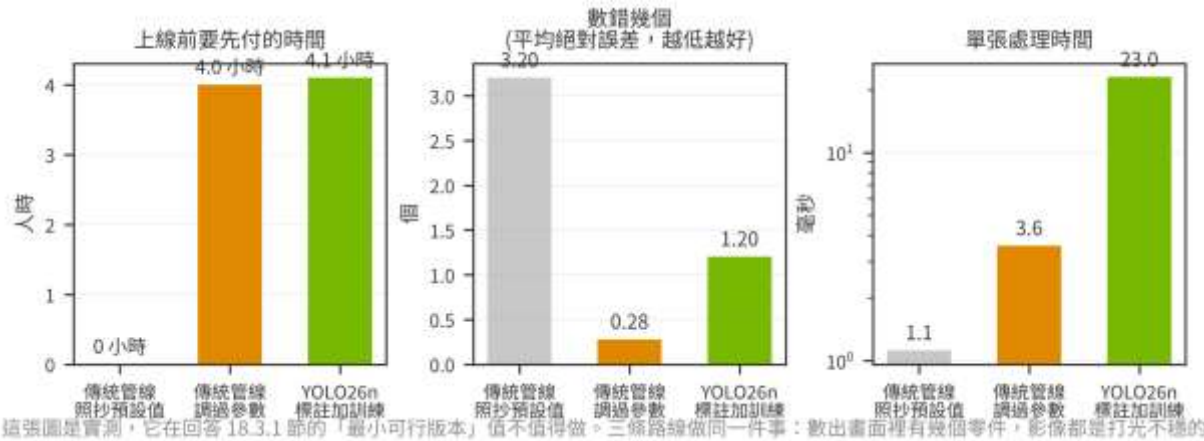


這是一張規劃用的說明圖，週數可依你的學期調整。它要強調的只有一件事：每一格的下半部寫的是「交付物」，而不是「做了什麼」。「這兩週我讀了很多論文」不是里程碑，「這兩週我做出一個會動但很爛的版本，準確率 31%」才是。理由在 18.3.2 節說過，而圖 18-3 剛剛量過：改進來自上一輪量到的結果，所以你必須儘早有一個可以量的東西。M2 是整個時程裡最關鍵的一格——它的任務不是做得好，而是「證明這條路走得通」。若 M2 做不出來，你還有十三週可以換方向;若拖到 M4 才發現，就沒有退路了。

圖 18-8　里程碑：一學期要繞幾圈

表 18-1 專題里程碑規劃

請特別重視 M2 的「粗糙原型」：許多專題失敗的原因，是把太多時間花在規劃與準備，直到後期才發現核心技術根本行不通。先做一個能動的版本(哪怕只有百分之三十的準確率)，你就能及早發現真正的困難點，並據此調整方向——這正是遊玩精神的工程版本。



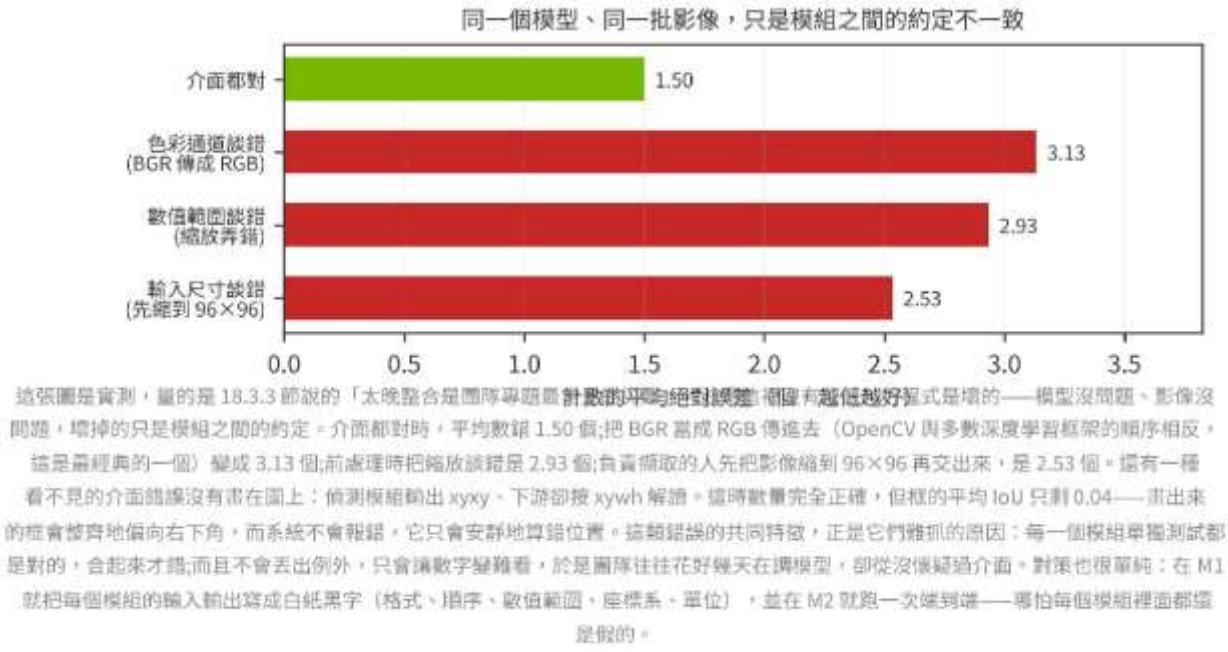
這張圖是實測，它在回答 18.3.1 節的「最小可行版本」值不值得做。三條路線做同一件事：數出畫面裡有幾個零件，影像都是打光不穩的那種。灰色是照抄範例預設值的傳統管線——零準備、寫完就能跑，但平均數錯 3.2 個。橘色是同一條管線，花大約半天把參數調過一輪（圖 18-3 的那五輪），平均誤差降到 0.28 個。綠色是標了 256 張、訓練過的偵測器，準備成本約 4.1 小時，平均誤差 1.20 個。這個結果值得停下來想一想：在「數有幾個」這件事上，花半天調參數的傳統管線，比花四小時標註加訓練的偵測器還準，而且快了 6 倍。要公平地說明為什麼：偵測器在這裡只給了很小的訓練預算（圖 18-6 的相同步數），而且它的工作點（信心門檻）也只是在開發集上粗調過;給它更多資源它會更好。但這恰恰是 M2 的論點——在你還沒有資源的那兩週，最小可行版本讓你立刻知道這個任務的基準線在哪裡、難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。

圖 18-9　最小可行版本有多便宜

18.3.3 團隊協作與工程實務

三個實務建議，它們是專業工程師的基本功：

- 版本管理：用 Git 管理程式碼與文件。它不只是備份，更讓你能安心嘗試(改壞了隨時可以回復)、讓多人能平行開發、也讓你能回頭檢視「當時為什麼這樣改」。
- 分工與整合：依模組分工(如影像處理、模型訓練、硬體整合、介面)，但務必約定好模組之間的介面(輸入什麼、輸出什麼格式)，並儘早進行整合測試。太晚整合是團隊專題最常見的災難。

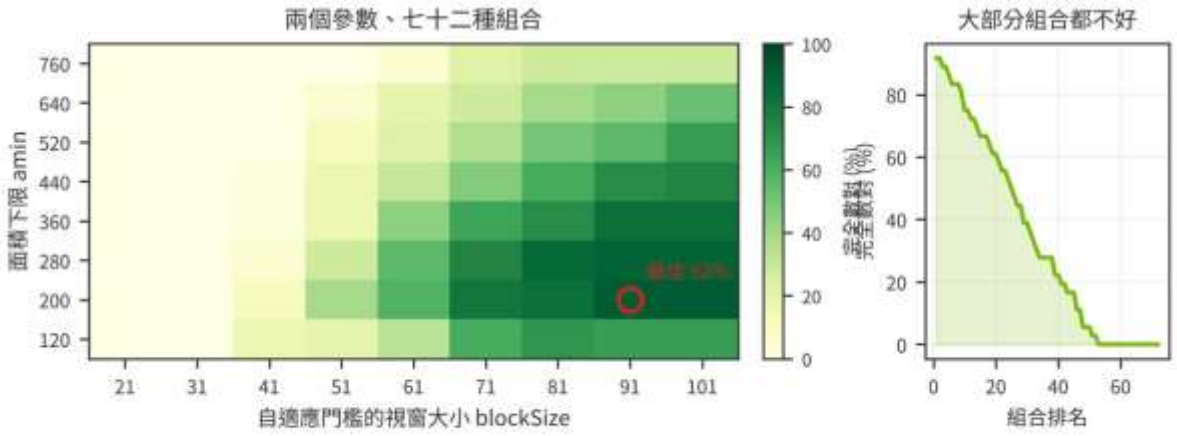


這張圖是實測，量的是 18.3.3 節說的「太晚整合是團隊專題最常見的災難」：四個模組都各自通過單元測試、計數的平均絕對誤差（MAE）越低越好，整合方式是壞的——模型沒問題、影像沒問題，壞掉的只是模組之間的約定。介面都對時，平均數錯 1.50 個;把 BGR 當成 RGB 傳進去（OpenCV 與多數深度學習框架的順序相反，這是最經典的一個）變成 3.13 個;前處理時把縮放談錯是 2.93 個;負責擷取的人先把影像縮到 96×96 再交出來，是 2.53 個。還有一種看不見的介面錯誤沒有畫在圖上：偵測模組輸出 xyxy、下游卻按 xywh 解讀。這時數量完全正確，但框的平均 IoU 只剩 0.04——畫出來的框會整齊地偏向右下角，而系統不會報錯，它只會安靜地算錯位置。這類錯誤的共同特徵，正是它們難抓的原因：每一個模組單獨測試都是對的，合起來才錯;而且不會丟出例外，只會讓數字變難看，於是團隊往往花好幾天在調模型，卻從沒懷疑過介面。對策也很單純：在 M1 就把每個模組的輸入輸出寫成白紙黑字（格式、順序、數值範圍、座標系、單位），並在 M2 就跑一次端到端——哪怕每個模組裡面都還是假的。

圖 18-10　整合：三個介面不一致的真實代價

- 工程筆記：記錄每次實驗的參數、結果與觀察。三個月後你絕對不會記得「那次準確率 92% 是用什麼設定跑出來的」。這份筆記也是期末報告最寶貴的素材。這兩條建議都可以量，而且量出來的數字比建議本身更有說服力。先看整合。圖 18-10 做了一個實驗：模型沒換、影像沒換、程式也沒有任何一行是壞的，只是把模組之間的約定弄錯。介面都對時平均數錯 1.50 個零件;把 BGR 當成 RGB 傳進去（OpenCV 與多數深度學習框架的通道順序相反，這是最經典的一個）變成 3.13 個;數值範圍談錯是 2.93 個;負責擷取的人先把影像縮到

96×96 再交出來是 2.53 個。還有一種更陰險的：偵測模組輸出 xyxy、下游卻按 xywh 解讀——數量完全正確，框的平均 IoU 卻只剩 0.04，畫出來會整齊地偏向右下角，而系統不會報任何錯。這類錯誤難抓的原因正是每個模組單獨測都對、合起來才錯，而且症狀只是「數字有點難看」。再看筆記。圖 18-11 把兩個參數交叉掃過 72 種組合：最好的一格是 91.7%，中位數只有 27.8%，而落在最佳值五個百分點以內的只有 4 格。參數空間的好區域就是這麼小，所以「我隨便試了幾個值」等於沒調過;而三個月後要回答那個 91.7% 是怎麼跑出來的，只有紀錄救得了你。順帶一提，那張圖還顯示參數之間會互相影響（面積下限可以調得很小，是因為前一步的開運算已經清掉雜訊了），所以筆記要記的是整組設定，而不是單一數字。



這張圖是實測，也是 18.3.3 節「寫工程筆記」那一條的理由。左圖把兩個參數（自適應門檻的視窗大小、連通元件的面積下限）交叉掃過 72 種組合，每一格都真的跑過 36 張影像。最好的一格是 92%，而 72 格裡只有 4 格落在最佳值的五個百分點之內——右圖把同樣的資料排序，可以看到絕大多數的組合都不好。請把這張圖當成一個地形圖：你在專題裡做的每一次「調調看」，都是在這個地形上踩一個點。兩件事因此變得很實際。第一，你不可能記得住。三個月後要回答「那次 92% 是怎麼跑出來的」，只有筆記救得了你——而且期末報告正需要這張表。第二，地形有結構，而且結構會說話。好的區域集中在「視窗夠大、面積下限不要太高」那一角；視窗太小時自適應門檻會把零件內部也切開，面積下限拉太高則開始把真正的零件濾掉。值得注意的是面積下限調得很小並沒有被懲罰——因為前一步的開運算已經把雜訊點清掉了。這就是參數之間的交互作用：單獨調一個參數得到的結論，換一個前處理就可能不成立，所以筆記要記的是「整組設定」，而不是單一數字。順帶一提，這整張圖跑完只要幾十秒——傳統管線最大的優勢就在這裡：你可以把整個參數空間掃過一遍，而深度學習的每一格都要重訓一次（圖 18-7 量過那個代價）。

圖 18-11　工程筆記：參數空間長什麼樣子

18.3.4 推導：一學期到底有多少工時，以及它會被什麼吃掉

18.3.1 節要你誠實評估時間，但沒說怎麼估。這一小節把它算出來——因為多數專題不是敗在技術，而是敗在一開始就把時間算錯了。

先算總額。一個四人小組，一學期十五週，每人每週實際投入三小時（這個數字請自己誠實填，它要扣掉你其他五門課、打工與社團）：

```
總工時 = 人數 × 週數 × 每人每週時數
       = 4 × 15 × 3 = 180 人時
```

一百八十人時聽起來很多。但它會先被三件事吃掉，而且是必然的。第一是協作本身：開會、同步進度、解釋自己做了什麼、合併彼此的程式碼——這在四人小組裡通常要佔掉一到兩成，而且人愈多佔比愈高。第二是環境與工具：裝套件、版本衝突、某個人的電腦跑不動、相機驅動不認。第三是文件與展示，也就是 M5 那一格。保守地各抓 15%、10%、15%，剩下的是：

```
可用於開發的工時 = 180 × (1 − 0.15 − 0.10 − 0.15)
                 = 180 × 0.60 = 108 人時
```

接著把本章量到的數字填進去。圖 18-5 量過：這個零件檢測站每張影像平均有十四個框要畫，以熟手每框四秒估算，標註兩百五十六張約四小時;而你的題目若要框遮擋的果實或畫出瑕疵的輪廓，一張十分鐘都有可能。同一張圖也量過訓練：在筆電 CPU 上每訓練一次要五到七分鐘，而你不會只訓練一次——圖 18-11 那張參數地形圖提醒你，每一次「調調看」都是地形上的一個點，傳統管線掃七十二個點只要幾十秒，深度學習掃七十二個點要重訓七十二次。

把這些擺在一起，一百零八人時的分配大概長這樣：標註與資料整理十五人時、去現場重新蒐集一輪十人時（圖 18-7 說明了為什麼這一筆不能省）、核心開發與調參四十人時、系統整合二十人時（圖 18-10 說明了為什麼不能少）、實地測試與退化量測十人時（圖 18-19 的那六格）。加起來已經滿了——而且這裡面沒有任何一格是留給「重做」的。

這就帶出本小節真正的結論，也是 18.3.1 節那個「最小可行版本」的算術理由。既然預算裡沒有重做的空間，那麼唯一能降低風險的做法，就是讓「發現做不到」這件事儘早發生。若核心技術行不通，在第四週發現，你還有十一週可以換方向;在第十二週發現，你只能交一份失敗的報告。把這句話翻成排程的語言：最不確定的那一段

要排在最前面，而不是最後面。這與一般人的直覺相反——大家習慣先做有把握的部分（介面、簡報、架構圖），因為那會讓人覺得有進度;但有進度跟降低風險是兩件事。

最後給一個很實際的檢查法，可以在 M1 就用：把你的系統拆成模組，為每一個模組標上「我有多確定做得出來」。只要有任何一個標著「不確定」，M2 的任務就是把那一個先試掉——不是把它做完，而是做一個粗糙到不行、但足以回答「行不行」的版本。這正是圖 18-9 的意義：傳統管線的那條路線準備成本是零，所以它幾乎永遠值得在第二週先跑一遍，哪怕你最後不會用它。它給你的不只是一個基準線，更是一個「這個任務到底難在哪裡」的答案。

18.4 專題方向與技術地圖

本節把全書十七章的技術整理成一張「技術地圖」，幫助你在選定題目後，知道該回去複習哪幾章。

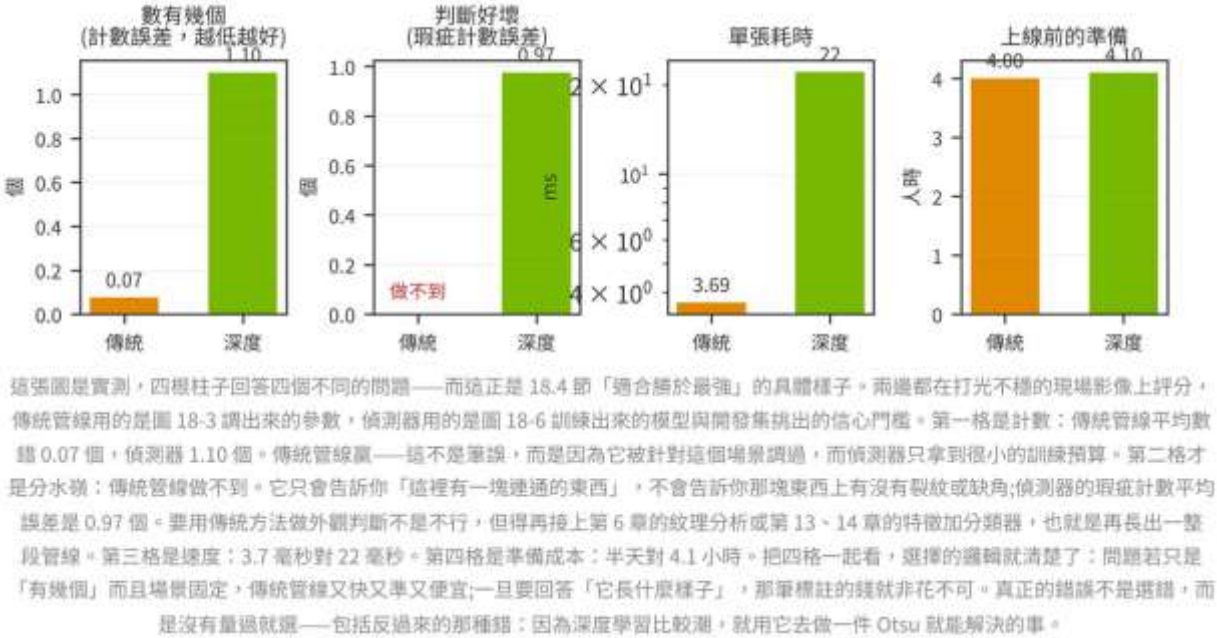


這是一張索引，不是實測。18.4 節說「不要為了用最新的技術而選題，要為了解決問題而選技術」，這張表就是那句話的操作方式：先把你的系統拆成「它到底要回答什麼問題」，再沿著那一列回去找工具。請特別注意第一列與第二列的差別——它是本章用整個零件檢測站量出來的一件事：「數有幾個」很快就能做到堪用（圖 18-6 顯示八張標註就有七成以上），「判斷好壞」卻要多得多的資料。所以當你的題目同時需要這兩件事時，請把它們當成兩個里程碑，而不是一個。另外，每一列的第二欄都同時列了傳統與深度學習的路線，這是刻意的：圖 18-13 會把兩者擺在一起量，而結論不是誰比較好，是它們貴在不同的地方。

圖 18-12　技術地圖：你的題目要回去複習哪幾章

表 18-2 專題方向與技術地圖

選題時的一個重要提醒：不要為了用最新的技術而選題，要為了解決問題而選技術。承第 9、10、13、16 章反覆強調的判準——一個用 Otsu 加連通元件就能穩定運作的零件計數系統，價值不亞於一個用上最新模型卻不穩定的系統。工程的價值在於解決問題，不在於炫技。這句話本書說過很多次，這裡用一個完整的實測把它釘下來。圖 18-13 讓兩條路線做同一件事，在同一批現場影像上評分。計數上傳統管線平均錯 0.07 個、偵測器錯 1.10 個——傳統管線贏;速度上是 3.7 毫秒對 22 毫秒;準備成本是半天對四小時。但第二格是分水嶺：判斷零件好壞，傳統管線做不到。它只會告訴你「這裡有一塊連通的東西」，不會告訴你那塊東西上有沒有裂紋或缺角。所以結論不是誰比較好，而是它們貴在不同的地方，而選擇的邏輯很清楚：問題若只是「有幾個」而且場景固定，傳統管線又快又準又便宜;一旦要回答「它長什麼樣子」，那筆標註的錢就非花不可。

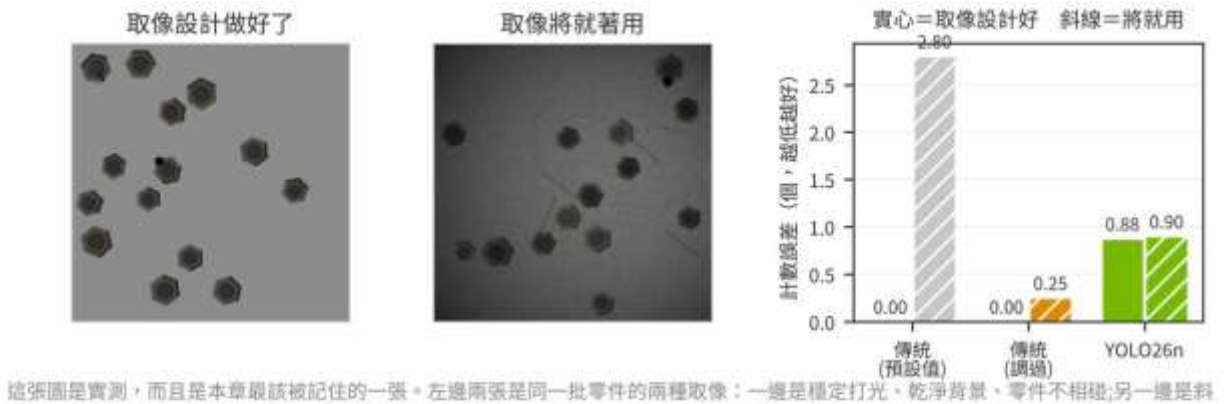


這張圖是實測，四根柱子回答四個不同的問題——而這正是 18.4 節「適合誰於最強」的具體樣子。兩邊都在打光不穩的現場影像上評分，傳統管線用的是圖 18-3 調出來的參數，偵測器用的是圖 18-6 訓練出來的模型與開發集挑出的信心門檻。第一格是計數：傳統管線平均數錯 0.07 個，偵測器 1.10 個。傳統管線贏——這不是筆誤，而是因為它被針對這個場景調過，而偵測器只拿到很小的訓練預算。第二格才是分水嶺：傳統管線做不到。它只會告訴你「這裡有一塊連通的東西」，不會告訴你那塊東西上有沒有裂紋或缺角;偵測器的瑕疵計數平均誤差是 0.97 個。要用傳統方法做外觀判斷不是不行，但得再接上第 6 章的紋理分析或第 13、14 章的特徵加分類器，也就是再長出一整段管線。第三格是速度：3.7 毫秒對 22 毫秒。第四格是準備成本：半天對 4.1 小時。把四格一起看，選擇的邏輯就清楚了：問題若只是「有幾個」而且場景固定，傳統管線又快又準又便宜;一旦要回答「它長什麼樣子」，那筆標註的錢就非花不可。真正的錯誤不是選錯，而是沒有量過就選——包括反過來的那種錯：因為深度學習比較潮，就用它去做一件 Otsu 就能解決的事。

圖 18-13　兩條路線的四面比較

還有一個比選模型更早、槓桿也更大的決定。圖 18-14 量了同一批零件的兩種取像。取像設計做好時（穩定打光、乾淨背景、零件不相碰），連照抄預設值的傳統管

線都是零誤差;取像將就著用，同一段程式就錯到 2.80 個。要把它救回來，你可以花半天調參數（降到 0.25 個）或花幾小時標註與訓練（0.90 個）——兩者都有效，但都比不上一開始就把光打好，而那幾乎不花錢。這正是第 7、8、17 章一再說的：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然也要誠實：果園、戶外、移動中的機器人根本沒有「把光控制好」這個選項，那時標註的錢就是你必須付的。所以專題的第一個決定不是選模型，而是問：我的現場，屬於哪一種?



這張圖是實測，而且是本章最該被記住的一張。左邊兩張是同一批零件的兩種取像：一邊是穩定打光、乾淨背景、零件不相碰;另一邊是斜向打光造成的明暗不均、鏡頭暗角、背景刮痕、零件亮度不一、輕微失焦與雜訊——也就是「相機層便宜一架」的樣子。右圖量的是計數誤差（越低越好），三組、各兩種取像條件。先看最左邊那一組：取像設計做好時，連照抄預設值的傳統管線都只錯 0.00 個;取像將就著用，同一段程式錯到 2.8 個。一盞穩定的光源與一塊素色背板，就是這兩根柱子之間的差距——而它幾乎不花錢。另外兩組是「取像救不回來時」你能做的兩件事：花半天調參數（中間一組，把誤差拉回 0.25 個），或花幾小時標註與訓練（右邊一組，0.90 個）。兩者都有效，但都比不上一開始就把光打好。這正是第 7、8、17 章一再說的：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然也要誠實：果園、戶外、移動中的機器人根本沒有「把光控制好」這個選項，那時右邊那兩組柱子就是你必須付的錢。所以專題的第一個決定不是選模型，而是問：我的現場，到底屬於哪一種?

圖 18-14　把錢花在打光上、調參上，還是標註上

18.5 成果展示與評估

18.5.1 如何做一場好的展示

技術做得好，還要說得清楚——這是工程師常被低估的能力。一場好的專題展示，建議這樣安排：



圖 18-15　一場好的展示：順序與時間

1. 從問題出發：先說「為什麼要做這個」，用一個具體的情境或痛點抓住聽眾。不要一開場就講你用了什麼模型。
2. 展示成果：盡早讓聽眾看到「它會動」——現場 demo 或實測影片，比任何投影片都有說服力。
3. 說明方法：再回頭解釋你怎麼做到的，重點放在關鍵決策與取捨(為什麼選這個方法而非那個)，而非流水帳。
4. 誠實呈現限制：說明系統在什麼情況下會失敗、目前的準確率、還有什麼沒做到。誠實不會扣分，反而顯示你真正理解自己的系統——這也呼應第 7、16 章的工程倫理。
5. 反思與展望：分享你學到什麼、若重來會怎麼做、下一步可以往哪走。

18.5.2 評量向度

本專題的評量對應 4P 精神，不只看最終成果，更看過程與成長。在看向度表之前，先講一件常被忽略的事：你要怎麼報你的數字。圖 18-16 拿同一批模型量了三種指標，結果彼此矛盾。mAP@0.5 一路在 78% 到 84% 之間，相當好看;但「實際有瑕疵的零件裡抓出來幾成」只有 55% 到 62%，始終低二十個百分點以上;而計數誤差又是另一

條曲線，甚至不是單調的。也就是說，一個 mAP 看起來八成的系統，可能正讓四成的不良品流到客戶手上。三個指標、三種結論，所以驗收指標必須在 M1 就寫清楚，而且要從使用者的痛點反推，不是從你會算的東西挑。另外兩個會讓數字偏離真相的陷阱也要先說。一是類別不平衡：圖 18-18 的訓練集有 2842 個良品、只有 643 個瑕疵，結果良品 AP 93.2%、瑕疵 73.3%，而兩類平均 83.2% 離良品很近、離瑕疵很遠;召回率的差距更大，99.6% 對 60.0%。平均值會把你最在乎的那一類藏起來，所以永遠要報逐類別的數字。二是資料外洩：圖 18-17 故意讓二十張驗證影像也進訓練集，mAP 量到 88.3%;同一個模型在真正沒看過的影像上只有 81.0%，瑕疵類更從 80.2% 掉到 69.9%。那些百分點憑空出現，而且不會發出任何警告。以下是評量向度：

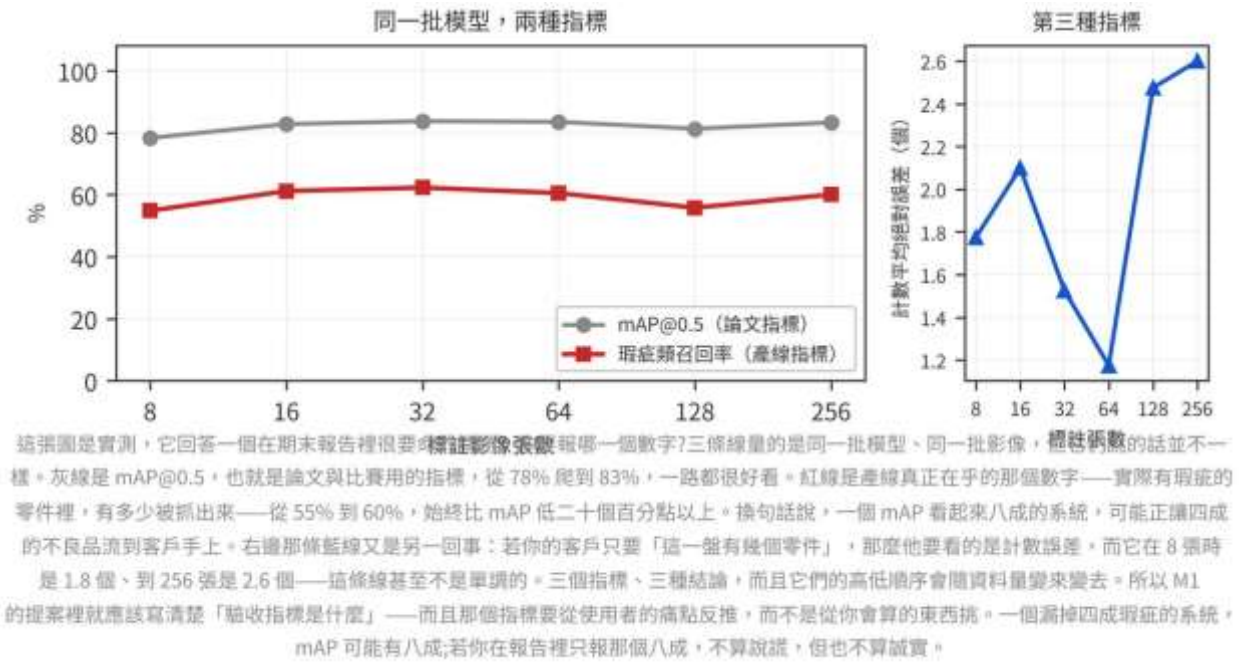


圖 18-16　指標選錯，你會看到完全不同的世界

表 18-3 專題評量向度

請注意最後一列「反思與倫理」。承第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤、第 16 章的 Redmon 故事、第 17 章的隱私設計——在你的專題報告中，請務必回答三個問題：誰會從這個系統受益?誰可能受到傷害?如果它出錯了，後果由誰承擔?這不是加分題，而是專業工作者的基本功。

18.5.3 推導：你測了幾張，那個準確率才算數

期末報告裡最常見的一句話是：「我們測了二十張，全部正確，準確率 100%。」這句話的問題不在於測試，而在於它把一個很小的樣本說成了一個確定的事實。這一小節把「那個數字可以相信到什麼程度」算出來，它承接第 14.5.3 節的標準誤，而且只需要一條式子。

你量到的準確率是一個比例。假設系統真正的成功率是 p，你測了 n 次，那麼量到的比例 $\hat{p}$ 會在 p 附近擺盪，擺盪的幅度是：

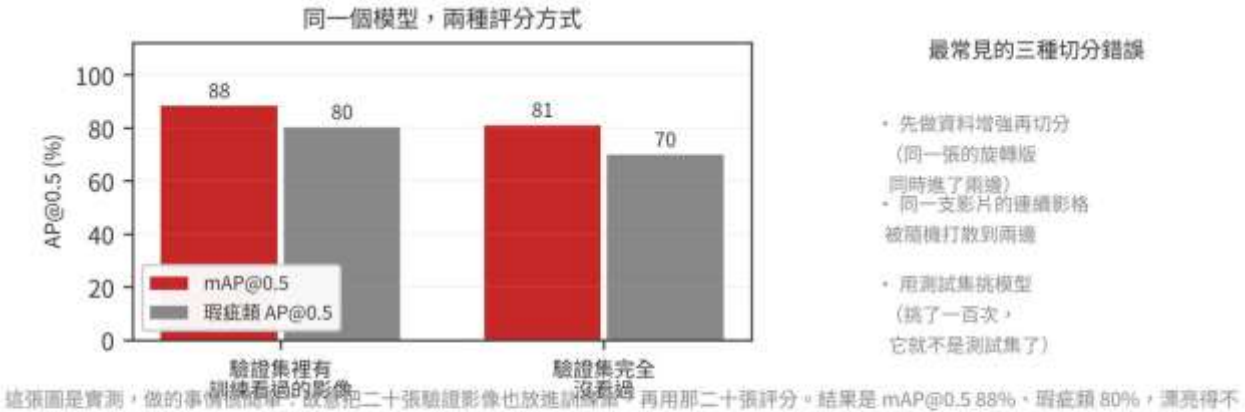
```
標準誤 SE = √( p(1−p) / n )
約 95% 信賴區間 ≈ p̂ ± 1.96 × SE
```

代幾個數字進去，結論會讓人清醒。測 20 張、量到 90%：SE = √(0.9×0.1/20) = 0.067，區間約是 77% 到 100%。也就是說，你報的「九成」和「八成」在統計上分不出來。測 100 張、量到 90%：SE = 0.03，區間約 84% 到 96%。測 500 張：SE = 0.013，區間約 87% 到 93%。要把區間縮小一半，樣本數要變四倍——因為 n 在根號裡面。這條平方根關係值得記住，它決定了你該測多少、也決定了測到某個程度就該停。

那「二十張全對」呢?這個情況特別，因為 $\hat{p}$ = 1 時上面那條式子給出 SE = 0，顯然不對。統計上有一個好用的近似叫三倍法則（rule of three）：若 n 次測試都沒有失敗，那麼真正的失敗率上限約是 3/n。

```
n 次全對 → 失敗率的 95% 上限 ≈ 3 / n
  n = 20  → 15%
  n = 100 → 3%
  n = 300 → 1%
```

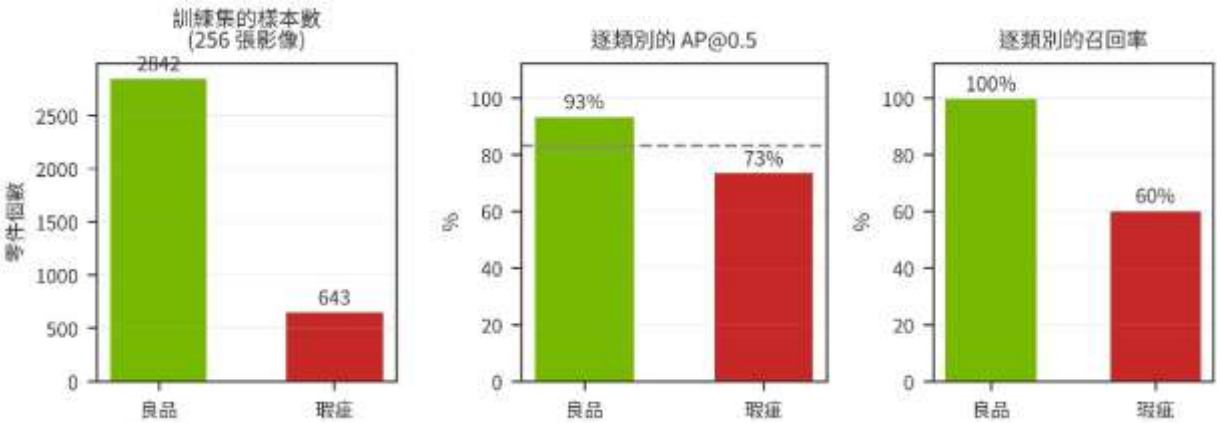
所以「二十張全對」真正的意思是：「失敗率有九成五的把握低於 15%」——這和 100% 差得很遠。若你的系統要宣稱不良率低於 1%，光是驗收就得測三百張以上，而且那三百張必須是模型沒看過的（圖 18-17）。



這張圖是實測，做的事情很簡單：訓練時把二十張驗證影像也放進訓練集，再用那二十張評分。結果是 mAP@0.5 88%、瑕疵類 80%，漂亮得不得了。接著拿同一個模型去評真正沒看過的影像：81% 與 70%，分別掉了 7 與 10 個百分點。這 7 個百分點是憑空出現的，而且它不會發出任何警告——程式不會報錯、損失曲線很漂亮、驗證分數還一路上升。第 14 章與第 16 章都提過這件事，這裡再提一次是因為它在專題裡特別容易發生：右邊列的三種情況都不是粗心，而是看起來完全合理的操作。尤其是第三種——你每次「用測試集看看這版好不好」然後決定留哪一版，就在把測試集的資訊搬進模型裡，這和圖 18-3 裡開發集與測試集愈拉愈開是同一個機制。專題的做法：在 M2 就把資料切成三份並寫進文件，測試集鎖起來，整學期只在 M5 驗收時開一次。

圖 18-17　切分錯了，你會量到一個不存在的好成績

還有兩件事會讓情況比式子說的更糟，必須一起講。第一，上面的算式假設每次測試互相獨立。若你的三百張影像是同一段影片的連續影格，它們彼此高度相似，等效的樣本數遠小於三百——這也是圖 18-17 右邊列出的切分錯誤之一。第二，少數類別的樣本數才是真正的瓶頸。圖 18-18 量過：訓練集裡良品有兩千八百多個、瑕疵只有六百多個，所以「瑕疵召回率」那個數字的區間，比整體準確率寬得多。要報一個可信的瑕疵召回率，你需要的是「夠多的瑕疵」，而不是「夠多的影像」。



這張圖是實測，追的是圖 18-6 留下的那條線索。左圖是訓練集真正的樣子：256 張影像裡有 2842 個良品、只有 643 個瑕疵，比例約 18%——這不是設計失誤，真實產線的不良率本來就低，這也正是瑕疵檢測難的根本原因：你最想抓的那一類，樣本最少。中圖是後果：良品 AP 93%、瑕疵 AP 73%，而兩類平均是 83%（灰色虛線）。注意平均值離良品很近、離瑕疵很遠——因為它是「兩類取平均」，而若你改用「所有樣本一起算」的指標（例如整體準確率），少數類別會被淹得更徹底。右圖是召回率，也就是「實際的瑕疵裡抓到幾成」：60%。對一條產線來說，這個數字才是真正要負責的那個。專題上的三個實作建議：一、永遠報逐類別的數字，不要只報平均；二、排標註預算時針對少數類別排（要的是「兩百個瑕疵樣本」，不是「兩百張影像」）；三、若少數類別實在收集不到，考慮換個問法——用第 14 章的異常偵測思路「學會正常長什麼樣，不正常就報警」，往往比硬訓練一個樣本極少的類別實際。

圖 18-18　平均值藏起來的那一類

最後把它變成專題可以直接照做的三句話。一、報數字時一併報樣本數，寫「在 120 張未見影像上 mAP@0.5 為 86%」，而不是「準確率 86%」。二、小樣本就不要報到小數點後一位——那個精度是假的。三、若某一類的樣本數少於三十，誠實寫出來並說明這是目前的限制。這三句話不會讓你的成績變差，反而正是 18.5.2 節那一列「誠實呈現限制」要看的東西。

18.6 專題案例參考

以下四個案例示範不同類型的專題如何整合全書技術。它們不是標準答案，而是幫助你想像可能性。

18.6.1 案例一：果園產量估計系統

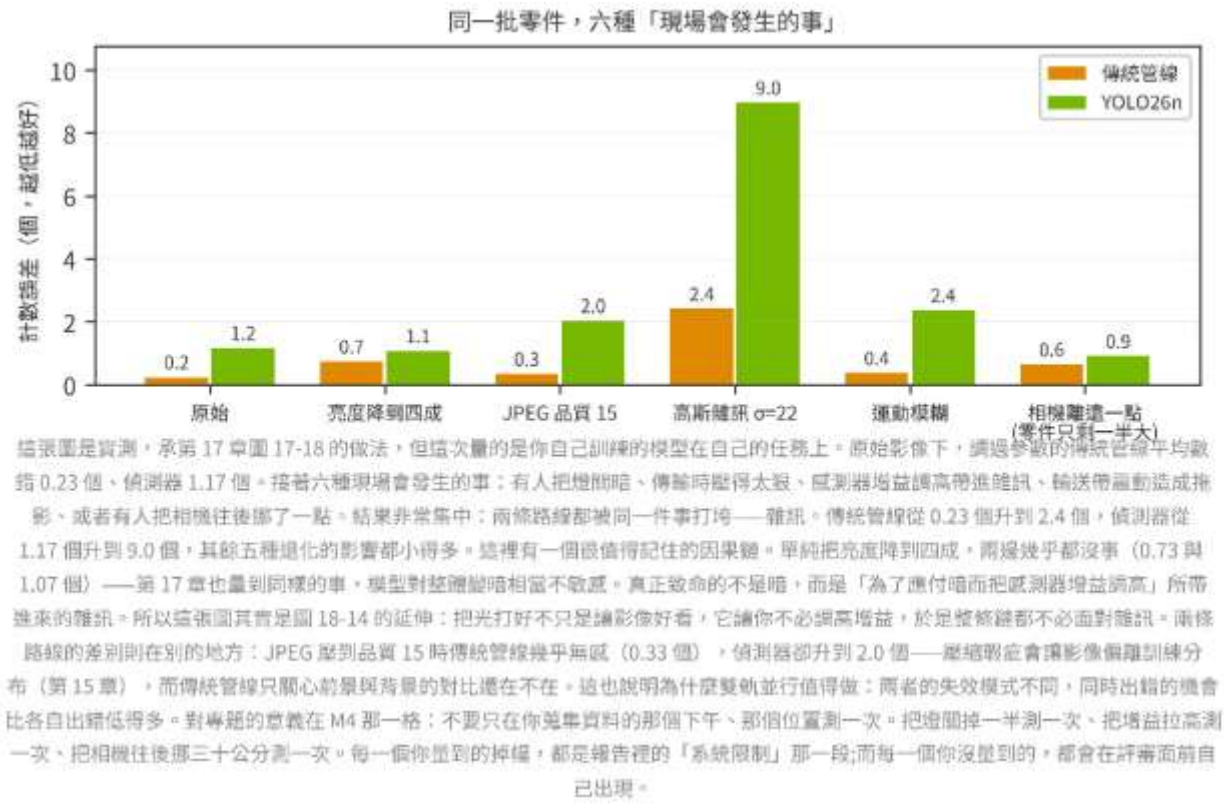
問題：小農難以在採收前準確估計產量，影響銷售規劃與人力調度。做法：用手機或樹莓派 5 配 AI Camera 拍攝果樹 → 以 YOLO 偵測並計數果實(第 16 章)→ 用第 8 章的色彩分析判斷成熟度 → 用第 4 章的相機校正估計果實大小 → 統計輸出估產報告。

技術挑戰與對策：果實相互遮擋(可結合多角度拍攝，或訓練模型學習遮擋樣態)、光線變化劇烈(第 5、8 章的前處理)、標註資料不足(第 15 章的遷移學習與資料增強)。最小可行版本：先做單一品種、固定距離拍攝的計數，再逐步擴充。

18.6.2 案例二：工件瑕疵檢測站

問題：人工目視檢測易疲勞、標準不一致。做法：固定打光與相機(第 17 章的取像設計)→ 前處理拉平背景(第 5 章 CLAHE 或第 10 章頂帽)→ 雙軌檢測：傳統管線(第 6 章頻率域紋理異常、第 9、10 章分割與型態學)與深度學習(第 16 章 YOLO)並行 → 比較兩者的準確率、速度與對「未見過瑕疵」的表現。

這個題目的教育價值在於：它讓你親身體驗全書反覆強調的判準——傳統方法不需瑕疵樣本、可解釋、運算輕;深度學習強健、能處理複雜形態。透過實測數據來論證「什麼時候該用什麼」，遠比背誦結論深刻。



這張圖是資測，承第 17 章圖 17-18 的做法，但這次量的是你自己訓練的模型在自己的任務上。原始影像下，調過參數的傳統管線平均數錯 0.23 個、偵測器 1.17 個。接著六種現場會發生的事：有人把燈關暗、傳輸時壓得太狠、感測器增益調高帶進雜訊、輸送帶晃動造成拖影、或者有人把相機往後挪了一點。結果非常集中：兩條路線都被同一件事打垮——雜訊。傳統管線從 0.23 個升到 2.4 個，偵測器從 1.17 個升到 9.0 個，其餘五種退化的影響都小得多。這裡有一個很值得記住的因果鏈。單純把亮度降到四成，兩線幾乎都沒事（0.73 與 1.07 個）——第 17 章也量到同樣的事，模型對整體變暗相當不敏感。真正致命的不是暗，而是「為了應付暗而把感測器增益調高」所帶進來的雜訊。所以這張圖其實是圖 18-14 的延伸：把光打好不只是讓影像好看，它讓你不必調高增益，於是整條鏈都不必面對雜訊。兩條路線的差別則在別的地方：JPEG 壓到品質 15 時傳統管線幾乎無感（0.33 個），偵測器卻升到 2.0 個——壓縮瑕疵會讓影像偏離訓練分布（第 15 章），而傳統管線只關心前景與背景的對比還在不在。這也說明為什麼雙軌並行值得做：兩者的失效模式不同，同時出錯的機會比各自出錯低得多。對專題的意義在 M4 那一格：不要只在你蒐集資料的那個下午、那個位置測一次。把燈關掉一半測一次、把增益拉高測一次、把相機往後挪三十公分測一次。每一個你提到的掉幅，都是報告裡的「系統限制」那一段;而每一個你沒提到的，都會在評審面前自己出現。

圖 18-19　實驗室很準，現場為什麼崩

18.6.3 案例三：互動式手勢控制機器人

問題：讓人以自然的手勢指揮機器人，而非透過遙控器。做法：以 YOLO Pose 或手部關鍵點偵測取得手勢(第 16 章)→ 定義手勢語彙(如張開手掌代表停止、指向代表前進方向)→ 加入時序判斷避免誤觸發(第 12 章)→ 透過 ROS 2 發布控制指令給機器人(第 17 章)→ 在 Jetson 上即時執行。

這個案例最能體現 Play 的精神——手勢的設計本身就是一種創作，你可以玩出各種有趣的互動方式。工程重點在即時性(第 17 章的延遲預算)與可靠性(誤判會讓機器人做出非預期動作，須設計安全機制)。

18.6.4 案例四：低光影像增強比較研究

問題：夜間或室內昏暗環境下拍攝的影像品質不佳，哪種方法最有效?做法：自建一組含「暗處拍攝」與「充足光線參考」配對的影像資料集 → 分別實作與比較：Gamma 校正與 CLAHE(第 5 章)、多影格平均(第 5 章夜景模式)、維納濾波與非局部平

均(第 7 章)、小波門檻化(第 11 章)、深度學習增強模型(第 15 章)→ 以 PSNR、SSIM(第 3 章)與主觀評分量化比較。

這是一個「研究型」而非「系統型」的專題，適合對演算法本身有興趣的同學。它的價值在於系統性的比較與誠實的數據呈現——這正是工程研究的核心素養，也直接對應 M3 里程碑要求的量化評估。

18.6.5 案例五：智慧鏡頭人臉辨識（樹莓派 5 + AI Camera）

情境：社團教室或實驗室的門禁，要求是即時、斷網也能運作，而且不把任何人的臉傳出去。這個題目之所以適合當期末專題，是因為它同時碰到本書三條主線：第 10、13 章的人臉偵測與特徵、第 15 章的嵌入向量，以及第 17 章的感測器上推論與隱私設計。

架構依 17.2.1 節的兩層切開。第一層在 AI Camera 的 IMX500 感測器裡執行人臉偵測，主機只拿到「有沒有人、框在哪裡」的張量後設資料;第二層只在偵測到人臉的那一瞬間才啟動——樹莓派取出那一小塊影像區域，算出嵌入向量，與已註冊的向量比對，成功就開門並記錄時間。請注意這個切法買到的東西：絕大多數的影格連解碼都沒有，而「是誰」這個高算力、低頻率的問題留給主機。這正是 17.1.2 節三層架構最乾淨的一個實例。

程式的骨架分成兩段。第一段在你的電腦上做，把模型轉成感測器的格式;第二段在樹莓派上做，讀取後設資料並接上比對流程。

```
# ── 在電腦上：把偵測模型轉成 IMX500 的部署格式 ──
from ultralytics import YOLO

model = YOLO("yolo11n.pt")        # 注意：目前支援到 YOLO11n 這一級
model.export(format="imx", data="coco8.yaml")
#   轉檔會自動做 INT8 量化（PTQ），產出 yolo11n_imx_model/
#   裡面的 packerOut.zip 就是要放到樹莓派上的部署包

# ── 在樹莓派 5 上：安裝韌體與工具 ──
#   sudo apt update && sudo apt full-upgrade
#   sudo apt install imx500-all
```

```
#   sudo reboot
```

上了樹莓派之後，重點是「只在需要時才讓影像進入主機」。以 Picamera2 為例，主迴圈大致是這個形狀：

```
from picamera2 import Picamera2

picam = Picamera2()
picam.start()

while True:
    # 感測器已經把推論做完了，這裡拿到的是「結果」而不是「影像」
    meta = picam.capture_metadata()
    faces = parse_detections(meta)       # 解析張量後設資料

    if not faces:
        continue                         # 絕大多數影格在這裡就結束了

    # 只有偵測到人臉時，才真的去取一張影像、而且只取框內那一塊
    frame = picam.capture_array()
    for (x1, y1, x2, y2) in faces:
        patch = frame[y1:y2, x1:x2]
        vec = embed(patch)               # 第 15 章的嵌入向量
        who, score = match(vec, registry) # 與已註冊的向量比對
        if score > THRESHOLD:
            open_door(who)
        else:
            notify_admin()
    del frame                            # 比對完即丟，不落地
```

這個案例的三個工程重點，剛好對應 M1 到 M5 要交代的事。一是模型的額度：人臉偵測模型必須塞進感測器上那很小的記憶體額度，而且只能是 INT8，所以這一層只做「有沒有臉」，不做「是誰」（17.2.1 節的三條硬規定）。二是驗收指標：門禁屬於「寧可擋錯也不要放錯」的場景，所以要報的是「冒名通過率」而不是整體準確率——承圖 18-16 的教訓，這個指標要在 M1 就寫進提案。三是樣本數：每個註冊者能拍到的臉通常只有幾十張，少數類別的樣本數才是瓶頸（圖 18-18 與推導 18.5.3）。

最後是倫理，而且這一題不能留到 M5。人臉是高度敏感的個人資料，所以圖 18-20 的三個問題必須在 M1 就回答，因為那時架構還改得動。具體的設計決策包括：公告蒐集範圍與目的、提供不使用人臉的替代方式（卡片或密碼）、只儲存比對用的特徵向量而非影像、設定保存期限並定期刪除、以及保留一條人工覆核與申訴的通道。請注意這些決定本身就是技術決定——「只存向量不存影像」這一條，直接決定了你

的資料庫長什麼樣子。

18.6.6 案例六：開放語意的現場巡檢（Jetson Orin Nano Super + NanoOWL）

情境：一台巡檢機器人要在廠區或校園裡找出「這週要找的東西」，而那個清單一直在變——這週找沒戴安全帽的人，下週找倒地的推車，再下週找堆在逃生通道上的紙箱。用第 16 章的做法，每換一次清單就要標註與重訓一輪，以一學期的工時預算（推導 18.3.4）根本跟不上。這正是開放詞彙偵測（open-vocabulary detection）存在的理由：類別不再是訓練時焊死的，而是執行時用一段文字決定的。

架構：Jetson Orin Nano Super 作為主控 → 相機節點發布影像 → NanoOWL 節點依文字提示執行開放詞彙偵測 → 把結果發布成 ROS 2 訊息 → 決策節點記錄位置與時間並回報（第 17.5 節的節點與話題）。提示詞從設定檔讀入，現場就能改，不必回實驗室。NanoOWL 的作法是把 OWL-ViT 的影像編碼器用 TensorRT 編成引擎檔，所以第一次執行前要先在機器上建好引擎：

```
# ── 先建好 TensorRT 引擎（在 Jetson 上執行一次即可）──
#   python3 -m nanoowl.build_image_encoder_engine \
#       data/owl_image_encoder_patch32.engine

from nanoowl.owl_predictor import OwlPredictor

predictor = OwlPredictor(
    "google/owlvit-base-patch32",
    image_encoder_engine="data/owl_image_encoder_patch32.engine")

# 類別就是這一行字，改字就換了一組類別
PROMPT = ["a person without a helmet", "a cardboard box"]

output = predictor.predict(image=image, text=PROMPT, threshold=0.1)
for box, label, score in zip(output.boxes, output.labels, output.scores):
    report(PROMPT[label], box, score)
```

NanoOWL 另外提供一個樹狀提示（tree prediction）的語法，可以把偵測與分類巢狀地組合起來，這在巡檢上很好用。方括號是「在這個東西裡面再找」，圓括號是「把它分成哪幾類」：

```
# 先找出每一張臉，再在每一張臉的區域裡找五官
```

```
#   --prompt="[a face [a nose, an eye, a mouth]]"

# 先找出每一個人，再判斷他有沒有戴安全帽
#   --prompt="[a person (wearing a helmet, not wearing a helmet)]"

# 把整張影像分類成室內或室外
#   --prompt="(indoors, outdoors)"
```

這個案例真正的教學價值在於它的限制，而不是它的炫目，而三個限制剛好都對應本章前面量過的東西。第一，提示詞是一個參數。模型比對的是那串字元產生的向量，不是你心裡的概念——單數與複數、加不加冠詞、換一個同義詞，結果可能天差地別。所以提示詞必須在開發集上調、在另一份從頭到尾不碰的測試集上報（圖 18-3 的開發集與測試集差距，在這裡會以更誇張的形式出現）。第二，它比同級的封閉詞彙模型重，因為每次推論都要同時處理影像與文字兩邊;這會直接吃掉第 17.4.2 節的延遲預算，所以巡檢機器人通常是「停下來看」而不是「邊走邊看」。第三，它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。

務實的做法因此是把它當成第一道篩子，而不是最終的判斷。先用開放詞彙快速找出「可能有問題的畫面」，再交給人工複核、或交給一個針對該類別訓練過的小模型確認。等到某一個項目變成長期固定的檢查項（例如安全帽），就值得為它標註資料、訓練一個封閉詞彙的模型正式上線。這正是 18.3.1 節「最小可行版本」的精神：先把路走通，再把最爛的那一段換掉——而開放詞彙讓你在還沒有任何標註資料的第二週，就能先把路走通一次。

順帶一提，這個案例也示範了一個選題上的判斷。若你的題目要找的東西從一開始就固定（例如只數零件），那麼開放詞彙是多餘的，用第 16 章的做法又快又準;它的價值只在「清單會變」這件事上。所以在提案時要先問清楚：我的需求方到底會不會改清單?多久改一次?這個問題的答案，決定了你該不該走這條路。

18.7 全書技術整合地圖

走到這裡，你已經走完了從像素到系統的完整旅程。這張表回顧全書的四篇架構，

也是你規劃專題時的索引。

表 18-4 全書四篇架構與核心能力

也請記得那些貫穿全書、比任何單一技術更重要的思想：換一個域看問題(第 6 章)、多解析度與由粗到細(第 11、13、15 章)、投票對抗雜訊(第 9、13 章)、正則化對抗病態(第 7、14 章)、精確率與召回率的取捨(第 9、10、14、16 章)、以及「適合勝於最強」(第 9、10、11、13、16、17 章)。技術會更迭，這些思考方式不會。而本章用一個零件檢測站，把其中幾個判準第一次完整地量了出來：取像設計的槓桿最大（圖 18-14）、「數有幾個」與「長什麼樣子」難度差一個量級（圖 18-6、18-13）、資料的瓶頸是變異而不是張數（圖 18-7）、指標選錯會看到完全不同的世界（圖 18-16）、而最便宜的那條路往往在第二週就能跑起來（圖 18-9）。接下來的推導 18.7.1 會把這些收斂成一個可以照著跑的六步流程，作為你面對下一個新題目時的起手式。

18.7.1 推導：把全書的判準收斂成一個可以照做的流程

走到這裡，你手上有十八章的工具與六個反覆出現的想法。但真正的問題是：面對一個新的題目，第一步該做什麼?這一小節把全書的判準收斂成一個流程。它不是標準答案，而是一個可以照著跑一遍、並且每一步都有本章實測數字支撐的檢查順序。

第一步：先問取像端能不能控制。這一步排在最前面，是因為它的槓桿最大，而學生幾乎總是跳過它直接去挑模型。圖 18-14 的實測：同一個計數任務，穩定打光、乾淨背景、零件不相碰時，傳統管線與深度學習幾乎沒有差別，兩者都接近全對;把取像條件放掉，傳統管線直接崩掉。換句話說，一盞穩定的光源可能抵得過幾百張標註。若你的場域是產線、檢測站、實驗台——先把光與背景固定下來。若是果園、戶外、移動中的機器人——這個選項不存在，直接跳到第二步。

第二步：問系統要回答的是「有幾個、在哪裡」還是「長什麼樣子」。這兩件事的難度差了一個量級。圖 18-6 的實測：「找到零件」八張標註就有七成以上，而「判斷它壞了沒有」要幾十倍的資料才慢慢爬上來。圖 18-13 說得更直接：傳統管線能數，

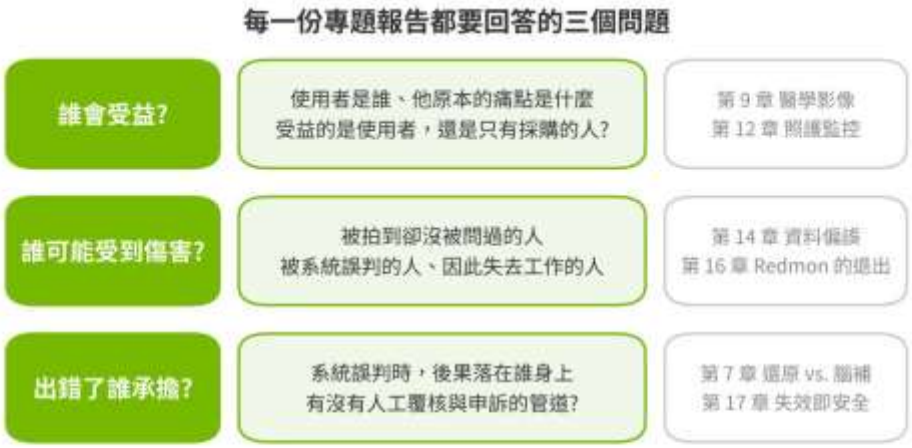
但不能判斷外觀;要用傳統方法做外觀判斷，就得再接上第 6 章的紋理分析或第 13、14 章的特徵加分類器，也就是再長出一整段管線。所以若你的題目同時需要這兩件事，請把它們切成兩個里程碑。

第三步：估資料的帳，而不是估模型的帳。圖 18-7 的實測結論很硬：同一批資料上多跑四倍的訓練步數，買到的進步遠小於把資料加到四倍。而圖 18-18 進一步說：要加的是少數類別。所以在 M1 的提案裡，資料那一欄要寫的是「我能拿到幾個瑕疵樣本」，而不是「我能拍幾張照片」。

第四步：定驗收指標，而且從使用者的痛點反推。圖 18-16 的實測：同一批模型，mAP@0.5 一路好看，而「實際的瑕疵裡抓到幾成」始終難看得多，計數誤差又是另一條曲線。三個指標、三種結論。接著用 18.5.3 節的式子反推樣本數：要宣稱的精度愈高，要測的張數愈多，而且是平方關係。

第五步：排延遲與硬體的帳。這一步直接沿用第 17 章：先確定任務允許的總延遲，再回頭分配給各環節，並且用第 99 百分位而不是平均值。圖 18-5 量過本章兩條路線的單張耗時差距約兩個數量級——這個數字決定你要不要 Jetson、要不要降解析度。

第六步：把失敗先想一遍。圖 18-19 列了六種「現場會發生的事」，每一種都在 M4 的實地測試裡量得出來。量到的每一個掉幅，都是報告裡「系統限制」那一段;沒量到的，會在評審面前自己出現。同時回答圖 18-20 的三個倫理問題——而且要在 M1 回答，因為那時架構還改得動。



這是一張檢核表，不是實測。18,5.2 節說它「不是加分題，而是基本功」，理由在右邊那一欄：這三個問題不是期末才冒出來的，它們從第 7 章就開始在這本書裡反覆出現。要讓這份檢核表不流於形式，有一個很有效的做法：把它放進 M1，而不是 M5。在還能改架構的時候問「誰可能受到傷害」，你可能會決定把人臉在端上就模糊掉、決定不儲存原始影像、或決定讓系統只輸出數量而不輸出身分——這些都是設計決策，到了 M5 就改不動了。這也呼應第 17 章的那句話：最有力的隱私保護不是承諾不看，而是技術上沒有東西可看。

圖 18-20　倫理三問：不是加分題

把六步連起來：控制取像 → 分清任務類型 → 估資料的帳 → 定指標與樣本數 → 排延遲預算 → 把失敗想一遍。你會發現這個順序裡，模型的選擇排得非常後面。這不是因為模型不重要，而是因為前面每一步都會改變「你需要什麼模型」這個問題的答案——甚至常常會讓答案變成「不需要模型」。這就是「適合勝於最強」在一張流程圖上的樣子，也是這門課十八章下來，最希望你帶走的那種判斷力。



圖 18-21　從像素到世界：全書的整合地圖

程式實作 Lab 18：期末專題

Lab 18-1(M1)提案與可行性評估

目標：找到一個你真心想做的題目。(a)腦力激盪：每人先獨立列出五個想做的題目，不要自我審查;(b)組內分享並票選，依「熱情」與「可行性」兩軸把候選題目畫在座標圖上;(c)對選定的題目做調查：是否已有類似作品?他們怎麼做?你的切入點是什麼?(d)畫出系統架構草圖：輸入是什麼、經過哪些處理、輸出什麼、用什麼硬體;(e)定義最小可行版本與延伸目標;(f)產出提案簡報，包含問題描述、目標、方法、時程與分工。

Lab 18-2(M2)資料蒐集與原型

目標：及早驗證核心技術可行。(a)蒐集資料：注意第 14 章的教訓——不同類別要在相同條件下拍攝，避免模型學到「捷徑」;記錄拍攝條件;(b)標註並依第 14 章原則切分訓練/驗證/測試集;(c)做出最小可行原型，哪怕準確率很低、介面很醜——重點是

「能從頭跑到尾」;(d)記錄第一次的量化結果作為基準線;(e)反思：實際做過之後，原本的規劃有哪些需要調整?

Lab 18-3(M3-M4)核心開發與整合部署

目標：把原型變成可用的系統。(a)迭代改進核心演算法，每次改動都記錄參數與結果(工程筆記);(b)量化評估：依任務選擇適當指標(準確率、mAP、PSNR 等)，並誠實記錄失敗案例;(c)系統整合：把各模組串接，進行端到端測試;(d)效能最佳化與部署(第 17 章)：量測延遲與 FPS，依需要最佳化;(e)實地測試：把系統帶到真實環境(果園、實驗室、走廊)測試，記錄與實驗室結果的差異——這往往是最有價值的學習;(f)依測試結果再迭代一輪。

Lab 18-4(M5)展示、文件與反思

目標：完整交付並沉澱學習。(a)撰寫技術文件：系統架構、環境安裝、使用說明、程式碼結構——標準是「另一個人照著做能重現你的成果」;(b)準備展示：依 18.5.1 節的五個步驟編排，務必有現場 demo 或實測影片;(c)誠實呈現限制與失敗案例;(d)回答三個倫理問題：誰受益、誰可能受害、出錯了誰承擔;(e)個人反思：這學期你學到最重要的一件事是什麼?若重來一次會怎麼做?接下來想繼續學什麼?(f)把成果整理成作品集——這是你日後求職、升學最有力的證明。

Lab 18-5 建立你自己的基準線與學習曲線（M2 的量化版）

目標：把 M2 從「做出一個能跑的東西」升級成「做出一個能回答問題的東西」。本 Lab 的每一步都對應本章的一張實測圖，你要在自己的題目上重做一遍。(a)最小可行版本：用不需要訓練的方法（門檻、色彩、影格差分、樣板比對，視你的題目而定）做出第一個端到端版本，並量出它的基準線數字與單張耗時（對應圖 18-9）;(b)調參一輪：選兩個最關鍵的參數做網格掃描，畫出你自己的參數地形圖，記錄最佳組合、中位數與最差值（對應圖 18-11）;注意要在開發集上挑，在另一份測試集上報;(c)學習曲線：若你的題目需要訓練，從少量標註開始（例如 8、16、32、64 張），每一次都

給相同的梯度步數，畫出你自己的學習曲線（對應圖 18-6）;(d)判斷瓶頸：曲線若還在上升，繼續標註;若已經打平，改做變異實驗——換一批不同光線 / 角度 / 位置的影像再訓練一次，比較結果（對應圖 18-7）;(e)寫下結論：以一頁說明「要達到驗收標準，我還需要什麼」，並據此修正 M3 到 M5 的排程。討論：若 (a) 的最小可行版本已經達到驗收標準，你會怎麼做?請注意這是好消息而不是壞消息——把省下來的時間花在穩健性與使用者介面上，價值遠高於硬把一個深度模型塞進去。

Lab 18-6 驗收與交付：讓你的數字站得住腳

目標：把 M5 從「做一場簡報」升級成「交付一個可被檢驗的系統」。(a)樣本數反推：先寫下你要宣稱的精度（例如「漏檢率低於 5%」），用推導 18.5.3 的標準誤或三倍法則算出需要測幾張，並誠實檢查你手上的測試集夠不夠大、少數類別有幾個樣本;(b)鎖定測試集：確認測試集從 M2 之後就沒有被用來挑過任何東西，並檢查三種常見的外洩（增強後才切分、連續影格被打散、用測試集挑模型，見圖 18-17）;(c)逐類別報告：列出每一類的精確率、召回率與樣本數，不要只報平均（對應圖 18-18）;(d)退化測試：設計至少五種你的現場真的會發生的退化（光線、雜訊、壓縮、模糊、距離、遮擋……），各測一次並製成表（對應圖 18-19）;每一個掉幅都要寫進報告的「系統限制」那一段;(e)失效設計：為最嚴重的那兩種退化設計偵測與因應方式（例如影像品質檢查、信心分數監控、人工覆核觸發條件，承第 17 章的失效即安全）;(f)倫理三問：書面回答誰受益、誰可能受害、出錯了誰承擔，並指出你在架構上做了哪些對應的設計決策（對應圖 18-20）;(g)可重現性：整理出一份文件，標準是「另一個人照著做能重現你的數字」——包含資料如何切分、參數的完整設定、環境與版本。討論：(d) 裡若有某一種退化讓系統完全失效，而你已經沒有時間修了，該怎麼辦?請思考為什麼「在報告裡誠實寫出來並說明因應方式」比「不提它」更專業，也更安全。

本章重點整理

- 4P 創意學習：Projects(在創造中學習)、Passion(投入真心在乎的題目)、Peers(協作與回饋)、Play(勇於嘗試、不怕失敗);源自 MIT Media Lab 的終身幼兒園研究群。
- 創意學習螺旋：想像、創造、遊玩、分享、反思，循環迭代且每繞一圈都站得更高;對應專題的多次迭代與里程碑規劃。
- 選題落在「熱情」與「可行性」的交集;可行性須誠實評估時間、資料、硬體與技術落差，並設定最小可行版本以確保底線。
- 里程碑 M1 提案、M2 資料與原型、M3 核心功能、M4 整合部署、M5 展示反思;每個里程碑都要有可展示的成果，並儘早做出粗糙原型以及早發現困難。
- 團隊實務：用 Git 版本管理、依模組分工但約定好介面並儘早整合、撰寫工程筆記記錄實驗參數與結果。
- 展示應從問題出發、盡早展示成果、說明關鍵取捨、誠實呈現限制、並反思展望;評量對應 4P，涵蓋技術、問題意識、協作、迭代、溝通與倫理反思。

習題

1. 請說明 4P 各自的意義。為什麼說「熱情」是其中最容易被忽略卻最關鍵的一項?

   提示與參考解答：Projects：在動手創造具體事物的過程中學得最深。Passion：投入真心在乎的題目。Peers：與同伴協作、互相回饋。Play：願意嘗試、不怕失敗的實驗心態。熱情最關鍵是因為它決定了你願意投入多少時間、承受多少挫折——而學習的深度往往正比於投入;它也最易被忽略，因為傳統作業多是老師指定題目，學生沒有選擇的餘地。

2. 承叫獸開講：Resnick 為什麼認為「幼兒園」是千年來最重要的發明?他對學校教育的主張是什麼?

   提示與參考解答：因為幼兒園是教育史上的根本創新——不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解，這是創造力的搖籃。他的

主張：不該讓幼兒園變得像其他學校(近年幼兒園小學化，積木被練習卷取代)，而該讓其他學校、甚至整個人生變得更像幼兒園——保留那套學習方式的精髓(即 4P)。

3. 請說明創意學習螺旋的五個階段。為什麼稱為「螺旋」而非「循環」?

提示與參考解答：五階段：想像(產生想法)、創造(動手做出來)、遊玩(測試試探)、分享(給人看、聽回饋)、反思(想想哪裡好壞與為什麼)，然後回到想像。稱為螺旋是因為每繞一圈都回到起點但站在更高位置——想法更成熟、技術更純熟、理解更深。它提醒我們專題不應期待一次做對，而要規劃多次迭代。

4. 好的專題題目應落在哪兩個條件的交集?若只有其中之一會發生什麼?

提示與參考解答：落在「熱情(你真心想做)」與「可行性(時間與資源內做得出來)」的交集。只有熱情而不可行：會做到一半崩潰、無法交付。只有可行而無熱情：會做得索然無味、成果平庸、學習有限。可行性須誠實評估時間、資料、硬體、技術落差四項。

5. 什麼是「最小可行版本」?為什麼 M2 階段就要做出粗糙原型，而不是先完整規劃?

提示與參考解答：最小可行版本是一個「一定做得出來」的核心功能，確保專題有底線，行有餘力再往上加。及早做粗糙原型的理由：許多專題失敗是因為把太多時間花在規劃準備，後期才發現核心技術行不通。先做能從頭跑到尾的版本(哪怕準確率很低)，就能及早發現真正的困難點並調整方向——這是遊玩精神的工程版本(快速原型)。

6. 團隊專題有哪三項工程實務建議?為什麼「太晚整合」是常見的災難?

提示與參考解答：一、用 Git 版本管理(不只備份，更讓你安心嘗試、多人平行開發、回顧修改理由)。二、依模組分工但務必約定介面(輸入輸出格式)並儘早整合測試。三、撰寫工程筆記記錄每次實驗的參數、結果與觀察。太晚整合的災難：各模組單獨都能跑，合起來卻因介面不符、時序問題、環境衝突而全面卡關，而此時已無時間修正。

7. 一場好的專題展示建議依什麼順序安排?為什麼「誠實呈現限制」不會扣分?

提示與參考解答：順序：一、從問題出發(為什麼要做，用具體情境抓住聽眾);二、盡早展示成果(demo 或實測影片最有說服力);三、說明方法(重點在關鍵決策與取捨，

而非流水帳);四、誠實呈現限制(何時會失敗、目前準確率、還有什麼沒做到);五、反思與展望。誠實不扣分，反而顯示你真正理解自己的系統——這也是第 7、16 章工程倫理的體現;隱藏限制的系統反而危險。

8. 在選擇專題技術時，為什麼「不要為了用最新技術而選題」?請以本書的判準說明。

   提示與參考解答：因為工程的價值在於解決問題，不在炫技。承第 9、10、11、13、16、17 章反覆強調的「適合勝於最強」：一個用 Otsu 加連通元件就能穩定運作的零件計數系統，價值不亞於用上最新模型卻不穩定的系統;受控環境用傳統方法可能又快又省又可解釋，複雜開放場景才需要深度學習。應為了解決問題而選技術。

9. 你的專題報告中應回答哪三個倫理問題?請以本書各章的倫理線索說明為什麼這是「基本功」而非加分題。

   提示與參考解答：三問題：誰會受益?誰可能受到傷害?如果它出錯了，後果由誰承擔?這是基本功的理由：第 7 章「還原 vs. 腦補」提醒區分找回與編造的資訊、第 9 章醫學影像須醫師確認、第 12 章照護監控涉及隱私尊嚴、第 14 章資料偏誤讓模型學到捷徑、第 16 章 Redmon 因軍事與隱私疑慮退出、第 17 章端上處理保護隱私——技術能力愈強，對後果的想像力就愈是專業素養不可分割的一部分。

10. 請從表 18-2 選一個專題方向，列出你會用到本書哪些章節的技術，並說明各自負責什麼環節。

    提示與參考解答：本題為開放題，評分重點在技術鏈的完整與合理。範例(果園產量估計)：第 4 章相機校正(像素換算實際尺寸以估果實大小)、第 5 章 CLAHE 與白平衡(戶外光線變化前處理)、第 8 章色彩分析(成熟度判斷)、第 14 章資料切分與評估、第 15 章遷移學習與資料增強(樣本不足)、第 16 章 YOLO(果實偵測計數)、第 17 章邊緣部署(田間無網路須機上運算)。

11. 請說出三個貫穿本書、比任何單一技術更重要的思想，並各舉兩章說明它如何反覆出現。

    提示與參考解答：可任選三個：一、換一個域看問題(第 6 章頻率域解週期雜訊、第 11 章小波補位置資訊)。二、多解析度由粗到細(第 11 章小波金字塔、第 13 章 SIFT 尺度空間、第 15 章 CNN 層級特徵)。三、投票對抗雜訊(第 9 章霍夫轉換、第 13 章

RANSAC)。四、正則化對抗病態(第 7 章維納濾波、第 14 章對抗過度擬合)。五、精確率與召回率的取捨(第 9 章 Canny 雙門檻、第 10 章 Viola-Jones 參數、第 14、16 章評估指標)。六、適合勝於最強(第 10 章的 Haar 級聯、第 11 章 JPEG 勝過 JPEG 2000、第 17 章邊緣模型選擇)。

12. (反思題)請回顧本學期：你學到最重要的一件事是什麼?哪一章的內容最出乎你意料?接下來你想繼續學什麼?請具體說明並規劃你的下一步。

提示與參考解答：本題無標準答案，重點在反思的具體與真誠。好的回答應：指出具體的學習事件(而非泛泛而談「學到很多」)、說明它為什麼改變了你的理解、並提出可執行的下一步(如深入某個主題、參加競賽、做開源貢獻、修習進階課程如 3D 視覺/SLAM/生成式模型、或把專題延伸為更完整的作品)。

13. 為什麼 18.3.2 節堅持「每個里程碑都要有可展示的成果」?「這兩週我讀了很多論文」為什麼不算一個里程碑?

提示與參考解答：因為創意學習螺旋的五個階段裡，「遊玩、分享、反思」這三個都需要有一個能動的東西才跑得起來——沒有東西可以玩，就沒有東西可以反思，於是整個螺旋停在「想像」與「創造」之間空轉。圖 18-3 把這件事量了出來：那五輪參數調整，每一輪的改進都來自上一輪量到的結果，而不是來自更多的思考。沒有第 0 輪那個難看的數字（測試集 8%），就不會有第 1 輪的方向。讀論文當然必要，但它是「創造」的輸入，不是螺旋的一圈。一個可操作的判準：里程碑的交付物要能讓別人問出「它在什麼情況下會失敗」這個問題。進度報告沒辦法被這樣問，一個會動但很爛的原型可以。

14. 圖 18-3 的五輪調參裡，哪一輪帶來了幾乎全部的進步?這對「卡住的時候該做什麼」有什麼啟示?

提示與參考解答：第 1 輪，也就是把全域門檻換成自適應門檻那一次——測試集從 8% 跳到 83%，後面四輪加起來只再移動了幾個百分點。啟示是：進步的分布通常極不平均，而大的那一步來自「換方法」，不是「換數字」。這裡的原因可以說得很具體：全域門檻假設整張影像的亮度一致，而這批影像有斜向打光與鏡頭暗角，這個假設根本不成立;再怎麼微調那個門檻值，都救不了一個錯的假設。所以卡住時該先問的是「我這一步的假設對不對」，而不是「這個參數再調大一點會不會好一些」。一個實用的習慣：每次調參前先寫下「我預期它會怎麼變、為什麼」，若結果和預期不符，那通常表示你對系統的理解有誤——而那個誤解才是真正要修的東

西。

15. 你的系統在 20 張測試影像上全部正確。可以在報告裡寫「準確率 100%」嗎?請用推導 18.5.3 的式子說明，並算出要宣稱「失敗率低於 1%」需要測幾張。

提示與參考解答：不可以。比例的標準誤是 SE = √(p(1−p)/n)，但 p̂ = 1 時這條式子給出 0，顯然不合理;這時要用三倍法則：n 次全對時，真正的失敗率上限約是 3/n。n = 20 時上限是 15%——所以「20 張全對」真正的意思是「有九成五的把握失敗率低於 15%」，離 100% 很遠。要宣稱失敗率低於 1%，需要 3/n < 0.01，即 n > 300 張。而且這 300 張必須是模型沒看過的（圖 18-17），也必須彼此獨立——若它們是同一段影片的連續影格，等效樣本數遠小於 300。順帶算一個常見的情況：測 60 張、量到九成，SE = √(0.9×0.1/60) ≈ 0.039，也就是三到四個百分點;這正是圖 18-3 裡 93% 與 90% 分不出高下的原因。報告的寫法：「在 120 張未見影像上 mAP@0.5 為 86%」，而不是「準確率 86%」。

16. 圖 18-6 的學習曲線在 32 張之後就打平了。這是不是表示「32 張就夠了」?請說明打平真正的意思。

提示與參考解答：不是。打平的意思是「這批資料的變異用完了」，不是「資料夠了」。該實驗的影像全部由同一個生成器產出：同一種零件、同一個尺寸範圍、同樣那幾種瑕疵、同一種退化方式。第 33 張和第 3 張長得太像，所以模型學不到新東西。圖 18-7 右半把這件事證明了：128 張理想取像條件下的影像，在現場影像上只有 66.9% 的 mAP;而 32 張真正的現場影像有 77.6%——四分之一的張數，高出 10.7 個百分點，瑕疵類更高出 16.9 個百分點。所以判斷「資料夠不夠」的正確問法不是數張數，而是列情況：我涵蓋了幾種光線、幾種背景、幾種角度、幾種距離、每一種瑕疵型態各有幾個樣本?任何一格是零，那一格就是你的系統將來會出錯的地方。實務上還有一個訊號可以用：若把訓練集加倍而驗證分數不動，下一步該做的是去新的情境蒐集資料，而不是繼續在原地加量。

17. 學習曲線打平之後，加算力有用嗎?請依圖 18-7 的實測說明，並指出真正有效的那個變因是什麼。

提示與參考解答：幾乎沒用。圖 18-7 左半實測：32 張基準步數是 mAP 83.7% / 瑕疵 74.3%;同一批 32 張跑四倍的梯度步數只動到 84.2% / 74.0%;張數加到 128 張是 81.3% / 69.8%。三者在量測噪聲範圍內，算力與張數都已經不是瓶頸。真正有效的變因是資料涵蓋的「情境變異」：把訓練影像換成與使用現場相同條件的影像，

mAP 從 66.9% 升到 77.6%。道理很單純——模型學得到的只有你給它看過的變異。在實驗室拍一千張，它就只學會實驗室。這也是 18.3.1 節「在真實環境蒐集訓練資料」那句話的量化版本，以及第 15 章分布偏移在專題尺度上的樣子。對排程的意義：資料蒐集要在 M2 就去現場做一次，不要等到 M4 整合時才發現訓練集和現場是兩回事——那時已經沒有時間重新標註了。

18. 請用推導 18.3.4 的方法，為一個四人小組估算一學期可用於開發的工時，並說明為什麼「沒有重做的空間」會決定專題的排程順序。

提示與參考解答：總工時 = 人數 × 週數 × 每人每週時數。以 4 人 × 15 週 × 每週 3 小時計算是 180 人時。扣掉協作（開會、同步、合併程式）約 15%、環境與工具約 10%、文件與展示約 15%，實際可用於開發的約 180 × 0.6 = 108 人時。把圖 18-5 的實測填進去：本章這個零件任務每張平均 14.1 個框，以每框四秒估算，標 256 張約 4 小時;每次訓練 5 到 7 分鐘，而你不會只訓練一次。再加上資料蒐集、核心開發、系統整合（圖 18-10 說明了為什麼不能省）、實地測試（圖 18-19 的六格），108 人時就滿了。關鍵在於：這個預算裡沒有任何一格留給「重做」。既然不能重做，唯一降低風險的方式就是讓「發現做不到」儘早發生，所以最不確定的那一段要排在最前面。這與直覺相反——大家習慣先做有把握的部分（介面、架構圖、簡報），因為那看起來有進度，但有進度和降低風險是兩件事。

19. 為什麼 M2 要先做一個「粗糙但能從頭跑到尾」的版本?請以圖 18-9 的實測說明它的投資報酬率。

提示與參考解答：圖 18-9 比了三條路線在同一批現場影像上的計數表現。照抄預設值的傳統管線：零準備、寫完就能跑，平均數錯 3.20 個。同一條管線花約半天調過參數：誤差降到 0.28 個。標註 256 張再訓練的偵測器：準備成本約 4.1 小時，誤差 1.20 個。也就是說，在「數有幾個」這件事上，半天的調參打敗了四小時的標註加訓練，而且快了約二十倍。要公平地補充：偵測器在此只拿到很小的訓練預算，信心門檻也只在開發集上粗調過，給它更多資源它會更好——而且它能做傳統管線做不到的事（圖 18-13 的第二格）。但這正是 M2 的論點：在你還沒有資源的那兩週，最小可行版本幾乎免費，卻立刻告訴你三件事——這個任務的基準線在哪裡、它到底難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。

20. 圖 18-10 的四種錯誤都不是程式寫錯，而是模組之間的約定不一致。請說明它們為什麼特別難抓，以及在 M1 與 M2 分別該做什麼來預防。

    提示與參考解答：四種錯誤的實測（計數平均絕對誤差，越低越好）：介面都對 1.50 個;把 BGR 當成 RGB 傳進去 3.13 個;數值範圍談錯 2.93 個;輸入尺寸談錯（先縮到 96×96 再交出來）2.53 個。另外還有一種數量完全正確、位置卻全錯的：偵測模組輸出 xyxy、下游按 xywh 解讀，框的平均 IoU 只剩 0.04。難抓的原因有三：一、每個模組單獨測試都是對的，只有合起來才錯;二、不會丟出例外，系統只會安靜地算錯，所以團隊往往花好幾天在調模型，從沒懷疑過介面;三、症狀是「數字有點難看」，而數字難看有一百種可能的解釋。M1 該做的：把每個模組的輸入輸出寫成白紙黑字——格式、通道順序、數值範圍、座標系、單位、影像尺寸。M2 該做的：跑一次端到端，哪怕每個模組裡面都還是假的（回傳固定值也行）。這兩件事加起來花不到半天，卻擋掉團隊專題最常見的災難。

21. 圖 18-11 掃過 72 種參數組合，最好的一格是 91.7%，中位數只有 27.8%。這對「調參」與「工程筆記」分別說明了什麼?另外，為什麼面積下限調得很小並沒有被懲罰?

    提示與參考解答：對調參而言，它說明參數空間的好區域很小——72 格裡只有 4 格落在最佳值的五個百分點之內，所以隨便試幾個值就宣稱「調過了」是沒有意義的。對工程筆記而言，它說明你不可能記得住：三個月後要回答「那次 91.7% 是怎麼跑出來的」，只有紀錄救得了你，而這張表本身就是期末報告最有價值的素材。至於面積下限：它調得很小卻沒有被懲罰，是因為前一步的開運算已經把雜訊點清掉了，所以沒有小面積的假目標需要靠面積下限來濾。這是參數之間的交互作用——單獨調一個參數得到的結論，換一個前處理就可能不成立。因此筆記要記的是「整組設定」，而不是單一數字;也因此報告裡寫「amin 設 200 最好」是沒有意義的，要寫「在自適應門檻 blockSize=91、開運算核 5×5 的前提下，amin 設 200 最好」。

22. 有人說「現在都用深度學習了，打光沒那麼重要」。請用圖 18-14 的實測回應這個說法。

    提示與參考解答：實測結果（計數平均絕對誤差，越低越好）：取像設計做好時，連照抄預設值的傳統管線都是 0.00 個;取像將就著用，同一段程式錯到 2.80 個。要把它救回來，你可以花半天調參數（降到 0.25 個）或花幾小時標註與訓練（0.90

個）——兩者都有效，但都沒有一開始就把光打好來得好，而且都貴得多。所以那個說法是倒果為因的：深度學習讓你「在無法控制取像時還有辦法」，而不是讓「控制取像變得不重要」。一盞穩定的光源、一塊素色背板、一個固定的工作距離，可能比多標五百張影像更有效，而且便宜得多。這與第 7、8、17 章一再強調的是同一件事：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然要誠實：果園、戶外、移動中的機器人沒有這個選項，那時標註的錢就必須花——所以專題的第一個決定不是選模型，而是問「我的現場屬於哪一種」。

23. 圖 18-16 顯示 mAP@0.5 一路在 78% 到 84% 之間，瑕疵類召回率卻只有 55% 到 62%。這個差距從哪裡來?期末報告該報哪一個?

提示與參考解答：差距來自兩件事。一、mAP 是兩個類別的平均，而良品那一類的 AP 有 93.2%，把平均往上拉（圖 18-18：平均 83.2% 離良品很近、離瑕疵很遠）。二、mAP 是對所有信心門檻積分出來的面積，而召回率是在一個實際工作點上的數字;前者描述「這個模型的潛力」，後者描述「你上線那天會發生什麼」。該報哪一個?報使用者在乎的那一個。對一條產線，要簽字負責的是「流出去的不良品有多少」，也就是瑕疵召回率;對一個計數應用，要看的是計數誤差（圖 18-16 右，而且它甚至不是單調的）。三個指標、三種結論，高低順序還會隨資料量變來變去。所以驗收指標必須在 M1 的提案裡就寫清楚，而且要從使用者的痛點反推，不是從你會算的東西挑。一個 mAP 八成、卻漏掉四成不良品的系統，若報告只寫那個八成，不算說謊，但也不算誠實。

24. 請說明圖 18-18 的類別不平衡與圖 18-17 的資料外洩，各自會讓你的驗收數字偏離真相多少，以及它們共同的對策是什麼。

提示與參考解答：類別不平衡（圖 18-18）：訓練集有 2842 個良品、643 個瑕疵，瑕疵只佔 18.5%。結果是良品 AP 93.2%、瑕疵 AP 73.3%，而兩類平均 83.2% 看起來還行;召回率的差距更大：99.6% 對 60.0%。偏離的方向是「樂觀」——平均值把你最在乎的那一類藏了起來。資料外洩（圖 18-17）：故意讓 20 張驗證影像也進訓練集，mAP 量到 88.3%、瑕疵類 80.2%;同一個模型在真正沒看過的影像上是 81.0% 與 69.9%，分別掉了 7.3 與 10.3 個百分點。這些百分點憑空出現，而且不會發出任何警告。共同的對策有三：一、永遠報逐類別的數字與樣本數，不要只報平均;二、在 M2 就把資料切成三份並寫進文件，測試集鎖起來，只在 M5 開一次;三、注意等效樣本數——同一支影片的連續影格、同一張影像的增強版本，都不算獨立的樣本（推導 18.5.3）。

25. （綜合設計題）請用推導 18.7.1 的六步流程，為你自己的專題題目完整走一遍，每一步都要寫出具體的答案與理由，並指出你目前最大的風險在哪一步。

提示與參考解答：本題為開放設計題，評分重點在六步是否都有具體答案（而非抽象描述），以及風險判斷是否誠實。六步與各自的檢查點——一、取像能不能控制?若能，先把光、背景、工作距離固定下來，並重測一次最簡單的方法（圖 18-14 顯示這一步的槓桿最大）。二、系統要回答「有幾個、在哪裡」還是「長什麼樣子」?兩者難度差一個量級（圖 18-6），同時需要就切成兩個里程碑。三、資料的帳：我能拿到幾個少數類別的樣本、涵蓋幾種情境?注意要的是變異不是張數（圖 18-7）。四、驗收指標是什麼、要測幾張?從使用者痛點反推指標（圖 18-16），再用三倍法則或標準誤反推樣本數（推導 18.5.3）。五、延遲與硬體的帳：總延遲上限多少、分配給哪些環節、用 p99 而非平均（第 17 章）。六、失敗與倫理：列出會發生的退化並各測一次（圖 18-19）、回答誰受益／誰受害／誰承擔（圖 18-20），而且要在 M1 回答。好的答案會指出一個具體的風險並附上驗證計畫，例如：「最大的風險在第三步——我只能在白天的溫室拍到資料，但系統要在清晨運作;所以 M2 的第一件事是去清晨拍 30 張，先確認模型在那個光線下還能不能用。」注意這個流程裡「選哪個模型」排得非常後面——因為前面每一步都會改變這個問題的答案，甚至常常讓答案變成「不需要模型」。

結語：從像素到世界

十八章的旅程到這裡告一段落。回頭看，我們從一張影像最基本的像素開始，學會了如何強化它、還原它、分析它的頻率、分割出物件、萃取特徵，再一路走到讓機器自己學習、自己辨識，最後把這一切裝進機器人裡，讓它能在真實世界中看見與行動。

但叫獸真正希望你帶走的，不是那些函式的用法——那些查得到、也會更新。我希望你帶走的是三樣東西。第一是判斷力：面對一個問題，知道有哪些工具可選、各自的代價是什麼、什麼情況下該用哪一個。這門課花了大量篇幅在比較與取捨，就是為了培養這件事。第二是連結的眼光：當你發現卷積、量化、多尺度、投票、正則化這些概念在不同章節中以不同面貌反覆出現時，你就開始像一個真正的工程師那樣思

考了——看見表象之下的共通結構。第三是責任感：每一章的故事裡都有人在做選擇，而技術愈強大，選擇就愈重要。

最後，請記得叫獸開講裡那些人。他們不是教科書上的名字，而是一個個在自己的處境裡，帶著熱情、好奇與不甘，把不可能變成可能的人。傅立葉的論文被退了稿，他繼續改;哈伯望遠鏡近乎失明，工程師用數學把影像救回來;一群創客翻譯著沒人看得懂的中文文件，意外打開了物聯網的大門。他們做的事，和你在期末專題裡要做的事，本質上是同一件事——只是規模不同而已。

所以，去做一個你真心想做的東西吧。和你的夥伴一起，帶著玩的心情，允許自己失敗、允許自己重來。這門課結束了，但那個螺旋不會停——想像、創造、遊玩、分享、反思，然後再一次想像。

祝你玩得開心。我們在你的作品裡見。

——機器人叫獸 李世淵

參考資料與延伸閱讀

- Resnick, M., Lifelong Kindergarten: Cultivating Creativity through Projects, Passion, Peers, and Play, MIT Press, 2017.(本章叫獸開講之史料來源;2018 年 PROSE 教育實務獎)
- MIT Media Lab, Lifelong Kindergarten Group:https://www.media.mit.edu/groups/lifelong-kindergarten
- Papert, S., Mindstorms: Children, Computers, and Powerful Ideas, Basic Books, 1980.(建構主義學習經典，Resnick 的師承)
- Scratch 程式學習平台：https://scratch.mit.edu
- 本書第 1 至 17 章之各章參考資料。
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)

索引

本索引標註各詞條主要出現的章次。中文詞條依注音符號排序,英文與縮寫詞條依字母排序。

一、中文詞條

白平衡 1、3、5、8、16-18
背景相減 9、12
比值測試 13
閉合 10、13
邊界框 14、16
邊界處理 5
邊緣偵測 1、4-7、9-10、13、15-16
邊緣運算 1-2、9-10、17
病態問題 7、14
不變矩 13
膨脹 7、9-10、13、15
頻譜 2-3、5-7、11
模型量化 3、17
摩爾紋 3、6
門檻法 9-10
描述子 6-7、13
模板比對 13
非極大值抑制 7、9、13、16
非監督式學習 14
非局部平均 6-7、11、18
反卷積 1、5-7
反向傳播 14-15
分水嶺法 9
仿射轉換 4
方塊效應 3、6、11
方向梯度直方圖 13-14
負片 2、5
傅立葉轉換 6、11
帶通濾波 6、13、15
單應性 4、13
等效直徑 13
低通濾波 3-4、6-7
點擴散函數 7
頂帽轉換 10
多解析度分析 6、11、13、15
多影格合成 5
對比度受限的自適應直方圖等化 5
對數轉換 3、5-6
斷開 10、13
端上運算 1-2、10、17

動態範圍 1、3、5-6、13
動作辨識 12
特徵匹配 4-5、7、12-13、16
特徵工程 13-14
特徵擷取 1、6-7、9-18
透視轉換 4、12-13
梯度 5-7、9-10、13-15
梯度下降 5-6、13、15
凸實度 13
填充率 13
推論引擎 17
吞吐量 17
桶狀畸變 4
同態濾波 6、10
同色異譜 8
內部參數 4
內插 3-6、8、12-13
擬合不足 14
逆濾波 7
鳥瞰圖 4
拉普拉斯運算子 5-7、9
離散餘弦轉換 6、11
劣化模型 5-7
連通性 3、9-10
連通元件標記 3、6、9-10、13
量化 1-9、11、13、15-18
濾波器核 5
率失真 3、11
高通濾波 6-7、9
高斯濾波 5-6、9
高斯雜訊 3、5-7、11
骨架化 3、10
過度分割 9
過度擬合 1、4、14-16、18
光流 5、8、12-13
可分離濾波 5-6、17
快速傅立葉轉換 6
孔徑問題 12-13
空間頻率 6
空間矩 13
開放詞彙偵測 1、16-18
霍夫轉換 1、4、9、13、16、18
灰階轉換 2-6
灰階世界 5、8-9
混疊 3-4、6、12
混淆矩陣 14
幾何轉換 1、4-6、12-18
級聯分類器 10、16
計算攝影 1、5、7

積分影像 10
激勵函數 15
機器學習 1、6、10、13-16、18
解析度 1、3、6-8、11-13、15-18
結構元素 3、10、13
角點偵測 4、7、13
交叉驗證 7、14
椒鹽雜訊 5-7、10
剪枝 17
監督式學習 14
鏡頭畸變 4
精確率 1、14、16-18
距離轉換 3、8-10
卷積 1、5-7、9、11-15、18
卷積定理 5-7
卷積神經網路 5-7、13-15
卷積層 1、5、15
均值濾波 5-7
均勻模式 13
齊次座標 4、17
遷移學習 1、15-16、18
侵蝕 7、9-10、15
強化學習 14
取樣 1、3-6、8、11-13、15、18
全連接層 15
區域成長 9
區域二值模式 13
稀疏光流 12-13
小波轉換 6、11
陷波濾波 3、6-7
像素 2-18
相位譜 6
相位相關法 5-6、12
相關 2-17
相機校正 2、4、7、10、12、16-18
型態學梯度 10
形狀特徵 5、9-10、13-14
虛擬色彩 3、8、12
旋轉不變性 13
訓練集 7、14
直方圖 2、5、7-9、12-14
直方圖等化 5、7、9、13
知識蒸餾 17
遮罩銳化 5
召回率 1、14、16-18
週期性雜訊 6-7
振幅譜 6
振鈴效應 5-7、11
偵測式追蹤 12、16

正則化 1、7、14-15、18
準確率 1、7-8、14-15、18
重投影誤差 4
中心矩 13
中值濾波 5-7
尺度不變性 13
池化層 15
遲滯連接 9
車牌辨識 1、4、10、13、18
稠密光流 8、12
創意學習螺旋 1、18
樹莓派 1-2、4-5、13、15、17-18
實例分割 9、14、16
視覺里程計 1、12、17
視訊 1-5、8-9、12-13、16-18
視訊穩定 4、12-13
手工特徵 13-15
神經網路 3-8、11-16
深度學習 1-2、5-18
數學型態學 10
數位浮水印 11
雙邊濾波 5-7
銳化 1、3、5-7
自適應中值濾波 7
姿態估計 1、12、14、16-17
資料集 1、14-16、18
資料增強 1、4-5、7-8、14-18
測試集 1、14-16、18
殘差連接 15
參數空間 9
色彩空間 2、5-6、8、11
色彩恆常性 8-9
散粒雜訊 3、5、7
損失函數 15
二值影像 3、7、9-10、13、15-17
延遲 2-3、10、12、17-18
驗證集 14-16
影格率 3、12、17
影格差分 12、17
影像拼接 4、6、13
影像分割 6-7、9-11、13、16
影像還原 1、3、5-7、9-11、14-15、17
影像融合 11
影像壓縮 3、6、11
物件偵測 1、7、9-10、12-14、16、18
物件追蹤 2、8、12、16
外部參數 4
維納濾波 1、6-7、11、18
位元深度 1-3、5、7、11

語意分割 9、16-17
約束最小平方 7
圓形度 9、13

二、英文與縮寫詞條

AI Camera 1-2、17-18
AlexNet 1、14-15、17
AOI 1、5-10、13、16-17
Bayer 3
BGR 2-3、5、8-10、12-13、17
Canny 1-2、4-7、9-10、12-14、16、18
CLAHE 5、8-10、12、18
CMYK 8
CNN 1、5-7、9、11-18
COCO 1、14、16
CT 1-18
DCT 3、6、11、17
DFT 5-6、13
Dropout 1、15
ESP32-CAM 1-2、4-5、8、10、12
FAST 7、13、16
FFT 6-7
FLANN 13
Gabor 6、13、15
Gamma 3、5、7、13-14、18
GoogLeNet 15
Haar 1-2、10-11、16-18
Harris 7、13
HDR 3、5
HOG 13-15
HSL 8
HSV 2、8-9、12、16
Hu 不變矩 13
ImageNet 1、14-15
IMX500 17-18
IoU 16
Jetson 1-2、5-7、13、15-18
JPEG 1-3、5-8、11、13、17-18
JPEG 2000 1、11、13、18
Keras 14-15
kNN 9、12-14
LBP 13
LeNet 14-17
LoG 5-13
Lucas-Kanade 12-13
mAP 3-4、8、14-18
MNIST 1、14-15

MobileNet 15-17
MRI 1、3、6-7、9、11
NanoOWL 17-18
NMS 16-17
NumPy 2-14、16
ONNX 16-17
OpenCV 1-10、12-13、15-17
ORB 4、7、13、17-18
Otsu 5-6、9-10、13、18
PSNR 1、3-7、11、18
PyTorch 2、14-15、17
RANSAC 1、4-5、13、16、18
RAW 2-3、5、7-8、11、13、17
ReLU 15
ResNet 15
RGB 2-3、8、11
ROI 2、6-7、9-10、12
ROS 2 1-2、4、17-18
SIFT 1-2、4-9、11、13-15、18
SLAM 1-2、13、17-18
Sobel 5-6、9-10、13、15
SORT 2、5、12、16
SSIM 1、3、5-7、11、18
SVM 13-15
TensorFlow 14-15、17
TensorRT 16-17
U-Net 9、16
VGGNet 15
WSQ 11
YCbCr 3、5-6、8、11
YOLO 1-2、4、7-10、12-18

── 全書完 ──